Simultaneous Localization and Mapping (SLAM)

How to Build a Vision-Based Navigation and Localization Drone



Editor: Lee Shihyuan (李世淵)

LINE OA @prof

Contents

Simultaneous Localization and Mapping (SLAM) / Nine Parts, Thirty-Four Chapters

Part I Introduction to SLAM and Autonomous Navigation (Chapters 1–3)

Chapter 1 What Is SLAM? 4

Chapter 2 SLAM Courses at Leading Universities 32

Chapter 3 Robot Navigation Systems 55

Part II Sensors for Localization and Navigation (Chapters 4–9)

Chapter 4 Indoor Positioning Technologies 81

Chapter 5 Radio and Satellite Navigation 107

Chapter 6 Geomagnetic Navigation and the Compass 132

Chapter 7 IMU Inertial Navigation 157

Chapter 8 An Introduction to LiDAR 186

Chapter 9 Cameras and Visual Navigation Sensors 211

Part III Mathematical Foundations of SLAM (Chapters 10–12)

Chapter 10 Coordinate Systems and Rigid-Body Motion 237

Chapter 11 Probability, Noise, and State Estimation 261

Chapter 12 Optimization and Graph-Based SLAM 288

Part IV 2D SLAM and the Cleaning Robot (Chapters 13–16)

Chapter 13 Building a 2D Map 314

Chapter 14 Gmapping: Particle Filter SLAM 339

Chapter 15 Cartographer: Google's 2D / 3D SLAM 364

Chapter 16 SLAM in the Cleaning Robot 392

Part V 3D LiDAR SLAM and Autonomous Inspection (Chapters 17–19)

Chapter 17 3D Point Clouds and LiDAR Mapping 421

Chapter 18 LOAM and LiDAR Odometry 451

Chapter 19 Autonomous Inspection Robots and Field Practice 480

Part VI Visual SLAM and the Vision-Navigated Robot (Chapters 20–24)

Chapter 20 An Introduction to Visual SLAM 509

Chapter 21 Epipolar Geometry and Camera Motion Estimation 539
Chapter 22 ORB-SLAM2 in Theory and in Practice 570
Chapter 23 ORB-SLAM3 and Visual-Inertial SLAM 601
Chapter 24 The Vision-Navigated Robot 630

Part VII Micro Robots, Drones and Edge AI SLAM (Chapters 25–27)

Chapter 25 NanoSLAM and Low-Power SLAM 662
Chapter 26 Drone SLAM and Visual-Inertial Navigation 693
Chapter 27 Multi-Robot SLAM and Cloud / Edge Collaboration 725

Part VIII Implementation in ROS / Python / C++ (Chapters 28–31)

Chapter 28 The SLAM Programming Environment 755
Chapter 29 Mapping with a 2D LiDAR Rover 786
Chapter 30 RGB-D / Stereo Camera SLAM in Practice 818
Chapter 31 SLAM Projects and Their Assessment 853

Part IX AI, VLMs and Semantic Navigation SLAM (Chapters 32–34)

Chapter 32 Semantic SLAM: Giving the Map a Meaning 889
Chapter 33 Training in Simulation and Sim-to-Real 912
Chapter 34 VLMs, Language Navigation and Embodied Intelligence 945

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Part I Introduction to SLAM and Autonomous Navigation

Chapter 1

What Is Simultaneous Localization and Mapping?

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Before turning to the material itself, we explain the role this chapter plays in the book as a whole and how it connects to the chapters that follow. This "map" appears at the start of every chapter so that you always know where you are and where you are going next—which is precisely the localization problem this book is about, transposed into the space of knowledge.

Figure 1-0 How Chapter 1 Connects to the Rest of the Book

[This chapter is responsible for] establishing the central problem of SLAM: how a robot can know where it is and draw the shape of its environment at the same time.

[Where it leads]

The six sensors of Section 1.3 → examined one by one in Part II (Chapters 4–9): indoor positioning, satellites, geomagnetic field, IMU, LiDAR, camera

The chicken-and-egg problem of Section 1.2 → solved by probabilistic filtering in Chapter 11 and by graph optimization in Chapter 12

The four applications of Section 1.4 → Part IV robotic vacuums, Part V autonomous inspection, Part VI AR / VR, Part VII UAVs

The map representations of Section 1.5 → occupancy grids in Chapter 13, point clouds and octrees in Chapter 17, semantic maps in Chapter 32

The front end and back end of Section 1.6 → formally defined in Chapter 12 and realized in actual systems in Chapters 15, 18, and 22

The technical challenges of Section 1.7 → the thread of "common failures" running through the whole book

[Prerequisites] None. This chapter requires no mathematics and no programming background—only curiosity.

Learning Objectives

This chapter is the starting point of the book. Its purpose is to give you an overall picture of Simultaneous Localization and Mapping (SLAM). After completing this chapter, you should be able to:

1. State the basic definition of SLAM, and explain in your own words what problem localization solves and what problem mapping solves.
2. Explain why localization and mapping are preconditions for each other, forming a chicken-and-egg problem of circular dependency, and how SLAM estimates both at the same time.
3. List the sensors commonly used on robots (wheel odometry, IMU, camera, LiDAR, radio, geomagnetic field) and state the physical quantity each one measures, together with its strengths and weaknesses.
4. Give examples of the role SLAM plays in autonomous driving, unmanned aerial vehicles, robotic vacuum cleaners, and AR / VR.
5. Distinguish among the output forms of a SLAM system: trajectory, 2D occupancy grid map, 3D point cloud, semantic map, and topological map.
6. Complete the hands-on activity for this chapter and experience firsthand the difference between "the world as the sensor sees it" and the actual map.

Core Concepts

Term (Chinese)	Term (English)	Explanation
同步定位與建圖	SLAM	The technique by which a robot builds a map of an unknown environment while estimating its own position within it.
定位	Localization	Estimating the position and orientation (pose)

Term (Chinese)	Term (English)	Explanation
		of the robot in a map or in world coordinates.
建圖	Mapping	Building a geometric or semantic representation of the environment from sensor data.
位姿	Pose	Position (x, y, z) together with orientation; in 2D commonly written (x, y, θ).
里程計	Odometry	Relative displacement accumulated from wheel encoders or sensors; drifts over time.
慣性量測單元	IMU	Composed of an accelerometer and a gyroscope; measures acceleration and angular rate.
光達	LiDAR	A sensor that measures range by laser time of flight, producing scan lines or point clouds.
閉環偵測	Loop Closure	Recognizing "I have been here before" on returning to a previously visited place, used to correct accumulated error.
佔據格地圖	Occupancy Grid Map	A 2D map that divides space into small cells and marks each as occupied, free, or unknown.
點雲	Point Cloud	A spatial representation composed of a large number of three-dimensional points, usually produced by a 3D LiDAR or a depth camera.
語意地圖	Semantic Map	A map that, beyond geometric shape, annotates meanings such as door, table, and corridor.
拓樸地圖	Topological Map	A map describing places and the passages between them as nodes and edges, similar to a metro route diagram.

1.1 The Basic Definition of SLAM: Localization + Mapping

Picture the following situation. You are blindfolded, led into a building you have never visited, and then the blindfold is removed. You have no floor plan in hand, and there are no signs on the walls. What would you do? Most people would first look around and make mental notes—"there is a door on the left, a long corridor ahead, a staircase on the right"—and then start walking. As you walk, a picture of the building gradually forms in your mind, and at the same time you always know roughly that "I am now about ten meters ahead and to the right of that door."

This seemingly natural process is one of the most classic and most difficult problems in robotics: Simultaneous Localization and Mapping (SLAM). The central question SLAM must answer can be stated in a single sentence:

> **The Central Question of This Book**
>
> With no map and no knowledge of its own position, how can a robot build a map, correct its own position, and keep moving, all at the same time?

Unpacked, SLAM is in fact the combination of two subproblems:

- Localization: given a map, estimate the robot's own pose within that map. For example, you open Google Maps on your phone and GPS tells you "you are at the campus gate"—that is localization.
- Mapping: given the robot's pose at every instant, stitch the environmental information measured by the sensors into a map. For example, a surveyor standing at a known station measures the surrounding terrain and draws a topographic map.

Taken separately, both problems already have mature solutions. The difficulty is that a robot in the real world often has neither: on entering a new environment it has no ready-made map to localize against, and no accurate pose with which to map. The spirit of SLAM is to let the robot estimate both unknowns at once—moving, observing, and using "where am I" and "what does the world look like" to correct each other, so that both estimates improve as it goes.

MIT's visual navigation course (VNAV, 16.485) defines SLAM as the problem of estimating, without prior knowledge of the environment and using sensor data alone, both the robot's pose (or its entire trajectory) and a representation of the environment. CMU's 16-833 goes a step further and stresses that SLAM must be carried out in real time under sensing uncertainty—sensors are noisy, wheels slip, lighting changes, and the robot never obtains perfect measurements. SLAM is therefore essentially a problem of making optimal estimates under uncertainty. This is why Part III of this book devotes three full chapters to the mathematical foundations of probability, filtering, and optimization.

Where the Name SLAM Comes From

The term SLAM originates in robotics research on the "stochastic mapping" problem carried out from the late 1980s through the 1990s. Around 1986, Smith, Self, and Cheeseman proposed a framework for estimating robot pose and landmark positions jointly by probabilistic means; subsequently Durrant-Whyte, Leonard, Dissanayake, and others established a complete theoretical foundation using the extended Kalman filter (EKF) and proved that the SLAM problem is convergent—as long as observation continues, the uncertainty in both the map and the localization steadily decreases. After the 2000s, particle filtering (such as Gmapping in Chapter 14

of this book) and graph optimization (such as Cartographer in Chapter 15 and ORB-SLAM in Chapter 22) successively became mainstream, and SLAM moved out of the laboratory into robotic vacuum cleaners, self-driving cars, and smartphone AR, becoming an indispensable core technology of modern intelligent robots.

1.2 Why Localization and Mapping Form a Chicken-and-Egg Problem

To understand why SLAM is hard, the key is to see clearly the circular dependency between localization and mapping:

- To build a map, you must first know where you are. A sensor measures "there is a wall three meters ahead"; this information is meaningful only if the robot's current pose is known—the same measurement will place that wall in completely different parts of the map depending on where the robot stands and which way it faces.
- To localize, you must first have a map. Localization is fundamentally a matter of matching: only by comparing what is seen now against features in the map can the robot infer its own position. Without a map there is nothing to match against.

This is a textbook chicken-and-egg problem: mapping requires localization, localization requires a map, each is a precondition for the other, and neither appears able to go first.

What Happens Without SLAM: Error Accumulation

Some students will ask: doesn't the robot have wheel odometry? If we count how many turns the wheels have made and how far it has traveled, can't we simply add it up

and know the position? In theory yes, but in practice odometry has a fatal weakness: drift. Wheels slip, carpet and tile have different friction, wheel diameters carry manufacturing error, and the small error of every step is accumulated into the position estimate and never disappears on its own. Over ten meters the discrepancy may be only a few centimeters, but after a full loop around one floor of a building the estimated position may already be off by several meters—and if such a pose is fed directly into mapping, the corridors come out curved, the rooms misaligned, and the same wall may even appear twice on the map.

The strategy by which SLAM breaks this deadlock can be summed up in one sentence: do not treat localization and mapping as two sequential steps, but as a single estimation problem in which the two correct each other. Concretely, a SLAM system usually works as follows:

1. Initialization: the robot takes its starting point as the origin. At this moment the map is empty, but the pose is certain (I am at the origin).
2. Motion and prediction: the robot advances a small step and predicts its new pose from wheel odometry or the IMU—this prediction carries error.
3. Observation and update: the sensors measure the surrounding environment; if the measured features agree with features already in the map, the degree of agreement is used in reverse to correct the pose, while newly observed regions are added to the map.
4. Loop-closure correction: when the robot completes a circuit and returns to a place it has already visited, the system recognizes "I have been here before" (loop closure detection). At that moment the accumulated error over the whole

trajectory and the whole map can be distributed and corrected in one pass, so that the map closes up.

In other words, SLAM does not make error vanish by magic; it continually uses the environment itself as a reference standard. Walls do not move and columns do not shift, so as long as the robot repeatedly observes the same environmental features, the drift can be pulled back. This idea runs through the entire book—from Bayesian filtering in Chapter 11 and pose-graph optimization in Chapter 12 to loop closure detection in Cartographer in Chapter 15, the same principle unfolds at different levels.

Figure 1-1 The chicken-and-egg problem, and what a map looks like without SLAM



The left panel draws the circular dependency as a loop: to map you must first know where you are, and to localize you must first have a map to match against. Each is a precondition for the other, and neither appears able to go first.

The middle and right panels show maps of the same robot, the same site, and the same route, built in two different ways. These are genuine simulation runs produced by code: the robot travels one loop along the corridor while a 2D LiDAR scans its surroundings, and the measured wall surfaces are marked into grid cells.

The middle panel uses wheel odometry alone to propagate the pose. The angular error

per step is extremely small (about one hundredth of a degree), and after one loop the endpoint deviates from the start by only 2.1 meters—which does not sound like much, yet the resulting map is already unusable: the whole figure is twisted, the corridor is curved, the same wall is drawn as two lines because the pose differed on the two passes, and the partition walls of the rooms are offset from one another. This is exactly the phenomenon described in the preceding paragraph.

In the right panel the poses have been corrected: the walls are straight, the rooms are square, and the map closes. Note that the sensor data in the two maps are identical—the only difference is where the robot believed itself to be. This sentence is worth remembering: when SLAM fails to produce a good map, the problem is almost always in the localization, not in the sensor.

Three Things to Take from This Figure

1. Map quality depends on pose quality. However accurate the sensor, if the pose is wrong the map is skewed.

2. Error does not appear suddenly; it accumulates step by step—every short segment in the middle panel looks reasonable, and what goes wrong is the sum.

3. SLAM's corrective power comes from the environment itself: walls do not move, so observing the same wall repeatedly pulls the drift back. Chapter 12 will show you how that correction is carried out mathematically.

Two Lines of Solution: A Preview of the Book's Main Thread

Faced with this mutual dependency, researchers over the past forty years have developed two principal lines of solution. They form the main thread from Part III

onward. You do not need to understand the details now, but knowing that these two paths exist will make the later chapters much clearer.

- The probabilistic filtering line: since we cannot be certain where the robot is, we retain many possibilities at once and use sensor data to eliminate implausible hypotheses one by one. This path leads to the Kalman filter and the particle filter, the subject of Chapter 11 and the theoretical basis of Gmapping in Chapter 14.
- The optimization line: write every position the robot has occupied and every constraint measured by the sensors into one enormous system of equations, and solve in a single pass for the configuration that best fits all observations. This path leads to graph optimization and bundle adjustment, the subject of Chapter 12 and the core of Cartographer in Chapter 15 and ORB-SLAM in Chapter 22.

These two lines do not replace each other. Chapter 11 explains the real-time advantage of filtering methods, and Chapter 12 explains why optimization methods can go back and correct historical errors. Chapters 14 and 15 will let you compare the two firsthand at the same site—and when you return to this section afterward, you will have a far more three-dimensional understanding of how the chicken-and-egg problem is solved.

1.3 How a Robot Senses the World: Wheel Odometry, IMU, Camera, LiDAR, Radio, and Geomagnetic Field

Human beings sense "where I am and what is around me" through the eyes, the ears, the vestibular system (the balance organ of the inner ear), and muscular proprioception. A robot needs corresponding senses. The sensors used in SLAM fall broadly into two classes:

- Proprioceptive sensors measure the robot's own state of motion—for example how far the wheels have turned (wheel odometry), or the linear acceleration and angular rate of the body (IMU). They need not observe the outside world, but their error accumulates.
- Exteroceptive sensors measure the environment around the robot—for example the image captured by a camera, the range measured by a LiDAR, the strength of a received radio signal, or the direction of the Earth's magnetic field. They provide an absolute or semi-absolute reference, but are easily disturbed by the environment.

The art of SLAM lies in fusing proprioceptive sensors, which drift but are continuous and smooth, with exteroceptive sensors, which do not drift but are noisy and can fail instantaneously, so that each compensates for the other's weakness. The six major sensor classes examined in depth in later chapters are introduced below.

1. Wheel Odometry

The most basic sensor on a wheeled robot. An encoder mounted on the motor shaft or wheel axle counts wheel revolutions; combined with the wheel diameter and the wheelbase, this yields the distance traveled and the angle turned. Wheel odometry has a high sampling rate, low cost, and continuous data; its drawback is the drift already mentioned—slip, wheel-diameter error, and uneven floors all cause error to accumulate without bound. In ROS, wheel odometry corresponds to the odom coordinate frame and is the source of the motion prediction in nearly every SLAM algorithm (see Chapters 10 and 29).

2. Inertial Measurement Unit (IMU)

An IMU (Inertial Measurement Unit) typically contains a three-axis accelerometer

and a three-axis gyroscope, and some units add a three-axis magnetometer. The accelerometer measures linear acceleration and the gyroscope measures angular rate; integrating them over time can in principle yield velocity, position, and attitude—this is inertial navigation. The IMU's sampling rate is very high (commonly 100 Hz to 1000 Hz) and it is highly sensitive to rapid motion, making it indispensable for unmanned aerial vehicles and for visual-inertial SLAM (such as ORB-SLAM3 in Chapter 23). Integration, however, amplifies noise and bias and produces rapid drift, so an IMU must almost always be fused with other sensors (see Chapter 7).

3. Camera

The camera carries the richest information at the lowest cost: a single image contains texture, color, shape, and even semantics (this is a door, that is a person). A monocular camera is inexpensive but cannot measure depth directly and suffers from scale ambiguity; a stereo camera computes depth from the disparity between the left and right images; an RGB-D camera (such as the RealSense) outputs the range of every pixel directly. SLAM built primarily on cameras is called Visual SLAM and is the main thread of Part VI. The camera's weakness is its dependence on illumination and texture—darkness, backlighting, blank white walls, and motion blur from rapid movement all cause visual SLAM to fail (see Chapters 9 and 20).

4. LiDAR

LiDAR (Light Detection and Ranging) emits laser pulses and measures the time of flight of the returned light, directly yielding precise geometric information of the form "there is something in this direction at this range." A 2D LiDAR scans a single plane by rotating and outputs a ring of range values (a LaserScan); it is standard equipment on robotic vacuum cleaners and teaching platforms. A 3D LiDAR (multi-beam) scans many

elevation angles at once and outputs a three-dimensional point cloud; it is the core sensor of self-driving cars and autonomous inspection robots. LiDAR's strengths are measurement accuracy and immunity to illumination; its drawbacks are higher cost, failure on glass and mirrored surfaces, and noise in rain or fog (see Chapter 8 and Part V).

5. Radio

GPS / GNSS satellite positioning is the best known form of radio positioning, but the signal is blocked indoors, in tunnels, and in urban canyons. Indoors, positioning can use Wi-Fi signal-strength fingerprinting, Bluetooth beacons, or ultra-wideband (UWB). The advantage of radio positioning is that it supplies an absolute position reference and does not drift; its drawbacks are limited accuracy (GPS on the order of several meters, Wi-Fi several meters, UWB ten to thirty centimeters) and susceptibility to multipath reflection. In practice it commonly serves as auxiliary information for SLAM or as a source of initialization (see Chapters 4 and 5).

6. Geomagnetic Field

The Earth is itself a large magnet, and a magnetometer (electronic compass) can measure the direction of the geomagnetic field, providing an absolute heading reference. More interesting still, the rebar and equipment inside a building distort the local magnetic field in characteristic ways, and these distortions form a "geomagnetic fingerprint" that can be used for indoor positioning. Geomagnetic sensing is entirely passive and requires extremely little power; its drawback is susceptibility to interference from ferromagnetic objects, which makes calibration mandatory (see Chapter 6).

Sensor	What It Measures	Principal Strength	Principal Weakness	Chapters
Wheel odometry	Wheel rotation → relative displacement	Inexpensive, continuous, high rate	Slip and drift; error accumulates	Chapters 10, 29
IMU	Acceleration, angular rate	High rate; sensitive to rapid motion	Integration drifts quickly	Chapters 7, 23
Camera	Images (texture, color, semantics)	Information-rich, low cost	Depends on illumination and texture	Chapters 9, 19–23
LiDAR	Laser ranging (2D / 3D)	Geometrically accurate; immune to illumination	Higher cost; fails on mirrored surfaces	Chapters 8, 13–18
Radio	GPS / Wi-Fi / UWB signals	Absolute position; no drift	Limited accuracy; multipath interference	Chapters 4, 5
Geomagnetic field	Direction and magnitude of the Earth's field	Passive; provides absolute heading	Ferromagnetic interference; requires calibration	Chapter 6

Table 1-1 Comparison of Sensors Commonly Used in SLAM

How the Sensors Are Developed in This Book

Each of the six sensor classes mentioned in this section has a dedicated chapter in Part II. The table below is given here so that you know where the in-depth treatment of each sensor can be found, and for convenient reference later.

Table 1-2 Sensors and Their Dedicated Chapters

Wheel odometry → Chapter 3 (dead reckoning), Chapter 13 (fusion with LiDAR), Chapter 29 (implementation and calibration)

IMU → Chapter 7 (principles, drift, attitude filtering, preintegration)

Camera → Chapter 9 (imaging model), Part VI (visual SLAM throughout)

LiDAR → Chapter 8 (principles and specifications), Part IV (2D), Part V (3D)

Radio → Chapter 4 (indoor positioning), Chapter 5 (satellite navigation)

Geomagnetic field → Chapter 6 (electronic compass and geomagnetic fingerprints)

Every chapter has the same structure: principle → specifications and limits → failure scenarios → complementarity with other sensors → hands-on activity.

This consistent structure exists so that you can compare sensors quickly rather than treating them as isolated pieces of knowledge.

One idea that runs through the entire book is planted here: no single sensor is universal. LiDAR is precise but cannot interpret glass; the camera is information-rich but fears darkness; the IMU responds quickly but drifts; satellite signals are accurate but do not reach indoors. Every chapter of Part II discusses honestly the conditions under which that sensor fails, and the experiment in Chapter 30 lets you see with your own eyes how LiDAR and camera each break down at the same site. Understanding

limitations is the core conviction of this book from its first page to its last.

1.4 How SLAM Relates to Autonomous Driving, UAVs, Robotic Vacuums, and AR / VR

Over the past two decades SLAM has gone from an obscure academic topic to a fiercely contested core technology, because it happens to be the common foundation of four major applications.

Autonomous Driving

A self-driving car requires centimeter-level localization accuracy—GPS error of several meters is enough to put the vehicle in the wrong lane. Self-driving systems therefore generally adopt a strategy of "high-definition map plus real-time localization": survey vehicles equipped with 3D LiDAR and cameras first build a high-definition map using SLAM techniques; while driving, the vehicle matches live sensor data against that map to localize precisely. In areas where no high-definition map exists (newly opened roads, parking structures), the vehicle must run SLAM in real time. The 3D LiDAR mapping and the LOAM family of algorithms introduced in Chapters 17 and 18 are precisely the technical backbone of localization and mapping for autonomous vehicles.

Unmanned Aerial Vehicles (Drones / UAVs)

When a UAV flies indoors, beneath bridges, or in tunnels where the GPS signal is blocked (GPS-denied environments), it can localize only with onboard sensors. Because payload and power are severely limited, UAVs favor the lightweight combination of camera plus IMU—that is, visual-inertial odometry (VIO) and visual-inertial SLAM. This

is the specialty of the research groups at ETH Zürich and UZH, and the reason MIT's VNAV course uses UAVs as its experimental platform. After 2015, consumer manufacturers such as DJI built VIO into their products, enabling drones to hover stably indoors; on the academic side the push continues toward smaller and more power-efficient systems, such as NanoSLAM in Chapter 25, which allows a 44-gram micro-UAV to run SLAM entirely onboard.

Robotic Vacuum Cleaners

The robotic vacuum is SLAM's most successful consumer application. Early machines used a random-bounce strategy—turn when you hit a wall—which produced a chaotic cleaning path with both repetition and omission. In 2010 Neato was the first to mount a 2D LiDAR on a vacuum, and in 2017 Xiaomi's first-generation LiDAR vacuum made the technology affordable, bringing SLAM into ordinary households on a large scale for the first time: the robot first builds a map of the room, then plans a boustrophedon path and covers the area zone by zone, greatly improving both efficiency and coverage. The room map, no-go zones, and point-and-clean features you see today when you open a vacuum's app are all SLAM underneath (see Chapter 16).

AR / VR (Augmented and Virtual Reality)

For AR glasses or phone-based AR to pin a virtual object onto a real tabletop, the device must know its own pose in space in real time—this is SLAM, although industry usually calls it "spatial localization" or "inside-out tracking." Apple's ARKit, Google's ARCore, and the headset tracking of Meta Quest are all built on visual-inertial SLAM at their core. VR headsets track environmental features with onboard cameras, replacing the external base stations required by earlier systems. It is fair to say that SLAM is what

turned AR / VR from a seated experience into one in which the user can walk freely.

Conceptual Summary

These four applications look worlds apart, yet underneath they are the same problem: estimating one's own pose in real time and understanding the surrounding space in an unknown or partially unknown environment.

The differences lie only in the sensor suite (LiDAR-centric or camera-centric), the computational resources (a vehicle computer or a phone chip), and the accuracy required (centimeters or decimeters).

This is why the book stresses one set of concepts across many platforms: master the core principles of SLAM and you can transfer freely among self-driving cars, UAVs, robotic vacuums, and AR.

1.5 From the Classroom to the Campus: Teaching Cases for SLAM at the High School and University Level

One distinctive feature of this book is that it grounds the study of SLAM in two real teaching settings: the high school AI and robotics course, and the university intelligent robotics program. The same core concepts are designed into activities of different depth according to the stage of study.

The High School Setting: Understandable and Buildable

At the high school level the emphasis is on building intuition and hands-on experience rather than rushing into derivations. Taking the classroom as the experimental site, the following progressive activities can be designed:

- Activity 1: Human SLAM. One student is blindfolded and directed verbally by classmates to walk around the classroom. While walking, that student narrates "where I think I am," and the others record on paper the gap between the actual and estimated positions, experiencing how error accumulates in the absence of external observation.
- Activity 2: First encounter with phone sensors. Use compass and accelerometer apps on a phone to observe sensor readings and understand the principles and the noise of inertial and geomagnetic sensing.
- Activity 3: Observing a robotic vacuum. Observe the cleaning path of a household robotic vacuum and the map in its app, and discuss how it knows which areas it has already cleaned.
- Activity 4: Mapping in simulation. Drive a virtual robot in a ROS simulation environment (such as the TurtleBot3 simulator) to build a map of the classroom, experiencing the complete SLAM workflow without any physical hardware.

The University Setting: Able to Derive, to Implement, and to Research

At the undergraduate and capstone level, the university intelligent robotics laboratory and the campus become the site, and the same set of concepts is driven deeper and extended further:

- Foundational implementation: run Gmapping and Cartographer with a 2D LiDAR teaching platform in the classroom and corridor (Chapters 14, 15, and 29) and compare the map quality of the two algorithms quantitatively.

- Advanced implementation: run RTAB-Map and ORB-SLAM3 with an RGB-D camera (Chapter 30) and compare the performance of visual SLAM and LiDAR SLAM at the same site.
- Capstone research: quadruped campus inspection, tightly coupled LiDAR-inertial mapping, vision-guided UAVs, and multi-robot collaborative mapping (Chapters 18, 22, and 24–26), connecting to graduate-level topics.

Chapter 31 of this book provides six complete capstone projects together with a grading rubric, covering the full spectrum from high school to university capstone work. Instructors are encouraged to show students this "learning map" at the beginning of the term, so that they know which visible, tangible robotic system each concept they are learning will ultimately lead to.

1.6 SLAM System Outputs: Trajectory, 2D Map, 3D Point Cloud, Semantic Map, and Topological Map

What exactly does a SLAM system produce when it has finished running? Depending on the needs of the application, the output takes one of five forms. Understanding these forms will help you quickly grasp, in later chapters, which kind of output a given algorithm is designed to produce.

1. Trajectory

The sequence of poses at every instant from start to finish. The trajectory is the most direct basis for evaluating SLAM accuracy: comparing the estimated trajectory against the true trajectory (ground truth, usually supplied by a motion capture system or high-precision GPS) yields the absolute trajectory error (ATE) and the relative pose error (RPE). Public datasets such as EuRoC, TUM, and KITTI (Chapters 26 and 30) all

provide ground-truth trajectories for benchmarking.

2. 2D Occupancy Grid Map

The floor is divided into cells of fixed size (for example 5 cm per cell), and each cell records the probability that it is occupied: black denotes obstacles (walls, table legs), white denotes free, traversable space, and gray denotes unknown regions not yet observed. This is the map form most commonly used by indoor wheeled robots and robotic vacuums, and it is the standard input to the ROS navigation stack (Chapter 13).

3. 3D Point Cloud

A spatial model composed of hundreds of thousands to hundreds of millions of three-dimensional points, each carrying coordinates (x, y, z) and optionally color or reflectance intensity. A 3D point cloud represents volumetric structure completely—staircases, ceiling conduits, tabletop heights—and is the basis for autonomous inspection, factory digital twins, and high-definition maps for self-driving cars (Chapter 17).

4. Semantic Map

On top of the geometric map, a "meaning label" is attached to each region or object: this patch is a door, that one a table, this is a corridor, that is a charging station. A semantic map lets the robot know not merely where the obstacles are but what those obstacles are, so that it can carry out tasks stated in human language such as "go to the registrar's office" or "avoid the stairs." The semantic map is where AI vision models (YOLO, SAM, VLM) meet SLAM, and it is the subject of Part IX (Chapters 32–34).

5. Topological Map

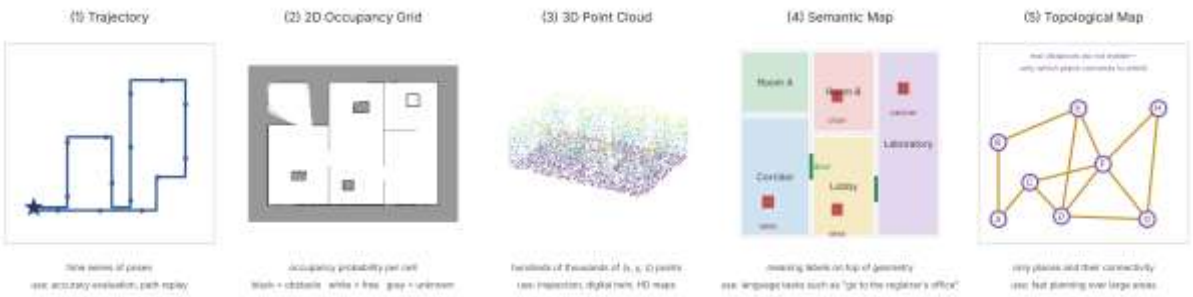
Discarding precise geometric coordinates, this form retains only places and the

connectivity between them, much as a metro route diagram ignores the true distances between stations and cares only about which station connects to which. A topological map requires very little memory and supports extremely fast path planning, which suits navigation over large areas; practical systems often adopt a hybrid map in which the topology is the skeleton and the grid map is the flesh.

Output Form	Data Structure	Typical Application	Representative Chapters
Trajectory	Time series of poses	Accuracy evaluation, path replay	Chapter 31
2D occupancy grid map	Probabilistic grid	Indoor navigation, robotic vacuums	Chapters 13–16
3D point cloud	Set of three-dimensional points	Spatial scanning, autonomous inspection, self-driving cars	Chapters 17, 18
Semantic map	Geometry + object labels	Semantic navigation, task-oriented robots	Chapters 24, 32
Topological map	Graph of nodes and edges	Fast planning over large areas	Chapters 3, 32

Table 1-3 The Five Output Forms of a SLAM System

Figure 1-2 Five Map Outputs for the Same Site



These five panels come from the same simulated site and the same scanning run; they differ only in what is kept and what is discarded. (1) The trajectory retains only the

robot's own sequence of poses and records nothing of the environment. (2) The occupancy grid map divides the floor into cells and records the occupancy probability of each cell; it is the protagonist of Part IV. (3) The point cloud retains three-dimensional structure and reveals wall heights and ceilings; it is the foundation of Part V. (4) The semantic map overlays meaning on geometry—this is a corridor, that is a laboratory, that block is a table; it is the subject of Chapter 32. (5) The topological map discards all geometry and keeps only places and their connectivity, like a metro route diagram.

The trade-off in information content deserves attention. Going from (1) to (5) is not a progression from worse to better—a topological map may hold one millionth of the data of a point cloud, but it is of no help whatsoever in steering a robot around a chair; conversely, for planning a route from the first-floor lobby to a third-floor laboratory, a point cloud is far too cumbersome. Practical systems commonly maintain several map types at once, using a topological map for global planning and a grid map for local obstacle avoidance—this is the "topology as skeleton, grid map as flesh" mentioned in Section 1.6.

You may treat this figure as a checklist of goals for the book: after Part IV you will have built (2) with your own hands, Part V gives (3), the visual SLAM of Part VI produces feature maps and variants of (3), and Chapter 32 produces (4). It does not matter if the details are unclear now; simply form the impression that SLAM has more than one kind of output.

Formulations and Procedures

1. The Probabilistic Definition of SLAM

Although the rigorous derivation waits until Chapter 11, this chapter states the

standard mathematical formulation of SLAM so that you have a precise anchor for what SLAM is estimating. Define the following symbols:

- x(t): the robot pose at time t, which in the 2D case is (x, y, θ).
- m: the map of the environment (which may be a grid map, a set of landmarks, or a point cloud).
- z(1:t): all sensor observations from time 1 to t (laser scans, image features, and so on).
- u(1:t): all control inputs or odometry readings from time 1 to t.

The full SLAM problem is to estimate the joint probability distribution over the entire trajectory and the map, given all observations and controls:

$$p(x(1:t), m \mid z(1:t), u(1:t))$$

The online SLAM problem estimates only the current pose and the map, marginalizing out the past trajectory, which suits real-time systems:

$$p(x(t), m \mid z(1:t), u(1:t))$$

In plain language: the answer SLAM seeks is "given everything I have seen and every path I have traveled, what are the most likely trajectory and map?" The reason it is written as a probability distribution rather than a single value is precisely that sensors are noisy and motion is imperfect—the answer SLAM returns always carries information about how certain it is. Particle-filter SLAM (Chapter 14), EKF-SLAM (Chapter 11), and graph-optimization SLAM (Chapter 12) are all, in essence, different numerical methods for solving this probabilistic problem.

2. The Standard Pipeline of a SLAM System

Modern SLAM systems generally adopt a front-end plus back-end architecture; the

pipeline is shown in Figure 1-3:

Figure 1-3 The Standard Pipeline of a SLAM System

Sensor data (laser scans / images / IMU / wheel odometry)

↓

Front end: feature extraction, data association, scan matching, odometry estimation

↓

Back end: filtering or graph optimization to estimate the optimal poses and map

↓

Loop closure: recognize revisited places and correct globally accumulated error

↓

Output: trajectory + map (2D grid map / 3D point cloud / semantic map)

The front end is responsible for turning raw sensor data into geometric constraints, and the back end for finding the most consistent answer under all those constraints, while loop closure detection is the key weapon against long-term drift. Part II of this book covers the sensors (the input to the front end), Part III the mathematics (the tools of the back end), and Parts IV through VI how various concrete systems realize this pipeline; from Part VII onward the book examines frontier extensions into miniaturization, multi-robot collaboration, and AI-based semantics.

1.7 The Technical Evolution of SLAM and the Contemporary Frontier

The preceding sections introduced the basic problem and the constituent parts of SLAM. This section widens the view to how the field has evolved over forty years and what is happening right now—which will help you understand where on that line of development each technique taught in the later chapters sits.

Evolution in Four Stages

Synthesizing recent survey literature, the development of SLAM can be divided roughly into four stages, and each stage has not replaced the one before it but has been layered onto it and reinforced it:

1. The hand-designed era (1986 to the 2010s): researchers used human understanding of geometry to hand-design feature detectors (such as SIFT and ORB in Chapter 20) and filtering algorithms (Chapter 11). This era established the mathematical skeleton of SLAM and is the principal content of Parts III through VI of this book.
2. The deep learning era (from about 2015): neural networks replaced some hand-designed modules—learned features, learned depth estimation, learned loop closure detection. The monocular depth estimation of Section 9.4 and the learned features of Section 20.7 both belong to this stage.
3. The radiance field era (from 2021): neural radiance fields (NeRF) and 3D Gaussian splatting (3DGS) redefined what a map is—a map is no longer merely a point cloud or a grid, but a field from which photorealistic images can be re-rendered.
4. The foundation model era (from 2023): vision-language models and robot

foundation models entered SLAM, moving robots from knowing shape to understanding meaning. This is the subject of Chapter 32.

How Radiance Fields Changed the Definition of a Map

The transition at the third stage deserves particular comment, because it changed a premise that had seemed self-evident: what a map should look like.

The map in traditional SLAM is an occupancy grid (Chapter 13) or a point cloud (Chapter 17)—these answer where something is, but cannot answer what the scene should look like when viewed from a given angle. NeRF represents the radiance field of the whole space implicitly with a neural network and can render photorealistic images from arbitrary viewpoints; 3DGS instead adopts an explicit representation, modeling the scene as a collection of three-dimensional Gaussian primitives, each with a position (mean), a covariance, a color, and an opacity.

The reason 3DGS rose so quickly in robotics is efficiency. Unlike implicit representations, which require querying a neural network at every point in space, the differentiable rasterization pipeline of 3DGS overcomes the bottleneck of slow training and rendering in large neural networks—precisely the obstacle that made NeRF-based systems difficult to use on robots. In addition, its explicit structure allows the map to be edited or deformed easily, which is a considerable advantage for map correction and adaptation to changing environments.

Clarification: Trade-offs Among Three Scene Representations

[Implicit] Approximates the radiance field with a neural network. Memory is small, but every query must run the network, so it is slow.

[Explicit] Renders directly from learned spatial features (voxels, hash grids, Gaussian primitives) with no neural component. Fast, but memory-hungry.

[Hybrid] Combines learned spatial features with a neural network. Both training and rendering are accelerated, and memory demand is likewise high.

For a robot the trade-off between speed and memory is always present—this is the same problem as the trade-offs of the milliwatt-class platforms in Chapter 25,

transposed to the level of map representation.

Four Directions at the Contemporary Frontier

Surveying the research of the past two years, the most active directions in SLAM can be grouped into four, and they make a practical map for any student who intends to go into research later:

- Robustness: enabling radiance-field SLAM to cope with dynamic environments and motion blur. Recent work includes dense neural SLAM designed specifically for blurred input and Gaussian mapping methods for evolving scenes; some systems also use the IMU to handle blur, rolling shutter, and severely noisy thermal imagery—which echoes the triple blow of high-speed flight discussed in Section 26.4.
- Collaboration: multiple robots building a Gaussian map together. It is worth noting that conventional collaborative SLAM has trended toward decentralization for scalability and robustness, yet because training three-dimensional Gaussians is computationally expensive, collaborative Gaussian

SLAM today is predominantly centralized—an instructive contrast with the architectural discussion in Section 27.3, and an illustration of how technology choices are governed by computational cost.

- Language: incorporating language capability into the map representation so that a robot can be queried about space in natural language. This is the subject of Chapter 32 and an extension of VLM navigation in Section 24.5.
- Foundation models: supporting monocular Gaussian SLAM with multi-view foundation models, or addressing fundamental questions such as what the best three-dimensional scene representation for a robot actually is. This line is pushing geometric SLAM and artificial intelligence toward the same meeting point.

Where This Book Stands

Having read this far, you may ask: if radiance fields and foundation models are so prominent, why does the greater part of this book still treat classical geometric methods?

The answer has three layers. First, practicality: the overwhelming majority of systems actually deployed in products today—robotic vacuums, self-driving cars, inspection quadrupeds, AR glasses—still have the geometric methods taught in this book as their backbone. They are mature, interpretable, and have well-defined computational requirements.

Second, foundation: radiance fields and foundation models have not replaced geometry; they are built on top of it. 3DGS SLAM still requires pose estimation, still requires loop closure correction, and still must contend with sensor limitations. Without understanding geometry, one cannot judge when these new methods will fail.

Third, pedagogy: the aim of this book is to train engineers who can exercise judgment, not users who chase terminology. The capstone projects of Chapter 31 and the discussion of Chapter 32 repeatedly stress the same point—technology will keep evolving, but the ability to understand limitations and choose the appropriate tool does not go out of date.

Case Studies

Case 1-1 The Xiaomi Robotic Vacuum: SLAM Enters Ordinary Homes

In 2017 Xiaomi released the first-generation Mijia robotic vacuum, which carried a rotating 2D LiDAR on top yet sold for less than half the price of conventional high-end machines. Its workflow is a complete demonstration of the SLAM loop introduced in this chapter: on start-up the LiDAR scans several thousand range points per second; as the robot drives it builds an occupancy grid map of the room with a particle filter algorithm while simultaneously localizing itself within that map; once the map is complete, the path planning module divides the rooms into zones and covers each zone with a boustrophedon path, ensuring neither repetition nor omission. This machine brought the term SLAM, which until then appeared only in research papers, into ordinary living rooms on a large scale for the first time. The lesson for the learner is this: SLAM is not some unreachable frontier technology—its core version runs on a processor of smartphone class, which is also why this book believes high school students can learn SLAM.

Case 1-2 The DARPA Subterranean Challenge: SLAM's Ultimate Proving Ground

From 2018 to 2021 the U.S. Defense Advanced Research Projects Agency (DARPA) held the Subterranean Challenge, which required robot teams to explore autonomously, build maps, and search for target objects in man-made tunnels, underground urban spaces, and natural caves. The underground environment combines every nightmare of SLAM: no GPS at all, dust and smoke that corrupt LiDAR returns, darkness that disables cameras, and narrow passages and vertical shafts that tax mobility. The competing teams—including groups from MIT, CMU, and ETH—generally used multi-sensor fusion of LiDAR, IMU, camera, and thermal imager, and formed heterogeneous teams of wheeled robots, quadrupeds, and UAVs that mapped collaboratively. The competition greatly advanced the practical maturity of LiDAR-inertial SLAM (Chapter 18) and multi-robot SLAM (Chapter 27), and it proved one thing: SLAM with a single sensor on a single robot is approaching maturity, and robustness in unknown, severe environments is the main battleground of the coming decade.

Case 1-3 The AR Measuring App on a Phone: Visual-Inertial SLAM in Your Pocket

Open the Measure app built into the iPhone, point the camera at a tabletop, move for a few seconds, and you can draw a virtual ruler in midair that conforms to the real world. Behind this is ARKit's visual-inertial SLAM: the phone camera tracks image feature points at 30 to 60 frames per second, the IMU measures motion several hundred times per second, and fusing the two yields a real-time estimate of the phone's six-degree-of-freedom pose together with a reconstruction of the planar structure of the tabletop. This case makes an excellent first classroom demonstration—no robot hardware is needed, since an industrial-grade SLAM system already lives inside every student's phone. Instructors can have students experiment with how the

ruler loses accuracy in front of a blank white wall (no texture), in a dim room, or when the phone is swung quickly, experiencing firsthand the camera limitations described in Section 1.3.

Case 1-4 From iMAP to 3DGS: What the Map Became in Five Years

The case studies in this book usually treat applications of mature technology; this one is an exception—it treats something now unfolding, so that you can feel the pace of change in this field.

In 2021 a work called iMAP demonstrated for the first time that real-time SLAM with a neural implicit representation was possible, and it is regarded as the starting point of radiance-field SLAM. In the few years since, related research has grown explosively: some work improved it with voxel feature grids, some switched to feature planes, and some combined point-cloud-based radiance fields. By the end of 2023, 3D Gaussian splatting had appeared and, thanks to its explicit representation and real-time rendering, rapidly became the new mainstream direction.

The speed of this change deserves attention: from a proof of concept to the research mainstay of an entire community took roughly three years. By contrast, ORB-SLAM, the protagonist of Chapter 22, was published in 2015 and remains the standard benchmark for visual SLAM to this day—undiminished after a decade.

The contrast raises a question worth pondering: what kind of technology endures, and what kind is quickly superseded? The author's observation is that methods which solve fundamental problems (epipolar geometry, graph optimization, loop closure detection) have very long lifetimes, whereas methods which solve questions of representation (in what format the map is stored, how the image is rendered) iterate

rapidly. This is also why the greater part of this book is devoted to the former.

The advice to students is this: understand each new technique as yet another representation, and ask what limitation of the older methods it removes and what price it pays (memory? computation? robustness?). With that framework, whatever new term appears next year, you will be able to place its value quickly.

Hands-On Activity

Activity 1-1 The World as the Sensor Sees It vs. the Actual Map

Purpose: to experience the gap between recording an environment from a first-person viewpoint and viewing it as a map from above, and to understand the data transformation problem that SLAM must solve.

Equipment

- One smartphone (for video recording), or one wheeled teaching robot.
- Two sheets of graph paper, a pencil, and a ruler (or tape measure).

Procedure

1. Form groups of three to four and choose an indoor route: for example, start at the rear door of the classroom, go straight along the corridor, turn right, pass the water fountain, and return to the front door.
2. First-person recording: one student holds the phone at walking height and records video over the entire route, narrating the landmarks seen along the way ("there is a fire extinguisher on the left," "a glass door ahead").
3. Draw a map from the record: back in the classroom, with the video as the only permitted reference and no return to the site, the group works together to

draw on graph paper the world as the sensor saw it—the route, the corners, the relative positions of landmarks—annotated with estimated distances.

4. Field measurement: return to the site with the tape measure and measure the corridor length, the positions of the corners, and the spacing between landmarks, then draw the actual map on the second sheet of graph paper.
5. Comparison and discussion: overlay the two drawings, measure where the error is largest, and discuss where the error came from—a misjudged sense of distance? an incorrectly estimated turning angle? landmarks recorded out of order?

Discussion Questions

1. Did the error in your drawn map grow larger or smaller the farther you traveled? How does this relate to the drift described in the text?
2. If the route returns to its starting point, did the endpoint of your drawing coincide with the start? If not, how should the whole map be corrected? Which term in the text does this operation correspond to?
3. Which landmarks in the video were easiest to recognize and which were easiest to confuse? What does this suggest about the kind of environment camera-based SLAM prefers?

Extended Challenge (Undergraduate)

Using the video recorded on the phone as input, run an open-source visual SLAM system on a computer (for example ORB-SLAM3 in monocular mode), output the estimated trajectory, and compare it three ways against your hand-drawn map and the surveyed map. This challenge previews the content of Chapters 22, 23, and 30.

Further Reading

1. C. Cadena et al., "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age," IEEE Transactions on Robotics, vol. 32, no. 6, 2016. (The authoritative survey of thirty years of SLAM; cited repeatedly throughout this book.)
2. H. Durrant-Whyte and T. Bailey, "Simultaneous Localization and Mapping: Part I & Part II," IEEE Robotics & Automation Magazine, 2006. (The classic introductory tutorial on SLAM; establishes the theoretical foundation.)
3. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (The bible of probabilistic robotics and the principal reference for the mathematical foundations in Part III.)
4. MIT OpenCourseWare, 16.485 Visual Navigation for Autonomous Vehicles (VNAV). (A complete, freely available course on visual navigation, with lecture notes and labs.)
5. CMU 16-833 Robot Localization and Mapping course website. (A graduate SLAM course organized around state estimation and optimization.)
6. Cyrill Stachniss (University of Bonn), Robot Mapping lecture series, freely available on YouTube. (Explains SLAM algorithms with intuitive animations; well suited to self-study.)
7. C. Campos et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE Transactions on Robotics, 2021. (The original paper on the protagonist of Chapter 23.)

A Guide to the Whole Book: A Map of Thirty-Four Chapters

As a closing to Chapter 1, a bird's-eye view of the whole book is given here. You need

not memorize it now, but whenever you feel lost later, you can come back and see which stage of the journey you are on.

Figure 1-4 The Learning Path Through the Nine Parts of the Book



Three Possible Reading Routes

- High school / introductory route: Chapters 1–3 to establish the concepts → Chapters 8 and 9 to learn about LiDAR and cameras → Chapters 13 and 16 to understand 2D mapping and robotic vacuums → Chapters 28 and 29 for hands-on simulation. No deep mathematics is required anywhere along the way, and most of the hands-on activities in each chapter can be completed in simulation.
- Full undergraduate route: read Parts I through VI in order, carrying out the implementation chapters of Part VIII in parallel. The mathematics of Part III is the critical threshold; pairing it with the hands-on activities of Chapters 11 and 12 is recommended for building intuition.

- Capstone / research route: skim Parts I through III quickly to establish a common vocabulary, then go deep according to your research direction—Part V for LiDAR, Part VI for vision, Part VII for miniature platforms, Part IX for AI integration—and converge with the project planning of Chapter 31.

Whichever route you choose, the rhythm of study this book recommends is the same: build intuition first, then understand the principle, then verify by doing, and finally seek out the failure boundary deliberately. Section 28.7 expands this rhythm into five concrete stages.

Summary

1. The core problem SLAM (simultaneous localization and mapping) must solve is this: with no map and no knowledge of its own position, a robot must move, build a map, and correct its own pose, all at the same time.
2. Localization requires a map and mapping requires localization, so the two form a chicken-and-egg circular dependency. SLAM's solution is to treat them as a single estimation problem in which observations of the environment continually correct both, with loop closure detection used to distribute long-accumulated error.
3. A robot's sensors divide into proprioceptive sensors (wheel odometry and IMU: continuous but drifting) and exteroceptive sensors (camera, LiDAR, radio, geomagnetic field: providing an environmental reference but each with its own weakness). The essence of SLAM is sensor fusion in which each compensates for the other's shortcomings.

4. The four major applications—autonomous driving, UAVs, robotic vacuums, and AR / VR—share the same SLAM foundation, differing only in sensor suite, computational resources, and accuracy requirements.
5. The output of a SLAM system takes five forms—trajectory, 2D occupancy grid map, 3D point cloud, semantic map, and topological map—each corresponding to different application needs.
6. Mathematically, SLAM estimates the joint probability distribution over the trajectory x and the map m given the observations z and the controls u; modern systems solve it with a front-end / back-end / loop-closure architecture.

In the next chapter we visit the SLAM courses of world-renowned universities including MIT, CMU, Stanford, ETH / UZH, and Bonn / Freiburg, to see how leading institutions arrange the learning path for this subject, and to distill from them this book's pedagogical main thread of sensors → mathematics → front end → back end → loop closure → capstone.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 2

A Guided Tour of SLAM Courses at World-Renowned Universities

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 1 established what problem SLAM addresses; this chapter answers a practical question: how do the best universities in the world teach this subject? This is more than a broadening of horizons—the overall architecture of this book is distilled from the common skeleton of those courses, so understanding this chapter is equivalent to holding the key to the organizing logic of the twenty-nine chapters that follow.

Figure 2-0 How Chapter 2 Connects to the Rest of the Book

[Building on the previous chapter] The six sensors, the chicken-and-egg problem, and the five map representations of Chapter 1 all have counterpart units in the courses surveyed here.

[This chapter is responsible for] touring the SLAM courses of five leading universities and distilling their common architecture.

[Where it leads]

2.1 MIT VNAV → IMU preintegration in Chapter 7, manifold optimization in Chapter 12, visual-inertial SLAM in Chapter 23

2.2 CMU 16-833 → probabilistic filtering in Chapter 11, factor graphs and GTSAM in Chapter 12

2.3 Stanford → navigation system architecture in Chapter 3, planning and control in Chapter 29

2.4 ETH / UZH → camera models in Chapter 9, epipolar geometry in Chapter 21, UAV VIO in Chapter 26

2.5 Bonn / Freiburg→ particle filtering in Chapter 11, occupancy grids in Chapter 13, Gmapping in Chapter 14

2.6 The common architecture → the basis on which the nine parts of this book are organized

[Prerequisites] None. This chapter is a survey and involves no mathematical

derivation.

Learning Objectives

A craftsman who wishes to do his work well must first sharpen his tools; a student who wishes to go deep must first survey the road. This chapter lets you stand on the shoulders of giants by touring the SLAM-related courses of five leading universities and seeing how first-rate institutions arrange the learning path for this subject. After completing this chapter, you should be able to:

1. Name the SLAM course, the principal instructor, and the pedagogical focus at each of the five academic centers: MIT, CMU, Stanford, ETH / UZH, and Bonn / Freiburg.
2. Compare the entry points taken by these courses—visual navigation, probabilistic inference, system integration, visual-inertial estimation, and probabilistic mapping—and explain their similarities and differences.
3. Summarize the common architecture of SLAM curricula: sensors → mathematics → front end → back end → loop closure → capstone, and map it onto the nine parts of this book.
4. Know where the free learning resources published by each school (OpenCourseWare, YouTube lecture videos, public lecture notes) can be found and how to use them.
5. Plan a self-study route through SLAM that suits your own level and goals.

Core Concepts

Keyword	English / Designation	Explanation
視覺導航	Visual Navigation	Robot navigation with the camera as primary sensor; the main thread of the MIT VNAV course.
狀態估測	State Estimation	The mathematical methods for inferring system state (pose, velocity, map) from noisy measurements.
機率圖模型	Probabilistic Graphical Model	A model expressing relations among random variables as nodes and edges; the theoretical language of the modern SLAM back end.
因子圖	Factor Graph	A probabilistic graphical model that represents measurements as factors; the core tool of CMU 16-833.
視覺里程計	Visual Odometry, VO	Estimating camera motion frame by frame from images alone; the final project of the UZH course.
視覺慣性里程計	VIO	Odometry fusing camera and IMU; the core technology of phone AR and UAVs.
流形最佳化	Manifold Optimization	The mathematical technique of optimizing on curved spaces such as the rotation group; a signature topic of MIT VNAV.
佔據格建圖	Occupancy Grid Mapping	The classic mapping method systematized by the Freiburg / Bonn courses.
開放式課程	OpenCourseWare, OCW	Complete course materials published free by a university; MIT OCW is the best-known example.
大規模開放線上課程	MOOC	Online courses on platforms such as edX and Coursera; the ETH Autonomous Mobile Robots course has a MOOC

Keyword	English / Designation	Explanation
		version.

2.1 MIT VNAV: Visual Navigation, Self-Driving Cars, UAVs, and Optimization

Course 16.485, Visual Navigation for Autonomous Vehicles (VNAV), offered by the Department of Aeronautics and Astronautics at MIT and taught by the SPARK Lab led by Professor Luca Carlone, is one of the most representative graduate courses in visual navigation anywhere in the world. Professor Carlone is himself a leading figure in SLAM—he is one of the principal authors of the 2016 SLAM survey listed in the Further Reading of Chapter 1.

Course Thread and Distinctive Features

The design of VNAV follows a clear logical line: begin from geometry, converge through optimization, and validate on UAVs. The content falls roughly into four blocks:

- Foundations of three-dimensional geometry: representations of rotation (rotation matrices, quaternions), rigid-body transformations, camera models, and two-view geometry—corresponding to Chapters 10 and 21 of this book.
- Optimization on manifolds: a rotation is not an ordinary vector and cannot simply be added or subtracted; optimization must be carried out on a curved space called a manifold. This is VNAV's most distinctive mathematical topic and the key to understanding modern SLAM back ends—Chapter 12 of this book introduces its spirit in more accessible terms.
- The visual front end: feature extraction and matching, visual odometry, place recognition, and loop closure detection—corresponding to Chapters 20 through 22 of this book.

- SLAM back end and systems: pose-graph optimization, bundle adjustment, visual-inertial fusion, and frontier extensions into semantic understanding—corresponding to Chapters 12, 23, and 32.

Pedagogy: Team Projects with UAVs

The most striking feature of VNAV is the arrangement of its laboratory work: students work in teams on real UAVs, completing week by week a sequence of experiments from attitude control and trajectory tracking through visual features and visual odometry to a complete navigation system, and closing the term with a competition-style project. This design, in which the mathematics of each week is immediately validated on a flying vehicle, is precisely the spirit emulated by the implementation chapters of Part VIII of this book.

The complete VNAV materials (lecture notes, assignments, and lab guides) are freely available on MIT OpenCourseWare (course 16.485, Fall 2020), making it one of the most complete free resources for self-study in visual navigation. Readers of this book are advised to take on the VNAV notes after finishing the mathematical foundations of Part III, which will make the effort go considerably further.

2.2 CMU Robot Localization and Mapping: Localization, Mapping, and Back-End Optimization

The Robotics Institute at Carnegie Mellon University (CMU) is the oldest and largest robotics research institution in the world; its course 16-833, Robot Localization and Mapping, is taught by Professor Michael Kaess. Professor Kaess is a representative figure in SLAM back-end algorithms—the iSAM / iSAM2 incremental smoothing algorithms and the GTSAM library, widely used in modern graph-optimization SLAM,

come from the research group he shares with Professor Frank Dellaert.

Course Thread and Distinctive Features

The course description of 16-833 states at the outset that localization and mapping are foundational capabilities for mobile robots operating in the real world, and that more challenging than either problem alone is their combination—SLAM; a robust solution must handle the uncertainty inherent in sensor measurements while providing localization and map estimates in real time. The course therefore centers on efficient probabilistic inference algorithms at the intersection of linear algebra and probabilistic graphical models, covering:

- The Bayes filter family: the Kalman filter, the extended Kalman filter (EKF-SLAM), the particle filter, and FastSLAM—corresponding to Chapters 11 and 14 of this book.
- Smoothing and graph optimization: factor graphs, least squares, and the incremental iSAM solver—corresponding to Chapter 12.
- Data association and loop closure: which landmark in the map does a given measurement actually correspond to? This is the step at which SLAM most easily goes wrong.
- State-of-the-art systems: in the later part of the course students read contemporary SLAM system papers and see how theory is reduced to practice.

A Comparison with MIT VNAV

If VNAV enters from vision and geometry, 16-833 enters from probability and inference: VNAV devotes substantial time to cameras and manifolds, while 16-833 treats filtering, factor graphs, and sparse linear algebra exhaustively. Taken together, the two courses cover the complete knowledge map of the modern SLAM front end

and back end—which is also why this book separates Part II (sensors) from Part III (mathematics) before integrating them into systems in Parts IV through VI.

2.3 Stanford Principles of Robot Autonomy: Localization, Perception, SLAM, and Planning

Principles of Robot Autonomy (course AA274A / CS237A) at Stanford University is offered jointly by the Department of Aeronautics and Astronautics and the Department of Computer Science and taught by the Autonomous Systems Laboratory (ASL) led by Professor Marco Pavone and colleagues. Unlike the previous two courses, this one is positioned not to dig deep into SLAM as a single topic but to walk through the entire system of an autonomous robot.

Course Thread and Distinctive Features

The Stanford course covers the complete technical stack of an autonomous robot, within which SLAM is the link that connects what comes before to what comes after:

- Motion and control: kinematic models, trajectory tracking, and feedback control—the basis of a robot that can move.
- Perception: camera models, image features, object detection, and machine learning for perception—a robot that can see.
- Localization and SLAM: Bayes filtering, Monte Carlo localization, and EKF-SLAM—a robot that knows where it is.
- Planning and decision making: path planning (A*, RRT), trajectory optimization, and task-level decision making—a robot that knows where to go and how to get there.

- System integration: ROS serves as the backbone all term, and at the end students integrate a complete autonomous navigation system in simulation and on a physical robot.

What This Means for the Present Book

Stanford's design answers an important question: what is SLAM for once you have learned it? The answer is that SLAM is the hub of an autonomous system, taking in sensor data from below and supporting path planning and task execution above. The navigation system overview of Chapter 3 and the navigation stack of Chapter 29 in this book adopt exactly this systems view, so that you always know what role the algorithm in front of you plays in the robot as a whole. This system-first, algorithm-second pedagogy suits project-based courses at the high school and undergraduate level particularly well.

2.4 ETH Zürich / UZH: Visual-Inertial Odometry, VIO, and MAV Navigation

ETH Zürich and the University of Zurich (UZH) are geographically adjacent, share faculty, and together form the heart of mobile robotics research in Europe. Two complementary course lineages are found here, representing the systems school and the vision school respectively.

ETH: Autonomous Mobile Robots

The Autonomous Systems Lab (ASL) at ETH, led by Professor Roland Siegwart, has long offered the course Autonomous Mobile Robots, whose textbook of the same name (MIT Press, by Siegwart, Nourbakhsh, and Scaramuzza) is one of the most widely used texts in mobile robotics, covering kinematics, perception, localization, and

navigation planning. The course is also offered as a MOOC on edX, free to learners worldwide. In addition, ASL's research results in visual-inertial estimation—open-source systems such as ROVIO and OKVIS—have become important infrastructure for localization on UAVs and mobile devices; we will meet them again in Chapters 23 and 26.

UZH: Vision Algorithms for Mobile Robotics

Professor Davide Scaramuzza of the Robotics and Perception Group (RPG) at UZH is an international leader in visual odometry and event camera research. His course Vision Algorithms for Mobile Robotics covers image formation, filtering, feature extraction, multi-view geometry, dense reconstruction, tracking, image retrieval, event-based vision, visual-inertial odometry, and SLAM—and the course website states outright that these are the algorithms behind Apple ARKit, Microsoft HoloLens, Meta Quest, and the NASA Mars helicopter.

The most elegant feature of this course is its building-block final project: each lecture is followed by a matching exercise in which students implement one module of a visual odometry pipeline (feature extraction, matching, pose estimation, triangulation, and so on), and at the end of the term all the blocks are assembled into a complete, runnable visual odometry pipeline validated on real datasets such as KITTI. All lecture videos and notes are freely available on the RPG website and YouTube. The knowledge structure of Chapter 9 and Chapters 20 through 21 of this book draws heavily on the ordering used in this course.

2.5 University of Bonn / Freiburg: Robot Mapping and the SLAM Courses of Cyrill Stachniss

The University of Freiburg and the University of Bonn in Germany are the founding centers of probabilistic robotics. Professor Wolfram Burgard (a co-author of Probabilistic Robotics) long taught Introduction to Mobile Robotics and Robot Mapping at Freiburg; his student Professor Cyrill Stachniss subsequently continued and extended that teaching line at the University of Bonn with a complete course centered on SLAM and environmental modeling.

Course Thread and Distinctive Features

The great strength of the Freiburg / Bonn line is that it presents probabilistic methods with exceptional system and exceptional gradualness:

- Probabilistic foundations: Bayes' rule, motion models, and observation models—corresponding to Chapter 11 of this book.
- Filter-based SLAM: EKF-SLAM, the UKF, particle filtering and FastSLAM, and the RBPF principle behind Gmapping—corresponding to Chapters 11 and 14.
- Graph-optimization SLAM: least squares, pose graphs, loop closure, and robustification—corresponding to Chapter 12.
- Mapping methods: occupancy grid mapping, laser scan matching, and map representations—corresponding to Chapter 13.

The Resource Friendliest to Self-Learners

Professor Stachniss has uploaded complete recordings of his courses over the years to YouTube, freely available, ranging from five-minute clips on a single concept to full ninety-minute lectures, with animations and blackboard derivations. SLAM learners worldwide regard them as the audiovisual resource best suited to self-study. It is worth noting in particular that Gmapping, the protagonist of Chapter 14 of this book, came out of the research of Stachniss together with Grisetti and Burgard—

watching the original authors explain it themselves after reading Chapter 14 is learning at its most authentic.

2.6 Mapping the Five Courses onto the Chapters of This Book

Before summarizing the common architecture, here is something practical: a unit-by-unit mapping of the core topics at the five schools onto the chapters of this book. The table serves two purposes—when you want to go deeper into a topic, it tells you which international course to consult; and when you take one of those online courses, it tells you which chapter of this book to refer back to.

Table 2-1 Course Units at the Five Schools and the Corresponding Chapters of This Book

[MIT VNAV: Visual Navigation and Optimization]

Image formation and camera models → Chapter 9

Optimization on manifolds → Chapter 10 (rotation representations), Chapter 12 (graph optimization)

IMU preintegration → Section 7.5

Visual-inertial odometry → Chapter 23

[CMU 16-833: Localization, Mapping, and the Back End]

Bayes and particle filtering → Chapter 11

EKF-SLAM → Section 11.6

Factor graphs and iSAM2 → Section 12.6, Chapter 27 (multi-robot)

Loop closure detection → Section 20.7, Section 22.5

[Stanford Robot Autonomy: System Integration]

Robot system architecture → Chapter 3

Perception and state estimation → Part II, Chapter 11

Path planning and control → Section 3.4, Section 29.6

[ETH / UZH: Vision and UAVs]

Feature detection and matching → Chapter 20

Multi-view geometry → Chapter 21

Visual-inertial fusion → Chapter 23

Micro aerial vehicle navigation → Chapter 26

[Bonn / Freiburg: Probabilistic Mapping]

Occupancy grid maps → Chapter 13

Scan matching and ICP → Section 13.3, Section 17.3

Gmapping and the RBPF → Chapter 14

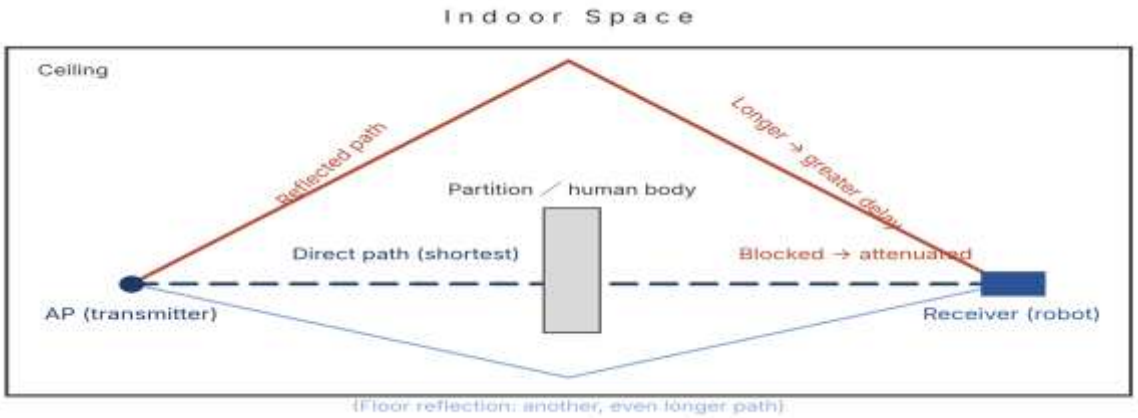
Graph-optimization SLAM → Chapter 12, Chapter 15

Looking at the table, you will notice one thing: no single school's course covers every topic. MIT leans toward visual-inertial estimation and optimization, Bonn toward probabilistic mapping, Stanford toward system integration. This is entirely reasonable—a fourteen-week course cannot exhaust an entire field, and each school

naturally selects according to its research strengths.

The editorial strategy of this book is therefore to take the strengths of each school and fill the gaps: the probabilistic mapping of Bonn forms the backbone of Parts III and IV; the visual-inertial work of MIT and Zurich is the main thread of Part VI; the factor graphs of CMU are the core of Chapter 12; the systems view of Stanford runs through Chapter 3 and Part VIII; and to these are added topics that courses in Taiwan actually need but international courses treat lightly—indoor positioning in Chapter 4, geomagnetic navigation in Chapter 6, robotic vacuums in Chapter 16, and the AI semantic navigation of Part IX.

Figure 2-1 Coverage Comparison Across the Five Courses



Drawing the mapping of Table 2-1 as a matrix makes the phenomenon described in the preceding paragraph immediately visible: five rows for the five schools, thirteen columns for the main topics of SLAM, dark cells for the core units of that course, light cells for topics covered, and blank cells for topics not covered.

Read across the rows and every school shows a clear center of gravity: the dark cells of MIT and the two Zurich institutions cluster on the left in camera models, multi-view geometry, and visual-inertial estimation; those of Bonn / Freiburg cluster in the

middle in probabilistic filtering, occupancy grid mapping, and scan matching; CMU falls on probability and factor graphs; and Stanford is the only one to treat system integration and ROS as core. This is exactly the "selection according to research strength" noted in Section 2.6.

Reading down the columns is more interesting still. Four columns are nearly blank throughout—indoor positioning together with satellite and geomagnetic navigation, consumer applications, AI semantic navigation, and miniature platforms (covered by only two schools). This does not mean these topics are unimportant; it means they either belong to industry rather than the research frontier, or evolve too quickly to have settled into a fourteen-week syllabus.

The last row is this book. It fills all thirteen columns, but you should understand that this does not make the book deeper than those courses—quite the opposite. The task of a textbook is to build a complete skeleton and a common vocabulary, while an international course provides depth in one area. The correct use of the two is complementary: use this book to cover the ground, and the courses to dig down.

2.7 The Common Architecture of SLAM Curricula: Sensors → Mathematics → Front End → Back End → Loop Closure → Capstone

Having visited five schools, if we pull the lens back we find something interesting: although the entry points differ—MIT from geometry, CMU from probability, Stanford from systems, Zurich from vision, Germany from mapping—the knowledge skeletons of these courses are strikingly consistent and can all be summarized in six stages:

Figure 2-2 The Common Architecture of SLAM Curricula

(1) Sensors: learn the data sources—camera, LiDAR, IMU—and their error characteristics

↓

(2) Mathematics: coordinate transformations, probability, filtering, optimization—the language for handling uncertainty

↓

(3) Front end: feature extraction, data association, scan matching, odometry

↓

(4) Back end: filtering or graph optimization to obtain the most consistent trajectory and map

↓

(5) Loop closure: recognize revisited places and correct globally accumulated error

↓

(6) Projects: integrate and validate on a real or simulated platform to complete a working system

The nine-part structure of this book is the result of expanding those six stages and localizing them: Part II corresponds to sensors; Part III to mathematics; Parts IV through VI realize the front end, back end, and loop closure in three kinds of system—2D LiDAR, 3D LiDAR, and vision; Parts VII and VIII correspond to the capstone; and Part IX extends to the rapidly evolving frontier of AI semantic navigation that the courses are still catching up with.

School / Course	Principal Instructor	Entry Point	Signature Feature	Corresponding Material in This Book
MIT 16.485 VNAV	Luca Carlone	Geometry and optimization	Manifold optimization, UAV projects	Chapters 10, 12, 21
CMU 16-833	Michael Kaess	Probabilistic inference	Factor graphs, iSAM / GTSAM	Chapters 11, 12
Stanford AA274A	Marco Pavone et al.	System integration	Full ROS stack, end-of-term integration	Chapters 3, 28, 29
ETH Autonomous Mobile Robots	Roland Siegwart	General survey of mobile robotics	Classic textbook, edX MOOC	Chapters 3, 7
UZH Vision Algorithms	Davide Scaramuzza	Vision and VIO	Building-block VO project, event cameras	Chapters 9, 20, 21
Bonn / Freiburg	Stachniss, Burgard	Probabilistic mapping	Fully public on YouTube, Gmapping	Chapters 11, 13, 14

Table 2-2 Overview of the Five SLAM Courses and Their Correspondence to the Chapters of This Book

Formulations and Procedures

1. Unifying the Five Courses Through the Estimation Problem

Although the five courses differ greatly in style, all of them revolve around the same central formula introduced in Chapter 1. Recall the probabilistic definition of online SLAM:

$$p(x(t), m \mid z(1:t), u(1:t))$$

The differences among the five courses amount to different strategies for attacking this one formula:

- CMU and Freiburg / Bonn concentrate on how to solve for this probability distribution: filtering methods (Chapter 11) update it recursively over time, while graph optimization (Chapter 12) converts it into a least-squares problem.
- MIT and UZH concentrate on where the observations z come from: how a camera image becomes a geometric constraint—features, epipolar geometry, and reprojection error (Chapters 20 and 21).
- Stanford concentrates on what to do with the solution: how pose and map are fed into path planning and control to constitute a complete autonomous system (Chapters 3 and 29).

2. A Procedure for Planning a Self-Study Route Through SLAM

Drawing together the resources of the five schools, this book recommends the following self-study route; you may choose your entry point according to where you start:

Figure 2-3 A Recommended Procedure for Planning Self-Study in SLAM

Starting point: do I have a background in programming and linear algebra?

├─ Not yet → begin with the ETH Autonomous Mobile Robots MOOC (concept-oriented, lowest barrier)

└─ Yes → proceed to the next step

↓

To take the vision route → UZH course videos + implement the visual odometry building blocks week by week

To take the probabilistic route → Stachniss's YouTube course + the derivations in Part III of this book

↓

Advanced depth → MIT VNAV OCW notes (manifold optimization) + CMU 16-833 (factor graphs)

↓

System integration → complete the implementation of Part VIII and the capstone of Chapter 31 using ROS

2.8 The Evolution of Open Education: From MOOCs to Embodied Intelligence Courses

The courses introduced in this chapter share one feature: their materials are public. This seems self-evident today, but it is a change of the past decade or so, and it is still unfolding. Understanding that evolution is useful when planning your own learning path.

Change in Three Stages

How knowledge is obtained has changed fundamentally over the past dozen years. The process began roughly in 2011, when professors at Stanford opened their artificial intelligence course online to everyone; before that, artificial intelligence could generally be studied only through a limited number of university courses or through books and papers, and those resources were concentrated in developed countries, so learners in emerging markets had difficulty reaching them. The appearance of massive open online courses began what is called the democratization of knowledge.

For a field like SLAM, which demands a great deal of background knowledge, this change was especially consequential. Before the 2010s, studying SLAM meant, in practice, gaining admission to one of a handful of laboratories; today any student in Taiwan with an internet connection can obtain the same lecture notes and assignments as a student at MIT. Professor Stachniss's posting of his entire course to a video platform, mentioned in Case 2-2 of this book, is a representative instance of this trend.

Stage Two: Hardware-Oriented Open Courses

Video-only courses have a limitation: students can follow the theory but have no robot on which to verify it. In response to this gap, open courses combined with physical platforms appeared. In March 2021, for example, the Duckietown Foundation released the first hardware-based massive open online course in artificial intelligence and robotics, titled "Self-Driving Cars with Duckietown," offered free on a public platform. The course has learners work hands-on so that a real robot makes its own decisions and completes broadly defined tasks, proceeding step by step from theory through implementation to deployment in simulation and on real robots; a robot platform was designed from scratch specifically to support this learning experience.

This model agrees with the design philosophy of Part VIII of this book: first get the workflow and the parameters right in simulation, then move to inexpensive physical hardware for verification. The containerized uniform environment recommended in Section 28.1 and the simulation-before-hardware path of Chapter 29 follow the same line of thought.

Stage Three: Embodied Intelligence Enters the Curriculum

In the past two years the content of such courses has changed again. Breakthroughs in large language models are being applied to robotics, and embodied intelligence allows robots to understand natural-language instructions and convert them into physical action; meanwhile organizations such as Google DeepMind and OpenAI are developing robot foundation models that learn general manipulation skills from enormous datasets, and understanding how to fine-tune such models for a specific application is becoming an essential skill.

At the same time another direction is entering the curriculum: because of the limitations of cloud connectivity, demand for on-device AI inference keeps rising, and modern courses increasingly cover model optimization techniques (quantization, pruning, knowledge distillation) together with edge AI frameworks and hardware acceleration. This is exactly the material discussed in Chapter 25 (milliwatt-class platforms) and Section 33.8 (edge deployment) of this book—these two topics were included precisely to align with this pedagogical trend.

Three Implications for How to Study

1. Simulation first: researchers have noted that simulation projects can teach roughly eighty percent of the necessary concepts at about one tenth the cost of hardware-based study; once the foundations are solid, hardware work proceeds

far more efficiently, whereas attempting complex projects too early leads to frustration. This agrees entirely with the recommendations of Part VIII.

2. Graduated challenge: the recommended learning structure advances level by level—basic motion (make the robot trace a square, a circle, a figure eight), sensor integration (add obstacle detection and avoidance), environment mapping (build a spatial map with SLAM), object manipulation (visual detection and grasping), and autonomous missions (integrate all capabilities into a complete task). In this book these five levels correspond to Chapter 3, Chapter 8, Chapters 13 through 15, Chapter 24, and Chapter 31 respectively.

3. Online courses and textbooks are complementary: online courses excel at dynamic demonstration and currency; textbooks excel at systematic organization and repeated reference. The recommended practice is to build the framework with this book, supplement intuition with online courses, and verify understanding with open-source projects—a division of labor that extends the method for reading open-source projects given in Section 28.6.

Case Studies

Case 2-1 From Course Assignment to Industry Change: The Story of GTSAM

The factor graphs and incremental smoothing taught in CMU 16-833 are not mere classroom theory. The GTSAM (Georgia Tech Smoothing and Mapping) library developed by Professors Kaess and Dellaert packages this theory into an efficient open-source C++ toolbox that is now standard equipment in robotics companies and laboratories worldwide—its traces appear in the back-end optimization of underwater

robots, self-driving cars, and AR devices alike. The lesson is this: the core assignment of a course (in 16-833, students implement a factor graph solver themselves) may be the prototype of tomorrow's industrial infrastructure. When Chapter 12 introduces g2o, Ceres, and GTSAM, you will use this tool that grew out of a classroom with your own hands.

Case 2-2 The SLAM Professor on YouTube: The Power of Open Knowledge

Beginning in 2013, Professor Stachniss published complete recordings of the Robot Mapping course from his Freiburg years and has continued to update them at Bonn, accumulating several hundred videos viewed millions of times. Countless self-learners around the world—including many graduate students and engineers in Taiwan—entered SLAM through these free videos. Likewise, MIT placed VNAV in full on OCW, UZH published all its notes and videos, and ETH launched a MOOC. This wave of openness at leading universities means that the barrier to studying SLAM has shifted from whether one can enter an elite school to whether one has a good learning map and sustained persistence. This book aims to be exactly that learning map, written in Traditional Chinese.

Case 2-3 A Taiwanese Student's Perspective: From the University Classroom to International Courses

For learners in Taiwan, the five courses make excellent material for a second, more advanced pass, but tackling the original materials directly usually meets two obstacles: the double barrier of mathematical notation and English terminology, and the absence of a locally operable experimental site. The strategy of this book is to build locally and connect internationally: first establish the concepts and the implementation firmly in

Chinese and in familiar settings (classroom mapping, campus inspection, robotic vacuums), and then point the Further Reading of each chapter to the corresponding unit of an international course. Take Gmapping in Chapter 14: first complete the mapping exercise with a robot in a university classroom, then go back and watch Stachniss explain the RBPF—most students find that the previously abstract particle filter suddenly becomes visible.

Case 2-4 Why a Textbook Is Still Needed: University Courses in the MOOC Era

This chapter has introduced a great many free online resources, which naturally raises a pointed question: if the courses of top universities can be watched for free, why enroll in a course at all? And why does this book need to exist?

The question is indeed debated seriously in education. Scholars have observed that as the digital transformation of teaching advances rapidly, universities have begun to face significant competition from massive open online courses and must therefore differentiate themselves in order to explain to students what the added value of a three- to five-year degree actually is.

In the author's view, textbooks and classrooms retain at least four kinds of value in this era that online video cannot easily replace:

- Localization: international courses are taught in English with European and American settings as examples. This book deliberately uses the professional terminology current in Taiwan (see the discussion of usage in Chapter 1), takes the university campus and the Taiwanese high school classroom as its sites, and includes topics of particular local concern (such as the behavior of geomagnetic navigation in steel-frame buildings in Chapter 6 and robotic vacuums in Chapter

16).

- Systematicity: online courses stand alone, and students must assemble the pieces themselves. The nine-part structure of this book, the "Where This Chapter Sits in the Book" section opening every chapter, and the cross-references between chapters all exist to supply that overall view.
- Referenceability: video makes it hard to look a formula or a parameter back up quickly. When you are stuck during implementation, a textbook's index, table of contents, and chapter numbering are far more efficient than rewatching a video.
- Transmission of failure experience: this is the aspect this book cultivates most deliberately. International courses usually present successful results, whereas every chapter here discusses common failures—the four failure modes of Section 14.6, the safety principles of Section 26.5, the debugging order of Section 29.7. This experience, drawn from real teaching practice, is something no video will tell you.

The position of this book is therefore clear: it does not compete with online courses but serves as their map and annotation. Students who use both—building the skeleton with this book and adding flesh with the international courses—will gain more than from any single source. That is also the real message behind the "power of open knowledge" emphasized in Case 2-2: resources are not the problem; organizing them is.

Hands-On Activity

Activity 2-1 Exploring International SLAM Course Resources and Writing a Comparison Report

Purpose: to locate and browse the public resources of the five schools firsthand, build your own map of learning resources, and practice reading English course websites.

Procedure

1. Form groups of two or three, each group claiming one school: MIT VNAV, CMU 16-833, Stanford AA274A, UZH Vision Algorithms, or the Bonn courses of Stachniss.
2. Search for and open that course's official website or its OCW / YouTube page, and find: the course title and number, the principal instructor, and the weekly syllabus of the most recent offering.
3. Choose one week whose topic title you can already understand (for example Kalman Filter or Feature Extraction), and read the first five pages of that week's notes or the first ten minutes of the video.
4. Prepare a one-page slide answering: what does this course assume students already know? What does it teach first? What do the assignments or projects look like? Which part corresponds to which chapter of this book?
5. Each group presents for three minutes, and the class together completes a comparison table of the five courses, posted in the classroom or on the course website as a reference map for the term.

Discussion Questions

1. Of the five courses, which has the lowest barrier to entry? Which demands the strongest mathematical background? On what evidence do you base your judgment?
2. If you could follow only one free resource alongside this book for self-study, which would you choose, and why?

3. (Undergraduate) Compare the project requirements of VNAV and 16-833: which form of assessment better tests whether a student has truly learned SLAM?

Extended Challenge (Undergraduate)

Download the first exercise from the UZH course website (usually on feature extraction or camera models), try to run it on your own computer, and record the environment setup problems you encounter and how you solve them—this record will prove useful when you build the SLAM development environment in Chapter 28.

Further Reading

1. MIT OpenCourseWare, 16.485 Visual Navigation for Autonomous Vehicles (VNAV), Fall 2020. (Complete lecture notes, assignments, and UAV lab guides, freely available.)
2. CMU 16-833 Robot Localization and Mapping course website (cs.cmu.edu/~kaess/teaching/16833). (Syllabi and reading lists from past years.)
3. Stanford AA274A / CS237A Principles of Robot Autonomy I course website and public lecture notes. (A full-stack course on autonomous robots.)
4. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011. (The textbook assigned in the ETH course; also available as an edX MOOC.)
5. UZH Robotics and Perception Group, Vision Algorithms for Mobile Robotics course page (rpg.ifi.uzh.ch/teaching.html). (All notes and videos public.)
6. Cyrill Stachniss, Robot Mapping / SLAM lecture series, YouTube (channel: Cyrill Stachniss). (The audiovisual SLAM resource best suited to self-study.)

7. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (The theoretical synthesis of the Freiburg line and the principal reference for Part III of this book.)
8. D. Scaramuzza and F. Fraundorfer, "Visual Odometry: Tutorial," IEEE Robotics & Automation Magazine, 2011. (The classic introductory tutorial on visual odometry, in two parts.)

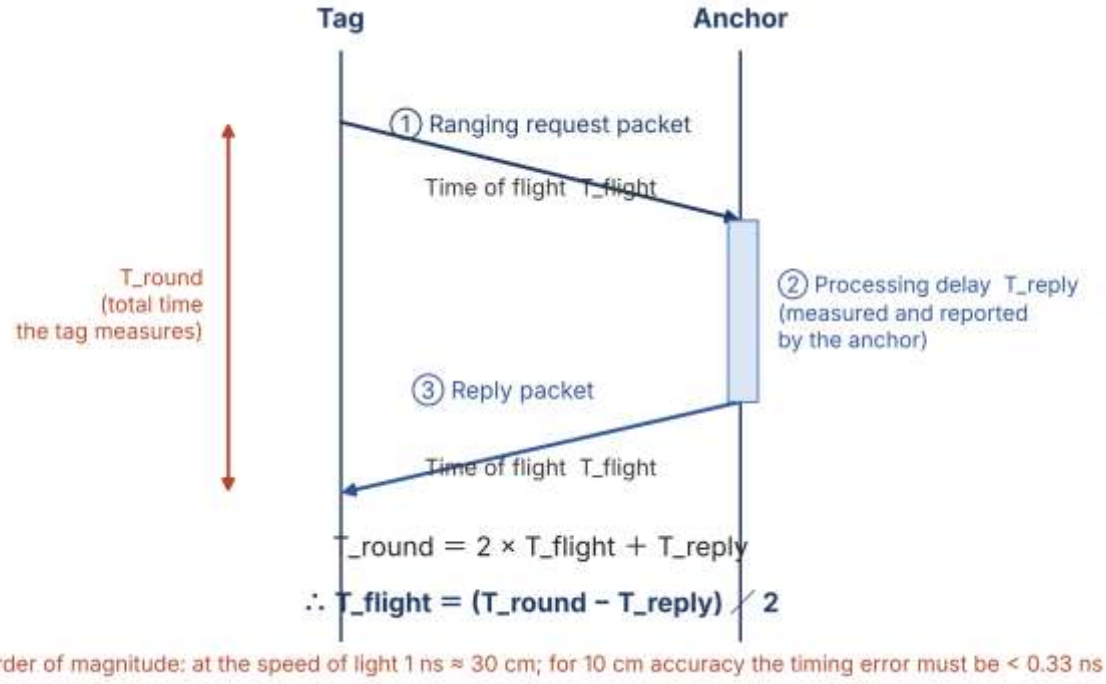
From a Tour of Courses to a System Architecture

To close this chapter, we turn the view back to the journey of this book itself.

Section 2.7 distilled the architecture common to the five courses: sensors → mathematics → front end → back end → loop closure → capstone. This architecture recurs not because the courses copy one another but because it reflects the internal logic of the SLAM problem—you must have data (sensors) before you can discuss how to process data (mathematics); you must be able to estimate the relation between adjacent instants (front end) before you can discuss global consistency (back end); loop closure is the fullest expression of the back end's power, and the capstone is the integrated test of every capability.

The nine parts of this book are that logic unfolded, with each link enlarged to its own full-length treatment:

Figure 2-4 The Common Architecture and the Nine Parts of This Book



[Material added by this book]

Part I Concepts and orientation (international courses usually enter the technology directly)

Part VII Platforms at the limit (micro-robots, UAVs, multi-robot collaboration)

Part IX AI semantic navigation (aligned with the pedagogical trend toward embodied intelligence)

In the next chapter we arrive at the first station of this architecture—not the sensors, but something above them: the complete architecture of a robot navigation system. Chapter 3 answers where SLAM sits within the whole navigation system and introduces the reactive, deliberative, and hybrid architectures, cost maps and path planning, and the differing navigation requirements of wheeled, tracked, quadruped, and aerial platforms. With that system blueprint in hand, the six sensors of Part II will each know where they belong.

Summary

1. MIT VNAV is organized around geometry and optimization on manifolds, runs a UAV project through the whole term, and has its complete materials freely available on OCW.
2. CMU 16-833 frames SLAM as real-time estimation under sensing uncertainty, concentrates on filtering, factor graphs, and the optimization back end, and is the birthplace of iSAM / GTSAM.
3. Stanford's robot autonomy course is distinguished by system integration: SLAM is taught within the complete perception–localization–planning–control stack, with ROS used throughout.
4. ETH's Autonomous Mobile Robots course (with its edX MOOC) offers the most accessible general entry point, while Scaramuzza's UZH course goes deep into vision and VIO through a building-block visual odometry project.
5. The Freiburg / Bonn line presents probabilistic mapping most systematically, and Professor Stachniss's YouTube courses are widely regarded as the free audiovisual resource best suited to self-study.
6. The five courses share one knowledge skeleton: sensors → mathematics → front end → back end → loop closure → capstone. The nine parts of this book follow that architecture and localize it to the Taiwanese campus setting.

In the next chapter we turn the lens from courses back to the robot itself and survey the complete architecture of a robot navigation system—the Sense–Think–Act loop, the taxonomy of maps and localization, and the place of SLAM within the navigation

system—preparing the ground for the tour of sensors in Part II.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 3

An Overview of Robot Navigation Systems

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 1 explained what problem SLAM solves, and Chapter 2 looked at how the world teaches the subject. But there is an easily overlooked premise: SLAM never exists in isolation—it is one link in a navigation system. This chapter draws the complete system blueprint so that you know what lies upstream of SLAM and where its output goes downstream.

This chapter closes Part I and is the key to understanding the architecture of the whole book: where in this system the sensors of Part II are mounted, which module the mathematics of Part III supports, and which components the implementation of Part VIII wires together—the answers are all in the architecture diagrams of this chapter.

Figure 3-0 How Chapter 3 Connects to the Rest of the Book

[Building on the previous chapters] The five map representations and six sensors of Chapter 1 are placed here within a complete system architecture.

[This chapter is responsible for] establishing the overall blueprint of a navigation system: how perception, decision, and action are connected, and where SLAM sits.

[Where it leads]

3.1 Sense–Think–Act → the layered thinking that runs through the book (recurring in Chapters 15, 18, 26, and 32)

3.2 Kinds of maps → occupancy grids in Chapter 13, point clouds in Chapter 17, feature maps in Chapter 20, semantic maps in Chapter 32

3.3 Kinds of localization → relocalization and the kidnapped robot problem in Chapter 11, multi-map operation in Chapter 23, AMCL in Section 29.5

3.4 Navigation system architecture → the full Nav2 implementation in Section 29.6, project integration in Chapter 31

3.5 The role of ROS → all of Part VIII (Chapters 27–30)

3.6 Platform differences → robotic vacuums in Chapter 16, quadrupeds in Chapter 18, UAVs in Chapter 26

[Prerequisites] The basic concepts of Chapter 1. This chapter still involves no

mathematics.

Learning Objectives

The previous two chapters introduced the central problem of SLAM and the teaching paths of leading universities. This chapter raises the view to survey the complete system architecture of an autonomous navigation robot, so that you hold a map of the whole system before going deep into any individual technique. After completing this chapter, you should be able to:

1. Explain the basic operating principle of a robotic system in terms of the Sense–Think–Act loop and give concrete examples of each stage.
2. Distinguish the data structures, strengths, weaknesses, and appropriate uses of the four principal map representations (occupancy grid map, feature map, point cloud, semantic map).
3. Distinguish the four modes of localization—relative localization, absolute localization, map-based localization, and relocalization—and describe the error characteristics of each.
4. Draw the architecture of a navigation system composed of the four modules SLAM, path planning, obstacle avoidance, and control, and describe the data flow among them.
5. Explain the core concepts of ROS / ROS 2 (nodes, topics, services) and the role they play in SLAM teaching and research.
6. Compare the differing navigation requirements of five platforms: the robotic vacuum, the service robot, the quadruped, the UAV, and the self-driving car.

Core Concepts

Term (Chinese)	Term (English)	Explanation
感知—決策—行動	Sense–Think–Act	The basic loop of a robotic system: sense the environment, make a decision, execute an action.
特徵地圖	Feature Map	A map that records only the positions of salient landmarks in the environment (corners, columns, markers).
相對定位	Relative Localization	Propagating pose by accumulating motion measurements (dead reckoning); error accumulates over time.
絕對定位	Absolute Localization	Measuring position directly with respect to a global reference frame (GPS, UWB); does not drift.
地圖定位	Map-based Localization	Estimating pose by matching live sensor data against an existing map, as in AMCL.
重定位	Relocalization	The ability to recover one's position within a map after becoming lost or being moved.
綁架機器人問題	Kidnapped Robot Problem	The extreme test of moving a robot abruptly to an unknown position, used to examine relocalization capability.
路徑規劃	Path Planning	Finding a feasible and optimal path from start to goal within a map.
避障	Obstacle Avoidance	Detecting and evading dynamic or temporary obstacles absent from the map.
代價地圖	Costmap	A navigation map in which obstacles are inflated by a safety margin, maintained in global and local layers.
機器人作業系統	ROS / ROS 2	A robotics software development

Term (Chinese)	Term (English)	Explanation
		framework providing inter-module communication, tools, and an ecosystem.
節點與話題	Node / Topic	The basic units of ROS: a node is a functional module, and a topic is a data channel between nodes.

3.1 Sense–Think–Act: Perception, Decision, Action

The operation of every autonomous robot—whether a robotic vacuum, a quadruped, or a self-driving car—reduces to one continually repeating loop: Sense, Think, Act. This loop executes from a few to several hundred times per second and constitutes the heartbeat by which a robot is alive.

- Sense: gather information through sensors. This includes sensing the external world (the LiDAR detects a wall ahead, the camera sees a pedestrian) and sensing the robot's own state (how far the wheels have turned, how far the body is tilted). The six major sensors of Part II all belong to this layer.
- Think: convert sensed information into action commands. This layer subdivides further: state estimation (where am I?—SLAM and localization), environmental understanding (what is around me?), task planning (where must I go?), path planning (how do I get there?), and motion planning (what is the next move?).
- Act: turn commands into actual output from motors, servos, and propellers, changing the state of the robot and of the world—which in turn changes what is sensed on the next cycle, so the loop continues.

Early robotics favored the strictly sequential Sense–Model–Plan–Act architecture, in which a complete world model had to be built before every action, making the system sluggish; in the 1980s Rodney Brooks proposed the behavior-based architecture, which skips modeling and lets perception drive behavior directly, giving fast reactions but making complex tasks difficult. Modern robots generally adopt a hybrid architecture: a fast reactive loop at the lower level handles obstacle avoidance and balance (milliseconds), while a deliberative planning layer above handles mapping and navigation (seconds)—and SLAM is a core member of that upper deliberative layer.

An everyday analogy: swerving on a bicycle to avoid a dog that darts out relies on reflex that bypasses conscious thought (reactive); planning how to ride from home to school relies on the map and the route in your head (deliberative). What is true of people is true of robots.

3.2 Kinds of Maps: Occupancy Grid, Feature Map, Point Cloud, and Semantic Map

Chapter 1 introduced the appearance of maps from the standpoint of output; this section compares them in depth from the standpoint of data structure, because the choice of map representation directly determines memory usage, algorithmic complexity, and the capabilities an application can have.

1. Occupancy Grid Map

Space is divided into a regular grid, and each cell stores an occupancy probability between 0 and 1. The advantages are a simple structure, fast queries, and natural support for path planning and obstacle avoidance; the drawbacks are that memory grows linearly with area, the resolution is fixed (with 5 cm cells a 3 cm object is invisible), and no semantics are carried. The 2D occupancy grid is the mainstream choice for indoor wheeled robots; the 3D version commonly uses an octree (such as OctoMap) for compressed storage, subdividing cells only where something exists.

2. Feature Map

This takes the opposite approach: instead of recording whether every cell is occupied, it records only the positions and descriptors of a small number of salient landmarks in the environment—wall corners, columns, AprilTags, visual feature points. The advantages are extreme compactness (a classroom may require only a few dozen

landmarks) and a natural fit with estimation theory such as EKF-SLAM; the drawbacks are that nothing is known about the space between landmarks, so the map cannot be used directly for obstacle avoidance, and that the environment must contain stable, recognizable features. The map points built by the ORB-SLAM family are essentially a visual feature map.

3. Point Cloud Map

This stores the raw three-dimensional points measured by the sensor directly, without gridding and without extracting features. The advantages are the highest fidelity and the ability to represent volumetric structure of arbitrary complexity; the drawbacks are an enormous data volume (often several gigabytes), the absence of a distinction between empty and unknown (does the absence of points mean nothing is there, or that nothing was scanned?), and the inconvenience of planning paths directly on it. In practice a point cloud often serves as raw material that is converted into a grid map or a mesh model as needed.

4. Semantic Map

Meaning is annotated on top of geometry: this region is the kitchen, that object is a chair, this line is a traversable corridor. A semantic map allows a robot to carry out tasks stated in human language and is the basis of service robots and of the AI semantic navigation of Chapter 32. It usually does not exist on its own but as a label layer overlaid on one of the three geometric maps above.

Map Type	Data Structure	Memory	Strengths	Limitations
Occupancy	Probabilistic	Medium	Convenient for	Fixed resolution;

Map Type	Data Structure	Memory	Strengths	Limitations
grid map	grid	(grows with area)	planning and obstacle avoidance; mature algorithms	no semantics
Feature map	List of landmarks	Very small	Compact; well suited to estimation theory	Cannot support obstacle avoidance; depends on features
Point cloud map	Set of three-dimensional points	Large	Highest fidelity; complete volumetric structure	Hard to plan on directly; no notion of free space
Semantic map	Geometry + labels	Depends on the underlying layer	Supports language-level tasks and human-robot interaction	Depends on the correctness of AI recognition

Table 3-1 Comparison of the Four Map Representations

3.3 Kinds of Localization: Relative, Absolute, Map-Based, and Relocalization

1. Relative Localization (Dead Reckoning)

Starting from a known origin, the current pose is propagated by accumulating the displacement of each step from wheel odometry and the IMU; this is also called dead reckoning. It is the method of sailors in the age of exploration: record heading and speed and accumulate to estimate the ship's position. Its advantages are self-sufficiency and rapid update; its fatal flaw is the drift emphasized repeatedly in Chapter 1—error only grows.

2. Absolute Localization

Position is measured directly with respect to a global coordinate frame: GPS gives latitude and longitude, UWB anchors give indoor coordinates, and a magnetic compass gives absolute heading. The advantage is bounded error that does not deteriorate over time; the drawbacks are dependence on external infrastructure (satellites, anchors), limited accuracy, and a usually low update rate.

3. Map-Based Localization

Given an existing map, the pose is inferred by matching live sensor data against the map. The classic example in the ROS ecosystem is AMCL (Adaptive Monte Carlo Localization): a large number of particles representing guessed positions are scattered over the map, and the particles whose expected laser scan best matches the actual scan are favored, gradually converging to the true pose (the principle is detailed in Chapter 11 and the implementation in Chapter 29). The reason a robotic vacuum need not remap on its second cleaning run is map-based localization.

4. Relocalization

If the robot is picked up and carried to another room (the famous kidnapped robot problem), or if visual SLAM loses track because of rapid rotation, the system must recover its position within the entire map without knowing where it is—this is relocalization. It is considerably harder than ordinary localization because there is no previous position to refer to and a global search is required. The ORB-SLAM family implements relocalization by retrieving similar images quickly with a bag-of-words model (Chapter 22); this capability shares its technical core with loop closure detection.

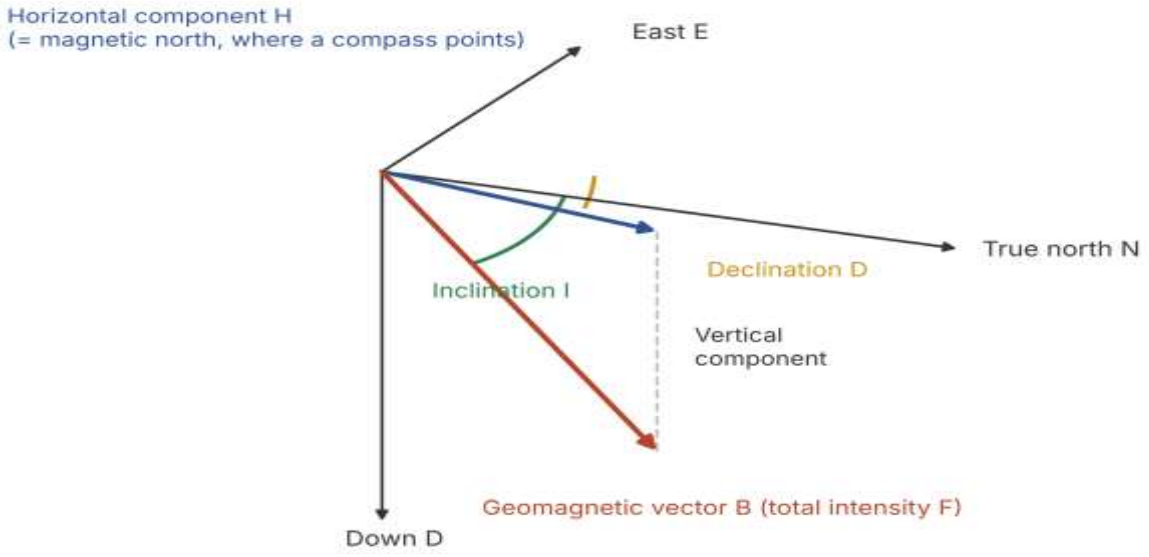
The four kinds of localization are not mutually exclusive but complementary layers: a practical system uses relative localization for high-rate short-term estimates, map-

based or absolute localization for drift-free long-term correction, and stands ready to invoke relocalization whenever it becomes lost—this is the sensor fusion philosophy expressed at the level of localization.

3.4 Navigation System Architecture: SLAM, Path Planning, Obstacle Avoidance, and Control

With a map and localization in hand, a robot still cannot walk to a destination. A complete navigation system consists of four modules connected by well-defined data interfaces:

Figure 3-1 Architecture and Data Flow of an Autonomous Navigation System



The left column of the figure shows the data flow through five modules connected in sequence, while the right column adds two pieces of information not easily seen at a glance in prose: the typical update rate of each module, and the actual component it corresponds to in ROS / Nav2 together with the relevant chapter of this book.

The differences in update rate deserve particular attention. Sensors output tens to hundreds of measurements per second, the controller runs fifty to one hundred times per second, and the global planner may run only once every several seconds—this is not a matter of importance but of the nature of each task: avoiding a wall must be immediate, whereas replanning an entire route can take its time. When tuning the system in Chapter 29, if the robot responds sluggishly or the path oscillates, the first thing to check is which rung of this frequency ladder has fallen behind.

The red loop arrow on the far left is the soul of the diagram: once the robot moves, the environment relative to it has changed, so it must sense again. Navigation is never a one-way pipeline but a closed loop that continually returns to itself—this is the full form of the Sense–Think–Act loop of Section 3.1.

- The SLAM / localization module is the eyes and the inner map of the whole system: without a correct pose, every module downstream works from a false premise—which is why this book devotes an entire volume to making this one module clear.
- The global planner operates on the static map and answers which general direction to take; graph search algorithms such as A* and Dijkstra are commonly used.
- The local planner operates at high rate on a local cost map covering a few meters around the robot and handles what the map does not contain: people walking by, a chair placed temporarily. The cost map inflates obstacles outward by a safety margin so that the robot does not scrape past them.

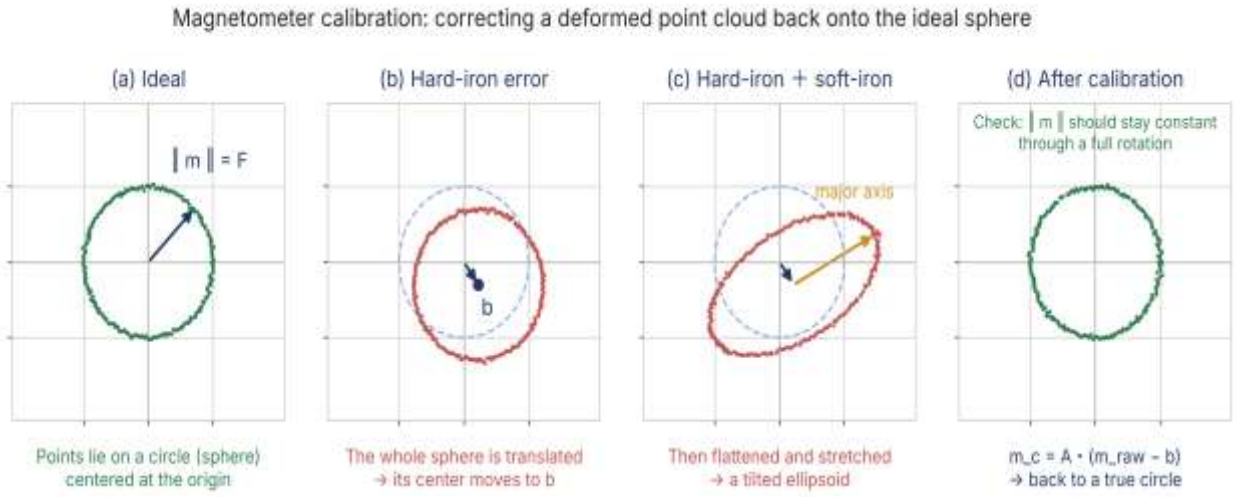
- The controller translates the geometric path into language the motors understand. For a differential-drive robot the command is usually a pair (v, ω): how fast to move forward and how fast to turn.

The concrete realization of this architecture in the ROS ecosystem is the well-known Navigation Stack (ROS 1) and Nav2 (ROS 2). Chapter 29 will have you run this entire pipeline on a physical robot.

SLAM's Upstream and Downstream: A Chart of Responsibilities

The four modules of the navigation system have been described. Here we clarify something students often confuse: where exactly does SLAM's responsibility end? What does it not do?

Figure 3-2 The Boundaries of SLAM's Responsibility, Upstream and Downstream



This figure divides SLAM's responsibility into four blocks, and from left to right they form a data chain: what upstream feeds it, what it is responsible for, what it is explicitly not responsible for, and who downstream consumes its output.

The contrast between the middle two blocks is the point of this section. Many beginners blame SLAM when the robot takes a wrong route, bumps into a chair, or

stops inaccurately, but these three belong respectively to the planner, local obstacle avoidance, and the controller. SLAM answers only two questions: where am I, and what does the environment look like.

This chart has a practical value: it lets you localize a problem quickly during debugging. Section 29.7 provides a systematic checking order whose logic follows exactly this upstream-to-downstream chain—if the robot misbehaves, first confirm that upstream (sensors, TF, timing) is sound, then look at SLAM itself, and only then check the downstream planning and control. Many students spend hours tuning planner parameters when the real problem is a mis-entered extrinsic calibration—precisely the cost of not understanding where responsibility lies.

3.5 The Role of ROS / ROS 2 in Teaching SLAM

If SLAM is the soul of navigation, ROS (Robot Operating System) is the skeleton that carries it. Despite the words "operating system" in the name, it is a robotics software framework that runs on top of Linux and solves one key problem: allowing modules written by different authors—sensor drivers, SLAM, planning, control—to be assembled like building blocks.

Three Core Concepts

- Node: an independent functional program. The LiDAR driver is a node, Gmapping is a node, the path planner is a node.
- Topic: the data channel between nodes, following the publish/subscribe model. The LiDAR node publishes the /scan topic and the SLAM node subscribes to it—neither needs to know the other, so long as they agree on the data format.

- Service and Action: the request-response pattern (querying a map) and the long-running task pattern (navigate to a goal, with progress reporting and cancellation).

Why Teaching SLAM Is Inseparable from ROS

First, a standardized interface makes algorithms hot-swappable: with the same robot and the same laser data, swapping one node lets you compare the map quality of Gmapping and Cartographer—this is the basis of the experiments designed in Part IV. Second, the rosbag recording tool captures sensor data for repeated playback, so students without a robot can still practice SLAM fully using public datasets. Third, the RViz visualization tool displays the map, pose, particle cloud, and planned path in real time, turning abstract algorithms into visible animation. Fourth, the mainstream open-source SLAM systems worldwide (Gmapping, Cartographer, RTAB-Map, and community wrappers of ORB-SLAM) all provide ROS interfaces, so standing on ROS means standing on the whole ecosystem.

From ROS 1 to ROS 2

ROS 1 was born in 2007 and reached end of official support in May 2025 with its final release, Noetic; new projects should use ROS 2 without exception. ROS 2 redesigned the underlying communication (adopting DDS) and supports real-time operation, multi-robot systems, and embedded platforms. A new version is released every year on May 23 (World Turtle Day): even years are long-term support releases (LTS, supported for five years) and odd years are regular releases. As of this writing, commonly used versions include Humble (2022, LTS), Jazzy Jalisco (2024, LTS), and the latest Lyrical Luth (2026, LTS, paired with Ubuntu 26.04). The implementation chapters of this book are based on LTS versions, and Chapter 28 explains environment

installation in detail. The good news pedagogically is that the concepts of nodes and topics introduced in this chapter carry over directly from ROS 1 to ROS 2—learn one and you know both.

Nav2's Standing Today: From Research Tool to Industry Standard

This section has described the role of ROS in teaching; here is an additional observation relevant to career planning: the navigation stack of the ROS ecosystem has evolved from a research tool into an industry standard.

As the successor to the ROS Navigation Stack, Nav2 builds on fifteen years of inheritance. It takes techniques from the autonomous vehicle field, scales them down, and redesigns and optimizes them for mobile and ground robots; it provides a complete set of functions—perception, planning, control, localization, visualization, and behavior—that compute an environment model from raw or preprocessed sensor and semantic data, plan paths dynamically, generate feasible motor commands, avoid obstacles, and organize higher-level robot behaviors.

Its degree of industrial adoption is worth noting: Nav2 is a production-grade, high-quality navigation framework trusted by more than a hundred companies worldwide; it enables mobile robots to complete user-defined application tasks in complex environments and applies to nearly every class of robot kinematics. Its capabilities are not limited to moving from point A to point B—intermediate poses can be specified, and other classes of task such as object following and complete coverage navigation can be expressed.

The last point is particularly meaningful for this book: complete coverage navigation is exactly the core task of the robotic vacuum in Chapter 16, and object

following echoes the visual navigation of Chapter 24. This shows that Nav2 is not merely a path planner but a composable task framework—Section 29.6 explains how it achieves this through the concept of behavior trees.

Study Note: The Market Value of ROS Skills

For students in Taiwan, practical competence in ROS 2 and Nav2 has several concrete values:

• A common industry language: whether the work is AGVs, service robots, agricultural robots, or inspection quadrupeds, most teams use ROS 2.

• A portfolio that accumulates: the capstone results of Chapter 31 can go straight onto a résumé and can be reproduced and verified by others.

• A bridge to international work: most of the assignments in the courses introduced in Chapter 2 also use ROS as their platform.

Part VIII of this book (Chapters 27–30) is therefore not merely a teaching unit but professional training—

the environment setup of Chapter 28, the complete navigation system of Chapter 29, and the capstone assessment of Chapter 31

together constitute a demonstrable portfolio of implementation work.

3.6 From the Robotic Vacuum to the Self-Driving Car: Navigation Requirements Across Platforms

The same navigation architecture takes on strikingly different forms on different platforms. Understanding how requirements shape a system is where the ability to design systems begins.

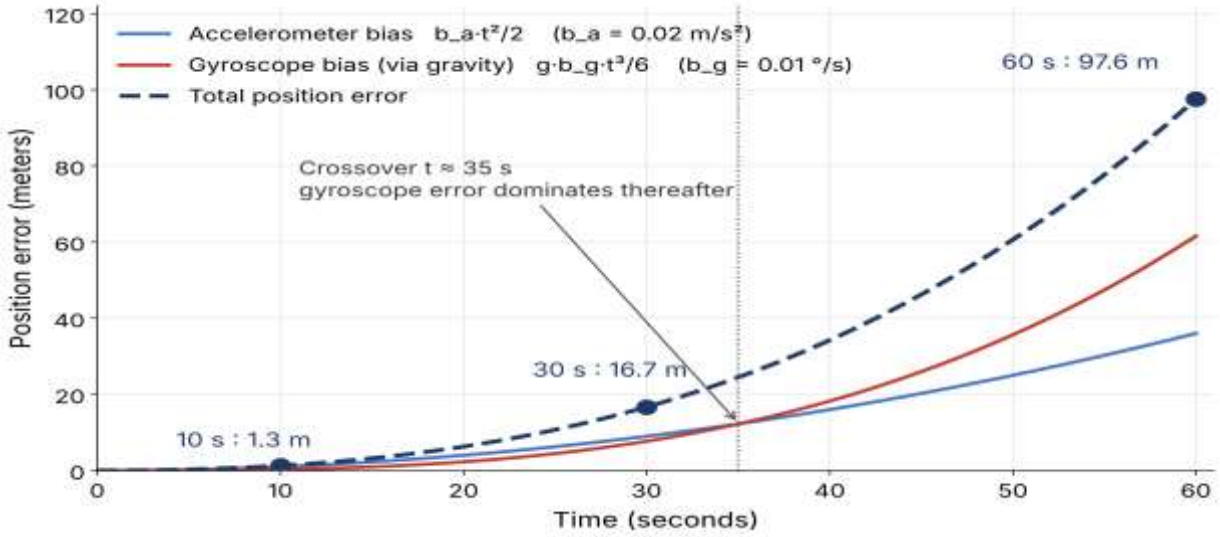
- Robotic vacuum: a 2D plane, low speed, a semi-static environment. The objective is not to go from A to B but to cover every square inch of floor (coverage path planning), and sensor cost is squeezed to the limit—one 2D LiDAR plus bump and cliff sensors must do everything (Chapter 16).
- Indoor service robot (food delivery, guiding): coexistence with people is the greatest challenge. It requires semantic understanding (elevator, reception desk), socially aware obstacle avoidance (not forcing a path through a crowd), and the ability to keep the map updated over long-term operation.
- Quadruped: able to climb stairs and step over obstacles, so navigation is upgraded from 2D to 2.5D / 3D—the map must contain terrain height and steppability, localization must stay stable under violent shaking, and tightly coupled LiDAR-inertial estimation (Chapter 18) becomes standard.
- UAV: full six degrees of freedom, high speed, and severe limits on payload and power. Lightweight visual-inertial solutions are preferred (Chapter 26), and latency tolerance is zero—half a second late with the pose and the aircraft has already hit the wall.
- Self-driving car: high speed, grave consequences, and large scale. It uses a high-definition map plus multiply redundant sensing (LiDAR, camera, radar, and RTK-GNSS are all indispensable), demands centimeter-level localization accuracy, and must pass automotive-grade safety validation.

Platform	Spatial Dimension	Speed	Core Difficulty	Typical Sensor Suite
Robotic vacuum	2D	Slow	Complete coverage; extreme cost limits	2D LiDAR + bump sensors + IMU
Service robot	2D + semantics	Slow	Coexistence with people; long-term map maintenance	2D LiDAR + camera + ultrasound
Quadruped	2.5D / 3D	Medium	Terrain understanding; vibration	3D LiDAR + IMU + depth camera
UAV	3D (6-DoF)	Fast	Weight and power; zero latency tolerance	Camera + IMU (VIO)
Self-driving car	2D over a large area	Very fast	Safety redundancy; centimeter accuracy	3D LiDAR + camera + radar + RTK

Table 3-2 Comparison of Navigation Requirements Across Five Platforms

The chapter sequence of this book advances along exactly this spectrum: Part IV starts from the most approachable robotic vacuum and 2D SLAM, Part V moves to quadrupeds and 3D LiDAR, and Parts VI and VII go on to visual navigation and UAVs, letting you climb the ladder of difficulty one rung at a time.

Figure 3-3 The Spectrum of Navigation Requirements Across Five Platforms



Once Table 3-2 is drawn as parallel columns, a clear spectrum emerges: from left to right, the spatial dimension rises from 2D to 3D, speed goes from slow to very fast, and the sensor suite expands from a single 2D LiDAR to LiDAR plus camera plus radar plus RTK.

Note the chapter labels along the bottom row—they are almost exactly the order of Parts IV through VII of this book. This is no coincidence: the book is deliberately arranged along this spectrum of difficulty, so that you begin with the most approachable, least expensive, and most easily verified robotic vacuum and move step by step to quadrupeds, UAVs, and multi-robot collaboration. Each step to the right raises the degrees of freedom, the speed, and the uncertainty that must be handled by one level.

Another observation worth savoring concerns the self-driving car: its spatial dimension returns to 2D (cars do not fly), yet large-scale 2D brings an entirely different set of difficulties—the map is too large to load in full, the speed leaves the algorithm only tens of milliseconds, and the consequences are grave enough to require multiple redundancy. This illustrates one thing: low dimensionality does not mean simplicity,

and what truly determines difficulty is the combination of requirements.

Formulations and Procedures

1. Error Accumulation in Dead Reckoning

Take a 2D differential-drive robot. Let the pose at time t be (x(t), y(t), θ(t)), and let linear velocity v and angular velocity ω be measured over one sampling period Δt. The dead-reckoning update equations are:

$$x(t+1) = x(t) + v \cdot \Delta t \cdot \cos\theta(t)$$

$$y(t+1) = y(t) + v \cdot \Delta t \cdot \sin\theta(t)$$

$$\theta(t+1) = \theta(t) + \omega \cdot \Delta t$$

Note that v and ω at every step carry measurement error, and that the error is transmitted to the position through the cosine and sine—particularly the angular error: a heading off by one degree puts the position off by about 17 centimeters after ten meters of straight travel, and every subsequent step builds on that skewed direction. This simple pair of equations is the mathematical explanation of why wheel odometry alone cannot support navigation, and a preview of the coordinate kinematics of Chapter 10 and the error propagation theory of Chapter 11.

2. Decomposing the Navigation Problem

The system architecture introduced in this chapter can be organized as the three questions of navigation and their corresponding modules:

Figure 3-4 The Three Questions of Navigation

Question 1: Where am I?

→ Localization / SLAM (Parts I through VI of this book)

Question 2: Where am I going?

→ Task and goal decision making (the capstones of Chapters 31 and 32)

Question 3: How do I get there?

→ Path planning + obstacle avoidance + control (the Nav2 implementation of Chapter 29)

These three questions come from a classic formulation in mobile robotics (Leonard and Durrant-Whyte): the task of this entire textbook is to answer the first question thoroughly and to equip you to connect that answer to the second and third.

3.7 The Contemporary Evolution of Navigation: From Modular Design to Foundation Models

What this chapter describes is the classical modular navigation architecture—perception, localization, planning, and control each with its own duty, connected by well-defined interfaces. This architecture is stable, interpretable, and easy to debug; it is the backbone of the overwhelming majority of deployed systems today and the object of the implementation work in Part VIII.

In the past two years, however, an entirely different line has appeared, and it is worth knowing that it exists and where it stands.

The Second Line: Navigation Foundation Models

In the traditional architecture every module is trained or designed separately; the new line attempts to accomplish navigation end to end with a single large model. Such models are called navigation foundation models or vision-language-action models.

Their range of capability is remarkable. One representative work, for example, provides a unified framework for several kinds of task—including image question answering, video question answering, and navigation; it is validated on benchmarks such as object search, tracking, and autonomous driving, and tested on a variety of real robot platforms including humanoids, quadrupeds, UAVs, and wheeled robots, indicating a possible path toward general-purpose navigation.

This property of one model across many platforms makes an interesting contrast with the discussion in Section 3.6: that section shows how greatly navigation requirements differ across platforms (wheeled robots need only 2D, quadrupeds need terrain assessment, UAVs need six degrees of freedom), while the foundation model line attempts to cover those differences with a single model. Section 24.5 and Chapter 32 return to this theme.

World Models: Letting the Robot Imagine the Future

Another emerging concept is the world model. Traditional navigation plans on the basis of the current map; a world model lets the robot predict what it will see if it moves in a given direction and choose its action accordingly. Recent research has applied vision-language world models to hierarchical planning and dual-horizon navigation prediction, and other work explores how a world model can evolve on its own in a continuous environment.

This line of thought will not be unfamiliar to readers of this book—the Bayes filter of Chapter 11 is itself a predict-update cycle, except that what is predicted is the pose rather than the image. A world model may be understood as extending the object of prediction from state to perception, a natural but powerful extension.

The Pace of Growth and a Pragmatic Judgment

The growth rate of this field deserves attention. One compilation notes that more than ninety robot foundation models were released between 2025 and 2026, spanning vision-language-action models, world models, humanoid foundation models, and policies for locomotion, navigation, and manipulation; and that submissions on vision-language-action models to top conferences grew roughly eighteenfold from 2025 to 2026.

How should a student respond to change at this pace? This book offers three recommendations:

1. Master the modular architecture first, because it is the mainstream of actual deployment and the basis for understanding new methods—a foundation model must still output a pose or waypoints, must still contend with sensor limitations, and must still avoid obstacles in the real world.
2. Understand which old problem the new method solves: the value of foundation models lies in generalization and a language interface, not in replacing geometric precision. Section 34.7 discusses their four major risks in detail, including hallucination, systematic misjudgment, latency, and safety.
3. Think in layers rather than replacement: the most pragmatic integration today has the foundation model handle high-level decisions (on the order of seconds and above) while the geometric system handles real-time control (milliseconds).

This is exactly the layered architecture proposed in Section 32.6 and a modern version of the Sense–Think–Act idea of Section 3.1.

Concept Summary: The Division of Labor Between the Two Lines

[Modular architecture] Parts I through VIII of this book

Strengths: interpretable, debuggable, well-defined computational requirements, no training data needed

Limitations: each module is optimized separately and global optimality is not guaranteed; weak semantic understanding

[Foundation model line] Chapter 32 of this book

Strengths: strong generalization, a language interface, portability across platforms

Limitations: high computational and memory demands, susceptibility to hallucination, difficulty of debugging and accountability

[Pragmatic integration] Let the foundation model understand the task at the upper layer and the modular architecture guarantee safety at the lower.

This principle of division recurs in Chapters 24, 27, and 32 of this book.

Case Studies

Case 3-1 A Day in the Life of a Food-Delivery Robot: The Whole Architecture Mobilized

Watch a food-delivery robot in a restaurant complete one errand and every concept of this chapter appears at once. Power-on: it loads the existing map and confirms with AMCL map-based localization that it is at the charging dock (map-based localization plus relocalization capability). Order received: the task system designates table 12 (the second question of navigation). Departure: the global planner computes a route on the occupancy grid map (A*), wheel odometry and the IMU supply high-rate poses along the way (relative localization), and the laser continually matches against the map to correct them (map-based localization). Midway a guest stands up and blocks the aisle: the local cost map updates instantly, the local planner traces an arc around the obstruction (obstacle avoidance), and the controller issues a new (v, ω). After delivery it returns to charge. Throughout the errand, the map built by SLAM is the foundation of everything—let the map be off by one meter and the robot will deliver the meal into a wall.

Case 3-2 The Kidnapping Test: Carrying a Robotic Vacuum into Another Room

Many students can run this classic experiment at home: let the vacuum work halfway through the living room, then pick it up and set it down in the bedroom. Early low-cost models go into complete disarray—believing themselves still in the living room, they bump repeatedly into bedroom walls. A model with relocalization capability pauses for several seconds, rotates once to rescan, matches the laser profile against the whole-home map in memory, and then teleports on the app's map to the correct bedroom position and resumes work. What happens in those few seconds is textbook solution of the kidnapped robot problem: discard the prior pose hypothesis, search globally, verify multiple hypotheses, converge. One act of picking the robot up and

putting it down is a free lecture in localization theory.

Case 3-3 From Willow Garage to Nav2: The Power of the ROS Ecosystem

Around 2010 the American company Willow Garage open-sourced the PR2 robot together with ROS and assembled the first-generation Navigation Stack—a complete combination of AMCL, cost maps, and global and local planners. Over the decade that followed, thousands of robots worldwide (including many homebuilt platforms in university laboratories in Taiwan) did not have to rewrite a navigation system but stacked their own applications on the same open-source building blocks. In the ROS 2 era, Nav2 went further by introducing a behavior tree task architecture and plug-in planners. This history illustrates what open-source ecosystems mean for robotics education: a student's energy can be concentrated on understanding principles and inventing applications rather than reinventing every wheel—which is why Part VIII of this book uses ROS 2 directly as its implementation backbone.

Case 3-4 Semantics Enters the Navigation Stack: When a Segmentation Model Meets Nav2

The modular architecture described in this chapter has been undergoing a quiet but important change in recent years: semantic information has begun to enter the core of the navigation stack.

A traditional cost map (Section 29.6) distinguishes only traversable from obstructed. Real-world traversability is far more nuanced—sidewalk and roadway are both flat ground and geometrically indistinguishable, yet semantically entirely different; grass can be driven over while a puddle is better avoided; glass does not exist for a LiDAR but is plainly visible to a camera.

Practical demonstrations have recently combined segmentation foundation models with Nav2 to perform semantic navigation outdoors by segmenting sidewalk from roadway, running on commodity-grade computing platforms. This means semantic navigation has moved from papers to deployable systems.

In the architecture of this book, this change occurs at the junction of two layers: the semantic layer (the AI vision models of Chapter 32) produces the judgment "this is sidewalk, that is roadway," and the geometric layer (the cost map of Chapter 29) converts that judgment into a traversal cost. The interface between them is simple—the semantic result becomes one layer of the cost map. This is precisely the advantage of a modular architecture: new capability can be added as a layer without rewriting the system.

There are two lessons here. First, the layered cost map design discussed in Section 29.6 is not merely an implementation detail but an architectural decision that accommodates future extension. Second, if you undertake a capstone project in semantic navigation (Chapters 24 and 31), the most pragmatic entry point is not to rewrite the navigation system but to add one semantic layer—an approach that is inexpensive, low in risk, and immediately visible in its effect.

Hands-On Activity

Activity 3-1 Dissecting a Navigation System and Designing One on Paper

Purpose: without writing code, carry out a complete dissection and specification of a navigation system with the eye of a systems designer, and build a global view.

Procedure

1. Form groups of three to four, and have each group draw one platform: robotic vacuum, food-delivery robot, campus inspection quadruped, indoor inspection UAV, or autonomous shuttle bus.
2. Requirements analysis: list the operating environment of that platform (indoor or outdoor? people present? terrain?), the speed requirement, and the severity of the consequences of failure.
3. Sensor selection: choose a combination from the six sensor classes of Table 1-1 in Chapter 1, and explain the purpose of each and the reason for rejecting the alternatives (including cost considerations).
4. Map and localization selection: choose a map representation from Table 3-1 and a localization strategy from Section 3.3, and justify both.
5. Draw the system architecture: following the model of Figure 3-1, draw the complete data flow for your platform and label the update rate of each module (which need milliseconds? which are fine at seconds?).
6. Each group presents for five minutes while the other groups play the customer and raise at least one awkward question (for example: what about glass partitions? what if a child pushes the robot away?).

Discussion Questions

1. In your platform, the failure of which module would have the most severe consequences? How should the system provide a backup for it?
2. If the budget were cut in half, which sensor would you give up first? What would the system's capability degrade to?

3. (Undergraduate) In the architecture you designed, which modules can use off-the-shelf components from ROS 2 / Nav2, and which must be developed yourselves?

Extended Challenge (Undergraduate)

Install ROS 2 and the TurtleBot3 simulator (installation steps are previewed in Chapter 28, and ample tutorials exist online), launch Nav2 in the Gazebo simulation world, observe the global path, local path, and cost map in RViz, and identify each component against Figure 3-1 of this chapter.

Further Reading

1. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011. (The principal reference for the systems view of this chapter; its Chapter 1 opens with the three questions of navigation.)
2. J. J. Leonard and H. F. Durrant-Whyte, "Mobile Robot Localization by Tracking Geometric Beacons," IEEE Transactions on Robotics and Automation, 1991. (One of the classic sources of the "where am I" formulation.)
3. R. Brooks, "A Robust Layered Control System for a Mobile Robot," IEEE Journal of Robotics and Automation, 1986. (The founding work of behavior-based architecture; essential for understanding how architectures evolved.)
4. S. Macenski et al., "The Marathon 2: A Navigation System," IROS 2020. (The official paper on the Nav2 navigation system.)
5. ROS 2 official documentation (docs.ros.org) and Nav2 official documentation (docs.nav2.org). (Browsing their conceptual introduction pages before the implementation chapters is recommended.)

6. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022. (The authoritative introduction to the design philosophy and industrial use of ROS 2.)

7. A. Hornung et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," Autonomous Robots, 2013. (The representative work on 3D occupancy grid mapping.)

From System Blueprint to Sensors: Closing Part I

Part I ends here. The division of labor among its three chapters is as follows: Chapter 1 established the problem (what SLAM must solve), Chapter 2 provided the learning map (how the world teaches it and how this book is organized), and this chapter drew the system blueprint (where SLAM sits in the whole).

These three chapters deliberately contain no mathematics and no code, the purpose being to establish a complete conceptual framework first. The author's teaching experience is that students who enter formulas and code too early tend to miss the forest for the trees, producing a system that runs but being unable to say why it is designed as it is. With this framework in place, the technical details that follow have somewhere to attach.

Figure 3-5 From Part I to Part II

[The Three Frameworks Established in Part I]

Problem framework → localization and mapping depend on each other and must be solved together (Chapter 1)

Learning framework → sensors → mathematics → front end → back end → loop closure → capstone (Chapter 2)

System framework → perception → localization and mapping → planning → control (this chapter)

[The Questions Part II Will Answer]

With what does a robot see the world? What can each sensor do, and when does it fail?

Chapter 4 Indoor positioning → Wi-Fi, UWB, Bluetooth, visual markers

Chapter 5 Satellite navigation → GNSS principles, urban canyons, fusion with inertial sensing

Chapter 6 Geomagnetic navigation → electronic compass, magnetic interference, geomagnetic fingerprints

Chapter 7 IMU → accelerometer, gyroscope, drift, attitude filtering

Chapter 8 LiDAR → time of flight, 2D and 3D, specifications and blind zones

Chapter 9 Camera → pinhole model, calibration, monocular, stereo, and RGB-D

[The Common Structure of Part II] Every chapter unfolds as principle → specifications → failure scenarios → complementarity → implementation.

Pay particular attention to the failure scenarios discussed in each chapter. Chapter 4 treats the fluctuation of Wi-Fi signals, Chapter 5 multipath in urban canyons, Chapter 6 magnetic distortion in steel-frame buildings, Chapter 7 the rate of IMU drift, Chapter

8 LiDAR's blindness to glass, and Chapter 9 the camera's curse of the blank white wall.

These are not negative asides but the basis of engineering judgment. By Section 30.7 you will watch LiDAR and camera each break down at the same site with your own eyes—and looking back then at every chapter of Part II, you will understand why this book stresses limitations from the very beginning.

Summary

1. An autonomous robot operates on the Sense–Think–Act loop; modern systems adopt a hybrid architecture in which a low-level reactive loop handles obstacle avoidance and balance while an upper deliberative planning layer carries SLAM and navigation.
2. The four map representations each have their domain: the occupancy grid favors planning and obstacle avoidance, the feature map is compact and well suited to estimation, the point cloud has the highest fidelity, and the semantic map supports language-level tasks; practical systems commonly mix them.
3. Localization divides into relative localization (high-rate but drifting), absolute localization (drift-free but limited in accuracy), map-based localization (matching against an existing map), and relocalization (global recovery after becoming lost); the four are complementary and together constitute a complete localization capability.
4. A navigation system strings four modules into a data flow—SLAM / localization, global planning, local obstacle avoidance, and motion control; SLAM is the precondition for every module downstream.

5. ROS / ROS 2 connects modules through the standardized interfaces of nodes, topics, and services, and together with the rosbag and RViz tools has become the de facto standard for teaching and research in SLAM; ROS 1 reached end of support in 2025, and the implementation work in this book is based on ROS 2 LTS releases.
6. From the robotic vacuum to the self-driving car, navigation requirements rise step by step with spatial dimension, speed, and safety demands, and the sensor suite and system design change accordingly; the chapters of this book are arranged along exactly this spectrum of difficulty.

With this, Part I, "Introduction to SLAM and Autonomous Navigation," comes to a close—you now know what problem SLAM is, how leading universities teach it, and where it sits within the robot as a whole. From the next chapter we enter Part II and take apart the robot's senses one by one, beginning with a question that seems paradoxical: why are more than thirty GPS satellites overhead of no help indoors?

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Part II Sensors for Localization and Navigation

Chapter 4

Indoor Positioning Technologies

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Part I has settled the SLAM problem, how the subject is taught around the world, and the overall blueprint of a navigation system. From this chapter we enter Part II and examine the sensors on a robot one by one—what question each can answer, and under what conditions each fails.

This chapter opens Part II, and its subject is indoor positioning. It has a crucial division of labor with the chapters that follow: satellite navigation in Chapter 5 fails indoors; the geomagnetic field in Chapter 6 and the IMU in Chapter 7 supply direction but not absolute position; and the LiDAR of Chapter 8 and the camera of Chapter 9 need a map before they can localize. The technologies treated here—Wi-Fi, Bluetooth, UWB, RFID, and visual markers—are the only class that gives absolute coordinates indoors directly.

This matters greatly for SLAM. Chapter 1 noted SLAM's two great difficulties: accumulated drift and loop closure detection. The technologies of this chapter supply, from outside, a drift-free absolute reference that becomes an additional constraint edge in the graph optimization of Chapter 12. In other words, this chapter is not a substitute for SLAM but an anchor for it.

Figure 4-0 How Chapter 4 Connects to the Rest of the Book

[Building on the previous chapters]

The system blueprint of Chapter 3 told us that localization is one module of navigation; this chapter supplies the solution for indoor settings.

What does the "absolute localization" of Section 3.3 rely on indoors? This chapter is the

answer.

[This chapter is responsible for]

Radio ranging (Wi-Fi / BLE / UWB) + landmark identification (RFID / QR / AprilTag) = absolute indoor position

[Where it leads]

→ Chapter 5 The outdoor counterpart: GNSS (complementary to this chapter, indoors and out)

→ Chapters 6 and 7 Direction sensors: geomagnetic field and IMU (supplying the heading and the high update rate this chapter lacks)

→ Chapters 9 and 21 The camera intrinsics and PnP mathematics behind AprilTag

→ Chapter 12 Writing the absolute observations of this chapter as a constraint edge in graph optimization

→ Chapters 24 and 29 Integrating AprilTag and Wi-Fi on a real robot

Learning Objectives

From this chapter we enter Part II, "Sensors for Localization and Navigation," and take apart the robot's various senses one at a time. The first stop is indoor positioning—the localization technology closest to your daily life, and the best starting point for understanding how absolute localization and SLAM complement each other. After completing this chapter, you should be able to:

1. Explain why the GPS signal fails indoors, and describe the physical causes of signal attenuation and multipath.
2. Describe the two-stage workflow of Wi-Fi RSSI fingerprint positioning—offline database construction and online matching—and implement a simple version.

3. Compare the three positioning modes of BLE beacons (proximity detection, trilateration, fingerprinting) and the accuracy of each.
4. Explain the source of UWB's high accuracy, and distinguish the two ranging architectures TWR and TDoA.
5. Compare RFID, NFC, QR codes, and visual markers (AprilTag / ArUco) as landmark-based positioning, in principle and in applicable setting.
6. Explain how indoor positioning and SLAM complement and fuse with each other, and know the trade-offs each technology makes in accuracy, cost, and deployment effort.

Core Concepts

Chinese Term	English Term	Explanation
接收訊號強度指標	RSSI	The signal strength measured at the receiver, which attenuates with distance; the cheapest basis for ranging.
指紋定位	Fingerprinting	Build a database of signal signatures for each position first, then match a live signal against it to find the most similar position.
多路徑效應	Multipath	The signal arrives by several paths after reflecting off walls, making the range too long or unstable.
非視距	NLOS	The transmitter and receiver are obstructed and the signal must penetrate or diffract; a principal cause of error.
三邊定位	Trilateration	Find the intersection from the distances to three or more known points to infer the unknown position.

Chinese Term	English Term	Explanation
三角定位	Triangulation	Infer position from angle information rather than distance.
超寬頻	UWB	A radio technology using extremely short pulses over an extremely wide spectrum, reaching decimeter or even centimeter ranging.
雙向測距	TWR	The tag and anchor communicate back and forth to measure time of flight, requiring no clock synchronization among anchors.
到達時間差定位	TDoA	Compare the difference in a signal's arrival time at several anchors; requires precise anchor synchronization and gives large capacity.
低功耗藍牙信標	BLE Beacon	A low-power Bluetooth device that broadcasts an identifier periodically; commonly used for indoor guidance.
視覺基準標記	Fiducial Marker	An artificial marker such as AprilTag or ArUco from which a relative pose can be resolved precisely from an image.
六自由度位姿	6-DoF Pose	Three axes of position plus three of rotation; a visual marker can resolve the complete pose at once.

4.1 Why GPS Alone Will Not Do Indoors

Walk into the basement parking garage of a department store and the blue dot on your phone map begins to wander, or simply reports that it cannot find you. Behind this everyday experience lie three physical limitations, one on top of another.

1. The Signal Is Simply Too Weak

GPS satellites orbit at medium altitude about twenty thousand kilometers above the surface and transmit at only about 25 to 50 watts—roughly the level of a household light bulb. After traveling twenty thousand kilometers to the ground the signal has attenuated to the order of −130 dBm, weaker than the ambient background noise, and the receiver must pull it out of that noise by correlating against a spreading code. A signal this fragile may lose several tens of decibels more passing through a reinforced concrete floor slab, which extinguishes what little remains.

2. Multipath Distorts the Range Measurement

Even when part of the signal leaks in through a window, it has usually bounced off walls, ceilings, and metal cabinets. GPS ranging works by multiplying signal time of flight by the speed of light, and a reflected path is longer than the direct path, so the computed range comes out too large. Worse, indoors the reflected path is often stronger than the direct one (which the wall has blocked), the receiver cannot tell them apart, and the position estimate therefore swings wildly—this is the main reason the blue dot jumps around in a parking garage.

The phenomenon is easy to understand geometrically: what reaches the receiver is not one signal but a set of copies with different delays and different strengths. If the ranging algorithm mistakes a reflected copy for the direct arrival, the computed range

is too long—and always too long, never too short, because a reflected path is necessarily farther than a straight one. This one-sided systematic bias cannot be averaged away over repeated measurements, which makes it the most intractable source of error in indoor radio positioning.

Figure 4-1 Indoor Multipath



Three paths arrive at the receiver at once: the direct wave is weakened by a partition or a human body, while the reflections off ceiling and floor may be stronger. If the ranging algorithm takes the latter, the computed range exceeds the true range—and always in that direction.

Why This Has Anything to Do with SLAM

The LiDAR of Chapter 8 and the visual ranging of Chapter 21 meet a similar problem of false correspondence: a wrong observation taken for a right one pulls the estimate off course.

The treatment is also shared—the RANSAC of Chapter 21 and the robust

kernels of Chapter 12 are, in essence, both saying: do not trust a few anomalous observations.

So although this chapter is about radio, the intuition you gain here—that outliers bias systematically to one side—applies equally in the geometric chapters that follow.

3. The Geometry Deteriorates

Satellite positioning requires at least four well-distributed satellites (three for position and one to solve for clock error). Indoors, the only satellites still visible are usually the few toward a window, so the geometric distribution is very poor. Positioning theory describes this with the dilution of precision (DOP): when the satellites cluster in one direction the DOP rises sharply, and even if every individual range is accurate, the error of the intersected position is amplified several times over.

A Clarification: The Trilemma of Indoor Positioning

① Accuracy: meter level or centimeter level? Each order of magnitude in accuracy usually costs an order of magnitude more.

② Cost: how much infrastructure must be installed in the environment (anchors, beacons, markers)?

③ Coverage: one building? one floor? one room? The larger the coverage, the higher the cost of deployment and maintenance.

Every technology introduced in this chapter is, in essence, a different answer to the trade-off among these three dimensions.

It is precisely because no single indoor positioning technology satisfies all three requirements at once that SLAM matters so much: SLAM requires no pre-installed infrastructure, and the robot can map and localize in an unfamiliar environment with its own sensors. Conversely, SLAM has the weaknesses of drift and relocalization—which is the subject of fusion treated in Section 4.7 at the end of this chapter.

4.2 Wi-Fi RSSI Fingerprint Positioning

Wi-Fi is the most ubiquitous indoor wireless infrastructure—access points (APs) are everywhere in classrooms, offices, and shopping malls, and every phone and laptop has a receiver built in. Since the signal is everywhere, can it be used for positioning? It can, but by a method entirely unlike GPS.

Why RSSI Cannot Be Converted Directly into Distance

The intuitive idea is that signal strength decays with distance, so measuring strength should give distance, and trilateration should then give the intersection point. In theory, signal strength in free space follows the log-distance path loss model:

$$RSSI(d) = RSSI(d_0) - 10 \cdot n \cdot \log_{10}(d / d_0)$$

where n is the path loss exponent (about 2 in free space, commonly 2 to 4 indoors because of obstruction). The problem is that indoors n varies sharply with wall material, foot traffic, and furniture layout: measured RSSI at the same distance can differ by more than 10 dB, which converts into range errors of several meters. The human body is also an excellent absorber—a phone held in the hand reads differently from one lying on a desk. Indoor Wi-Fi positioning therefore does not take the ranging route but the fingerprinting route.

Fingerprint Positioning: A Two-Stage Workflow

The core insight of fingerprint positioning is this: do not try to understand how the signal propagates; simply record what each place looks like. The workflow has two stages:

- Offline (training) stage: divide the site into grid points (for example one reference point every 2 meters), stand at each point for a few seconds, and record the MAC address and RSSI of every visible AP, forming one signal fingerprint. The fingerprints of all reference points constitute the fingerprint database, or radio map.
- Online (positioning) stage: the user's device scans the APs and RSSI values it sees at that moment, compares them for similarity against every fingerprint in the database, and takes the most similar one (or a weighted average of the top K) as the estimated position.

Figure 4-2 The Two Stages of Wi-Fi Fingerprint Positioning

[Offline stage: building the fingerprint database] (slow, but done only once)

(1) Select reference points across the site (for example one every 2 meters)

↓

(2) Stand at each reference point for a few seconds and record the MAC and RSSI of every

visible AP

↓

(3) Store as a database: position (x, y) ↔ fingerprint vector F = [$RSSI_1$, $RSSI_2$, …, $RSSI_n$]

[Online stage: positioning in real time] (fast, runs continuously)

(4) The device scans current RSSI values, giving the observation vector S

↓

(5) Compute the distance D(S, F_i) against every fingerprint F_i in the database

↓

(6) Take the K closest and average their coordinates with weights → output the estimated position

Key idea: the "distance" here is a distance in signal space, not a physical distance.

You will meet this two-stage structure repeatedly in later chapters. The geomagnetic fingerprint positioning of Chapter 6 is the same workflow with field strength substituted for RSSI; the bag-of-words loop closure detection of Chapter 20 follows the same logic—build a feature dictionary offline, compare similarity online. Even the particle weighting of Gmapping in Chapter 14 is, in essence, asking how closely the current observation resembles the observation this position on the map should produce.

Once you see this, you will recognize that localization is in large part matching. The only difference is what is being matched: signal strength, magnetic field, image features, or point cloud geometry.

The Matching Algorithm: K Nearest Neighbors

The most common matching method is KNN (K-nearest neighbors). Let the

currently measured fingerprint vector be S and the fingerprint of the i-th reference point in the database be F_i. The Euclidean distance between them is:

$$D(S, F_i) = \sqrt{\Sigma_j (S_j - F_{ij})^2}$$

where j runs over all commonly visible APs. Take the K reference points with the smallest D and average their coordinates, with weights, to obtain the estimated position. The reciprocal of distance is commonly used as the weight, giving WKNN.

Strengths and Limitations

- Strengths: zero hardware cost (it uses existing APs), broad coverage, and implementable on a phone, which makes it very well suited as a teaching entry point.
- Limitations: accuracy is usually 3 to 8 meters; building the fingerprint database is laborious (one floor may need several hundred reference points); and changes in the environment (an AP moved, partitions rebuilt, crowds) invalidate the database, so it must be rebuilt periodically.

Recent improvements include replacing KNN matching with deep learning, smoothing the trajectory with pedestrian dead reckoning (PDR) from the phone's IMU, and ranging directly with Wi-Fi RTT (round-trip time, IEEE 802.11mc)—the last of which can raise accuracy to 1 to 2 meters, but requires support in both the AP and the device.

4.3 Bluetooth Beacon / BLE Positioning

A Bluetooth Low Energy (BLE) beacon is a small device that runs for one to two years on a single coin cell and periodically broadcasts its own identifier. It became widespread after Apple introduced the iBeacon specification in 2013, and Google proposed the Eddystone specification as well. Compared with Wi-Fi APs, the advantage

of beacons is that you choose where to put them—both density and placement can be designed around the positioning requirement.

Three Modes of Use

- Proximity detection: the simplest and most reliable. No coordinates are computed; the system only judges whether the user is near a given beacon, in three coarse levels—immediate (within 10 cm), near (about 1 m), and far (within 10 m). Museum exhibit guides and in-store promotions commonly use this mode. For a robot it amounts to "I have passed a checkpoint."
- Trilateration: RSSI from three or more beacons is converted to distance and the intersection is solved. The theory is elegant, but because of the RSSI instability discussed in Section 4.2 the practical accuracy is usually 2 to 5 meters, and a good geometric layout of beacons is required.
- Fingerprinting: identical to the Wi-Fi fingerprint method, with beacons in place of APs. Because beacon density is under your control, accuracy is usually somewhat better than with Wi-Fi fingerprints.

Advanced: Bluetooth Direction Finding

From Bluetooth 5.1 onward, direction finding measures the angle of arrival (AoA) or angle of departure (AoD) of the signal with an antenna array. Because angle information is far more stable than RSSI, a well-designed system can push accuracy to the decimeter or even centimeter level—the most important recent advance in BLE positioning. The cost is array antenna hardware, which raises both price and deployment complexity.

Engineering Considerations

- Broadcast rate and battery life are a seesaw: ten broadcasts per second responds quickly but drains the cell, one per second saves power but produces noticeable positioning latency.
- BLE uses the 2.4 GHz band, shared with Wi-Fi and microwave ovens, so interference is significant in crowded venues (the human body has high water content and absorbs 2.4 GHz particularly strongly).
- Beacons must be inspected and have their batteries replaced on a schedule—a long-term maintenance cost that is easily overlooked and considerable when several hundred units are deployed.

4.4 Ultra-Wideband (UWB) Positioning

If Wi-Fi and BLE guess distance from signal strength, ultra-wideband (UWB) measures time directly and computes distance from it—which is the fundamental reason for its leap in accuracy.

Why UWB Is So Accurate

UWB transmits extremely short pulses lasting only a few nanoseconds, occupying a bandwidth of 500 MHz or more. Time resolution is inversely proportional to bandwidth: the shorter the pulse, the more precisely the receiver can determine the instant of arrival. Since light travels about 30 centimeters per nanosecond, resolving arrival time to the sub-nanosecond level yields range at the centimeter level.

More important still is immunity to multipath. A reflected signal arrives later than the direct one; with ordinary narrowband signals the waveform is drawn out, so direct and reflected arrivals overlap and cannot be separated. The UWB pulse is so short that the first arriving peak (the direct wave) is clearly separated from the later reflections,

and the receiver need only take that first peak—exactly the property most needed indoors.

Two Ranging Architectures

- Two-way ranging (TWR): the tag transmits to the anchor and the anchor replies; from the total round-trip time minus the processing time the tag computes the one-way time of flight and the distance. The advantage is that no time synchronization between anchors is needed; the drawback is that every measurement requires a round trip, so the channel congests when there are many tags.
- Time difference of arrival (TDoA): the tag broadcasts one way only, several anchors record their own times of arrival, and a server solves for position from the time differences. The advantages are that the tag only transmits and never receives, which is extremely power-efficient, and that system capacity is almost independent of the number of tags; the drawback is that every anchor must be synchronized to the nanosecond, which makes deployment demanding.

The Truth About Accuracy

Vendor literature often advertises "10 cm accuracy," which is indeed achievable under good line of sight (LOS) with an ideal anchor geometry. In a real site, however, error rises noticeably as soon as a body or a piece of equipment creates a non-line-of-sight (NLOS) condition; comparative studies show that the newer IEEE 802.15.4z standard can hold mean ranging error under NLOS within 0.2 meters, a clear improvement over the older standard. A pragmatic engineering expectation is decimeter level under good deployment, with errors of tens of centimeters or more where obstruction is severe.

Where UWB Stands Today

UWB has moved from the laboratory into consumer and industrial use: precise item finding with phones and tracking tags, automotive digital keys (secure ranging resistant to relay attacks), real-time location systems (RTLS) in factories and warehouses that keep forklifts from colliding, and even ball tracking in sports. For robots, UWB is particularly well suited as a drift-free absolute reference fused with SLAM—the subject of Section 4.7 and a common solution for indoor UAV positioning.

4.5 Landmark Positioning with RFID / NFC / QR Codes

The technologies of the previous three sections all try to estimate position continuously. This section introduces a different idea: discrete landmark positioning—not knowing the coordinates at every instant, but being absolutely certain of where you are when you pass a particular place. This approach is very cheap and very reliable, and is for that reason more popular in industrial settings.

RFID (Radio Frequency Identification)

A passive RFID tag has no battery of its own: it is powered by induction from the field emitted by the reader and returns its identifier, at a cost as low as a few NT dollars. By frequency band they divide into low frequency (LF, a few centimeters), high frequency (HF, about 10 cm), and ultra-high frequency (UHF, several meters). The classic approach for a warehouse AGV is to embed an array of RFID tags under the floor; a reader on the underside of the vehicle reads the identifier as it passes and corrects the position—accuracy depends on tag density, so denser laying means better accuracy.

NFC (Near Field Communication)

NFC can be regarded as a specialized subset of HF RFID with a working range of only about 4 centimeters. The short range is itself an advantage: a successful read means the device is practically touching the tag, so positional certainty is extremely high. It suits charging-dock alignment, tool-cabinet check-in and check-out, and other situations that require zero ambiguity.

QR Codes and Data Matrix Codes

The great advantage of the QR code is that it costs nothing—a sticker from an office printer is a landmark, and a damaged one is peeled off and replaced. This is exactly the approach used by Amazon's warehouse robots (the Kiva system): QR codes are laid out in a regular pattern across the floor, a downward-facing camera under the robot photographs them, and reading the identifier tells the robot which cell of the grid it occupies, while the position and rotation of the code within the image give the fine offset and heading. Hundreds of thousands of robots run in warehouses worldwide on this seemingly humble scheme.

Technology	Working Range	Cost	Information Provided	Typical Application
Passive RFID	Centimeters to several meters	Very low	Identifier	Warehouse floor tags, logistics tracking
NFC	About 4 centimeters	Very low	Identifier	Charging dock alignment, tool check-out
QR Code	Depends on the camera	Almost zero	Identifier + coarse pose	Warehouse robots, teaching experiments
AprilTag / ArUco	Centimeters to several meters	Almost zero	Complete 6-DoF pose	Robot navigation, camera calibration

Table 4-1 Comparison of Four Landmark-Based Positioning Technologies

4.6 Visual Landmarks and AprilTag / ArUco Positioning

A QR code can tell a robot which cell it is in, but what if we want more precise information—the complete three-dimensional position and orientation of the camera relative to the marker? That is where fiducial markers come in. AprilTag (developed at the University of Michigan) and ArUco (developed at the University of Córdoba in Spain and integrated into OpenCV) are the two most widely used systems.

Why a Full Pose Can Be Recovered

The principle is quite intuitive. The marker is a square of known side length, and the three-dimensional coordinates of its four corners are fully known in the marker's own frame (for a marker 10 cm on a side, the corners are (0,0,0), (0.1,0,0), (0.1,0.1,0), and (0,0.1,0)). When a camera photographs the marker, the two-dimensional pixel coordinates of those four corners can also be detected precisely. Given four 3D-to-2D point correspondences plus the camera intrinsics (introduced in detail in Chapter 9), the rotation and translation of the camera relative to the marker can be solved—what computer vision calls the PnP problem (perspective-n-point), examined in depth in Chapter 21.

The Detection Pipeline

1. Binarization: convert the grayscale image to black and white to bring out the high-contrast boundary of the marker.
2. Contour detection and quadrilateral filtering: find closed contours and keep those that approximate a square.

3. Perspective rectification: warp the obliquely imaged quadrilateral back into a square (a homography).
4. Code reading: read the bit pattern of the internal black-and-white cell array, match it against the dictionary to confirm the ID, and reject false detections using the built-in Hamming error correction.
5. Pose solution: solve PnP from the four corners and the camera intrinsics, outputting a 6-DoF pose.

AprilTag Compared with ArUco

- AprilTag: error correction and false-detection rejection are designed more rigorously, so it remains comparatively robust at long range, at low resolution, and under partial occlusion; it is preferred in academia and robotics.
- ArUco: fast detection, a dictionary of configurable size, and deep integration with OpenCV (a few lines of code suffice), which makes it very convenient for teaching and rapid prototyping.

Their Role in a Robotic System

- Absolute pose correction: place markers at the charging dock, by the elevator, and at the laboratory door, and the robot eliminates accumulated drift the moment it sees one—the most common use.
- Precise docking: actions requiring millimeter alignment, such as charging or picking and placing material, use a visual marker for the final stage of visual servoing (Chapter 24).
- UAV landing: a marker on the landing pad lets the aircraft touch down precisely.

- Ground truth for a SLAM system: in teaching experiments, densely placed markers can supply an approximate ground-truth trajectory for evaluating SLAM accuracy (used in the capstone projects of Chapter 31).

It is worth noting that a visual marker is essentially an artificial feature point guaranteed to be reliable. Understanding it is understanding a simplified version of the natural feature points of visual SLAM in Chapter 20—the difference being that natural features are not guaranteed stable, not guaranteed unique, and have no known size, which is precisely where the difficulty of visual SLAM lies.

4.7 Fusing Indoor Positioning with SLAM

By this point you will have noticed a pattern: the strengths and weaknesses of indoor positioning and of SLAM are exactly complementary.

Characteristic	Indoor positioning (Wi-Fi / BLE / UWB / markers)	SLAM
Error behavior	Bounded; does not accumulate with time	Accurate in the short term; drifts in the long term
Infrastructure	Must be deployed and maintained beforehand	None needed; usable on arrival
Initial position	Knows where it is at power-on	Must start from the origin or perform relocalization
Map information	Coordinates only; no environmental structure	Produces a complete map usable for obstacle avoidance and planning
Update rate	Lower (a few times per second)	Higher (tens of times per second)

Table 4-2 How Indoor Positioning and SLAM Complement Each Other

Three Levels of Fusion

- Loosely coupled: the two systems run independently, each outputting its own position estimate, and the two are then combined with weights by a Kalman filter or similar. This is the simplest to implement and the first choice for teaching and rapid prototyping.
- Tightly coupled: the raw measurements of the indoor positioning system (UWB ranges, marker corners) enter the SLAM back-end optimization directly as constraint terms and are solved together. Accuracy is best because all information is handled consistently within one probabilistic framework—a natural extension of the pose-graph optimization of Chapter 12, in which a UWB range is simply an edge between a pose and an anchor.
- Aiding initialization and relocalization: the most pragmatic use. SLAM runs on its own most of the time, and UWB or a visual marker supplies the initial pose at start-up or a relocalization cue after loss of track. At very low integration cost this solves SLAM's two most troublesome problems.

A Representative Architecture

The complete localization system of a campus service robot might look like this: 2D LiDAR SLAM as the mainstay, supplying pose and map ten times per second; AprilTags posted by the elevators and in the lobbies of every floor, so that the robot corrects its absolute pose each time it passes one; NFC at the charging dock to ensure error-free docking; and BLE beacons to decide which floor it is on when moving between levels (a question LiDAR SLAM cannot answer at all, since the laser profile of every floor may look identical). Each technology does its own job—which is the essence

of systems engineering: there is no universal sensor, only an appropriate combination.

Formulations and Procedures

1. The Mathematics of Trilateration

Let there be three anchors at known positions $A_1(x_1, y_1)$, $A_2(x_2, y_2)$, and $A_3(x_3, y_3)$, with measured tag-to-anchor distances d_1, d_2, and d_3. The tag position (x, y) then satisfies three circle equations:

$$(x - x_i)^2 + (y - y_i)^2 = d_i^2 \quad , \quad i = 1, 2, 3$$

Subtracting the second and third equations from the first cancels the squared terms and leaves two linear equations, from which (x, y) can be solved.

In the ideal error-free case the three circles meet at exactly one point; with real measurement error they bound a small triangular region instead. The solution is then obtained by least squares—find the point that minimizes the sum of squared deviations from the three circles:

$$\min \Sigma_i \left(\sqrt{(x - x_i)^2 + (y - y_i)^2}\right) - d_i)^2$$

This idea of using redundant measurements to fight noise is exactly the spirit of the SLAM back-end optimization of Chapter 12—you will find that from trilateration to bundle adjustment the mathematical skeleton is one and the same.

2. Computing Time of Flight in TWR

In two-way ranging, let T_round be the total time from the tag's transmission to its receipt of the reply, and T_reply the processing time at the anchor from receiving the signal to sending the reply. The one-way time of flight and the distance are then:

$$T_flight = (T_round - T_reply) / 2$$

$$d = c \times T_flight \quad (c \text{ is the speed of light, about } 3\times10^{8} \text{ m/s})$$

This shows how critical timing accuracy is: at the speed of light a 1-nanosecond error equals 30 centimeters of range error, and achieving 10 cm accuracy requires timing resolution better than 0.33 nanoseconds. It also explains why ordinary Wi-Fi and Bluetooth chips cannot do this—their clock resolution falls far short of what such a measurement demands.

Figure 4-3 Timing Relationships in Two-Way Ranging (TWR)



This figure explains both why UWB is accurate and why it is expensive. T_reply is measured and reported by the anchor itself, which cancels the effect of clock offset between the two ends—the chief advantage of TWR over one-way ranging, which requires strict clock synchronization. The price is two packets per range measurement, which limits the update rate, and the fact that every tag must take its turn conversing with the anchors, which makes system capacity the practical bottleneck.

3. A Procedure for Choosing an Indoor Positioning Technology

Figure 4-4 A Decision Procedure for Selecting an Indoor Positioning Technology

Question 1: what accuracy is required?

├─ Meter level will do → Wi-Fi / BLE fingerprint positioning (lowest cost)

├─ Decimeter level → UWB or BLE direction finding (anchors must be deployed)

└─ Centimeter level → UWB (with good LOS) or visual markers (at short range)

↓

Question 2: may equipment be installed in the environment?

├─ Not at all → SLAM is the only option (Parts IV through VI)

├─ Paper may be posted → QR Code / AprilTag (zero cost)

└─ Construction and wiring are possible → UWB anchors / an array of BLE beacons

↓

Question 3: is a continuous trajectory needed, or discrete correction?

├─ Continuous → Wi-Fi / BLE / UWB

└─ Discrete correction → RFID / NFC / visual markers + SLAM as the mainstay

4.8 The Contemporary Evolution of Indoor Positioning: A Race Beyond Accuracy

The technical skeleton described in the first seven sections has been fairly stable over the past decade, but several changes worth knowing appeared in 2025 and 2026. Interestingly, most of them are not about greater accuracy but about easier deployment and wider availability.

1. Bluetooth Channel Sounding: A Low-Cost Challenger to UWB

The Bluetooth 6.0 specification incorporates channel sounding, which replaces distance estimation from RSSI with phase-based ranging (PBR). Its theoretical accuracy is on the order of a meter, and under good conditions it can approach the decimeter level, at a cost and power consumption far below UWB, and along the evolutionary path of existing Bluetooth silicon.

Industry assessments, however, are quite sober: line-of-sight obstruction, multipath distortion, scalability, and battery life keep it from being a viable solution for large-scale indoor mobile positioning today. For students this is an instructive case—the theoretical accuracy of a technology and its deployability are two different things, and engineering selection always looks at the latter. That is precisely the habit of mind the selection flowchart in Section 4.7 means to convey.

2. Wi-Fi RTT and 802.11az: The Standard Exists, Adoption Does Not

Wi-Fi round-trip-time ranging (RTT, IEEE 802.11mc) can deliver sub-meter accuracy, but support remains the bottleneck—industry figures put it at roughly thirty percent of Android devices, and iOS does not expose it. An improved one-way RTT scheme works with any AP, at the price of accuracy falling back to three to five meters and requiring extensive post-processing and calibration. The new 802.11az standard brings positioning formally into the Wi-Fi specification and is expected to push accuracy to the decimeter level, but infrastructure replacement cycles are measured in years.

This leads to a judgment that matters to an engineer: in a campus or laboratory setting, the RSSI fingerprint method of Section 4.2, however unglamorous, is the only one that works on every device. That is why Exercise 4-1 in this chapter uses fingerprinting rather than RTT.

3. UWB: The Most Accurate, Still a Premium Fitting

UWB can localize to within thirty centimeters, making it the most accurate of all the wireless technologies in this chapter. Its adoption is limited by hardware cost, support only in premium iOS and Android models, and the infrastructure requirement of deployed anchors. In warehouses, hospitals, factories, and other applications where the site is fixed and the value at stake is high enough, UWB is already a mature solution; in consumer and teaching settings it remains expensive.

4. New Developments in Visual Markers: Making the Marker Invisible

The AprilTag and ArUco markers of Section 4.6 have one long-standing complaint against them: the black-and-white squares are ugly and look badly out of place on the wall of a hospital, an office, or a home. iMarkers, published in 2026, propose an interesting remedy—replacing printing ink with cholesteric spherical reflectors (CSR), which yields markers that are nearly invisible to the human eye yet clearly legible to a camera under particular polarization conditions.

The measured results are worth noting: detection range is comparable to printed ArUco markers of the same size, pose estimation error differs by only 1.3 to 3.9 percentage points, and recognition takes 21 to 105 milliseconds; in very low light below five lux the detection rate still reaches 78 to 89 percentage points, whereas printed markers drop to 6 percentage points. For a robot that must work a dim corridor or patrol at night, this is a substantive improvement.

Another line of work replaces the traditional image-processing pipeline with deep learning. YoloTag, for example, applies an object detection model to marker detection and proves more robust than the conventional binarization pipeline on platforms such as UAVs, which shake violently and often produce motion-blurred images. This belongs to the same thread as the role of YOLO in navigation in Chapter 24 and AI-assisted navigation in Chapter 32.

5. The Real Trend: From Tuning Toward Calibration-Free Operation

Taken together, these changes reveal a common direction: ease of deployment is displacing theoretical accuracy as the first criterion in system selection. Self-calibrating systems that require no fingerprint survey are gradually becoming the industry baseline, because the cost of collecting and maintaining fingerprints noted in Section 4.2 is often harder to accept in a real commercial setting than being a meter less accurate.

Three Criteria for Students

1. Before asking how accurate it must be, ask what the accuracy is for. Navigating to a doorway needs thirty centimeters; finding a particular room needs only three meters.

2. Beyond accuracy there are three further dimensions: deployment cost, device compatibility, and maintenance burden. Table 4-1 and the selection procedure of Section 4.7 are about balancing all four.

3. Absolute localization need not carry all the accuracy alone—fused with the IMU of Chapter 7 and the SLAM of Chapter 13, even three-meter Wi-Fi positioning suffices to support dependable navigation. This is the central claim

of Section 4.7 and the most important idea in this chapter.

Case Studies

Case 4-1 Amazon's Warehouse Robots: Global Logistics Carried on Stickers

The warehouse robots of Amazon Robotics (formerly Kiva Systems) are a textbook case of low technology solving a hard problem. Hundreds of thousands of orange transport robots move through warehouses worldwide, and yet their localization scheme is startlingly plain: QR codes are pasted across the floor at a fixed pitch, a downward-facing camera under the robot reads the identifier to confirm the cell, the offset and rotation of the code in the image give the fine pose, and between cells wheel odometry carries the short-range dead reckoning.

The brilliance of this scheme lies in exploiting the controllability of the environment to the full: the warehouse is built by the company itself, so the floor can be marked at will, the lighting can be provided, and the routes can be constrained to a grid. Given all that, why make the robot struggle with visual SLAM? A service robot, by contrast, enters someone else's building and cannot modify the environment, which is exactly why it must rely on SLAM. The contrast carries an important lesson: before choosing a localization technology, ask first how far you are able to modify the environment—this is often more decisive than the algorithm itself.

Case 4-2 A UWB Real-Time Location System in a Hospital

Large hospitals face a costly problem: mobile equipment such as infusion pumps and ultrasound carts is constantly "wheeled off somewhere," nursing staff spend a

great deal of time each day hunting for it, and the hospital ends up over-purchasing. The remedy offered by a UWB real-time location system (RTLS) is this: deploy a grid of anchors in the ceiling, attach one UWB tag to each piece of equipment, broadcast one way in a TDoA architecture, and have a server solve for all tag positions in real time and display them on the floor plan.

This case demonstrates the value of the TDoA architecture: an equipment tag only transmits and never receives, so one battery lasts for years, and system capacity is nearly independent of the number of tags, so tens of thousands of tags in one hospital still cause no congestion. The challenge in a hospital is the NLOS conditions created by metal equipment and foot traffic—which is also why practical accuracy often falls short of the advertised figure. With small adjustments the same architecture serves forklift collision avoidance in a factory, personnel safety-zone management in a warehouse, and absolute pose correction for a robot.

Case 4-3 An AprilTag Navigation Experiment in the Classroom

A typical teaching experiment in a university intelligent robotics classroom: post ten AprilTags on the four walls, with the world coordinates of each marker measured and recorded in advance. The teaching vehicle carries a single forward-facing camera, and whenever any marker appears in the field of view it can solve for its own 6-DoF pose relative to that marker and convert it into world coordinates—obtaining drift-free absolute localization instantly.

The teaching value of this experiment is very high. First, it lets students verify with their own hands the seemingly magical fact that four corner points plus the camera intrinsics suffice to recover a pose. Second, by covering half a marker, backing away,

and turning off the lights, students experience the fragile points of visual localization directly. Third, and best of all, running it in parallel with LiDAR SLAM and displaying both trajectories at once in RViz—the SLAM trajectory smooth but slowly drifting away, the AprilTag trajectory jumpy but always correct. That one picture is worth a thousand words and conveys the whole motivation for fusion in Section 4.7 at a glance.

Hands-On Activities

Exercise 4-1 A Wi-Fi Signal Heat Map for Classroom Positioning

Purpose: to carry out one complete fingerprint positioning workflow by hand—build the database, take measurements, match, and validate—and so understand both the capability and the limits of RSSI positioning. This exercise requires only a phone and suits high school and undergraduate groups alike.

Equipment

- A smartphone (on Android, install a Wi-Fi analyzer app; because iOS restricts this more tightly, use the AirPort utility instead or substitute BLE beacons).
- Graph paper or spreadsheet software, a tape measure, and tape for marking reference points.

Procedure

1. Site planning: choose a classroom, measure its length and width with the tape, mark grid points on the floor at a 2-meter pitch (an ordinary classroom gives about 12 to 20 reference points), and number them on the graph paper.
2. Offline database construction: stand at each reference point facing a fixed direction (toward the lectern throughout, say, to reduce differences caused by shadowing by the body), record the name and RSSI of the five strongest visible

APs, take three measurements per point and average them, and enter the result in the spreadsheet.

3. Drawing the heat map: for the strongest AP, fill in the RSSI value at each reference point on the graph paper as a color scale (red for strong, blue for weak), and observe whether the spatial distribution matches the expectation that signal weakens with distance from the AP.
4. Online positioning test: one student is led with eyes closed to an undisclosed reference point and measures the fingerprint there; the others estimate the position by KNN (compute the Euclidean distance to every fingerprint in the database and average the coordinates of the three closest).
5. Error statistics: repeat the test at least 8 times, record the estimated and true positions each time, and compute the mean and maximum error.
6. Interference experiment: have the whole class gather around the test point, measure the fingerprint of that same point again, and compare the change in RSSI with the change in positioning error.

Discussion Questions

1. What mean positioning error did you measure? Does it agree with the 3 to 8 meters stated in the text? Where is the reference point with the largest error, and why there?
2. By how much did the crowd increase the error? What does this imply for indoor navigation in a shopping mall at peak hours?
3. If the reference-point pitch were reduced from 2 meters to 1 meter, how much would accuracy improve? How much would the survey workload increase? Is the trade worth it?

4. (Undergraduate) If you were to fuse this system with the LiDAR SLAM of Chapter 14, which of the coupling approaches in Section 4.7 would you adopt, and why?

Extended Challenge (Undergraduate)

Write a WKNN positioning program in Python that reads the fingerprint database you built together with a live measurement and outputs the estimated coordinates and error statistics automatically; then compare accuracy for K = 1, 3, and 5, and try adding step detection from the phone's IMU to smooth the trajectory, observing whether the error falls.

Further Reading

1. P. Bahl and V. N. Padmanabhan, "RADAR: An In-Building RF-Based User Location and Tracking System," IEEE INFOCOM, 2000. (The founding work on Wi-Fi fingerprint positioning; essential reading.)
2. H. Liu et al., "Survey of Wireless Indoor Positioning Techniques and Systems," IEEE Transactions on Systems, Man, and Cybernetics, 2007. (A complete survey of indoor positioning technologies.)
3. IEEE Std 802.15.4z-2020, Enhanced Ultra Wideband (UWB) Physical Layers and Associated Ranging Techniques. (The current standard document for UWB ranging.)
4. E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," ICRA, 2011. (The original AprilTag paper, explaining the error-correction design.)
5. S. Garrido-Jurado et al., "Automatic Generation and Detection of Highly Reliable Fiducial Markers Under Occlusion," Pattern Recognition, 2014. (The original ArUco paper.)

6. The ArUco module tutorial in the official OpenCV documentation (docs.opencv.org). (Includes complete code examples; the first reference for implementation.)
7. Bluetooth SIG, Bluetooth Core Specification 5.1, the Direction Finding sections. (The specification for AoA / AoD direction finding.)

From Indoor Positioning to Navigation Sensors: Where This Chapter Stands

The technologies introduced in this chapter share one feature: every one of them requires that something be installed in the environment beforehand—an AP, a beacon, a UWB anchor, an RFID tag, or a visual marker. That is the source of their strength (a drift-free absolute position) and also their fundamental limitation (they fail entirely in an environment where nothing has been installed).

In the five chapters that follow we turn to a different class of sensor: ones that require no installation and work from physical law or from information already present in the environment. The GNSS of Chapter 5 relies on satellites, the geomagnetic sensing of Chapter 6 on the Earth's magnetic field, the IMU of Chapter 7 on inertia, and the LiDAR and camera of Chapters 8 and 9 on the geometry and texture of the surroundings. The price is that none of them can answer directly how many meters into the building you are.

This is exactly why SLAM exists. With neither installed infrastructure nor a map, a robot can only draw the map as it goes and localize itself with the map it has just drawn. The mathematics of Part III and the algorithms from Part IV onward all address this problem. What this chapter asks students to remember is this: if the environment permits

installation, do not be a hero—a thirty-dollar AprilTag is sometimes more dependable than an entire SLAM system.

Summary

1. GPS fails indoors for three reasons: the signal is extremely weak and penetrates building materials poorly, multipath reflection distorts the range measurement, and the deteriorated geometry of the visible satellites amplifies the error.
2. Because indoor propagation is complex, Wi-Fi RSSI is unsuited to direct ranging, so the mainstream approach is fingerprint positioning—build a signal database offline and match with KNN online—with accuracy of about 3 to 8 meters, zero hardware cost, and the burden of maintaining the database.
3. BLE beacons can be placed as the application requires and support three modes: proximity detection, trilateration, and fingerprinting; the direction finding introduced in Bluetooth 5.1 raises accuracy substantially using AoA / AoD, at the cost of an array antenna.
4. UWB measures time of flight directly with nanosecond pulses and can separate the direct wave from reflections, making it the most accurate wireless positioning technology; TWR needs no synchronization but has limited capacity, while TDoA is power-efficient and high-capacity but requires precisely synchronized anchors.
5. RFID, NFC, and QR codes provide discrete but highly reliable landmark positioning at low cost and suit sites where the environment can be modified, with Amazon's warehouse robots as the representative case.
6. Visual markers such as AprilTag and ArUco can recover a complete 6-DoF pose from four corner points and the camera intrinsics (the PnP problem), which

makes them powerful tools for absolute pose correction, precise docking, and SLAM accuracy evaluation.

7. Indoor positioning and SLAM are exactly complementary: the former has bounded error but needs infrastructure, while the latter is self-sufficient but drifts; fusion comes at three levels—loosely coupled, tightly coupled, and aiding initialization and relocalization.

In the next chapter we widen the view from indoors to outdoors and take up radio and satellite navigation: how do GPS and GNSS actually work? Why can RTK compress meter-level error to the centimeter level? And how does a campus patrol robot switch seamlessly between indoor and outdoor localization modes?

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 5

Radio Navigation and Satellite Navigation

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 4 settled absolute localization indoors; this chapter handles the outdoors—and in fact anywhere outdoors on Earth. Satellite navigation is the most widely covering positioning infrastructure in human history, and the only sensor on a robot that can tell it where on Earth it is without anything having been installed.

What this chapter really means to establish, however, is not that GPS is useful but three deeper ideas. First, the mathematical skeleton of GNSS—one more unknown than there are observations (the clock error), hence the need for one more satellite—is the same habit of mind as the state estimation of Chapter 11 and the graph optimization of Chapter 12. Second, geometry determines accuracy: the notion of DOP reappears in another guise in the epipolar geometry of Chapter 21 and the point cloud registration of Chapter 17. Third, no single sensor is dependable and fusion is the answer—the triple GNSS / INS / SLAM architecture of Section 5.6 is a miniature of the second half of this book.

Figure 5-0 How Chapter 5 Connects to the Rest of the Book

[Building on the previous chapter]

Chapter 4 Absolute localization indoors (installation required) → this chapter: absolute localization outdoors (no installation)

The multipath, NLOS, and deteriorating geometry of Chapter 4 acquire quantitative names here: UERE and DOP

[This chapter is responsible for]

The three-dimensional version of range intersection + an unknown clock = pseudorange positioning

Differential corrections and carrier phase = advancing from the meter level to the

centimeter level

Judging signal quality = when a robot should trust GNSS

[Where it leads]

→ Chapters 6 and 7 Filling in when GNSS fails: the geomagnetic field supplies heading, the IMU supplies short-term dead reckoning

→ Chapter 11 This chapter's "many observations for many unknowns" formalized as state estimation

→ Chapter 12 GNSS observations as absolute constraint edges in a pose graph

→ Chapters 18 and 26 The role of GNSS in outdoor LiDAR SLAM and UAV navigation

→ Chapter 26 Autonomous navigation in GNSS-denied environments

Learning Objectives

In the previous chapter we examined indoor positioning; in this one we step outdoors and examine the globe-spanning radio and satellite navigation systems. These are the principal source of a robot's absolute localization capability, and the key lesson for understanding why self-driving cars and UAVs still need SLAM. After completing this chapter, you should be able to:

1. Describe the three segments of GNSS (space, control, and user) and the basic principle of satellite positioning, and explain why at least four satellites are required.
2. Distinguish triangulation, trilateration, and time-difference-of-arrival positioning in principle and in applicable setting.
3. Explain carrier phase measurement and the principle of RTK, and understand why RTK can compress meter-level error to the centimeter level.

4. Compare the potential and the limitations of 5G, LoRa, Wi-Fi RTT, and other terrestrial radio positioning technologies for robotic applications.
5. List and explain the principal error sources in radio positioning: the ionosphere, the troposphere, multipath, obstruction, geometry, and interference.
6. Explain how GNSS fuses with an IMU and with SLAM, and design an indoor–outdoor hybrid navigation strategy for a campus patrol robot.

Core Concepts

Chinese Term	English Term	Explanation
全球衛星導航系統	GNSS	The collective name for the satellite positioning systems of various nations, including GPS, GLONASS, Galileo, and BeiDou.
虛擬距離	Pseudorange	The distance obtained by multiplying signal propagation time by the speed of light; called "pseudo" because it includes the receiver clock error.
星曆	Ephemeris	The satellite's orbital parameters, used to compute its position precisely at any instant.
精度稀釋因子	DOP	Describes how far satellite geometry amplifies positioning error; the smaller the value the better.
載波相位	Carrier Phase	Measurement of the phase of the carrier waveform, far more accurate than pseudorange and the basis of RTK.
整週波未定值	Integer Ambiguity	The unknown whole number of cycles in a carrier phase measurement; resolving it is what achieves centimeter accuracy.
即時動態定位	RTK	A technique achieving centimeter positioning

Chinese Term	English Term	Explanation
		by differential correction from a base station.
差分 GNSS	DGNSS	Uses a base station of known coordinates to compute error corrections and broadcasts them for rovers to use.
星基增強系統	SBAS	A wide-area augmentation system broadcasting corrections from geostationary satellites, such as WAAS and EGNOS.
網路 RTK	NTRIP	The communication protocol for transmitting RTK correction data over the internet.
GNSS 拒止環境	GNSS-denied	An environment where satellite signals are obstructed or jammed and unusable, such as indoors, in tunnels, or in urban canyons.
都市峽谷效應	Urban Canyon	The phenomenon of tall buildings on both sides obstructing satellites and causing severe multipath reflection.

5.1 The Basic Principles of GPS and GNSS

GPS (Global Positioning System) is the satellite positioning system the United States began building in the 1970s and brought to full operation in the 1990s. Today it is no longer the only choice: Russia's GLONASS, the European Union's Galileo, and China's BeiDou have all achieved global coverage, and together with the two regional systems—Japan's QZSS and India's NavIC—they are collectively called the Global Navigation Satellite System (GNSS). As of March 2026 the four major systems had about 130 operational satellites in orbit, of which about 31 belong to GPS, distributed over 6 orbital planes in medium Earth orbit at an altitude of about twenty thousand kilometers. Most smartphones manufactured after 2020 support all four systems at once—which is exactly why phone positioning has become noticeably more accurate in recent years: more satellites are available, so the geometry naturally improves.

The Three Segments

- Space segment: the constellation in orbit. Each satellite carries a high-precision atomic clock and continuously broadcasts its own identifier, a time stamp, and its orbital parameters (the ephemeris).
- Control segment: the network of ground monitoring stations and the master control station. These continuously track the satellites, correct orbit and clock deviations, and upload the latest ephemeris to the satellites.
- User segment: the receiver. It only receives and never transmits—which is the key to understanding GNSS: the satellite does not know where you are, the position is computed entirely by your own receiver, and the number of users can therefore be unlimited.

The Basic Principle: Intersection of Range Measurements

GNSS positioning is, in essence, three-dimensional trilateration. The satellite stamps the signal with "I transmitted this at time t"; the receiver notes its own time on reception, the difference gives the propagation time, and multiplying by the speed of light gives the range. Knowing the satellite's position (computed from the ephemeris) and the range, the receiver lies on a sphere centered on that satellite. Three spheres from three satellites intersect, and in theory the position is determined.

Why a Fourth Satellite Is Needed

The trouble is the clock. The atomic clock aboard the satellite is expensive and accurate to the nanosecond, but the receiver holds only a cheap quartz oscillator that may at any moment differ from satellite time by several microseconds. At the speed of light 1 microsecond equals 300 meters of range error—an error large enough that the three spheres fail to intersect at all.

The remedy is elegant: treat the receiver's clock error as a fourth unknown and solve for it along with the rest. With four unknowns (x, y, z, Δt), four equations are needed—that is, four satellites. This is also why the measured range, inaccurate because of the receiver clock, is not called a true range but a pseudorange.

A side effect of this design is that in completing a position fix a GNSS receiver also corrects its own clock to atomic-clock accuracy. Telecommunication base stations, power grids, and financial trading systems worldwide rely on GNSS for time synchronization—the importance of satellite navigation to modern society goes far beyond helping you find your way.

5.2 Triangulation, Trilateration, and Time-Difference Positioning

The geometric methods of radio positioning fall into three classes, and students must keep them apart—the three terms are often used interchangeably in the Chinese-language literature, yet their principles are entirely different.

1. Triangulation: Using Angles

Measure the bearing of the target relative to two known reference points; the intersection of the two rays is the target's position. Classical surveying, radio direction finding (RDF) in navigation at sea, and the Bluetooth AoA direction finding mentioned in Chapter 4 all belong to this class. Its characteristic is that it needs a directional antenna or an antenna array but no precise time measurement.

2. Trilateration: Using Distances

Measure the distances from the target to several known points and find the intersection of the spheres (or circles). GNSS and UWB TWR belong to this class. Its characteristic is that it needs accurate time or signal strength measurement but no knowledge of direction. Chapter 4 already presented the formulation for the two-dimensional case; GNSS is the three-dimensional version with the clock unknown added.

3. Time Difference of Arrival (TDoA): Using Time Differences

No absolute distance is measured, only the difference in the signal's arrival time at two base stations. The set of points with a fixed time difference forms, geometrically, a hyperbola (a hyperboloid in three dimensions), and the intersection of two or more such surfaces is the target's position—hence the alternative name hyperbolic positioning. The UWB TDoA architecture introduced in Chapter 4 and the LORAN long-wave navigation system of the Second World War era both use this method. Its characteristic is that the target end need only transmit, or only receive, which is

extremely power-efficient and gives large system capacity, at the price of requiring precise time synchronization among the base stations.

Method	Physical Quantity Measured	Geometric Locus	Requirements	Typical Application
Triangulation	Angle	Ray	A directional antenna	BLE AoA, radio direction finding
Trilateration	Absolute distance	Circle / sphere	Precise timing or known transmit power	GNSS, UWB TWR
TDoA	Distance difference	Hyperbola / hyperboloid	Precise synchronization among anchors	UWB TDoA, LORAN

Table 5-1 Comparison of Three Geometric Methods of Radio Positioning

Figure 5-1 Comparing the Loci of the Three Geometric Methods of Radio Positioning



This figure is the page of this section most worth remembering. The three methods differ not in technical detail but in what geometric shape a single measurement

confines the target to—a ray, a circle (sphere), or a hyperbola (hyperboloid). The essence of positioning is to intersect several such constraints.

It is worth noting that this idea—an observation confines the state to some manifold, and the intersection is then sought—is precisely the geometric intuition behind state estimation in Chapter 11 and graph optimization in Chapter 12. The scan matching of Chapter 13 and the epipolar geometry of Chapter 21 (where one correspondence confines the camera motion to an epipolar line) are, in essence, doing the same thing. If you build this intuition here, the parts that follow will come much more easily.

Dilution of Precision (DOP)

Whichever method is used, the geometric distribution of the reference points (satellites or base stations) directly affects accuracy. Imagine two satellites in almost the same direction in the sky: their spheres cross at a very shallow angle, and a small change in measurement error slides the intersection a long way along the tangent. Conversely, when satellites are spread evenly in all directions the crossing angles approach perpendicular and the error is confined to a small region.

This amplification is quantified by the dilution of precision (DOP): the actual positioning error is roughly the ranging error times the DOP value. Under open sky the DOP is often below 2, while in an urban canyon or indoors it may climb above 10—which is the quantitative explanation of the "deteriorating geometry" described in Chapter 4.

Figure 5-2 The Effect of Satellite Geometry on Positioning Accuracy (DOP)



Each satellite in the figure draws a semi-transparent band of ranging uncertainty—the width of the band is the ranging error, and its direction is perpendicular to the line of sight. In the left panel the satellites are spread in all directions and the three bands intersect in a small, nearly circular region; in the right panel they are crowded into one small patch of sky overhead, the bands run almost parallel, and the region of intersection is drawn out into a long, narrow slanted sliver. For the same ranging error, positional uncertainty in the right panel is several times greater.

This explains several things seen in practice: why GNSS accuracy in the vertical (height) is always two to three times worse than in the horizontal—there are no satellites below the ground, so the vertical geometry is inherently one-sided; why in an urban canyon "many satellites but terrible accuracy" occurs—number is not geometry; and why a patrol robot should use HDOP rather than the satellite count as its primary quality indicator.

The DOP Family and Its Thresholds

GDOP (geometric), PDOP (position), HDOP (horizontal), VDOP (vertical), TDOP (time)—robot navigation most often uses HDOP.

Rules of thumb in practice: HDOP < 2 is excellent, 2–5 usable, 5–10 marginal, and > 10 should be treated as untrustworthy.

These thresholds are used directly in the patrol system design of Section 5.7 and in the survey sheet of Exercise 5-1.

5.3 RTK-GNSS and High-Accuracy Outdoor Positioning

Ordinary phone GNSS is accurate to about 3 to 10 meters, which is enough to find your way but nowhere near enough for self-driving cars, automated agricultural machinery, or UAV surveying. Compressing the error to the centimeter level requires two key techniques: differential correction and carrier phase measurement.

1. The Idea of Differencing

A large part of GNSS error is common to a region: ionospheric and tropospheric delay, and small satellite clock and orbit deviations, are almost identical at two receivers several kilometers apart.

Differential GNSS (DGNSS) exploits this. A base station is set up at a position whose coordinates have been precisely surveyed; comparing the position it computes with the known true position lets it infer the present error, which it then broadcasts as a correction to nearby rovers. With the common error removed, the rover's accuracy improves from the meter level to the decimeter level.

2. Carrier Phase: The Key to the Leap in Accuracy

Ordinary GNSS uses the ranging code modulated onto the carrier, whose chip

length is on the order of hundreds of meters, so measurement resolution is naturally limited. The carrier itself, however, has a wavelength of only about 19 centimeters (the L1 band), and if its phase can be measured the resolution reaches a hundredth of a wavelength—the millimeter level.

The price is a new unknown: the receiver can measure the fractional part of a cycle precisely but does not know how many whole wavelengths the signal has traveled in all—this is the integer ambiguity problem. The core technique of an RTK system is to resolve that integer quickly and reliably (called ambiguity resolution, or obtaining a "fixed solution"). Once the fix succeeds, accuracy reaches the centimeter level; when it fails, only a "float solution" is maintained and accuracy falls back to the decimeter level.

3. The Architecture of an RTK System

Figure 5-3 The Architecture of an RTK-GNSS System

Base station: coordinates known, receives satellite signals → computes error corrections

↓ (transmitted by radio, 4G / 5G, or internet NTRIP)

Rover: receives satellite signals + the base station corrections

↓

Double differencing → common errors canceled → integer ambiguity resolved

↓

Fixed solution: centimeter positioning / Float solution: decimeter positioning

4. Network RTK and Practical Considerations

Building one's own base station is costly and limited in coverage, so network RTK developed: a national agency or a commercial operator builds a network of base stations and broadcasts correction data over the internet using the NTRIP protocol. In Taiwan, the National Land Surveying and Mapping Center of the Ministry of the Interior operates the e-GNSS system, which provides real-time positioning service across the country.

- Baseline length limit: the farther the rover is from the base station, the less the common-error assumption holds; a separation within a few tens of kilometers is generally recommended.
- Initialization time: reaching a fixed solution from power-on usually takes tens of seconds, and a re-fix is needed after loss of lock in a tunnel or under trees.
- Open sky is still required: RTK removes atmospheric and satellite errors and can do nothing about obstruction or multipath—RTK under trees or beside a tall building is just as unreliable. This point is crucial for robotic applications.

Worth mentioning is a new generation of techniques such as Galileo's High Accuracy Service (HAS), which can broadcast correction data directly by satellite and reach accuracy of about 20 centimeters with no ground base station—a rather practical compromise for wide-area outdoor robotics.

5.4 5G, LoRa, and Wi-Fi RTT in Robot Localization

A satellite signal comes from twenty thousand kilometers away and is inherently fragile. The signal from a terrestrial base station is far stronger—can it be used for positioning? This is a popular direction in the recent convergence of communication

and positioning.

1. 5G Positioning

5G included positioning in its design from the outset and has three advantages: millimeter waves and wide bandwidth give high time resolution, massive MIMO antenna arrays can measure angle of arrival, and the dense deployment of small cells shortens distances. 5G positioning combines time difference of arrival, angle of arrival, and round-trip time, and under ideal conditions reaches the meter or even sub-meter level. For factory AGVs and campus self-driving vehicles this means that communication and positioning share one infrastructure, which is highly attractive. The limitations are that accuracy depends strongly on cell density and that the carrier must expose the relevant capabilities.

2. LoRa Positioning

LoRa is a long-range, low-power IoT communication technology in which a single gateway can cover several kilometers. Its positioning capability comes from TDoA—several gateways record the difference in arrival time of the same packet. Accuracy is only on the order of tens to hundreds of meters, far short of the technologies above, but it wins on coverage, extremely low power, and low cost. Its place is wide-area coarse positioning: tracking farm equipment, monitoring assets in mountainous terrain, or the hybrid architecture in the Chapter 4 case study of "precise UWB ranging plus long-range LoRa backhaul."

3. Wi-Fi RTT (Round-Trip Time)

The Fine Timing Measurement (FTM) mechanism introduced in IEEE 802.11mc lets a device exchange time stamps with an AP and measure the round-trip time directly to obtain distance—fundamentally different from guessing distance from RSSI in Chapter

4. Accuracy reaches 1 to 2 meters and no fingerprint database is required. The limitation is that both AP and device must support FTM, and adoption remains the main bottleneck today.

Technology	Coverage	Typical Accuracy	Infrastructure Required	Role in Robotic Applications
GNSS	Global	3–10 meters	None (the satellites are already up there)	The outdoor mainstay
RTK-GNSS	Within tens of km of a base station	Centimeter level	A base station or a network service	Agricultural machinery, surveying, self-driving
5G	The coverage of the base stations	Meter to sub-meter	Densely deployed base stations	Campuses, factories
LoRa	Several kilometers	Tens to hundreds of meters	Gateways	Wide-area coarse tracking
Wi-Fi RTT	Tens of meters indoors	1–2 meters	APs supporting FTM	Indoor assistance

Table 5-2 Comparison of Radio Positioning Technologies for Robotic Applications

5.5 Error Sources in Radio Positioning: Multipath, Obstruction, and Interference

Only by understanding the errors can one judge when GNSS may be trusted and when a switch to SLAM is required. GNSS errors fall into three classes.

1. Propagation Path Errors (Removable by Differencing)

- Ionospheric delay: the ionosphere, roughly 50 to 1,000 kilometers above the surface, contains a great many free electrons and delays the signal. This is the

largest error source for a single-frequency receiver, reaching several meters and varying sharply with solar activity. A dual-frequency receiver can remove it by exploiting the difference in delay between frequencies—which is why dual-frequency GNSS is more expensive but far more accurate.

- Tropospheric delay: the delay caused by water vapor and pressure in the lower atmosphere, amounting to tens of centimeters up to a few meters, correctable by model estimation.
- Satellite clock and orbit errors: continuously corrected by the control segment, with residual error below about a meter.

2. Local Environmental Errors (Not Removable by Differencing)

- Multipath: the signal arrives after reflecting off buildings, the ground, or water, and the longer path makes the range too large. This is the principal error source in urban and campus settings and can reach several meters or more. Mitigation includes choke-ring antennas and signal quality screening algorithms, but it cannot be eliminated.
- Obstruction: buildings, tree canopies, and tunnels block the signal outright, leaving too few usable satellites or a degraded geometry. The royal palm and banyan canopies common on Taiwanese campuses are enough to drop RTK from a fixed solution back to a float solution.
- The urban canyon effect: with tall buildings on both sides, only a narrow strip of sky remains, the visible satellites cluster in one direction and the DOP degrades, while the wall surfaces create severe multipath—obstruction and multipath striking together. This is the most troublesome localization problem for a self-driving car in a city center.

3. Deliberate Interference

- Jamming: a strong signal swamps the faint satellite signal and makes the receiver lose lock. A small illegal jammer available on the market can disable GNSS within several tens of meters around it.
- Spoofing: counterfeit satellite signals are transmitted so that the receiver computes a wrong but normal-looking position. This is more dangerous than jamming—the system raises no error and simply guides you quietly to the wrong place. For UAVs and self-driving cars this is a serious security issue.

The existence of these three classes of error is the fundamental reason a robot cannot rely on GNSS alone. A robot patrolling under the shade of campus trees may at any moment meet a signal dropout, a jump of several meters, or a loss of the RTK fix—and if the navigation system depends entirely on GNSS, the robot loses its bearings at those moments. The remedy is the fusion of the next section.

5.6 Fusing Radio Navigation with the IMU and SLAM

In Chapter 4 we saw how indoor positioning and SLAM complement each other; the GNSS with IMU and SLAM fusion of this section is the same idea unfolded outdoors, and one of the most mature applications of sensor fusion today.

GNSS and the IMU: A Match Made in Heaven

Their error characteristics are exactly opposite:

- IMU: a very high update rate (hundreds of Hz), extremely accurate over short intervals, and completely immune to obstruction and interference; but integration drift makes the error accumulate rapidly with time (treated in detail in Chapter 7).

- GNSS: a low update rate (1 to 10 Hz) and noisy single measurements; but the error is bounded and does not drift over time.

The logic of fusion is therefore quite natural: let the IMU supply a continuous high-rate trajectory, and let GNSS periodically pull the accumulated drift back. This combination is called GNSS / INS integrated navigation and is standard equipment in aviation, shipping, missiles, and self-driving cars. It is usually implemented with a Kalman filter (Chapter 11), which continuously estimates and compensates the IMU bias, so that even when GNSS is briefly lost (in a tunnel or under trees) the IMU maintains usable accuracy for seconds to tens of seconds—an interval known as the dead-reckoning capability of inertial navigation.

Adding SLAM: Triple Insurance

GNSS / INS still fails in a long tunnel or indoors, and SLAM must then take over. The complete triple architecture is as follows:

Figure 5-4 The Triple GNSS / INS / SLAM Fusion Architecture

IMU (hundreds of Hz): supplies high-rate continuous motion estimation, accurate in the short term

+

GNSS (1–10 Hz): supplies absolute position outdoors, eliminating long-term drift

+

SLAM (10–30 Hz): supplies relative constraints from environmental features; the mainstay when GNSS fails

↓

Fusion filter / graph optimization back end (Chapters 11 and 12)

↓

A continuous, drift-free pose estimate available in all conditions

A Key Engineering Problem: Unifying the Coordinate Systems

Before fusion, an easily overlooked problem must be solved: GNSS gives latitude and longitude (WGS84 geographic coordinates), while SLAM uses local rectangular coordinates with the starting point as origin. The two must first be brought into one frame—the usual approach is to convert latitude and longitude into east-north-up (ENU) local rectangular coordinates with the first GNSS fix as the origin. Time synchronization (aligning the time stamps of all sensors) and extrinsic calibration (the relative mounting positions of GNSS antenna, IMU, and LiDAR) must also be handled. These unremarkable engineering details are often what decides whether an implementation succeeds—the coordinate transforms and TF tree of Chapter 10 treat

this subject in full.

Switching Trust

A practical system also needs an arbiter to decide which source to believe at a given moment. Common criteria include the number of usable satellites, the DOP value, the RTK solution status (fixed, float, or single-point), and whether the discrepancy between the GNSS and SLAM estimates has grown too large. As the robot moves from outdoors to indoors, the system should shift weight smoothly from GNSS to SLAM rather than jumping—which is the central challenge of the campus scenario in the next section.

5.7 Indoor–Outdoor Hybrid Navigation for a Campus Patrol Robot

Putting together the techniques of this and the previous chapter, we can design the complete navigation system of a campus patrol robot. What makes this scenario a classic is precisely that it covers every difficult situation.

Site Analysis: Four Environments on One Patrol

- The open sports field and parking lot: the sky is open, GNSS and RTK perform at their best, and they can serve as the mainstay of localization.
- Tree-shaded walkways and the sides of buildings: obstruction and multipath occur together, and GNSS may jump by several meters or lose its fix. Here SLAM must be primary and GNSS secondary, and the system must be able to judge that GNSS is not to be trusted.
- Breezeways and arcades: semi-indoor, with GNSS coming and going; the transition zone where errors are most likely.

- Indoor corridors and classrooms: GNSS fails completely, so LiDAR or visual SLAM alone carries the load, with the AprilTags or BLE beacons of Chapter 4 supplying floor information and absolute correction.

System Design Strategy

1. Layered maps: outdoors, a global map referenced to GNSS coordinates; indoors, a local SLAM map for each building; the two joined at the doorway as an anchor, forming a topological network of maps (where the topological maps of Chapter 3 come into their own).
2. Sensor configuration: an RTK-GNSS antenna (the outdoor mainstay) + an IMU (the backbone of continuity) + a 3D or 2D LiDAR (the SLAM mainstay) + a camera (visual markers and semantic recognition).
3. Mode-switching mechanism: decide jointly from the GNSS quality indicators (satellite count, DOP, solution status) and the SLAM matching quality, adjusting weights progressively in the transition zone rather than switching hard.
4. Reinforcing the transition zones: post an AprilTag at the entrance of each building as a bridge between the outdoor and indoor coordinate frames—the moment the robot sees the marker it knows its position in both frames at once, and the handover is seamless.
5. Failure fallback: if every means of localization goes wrong at once (strong interference together with an obstructed LiDAR, say), the robot should stop and report rather than drive on with a wrong pose—a basic principle of safety design.

This architecture is actually realized in the capstone project of Chapter 31. For now you need only grasp one central idea: navigation for a real-world robot is never a matter of choosing the single best technology but of letting several technologies run a

relay, each on the leg it runs best. That is exactly the purpose of introducing the six major sensors one at a time in Part II of this book—you must know the strengths and weaknesses of every team member to be a good coach.

Formulations and Procedures

1. The Pseudorange Equation

Let the receiver be at (x, y, z) with clock bias Δt, let the known position of the i-th satellite be (x_i, y_i, z_i), and let the measured pseudorange be ρ_i. Then:

$$\rho_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} + c \cdot \Delta t$$

where c is the speed of light. There are four unknowns—x, y, z, and Δt—so at least four satellites are needed to form four equations. Because the equations are nonlinear, in practice they are linearized by Taylor expansion and solved iteratively, or handled by least squares when there are redundant measurements from more than four satellites—with 10 satellites visible there are 10 equations for 4 unknowns, and the surplus information serves precisely to suppress noise.

2. Positioning Error and DOP

Positioning error can be approximately decomposed into the product of two factors:

$$\textit{positioning error} \approx \textit{UERE} \times \textit{DOP}$$

UERE (user equivalent range error) is the equivalent ranging error of a single satellite (including ionosphere, troposphere, multipath, and so on), while DOP is the geometric amplification factor. This compact relation points to two routes of improvement: reduce the ranging error (dual frequency, differential correction) or

improve the geometry (more constellations, a more open location). You can observe this directly with a GNSS viewer app on a phone: step beside a tall building and the satellite count falls, the DOP rises, and the error circle immediately grows.

3. The Carrier Phase Observation Equation

The carrier phase observable φ (in cycles) relates to the true range r as:

$$\lambda \cdot \varphi = r + c \cdot \Delta t + \lambda \cdot N + \text{error terms}$$

where λ is the carrier wavelength (about 19 centimeters for L1) and N is the integer ambiguity—an unknown integer. The whole essence of RTK lies in using double differencing between base station and rover to cancel most of the error terms and then resolving N as an integer quickly and reliably. Once N is fixed, λ · φ becomes a range measurement accurate to the millimeter, and that is the source of centimeter-level positioning.

4. A Procedure for Judging the Trustworthiness of a GNSS Fix

Figure 5-5 A Decision Procedure by Which a Robot Judges Whether to Trust GNSS

① Are there ≥ 6 visible satellites? No → do not trust; switch to SLAM as the lead

↓ Yes

② Is HDOP < 2.0? No → reduce the weight

↓ Yes

③ Is the RTK status a fixed solution? Float → medium weight; single-point → low weight

↓ Fixed

④ Is the discrepancy from the IMU / SLAM estimate below threshold? No → judge it a multipath jump and reject this measurement

↓ Yes

⑤ Accept as a high-weight absolute position measurement and correct the accumulated drift

5.8 The Contemporary Upheaval in Satellite Navigation: Stronger Signals and Stronger Interference

The GNSS architecture described in the first seven sections has been fairly stable since it took shape in the 1990s. Between 2025 and 2026, however, two opposing forces appeared in this field at once—one making positioning more accurate and more robust, the other making it less trustworthy. Both bear directly on robotic system design, and students should know them.

1. High Accuracy for Free: The Galileo High Accuracy Service

The RTK of Section 5.3 requires a base station and a communication link, which is the greatest obstacle to its spread. The European Union's Galileo High Accuracy Service (HAS) offers another route: precise ephemerides and clock corrections are broadcast free of charge in the satellite signal itself, so a receiver needs neither a network connection nor a subscription to reach about twenty centimeters of horizontal accuracy by precise point positioning (PPP).

This is good news for robotics teaching—it moves centimeter-class positioning a long way from professional equipment requiring a base station toward consumer-grade hardware. The price is that PPP takes longer to converge (usually several minutes) and does not fix instantly as RTK does, so it suits patrol or agricultural machinery running for long stretches rather than stop-and-go settings.

2. Low Earth Orbit Navigation Constellations: A Generational Gap in Signal Strength

A more fundamental change comes from low Earth orbit positioning, navigation, and timing (LEO-PNT). Traditional GNSS satellites sit in medium orbit at twenty thousand kilometers, and their signal reaches the ground at only about −130 dBm—weaker, as Section 5.1 noted, than the ambient noise. A LEO satellite is only about five hundred kilometers up, forty times closer, and the received power is accordingly far greater.

Take one representative commercial system: its planned maximum received power is about −136 dBW, which the operator claims to be a hundred times the strength of the traditional GPS L1 C/A signal; after an in-orbit software update the ranging accuracy of a single satellite improved from 4.2 centimeters to about 1.5 centimeters, and once the full constellation is deployed the target under open sky is about two centimeters

horizontally and four centimeters vertically. The constellation is planned at two hundred fifty-eight satellites, more than ten times the twenty-four-satellite baseline constellation of GPS; the first production satellites are expected to launch in October 2026, with service beginning in 2027.

For robots the significance of signal strength is not only greater accuracy but reception in more places—under tree shade, at the edge of a building, indoors near a window: the settings listed as failure zones in Section 5.5 may all become localizable again.

3. The Other Force: Jamming and Spoofing Are Becoming Routine

In the same period, however, the scale of deliberate interference with GNSS has far exceeded anything in the past. Observations by the experimental LEO satellite just described, from an altitude of five hundred kilometers, revealed widespread signal interference from Europe through the Middle East—from France in the west to the Pakistani border in the east; in the worst-affected regions GPS signal strength fell from a normal level of about forty decibels to only about ten. As the research team put it: "We expected to see some interference, but far more than we imagined."

This turns the "deliberate interference" of Section 5.5 from a short paragraph in a textbook into a routine condition that system design must confront head-on. Jamming leaves the receiver unable to acquire a signal, so the robot at least knows it has gone blind; spoofing is more dangerous—it sends a plausible false signal and lets the robot go confidently to the wrong place.

It is worth noting that the strong signal of a LEO constellation is itself a countermeasure: measurements indicate that its stronger signal can shrink the area a jammer is able to disable by as much as ninety-five percent. The operator also concedes, however, that a

nearby or high-power jammer may still succeed under unfavorable conditions—no single system is immune.

4. The Terrestrial Network Fills In: 5G Positioning

The 5G positioning introduced in Section 5.4 is also advancing. From Release 17 onward the 3rd Generation Partnership Project (3GPP) brought high-accuracy positioning formally into the specification, and Release 18 further strengthened carrier phase positioning and sidelink positioning—the latter letting devices range against each other directly without routing everything through a base station. This is a development worth noting for the multi-robot collaboration of Chapter 27: a group of robots can range against one another and form a local relative-positioning network.

5. The Conclusion for a Robotics Engineer

Taking these four developments together yields a clear conclusion: GNSS is becoming both stronger and more fragile at once. Stronger, because signal power, correction services, and multi-system multi-band reception are all improving; more fragile, because the electromagnetic environment it inhabits is no longer simple.

And the engineering response is exactly what Section 5.6 of this chapter has already set out: never let any single sensor bear the burden of localization alone. However good GNSS becomes, a robot should be able to detect when it cannot be trusted and at that moment switch gracefully to IMU dead reckoning and SLAM.

A Practical Criterion: Three Lines of Defense Against Spoofing

1. Plausibility check: does the position jump instantaneously? Does the speed exceed the robot's physical limit? (The IMU of Chapter 7 provides exactly the

independent motion reference needed.)

2. Consistency check: does the discrepancy between the GNSS position and the SLAM estimate persistently exceed a threshold? (This is the heart of the decision procedure of Figure 5-5.)

3. Signal-level check: is the carrier-to-noise ratio (C/N_0) anomalously uniform? Real satellites at differing elevations should differ noticeably, while a spoofed signal is often too consistent.

Case Studies

Case 5-1 Self-Driving Farm Machinery: RTK's Most Successful Commercial Application

In the years when self-driving cars were still being tested, RTK-guided tractors were already in large-scale commercial operation in farm fields worldwide. Why agriculture became the perfect stage for RTK is well worth considering: fields are open and unobstructed, so satellite signal quality is excellent; the working path consists of regular straight passes back and forth and needs no complex decision-making; and yet accuracy requirements are very high—a few centimeters of error in row spacing causes overlapping or missed sowing and accumulates into considerable losses in yield and cost.

RTK lets a tractor drive its planned route automatically to centimeter accuracy, with the operator intervening only to turn at the headland (newer systems automate the turns as well). This case shows an important engineering judgment: when the environment is extremely favorable to a given technology, use that technology to the full rather than bolting on sensors you do not need for the sake of technical

completeness. Conversely, drive the same tractor under an orchard canopy and RTK fails at once—only then is LiDAR or visual SLAM needed to take over. Technology selection is always decided by the site.

Case 5-2 Self-Driving Car Localization in an Urban Canyon

A self-driving car in a city center faces the harshest test of GNSS: tall buildings on both sides cut the sky into a narrow slit, the visible satellites drop from more than ten to three or four clustered in one direction, and the DOP degrades several-fold; worse still are the mirror reflections of glass-curtain towers—the receiver may be receiving a signal bounced back from the building opposite, the computed position may be off by tens of meters, and the system usually does not know it is wrong.

The mainstream solution in self-driving cars is to reduce dependence on GNSS drastically: a pre-surveyed high-definition map serves as the reference, the onboard LiDAR and cameras localize by matching map features in real time (lane lines, curbs, sign poles, building outlines), and GNSS serves only for initialization and coarse regional determination. This is exactly where the commercial value of the 3D LiDAR mapping techniques of Chapters 17 and 18 lies—when the signal overhead is unreliable, the structure of the ground around you is the most faithful reference.

Case 5-3 UAV Flight in GPS-Denied Conditions

A consumer UAV can hover stably on GNSS in the open, but as soon as it flies beneath a bridge, into an indoor plant, or into a forest it enters what is called a GNSS-denied environment. Early UAVs would drift out of control in such conditions, or even trigger a return-to-home that flew them into an obstacle.

The modern solution is visual-inertial odometry (VIO): a downward- or forward-

facing camera tracks texture features on the ground or in the surroundings, motion is estimated together with the IMU, and no external signal is relied upon at all (treated in detail in Chapter 26). This also explains why research on UAV visual localization is concentrated at ETH and UZH (Chapter 2)—their central question is precisely how a robot can fly on its own eyes when satellites cannot help. For the reader of this book this is a clear thread: this chapter is about using the signal well when there is one, and Parts VI and VII are about what to do when there is none.

Hands-On Activities

Exercise 5-1 A GNSS Signal Quality Survey of the Campus

Purpose: to measure satellite signal quality at various points on campus with a phone, verify first-hand the effect of obstruction, multipath, and DOP on positioning accuracy, and produce a GNSS trustworthiness map to serve as the basis for route planning in the patrol robot project of Chapter 31.

Equipment

- A smartphone with an app that displays satellite information (search for tools of the "GNSS viewer" or "GPS Test" kind; it must show the satellite count, signal strength, and DOP or an accuracy estimate).
- A campus floor plan, a recording sheet, and a tape measure (optional).

Procedure

1. Choosing points: select at least 8 representative measurement points across the campus, covering four environments—an open sports field, a tree-shaded walkway, the north wall of a building, a breezeway or arcade, and an indoor corridor, with at least one point each.

2. Measuring: hold the phone still for 60 seconds at each point, then record the total number of visible satellites, the number actually used in the fix, the distribution of signal strengths, the HDOP value (or the accuracy estimate the app shows), and the latitude and longitude displayed.
3. Repeatability test: measure again at the same point after an interval of 5 minutes and compare the difference in latitude and longitude between the two—that difference is the repeatability error of the fix.
4. Drawing the trustworthiness map: classify each point into three grades by satellite count and DOP (green: trustworthy, GNSS may lead; yellow: downweight, with SLAM assisting; red: not trustworthy, switch to SLAM) and mark them on the campus plan.
5. Observing multipath: stand for 2 minutes beside a glass-curtain tower and for 2 minutes in the open, compare how far the computed position jumps, and record whether the distribution of satellite signal strengths looks anomalous in each case.
6. Summary report: using the trustworthiness map, plan a patrol route for a campus patrol robot, indicating which localization method should lead on each stretch and where AprilTag correction points should be placed.

Discussion Questions

1. At which point was the repeatability error largest? What were the satellite count and DOP value there? Which class of error source from Section 5.5 of this chapter does it correspond to?
2. Which has the greater effect on positioning, shade from trees or the side of a building? What do you think is the reason (hint: foliage is partly transmissive, while a wall blocks completely but reflects)?

3. If this patrol robot carried GNSS alone and no SLAM, on which stretches of your planned route would it run into trouble, and with what consequences?
4. (Undergraduate) Following the decision procedure of Figure 5-5, set the thresholds for your own campus (satellite count, HDOP, positional discrepancy) and justify each choice.

Extended Challenge (Undergraduate)

Write a program in Python that parses the NMEA sentences (GGA, GSA, GSV) output by a phone or a USB GNSS receiver, records satellite count, HDOP, and the fix quality indicator automatically, and plots time series and trajectory. Then try computing the standard deviation of the position scatter at a static point to quantify the difference in accuracy among environments. This program can be carried over directly into the patrol project of Chapter 31.

Further Reading

1. P. Misra and P. Enge, Global Positioning System: Signals, Measurements, and Performance, 2nd ed., Ganga-Jamuna Press, 2011. (The standard textbook on GNSS principles.)
2. B. Hofmann-Wellenhof, H. Lichtenegger, and E. Wasle, GNSS – Global Navigation Satellite Systems, Springer, 2008. (A complete reference covering the four major systems and RTK.)
3. P. D. Groves, Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, 2nd ed., Artech House, 2013. (The authoritative work on GNSS / INS integrated navigation; the principal reference for Section 5.6.)

4. The e-GNSS real-time kinematic positioning system website of the National Land Surveying and Mapping Center, Ministry of the Interior. (Official information and usage instructions for Taiwan's network RTK service.)
5. The RTKLIB open-source project (rtklib.com). (Free GNSS and RTK processing software, usable for teaching experiments and post-processing analysis.)
6. European Union Agency for the Space Programme, the official Galileo High Accuracy Service (HAS) documentation. (The new generation of satellite-broadcast correction service.)
7. 3GPP TS 38.305, NG-RAN Stage 2 Functional Specification of User Equipment Positioning. (The standard specification for 5G positioning.)

From Satellites to Sensors: Where This Chapter Stands

Chapter 4 and this chapter together answer one question: how does a robot know where in the world it is? The answer is that if the environment allows it—infrastructure installed indoors, open sky outdoors—let external infrastructure tell it. That is the least troublesome approach and the one least prone to drift.

But these two chapters also mark out the boundary. Where there is no installed infrastructure indoors and no sky outdoors—tunnels, underground garages, the depths of a warehouse, forests, buildings after a disaster, the surface of the Moon—a robot must rely entirely on itself. Chapters 6 through 9 introduce the internal senses available to it one at a time: the magnetic compass, the inertial measurement unit, LiDAR, and the camera. None requires any external installation, and all share one fatal weakness: they drift.

And the only way to eliminate drift is for the robot to recognize that it has returned to

a place it has been. That is loop closure detection, and it is the reason SLAM is possible at all. The mathematics of Part III and the algorithms from Part IV onward all prepare the way for it. The central judgment this chapter leaves with the student is this: when GNSS is available, use it; when it is not, know that it is not—the latter is the truly difficult and truly important engineering capability.

Summary

1. GNSS comprises three segments—space, control, and user; the receiver only receives and never transmits, performs three-dimensional trilateration with pseudoranges, and needs at least four satellites because the clock bias must be solved for at the same time. As of 2026 the four major systems had about 130 satellites in orbit in all.
2. Radio positioning has three geometric methods: triangulation uses angles, trilateration uses distances, and TDoA uses distance differences; the geometric distribution of the reference points is quantified by DOP, and the positioning error is roughly the ranging error times the DOP.
3. RTK combines differential correction with carrier phase measurement to achieve centimeter-level positioning, the key being resolution of the integer ambiguity; a fixed solution is centimeter-level and a float solution decimeter-level, and neither can do anything about obstruction and multipath.
4. Each terrestrial radio positioning technology has its place: 5G suits campuses and factories, LoRa suits wide-area coarse tracking, and Wi-Fi RTT can provide meter-level indoor ranging—but all are limited by how widely the infrastructure has been adopted.

5. GNSS errors divide into propagation errors removable by differencing (ionosphere, troposphere, satellite clock and orbit), local errors that cannot be removed (multipath, obstruction, urban canyons), and deliberate interference (jamming and spoofing).
6. GNSS and the IMU have complementary error characteristics and together form a mature integrated navigation system; adding SLAM yields a triple architecture that maintains continuous, drift-free localization outdoors, in transition zones, and indoors alike.
7. A campus patrol robot must adopt a different primary means of localization in each zone of the environment and use visual markers as the bridge between the indoor and outdoor coordinate frames; the central idea of the system design is to let several technologies run a relay, each on the leg it runs best.

In the next chapter we take up one of the oldest means of navigation still in service—the Earth's magnetic field. From the mariner's compass to the phone's compass app, from correcting ferromagnetic interference to indoor geomagnetic fingerprint positioning, we will see how the planet's own field supplies a robot with an absolute directional reference.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 6

Geomagnetic Navigation and the Electronic Compass

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapters 4 and 5 answered the question "where am I"; from this chapter Part II turns to another question, equally basic and more easily overlooked: "which way am I facing?"

Position and orientation are two different things. Even if a robot knows it is standing in the middle of a corridor, it still cannot move a step without knowing which way it faces—it can neither translate "go forward three meters" into the right action nor correctly mark on the map a sensor reading of "a wall two meters ahead." Chapter 10 formalizes this as pose: position plus orientation, six degrees of freedom of which none may be missing.

The magnetometer of this chapter is the only sensor on a robot that supplies an absolute heading—it needs no map, needs no installation, and does not drift. The price is that it is extremely easy to disturb, and the source of the disturbance is often the robot itself. This character of being uniquely absolute yet easily contaminated gives it a very particular role in sensor fusion: it contributes little most of the time but is irreplaceable at the critical moment.

Figure 6-0 How Chapter 6 Connects to the Rest of the Book

[Building on the previous chapters]

Chapter 4 Wi-Fi fingerprint positioning → Section 6.3 of this chapter: the same method with magnetic field strength substituted

Chapter 5 GNSS gives position but cannot give heading at rest → this chapter supplies that piece

[This chapter is responsible for]

The three geomagnetic parameters (F, D, I) → tilt compensation → magnetic heading

→ true heading

Hard-iron and soft-iron calibration: correcting a deformed point cloud back to an ideal sphere

Turning "interference" into a "fingerprint": indoor geomagnetic positioning

[Where it leads]

→ Chapter 7 The IMU: the gyroscope is accurate in the short term and the magnetometer in the long term, and together they form an AHRS

→ Chapter 10 Heading angle becomes formally part of the pose

→ Chapter 14 Geomagnetic fingerprint matching ≈ the observation model of a particle filter

→ Chapter 26 The consequences of a failed UAV compass calibration (Case 6-1)

→ Chapter 29 Mounting location and interference troubleshooting for the IMU and magnetometer on a physical vehicle

Learning Objectives

The compass is one of humanity's oldest navigational instruments, and yet it remains active in every phone and every UAV. This chapter examines how the Earth's magnetic field supplies a robot with an absolute directional reference, and why indoors it turns from a clean direction sensor into a distinctive positional fingerprint. After completing this chapter, you should be able to:

1. Explain the origin of the Earth's magnetic field and the three parameters that describe it: declination, inclination, and field intensity.
2. Explain how a magnetometer measures the field, and compute a heading angle from a three-axis magnetometer together with tilt compensation.

3. Explain the principle of indoor geomagnetic fingerprint positioning, and account for why "interference" becomes instead a source of positional information.
4. Distinguish the causes of hard-iron and soft-iron error, and carry out practical calibration procedures such as the figure-eight calibration.
5. Describe the role geomagnetic sensing plays in phones, UAVs, and quadruped robots, and its known limitations.
6. Analyze the error characteristics of geomagnetic sensing, the IMU, and visual SLAM, and explain the logic of fusing them complementarily.

Core Concepts

Chinese Term	English Term	Explanation
地磁場	Geomagnetic Field	The field produced by convection of liquid metal in the Earth's outer core, with an intensity of about 25 to 65 microtesla.
磁偏角	Declination	The horizontal angle between magnetic north and true north, varying with place and time.
磁傾角	Inclination / Dip	The angle between the field lines and the horizontal plane; near 0 degrees at the equator and near 90 at the poles.
磁力計	Magnetometer	A sensor measuring field strength and direction, mostly by the Hall effect or magnetoresistance.
電子羅盤	Electronic Compass	A module combining a magnetometer and an accelerometer to output a heading angle.
航向角	Heading / Yaw	The angle between the robot's direction of travel and a reference north.

Chinese Term	English Term	Explanation
傾斜補償	Tilt Compensation	Using the attitude measured by the accelerometer to project the magnetometer readings back into the horizontal plane.
硬鐵誤差	Hard-iron Error	A fixed offset caused by permanently magnetized material on the platform, displacing the center of the measurement sphere from the origin.
軟鐵誤差	Soft-iron Error	Ferromagnetic material distorts the external field, deforming the measurement sphere into an ellipsoid.
地磁指紋	Magnetic Fingerprint	The local field distortion caused by a building's steel structure, usable as a positional signature.
世界地磁模型	WMM	The global field model maintained jointly by NOAA and BGS, updated every five years.
磁暴	Geomagnetic Storm	A violent field disturbance driven by solar activity, which temporarily degrades compass accuracy.

6.1 The Earth's Magnetic Field and the Magnetometer

The Earth is an enormous magnet. The field comes not from some solid lodestone but from the convective motion of hot liquid iron and nickel in the planet's outer core—the searing conductive fluid forms circulating currents under the combined action of rotation and thermal convection and, like a giant dynamo, continuously generates a magnetic field. This mechanism is called the geodynamo.

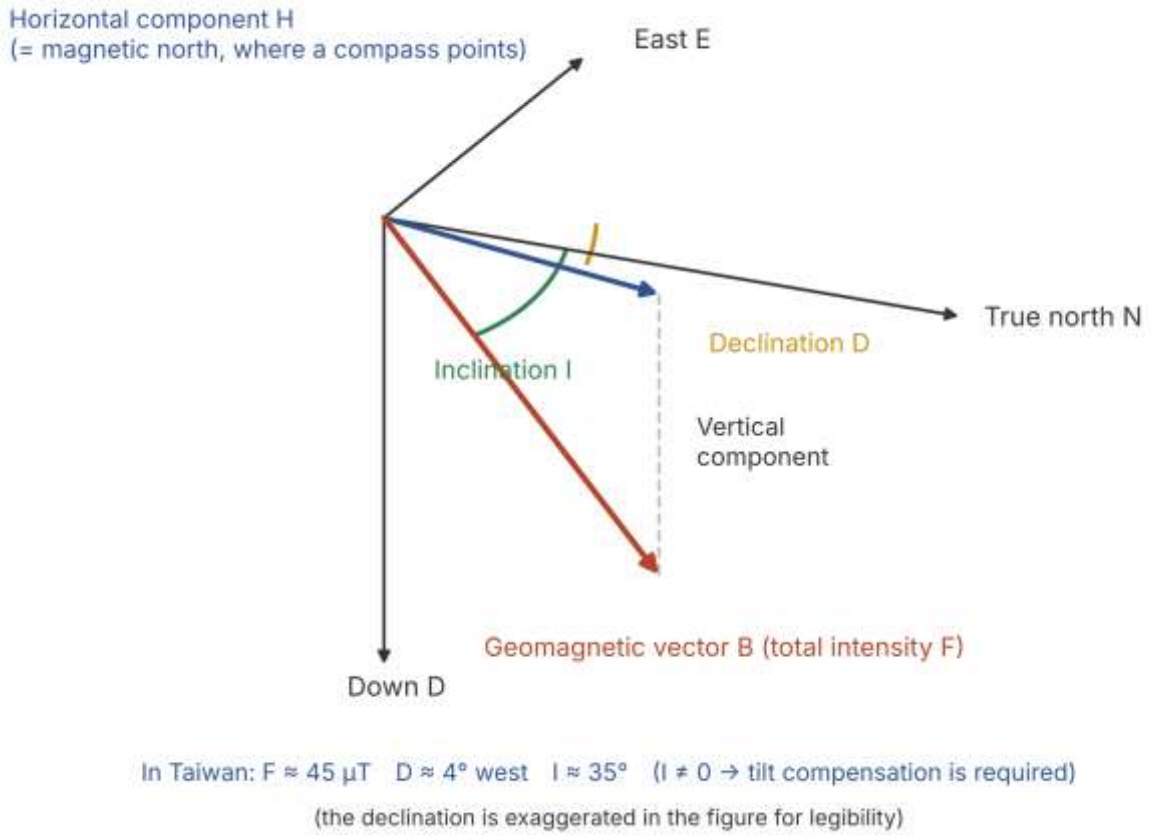
The Three Parameters That Describe the Geomagnetic Field

At any point on the surface the geomagnetic field is a three-dimensional vector, usually described by three parameters:

- Total intensity: between about 25 and 65 microtesla (μT), weaker near the equator and stronger at high latitudes. In Taiwan it is around 45 μT. By comparison, the surface field of a refrigerator magnet can reach several thousand microtesla—the geomagnetic field is in fact extremely weak, which is exactly why it is so easily disturbed.
- Declination: the horizontal angle between magnetic north and true north. Maps and navigation software use true north while a compass points to magnetic north, and the difference is the declination. In Taiwan the declination is about 4 degrees west (varying slightly with place and year), meaning the "north" a compass indicates lies about 4 degrees west of true north.
- Inclination: the angle between the field lines and the horizontal plane. Near the magnetic equator it approaches 0 degrees (field lines parallel to the ground) and near the magnetic poles 90 degrees (field lines entering the ground vertically). In Taiwan it is about 35 degrees—a parameter that matters greatly to a robot, because it means the geomagnetic vector has a substantial vertical

component, and tilt compensation is required before the heading can be computed correctly.

Figure 6-1 The Geomagnetic Vector and Its Three Descriptive Parameters



This figure lays out the geometric relation among the three parameters at a stroke. The geomagnetic vector B points from the origin obliquely downward (in the northern hemisphere), and its projection H onto the horizontal plane is the magnetic north a compass indicates; the horizontal angle between H and true north is the declination D, and the angle between B and the horizontal plane is the inclination I.

Note in particular that the inclination in Taiwan is about 35 degrees—which means the geomagnetic vector has a substantial vertical component. Tilt the robot even slightly and that vertical component "leaks" into the readings of the horizontal axes and the heading comes out wrong. This is exactly why the tilt compensation of Section 6.2 exists, and why an electronic compass can never work without an accompanying

accelerometer (Chapter 7). Near the magnetic equator, where I approaches zero, tilt compensation matters far less; at high latitudes, where I approaches 90 degrees, the horizontal component H becomes too small and the compass all but fails—the magnetic blackout zone mentioned in Section 6.1.

The Field Drifts: The World Magnetic Model

Because the motion of the outer-core fluid is not steady, the geomagnetic field changes continuously and slowly, and the north magnetic pole in particular keeps moving—over recent decades it has drifted from northern Canada toward Siberia, and noticeably faster since the late 1990s. Precise navigation therefore requires a continuously updated model.

The World Magnetic Model (WMM), maintained jointly by the U.S. NOAA National Centers for Environmental Information (NCEI) and the British Geological Survey (BGS), is that standard. It is updated every five years; the current edition, WMM2025, was released in December 2024 and is valid through the end of 2029, and a high-resolution edition, WMMHR2025, is published alongside it. The model is used by NATO, the U.S. Department of Defense, aviation and marine systems, and the compass app on every phone—the "true north" shown on your phone is this model performing the declination correction behind the scenes.

The model also marks the magnetic blackout zones near the poles, where the horizontal field intensity is too low for compass navigation to be reliable. In addition, magnetic storms driven by solar activity make the declination depart temporarily from the model prediction—an effect especially pronounced for aviation operations at high latitudes.

How a Magnetometer Measures

- The Hall effect: when current flows through a conductor subjected to a perpendicular magnetic field, the carriers deflect to one side and a transverse voltage appears whose magnitude is proportional to the field strength. The principle is intuitive and the cost low, but sensitivity is relatively poor.
- Magnetoresistance: the resistance of certain materials changes with the magnetic field. Anisotropic magnetoresistance (AMR) and tunneling magnetoresistance (TMR) offer high sensitivity and low power consumption and are the schemes commonly used in phones and robots today.

A modern MEMS magnetometer integrates three-axis sensing elements in a chip a few millimeters square, often packaged together with a three-axis accelerometer and a three-axis gyroscope to form what is called a nine-axis IMU module. Note that the raw output of a magnetometer is the three field components in the sensor frame, which is still a considerable amount of processing away from a heading angle—the subject of the next section.

6.2 The Electronic Compass and the Heading Angle

The needle of a traditional compass is a freely pivoting magnetized bar, held level by gravity and pointed to magnetic north by magnetic force. An electronic compass has no moving parts and must accomplish the same thing with mathematics—and a pitfall many beginners fall into is hidden here.

The Simplest Case: Perfectly Level

If the magnetometer happens to be held level, the heading needs only the two horizontal components:

$$\psi = atan2(-m_Y, m_x)$$

where m_x and m_Y are the two horizontal-axis readings of the magnetometer and ψ is the magnetic heading. atan2 is used rather than atan so that all four quadrants are resolved correctly.

The Real Problem: The Robot Tilts

The trouble is that a robot can hardly stay level forever—a vehicle crossing a slope, a UAV pitching forward to accelerate, a quadruped climbing stairs: the sensor tilts with the platform. Because the inclination in Taiwan is about 35 degrees, the geomagnetic vector has a substantial vertical component, and once the sensor tilts that component leaks into the two horizontal axes and badly biases the heading. In practice a tilt of 10 degrees can produce a heading error of several to more than ten degrees.

The Remedy: Tilt Compensation

The remedy is to measure the direction of gravity with an accelerometer first, deduce the roll angle φ and pitch angle θ, and then rotate the magnetometer readings back into the horizontal plane:

$$m_x' = m_x \cdot \cos\theta + m_z \cdot \sin\theta$$

$$m_Y' = m_x \cdot \sin\varphi \cdot \sin\theta + m_Y \cdot \cos\varphi - m_z \cdot \sin\varphi \cdot \cos\theta$$

$$\psi = atan2(-m_Y', m_x')$$

This is why an electronic compass module must always include an accelerometer—a magnetometer alone cannot work correctly while tilted. It also explains something familiar from daily life: a phone compass app usually asks you to hold the phone level, because although tilt compensation corrects for attitude, an accelerometer on a moving platform measures acceleration together with gravity, which spoils the attitude estimate and in turn makes the heading unstable.

From Magnetic Heading to True Heading

The last step is to apply the declination correction:

$$\textit{true heading} = \textit{magnetic heading} + \textit{declination}$$

In Taiwan the declination is about 4 degrees west (recorded by convention as a negative value), so the true heading is roughly the magnetic heading minus 4 degrees. The precise declination can be looked up from the WMM by latitude, longitude, and date—a small example of GNSS and geomagnetic sensing working together: you must know roughly where you are (GNSS) before you can look up the right declination correction.

6.3 Indoor Geomagnetic Fingerprint Positioning

The previous two sections treated interference as the enemy—the rebar in a building distorts the geomagnetic field and throws the compass off. But consider it from another angle: since the pattern of distortion differs at every position and is quite stable, are those distortions not themselves an excellent positional signature? This inversion is the central insight of indoor geomagnetic positioning.

The Principle: Turning a Bug into a Feature

The rebar, steel beams, elevator shafts, and heavy machinery of a modern building are all made of ferromagnetic material and distort the geomagnetic field locally. These distortions have three properties highly favorable to positioning:

- Spatial distinctiveness: the field vector (magnitude and direction) differs from position to position, forming a fingerprint-like signature.

- Temporal stability: building structure does not move, so the field distribution is unchanged for months or even years—far more stable than a Wi-Fi signal (APs get relocated, and crowds cause fluctuations).
- Zero infrastructure: no beacon or base station need be installed; only a database must be built. This is its greatest advantage over UWB and BLE.

The Workflow: Directly Descended from Wi-Fi Fingerprinting

The workflow is almost identical to the Wi-Fi fingerprint positioning of Chapter 4: in the offline stage, field signatures are collected along corridors or at grid points to build the database; in the online stage, a live measurement is matched against the database to find the most similar position.

One Crucial Difference: Too Few Dimensions of Information

Geomagnetic fingerprints have an inherent weakness, however. A Wi-Fi fingerprint at one point may contain a dozen APs, forming a feature vector of a dozen dimensions with strong discriminating power; a geomagnetic fingerprint has only three components (and often only the one dimension of total intensity), so its discriminating power is comparatively thin—it is not at all unusual for two points ten meters apart in a corridor to have exactly the same field strength.

For this reason practical geomagnetic positioning rarely matches single points and instead matches trajectories: the user or robot records a sequence of field readings while walking, and that whole stretch of "field curve" is matched against the database. A curve is far more distinctive than a single value, much as a phrase of melody is easier to recognize than one note. Common methods include the particle filter (Chapter 11) and dynamic time warping (DTW).

Practical Accuracy and Limitations

- Accuracy: combined with trajectory matching and pedestrian dead reckoning (PDR), indoor accuracy reaches about 1 to 3 meters, comparable to or better than Wi-Fi fingerprinting.
- Survey cost: as with Wi-Fi fingerprinting, collection requires walking the site by hand, which is the greatest obstacle to deployment.
- Environmental change: relocating heavy metal equipment or remodeling the building invalidates part of the database; elevator movement and the starting and stopping of large motors also cause momentary disturbance.
- Magnetic storms: although solar activity affects the geomagnetic field, indoor positioning matches a local relative pattern rather than an absolute value, so the effect is comparatively slight.

6.4 Ferromagnetic Interference and Hard-Iron / Soft-Iron Calibration

The magnetometer is one of the sensors most in need of calibration, for a simple reason: the geomagnetic field is only about 45 μT, while the fields produced by the robot's own motors, battery, steel screws, and metal chassis are often far larger. Without calibration the compass reads the robot's own field rather than the Earth's.

The Ideal Case: A Perfect Sphere

First establish a key geometric intuition. Rotate a magnetometer slowly through every direction, collect all three-axis readings, and plot them in three-dimensional space. Since the magnitude of the geomagnetic field is fixed and only its direction changes, those points should in theory lie on a sphere centered at the origin with radius equal to the geomagnetic intensity. The whole task of calibration is to correct the

deformed cloud of measured points back onto that ideal sphere.

Hard-Iron Error: A Displaced Center

Hard-iron error comes from objects on the platform that produce a fixed magnetic field—permanent magnets, magnetized steel parts, motors. They superimpose a constant field vector in the sensor frame, and the effect is to translate the whole sphere away from the origin.

Correction is comparatively simple: find the maximum and minimum of the measured cloud on each axis and take the midpoint as the offset:

$$b = (m_{max} + m_{min}) / 2$$

$$m_{corrected} = m_{raw} - b$$

Soft-Iron Error: The Sphere Becomes an Ellipsoid

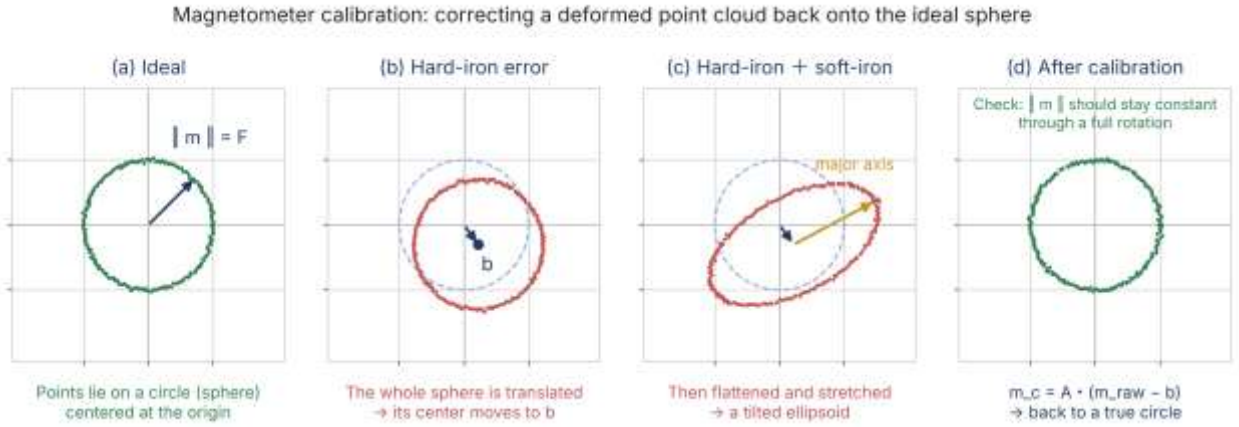
Soft-iron error comes from ferromagnetic material that produces no field of its own but distorts an external one—iron plate, steel beams, the chassis. Such material concentrates field lines in some directions and spreads them in others, and the effect is to flatten or stretch the sphere into an ellipsoid, possibly tilted as well.

Correction requires a 3×3 transformation matrix to pull the ellipsoid back into a true sphere:

$$m_{corrected} = A \cdot (m_{raw} - b)$$

The matrix A is usually estimated from the measured point cloud by an ellipsoid fitting algorithm. This is also why full magnetometer calibration requires data from every direction—turning only in the horizontal plane cannot constrain deformation in the vertical.

Figure 6-2 The Geometric Meaning of Hard-Iron and Soft-Iron Error



These four panels contain the entire spirit of this section. (a) Ideally, turning the magnetometer through every direction puts all readings on a sphere centered at the origin with radius equal to the total geomagnetic intensity. (b) Hard-iron error translates the whole sphere—its shape is unchanged, only its center has moved to b. (c) Soft-iron error further flattens, stretches, and rotates the sphere into a tilted ellipsoid. (d) Calibration subtracts the offset and then applies a matrix that pulls the ellipsoid back into a true sphere.

Once this figure is understood, three practical matters become self-evident: why turning only in the horizontal plane is not enough (that constrains only one cross-section of the ellipsoid and leaves vertical deformation entirely unobserved, hence the figure-eight with flips); why calibration must be done in the final installed configuration (b and A are properties of the whole robot, and calibrating a sensor removed from it measures a different set of parameters); and why "turn through a full rotation and see whether the total intensity stays constant" is the best verification (successful calibration is equivalent to the cloud returning to a true sphere, and a true sphere is by definition one whose radius is everywhere the same).

A Common Misconception

Many students think calibration means pointing north. It does not—calibration requires no knowledge whatever of which way north lies.

Calibration does one thing only: it corrects the shape of the measured point cloud back into a sphere. Which direction on that sphere corresponds to true north is the work of the tilt compensation of Section 6.2 and the declination correction of Section 6.1.

This division of labor—correct the sensor's own systematic error first, then speak of its relation to the world—reappears with exactly the same structure in the camera calibration of Chapter 9 (intrinsics versus extrinsics).

Practical Calibration Procedure

1. Figure-eight calibration: trace a figure eight in the air with the device while rotating it, so that the sensor covers as many three-dimensional attitudes as possible. This is exactly what a phone app is asking for when it says to wave the phone.
2. Calibrate in the final installed configuration: the magnetometer must be mounted on the robot, with all wiring connected and the battery in place, before calibration—because hard-iron error is created by the whole robot, and calibrating the sensor on its own is meaningless.
3. Stay away from environmental sources: calibrate away from reinforced walls, computers, and large metal tables, or the soft-iron effect of the environment will be recorded as if it were error of the platform itself.
4. Verification: turn through a full rotation after calibration and check whether the total field intensity stays steady (it should be close to the local geomagnetic

intensity with little variation); this is the simplest indicator of whether calibration has succeeded.

Dynamic Interference: The Part Calibration Cannot Save

One class of interference cannot be dealt with by calibration beforehand: the field produced by changing motor current. As the robot accelerates the current rises and the field changes with it—a dynamic error tied to the state of motion. Mitigations include mounting the magnetometer far from the motors and power leads (which is why UAVs often place the compass on an extended arm or atop the GPS antenna), building a compensation model from the current reading, or reducing the magnetometer's weight in the filter during high-current intervals.

6.5 The Role of Geomagnetic Navigation in Phones, UAVs, and Quadruped Robots

Phones

The magnetometer in a phone carries three duties: showing direction in the compass app, setting the view heading in a map app (so the map rotates as you turn), and establishing the initial bearing in AR applications. It also supplies a drift-free heading reference in sensor fusion—on the gyroscope alone a phone's orientation estimate would depart noticeably within minutes.

UAVs

The UAV is the platform that depends most on the compass and is most troubled by its problems. The flight controller needs heading to interpret GNSS position changes correctly and to carry out directional control; once the compass is wrong, the classic circular flight known as the toilet bowl effect appears—the UAV believes it is flying

forward while it actually circles outward in ever-widening loops.

- Pre-flight calibration is standard procedure and must be performed near the actual launch point.
- Never calibrate or take off near reinforced structures, iron fencing, an underground garage, or a large metal roof.
- Professional models often carry two compasses that check each other, and switch automatically to a GNSS heading (inferred from the direction of motion) when anomalous readings are detected.

Quadrupeds and Ground Robots

A quadruped or wheeled robot changes attitude sharply, carries many metal parts, and has high-power motors, so a magnetometer works in a rather hostile environment. Indoors, therefore, most systems deliberately reduce the magnetometer's weight or dispense with it altogether—heading is supplied instead by the scan matching result of LiDAR or visual SLAM, because in a structured indoor environment the direction of the walls is far more dependable than the geomagnetic field.

Conversely, in open outdoor settings with little metal (farmland, grassland, large outdoor sites), the geomagnetic field remains the most economical and dependable source of heading. This confirms once again an idea this book stresses repeatedly: there is no sensor that is absolutely better or worse, only settings it suits and settings it does not.

6.6 How the Geomagnetic Field, the IMU, and Visual SLAM Complement One Another

Placed within the localization system as a whole, the role of geomagnetic sensing

can be summed up in one sentence: the only low-cost passive sensor that supplies an absolute heading.

Sensor	Information Provided	Error Characteristics	Principal Weakness
Gyroscope	Angular rate (integrated for change of angle)	Very accurate in the short term; drifts in the long	Accumulation of bias drift
Accelerometer	Direction of gravity (roll, pitch)	No long-term drift	Contamination by motional acceleration
Magnetometer	Direction of magnetic north (heading)	No long-term drift	Ferromagnetic interference; needs calibration
Visual / LiDAR SLAM	Relative pose and environmental structure	Accurate in the short term; drifts in the long	Depends on environmental features
GNSS	Absolute position (heading inferable)	Bounded error	Fails indoors; low update rate

Table 6-1 The Role of Each Sensor in Attitude and Heading Estimation

Three-Way Complementarity: The Logic of an AHRS

The attitude and heading reference system (AHRS) is precisely the classic design that combines the first three:

- The gyroscope supplies smooth, high-rate angular change and is the backbone of attitude estimation.

- The accelerometer corrects roll and pitch drift using the direction of gravity—but it constrains heading (rotation about the vertical axis) not at all, because rotating about the gravity axis does not change the direction of gravity.
- The magnetometer fills exactly that gap: it corrects heading drift using the direction of magnetic north.

The division of labor can be remembered this way: the gyroscope handles everything in the short term, the accelerometer handles up and down in the long term, and the magnetometer handles the compass points in the long term. The complementary filter and the extended Kalman filter (Chapters 7 and 11) are the mathematical tools that realize this division.

After SLAM Is Added

Visual or LiDAR SLAM can also supply heading, and in a structured indoor environment it is often more dependable than a magnetometer. A common strategy in modern indoor robots is therefore:

5. At start-up, let the magnetometer supply a coarse initial heading so that SLAM has a sensible starting point (keeping the map from being built in an entirely random direction).
6. During operation, use the SLAM scan matching result as the primary source of heading, with the magnetometer downweighted or used only to monitor for anomalies.
7. On entering an outdoor or unstructured open area (where the risk of SLAM failure is high), raise the magnetometer's weight again.

8. If the magnetometer reading and the SLAM estimate disagree substantially and persistently, raise an alert—this usually means either that a large metal object is nearby or that SLAM has already lost track.

This design pattern of adjusting trust by situation is exactly the same as the GNSS trustworthiness decision of Figure 5-5 in Chapter 5. By this point you should be able to feel that although the chapters of Part II appear to treat different sensors, they repeatedly treat the same subject: know the error characteristics of each sensor, then let each contribute the right information at the right moment.

Formulations and Procedures

1. The Component Representation of the Geomagnetic Vector

The geomagnetic vector B can be decomposed in the local horizontal frame (north N, east E, down D) as:

$$B = (B_N, B_E, B_D)$$

from which the three commonly used parameters follow:

$$\text{horizontal component} \quad H = \sqrt{(B_N^2 + B_E^2)}$$

$$\text{declination} \quad D = \operatorname{atan2}(B_E, B_N)$$

$$\text{inclination} \quad I = \operatorname{atan2}(B_D, H)$$

$$\text{total intensity} \quad F = \sqrt{(B_N^2 + B_E^2 + B_D^2)}$$

Taking Taiwan as the example, with total intensity F of about 45 μT and inclination I of about 35 degrees, substitution gives a horizontal component $H \approx F \cdot \cos I \approx 37$ μT and a vertical component of about 26 μT. The vertical component is a considerable fraction of the whole, which is the quantitative reason Section 6.2 insists that tilt compensation cannot be omitted.

2. The Magnetometer Calibration Model

The complete calibration model handles hard-iron and soft-iron error at once:

$$m_corrected = A \cdot (m_raw - b)$$

where b is the 3×1 hard-iron offset vector (the displacement of the sphere's center) and A the 3×3 soft-iron correction matrix (restoring the ellipsoid to a true sphere). The indicator of calibration quality is this: after calibration, the total field intensity should stay steady in any attitude:

$$||\, m_corrected \,|| \approx F \quad \textit{(the local total geomagnetic intensity, a constant)}$$

In practice one computes the standard deviation of ||m|| over a stretch of rotation data, and the smaller the value the more successful the calibration. This is a practical indicator students can compute, and judge, immediately in the laboratory.

3. The Complete Processing Chain of an Electronic Compass

Figure 6-3 The Processing Chain of an Electronic Compass from Raw Readings to True Heading

① Raw three-axis magnetometer readings m_raw

↓

② Hard-iron and soft-iron calibration: $m = A \cdot (m_raw - b)$

↓

③ Attitude from the accelerometer: roll φ, pitch θ

↓

④ Tilt compensation: project m back into the horizontal plane to obtain m_x', m_Y'

↓

⑤ Compute the magnetic heading: $\psi = \text{atan2}(-m_Y', m_x')$

↓

⑥ Apply the declination correction (looked up from the WMM by latitude and longitude) → true heading

↓

⑦ Fuse with the gyroscope (complementary filter or EKF) → a smooth, drift-free heading output

6.7 Contemporary Developments in Geomagnetic Navigation: From Manual to Automatic Calibration

Geomagnetic sensing is an old technology—the compass has a history of nearly a thousand years. Its application in robotics, however, has taken two notable turns in

recent years: the resolution of the model has improved greatly, and calibration has moved from a manual ritual to something done automatically.

1. A Finer Geomagnetic Model: WMMHR

The World Magnetic Model (WMM) mentioned in Section 6.1 was released in its WMM2025 edition in December 2024. Published alongside it was the high-resolution edition WMMHR2025, whose spatial resolution near the equator is about three hundred kilometers—roughly a tenfold improvement over the standard edition's three thousand three hundred kilometers.

This difference matters little for ordinary navigation, but for work requiring a high-accuracy heading—surveying, aviation, and long-range autonomous navigation aided by the geomagnetic field—its meaning is clear: the finer the model, the more accurate the local declination correction and the smaller the error in true heading. NOAA recommends that users move to the high-resolution edition.

It is worth remarking that this model is itself a kind of map—only what it draws is not the position of walls but the distribution of a magnetic field. You may regard it as a global-scale prior map maintained by national institutions; the indoor geomagnetic fingerprint map of Section 6.3 is the same concept shrunk to the scale of a building and built by the user. The occupancy grid map to be built from Chapter 13 onward is a map of yet another scale and content. The question of what a map is turns out to be much broader than intuition suggests.

2. No More Waving the Phone: Calibrating Automatically During SLAM

The figure-eight calibration described in Section 6.4 is the greatest practical obstacle to using a magnetometer—it demands an unnatural motion of the user, and in many

settings it simply cannot be done. A sensor mounted on a train, in an elevator, or on a high-speed platform cannot be picked up and waved; nor will a cleaning robot roll itself over.

The remedy proposed in recent research is to bring calibration into SLAM itself. A representative 2024 work proposes a "simultaneous localization, mapping, and calibration" architecture in which a factorized particle filter estimates the magnetometer bias along with the pose and the magnetic field map—concretely, each particle maintains its own Kalman filter to track the bias, and loop closures are determined by spatial neighborhood analysis. Experiments on mobile robots in a shopping mall, an office, and an apartment produced bias estimates comparable to the calibration method built into the operating system, with no calibration motion required of the user at all.

The structure of this method repays close study: its skeleton is precisely the Rao-Blackwellized particle filter used by Gmapping in Chapter 14—the pose is represented by particles, and the remaining quantities (the map in Gmapping, the map plus the bias here) are maintained analytically for each particle. In other words, this is the algorithm you will implement with your own hands in Part IV, applied to a different object. It confirms once again a point this book stresses repeatedly: the mathematical skeleton of SLAM is general, and replacing the sensor and the observation model lets it solve an entirely new problem.

3. Deep Learning Enters Geomagnetic Fingerprint Positioning

Section 6.3 noted that the fundamental difficulty of geomagnetic fingerprinting is having too few dimensions of information—only three numbers at each position (and in practice often only the single number of total intensity), so different positions collide

very easily. Recent work turns to sequence models: rather than a single reading, it considers the shape of the whole curve of field variation over the robot's motion and matches it with a neural network.

Such methods have advanced from early convolutional networks to architectures combining attention mechanisms with long short-term memory networks, and show better robustness than traditional KNN matching in complex indoor environments. Their central insight agrees with the visual bag of words in Chapter 20 and the ORB-SLAM2 loop closure detection in Chapter 22: a single observation is unreliable, but a sequence of observations is highly distinctive.

Threading the Three Together

A finer model (WMMHR) → a more accurate absolute heading; automatic calibration (built into SLAM) → no more manual ritual; sequence learning → localization even from low-dimensional observations.

Their common direction is this: hand the steps that once required human intervention to algorithms that resolve them while running.

That is exactly the spirit of the implementation chapters in Part VIII of this book—a good robotic system should leave the user responsible only for saying where to go, and everything else to the system.

Case Studies

Case 6-1 The UAV "Toilet Bowl Effect": A Textbook Case of Compass Error

A vivid term circulates among UAV pilots: the toilet bowl effect—in position-hold

mode the aircraft not only fails to hold station but circles outward in ever-widening loops, like water going down a bowl. This is almost always a compass problem.

The cause can be understood as follows. The flight controller believes the nose points north while it actually points northeast. When GNSS shows the aircraft drifting east, the controller commands a correction to the west—but what it takes for west is actually northwest. Every correction therefore carries a fixed angular bias, and instead of being eliminated the position error is pushed around tangentially, tracing a circle.

Common causes include calibrating on a reinforced balcony or a metal car roof, rebar or buried utilities beneath the launch point, and magnetic accessories fitted to the aircraft (some camera gimbals and speakers contain permanent magnets). The teaching value of this case is that it shows clearly how heading error and position error couple—get the heading wrong and every position-based control decision is biased systematically to the same side. This idea reappears in the coordinate transforms of Chapter 10 and the navigation implementation of Chapter 29.

Case 6-2 From Interference to Information: Commercial Attempts at Indoor Geomagnetic Positioning

In the mid-2010s companies such as Finland's IndoorAtlas proposed commercial indoor positioning based on geomagnetic fingerprints, briefly regarded as the ideal "no hardware at all" solution—no beacons to install, no reliance on Wi-Fi, nothing but the magnetometer already in the phone. In guidance applications for shopping malls, airports, and exhibition halls the idea did demonstrate meter-level positioning.

Commercialization also exposed its practical limits, however: the fingerprint database had to be built by walking the site, and a large mall required tens of hours of

survey work; once the mall remodeled its units or replaced large metal display racks, part of the database had to be resurveyed; and the discriminating power of the geomagnetic field alone was insufficient, so in practice Wi-Fi, Bluetooth, and phone PDR still had to be fused for stable operation.

This case offers students two lessons. First, zero hardware cost is not zero deployment cost, and data collection and maintenance are often the real barrier—exactly the conclusion reached for Wi-Fi fingerprint positioning in Chapter 4. Second, in a real product it is rare for any single technology to carry the requirement alone, and fusion is very nearly the inevitable answer.

Case 6-3 A Magnetic Field Map of the Classroom: Drawing the Rebar

A demonstration experiment highly popular in both university and high school classrooms: have students carry a phone with a magnetometer app slowly across the classroom floor and along the walls, watching the field strength reading change.

The result tends to impress. In the open center of the room the reading holds steady near 45 μT; step close to a column or a load-bearing wall and it may jump instantly above 60 μT; hold the phone against the metal legs of a computer desk or the iron frame of a blackboard and the reading may double or plunge. Plotting the measured values by position as a color map makes an invisible map of the rebar emerge—the skeleton of the building developed by the magnetic field.

This one experiment accomplishes three teaching aims at once: it lets students feel directly how weak and easily disturbed the geomagnetic field is (Section 6.1), it makes clear why an indoor compass cannot be trusted (Section 6.4), and at the same time it lets them witness the spatial stability and distinctiveness of those very disturbances,

which is the basis of geomagnetic fingerprint positioning (Section 6.3). One set of data explains both why the field cannot be used and why it can—which is the central conceptual turn of this chapter.

Hands-On Activities

Exercise 6-1 Magnetometer Calibration and a Geomagnetic Fingerprint Map of the Classroom

Purpose: to carry out a complete magnetometer calibration and measure the geomagnetic distribution of a classroom, experiencing at the same time the dual identity of the geomagnetic field as a heading sensor and as a positional fingerprint. This exercise can be done with a phone alone; undergraduates may extend it to Arduino or ESP32 hardware.

Equipment

- A smartphone with a sensor app that displays raw three-axis magnetometer values (search for tools of the "magnetometer" or "sensor logger" kind).
- A classroom floor plan, a recording sheet, and colored pencils or spreadsheet software.
- (Optional, undergraduate) An ESP32 or Arduino with a nine-axis module such as the HMC5883L, QMC5883L, or MPU-9250.

Part One: Calibration and Heading Verification

1. Uncalibrated baseline test: hold the phone in the center of the classroom, rotate slowly in place through one full turn, and record the maximum and minimum of the total field strength ||m||. In theory the total strength should

stay constant through the rotation, so the measured range reflects the magnitude of the uncalibrated error.

2. Figure-eight calibration: following the app or system instructions, trace a figure eight in the air with the phone while flipping it through various angles for about 20 seconds, so that the sensor covers as many attitudes as possible.
3. Post-calibration verification: repeat step 1 and compare the range of total strength before and after (the standard deviation may be computed) to verify the calibration.
4. Tilt experiment: hold the phone level facing the blackboard and record the heading; then pitch the phone forward about 30 degrees while keeping the same facing, and observe how much the heading reading changes. Discuss the relation to tilt compensation in Section 6.2.
5. Interference experiment: bring the phone near a laptop, a metal chair leg, a fire hydrant, and a power outlet in turn, recording how much the total field strength and the heading change.

Part Two: A Geomagnetic Fingerprint Map

6. Mark grid points on the classroom floor at a 1.5- to 2-meter pitch (about 15 to 20 points) and number them on the floor plan.
7. Pause for 5 seconds at each grid point in a fixed attitude (phone level, facing the same direction), record the total field strength and the three axis components, and take three measurements per point and average them.
8. Draw the color map: fill in the floor plan using total field strength as the color value (blue for low, red for high) and observe whether the outlines of columns, load-bearing walls, or heavy metal equipment emerge.

9. Blind-test verification: one student is led with eyes closed to a grid point, measures the field, and the others match it against the database to infer the position; repeat 6 times and tabulate the success rate and mean error.
10. Advanced trajectory matching: have one student walk one circuit along the edge of the classroom while recording the field sequence continuously, and observe whether this "field curve" makes it easier to identify which stretch of the route was walked than a single-point value does.

Discussion Questions

1. What was the range of total field strength through one rotation before and after calibration? What does the difference tell you?
2. How large was the heading error produced by a 30-degree tilt? If a robot climbed a 15-degree slope without tilt compensation, what would go wrong in its navigation?
3. What was your blind-test success rate? At which grid points did most of the failures occur? Which limitation mentioned in Section 6.3 does this confirm?
4. Compare the discriminating power of single-point matching with that of trajectory matching, and explain why practical systems prefer the latter.
5. (Undergraduate) Judging from your classroom's magnetic field map, is this room suited to using a magnetometer to supply a robot's heading? If not, what method would you use instead (hint: see Section 6.6)?

Extended Challenge (Undergraduate)

Using an ESP32 with a nine-axis module, write a program that does the following: read the raw three-axis magnetometer and accelerometer values; implement hard-iron calibration (the min-max method) and soft-iron calibration by ellipsoid fitting;

implement tilt compensation and output the true heading (including the declination correction for Taiwan); and finally fuse gyroscope data with a complementary filter, comparing the smoothness and long-term stability of the heading output before and after fusion. This program can serve directly as the basis for IMU attitude estimation in Chapter 7.

Further Reading

1. NOAA NCEI, the official World Magnetic Model 2025 (WMM2025) website and technical report. (Includes an online declination calculator for looking up values anywhere in Taiwan.)
2. British Geological Survey, Geomagnetism teaching resources (geomag.bgs.ac.uk). (Fundamentals of geomagnetism and an account of the model.)
3. M. Kok, J. D. Hol, and T. B. Schön, "Using Inertial Sensors for Position and Orientation Estimation," Foundations and Trends in Signal Processing, 2017. (A complete tutorial on inertial and magnetic sensor fusion, freely available.)
4. B. Li et al., "How Feasible Is the Use of Magnetic Field Alone for Indoor Positioning?," IPIN, 2012. (A representative study of the feasibility of geomagnetic indoor positioning.)
5. J. Haverinen and A. Kemppainen, "Global Indoor Self-localization Based on the Ambient Magnetic Field," Robotics and Autonomous Systems, 2009. (An early classic paper on geomagnetic fingerprint positioning.)
6. J. F. Vasconcelos et al., "Geometric Approach to Strapdown Magnetometer Calibration in Sensor Frame," IEEE Transactions on Aerospace and Electronic

Systems, 2011. (A rigorous mathematical treatment of magnetometer calibration.)

7. The official Compass Calibration documentation of the PX4 and ArduPilot open-source flight stacks. (Practical guidance and troubleshooting for UAV compass calibration.)

From Direction to Inertia: Where This Chapter Stands

This chapter has given the robot an absolute sense of direction and also exposed its fragility: a reinforced wall, a running motor, even the change in current as the robot itself accelerates, is enough to make it point the wrong way. The magnetometer's update rate is also low, so it cannot capture rapid rotation.

The IMU of the next chapter fills exactly these two gaps. A gyroscope updates hundreds of times per second and is extremely sensitive to attitude change over short intervals, and it is wholly immune to magnetic disturbance; but its output must be integrated, and the error accumulates and drifts with time. Hence an elegant division of labor: the gyroscope handles the short term and the magnetometer the long term, and together with the direction of gravity from the accelerometer they form the attitude and heading reference system (AHRS)—exactly the arrangement set out in Table 6-1 of Section 6.6, and the problem that the attitude filtering of Chapter 7 treats formally.

By this point you should have caught the rhythm of Part II: each chapter introduces one sensor, says what it can give and under what conditions it lies, and then points out how the sensor of the next chapter fills that gap. When this chain ends at Chapter 9, every sense on the robot will be in place—and what Part III then teaches is the mathematics of fusing these mutually contradictory readings into the single most reasonable answer.

Summary

1. The geomagnetic field arises from the convection of liquid metal in the Earth's outer core; at only about 25 to 65 μT it is far weaker than the fields produced by everyday metal objects and is therefore extremely easy to disturb; it is described by three parameters—declination, inclination, and total intensity.
2. The north magnetic pole continues to drift toward Siberia, so precise navigation must rely on the World Magnetic Model (WMM); the current edition, WMM2025, was released in December 2024 and is valid through the end of 2029, with a high-resolution edition WMMHR2025 published alongside it.
3. An electronic compass must combine an accelerometer for tilt compensation—the inclination in Taiwan is about 35 degrees, so the vertical component is substantial and uncompensated tilt produces significant heading error; the declination correction is applied last to obtain the true heading.
4. The field distortion caused by indoor steel structure is interference for navigation yet a stable and distinctive positional fingerprint for localization; because a single point carries few dimensions of information, practice favors trajectory matching with a particle filter, giving accuracy of about 1 to 3 meters.
5. The geometric essence of magnetometer calibration is to restore a deformed point cloud to a true sphere: hard-iron error displaces the center (corrected by the offset vector b) and soft-iron error turns the sphere into an ellipsoid (corrected by the matrix A); calibration must be performed in the final installed configuration and away from environmental disturbance.
6. The UAV depends on the compass most and suffers most from it, and a compass error leads to the toilet bowl effect; indoor robots commonly

downweight the magnetometer and take heading from SLAM scan matching instead.

7. In the AHRS architecture the gyroscope handles everything in the short term, the accelerometer handles up and down in the long term, and the magnetometer handles the compass points in the long term; once SLAM is added, the magnetometer often retreats to initialization and anomaly monitoring.

In the next chapter we go deeper into the central sensor of Part II—IMU inertial navigation. How do an accelerometer and a gyroscope measure motion? Why does integration produce drift? And what are attitude filtering, IMU preintegration, and tightly coupled visual-inertial estimation? These will be important groundwork for the later chapters on visual SLAM and UAV navigation.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 7

IMU Inertial Navigation

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

The magnetometer of Chapter 6 gave the robot an absolute direction, but it is slow and fragile. The inertial measurement unit of this chapter supplies the opposite character: extremely fast and wholly immune to outside disturbance, yet badly wrong as soon as any length of time passes.

The IMU holds a peculiar place in this book. It is one of the sensors of Part II and also one of the protagonists of the visual-inertial SLAM of Part VI. More importantly, it is the first sensor in the book that can only be used by integration—which brings out the central contradiction of SLAM: dead reckoning is fast and continuous yet must drift, while observation is slow and discrete yet does not drift. The whole book, from the filtering of Chapter 11 and the graph optimization of Chapter 12 to the loop closure detection from Chapter 20 onward, addresses different faces of this one contradiction.

What this chapter asks you to attend to, then, is not the physics of MEMS but the law of error growth in Section 7.3 and the idea of preintegration in Section 7.5. The former explains why the IMU cannot be used alone, and the latter is the key engineering technique that lets modern visual-inertial systems run in real time.

Figure 7-0 How Chapter 7 Connects to the Rest of the Book

[Building on the previous chapter]

Chapter 6 The magnetometer: absolute and drift-free, but slow and easily disturbed

This chapter The IMU: fast and undisturbed, but bound to drift → the two complement each other as an AHRS

[This chapter is responsible for]

The measurement principles of specific force and the Coriolis force → attitude solution

→ integration drift (t^2 and t^3)

Attitude filtering: complementary → Kalman → EKF

Preintegration: letting a high-rate IMU enter a low-rate optimization problem

[Where it leads]

→ Chapter 11 This chapter's complementary filter and EKF formally subsumed into the Bayesian filtering framework

→ Chapter 12 Preintegration becomes an edge in a pose graph (foreshadowed in Section 7.5)

→ Chapter 18 LIO: LiDAR-inertial odometry

→ Chapter 23 The tightly coupled visual-inertial estimation of ORB-SLAM3 (the full version of Section 7.7)

→ Chapter 26 UAVs: the IMU is the heart of the flight controller, and its failure means a crash

Learning Objectives

The IMU is the busiest sensor on a robot—it reports the state of motion hundreds of times per second—and also the one most apt to lie. This chapter goes deep into the principles, limitations, and modern remedies of inertial navigation, material that directly supports the visual-inertial SLAM of Part VI and the UAV navigation of Part VII. After completing this chapter, you should be able to:

1. Explain the MEMS measurement principles of the accelerometer and the gyroscope, and explain why an accelerometer still reads about 9.8 at rest.
2. Define the three attitude angles roll, pitch, and yaw, and compute roll and pitch from an accelerometer.

3. Explain the cause of integration drift, and explain why position error grows with the square or even the cube of time.
4. Explain the basic ideas of the complementary filter and the Kalman filter, and compare the settings each suits.
5. Explain what problem IMU preintegration solves, and why it is the key technique in modern visual-inertial SLAM.
6. State the four major contributions of the IMU in visual-inertial SLAM, and explain the difference between loose and tight coupling.
7. Explain how ORB-SLAM3 completes IMU initialization by maximum a posteriori estimation, and why initialization requires motion excitation.

Core Concepts

Chinese Term	English Term	Explanation
慣性量測單元	IMU	A sensor module integrating an accelerometer and a gyroscope (and sometimes a magnetometer).
比力	Specific Force	What an accelerometer actually measures: motional acceleration minus gravitational acceleration.
科氏力	Coriolis Force	The physical effect by which a MEMS gyroscope measures angular rate.
姿態角	Roll / Pitch / Yaw	Roll, pitch, and yaw, describing a rigid body's orientation in three-dimensional space.
零偏	Bias	The fixed output error a sensor shows with zero input, which varies with temperature and time.

Chinese Term	English Term	Explanation
隨機游走	Random Walk	The random drift produced by integrating white noise, with error growing as the square root of time.
積分漂移	Integration Drift	Tiny errors accumulating continuously through integration, so the estimate departs gradually from the truth.
互補濾波器	Complementary Filter	Fuses gyroscope and accelerometer by complementary high-pass and low-pass filtering.
卡爾曼濾波器	Kalman Filter	The optimal linear recursive estimator under the assumption of Gaussian noise.
擴展卡爾曼濾波	EKF	A method applying the Kalman filter after linearizing a nonlinear system.
IMU 預積分	IMU Preintegration	Integrating the IMU measurements between two keyframes in advance into a single constraint, avoiding repeated computation.
緊耦合	Tightly Coupled	Solving the raw IMU measurements and the visual measurements within one optimization problem.
最大後驗估測	MAP	Finding the most probable state estimate given the observations; the theoretical basis of ORB-SLAM3's initialization.
尺度不確定性	Scale Ambiguity	A monocular camera cannot determine true size, so scale must be supplied by an IMU or an object of known size.

7.1 The Accelerometer, the Gyroscope, and the Magnetometer

At the core of an IMU (inertial measurement unit) are two kinds of sensor: an accelerometer measures linear acceleration and a gyroscope measures angular rate. The magnetometer introduced in Chapter 6 is often packaged with them to form what is called a nine-axis module. The division of labor among the three is exactly the one given in Section 6.6: the gyroscope handles everything in the short term, the accelerometer handles up and down in the long term, and the magnetometer handles the compass points in the long term.

1. The Accelerometer: A Mass on a Spring

The principle of a MEMS accelerometer can be understood from a very simple model: a tiny proof mass is suspended within the chip on elastic beams. When the chip accelerates, inertia displaces the mass relative to the frame, that displacement changes the gap between capacitor plates, and measuring the change in capacitance with a circuit converts it into acceleration. The whole structure is etched into a few square millimeters of silicon, at a cost as low as a few tens of NT dollars.

There is a key idea here that almost every beginner finds puzzling: set an accelerometer at rest on a desk and it reads not 0 but about 9.8 m/s² along the vertical. Why should something at rest have an acceleration?

A Clarification: An Accelerometer Measures Specific Force, Not Acceleration

The physical quantity an accelerometer actually measures is specific force, defined as:

a_measured = a_true - g

where a_true is the platform's true acceleration relative to an inertial frame and g is the gravitational acceleration vector.

Intuitively: an accelerometer measures how hard the spring is pushing the proof mass. At rest the spring must exert force against gravity to hold the mass up, so the reading is about 9.8;

in free fall the mass descends together with the housing and the spring bears no force at all, so the accelerometer reads 0 instead—which is the physical essence of weightlessness aboard a space station.

Conclusion: before using an accelerometer you must know the direction of gravity and subtract it to obtain the true motional acceleration. And the direction of gravity must in turn be determined from the attitude—

this circle, in which attitude and acceleration depend on each other, is one of the roots of the difficulty of inertial navigation.

2. The Gyroscope: Vibration and the Coriolis Force

A MEMS gyroscope measures angular rate. Its principle uses the Coriolis force: a proof mass inside the chip is driven in high-frequency reciprocating vibration, and when the whole assembly rotates the vibrating mass experiences a Coriolis force perpendicular to the direction of vibration, producing a detectable lateral displacement whose magnitude is proportional to the angular rate.

Note in particular that a gyroscope outputs angular rate (degrees turned per

second), not angle. To obtain an angle it must be integrated over time—and that is precisely the source of the drift problem (Section 7.3).

3. Grades and Specifications

IMUs divide by accuracy into several grades, with prices differing by factors of thousands:

Grade	Typical Bias Stability	Price Range	Typical Application
Consumer MEMS	Tens of degrees per hour	Tens to hundreds of NT dollars	Phones, toys, teaching modules
Industrial MEMS	A few degrees per hour	Thousands to tens of thousands	UAVs, robots, AGVs
Tactical grade	0.1 to 1 degree per hour	Hundreds of thousands	Guidance, surveying, self-driving cars
Navigation grade	Below 0.001 degree per hour	Millions and above	Aviation, submarines, spaceflight

Table 7-1 IMU Accuracy Grades and Applications

The MPU-6050, MPU-9250, ICM-20948, and BNO055 modules used in the teaching experiments of this book fall between consumer and industrial grade; UAV flight controllers (the Pixhawk family) mostly use industrial grade and often carry several units that check each other. Understanding the difference between grades matters: the same algorithm can dead-reckon independently for minutes on a tactical-grade IMU and may lose accuracy in seconds on a consumer module—which determines how much trust the system design may place in the IMU.

7.2 Attitude Estimation: Roll, Pitch, and Yaw

Attitude describes the orientation of a rigid body in space. The Euler angle representation customary in aviation has three angles, which you may remember by thinking of an aircraft:

- Roll (φ): rotation about the fore-and-aft axis—the wings rock up and down and the fuselage banks.
- Pitch (θ): rotation about the lateral axis—the nose rises or drops.
- Yaw (ψ): rotation about the vertical axis—the nose turns left or right, which is the heading angle discussed in Chapter 6.

Obtaining Roll and Pitch from the Accelerometer

When the platform is at rest or in uniform motion, what the accelerometer measures is the components of gravity on its three axes. Since gravity necessarily points toward the center of the Earth (straight down), comparing the three readings reveals how far the sensor is tilted:

$$\varphi = atan2(a_y, a_z)$$

$$\theta = atan2(-a_x, \sqrt{(a_y^2 + a_z^2)})$$

A Key Limitation: The Accelerometer Cannot Reach Yaw

There is a limitation that must be kept firmly in mind: an accelerometer cannot measure yaw. The reason is intuitive—lay the device flat on a desk and rotate it in place, and the direction of gravity never changes, so the three readings never change and the accelerometer is entirely insensible of the rotation. This is exactly why the magnetometer of Chapter 6 is indispensable, and why purely visual SLAM needs some other source for an initial heading.

A Second Limitation: Acceleration Contamination

The formulas above presuppose that the accelerometer measures gravity alone. But as soon as the robot accelerates, turns, or vibrates, the reading is the superposition of gravity and motional acceleration, and the attitude angles computed from it are badly distorted. A vehicle accelerating hard is misjudged by these formulas as nose-up—because a forward acceleration is, to the sensor, indistinguishable from the component of gravity after a tilt.

This limitation yields an important conclusion: an accelerometer is suited to supplying a long-term average attitude reference but cannot be used for instantaneous attitude estimation. The gyroscope is exactly the opposite—accurate in the short term, drifting in the long. Their complementarity is the entire motivation for the attitude filtering of Section 7.4.

Gimbal Lock in Euler Angles

Euler angles are intuitive but have a famous defect: when the pitch angle approaches ±90 degrees (nose straight up or straight down), the roll and yaw axes coincide and a degree of freedom is lost—gimbal lock. For UAV aerobatics or a robot arm this is fatal. Practical systems therefore mostly represent attitude internally with quaternions, which have no singularity and are computationally efficient, if harder to picture. A full account of quaternions is left to Chapter 10; for this chapter it is enough to build intuition with Euler angles.

7.3 The Problem of Integration Drift

The beautiful promise of inertial navigation is that an IMU alone lets a robot know its pose entirely on its own—no satellites, no landmarks, no external signal of any kind. The promise holds perfectly in theory and is almost impossible to realize alone in

practice. The reason is one word: integration.

Double Amplification by Integration

Deriving a pose from IMU readings requires integration:

$$\text{angular rate } \omega \xrightarrow{\text{(integrate once)}} \text{attitude angle}$$

$$\text{acceleration } a \xrightarrow{\text{(integrate once)}} \text{velocity} \xrightarrow{\text{(integrate again)}} \text{position}$$

Integration is an operation that only takes in and never lets out: every tiny error at every instant is added permanently into the result and never cancels itself. Worse, position requires two integrations, so the error is amplified twice over.

The Mathematics of Error Growth

Suppose the accelerometer has a constant bias error b (0.01 m/s², say, which is quite common on a consumer module). Then after a time t:

$$\text{velocity error} = b \cdot t$$

$$\text{position error} = (1/2) \cdot b \cdot t^2$$

Put numbers in to get a feel for it: with $b = 0.01$ m/s², the position error after 10 seconds is about 0.5 meters, after 60 seconds about 18 meters, and after 300 seconds as much as 450 meters. And this is only the accelerometer bias; gyroscope bias makes attitude error grow linearly with time, and once the attitude is wrong gravity is not fully removed, so the residual gravity component is taken for motional acceleration and integrated twice—a path whose position error grows as t^3, and the dominant error source in practice.

An Intuitive Account: How Attitude Error Poisons Position Estimation

Suppose the attitude estimate is off by just 1 degree. What the system takes

for vertical then differs from true vertical by 1 degree,

and after gravity is removed a horizontal component remains: $9.8 \times \sin(1°) \approx 0.17$ m/s².

This nonexistent acceleration, integrated twice, produces about 8.5 meters of position error in 10 seconds.

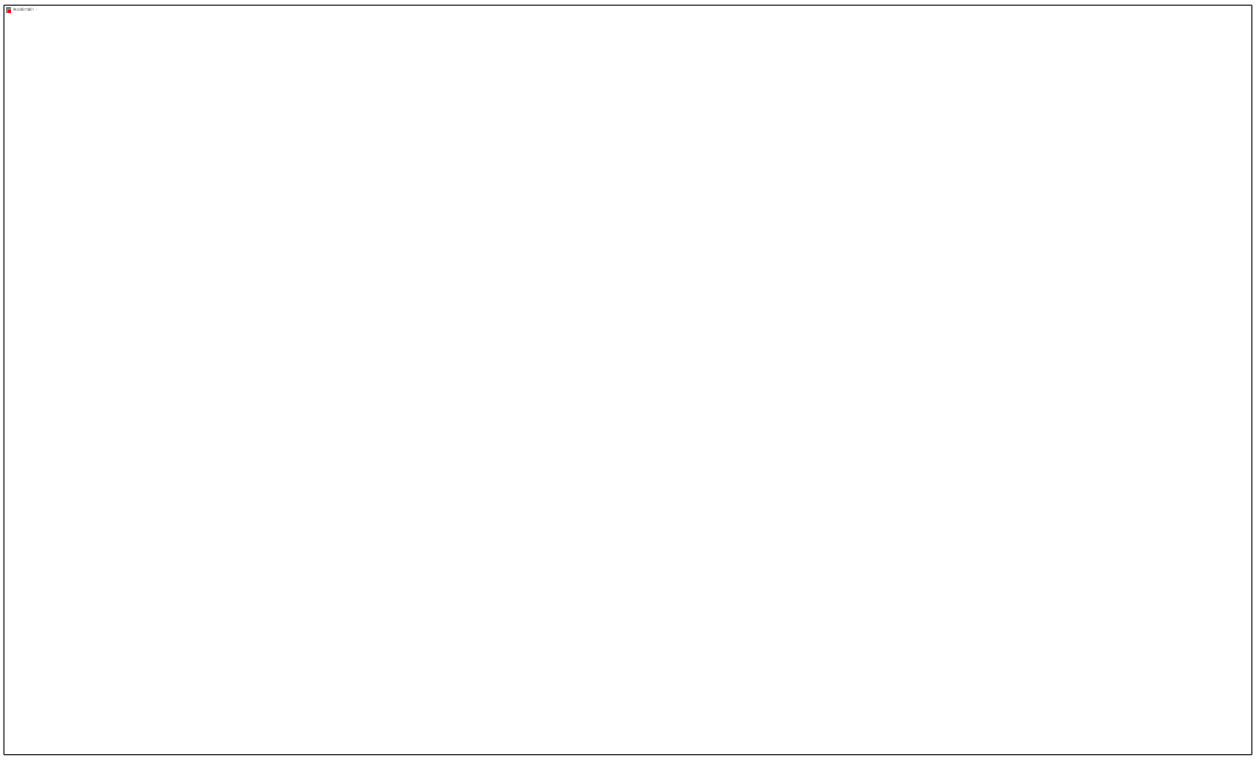
Conclusion: in inertial navigation, attitude accuracy matters more than acceleration accuracy. This is exactly why the attitude filtering of Section 7.4 is so important,

and why every practical system must bring in external information (magnetometer, GNSS, vision) to constrain the attitude.

Three Sources of Error

- Bias: a constant output offset with zero input. It is not a constant but varies with temperature (especially noticeably during warm-up) and wanders slowly with time. It is the principal error source and the quantity every filter and optimization algorithm sets out to estimate online.
- Noise: random high-frequency disturbance. After integration it becomes a random walk, with error growing as the square root of time—compared with the linear or quadratic growth of bias, this term is in fact not the main problem.
- Scale factor and axis non-orthogonality errors: departure of the actual sensitivity from its nominal value, and three axes not exactly perpendicular. Both can be reduced substantially by calibration beforehand.

Figure 7-1 Position Error Growth in Pure Inertial Dead Reckoning with a Consumer-Grade MEMS IMU



This figure makes the whole point of Section 7.3 with concrete numbers. It uses bias levels typical of consumer MEMS: an accelerometer bias of 0.02 meters per second squared (about 2 mg) and a gyroscope bias of 0.01 degrees per second. The two error curves have entirely different shapes—the error from accelerometer bias grows as t^2, while gyroscope bias produces t^3 growth by the route of an attitude error leaving gravity imperfectly removed.

Note the crossing point: at about thirty-five seconds the gyroscope-induced error overtakes the accelerometer's and dominates completely thereafter. The total error is about 1.3 meters at ten seconds, about 16.7 meters at thirty, and close to a hundred meters by sixty. One minute, one hundred meters—which is why nobody navigates on a consumer-grade IMU alone.

The figure also explains two things of great engineering importance. First, why nearly

all the cost of a high-end IMU goes into the gyroscope: because the t^3 term dominates in the end, reducing gyroscope bias pays far more than reducing accelerometer bias. Second, why the key parameter of a visual-inertial system is how often a visual observation arrives: pull the IMU back on course while the error is still small (within a keyframe interval of a fraction of a second, say) and this curve never reaches its right-hand half. That strategy is exactly how ORB-SLAM3 in Chapter 23 attains centimeter accuracy on a UAV.

An Order of Magnitude Worth Remembering

Attitude error $\propto t$ (linear), velocity error $\propto t^2$, position error $\propto t^3$—each further integration raises the power of time by one.

This law of integration amplifying error is not the IMU's alone. The wheel odometry of Chapter 13 and the visual odometry of Chapter 20 are all dead-reckoning sensors, and none escapes accumulated drift.

They differ only in rate of growth: wheel odometry roughly linear to quadratic, visual odometry roughly linear, and the IMU cubic—so the IMU drifts most fiercely and most needs frequent correction.

The Practical Conclusion

The right use of an IMU is not to navigate with it alone but to fill the gaps of other sensors. The usable dead-reckoning interval of a consumer-grade IMU is on the order of seconds, an industrial-grade unit reaches tens of seconds, and only tactical grade and above lasts for minutes. The central question of system design is therefore always this: before the IMU drifts beyond acceptable bounds, when will the next external correction arrive? Whether that correction comes from GNSS (Chapter 5), LiDAR scan

matching (Chapter 13), or visual features (Chapter 20), the reasoning is the same.

7.4 Attitude Filtering: The Complementary Filter, the Kalman Filter, and the EKF

Since the gyroscope is accurate in the short term and drifts in the long, while the accelerometer is accurate in the long term and contaminated in the short, the answer suggests itself: fuse the two and take the best of each. That is the task of attitude filtering.

1. The Complementary Filter: Simplest and Effective

The idea of the complementary filter is wonderfully direct: high-pass the integrated gyroscope result (trust its rapid changes), low-pass the angle computed from the accelerometer (trust its long-term average), and add the two with weights:

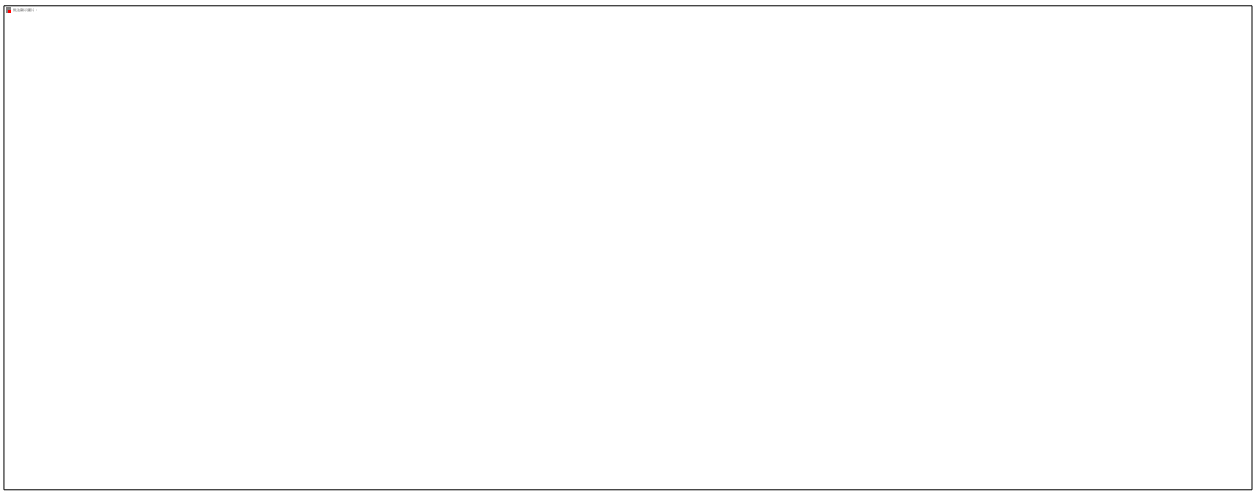
$$\theta = \alpha \cdot (\theta_{prev} + \omega \cdot \Delta t) + (1 - \alpha) \cdot \theta_{accel}$$

where α is usually taken between 0.95 and 0.99. A larger α means greater trust in the gyroscope, with a smoother output but weaker resistance to drift; a smaller α follows the accelerometer more quickly but is more easily disturbed by vibration.

- Advantages: a few lines of code, almost no computational cost, and a single parameter with a clear physical meaning—very well suited to Arduino-class microcontrollers and teaching experiments.
- Limitations: no statistical model of error, no ability to estimate bias, and no information about how uncertain the estimate is.

Figure 7-2 The Time-Domain Behavior and Frequency-Domain Meaning of the

Complementary Filter



The left panel is what you will plot with your own hands in Exercise 7-1. The light blue trace is the attitude computed from the accelerometer alone: centered on the truth over the long run but extremely noisy; the red trace is pure gyroscope integration: a beautifully smooth curve that departs steadily upward; and the green trace is the complementary filter output, nearly coincident with the black dashed line of true attitude.

The right panel explains where the word "complementary" comes from. The complementary filter in fact performs a division of labor in frequency: the accelerometer signal passes a low-pass filter (keeping only slow changes and rejecting vibration noise), the gyroscope signal passes a high-pass filter (keeping only fast changes and rejecting slow drift), and the responses of the two paths are exactly complementary, summing to unity at every frequency.

The corner frequency is set by α and the sampling period: the time constant is $\tau = \alpha\Delta t / (1 - \alpha)$. With $\alpha = 0.98$ and $\Delta t = 0.01$ second, τ is about 0.49 second and the corner frequency about 0.32 hertz. This gives you a physical basis for tuning: attitude changes slower than that are judged by the accelerometer, faster ones by the gyroscope. If the robot moves violently and acceleration contamination is severe, raise α (trust the

gyroscope longer); if the gyroscope is of poor quality and drifts quickly, lower α.

This idea of allocating trust according to signal characteristics recurs in more elaborate forms throughout the rest of the book: the Kalman gain of Chapter 11 is an α computed automatically from the uncertainties, the information matrix on each edge in the graph optimization of Chapter 12 is the matrix version of the same thing, and the judgment in the multi-robot systems of Chapter 27 of whether to trust oneself or a teammate is, in essence, the same question.

2. The Kalman Filter: The Optimal Linear Estimator

The Kalman filter raises the problem to the level of probability. It estimates not only what the attitude is but also how uncertain that estimate is (represented by a covariance matrix), and at each step it does two things:

1. Predict: propagate the attitude to the next instant using the gyroscope reading, while letting the uncertainty grow—because integration accumulates error.
2. Update: correct the prediction with the accelerometer observation. The strength of the correction is set by the Kalman gain: if the prediction is very uncertain and the observation noise small, the observation is trusted more; otherwise the prediction is.

This predict-update cycle is a miniature of the standard SLAM pipeline of Chapter 1 and the core mathematics that Chapter 11 unfolds in depth. A great advantage of the Kalman filter is that bias can be included in the state vector and estimated along with everything else—the system learns automatically how far this particular gyroscope is off today, which the complementary filter cannot do.

3. The Extended Kalman Filter (EKF): Handling Nonlinearity

The standard Kalman filter requires a linear system, but attitude kinematics is full of trigonometric functions and quaternion products and is nonlinear in essence. The EKF's remedy is to linearize the system by a first-order Taylor expansion about the current estimate (computing a Jacobian) and then apply the Kalman filter equations.

The EKF is the most widely used method of attitude estimation in robotics: UAV flight stacks (EKF2 and EKF3 in PX4 and ArduPilot), the robot_localization package in ROS, and sensor fusion in phones are nearly all EKFs behind the scenes. Its limitations are that linearization may be inaccurate when errors are large, and that it needs a sensible initial estimate to converge.

Method	Computational Demand	Estimates Bias	Provides Uncertainty	Suited To
Complementary filter	Very low	No	No	Teaching, microcontrollers, simple platforms
Kalman filter	Medium	Yes	Yes	Linear systems, simplified attitude models
EKF	Medium to high	Yes	Yes	UAV flight control; the mainstream choice in robotics
Mahony / Madgwick	Low	Partly	No	Common in open-source flight stacks and AHRS modules

Table 7-2 Comparison of Common Attitude Filtering Methods

A note in addition: the Mahony and Madgwick filters are practical schemes between the complementary filter and the Kalman filter, representing attitude by

quaternion and correcting by gradient descent or proportional-integral control; they are computationally light yet perform respectably, are extremely common in open-source flight stacks and AHRS modules, and make a good first stop for more advanced implementation.

7.5 The Concept of IMU Preintegration

This section enters the key technique of modern visual-inertial SLAM. The material is comparatively advanced; a high school course may take away only the intuition of what problem it solves, while undergraduates should understand its design logic.

The Starting Point: A Mismatch of Rates

A camera in visual SLAM runs at about 20 to 30 frames per second, while an IMU runs at 200 to 1,000 samples per second. That means several tens to over a hundred IMU measurements lie between two consecutive images. How is this mass of high-rate data to be turned into a single constraint the back-end optimizer can use?

The Fatal Flaw of the Naive Approach

The most intuitive approach is to integrate the IMU measurements over that interval directly, obtain the change of pose from frame i to frame j, and add it as a constraint in the optimization.

But there is a serious problem here. Integrating acceleration requires first removing gravity, removing gravity requires knowing the attitude, and the attitude is tied to the pose estimate of frame i. During optimization the pose estimate of frame i is updated repeatedly—and every update requires recomputing the entire IMU integral. With over a hundred measurements and dozens of iterations, the computation becomes completely unmanageable in real time.

The Central Insight of Preintegration

The ingenuity of preintegration lies in expressing the integral in a form independent of the initial pose.

Concretely, instead of integrating the change of pose in the world frame, one integrates the relative motion in the body frame of frame i. These relative quantities depend only on the raw IMU readings and the bias estimates and are entirely independent of frame i's position and orientation in the world. However the pose of frame i is updated during optimization, the preintegration result needs no recomputation—only one simple coordinate transformation at the outermost level.

Figure 7-3 The Problem Preintegration Solves

[The naive approach]

The pose of frame i is updated → the direction of gravity changes → the whole IMU interval must be reintegrated → the computation explodes

[The preintegration approach]

Preintegrate the relative motion (independent of the initial pose) → computed once only

↓

However the pose of frame i changes during optimization, only one outer coordinate transformation is needed

↓

The computation falls sharply and real-time execution becomes possible

What Preintegration Produces

Preintegration between two keyframes produces three relative quantities and one uncertainty:

- Relative rotation ΔR: how far the body turned over the interval.
- Relative velocity change Δv: how much the velocity changed.
- Relative position change Δp: how far it moved in the initial body frame.
- The covariance matrix: how trustworthy this constraint is, which sets its weight in the optimization.

Handling the Bias

One more detail: the bias must be removed before integration, yet the bias is itself an unknown to be estimated. If the bias estimate changes during optimization, must the preintegration be recomputed after all? The remedy in preintegration theory is to compute at the same time the first-order partial derivatives (the Jacobians) of the preintegration result with respect to the bias, so that a small change in bias needs only a quick linear correction rather than reintegration.

Preintegration theory was introduced by Forster et al. in 2015 with a rigorous treatment of the manifold structure of the rotation group, and it is now a standard component of every mainstream visual-inertial system—VINS-Mono, OKVIS, and ORB-SLAM3 without exception. This also answers why the MIT VNAV course of Chapter 2 devotes so much space to optimization on manifolds: a rotation is not an ordinary vector, and handling it requires dedicated mathematical tools.

7.6 The Role of the IMU in Visual-Inertial SLAM

Why must visual SLAM add an IMU? The answer is not as simple as one more sensor being one more insurance policy—what the IMU supplies is several things that purely visual methods cannot do even in principle.

Four Major Contributions of the IMU

3. It supplies absolute scale: a monocular camera has an inherent scale ambiguity—the same image sequence could be large motion in a large room or small motion in a doll's house, and vision alone cannot tell. The IMU measures acceleration in physical units, and integrating it supplies true scale. This is the single most important value of an IMU in a monocular visual-inertial system.

4. It supplies the direction of gravity: the accelerometer's long-term average points along gravity, letting the system establish which way is down. This makes roll and pitch observable and lets the map be set upright—crucial for UAV control and AR applications.
5. It fills the gaps when vision fails: when rapid rotation blurs the image, the lens is briefly obscured, or the robot enters a white-walled or dark area, the visual front end loses track. The IMU can still dead-reckon usefully for a few seconds, carrying the system through the blank and back onto the track—which raises robustness substantially.
6. It supplies high-rate output: 30 frames per second is too slow for flight control. The IMU lets the system output pose at hundreds of Hz, meeting the needs of a real-time control loop.

What Vision Gives Back to the IMU

The partnership runs both ways. Vision supplies exactly what the IMU lacks most: drift-free observations of position and attitude. Through visual constraints the system can continuously estimate and compensate the IMU bias, bringing the drift nightmare of Section 7.3 under control. Their relationship can be summed up thus: the IMU gives vision scale and gravity, and vision gives the IMU calibration and constraint.

Loose Coupling and Tight Coupling

- Loosely coupled: the visual and inertial systems each compute a pose independently and the two results are then fused with weights. The advantages are modularity, ease of implementation, and the ability to reuse an existing visual SLAM system directly; the drawback is that information has already been compressed within each stage, so the raw measurements cannot be revisited at fusion time and accuracy is limited.

- Tightly coupled: the visual reprojection errors and the IMU preintegration constraints are placed in one optimization problem and pose, velocity, bias, and map points are solved together. The advantage is that all information is handled consistently within one probabilistic framework, with markedly better accuracy; the drawbacks are implementation complexity and computational cost.

Modern high-accuracy systems use tight coupling without exception. This again echoes the fusion discussion in Section 4.7 of Chapter 4 and in Chapter 5—by now you will have noticed that "tight coupling beats loose coupling" is a general principle running through this whole book: the later information is compressed, the better the optimum that can be found.

7.7 ORB-SLAM3 and Tight Coupling with the IMU

ORB-SLAM3 is the most complete open-source visual-inertial SLAM system available, developed by a team at the University of Zaragoza in Spain. It was the first system to support purely visual, visual-inertial, and multi-map SLAM at once while accommodating monocular, stereo, and RGB-D cameras and both pinhole and fisheye lens models. This section builds the overall picture; a full dissection of the system is left to Chapter 23.

The Core Innovation: Maximum a Posteriori Estimation Throughout

The most important contribution of ORB-SLAM3 is a tightly coupled visual-inertial system built entirely on maximum a posteriori (MAP) estimation—the IMU initialization stage included. This lets the system run robustly in real time indoors and out and in scenes large and small, with accuracy about two to ten times better than earlier methods.

Why IMU Initialization Is So Difficult

At start-up a cluster of unknowns must be determined at once: the true scale of the map, the direction of gravity, the accelerometer and gyroscope biases, and the initial velocity. These quantities are coupled to one another, and not all of them are immediately observable.

The initialization procedure of earlier systems (such as ORB-SLAM-VI) took about 15 seconds to complete, during which accuracy was poor, badly affecting robustness and the user's experience. ORB-SLAM3 reformulates initialization as an optimization problem—solving within the MAP framework for the most probable values of scale, gravity direction, bias, and velocity at once—which makes initialization far faster and more reliable.

Motion Excitation: A Necessary Condition for Initialization

There is a point here that must be known in practice: IMU initialization requires sufficient motion excitation. The reason is that if the platform moves at constant velocity in a straight line, or stands still, the accelerometer reads essentially nothing but gravity and the system cannot separate scale from bias—mathematically these quantities are said to be unobservable.

In practice, therefore, after starting visual-inertial SLAM one should first perform a stretch of motion with accelerations, decelerations, and rotations (hold the camera and move it a little forward and back and side to side while rotating it, for a few seconds) before the system can complete initialization. This is one of the most frequent problems students meet while implementing ORB-SLAM3 in Chapter 30: "why does initialization keep failing?" The answer is usually that there was not enough motion.

The IMU within the Three-Thread Architecture

- Tracking thread: extract ORB features from each frame and estimate the camera pose by minimizing reprojection and inertial error; in visual-inertial mode it also estimates velocity and the IMU biases.
- Local mapping thread: insert keyframes, cull redundant information, and run local bundle adjustment—which in inertial mode is visual-inertial BA, completing and continuously refining the IMU parameters by MAP estimation.
- Loop closing and map merging thread: detect loop closures and map overlap and perform global correction.

The Multi-Map System: Losing Track Is No Longer a Disaster

The second innovation of ORB-SLAM3 is its multi-map architecture (the Atlas). When the system loses track because visual information has been poor for a long stretch, it does not collapse but opens a new map and continues; later, on returning to a region it has visited, it merges the new and old maps seamlessly. This is extremely practical for a robot running for long periods—Chapter 23 discusses the mechanism in full.

Accuracy

On public datasets, the stereo-inertial mode of ORB-SLAM3 attains an average accuracy of about 3.5 centimeters on the EuRoC UAV dataset; on the TUM-VI room sequences, recorded by hand with rapid motion to represent AR and VR scenarios, accuracy reaches about 9 millimeters. The order of magnitude speaks for itself: a cheap camera plus a cheap IMU, under the right algorithm, can attain surprising accuracy—and that is exactly what this book hopes you will verify with your own hands.

Formulations and Procedures

1. The Basic Kinematic Equations of Inertial Navigation

Let the position in the world frame be p, the velocity v, and the attitude rotation matrix R; let the IMU measure angular rate ω and specific force a, with gravity vector g. The continuous-time update equations are:

$$\dot{R} = R \cdot [\omega]_{\times}$$

$$\dot{v} = R \cdot a + g$$

$$\dot{p} = v$$

where $[\omega]_{\times}$ denotes the skew-symmetric matrix of the angular rate. These three equations are the whole of inertial navigation—and precisely because they must be integrated twice, the drift problem of Section 7.3 arises. In discretized form for implementation:

$$R(k+1) = R(k) \cdot Exp(\omega \cdot \Delta t)$$

$$v(k+1) = v(k) + (R(k) \cdot a + g) \cdot \Delta t$$

$$p(k+1) = p(k) + v(k) \cdot \Delta t + (1/2) \cdot (R(k) \cdot a + g) \cdot \Delta t^2$$

2. Error Growth Caused by Bias

With accelerometer bias b_a and gyroscope bias b_g, the principal error terms grow as:

$$attitude\ error \approx b_g \cdot t \quad (linear)$$

$$velocity\ error \approx b_a \cdot t + g \cdot b_g \cdot t^2 / 2$$

$$position\ error \approx b_a \cdot t^2 / 2 + g \cdot b_g \cdot t^3 / 6 \quad (the\ cubic\ term\ dominates)$$

Note the t^3 term containing g in the third expression: gyroscope bias becomes, by the route of attitude error → gravity imperfectly removed → spurious acceleration, the

dominant error source in long-duration inertial navigation. This expression explains why nearly all the cost of a high-end IMU goes into the gyroscope.

3. The Complete Implementation Procedure of a Complementary Filter

Figure 7-4 The Attitude Estimation Flow of a Complementary Filter

① Read the IMU: three-axis acceleration a, three-axis angular rate ω

↓

② Compute attitude from the accelerometer: $\varphi_{acc} = \operatorname{atan2}(a_y, a_z)$, $\theta_{acc} = \operatorname{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$

↓

③ Integrate the gyroscope: $\varphi_{gyro} = \varphi_{prev} + \omega_x \cdot \Delta t$ (likewise for pitch)

↓

④ Complementary fusion: $\varphi = \alpha \cdot \varphi_{gyro} + (1-\alpha) \cdot \varphi_{acc}$ (α about 0.95 to 0.99)

↓

⑤ Yaw ψ: supplied by the magnetometer (Chapter 6), or integrated from the gyroscope with its drift accepted

↓

⑥ Output the attitude (φ, θ, ψ) and update φ_{prev} and θ_{prev} for the next round

4. The Cost Function of Tightly Coupled Visual-Inertial Estimation

A tightly coupled system places all information in one optimization problem,

whose cost function has the structure:

$$\min \Sigma \| \text{visual reprojection error} \|^2 + \Sigma \| \text{IMU preintegration error} \|^2 + \| \text{prior error} \|^2$$

The first term requires the projected position of a map point in the image to be as close as possible to the feature position actually observed; the second requires the change of pose between adjacent keyframes to agree as closely as possible with what IMU preintegration predicts; and the third retains the information of earlier optimizations. The variables to be solved for include the poses, velocities, and IMU biases of all keyframes together with the coordinates of the map points.

This is exactly the full form of the graph optimization of Chapter 12—for now you need understand only one thing: tight coupling means letting the error terms of different sensors contend with one another within a single expression, so that the answer found is the most reasonable one for all the observations together.

7.8 Contemporary Developments in Inertial Sensing: When the IMU Meets Deep Learning

The IMU is a field mature to the point of dullness—the physics of MEMS has not changed in decades, and the mathematics of preintegration largely settled between 2015 and 2017. Between 2024 and 2026, however, a new line of work appeared that students should know: treating IMU error by learning.

1. The Crux of the Problem: Bias Is Not a Constant

Recall from Section 7.3 that the principal source of IMU error is bias. The traditional approach treats bias as a slowly varying random walk and estimates it with a filter or an optimizer—which is what both the EKF of Section 7.4 and ORB-SLAM3 of Section 7.7

do.

Real bias behavior is more complex, however: it varies with temperature, supply voltage, and mechanical stress, and it is entangled with the sensor's own nonlinearities. Describing it with a simple random walk model is in essence a compromise. That is exactly where deep learning enters—if the variation of bias has structure, why not let a model learn it?

2. Learning Bias Dynamics: Modeling with Neural Differential Equations

A representative 2025 work proposes modeling the continuous-time dynamics of bias directly with a neural ordinary differential equation (neural ODE) rather than treating it as a random walk. The design contains several ingenuities worth noting.

First, training requires no ground truth for the bias—which cannot be measured directly at all; the researchers instead build the loss function from pose ground truth and let the model learn the bias indirectly. Second, because IMU kinematics is defined on a matrix Lie group (a concept formally introduced in Chapter 10), the authors extend the neural differential equation into a Lie-algebraic form so that the model output satisfies the geometric constraints of rotation by construction. Third, the discrete IMU measurements are interpolated into a continuous control input by cubic Hermite splines, and a hierarchical training strategy is adopted that takes the gyroscope first and the accelerometer second.

On the EuRoC and TUM-VI datasets (the two you will meet in Section 7.6 and in the exercises) and in physical experiments, the method clearly improved the error of pure IMU integration and the overall performance of visual-inertial odometry, and showed the ability to generalize across datasets.

3. A Foundation Model for Inertial Localization

Learning-based IMU methods have long shared one weakness: a model trained on a particular dataset fails on another platform. A 2025 CVPR work attempts to resolve this along foundation-model lines—first pretraining on more than a hundred hours of multi-platform data to acquire general knowledge of motion; then fine-tuning for a new task with low-rank adaptation (LoRA), with only about 1.1 million trainable parameters; and finally performing online test-time adaptation at deployment so the model keeps learning as it runs.

The result is about a thirty-six percent improvement in absolute trajectory error over models specialized to each case, while running in real time at two hundred frames per second on a robot. This three-stage structure of pretraining, fine-tuning, and online adaptation is just like the vision-language-action models to be introduced in Chapters 33 and 34, which suggests that the subfields of robotics are converging on a single methodology.

4. How This Line of Work Should Be Regarded

This book's position is pragmatic: learning-based methods are, at present, a way to improve the quality of the IMU's input rather than a replacement for the preintegration of Section 7.5 or the tightly coupled optimization of Section 7.7. The most valuable use in practice is to place the learned model at the very front—denoise the raw readings and estimate the bias more accurately first, then feed the existing geometric system.

This division of labor has two merits: the interpretability and mathematical guarantees of the geometric system are preserved, while the learned model is responsible only for what it does best, finding in data the regularities that are hard to model. As you will see in Chapter 32, this is currently the most mature pattern for integrating AI with SLAM.

A Reminder for Students

However powerful the model, the physical fact of Section 7.3 does not change: integration must amplify error, and the t^3 term will dominate in the end.

Learned denoising can lower the bias by an order of magnitude, which means shifting the curve of Figure 7-1 to the right—not flattening it.

What can actually flatten the curve is only a drift-free external observation: vision, LiDAR, GNSS, loop closure. That is the fundamental reason SLAM exists and the entire content of the last six parts of this book.

Case Studies

Case 7-1 Submarines and Missiles: The Original Battlefield of Inertial Navigation

The first large-scale applications of inertial navigation appeared aboard the submarines and intercontinental missiles of the Cold War. A submerged submarine receives no external signal whatsoever and must hold its course for weeks on pure inertial navigation; a missile in flight cannot rely on an external signal that may be jammed and likewise can only dead-reckon on its own.

These systems used navigation-grade components such as mechanical or ring laser gyroscopes—the size of a briefcase, weighing tens of kilograms, costing millions of dollars, with bias stability a million times better than a phone IMU. Even so, a submarine must still surface periodically or correct by other means—because the mathematics of Section 7.3 is merciless: however small an error, multiplied by a long enough time it becomes a large one.

This historical comparison is highly instructive: the MPU-6050 in your hand, costing a few tens of dollars, works on exactly the same physical principles as those astronomically expensive systems, differing only in accuracy grade. And so the emphasis of this book is not on buying a better IMU but on using cleverer algorithms and sensor fusion to extract the greatest value from a cheap one—which is the greatest contribution of the past two decades of SLAM research.

Case 7-2 The Phone's Pedometer and Automatic Screen Rotation

The phone in your pocket demonstrates both the applications and the limitations of inertial sensing every day. Automatic screen rotation relies on the accelerometer to determine the direction of gravity—which is why, lying in bed with the phone held flat, it so often rotates the wrong way: gravity is then perpendicular to the screen and the roll angle is mathematically unobservable (the limitation of Section 7.2 appearing in daily life).

The pedometer shows another line of thinking: it does not integrate acceleration to obtain position (which, as Section 7.3 explains, would quickly lose accuracy) but detects periodic peaks in the acceleration signal to count steps, then multiplies by an estimated stride length to get distance. The method neatly sidesteps integration drift—rather than integrate, recognize a pattern. This is the central idea of pedestrian dead reckoning (PDR) and is often paired with the Wi-Fi fingerprint positioning of Chapter 4 and the geomagnetic fingerprint positioning of Chapter 6.

The lesson for students is important: faced with a sensor's inherent limitation, there is a route besides using a better filter—use the sensor differently, and do not ask it to do what it cannot.

Case 7-3 The EuRoC Dataset: A Common Examination Hall for Visual-Inertial SLAM

The EuRoC MAV dataset, released by a team at ETH Zürich, was recorded with a micro aerial vehicle carrying a stereo camera and an IMU through a machine hall and an industrial plant, with high-accuracy ground truth from a motion capture system and laser scanning. It has become the standard benchmark of visual-inertial SLAM—nearly every paper in the field reports its score on EuRoC.

The dataset deliberately includes sequences from easy to hard: from slow, steady flight to rapid, aggressive maneuvers with visibly blurred images. This design lets researchers see clearly under what conditions an algorithm begins to fail. The average accuracy of about 3.5 centimeters attained by ORB-SLAM3's stereo-inertial mode is its score on this benchmark.

For students the value of a public dataset is this: with no UAV and no motion capture system, a single computer is enough to run the same experiments as the world's leading research groups and compare the results. The exercises of Chapter 30 use the EuRoC dataset directly—the most concrete practice of standing on the shoulders of giants.

Hands-On Activities

Exercise 7-1 Witnessing Integration Drift First-Hand

Purpose: to experience the drift of inertial navigation through actual measurement and computation, and to implement a complementary filter and compare the result before and after fusion. This is the single most important experiment in Part II of this book—it will give you a visceral sense of why SLAM is needed.

Equipment

- A smartphone (with an app that can record and export raw sensor data), or an ESP32 or Arduino with an MPU-6050 or MPU-9250 module.
- A computer (with a Python environment including numpy and matplotlib), a tape measure, and a flat desktop.

Part One: Static Drift Test

1. Lay the device completely still and flat on the desk, record 60 seconds of raw three-axis acceleration and angular rate, and export the data.
2. Compute the bias: take the mean of the first 10 seconds. In theory, at rest the angular rate should be 0 and the acceleration should be gravity alone, so the measured departure is the bias. Record the three-axis gyroscope bias (converted to degrees per second).
3. Integration experiment (bias not removed): integrate the 60 seconds of angular rate directly into angle. The device did not move at all—by how many degrees does the plotted angle depart?
4. Integration experiment (bias removed): subtract the bias computed in step 2 and integrate again, then compare the two curves. By how much did the error fall? How much remains? What causes what remains (hint: the three sources of error in Section 7.3)?
5. Position drift: remove gravity and integrate the acceleration twice to obtain position. With the device at rest for 60 seconds, how far did the computed "position" travel? Does it agree with the theoretical estimate of Section 7.3?

Part Two: Motion Test and Filtering

6. Push the device along the desk in a straight line for 1 meter and stop, recording data throughout. Compute the distance traveled by double integration, compare it with the 1 meter measured by tape, and compute the percentage error.
7. Tilt the device slowly to about 30 degrees and return it to level, repeating several times and recording the data.
8. In Python, compute three attitude curves separately—pure gyroscope integration, pure accelerometer computation, and complementary filter fusion ($\alpha = 0.98$)—and plot them on the same axes for comparison.
9. Vibration test: tap the desk lightly to create vibration and observe how the three curves differ in response—which one jumps violently? Which is smoothest? Which is most dependable in the long run?
10. Parameter experiment: set α to 0.90, 0.95, 0.98, and 0.999 in turn, compare the smoothness and the responsiveness of the output, and identify the value you consider best, with reasons.

Discussion Questions

1. After 60 seconds at rest, by how many meters did your computed "position" drift? What would happen to a robot navigating on that result?
2. The error fell markedly once the bias was removed, which shows that bias is the principal error source. But bias varies with temperature and time—what does this imply for the practice of calibrating once at power-on and leaving it at that?
3. In the 1-meter straight-line experiment, what was your percentage error? Did the error come mainly from the acceleration phase or the deceleration phase, and why?

4. Comparing the three attitude curves, explain how the respective strengths and weaknesses of the gyroscope and the accelerometer appear concretely in the plot.
5. (Undergraduate) Using the bias values you measured, apply the formulas of Section 7.3 to estimate how long it takes for the position error to reach 1 meter. What does that interval mean for robotic system design?

Extended Challenge (Undergraduate)

First: implement the Madgwick or Mahony filter (public reference code exists for both), represent attitude by quaternion, and compare its long-run stability with the complementary filter. Second: download one sequence of the EuRoC dataset, read its IMU data and ground-truth trajectory, implement pure inertial dead reckoning and compare it with the truth, and plot the growth of error with time to verify the t^2 and t^3 theory of Section 7.3. This experiment leads directly into the ORB-SLAM3 implementation of Chapter 30.

Further Reading

1. P. D. Groves, Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, 2nd ed., Artech House, 2013. (The authoritative textbook on inertial navigation; Chapters 4 and 5 are the standard reference for IMU error models.)
2. M. Kok, J. D. Hol, and T. B. Schön, "Using Inertial Sensors for Position and Orientation Estimation," Foundations and Trends in Signal Processing, 2017. (A complete tutorial, freely available and with code examples; strongly recommended for undergraduates.)

3. C. Forster et al., "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," IEEE Transactions on Robotics, 2017. (The definitive paper on IMU preintegration theory and the principal basis of Section 7.5.)
4. C. Campos et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE Transactions on Robotics, 2021. (The principal basis of Section 7.7, dissected in full in Chapter 23.)
5. S. Madgwick, "An Efficient Orientation Filter for Inertial and Inertial/Magnetic Sensor Arrays," Technical Report, University of Bristol, 2010. (The original Madgwick filter document, with complete code.)
6. M. Burri et al., "The EuRoC Micro Aerial Vehicle Datasets," International Journal of Robotics Research, 2016. (The paper describing the EuRoC dataset.)
7. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE Transactions on Robotics, 2018. (Another classic tightly coupled visual-inertial system, worth reading alongside ORB-SLAM3.)

From Inertia to the Outside World: Where This Chapter Stands

The magnetometer and IMU introduced in Chapter 6 and this chapter are together called proprioceptive sensors—what they perceive is the robot's own state: which way I face, how fast I am turning, how great an acceleration I feel. They need no environment and can therefore work in any environment.

But they cannot see the world. The IMU does not know there is a wall ahead, and the magnetometer does not know whether this corridor has been walked before. The LiDAR of Chapter 8 and the camera of Chapter 9 belong to the other class—exteroceptive sensors, which measure the geometry and appearance of the

environment directly. The division of labor between these two classes is the basic architecture of any robotic perception system: proprioceptive sensors supply a continuous, high-rate but drifting estimate, and exteroceptive sensors supply a low-rate but stable correction.

In the next chapter we turn to LiDAR. Rather than guessing how far it has moved as an IMU does, it measures with a laser directly that the wall is three meters away. From the time-of-flight principle and the difference between 2D and 3D LiDAR to its role in cleaning robots, warehouse AGVs, and self-driving cars, Chapter 8 raises the curtain on the 2D SLAM exercises of Part IV.

Summary

1. An accelerometer measures specific force (motional acceleration minus gravity), so it reads about 9.8 at rest and 0 in free fall; a gyroscope measures angular rate using the Coriolis force, and its output must be integrated to obtain an angle.
2. An accelerometer can resolve roll and pitch from the direction of gravity but is entirely insensible of yaw, and it is contaminated by motional acceleration when the platform accelerates; Euler angles suffer gimbal lock as pitch approaches ±90 degrees, so practice mostly uses quaternions instead.
3. Integration drift is the fundamental limitation of inertial navigation: accelerometer bias makes position error grow as t^2, while gyroscope bias produces t^3 growth by the route of an attitude error leaving gravity imperfectly removed; attitude accuracy matters more than acceleration accuracy.
4. Attitude filtering fuses the gyroscope (accurate in the short term) with the accelerometer (accurate in the long term): the complementary filter is simple

and efficient but cannot estimate bias; the Kalman filter and EKF estimate bias and uncertainty together and are the mainstream choice in UAV flight control and robotics.

5. IMU preintegration integrates the high-rate measurements between two keyframes into relative quantities independent of the initial pose, so that nothing need be recomputed during optimization—the key technique that lets modern visual-inertial SLAM run in real time.
6. The IMU makes four major contributions in visual-inertial SLAM: absolute scale, the direction of gravity, filling gaps when vision fails, and high-rate output; vision gives back drift-free observations that correct the IMU bias. Tight coupling is more accurate than loose coupling.
7. ORB-SLAM3 performs tightly coupled visual-inertial SLAM on a maximum a posteriori basis throughout, IMU initialization included, with accuracy about two to ten times better than earlier methods; initialization requires sufficient motion excitation, without which scale and bias are unobservable.

In the next chapter we turn to a quite different sensor—LiDAR. Rather than guessing how far it has moved as an IMU does, it measures with a laser directly that the wall is three meters away. From the time-of-flight principle and the difference between 2D and 3D LiDAR to its role in cleaning robots, warehouse AGVs, and self-driving cars, Chapter 8 raises the curtain on the 2D SLAM exercises of Part IV.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 8

An Introduction to LiDAR

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

The magnetometer and IMU of Chapters 6 and 7 are proprioceptive sensors—they perceive the robot's own state but cannot see the world. From this chapter Part II turns to exteroceptive sensors: those that measure the geometry and appearance of the environment directly.

LiDAR is the most direct of them. It does not dead-reckon, integrate, or guess; it measures with a laser that there is something in that direction, three meters away. This character of measuring geometry directly makes it the protagonist of Parts IV and V of this book—from the occupancy grid map of Chapter 13 and Gmapping of Chapter 14 to Cartographer of Chapter 15 and the 3D point clouds and LOAM of Chapters 17 and 18, all built on the data introduced here.

The role of this chapter, then, is to lay a foundation of specifications and intuition. Having read it, you should be able to look at a LiDAR datasheet and judge at once whether that unit can see a chair leg, how fine the resulting map will be, and whether it will come to grief in a glass-curtain lobby. These judgments will be in daily use when you build maps with your own hands in Chapter 29.

Figure 8-0 How Chapter 8 Connects to the Rest of the Book

[Building on the previous chapters]

Chapters 6 and 7 Proprioceptive sensors: fast and continuous, but drifting

This chapter Exteroceptive sensors: slow and discrete, but drift-free → only together

can they eliminate accumulated error

[This chapter is responsible for]

Time-of-flight ranging → scanning → the LaserScan and PointCloud data formats

The physical meaning of the specifications: measurement range, angular resolution, scan rate, reflectivity

[Where it leads]

→ Chapter 13 LaserScan + pose → occupancy grid map (the first use of this chapter's data)

→ Chapters 14 and 15 Gmapping and Cartographer: the two mainstays of 2D LiDAR SLAM

→ Chapter 16 Cleaning robots: the full discussion of the height blind spot of Case 8-1 in this chapter

→ Chapters 17 and 18 3D point cloud registration and LiDAR odometry

→ Chapter 29 Mapping with a physical vehicle: where this chapter's judgment of specifications becomes practice

Learning Objectives

Every sensor in the previous chapters has been guessing—odometry guesses displacement, the IMU guesses motion, Wi-Fi guesses distance. The LiDAR of this chapter is different: it measures with laser light, directly, that there is something in that direction at that distance. This precise geometric measurement makes LiDAR one of the most important sensors in both 2D and 3D SLAM. After completing this chapter, you should be able to:

1. Explain the time-of-flight (ToF) ranging principle and why LiDAR can provide precise range measurement.

2. Distinguish the structural differences between 2D LiDAR and 3D multi-beam LiDAR, and the merits and drawbacks of mechanical and solid-state scanning mechanisms.
3. Describe the content of and difference between the LaserScan and PointCloud data formats.
4. Explain how angular resolution, measurement range, accuracy, and reflectivity affect LiDAR's performance in practice.
5. Give examples of the role of LiDAR in indoor mapping, warehouse AGVs, and self-driving cars.
6. Compare the strengths and weaknesses of LiDAR against the camera and the IMU, and explain why practical systems so often fuse all three.
7. Know the low-cost LiDAR platforms suited to teaching, in preparation for the 2D SLAM exercises of Part IV.

Core Concepts

Chinese Term	English Term	Explanation
光達	LiDAR	A sensor measuring distance with laser light; the name stands for light detection and ranging.
飛行時間	Time of Flight, ToF	Distance computed by multiplying the time from emission to the return of the reflection by the speed of light.
脈衝式測距	Pulsed / dToF	Emits a short pulse and measures the round-trip time directly; suited to long ranges.
調頻連續波	FMCW	Modulates the laser frequency continuously and can measure range and velocity at once.

Chinese Term	English Term	Explanation
雷射掃描	LaserScan	The data format of one revolution of range values output by a 2D LiDAR.
點雲	Point Cloud	The set of three-dimensional points output by a 3D LiDAR.
線數	Channels / Lines	The number of laser beams in the vertical direction of a 3D LiDAR, such as 16, 32, 64, or 128.
視野角	Field of View, FoV	The angular range a LiDAR can scan, horizontal and vertical.
角解析度	Angular Resolution	The angular interval between adjacent measurement points; the smaller, the denser.
反射率	Reflectivity	The proportion of laser light a surface reflects, affecting detectable range and signal strength.
固態光達	Solid-State LiDAR	A LiDAR without a large rotating mechanism, including MEMS, flash, and OPA architectures.
多普勒效應	Doppler Effect	The physical phenomenon by which FMCW LiDAR measures a target's radial velocity directly.

8.1 The Basic Principle of LiDAR: Time of Flight and Scanning

LiDAR stands for light detection and ranging. Its central idea descends directly from the GNSS of Chapter 5 and the UWB of Chapter 4: transmit a signal, measure the round-trip time, and multiply by the signal's speed to obtain distance. The only differences are that LiDAR uses laser light, over far shorter distances and with far greater accuracy.

1. Pulsed Time of Flight (dToF)

The most intuitive ranging method is direct time of flight (dToF): emit an extremely short laser pulse, time the interval t from emission to the return of the reflection, and the distance is:

$$d = (c \times t) / 2$$

where c is the speed of light and the division by 2 accounts for the light having traveled out and back. The formula is simple but the demand on timing accuracy is extreme—at the speed of light 1 nanosecond equals 30 centimeters, so centimeter accuracy requires timing to about 0.07 nanosecond. This is precisely the technical barrier of a LiDAR chip and why it is more expensive than ultrasonic or infrared ranging. The advantages of dToF are that energy is concentrated in a short pulse and long ranges can be measured, which makes it the mainstream for long-range automotive LiDAR.

2. Frequency-Modulated Continuous Wave (FMCW)

Another increasingly important method is frequency-modulated continuous wave (FMCW): rather than a pulse, a laser is emitted continuously whose frequency varies linearly with time (a chirp), and the returning signal is mixed with the signal being

emitted at that moment to measure the frequency difference (the beat frequency). That difference encodes distance and velocity at once—the most fascinating property of FMCW.

The greatest advantage of FMCW is that the Doppler effect lets it measure the radial velocity of every point directly, with no need to compare successive measurements as a pulsed system must; and because it uses coherent detection it can reject ambient light and other LiDARs almost entirely. The price is a narrow-linewidth laser source, which is difficult to manufacture and comparatively expensive, so FMCW is used mainly in premium automotive LiDAR and in research today. Recent advances in silicon photonics have realized FMCW systems with about 120 meters of detection range and 0.3 meters of accuracy on a micron-scale platform, and these are regarded as an important direction for the next generation.

3. Why Scanning Is Necessary

A single laser beam measures the distance in only one direction at a time—that is merely a point. To build the outline or the three-dimensional shape of an environment, the beam must be swept across different directions, which is the task of the scanning mechanism. The manner of scanning determines a LiDAR's form and price and is the subject of Section 8.2.

A Clarification: The Differences Among LiDAR, Radar, and Ultrasound

All three work on the same principle (emit a signal, measure the round-trip time) and differ in the wave used:

• LiDAR: laser light, short in wavelength and excellent in directionality, reaching

centimeter to millimeter accuracy and high resolution; its weaknesses are glass, mirrors, rain, and fog.

• Radar: radio waves, highly penetrating, unaffected by rain and fog, and able to measure speed; its weaknesses are low resolution and difficulty in resolving an object's shape.

• Ultrasound: sound waves, extremely cheap and dependable at short range; its weaknesses are small range, low accuracy, and susceptibility to wind and sound-absorbing materials.

Self-driving cars often use all three together, precisely for their complementarity—entirely consistent with the sensor fusion idea this book stresses repeatedly.

8.2 2D LiDAR, 3D LiDAR, and Multi-Beam Lasers

By scanning dimension, LiDAR divides into 2D and 3D; by scanning mechanism, into mechanical and solid-state. These two axes of classification determine a unit's capability, form, and price.

1. 2D LiDAR: Scanning One Plane

A 2D LiDAR rotates a single beam through one turn in a horizontal plane (commonly a 360-degree or 270-degree field of view), measures the distance in every direction in that plane, and outputs one revolution of range values. It is like a laser ruler sweeping horizontally across space at a fixed height.

The advantages of 2D LiDAR are a small data volume, light computation, and an affordable price (teaching models cost a few thousand to ten-odd thousand NT dollars),

which make it standard equipment on cleaning robots, teaching vehicles, and indoor AGVs, and the protagonist of the Gmapping and Cartographer exercises of Part IV of this book. Its inherent limitation is that it sees only one horizontal slice—mounted to scan at a height of 20 centimeters, it cannot see a tabletop at 30 centimeters and cannot detect an overhead bar or a low threshold. This limitation recurs throughout Section 8.5 and the discussion of cleaning robots in Chapter 16.

2. 3D LiDAR: Multiple Beams and Scanning

Obtaining three-dimensional information requires measuring in the vertical direction as well. 3D LiDAR achieves this chiefly in two ways:

- Multi-beam: several sets of laser emitters and receivers are stacked vertically, each set covering one elevation angle, and all rotate and scan together. Common specifications are 16, 32, 64, and 128 beams—the more beams, the denser the vertical sampling and the finer the three-dimensional reconstruction, and the higher the price. This is the mainstream for self-driving cars and autonomous patrol robots.
- Single-beam vertical scanning: an oscillating mirror or rotating prism makes a single beam (or a few beams) sweep vertically as well, covering the vertical field of view with fewer laser components and so reducing cost.

3. Mechanical versus Solid-State

Traditional LiDAR rotates the whole optical assembly or a mirror with a motor and is called mechanical. Its field of view is large and the technology mature, but its drawbacks are clear: bulk, rotating parts subject to wear, limited life in a vibrating environment, and high cost.

Solid-state LiDAR seeks to reduce or eliminate large rotating mechanisms, chiefly

in three architectures:

- MEMS: a micromirror on a chip deflects the laser beam to scan line by line. Strictly speaking this is semi-solid-state, since the tiny mirror still moves; it is currently the solid-state approach with the largest shipments.
- Flash: like a camera flash, the whole scene is illuminated at once and an array of SPAD or APD sensors receives all the returns, with no moving parts at all; the advantage is the absence of scanning latency, and the limitation is that spreading the illumination energy shortens the range.
- OPA (optical phased array): adjusting the phase of many waveguide emitters steers the beam purely electronically within microseconds, with no mechanical motion whatever; it is the most advanced technically, though its readiness for volume production is still advancing.

The overall trend in solid-state LiDAR is smaller size, greater vibration tolerance, and lower cost, and it is moving steadily from premium automotive use into robotics and consumer applications. For the reader of this book, understanding this progression from a mechanical present to a solid-state future helps in reading the bewildering array of products on the market.

8.3 LiDAR Data Formats: LaserScan and PointCloud

What a sensor measures must finally be handed to a SLAM algorithm in a particular format. LiDAR has two standard formats in the ROS ecosystem, corresponding to 2D and 3D.

1. LaserScan: The Data Format of 2D LiDAR

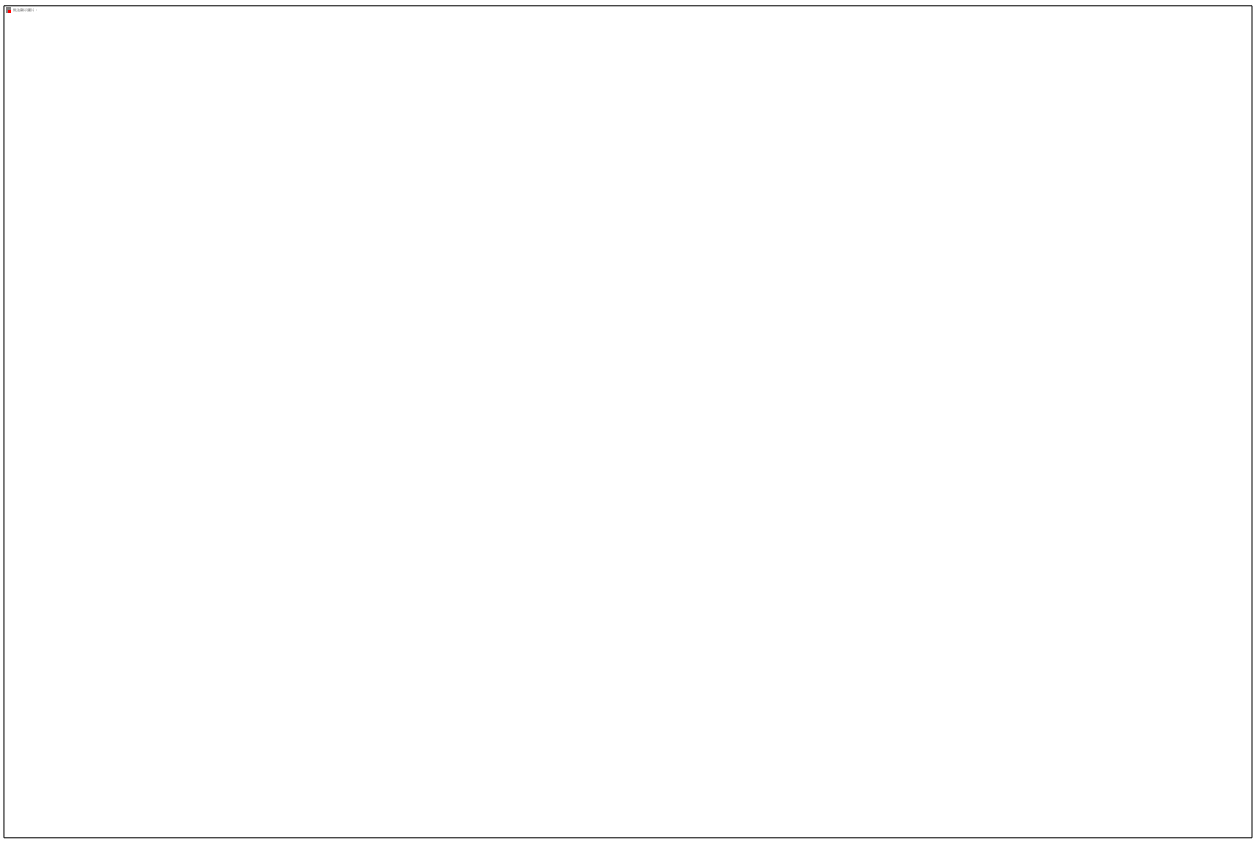
A 2D LiDAR outputs one revolution of range values, represented in ROS by the

sensor_msgs/LaserScan message. At its core is an array of ranges together with a few parameters describing the scan:

- angle_min, angle_max: the starting and ending angles of the scan.
- angle_increment: the angular interval between adjacent measurement points (that is, the angular resolution).
- range_min, range_max: the nearest and farthest trustworthy measurement distances.
- ranges[]: the array of range values—the body of the data, each element being the distance measured at the corresponding angle.
- intensities[]: the array of return intensities (optional).

Because the angle increases regularly, LaserScan need not store an angle for every point: the range array plus a starting angle and an angular interval suffices to recover each point's direction. This compact representation makes 2D data very light—one revolution is usually only a few hundred to a thousand range values. To plot a point in a coordinate frame requires only the simple conversion from polar to rectangular coordinates (the formulation in Section 8.5).

Figure 8-1 The Scan Geometry of 2D LiDAR and the LaserScan Data Structure



This figure puts every field of the LaserScan message back into its geometric meaning. The LiDAR sits at the origin and emits a beam starting at angle_min and every angle_increment thereafter until angle_max; each beam returns one range, stored in order in the ranges array. So the number ranges[i] carries no directional information in itself—the direction must be recovered from the index i: θ_i = angle_min + i × angle_increment.

This is one of the most frequent beginner's mistakes: plot the ranges array directly and you get a contorted curve rather than the shape of the room. The two conversion formulas at the bottom of the figure must be applied to turn polar (r, θ) into rectangular (x, y) before the point cloud looks like a room. You will do this with your own hands in Exercise 8-1, and the act of marking these points into cells in Chapter 13 is the first step of mapping.

The figure also deliberately shows two failure modes. The glass section of the right-hand wall lets the laser pass straight through with no return; the deep black light-absorbing material on the lower left wall absorbs nearly all the light and the return is too weak to read. The result is the same in both cases—the range value in that direction becomes infinite or zero, and in SLAM's eyes there is nothing there. An opening that should not exist appears in the map, and the robot may walk straight into a glass door.

Three Practical Countermeasures to the Glass Problem

1. Add an ultrasonic or infrared proximity sensor dedicated to detecting glass—the standard practice in cleaning robots and service robots.

2. Apply visible marking tape or an AprilTag to the glass (Section 4.6), solving recognition for people and robots at once.

3. Annotate a virtual wall on the map by hand (Nav2's keepout zone, implemented in Chapter 29) so that path planning avoids it outright.

Remember one principle: what the sensor cannot see, no algorithm however powerful can recover—an iron law that still holds when AI-assisted navigation is discussed in Chapter 32.

2. PointCloud: The Data Format of 3D LiDAR

The data of a 3D LiDAR is a great many points in three-dimensional space, represented in ROS by the sensor_msgs/PointCloud2 message. Unlike LaserScan, every point stores its own three-dimensional coordinates (x, y, z) directly, with optional additional fields for return intensity, beam index, time stamp, and the like.

The data volume of a point cloud is far larger than that of a LaserScan: a 64-beam LiDAR can produce more than a million points per second. This raises two engineering

issues—heavy transmission and storage requirements, and the downsampling usually needed before further processing (the voxel grid of Chapter 17). Point cloud processing has a dedicated open-source library, PCL (the Point Cloud Library), used extensively in Part V.

Characteristic	LaserScan (2D)	PointCloud2 (3D)
Data dimension	One revolution of range values (planar)	A set of three-dimensional points (volumetric)
Volume per message	Hundreds to a thousand points	Tens of thousands to millions of points
Point representation	Range + regular angle	x, y, z stored directly
Computational burden	Light	Heavy; usually needs downsampling
Typical source	A rotating 2D LiDAR	A multi-beam or solid-state 3D LiDAR
Use in this book	2D SLAM in Chapters 13 through 16	3D SLAM in Chapters 17 and 18

Table 8-1 Comparison of the LaserScan and PointCloud2 Data Formats

8.4 LiDAR Resolution, Angular Resolution, Measurement Range, and Reflectivity

Selecting or evaluating a LiDAR requires reading its datasheet. This section explains the four most critical parameters, which directly determine the quality of a SLAM map.

1. Measurement Range and Accuracy

- Measurement range: the nearest and farthest measurable distances. Teaching-grade 2D LiDARs commonly span 0.15 to 12 meters, while long-range automotive units reach beyond 200 meters.
- Accuracy: how close a measurement is to the true distance, commonly specified as ±2 to ±3 centimeters.
- Precision: the consistency of repeated measurements of the same target. Accuracy and precision are two different things—a ruler that always reads 5 centimeters long is inaccurate yet may be very precise, and a systematic bias can be calibrated out.

2. Angular Resolution: How Fine the Map Can Be

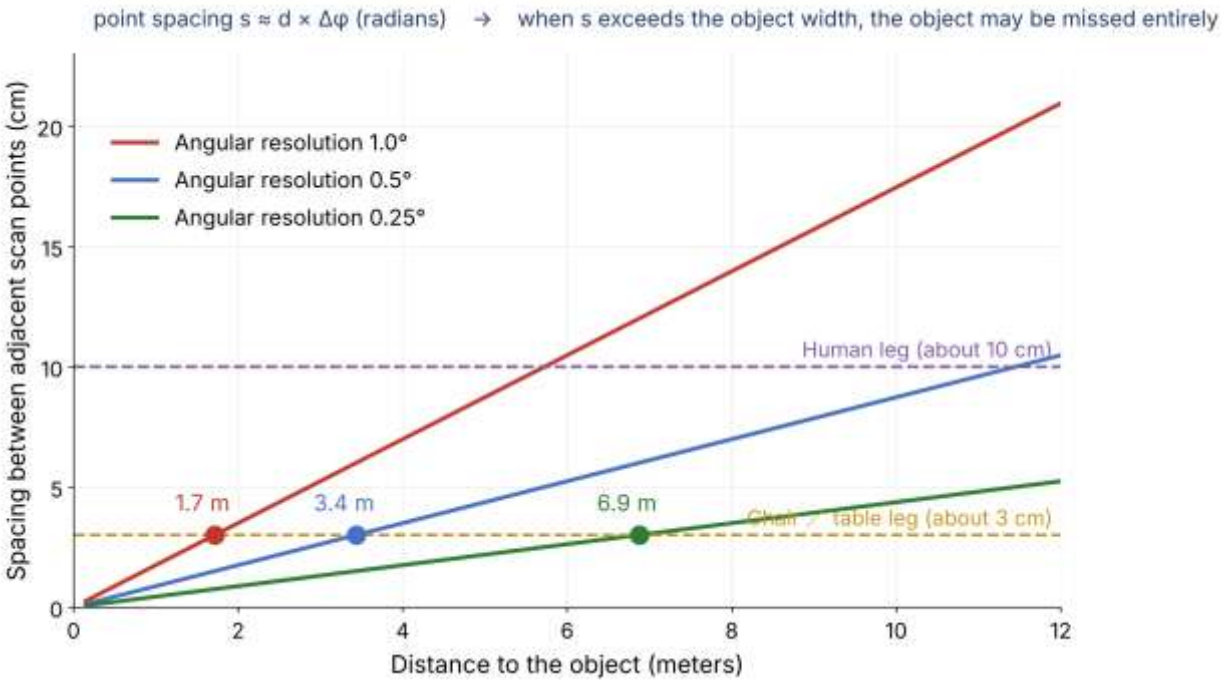
Angular resolution is the angular interval between adjacent measurement points, and it determines how fine a structure the map can represent. A key geometric fact lies here: for the same angular resolution, the farther away, the greater the actual spacing between adjacent points.

$$\textit{spacing between adjacent points} \approx \textit{distance} \times \textit{angular resolution (in radians)}$$

For example: at an angular resolution of 1 degree and a distance of 5 meters, adjacent points are about 8.7 centimeters apart—which means a chair leg 5 centimeters wide at 5 meters may fall entirely between two beams and go completely undetected. This explains why small distant objects are easily missed by LiDAR and is the key to the discussion in Section 8.5 and in Chapter 16 of why a cleaning robot runs into table legs.

Figure 8-2 The Relation Among Angular Resolution, Distance, and Detectable

Object Size



Angular resolution is the number on a datasheet most easily skipped over and most decisive of map quality. Its effect is simple: the spacing between adjacent scan points grows linearly with distance, $s \approx d \times \Delta\varphi$. But put concrete numbers in and the conclusion is striking.

Take a chair leg 3 centimeters wide. An entry-level LiDAR with 1-degree angular resolution is guaranteed to catch at least one point on it only within 1.7 meters; 0.5 degree extends that to 3.4 meters; and 0.25 degree to 6.9 meters. In other words, in the same classroom with the same chair, changing the LiDAR may turn four clear points on the map into nothing at all.

This explains several things seen in practice: why maps built with a cheap LiDAR have straight walls but fragmented furniture; why cleaning robots so often run into chair legs (the height blind spot of Case 8-1 is only one reason, and angular resolution is another); and why the occupancy grid map of Chapter 13 needs accumulated observations—a small object missed in one scan is recovered by moving closer and scanning again.

When purchasing or reading a datasheet you may treat this figure as a lookup table: decide first how small an object you must detect and at what distance, then work back to the angular resolution required. This habit of inferring specifications from requirements is the same engineering discipline as the indoor positioning selection procedure of Section 4.7.

3. Scan Rate

The scan rate (10 Hz means ten revolutions per second) affects the quality of mapping while the robot moves. If the rate is too low and the robot moves quickly, the robot has traveled some distance between the start and the end of one revolution, producing motion distortion—a problem that must be handled explicitly in the 3D LiDAR SLAM of Chapter 18.

4. Reflectivity: An Overlooked but Critical Variable

LiDAR depends on the target to reflect the laser back, so the target's reflectivity is crucial. A white wall is highly reflective and easy to detect; a black object absorbs most of the laser and reflects weakly, which shortens the detectable range sharply—a LiDAR rated at 12 meters may manage only a few meters against a black sofa. More troublesome still:

- Mirrors and glass: the laser is reflected elsewhere rather than back, so the LiDAR sees through the glass or measures a virtual image in the mirror, producing errors in the map—one of the most common causes of failure in indoor LiDAR SLAM.
- Highly reflective objects: metal and retroreflective signs may produce excessively strong returns or multiple reflections.

- Liquid surfaces: water and polished floors may reflect specularly and cause missed measurements.

The maximum range on a LiDAR datasheet is therefore usually quoted under a particular reflectivity (10% or 90%, say)—be sure to note that premise when reading specifications, or actual performance will fall well short of expectation.

8.5 LiDAR in Indoor Mapping, Warehouse AGVs, and Self-Driving Cars

1. Indoor Mapping and Service Robots

2D LiDAR is the mainstay of indoor mapping. Cleaning robots, food-delivery robots, and guide robots mostly build occupancy grid maps with one horizontal 2D LiDAR and Gmapping or Cartographer (Part IV). Indoor environments are clearly structured (walls, doors, corridors), and LiDAR's precise geometric measurement builds crisp, stable maps—something visual methods can hardly match against white walls or in dim light. But the glass partitions, mirrors, and single-height slice described above also defeat pure 2D LiDAR repeatedly in modern glass-curtain buildings, so in practice ultrasonic or infrared sensors are often added to fill the gap.

2. Warehouse AGVs and Factory Automation

The warehouse automated guided vehicle (AGV) is one of LiDAR's most mature industrial applications. Besides SLAM navigation, LiDAR doubles as a safety guard—many industrial units carry safety certification and can trigger slowing or stopping the instant a person enters a hazardous area. A factory environment is comparatively controlled (level floors, steady lighting, a known layout), which lets LiDAR achieve its greatest reliability. This echoes the idea of the Amazon case in Chapter 4: the more

controllable the environment, the simpler the technology selection can be.

3. Self-Driving Cars

3D LiDAR is the core sensor of most self-driving cars (Tesla's purely visual approach being the famous exception). It provides precise three-dimensional structure for obstacle detection, determination of drivable area, and localization by matching against a high-definition map (the urban canyon case of Chapter 5). The demands a self-driving car places on LiDAR are the most severe of all: long range (to see far ahead at highway speed), high resolution (to tell a pedestrian from a utility pole), a high update rate, and strict automotive-grade reliability. That is why automotive LiDAR still carries a high unit price (a few hundred US dollars each) and why it is evolving aggressively toward solid-state and FMCW designs.

4. Other Emerging Applications

- Surveying and spatial scanning: backpack or handheld LiDAR scans buildings, historic sites, and construction sites to produce high-accuracy 3D models (Chapters 15 and 17).
- Agriculture and forestry: UAV-borne LiDAR penetrates the canopy to measure tree height and terrain.
- Quadrupeds and patrol: 3D LiDAR tightly coupled with an IMU for autonomous patrol indoors and out without GNSS (Chapter 18).

8.6 Comparing LiDAR with the Camera and the IMU

Placing LiDAR in the panorama of Part II's sensors shows its position more clearly. Table 1-1 of Chapter 1 already gave an overview; here we make a deeper comparison for the tasks of mapping and localization.

Item of Comparison	LiDAR	Camera	IMU
Measures directly	Distance (geometry)	Brightness / color	Acceleration / angular rate
Depth information	Direct and precise	Must be inferred (no scale in monocular)	None
Affected by light	Almost not at all (active light source)	Severely (depends on ambient light)	Not at all
Semantic information	None (shape only)	Rich (objects recognizable)	None
Data volume	Medium to large	Large	Small
Cost	Medium to high	Low	Low to medium
Principal weakness	Glass and mirrors, rain and fog, no semantics	Light, no scale, no depth	Drift

Table 8-2 LiDAR, Camera, and IMU Compared on Mapping and Localization Tasks

The Logic of Their Complementarity

The table shows clearly why the three are naturally complementary:

- LiDAR gives precise geometry and depth but cannot tell what a thing is, and it fears glass and mirrors.
- The camera gives rich semantics and color but no dependable depth or scale, and it fears darkness and backlighting.
- The IMU gives high-rate motion information to fill the gaps but drifts.

The most robust systems therefore often use all three: LiDAR SLAM builds a precise geometric skeleton, the camera supplies semantic labeling and loop closure

recognition, and the IMU compensates during LiDAR motion distortion and rapid movement. The LiDAR-inertial tight coupling (LIO) of Chapter 18 is the representative of LiDAR plus IMU, and the semantic SLAM of Chapter 32 the representative of LiDAR (or camera) plus AI.

LiDAR versus Vision: A Debate with No Standard Answer

In the self-driving field, whether LiDAR should be used at all was once a fiercely contested question. Advocates of pure vision argue that humans drive with eyes alone, and that cameras are cheap, carry semantics, and can read signs; advocates of LiDAR argue that the dependable depth it provides is essential safety redundancy and should not be sacrificed to save cost. The debate remains unsettled, but it offers an important lesson: sensor selection is never a purely technical question and involves cost, safety, regulation, and business strategy as well. As an engineer, understanding the boundaries of each sensor's capability is what allows a well-founded judgment in a concrete situation—which is the common purpose of all seven chapters of Part II.

8.7 Low-Cost LiDAR and Teaching Robot Platforms

The good news is that LiDAR has become cheap enough to enter the classroom. This section introduces LiDARs and platforms suited to teaching, in preparation for the exercises of Part IV.

1. Entry-Level 2D LiDAR

The spread of low-cost 2D LiDAR in recent years is what has allowed SLAM education to reach undergraduates and even high school students. Such units mostly use triangulation or low-cost ToF principles, scan 360 degrees with a range of about 8 to 12 meters, start at a few thousand NT dollars, and come with complete ROS drivers—

plug one in and it publishes LaserScan, ready to feed directly to Gmapping or Cartographer. Their accuracy and stability fall short of industrial models but are more than adequate for teaching.

2. Teaching Robot Platforms

- The TurtleBot family: the official standard teaching platform of ROS, carrying a 2D LiDAR and a complete software stack; it is the most widely used platform in SLAM teaching worldwide, with abundant tutorial material online. The exercises of Chapter 29 in this book take it as the principal reference.
- A self-built ROS vehicle: assembled from a differential-drive chassis, motor encoders, and a microcontroller with a low-cost 2D LiDAR and a single-board computer (a Raspberry Pi or Jetson Nano, say). The cost is lower and the learning deeper, which suits undergraduate projects—students handle every link themselves, from the TF frames of Chapter 10 to publishing odometry in Chapter 29.
- A simulation platform: no hardware at all. A virtual LiDAR vehicle can be placed in a simulation environment such as Gazebo or Foxglove to run complete SLAM experiments (Chapter 28). This lets high school courses and large classes give everyone hands-on work, and it is the entry route this book strongly recommends.

3. Advanced: Teaching with 3D LiDAR

Low-beam-count 3D LiDARs (16 beams, say) have also fallen sharply in price in recent years and, paired with an edge computing platform such as a Jetson, can already serve undergraduate 3D SLAM projects (Chapters 17 and 18) and university autonomous patrol research. For high school and introductory undergraduate work,

however, starting from 2D LiDAR or simulation remains advisable: build the core concepts of SLAM firmly first, then enter the world of 3D.

Whichever platform is chosen, this book's advice is the same: understand the pipeline in simulation first, verify it with cheap hardware next, and pursue accuracy and scale only last. Every exercise in Part IV marks out both a simulation route and a hardware route so that students with differing resources can all take part. In the next chapter we take leave of LiDAR's active ranging and turn to the sensor richest in information and most in need of mathematical treatment—the camera—laying the ground for the visual SLAM of Part VI.

Formulations and Procedures

1. Time-of-Flight Ranging

The basic ranging formula of pulsed LiDAR:

$$d = (c \times t) / 2$$

where $c \approx 3\times10^8$ m/s is the speed of light, t the round-trip time of the pulse, and the division by 2 accounts for the out-and-back path. From it the requirement on timing accuracy follows:

$$\text{range error } \Delta d = (c \times \Delta t) / 2$$

Reaching 1 centimeter (0.01 m) of accuracy requires $\Delta t \approx 0.067$ nanosecond—which shows the technical challenge of a LiDAR timing circuit and explains why an accurate LiDAR is far from cheap.

2. Coordinate Conversion of a LaserScan Point

The i-th measurement of a 2D LiDAR is given in polar coordinates (range r_i, angle

θ_i), where the angle follows from the scan parameters:

$$\theta_i = angle_min + i \times angle_increment$$

Converted to rectangular coordinates in the LiDAR frame (for mapping and display):

$$x_i = r_i \times \cos\theta_i$$

$$y_i = r_i \times \sin\theta_i$$

This simple polar-to-rectangular conversion is the first step by which every 2D SLAM system turns a revolution of range values into a revolution of points on a map. The scan matching of Chapter 13 and Gmapping of Chapter 14 are both built upon it.

3. Angular Resolution and Detectable Object Size

The spacing of two adjacent beams at distance d determines the smallest object size that can be detected:

$$point\ spacing\ s \approx d \times \Delta\varphi \quad (\Delta\varphi \text{ being the angular resolution in radians})$$

An object narrower than s may fall entirely between two beams and be missed. For instance, with $\Delta\varphi = 1°$ (≈ 0.0175 radian) and $d = 5$ m, $s \approx 8.7$ cm. This expression is the key to understanding how finely a LiDAR can see and quantifies the phenomenon that small distant objects are easily missed.

4. The LiDAR Data Processing Pipeline

Figure 8-3 The Data Flow of LiDAR from Emission to SLAM Input

① The laser emits a pulse or a frequency-modulated wave

↓

② The target reflects and the sensor receives the return

↓

③ Compute the time of flight → range d (dToF), or the beat frequency → range + velocity (FMCW)

↓

④ The scanning mechanism supplies the angle → the direction of each point

↓

⑤ Assemble the data: 2D → LaserScan (range + angle) / 3D → PointCloud (x, y, z)

↓

⑥ Convert polar to rectangular coordinates → hand to the SLAM front end (scan matching, feature extraction)

8.8 The Contemporary Upheaval in LiDAR: Collapsing Prices and the Move to Solid State

The technical architecture described in the first seven sections of this chapter is quite stable, but the industry itself is undergoing violent change. What students most need to know is not some new principle but a simple fact: LiDAR is turning from expensive research equipment into a cheap standard component.

1. Three Orders of Magnitude in a Decade

Line up a few dates and the trend is unmistakable. In 2016 and 2017 the 64-beam mechanical LiDAR commonly used in self-driving research cost about eighty thousand US dollars per unit; in 2020 a leading solid-state product cost about nineteen hundred dollars; in 2022 one mainstream product fell to about five hundred dollars; and by 2026 the same family had entered volume production below two hundred dollars, with an average price in the Chinese market of about one hundred forty-five dollars. Other manufacturers are targeting under two hundred dollars and planning a push toward one hundred.

What drives the change is manufacturing scale, in-house production of key components, and capacity expansion—a trajectory just like the cost curves of solar modules and smartphone camera modules before it. One major manufacturer has begun volume delivery against orders for more than four million units and plans to raise annual capacity to four million.

The significance for teaching and research is immediate: the threshold for the low-cost LiDAR described in Section 8.7 keeps falling. The 3D LiDAR that only a research laboratory could afford five years ago is now within the budget of an undergraduate project or even a high school club. The 3D LiDAR SLAM of Chapters 17 and 18 has gone from something understood by reading papers to something you can actually try.

2. Shipment Mix: Robotics Is Catching Up with Automotive

The numbers also show a change on the application side. Take one major manufacturer's 2025 shipments: about 1.62 million units for the year, more than tripling year on year; of these about 1.38 million went to advanced driver assistance systems and about 239,000 to robotics—with the latter growing more than fourfold year on year, far faster than automotive. The company further projects total shipments of three

to three and a half million units in 2026.

Robotic LiDAR growing faster than automotive reflects the spread of cleaning robots, delivery robots, warehouse AGVs, and patrol robots. These are exactly the applications that Parts IV, V, and VIII of this book address—what you are studying happens to be the segment whose demand is growing fastest.

3. Going Solid State: From Spinning to Not Spinning

Section 8.2 compared mechanical and solid-state designs. In industry that transition is now largely settled: the price collapse has occurred almost entirely in solid-state architectures, because having no rotating mechanism means fewer precision parts, longer life, smaller size, and the possibility of mass production by semiconductor processes.

For a system designer the shift brings one conceptual adjustment. A mechanical LiDAR offers a full 360-degree field of view, while a solid-state unit usually covers only a forward sector. This means a single sensor can no longer see everything—and the remedy is to fit several. Extrinsic calibration of multiple sensors (their relative poses) therefore changes from an occasional requirement into a subject every robot must handle. The point cloud registration of Chapter 17 and the coordinate transforms of Chapter 10 are the tools for exactly this.

4. Frequency-Modulated Continuous Wave and "4D LiDAR"

The FMCW architecture mentioned in Section 8.1 is regarded as the direction of the next generation. Its key advantage is measuring radial velocity directly while measuring range—each point carries a velocity component in addition to x, y, and z, which is why it is often called 4D LiDAR.

Its significance for SLAM is worth pondering. From Chapter 17 onward one difficulty recurs: dynamic objects (people walking, vehicles driving) contaminate registration and make the robot believe it has moved. If the sensor itself can tell you which points are in motion, rejecting dynamic objects changes from a difficult algorithmic problem into a simple threshold test. FMCW is also naturally resistant to interference from sunlight and from other LiDARs—a practical matter in an era when LiDARs are about to fill the streets.

Three Judgments for Students

1. Do not design systems on old prices. The argument that LiDAR is too expensive so we will use a camera holds over a far narrower range in 2026 than it did five years ago.

2. But cheap does not mean suitable. The price collapse has occurred mainly in forward-facing solid-state units of automotive specification; the small 360-degree LiDAR an indoor robot needs is still another market.

3. The physical meaning of a specification does not change with price. The angular resolution limit of Figure 8-2 and the failures on glass and black surfaces hold just as firmly on a two-hundred-dollar LiDAR as on an eighty-thousand-dollar one.

Case Studies

Case 8-1 The Cleaning Robot's Invisible Trap: The Height Blind Spot of 2D LiDAR

The cleaning robot in many a student's home has played out the same drama: it deftly skirts the sofa and the walls yet always runs headlong into the legs of the dining

chairs, or gets stuck at the edge of the baseboard. The robot is not stupid; the inherent limitation of 2D LiDAR is showing itself.

A 2D LiDAR mounted at some fixed height on top of the robot (8 centimeters, say) scans horizontally and can see obstacles only in that one slice. A slender chair leg may occupy only one or two beams at that height, and if it is also somewhat far away (the angular resolution problem of Section 8.4) it may fall entirely into the gap between beams and be missed. Lower obstacles still (cables, thresholds, a pet bowl) are not in the scan plane at all and the LiDAR is wholly insensible of them.

This case vividly explains why modern cleaning robots have taken to adding other sensors—infrared or ultrasonic obstacle detection at the front, cliff sensors underneath, even a 3D ToF camera instead (Chapter 16). It also foreshadows an important idea: a SLAM map contains only the world the sensor can see, and the physical limitations of the sensor directly determine the completeness of the map and the safety of navigation.

Case 8-2 The LiDAR Dispute in Self-Driving: The Divergence of Waymo and Tesla

A famous divergence of technical approach exists in the self-driving industry. The mainstream camp represented by Waymo covers the roof and body with several 3D LiDARs and holds that the dependable depth LiDAR provides is the cornerstone of safety; the other camp, represented by Tesla, follows a purely visual route and holds that enough cameras plus powerful AI can drive by sight as a human does, at far lower cost and with mass production in view.

Both sides have their reasons: the LiDAR camp stresses poor lighting and safety redundancy, the vision camp cost, semantic understanding, and scalability. As

automotive LiDAR continues to fall in price and go solid-state and as visual AI continues to advance, the debate keeps evolving. For the reader of this book the point is not who is right but reading the trade-off behind it: this is the best teaching material for the idea in Section 8.6 that sensor selection is a combined judgment of technology, cost, safety, and business.

Case 8-3 Backpack 3D Scanning: Giving a Building a Digital Twin

Put a multi-beam 3D LiDAR, an IMU, and a computing unit in a backpack, have an operator walk through a building wearing it, and a three-dimensional point cloud model of the whole structure is built in real time—this is backpack mobile mapping, an application Google's Cartographer system demonstrated from the start (Chapter 15).

As the operator walks corridors, stairs, and rooms, the system stitches each instant's LiDAR scan together in real time by SLAM, finally producing a high-accuracy digital twin usable for building information modeling (BIM), indoor navigation, and heritage preservation. This case threads several themes of the book together: ToF ranging (this chapter), IMU compensation of scan distortion (Chapter 7), 3D point cloud processing (Chapter 17), and loop closure correction (Chapter 15). It also shows that the value of SLAM lies not only in making a robot move but in letting people digitize real space with unprecedented efficiency—a direction with great potential for university field projects.

Hands-On Activities

Exercise 8-1 Visualizing LiDAR Data and Reading a Datasheet

Purpose: using real or simulated LiDAR data, to understand the structure of LaserScan, to perform the polar-to-rectangular conversion by hand so that a revolution of range

values becomes the outline of the environment, and to learn to read a LiDAR datasheet. This exercise offers both a simulation route and a hardware route, suiting classes with different resources.

Route A: Simulation (no hardware required; suits a whole class)

1. Install ROS 2 and the TurtleBot3 simulation environment on a computer (see Chapter 28 for detailed steps, or follow the official tutorial) and launch the Gazebo world and the virtual vehicle.
2. Display the /scan topic in RViz and observe how the red points of the LiDAR scan hug the walls as the vehicle moves.
3. Inspect the content of the LaserScan message from the command line, find angle_min, angle_increment, range_max, and the rest, record them, and compute this virtual LiDAR's field of view and total point count.
4. Record a rosbag and export one revolution of the ranges array.

Route B: Hardware (for classes with a low-cost 2D LiDAR)

5. Connect the 2D LiDAR to a computer, start its ROS driver, and confirm that the /scan topic is publishing normally.
6. Set the LiDAR still in the center of the classroom and use RViz to observe whether the scan outline hugs the four walls and the desks and chairs.
7. Place a white cardboard sheet, a black backpack, and a sheet of glass (or a mirror) in turn 2 meters in front of the LiDAR, compare how the three appear in RViz, and record the anomalies caused by the glass and the mirror.
8. Record and export one revolution of scan data for the next step's analysis.

Common Part: Data Analysis (Python)

9. Read in the exported ranges array and convert each (range, angle) pair into rectangular (x, y) coordinates by the formulation of Section 8.5.
10. Plot all the points with matplotlib and observe whether the plan outline of the environment is reproduced (walls, desks, the doorway).
11. Compute and mark: what is the actual spacing between adjacent points for this LiDAR at 3 meters and at 6 meters? Discuss the effect on detecting small objects.
12. Reading a datasheet: find the datasheet of a real LiDAR (teaching-grade or automotive), and identify its measurement range, angular resolution, scan rate, and the reflectivity premise under which the maximum range is quoted.

Discussion Questions

1. In the point cloud outline you plotted, where is it clearest? Where are there gaps or stray points? What in the environment do the gaps most likely correspond to (glass? a black object? too far away?)?
2. (Route B) What anomalies did the glass and the mirror produce in the LiDAR data? If a cleaning robot mapped with this LiDAR, what would happen when it met a floor-to-ceiling window?
3. How much do the point spacings at 3 meters and at 6 meters differ? To detect a 5-centimeter-wide chair leg dependably, within what distance can this LiDAR manage it?
4. Based on the comparison in this chapter (Table 8-2), if you were choosing the primary sensor for a campus service robot, would you choose LiDAR or a camera? Why? What sensor would you need to add?

Extended Challenge (Undergraduate)

First: write a ROS 2 node that subscribes to /scan, converts LaserScan into a point cloud in real time and publishes it, and displays the raw scan and the converted result together in RViz. Second: implement a simple nearest-obstacle alert—find the minimum range and its direction in each revolution and issue a warning when it falls below a safety threshold; this is the prototype of the obstacle avoidance of Chapter 29.

Further Reading

1. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011. (The sensor material in Chapter 4 contains a systematic introduction to ranging principles.)
2. T. Raj et al., "A Survey on LiDAR Scanning Mechanisms," Electronics, 2020. (A complete comparison of scanning mechanisms—mechanical, MEMS, flash, and OPA.)
3. You Li and Javier Ibanez-Guzman, "Lidar for Autonomous Driving," IEEE Signal Processing Magazine, 2020. (An authoritative survey of automotive LiDAR principles and systems.)
4. The official ROS documentation of the sensor_msgs/LaserScan and sensor_msgs/PointCloud2 message definitions. (The first-hand specification of the data formats.)
5. The official Point Cloud Library (PCL) tutorials (pointclouds.org). (The standard tool for 3D point cloud processing, used in the exercises of Part V.)
6. The technical documentation of the SLAMTEC RPLIDAR family and the Livox family. (Specifications and ROS driver examples for entry-level 2D and low-cost 3D LiDAR.)

7. The ROBOTIS TurtleBot3 e-Manual (emanual.robotis.com). (Complete implementation guidance for the teaching platform, with SLAM examples.)

From Range to Image: Where This Chapter Stands

LiDAR has given the robot a nearly ideal geometric sense: direct, precise, and indifferent to illumination. But it has three essential limitations. First, it sees only geometry and not color, text, or semantics—it knows there is a rectangular solid ahead but not whether it is a door or a cabinet. Second, it degenerates in a featureless corridor or an empty hall: every direction looks the same and scan matching has no way to tell how far it has traveled (a problem formally discussed in Section 17.5). Third, it is expensive, heavy, and power-hungry—though as Section 8.8 described, that is changing fast.

The camera of Chapter 9 is the opposite on all three counts: it is cheap, light, and frugal with power, and it is rich in semantic information—color, texture, text, object class. The price is that it cannot measure absolute scale (monocular), it fears darkness and backlighting, and it fears white walls. The complementarity of these two sensors is the heart of Table 8-2 in Section 8.6 and the reason Parts V and VI take LiDAR and vision respectively as their protagonists.

And the fusion route represented by ORB-SLAM3 in Chapter 23 and LIO-SAM in Chapter 18 states the industry's real answer: where the budget allows, fit both. Chapter 32 goes further still—when a vision-language model can understand that a thing is a door, LiDAR's geometric precision and the camera's semantic understanding will become one, forming a navigation system that truly understands what people say.

Summary

1. LiDAR measures distance by time of flight: the pulsed form (dToF) measures the round-trip time directly, and its concentrated energy suits long ranges; FMCW modulates frequency continuously and can measure range and velocity at once with good interference rejection, though at higher cost, and it is an important direction for the next generation.
2. A 2D LiDAR scans a single horizontal plane with light data and a low price and is the mainstay of indoor mapping; a 3D LiDAR obtains three-dimensional information by multiple beams or vertical scanning; scanning mechanisms are moving from mechanical to solid-state schemes such as MEMS, flash, and OPA, trending smaller, more vibration-tolerant, and cheaper.
3. A 2D LiDAR outputs LaserScan (a range array plus regular angles: compact and light) and a 3D LiDAR outputs PointCloud (three-dimensional coordinates stored directly: voluminous and in need of downsampling); the two correspond to Parts IV and V of this book respectively.
4. The key specifications include measurement range, accuracy and precision, angular resolution (which determines the smallest detectable object, with points growing sparser at greater distance for the same angular resolution), scan rate (too low a rate causes motion distortion), and reflectivity (black objects and glass or mirrors being the principal causes of failure).
5. LiDAR is widely used in indoor mapping, warehouse AGVs (doubling as a safety guard), and self-driving cars (three-dimensional perception and map-based localization), as well as in emerging fields such as backpack surveying, agricultural and forestry measurement, and quadruped patrol.
6. LiDAR gives precise geometry but no semantics and fears glass and mirrors; the camera gives rich semantics but no dependable depth or scale; the IMU gives

high-rate motion but drifts; the three are naturally complementary and the most robust systems often use them together.

7. Low-cost 2D LiDAR and teaching platforms such as TurtleBot, together with wholly free simulation environments, have allowed SLAM education to reach undergraduates and high school students; this book recommends the learning path of simulation first, cheap hardware next, and accuracy last.

In the next chapter we turn to the sensor richest in information—the camera. From the pinhole camera model, lens distortion, and calibration to the differences among monocular, stereo, and RGB-D cameras, Chapter 9 explains how an image becomes geometric information rather than a heap of pixels, laying the foundation for the visual SLAM of Part VI.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 9

Cameras and Visual Navigation Sensors

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

The LiDAR of Chapter 8 measures geometry directly, precisely and dependably. The camera of this chapter is exactly the opposite—it measures no distance at all, yet it brings what LiDAR can never give: color, texture, text, and semantics.

This is the last chapter of Part II and the turning point of the whole book. The sensors accumulated over the first eight chapters are now all present: absolute position (Chapters 4 and 5), absolute direction (Chapter 6), high-rate motion (Chapter 7), precise geometry (Chapter 8), and rich semantics (this chapter). Every one of them has its moments of lying, and not one can accomplish navigation alone.

What Part III teaches, therefore, is the mathematics of fusing these mutually contradictory readings into the single most reasonable answer—coordinates and pose in Chapter 10, probability and state estimation in Chapter 11, and optimization and graph theory in Chapter 12. The content of this chapter is especially crucial: the pinhole model and the intrinsic matrix K are the bedrock of the whole of Part VI's visual SLAM, and K appears in every expression of the epipolar geometry of Chapter 21, ORB-SLAM2 of Chapter 22, and ORB-SLAM3 of Chapter 23.

Figure 9-0 How Chapter 9 Connects to the Rest of the Book

[Building on the previous chapter]

Chapter 8 LiDAR: geometrically precise, but without semantics, degenerate in corridors, and costly

This chapter The camera: cheap, light, and frugal with power, and rich in semantics, but

without scale and afraid of darkness and white walls

[This chapter is responsible for]

The pinhole model → the intrinsic matrix K → distortion and calibration

The three ways a camera obtains depth: monocular, stereo, RGB-D

The four raw materials of vision: feature points, edges, optical flow, depth

[Where it leads]

→ Section 4.6 The K that AprilTag pose solution requires, completed here

→ Chapter 20 Feature-based and direct methods: the full unfolding of Section 9.6 of this chapter

→ Chapter 21 Epipolar geometry: motion estimation between two images

→ Chapters 22 and 23 ORB-SLAM2 and ORB-SLAM3

→ Chapter 30 Mapping on real hardware with RGB-D and stereo cameras

→ Chapters 32 and 34 Semantic SLAM and VLM language navigation: the full version of Section 9.7 of this chapter

Learning Objectives

The camera is the richest sensor in information and the cheapest—one image contains shape, texture, color, and semantics at once. But an image is in essence a three-dimensional world flattened into two, and recovering the lost depth is precisely the central challenge of visual SLAM. This is the last chapter of Part II and the entrance to the visual SLAM of Part VI. After completing this chapter, you should be able to:

1. Explain the pinhole camera model and write the mathematical relation by which a three-dimensional point projects into a two-dimensional image.

2. Explain the meaning of camera intrinsics and extrinsics, and account for the cause of lens distortion and the purpose of camera calibration.
3. Compare the ways monocular, stereo, and RGB-D cameras obtain depth, with the merits and drawbacks of each.
4. Describe the differences and uses of three kinds of visual information: feature points, edges, and optical flow.
5. Analyze the advantages and limitations of visual navigation relative to LiDAR, the scale ambiguity problem above all.
6. Describe the line of knowledge that leads from image processing into visual SLAM.
7. Gain a first acquaintance with vision-language models (VLMs) and the concept of semantic navigation, in preparation for Chapter 32.

Core Concepts

Chinese Term	English Term	Explanation
針孔相機模型	Pinhole Camera Model	The geometric model describing how a three-dimensional point projects through the optical center onto the image plane.
焦距	Focal Length	The distance from the optical center to the image plane, which sets magnification and field of view.
相機內參	Intrinsic Parameters	The camera's own imaging parameters—focal length, principal point, and so on—which form the intrinsic matrix K.
相機外參	Extrinsic Parameters	The camera's position and orientation in world coordinates (rotation R and

Chinese Term	English Term	Explanation
		translation t).
鏡頭畸變	Lens Distortion	Image deformation caused by the lens, divided into radial and tangential distortion.
相機校正	Camera Calibration	The procedure for measuring and estimating the intrinsics and the distortion coefficients.
視差	Disparity	The horizontal displacement of the same point between the left and right images, used for stereo depth.
基線	Baseline	The distance between the two lenses of a stereo camera, which affects depth range and accuracy.
特徵點	Feature / Keypoint	A salient point in an image that is easy to recognize and repeatably detectable, such as a corner.
光流	Optical Flow	The movement of pixel brightness over time in an image, which reflects motion.
尺度不確定性	Scale Ambiguity	The inherent limitation that a monocular camera cannot determine true size.
視覺語言模型	VLM	An AI model able to understand images and language at once, used in semantic navigation.

9.1 The Pinhole Camera Model

Understanding how a camera sees the world requires first understanding an old and elegant model—the pinhole camera. Its prototype is the camera obscura: a small hole in one face of a sealed box through which light from outside projects an inverted image on the opposite inner wall. This seemingly simple device contains the core geometry of all camera imaging.

The Geometry of Projection

Place the pinhole (the optical center) at the origin with the optical axis along Z pointing forward. Light from a three-dimensional point P = (X, Y, Z) in space passes through the optical center and strikes an image plane at focal length f. By similar triangles the coordinates of the projected point are:

$$x = f \cdot X / Z$$

$$y = f \cdot Y / Z$$

Simple as they are, these two expressions lay bare the source of every difficulty in visual SLAM—note that the denominator in each is Z, the depth. This means:

- Three dimensions are flattened into two: of the three coordinates (X, Y, Z), only two (x, y) survive projection, and the dimension of depth Z has been divided away.
- Depth information is lost: a small object at 2 meters and a large object at 4 meters may occupy exactly the same size in the image. From one image alone, a near small thing cannot be distinguished from a far large one.

This is the geometric essence of the scale ambiguity discussed in depth in Section 9.5. Recovering that divided-away Z is the common goal of stereo cameras, RGB-D

cameras, and all the techniques of visual SLAM.

From Physical Coordinates to Pixel Coordinates

The (x, y) computed above are physical coordinates on the image plane (in millimeters). But a digital image uses pixel coordinates—origin at the top left, units in pixels. The conversion needs three things: the scale that converts millimeters to pixels (fx and fy, the focal length expressed in pixels) and the position of the image center (the principal point) (cx, cy). The complete projection relation can be written in matrix form:

$$[u\ v\ 1]^T = (1/Z) \cdot K \cdot [X\ Y\ Z]^T$$

where K is called the camera intrinsic matrix:

$$K = [\,fx\ 0\ cx;\ 0\ fy\ cy;\ 0\ 0\ 1\,]$$

This matrix K is the most basic and most frequently appearing object in visual SLAM. The AprilTag solution of Chapter 4, the epipolar geometry of Chapter 21, and ORB-SLAM of Chapter 22 all use it. And the values in K must be obtained by camera calibration (Section 9.2).

Figure 9-1 The Pinhole Camera Model: From a Point in Space to a Pixel



The left panel is the starting point of all visual geometry, and it is in fact nothing but a pair of similar triangles. The distance from the optical center to the image plane is the

focal length f and the distance from the optical center to the point in space is the depth Z; the two triangles are similar, so x / f = X / Z, that is, x = f·X / Z. All the complicated mathematics of visual SLAM traces back, in the end, to these two lines.

Note in particular that Z in the denominator. It is the source of every difficulty in this chapter and indeed in the whole of Part VI: Z is divided away in projection, so the same pixel may come from a small thing nearby or a large thing far off. This is the scale ambiguity of Section 9.3, and the reason the IMU of Chapter 7 matters so much to monocular vision—the IMU measures acceleration in physical units and can restore the lost scale.

The right panel deals with something else: a camera outputs pixels, not meters. The origin of pixel coordinates is at the top left of the image, and the point the optical axis passes through (the principal point) is usually not exactly at the center—the assembly of sensor and lens always has some offset. Translations cx and cy must therefore be added, and fx and fy used to convert physical units into pixels (the two may differ slightly, since pixels are not necessarily square).

Arrange these four numbers fx, fy, cx, cy into a matrix and you have the intrinsic matrix K. So K is nothing abstract—it is the four numbers describing how this camera flattens the world into pixels. And the calibration of Section 9.2 does exactly this: it measures those four numbers (plus the distortion coefficients).

A Distinction Often Confused

The intrinsics describe the camera itself: focal length, principal point, distortion.

So long as the lens is not changed and the focus not adjusted, they stay fixed.

The extrinsics describe where the camera is: the rotation R and translation t

relative to the world frame. The moment the robot moves, they change.

What visual SLAM solves for is precisely the extrinsics—the R and t of every frame are the robot's pose. The intrinsics must be calibrated beforehand, or every extrinsic solved for will be wrong.

This division of labor has exactly the same structure as the magnetometer calibration of Section 6.4: correct the sensor itself first, then speak of its relation to the world.

9.2 Lens Distortion and Camera Calibration

The pinhole model assumes light travels in perfect straight lines, but a real camera uses a glass or plastic lens. A lens gathers more light and makes the image brighter and sharper, but it also introduces distortion—geometric deformation of the image.

Two Kinds of Distortion

- Radial distortion: caused by the shape of the lens, growing more severe with distance from the image center. Barrel distortion bows straight lines outward (pronounced in wide-angle and fisheye lenses) and pincushion distortion bows them inward. Photograph a brick wall with a phone's wide-angle lens and the mortar lines near the edges bend visibly—that is radial distortion.
- Tangential distortion: caused by the lens and the sensor not being mounted perfectly parallel, making some regions of the image appear skewed and stretched. It is usually milder than radial distortion.

Distortion is fatal to visual SLAM: all of SLAM's geometric computation assumes the pinhole model (straight-line projection), and if distortion is not corrected first, a line that is truly straight is curved in the image, every geometric inference based on lines

and angles goes systematically wrong, and the resulting map comes out warped.

Camera Calibration: Finding K and the Distortion Coefficients

The purpose of camera calibration is to measure this camera's intrinsic matrix K and its distortion coefficients. The most classic method is the checkerboard calibration proposed by Zhengyou Zhang in 2000, whose principle descends directly from the PnP of AprilTag in Chapter 4:

1. Prepare a checkerboard calibration target of known dimensions (the side length of each square known).
2. With the camera to be calibrated, photograph the checkerboard ten to twenty times from different angles and distances.
3. The algorithm automatically detects the pixel coordinates of the checkerboard corners in each image.
4. Since the true three-dimensional coordinates of the checkerboard corners are known (regularly arranged on a plane) and their image coordinates have been measured, one can infer the K and the distortion coefficients that best fit these correspondences (another least-squares optimization problem).
5. Use the recovered distortion coefficients to undistort the images, correcting the curved image back into conformity with the pinhole model.

OpenCV has complete calibration tools built in (calibrateCamera and related functions), and ROS has the camera_calibration package—wave the camera at a checkerboard and it completes automatically. This is the first step of every visual SLAM implementation and will certainly be used when implementing RGB-D SLAM in Chapter 30.

A Practical Reminder: Calibration Is Not Done Once and for All

• Changing the lens, adjusting the focus, or even a knock that shifts the lens invalidates the old calibration, and it must be redone.

• Calibration images should cover every region of the frame and a variety of tilt angles—and above all do not forget the corners, where radial distortion is most severe.

• The calibration error (the reprojection error) should be below 1 pixel to count as good; if it is too large, usually the checkerboard shots were not varied enough or corners were detected wrongly.

• Industrial cameras often ship with calibration parameters, but for applications demanding high accuracy it is still advisable to calibrate and verify them yourself.

9.3 Monocular, Stereo, and RGB-D Cameras

Section 9.1 explained that a single image loses depth. Three mainstream routes recover it, corresponding to three kinds of camera.

1. The Monocular Camera: Simplest and Hardest

One camera, one image. The hardware is the simplest, cheapest, most frugal with power, and most ubiquitous (every phone has one). But as described, a single image cannot give depth directly—which brings the famous scale ambiguity (Section 9.5).

How, then, does a monocular camera do SLAM? The answer is by using motion: as the camera moves it photographs the same scene from different positions, much as a person can sense distance with one eye closed by rocking the head side to side. From

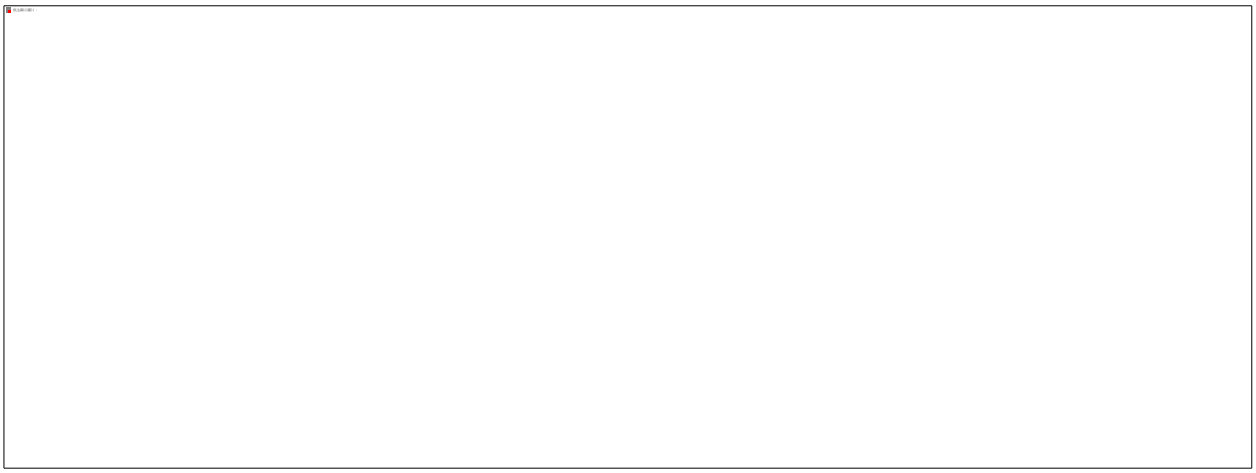
the changing parallax across successive frames, depth can be triangulated—but that depth has only relative proportion and no absolute scale (there is no telling whether it is a real meter or a centimeter in a doll's house) unless other information is brought in (an IMU, a known object size, or wheel odometry). This is precisely the value of the IMU of Chapter 7 in supplying absolute scale to monocular vision.

2. The Stereo Camera: Imitating Two Eyes

Two cameras are mounted side by side at a fixed, known separation (the baseline) and photograph simultaneously. The same point in space falls at different horizontal positions in the left and right images, and the difference is called the disparity. Near objects have large disparity and far objects small—hold a finger before your eyes and close each eye in turn and the finger jumps a long way against the background, while a distant mountain barely moves; this is the same principle.

From the disparity together with the known baseline and focal length, depth follows directly by triangular geometry (the formulation in Section 9.6). The advantages of stereo are that a single (left-right) pair yields depth, that it has absolute scale, and that it works indoors and out. The limitations are that the relative positions of the two cameras must be calibrated precisely, that it fails on untextured surfaces (a white wall) where no correspondence can be found, and that depth accuracy falls off rapidly with distance (the shorter the baseline, the faster the falloff).

Figure 9-2 The Geometry of Stereo Depth and the Growth of Depth Error



The left panel shows why stereo depth works. The same point in space falls at different horizontal positions in the left and right images, and that difference is the disparity d. The nearer the point, the larger the disparity; the farther, the smaller. Similar triangles give $Z = f \cdot b / d$—focal length and baseline are known constants, so measuring the disparity is measuring the depth.

The right panel shows its limitation, and this is the thing a datasheet does not show and yet decides whether the system succeeds. Differentiating $Z = f \cdot b / d$ with respect to d gives $\Delta Z \approx (Z^2 / f \cdot b) \cdot \Delta d$: depth error grows with the square of distance. With a focal length of 700 pixels and a disparity measurement error of 0.2 pixel, a small stereo module with a 6-centimeter baseline already has 10 centimeters of depth error at 4.6 meters; stretch the baseline to 12 centimeters and that extends to 6.5 meters; 25 centimeters reaches 9.4 meters.

This explains three things seen in practice. First, why the stereo cameras on a self-driving car have such wide baselines—because they must see a hundred meters ahead. Second, why the dual lenses of a phone can only produce portrait-depth effects within a few meters (Case 9-1) and cannot be used for navigation. Third, why RGB-D cameras are nearly always labeled with an effective range of 0.3 to 5 meters—beyond that the

error becomes unusable.

When choosing a camera you may treat this figure as a lookup table: decide first at what distance you need what accuracy, then work back to the baseline required. This process of inferring specifications from requirements is exactly isomorphic to the judgment of LiDAR angular resolution in Section 8.4—the physical principles of the two sensors differ, but the engineering reasoning is one and the same.

3. The RGB-D Camera: Active Depth Measurement

An RGB-D camera outputs a color image (RGB) and the depth (D) of every pixel at once. It obtains depth actively, by two mainstream technologies:

- Structured light: a known infrared pattern (a dense array of dots, say) is projected onto the scene and depth is inferred from how far the pattern is deformed. The first-generation Microsoft Kinect used this method; accuracy is high at short range, but it fails in strong light and the range is short.
- Active IR stereo: essentially stereo ranging, but with an additional projected infrared texture pattern to solve the problem of finding no correspondence on untextured surfaces such as white walls. The Intel RealSense D400 family uses this method and works indoors and out.

The greatest advantage of an RGB-D camera is dense depth out of the box, with no need to perform complicated depth estimation oneself, which makes it the ideal input for indoor RGB-D SLAM (RTAB-Map and ORB-SLAM3 in Chapter 30). The limitations are that most depend on infrared and so perform poorly outdoors in strong sunlight (the sun's infrared swamps the projected pattern), that the effective range is usually 0.2 to 10 meters, and that they are equally helpless against glass and mirrors (the same weakness as the LiDAR of Chapter 8).

Type	How Depth Is Obtained	Absolute Scale	Principal Advantage	Principal Limitation
Monocular	Motion + triangulation	None (must be supplied externally)	Cheapest, most ubiquitous, most frugal with power	Scale ambiguity; requires motion
Stereo	Triangular geometry from left-right disparity	Yes	Depth from a single frame; works indoors and out	Needs calibration; fears untextured surfaces; falls off at distance
RGB-D	Structured light / active IR stereo	Yes	Outputs dense depth directly	Fears strong light; limited range; fears glass

Table 9-1 Comparison of How Three Kinds of Camera Obtain Depth

9.4 Feature Points, Edges, Optical Flow, and Depth Estimation

What a camera gives is a whole image of several hundred thousand pixels. But SLAM cannot, and need not, process every pixel; it extracts the useful information from the image. This section introduces the three most important kinds of visual information.

1. Feature Points (Keypoints): The Landmarks of Visual SLAM

A feature point is a salient point in the image that is easy to recognize and still recognizable from another angle—usually a corner (where two edges meet), such as the corner of a window frame, the junction of floor tiles, or the corner of a letter on a poster.

Why are corners preferred? Imagine choosing a point on a white wall and trying to find it again in the next frame—everywhere on a white wall looks the same and it cannot

be found at all; choose a point on the edge of a straight line and the perpendicular direction can be determined but the position along the line cannot (this is called the aperture problem); only a corner is unique in both directions and can be relocated precisely. This is the bedrock of feature-based visual SLAM (Chapters 20 and 22): treat corners as landmarks in the environment and track their movement across successive images to infer the camera's motion. ORB, SIFT, SURF, and FAST are all well-known methods of feature detection and description, treated in detail in Chapter 20.

2. Edges: The Outlines of Structure

An edge is a place of abrupt brightness change in the image, usually corresponding to the outline of an object or the junction of different surfaces. Edge information plays an important part in some direct-method and line-feature SLAM systems, especially in man-made environments poor in feature points (an empty corridor, a scene with nothing but wall lines), where lines are instead the stable cue.

3. Optical Flow: The Motion of Pixels

Optical flow describes the movement of brightness patterns in an image over time—as the camera or an object moves, the texture in the frame flows. Picture raindrops streaming backward on a car window: that is optical flow visualized. Optical flow can be used to estimate motion, track features, and detect dynamic objects. It is the core of direct-method visual odometry and the key technique in UAV position hold (optical flow positioning, Chapter 26).

4. Depth Estimation

Besides the stereo disparity and the active RGB-D sensing described above, deep learning has recently brought a breakthrough in monocular depth estimation: a neural network is trained to guess the relative depth of every pixel from a single image. Such

models (Depth Anything, mentioned in Chapter 32, for example) exploit prior knowledge learned from a great many images—they know that the sky is far away, that a floor recedes into the distance, that a person is bigger than an ant. Although it is guessing and mostly yields only relative depth, it is already practical enough for many applications and is an important complement to traditional geometric methods.

These four kinds of information are not mutually exclusive: modern visual SLAM commonly mixes them—feature points for a robust localization skeleton, optical flow for fast tracking, and depth (whether from stereo, RGB-D, or a neural network) to reconstruct a dense map.

9.5 The Advantages and Limitations of Visual Navigation

Placing the camera in the panorama of Part II's sensors (Table 8-2 of Chapter 8 already made a first comparison), this section goes deeper into the advantages and the soft spots peculiar to visual methods.

The Distinctive Advantages of Visual Navigation

- Richest in information: one image contains shape, texture, color, and semantics at once. LiDAR and the IMU cannot give this at all—LiDAR knows only that there is something three meters away, while a camera can know that it is a red door. This is precisely why the semantic navigation of Chapter 32 is built on vision.
- Lowest cost and most ubiquitous: a camera is one to two orders of magnitude cheaper than a LiDAR, is everywhere, and is mature in volume production.
- Passive sensing and low power: unlike LiDAR, which must emit a laser actively, a camera only receives ambient light, which saves power and avoids mutual

interference—particularly suited to UAVs and micro robots (Chapters 25 and 26).

- Light: for a UAV severely constrained in payload and power, a camera plus an IMU is the ideal combination.

The Inherent Limitations of Visual Navigation

- Scale ambiguity: the most fundamental soft spot of a monocular camera. As Section 9.1 explained, projection divides depth away, a monocular image cannot know absolute size, and scale must be supplied by an IMU, a stereo baseline, or an object of known size.
- Dependence on light and texture: the most fatal weakness of visual methods. Darkness, backlighting, and overexposure blind the camera; untextured surfaces such as white walls and plain-colored floors yield no feature points; and image blur from rapid motion also breaks tracking. LiDAR, by contrast, carries its own light source and does not fear the dark.
- Computational burden: image data is voluminous, and feature extraction and matching require considerable computing power—which is where edge computing platforms come in (the Jetson of Chapter 33).
- Fear of dynamic scenes: people and vehicles moving through the frame disturb visual SLAM, which assumes a static environment.

A Summary of the Idea: Vision and LiDAR as Complements

LiDAR: precise geometry, carries its own light and does not fear the dark, but has no semantics, fears glass and mirrors, and is more expensive.

The camera: rich semantics, cheap and light, but no dependable scale or depth,

and fears darkness and untextured surfaces.

Their weaknesses hardly overlap—which is exactly the fusion logic of "LiDAR builds the geometric skeleton, the camera applies the semantic labels" (Chapter 32),

and confirms once more the central conviction of Part II: there is no all-powerful sensor, only an appropriate combination.

9.6 From Image Processing to Visual SLAM

Many students have studied image processing before taking this course—filtering, edge detection, thresholding, morphological operations, and so on. This section explains how those fundamentals lead step by step to visual SLAM, joining the knowledge of the two courses together.

The Ladder of Knowledge

6. Image processing: operating on the pixels of a single image—removing noise, enhancing contrast, detecting edges. This is the level of making an image clearer.
7. Feature extraction: pulling corners and descriptors out of an image, condensing a heap of pixels into a few identifiable points. This is the material of Chapter 20.
8. Multiple-view geometry: analyzing the geometric relations among several images of the same scene and inferring camera motion and scene structure from the movement of feature points. This is the epipolar geometry of Chapter 21.

9. Visual odometry (VO): chaining the previous step frame by frame to estimate the camera trajectory continuously—but with accumulating drift.
10. Visual SLAM: adding mapping, loop closure detection, and global optimization on top of VO to eliminate long-term drift and obtain a consistent trajectory and map. This is the subject of Part VI.

This ladder echoes the order of exposition in the UZH course of Chapter 2: image formation → features → multiple-view geometry → visual odometry → SLAM. Each rung is the foundation of the next and none may be skipped. This book treats image processing as prerequisite knowledge and enters visual SLAM formally from the features of Chapter 20; but the camera model and calibration laid down in this chapter are the premise of understanding everything that follows.

Two Schools: Feature-Based and Direct

Visual SLAM has two main technical routes; the concepts are established here and detailed in Chapter 20:

- Feature-based: extract and match feature points and solve for motion from their geometric relations. Robust and mature, with the ORB-SLAM family as its representative.
- Direct: extract no features and estimate motion directly from pixel brightness change (the optical flow idea). It has the advantage in weakly textured scenes but is sensitive to changes in lighting, with LSD-SLAM and DSO as its representatives.

9.7 A First Look at VLMs, AI Vision Models, and Semantic Navigation

This section previews Part IX (Chapters 32–34, AI semantics and language navigation). Traditional visual SLAM answers a geometric question: where is the camera and what shape is the environment? An AI vision model can further answer a semantic question: what is in the frame and what does it mean? This is precisely the camera's most precious value relative to LiDAR—it carries semantics.

From Geometry to Semantics

A robot doing only traditional SLAM builds a map that says there is a wall here and an obstacle there; with AI visual recognition added, the map is upgraded to this is a door, that is a staircase, over there is a fire extinguisher—a semantic map (mentioned in Chapters 1 and 3). This upgrade lets a robot carry out tasks at the level of human language.

Key AI Vision Models (detailed in Chapter 32)

- YOLO: real-time object detection, drawing boxes in the image around this is a person, that is a chair; the mainstay of campus scene recognition (a focus of the supplementary material and the university course).
- SAM (Segment Anything): segmenting an image precisely into regions, object by object.
- Depth Anything: estimating depth from a single image (mentioned in Section 9.4).
- CLIP and DINO: understanding the semantic content of an image and linking images with text.

VLMs: Letting a Robot Both See and Understand

A vision-language model (VLM) can understand images and natural language at once. This lets a robot accept spoken tasks: "walk to the whiteboard," "avoid the

backpack on the floor," "help me find the elevator"—the VLM analyzes the camera view, understands the language instruction, and converts the task into a navigation goal.

This is the paradigm shift now under way in robot navigation: from given coordinates, go there to understand what people say, understand the scene, and decide autonomously how to go. It is also why this chapter stands at the end of Part II—the camera is not just one more ranging sensor but the robot's window onto understanding the world. Chapter 32 unfolds this exciting direction in full, including how to collect and annotate training data, how to train a campus scene recognition model, and how to integrate a VLM with ROS 2.

Part II, "Sensors for Localization and Navigation," is now complete. You have met the robot's six major senses: odometry and the IMU perceive its own motion (Chapter 7), radio and satellites and the geomagnetic field supply absolute references (Chapters 5 and 6), indoor positioning technologies fill the indoor gap (Chapter 4), LiDAR measures geometry precisely with a laser (Chapter 8), and the camera brings the richest visual and semantic information (this chapter). In the next part we turn from what the sensors give to how the mathematics handles this noisy data—entering the mathematical foundations of SLAM.

Formulations and Procedures

1. The Complete Pinhole Projection Equation

The position of a three-dimensional point P_w in world coordinates is transformed into camera coordinates by the extrinsics (rotation R, translation t) and then projected into pixel coordinates by the intrinsics K, the complete relation being:

$$s \cdot [u\ v\ 1]^T = K \cdot [R \mid t] \cdot [X_w\ Y_w\ Z_w\ 1]^T$$

where s is the scale factor (equal to that point's depth Z in the camera frame). This expression threads the three central concepts of this chapter together: the intrinsics K (9.1), the extrinsics [R|t] (the camera's pose in the world, which is exactly what SLAM solves for), and the scale s (the source of the scale ambiguity of 9.5). All of visual SLAM is, in essence, solving for the camera poses [R|t] and the three-dimensional point coordinates simultaneously across a great many such projection equations.

2. The Stereo Depth Formula

Depth computation with a stereo camera is the most intuitive formula in visual ranging. With baseline b, focal length f (in pixels), and disparity d (in pixels) for a point between the left and right images, the depth Z is:

$$Z = f \cdot b \,/\, d$$

This expression reveals three important facts:

- The larger the disparity d (a near object), the smaller the depth Z; as the disparity approaches 0 (a far object) the depth approaches infinity—so stereo cannot measure distant objects accurately.
- The larger the baseline b, the greater the disparity at a given distance, the more accurate the depth, and the farther the measurable range (which is why the RealSense D455 enlarged its baseline to improve accuracy at distance).
- Depth error is proportional to the square of distance: double the distance and the error quadruples. This is the common fate of all passive visual depth measurement.

3. The Growth of Depth Error with Distance

For a disparity measurement error Δd, the depth error is approximately:

$$\Delta Z \approx (Z^2 / (f \cdot b)) \cdot \Delta d$$

The Z^2 in this expression explains why stereo and RGB-D cameras all advertise accuracy at short range and rapid falloff at long. For instance, a camera with 1 centimeter of error at 1 meter may have as much as 16 centimeters at 4 meters. This expression is an important basis for choosing a camera and setting the working range of a robot.

4. The Processing Pipeline of a Camera in Visual SLAM

Figure 9-3 The Data Flow from Camera to Visual SLAM

① The camera captures an image (monocular / stereo / RGB-D)

↓

② Undistortion: correct the image with the distortion coefficients from calibration (9.2)

↓

③ Feature extraction: detect corners and compute descriptors (9.4, Chapter 20)

↓

④ Feature matching / optical flow tracking: find corresponding points between adjacent frames

↓

⑤ Geometric solution: infer camera motion from the correspondences and the intrinsics K (Chapter 21)

↓

⑥ Triangulation / depth: reconstruct three-dimensional points (monocular needs motion, stereo uses disparity, RGB-D gives it directly)

↓

⑦ Back-end optimization + loop closure: eliminate drift and output trajectory and map (Chapters 22 and 23)

9.8 The Contemporary Upheaval in Visual Sensing: When Geometry Is Replaced by Models

This chapter has been about geometry: the pinhole model, the intrinsic matrix, the

disparity formula. These expressions have hardly changed since they took shape in the 1990s and remain the foundation of every system in Part VI. But between 2024 and 2026 something happened in computer vision that may rewrite that foundation, and students must know of it.

1. Monocular Depth Is No Longer an Impossible Task

Section 9.3 said a monocular camera cannot obtain depth, and in the purely geometric sense that remains correct—one image genuinely cannot determine scale mathematically. But recent depth estimation foundation models accomplish something else: they learn from a great deal of data how deep this kind of scene ought to look.

What they rely on is not geometry but prior knowledge: a door is about two meters tall, a corridor about one and a half meters wide, a person about one meter seventy. Having seen enough of the world, a model can produce a quite reasonable depth map from a single photograph, even an absolute depth in meters.

The significance for robots is practical: the RGB-D camera of Chapter 30 fails in strong sunlight (noted in Section 9.3), while a monocular depth model is not so limited; it can also supply a scale reference for initializing the monocular SLAM of Chapter 20. But keep its nature firmly in mind—it is a guess, not a measurement. Confronted with a scene it has never seen (a doll's house, a cavernous exhibition hall), it will give a wrong answer with confidence.

2. VGGT: Replacing the Whole SfM Pipeline with One Model

A more fundamental shock comes from the visual geometry grounded transformer presented in the best paper at CVPR 2025. Its approach is extremely direct: given one

to several hundred images, a single forward pass of the model outputs camera intrinsics and extrinsics, depth maps, point clouds, and three-dimensional point tracks all at once—the whole process in under a second.

Contrast this with the traditional pipeline. Obtaining the same result conventionally requires detecting feature points (Section 9.4) → matching → estimating epipolar geometry (Chapter 21) → triangulation → running bundle adjustment (Chapter 12) in repeated iterations. That pipeline is what Part VI needs four chapters to cover. And the research shows that, without any geometric optimization as post-processing, the model's performance already exceeds that of traditional methods which do post-process.

Must Part VI then still be studied? This book's answer is emphatic: yes, and all the more so. There are three reasons. First, such models currently target offline reconstruction rather than real-time SLAM, and what a robot needs is continuous localization thirty times a second, not a one-off scene reconstruction. Second, they have no loop closure, no long-term consistency, and no uncertainty output—and those three things are the core of SLAM. Third, and most important: someone who does not understand the geometry cannot tell when the model is talking nonsense.

3. The Event Camera: A Sensor That Takes Another Road Entirely

The event camera also deserves mention. A conventional camera outputs complete frames at a fixed rate; an event camera lets every pixel operate independently—reporting an event only when the brightness change exceeds a threshold, with time resolution down to the microsecond and dynamic range far beyond an ordinary camera.

This answers exactly the two great visual limitations listed in Section 9.5: motion blur

and violent changes in illumination. A conventional camera photographs a blur while a UAV tumbles at speed, whereas an event camera still tracks clearly; a conventional camera is wholly overexposed the instant one bursts out of a tunnel into daylight, while an event camera is barely affected. This line of work is mentioned again in Chapter 26 on UAV visual-inertial navigation.

The price is that its output is not an image but a stream of events, so none of the algorithms of Section 9.4 can be applied directly and all must be redesigned. This keeps it largely in the research domain to this day—but in high-speed, high-dynamic-range settings it is currently the only answer.

Three Judgments for Students

1. Geometry is not obsolete. A pose output by a model must still be interpreted with the K of this chapter, must still be expressed in the coordinate frames of Chapter 10, and must still pass through the optimization of Chapter 12 to be globally consistent.

2. The model's most valuable place is the front end. Use it to replace the fragile handcrafted steps—feature matching, monocular initialization, depth priors—while keeping the geometric optimization of the back end. This agrees exactly with the conclusion on learning-based IMUs in Section 7.8.

3. Judgment comes from geometry. Someone who knows that $\Delta Z \propto Z^2$ will know to be suspicious when a model says the wall is fifty meters away; someone who does not will simply accept it.

Case Studies

Case 9-1 Phone Portrait Mode: The Everyday Magic of Two Lenses

The dual-lens phone in your pocket demonstrates the principles of this chapter every day. Turn on portrait mode and the subject is sharp while the background is softly blurred—behind that effect is stereo depth measurement: the two lenses photograph simultaneously across a tiny baseline, the phone computes the disparity of every pixel to estimate a depth map, decides which pixels are the near subject and which the far background, and then applies a simulated bokeh blur to the background.

This case also exposes the limitation of stereo. Portrait mode often fails at the edge of hair, on openwork objects, or against a plain background—because disparity is hard to estimate accurately there (no texture, detail too fine). This is the everyday embodiment of what Section 9.3 said about stereo fearing untextured surfaces. In recent years phones have moved to a hybrid of two-lens disparity, AI semantic segmentation, and neural network depth estimation, which answers exactly to the trend of mixing several kinds of visual information noted in Section 9.4. Next time you use portrait mode, observe where it goes wrong—it is the most personal experiment in visual depth there is.

Case 9-2 The Mars Helicopter Ingenuity: Interplanetary Navigation on Vision Alone

NASA's Mars helicopter Ingenuity completed dozens of flights on Mars, making history as the first powered flight on another planet. There is no GPS on Mars and no usable magnetic compass (Mars has almost no global magnetic field), so how did it know where it was and fly so steadily?

The answer is visual-inertial navigation: a downward-facing monocular camera under the belly photographed the ground at tens of frames per second, tracking

feature points on rocks and surface texture (Section 9.4), with an IMU to estimate motion—precisely the technique the UZH research group mentioned in Chapter 2 excels at and the core of the UAV navigation of Chapter 26. The thin Martian atmosphere (about 1% of Earth's density) makes flight extremely unstable and places severe demands on the immediacy and robustness of navigation. Ingenuity's success proved one thing: in an extreme environment with no external signal whatever, a cheap camera plus an IMU can carry dependable autonomous navigation. This is also why this book attaches such importance to visual-inertial methods—they are the key to a robot leaving infrastructure behind and becoming truly autonomous.

Case 9-3 The Curse of the White Wall: A Classic Failure of Visual SLAM

A scene played out repeatedly in the laboratory: a student eagerly runs visual SLAM on a camera-equipped robot in the classroom, and everything goes well—until the robot turns toward a clean white wall, the system loses track at once, and the map collapses.

The cause is just what Sections 9.4 and 9.5 stressed repeatedly: there is not a single feature point on a white wall, every part of what the camera sees looks the same, and the system has no way to judge whether it has moved or how far. Along with the IMU's need for motion excitation at initialization in Chapter 7 and LiDAR's fear of glass in Chapter 8, this is one of SLAM's three classic environmental traps.

The remedies echo the fusion thinking of the whole book: add an IMU and dead-reckon through the few seconds in which vision has lost track (Chapter 7); or switch to LiDAR, for which a wall is instead a crisp plane; or project texture (the active-infrared approach of RGB-D). The teaching value of this case is that it lets students feel directly

how a sensor's limitation becomes a system's failure—understanding the limitations matters more than admiring the capabilities.

Hands-On Activities

Exercise 9-1 Camera Calibration and a First Experience of Visual Depth

Purpose: to carry out a camera calibration by hand, observe the effect of distortion correction, and experience feature point detection and stereo depth measurement. This exercise can be done with an ordinary webcam or a phone and is a warm-up for the visual SLAM exercises of Chapter 30.

Equipment

- A webcam or phone camera, and a computer (with a Python and OpenCV environment).
- A printed checkerboard calibration target (downloadable from the OpenCV site; mount it on a rigid board to keep it flat).
- (Optional) A stereo or RGB-D camera (a RealSense, say) for the advanced depth work.

Part One: Calibration and Undistortion

1. Photograph the checkerboard 15 to 20 times from different angles and distances, covering as much of the frame and as many tilts as possible.
2. Use OpenCV's findChessboardCorners to detect the corners and calibrateCamera to obtain the intrinsic matrix K and the distortion coefficients, and record them.
3. Examine the K matrix: are fx and fy close to each other? Are cx and cy about half the image width and height? Consider the physical meaning of these numbers.

4. Use the recovered parameters to undistort an image containing straight lines (a brick wall or a bookshelf, say), place the before and after side by side, and observe whether the curved lines near the edges have been straightened.
5. Check whether the reprojection error is below 1 pixel; if it is too large, add more calibration images or reshoot and try again.

Part Two: Feature Point Detection

6. Using OpenCV's ORB detector, extract and draw feature points on an image of the classroom or a desktop, and observe where they concentrate (corners, richly textured areas) and where they are absent (white walls, plain-colored desktops).
7. Photograph the same scene twice (moving the camera slightly), find corresponding points by feature matching and draw the connecting lines, and observe which matches are correct and which are wrong.
8. Deliberately photograph a large expanse of white wall and observe that ORB finds almost no feature points—verifying "the curse of the white wall" first-hand (Case 9-3).

Part Three: Experiencing Depth (choose according to equipment)

9. With an RGB-D camera: read and display the depth map (far and near rendered as a color scale), point the camera at a desktop and then into the distance and observe how the depth values change; then point it at glass or a mirror and observe the missing or erroneous depth data.
10. With a monocular camera only: photograph an object of known size (a sheet of A4 paper, say) at various distances, record its size in pixels in the image, and

verify the pinhole projection relation that doubling the distance halves the pixel size (Section 9.1).

Discussion Questions

1. How pronounced is the change in the straight lines near the image edges before and after undistortion? If you ran visual SLAM without undistorting first, what problems would you expect in the map?
2. Where in your classroom do the ORB feature points concentrate? Which areas have almost none? What does this suggest about where a robot should look so as not to lose track?
3. (With RGB-D) What went wrong with the depth data on glass and mirrors? Is this the same as the weakness of LiDAR in Chapter 8? Explain the common cause behind both.
4. Based on Table 9-1 of this chapter, would you choose a monocular, stereo, or RGB-D camera for an indoor service robot? Would you pair it with an IMU? Give your reasons.

Extended Challenge (Undergraduate)

First: with the calibrated camera, implement a program that estimates the camera pose from an ArUco marker (echoing Chapter 4) and displays the camera's 6-DoF pose relative to the marker in real time—joining Chapter 4 to this one. Second: if a stereo camera is available, implement disparity computation (OpenCV's StereoBM or StereoSGBM), produce a depth map, and use the formulation of Section 9.6 to verify the depth accuracy for an object at a known distance, observing whether the error grows with the square of distance. This program leads directly into Chapter 30.

Further Reading

1. R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004. (The authoritative bible of multiple-view geometry and the principal reference for Chapter 21.)
2. R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022. (The standard textbook of computer vision, freely available, with complete chapters on camera models and calibration.)
3. Z. Zhang, "A Flexible New Technique for Camera Calibration," IEEE TPAMI, 2000. (The original paper on Zhang's checkerboard calibration and the basis of Section 9.2.)
4. The official OpenCV tutorial on Camera Calibration and 3D Reconstruction (docs.opencv.org). (With complete code examples; the first choice for implementation.)
5. L. Keselman et al., "Intel RealSense Stereoscopic Depth Cameras," CVPR Workshops, 2017. (The principles and specifications of the RealSense depth camera family and the basis of Section 9.3.)
6. Xiang Gao and Tao Zhang, Fourteen Lectures on Visual SLAM, Publishing House of Electronics Industry. (The most popular introduction to visual SLAM in the Chinese-speaking world, covering the complete line from camera models to SLAM.)
7. UZH Robotics and Perception Group, the course Vision Algorithms for Mobile Robotics (introduced in Chapter 2). (A complete free course from image formation to visual SLAM.)

The Close of Part II: From Sensors to Mathematics

Part II ends here. Over six chapters the robot's senses have all been assembled, and you will have noticed a structure that recurs: every sensor has the situations it excels in and the situations in which it must fail, and the failure situations of different sensors happen not to overlap.

Indoor positioning needs installed infrastructure, GNSS needs the sky, the magnetometer fears rebar, the IMU drifts, LiDAR has no semantics and degenerates in corridors, and the camera has no scale and fears darkness and white walls. Lay them out in a table and one fact is clear: no single sensor can accomplish navigation alone, but their combination can.

The question then becomes: when five sensors give mutually contradictory readings at once, which should the robot believe? The answer of Chapter 11 is probability—every reading carries its own uncertainty, and the result of fusion is the estimate with the least uncertainty. But before these readings can be compared within one expression, they must first be expressed in one coordinate frame and position and orientation must be described by one set of mathematics.

That is the work of the next chapter. Chapter 10 introduces coordinate systems, the various representations of rotation (rotation matrices, Euler angles, quaternions, Lie groups), and rigid-body motion—it is the most mathematically dense chapter in the book and the toolbox every chapter thereafter uses. Read it slowly if need be, but do read it until you understand it.

Summary

1. The pinhole camera model describes how a three-dimensional point projects into a two-dimensional image; the denominator of the projection formula is

the depth Z, meaning that depth is divided away in imaging—the geometric root of scale ambiguity in visual SLAM. The intrinsic matrix K is the most basic object in visual SLAM.

2. Real lenses have radial and tangential distortion, which bends straight lines and violates the pinhole assumption; camera calibration (Zhang's checkerboard method, for instance) measures the intrinsics K and the distortion coefficients, and undistortion is the first step of all visual SLAM.
3. The three kinds of camera obtain depth in different ways: monocular triangulates from motion but has no absolute scale; stereo uses left-right disparity and has absolute scale but fears untextured surfaces; RGB-D projects actively to obtain dense depth but fears strong light and glass.
4. Visual SLAM extracts three kinds of information from an image: feature points (corners, used as landmarks), edges (outlines), and optical flow (pixel motion); monocular depth estimation by deep learning is a newly emerging complement. Modern systems commonly mix them.
5. The camera is richest in information, lowest in cost, passive, frugal with power, and light; but it has scale ambiguity, depends heavily on light and texture (failing in darkness, against white walls, and under blur), carries a heavy computational load, and fears dynamic scenes. Its weaknesses are nearly complementary to LiDAR's.
6. The ladder of knowledge runs: image processing → feature extraction → multiple-view geometry → visual odometry → visual SLAM; visual SLAM divides into the feature-based school (ORB-SLAM) and the direct school (DSO).
7. The camera carries semantics and is the window onto semantic navigation: AI models such as YOLO, SAM, Depth Anything, and CLIP, together with VLMs, let

a robot advance from knowing shapes to understanding a scene and understanding what people say—the subject of Chapter 32.

Part II, "Sensors for Localization and Navigation," is now complete—we have met all of the robot's senses. But what every sensor gives is raw data carrying noise: odometry drifts, LiDAR has stray points, the camera mismatches features. The next part, "The Mathematical Foundations of SLAM," answers a fundamental question: faced with these imperfect measurements, how does a robot make the most reasonable estimate mathematically? We will begin with coordinate frames and rigid-body motion and build the mathematical tools for handling uncertainty step by step.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Part III The Mathematical Foundations of SLAM

Chapter 10

Coordinate Systems and Rigid-Body Motion

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Part II has introduced all the sensors, but each of them speaks for itself—the LiDAR says "there is a wall three meters ahead," the camera says "there is a corner at image coordinates (320, 240)," and the IMU says "it turned 0.3 degrees about its own X axis." These statements do not even share units or a reference, and they cannot be compared at all.

What Part III does is make them speak one language. This chapter is the first and most basic step: establish coordinate systems and learn to move any piece of data from one frame into another.

It is the most mathematically dense chapter in the book, and also the one that repays the most. From the next chapter onward, the symbols of this chapter appear in every expression: the state x of Chapter 11 is the pose of this chapter, the variables optimized in Chapter 12 are a string of poses, placing LiDAR points into a map in Chapter 13 uses the 2D transformation of this chapter, the ICP of Chapter 17 solves for a T, and ORB-SLAM2 in Chapter 22 tracks a T for every frame. One might say that every chapter after this one is solving for some kind of T.

Figure 10-0 How Chapter 10 Connects to the Rest of the Book

[Building on the previous chapters]

Chapters 4 through 9 Six classes of sensor, each speaking in its own frame and its own units

This chapter Establishing a common language: frames, pose, transformation matrices

[This chapter is responsible for]

The three levels of frame: world → robot → sensor

Homogeneous coordinates: writing "rotation + translation" as a single matrix multiplication

Three representations of rotation and the trade-offs among them (Table 10-1)

The TF tree: how ROS manages the whole tree of frames

[Where it leads]

→ Chapter 11 The pose becomes the "state x" to be estimated

→ Chapter 12 The nodes of a pose graph are the T of this chapter

→ Chapter 13 Putting LiDAR points into a map: a direct application of the formulation of Section 10.3

→ Chapter 17 ICP registration: solving for a T between two point clouds

→ Chapters 28 and 29 A faulty TF tree is the most common failure in ROS mapping

Learning Objectives

Part II introduced all of the robot's senses, but every sensor reports its data in its own frame—the camera says "there is something two meters ahead," the LiDAR says "there is a wall three meters to my left." To assemble this information into one consistent map, one must first learn to convert precisely among different frames. This chapter is the first stop of Part III and the bedrock of all the mathematics of SLAM. After completing this chapter, you should be able to:

1. Describe the relationship among the world, robot, and sensor frames, and explain why SLAM needs several frames.
2. Use homogeneous coordinates to express rotation and translation together as a single matrix operation.
3. Write the representation of a 2D pose (x, y, θ) and the coordinate transformation formulas.
4. Describe the three representations of 3D rotation—rotation matrix, Euler angles, and quaternion—with their merits, drawbacks, and proper settings.
5. Describe the composition and inversion of coordinate transformations, and understand the concept of the TF tree.
6. Describe the meaning of and the relations among the standard ROS frames map, odom, base_link, and camera_link.

Core Concepts

Chinese Term	English Term	Explanation
座標系	Coordinate Frame	The reference against which position and direction are described; SLAM has several at once.
位姿	Pose	Position plus attitude; (x, y, θ) in 2D, position plus three axes of rotation in 3D.
剛體運動	Rigid Body Motion	Motion that does not change an object's shape or size, consisting only of rotation and translation.
齊次座標	Homogeneous Coordinates	A coordinate representation with one extra dimension, letting rotation and translation be expressed by a single

Chinese Term	English Term	Explanation
		matrix.
旋轉矩陣	Rotation Matrix	An orthogonal matrix describing rotation, belonging to the special orthogonal group SO(3).
歐拉角	Euler Angles	Rotation expressed by the three angles roll, pitch, and yaw; intuitive but subject to gimbal lock.
四元數	Quaternion	Rotation expressed by four numbers; free of singularities and computationally efficient, the standard representation in SLAM.
變換矩陣	Transformation Matrix	A 4×4 matrix containing both the rotation R and the translation t, belonging to SE(3).
萬向鎖	Gimbal Lock	The singular phenomenon in which Euler angles lose a degree of freedom at particular attitudes.
座標轉換樹	TF Tree	The tree structure that manages the relations among frames in ROS.
里程計座標	odom frame	A reference frame that is continuous and smooth but drifts over the long run.
地圖座標	map frame	A global reference frame that does not drift but may jump.

10.1 World, Robot, and Sensor Frames

Picture a robot in a classroom with a camera mounted on its head, which sees "a chair two meters straight ahead." A question hides in that statement: straight ahead of what? Of the camera, of the robot's body, or of the classroom? The three answers differ—turn the robot and "ahead" relative to the camera changes, while the chair's position relative to the classroom does not. This is why SLAM must handle several frames at once.

Three Levels of Frame

- World frame: a fixed global reference, usually with its origin at the robot's starting point at power-on or at some corner of the map. The absolute position of everything on the map is referred to it. It is the frame into which SLAM ultimately wants to unify everything.
- Robot (body) frame: a frame fixed to the robot and moving with it, with its origin usually at the center of the chassis. In ROS it is called base_link. The robot's motion is, in essence, the motion of this frame relative to the world frame.
- Sensor frame: fixed to each sensor. The camera has a camera frame and the LiDAR a LiDAR frame, with the origin at each sensor's measurement center. What a sensor measures directly is expressed in its own frame.

Why So Many Levels Are Needed

The key is this: where a sensor is mounted on the robot is fixed (the camera is always 20 centimeters forward of the head, the LiDAR always 15 centimeters above the chassis), while the robot's position relative to the world changes constantly. Separating the fixed mounting relationship from the changing robot pose is what keeps the

mathematics clear.

So a piece of sensor data entering the map takes two steps: first from the sensor frame into the robot frame (using the fixed mounting relationship, called the extrinsics, mentioned in Chapter 5), then from the robot frame into the world frame (using the robot's current pose, which is exactly what SLAM estimates). Understanding this sensor → robot → world chain is a prerequisite for reading any SLAM system. The rest of this chapter assembles the mathematical tools for doing the transforming.

10.2 Homogeneous Coordinates

A coordinate transformation involves two actions: rotation (turning a direction) and translation (moving a position). The nuisance is that rotation is multiplication (a matrix times a vector) while translation is addition (a vector plus a vector), and the two differ in form, which makes chaining several transformations verbose. Homogeneous coordinates are an elegant trick that unifies the two into a single matrix multiplication.

The Magic of One Extra Dimension

The method is simple: append a 1 to the original coordinates. A 2D point (x, y) is written (x, y, 1) and a 3D point (x, y, z) is written (x, y, z, 1).

The benefit is that translation can now be written into the matrix as well. In 2D, "rotate by θ then translate by (tx, ty)" can be merged into a single 3×3 matrix multiplication:

$$[x';y';1] = [\cos\theta \ -\sin\theta \ tx;\sin\theta \ \cos\theta \ ty;0 \ 0 \ 1]\cdot[x;y;1]$$

This 3×3 matrix is called the 2D transformation matrix (or homogeneous transformation matrix). Its upper-left 2×2 block is the rotation, its upper-right column the translation, and a row (0 0 1) is appended at the bottom. What once took two

steps—multiply by the rotation, then add the translation—is now done in one matrix multiplication.

Why This Matters

- Chaining transformations becomes a product: whether the robot has moved through several segments or the data passes through several layers of sensor → robot → world, one need only multiply the transformation matrices in order, with no separate handling of rotation and translation.
- The inverse transformation is the inverse matrix: to convert world coordinates back into robot coordinates, take the inverse of the transformation matrix.
- Uniform implementation: all coordinate handling is matrix multiplication, which keeps the code concise and less error-prone and lends itself to GPU acceleration.

The 3D case is entirely analogous, with a 4×4 matrix instead: the upper-left 3×3 block is the rotation matrix R, the upper-right 3×1 column the translation vector t, and a row (0 0 0 1) is appended at the bottom. This 4×4 matrix is one of the most central mathematical objects in SLAM: it carries the complete pose information of where the robot faces and where it is.

A Clarification: Two Ways of Reading a Transformation Matrix

The same transformation matrix T admits two readings, which beginners often confuse:

① Coordinate transformation: converting the coordinates of the same point in frame A into its coordinates in frame B.

② Rigid-body motion: actually moving and rotating a point to a new position (within the same frame).

The two have the same mathematical form but opposite meanings (turning a frame one way is the same as turning a point the other way).

In SLAM it is mostly the first—we do not really move the world, we describe the same thing in another frame.

10.3 The 2D Pose: x, y, θ

Cleaning robots and teaching vehicles move in a plane, and a 2D pose suffices to describe them. This is the best starting point for understanding 3D, because 2D has only one rotational degree of freedom and everything is intuitive.

Three Numbers Describe Everything

The pose of a robot in a plane is completely described by three numbers:

- x, y: the robot's position in the world frame.
- θ: the robot's heading angle, the angle between the robot's forward direction and the world frame's X axis.

This is exactly the quantity estimated by the dead reckoning of Chapter 3 and the Gmapping particles of Chapter 14. In 2D SLAM, every particle and every pose node is just such a triple (x, y, θ).

A Worked Example of 2D Coordinate Transformation

Suppose the LiDAR measures a point p_robot = (a, b) in the robot frame (a ahead of the robot, b to its left), and the robot's pose in the world frame is (x, y, θ). Then the

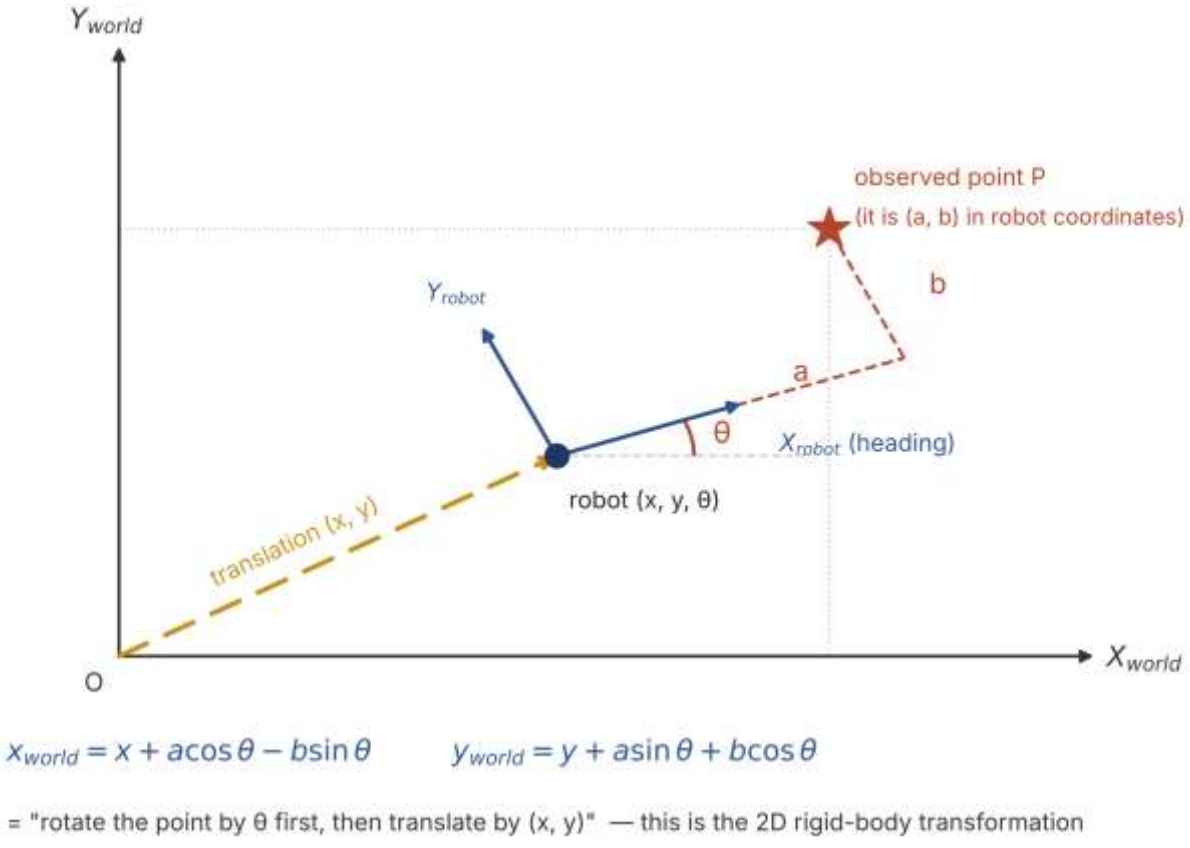
position of this point in the world frame is:

$$x_world = x + a \cdot \cos\theta - b \cdot \sin\theta$$

$$y_world = y + a \cdot \sin\theta + b \cdot \cos\theta$$

This is exactly the computation performed for every point when "plotting one revolution of LiDAR ranges into a map" in Chapter 8. Written in homogeneous coordinates, it is the 3×3 transformation matrix of the previous section times (a, b, 1). You may verify that when θ = 0 (the robot facing due east) and the pose is at the origin, the formulas reduce to x_world = a and y_world = b, as intuition requires.

Figure 10-1 2D Rigid-Body Transformation: Moving a Point from Robot Coordinates into World Coordinates



This figure is the geometric source of the most frequently used pair of expressions in the book. The robot is at (x, y) in the world frame, with its heading at angle θ to the world X axis. It measures a point with its LiDAR and records it in its own frame as (a, b)—

a ahead, b to the left. The question is: where is that point on the world map?

The answer comes in two steps. First rotate: the robot's axes are turned by θ relative to the world, so (a, b) must first be multiplied by the rotation matrix, giving ($a\cdot\cos\theta - b\cdot\sin\theta$, $a\cdot\sin\theta + b\cdot\cos\theta$). Then translate: add the robot's own position (x, y). Together these give the two expressions at the bottom of the figure.

Be sure to remember that the order is rotate first, then translate, and it cannot be reversed—a point that recurs in the ICP of Chapter 17 and the pose updates of Chapter 22. Reversing the order gives an entirely different result, and the error is often not obvious: it is barely visible while the robot is near the origin, but once it travels far the whole map goes crooked. Many students who find in Chapter 29 that "the map grows more and more skewed as the robot drives" are seeing exactly this.

An Exercise That Ties the Ideas Together

Reverse these two expressions: given a point's position in the world and the robot's pose, how would you compute its (a, b) in robot coordinates?

The answer is to multiply by the inverse transformation—and Section 10.6 will tell you that the inverse need not really be computed as a matrix inverse, since $T^{-1} = [\, R^T \quad -R^T t \,]$.

Both directions are used in SLAM: mapping uses the forward direction (moving observations into the map) and localization the reverse (moving map points into the sensor's view for comparison). Chapters 13 and 14 are exactly this pair.

Two Practical Pitfalls with Angles

- Angle wrapping: θ is periodic, and +180 degrees and −180 degrees are the same direction. When computing the difference of two angles, the result must

be wrapped back into the interval $(-\pi, \pi]$, or absurdities appear such as "turned 350 degrees" where "turned 10 degrees the other way" was meant. This is one of the most common bugs in 2D SLAM code.

- Radians and degrees: mathematical operations always use radians, while people read degrees. Units must be kept consistent in code; mixing them causes errors that are hard to notice.

10.4 The 3D Pose: Position and Attitude

UAVs, quadrupeds, and handheld cameras move in three-dimensional space and need a complete 3D pose. The 3D position is simple—just three numbers (x, y, z); what is genuinely complicated is 3D attitude: describing which way a body faces takes three rotational degrees of freedom, and there turn out to be several ways of representing those three, each with merits and drawbacks. This is the mathematical heart of the chapter.

Six Degrees of Freedom

A rigid body in three-dimensional space has six degrees of freedom (6-DoF): three translations (along X, Y, and Z) plus three rotations (about X, Y, and Z). The "6-DoF pose" that recurs in Chapters 7 and 23 means exactly these six numbers. The three positional degrees of freedom are uncontroversial, so the next three sections concentrate on the three representations of rotation.

Why is rotation so troublesome? Because a rotation is not an ordinary quantity. Two translations may be added in any order with the same result; but composing two rotations depends on the order—rotate 90 degrees about X and then 90 about Y, and the result differs entirely from rotating about Y first and then X (try it with your phone).

This non-commutativity makes the mathematics of rotation far subtler than that of translation, and it is why the MIT VNAV course of Chapter 2 emphasizes optimization on manifolds—the space of rotations is curved, not a flat vector space.

10.5 Rotation Matrices, Euler Angles, and Quaternions

This is the most important section of the chapter. 3D rotation has three principal representations, and every one of them appears in SLAM systems, so you must understand the role of each.

1. The Rotation Matrix: The Theoretical Cornerstone

A rotation is represented by a 3×3 matrix R. It rotates a vector into a new direction, and the operation is a matrix times a vector. A rotation matrix has strict mathematical properties: it is orthogonal (its columns are mutually perpendicular and of unit length) and its determinant is +1, and the set of such matrices is called the special orthogonal group SO(3).

- Advantages: mathematically perfect, directly usable in coordinate transformation, free of singularities, and composition of rotations is simply matrix multiplication.
- Drawbacks: it uses 9 numbers to represent a rotation with only 3 degrees of freedom, which is highly redundant; and in storage and optimization the orthogonality constraint must be maintained, so one cannot freely add to or subtract from the 9 numbers (or the result is no longer a legitimate rotation).

2. Euler Angles: The Most Intuitive, with a Trap

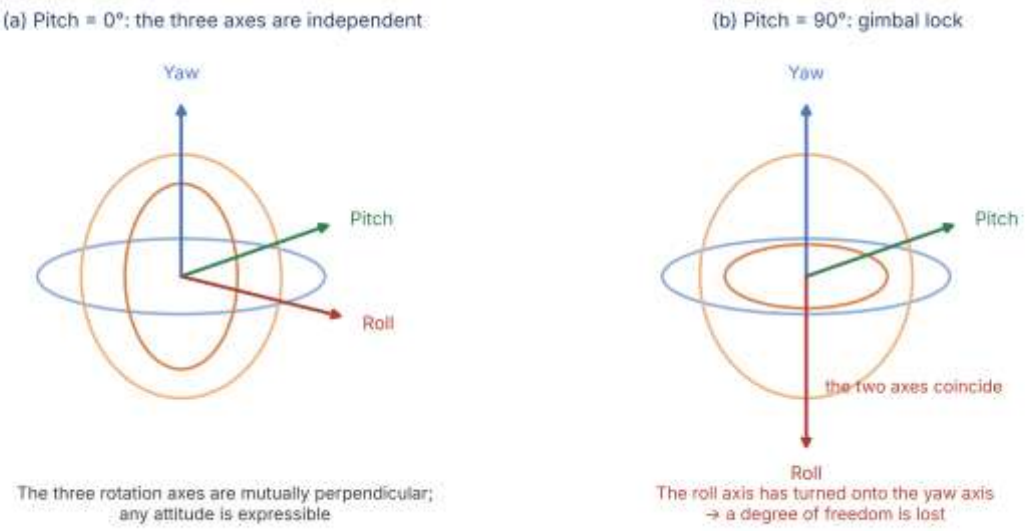
A rotation is represented by three angles—roll φ, pitch θ, and yaw ψ (introduced in Chapter 7). This is the representation closest to human intuition: "nose up 10 degrees,

banked 5 degrees right, heading east" is understood at once.

- Advantages: intuitive, using only 3 numbers (no redundancy), and convenient for people to understand and for interface display.
- Drawbacks: gimbal lock—when the pitch angle approaches ±90 degrees the roll and yaw axes coincide and a degree of freedom is lost (detailed in Chapter 7); furthermore, rotations cannot be composed by simple addition, and the same rotation may have several sets of angles (periodicity), which causes trouble in numerical optimization.

Euler angles are therefore suited to being shown to people (interface display, debugging) but not to being computed with by machines (internal computation). This is an important engineering division of labor.

Figure 10-2 Gimbal Lock: Why Euler Angles Lose a Degree of Freedom



Gimbal lock is the most notorious defect of Euler angles, yet it is not hard to understand—seeing the relationship of the three axes is enough. In the left panel the pitch angle is zero and the yaw, pitch, and roll axes are mutually perpendicular, so any attitude can be expressed uniquely by three angles.

The right panel turns the pitch angle to 90 degrees. Notice what has happened: the roll axis has been carried around by the pitch axis until it coincides with the yaw axis. Changing the yaw angle and changing the roll angle now produce exactly the same motion—the three parameters have only two effective degrees of freedom left, and a whole direction in attitude space has become inexpressible.

This is not an implementation flaw but a mathematical necessity: any method describing three-dimensional rotation with three numbers must have singularities. Avoiding them requires more than three numbers—a rotation matrix uses nine (with six constraints) and a quaternion four (with one). That is why Section 10.5 says the quaternion is the practical standard in SLAM.

The Apollo 11 navigation platform of Case 10-1 came to grief over exactly this. And on a robot it causes trouble just the same: when a UAV makes a steep dive or climb, when a robot arm's joints reach a particular attitude, or when a quadruped tips over, storing attitude as Euler angles may produce numerical blow-up or sudden attitude jumps. We shall meet it again when UAV flight control is discussed in Chapter 26.

3. The Quaternion: The Practical Standard in SLAM

A quaternion represents a rotation with four numbers (w, x, y, z) and may be understood as a compact encoding of "rotate by some angle about some axis." It is unintuitive at first sight, yet it is the rotation representation used internally and uniformly in modern SLAM, UAVs, games, and animation.

- Advantages: no gimbal lock (no singularities), only 4 numbers (minimal redundancy), composition of rotations is just quaternion multiplication (computationally efficient), numerically stable, and convenient for interpolation (turning smoothly from one attitude to another).

- Drawbacks: unintuitive, as a person can hardly read off what direction (w, x, y, z) means; and a unit-length constraint must be maintained (but only one constraint, far simpler than the orthogonality constraints of a rotation matrix).

The division of labor in practice is very clear: quaternions for internal computation and storage, Euler angles for interface display and human reading, and rotation matrices for coordinate transformation. The three interconvert, and the tf2 package in ROS and every mathematical library have conversion functions built in. You need not memorize the multiplication rules for quaternions, but you must understand why practice uses them—the answer is that they avoid both the redundancy of the rotation matrix and the gimbal lock of Euler angles.

Representation	Count of Numbers	Singularities	Principal Advantage	Principal Use
Rotation matrix	9 (3×3)	None	Direct in coordinate transformation, theoretically perfect	Coordinate transformation operations
Euler angles	3	Yes (gimbal lock)	Most intuitive, easy for people to read	Interface display, debugging
Quaternion	4	None	Compact, efficient, convenient to interpolate	Internal computation and storage

Table 10-1 Comparison of Three Representations of 3D Rotation

10.6 Coordinate Transformation and the TF Tree

With the transformation matrix as a tool, let us see how a real robotic system manages the relations among its many frames. The answer is a tree structure—the TF

tree.

Composition and Inversion of Transformations

First review two key operations, the basis on which the TF tree works:

- Composition: given the transformation from A to B and that from B to C, multiplying the two matrices gives the transformation from A to C. This lets us carry sensor data all the way into world coordinates.
- Inversion: given the transformation from A to B, its inverse matrix is the transformation from B to A.

With these two operations, knowing the transformation between any two adjacent frames suffices to compute the relation between any two frames at all—which is precisely the central idea of the TF tree.

The TF Tree: A Family Tree of Frames

On a real robot there may be a dozen or more frames: world, map, odometry, chassis, each wheel, the LiDAR, the camera, every link of a robot arm, and so on. ROS manages their relations with a tree structure (the TF tree, the transform tree).

- Each frame is a node of the tree.
- Between each parent and child node a transformation is recorded (parent to child).
- When the relation between any two frames is queried, the system automatically composes (multiplies) the transformations along the path through the tree.

The ingenuity of this design is that each transformation need only be published by the module that knows it best. The LiDAR driver publishes the fixed transformation from chassis to LiDAR (from the mounting measurements), the SLAM system publishes

the transformation from map to chassis (from the localization estimate), and odometry publishes the transformation from odom to chassis. Each module minds its own business, and the TF tree integrates them automatically into a complete network of frame relations. You will set up this tree with your own hands in the Chapter 29 exercises.

Time Stamps: A Key Detail

TF manages not only spatial relations but time. Because the robot is moving, every transformation varies with time, so the TF system stamps each transformation with a time and a query can ask "where was the camera relative to the map 0.1 second ago." This is crucial for sensor fusion—camera and LiDAR data arrive at different times and must be aligned using the transformation at the correct instant, or noticeable error appears during fast motion. This echoes the engineering problem of time synchronization in the fusion discussion of Chapter 5.

10.7 map, odom, base_link, and camera_link in ROS

ROS defines a standard naming and hierarchy of coordinate frames for mobile robots, and almost every ROS navigation system follows it. Understanding this convention is basic literacy for reading and implementing SLAM systems.

The Standard Frame Hierarchy

The standard chain, from top to bottom, is: map → odom → base_link → the individual sensor frames.

Figure 10-3 The ROS Standard Frame Hierarchy

map (the map frame): globally fixed and drift-free, but it jumps

↓ (published by the SLAM or localization system)

odom (the odometry frame): continuous and smooth, but drifts over the long run

↓ (published by odometry)

base_link (the robot body): fixed at the center of the robot's chassis

↓ (published from the fixed mounting relationship, that is, the extrinsics)

laser / camera_link / imu_link (the individual sensor frames)

Why Both map and odom Are Needed

This is the most ingenious and the most confusing part of the whole design. Why not use a single frame instead of splitting it into map and odom? Because they solve different needs and are complementary:

- odom (the odometry frame): obtained by integrating odometry and the IMU. Its character is continuous, smooth, and free of jumps—very well suited to short-term motion control and obstacle avoidance. But as Chapters 3 and 7 described, it drifts over the long run and grows less accurate the farther the robot travels.
- map (the map frame): obtained from SLAM or a localization system (AMCL, say). Its character is long-term correctness without drift, because it corrects itself by matching against the map. But it jumps: when SLAM suddenly discovers "I am actually here" and corrects the position, the robot's coordinates in map shift instantaneously.

The wisdom of keeping both layers is this: modules that need smoothness (obstacle avoidance control) use odom, and modules that need accuracy (global path planning) use map. What a SLAM system actually publishes is the map-to-odom transformation—it continually adjusts that transformation to compensate for odometry drift, while base_link's motion relative to odom remains smooth. This design lets the two apparently conflicting requirements of smoothness and accuracy coexist elegantly.

Sensor Frames and the Extrinsics

Each sensor frame below base_link (laser, camera_link, imu_link, and so on) has a fixed transformation relative to base_link, determined by where the sensor is mounted on the robot—the extrinsics mentioned in Chapter 5. These transformations are usually written in the robot's URDF description file (Chapter 29) or published by a static transform publisher. Measuring these extrinsics accurately is the basis of the system's accuracy—a mounting position off by a few centimeters or a few degrees blurs or misaligns the resulting map.

You now have the first set of mathematical tools of SLAM: how to convert poses precisely between frames. But every measurement in the real world carries noise—wheels slip, LiDAR has error, cameras mismatch features. The next chapter, "Probability, Noise, and State Estimation," teaches how to describe these uncertainties in the language of probability and make the most reasonable estimate within them. This is the pivotal turn by which SLAM moves from geometry to estimation.

Formulations and Procedures

1. The 3D Homogeneous Transformation Matrix

3D rigid-body motion is represented by a 4×4 homogeneous transformation matrix T containing both the rotation R (3×3) and the translation t (3×1):

$$T = [\,R \;\; t\,;\, 0 \;\; 1\,]$$ *(upper left 3×3 is R, upper right 3×1 is t, lower left is 0, lower right is 1)*

To convert a point p_A in frame A (in homogeneous coordinates) into frame B:

$$p_B = T_BA \cdot p_A$$

Here T_BA reads "the transformation from A to B." The order in which subscripts are written is the easiest thing to get wrong in SLAM; adopt one convention and hold to it throughout.

2. Composition and Inversion of Transformations

Composition: chain A→B with B→C into A→C:

$$T_CA = T_CB \cdot T_BA$$

Note the order of multiplication: adjacent subscripts must meet (B next to B). Inversion:

$$T_AB = (T_BA)^{-1}$$

The inverse of a homogeneous transformation matrix has a shortcut formula, with no need for a general 4×4 matrix inversion:

$$T^{-1} = [\,R^T \;\; -R^T t\,;\, 0 \;\; 1\,]$$

Exploiting the orthogonality of the rotation matrix (its inverse equals its transpose) makes inversion very efficient.

3. Composing 2D Poses (Chaining Odometry)

After the robot moves from pose $p_1 = (x_1, y_1, \theta_1)$ through a relative motion (Δx, Δy,

Δθ, expressed in the body frame of p_1), the new pose is:

$$x_2 = x_1 + \Delta x \cdot \cos\theta_1 - \Delta y \cdot \sin\theta_1$$

$$y_2 = y_1 + \Delta x \cdot \sin\theta_1 + \Delta y \cdot \cos\theta_1$$

$$\theta_2 = \text{normalize}(\theta_1 + \Delta\theta) \quad (\text{normalize means wrapping back into } (-\pi, \pi])$$

This is the concrete implementation of the dead reckoning of Chapter 3 and the motion update of Gmapping in Chapter 14. The normalize step is the angle wrapping emphasized in Section 10.3 and must not be omitted.

4. The Complete Pipeline by Which Sensor Data Enters a Map

Figure 10-4 The Chain of Transformations by Which One Sensor Measurement Enters the World Map

① Sensor measurement: point p in the sensor frame (camera_link, say)

↓ × T(base_link ← camera_link) the extrinsics, fixed

② Point p in the robot frame (base_link)

↓ × T(odom ← base_link) odometry

③ Point p in the odometry frame (odom)

↓ × T(map ← odom) the SLAM correction

④ Point p in the world map frame (map) → written into the map

The whole chain is a product of several transformation matrices, and the TF tree performs this computation automatically.

10.8 Contemporary Choices of Rotation Representation: From Engineering to Learning

Section 10.5 compared rotation matrices, Euler angles, and quaternions. That comparison is already complete in the context of traditional SLAM, but two recent developments deserve a supplement—one concerning engineering practice and one concerning deep learning.

1. A Quantitative Comparison: The Real Cost of Each Representation

Recent research has made a systematic comparison of seven principal representations of three-dimensional rotation—Euler angles, axis-angle vectors, quaternions, rotation matrices, the exponential map, the six-dimensional continuous representation, and probabilistic representations such as the matrix Fisher and Bingham distributions—evaluating them on continuity, gimbal lock, computational efficiency, storage, and interpolation quality.

Its conclusion agrees with Section 10.5 of this book: for general purposes the quaternion strikes the best overall balance—computationally efficient, compact in storage, smoothly interpolable by spherical linear interpolation (SLERP), and free of gimbal lock. In concrete figures, composing quaternions takes about 34 microseconds and occupies 32 bytes; rotation matrices about 306 microseconds and 72 bytes; the six-dimensional continuous representation about 422 microseconds and 48 bytes. The quaternion is thus nearly an order of magnitude faster at composition, a substantive difference in a SLAM back end that composes poses tens of thousands of times per second.

2. But Inside a Neural Network the Quaternion Is Not Best

The second development bears closely on Chapter 32. When a robot uses a neural network to predict attitude directly—regressing camera pose from an image, say, or having a vision-language-action model output a target attitude for a manipulator—the quaternion becomes a problem instead.

The reason is continuity. A neural network is trained by gradient descent and requires the output space to vary continuously with the input; but quaternions have a double-cover property (q and −q represent the same rotation), and Euler angles have the discontinuity of gimbal lock besides. These discontinuities feed the network contradictory gradients during training and make learning unstable.

The remedy is the six-dimensional continuous representation: output the first two column vectors of the rotation matrix and recover the full matrix by Gram-Schmidt orthonormalization. This representation is continuous over the space of rotations and has no singularity anywhere. The research cited above reports that in neural attitude estimation tasks it reduces error by about six to fourteen percent relative to quaternions.

This brings out an idea that matters to students: there is no best rotation representation, only the representation best for this task. A SLAM back end needs fast composition, so it uses quaternions; a neural network needs continuity and differentiability, so it uses the six-dimensional representation; people need to read it, so the user interface shows Euler angles. The same rotation puts on different clothes for different purposes.

3. Lie Groups: Making Optimization Proceed Correctly on the Manifold

There is another line, standard in modern SLAM yet often skipped in introductory texts: regard rotations and poses as Lie groups (SO(3) and SE(3)) and their small changes as

vectors in a Lie algebra.

Why is this needed? Because the optimization of Chapter 12 must differentiate with respect to pose, must compute how far two poses differ, and must add a small increment to a pose. But rotation matrices cannot simply be added—the sum of two rotation matrices is no longer a rotation matrix. The Lie algebra provides the correct procedure: add, subtract, and differentiate in the tangent space of the manifold, then send the result back to the manifold by the exponential map.

In practice you need not derive any of this from scratch—libraries such as Sophus and manif already encapsulate the operations of SO(3) and SE(3), and optimization frameworks such as g2o, Ceres, and GTSAM all have manifold parameterizations built in. Chapters 12 and 23 will use them concretely. For now, remember one sentence: a rotation is not a vector and cannot simply be added. The warning in Section 10.3 that angles cannot be averaged directly is the two-dimensional version of that same sentence.

A Quick Reference for Choosing

User interfaces, specification sheets, numbers people must read → Euler angles (but never stored internally as such)

SLAM internal state, sensor drivers, ROS messages → quaternions

Differentiation and incremental updates in optimization → Lie algebras (so(3) / se(3))

The output layer of a neural network → the six-dimensional continuous representation

Point clouds and geometric operations → rotation matrices (directly

multipliable)

Case Studies

Case 10-1 Apollo 11's Gimbal Lock Scare

Gimbal lock is not an abstract mathematical term; it once caused a real crisis during a mission to the Moon. The inertial navigation platform of the Apollo spacecraft was supported by three mechanical rings (gimbals) and tracked the spacecraft's attitude in the manner of Euler angles. When the three rings approached coplanarity, gimbal lock would occur—a rotational degree of freedom lost, and the navigation platform potentially losing its attitude reference.

The engineers knew the risk well, designed a warning indicator on the instrument panel, and planned the flight attitudes to stay clear of the dangerous region. Astronaut Michael Collins once complained half-jokingly about the restriction. In hindsight this was the inherent defect of representing attitude with Euler angles manifesting itself in an extreme mission. Modern spacecraft, UAVs, and robots long ago moved to quaternions precisely to eliminate the problem.

The lesson for students is that the choice of representation is not a matter of pure mathematical taste; it has real engineering consequences. The seemingly dry comparison of three rotation representations in Section 10.5 carries the wisdom countless engineers bought with hard experience—which is why this book insists on explaining in its mathematical chapters why a choice is made, not merely how to compute.

Case 10-2 The Frame Tree in RViz: Making the Abstract Concrete

Students meeting the TF tree for the first time often find it abstract and hard to grasp. But in RViz, the visualization tool of ROS, the frame tree becomes something visible: each frame is displayed as a little set of red, green, and blue axes (for X, Y, and Z); as the robot moves, the base_link axes move with it; and with the LiDAR mounted ahead of the robot, the laser axes stay fixed a short distance in front of base_link.

More instructive still is observing errors. If the extrinsics are set wrong (the LiDAR's mounting direction entered backwards, say), the point cloud of the LiDAR scan in RViz will be displaced from the real environment—the wall is clearly ahead, yet the points are drawn behind or to the side. By adjusting the extrinsics and watching in real time whether the point cloud hugs the environment, students understand the meaning of coordinate transformation very directly. This is the best experiment for threading every abstract concept of Sections 10.1 through 10.7 together at once, and the Chapter 29 exercises use it extensively.

A common debugging maxim in university classrooms is: "if the map is crooked, look at TF first." Many SLAM mapping failures have their root not in the algorithm but in misconfigured frames—extrinsics entered wrongly, time stamps unaligned, a broken TF tree. Understanding this chapter saves students countless wasted hours of debugging.

Case 10-3 Forward Kinematics of a Robot Arm: The Same Mathematics on Another Stage

The coordinate transformation mathematics of this chapter is not used only in mobile robots; it is also the core of robot arms. A six-axis arm has six joints between base and gripper, and the rotation of each joint is a rotational transformation. Multiplying the transformation matrices in order—base to first link, first link to second,

and so on to the gripper—computes the pose of the gripper in space given the six joint angles, which is called forward kinematics.

This is a direct application of "composing transformations is just multiplying matrices" from Section 10.6. Students taking a robot arm course alongside this one will find that both use the same homogeneous transformation mathematics—no coincidence, but because "how a rigid body moves in space" is the common language of robotics. Mastering this chapter lays the mathematical foundation for mobile robots and robot arms alike. This also echoes the shared sensor → mathematics → systems architecture of the university courses in Chapter 2: mathematics is the common denominator of all robotics.

Hands-On Activities

Exercise 10-1 Working Coordinate Transformations by Hand and Observing the TF Tree

Purpose: to master 2D coordinate transformation through pencil-and-paper computation and verification in code, and to observe a real TF tree in ROS so that the abstract concepts become concrete. This exercise has three parts—paper, code, and ROS—and suits teaching at various depths.

Part One: Pencil and Paper (everyone)

1. A robot has pose ($x = 3$, $y = 2$, $\theta = 90°$) in the world frame. Its LiDAR measures a point in the body frame ($a = 2$ ahead, $b = 0$ to the left). Compute this point's world coordinates by hand with the formulas of Section 10.3, and draw the figure on graph paper to check that the answer is sensible.

2. For the same robot, the LiDAR measures another point (a = 1, b = 1). Compute its world coordinates by hand and confirm with a drawing.
3. Angle wrapping exercise: the robot's heading is $\theta_1 = 170°$ and it turns right by $\Delta\theta = 30°$. Direct addition gives 200°; what should it be after wrapping back into $(-180°, 180°]$? Explain why not handling this causes trouble.

Part Two: Verification in Python (everyone)

4. Build the 3×3 homogeneous transformation matrix of Section 10.2 with numpy, redo the computation of Part One as a matrix multiplication, and verify that the results agree.
5. Implement composition of transformations: given the world→robot and robot→LiDAR transformations, multiply them to obtain world→LiDAR, and verify.
6. Implement inversion: take the inverse of the world→robot transformation and verify that it equals the robot→world transformation (compare the shortcut formula of Section 10.2 with a direct numpy inverse).
7. (Undergraduate) Implement the mutual conversion of Euler angles, rotation matrices, and quaternions (scipy.spatial.transform.Rotation may be used); take an attitude with pitch near 90° and observe how the Euler angle representation becomes anomalous, verifying gimbal lock first-hand.

Part Three: Observing the ROS TF Tree (for those with a ROS environment)

8. Launch the TurtleBot3 simulation (Chapter 28) or any ROS system with TF, use view_frames or rqt_tf_tree to produce a diagram of the TF tree, and identify the hierarchical relations among map, odom, base_link, and the sensor frames.

9. Display TF in RViz, move the robot, and observe which axes move with it (base_link and its child frames) and which do not (map).
10. Use the tf2 query tools to look up the transformation of camera_link relative to map, querying once before and once after the robot moves, and observe how the values change.

Discussion Questions

1. In Part One, when the robot's heading is 90° (due north), which direction in the world frame does the body's "ahead" correspond to? Does your computed result match intuition?
2. Why does a SLAM system publish the map→odom transformation rather than map→base_link directly? How does this design help reconcile smoothness with accuracy?
3. (Undergraduate) What anomaly did you observe in the gimbal lock experiment? If a UAV's flight controller used Euler angles rather than quaternions, what might happen during an aerobatic roll?
4. If the map a robot builds is always skewed, or the point cloud does not line up with the walls, what would you check first based on this chapter?

Extended Challenge (Undergraduate)

Write a ROS 2 node that publishes a static transformation of a custom sensor relative to base_link (deliberately setting a known rotation and translation), display that sensor's simulated point cloud in RViz, and adjust the extrinsic values while watching in real time how the cloud moves, until it sits where expected. This exercise corresponds directly to configuring a robot's TF in the Chapter 29 exercises and lays the coordinate groundwork for the 2D SLAM vehicle implementation.

Further Reading

1. Xiang Gao and Tao Zhang, Fourteen Lectures on Visual SLAM, Lecture 3 (rigid-body motion in three-dimensional space) and Lecture 4 (Lie groups and Lie algebras), Publishing House of Electronics Industry. (The clearest introduction to the rotation mathematics of SLAM in the Chinese-speaking world.)
2. K. M. Lynch and F. C. Park, Modern Robotics: Mechanics, Planning, and Control, Cambridge University Press, 2017. (The authoritative textbook on rigid-body motion and coordinate transformation, with a free electronic edition and online course.)
3. J. Solà, J. Deray, and D. Atchuthan, "A Micro Lie Theory for State Estimation in Robotics," arXiv, 2018. (A friendly introduction to rotation groups and optimization on manifolds, leading into Chapter 12.)
4. The official ROS tf2 tutorials (docs.ros.org). (Implementation guidance for the TF tree, with complete examples.)
5. REP 105: Coordinate Frames for Mobile Platforms (ros.org/reps). (The official definition of the ROS standard frames map, odom, and base_link.)
6. 3Blue1Brown, the "Quaternions and 3D rotation" video series (YouTube). (Explains quaternions intuitively with superb animation; strongly recommended.)
7. R. Siegwart et al., Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011. (Chapters 2 and 3 place frames and kinematics in their robotics context.)

From Certainty to Uncertainty: Where This Chapter Stands

This chapter has accomplished one thing: making the data of all sensors expressible in

one frame. But it has quietly assumed a premise that does not hold in the real world—that every number is exact.

The expressions of Figure 10-1 assume the robot's pose (x, y, θ) is known and correct. But Chapter 7 told us the IMU drifts, Chapter 5 told us GNSS error is amplified by DOP, and the wheel odometry of Chapter 13 slips. In reality a robot never knows exactly where it is—it has only an estimate carrying an uncertainty.

Likewise, the observed point P in Figure 10-1 is not an exact point either. The LiDAR of Chapter 8 has ranging error and limited angular resolution, and the camera of Chapter 9 has pixel quantization and feature localization error. Every observation is a blur.

So Chapter 11 replaces every equals sign of this chapter with a probability distribution: a pose is no longer three numbers but a Gaussian distribution; an observation is no longer a point but a measurement with a covariance. And the work of fusing several sensors becomes "multiply several distributions together and find the most probable answer." That is Bayesian filtering, the common skeleton of the Kalman filter and the particle filter. The transformation matrices of this chapter do not disappear—they become the motion model and the observation model within the probabilistic framework.

Summary

1. SLAM handles three levels of frame at once—world, robot, and sensor; the sensor mounting relationship is fixed (the extrinsics) while the robot pose varies (the quantity SLAM estimates), and a piece of data entering the map passes through the chain sensor → robot → world.

2. Homogeneous coordinates add one dimension to unify rotation (multiplication) and translation (addition) into a single matrix multiplication; 2D uses a 3×3 and 3D a 4×4 transformation matrix, so that chaining transformations becomes a product and inversion becomes taking an inverse matrix.
3. A 2D pose is described by the three numbers (x, y, θ), with coordinate transformation given by formulas containing cosθ and sinθ; in practice angle wrapping (back into −π to π) must be handled and radians and degrees kept consistent.
4. A 3D rigid body has six degrees of freedom (three translational, three rotational); composition of rotations is non-commutative and the space they form is curved, which is the root of the complexity of rotation mathematics and of optimization on manifolds.
5. The three representations of 3D rotation have their own roles: the rotation matrix (9 numbers, no singularity, used in coordinate transformation), Euler angles (3 numbers, intuitive but with gimbal lock, used in human interfaces), and the quaternion (4 numbers, no singularity, efficient, used for internal computation and storage).
6. The TF tree manages many frames in a tree structure, each transformation published by the module that knows it best, with transformations composed automatically along the path on query; transformations carry time stamps to support temporal alignment in sensor fusion.
7. The ROS standard frames run map → odom → base_link → sensors: odom is continuous and smooth but drifts, while map is accurate but jumps; SLAM

publishes the map→odom transformation to compensate for drift, letting smoothness and accuracy coexist elegantly.

This chapter has assembled the first set of mathematical tools of SLAM—deterministic coordinate transformation. But the real world is full of uncertainty: odometry slips, LiDAR has stray points, cameras mismatch features. The next chapter, "Probability, Noise, and State Estimation," introduces the language of probability and teaches a robot to say "I am probably here, but I am not quite sure" and to make the most reasonable estimate amid noise. This is the pivotal step by which SLAM moves from geometry to estimation, and the common starting point of Bayesian filtering, the Kalman filter, and the particle filter.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 11

Probability, Noise, and State Estimation

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 10 made the data of all sensors expressible in one frame, but it assumed every number was exact. This chapter dismantles that assumption.

A real robot never knows exactly where it is. Wheels slip, the IMU drifts, the LiDAR has ranging error, and the camera localizes feature points imprecisely. More troublesome still, these errors contradict one another—odometry says it traveled three meters while LiDAR matching says only two point seven. Which should be believed?

The answer this chapter gives is: both, weighted by how far each can be trusted. And for trustworthiness to be computable, every estimate must first be written as a probability distribution. So a pose is no longer three numbers but a Gaussian distribution; an observation is no longer a point but a measurement with a covariance; and fusion is no longer averaging but multiplying distributions.

This is the most crucial conceptual turn in the book. The optimization of Chapter 12, Gmapping in Chapter 14, Cartographer in Chapter 15, and ORB-SLAM3 in Chapter 23 all rest on the framework of this chapter—they differ only in how they represent that distribution and how they find the most probable answer.

Figure 11-0 How Chapter 11 Connects to the Rest of the Book

[Building on the previous chapter]

Chapter 10 A pose = three (or six) exact numbers

This chapter A pose = a probability distribution carrying an uncertainty

[This chapter is responsible for]

The Gaussian distribution and covariance: describing "roughly where I am" with a mean plus

an ellipse

The motion model and the observation model: the two sources of prediction and correction

Bayesian filtering → Kalman → EKF → particle filter

[Where it leads]

→ Chapter 12 Another way to solve the same problem: optimization (the counterpart to filtering)

→ Chapter 13 Every cell of an occupancy grid map is a probability too

→ Chapter 14 Gmapping = a particle filter + a map carried by every particle

→ Chapter 16 AMCL: relocalization in a cleaning robot is the MCL of this chapter

→ Chapter 23 The MAP estimation of ORB-SLAM3 is the optimization version of this chapter's ideas

Learning Objectives

In the previous chapter we learned to convert precisely between frames—that was deterministic mathematics. But the real world is not so clean: odometry slips, LiDAR has stray points, cameras mismatch features. This chapter introduces the language of probability and teaches a robot to say "I am probably here, but I am not quite sure" and to make the most reasonable estimate amid noise. This is the pivotal turn by which SLAM moves from geometry to estimation, and the mathematical core of the book. After completing this chapter, you should be able to:

1. Explain why SLAM must handle uncertainty probabilistically rather than pursue a single exact answer.
2. Explain the meaning of the mean and variance of a Gaussian distribution, and understand how covariance describes uncertainty in two or more dimensions.

3. Explain how the motion model and the observation model describe a robot's prediction and perception probabilistically.
4. Describe the recursive predict-update structure of Bayesian filtering and understand that it is the common skeleton of every filter.
5. Describe how the Kalman filter works and explain how the Kalman gain balances prediction against observation.
6. Explain how the EKF handles nonlinearity, together with the basic concept and limitations of EKF-SLAM.
7. Explain the principle of the particle filter and Monte Carlo localization, and compare it with the Kalman family.

Core Concepts

Chinese Term	English Term	Explanation
機率分布	Probability Distribution	Describes the probability of each possible value of a random variable.
高斯分布	Gaussian Distribution	The bell-shaped normal distribution, described completely by a mean and a variance.
均值	Mean	The center of a distribution, representing the most likely estimate.
變異數	Variance	The spread of a distribution, representing the magnitude of the uncertainty.
共變異數	Covariance	Describes the correlation and joint uncertainty among multidimensional random variables.
運動模型	Motion Model	Describes how the state changes after the robot executes a control (noise included).

Chinese Term	English Term	Explanation
觀測模型	Observation Model	Describes what the sensor would measure in a given state (noise included).
貝氏濾波	Bayes Filter	The general framework estimating state recursively through the two steps of prediction and update.
卡爾曼濾波	Kalman Filter	The optimal recursive estimator for a linear Gaussian system.
卡爾曼增益	Kalman Gain	The weight determining whether to believe the prediction or the observation.
擴展卡爾曼濾波	EKF	Applies the Kalman filter after linearizing a nonlinear system.
粒子濾波	Particle Filter	A nonparametric filter approximating an arbitrary distribution with a large number of samples.
蒙地卡羅定位	MCL	Localization within a known map by particle filter—that is, AMCL.

11.1 Why SLAM Must Handle Uncertainty

Recall the striking experiment of Chapter 7: a device completely at rest, whose "position" computed by integrating the IMU drifted by several meters within 60 seconds. Think too of the Wi-Fi positioning of Chapter 4, the GPS jumps of Chapter 5, and the camera mismatches of Chapter 9—not one sensor gives a true value; what each gives is a guess carrying noise.

There are two attitudes one may take toward this reality. The naive approach is to pretend the sensors are accurate and use their readings directly. That is exactly why dead reckoning fails—take every erroneous measurement for truth and accumulate it, and the error accumulates without end.

The mature approach is to admit that every measurement is uncertain, quantify that uncertainty, and then strike the best balance among several uncertain pieces of information. This is the central idea of probabilistic robotics and the mathematical backbone of modern SLAM.

The Key Idea: What Is Estimated Is a Distribution, Not a Value

This is the most important conceptual shift in the chapter. Traditional thinking says "the robot is at (3.5, 2.1)"; probabilistic thinking says "the robot is most likely near (3.5, 2.1), but it may also be anywhere within an ellipse around it, with likelihood increasing toward the center."

In other words, what SLAM estimates is not a point but a whole probability distribution—it tells us both the best guess and how trustworthy that guess is. That trustworthiness is crucial, because it determines how far the next step should believe this estimate. The probabilistic definition of SLAM given in Chapter 1, p(x, m | z, u), is

precisely saying "what the probability distribution of the state looks like, given all observations and controls."

An Intuitive Analogy: Walking Blindfolded

Imagine walking blindfolded through a familiar room. With each step you roughly know how far you have advanced (motion),

but not exactly—perhaps a little more, perhaps a little less (motion noise). Your grasp of your own position grows vaguer and vaguer.

Then your hand touches the corner of a wall (an observation). That definite landmark makes your grasp of your position sharp at once.

SLAM is doing exactly this: uncertainty enlarges while walking (prediction) and contracts on touching a landmark (update).

What this chapter teaches is how to describe these two actions—blurring and sharpening—precisely in mathematics.

11.2 The Gaussian Distribution: Mean, Variance, and Covariance

Describing uncertainty mathematically requires a distribution. The one most used in SLAM is the Gaussian (or normal) distribution, because it has two irreplaceable advantages: it is completely described by only two parameters, and it remains Gaussian under a variety of operations (which is mathematically very friendly).

The One-Dimensional Gaussian: A Bell Curve

The one-dimensional Gaussian is the familiar bell curve, determined completely by two parameters:

- The mean μ (mu): the center of the curve, representing the most likely value—our best estimate.
- The variance σ^2 (sigma squared): how fat or thin the curve is, representing the magnitude of the uncertainty. A small variance gives a tall, narrow curve, meaning high confidence; a large variance gives a short, wide one, meaning great uncertainty.

The standard deviation σ is the square root of the variance and shares units with the measurement, which is more intuitive. "The robot is at 3.5 meters with a standard deviation of 0.2 meters" means that the best estimate is 3.5 and the truth falls between 3.3 and 3.7 with about 68% probability (the one-standard-deviation range of a Gaussian).

The Multivariate Gaussian: From a Point to an Ellipse

A robot's position is two-dimensional (x, y), so the uncertainty becomes two-dimensional as well. The mean is then a point (the most likely position), while the uncertainty is described by a covariance matrix.

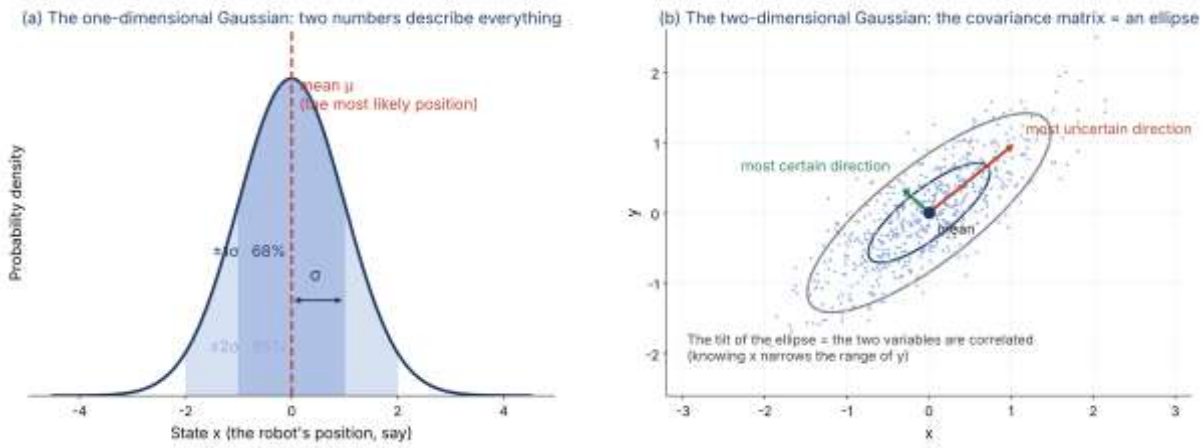
The geometric meaning of the covariance matrix is very intuitive: it corresponds to an uncertainty ellipse. The ellipse's center is the mean, its size reflects how uncertain the estimate is, and its shape and orientation reflect which directions are more uncertain.

- Diagonal elements: the variance along each axis (how uncertain in x, how uncertain in y).

- Off-diagonal elements (the covariances): the correlation between x and y. A positive value means "when x runs large, y tends to run large too," and the ellipse tilts.

Take an example from SLAM: a robot driving straight along a corridor is very uncertain about how far it has traveled (along the corridor, where error accumulates) but quite certain about whether it has strayed from the centerline (perpendicular, where the walls keep correcting it). Its uncertainty ellipse is therefore a long, narrow one stretched along the corridor. This ellipse keeps deforming as the robot moves and observes—every step of SLAM is, in essence, updating this ellipse.

Figure 11-1 The Gaussian Distribution: From a One-Dimensional Bell Curve to a Two-Dimensional Uncertainty Ellipse



The left panel is the one-dimensional case. The whole curve is determined by just two numbers: the mean μ (the most likely position) and the standard deviation σ (how uncertain it is). 68% of the probability falls within ±1σ and 95% within ±2σ—two numbers worth memorizing, because judging in practice whether an estimate is accurate enough uses them constantly.

The right panel is the two-dimensional case, and that ellipse recurs throughout the rest of this book. Note that it is tilted—which means the errors in x and y are correlated. The

red arrow points along the ellipse's major axis, the most uncertain direction; the green arrow along the minor axis, the most certain.

This tilt has a concrete physical meaning in SLAM. As a robot advances along a corridor, its lateral position can be locked precisely by the walls on either side, while its longitudinal position can only accumulate from odometry—so the uncertainty ellipse is stretched along the corridor. The degeneracy of Section 17.5 and the "distant points measure poorly" of Chapter 21 both draw this same elongated ellipse.

Remember one thing in particular: a covariance matrix is not a heap of abstract numbers, it is this ellipse. The diagonal elements set its size along each axis and the off-diagonal elements set how far it tilts. When you see the symbols P and Σ hereafter, an ellipse should come to mind at once.

Why the Gaussian Is Preferred

- Two parameters suffice: mean and covariance describe the uncertainty completely, and both storage and computation are light.
- Closure under operations: the product of two Gaussians is Gaussian, and a Gaussian under a linear transformation remains Gaussian. This lets every step of a filter stay Gaussian rather than growing ever more complicated, and it is the mathematical root of the Kalman filter's elegance.
- The central limit theorem: many independent small errors superimposed tend toward a Gaussian. Sensor noise is often precisely the sum of many tiny factors, so the Gaussian assumption often holds in practice.

A caution, however: the Gaussian assumption is not universal. When the uncertainty is multimodal (the robot may be in one of two identical-looking rooms but does not know which), a single Gaussian cannot describe it—which is exactly why the

particle filter enters in Section 11.7.

11.3 Sensor Noise and the Motion Model

Placing a robot in a probabilistic framework requires two models, which are the inputs to every filter. The two describe how the robot moves and how it sees.

1. The Motion Model

The motion model answers: "after the robot executes some control (advance one meter, turn 30 degrees left), what will the new pose be?" The key is that the answer is not a definite pose but a probability distribution, because executing a control carries noise (wheel slip, uneven ground).

Mathematically it is written p(x_t | x_{t−1}, u_t): the probability distribution of the new pose x_t given the previous pose x_{t−1} and the control u_t. The effect of the motion model is to enlarge the uncertainty—every step inflates the uncertainty ellipse a little, because fresh motion noise has accumulated. This is exactly the 2D pose composition formula of Chapter 10 with a noise term added.

2. The Observation Model

The observation model answers: "if the robot really is at some pose, what should the sensor measure?" Here too the answer is a probability distribution, because the sensor is noisy.

Mathematically it is written p(z_t | x_t): the probability distribution of the observation z_t given the pose x_t. For instance, "if the robot is here, the LiDAR should measure a wall 2 meters ahead," but the actual measurement will jitter around 2 meters. The effect of the observation model is to contract the uncertainty—a definite observation (touching a corner) greatly narrows the range of possible positions.

The Division of Labor Between the Two Models

These two models correspond exactly to the two actions in the blindfolded-walking analogy of Section 11.1: the motion model enlarges the uncertainty (the longer you walk, the vaguer it gets), and the observation model contracts it (touching a landmark makes it sharp again). The whole process of SLAM is these two models alternating—move a step, take a look, move a step, take a look. The Bayesian filter of the next section expresses that alternation precisely in mathematics.

	Motion Model	Observation Model
Mathematical form	p(x_t | x_{t−1}, u_t)	p(z_t | x_t)
Question answered	Where it will be after moving	What would be seen from here
Inputs	The previous pose + the control command	The current pose
Source of noise	Slip, terrain, wheel diameter error	Sensor measurement error
Effect on uncertainty	Enlarges (inflates)	Contracts (shrinks)
Corresponding analogy	Walking blindfolded	Touching a wall corner

Table 11-1 The Motion Model and the Observation Model Compared

11.4 The Bayes Filter

The Bayes filter is the core of this chapter—it is the common skeleton of the Kalman filter, the EKF, the particle filter, and every other filter. Once it is understood, the sections that follow are merely different implementations of it.

Belief: A Probabilistic Description of the State

The central object a Bayes filter tracks is called the belief, written bel(x_t)—the probability distribution of the current state given all information so far. The belief is the mathematical substance of the uncertainty ellipse described in Section 11.1. The task of the Bayes filter is to update that belief whenever a new piece of data arrives.

A Recursion of Two Steps

The Bayes filter cycles through two steps, corresponding exactly to the two models of the previous section:

1. Prediction: the robot executes control u_t, and the motion model pushes the belief forward. This step enlarges the uncertainty—"I moved, so I am less sure where I am." Mathematically it integrates the previous belief against the motion model (summing over all possible previous poses).
2. Update (correction): the sensor yields observation z_t, and the observation model corrects the belief. This step contracts the uncertainty—"I saw a landmark, so I am surer where I am." Mathematically it multiplies the predicted belief by the observation model and normalizes.

This predict-update cycle is the very same thing as the standard SLAM pipeline of Figure 1-3 in Chapter 1 and the predict-update of the Kalman filter in Chapter 7, now given a precise probabilistic statement. Fix this two-step structure firmly in mind: it is the key to understanding every recursive estimator.

Figure 11-2 The Recursive Cycle of Bayesian Filtering

The previous belief bel(x_{t−1})

↓ ① Prediction: apply the motion model p(x_t | x_{t−1}, u_t)

→ the belief widens (uncertainty enlarges)

The predicted belief (the prior)

↓ ② Update: multiply by the observation model p(z_t | x_t) and normalize

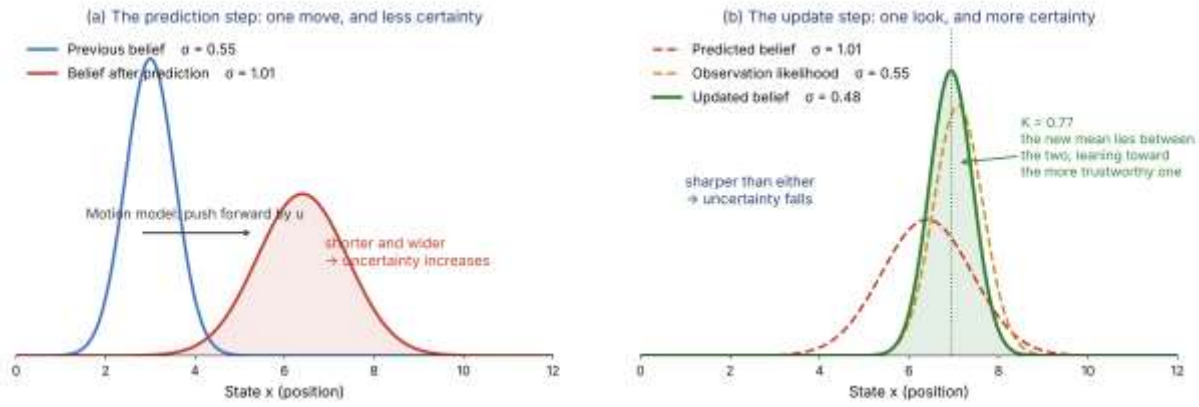
→ the belief narrows (uncertainty contracts)

The new belief bel(x_t)

↓

(back to the top for the next instant)

Figure 11-3 The Two Steps of Bayesian Filtering: Prediction Widens the Distribution, Update Sharpens It



This figure is the one most worth understanding in the chapter—arguably in the whole book. It lays out the entire Bayesian filtering cycle in a one-dimensional example.

The left panel is the prediction step. The robot was fairly sure it was at position 3 ($\sigma = 0.55$), and then odometry says "moved forward by 3.4." The mean of the belief moves forward to 6.4, but the curve also becomes shorter and wider (σ grows to 1.01)—

because motion itself carries noise, and the farther one travels the less certain one becomes. This is the integration drift of Chapter 7 and the odometry error of Chapter 13 expressed in probabilistic language: with dead reckoning alone, uncertainty can only rise monotonically.

The right panel is the update step. The sensor now reports an observation placing the position near 7.1 ($\sigma = 0.55$). Multiplying the predicted belief by the observation likelihood and normalizing gives a posterior belief lying between the two (6.94) and sharper than either (σ falls to 0.48).

Where between the two it lands is set by the Kalman gain. In this example $K = 0.77$, meaning the new mean has moved 77% of the way toward the observation—because the observation ($\sigma = 0.55$) is more trustworthy than the prediction ($\sigma = 1.01$). If the observation noise were large, K would approach 0 and the robot would nearly ignore the observation; conversely, if the observation were extremely accurate, K would approach 1 and the robot would nearly accept it outright. Section 11.5 gives the full formula for K, but its meaning is in this figure.

Three Conclusions Worth Remembering

1. The uncertainty after an update is always smaller than before it. This is the fundamental value of fusion—one more independent observation, one degree less uncertainty.

2. The fused mean always lies between the two sources and never outside that range. So a Kalman filter can never be "more absurd than either source."

3. The whole cycle has only these two steps: prediction widens, update sharpens. The particle filter of Chapter 14, the scan matching of Chapter 15,

and the visual-inertial optimization of Chapter 23 are all doing the same thing in different representations.

Bayes' Rule: The Basis of the Update Step

The mathematics behind the update step is the famous Bayes' rule of probability theory. Its spirit is to combine the prior belief (what the prediction step produced, before the observation was seen) with the evidence the observation provides, giving the posterior belief (a more precise belief after the observation).

$$\textit{posterior} \propto \textit{observation likelihood} \times \textit{prior belief} \qquad bel(x_t) \propto p(z_t \mid x_t) \cdot \textit{predicted}$$

This expression reads: the updated belief is proportional to the observation model times the predicted belief. It is the theoretical root of the update step of every filter. The Bayes filter is itself an abstract framework; in actual computation arbitrary distributions cannot be handled directly—the different filters are different answers to the questions of what form the belief takes and how the integral and the product are computed. The next three sections give three such answers.

11.5 The Kalman Filter

The Kalman filter (KF) is the exact solution of the Bayes filter under the assumption of a linear system with Gaussian noise. Under that assumption the belief is always Gaussian, so the whole filter need track only two quantities: the mean (the best estimate) and the covariance (the uncertainty). This makes it extremely efficient and has made the Kalman filter one of the most successful algorithms in engineering history—from Apollo lunar navigation to the phone in your pocket, it is everywhere.

The Two Steps of the Kalman Filter

It follows the Bayes filter architecture exactly, only with every operation carried out in the world of Gaussians:

3. Prediction: push the mean forward with the motion model (the robot moved, so the estimated position moves with it) and enlarge the covariance (adding motion noise, so uncertainty grows).
4. Update: correct the mean with the observation and shrink the covariance. The strength of the correction is set by the Kalman gain.

The Kalman Gain: The Most Ingenious Concept in the Chapter

The Kalman gain (K) is the soul of the filter. It answers a key question: when the prediction and the observation disagree, which should be believed, and by how much?

Its logic is entirely intuitive:

- If the prediction is very uncertain (large covariance) and the observation accurate (small noise): the gain is large and the observation is believed more—"I was not sure anyway, and since the sensor is so definite, let us go with it."
- If the prediction is confident and the observation noisy: the gain is small and the prediction is believed more—"I am quite sure of my own dead reckoning; this very noisy measurement is just a reference."

In other words, the Kalman filter does not stolidly take an average but weights by the trustworthiness of each side—whichever is more trustworthy is listened to more. This idea of weighting by trustworthiness is the mathematical embodiment of the fusion spirit that recurs throughout this book (Chapters 4, 5, and 7). The attitude filtering of Chapter 7 and the GNSS/INS integrated navigation of Chapter 5 both have this gain working underneath.

A Summary of the Idea: What a Kalman Filter Does

In one sentence: predict the present from the previous estimate plus the motion, correct it with the sensor observation,

and fuse the two weighted by their respective trustworthiness (covariance) to obtain an estimate more accurate than either alone.

The key insight: fusing two noisy pieces of information yields a result more accurate than either one by itself—

which is why multi-sensor fusion always beats using only the best single sensor.

11.6 The Extended Kalman Filter and EKF-SLAM

The Kalman filter is beautiful but has a strict premise: the system must be linear. Unfortunately the robot's world is full of nonlinearity—as Chapter 10 showed, a pose update contains cosθ and sinθ, and the observation model "how far would the LiDAR measure if I were here" is nonlinear too. The standard Kalman filter cannot handle this directly. The EKF (extended Kalman filter) was created for exactly this.

The EKF's Central Trick: Local Linearization

The EKF's idea is very pragmatic: since the system is nonlinear, approximate the curve by its tangent near the current estimate. Mathematically this is a first-order Taylor expansion, and the resulting matrix of slopes is called the Jacobian. Once linearized, the standard Kalman filter formulas apply.

This approximation works well when the nonlinearity is not too severe and the uncertainty not too large, and it is the most widely used estimation method in robotics—the UAV flight-control EKF mentioned in Chapter 7 and the robot_localization package in ROS both rest on it. But its approximate nature brings limits too: when the nonlinearity is strong or the uncertainty large, the tangent departs too far from the curve and the estimate may lose accuracy or even diverge.

EKF-SLAM: Putting the Map into the State as Well

How is the EKF used for SLAM? The key step is to put the robot's pose and the positions of all landmarks into one state vector and estimate them together with a single Gaussian distribution (a single large covariance matrix).

This design captures the essence of SLAM ingeniously—the coupling of localization and mapping emphasized in Chapter 1. When the robot observes a landmark and corrects its own position, the correlations in the covariance matrix propagate that correction to all related landmarks and poses. This is precisely the mathematical mechanism by which a loop closure flattens the error of a whole map at once: on returning to a familiar place, one strong observation corrects the robot and every landmark along the way together, through the correlations in the covariance.

The Historical Standing and the Limitations of EKF-SLAM

EKF-SLAM was the first complete and workable solution in the history of SLAM (in the 1990s), laid the theoretical foundation of the whole field, and holds an honored place. But it has two fatal limitations that gradually pushed it out of the mainstream:

- Computational complexity: the size of the covariance matrix grows as the square of the number of landmarks. With many landmarks (thousands), the

matrix operations become too slow for real time—a hard limit for large scenes with tens of thousands of landmarks.

- Accumulating linearization error: the EKF makes a linear approximation at every step, and the errors accumulate, which over long runs or large scenes may make the estimate inconsistent.

These two limitations gave rise to the two mainstream lines that followed: the particle filter of Chapter 14 (handling it another way) and graph optimization in Chapter 12 (replacing recursive filtering with batch optimization). One may say that understanding the limitations of EKF-SLAM is understanding why modern SLAM turned to graph optimization—which makes this a pivotal chapter between what came before and what comes after.

11.7 The Particle Filter and Monte Carlo Localization

The whole Kalman family assumes the belief is a single Gaussian. But as the end of Section 11.2 noted, in some situations the belief is multimodal—the robot may be in one of two identical rooms, and the belief should have two peaks. A single Gaussian cannot describe this. The particle filter offers an entirely different approach.

Representing a Belief with a Crowd of Samples

The central insight of the particle filter is this: rather than describing the distribution with a mathematical formula, approximate it with a large crowd of samples (particles). Each particle represents one hypothesis that "the robot might be here." Where the particles are dense, the likelihood is high.

The greatest advantage of this representation is that it can describe a distribution of any shape—multimodal, irregular, non-Gaussian, all are fine. This the Kalman family

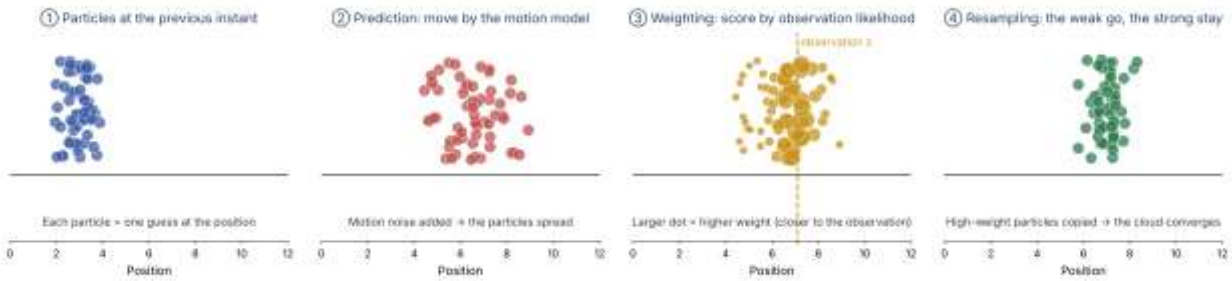
cannot do.

The Three Steps of the Particle Filter

It too follows the Bayes filter architecture, but realized with particles:

5. Prediction (move the particles): each particle moves according to the motion model, with its own random noise added. The once-clustered cloud of particles spreads out—corresponding to the enlargement of uncertainty.
6. Update (weigh them): the observation model scores each particle (assigns a weight). Particles that agree with the observation (the wall expected at this position matches what was measured) score high, and those that do not score low.
7. Resampling: particles are drawn afresh according to their weights—high-scoring particles are copied several times and low-scoring ones are eliminated. The cloud gradually gathers at the most likely position. This is the essence of the particle filter: survival of the fittest.

Figure 11-4 The Four Steps of the Particle Filter: Representing a Probability Distribution with a Crowd of Samples



The particle filter abandons describing a distribution with a formula and represents it with a crowd of samples instead. Every particle is a complete guess at the state—in the localization problem, a hypothesis that "the robot might be here, facing this way." Where the particles are dense, the probability is high.

The four steps correspond exactly to the two steps of Figure 11-3. ① is the particle cloud at the previous instant. ② Prediction: each particle moves forward on its own according to the motion model with random noise added—the result is that the particles spread, corresponding to the "widening" in the Gaussian version. ③ Weighting: each particle is scored by the observation likelihood, and those agreeing with the observation receive high weight (drawn as larger dots). ④ Resampling: particles are drawn anew by weight, high-weight particles copied several times and low-weight ones eliminated, so the cloud converges—corresponding to the "sharpening" in the Gaussian version.

What, then, makes it better than the Kalman filter? The key is that the particle cloud may take any shape. The Kalman filter is locked into a unimodal Gaussian, while real localization problems are often multimodal: a robot at power-on may be anywhere in the building; walking a symmetric corridor, the observations at two positions look identical. In such situations "a mean plus an ellipse" simply cannot express the belief, while a particle cloud can keep a group of particles in each of two places and decide which to keep once more observations arrive.

The price is computation. The number of particles must grow exponentially with the dimension of the state, so the particle filter is very practical for 2D localization (three degrees of freedom) but cannot cope in high dimensions. That is exactly why Gmapping in Chapter 14 must adopt the Rao-Blackwellization trick—particles only for the pose, with the map handled analytically.

Monte Carlo Localization (MCL) and AMCL

Applying the particle filter to localization within a known map gives Monte Carlo localization (MCL), whose implementation in ROS is called AMCL (mentioned in

Chapter 3). Its operation is extremely intuitive: not knowing where the robot is at first, scatter particles everywhere on the map; as the robot moves and scans, particles inconsistent with the observations are eliminated and the cloud gradually converges on the true position.

MCL inherently solves the kidnapped robot problem of Chapter 3: even if the robot is carried away, so long as some particles remain scattered across the map they can win out at the new position and let the robot find itself again. This is hard for a single-Gaussian Kalman filter to do—and it is the mathematical reason a cleaning robot can relocalize after being carried into another room.

The Particle Filter and SLAM: A Preview of Gmapping

Applying the particle filter to SLAM (rather than localization alone) requires an ingenious trick called the Rao-Blackwellized particle filter (RBPF)—each particle represents not only a pose but carries a whole map of its own. This is the core of Gmapping in Chapter 14 and the protagonist of the 2D SLAM of Part IV. This chapter sets out the principle of the particle filter clearly so that Chapter 14 can concentrate on how it builds maps.

Characteristic	The Kalman / EKF family	The particle filter
Belief representation	A single Gaussian (mean + covariance)	A crowd of particle samples
Distributions it can handle	Gaussian only (unimodal)	Any shape (multimodal possible)
Nonlinearity	The EKF relies on linearized approximation	Naturally supported, no linearization needed
The kidnapped	Hard to handle	Inherently solvable

Characteristic	The Kalman / EKF family	The particle filter
robot problem		
Computational cost	Low (in low dimensions)	High (many particles needed)
Representative applications	GNSS/INS, EKF-SLAM	AMCL, Gmapping

Table 11-2 The Kalman Family Compared with the Particle Filter

You now command the core estimation tools of SLAM: describing uncertainty with probability, handling it with the predict-update of Bayesian filtering, and three concrete implementations in the Kalman filter, the EKF, and the particle filter. These belong to the recursive filtering route—update once for every piece of data. But modern SLAM has another, more powerful route: rather than processing data one piece at a time, accumulate a batch and optimize it all at once. The next chapter, "Optimization and Graph-Based SLAM," introduces the route that dominates contemporary SLAM—pose graphs, bundle adjustment, and g2o, Ceres, and GTSAM.

Formulations and Procedures

1. The One-Dimensional Gaussian Distribution

The probability density function of a one-dimensional Gaussian:

$$p(x) = (1 / \sqrt{(2\pi \sigma^2)}) \cdot \exp(-(x-\mu)^2 / (2\sigma^2))$$

where μ is the mean (the center of the distribution, the best estimate) and σ^2 the variance (the magnitude of the uncertainty). About 68% of the probability falls within μ±σ, 95% within μ±2σ, and 99.7% within μ±3σ—this "68-95-99.7 rule" is the most practical intuition for reading uncertainty.

2. Bayes' Rule

The theoretical core of state estimation:

$$p(x \mid z) = p(z \mid x) \cdot p(x) \;/\; p(z)$$

Read as: posterior = (likelihood × prior) / evidence. In filtering, p(x) is the prior belief from the prediction step, p(z|x) is the observation model, and p(x|z) is the updated posterior belief. The denominator p(z) is only a normalizing constant (ensuring the probabilities sum to 1), so it is often written in proportional form:

$$p(x \mid z) \propto p(z \mid x) \cdot p(x)$$

3. The Two Steps of the Bayes Filter

Prediction step (apply the motion model and integrate over the previous state):

$$\overline{bel}(x_t) = \int p(x_t \mid x_{t-1}, u_t) \cdot bel(x_{t-1})\, dx_{t-1}$$

Update step (multiply by the observation model and normalize):

$$bel(x_t) = \eta \cdot p(z_t \mid x_t) \cdot \overline{bel}(x_t)$$

where $\overline{bel}$ is the predicted belief (the prior), bel the updated belief (the posterior), and η a normalizing constant. The Kalman filter, the EKF, and the particle filter are all concrete realizations of these two expressions under different representations.

4. The Core Kalman Filter Equations (Conceptual Form)

Prediction:

$$\text{mean: } \bar{x} = F \cdot \hat{x} \qquad \text{covariance: } \bar{P} = F \cdot P \cdot F^T + Q$$

Update:

$$\text{Kalman gain: } K = \bar{P} \cdot H^T \cdot (H \cdot \bar{P} \cdot H^T + R)^{-1}$$

$$\text{mean: } \hat{x} = \bar{x} + K \cdot (z - H \cdot \bar{x}) \qquad \text{covariance: } P = (I - K \cdot H) \cdot \bar{P}$$

where F is the motion model, H the observation model, Q the motion noise, and R the observation noise. Look closely at the expression for the gain K: when the observation noise R is small (the observation is accurate), K grows and the update trusts the observation more (the innovation $z - H\cdot\hat{x}^{-}$, the difference between observation and prediction, is multiplied by a large K); when R is large, K shrinks and the prediction is trusted more. This expression gives precise mathematical form to the "weighting by trustworthiness" intuition of Section 11.5.

5. A Procedure for Choosing Among the Three Filters

Figure 11-5 A Decision Procedure for Choosing a Filter

Question 1: is the system linear?

├─ Yes → the Kalman filter (KF): optimal and fastest

└─ No → go to the next question

↓

Question 2: is the nonlinearity mild, the belief unimodal, and a good initial estimate available?

├─ Yes → the extended Kalman filter (EKF): efficient, by linearized approximation

└─ No (strong nonlinearity / multimodality / the kidnapped robot problem) → go to the next question

↓

Question 3: is the state dimension low and a large number of particles affordable?

├─ Yes → the particle filter (PF / MCL): can handle any distribution

└─ No (high dimension, large scene) → consider the graph optimization of Chapter 12

11.8 Contemporary Developments in State Estimation: Filtering Has Not Left, It Has Moved

The Kalman family and the particle filter introduced in this chapter are often described in textbooks as classical methods, as though they had been superseded by the graph optimization of Chapter 12. That impression is only half right, and the distinction is

worth drawing carefully.

1. The Back End Really Has Been Taken Over by Optimization

In the back end of SLAM—the layer that maintains the whole trajectory and the whole map—optimization has indeed won out completely. The reason was given in Section 11.6: the covariance matrix of EKF-SLAM grows as the square of the number of landmarks and cannot cope beyond a few thousand; and the linearization is performed only once, so an early error cannot be revisited.

The graph optimization of Chapter 12 retains all historical poses and observations, can relinearize repeatedly, and can correct an entire trajectory at once on loop closure. Add incremental solvers such as iSAM2, which update only the affected part, and optimization is no longer at a disadvantage in speed either. ORB-SLAM3, Cartographer, and LIO-SAM today all take this route without exception.

2. But the Front End and the High-Rate Loops Remain the Domain of Filtering

Zoom in, however, to real-time estimation several hundred times a second, and filtering remains irreplaceable. Attitude estimation in a UAV flight controller, body state estimation in a quadruped, and the high-rate IMU dead reckoning of Chapter 7 are all still run by an EKF—because optimization needs a batch of data accumulated before it can solve, and a control loop cannot wait.

This yields a practical division of labor that students should internalize: high-rate, low-dimensional, real-time → filtering; low-rate, high-dimensional, globally consistent → optimization. A complete robotic system has both, layered side by side rather than chosen between. LIO-SAM in Chapter 18 and ORB-SLAM3 in Chapter 23 are both such layered architectures.

3. The Invariant EKF: Making Filtering Dependable Again

Filtering itself is advancing too. Section 11.6 noted that the fundamental weakness of the EKF is linearization error—when the estimate departs far from the truth, the Jacobian is computed at the wrong place and the filter may diverge. The recent invariant extended Kalman filter (InEKF) exploits the Lie group structure mentioned in Section 10.8 and defines the state on a matrix Lie group; under certain conditions its error dynamics are independent of the estimate, which gives it better convergence properties.

This line has been especially successful on legged robots and has become the standard approach to body state estimation on many quadrupeds and humanoids. For students it is a fine illustration: the Lie groups that looked abstract in Chapter 10 translate directly here into a more stable filter.

4. Neural Compensation: Letting a Filter Handle What It Cannot Model

The newest line hangs a learned model alongside the filter to handle the cases where the model assumptions break down. A 2026 study on legged robots is a typical example: it adds a neural compensator built around an attention mechanism on top of an invariant EKF.

Its design logic repays study. State estimation for a legged robot relies on the assumption that a planted foot does not slip, but that assumption collapses on gravel or low-friction surfaces. The study treats the degree of foot slip as an explicit conditioning variable, encodes the filter's state history and the degree of slip separately with GRU autoencoders, uses cross-attention to produce a compensation adjusted to the slip situation, and finally adds a nine-dimensional residual correction to the filter's posterior state.

The effect is considerable: position error on gravel fell by about 93% relative to the strongest baseline, on low-friction Teflon panels by about 49%, and in outdoor grass tests of about a hundred meters by about 55% to 73%, while the whole still runs in real time at 580 hertz.

Note the structure—identical to the learning-based IMU of Section 7.8 and the depth estimation models of Section 9.8: keep the geometric and probabilistic skeleton and let the neural network compensate for the parts physics cannot write down. This is already the most robust pattern for integrating AI in robotics today, and Chapter 32 discusses it again from a higher vantage.

These Four Developments in One Sentence

Filtering has not been superseded; it has merely withdrawn from taking charge of the whole map to taking charge of the immediate moment—and in that position it remains irreplaceable.

You are not studying this chapter as archaeology: writing UAV attitude estimation in Chapter 26, tuning AMCL parameters in Chapter 29, and understanding in Chapter 16 why a cleaning robot suddenly remembers where it is after getting lost all use what is in this chapter.

Case Studies

Case 11-1 The Moon Landing: The Kalman Filter's Battle of Renown

The Kalman filter was introduced by Rudolf Kálmán in 1960, and its first world-shaking application was the Apollo lunar program. A spacecraft traveling between Earth and Moon required a navigation system that fused noisy data from inertial

measurement, ground tracking, and other sources to estimate position and velocity in real time—the perfect stage for the Kalman filter's "fuse many sources by trustworthiness."

The spacecraft's computing power was extremely limited at the time (far less than a pocket calculator today), but the Kalman filter needs to track only two quantities, the mean and the covariance, and computes recursively (with no need to store all past data), so it could run in real time on such straitened hardware. One may fairly say that without the Kalman filter, Apollo's navigation would have been nearly impossible.

This case echoes Apollo's gimbal lock in Chapter 10—one and the same lunar mission drove the development of both rotation representation and state estimation. It also offers a lesson: these seemingly abstract probabilistic tools are the practical bedrock on which humanity walked on the Moon. More than sixty years later, the positioning in your phone and the navigation of your cleaning robot still use the same mathematics.

Case 11-2 The Cleaning Robot's "Particle Cloud": Visualizing AMCL

Many cleaning robot apps that display a map will, in a debug or advanced mode, show a vivid scene: at power-on, a great many small dots (particles) representing possible positions are scattered over the map; as the robot moves a few steps and scans a few times, those dots converge rapidly from "scattered over the whole map" into "a tight little cluster"—the robot has worked out where it is.

This is exactly a live visualization of the AMCL of Section 11.7. Scattered particles mean "I am not yet sure where I am" and a converged cluster means "now I am sure." Carry the robot into another room and the cluster will first become confused

(inconsistent with the observations) and then regather at the correct new position—the solving of the kidnapped robot problem unfolding before your eyes.

The teaching value of this case is that it turns the chapter's most abstract idea, a probability distribution, into a visible cloud of particles. In university classrooms, instructors often have students watch the particles evolve in AMCL within RViz, and alongside the theory of this chapter the abstract "contraction of belief" becomes concrete at once. It is another example of this book's emphasis on understanding the principle first and then observing it by hand.

Case 11-3 Sensor Fusion in a Self-Driving Car: The Gain Balancing Trust on a Real Road

A self-driving car carries GPS, an IMU, wheel encoders, LiDAR, and cameras at once, each with its own account of where the car is and each dependable at different moments. The Kalman family (usually an EKF or a variant) is the mainstay of fusing this information.

The Kalman gain of this chapter is embodied here very concretely: when the vehicle enters a tunnel and GPS loses lock (the observation noise R soars), the filter automatically lowers its trust in GPS and leans on the dead reckoning of the IMU and the wheel encoders (the integrated navigation of Chapter 5); after leaving the tunnel and GPS recovers, the filter gradually shifts trust back. This dynamic adjustment of trust by situation needs no hard-coded rules from an engineer; it follows naturally from the mathematics of the covariance and the gain.

This is precisely the fundamental advantage of the probabilistic approach over hard-coded rules: it lets the system weigh each information source by its present

trustworthiness and adapt gracefully to any situation. The idea runs through the whole book—from the indoor positioning fusion of Chapter 4 and the GNSS/INS of Chapter 5 to the attitude filtering of Chapter 7 and the unified mathematical statement of this chapter, you should be able to feel that Parts II and III are really treating different faces of one thing.

Hands-On Activities

Exercise 11-1 Building a One-Dimensional Kalman Filter and Particle Filter by Hand

Purpose: using the simplest one-dimensional example, to implement a Kalman filter and a particle filter by hand and see with your own eyes how uncertainty widens and contracts and how a particle cloud converges. This exercise is mainly in Python and is the key step to understanding all the theory of this chapter.

Part One: A One-Dimensional Kalman Filter (everyone)

1. Set up the scenario: a robot on a one-dimensional track, true position unknown. It has a drifting velocity sensor (the motion model, with large noise) and a position sensor that reports occasionally (the observation model, with moderate noise). Generate a stretch of noisy simulated data in code.
2. Implement the prediction step of the Kalman filter: at each step update the mean position with the velocity and enlarge the variance. Plot the estimated position and the uncertainty band (mean ± standard deviation) against time, and observe how the band keeps widening with prediction alone.
3. Add the update step: whenever the position sensor reports, correct the mean with the observation and shrink the variance. Observe how the uncertainty band contracts at each observation.

4. Plot the estimated trajectories of pure prediction (dead reckoning) and the complete Kalman filter together, compare them with the truth, and quantify how much error the Kalman filter removed.
5. Experiment with the Kalman gain: set the observation noise R very large and then very small, and observe how the gain changes and how the estimated trajectory leans more toward prediction or toward observation.

Part Two: A One-Dimensional Particle Filter (everyone, or undergraduate)

6. Scatter 500 particles (uniformly distributed) along the one-dimensional track, representing "not knowing where the robot is at first."
7. Implement the three steps: at each step move all particles by the motion model (with noise added), weight each particle by the observation, and resample by weight.
8. Plot a histogram of the particle distribution every few steps and observe how the cloud converges from "spread over the whole track" to "near the true position."
9. Kidnapping test: move the true position elsewhere abruptly partway through and observe how the cloud first becomes confused and then regathers at the new position.

Part Three: A Multimodal Scenario (undergraduate)

10. Design a symmetric corridor scenario in which the observations at two locations are identical (a symmetric environment). Handle it with a particle filter and observe how the cloud maintains two peaks—verifying first-hand the claim of Section 11.7 that a particle filter can handle multimodality and a Kalman filter cannot.

11. Consider: what would happen if a Kalman filter handled this symmetric scenario? (Hint: a single Gaussian would place the mean midway between the two peaks—a position the robot cannot possibly occupy.)

Discussion Questions

1. In Part One, how much error did the complete Kalman filter remove relative to pure dead reckoning? Where does that improvement come from?
2. When you set the observation noise R very small, what happened to the estimated trajectory? Does this match the account of the Kalman gain in Section 11.5?
3. In the particle filter kidnapping test, how long did the cloud take to regather? What would happen with only 50 particles rather than 500? What trade-off regarding the number of particles does this reveal?
4. (Undergraduate) In the symmetric corridor scenario the particle filter maintained two peaks while a Kalman filter would fail. What practical lesson does this offer about choosing a filter?

Extended Challenge (Undergraduate)

Extend the one-dimensional Kalman filter to two dimensions (x, y), track the covariance matrix, and plot the uncertainty ellipse at every step with matplotlib; observe how the ellipse stretches along the direction of travel as the robot moves in a straight line and how it contracts after an observation. This two-dimensional visualization corresponds directly to Section 11.2 and builds the intuition for Gmapping in Chapter 14 and the AMCL exercises of Chapter 29.

Further Reading

1. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (The bible of probabilistic robotics and the authoritative source for every topic in this chapter; Chapters 2 through 8 are especially relevant.)
2. Cyrill Stachniss, the Bayes Filter, Kalman Filter, and Particle Filter video series from his SLAM course (YouTube). (Explains all three filters intuitively with animation; the first choice for self-study.)
3. R. Faragher, "Understanding the Basis of the Kalman Filter via a Simple and Intuitive Derivation," IEEE Signal Processing Magazine, 2012. (The friendliest intuitive derivation of the Kalman filter.)
4. R. Labbe, "Kalman and Bayesian Filters in Python" (a free e-book on GitHub). (With complete runnable code; the best companion for implementing this chapter.)
5. D. Fox et al., "Monte Carlo Localization for Mobile Robots," ICRA, 1999. (The original paper on Monte Carlo localization.)
6. The filtering units of the CMU 16-833 Robot Localization and Mapping course (introduced in Chapter 2). (Graduate-level material on EKF-SLAM and the particle filter.)
7. Xiang Gao and Tao Zhang, Fourteen Lectures on Visual SLAM, the first part of Lecture 6 (nonlinear optimization). (The bridge from the filtering of this chapter to the graph optimization of Chapter 12.)

From Filtering to Optimization: Where This Chapter Stands

This chapter has written the SLAM problem as a probabilistic one and given three ways of solving it. But all three share one character: they are recursive—they keep only the current belief, update it whenever new data arrives, and then discard the old data.

The advantage is fixed memory and steady computation, very well suited to a real-time system. But the price is equally clear: once an estimate at some step goes wrong, the error is pressed into the current belief and can never be revisited. When the robot comes all the way around to its starting point and finds the map does not line up, a filter can only correct slowly from that moment on; it cannot go back and straighten the whole stretch of trajectory in between.

What Chapter 12 introduces is the opposite approach: throw nothing away. Treat the pose at every instant as a variable to be solved for, treat every observation as a constraint, and put them all into one enormous optimization problem to be solved at once. The moment loop closure detects "I am back at the start," that new constraint pulls the entire trajectory back to the right place—something filtering can never do.

Interestingly, the two have the same mathematical root: the maximum a posteriori estimate that optimization seeks and the peak of the posterior distribution that filtering pursues are, under Gaussian assumptions, the same thing. Chapter 12 explains this equivalence. So nothing learned in this chapter goes to waste—it has merely changed its manner of computation.

Summary

1. SLAM must handle uncertainty probabilistically: every sensor gives a noisy guess, and the mature approach is to quantify the uncertainty and then strike the best balance. The key idea is that what is estimated is a probability distribution (a best guess plus a trustworthiness), not a single value.
2. A Gaussian distribution is described completely by two parameters, the mean (the best estimate) and the variance (the uncertainty); in several dimensions the

covariance matrix corresponds to an uncertainty ellipse. The Gaussian is the first choice in SLAM because two parameters suffice, it is closed under the relevant operations, and the central limit theorem supports it—but it cannot describe a multimodal distribution.

3. The motion model p(x_t|x_{t−1},u_t) describes where motion leads and enlarges the uncertainty; the observation model p(z_t|x_t) describes what would be seen from here and contracts it. The two are the inputs to every filter.
4. The Bayes filter is the common skeleton of every filter, recursing through two steps—prediction (apply the motion model, the belief widens) and update (apply the observation model, the belief narrows)—with the update grounded in Bayes' rule: posterior ∝ likelihood × prior.
5. The Kalman filter is the optimal solution under linear Gaussian assumptions and need track only the mean and the covariance; the Kalman gain balances prediction against observation by their trustworthiness—the mathematical core of why multi-sensor fusion beats any single sensor.
6. The EKF handles nonlinearity by local linearization with a Jacobian and is the most widely used estimator in robotics; EKF-SLAM places poses and landmarks in one state and estimates them jointly, embodying the coupling of localization and mapping, but its covariance grows as the square of the number of landmarks and its linearization error accumulates—which gave rise to the graph optimization route.
7. The particle filter approximates an arbitrary distribution with a crowd of samples through three steps—prediction, weighting, resampling—and can handle multimodality and nonlinearity and inherently solve the kidnapped

robot problem; Monte Carlo localization (AMCL) is its localization application and the Rao-Blackwellized version (Gmapping) its SLAM application.

The filters of this chapter are all recursive—update once per piece of data and then discard it. That is efficient, but it also means that once past information has been compressed into the current belief it can never be revisited. The next chapter, "Optimization and Graph-Based SLAM," introduces the other route: retain a batch of poses and observations, build them into a pose graph, and optimize all at once for the most consistent solution. This route is more accurate and handles loop closure better, and it is the mathematical foundation of contemporary mainstream systems such as ORB-SLAM and Cartographer.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 12

Optimization and Graph-Based SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 11 wrote SLAM as a probabilistic problem and gave three filtering solutions. But filtering has one common character: it keeps only the current belief and discards the old data at every update. That is economical in real time but costly in correctness—once an error has been pressed into the belief, it can never be revisited.

What this chapter introduces is the opposite approach: throw nothing away. Treat the pose at every instant as a variable to be solved for, treat every observation as a constraint, and put them all into one optimization problem to be solved at once. When the robot comes around to its starting point and finds the map does not line up, the new loop closure constraint can straighten the entire trajectory—something filtering can never do.

This is the mainstream architecture of modern SLAM. The 2D SLAM of Chapters 14 and 15, the LiDAR-inertial odometry of Chapter 18, and the ORB-SLAM family of Chapters 22 and 23 all have the graph optimization of this chapter as their back end. Of the three chapters in Part III, this is the one that connects most directly to implementation.

Figure 12-0 How Chapter 12 Connects to the Rest of the Book

[Building on the previous chapters]

Chapter 10 The pose T → becomes a node of the graph here

Chapter 11 Probability and information → becomes the information matrix Ω on an edge here

Chapter 11's filtering "discards history" → this chapter "retains all history"

[This chapter is responsible for]

The front-end/back-end division → the pose graph → least squares → bundle

adjustment

Sparsity: why even tens of thousands of nodes remain solvable

g2o, Ceres, and GTSAM: the tools you will actually use

[Where it leads]

→ Chapter 15 Cartographer's branch-and-bound loop closure + a pose graph back end

→ Chapter 17 An ICP registration result = one edge of a pose graph

→ Chapter 18 LIO-SAM: a factor graph of LiDAR, IMU, and GNSS

→ Chapters 22 and 23 ORB-SLAM's local BA and global BA

→ Section 7.5 What IMU preintegration produces is one edge of this chapter's graph

Learning Objectives

The filters of the previous chapter are recursive—update once per piece of data and then discard it. This chapter introduces the other route: retain a batch of poses and observations, build them into a graph, and optimize all at once for the most consistent solution. This graph optimization route is more accurate and handles loop closure better; it is the mathematical foundation of contemporary mainstream systems such as ORB-SLAM and Cartographer, and the conclusion of Part III. After completing this chapter, you should be able to:

1. Describe the division of labor between the SLAM front end and back end, and understand that graph optimization belongs to the back end.
2. Describe what the nodes and edges of a pose graph represent, and read how such a graph describes a SLAM problem.
3. Explain why SLAM can be cast as a least-squares problem.
4. Explain the concept of bundle adjustment and its role in visual SLAM.

5. Know the place and purpose of the three major optimization libraries g2o, Ceres, and GTSAM.
6. Explain why modern SLAM mostly adopts graph optimization rather than the filters of the previous chapter.

Core Concepts

Chinese Term	English Term	Explanation
前端	Front-End	Turns raw sensor data into geometric constraints (features, matches, odometry).
後端	Back-End	Finds the most consistent poses and map under all the constraints.
位姿圖	Pose Graph	A graph structure with poses as nodes and constraints as edges.
節點	Node / Vertex	A quantity to be estimated, such as a robot pose or a landmark position.
邊	Edge / Constraint	A relative measurement constraint between two nodes.
最小平方	Least Squares	The method of finding the solution minimizing the sum of squared errors.
殘差	Residual	The difference between a measured value and a predicted value.
光束法平差	Bundle Adjustment, BA	Visual optimization of camera poses and three-dimensional points together.
重投影誤差	Reprojection Error	The gap between a three-dimensional point's projection into an image and the pixel actually observed.
資訊矩陣	Information Matrix	The inverse of the covariance, representing a constraint's trustworthiness weight.

Chinese Term	English Term	Explanation
稀疏性	Sparsity	Each constraint involves only a few variables, which lets a large problem be solved efficiently.
圖優化函式庫	g2o / Ceres / GTSAM	Open-source tools for solving graph optimization problems.

12.1 The Division of Labor Between Front End and Back End

Modern SLAM systems generally adopt a two-stage front-end-plus-back-end architecture (the common skeleton of the university courses of Chapter 2 and the core of Figure 1-3 in Chapter 1). Understanding this division is the first step to reading this chapter.

The Front End: Turning Sensor Data into Geometric Constraints

The front end faces the sensors directly, and its job is to turn raw data into geometric constraints—relations such as "between position A and position B the robot moved so far" or "a certain landmark was seen at position A."

- For LiDAR SLAM: the front end performs scan matching (Chapter 13) to compute how far this frame moved relative to the previous one.
- For visual SLAM: the front end performs feature extraction and matching (Chapters 9 and 20) to compute the camera motion and the landmarks seen.
- The front end is also responsible for loop closure detection: recognizing "I have been to this place" and producing a crucial loop closure constraint.

The Back End: Finding the Most Consistent Solution Under All Constraints

The back end does not touch the sensors directly; it receives the heap of geometric constraints the front end produces, and its task is to find a set of poses and a map that satisfy all the constraints as nearly as possible at once.

Here is the difficulty: these constraints often contradict one another. Odometry says "I moved 10 meters from A to B," while loop closure detection says "B is in fact right beside A"—the two constraints are at odds. The back end's job is to find the best compromise among such contradictory constraints so that the total error is least. That

is the subject of this chapter.

Two Routes for the Back End

Historically there have been two routes to finding that best compromise:

- The filtering route (Chapter 11): process recursively, one piece at a time, as represented by the EKF and the particle filter. Its advantages are real-time operation and fixed memory; its drawback is that past information, once compressed, cannot be revisited.
- The optimization route (this chapter): retain a batch of constraints and solve the whole problem at once, as represented by graph optimization. Its advantages are high accuracy, repeated correction, and elegant handling of loop closure; its price is greater computation.

Over the past decade and more, the optimization route has become mainstream thanks to its accuracy and its handling of loop closure. This chapter sets out the mathematics and tools of that route.

12.2 The Concept of the Pose Graph

The central data structure of graph-based SLAM is a pose graph. It turns the abstract SLAM problem into an intuitive graph—the very graph of nodes and edges studied in a data structures course.

Nodes and Edges

- Node (vertex): represents a quantity to be estimated. In the simplest pose graph, each node is the robot's pose at some instant (the x, y, θ of Chapter 10, or a 6-DoF pose). In versions that include landmarks, landmark positions are nodes as well. These are the unknowns we want to solve for.

- Edge (constraint): represents a relative measurement constraint between two nodes. It records what relation some sensor says should hold between those two nodes.

Two Crucial Kinds of Edge

A pose graph has two kinds of edge of different origin but equal importance:

- Odometry edges (sequential edges): connect two poses adjacent in time, coming from odometry or scan matching—"I moved this much from frame t to frame t+1." These edges form a chain that strings the poses together.
- Loop closure edges: connect two poses not adjacent in time but reunited in space, coming from loop closure detection—"where I am now is the same place as frame 5 long ago." Such an edge forms a loop in the graph and is the key to eliminating accumulated drift.

Why Loop Closure Edges Matter So Much

Imagine a robot going once around a building and returning to its starting point. The odometry edges form a chain accumulating error all the way, so that after the circuit a noticeable gap appears between the endpoint and the start (the drift of Chapter 1). Loop closure detection then produces an edge declaring "the endpoint is the start"—and this edge contradicts the entire odometry chain.

The magic of back-end optimization is to spread that contradiction evenly over the whole trajectory: it nudges every pose along the way so that the endpoint returns to the start while disturbing the odometry constraints as little as possible. The result is that the whole trajectory is straightened and the accumulated drift is removed at a stroke. This is why loop closure is so crucial to SLAM, and a great advantage of graph optimization over filtering: it can go back and correct every historical pose.

Figure 12-1 The Structure of a Pose Graph

Nodes: ①—②—③—④—⑤—⑥—⑦—⑧ (each one a pose)

└——odometry edges (adjacent, like a chain)——┘

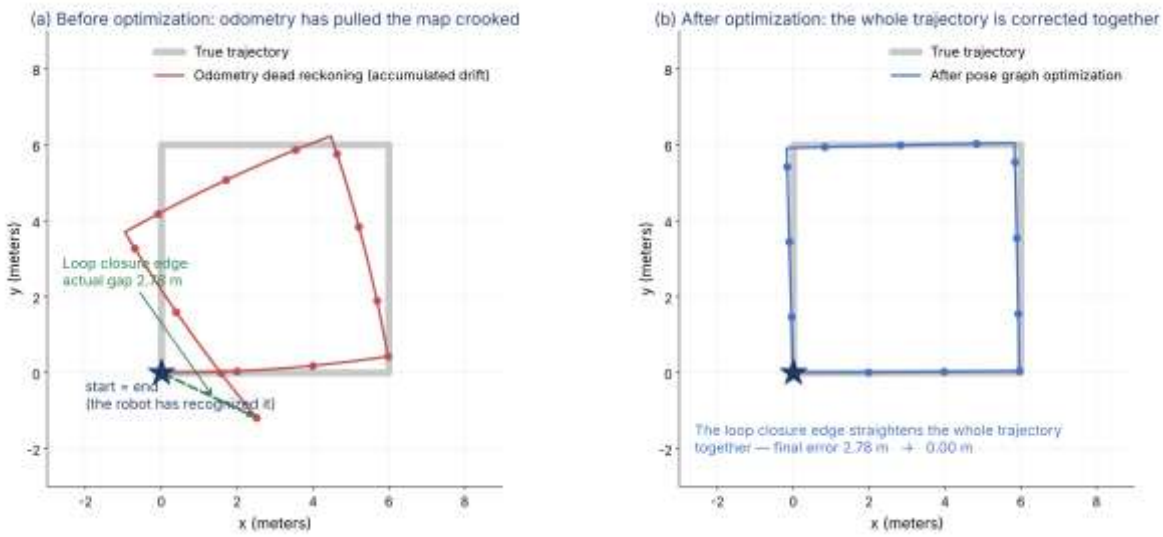
① ←————loop closure edge————→ ⑧

(frame ⑧ is detected to have returned to the position of frame ①)

Before optimization: the odometry chain has accumulated drift and ⑧ does not line up with ①

After optimization: the loop closure edge spreads the contradiction over ①–⑧ and the whole trajectory is straightened

Figure 12-2 The Power of Loop Closure: How One Edge Straightens a Whole Map



This figure is actually computed, not a sketch. A robot is simulated walking once around a 6-meter square, with odometry carrying a systematic angular bias of about 0.013

radian per step—a very typical source of error in real wheeled robots, arising for instance from a slight difference in the diameters of the left and right tires.

The left panel is the result of odometry alone. Note its shape: each leg is still fairly straight and the corners are close to 90 degrees, so it looks entirely plausible—yet the whole has rotated askew. On returning to the start after a full circuit, the dead-reckoned position is 2.56 meters from the true starting point. This is what the integration drift of Chapter 7 and the "prediction can only raise uncertainty" of Chapter 11 look like in a plane.

The crucial step occurs the moment the robot recognizes "I am back at the start." That recognition is performed by the front end—scan matching in Chapters 14 and 15, the visual bag of words in Chapters 20 and 22. The front end's conclusion is written as a loop closure edge: the relative pose between node N and node 0 should be zero.

The right panel is the result of optimizing with that edge added. The whole trajectory is straightened together, the final error falls from 2.56 meters to 0.00 meters, and the mean error along the route falls from 1.18 meters to 0.04 meters. Note the word together—it is not only the last node that is corrected; every node in between shares in the correction. This is the most fundamental advantage of graph optimization over filtering: it can go back and rewrite history.

Why Filtering Cannot Do This

At step k a filter has already compressed everything before step k into one mean and one covariance; the positions of the intermediate nodes no longer exist in the state and so cannot be corrected.

Graph optimization retains all N nodes as variables, so the new information

from a loop closure edge can propagate back along the chain of odometry edges and move every node a little.

The price is more variables and storing all of history. That is the trade-off of Section 12.7, and it is why practice often layers the two together: filtering in the front end to maintain high rate, graph optimization in the back end to maintain global consistency.

12.3 Nodes, Edges, and Constraints

This section looks more closely at the constraints that edges represent and at how to quantify the degree to which a constraint is satisfied—the key to turning a pose graph into a solvable problem.

The Residual: The Difference Between Measurement and Prediction

Every edge carries a measured value (what relation the sensor says should hold between the two nodes). From the current estimates of the two nodes' poses we can also compute a predicted value (what relation actually holds between them under the current estimate). Their difference is called the residual:

$$\textit{residual} = \textit{measured value} - \textit{predicted value}$$

A zero residual means this edge's constraint is perfectly satisfied; the larger the residual, the more the current pose estimate violates that constraint. The back end's goal is to adjust all the nodes so that the residuals of all the edges are as small as possible.

Not All Constraints Are Equally Trustworthy: The Information Matrix

A key detail here: different edges differ in trustworthiness. An edge from high-

accuracy LiDAR scan matching is far more trustworthy than one from slipping odometry. The optimization should respect the trustworthy edges more.

That trustworthiness is described by an information matrix—which is exactly the inverse of the covariance matrix of Chapter 11. A large covariance (uncertain) corresponds to a small information matrix (low weight); a small covariance (certain) to a large information matrix (high weight). Trustworthy constraints thus speak louder in the optimization. This joins graph optimization perfectly to the probabilistic framework of Chapter 11: the two are different solutions to the same probabilistic problem.

The Uniformity of Constraints

The beauty of the pose graph is that it unifies constraints of every origin—odometry, scan matching, loop closure, even the UWB ranging of Chapter 4, the GNSS of Chapter 5, and the IMU preintegration of Chapter 7—into edges between nodes. Whatever sensor a constraint comes from, in the graph it is an edge with a residual and an information matrix. This uniformity is why graph optimization fuses multiple sensors so elegantly, and it echoes the tight coupling idea recurring in earlier chapters: put all the raw constraints into one optimization problem and solve them together.

12.4 Least Squares

Now let us gather the foregoing into a definite mathematical problem. Since the residuals of all edges are to be made as small as possible, the natural approach is to find a set of node values minimizing the sum of the squares of all residuals. This is least squares.

Why the Sum of Squares

Why the sum of squares rather than the sum of absolute values of the residuals?

For two reasons:

- A mathematical reason: the square function is differentiable everywhere and has a pleasant form, which makes it convenient to find a minimum by calculus (setting the derivative to zero). The absolute value is not differentiable at zero and is awkward to handle.
- A probabilistic reason: when the noise is Gaussian (Chapter 11), minimizing the sum of squared residuals is mathematically equivalent to maximizing the posterior probability (finding the most probable solution). In other words, least squares is not chosen arbitrarily; it is the optimal estimate under the Gaussian assumption. This shows once again that graph optimization and the probabilistic framework of Chapter 11 are the same thing.

With the information matrix added as a weight, the more precise statement is "minimize the weighted sum of squared residuals"—trustworthy edges carry greater weight and influence the sum more.

How It Is Solved: Successive Approximation

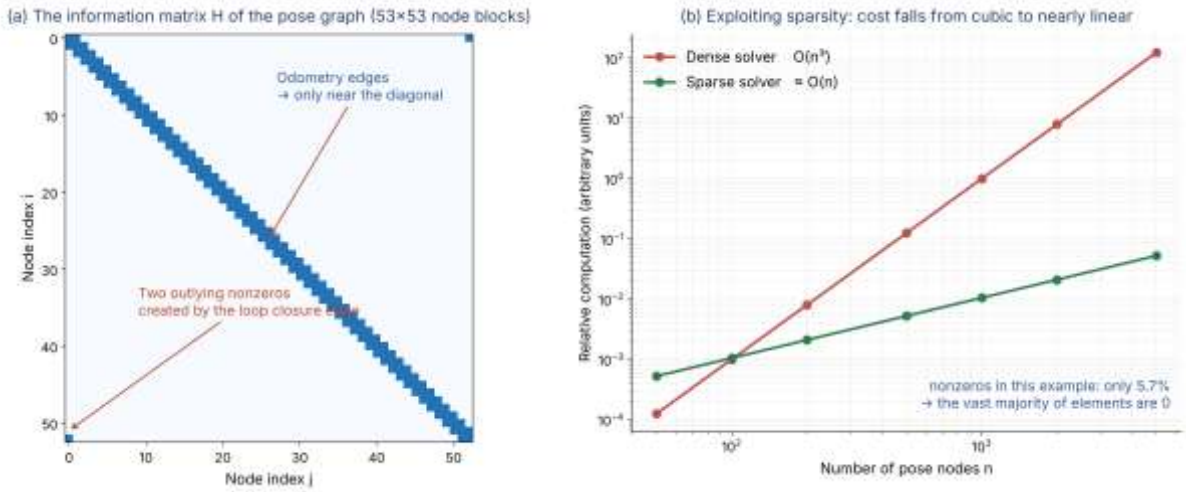
This least-squares problem is usually nonlinear (because the residuals contain cosθ, sinθ, and the like, from Chapter 10) and cannot be solved in one step. In practice it is approached iteratively:

1. Begin from an initial guess (usually the rough trajectory strung together from odometry).
2. Linearize the problem near the current estimate (computing a Jacobian, the same trick the EKF used in Chapter 11).
3. Solve a linear system to obtain a step correcting in the direction of smaller error.

4. Update the estimate and return to step 2, repeating until the residual no longer falls appreciably (convergence).

Common iterative methods are Gauss-Newton and Levenberg-Marquardt; the latter is more robust and is the default choice in many SLAM systems. You need not compute these by hand—that is precisely what the libraries g2o, Ceres, and GTSAM do for us (Section 12.6).

Figure 12-3 Sparsity: The Secret of Solving Tens of Thousands of Nodes in Real Time



The left panel plots the structure of the information matrix of the pose graph of Figure 12-2, with dark cells marking nonzeros. Nearly all nonzero elements are crowded near the diagonal—because each odometry edge connects only two adjacent nodes and involves no others. The only exceptions are the two isolated blocks at upper right and lower left, created by the loop closure edge that joins the first node to the last.

In this small example of 53 nodes the nonzero fraction is already only 5.7%; raise the node count into the thousands and it falls below one part in a thousand. In other words, this matrix is almost entirely zero.

The right panel shows why that matters. A general dense solver costs $O(n^3)$ operations,

so a thousand nodes require on the order of a billion operations and ten thousand nodes a trillion—quite impossible in real time. But a sparse solver (sparse Cholesky factorization, the conjugate gradient method, and the like) handles only the nonzeros and costs close to O(n), so even tens of thousands of nodes are solved within tens of milliseconds.

This is not a detail but the engineering basis on which modern SLAM exists at all. More than half of the value of the three libraries g2o, Ceres, and GTSAM of Section 12.6 lies in exploiting sparsity to the utmost. When you compile ORB-SLAM2 in Chapter 22 and install g2o, recall this figure—a matrix of this shape is what it handles.

Sparsity: The Secret That Makes Large Problems Solvable

A pose graph of a large scene may have tens of thousands of nodes, corresponding to tens of thousands of unknowns. Solving a linear system that large directly would be computationally infeasible. Fortunately the SLAM problem has a saving property: sparsity.

Sparsity means that each edge involves only two nodes rather than all of them. Hence in the large matrix after linearization the great majority of elements are zero. Exploiting that structure, specialized sparse matrix algorithms can solve extremely quickly—which is precisely what EKF-SLAM (Chapter 11) cannot do and graph optimization can: the EKF's covariance matrix becomes dense (every landmark becomes correlated with every other), while graph optimization stays sparse. This is the mathematical root of graph optimization's ability to handle large scenes.

12.5 Bundle Adjustment

The pose graph above handles constraints between poses. In visual SLAM there is

a finer optimization problem that handles camera poses and three-dimensional map points at once—bundle adjustment (BA). It is the source of accuracy in visual SLAM and the core of ORB-SLAM in Chapters 22 and 23.

Where the Name Comes From

"Bundle" refers to the many rays emanating from each three-dimensional point toward each of the cameras that see it. BA's task is to adjust all camera poses and all three-dimensional point positions so that these bundles of rays align as nearly as possible—that is, so that each three-dimensional point projects into an image as close as possible to the pixel actually observed. That difference between projection and observation is called the reprojection error.

The Reprojection Error: The Residual of Visual SLAM

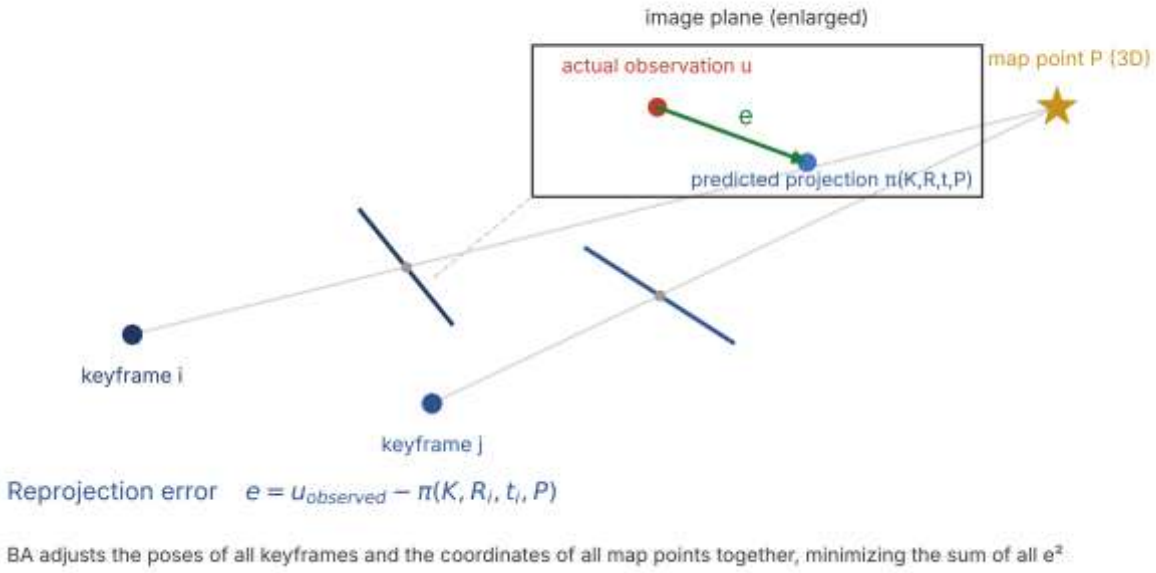
The reprojection error is BA's central residual, and it puts the camera projection model of Chapter 9 to work:

5. Take a three-dimensional map point and project it into the image with the currently estimated camera pose and the intrinsics K (Chapter 9), obtaining a predicted pixel position.
6. That map point also has an actually observed pixel position in that image (obtained by front-end feature extraction).
7. The difference between the two (in pixels) is the reprojection error.

BA adjusts all camera poses and all three-dimensional points at once to minimize the sum of the squares of all reprojection errors. This is an enormous least-squares problem—a scene may have hundreds of camera poses and tens of thousands of three-dimensional points—but it too is solved efficiently through sparsity (each point

is seen by only a few cameras).

Figure 12-4 The Reprojection Error: The e That Bundle Adjustment Minimizes



The difference between bundle adjustment and a pose graph lies in the definition of the residual. A pose graph's residual is a relative relation between two poses; BA's residual is the green arrow in the figure: project the three-dimensional map point P into the image with the currently estimated camera pose and intrinsics, and take the gap between that projection and the feature point actually detected.

This definition has two important properties. First, it is measured directly in pixels—the error is "so many pixels off," which is very intuitive and convenient for setting thresholds (the outlier test commonly used in ORB-SLAM is a reprojection error exceeding some number of pixels). Second, it involves both camera pose and map point coordinates, so BA adjusts the two together—which is where the "bundle" in its name comes from: all the bundles of rays are adjusted together.

Read this figure alongside Figure 9-1 of Chapter 9. There the function π computed a pixel position from a known camera pose and map point; here the direction is

reversed—given the pixel position, adjust the camera pose and the map point so that the computed pixel position comes as close as possible to the actual observation. That is what the whole back end of visual SLAM does.

BA has far more variables than a pose graph—each map point is three variables, and a medium-sized scene easily has tens of thousands of points. Fortunately its sparse structure is even nicer: map points have no direct edges among themselves and are connected only through cameras. Eliminating with the Schur complement by exploiting that structure lets one solve for the camera poses first and back-substitute the map points, which is the key trick that lets BA run in real time; Chapter 22 returns to it.

The Relationship Between BA and the Pose Graph

- Bundle adjustment: optimizes camera poses and three-dimensional points together with the highest accuracy, but has many variables and heavy computation. Suited to local use (the Local Mapping thread of Chapter 22) or to critical moments.
- Pose graph: optimizes poses only (marginalizing the three-dimensional points away), with few variables and high speed. Suited to large-scale loop closure correction (the Loop Closing of Chapter 22).

Modern visual SLAM combines the two ingeniously: local BA maintains high accuracy in ordinary operation, while a pose graph straightens a large stretch of trajectory quickly on loop closure. This "local BA + global pose graph" architecture is the classic design of the ORB-SLAM family, unfolded in full in Chapters 22 and 23. The theory of BA is also exactly what the MIT VNAV and CMU 16-833 courses of Chapter 2 emphasize—you should now be able to see why those leading courses devote so much space to optimization.

12.6 g2o, Ceres, and GTSAM

The mathematics above looks complicated, but the good news is that you hardly need to implement the iterative solving and sparse matrix operations yourself. Three mature open-source libraries have done the heavy lifting. Knowing where each stands is essential knowledge for entering SLAM implementation.

The Three Major Libraries

- g2o (general graph optimization): a C++ library designed specifically for graph optimization, developed by the SLAM community. Its usage is very close to the concepts of this chapter—you define nodes (poses, landmarks) and edges (constraints) and it solves for you. ORB-SLAM2 uses g2o. For a learner, g2o's architecture corresponds most closely to this chapter's pose graph concepts and is a good starting point for understanding graph optimization.
- Ceres Solver: a general nonlinear least-squares library developed by Google, not limited to SLAM. It is powerful, well documented, and convenient in its automatic differentiation (no need to compute Jacobians by hand), and it is widely used for optimization problems of every kind. Cartographer (Chapter 15) uses Ceres.
- GTSAM (Georgia Tech Smoothing and Mapping): developed at the Georgia Institute of Technology (Professor Kaess of CMU 16-833, mentioned in Chapter 2, is a core contributor), built around the concept of a factor graph and providing incremental solving algorithms such as iSAM2—able to update efficiently as new data arrives rather than recomputing from scratch each time.

The Factor Graph: An Equivalent Point of View

The factor graph used by GTSAM is a more general formulation of the pose graph.

In a factor graph the variables to be estimated are one kind of node and each constraint (called a factor) is another kind, connected to the variables it involves. It is mathematically equivalent to a pose graph, but its formulation is more flexible and accommodates sensor constraints of every sort naturally. As Chapter 2 noted, the factor graph is the central tool of the CMU 16-833 course—another sign that the content of this chapter aligns fully with leading international courses.

Library	Developer	Central Concept	Representative User
g2o	The SLAM community	General graph optimization	ORB-SLAM2
Ceres	Google	General nonlinear least squares	Cartographer
GTSAM	Georgia Tech	Factor graphs + incremental solving	LIO-SAM and others

Table 12-1 The Three Major Graph Optimization Libraries Compared

The advice for readers of this book is: at the high school and introductory undergraduate stage, understanding the pose graph–residual–least squares concepts suffices, with no need to go deep into library details; at the undergraduate project and research stage, start with g2o (closest in concept to this chapter) and then choose Ceres or GTSAM as the project requires. The exercises of Chapters 22 and 30 put these tools to actual use.

12.7 Why Modern SLAM Commonly Uses Graph Optimization

To close the chapter, let us answer a question running through Chapters 11 and 12:

since the filters of Chapter 11 can already do SLAM, why have nearly all modern mainstream systems moved to graph optimization? This contrast is the key to understanding where contemporary SLAM is going.

Four Advantages of Graph Optimization

8. It can go back and correct history: a filter compresses the past into the current belief and cannot revisit it; graph optimization retains the whole graph and can correct all historical poses at once on loop closure, flattening drift elegantly. This is the largest source of the difference in accuracy.
9. Repeated relinearization gives higher accuracy: the EKF linearizes each measurement only once (and cannot redo it); graph optimization relinearizes repeatedly through its iterations, approaching the true solution step by step and greatly reducing the linearization error noted in Chapter 11.
10. Exploiting sparsity, it handles large scenes: graph optimization's matrices stay sparse and, with sparse algorithms, handle tens of thousands of nodes; EKF-SLAM's covariance matrix becomes dense and bogs down as it grows with the square of the number of landmarks.
11. It fuses multiple sensors uniformly: every constraint is an edge of the graph, so odometry, vision, LiDAR, IMU, and GNSS can be tightly coupled naturally within one problem (the tight coupling idea of Chapter 7).

The Price of Graph Optimization

Graph optimization is not without cost. It must retain more historical data (greater memory), a single batch solve is computationally heavier, and implementation is more complex. In the early days, limited computing power made filtering the mainstream; it was the growth of computing power and the maturing of sparse solvers that made

graph optimization possible and eventually victorious. This is also a reminder: the mainstream of a technology shifts as hardware and algorithms evolve, and understanding why matters more than memorizing what is used now.

Item of Comparison	Filtering (Chapter 11)	Graph optimization (this chapter)
Manner of processing	Recursive, updating one piece at a time	Batch, optimizing all at once
Correcting history	Cannot go back	Can correct all of history
Linearization	Once per measurement	Repeatedly through the iterations
Handling loop closure	Weaker	Its strength
Large scenes	The EKF is limited (dense)	Exploits sparsity, scales well
Computation and memory	Fixed and light	Heavier
Representative systems	AMCL, early EKF-SLAM	ORB-SLAM, Cartographer

Table 12-2 An Overall Comparison of Filtering and Graph Optimization

Part III, "The Mathematical Foundations of SLAM," is now complete. You have assembled the full mathematical toolbox of SLAM: the coordinate transformations of Chapter 10 let us convert precisely between frames, the probability and filtering of Chapter 11 let us handle uncertainty, and the optimization and graph optimization of this chapter let us find the most consistent global solution. These three chapters are abstract, but they are the common bedrock of every concrete system that follows. From the next part onward we put this mathematics to use—Part IV begins with the most

approachable subject, 2D SLAM and the cleaning robot, and shows how Gmapping builds maps with the particle filter of Chapter 11 and how Cartographer closes loops with the graph optimization of this chapter. Theory and practice are about to meet on a real robot.

Formulations and Procedures

1. The Least-Squares Objective of SLAM

At the core of graph-based SLAM is finding a set of states X (all poses and landmarks) minimizing the weighted sum of squared residuals:

$$X^* = \operatorname{argmin} \Sigma_k \; e_k(X)^T \cdot \Omega_k \cdot e_k(X)$$

where e_k is the residual of the k-th edge (the difference between measurement and prediction), Ω_k the information matrix of that edge (its trustworthiness weight, the inverse of the covariance), and X* the optimal solution. This expression condenses all of SLAM into one sentence: find a set of poses and a map minimizing the weighted contradiction among all constraints. Maximizing probability as in Chapter 11 is, under Gaussian assumptions, exactly equivalent to this least-squares problem.

2. The General Form of the Residual

In a pose graph, an edge connecting nodes i and j has residual:

$$e_{ij} = \text{measured relative pose } (z_{ij}) \ominus \text{predicted relative pose } (x_i, x_j)$$

The symbol $\ominus$ denotes "subtraction" in pose space (Chapter 10)—since a pose contains a rotation, this subtraction is not ordinary subtraction but must be performed on the rotation group (the manifold operations of MIT VNAV in Chapter 2). A zero residual means the constraint is perfectly satisfied.

3. The Reprojection Error (Bundle Adjustment)

In visual SLAM, the reprojection error of a three-dimensional point P in the image of the i-th camera is:

$$e = u_observed - \pi(K, R_i, t_i, P)$$

where u_observed is the pixel coordinate actually observed and π(·) is the projection function of Chapter 9 (projecting the three-dimensional point P into the image with camera pose R_i, t_i and intrinsics K). BA minimizes the sum of squares of all reprojection errors over all camera poses and all three-dimensional points at once.

4. The Iterative Solving Procedure (Gauss-Newton / LM)

Figure 12-5 The Iterative Solving Procedure of Graph Optimization

① Initialization: string together a rough set of poses from odometry (the initial guess X_0)

↓

② Linearization: compute all residuals and Jacobians near the current X

↓

③ Build the linear system: $H \cdot \Delta x = -b$ (H being the sparse information matrix)

↓

④ Sparse solve: exploit sparsity to obtain the correction Δx quickly

↓

⑤ Update: $X \leftarrow X \oplus \Delta x$ (updating in pose space)

↓

⑥ Converged? No → return to ② / Yes → output the optimal solution X*

This whole procedure is what g2o, Ceres, and GTSAM carry out for us behind the scenes. Understanding the procedure is enough; there is no need to implement it by hand—just as driving does not require building the engine.

12.8 Contemporary Developments in Graph Optimization: Faster, Larger, and Less Afraid of Error

The framework introduced in this chapter has had essentially the same skeleton since it took shape around 2010. But it has kept advancing in three directions—speed, scale, and tolerance of erroneous data.

1. Incremental Solving: No Need to Recompute the Whole Graph Every Time

The most immediate question is this: a node is added with every step the robot takes, so must the whole graph be re-solved each time? GTSAM, mentioned in Section 12.6, answers with iSAM2—it organizes the factor graph into a Bayes tree, so that when a new edge is added only the small affected set of branches must be recomputed while the rest stays as it was.

This turns optimization from batch processing into real-time incremental updating, and it is the key to running graph optimization on a robot. It also explains a familiar phenomenon: the CPU is relaxed during ordinary mapping and suddenly becomes busy the moment a loop closure is detected—because a loop closure edge disturbs the whole tree.

2. Global Methods Return to the Stage

On the offline reconstruction side, a notable reversal has occurred recently. For a long time incremental reconstruction (adding images one at a time with repeated BA) was considered the most accurate but extremely slow, while global methods were fast but insufficiently stable. A 2024 ECCV work redesigned the global pipeline—view graph calibration, global positioning, and structure refinement in three steps, with camera positions and image points estimated together in the global positioning stage.

The result is striking: on a large dataset of thirty-six thousand images with cameras spanning over two hundred fifty meters, it completed reconstruction in 5.5 hours with ninety percent recall within one meter, whereas the traditional incremental method ran for more than seven days for only about fifty percent recall. Accuracy was comparable or better, at speeds several orders of magnitude greater.

The significance for students: if a project needs to reconstruct a campus or a building from thousands of photographs (a frequent need in the projects of Chapters 17 and 31), practical tools now exist and one need no longer wait days.

3. Robust Kernels: When the Front End Supplies a Wrong Loop Closure

Graph optimization has a fatal weakness, already mentioned under "the uniformity of constraints" in Section 12.3: it assumes every edge is correct. But the front end makes mistakes—the visual bag of words of Chapter 20 may mistake two similar-looking corridors for the same place, and the point cloud registration of Chapter 17 may converge to a wrong solution in a symmetric structure.

How serious is one wrong loop closure edge? It acts like a powerful rubber band, yanking two stretches of trajectory that are not in fact adjacent together and distorting the whole map on the spot. And because least squares penalizes large residuals quadratically, that erroneous edge has the loudest voice of all—an inherent fragility of the least-squares method.

Countermeasures come at two levels. The first is a robust kernel: replace the squared penalty on the residual with a more slowly growing function such as Huber or Cauchy, so that the influence of large residuals is suppressed and outliers no longer dominate the solution. The second is more thorough—let the optimization itself decide whether to believe an edge, for instance by attaching a switch variable to each loop closure edge that turns off automatically if it conflicts severely with the other constraints.

This matters a great deal in practice. If, while mapping with real hardware in Chapter 29, you find that "the map was fine and then suddenly distorted entirely," the first thing to suspect is a misjudged loop closure. And turning on the solver's robust kernel will

often resolve it.

4. Combination with Learning-Based Methods

Finally, it is worth remarking that the division of labor between graph optimization and deep learning is settling into shape. The most mature pattern today is not to replace optimization with a network but to let the network take the front end—detecting and matching features, judging loop closures, estimating depth priors—and hand the results to a traditional factor graph back end to solve.

The reasoning for this division is exactly that of Sections 7.8, 9.8, and 11.8: the geometric and probabilistic structure of the back end supplies interpretability, uncertainty, and mathematical guarantees, while the perceptual part of the front end goes to models that excel at learning regularities from data. ORB-SLAM3 in Chapter 23 and AI-assisted navigation in Chapter 32 can both be understood within this framework.

A Practical Order of Debugging

The map is distorted → suspect a wrong loop closure edge first (turn on the robust kernel, or raise the verification threshold for loop closures).

The map is skewed overall but locally correct → suspect a systematic bias in odometry (as in the left panel of Figure 12-2); the wheel diameter or wheelbase needs calibrating (Chapter 29).

No loop closure is detected → the problem is in the front end, not the back end (the branch and bound of Chapter 15, the visual bag of words of Chapter 20).

Solving is slow → check whether a dense solver has been used by mistake, or whether the node count calls for keyframe selection.

Case Studies

Case 12-1 The Power of Loop Closure: How a Crooked Map Is Straightened

This is the most dramatic demonstration of graph optimization, and nearly every SLAM classroom shows it. A robot goes once around a building, mapping with odometry and scan matching alone. Halfway round, the map still looks reasonable; but as the circuit nears completion, accumulated drift makes the map skew—a corridor that should close has its end offset from its start by several meters, and the same wall is even drawn as two diverging lines.

Then loop closure detection fires: the system recognizes "what I see now is the lobby I started from" and produces a loop closure edge. Graph optimization runs at once—and on screen the students watch the whole crooked map snap straight: the diverging walls become one, the offset corridor joins up, and the whole trajectory turns regular. In that instant the abstract description of Section 12.2, "spreading the contradiction over the whole trajectory," becomes a living picture before their eyes.

The impact of this case is that it lets students see with their own eyes how one edge corrects an entire map. This a filter cannot do—a filter has not retained its historical poses and cannot go back. The whole value of graph optimization is laid bare in that single moment of straightening. When implementing Cartographer in Chapter 15, students will trigger that moment with their own hands.

Case 12-2 The Lineage of GTSAM: From Classroom to Global Industry

As noted when introducing the CMU 16-833 course in Chapter 2: the GTSAM library developed by Professors Michael Kaess and Frank Dellaert packaged the theory

of factor graphs and incremental smoothing into an open-source tool, and it is now standard equipment in robotics companies and laboratories worldwide. The content of this chapter is the key to understanding what GTSAM does.

One key innovation of GTSAM is iSAM2 (incremental smoothing and mapping). Traditional graph optimization must re-solve the whole problem whenever new data arrives; iSAM2 cleverly updates only the part affected by the new data, which raises efficiency greatly and lets large-scale, long-duration SLAM run in real time. This addresses the pain point of "graph optimization is computationally heavy" raised in Section 12.7.

This case ties several threads of the book together: the leading university courses of Chapter 2, the graph optimization mathematics of this chapter, and real-world industrial application. It tells students that the mathematics they are studying is not an ivory-tower toy but the actual engine driving underwater robots, self-driving cars, and AR devices. Understanding this chapter puts you on the same footing of knowledge as those leading engineers.

Case 12-3 From Surveying to SLAM: The Century-Old Lineage of Bundle Adjustment

Bundle adjustment is often mistaken for a recent invention of computer vision, but its lineage traces back to nineteenth-century photogrammetry. The surveyors of that era reconstructed terrain from several aerial photographs and faced exactly the same problem: how to adjust camera positions and ground points so that the projections in all the photographs agree. The adjustment theory they developed is the direct ancestor of today's BA.

This history offers two lessons. First, the ideas of least squares and adjustment endure—from nineteenth-century surveying through twentieth-century Apollo navigation (Chapters 10 and 11) to twenty-first-century SLAM, humanity's core method for handling several erroneous measurements descends in one line. Second, knowledge across fields often converges: surveying, photogrammetry, computer vision, and robotics all meet at the point of BA. This echoes the observation in Chapter 10 that robot arms and mobile robots share one set of mathematics—mathematics is the common language of every engineering field, and mastering the fundamentals opens doors everywhere.

Hands-On Activities

Exercise 12-1 Optimizing a Pose Graph by Hand

Purpose: through reasoning on paper and implementation in code, to understand first-hand how a pose graph is straightened and to feel the power of a loop closure constraint. This exercise has a paper part and a coding part and suits teaching at various depths.

Part One: A Pose Graph on Paper (everyone)

1. On graph paper, draw a square patrol: starting from the origin, odometry says "4 squares right, 4 up, 4 left, 4 down" and the robot should return to the origin. First draw the four corner poses according to odometry.
2. Add drift: suppose each corner turns a little too little (85 degrees rather than 90, say), redraw the trajectory, and observe how the endpoint fails to return to the origin (a drift gap appears).

3. Add the loop closure edge: declare that the endpoint is the start. Reason intuitively: to bring the endpoint back to the start while disturbing the lengths of the four legs as little as possible, how should the four corners be adjusted? Draw the straightened trajectory by hand.
4. Discussion: the loop closure edge spreads the error over the four corners rather than concentrating the correction at one. Why is spreading better than concentrating?

Part Two: Pose Graph Optimization in Python (undergraduate)

5. Build a simple 2D pose graph in Python: several pose nodes, odometry edges connecting them (with simulated drift), and one loop closure edge.
6. Implement the residual computation: for each edge, compute the difference between the measured relative pose and the currently estimated relative pose.
7. Solve with an off-the-shelf optimizer (scipy.optimize.least_squares, or the Python interface of g2o or GTSAM) to minimize the sum of squared residuals.
8. Plot the trajectories before and after optimization together and observe how the drifted trajectory is pulled back into closure by the loop closure edge.
9. Experiment with the information matrix: set the weight (information matrix) of the loop closure edge large and then small, and observe how the strength of the straightening changes. With a large weight the system believes the loop closure more and pulls harder.

Part Three: Observing Loop Closure in Real SLAM (for those with a ROS environment)

10. Using Cartographer (Chapter 15) or another SLAM system supporting loop closure, drive the robot once around a simulated environment and watch in

RViz in real time: the map drifts gradually during the circuit, and the instant loop closure fires the map is straightened.

11. Record that moment of straightening, compare it with Case 12-1 and Section 12.2, and identify which edges are odometry edges and which is the loop closure edge.

Discussion Questions

1. In Part One, how did you adjust the four corners after adding the loop closure edge? Compared with charging all the error to the final leg, which map is more reasonable when the optimization spreads the error, and why?
2. In Part Two, how did the strength of the straightening change when you raised the weight of the loop closure edge? If the loop closure detection were in fact a misjudgment (a false loop closure), what would a high weight cause?
3. This chapter says graph optimization can go back and correct history while the filters of Chapter 11 cannot. From your experimental results, explain how that difference affects the quality of the final map.
4. (Undergraduate) Why does sparsity let a large pose graph be solved quickly? What would happen if every edge involved all the nodes (no sparsity)?

Extended Challenge (Undergraduate)

Install the Python bindings of g2o or GTSAM, load a public standard pose graph dataset (the intel or manhattan datasets in g2o format, for example), run the optimization, and visualize the difference before and after with matplotlib. Then try adding a wrong loop closure edge and observe how it wrecks the whole map—this will help you understand why Chapters 20 and 22 take such pains over robust loop closure detection and why robust kernels matter.

Further Reading

1. G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, "A Tutorial on Graph-Based SLAM," IEEE Intelligent Transportation Systems Magazine, 2010. (The most classic introduction to graph-based SLAM and the principal basis of this chapter.)
2. Xiang Gao and Tao Zhang, Fourteen Lectures on Visual SLAM, Lecture 6 (nonlinear optimization) and Lectures 9–10 (the back end, BA, and pose graphs). (The clearest material on implementing graph optimization in the Chinese-speaking world.)
3. R. Kümmerle et al., "g2o: A General Framework for Graph Optimization," ICRA, 2011. (The original paper on the g2o library.)
4. F. Dellaert and M. Kaess, "Factor Graphs for Robot Perception," Foundations and Trends in Robotics, 2017. (The authoritative work on GTSAM and factor graphs, freely available.)
5. B. Triggs et al., "Bundle Adjustment — A Modern Synthesis," Vision Algorithms, 2000. (The authoritative survey of bundle adjustment.)
6. The official Ceres Solver documentation (ceres-solver.org) and the official GTSAM tutorials (gtsam.org). (First-hand implementation guidance for the three major libraries.)
7. The factor graph and graph optimization units of the CMU 16-833 Robot Localization and Mapping course (introduced in Chapter 2). (Graduate-level material on graph optimization.)

The Close of Part III: From Mathematics Back to the Robot

The three chapters of Part III end here. Chapter 10 gave the language for expressing

pose, Chapter 11 the language for describing uncertainty, and this chapter the method of solving. Together they are the mathematical skeleton of all modern SLAM.

Looking back, these three chapters in fact answer three aspects of one question. "Where am I?"—Chapter 10 says it is a pose T; Chapter 11 says it is really a distribution; this chapter says that to find it, put all the constraints together and solve a least-squares problem.

From the next chapter, Part IV puts these tools genuinely to work. Chapter 13 turns LiDAR scan data into an occupancy grid map—using the coordinate transformations of Chapter 10, the probabilistic updates of Chapter 11, and this chapter's idea of finding a consistent solution from many observations all at once. Gmapping in Chapter 14 is an implementation of the particle filter, and Cartographer in Chapter 15 an implementation of this chapter's pose graph.

In other words, what Part III teaches is not "mathematics" but the tools every chapter of the next six parts will use. If a concept still feels abstract now, do not worry—when you watch a crooked map snap straight at the moment of loop closure in Chapter 29, everything in this chapter will become concrete.

Summary

1. Modern SLAM adopts a front-end-plus-back-end architecture: the front end turns sensor data into geometric constraints (scan matching, feature matching, loop closure detection) and the back end finds the most consistent solution under all the (often contradictory) constraints; graph optimization belongs to the back end.

2. A pose graph describes SLAM with nodes (the poses and landmarks to be estimated) and edges (relative measurement constraints); odometry edges string poses together like a chain, while loop closure edges form loops and are the key to eliminating accumulated drift.
3. Each edge has a residual (measurement minus prediction) and an information matrix (a trustworthiness weight, the inverse of the covariance); graph optimization unifies sensor constraints of every kind into weighted edges, realizing tight coupling of multiple sensors.
4. SLAM is cast as a least-squares problem: find a set of states minimizing the weighted sum of squared residuals; the square has both a mathematical justification (differentiability) and a probabilistic one (equivalence to maximum a posteriori under Gaussian assumptions), it is solved iteratively by Gauss-Newton or LM, and sparsity makes large scenes tractable.
5. Bundle adjustment optimizes camera poses and three-dimensional points together by minimizing reprojection error and is the source of accuracy in visual SLAM; modern systems combine local BA (high accuracy) with a global pose graph (fast loop closure).
6. The three major libraries do the solving for us: g2o (general graph optimization, used by ORB-SLAM2), Ceres (Google's general least-squares library, used by Cartographer), and GTSAM (factor graphs plus incremental iSAM2); the factor graph is a generalization of the pose graph.
7. Modern SLAM mostly adopts graph optimization because it can go back and correct history, raise accuracy by repeated relinearization, exploit sparsity to handle large scenes, and fuse multiple sensors uniformly; the price is greater

memory and computation, and it became mainstream only after hardware and algorithms had advanced.

Part III, "The Mathematical Foundations of SLAM," is complete. We have assembled the full mathematical toolbox: coordinate transformation (Chapter 10), probability and filtering (Chapter 11), and optimization and graph optimization (this chapter). These three chapters are abstract yet are the bedrock of everything. From the next part onward the theory lands on real robots—Part IV begins with the most approachable subject, 2D SLAM and the cleaning robot, and you will see how Gmapping applies the particle filter of Chapter 11 and how Cartographer applies the graph optimization of this chapter. Mathematics and practice are about to meet.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Part IV 2D SLAM and the Cleaning Robot

Chapter 13

Building a 2D Map

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Part III assembled all the tools: the coordinate transforms of Chapter 10, the probabilistic estimation of Chapter 11, and the optimization of Chapter 12. From this chapter, Part IV puts those tools together into a system that actually runs.

And the first thing to do is turn what the sensors measure into a map. The occupancy grid map of this chapter is the first map format in this book that can really be written to a file and handed to a navigation system—the SLAM systems of Chapters 14 and 15 output it, the cleaning robot of Chapter 16 plans its cleaning path on it, and Nav2 in Chapter 29 takes it as the standard input of the navigation stack.

This chapter makes one deliberate simplification: it assumes the pose is known. That of course does not hold—Chapter 1 called it a chicken-and-egg problem—but getting clear first on how to build a map with a known pose lets Chapters 14 and 15 concentrate on what to do when the pose is not accurate. This decomposition is pedagogically necessary.

Figure 13-0 How Chapter 13 Connects to the Rest of the Book

[Building on the previous chapters]

Chapter 10 Coordinate transforms → move LiDAR points from the body frame into the world frame

Chapter 11 Probability → each cell stores not 0 or 1 but an occupancy probability

Chapter 8 LiDAR specifications → angular resolution determines how finely the map

can be drawn

[This chapter is responsible for]

The data structure of the occupancy grid map → log-odds updating → ray casting

Scan matching (ICP) → fusing odometry with LiDAR

The resolution trade-off and evaluating map quality

[Where it leads]

→ Chapter 14 Gmapping: estimating the pose as well (the particle filter route)

→ Chapter 15 Cartographer: submaps + branch-and-bound loop closure (the optimization route)

→ Chapter 16 The cleaning robot: the consumer application of occupancy grid maps

→ Chapter 17 The three-dimensional version of ICP: point cloud registration

→ Chapter 29 Nav2 plans paths using this chapter's map as input

Learning Objectives

The first three parts laid the conceptual and mathematical foundations; from this part onward, theory comes down to earth on a real robot. Part IV focuses on the most approachable and most familiar form of SLAM—2D SLAM, exactly the technology a cleaning robot uses. This chapter first makes clear what a 2D map is and how it is built, preparing the way for Gmapping and Cartographer in the two chapters that follow. After completing this chapter, you should be able to:

1. Describe the structure of an occupancy grid map and the meaning of each cell's occupancy probability and how it is updated.
2. Explain how laser scan matching estimates a robot's relative motion.

3. Explain why odometry and LiDAR should be fused, and how the two complement each other.
4. Explain how the choice of map resolution affects map quality, memory, and computation.
5. Analyze the challenges of 2D mapping in typical indoor scenes such as corridors, classrooms, and offices.
6. Explain how to assess the quality of a 2D map and recognize common problems such as distortion, breaks, and drift.

Core Concepts

Chinese Term	English Term	Explanation
佔據格地圖	Occupancy Grid Map	Divides space into regular cells, each recording the probability of being occupied.
佔據機率	Occupancy Probability	The probability that a cell is occupied by an obstacle, between 0 and 1.
對數勝算比	Log-odds	Expresses a probability logarithmically, turning map updating into simple addition and subtraction.
掃描匹配	Scan Matching	Compares two successive laser scans to estimate the robot's relative motion.
ICP	Iterative Closest Point	The classic scan matching algorithm of repeatedly finding correspondences and aligning.
里程計	Odometry	Relative displacement estimated from wheel speed; continuous but drifting.
光線投射	Ray Casting	The method of updating the cells along a laser beam's path.

Chinese Term	English Term	Explanation
地圖解析度	Map Resolution	The actual side length each cell represents, 5 centimeters for example.
自由空間	Free Space	A region the laser has passed through and confirmed to be unobstructed.
未知區域	Unknown	A region no sensor has yet observed.
地圖漂移	Map Drift	Gradual displacement or deformation of the map caused by accumulated error.
逆感測器模型	Inverse Sensor Model	The model inferring the occupancy probability of each cell along the way from one measurement.

13.1 The Occupancy Grid Map

Chapters 1 and 3 gave a first acquaintance with the occupancy grid map; this section goes deeper into how it works—the central map representation of 2D SLAM, and what underlies the room map you see in a cleaning robot's app.

Dividing Space into a Checkerboard

The idea of the occupancy grid map is utterly plain: divide the plane the robot moves in into square cells (like a checkerboard or graph paper) and record in each cell one number—the probability that this cell is occupied by an obstacle.

- A probability near 1 (conventionally drawn black): almost certainly an obstacle, such as a wall or a table leg.
- A probability near 0 (drawn white): almost certainly empty and traversable.
- A probability of 0.5 (drawn gray): unknown—no sensor has yet seen this cell.

Distinguishing these three states (occupied, free, unknown) is crucial, and "unknown" above all—it is not the same as "free." The large gray expanse at the edge of a map marks places the robot has not explored, and treating it as traversable would make path planning assume it is empty and go wrong. This is where the occupancy grid map improves on a simple binary obstacle/no-obstacle map.

Each Cell Handled Independently

The occupancy grid map makes one key assumption: the occupancy states of the cells are mutually independent. This does not hold exactly in mathematics (adjacent cells are obviously related—a wall is continuous), but the simplification makes computation extremely efficient: each laser measurement needs only the few cells it struck to be updated, with no account taken of correlations among cells. This is another

instance of the engineering practice of trading a reasonable simplification for tractable computation, in the same spirit as the Gaussian assumption of Chapter 11.

The Premise: The Pose Must Be Known First

An idea that recurs from Chapter 1 must be stressed here: building an occupancy grid map presupposes knowing the robot's pose at every instant. For a laser measurement saying "there is a wall three meters ahead" can be assigned to a cell on the map only if one knows where the robot is and which way it faces.

Which raises the question—where does the pose come from? This is exactly SLAM's chicken-and-egg problem. This chapter assumes for now that the pose is supplied somehow (by odometry plus scan matching, say) and concentrates on how to build the map given a pose; how to estimate the pose well at the same time is the core of Gmapping and Cartographer in Chapters 14 and 15. Separating mapping from localization for the moment is an effective strategy for learning SLAM.

13.2 Updating the Occupancy Probability: Log-Odds

A robot does not draw its conclusion from one look—it scans the same place repeatedly, each scan supplying new evidence, and the map should accumulate that evidence. This section explains how the accumulation is done, and an elegant trick that makes the computation simple.

Accumulating Evidence: Bayesian Updating

The occupancy probability of each cell is updated with every observation. If a cell is struck by the laser many times in succession (each saying "there is something here"), its occupancy probability approaches 1; if the laser passes through it many times in succession (each saying "this is empty"), the probability approaches 0. This is exactly

the Bayesian updating of Chapter 11 in application—correcting an existing belief with new evidence.

But updating the probability directly requires repeated multiplication and division (and normalization), which is cumbersome, and the probability is confined between 0 and 1, where values near the boundaries easily become numerically unstable. Practice therefore adopts an elegant trick.

Log-Odds: Turning Multiplication into Addition

The trick is to rewrite the probability in log-odds form. The odds are the occupancy probability divided by the free probability, and taking the logarithm gives the log-odds. The benefits are immediate:

- Updating becomes addition: each new observation needs only its log-odds increment added—multiplication and division become addition and subtraction, and the computation is extremely fast.
- The numerical range is unbounded: log-odds may be any real number (positive infinity for certainly occupied, negative infinity for certainly free, 0 for an even chance), no longer pinned against the boundaries of 0 and 1, and numerically stable.
- Convert back to probability only when needed: accumulate in log-odds throughout and convert back to a probability between 0 and 1 only for display or decision-making.

This device of taking logarithms to turn multiplication into addition appears everywhere in engineering and science (the decibel and pH among them), and the occupancy grid map is only one instance. You need not memorize the formula, but you should understand the motivation: to make accumulating a great many observations

both fast and stable.

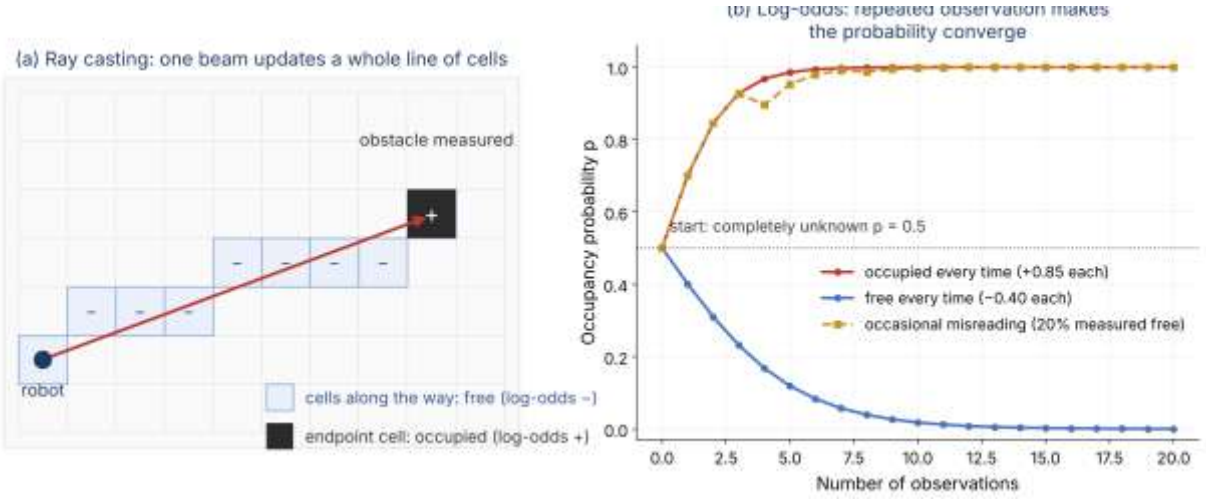
Ray Casting: One Measurement Updates a Whole Line

There is a detail here that is easily overlooked and very important. When the laser measures "a wall three meters ahead," it in fact updates a whole line of cells, not just the one at the wall:

- The cell at the laser's endpoint (the wall): raise the occupancy probability (there is something here).
- All the cells the laser passed through along the way (within three meters): lower the occupancy probability (the laser got through, so these places are empty).

This process of updating the cells along the laser's path is called ray casting, and the model behind it is the inverse sensor model—inferring the occupancy state of each cell along the way from one measurement. It is exactly this mechanism that lets the robot know not only where the walls are but also which space is definitely traversable, which is crucial for the path planning of Chapter 29.

Figure 13-1 Ray Casting and Log-Odds Updating



The left panel shows the geometry of ray casting. One laser beam measuring "something at three meters" in fact carries two kinds of information: there is an obstacle

in the endpoint cell (add a positive log-odds), and nothing blocked the way in each cell passed through (add a negative log-odds to each). So one beam updates not one cell but a whole line—which is why one circuit of a room is enough for the entire open area to be swept white.

The right panel explains why log-odds is used rather than storing the probability directly. The red curve is a cell measured as occupied every time: after the first observation the probability jumps from 0.5 to about 0.70, after three it exceeds 0.90, and after five it approaches 0.99. The blue curve is a cell measured as free every time, whose probability falls rapidly toward 0.

Most worth noting is the dashed yellow curve: this cell is measured as occupied most of the time, but with a 20% chance of being misread as free because of noise or a reflection. The curve dips only briefly and is pulled back—a single misreading cannot overturn accumulated evidence. This is the fundamental advantage of a probabilistic map over "mark it black when you see it," and the concrete realization of the Bayesian updating of Chapter 11 on a two-dimensional grid.

Why Take Logarithms

Bayesian updating is itself multiplication: new odds = old odds × measurement odds. Repeated multiplication readily underflows numerically, and it is slower.

After taking logarithms, multiplication becomes addition and each update needs only one integer or floating-point addition—a decisive difference for a real-time system updating tens of thousands of cells per second.

There is a hidden benefit as well: log-odds can be clamped directly (to ±6, say),

which keeps a cell observed a thousand times from locking up so that the map can never reflect environmental change. The dynamic updating of Section 13.5 relies on exactly this mechanism.

13.3 Laser Scan Matching

The previous section assumed the pose was known. But how is the pose actually estimated? Besides odometry, the LiDAR itself can supply a powerful cue to the pose—this is scan matching, the core technique of the 2D SLAM front end.

The Central Idea: Align Two Scans

The idea of scan matching is quite intuitive. The robot scans once at position A, advances a small step to position B, and scans again. If the environment has not changed (the wall is still that wall), then the displacement and rotation of the scan at B relative to the scan at A reflect the robot's motion from A to B.

In other words: find a rotation and translation that lay the scan at B onto the scan at A so that the two agree as closely as possible—that rotation and translation are the robot's relative motion. This is an optimization problem (in the spirit of Chapter 12): adjust the pose so as to minimize the discrepancy between the two scans.

ICP: The Most Classic Scan Matching

The best-known scan matching algorithm is ICP (iterative closest point). Its procedure is a loop of repeated approximation:

1. Correspondence: for each point in scan B, find the nearest point in scan A and pair them.

2. Alignment: compute a rotation and translation that minimize the total distance over all paired points.
3. Update and iterate: apply that transformation, then return to step 1 to find correspondences afresh, repeating until no further improvement occurs (convergence).

The name of ICP describes its operation precisely: iteratively find the closest points and align. It is simple and effective, the cornerstone of point cloud registration, and it reappears in the 3D mapping of Chapter 17.

The Strengths and Pitfalls of Scan Matching

- Strength: LiDAR is precise, and the relative motion estimated by scan matching is usually far more accurate than odometry, especially while turning, where it is unaffected by wheel slip.
- Pitfall one—a repetitive environment: in scenes that look the same everywhere, such as corridors and empty halls, scan matching cannot find a unique alignment (slide along the corridor and the scans look much the same), and the estimate goes wrong. This is the same class of problem as "the curse of the white wall" in Chapter 9: a lack of distinctive features.
- Pitfall two—the initial guess: ICP needs a good initial guess to converge to the right solution, or it may fall into a wrong local optimum. This is exactly where odometry comes in—the next section explains.

13.4 Fusing Odometry with LiDAR

Part II stressed sensor fusion repeatedly, and 2D SLAM is the cleanest demonstration of the idea: the strengths and weaknesses of odometry and LiDAR are exactly complementary, and together they far surpass either alone.

How the Two Complement Each Other

Characteristic	Odometry (wheel speed)	LiDAR scan matching
Data continuity	Continuous, high-rate, smooth	Limited by the scan rate
Short-term accuracy	Fair	High (especially while turning)
Long-term behavior	Accumulating drift	Comparatively stable
Where it fails	Slip, uneven floors	Repetitive environments, insufficient features
Initial guess required	Not required	Required (or it readily diverges)

Table 13-1 How Odometry and LiDAR Scan Matching Complement Each Other

The Logic of Fusion: Odometry Gives the Starting Point, LiDAR Corrects It

Do you see the key? Odometry needs no initial guess and is continuous and smooth, which is exactly the starting point that scan matching—which does need an initial guess—requires; and scan matching is precise and does not drift, which is exactly what corrects odometry's drift. The two fit together perfectly:

4. The robot moves one step and odometry first gives a coarse estimate of the relative motion (the initial guess).
5. Starting from that guess, scan matching (ICP) is run to refine a more accurate relative motion.
6. The refined pose is used to update the map (Section 13.2) and serves as the basis for the next step.

This cycle of "odometry predicts, LiDAR corrects" is the concrete embodiment of the predict-update cycle of Bayesian filtering in Chapter 11: odometry performs the prediction (the motion model) and LiDAR the update (the observation model). You should be able to feel the abstract mathematics of Part III turning here into an operable mapping procedure. Gmapping in Chapter 14 and Cartographer in Chapter 15 are both refinements of this basic cycle.

What to Do When the Wheels Slip

When the wheels slip, odometry suddenly reports a large, wrong displacement. The value of scan matching stands out all the more here: even if odometry says "I advanced half a meter," so long as the LiDAR scan shows the environment has barely changed, scan matching pulls the estimate back to "I hardly moved at all." This ability to overrule a wrong motion estimate with an observation of the environment is the fundamental reason SLAM is more robust than dead reckoning on odometry alone.

13.5 Map Resolution and Updating

In building a 2D map, one key decision must be made: how large an actual area does each cell represent? This is the map resolution, and it drives a three-way trade-off among map quality, memory, and computation.

The Resolution Trade-Off

Map resolution means the actual side length of each cell, commonly 5 centimeters indoors. The choice is a typical engineering trade-off:

- High resolution (small cells, 1 centimeter say): the map is fine and can represent small structures (table legs, door frames), but memory use grows explosively

(the number of cells varies inversely with the square of the side length—halve the side and the count quadruples) and computation slows.

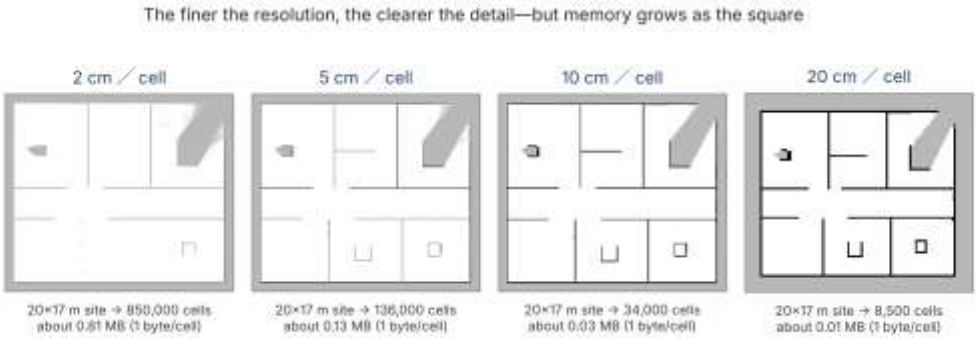
- Low resolution (large cells, 20 centimeters say): memory is saved and computation is fast, but the map is coarse, small obstacles may be smeared away within a cell, and wall positions are imprecise.

How to choose? It depends on the application. A cleaning robot need not resolve a centimeter of detail and 5 centimeters is ample; a robot requiring precise docking may need higher resolution. This echoes the discussion of LiDAR angular resolution in Chapter 8—however high the map resolution, it can never exceed the resolving power of the sensor itself, and the two must be matched.

The Reality of Memory

To give a concrete sense of memory use: a room 20 meters by 20 meters at 5-centimeter resolution is 400 × 400 = 160,000 cells. Raise the resolution to 1 centimeter and it becomes 2000 × 2000 = 4 million cells—fully 25 times as many. For a large area (a whole building) this growth is considerable, which is why Chapter 3 noted that 3D maps are compressed with an octree (OctoMap). Because a 2D occupancy grid covers only a plane, the memory pressure is far lower, which is one reason it suits teaching and cleaning robots.

Figure 13-2 The Resolution Trade-Off: One Site at Four Cell Sizes



These four panels are the same scan and the same data with only the cell size changed. At 2 centimeters the walls are finely drawn and the furniture outlines clear; at 20 centimeters the walls become thick black blocks, the table and chair legs blur together entirely, and even the doorway opening is nearly filled in.

But the price is written beneath each panel. For a site of 20 × 17 meters, 2 centimeters requires 850,000 cells and about 0.81 MB; 5 centimeters falls to 136,000; 10 centimeters leaves only 34,000; and 20 centimeters, 8,500. Halve the cell side and the count quadruples—memory grows as the square, and the computation needed for updating grows with it.

The common practical choice is 5 centimeters, and that is no accident: it is just fine enough to resolve an ordinary door frame (about 80 to 90 centimeters), a walkway (about 1 meter), and the gaps between table legs, while the memory remains within what an embedded platform can bear. Cleaning robots often use 5 to 10 centimeters to save memory; a warehouse AGV that must pass through narrow aisles may go to 2 or 3 centimeters.

The method of selection is exactly the same as judging LiDAR angular resolution in Section 8.4: ask first how narrow a gap you must resolve, then work back to the cell size. And the two must be matched—storing data from a LiDAR of 1-degree angular

resolution in 2-centimeter cells yields only a map full of holes, because the sensor simply cannot supply information that fine.

Dynamic Updating: A Map Is Not One Photograph

A map must be updated continuously, for two reasons:

- Exploring new areas: the robot reaches places it has not been and fills unknown gray cells in as free or occupied.
- Correcting old estimates: repeated observation of the same area accumulates evidence that corrects previously uncertain cells (the Bayesian updating of Section 13.2).

But this brings a subtle problem: what if the environment itself changes—if someone moves a chair? If the map's memory of old observations runs too deep (log-odds accumulated too high), it is hard to update to the new state; if the memory is too shallow, noise disturbs it easily. In practice the log-odds are clamped to upper and lower limits, which keeps the map somewhat plastic and able to reflect change. This foreshadows the discussion of dynamic environments for the cleaning robot in Chapter 16.

13.6 2D Mapping in Corridors, Classrooms, and Offices

With the theory covered, we turn to real indoor scenes. Different spatial structures pose quite different challenges for 2D mapping. Understanding them lets you anticipate where trouble will arise, which is important preparation before implementation.

The Classroom: A Friendly Scene for 2D SLAM

An ordinary classroom is the ideal scene for 2D SLAM: four walls with clear

boundaries, desks and chairs offering rich geometric features, and an enclosed space of modest size. Scan matching runs stably here and the map comes out clear. This is also why the hands-on activities in this book mostly use a classroom as the site (Chapters 14 and 29)—it lets students build a first experience of success under the most favorable conditions.

The Corridor: Scan Matching's Nightmare

A long straight corridor is the classic difficulty of 2D SLAM and the archetype of the repetitive-environment pitfall of Section 13.3. A corridor has parallel long walls on both sides, and moving along it the laser scan barely changes—scan matching cannot tell how far along the corridor the robot has traveled (that direction lacks constraint) and can determine only whether it has strayed from the centerline (the perpendicular direction is constrained by the walls).

This is exactly the source of that uncertainty ellipse stretched along the corridor in Section 11.2 of Chapter 11. Here the value of odometry stands out—it supplies the estimate of motion along the corridor and fills in the constraint scan matching lacks. This is one more instance of why fusing odometry and LiDAR (Section 13.4) is indispensable. Mapping a long corridor on LiDAR alone almost always fails.

The Office: Dynamics and Glass

An open-plan office combines several difficulties: glass partitions let the LiDAR see through or produce false reflections (the glass problem of Chapter 8), people moving about cause dynamic disturbance, and similar cubicle arrangements are easily confused. Such environments often require finer parameter tuning or additional sensors.

Lobbies and Atriums: The Difficulty of Open Space

The problem with large open spaces (a high-ceilinged lobby, say) is that LiDAR has a limited measurement range (about 8 to 12 meters for teaching models, Chapter 8), and when the space is larger than that range the robot in the middle may find no wall in any direction and lose its reference entirely. Scan matching then has nothing to work with and odometry alone must carry the load, with error accumulating rapidly.

Scene	Principal Challenge	Key Remedy
Classroom	Almost none (an ideal scene)	Map directly
Long corridor	Repetitive environment, no constraint along the corridor	Rely on odometry to supply the longitudinal estimate
Office	Glass, moving people, similar cubicles	Parameter tuning, multiple sensors
Large open hall	Beyond LiDAR range, nothing to scan all round	Carry on with odometry, extend the range, or add landmarks

Table 13-2 Mapping Challenges in Typical Indoor Scenes

13.7 Assessing Map Quality: Distortion, Breaks, and Drift

With the map built, how does one judge whether it was built well? This is an extremely important and often neglected part of practice. This section teaches you to recognize three common symptoms in a map and how to assess quality—the basis, too, of the project grading rubric of Chapter 31.

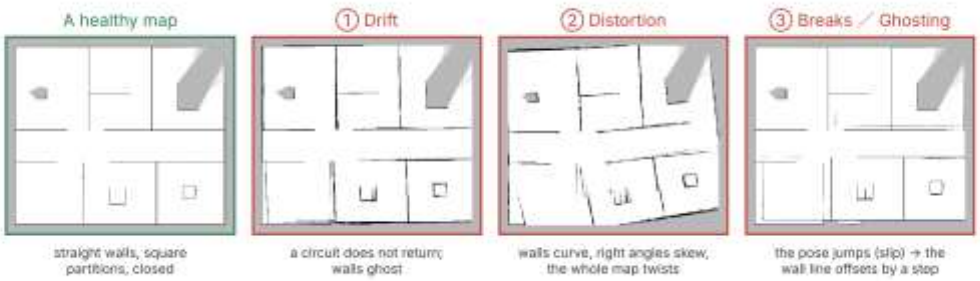
Three Common Symptoms

- Drift: the most common problem. Accumulated error gradually displaces the map, the classic symptom being that a circuit does not return to its starting

point—a loop that should close leaves the end offset from the beginning. This is exactly what the loop closure of Chapter 12 exists to fix, and the basis for judging whether loop closure detection is needed.

- Distortion: the geometry of the map is warped, straight walls become curved, right angles become skewed, and room proportions are falsified. This mostly comes from systematic error in scan matching or a scale error in odometry (an inaccurate wheel-radius calibration).
- Breaks and ghosting: the same wall appears twice on the map (a ghost), or wall lines break apart. Usually the pose estimate jumped wrongly at some point, so the same structure was drawn in two places. It happens most readily in corridors and open areas.

Figure 13-3 Three Common Symptoms in a Map



These four panels likewise come from one set of sensor data and differ only in the pose. The leftmost is how it should look with a correct pose: straight walls, square partitions, a closed outer boundary.

① Drift: the pose shifts slowly and persistently, so the same wall is drawn in different places at different times and finally appears as two parallel lines; and on returning to the same spot after a circuit, start and end do not line up. This is the classic symptom of no loop closure—the robot has come back and does not know it.

② Distortion: the pose carries a persistent angular bias and the whole map is progressively twisted. The symptom is that every short stretch looks normal while the whole is askew: the corridor is no longer straight and the room becomes a rhombus. The cause is usually an inaccurate wheel-radius or wheelbase calibration (actually calibrated in Chapter 29), or a failure of scan matching in a long corridor.

③ Breaks and ghosting: the pose jumps suddenly at some instant—the robot slips, is pushed by someone, or scan matching converges to a wrong solution. The symptom is easy to recognize: the wall is cleanly offset by a step at some position, as though the map had been cut and glued back without aligning.

Inferring the Cause from the Symptom

Seeing drift (ghosted walls, start and end not matching) → the loop closure was not detected, or was detected and not accepted. The branch and bound of Chapter 15 and the visual bag of words of Chapter 22 exist to solve exactly this.

Seeing distortion (curved walls, skewed right angles) → suspect a systematic odometry bias first (wheel radius or wheelbase), then degeneracy of scan matching in a long corridor.

Seeing breakage (a wall line cleanly offset by a step) → find what happened at that moment: slip, a collision, or dropped sensor frames.

This ability to read symptoms and infer causes will be in daily use while mapping on hardware in Chapter 29—and what it requires is precisely an eye that has seen what a normal map looks like.

How to Assess Map Quality

Assessment methods divide into the subjective and the objective:

- Visual inspection (qualitative): are the walls straight? Are the right angles square? Does the loop close? Is there ghosting or breakage? This is the quickest first step, and an experienced eye can tell a good map from a bad one at a glance.
- Comparison with measurement (quantitative): measure a few key distances on the map (corridor length, room width), compare them with tape measurements, and compute the percentage error. This is the objective way to quantify map accuracy.
- Loop consistency: have the robot go around a loop back to its starting point and measure the gap between "start" and "end" on the map. The smaller the gap, the better drift has been controlled.
- Comparison with ground truth (research grade): in an environment with a motion capture system or a precisely known map, overlay the estimated map on the truth and compare (used in the Chapter 31 project).

Developing the habit of assessing quality as soon as a map is built is crucial. A map that looks complete but is in fact distorted will throw all subsequent path planning (Chapter 29) off—the robot will drive into a wall following a wrong map. This echoes the idea of Chapter 8 that a sensor's limitation becomes the system's failure: map quality directly decides the success or failure of the whole navigation system.

By this point you understand what a 2D map is, how to build one, and how to assess it. But this chapter deliberately separated mapping from localization and assumed the pose was known. Real SLAM must solve both at once—and the next chapter brings in Gmapping, which uses the particle filter of Chapter 11 to let every particle carry its own map, localizing and mapping at once and untying the chicken-and-egg knot elegantly.

Theory and algorithm are at last about to become one.

Formulations and Procedures

1. Log-Odds Updating

For a cell with occupancy probability p, the log-odds are defined as:

$$l = \log(p / (1 - p))$$

With each new observation, the new log-odds require only an addition:

$$l_{new} = l_{old} + l_{measurement}$$

where l_measurement is the evidence this observation brings (positive for an occupied observation, negative for a free one). This is the mathematical core of "turning multiplication into addition" in Section 13.2. For display or decision-making, convert back to a probability:

$$p = 1 - 1 / (1 + \exp(l))$$

A positive l favors occupancy, a negative l favors free space, and zero is an even chance. In practice l is clamped to upper and lower limits to keep the map plastic to change.

2. The Optimization Objective of Scan Matching

Scan matching seeks a relative pose T (rotation plus translation) that minimizes the total error after aligning the current scan S to the reference scan R:

$$T^* = \operatorname{argmin} \Sigma_i \| T \cdot s_i - r_{j(i)} \|^2$$

where s_i is a point of the current scan and $r_{j(i)}$ its nearest corresponding point in the reference scan. This is a least-squares problem (Chapter 12), and ICP approaches its solution by repeating find correspondences—solve—iterate.

3. The Procedure for Mapping with Fused Odometry and LiDAR

Figure 13-4 One Step of 2D SLAM Mapping (this chapter's version, with loop closure handled separately)

① The robot moves one step

↓

② Odometry estimates a coarse relative motion (the initial guess)

↓

③ Starting from that guess, scan matching (ICP) refines the relative motion

↓

④ Update the robot pose (accumulate the refined relative motion)

↓

⑤ Ray casting: update the cells along the laser path in log-odds

↓

⑥ Return to ① for the next step (checking periodically for loop closure → Chapters 14 and 15)

4. Estimating Map Memory

The total number of cells in an occupancy grid map is:

$$\textit{number of cells} = (\textit{width} \,/\, \textit{resolution}) \times (\textit{height} \,/\, \textit{resolution})$$

For example, a 20 m × 20 m room at a resolution of 0.05 m: cells = 400 × 400 = 160,000. Halve the resolution (to 0.025 m) and the count quadruples (640,000)—this expression quantifies the quadratic relation between resolution and memory in Section

13.5.

13.8 Contemporary Practice in 2D Mapping: From a One-Off Map to a Lifelong Map

The mathematics of the occupancy grid map took shape in the 1980s, and the log-odds updating introduced in this chapter has not changed since. But how a map is used has changed considerably over the past decade, and that change bears directly on the tools you will use in Chapter 29.

1. From a Static Image File to a Continuously Updatable Pose Graph

The traditional practice is to map once, save a PGM image file plus a YAML configuration, and thereafter load that static map to localize while navigating. The procedure is simple but has an obvious problem—environments change. Move the desks and chairs, rearrange the shelving, redo the partitions, and the map is out of date with nothing for it but to map the whole place again.

slam_toolbox, the current mainstream tool in the ROS 2 ecosystem, proposes a different approach: it saves not just the map but the whole pose graph (the nodes and edges of Chapter 12), serialized. On the next start-up that pose graph is loaded, and the robot may choose to keep mapping, to localize only, or to merge new and old maps. This is called lifelong mapping.

A few specifications worth knowing: its synchronous mode runs at more than five times real time in a building of about 2,800 square meters (thirty thousand square feet), and still at three times at about 5,600 square meters; the largest known synchronous mapping case is about 18,600 square meters. The asynchronous mode suits larger spaces at the price of selectively discarding scans it cannot process in time. The

underlying optimization solver uses Ceres by default and can be switched to g2o—precisely those introduced in Section 12.6.

2. Localization Without a Static Map

slam_toolbox also offers a mode called elastic pose graph localization: after loading a serialized map, it matches incoming scans against a buffer of recent scans rather than permanently altering the underlying map. It likewise accepts an initial pose set in RViz and is used much like the AMCL to be introduced in Chapter 16.

The significance for students is that the AMCL of Chapter 16 (particle filter localization) is no longer the only option. The tool's own author nonetheless advises beginners to start with AMCL—because its behavior is intuitive, its documentation plentiful, and its problems easy to diagnose. This book's arrangement follows that advice.

3. Merging Several Maps

There is one more very useful capability in practice: merging the results of several mapping runs into one globally consistent map. A large site is hard to cover in one run—the battery runs out, some areas are temporarily inaccessible—so mapping in stages and merging afterward has become standard practice. Merging can be done by dragging into alignment by hand in RViz, or automatically with kinematic tools.

The mathematics behind this is the pose graph optimization of Chapter 12: join the two pose graphs, add a few manually specified edges saying "these two nodes are the same place," and solve again. Once Chapter 12 is understood, this feature is no longer mysterious.

4. Beyond the Occupancy Grid

Finally it is worth mentioning that the occupancy grid is not the only two-dimensional

map representation. Several others have gradually spread through navigation stacks in recent years: a costmap inflates obstacles outward and stores directly whether a place is traversable rather than whether something is there; a distance field (ESDF) stores in each cell the distance to the nearest obstacle, making obstacle avoidance and path smoothing more efficient to compute; and a semantic layer overlays information such as "this is glass," "these are stairs," "this is a keep-out zone" on top of the geometry—the layered costmap of Nav2 mentioned in Section 3.6 is exactly this architecture.

But nearly all of these representations are built on, or coexist with, the occupancy grid. So what this chapter teaches remains the foundation: once you understand that each cell stores a probability and accumulates evidence in log-odds, every variant is only another way of storing it.

Three Reminders for Students

1. Maps go out of date. Any robot meant to run for a long time must handle map updating rather than assume the environment never changes.

2. Saving the pose graph is worth more than saving an image file. An image can only be looked at; a pose graph can be optimized further, merged, and corrected in retrospect.

3. This chapter's simplification (a known pose) is lifted in the next. From Chapter 14, pose and map are estimated together—and that is what SLAM really looks like.

Case Studies

Case 13-1 That Map in the Cleaning Robot App: The Occupancy Grid in Daily Life

Open any cleaning robot app that supports mapping, and the room map you use to set keep-out zones and designate rooms is the occupancy grid map of this chapter underneath. The black wall lines are cells of high occupancy probability, the white cleanable area is free space of low occupancy probability, and places the robot has not visited are blank (unknown).

Watching this map grow is instructive: during the robot's first cleaning the map grows bit by bit—drawn where it goes and where it scans—and the unknown area is gradually filled in. This is the visualization of "dynamic updating—exploring new areas" in Section 13.5. If the robot slips somewhere or runs into black furniture (invisible to LiDAR, Chapter 8), you may see a skewed wall line or a gap in the map—the drift and breakage of Section 13.7, appearing live in your own living room.

A more advanced observation: some cleaning robots produce a more accurate and smoother map on a second cleaning of the same home. This is because more observations have accumulated and the estimate of each cell is more confident (the Bayesian accumulation of Section 13.2), and a loop closure correction (Chapter 12) may even have been performed. A cleaning robot is in fact a portable teaching aid for every theory in this part.

Case 13-2 Coming to Grief in a Long Corridor: A Real Teaching Failure

A scene played out almost every year in university classes: students run 2D SLAM mapping smoothly in the classroom, drive the robot confidently into a long corridor, and the map blows up instantly—the corridor drawn now long, now short, wall lines twisted and overlapping, the robot thoroughly lost.

The cause is the repetitive environment stressed in Sections 13.3 and 13.6: a long corridor lacks distinctive features along its length, and scan matching cannot determine how far along it the robot has traveled. If odometry is also inaccurate because the floor material changed (tile giving way to carpet), both sources of information fail and mapping must fail with them.

This failure is in fact the best lesson there is. It lets students feel for themselves that SLAM is not omnipotent and that the structure of the environment directly decides success or failure. The remedies echo the whole book: fuse dependable odometry (Section 13.4), use a longer-range LiDAR that can see the end of the corridor, or post AprilTags along it (Chapter 4) to supply distinctive landmarks. Understanding why it failed teaches more than a smooth mapping run in the classroom—which is why this book discusses limitations and failures honestly in every chapter.

Case 13-3 From 2D to Intelligence: The Evolution of the Cleaning Robot

To review the evolution of the cleaning robot is to review the spread of 2D SLAM. The earliest models (around 2002) had no map at all and used random bump-and-turn—a disorderly cleaning path, both repetitive and incomplete (Chapters 3 and 16). In 2010 Neato first fitted a 2D LiDAR to a cleaning robot, letting it build an occupancy grid map and plan a systematic path. In 2017 Xiaomi popularized the LiDAR cleaning robot at an affordable price (the case in Chapter 1), bringing the occupancy grid map into countless homes.

Recent models go further: a 3D ToF camera recognizes and avoids cables, shoes, and pet accidents (making up for the height blind spot of 2D LiDAR, Chapter 8); AI recognizes room types and zones automatically; and some even remember the maps

of several floors. This line of evolution shows clearly that the 2D occupancy grid map is the foundation, while real products keep layering on sensors and AI to make up for the limits of any single technology—exactly the fusion idea running through this book, and a preview of the AI semantic navigation of Part IX. The 2D mapping you learn today is the starting point for understanding all of it.

Hands-On Activities

Exercise 13-1 Building an Occupancy Grid Map by Hand

Purpose: to implement occupancy grid updating from scratch in code and so understand log-odds, ray casting, and scan matching first-hand. The exercise is mainly in Python, with a simulation route and a data route, and is a direct warm-up for the Gmapping exercise of Chapter 14.

Part One: Log-Odds Updating (everyone)

1. Create a two-dimensional array with numpy as the map (200×200 cells, say, each initialized to log-odds 0 for unknown).
2. Simulate a single laser beam: given the robot's position, the beam direction, and the measured range, implement ray casting—decrease the log-odds of the cells along the way (free) and increase that of the endpoint cell (occupied).
3. Convert the log-odds back to probabilities and display the map in grayscale with matplotlib (black = occupied, white = free, gray = unknown), confirming that one beam draws a white line to a black dot.
4. Simulate a full revolution (360 beams): place the robot at the center of the room, cast a beam toward each simulated wall surface, and observe whether the room outline is drawn.

5. Repeated observation: have the robot scan several times from the same position (with random noise added) and observe how the wall lines become more certain as observations accumulate (log-odds accumulate and the color darkens).

Part Two: Mapping While Moving (everyone, or undergraduate)

6. Design a simple simulated room (define a few walls in code), move a virtual robot along a planned route, update the map at each step from the known pose and the simulated LiDAR, and watch the complete room map grow step by step.
7. Add odometry noise: deliberately give the robot's pose estimate an accumulating error (drift) and observe how the resulting map deforms and the wall lines ghost—reproducing the symptoms of Section 13.7 first-hand.
8. Resolution experiment: map the same scene at resolutions of 2, 5, and 20 centimeters and compare map detail, memory use (cell count), and computation time.

Part Three: Mapping in ROS, Real or Simulated (for those with a ROS environment)

9. In the TurtleBot3 simulation (Chapter 28), launch the built-in 2D SLAM (slam_toolbox or gmapping), teleoperate the robot once around the simulated classroom, and watch the occupancy grid map grow in RViz in real time.
10. Drive the robot into a simulated long corridor and observe whether the corridor difficulty described in Section 13.6 appears (the map deforming along the corridor).

11. Once the map is built, assess its quality by the methods of Section 13.7: inspect visually whether the walls are straight, and measure the room dimensions on the map against the simulation settings.

Discussion Questions

1. In Part One, how does the color (occupancy probability) of the same wall differ after 1 observation and after 10? What does this show about the accumulation of log-odds?
2. After adding odometry drift in Part Two, which symptom appeared (distortion, breakage, or drift)? Which technique from Chapter 12 would you use to correct it?
3. Going from 5-centimeter to 2-centimeter resolution, by what factor did the cell count increase? Did the changes in memory and computation time match the quadratic relation of Section 13.5?
4. (With ROS) What went wrong with the long corridor map? Comparing with Section 13.6, which remedy do you think is most effective—adding odometry, using a longer-range LiDAR, or posting AprilTags? Why?

Extended Challenge (Undergraduate)

Implement a simplified ICP scan matcher: given two simulated scans with a known relative displacement, estimate that displacement by the procedure of finding nearest correspondences, solving for the best rotation and translation, and iterating, then compare with the truth. Then test your ICP in a long-corridor setting (too few features along its length) and observe why its estimate along the corridor goes wrong—which will give you a deep understanding of why Gmapping (Chapter 14) uses a particle filter rather than relying on scan matching alone.

Further Reading

1. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (Chapter 9, Occupancy Grid Mapping, is the authoritative basis of this chapter.)
2. Cyrill Stachniss, the Occupancy Grid Maps, Scan Matching, and ICP videos in his SLAM course (YouTube). (Explains this chapter's core with animations; the first choice for self-study.)
3. P. Besl and N. McKay, "A Method for Registration of 3-D Shapes," IEEE TPAMI, 1992. (The original paper on ICP scan matching.)
4. A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation," Computer, 1989. (The pioneering paper on occupancy grid maps.)
5. The slam_toolbox open-source project (GitHub) and the official ROS documentation. (The mainstream modern ROS 2 tool for 2D SLAM, superseding the old gmapping; the first choice for implementation.)
6. The SLAM chapters of the ROBOTIS TurtleBot3 e-Manual (emanual.robotis.com). (Complete implementation steps for 2D mapping.)
7. Xiang Gao and Tao Zhang, the map-related chapters of Fourteen Lectures on Visual SLAM. (Though focused on vision, its discussion of map representations complements this chapter.)

From a Known Pose to Estimating Both: Where This Chapter Stands

This chapter has completed the second half of SLAM: given the pose, turn sensor data into a map. That half is in fact quite direct—ray casting plus log-odds, and under a hundred lines of code will do it.

The difficult part is the other half. This chapter assumed throughout that the pose was known and correct, and Figure 13-3 has already shown how badly the map breaks once that assumption fails. A real robot has nobody to tell it where it is—it has only a drifting odometer and the scan matching of Section 13.3.

Scan matching is in fact already an embryo of SLAM: it corrects the pose in reverse by aligning the current scan to the map. But it has two weaknesses. First, it sees only locally—in the long corridor of Section 13.6 it can slide along and still match, so it cannot judge how far it has gone. Second, it has only one answer—once a match is wrong, there is no mechanism to go back.

Gmapping in Chapter 14 solves the first weakness with the particle filter of Chapter 11: it keeps dozens of pose hypotheses at once and lets the observations eliminate the implausible ones one by one. Cartographer in Chapter 15 solves the second with the pose graph optimization of Chapter 12: it keeps all history and corrects it in one pass at a loop closure. Read the two chapters together and you have a direct contest between the two routes of Chapters 11 and 12 on the same problem—judged by exactly the map-quality criteria taught in Section 13.7.

Summary

1. An occupancy grid map divides the plane into cells, each recording an occupancy probability in three states (occupied, free, unknown); "unknown" is not "free," which is where it improves on a binary map. Mapping presupposes a known pose, and this chapter separates mapping from localization to concentrate on how to map given a pose.

2. The occupancy probability accumulates by Bayesian updating with each observation; practice uses log-odds, which turns multiplication into addition and gives an unbounded, stable numerical range; one laser measurement updates a whole line through ray casting—the endpoint cell (occupied) and the cells along the way (free).
3. Scan matching estimates relative motion by comparing two successive laser scans, the classic algorithm being ICP (correspond—solve—iterate); it is precise but fears repetitive environments (corridors) and needs a good initial guess.
4. Odometry and LiDAR are exactly complementary: odometry is continuous and needs no initial guess but drifts, while LiDAR is precise and drift-free but needs an initial guess and fears repetitive environments; the logic of fusion is "odometry predicts, LiDAR corrects," the embodiment of the predict-update cycle of Bayesian filtering, and it can overrule a wrong motion estimate caused by slip with an observation of the environment.
5. Map resolution is a three-way trade-off among quality, memory, and computation, with the cell count varying inversely with the square of the resolution; the map must be updated dynamically to explore new areas and correct old estimates, and the log-odds are clamped to keep it plastic to change.
6. Different scenes pose different challenges: the classroom is ideal, a long corridor is hardest because it is unconstrained along its length (odometry must fill in), an office has glass and dynamics, and a large open hall exceeds the LiDAR's range; anticipating the challenge is the key to a successful implementation.

7. The three symptoms of a map are drift (a circuit does not return to its starting point), distortion (warped geometry), and breaks or ghosting (one wall drawn as two lines); assessment methods include visual inspection, comparison with measurement, loop consistency, and comparison with ground truth. Map quality directly decides the success of navigation.

This chapter deliberately separated mapping from localization and assumed a known pose. But real SLAM must untie both at once—exactly the chicken-and-egg problem of Chapter 1. The next chapter brings in Gmapping: using the particle filter of Chapter 11, it lets every particle carry its own trajectory and its own map and runs a survival of the fittest among many hypotheses, accomplishing localization and mapping at once and elegantly. The particle filter theory of Part III is about to show what it can do in the first complete SLAM system.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 14

Gmapping: Particle Filter SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 13 built a map, but it rested on an assumption that does not hold—that the pose was known. This chapter removes that assumption and confronts the chicken-and-egg problem posed in Chapter 1 head-on.

Gmapping's answer is the particle filter of Chapter 11, with one key twist: rather than letting the particles represent the joint distribution over pose and map, it lets each particle carry only one pose trajectory, with the map built by that particle itself by the method of Chapter 13. This decomposition is called Rao-Blackwellization and is the core of this chapter.

This chapter and Chapter 15 form a pair of controlled experiments: on the same problem (2D LiDAR SLAM), Gmapping takes the filtering route of Chapter 11 and Cartographer the optimization route of Chapter 12. Read both and then look again at the comparison of filtering and graph optimization in Section 12.7, and that table will no longer be an abstract list.

Figure 14-0 How Chapter 14 Connects to the Rest of the Book

[Building on the previous chapters]

Chapter 11 The particle filter → the skeleton of this chapter (predict, weight, resample)

Chapter 13 Occupancy grid mapping → each particle builds its own

Section 13.3 Scan matching → the key component for improving the proposal distribution in this chapter

[This chapter is responsible for]

Rao-Blackwellization: splitting the joint distribution into "particles for the pose, analytic for

the map"

Particle count, weights, resampling, and particle degeneracy

Gmapping's two key improvements: an improved proposal distribution and selective resampling

[Where it leads]

→ Chapter 15 Cartographer: the optimization solution to the same problem (the control group)

→ Chapter 16 AMCL: the particle filter applied to localization in a known map

→ Section 6.7 Automatic magnetometer calibration: the same RBPF applied to a different object

→ Chapter 29 Tuning Gmapping parameters on a physical vehicle

Learning Objectives

In the previous chapter we built a map assuming the pose was known, but real SLAM must untie localization and mapping at once. This chapter introduces the first complete SLAM system—Gmapping. Using the particle filter of Chapter 11, it lets every particle carry its own trajectory and its own map and runs a survival of the fittest among many hypotheses, untying the chicken-and-egg knot of Chapter 1 elegantly. After completing this chapter, you should be able to:

1. Describe the basic ideas of Gmapping and its place in the history of SLAM.
2. Explain the central idea of the Rao-Blackwellized particle filter: why letting each particle carry a map unties the chicken-and-egg problem.
3. Explain the roles of particle count, weight, and resampling, and the concepts of particle degeneracy and effective sample size.

4. Explain Gmapping's two key improvements: the improved proposal distribution and selective resampling.
5. Describe the complete procedure for running Gmapping in ROS and the prerequisites it requires.
6. Recognize common mapping failures—wheel slip, repetitive scenes, LiDAR occlusion—and propose remedies.
7. Complete the exercise of building a classroom map with a teaching vehicle or a simulator.

Core Concepts

Chinese Term	English Term	Explanation
粒子濾波 SLAM	RBPF-SLAM	A SLAM method estimating trajectory and map together with a particle filter.
Rao-Blackwell 化	Rao-Blackwellization	Decomposing the problem so that one part is sampled with particles and the other solved analytically.
粒子	Particle	One complete hypothesis: a trajectory together with its corresponding map.
權重	Weight	How well that particle's hypothesis agrees with the sensor observations.
重採樣	Resampling	The process of copying high-scoring particles and eliminating low-scoring ones by weight.
粒子退化	Particle Depletion	The loss of particle diversity caused by excessive resampling.
有效粒子數	Effective Sample Size	A measure of particle diversity, used to decide whether to resample.

Chinese Term	English Term	Explanation
提議分布	Proposal Distribution	The probability distribution generating candidate positions for the particles at the next step.
掃描匹配	Scan Matching	Comparing laser scans to refine the pose; Gmapping uses it to improve the proposal distribution.
選擇性重採樣	Selective Resampling	Resampling only when the effective sample size is too low, to avoid depletion.
地圖伺服器	map_server	The ROS tool for saving and loading map files.
地圖檔	pgm / yaml	The image and description file formats of a ROS map.

14.1 The Basic Ideas of Gmapping

Gmapping is one of the best-known and most widely used open-source systems in 2D laser SLAM. It was proposed by Grisetti, Stachniss, and Burgard in 2007—the German Freiburg and Bonn probabilistic robotics group introduced in Chapter 2. After finishing this chapter, watching Professor Stachniss explain particle filter SLAM himself on YouTube gives the rare experience of hearing it from the original author.

What Problem Gmapping Solves

Chapter 13 deliberately assumed a known pose and concentrated on mapping. But a real robot's pose is exactly what is unknown—and if the pose is wrong the map is wrong with it. Gmapping must answer two mutually dependent questions at once: what was the trajectory the robot traveled? And what does the map of the environment look like?

Its answer is most ingenious: do not try to find the one correct trajectory, but maintain dozens to hundreds of possible trajectory hypotheses at once, each building its own map; then see which hypotheses agree best with the sensor observations, let those survive and multiply, and eliminate the rest. This is the direct application of the survival-of-the-fittest idea of the particle filter in Chapter 11.

Inputs and Outputs

- Inputs: 2D laser scans (LaserScan, Chapter 8), odometry (Chapters 3 and 13), and the robot's coordinate frame configuration (TF, Chapter 10).
- Outputs: an occupancy grid map (Chapter 13) and the robot's pose in that map (that is, the map→odom transform, Section 10.7).

Note this list of inputs and outputs: it draws on almost every preceding chapter.

That is exactly the point of this part—theory converges here into a system that can really be run.

Its Place in History

Gmapping was for years the default choice for 2D SLAM in the ROS ecosystem, and countless university laboratories and teaching platforms began with it. In recent ROS 2 environments, graph-optimization solutions such as slam_toolbox (Chapter 12) have gradually become mainstream, but Gmapping's teaching value is undiminished: it is the clearest example of how a particle filter does SLAM, and the best bridge from the theory of Chapter 11 to a real system.

14.2 The Rao-Blackwellized Particle Filter

This is the central idea of the chapter. Gmapping's full name is Rao-Blackwellized particle filter SLAM, and behind that mouthful lies a very clever decomposition.

The Difficulty of Applying a Particle Filter to SLAM Directly

Recall Chapter 11: a particle filter approximates a probability distribution with a set of samples. Applied to SLAM directly, every particle would have to represent one hypothesis of both the robot's pose and the entire map. The trouble is that a map has tens of thousands of cells—tens of thousands of unknowns. Sampling such a high-dimensional space with particles would need an astronomical number of them, which is entirely impracticable. This is called the curse of dimensionality.

Rao-Blackwellization: A Clever Decomposition

The central insight of Rao-Blackwellization is to split the problem in two: sample one part with particles and compute the other analytically.

The key observation is the conclusion of Chapter 13: given the robot's complete trajectory, building the map is a deterministic and simple matter (log-odds updating will do, with no sampling required). In other words, the map need not be guessed at all; it can be computed directly from the trajectory.

The decomposition follows naturally:

- The part sampled with particles: the robot's trajectory (only three numbers per step: x, y, θ). This is low-dimensional and well within a particle filter's reach.
- The part computed analytically: the map. Each particle takes its own trajectory and computes the corresponding map directly by the method of Chapter 13.

The Core Idea: Every Particle Is a Parallel World

Imagine 100 parallel worlds, in each of which the robot walked a slightly different route (a trajectory hypothesis),

and so each drew a slightly different map.

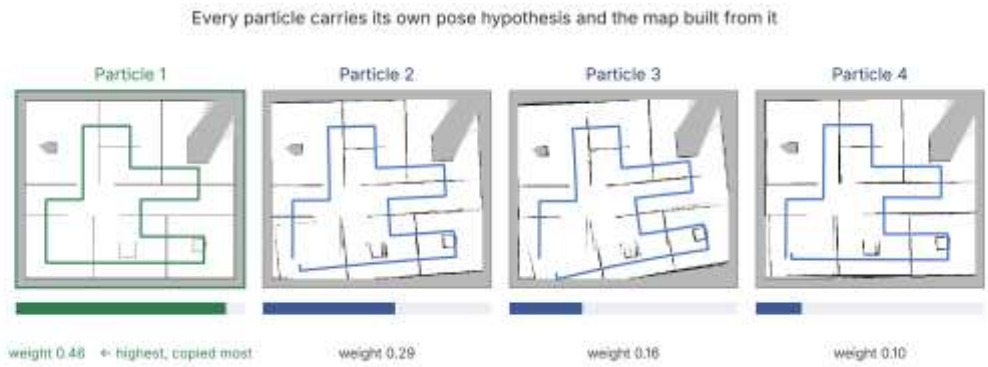
When a new laser scan arrives, every world asks itself: given my map and my position, is this scan plausible?

Plausible worlds score high (high weight); implausible ones score low.

At resampling, high-scoring worlds are copied and low-scoring ones vanish. What survives is the hypothesis most consistent with the observations.

That is Gmapping: finding the correct trajectory and map at once through competition among parallel worlds.

Figure 14-1 Rao-Blackwellization: Every Particle Carries Its Own Map



These four panels were produced from the same LiDAR data and differ only in the trajectory each of four particles hypothesizes—each particle has a different odometry error model, so each walks a different path and builds a different map.

Particle 1's hypothesis is closest to the truth, and its map has straight walls and square partitions; particle 4's hypothesis deviates most, and its map is twisted throughout with blurred walls. The weight is computed from exactly how well this laser scan agrees with my map—weights of 0.46, 0.29, 0.16, and 0.10, in exactly the order of map quality.

This is the whole spirit of Rao-Blackwellization. A particle need not guess pose and map at once (a dimension far too high to handle); it need only guess a trajectory, and the map is the inevitable consequence, computed by the method of Chapter 13. The particle filter thus goes from impossible to feasible.

Note one thing in particular: what is eliminated is not only the wrong pose but the wrong map with it. At resampling, particle 1 is copied several times and particle 4 deleted, and its skewed map vanishes along with it. This is the greatest difference between SLAM and localization alone—a wrong hypothesis contaminates pose and map together, so the whole set must be eliminated together.

A Frequently Asked Question: Is There Enough Memory?

Every particle carrying a map sounds memory-hungry—and it is. For 30 particles, a 20 × 17 meter site, and 5-centimeter resolution, the total is 30 × 136,000 = 4.08 million cells, about 4 MB.

That is nothing on a laptop, but on an embedded platform with only tens of MB of memory it is a real limit. It is also why Gmapping's particle count is usually set in the dozens rather than the thousands.

There is one implementation trick: at resampling, copied particles may share the same map (copy-on-write, duplicated only when written to), which saves a great deal of memory.

Why This Unties the Chicken-and-Egg Problem

The chicken-and-egg problem of Chapter 1 is that mapping needs a pose and localization needs a map. The RBPF's solution is that neither need come first. Each particle first hypothesizes a trajectory (which needs no map), builds a map from that trajectory (which needs only the trajectory), then evaluates how good the hypothesis is using the observations (which needs a map—and the map has just been built). The whole cycle is self-consistent and requires neither side to exist first. This elegant decomposition is Gmapping's important contribution to the history of SLAM.

14.3 Particle Count, Weights, and Resampling

This section goes into three practical essentials of the RBPF: how many particles to use, how to score them, and when to eliminate them.

1. Particle Count: Accuracy Against Computation

The particle count is Gmapping's most important parameter, and the trade-off is intuitive:

- Many particles (100, say): more hypotheses are covered and the system is more robust to complex environments and greater uncertainty, but every particle must maintain a map, so memory and computation cost more.
- Few particles (5 to 30, say): fast and light, but hypothesis coverage is insufficient, and in the face of large drift or a repetitive environment every particle may guess wrong with no way back.

In teaching practice, 30 particles usually suffice for an ordinary classroom or corridor; in a large environment or with poor odometry it can be raised to 50 or 100. One important reminder here: particle count times map size determines memory use—100 particles each carrying a 400×400 map is 16 million cells. This is the fundamental reason Gmapping is hard to use on very large sites, and one motivation for the later ascendancy of graph optimization methods (Cartographer, Chapter 15).

2. Weights: Scoring Hypotheses with Observations

A particle's weight represents how well this hypothesis agrees with the latest observation. It is computed by using that particle's pose and map to predict what should be measured and comparing with the actual laser scan.

If a particle's map says "a wall three meters ahead" and the actual scan also measures three meters, the two agree and the weight is high; if a particle's map says "ahead is empty" while a wall is actually measured, that particle's pose hypothesis is clearly off and its weight is low. This is the concrete realization of the observation model of Chapter 11.

3. Resampling: Survival of the Fittest, and Its Pitfall

Resampling copies high-scoring particles and eliminates low-scoring ones by weight, so that the particle set gradually gathers around the correct hypothesis. But a dangerous pitfall lurks here: particle depletion.

If resampling occurs at every step, the diversity of the particles is lost rapidly—in the end all of them may be copies of a single ancestor and nearly identical. The particle filter then loses the point of maintaining several hypotheses. Worse: if that heavily copied ancestor in fact guessed wrong, the system has no other hypothesis left to save it and SLAM fails completely.

This problem is especially severe in SLAM, because once a wrong trajectory hypothesis monopolizes the set, the corresponding wrong map is settled too and cannot be corrected afterward (the weakness of filtering relative to graph optimization noted in Chapter 12). Gmapping's second key improvement exists precisely to counter it—see the next section.

14.4 Gmapping's Two Key Improvements

A particle filter with one map per particle and nothing more would be impractical, because it would require a great many particles. Gmapping succeeds thanks to two improvements that cut the required particle count sharply. Understand these two and you understand the essence of Gmapping.

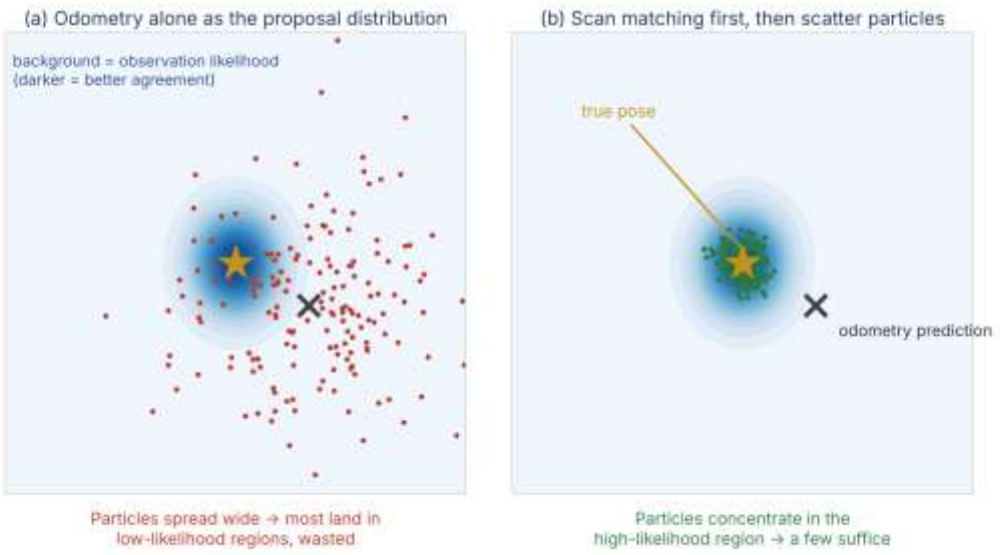
Improvement One: Improving the Proposal Distribution with Scan Matching

The proposal distribution is the probability distribution governing where to scatter the particles at the next step. The plainest approach uses odometry alone: the particles move by odometry and then spread out with random noise added.

The trouble is that odometry noise is often large, so the particles spread widely while the truly correct pose lies in only a small part of that area. Most particles are wasted at obviously wrong positions, so a great many are needed for a few to land by chance in the right place.

Gmapping's improvement is to refine the pose first with the scan matching of Chapter 13 and then scatter the particles over a far smaller area centered on the refined result.

Figure 14-2 Improving the Proposal Distribution



The background of both panels is the same observation likelihood—the darker the color, the better a laser scan taken from that position would agree with the map. The gold star is the true pose and the gray cross the position predicted by odometry, with a clear discrepancy between them.

The left panel is the plain approach: scatter particles about the odometry prediction. Because odometry noise is large, the particles spread over an area far wider than the high-likelihood region, and the great majority land at nearly impossible positions—

they will receive weights near zero and the round of computation is wasted.

The right panel is Gmapping's approach: find the position of highest likelihood first by the scan matching of Section 13.3, then scatter a small handful of particles about it. Nearly every particle lands within the dark region, and every one is a meaningful hypothesis.

The effect in numbers is striking: the particle count required falls from thousands to dozens. This is not merely a saving in computation but the key that let Gmapping run in real time on the embedded computers of its day. Compare this figure with the four steps of the particle filter in Figure 11-4 of Chapter 11 and you will see that what has been improved is step ②, prediction—blind scattering replaced by informed scattering.

The effect of this improvement is striking: it lets Gmapping achieve with dozens of particles what the plain method needs thousands to achieve. You should be able to see that this is the extended application of the "odometry predicts, LiDAR corrects" idea of Section 13.4.

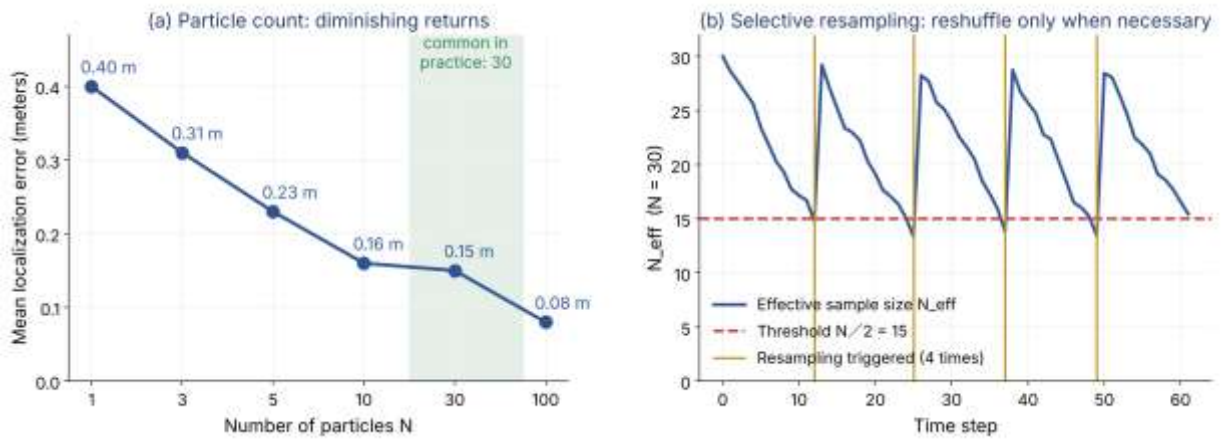
Improvement Two: Selective Resampling

Against the particle depletion problem of Section 14.3, Gmapping adopts selective resampling: rather than resampling at every step, it first assesses the diversity of the particles and resamples only when that diversity has fallen too low.

The measure of diversity is called the effective sample size (ESS). If all particles have similar weights (meaning all are about equally plausible), the ESS is high, diversity is good, and no resampling is needed; if a few particles have very high weights and the rest very low (meaning the contest has been decided), the ESS is low, and only then is resampling performed to concentrate resources.

This strategy strikes a balance between preserving diversity and concentrating on the correct hypothesis, and greatly slows particle depletion. These two improvements—an improved proposal distribution to cut the required particle count, and selective resampling to avoid depletion—are the core contributions of the Gmapping paper and the reason it became a practical system.

Figure 14-3 The Particle-Count Trade-Off and Selective Resampling



The left panel shows actual results: the same trajectory and the same noise settings with only the particle count changed. With 1 particle the mean localization error is about 0.40 meters—which is in fact no filtering at all, just dead reckoning on a single hypothesis; 5 particles bring it to 0.23 meters; 10 to 0.16; 30 to 0.15; and even 100 only to 0.08.

Note that there is almost no improvement from 10 to 30, and that going from 30 to 100 (more than tripling the computation) buys only 0.07 meters. This is diminishing returns—the quantitative version of "thirty particles are both enough and not enough" in Case 14-2. Gmapping's default particle count in practice falls near this sweet spot.

The right panel shows selective resampling at work. The blue curve is the effective sample size N_eff over time: just after a resampling all weights are equal and N_eff

equals the total of 30; as observations accumulate the weight concentrates on a few particles and N_eff falls steadily. Only when it drops below the threshold N / 2 = 15 (the red dashed line) is resampling triggered (the yellow vertical lines), five times over the whole trajectory.

This sawtooth curve makes the particle depletion of Section 14.3 quite clear: resample at every step and diversity is destroyed over and over, so a correct hypothesis may be deleted before it has had a chance to prove itself; never resample and all the weight concentrates on one particle while the rest become ornaments. Selective resampling strikes the balance between these extremes.

14.5 The Procedure for Running Gmapping in ROS

With the theory covered, this section describes the complete procedure for running Gmapping in ROS. It is also a preview of the implementation chapter, Chapter 29.

Prerequisites: Three Things, None of Which May Be Missing

When students fail at their first Gmapping run, the cause is almost always a missing prerequisite:

1. Laser scan data: the LiDAR driver publishing the /scan topic normally (the LaserScan format of Chapter 8).
2. Odometry data: the robot publishing the odom topic and the odom→base_link transform (Section 10.7). This requires wheel encoders and correct wheel-radius and wheelbase parameters.
3. A complete TF tree: the fixed base_link→laser transform above all (the extrinsics of the LiDAR mounting, Chapter 10). Set it wrong and the scan points are

displaced relative to the robot and the map must come out askew—which is the origin of the Chapter 10 maxim that a skewed map means checking TF first.

The Mapping Procedure

4. Start the robot's low-level drivers (motor control, encoder odometry, LiDAR).
5. Confirm the prerequisites: use the tools to check that /scan carries data, that the TF tree is complete and unbroken, and that odometry changes as the robot moves.
6. Launch the Gmapping node and set the key parameters (particle count, map resolution, update distance thresholds, and so on).
7. Open RViz and display the map, the laser scan, TF, and the particle cloud so that mapping can be watched in real time.
8. Teleoperate the robot slowly through the environment, covering every area to be mapped. Go slowly—this is one of the keys to success.
9. Watch the map grow: are the wall lines clear? Is there ghosting? Does the loop close after a circuit?
10. When mapping is complete, save the map with map_saver, which produces a .pgm image file and a .yaml description file.
11. For later navigation, load this map with map_server and localize with AMCL (Chapters 3 and 11).

Key Parameters

Parameter	Meaning	Tuning Advice
Particle count	How many trajectory hypotheses to	30 for a classroom; raise to 50–100 for a large environment or poor odometry

Parameter	Meaning	Tuning Advice
	maintain	
Map resolution	The actual side length each cell represents	0.05 m is common indoors (Section 13.5)
Update distance / angle threshold	How far it must move before an update	Too large misses detail, too small wastes computation
Maximum usable laser range	Measurements beyond this are discarded	Slightly below the LiDAR's nominal range is more robust
Odometry noise parameters	How inaccurate odometry is assumed to be	Raise on a platform that slips badly

Table 14-1 Principal Gmapping Parameters and Tuning Advice

14.6 Common Failures: Wheel Slip, Repetitive Scenes, LiDAR Occlusion

This section may be the most useful in the chapter for implementation. Failure in SLAM is the norm rather than the exception, and recognizing failure modes and treating them appropriately is the core of engineering ability.

Failure One: Wheel Slip and Inaccurate Odometry

Symptom: the map shows clear misalignment or ghosting, especially at turns; the robot believes it has traveled far and the map is stretched.

Cause: wheels slipping on a smooth floor, an abrupt change in friction at the boundary between carpet and tile, or wrong wheel-radius or wheelbase parameters (this last is most often overlooked and produces a systematic scale error that enlarges or shrinks the whole map proportionally).

- Remedy: calibrate wheel radius and wheelbase properly (drive the robot 5 meters straight and turn it 10 times in place, comparing odometry readings with measurements); slow down to reduce slip; and raise Gmapping's odometry noise parameters so the system trusts odometry less and relies more on scan matching.

Failure Two: Repetitive Scenes and Insufficient Features

Symptom: a long corridor drawn now long, now short, with overlapping wall lines; the robot thoroughly lost in a large open hall. This is exactly the corridor difficulty of Section 13.6.

Cause: a lack of distinctive geometric features along the corridor means scan matching cannot determine the amount of longitudinal motion; in open space everything around exceeds the LiDAR's range and there is nothing to match.

- Remedy: ensure odometry is of good quality (it is then the only source of longitudinal information); raise the particle count to maintain more hypotheses; place temporary obstacles such as cardboard boxes along the corridor to provide features (a technique commonly used in teaching); or use a longer-range LiDAR.

Failure Three: LiDAR Occlusion and Invisible Obstacles

Symptom: gaps appear in the map where there should be none; the robot runs into something not drawn on the map.

Cause: glass and mirrors let the laser pass through or reflect it (Chapter 8); black light-absorbing objects reflect weakly; an obstacle is not at the height of the LiDAR's scan plane (the height blind spot of Chapter 8 and the chair-leg problem of Case 8-1); or someone standing in front of the LiDAR creates a moving occlusion.

- Remedy: clear the site while mapping and keep people from walking around the robot; glass partitions may be temporarily papered over or shielded with cardboard boxes during mapping; and recognize that a 2D map reflects only one height slice, so navigation must be supplemented with other sensors.

Failure Four: Moving Too Fast

Symptom: scan matching cannot keep up, and the map is suddenly misaligned or even blows up.

Cause: the robot moves or turns too quickly, the difference between successive scans is too large, and the convergence range of scan matching is exceeded (ICP needs a good initial guess, Section 13.3).

- Remedy: slow down, especially while turning. This is the simplest and most effective measure of all—in teaching experience, most mapping failures are solved by simply driving more slowly.

Failure Mode	Typical Symptom	Principal Remedy
Wheel slip	Map misalignment, ghosting, distorted scale	Calibrate wheel radius and wheelbase, slow down, raise odometry noise
Repetitive scene	Corridor deformed, robot lost	Ensure odometry quality, add particles, add temporary features
LiDAR occlusion	Gaps in the map, hitting obstacles not drawn	Clear the site, deal with glass, recognize the height blind spot
Moving too fast	Map suddenly misaligned or blown up	Slow down, especially while turning

Table 14-2 Common Gmapping Failure Modes and Remedies

14.7 Teaching Practice: Building a Classroom Map with a Small Vehicle

This section describes how to turn Gmapping into a workable class session, with suggestions at two levels of depth for high school and undergraduate courses. Detailed steps appear in this chapter's hands-on activities.

The High School Version: Understand First, Then Do

A high school course should center on observation and experience without going deep into the mathematics:

- Use a simulator first (TurtleBot3 in Gazebo, Chapter 28) so that every student can operate one without being limited by the amount of hardware.
- Focus on three things: how the map grows as the robot moves, how the particle cloud spreads and converges with motion, and how the map goes wrong when driven too fast.
- Demonstrate failure deliberately: have students drive the robot very fast and watch the map blow up before their eyes—the shock of failure builds intuition better than the smoothness of success.
- Connect it to daily life: compare with the map in a home cleaning robot's app (Case 13-1) and see that the two are the same thing.

The Undergraduate Version: Tune the Parameters, Understand the Mechanism

Undergraduates should go further, operating a physical platform and understanding the effect of the parameters:

- Complete the TF configuration and odometry calibration by hand to feel the importance of the prerequisites (this often takes half the debugging time).
- Experiment with the parameters systematically: vary the particle count, map resolution, and odometry noise, and compare map quality quantitatively (by the assessment methods of Section 13.7).
- Map both a classroom and a corridor and check the prediction of Section 13.6—that the corridor is indeed harder.
- After mapping, go on to run AMCL localization and navigation (Chapter 29), completing the full chain of mapping → localization → navigation.

Teaching Notes

From years of teaching experience, three points are recommended to instructors. First, explain the prerequisites and failure modes clearly before letting students start—otherwise the class drowns in debugging. Second, stress going slowly: speed is the single largest variable in a beginner's success rate. Third, be sure to include a step for assessing map quality (Section 13.7), so that students form the habit of not merely building a map but judging whether it is good. These three raise the success rate of a SLAM practical considerably.

You have now been through a first complete SLAM system. Gmapping unties the chicken-and-egg problem elegantly with a particle filter, but it has clear limits: the memory pressure of particle count times map size makes large scenes hard to handle, and a filtering method cannot go back and correct history (Chapter 12). The Cartographer of the next chapter takes another road—using the graph optimization and loop closure detection of Chapter 12 to make large-area mapping and a one-click straightening possible. Comparing the two will give you a deeper understanding of the

trade-off between the filtering and optimization routes.

Formulations and Procedures

1. The Probabilistic Factorization of Rao-Blackwellization

SLAM must estimate the joint distribution over trajectory and map (Chapter 1). The key of the RBPF is to split it into two factors:

$$p(x(1:t), m \mid z(1:t), u(1:t)) = p(m \mid x(1:t), z(1:t)) \cdot p(x(1:t) \mid z(1:t), u(1:t))$$

The second factor on the right is the distribution over trajectories—low-dimensional, sampled with particles; the first is the distribution over maps given a trajectory—which Chapter 13 showed can be computed directly in log-odds with no sampling. This expression is the mathematical statement of the decomposition of Section 14.2 and a distillation of all of Gmapping's ingenuity.

2. The Particle Weight

The weight of the i-th particle is updated as:

$$w^{(i)} \propto p(z_t \mid m^{(i)}, x_t^{(i)})$$

Read: the weight is proportional to the likelihood of observing z_t given that particle's map $m^{(i)}$ and pose $x_t^{(i)}$. The better the scan agrees with the map, the higher the weight. This is the concrete form of the observation model of Chapter 11 within SLAM.

3. Effective Sample Size and Selective Resampling

The measure of particle diversity:

$$N_eff = 1 \;/\; \Sigma_i \, (w^{(i)})^2 \quad \textit{(with the weights normalized)}$$

If all particles have equal weight (diversity at its best), N_eff equals the total count

N; if the weight concentrates on a single particle, N_eff approaches 1. Gmapping's selective resampling rule is:

$$\textit{resample only if } N_{eff} < N / 2$$

This simple test is the key mechanism for avoiding particle depletion in Section 14.4.

4. One Complete Step of Gmapping

Figure 14-4 The Processing of One Gmapping Step

① Obtain the new odometry reading and laser scan

↓

② For each particle: take odometry as the initial guess and run scan matching to refine the pose

↓

③ Sample within a small neighborhood of the refined result for that particle's new pose (the improved proposal distribution)

↓

④ Compute the weight: compare the scan that particle's map predicts with the actual scan

↓

⑤ Each particle updates its own map with its own new pose (log-odds, Chapter 13)

↓

⑥ Compute N_eff; if it is below the threshold, resample (selective resampling)

↓

⑦ Output the map and pose of the highest-weight particle → return to ①

5. Estimating Memory

Gmapping's memory requirement is the particle count times the size of a single map:

$$\textit{total cells} = \textit{particle count} \times (\textit{width} \,/\, \textit{resolution}) \times (\textit{height} \,/\, \textit{resolution})$$

For example, 50 particles, a 20 m × 20 m room, and 0.05 m resolution: 50 × 400 × 400 = 8 million cells. This expression quantifies the limitation noted in Section 14.3—and is the fundamental reason Gmapping scales poorly to large scenes.

14.8 Gmapping Today: Tools Become Obsolete, Ideas Do Not

Let us be honest with students: to build a 2D map in ROS 2 today, most people would not choose Gmapping. It is a package of the ROS 1 era, and the mainstream of the official ecosystem has moved to the slam_toolbox introduced in Section 13.8. Why, then, does this book devote a whole chapter to it?

1. Because the RBPF Has Not Gone Away

The Gmapping package will retire, but the Rao-Blackwellized particle filter behind it is alive in three places.

First, the AMCL of Chapter 16—cleaning robots and service robots localizing in a known map use exactly a particle filter, simply without having to build the map any more. Second, the localization modules of consumer cleaning robots adopt this route extensively, because it performs stably on low computing resources and is simple to implement. Third, it continues to be carried over to new problems: the automatic magnetometer calibration of Section 6.7 is an example—a factorized particle filter estimating pose, magnetic field map, and sensor bias at once, with a skeleton identical to this chapter's.

In other words, what this chapter teaches is not how to use a piece of software but a structure for solving problems: when part of the state can be solved analytically given the rest, factor it out and use particles only on the genuinely hard part. This structure will prove useful again and again in later research.

2. Its Weakness Is Exactly the Subject of the Next Chapter

Several comparative studies of 2D SLAM reach the same conclusion: in corridors, large sites, and scenes requiring loop closure, methods with an optimization back end (the Cartographer family) are clearly better than Gmapping in map accuracy. The reason has been foreshadowed in this chapter—a particle filter is recursive, and once every particle has strayed at some step there is no way back; and resampling itself keeps consuming diversity.

This is not poor implementation but a structural limitation of the filtering route, as the summary of Chapter 11 already said. The Cartographer of Chapter 15 fills that gap with the pose graph optimization of Chapter 12: keep all history and correct it in one pass at a loop closure. Reading the two chapters in sequence gives a felt understanding of why nearly every modern SLAM back end uses optimization, rather than merely a memorized conclusion.

3. The Particle Filter Is Itself Advancing

It is worth noting that research on particle filters has not stood still, and recent improvements run broadly in three directions. First, learning the proposal distribution: a neural network learns from data where to scatter the particles, replacing this chapter's scan matching. Second, parallelization: particles are mutually independent and naturally suited to a GPU, which makes thousands of particles no longer a burden today. Third, hybrids with optimization: particles handle multimodality and uncertainty while optimization handles local refinement—a layered approach visible in LIO-SAM in Chapter 18 and ORB-SLAM3 in Chapter 23.

Three Judgments for Students

1. Do not learn only the tool when learning an algorithm. How slam_toolbox is used will change within a few years, but the structure of factorizing and representing multimodality with particles will not.

2. Filtering and optimization do not displace each other. Section 11.8 said it: filtering for high-rate real-time work, optimization for global consistency; a complete system has both.

3. This chapter's exercise is still worth doing. Even if the final product does not use Gmapping, watching thirty particles converge step by step and N_eff drop below the threshold to trigger a resampling is the fastest route to understanding the whole family of probabilistic SLAM.

Case Studies

Case 14-1 The Original Authors' Classroom: From Paper to YouTube

Gmapping's three authors—Grisetti, Stachniss, and Burgard—are core members of the Freiburg and Bonn probabilistic robotics group introduced in Chapter 2. Interestingly, Professor Stachniss later recorded the complete derivation of particle filter SLAM and published it free on YouTube, where it has become shared teaching material for SLAM learners worldwide.

This means that after finishing this chapter you can go straight to the algorithm's originator explaining it himself—he shows with animations how the particles spread, how they are weighed by observations, and how resampling makes them converge. This combination of reading a textbook for the architecture and watching the original author for the intuition is the best practice of the learning route recommended in Chapter 2.

This case also echoes "the power of open knowledge" in Chapter 2: an algorithm published in a top journal in 2007 is available today to any high school or university student in Taiwan—the original paper, the open-source code, and a lecture by the author, all free. The barrier to learning SLAM has moved from whether one can get into a famous school to whether one has a good map of the subject and sustained persistence—exactly the role this book hopes to play.

Case 14-2 A Classroom of Thirty Particles: Why Gmapping Is Both Enough and Not Enough

A typical observation in university teaching experiments: mapping an ordinary classroom with 30 particles gives a quite beautiful result—straight wall lines, a closed loop, clear detail. But take the same settings to a whole floor (tens of meters across, several rooms and a long corridor) and map quality falls noticeably, or fails outright.

The reason is twofold. First, memory and computation: as the scene grows, the cell count of each map explodes, and multiplied by the particle count the burden is heavy (formulation 5). Second, and more fundamental: a larger scene means a longer trajectory and more accumulated drift, requiring more particles to cover the possible hypotheses; but with more particles the computation cannot keep up—a dilemma.

More crucially: when the robot completes a large circuit back to its starting point, even if the loop closure is detected, a particle filter cannot go back and correct the whole historical trajectory as graph optimization can—every particle's map was long since drawn along a wrong trajectory and cannot be redone (the central argument of Chapter 12). This is exactly the problem Cartographer in Chapter 15 sets out to solve. Performing this "classroom succeeds, whole floor fails" comparison with your own hands gives a deep sense of the difference between the two chapters.

Case 14-3 Particles Inside the Cleaning Robot: The Theory Is in Your Home

A case in Chapter 11 noted that many cleaning robots have a debug mode in which the particle cloud can be seen converging. This chapter takes it further: a cleaning robot's first mapping and its later cleanings in fact correspond to two different techniques.

Entering an unfamiliar environment for the first time, the robot must localize and map at once—the SLAM of this chapter. Once the map is built, every later cleaning need only localize within a known map, which is the Monte Carlo localization (AMCL) of Chapters 3 and 11. Both share the particle filter skeleton and differ in whether the map is known: SLAM's particles each carry a growing map, while AMCL's particles share one fixed map.

Understanding this distinction matters, because it corresponds to two different tools and procedures in ROS (Section 14.5): run Gmapping during mapping and save the map; load the map with map_server during navigation and run AMCL to localize. The exercises of Chapter 29 walk through this chain in full. Next time you watch the cleaning robot at home explore gingerly on its first run and set to work confidently on its second, you will know it has switched algorithms.

Hands-On Activities

Exercise 14-1 Building a Classroom Map with Gmapping

Purpose: to run a complete particle filter SLAM mapping session, observe the evolution of the particle cloud, and understand Gmapping's mechanism and limits through parameter experiments and reproduced failures. The exercise offers both a simulation

route and a hardware route.

Route A: Mapping in Simulation (the whole class, no hardware needed)

1. Launch the TurtleBot3 Gazebo simulation environment and the SLAM node (Chapter 28 has the complete installation steps).
2. Open RViz and display at once: the occupancy grid map, the laser scan, the TF axes, and the particle cloud (if the tool supports it).
3. Teleoperate the robot slowly around the simulated room with the keyboard, watching the map grow cell by cell and the unknown gray region fill in as you go.
4. Observe the particle cloud: how does it spread while the robot drives straight? How does it converge at a turn or on seeing a distinct feature? Compare with the theory of Chapter 11 and Section 14.3.
5. Speed experiment: map once very slowly, then map again at top speed, compare the quality of the two maps, and reproduce the "moving too fast" failure of Section 14.6.
6. Save the map and assess its quality by the methods of Section 13.7: are the walls straight? Does the loop close? Measure the room dimensions on the map against the simulation settings.

Route B: Mapping with a Physical Vehicle (for classes with hardware)

7. Pre-checks: confirm that /scan carries data, that the TF tree is complete (base_link→laser above all), and that odometry changes with motion. This step may not be skipped.
8. Odometry calibration: drive the vehicle 3 meters straight and compare the odometry reading with a tape measurement; turn it 10 times in place and

compare the angle reading. If the error is large, correct the wheel-radius and wheelbase parameters.

9. Clear the site: ask students to withdraw to the edges of the room, remove objects that might occlude the LiDAR, and temporarily paper over any glass partitions.
10. Teleoperate the vehicle slowly once around the classroom (including all four corners), return to the starting point, and observe whether the loop closes.
11. Save the map files (.pgm and .yaml), measure the classroom dimensions on the map against tape measurements, and compute the percentage error.

Part Two: Parameter Experiments (Undergraduate)

12. Particle count experiment: map the same site with 5, 30, and 100 particles and compare map quality against computational burden (observe CPU usage and update rate).
13. Resolution experiment: map at resolutions of 0.02, 0.05, and 0.10 m and compare detail against memory use, verifying the estimate of formulation 5.
14. The corridor challenge: drive the robot into a long corridor and map it, checking the predictions of Sections 13.6 and 14.6 and observing whether distortion appears. Try placing cardboard boxes along the corridor to provide features, map again, and compare the improvement.
15. Slip experiment: lay a smooth board on the floor or lift one wheel briefly to induce slip, and observe how the map goes wrong.

Discussion Questions

1. How far do the maps built at high and low speed differ? From the standpoint of Section 14.6 and Section 13.3, explain why driving slowly is so effective.

2. Going from 5 particles to 100, how much did map quality improve? How much did the computational burden increase? What do you think is the most suitable particle count for your site?
3. What went wrong mapping the corridor? Did the cardboard boxes improve it? Which claim of Section 14.6 does this confirm?
4. If the robot completes a large circuit back to its starting point and the map does not close, can Gmapping correct the whole map after the fact? Why not? (Hint: Chapter 12 and Case 14-2.)

Extended Challenge (Undergraduate)

First: implement a minimal RBPF simulation in Python—define a virtual room in code, create 20 particles (each with a pose and a small map), implement the loop of move with noise → weigh → selectively resample → each updates its own map, draw every particle's map side by side, and watch the correct hypothesis win. Second: after mapping, go on to launch map_server and AMCL so the robot localizes in the map it built and performs navigation (Chapter 29), completing the full chain of mapping → localization → navigation.

Further Reading

1. G. Grisetti, C. Stachniss, and W. Burgard, "Improved Techniques for Grid Mapping with Rao-Blackwellized Particle Filters," IEEE Transactions on Robotics, 2007. (The original Gmapping paper and the principal basis of this chapter.)
2. M. Montemerlo et al., "FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem," AAAI, 2002. (The pioneering work on RBPF-SLAM, a useful counterpart to Gmapping.)

3. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (Chapter 13 gives the complete theory of FastSLAM and particle filter SLAM.)
4. Cyrill Stachniss, the Particle Filter and FastSLAM course videos (YouTube). (Explained by the original author; best read alongside this chapter.)
5. The OpenSLAM Gmapping project and the documentation of the ROS slam_gmapping package. (Original source code and parameter descriptions.)
6. The slam_toolbox open-source project (GitHub). (Modern 2D SLAM based on graph optimization in ROS 2; useful for comparative experiments against Gmapping.)
7. The SLAM chapters of the ROBOTIS TurtleBot3 e-Manual. (Complete operating instructions for mapping in simulation and on hardware.)

From Particles to Graphs: Where This Chapter Stands

This chapter has shown SLAM working properly for the first time: pose and map estimated together, and estimated rather well. It is the first complete system of Part IV.

But the curve in the left panel of Figure 14-3 states its ceiling honestly. Even raising the particle count to a hundred brings the mean error only to 0.08 meters, and the computation is already more than triple that of thirty. The more fundamental problem is that every correction a particle filter makes happens in the present—it has no mechanism whatever for going back and rewriting the trajectory it has traveled.

Imagine the robot completing a circuit of a whole building back to its starting point and finding that what it sees does not match the starting point recorded on the map. All Gmapping can do is correct slowly from that moment on; the error of the whole circuit, already written into every particle's map, can never be taken back out. This is the

drift symptom of Section 13.7, and its remedy is loop closure.

The Cartographer of Chapter 15 solves this with three things: accumulating scans into submaps to reduce the size of the pose graph, searching the whole map efficiently for loop closure candidates by branch and bound, and then correcting the positions of all submaps in one pass by the pose graph optimization of Chapter 12. In the same classroom with the same vehicle, Chapter 15 will let you see for yourself how the two maps differ—and looking back at this chapter then, both the merits and the limits of Gmapping will have become very concrete.

Summary

1. Gmapping was proposed by Grisetti, Stachniss, and Burgard in 2007 and is one of the best-known 2D laser SLAM systems; its inputs are laser scans, odometry, and TF, and its outputs an occupancy grid map and the map→odom transform.
2. Rao-Blackwellization splits SLAM in two: the trajectory (low-dimensional, sampled with particles) and the map (solvable analytically given the trajectory); each particle is a parallel world carrying one trajectory and one map, competing on observation scores and untying the chicken-and-egg problem elegantly.
3. The particle count trades accuracy against computation (about 30 for a classroom); the weight represents how well a hypothesis agrees with the observation; resampling lets good hypotheses survive, but excessive resampling causes particle depletion, letting a wrong hypothesis monopolize the set irrecoverably.
4. Gmapping's two key improvements: improving the proposal distribution with scan matching (scattering particles at plausible positions and cutting the

required count from thousands to dozens), and selective resampling (resampling only when the effective sample size falls below a threshold, to avoid depletion).

5. The three prerequisites of a ROS implementation, none of which may be missing: /scan laser data, odometry with the odom→base_link transform, and a complete and correct TF tree (the base_link→laser extrinsics above all); after mapping, save with map_saver as .pgm and .yaml.
6. The four major failure modes: wheel slip (calibrate wheel radius and wheelbase, raise odometry noise), repetitive scenes (corridors—rely on odometry and add features), LiDAR occlusion (glass, black objects, the height blind spot), and moving too fast (slowing down is the most effective single measure).
7. Gmapping's fundamental limitation is that memory grows as particle count times map size, which scales poorly to large scenes; and as a filtering method it cannot go back and correct the historical trajectory and map—exactly the problem the optimization route of the next chapter sets out to solve.

This chapter completed a first full SLAM system. Gmapping unties the chicken-and-egg problem elegantly with a particle filter, but it also shows us the ceiling of the filtering route plainly: large scenes are too much for it, and history cannot be corrected. The next chapter brings in Cartographer—developed by Google, it organizes space into submaps, performs loop closure correction with the graph optimization of Chapter 12, maps large areas in real time, and straightens a whole skewed map at one stroke the instant a loop closure is detected. That striking picture from Chapter 12 is about to reappear in a real system.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 15

Cartographer: Google's 2D / 3D SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

At this point in Part IV, each of the two main lines of theory has at last grown into a complete system. The Bayesian filtering of Chapter 11 grew into Gmapping in Chapter 14; the graph optimization of Chapter 12 grows into Cartographer in this chapter. The two chapters are not parallel substitutes but two answers to the same problem within two mathematical frameworks, which is why reading them together yields twice what reading them separately would.

Looking back, this chapter draws on nearly the whole stock of the first three parts: the LiDAR specifications of Chapter 8 determine the quality of the material for scan matching, the IMU of Chapter 7 is all but a necessity in 3D mode, the SE(2) pose composition of Chapter 10 is the underlying language of stitching submaps, the probabilistic grid idea of Chapter 11 was made concrete in Chapter 13 as the internal representation of a submap, and the sparse pose adjustment of Chapter 12 is the engine of this chapter's global SLAM itself. Looking forward, the .pbstream maps and loop closure correction produced here will be drawn upon again and again in the navigation of Part V, the multi-robot work of Part VI, and the field applications of Part VII.

Figure 15-0 The Connection Map of Chapter 15

The stock of theory from the first three parts

Chapter 7 IMU Chapter 8 LiDAR Chapter 10 SE(2) pose

| | |

└──────┼──────┘

▼

Ch. 11 Probabilistic filtering ── Ch. 13 Occupancy grid ── Ch. 12 Graph optimization

| | |

▼ ▼ ▼

Chapter 14 Gmapping [Chapter 15 Cartographer]

(the filtering route) (the optimization route, this chapter)

|

▼

Part V Navigation Part VI Multi-robot collaboration Part VII Field applications

Learning Objectives

The Gmapping of the previous chapter untied the chicken-and-egg problem with a particle filter but also exposed the ceiling of the filtering route: large scenes are too much for it, and history cannot be corrected. This chapter introduces Cartographer, developed by Google—it organizes space into submaps, performs loop closure correction with the graph optimization of Chapter 12, maps large areas in real time, and straightens a whole skewed map at one stroke the instant a loop closure is detected. After completing this chapter, you should be able to:

1. Describe Cartographer's two-subsystem architecture and the division of labor between local SLAM and global SLAM.
2. Explain the concept of a submap and why scans are grouped into submaps.

3. Explain the purpose of and difference between local and global scan matching.
4. Explain how branch and bound makes real-time loop closure detection possible.
5. Describe the procedure and key settings for running 2D Cartographer mapping in ROS.
6. Describe 3D Cartographer and its application to backpack spatial modeling.
7. Compare Cartographer with Gmapping in architecture, strengths and weaknesses, and suitable settings.

Core Concepts

Chinese Term	English Term	Explanation
子地圖	Submap	A small probability grid map made of a few dozen scans; Cartographer's basic unit.
局部 SLAM	Local SLAM	The front end: inserts new scans into the current submap; locally consistent but drifting.
全域 SLAM	Global SLAM	The back end: seeks loop closure constraints in the background and optimizes the whole pose graph.
機率格	Probability Grid	The internal representation of a submap, each cell recording an occupancy probability.
掃描匹配	Scan Matching	The process of aligning a new scan to the current or an existing submap.
閉環約束	Loop Closure Constraint	An edge connecting revisited places, used to eliminate accumulated error.

Chinese Term	English Term	Explanation
分支定界	Branch and Bound	An algorithm that searches efficiently for the best match and rejects impossible solutions.
多解析度預算格	Precomputed Grids	Maps precomputed at several resolutions to accelerate the branch-and-bound search.
稀疏位姿調整	Sparse Pose Adjustment	The optimization performed on the graph formed by submap and scan poses.
非線性最小平方	Nonlinear Least Squares	The form of optimization Cartographer solves with Ceres.
背包式建模	Backpack Mapping	Carrying a sensor suite on foot to scan the space of a building.
穩健核函數	Robust Kernel	An optimization technique that reduces the influence of a false loop closure.

15.1 The Architecture of Cartographer

Cartographer was developed by Google and open-sourced in 2016, and the title of its core paper names its headline feature directly: "Real-Time Loop Closure in 2D LIDAR SLAM." That name itself declares the fundamental difference from Gmapping—loop closure detection and graph optimization are built into the core of the system, and they run in real time.

Two Subsystems: Local and Global

The most important thing in understanding Cartographer is to grasp its two-subsystem architecture. It may be regarded as two independent but related subsystems: local SLAM (also called the front end or the local trajectory builder) is responsible for building a sequence of submaps, each locally consistent, and we accept that local SLAM will drift over time; global SLAM runs in a separate thread and looks for loop closure constraints among the submaps.

- Local SLAM (the front end): matches each new scan and inserts it into the current submap. It aims at real-time operation and local accuracy, in the full knowledge that drift will accumulate over time—a deliberate design trade-off.
- Global SLAM (the back end): in a background thread, matches scans against nearby submaps to establish loop closure constraints, and continuously optimizes the constraint graph formed by submap and scan poses.

The key is that both run at once. The foreground maps quickly so the operator sees results immediately, while the background quietly seeks loop closures and restores global consistency. This is why it is real time, and the greatest architectural difference from Gmapping.

Figure 15-1 Cartographer's Two-Subsystem Architecture

[Local SLAM (the front end)—running in the foreground]

New scan → scan matching to the current submap → insert into the submap → seal the submap when complete

Character: real-time, locally consistent, drifting

[Global SLAM (the back end)—a background thread]

Scan ↔ matched against nearby existing submaps → a good match establishes a loop closure constraint

↓

Submap poses and scan poses form a pose graph (Chapter 12)

↓

Every few seconds, solve the nonlinear least-squares problem with Ceres → the whole map is straightened

Both run at once: the foreground gives a real-time preview, the background guarantees global consistency

The Connection to Chapter 12

You should recognize it at once: what global SLAM does is exactly the graph optimization of Chapter 12. It combines submaps formed by local scan matching (probability grids refined by a nonlinear least-squares optimizer based on Ceres) with

a sparse pose graph carrying branch-and-bound loop closure detection, achieving accurate real-time 2D SLAM on ordinary hardware. Chapter 12's mention that Cartographer uses the Ceres library is confirmed here. Theory and system formally meet in this chapter.

15.2 Submaps

The submap is Cartographer's most distinctive design. Understand why it exists and you understand why Cartographer can handle large scenes.

What a Submap Is

A submap is a small probability grid map (the occupancy grid of Chapter 13) accumulated from a few dozen consecutive laser scans. As the robot moves, new scans are continuously inserted into the current submap; once the configured number of scans has accumulated, that submap is finished and sealed, and the system begins building the next. The environment is thus cut into a series of partly overlapping small maps.

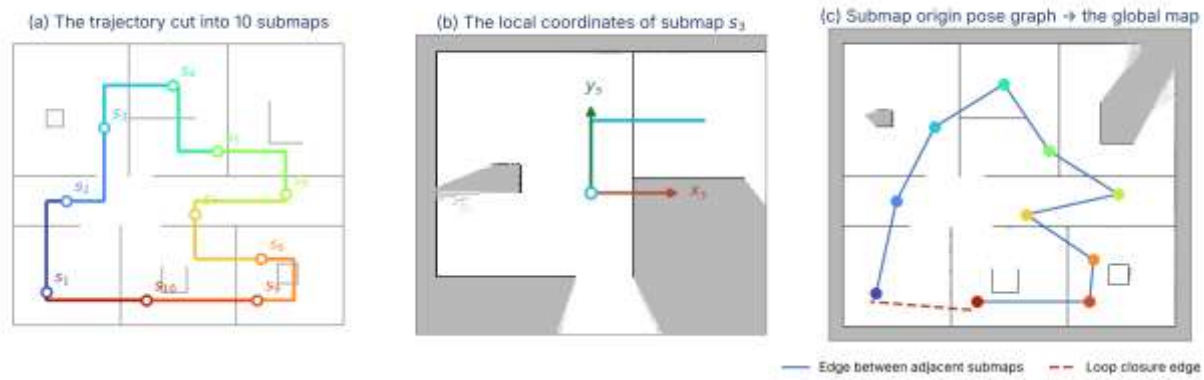
Why Submaps: Three Reasons

1. Bounding the accumulation of error: because a scan is matched only against a submap containing a few recent scans, this approach accumulates error slowly; over merely a few dozen consecutive scans the accumulated error is very small. In other words, within a submap's brief lifetime the drift is negligible—each submap is internally clean.
2. Providing a unit for optimization: in global optimization there is no need to adjust every scan, only the poses of the submaps. One submap represents

dozens of scans, which reduces the node count of the pose graph sharply and lets the optimization finish in real time. This is the key to efficiency.

3. Serving as the object of loop closure comparison: in loop closure detection, the current scan is compared against past submaps. A submap carries more information than a single scan and is more stable, so the comparison is more reliable.

Figure 15-2 Three Aspects of the Submap: Segmentation, Local Coordinates, Stitching



This figure breaks the abstract word "submap" into three aspects the eye can confirm. The left panel shows one trajectory cut into ten segments, each a submap s_1 through s_{10}; the open circles mark each submap's origin pose, the position and heading of the robot when that stretch began. Note that the division is not by distance traveled but by how many scans have accumulated—Cartographer seals a submap by scan count precisely to keep the internal error of each submap at a negligible magnitude.

The middle panel takes the third submap on its own and places it in its own local coordinate frame. The red and green arrows are that submap's x and y axes, with the origin always at the start of that stretch. Inside the submap there is no knowledge whatever of where it is in the world; it remembers only where this wall and that corner are relative to its own origin. This deliberate amnesia is the key to Cartographer's ability

to cut a large scene small: let the local be local and the global be global.

The right panel is the stitched result. Blue lines join the origins of adjacent submaps into a pose chain; the red dashed line is the loop closure edge, pulling the last submap back to the first. The variables actually optimized are only the ten colored dots in the figure—the poses of ten submap origins, thirty unknowns in all. By comparison this trajectory itself has nearly sixty time steps; optimizing the pose of every scan would double the unknowns, and the computation of the optimization grows roughly with their number. This is the concrete meaning of "providing a unit for optimization" in Section 15.2.

Take the three panels together and the submap's role is clear: it is at once an isolation chamber for error (local error does not leak out), a Lego brick for optimization (one brick representing dozens of scans), and the object of loop closure comparison (carrying enough information to be more stable than a single scan). One design solves three problems at once, and that is the engineering taste most worth learning from Cartographer.

How Large Should a Submap Be?

Too small: the number of submaps explodes, the pose graph gains nodes, and global optimization slows; and each submap contains too few geometric features, so loop closure comparison misjudges easily.

Too large: the scan matching error accumulated within a submap becomes significant, the premise that local error is negligible is broken, and wall lines remain askew even after loop closure straightening.

A practical starting point: indoor 2D sites commonly seal a submap every 70 to

100 scans (num_range_data in Table 15-1); reduce it where features are sparse, as in corridors, and raise it in cluttered rooms.

How to judge: look at the overlap between submaps in RViz. If the wall lines of adjacent submaps coincide, the size is right.

The Contrast with Gmapping

The philosophical difference between the two systems is clear here. Gmapping has every particle maintain one complete map (Chapter 14), with memory growing as particle count times map size; Cartographer maintains a single map stitched from many submaps and restores global consistency by adjusting submap poses.

The consequence is direct: Cartographer's memory requirement is proportional to the size of the scene (not multiplied by a particle count), so it can handle a whole building; and because it keeps the submaps and the pose graph, it can go back and correct—exactly the core advantage of graph optimization over filtering noted in Chapter 12.

15.3 Scan Matching

There are two kinds of scan matching in Cartographer with quite different purposes. Keeping them apart is the key to understanding how the system works.

1. Local Scan Matching: Insertion into the Current Submap

This is the front end's everyday work. For each new scan the system must decide where it should be pasted into the current submap. It takes an initial guess from odometry or the IMU (the idea of Section 13.4) and refines it by optimization so that the scan points agree as closely as possible with the submap's probability grid.

Cartographer formulates this match as a nonlinear least-squares problem and solves it with Ceres (Chapter 12). This differs from traditional ICP (Section 13.3)—rather than finding point-to-point correspondences, it directly maximizes the extent to which scan points fall into cells of high occupancy probability, which is less sensitive to noise.

2. Global Scan Matching: Finding Loop Closures

This is the back end's work and far harder. The question is: does the current scan agree with some submap built long ago? If it does, a loop closure has been found.

The difficulty lies in the search space. A loop closure candidate may be at any position and any angle, and because drift has already accumulated the initial guess may be badly off—the search cannot be confined to a small neighborhood. Brute-force search over every possible position and angle would be far too expensive to run in real time.

This is the problem Cartographer's most ingenious technical contribution addresses, and the subject of the next section.

Practical Considerations in Bounding the Search

Even with an efficient algorithm, the system must control the amount of computation. To limit the number of constraints (and the computation), Cartographer considers only a subsampled set of all nearby nodes when building constraints, governed by a sampling ratio constant; sample too little and constraints may be missed and loop closure fail, sample too much and global SLAM slows and real-time loop closure becomes impossible. This is an important trade-off in tuning.

15.4 Loop Closure Detection and Global Optimization

This section is the technical core of the chapter: how Cartographer achieves real-

time loop closure—exactly the achievement its paper's title claims.

Branch and Bound: Pruning Impossible Branches

Faced with a vast search space, Cartographer uses branch and bound. It operates on grid maps at several resolutions with a branch-and-bound mechanism that efficiently rejects incorrect matches.

The central idea of branch and bound can be understood by an everyday analogy: suppose you must find a person in a whole building. The foolish method is to search every room; the clever one is to glance quickly over each floor first (low resolution), and if some floor obviously cannot hold anyone (the whole floor is under renovation, say), skip it entirely without searching room by room—that is pruning.

Cartographer does precisely this: it downsamples the existing submaps to several resolutions (a quarter and a half size, say) as inputs to scan matching, begins the match at low resolution with a coarse computation, and uses the best preliminary score to skip a great many impossible translation-rotation combinations at higher resolution, repeating until full resolution is reached. The system precomputes multi-resolution grids of sizes 1, 4, 16, and 64 to support this search.

Why This Achieves Real-Time Operation

The key is the notion of an upper bound. The score computed on a low-resolution map is an upper bound (never an underestimate) on all possible match scores for that region at high resolution. So if a region's score at low resolution is already below the best answer found so far, it can never do better at high resolution—the whole region can be skipped safely without being expanded.

This reduces the search from all possible combinations to a few promising

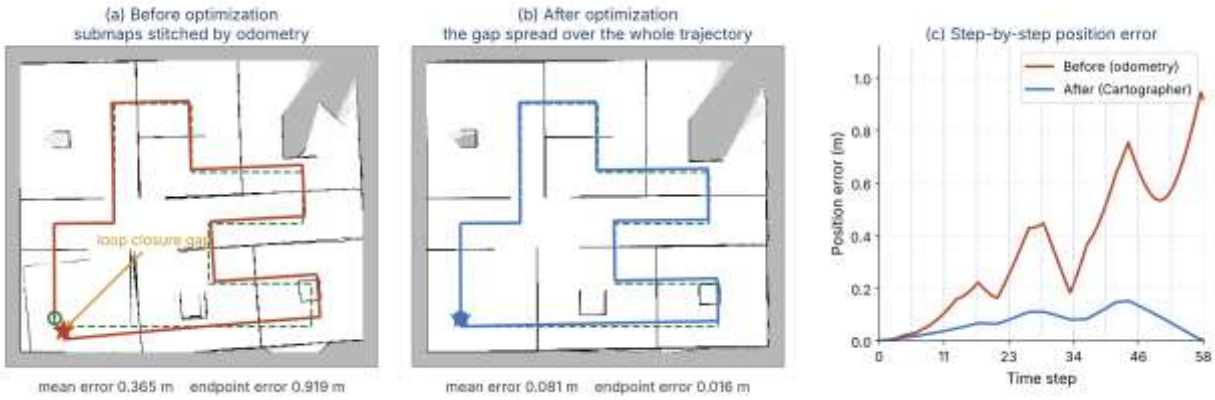
branches, and real-time loop closure becomes possible. The mechanism is explained in detail in the Cartographer paper, and its dedicated fast correlative scan matcher was designed precisely to make scan matching for real-time loop closure possible.

Global Optimization: Spreading the Contradiction

Once loop closure constraints are found, we enter the graph optimization of Chapter 12. Loop closure optimization, like scan matching, is formulated as a nonlinear least-squares problem, which lets the system add residuals easily to incorporate further data; every few seconds the system computes a solution with Ceres.

What this optimization adjusts are the submap poses and the scan poses, and it is called sparse pose adjustment. When the loop closure constraint and the odometry constraints contradict each other, the optimization spreads the contradiction evenly over the whole trajectory—the picture from Chapter 12 of a whole skewed map snapping straight is played out here for real.

Figure 15-3 Before and After Loop Closure: How One Constraint Straightens a Whole Map



This is the figure in the chapter most worth pausing over, because it turns "the most striking three seconds in class" of Case 15-1 into quantifiable numbers. Panel (a) is before optimization: the submaps are stitched together purely by odometry's dead

reckoning. The whole map is visibly skewed, the outer walls rotated bodily along a diagonal, and the end of the robot's circuit (the star) has not returned to the starting point (the circle), the two differing by 0.92 meters. The gold arrow points to that gap—not the sensor's fault but the result of dozens of tiny angular errors accumulating along the way.

Panel (b) is after one loop closure edge has been added and the pose graph solved afresh. Note the nature of the change: the end point is not forcibly dragged back to the start; rather, that 0.92 meters of contradiction is spread back over the whole trajectory according to the confidence of each constraint. The result is a map turned square with wall lines converging into single lines, the endpoint error falling to 0.016 meters and the mean error from 0.365 to 0.081—a reduction of nearly eighty percent. This is what "the covariance decides who yields" from Section 12.3 looks like on real data.

Panel (c) plots the step-by-step error of both trajectories together and is the most instructive panel of all. The red curve (before optimization) shows the classic shape of drift: nearly zero at first, rising gently in the middle, and shooting up at the end—error grows not linearly but as an angular deviation is amplified by the remaining distance. The blue curve (after optimization) is pressed into a gentle arch: the maximum appears in the middle of the trajectory (about 0.15 meters) while both ends, anchored by the start and the loop closure edge, approach zero.

This arch is worth remembering, for it is the fingerprint of pose graph optimization. When a trajectory has strong constraints at both ends and only odometry strung between, the error distribution must be low at the ends and high in the middle. It also tells us something practical: if the site permits two or more loops, there will be two or more loop closure edges and the arch will be pressed lower still. That is why the

mapping procedure of Section 15.5 insists on completing the whole loop, and it foreshadows the comparative experiment of the next section.

Loop Closure Straightening and the Map Suddenly Jumping

Why it jumps: global optimization runs every few seconds, and once a loop closure constraint is found the poses of the whole set of submaps are updated at once, so on screen the whole map rearranges instantly—the three seconds of Case 15-1.

Why navigation must be careful: the map and the map→odom transform change at the same moment, and a path being executed may become invalid instantly. The navigation stack of Part V must tolerate such a discontinuity, which is why mapping and navigation are commonly separated into two phases in practice.

How to confirm straightening succeeded: look at the seam of the loop. Before straightening the wall will ghost or be offset (the symptoms of Section 13.7); after, it should converge into a single line.

What to do if it does not straighten: usually the loop closure score threshold is too high or the sampling ratio too low (Table 15-1), so loosen the threshold and try again; but too loose a threshold invites false loop closures, so tune it together with the robust kernel of Section 15.4.

Robustification: Preventing the Catastrophe of a False Loop Closure

Loop closure detection can err (two rooms that look alike, say). One wrong loop closure constraint will drag the map forcibly to a wrong position with catastrophic consequences (mentioned in the Chapter 12 hands-on activity). Cartographer

therefore uses a robust kernel in the optimization to downweight outlying constraints, so that a few bad matches cannot ruin the whole map. This echoes the Chapter 12 extended challenge of adding a false loop closure edge—now you know how a practical system guards against it.

15.5 Implementing 2D Cartographer

This section covers the practical points of running 2D Cartographer mapping in ROS. Read alongside the Gmapping procedure of Section 14.5, the similarities and differences become clear.

Prerequisites

The same three requirements as Gmapping still apply: laser scans (/scan), odometry, and a complete TF tree (Chapter 10). But Cartographer differs in two respects:

- The IMU matters more: Cartographer can make effective use of the attitude information an IMU provides (Chapter 7), and in 3D mode it is all but required. In 2D mode, having an IMU improves mapping stability noticeably.
- Odometry is not strictly necessary: because its local scan matching is stronger, Cartographer can work without odometry in feature-rich environments. But where features are sparse, as in corridors (Section 13.6), odometry remains critical.

Configuration Files: Lua

A distinctive feature of Cartographer (and a barrier for beginners) is that it uses configuration files in the Lua language, with far more parameters than Gmapping. Most local SLAM options are found in the trajectory_builder_2d.lua and

trajectory_builder_3d.lua configuration files for 2D and 3D respectively.

The key parameter groups include:

Parameter Group	What It Controls	Direction of Tuning
Submap scan count	How many scans accumulate in each submap	Too few makes the submap unstable, too many lets drift into the submap
Resolution	The cell side length of the probability grid	0.05 m is common indoors (Section 13.5)
Local matching weights	The relative importance of each residual in scan matching	Raise the odometry weight where odometry is dependable
Loop closure sampling ratio	How many nodes are considered in building constraints	Too low misses loop closures, too high slows real-time operation (Section 15.3)
Loop closure score threshold	How high a match score is accepted as a loop closure	Too low misjudges easily, too high detects no loop closures
Optimization period	How often global optimization runs	Usually once every few seconds

Table 15-1 Principal Parameter Groups of 2D Cartographer

The Mapping Procedure

4. Confirm that sensor data and the TF tree are normal (as in Section 14.5).
5. Prepare the Lua configuration file, adjusting the sensor configuration to the platform (IMU or not, odometry or not, number of LiDARs).
6. Launch the Cartographer node and RViz, displaying the submaps, the trajectory, and the laser scan.

7. Teleoperate the robot slowly to map. Pay particular attention on completing the loop back to the starting point to whether a loop closure is triggered—the moment in this chapter most worth watching.
8. When mapping is complete, finish the trajectory and run the final optimization, then export the map file (in .pbstream format, convertible to the ROS standard .pgm and .yaml).

What to Watch: Submaps and Loop Closure

In RViz, Cartographer displays the individual submaps distinguishably, so you can clearly see that the map is stitched together from submaps. The finest moment is the instant a loop closure triggers: a trajectory and map that were offset suddenly jump into the correct position. Be sure to allow class time for students to witness this moment—it is the most powerful concretization of the abstract theory of Chapter 12.

15.6 3D Cartographer and Backpack Spatial Modeling

Cartographer does not do only 2D. It also supports 3D SLAM, and one of Google's original motivations for developing it was precisely the three-dimensional surveying of indoor spaces.

How 3D Mode Differs

- Sensors: a 3D LiDAR (multi-beam, Chapter 8) is required. Google's system originally carried two Hokuyo laser scanners and an inertial measurement unit, and later versions used two Velodyne PUCK scanners.
- The necessity of an IMU: in 3D mode the estimation of attitude (roll and pitch, Chapter 7) is crucial, and an IMU is all but required.

- Submaps become 3D: the probability grid goes from planar to volumetric voxels, and memory requirements rise sharply.
- Computational burden: 3D scan matching and optimization cost far more than in 2D, and demand more of the hardware.

Backpack Spatial Modeling

Case 8-3 of Chapter 8 mentioned backpack mobile scanning, and Cartographer is the representative of such systems. An operator carries a backpack fitted with LiDAR and an IMU and builds a three-dimensional model of a whole building in real time while walking.

The application matters because it solves the pain point of traditional surveying: a conventional static laser scanner must be set up at each station, scan, and be carried to the next—slow and laborious; a backpack system scans wherever it walks, raising efficiency tens of times over.

- Applications: building information model (BIM) creation and verification, digital preservation of historic sites and cultural assets, plant facility management, producing indoor navigation maps, and post-disaster structural assessment.
- What it means for university sites: this is a project direction of great potential—scan campus buildings, laboratories, or plants with 3D SLAM to produce a digital twin for teaching and research, putting the 3D point cloud techniques of Part V (Chapters 17 and 18) to practical use.

The Extension to Continuous-Time SLAM

It is worth mentioning that the academic community continues to improve on Cartographer's foundation. Some research groups have pointed out, for instance, that treating scans as a continuous-time trajectory rather than snapshots at discrete

instants further improves 3D mapping quality—work built precisely on a deep understanding of Cartographer's architecture. For students intending to do research, understanding this chapter is the ticket into the field.

15.7 Comparing Cartographer with Gmapping

The two protagonists of this part have now both been introduced. This section makes a systematic comparison, and it is the most important table in Part IV.

Item of Comparison	Gmapping (Chapter 14)	Cartographer (this chapter)
Theoretical basis	Particle filter (Chapter 11)	Graph optimization (Chapter 12)
Map organization	One complete map per particle	A series of submaps stitched together
Loop closure handling	No dedicated mechanism; hard to correct afterward	A core feature; can straighten the whole map in real time
Memory	Particle count × map size	Proportional to the size of the scene
Large scenes	Limited	Its strength; can map a whole building
3D support	None (2D only)	Supports 2D and 3D
Use of an IMU	Not used	Integrates effectively; all but required in 3D
Parameter complexity	Fewer; easy to start with	More (Lua configuration files); a higher barrier
Teaching value	The best example for understanding particle filters	The best example for understanding graph optimization and loop closure

Table 15-2 A System Comparison of Cartographer and Gmapping

Figure 15-4 Same Site, Same Route: The Two Systems Head to Head

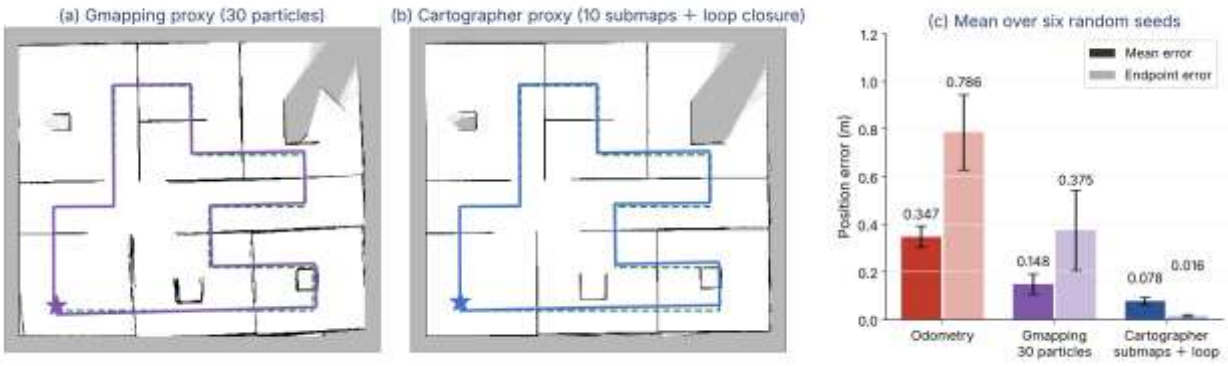


Table 15-2 lists the architectural differences between the two systems; this figure converts those differences into numbers. The experimental conditions are identical: the same 12 × 9 meter L-shaped indoor site, the same route once around and back to the start, and the same odometry noise parameters. Panel (a) is a Gmapping proxy from Chapter 14 (30 particles), panel (b) a Cartographer proxy from this chapter (10 submaps plus one loop closure edge), and panel (c) the statistics over six different random seeds.

Look at the maps first. Both are far better than the odometry-only result, but they differ at the seam: Gmapping's map still shows a slight misalignment where the loop joins, and the outer walls carry a slight overall tilt; Cartographer's map has straight walls and a cleanly closed loop. The reason is in the numbers of panel (c)—Gmapping's endpoint error averages 0.375 meters with a standard deviation of 0.168, while Cartographer's is 0.016 meters with a standard deviation of only 0.004. More than a twenty-fold difference, and the latter barely fluctuates across seeds.

The comparison of mean error is just as clear: odometry 0.347, Gmapping 0.148, Cartographer 0.078 meters—that is, the particle filter cuts drift by about sixty percent, and submaps plus loop closure cut nearly half again on top of that. But more notable

still is the length of the error bars. Gmapping's endpoint error has a standard deviation of 0.168 meters, close to half its own mean, which is exactly the randomness of a particle filter: with the same parameters it is sometimes lucky and sometimes exhausts its particles. Cartographer's error bars are almost too short to see.

There is a structural reason behind this difference worth thinking through: Gmapping is a filter and can only move forward—history already resampled away cannot be corrected, so once endpoint error has accumulated it must be carried along; Cartographer keeps the whole pose graph, and the moment a loop closure appears the poses of all past submaps can be readjusted. Whether history can be rewritten is the most fundamental divide between the filtering and optimization routes, and the common source of every entry in Table 15-2.

An Honest Note About This Comparative Experiment

This is a teaching proxy implementation, not a performance benchmark of the Gmapping and Cartographer source code. Both were rewritten for this book's simulated site to bring out the architectural difference, and the performance of the actual packages depends heavily on parameters, sensors, and site.

The result does not hold under all conditions. If the site permits no loop (a corridor that runs straight to a dead end, say), Cartographer has no loop closure to use and its advantage narrows sharply—which is exactly what the third discussion question asks you to consider.

Particle count and submap count both change the result. This figure fixes 30 particles and 10 submaps, and you may sweep these two parameters yourself in Exercise 15-1.

Rerun this on the real packages in Part Two of Exercise 15-1 and compare your

measured numbers with this figure. Different numbers are normal; what matters is whether the trend agrees.

How to Choose

- A small site, teaching, understanding particle filters: Gmapping. Few parameters and intuitive concepts make it a good entry point.
- A large site, loop closure needed, a whole building to map: Cartographer. Its architecture was built for this.
- 3D modeling and backpack scanning: Cartographer 3D.
- Modern 2D SLAM in a ROS 2 environment: besides Cartographer, consider slam_toolbox—likewise based on graph optimization, with simpler parameters, and quite popular in the ROS 2 community in recent years.

What the Two Chapters Teach

Reading Chapters 14 and 15 side by side, you will see that they are the embodiments of the two theoretical routes of Chapters 11 (filtering) and 12 (optimization). Gmapping shows the ingenuity and the ceiling of the particle filter; Cartographer shows how graph optimization breaks through that ceiling. This is not merely a comparison of two tools but a miniature of more than a decade of technical evolution in SLAM.

You now command the two major systems of 2D SLAM. In the next chapter we turn the lens to the application closest to daily life—the cleaning robot. From random bump-and-turn to SLAM-based planning, from room segmentation to coverage path planning, we see how these theories operate in your own living room, and understand the real challenges that furniture, mirrors, and low obstacles pose in a home.

15.8 After Cartographer: The New Landscape of 2D SLAM, 2024–2026

Cartographer's paper appeared in 2016, which makes it, as this book is written, a system in its tenth year. Ten years is enough for a change of dynasty in robotics, yet Cartographer's core design—submaps, branch and bound, sparse pose adjustment—remains the standard answer in 2D LiDAR SLAM, and nearly every successor system makes local substitutions on that skeleton. This section surveys four main developments of the past two years so that you know where the frontier lies once this chapter is done.

1. A Shift in Maintenance: slam_toolbox Becomes the Practical Default in ROS 2

The largest change is not in the algorithm but in the ecosystem. As ROS 2 became mainstream, slam_toolbox, developed by Steve Macenski, gradually displaced Cartographer as the default 2D SLAM of most ROS 2 projects. It likewise takes the optimization route and likewise builds a pose graph by scan matching, but its parameters are far simpler than Cartographer's Lua configuration, and it has built in two capabilities in which Cartographer is weaker: map serialization and continued mapping (saving a completed pose graph and resuming from it at the next start-up), and a pure localization mode (loading an existing map to localize only, without updating it).

The implication for teaching cuts both ways. On one hand, if your goal is to get a project running on ROS 2, slam_toolbox is usually the path of least resistance; on the other, Cartographer remains the best teaching material for understanding why the design is as it is—its paper gives the clearest derivation of the branch-and-bound upper bound,

and many of slam_toolbox's design choices in fact presume a reader who already knows what Cartographer is doing. This book's advice: learn the principles with Cartographer, do the project with slam_toolbox.

2. Lifelong and Multi-Session Mapping: A Map Is No Longer a One-Off

The second development is an expansion of the problem statement itself. Traditional SLAM assumes one run, one map, and done; but a real site changes: furniture is moved, shelving rearranged, the season alters the lighting. Lifelong mapping addresses exactly this—how a robot walking the same site repeatedly can fold new observations into the old map while deleting obstacles that no longer exist, rather than letting the map grow ever dirtier. The serialization and resumption capabilities of slam_toolbox were designed with this in view.

Technically this turns the pose graph from a single trajectory into a collection of trajectories from several sessions, and loop closures occur not only within one run but between runs on different days (called multi-session SLAM). This line leads directly to the multi-robot collaboration of Part VI: several robots mapping at once and then merging their pose graphs is, in essence, the spatial version of multi-session SLAM.

3. Jointly Optimizing Pose and Map: The Occupancy-SLAM Line

The third is a theoretical challenge aimed directly at Cartographer's architecture. Cartographer's practice is to compute the pose by scan matching first and then insert the scan into the map—the pose is a variable and the map a by-product of it. Occupancy-SLAM, proposed by a team at the University of Technology Sydney (published at Robotics: Science and Systems 2024, with an improved version in 2025), argues that this division of labor is itself the bottleneck on accuracy: since the quality

of the map depends on the pose, correction of the pose should likewise be driven back by the consistency of the map, and so robot pose and the value of every occupancy cell should be treated as variables of one optimization problem and solved together.

The idea is mathematically natural; the difficulty is scale: a 2D occupancy grid easily runs to hundreds of thousands of cells, and treating each as a variable makes the problem several orders of magnitude larger than a traditional pose graph. The team's contribution is precisely in finding the sparse structure of this large problem so that it converges in acceptable time. If you have read the discussion of sparsity in Chapter 12 (Figure 12-3), you will recognize this as the same idea enlarged—sparsity is always the watershed of whether a large optimization can be practical.

4. Learning-Based Loop Closure Detection: From Geometric Matching to Scene Descriptors

The last development is in loop closure detection. Cartographer's branch and bound is a purely geometric method: it searches exhaustively over multi-resolution grids and accelerates by pruning with an upper bound. Its advantages are that it requires no training, depends on no dataset, and behaves entirely predictably; its drawback is that once the search range widens (the robot is picked up and set down on another floor, say—the kidnapping problem of Chapter 11) the computation still explodes.

The mainstream alternative in recent years is the scene descriptor: compress a scan into a fixed-length vector fingerprint, so that loop closure detection becomes a nearest-neighbor search in a fingerprint database, independent of how far the robot has traveled. The Scan Context family represents this route in 3D, and work in 2024 and 2025 extends it to loop closure from point cloud density images and to surveys that treat place recognition systematically as a problem in its own right. In 2D, limited by

how little information a single-beam scan carries, descriptors are less discriminative than in 3D, so branch and bound has not been displaced there—which also explains why the technical core of this chapter still repays close reading today.

Four Threads, One Common Theme

Engineering (slam_toolbox): packaging Cartographer's principles more usably, adding serialization and pure localization.

Problem statement (lifelong mapping): the map turns from a one-off product into an asset requiring long-term maintenance.

Mathematics (Occupancy-SLAM): drawing the map itself into the optimization variables, challenging the division of pose first, map second.

Perception (scene descriptors): replacing geometric exhaustive search in loop closure detection with feature retrieval, to handle large areas and the kidnapping problem.

The common theme: Cartographer's skeleton (submaps + pose graph + loop closure) has not been overturned; what is replaced is each component on that skeleton. Learn the skeleton and every later replacement of a part becomes readable.

Looking Back and Forward

This chapter has turned the graph optimization of Chapter 12 from a set of equations into a system that can be run down a corridor. Looking back at the arrangement of Part IV: Chapter 13 taught how to turn scans into a map, Chapter 14 untied the chicken-and-egg problem with a particle filter, and Chapter 15 demonstrated the other road—keep the history and allow it to be corrected. The numbers of Figure 15-4 show this

road's advantage at a site with a loop, and Section 15.8 states its boundary honestly: with no loop, loop closure has nothing to do.

From Chapter 16 the lens turns from what an algorithm looks like to how these algorithms survive inside a real product. The cleaning robot is the best first case: cheap, mass-produced, running every day in tens of millions of homes, and yet obliged to do everything this chapter discussed on an extremely small computing budget.

Formulations and Procedures

1. The Optimization of Local Scan Matching

When inserting a new scan into a submap, seek a pose ξ that makes the scan points fall as far as possible in places of high occupancy probability in the submap:

$$\xi^* = \operatorname{argmin}_{\xi} \sum_k \left(1 - M(T_\xi \cdot h_k)\right)^2$$

where h_k is the k-th point of the scan, T_ξ the pose transform carrying it into submap coordinates (Chapter 10), and M(·) the occupancy probability of that position in the probability grid. The expression means: make every scan point land where the map believes there is an obstacle. This is a nonlinear least-squares problem (Chapter 12), solved with Ceres.

2. Global Optimization (Sparse Pose Adjustment)

Global SLAM optimizes the submap poses Ξ^m and the scan poses Ξ^s to minimize the weighted residual of all constraints:

$$\operatorname{argmin} \tfrac{1}{2} \sum_{ij} \rho\left(E^2(\xi_i^m, \xi_j^s; \Sigma_{ij}, \xi_{ij})\right)$$

where E is the residual (the difference between the actual and the measured relation of the j-th scan to the i-th submap), Σ_{ij} the covariance of that constraint (its

confidence, Section 12.3), and ρ a robust kernel—which keeps a misjudged loop closure constraint from ruining the whole map (Section 15.4). This expression has exactly the form of the least-squares objective of Chapter 12, with the nodes replaced by submaps and scans.

3. Branch-and-Bound Search

The search space of global scan matching is defined as a three-dimensional window (translation x, translation y, rotation θ):

$$W = \{ -w_x, \ldots, w_x \} \times \{ -w_y, \ldots, w_y \} \times \{ -w_\theta, \ldots, w_\theta \}$$

Branch and bound proceeds by repeating branch—bound—prune:

Figure 15-5 The Branch-and-Bound Loop Closure Search

① Precompute multi-resolution grids (sizes 1, 4, 16, 64)

↓

② Starting at the lowest resolution, cut the search space into several large blocks (branch)

↓

③ For each block, compute an upper bound on the score on the low-resolution grid (bound)

↓

④ If a block's upper bound < the best score so far → prune the whole block and do not expand it (prune)

↓

⑤ Otherwise subdivide the block and repeat ②–④ at higher resolution

↓

⑥ On reaching full resolution with a score above the threshold → accept as a loop closure constraint

4. The Complete Data Flow of Cartographer

Figure 15-6 Cartographer End to End

Sensors: laser scans + IMU + (optionally) odometry

↓

[Local SLAM] scan matching → insert into the current submap → seal when full and open a new one

↓ (producing a local trajectory, which drifts)

[Global SLAM, a background thread]

Sample nodes → branch-and-bound matching against existing submaps → find loop closure constraints

↓

Form a pose graph (submap poses + scan poses)

↓

Every few seconds, run sparse pose adjustment with Ceres (including a robust kernel)

↓

Output: a globally consistent map and trajectory (the whole map straightened at a loop closure)

Case Studies

Case 15-1 The Moment of Loop Closure: The Most Striking Three Seconds in Class

Case 12-1 of Chapter 12 described the picture of a loop closure straightening a map, and Cartographer makes that picture an experiment reproducible in class.

The typical teaching scene runs like this: students teleoperate the robot once around the department building. All goes well for the first half and the map is neat; by the second half the wall lines begin to diverge from what was built at the start, obvious ghosting appears in the corridor, and puzzlement spreads across the students' faces. Then, the instant the robot returns near its starting point—the background thread finds the loop closure constraint and global optimization runs—the whole map reassembles on the screen in a flash, the ghosts merge into one, the offset corridor joins up, and the trajectory forms a beautifully closed loop.

Those three seconds condense the graph optimization of Chapter 12 and the branch and bound and sparse pose adjustment of Section 15.4 into a single visible instant. Teaching experience shows that students who have seen this with their own eyes understand why graph optimization is used far better than any lecture can convey. Instructors are advised to arrange enough space for the robot to complete a full loop—the class time in this chapter most worth investing.

Case 15-2 Why Google Built Cartographer: Ambitions for Indoor Maps

Google did not develop Cartographer out of purely academic interest. By then Google had mapped the world's roads with Street View cars, but indoor spaces—malls, airports, stations, museums—remained blanks on the map. Traditional surveying was too slow and too expensive for indoor mapping, and a technology that could map while walking was required.

This explains several of Cartographer's design orientations: why it must be real time (the operator needs to know at once what has not yet been scanned), why it must handle large scenes (a whole mall), why a backpack system was built (anywhere a

person can walk must be scannable), and why loop closure matters so much (walking a large area must accumulate drift).

This case offers an important lesson: a technology's design orientation is often decided by application requirements. Gmapping came out of an academic laboratory and pursued algorithmic elegance; Cartographer came out of industrial need and pursued practicality at large sites. Neither is better—only more or less suitable, which confirms once again the idea of Chapter 8 that sensor selection is a combined judgment.

Case 15-3 Experiments in Sensor Configuration: What Configuration Suits What Site

One research group tested systematically how different sensor combinations affect Cartographer's performance, with instructive results: in a cluttered room, three 2D LiDARs with an IMU and odometry gave the lowest error; in a long corridor, a single 3D Velodyne LiDAR with an IMU performed best.

The result agrees entirely with earlier discussions in this book. In a cluttered room, several 2D LiDARs scanning from different angles cover more geometric features and reduce occlusion (the height blind spot of Chapter 8); the difficulty of a long corridor is a lack of longitudinal constraint (Section 13.6), and a 3D LiDAR can see the structural variation of ceiling and floor, filling exactly that gap.

The lesson for university projects and undergraduate practicals is: do not chase the most expensive sensor blindly, but choose the configuration by the character of the site. This also makes an excellent project topic—measure the mapping quality of different sensor combinations at various campus sites (classroom, corridor, atrium) and

compare quantitatively by the methods of Section 13.7. Such a study has both teaching value and practical significance.

Hands-On Activities

Exercise 15-1 Cartographer Mapping and Watching the Loop Closure

Purpose: to run 2D Cartographer mapping, watch submaps form and the instant a loop closure straightens the map, and run a comparative experiment against the Gmapping of Chapter 14. This is the most important exercise in Part IV.

Part One: Mapping in Simulation and Watching the Loop Closure (the whole class)

1. Install and launch Cartographer's 2D configuration in the TurtleBot3 Gazebo simulation environment (Chapter 28).
2. Display the submaps and the trajectory in RViz, move within a small area first, and observe how submaps form one by one and partly overlap.
3. Teleoperate the robot once around a complete loop of the simulated world, watching throughout: the map is neat in the first half, and misalignment begins to appear in the second.
4. The key moment: on returning near the starting point, watch for whether the loop closure triggers and the map is straightened. Record this instant and replay it for discussion in class.
5. If no loop closure triggers, try adjusting the loop closure score threshold and the sampling ratio (Table 15-1) and run again.

Part Two: The Comparative Experiment—Cartographer versus Gmapping (the core)

6. In the same simulated world and along the same route, map once with Gmapping (Chapter 14) and once with Cartographer.
7. Assess both maps by the methods of Section 13.7: are the walls straight? Does the loop close? Is there ghosting? Measure key dimensions on the map against the simulation settings.
8. Place the two maps side by side, mark the regions where they differ most, and explain why (hint: Table 15-2).
9. Observe resource use: compare the CPU and memory consumption of the two against the discussion of formulation 5 in Chapter 14 and Section 15.2.

Part Three: Mapping on a Physical Platform (for classes with hardware)

10. Prepare the Lua configuration file and adjust it to your platform (IMU or not, odometry or not, LiDAR model and mounting position).
11. Map the classroom first to verify the configuration, then take on a larger site—ideally a corridor or a floor of the department building that permits a complete loop.
12. Walk slowly and be sure to complete the whole loop back to the starting point, watching for the loop closure.
13. Export the map and assess its quality, comparing with the Gmapping result at the same site.

Discussion Questions

1. Did you observe the instant the loop closure straightened the map? Which areas of the map changed most before and after, and why those?
2. In the comparative experiment, in what respects did the Cartographer and Gmapping maps differ most? Which entries of Table 15-2 does this confirm?

3. If the site has no loop (merely a corridor that comes to a dead end, say), can Cartographer's loop closure mechanism still do anything? Does it still have an advantage over Gmapping then?
4. (Undergraduate) Section 15.3 noted that too low a loop closure sampling ratio misses loop closures while too high a ratio slows real-time operation. Adjust this parameter in practice, record how loop closure detection rate and system latency change, and find a suitable value for your site.

Extended Challenge (Undergraduate)

First: design a false loop closure experiment—build two rooms of nearly identical appearance in the simulated world and observe whether Cartographer misjudges a loop closure; if it does, observe whether the robust kernel lessens the severity. This will deepen your understanding of the robustification of Section 15.4. Second: if your school has a 3D LiDAR, try Cartographer's 3D mode by scanning a laboratory or a flight of stairs to produce a three-dimensional point cloud model, experiencing the idea of backpack modeling (Section 15.6) and preparing for the 3D SLAM of Part V (Chapters 17 and 18).

Further Reading

1. W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-Time Loop Closure in 2D LIDAR SLAM," ICRA, 2016. (The original Cartographer paper and the principal basis of this chapter.)
2. The official Cartographer documentation (google-cartographer.readthedocs.io) and the Cartographer ROS documentation. (Includes the "Algorithm walkthrough for tuning" guide; essential for implementation.)

3. Google Open Source Blog, the "Cartographer" release announcement. (Explains the development motivation and the background of indoor surveying.)
4. J. Elseberg and A. Nüchter, "Improving Google's Cartographer 3D Mapping by Continuous-Time SLAM," ISPRS Archives, 2017. (Research improving the 3D mode, with measurements from a backpack system.)
5. The slam_toolbox open-source project (GitHub). (Another graph-optimization 2D SLAM in ROS 2, useful for comparison with Cartographer.)
6. The official Ceres Solver documentation (ceres-solver.org). (The optimization library Cartographer uses, echoing Chapter 12.)
7. Cyrill Stachniss, the Graph-Based SLAM course videos (YouTube). (The theoretical basis of graph optimization SLAM; good read alongside this chapter.)

Summary

1. Cartographer was developed by Google and open-sourced in 2016, its core feature being real-time loop closure; it divides into two concurrently running subsystems: local SLAM (the front end, building submaps that are locally consistent but drift) and global SLAM (the back end, seeking loop closure constraints in the background and optimizing the pose graph).
2. A submap is a small probability grid map made of a few dozen scans; the three reasons for submaps are bounding the accumulation of error, providing a unit for optimization (the node count falls sharply, so real-time operation is possible), and serving as a stable object for loop closure comparison. Memory is proportional to the size of the scene, so a whole building can be handled.

3. The system has two kinds of scan matching: local matching inserts a new scan into the current submap (solving a nonlinear least-squares problem with Ceres); global matching searches for loop closures, is far harder, and must limit the number of sampled nodes to balance loop closure success against real-time operation.
4. Branch and bound is the key to real-time loop closure: on precomputed multi-resolution grids it computes an upper bound on the score at low resolution and prunes whole regions that cannot win, cutting the search sharply; once constraints are found, sparse pose adjustment straightens the whole map, and a robust kernel prevents a false loop closure from causing catastrophe.
5. A 2D implementation requires laser, TF, and (preferably) an IMU; Cartographer is configured by Lua files with a good many parameters, the key ones being the submap scan count, the resolution, the loop closure sampling ratio, and the score threshold; be sure to complete the whole loop to observe loop closure.
6. 3D mode requires a 3D LiDAR and an IMU, with voxel-grid submaps and a heavy computational load; backpack spatial modeling lets an operator build a three-dimensional model of a whole building while walking, applied to BIM, heritage preservation, and facility management—a project direction of great potential at a university site.
7. Compared with Gmapping: Cartographer is based on graph optimization rather than a particle filter, on submaps rather than one map per particle, with loop closure as a core feature, memory proportional to the scene, and support for 3D and the IMU, though with more complex parameters. The two are the embodiments of the two theoretical routes of Chapters 11 and 12.

The two major technical protagonists of Part IV are now complete. In the next chapter we take these techniques back to the setting closest to daily life—the cleaning robot. From random bump-and-turn to SLAM-based planned cleaning, from room segmentation and keep-out zones to coverage path planning, and facing honestly the real challenges of a home: chair and table legs, mirrors, low obstacles, and pets. You will find that every theory of this part is quietly at work in your own living room.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 16

SLAM in the Cleaning Robot

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

This chapter closes Part IV and is the first time in the book that the technology accumulated over fifteen chapters is placed as a set inside a product you can buy and hold. The previous three chapters were about the algorithms themselves—how an occupancy grid is built, how particles are filtered, how a pose graph is solved; this chapter asks another question: what happens when those algorithms must run on a machine costing twenty thousand NT dollars, with a computing chip no better than a phone's, working alone for eight hours a day in an unfamiliar home? The answer is usually not that the algorithm is not good enough but that the real world is far more complicated than the corridor in a paper.

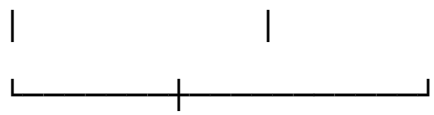
So this chapter should be read differently from the previous three. Those built tools from the bottom up; this one dissects a product from the top down: which tools it uses, where it compromises, and how it makes up for them. Having read it, look back at the angular resolution of Chapter 8, the map symptoms of Chapter 13, and the slip problem of Chapter 14, and you will find that every limitation that seemed abstract at the time corresponds to some concrete behavior of the cleaning robot in your home.

Figure 16-0 The Connection Map of Chapter 16

The first three parts: sensing and mathematical tools

Chapter 3 Navigation architecture Chapter 8 2D LiDAR Chapter 11 Probabilistic

estimation



▼

Part IV: Chapter 13 Occupancy grid → Chapter 14 Gmapping

→ Chapter 15 Cartographer

▼

[Chapter 16 The cleaning robot: where technology meets the ground]

Coverage path planning · room segmentation · keep-out zones · sensor fusion

|

▼

Part V 3D SLAM Part VIII Navigation planning Part IX Projects

Learning Objectives

The theory of Part IV comes at last back to the setting closest to daily life—your own living room. The cleaning robot is the most successful popularization of SLAM technology, and nearly every idea of the preceding fifteen chapters has its counterpart in this small disc of a machine. This chapter closes Part IV and is a lesson in bringing theory home. After completing this chapter, you should be able to:

1. Explain why a cleaning robot needs SLAM, and what the consequences are without it.
2. Compare random bump-and-turn cleaning with SLAM-based planned cleaning, and understand the quantitative meaning of coverage and efficiency.
3. Explain how a LiDAR cleaning robot builds a map of a home, and connect it to the techniques of Chapters 13 through 15.

4. Explain the principles of room segmentation, keep-out zones, and coverage path planning.
5. List the sensor combination of a cleaning robot and describe the role of each and how they complement one another.
6. Analyze the real challenges of a home: chair and table legs, mirrors, low obstacles, and a dynamic environment.
7. Understand the basic strategies and evaluation metrics of coverage path planning from a cleaning route.

Core Concepts

Chinese Term	English Term	Explanation
覆蓋路徑規劃	Coverage Path Planning	Planning a path that visits every traversable position in a region.
弓字形路徑	Boustrophedon Path	A back-and-forth cleaning route like plowing a field; the basic form of coverage planning.
隨機碰撞式	Random Bounce	The strategy of early cleaning robots: turn on hitting an obstacle, with no map and no planning.
覆蓋率	Coverage Rate	The proportion of the cleanable area actually cleaned.
重複率	Redundancy Rate	The proportion of positions cleaned more than once; the lower the more efficient.
房間分割	Room Segmentation	Automatically dividing a whole map into meaningful room blocks.
禁區	No-go Zone	An area the user marks on the map that the robot may not enter.

Chinese Term	English Term	Explanation
虛擬牆	Virtual Wall	A virtual boundary on the map, functioning much like a keep-out zone.
懸崖感測器	Cliff Sensor	An infrared sensor detecting the floor ahead suddenly ending (stairs).
3D 飛時測距	3D ToF	A sensor measuring the shape of obstacles ahead three-dimensionally.
結構光	Structured Light	Projects a light pattern to read the shape and distance of obstacles.
多樓層地圖	Multi-floor Map	Remembers the maps of several floors and recognizes the current floor automatically.

16.1 Why a Cleaning Robot Needs SLAM

The question seems simple at first: is it not just a matter of sweeping the floor clean? But think it through, and sweeping the whole floor of a home once without repeating too often requires answering the three navigation questions of Chapter 3: where am I? Where am I going? How do I get there?

Three Difficulties Without SLAM

- Not knowing what has been cleaned: without a map and localization the robot cannot record that it has already covered a patch. The result is that some places are cleaned five times and others not at all.
- Not knowing when it is finished: without a map there is no knowing how large the home is or how much remains. Early models could rely only on a timer—thirty minutes and knock off, regardless of whether anything was finished.
- Not knowing the way home: when the battery runs low it must return to the dock, and without a map it can only wander and hope, often stranding itself halfway.

Four Capabilities SLAM Brings

1. Systematic cleaning: only with a map can a boustrophedon route be planned to cover zone by zone, raising coverage sharply and lowering repetition (Section 16.2).
2. Resuming after a break: when the battery runs low it docks, and after charging it can return to where it left off—which requires accurate localization and a remembered map.

3. User interaction: showing the room map in an app lets the user say "clean the kitchen only," set keep-out zones, and point to where it should clean. This is the product value a map brings.
4. Consistency across cleanings: a second cleaning need not explore afresh and can simply load the existing map and localize (the AMCL of Case 14-3 in Chapter 14).

Threading the Earlier Chapters Together

The cleaning robot is almost a physical review of the first fifteen chapters of this book: it senses the environment with the 2D LiDAR of Chapter 8, records space in the occupancy grid map of Chapter 13, localizes and maps at once with the SLAM algorithms of Chapter 14 or 15, integrates sensors with the coordinate transforms of Chapter 10, and relocalizes with the particle filter of Chapter 11 (the kidnapping problem of Chapter 3). To understand the cleaning robot is to understand the whole of Part IV.

16.2 Random Bump-and-Turn Cleaning versus SLAM-Based Planned Cleaning

The evolution of the cleaning robot is a miniature of the move from reactive to deliberative (the architectural discussion of Chapter 3).

Random Bump-and-Turn: Reactive Architecture Taken to the Limit

When iRobot launched the first-generation Roomba in 2002, it used a random bump-and-turn strategy: drive straight, turn through a random angle on hitting something, and keep going, alternating with a few simple behaviors such as spiraling and wall-following.

The inspiration for this design in fact comes from the behavior-based architecture mentioned in Chapter 3—no model, no planning, perception driving behavior directly. Its advantages are that it is extremely cheap, the program is simple, and it cannot fail because of a wrong map; its drawbacks are equally plain: a disorderly route, uneven coverage, and a very long time.

Interestingly, the strategy is not mathematically baseless: given long enough, a random walk will in theory cover any connected region. The trouble is that long enough may mean an hour—and there is never any guarantee that cleaning has actually finished.

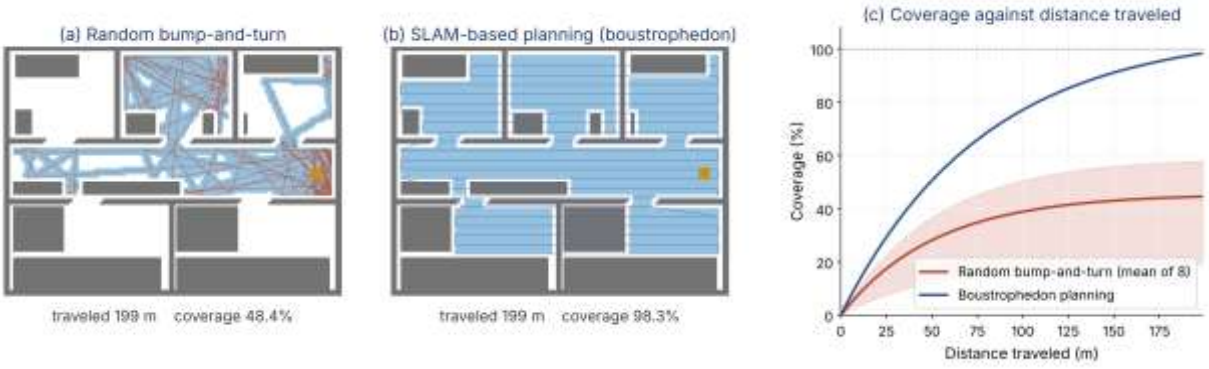
SLAM-Based Planning: The Victory of Deliberative Architecture

In 2010 Neato first fitted a 2D LiDAR to a cleaning robot, letting it build an occupancy grid map and plan a systematic route. Modern models are no longer merely rotating brushes steered by random bumps but run on-device AI to build maps, avoid obstacles, and adapt to real domestic life.

The core of planned cleaning is the boustrophedon path (the word means "as the ox turns in plowing"): the robot cleans in a straight line to the end in one direction, turns 180 degrees, moves over by one brush width, and sweeps back the other way, repeating to cover the whole area.

Figure 16-1 Same Apartment, Same 199 Meters: The Results of Two Cleaning

Strategies



This figure turns the verbal differences of Table 16-1 into measurable numbers through one comparative experiment. The scene is a three-bedroom apartment of 9.0 × 6.6 meters with a master bedroom, a second bedroom, a study, a living room, a kitchen, and a hallway, furnished with a bed, a sofa, a desk, and a counter; subtracting the wall and furniture margins the robot's radius cannot enter, the cleanable area is 27.5 square meters. Both strategies start from the dock on the right (the gold square) and both travel 199 meters—that is, battery consumption is the same.

Panel (a) is random bump-and-turn: drive straight and turn through a random angle on contact. The path shows the classic tangle, and the result is only 48.4% covered. More notable is where it covered—the master bedroom, the hallway, and the kitchen are clearly covered while the second bedroom, the study, and the living room are nearly blank. The reason is concrete: the doorways are only 85 centimeters wide, leaving a passable gap under 60 centimeters after the robot's radius, and the probability that a random heading passes exactly through that gap is low. A random strategy does not "get there eventually"; it gets trapped in the first few rooms it happens to enter.

Panel (b) is SLAM-based planning: the map is first used to cut the space into rooms (Figure 16-2 in the next section), then a boustrophedon path is run in each room with the shortest connections between them. The same 199 meters gives 98.3% coverage.

The neat rows of back-and-forth lines on the path plot are the origin of the name boustrophedon; the unswept white margin around the furniture is the physical limit imposed by the robot's radius—which also explains why 100% coverage is hard to reach in practice.

Panel (c) plots coverage against distance traveled and is the most informative panel in the figure. The solid red line is the mean of eight random trials, and the pale red band the range from best to worst. The two curves nearly coincide over the first 25 meters—reasonably enough, since at the start there are unswept places everywhere and even wandering pays. But from about 40 meters they diverge: the boustrophedon keeps climbing linearly toward 100% while the random strategy quickly flattens, with final coverage across the eight trials falling between 19.5% and 59.0%.

The width of that pale red band is more worth discussing than the mean itself. It represents the fact that with the same machine, the same program, and the same time, today's result and tomorrow's may differ threefold. For an appliance used every day, unpredictability is more fatal than low efficiency—the user cannot tell whether cleaning has finished. This is the quantitative version of the difficulty of "not knowing when it is finished" in Section 16.1, and the greatest value SLAM brings to the cleaning robot: not merely cleaning better but cleaning predictably.

A Note About This Simulation

This is a simplified simulation for teaching: the robot is treated as a circle 26 centimeters in diameter with an effective brush width of 26 centimeters, excluding the extra coverage of a side brush.

The random strategy is implemented as drive straight, turn through a random

angle on contact. A real Roomba also interleaves spiral and wall-following modes and would achieve somewhat higher coverage than this figure, but the structural problem of being trapped in a multi-room home remains the same.

The boustrophedon plan traveled 199 meters, and the theoretical shortest covering distance is 27.5 ÷ 0.26 ≈ 105.6 meters, so the repetition ratio is about 1.88. That figure is middling by this chapter's metrics—turns, detours around furniture, and travel between rooms are all included.

You can reproduce this figure yourself in Part Three of Exercise 16-1 and try to press the repetition ratio below 1.5.

Item of Comparison	Random bump-and-turn	SLAM-based planning
Builds a map?	No	Yes (an occupancy grid map)
Cleaning route	Random and disorderly	Boustrophedon and systematic
Coverage	Not guaranteed; depends on time	High; can approach complete coverage
Repetition ratio	High	Low
Time required	Long	Short
Can resume after a break?	No	Yes
Cost	Very low	Higher (a LiDAR or camera is needed)
Tolerance of map error	Unaffected	A wrong map means wrong cleaning

Table 16-1 A Comparison of the Two Cleaning Strategies

Note the last row: planning is not without drawbacks. If SLAM mapping fails (the map symptoms of Section 13.7), the robot cleans according to a wrong map and may miss a whole room instead. This echoes the conclusion of Chapter 13: map quality directly decides whether the system succeeds.

16.3 The LiDAR Cleaning Robot and 2D LiDAR Mapping

Case 1-1 of Chapter 1 noted that in 2017 Xiaomi launched a LiDAR cleaning robot at an affordable price, bringing SLAM technology into ordinary homes in quantity for the first time. This section looks closely at how it works.

Hardware Configuration

A typical LiDAR cleaning robot has a raised round turret on top containing a rotating 2D LiDAR (Chapter 8), scanning at about the height of the body's top (8 to 10 centimeters above the floor). It rotates several times a second and measures ranges in hundreds of directions around it.

Cost control is worth noting: the LiDAR in a domestic cleaning robot mostly uses low-cost triangulation ranging, with a range of about 6 to 8 meters and accuracy of a few centimeters—far short of industrial models but ample for indoor mapping. This is an instance of the Chapter 8 principle that resolution must match the application requirement.

The Mapping Procedure: Chapters 13 through 15 in Practice

5. Entering an unfamiliar environment for the first time: the robot runs SLAM, building an occupancy grid map and estimating its own pose as it moves

(Chapter 13). Most models first follow the walls around once to establish the outline of the home, then deal with the interior.

6. Choice of algorithm: early models mostly used particle filter methods (Chapter 14); in recent years, as computing chips have improved, more and more use graph optimization with loop closure detection (Chapter 15), making maps of larger homes more consistent.
7. Map storage: the completed map is stored in the machine and in the cloud, and loaded directly for later cleanings.
8. Later cleanings: after loading the map it localizes (AMCL-type methods, Chapter 11) without remapping, and can plan routes directly.

A Recent Trend: LiDAR and Vision Together

Modern premium models no longer rely on LiDAR alone. The industry's practice is for the laser LiDAR to build the map and track the robot's position while an AI camera handles the visual SLAM side—recognizing objects on the floor and deciding what to do about them. This is the commercial practice of "LiDAR gives geometry, the camera gives semantics" from Chapter 9.

This combination answers directly to the complementarity analysis of Table 8-2 in Chapter 8: LiDAR does not fear the dark and is geometrically precise but does not know what a thing is; a camera can recognize objects but fails in dim light and on untextured surfaces. Some manufacturers therefore add an LED beside the camera to preserve vision in dark or low spaces.

16.4 Room Segmentation, Keep-Out Zones, and Coverage

Having a map is only the first step. Three further functions are needed to make the

map genuinely useful, and together they constitute what the user perceives as intelligence.

1. Room Segmentation

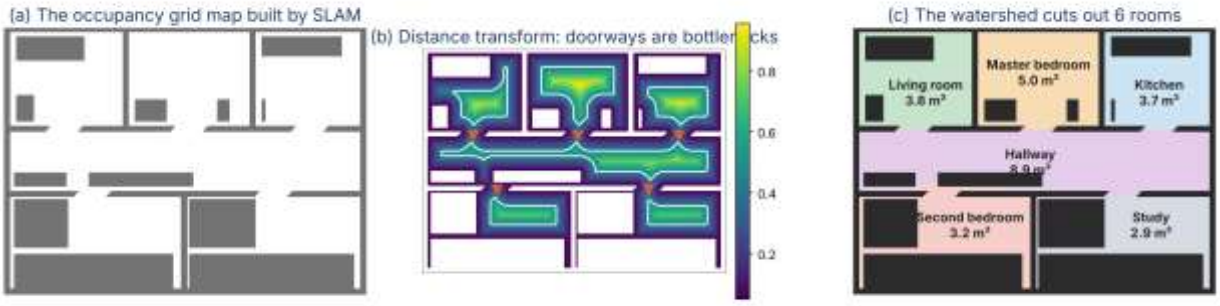
Room segmentation automatically divides a whole occupancy grid map into meaningful blocks such as living room, bedroom, and kitchen. This is crucial to the user experience—without segmentation the user can only say "clean everything"; with it, the user can say "clean the kitchen only."

The basic idea of segmentation is to find the bottlenecks of the space: at doorways and narrow passages the width of free space narrows abruptly, and these positions are natural boundaries. Common methods include using a distance transform to find the core region of each room and then expanding outward, or detecting the geometric features of door frames.

Having built a map, modern models can label rooms, exclude keep-out zones, and even remember which areas need cleaning more often. This already touches on the semantic map idea mentioned in Chapter 3—the robot knows not only that a place is traversable but that it is the kitchen. Chapter 32 discusses semantic labeling with AI in full.

Figure 16-2 Room Segmentation: Finding the Doorway Bottleneck with a Distance

Transform



Room segmentation sounds like something requiring artificial intelligence—after all, which room is the bedroom is a semantic question. But this figure shows that two steps of purely geometric computation do the job rather well. Panel (a) is the occupancy grid map built by SLAM, at this point merely a collection of black and white cells with no notion of a room.

Panel (b) is the key step: the distance transform. It computes for every free cell how far it is from the nearest wall, the brighter the color the more open. One pattern is immediately visible—the center of a room is bright (far from walls) while a doorway is a dark narrow slot (walls on both sides, near to both). The five red triangles mark this apartment's doorways, and all of them appear as bottlenecks in the distance transform. The white contour marks the threshold of 0.45 meters: the doorway values are about 0.42 meters and are just cut off, so the cores of the rooms separate naturally into distinct blocks.

Panel (c) is the second step: using those cores as seeds for a watershed expansion, letting each core grow back along the gradient of the distance transform until they meet at the doorways. The result is six blocks corresponding exactly to the master bedroom, second bedroom, study, hallway, living room, and kitchen, with their areas computed as well. Not a scrap of machine learning was used in the process—only the

distance transform and the watershed, each one line of code in OpenCV.

The product significance of this result is direct: with segmentation the app can display "living room, 3.8 square meters" and let the user tap "clean the kitchen only"; with segmentation, coverage planning can proceed room by room (which is exactly what Figure 16-1(b) does) rather than cutting long strips across the whole plane. But it must be said honestly that a purely geometric method has its limits: where an open kitchen flows into the living room there is no door and thus no bottleneck to find, and such layouts often get merged into one block or cut in the wrong place—which is exactly why modern models have begun to bring in semantic information.

How to Choose the Distance Transform Threshold

The physical meaning of the threshold is that a room's core must be at least this far from a wall. It must be greater than half the doorway width and less than half the room width.

In this figure the doorway is 0.85 meters wide (half-width about 0.42) and the narrowest room has a half-width above about 0.6 meters, so 0.45 meters is a safe middle value.

Threshold too low: the doorway is not cut and adjacent rooms merge into one block (which happens below 0.40 in this chapter's simulation).

Threshold too high: a room's core is fragmented by furniture and one room splits into several blocks, which must then be merged by a minimum-area filter.

Doorway widths are fairly standard in practice (80 to 90 centimeters in most Taiwanese homes), so this threshold can be hard-coded; what is genuinely hard are open-plan layouts and doorless passages.

2. Keep-Out Zones and Virtual Walls

A keep-out zone lets the user mark on the app map an area the robot may not enter—the pet's bowl, the child's block area, a tangle of cables. A virtual wall draws a line that may not be crossed.

Technically this is adding artificial obstacles to the costmap (Chapter 3): in path planning the keep-out zone is treated as impassable. It is a clever design—nothing in SLAM or the sensors need change, only a layer of user-defined constraints added at the planning level. You might consider that this is in fact a way of injecting the user's knowledge into a robotic system.

3. Coverage Path Planning

Coverage path planning (CPP) differs essentially from ordinary path planning:

- Ordinary path planning (Chapters 3 and 29): find the shortest path from A to B, the goal being arrival.
- Coverage path planning: find a path that visits every traversable position, the goal being to miss nothing.

The cleaning robot is the archetypal application of the latter. The basic strategy is to divide the map into obstacle-free strips, run a boustrophedon path within each, and connect the strips in order. The robot uses its onboard sensors and (where available) a preloaded floor plan outline to plan a coverage path reaching every accessible room, corridor, and space.

Evaluation Metrics

- Coverage: the area actually cleaned divided by the cleanable area. The ideal is near 100%, but under furniture and in corners complete coverage is often hard.

- Repetition ratio: total distance traveled divided by the theoretical shortest covering distance. The closer to 1 the more efficient.
- Cleaning time: the time to finish the whole home, the metric the user feels most directly.

These three metrics pull against one another: raising coverage usually raises repetition and time. The engineering trade-off is the core of product design. It is also a quantitative line of thinking the project grading rubric of Chapter 31 may draw on.

16.5 A Cleaning Robot's Sensors: LiDAR, IMU, Bumper, Cliff Sensors

The sensors on a cleaning robot are almost a condensed version of Part II of this book. This section describes the role of each and how they complement one another.

Sensor	Role	Corresponding Chapter
2D LiDAR	The mainstay of mapping and localization, measuring ranges all round	Chapter 8
IMU	Detects turning and tilt, assisting pose estimation	Chapter 7
Wheel encoders	Provide odometry, the source of SLAM's motion prediction	Chapters 3 and 13
Bumper	The last line of defense when the LiDAR has not seen something	This chapter
Cliff sensors	Downward infrared, preventing a fall down the stairs	This chapter
Infrared /	Close-range obstacle detection, filling	Chapter 8

Sensor	Role	Corresponding Chapter
ultrasonic	the LiDAR's blind spot	
AI camera	Recognizes objects and semantics, avoiding cables and pet accidents	Chapters 9 and 32
3D ToF / structured light	Measures the three-dimensional shape of obstacles ahead	Chapters 8 and 9

Table 16-2 The Sensor Suite of a Cleaning Robot

Why So Many Sensors Are Needed

The answer is the height blind spot of Case 8-1 in Chapter 8: a 2D LiDAR scans one horizontal slice and cannot see anything below or above it. Therefore:

- Low obstacles (cables, slippers, pet toys): below the LiDAR's scan plane and entirely invisible, so forward-facing infrared, 3D ToF, or a camera is required.
- Overhanging obstacles (hanging curtains, the edges of low furniture): may sit just above or below the scan plane, with unstable detection.
- Stairs and drop-offs: a horizontal LiDAR scan cannot detect the floor suddenly ending at all, so downward cliff sensors are required—an indispensable safety device on a cleaning robot.
- Glass and mirrors: the old problem of Chapter 8, with the laser passing through or reflecting away, so the bumper must serve as the backstop.

Figure 16-3 A Height Profile: The Things a 2D LiDAR Cannot See



Table 16-2 lists eight sensors, and a beginner's first question is usually: what are so many of them for? This side-view profile is the answer. The red dashed line is the 2D LiDAR's scan plane, about 9 centimeters above the floor—note that it is a plane, not a field of view. The LiDAR knows everything on that plane and is wholly ignorant of everything off it. Chapter 8 of this book called it the height blind spot, and a home is where the height blind spot is most fatal.

Reading left to right, four typical failures. Cables and slippers are only three or four centimeters high and fall entirely below the scan plane; the LiDAR does not merely see them poorly but has no idea they exist—the physical reason a cleaning robot tangles itself in cables and drags a sock through the whole house. A dining chair is a different failure: wide above and narrow below, the scan plane cuts only the slender legs, and the angular resolution formula of Chapter 8 tells us that at sufficient distance the spacing of adjacent beams exceeds the leg's width and the whole leg falls between two beams and is missed. Case 16-2, "why your cleaning robot always hits that chair," is about exactly this geometric fact.

Floor-to-ceiling windows and mirrors are the optical difficulty stressed repeatedly in Chapter 8: the laser either passes straight through or is reflected elsewhere, producing either a room on the map that does not exist at all or an invisible wall. Stairs and drop-

offs are more dangerous still—a horizontal scan cannot detect the floor suddenly ending, and without another sensor the robot would walk straight down the stairs.

The three colored regions in the figure are the answer that fills the gaps, and their placement itself explains the design logic: the 3D ToF and the AI camera open a conical field of view forward and upward, covering the three-dimensional shape above and below the scan plane; the cliff sensors point straight down, covering the floor itself; and the bumper sits on the front edge of the body as the last line of defense—when every other sensor has failed, at least physical contact remains. These three layers, from far to near and from precise to coarse, are the most everyday instance of the sensor fusion idea of Part II: there is no all-powerful sensor, only an appropriate division of labor.

An Observation You Can Make Yourself

Lay a book flat on the floor and see whether your cleaning robot avoids it—a book is about 3 centimeters thick, below the scan plane.

Replace it with a cardboard box (more than 10 centimeters tall) and it will usually be skirted cleanly—because it has entered the scan plane.

This comparison needs only two things already in the house, yet it lets you confirm with your own eyes that the scan plane is a real line.

A further observation: replace the book with a slender chair leg or the base of a floor lamp and place it at various distances to test the effect of the angular resolution of Chapter 8.

Modern Premium Configurations

The sensor suites of recent premium models have grown richer. Some models pair a single AI camera with 3D structured light to form an advanced obstacle detection

system that recognizes a great many common obstacle types. The sensors each have their job: an LED keeps the camera's vision in dark or low spaces, 3D structured light reads the shape of obstacles directly ahead, and 3D ToF measures height and width at once while holding up in strong light.

This evolution confirms again the core conviction of this book: there is no all-powerful sensor, only an appropriate combination. The cleaning robot is the most everyday teaching material for sensor fusion (Part II).

16.6 SLAM Challenges in a Home: Furniture, Mirrors, Low Obstacles

A laboratory classroom is a friendly scene for SLAM (Section 13.6), but a home is another matter. This section faces the challenges of a real home honestly—the part of the chapter you can most readily confirm by watching your own cleaning robot.

1. Slender Chair and Table Legs

Case 8-1 of Chapter 8 discussed this in detail: a dining chair's legs are slender and at a distance may fall entirely between two beams and be missed (the angular resolution formula of Chapter 8). The result is that the leg is not on the map and the robot drives straight into it.

- Remedy: at close range the angular resolution suffices, with the bumper as a backstop; modern models use 3D ToF or a camera to fill in detection at close range.

2. Mirrors and Floor-to-Ceiling Windows

A mirror lets the LiDAR "see" the virtual image within it and draw on the map a room that does not exist at all; a floor-to-ceiling window lets the laser pass straight

through and the robot believes it can go there. This is the LiDAR weakness stressed repeatedly in Chapter 8, and it is extremely common in homes.

- Remedy: have the user set a keep-out zone by hand in the app (Section 16.4); some models detect glass with ultrasound (sound waves reflect off glass); and cover mirrors temporarily during mapping.

3. Low and Dynamic Obstacles

Cables, socks, pet toys, and pet accidents on the floor are the cleaning robot's most famous sources of disaster. They are below the LiDAR's scan plane and in a different place every day—they will not be on the map.

- Remedy: this is exactly where an AI camera earns its keep. The sensing system fuses data from LiDAR, camera, and sometimes a structured-light sensor to detect walls, furniture legs, toys, and even cables, and it does so continuously.

4. A Dynamic Environment: Homes Change

A chair is pulled out, a suitcase is left in the living room, a door opens and closes—a home changes every day. This is a severe test for SLAM, because the map must both remember permanent structure (walls) and forget temporary objects (the map plasticity of Section 13.5).

- Remedy: clamp the occupancy probability to keep the map plastic (Section 13.5); distinguish a permanent map (walls and furniture) from a temporary obstacle layer (objects detected during this cleaning); and some models tally over several cleanings which obstacles are permanent.

5. Changes in Floor Material

Going from tile to carpet changes the friction and slip characteristics of the wheels

and degrades odometry (the slip problem of Section 14.6); carpet also lets the robot sink in, changing the body height and so the LiDAR's scan plane.

- Remedy: raise the noise assumed for odometry in SLAM and rely more on scan matching; some models detect carpet and adjust behavior (raising suction, lifting the mop).

Challenge in the Home	Root Cause	Principal Remedy
Slender chair and table legs	Insufficient angular resolution; missed at a distance	Close-range detection, bumper, 3D ToF
Mirrors and floor-to-ceiling windows	The laser passes through or reflects, producing a virtual image	User-set keep-out zone, ultrasound, temporary covering
Low dynamic obstacles	Below the LiDAR's scan plane	AI camera recognition, 3D structured light
The environment changes daily	The map must both remember and forget	Clamp the probability, layer the map, tally over several runs
Carpet and changes of floor	Degraded odometry, changed body height	Raise the odometry noise, rely on scan matching

Table 16-3 SLAM Challenges in a Home and Their Remedies

16.7 Teaching Activity: Understanding Coverage Path Planning from a Cleaning Route

This section describes how to turn coverage path planning into a lesson requiring no hardware yet highly instructive. It is among the activities this book considers best

suited to a high school classroom, and undergraduates can extend it into a programming exercise.

Why Start from the Cleaning Route

Coverage path planning is a problem everyone has an intuition for and that proves not simple the moment one tries it. Everyone has swept and mopped a floor and knows intuitively to go row by row rather than wander—but defining precisely how to arrange the rows most efficiently, how to go around an obstacle, and how to be sure nothing is missed brings one into the domain of algorithms.

A Paper Activity (suitable for high school)

9. Draw a room on graph paper: a wall for the outer boundary, a few obstacles inside (representing a sofa and a table), and one doorway.
10. Have students plan a route with a pen, required to pass through every cell at least once and to start and end at the doorway, recording the total number of steps.
11. Compare the class's routes: whose took the fewest steps? How much do the shortest and longest differ? Those extra steps are the repetition ratio made concrete.
12. Introduce the boustrophedon strategy: demonstrate cutting the room into several strips, running a boustrophedon within each and connecting them, and compare its efficiency with the intuitive routes.
13. Add variables: if a column stands in the middle of the room, how must the boustrophedon be adjusted? What if the doorway is in a corner rather than the middle?

Observing a Home Cleaning Robot (extended assignment)

14. Observe and record the cleaning route of the robot at home (or in a video): is it random or boustrophedon? How does it handle furniture?
15. Photograph the map in the app and mark which areas are segmented as one room, and whether there is any obvious map error (the symptoms of Section 13.7).
16. Create a challenge deliberately: put a pair of slippers or a (safe) length of cord on the floor and observe whether the robot detects it and how it responds.
17. Pick it up and set it down in another room, and observe the relocalization process (the kidnapping problem of Chapter 3, Case 11-2 of Chapter 11).

Programming Directions for Undergraduates

- Implement boustrophedon coverage planning in Python: take an occupancy grid map with obstacles as input, output a coverage path, and compute coverage and repetition ratio.
- Implement room segmentation: find the doorway bottlenecks with morphological operations or a distance transform and cut the map into blocks.
- In a ROS simulation, connect your own coverage planner to the map built in Chapter 14 or 15 and have a virtual robot perform a complete cleaning.

Part IV, "2D SLAM and the Cleaning Robot," is now complete. We began with the occupancy grid map of Chapter 13, passed through the particle filter of Gmapping in Chapter 14 and the graph optimization and loop closure of Cartographer in Chapter 15, and in this chapter saw how these techniques come down to earth in a home. You should be able to feel that SLAM is not some unreachable frontier technology: its core version is quietly running in your own living room.

In the next part we raise the dimension from the plane to the solid. Part V, "3D

LiDAR SLAM and Autonomous Patrol," introduces three-dimensional point clouds, ICP and NDT registration, the LOAM family of algorithms, and how quadrupeds and patrol robots localize and map in complex three-dimensional space. The difficulty rises, but you have all the necessary groundwork in place.

16.8 The Next Step for Home Cleaning Robots: Four Turns in 2024–2026

The first seven sections of this chapter described a mature combination of technologies: 2D LiDAR mapping, room segmentation, boustrophedon coverage. That combination largely settled after 2017, and the real changes have occurred at its boundaries—once the industry found that the technical bottleneck in cleaning a floor was no longer SLAM but what lies outside SLAM, the direction of innovation turned entirely. This section surveys four such turns.

1. From Going Around Obstacles to Taking Them Away

The most dramatic change came at CES in early 2025: Roborock unveiled the Saros Z70, fitting the cleaning robot with a foldable five-axis arm, OmniGrip, able to pick up small items under 300 grams from the floor—socks, small towels, tissues, slippers—place them where directed, and then come back and clean that patch of floor.

From this book's standpoint, the significance lies not in the arm itself but in the fact that its demands on the map are an entire level higher. To pick up a sock, the robot must know not merely that there is an obstacle there (which an occupancy grid can do) but that it is a sock, that its three-dimensional shape is thus, that it can be gripped, and where it should be put. That goes far beyond the expressive power of the occupancy grid of Chapter 13 and into the territory of semantic maps and object-level

representations. In other words, the real innovation forced out by the arm is an upgrade in the map format.

2. From Geometric Maps to Semantic Maps

Following on from the last point: the mainstream of recent research is to overlay a layer of "what this is" on top of the geometric map. Work on open-vocabulary 3D scene graphs connects the recognition ability of vision-language models to three-dimensional maps, letting a robot respond to tasks described in natural language rather than by coordinates, such as "go to the table beside the sofa."

For a cleaning robot the value of this line is very practical. The room segmentation of Section 16.4 can only cut by geometric bottlenecks and fails at an open kitchen; if the map itself carries object labels, "the area with the counter and the refrigerator is the kitchen" becomes a usable criterion. Keep-out zones are the same—better that the robot recognize the pet's bowl and avoid it than that the user draw a box by hand in the app. The probabilistic framework of Part III handles whether something is there; a semantic map handles what it is, and the two are additive rather than substitutes.

3. The Standardization of Coverage Path Planning: From In-House Algorithms to Open-Source Infrastructure

The third turn is in the software ecosystem. Coverage path planning was for years each manufacturer's secret recipe, but the ROS 2 ecosystem has lately acquired a general coverage planning server (opennav_coverage, built on Fields2Cover) that organizes the strategies of this chapter into configurable options: the path order may be BOUSTROPHEDON, SNAKE, or SPIRAL; the scan angle may be specified or found automatically by brute-force search, and the scoring criterion for "best" is exactly the

three metrics of this chapter—total length, coverage, or number of rows.

This correspondence repays close attention: the evaluation metrics discussed in Section 16.4 become selectable objective functions directly in this open-source tool, and the boustrophedon-versus-spiral comparison the extended challenge asks for is here a configuration parameter. This shows that the content of this chapter is not idle product observation but an engineering field supported by standardized tools. It was designed originally for agricultural machinery (field operations), but the mathematical structure of the coverage problem is identical to indoor cleaning.

4. The Cleaning Robot Chassis as a Standard Platform for Research and Teaching

The last turn concerns you most. As Case 16-3 describes, the cleaning robot chassis is gradually becoming the first choice for a low-cost research platform: the chassis is mature, motor and odometry interfaces are ready-made, the size suits indoor work, the price is far below a research-grade robot, and the flat top makes it easy to add sensor mounts. For university projects and high school clubs on a limited budget, this is a practical route to actually running the theory of sixteen chapters.

This book's advice is: do not chase expensive equipment from the start. A programmable cleaning robot chassis plus one low-cost 2D LiDAR is enough to reproduce everything in Chapters 13 through 16—mapping, loop closure, room segmentation, coverage planning. Put the budget and time saved into the algorithms and field experiments; that is where a project's real value lies.

The Common Signal of the Four Turns

Hardware (the robot arm): cleaning robots begin to act on things rather than

merely go around them.

Representation (semantic maps): the map rises from where something is to what it is, filling the blind spot of purely geometric methods.

Software (open-source coverage planning): coverage path planning goes from each firm's own recipe to a configurable standard component.

Teaching (research chassis): mass-produced consumer hardware flows back as a low-cost platform for research and teaching.

The common signal: SLAM itself is no longer the bottleneck. The technology of the first seven sections remains the foundation, but the competition has moved above that foundation—what more information can be put into the map, and what the robot can do with it.

Looking Back and Forward

Part IV ends here. Looking back at the arrangement of these four chapters: Chapter 13 turned scans into a map, Chapter 14 untied the chicken-and-egg problem with a particle filter, Chapter 15 made history correctable with graph optimization, and Chapter 16 put all three inside a real product and watched how it fares before an 85-centimeter doorway, a 3-centimeter cable, and a mirror. The three sets of experiments and the profile of Figures 16-1 through 16-3 all say the same thing: the elegance of an algorithm is finally accepted or rejected by the geometry of the real world.

From Chapter 17 we raise the dimension from the plane to the solid. Part V, "3D LiDAR SLAM and Autonomous Patrol," introduces three-dimensional point clouds, ICP and NDT registration, the LOAM family of algorithms, and patrol applications. That lonely scan plane in Figure 16-3 is exactly the problem the next part sets out to solve—when

a scan is no longer at a single height, the robot can at last see the whole world.

Formulations and Procedures

1. Coverage and Repetition Ratio

The two core metrics for assessing cleaning efficiency:

$$\text{coverage} = \text{area actually cleaned} \;/\; \text{total cleanable area} \times 100\%$$

$$\text{repetition ratio} = \text{actual distance traveled} \;/\; \text{theoretical shortest covering distance}$$

The theoretical shortest covering distance is roughly the cleanable area divided by the brush width—the distance a perfect boustrophedon would take to cover the whole area without repetition. A repetition ratio of 1 means perfect non-repetition; in practice, with turns, detours around obstacles, and returns to charge, it usually falls between 1.2 and 2.0, while random bump-and-turn may exceed 5.

2. The Number of Rows in a Boustrophedon Path

In an unobstructed rectangle of width W and length L, a boustrophedon at brush width w requires rows and distance:

$$\text{rows } n = \lceil W \;/\; w \rceil$$

$$\text{total distance} \approx n \times L + (n - 1) \times w$$

The second term is the lateral movement at each turn. This expression explains why a wider brush is more efficient—fewer rows and fewer turns. It is also the motivation for widening the brush head in product design.

3. The Complete Operating Procedure of a Cleaning Robot

Figure 16-4 The Complete Flow of One Cleaning Run

① Set out from the dock and load the existing map (or enter SLAM mapping mode on the first run)

↓

② Localize: confirm the position in the map by an AMCL-type method (Chapter 11)

↓

③ Room segmentation and task planning: decide the cleaning order and exclude keep-out zones (Section 16.4)

↓

④ Coverage path planning: generate a boustrophedon route within the current room

↓

⑤ Clean: move along the path while

• continuing to localize and update the map (SLAM)

• avoiding obstacles in real time (camera / 3D ToF / bumper / cliff sensors)

• recording the area already cleaned

↓

⑥ Battery low? → plan a return to the dock and, once charged, resume from the point of interruption

↓

⑦ All rooms complete → return to the dock and upload the map and the cleaning record

4. The Basic Approach to Room Segmentation

Figure 16-5 The Distance Transform Method of Room Segmentation

① Run a distance transform on the occupancy grid map: each free cell records its distance to the nearest obstacle

↓

② The center of a room has a large value (open) while doorways and passages have small ones (narrow)

↓

③ Take the regions above a threshold as the core seeds of the rooms

↓

④ Expand outward from each core until they meet → the boundary is the room division

↓

⑤ Output the segmented room blocks for the user to name and assign cleaning individually

Case Studies

Case 16-1 Twenty Years of Evolution: From Roomba to Physical AI

The twenty years of the cleaning robot are a miniature of the popularization of robotics. The first-generation Roomba appeared in 2002 with a random bump-and-turn strategy—no map, no localization, purely the commercialization of the behavior-

based architecture of Chapter 3. In 2010 Neato first introduced 2D LiDAR and SLAM. In 2017 Xiaomi popularized the LiDAR model at an affordable price (Case 1-1 of Chapter 1).

And by 2026 the situation has changed in kind again. This generation of home robots is classified as physical AI—no longer merely rotating brushes steered by random bumps, but running on-device AI that builds maps, avoids obstacles, and adapts to the shape of real domestic life. They scan rooms with LiDAR and camera SLAM and build 2D or 3D maps of walls, furniture, and habitual traffic lines; having built the map, they can label rooms, exclude keep-out zones, and even remember which areas need cleaning more often.

This line of evolution shows clearly that SLAM is the foundation while real products keep layering on sensors and AI to break through the limits of any single technology. From the occupancy grid of Chapter 13 to the semantic labels and behavioral adaptation of this chapter, what you see is the complete thread from Part IV to Part IX (AI semantic navigation) of this book.

Case 16-2 Why Your Cleaning Robot Always Hits That Chair

This is an everyday phenomenon nearly every user has met and the one that best confirms this book's theory. The robot deftly skirts the sofa and the walls yet always drives headlong into the dining chair's legs.

The reason is threefold. First, the angular resolution problem of Chapter 8: a slender chair leg several meters away may fall entirely between two beams and be missed. Second, the height of the scan plane: some chair legs are wide above and narrow below, and the LiDAR happens to scan at the narrowest part. Third, dynamics:

the chair is pulled to a different place every day, and the old position recorded on the map is long out of date.

The remedies correspond to the three levels: rescan at close range (angular resolution improves with distance), add 3D ToF or a camera to see the whole shape, and keep the map plastic so that it reflects the chair's movement (Section 13.5). Next time you see the robot hit a chair leg, consider which chapter of this book each of the three causes corresponds to—the most personal SLAM review there is.

Case 16-3 The Role of the Cleaning Robot Platform in Research and Teaching

Interestingly, the cleaning robot is not only a consumer product but a practical platform for research and teaching. Some research groups have used a programmable cleaning robot chassis as an unmanned ground vehicle, paired it with a low-cost 360-degree 2D LiDAR, and implemented SLAM and coverage path planning research under ROS.

The advantages of such a platform are very practical: the chassis is mature and stable, motor and odometry interfaces are ready-made, the small size suits indoor work, the price is far below a research-grade robot, and the structure readily accepts additional sensor mounts. For a university project or an advanced high school club this is a feasible low-cost route—a consumer chassis with a teaching-grade LiDAR and a single-board computer makes a research platform capable of running the algorithms of Chapters 14 and 15.

This case echoes the discussion of teaching platforms in Section 8.7 of Chapter 8 and offers a lesson: robotics research need not demand expensive equipment, and

making good use of mature consumer hardware as a base while putting one's energy into algorithms and application innovation is often the smarter strategy.

Hands-On Activities

Exercise 16-1 Coverage Path Planning: From Paper to Program

Purpose: to understand the strategies and evaluation metrics of coverage path planning at three levels—on paper, by observation, and in code—and to connect all the theory of Part IV to lived experience.

Part One: Coverage Planning on Paper (everyone, no equipment needed)

1. Draw a 12 × 10 cell room on graph paper with two 2 × 3 obstacles inside (a sofa and a table) and one cell on an edge as the charging dock.
2. Each student plans a route independently, required to pass through every free cell at least once and return to the dock. Record the total steps.
3. Tabulate for the class: what is the fewest number of steps? The most? Compute each student's repetition ratio (total steps ÷ number of free cells).
4. Demonstrate the boustrophedon strategy: cut the room into strips around the obstacles, run a boustrophedon in each, and connect them. Compare the step count with the intuitive routes.
5. Challenge: if the two obstacles are moved side by side into the middle of the room, how must the boustrophedon be adjusted? Draw your solution.

Part Two: Observing a Real Cleaning Robot (homework)

6. Observe the cleaning route of the robot at home (or in an online video), judge whether it is random or planned, and describe how it handles furniture.

7. Screenshot the room map in the app and mark: is the room segmentation reasonable? Are there any of the map symptoms of Section 13.7 (distortion, breakage, drift)?
8. Low obstacle test: put a pair of slippers (a safe object) on the floor and observe whether the robot avoids, pushes, or gets stuck on them, explaining with the knowledge of Section 16.6.
9. Kidnapping test: move the robot to another room mid-cleaning and time how long relocalization takes (the kidnapping problem of Chapter 3, Case 11-2 of Chapter 11).

Part Three: Programming (Undergraduate)

10. Read an occupancy grid map into Python (one produced by the Chapter 13 exercise, or defined yourself) and implement boustrophedon coverage path planning: cut the map into obstacle-free strips, generate a back-and-forth path in each, and connect the strips.
11. Compute and output the coverage and repetition ratio, and visualize the path on the map with matplotlib.
12. Implement room segmentation: use the distance transform method of Figure 16-5 to cut the map into rooms automatically and display them in different colors.
13. Add a keep-out zone feature: let the user specify a rectangular area, exclude it automatically in planning, and verify that the path indeed avoids it.
14. (Advanced) In a ROS simulation, connect your coverage planner to the map built in Chapter 14 or 15 and have a virtual robot perform a complete cleaning.

Discussion Questions

1. In Part One, how many steps did the boustrophedon route save over the intuitive routes? In a real 100-ping home, how much cleaning time would that difference amount to?
2. What map symptoms did you find while observing a real robot? Judging by Section 13.7, what are their most likely causes?
3. How long did relocalization take in the kidnapping test? If the robot were placed in a room that looks like its original location (two bedrooms of identical layout, say), what would you expect to happen? (Hint: the multimodal distribution of Chapter 11.)
4. Following Table 16-3, if you were designing a cleaning robot for a home with pets, which sensors would you strengthen first, and why?

Extended Challenge (Undergraduate)

First: compare two coverage strategies—boustrophedon and spiral (circling inward from the outside)—on the same map for coverage, repetition ratio, and number of turns, and analyze what shape of room each suits. Second: implement resuming after a break—interrupt the path halfway, record the completed area, and on restart clean only the remainder, verifying that coverage still reaches 100%. This feature requires localization, the map, and task state management at once, and is a good topic for integrating everything in Part IV.

Further Reading

1. H. Choset, "Coverage for Robotics – A Survey of Recent Results," Annals of Mathematics and Artificial Intelligence, 2001. (The classic survey of coverage path planning, including boustrophedon decomposition.)

2. E. Galceran and M. Carreras, "A Survey on Coverage Path Planning for Robotics," Robotics and Autonomous Systems, 2013. (A complete review of coverage planning, suitable as a project reference.)
3. R. Bormann et al., "Room Segmentation: Survey, Implementation, and Analysis," ICRA, 2016. (A comparative study of room segmentation methods, with open-source implementations.)
4. S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, MIT Press, 2005. (The theoretical basis of occupancy grids and Monte Carlo localization, echoing this chapter's mapping and relocalization.)
5. R. Chuchro, "Simulating Coverage Path Planning with Roomba," arXiv, 2022. (Explores coverage planning with reinforcement learning and compares it with a cleaning robot's built-in algorithm.)
6. The Coverage Server documentation of ROS Navigation2 (docs.nav2.org). (The coverage path planning tool of the ROS 2 ecosystem.)
7. The official OpenCV tutorials on the distance transform and morphological operations. (The technical basis for implementing room segmentation.)

Summary

1. A cleaning robot needs SLAM to answer what has been cleaned, when it is finished, and how to get home; SLAM brings four capabilities—systematic cleaning, resuming after a break, user interaction, and consistency across cleanings—and is a physical review of the first fifteen chapters of this book.
2. Random bump-and-turn is the commercialization of reactive architecture (the 2002 Roomba), with a disorderly route and no guarantee of coverage; SLAM-based planning maps first and then runs a boustrophedon path, with high

coverage and a low repetition ratio, though a map error leads directly to a cleaning error.

3. A LiDAR cleaning robot builds an occupancy grid map with a low-cost 2D LiDAR, running SLAM on the first cleaning and loading the map to localize thereafter; modern premium models use LiDAR and an AI camera together—LiDAR to map and localize, the camera to recognize objects on the floor—the commercial practice of "LiDAR gives geometry, the camera gives semantics."
4. Room segmentation cuts the map into blocks by finding spatial bottlenecks such as doorways and already touches on the semantic map idea; a keep-out zone injects the user's knowledge into the costmap; coverage path planning seeks to visit every cell rather than reach a point, and is assessed by the three metrics of coverage, repetition ratio, and cleaning time.
5. The sensor suite comprises a 2D LiDAR, an IMU, wheel encoders, bumper, cliff, infrared or ultrasonic sensors, an AI camera, and 3D ToF; the reason for such variety is the 2D LiDAR's height blind spot—low obstacles, stairs and drop-offs, and glass and mirrors all require other sensors to fill the gap.
6. The five challenges of a home: slender chair and table legs (insufficient angular resolution), mirrors and floor-to-ceiling windows (the laser passing through or reflecting), low and dynamic obstacles (below the scan plane), an environment that changes daily (the map must both remember and forget), and changes in floor material (degraded odometry).
7. Coverage path planning is excellent teaching material—everyone has an intuition for it and it proves not simple the moment one tries—and can be taught at three levels, from planning on paper through observing a real robot to programming, making the best bridge between theory and daily life.

Part IV, "2D SLAM and the Cleaning Robot," ends here. We began with the occupancy grid map, passed through the particle filter of Gmapping and the graph optimization and loop closure of Cartographer, and finally saw how these techniques come down to earth in a home—SLAM is not some unreachable frontier technology; it is quietly running in your own living room. In the next part we raise the dimension from the plane to the solid: Part V, "3D LiDAR SLAM and Autonomous Patrol," introduces three-dimensional point cloud processing, ICP and NDT registration, the LOAM family of algorithms, and how quadrupeds and patrol robots localize and map in complex three-dimensional space.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Part V 3D LiDAR SLAM and Autonomous Inspection

Chapter 17

3D Point Clouds and LiDAR Mapping

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Part V begins here, and what this chapter does is lift everything accumulated in the first four parts out of the plane and into three dimensions. The change sounds like no more than adding a z coordinate, but more is altered than one would expect: the map is no longer a picture that can be printed but tens of millions of three-dimensional points; registration is no longer three degrees of freedom but six; the volume of data jumps from a few megabytes to a few gigabytes. The four tools of this chapter—downsampling, ICP, NDT, and map representation—exist precisely to cope with that jump in scale.

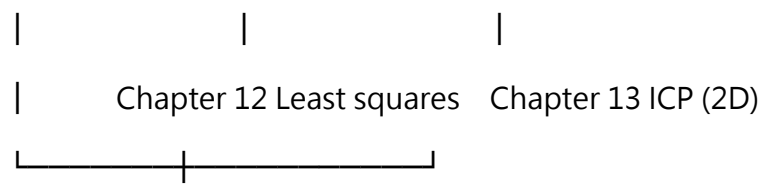
It is worth noting that this chapter contains almost no new mathematics. ICP is the three-dimensional version of Section 13.3, the core of NDT is the multivariate Gaussian distribution of Chapter 11, the optimization behind registration is the least squares of Chapter 12, and the coordinate transformation is the rigid-body motion of Chapter 10. What is genuinely new is only scale—and the engineering problems that scale brings. That is also why this chapter devotes a whole section to downsampling: in a three-dimensional world, how to make data smaller without distorting it is a subject in its own right.

Figure 17-0 The Link Map of Chapter 17

Existing tools from the first four parts

Chapter 8 3D LiDAR Chapter 10 Rigid-body transformation Chapter 11 Gaussian

distribution



【Chapter 17 3D point clouds: the new problems that scale brings】

Downsampling ─ ICP / NDT registration ─ Map representation ─ Colorization



Learning Objectives

The 2D SLAM of Part IV lets a robot localize and map on a plane, but the real world is three-dimensional: stairs have height, ceilings carry pipework, a legged robot must step over obstacles, and a drone must thread through three-dimensional space. This chapter raises the dimension from the plane to the solid, introducing the processing and registration of three-dimensional point clouds and opening Part V's treatment of 3D SLAM and autonomous inspection. After studying this chapter, you should be able to:

1. Describe the data structure, sources, and characteristics of a 3D point cloud, and understand how it differs fundamentally from a 2D occupancy grid map.
2. Explain the principle and the necessity of voxel downsampling, and estimate its effect.
3. Describe the procedure, the variants, and the limitations of ICP point cloud registration.

4. Explain the central idea of NDT and compare it with ICP in terms of differences and suitable applications.
5. Describe the ways a 3D map can be represented and the procedure for scanning an indoor space.
6. Explain the principle of point cloud colorization and the applications of colored three-dimensional visual modeling.
7. Give examples of the practical value of 3D spatial modeling in factories, on campuses, and at exhibition venues.

Core Concepts

中文名詞	English Term	Explanation
點雲	Point Cloud	Spatial data made up of a large number of three-dimensional points, each with x, y, and z coordinates.
體素	Voxel	A cubic cell in three-dimensional space, the counterpart in three dimensions of the 2D pixel.
體素降採樣	Voxel Grid Downsampling	Cutting space into voxels and keeping one representative point per cell to reduce the count.
點雲配準	Point Cloud Registration	Finding the rotation and translation between two point clouds that bring them into alignment.
迭代最近點	ICP	The classic registration algorithm that repeatedly finds corresponding points and solves for the best transformation.
點對面 ICP	Point-to-Plane ICP	A variant of ICP taking the point-to-plane distance as the error, which converges

中文名詞	English Term	Explanation
		faster.
常態分布轉換	NDT	A registration method that converts a point cloud into Gaussian distributions within voxels and then matches.
法向量	Normal Vector	The vector describing the orientation of the surface a point lies on, used in point-to-plane registration and in shading.
KD 樹	KD-Tree	The spatial data structure that accelerates nearest-neighbor search, the key to ICP's efficiency.
八叉樹	Octree	A tree structure representing three-dimensional space by recursive subdivision, used to compress a map.
點雲顏色化	Colorization	Projecting the color of camera imagery onto a point cloud to produce a colored model.
數位分身	Digital Twin	A high-accuracy three-dimensional digital model of a real space or piece of equipment.

17.1 Point Cloud Data

Chapter 8 introduced the point cloud as the output format of a 3D LiDAR; this section goes deeper into its structure and characteristics, which are the data foundation of all of Part V.

What Is a Point Cloud

A point cloud is a set made up of a large number of three-dimensional points. In its most basic form, each point carries only three numbers—x, y, and z coordinates—meaning "there is a surface at this position in space." Plot several hundred thousand such points and the three-dimensional shape of an object and its surroundings emerges.

Each point may also carry additional fields:

- Intensity: the strength of the laser return, which reflects the material and the lightness of the surface. Metal and retroreflective signs give a high intensity, black objects a low one.
- Color (RGB): if camera imagery has been fused in, each point can carry a color, forming a colored point cloud (Section 17.6).
- Ring: in a multi-beam LiDAR, which laser line the point came from. This information is very important in the LOAM feature extraction of Chapter 18.
- Timestamp: the precise instant at which the point was measured, used for motion compensation (Chapter 18).

Three Characteristics of a Point Cloud

- Unordered: a point cloud is a set of points with no fixed ordering. This is unlike the regular pixel grid of an image—convolution can be applied directly to an

image but not to a point cloud. It is also one of the great difficulties of processing point clouds with deep learning.

- Sparse and uneven: points are dense nearby and sparse far away (the angular resolution formula of Chapter 8); and only surfaces carry points, with the interior of objects and anything not scanned left empty.
- Enormous in volume: a 64-beam LiDAR can produce more than a million points per second. A ten-minute scan is several hundred million points—which is why Section 17.2 must downsample.

The Fundamental Difference from a 2D Occupancy Grid Map

Characteristic	2D Occupancy Grid Map (Chapter 13)	3D Point Cloud
Data structure	A regular grid with one probability per cell	An unordered set of points
Spatial representation	The three states of occupied, free, and unknown	Only "a surface is here"
Unknown region	Marked explicitly (in gray)	Cannot tell "nothing there" from "not scanned"
Memory	Proportional to area	Proportional to point count, usually far larger
Usable directly for planning	Yes	Not conveniently; conversion is needed
Fidelity of detail	Limited by resolution	Very high; the raw measurements are preserved

Table 17-1 2D Occupancy Grid Map Compared with a 3D Point Cloud

Pay particular attention to the row for the unknown region. Chapter 13 stressed

that an occupancy grid map can tell "definitely empty" from "unknown," which is crucial for path planning; but a bare point cloud has no such ability—a place with no points may be truly empty or may simply never have been scanned. This is exactly why point clouds are so often converted into a 3D occupancy grid (an octree, say) in practice (Section 17.5).

17.2 Voxel Grid Downsampling

Faced with data that routinely runs to tens of millions of points, the first thing to do is almost always to downsample—to reduce the number of points while preserving the shape as far as possible. Voxel downsampling is the most common and the most intuitive method.

The Principle: One Representative per Cell

A voxel is the three-dimensional counterpart of a pixel—space cut into small cubes of fixed side length. The procedure of voxel downsampling is extremely simple:

1. Cut the space occupied by the whole point cloud into a voxel grid of side length d.
2. Assign each point to the voxel it falls in.
3. Output one representative point per voxel—usually the centroid (the mean coordinate) of all points in that voxel.
4. Ignore empty voxels entirely.

The result is that the many points of an originally dense region are merged into a few, while sparse regions are barely affected. The point cloud as a whole becomes uniform and light, yet the shape is largely preserved.

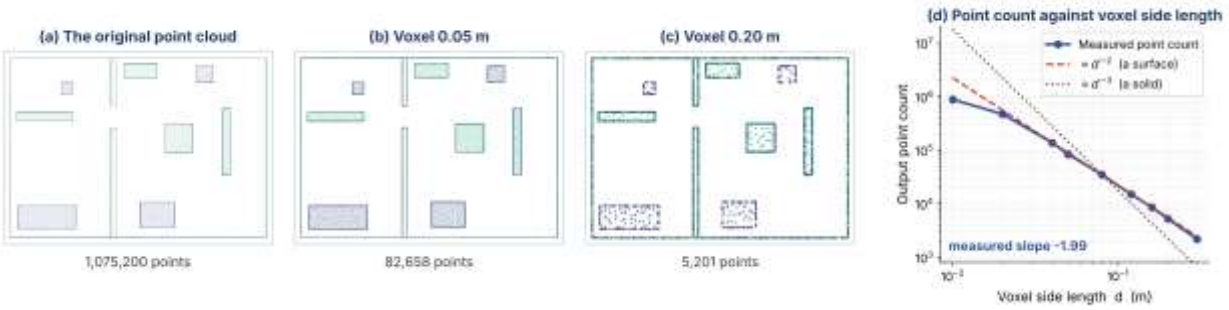
Centroid or Cell Center

Here lies an implementation difference that looks minor but matters. Taking the centroid (the center of mass) of all points in a voxel reflects the true surface position more faithfully; taking the voxel center for speed is faster but less accurate. Comparative studies show that exact voxel downsampling gives results all but identical to the standard implementation, while the approximate version may produce spurious points—as many as twice the point count. For mapping where accuracy matters, the exact centroid version is recommended.

The Effect and the Price of Downsampling

- Effect: the larger the voxel side length d, the fewer points are kept. The reduction in point count is roughly inversely proportional to the cube of d—double the side length and the count falls to about an eighth. This is the inevitable consequence of three-dimensional space.
- Price: loss of detail. If the voxel side length exceeds the characteristic size of an object (a 20-centimeter voxel against a 5-centimeter pipe, say), that feature disappears.
- Trade-off: d should be slightly smaller than the smallest feature size you care about. Indoor mapping commonly uses 5 to 10 centimeters; large outdoor areas can use 20 to 50 centimeters.

Figure 17-1 Voxel Downsampling: A Visible Trade-off and a Law You Can Check



This figure comes from a genuine simulated scan: an indoor space of 10 × 7 × 2.8 meters, scanned from seven positions by a 64-beam LiDAR and merged, for 1,075,200 points in all. Panels (a) to (c) are top views (the floor and ceiling are filtered out of the display so the structure can be seen clearly), with color representing height. The walls, the central partition and the doorway, and the outlines of the bookcase, the workbench, and the equipment rack are all visible.

Comparing the three panels makes the trade-off of downsampling clear. At a voxel side length of 0.05 meters (panel b), the point count falls from 1.075 million to 82,658, a reduction of more than ninety percent, yet the wall lines, the furniture outlines, and the position of the doorway are all preserved—this is what Section 17.2 means by "the point cloud as a whole becomes uniform and light, yet the shape is largely preserved." At 0.20 meters (panel c), only 5,200 points remain, the walls begin to break up into strings of disconnected points, the furniture degenerates into a few patches of color, and the doorway is already hard to make out. The three pipes of 18 centimeters' diameter on the ceiling of this chapter are close to vanishing at this scale as well.

Panel (d) turns all this into a law you can check by arithmetic. The horizontal axis is the voxel side length and the vertical axis the output point count, both on logarithmic scales. The measured points fall on a straight line of slope −1.99—as near as makes no

difference to the −2 predicted by Equation 1, and clearly away from the −3 of a solid volume (the dotted line in the figure). This confirms the easily overlooked point of Section 17.2: what the LiDAR scans is the surface of an object rather than its interior, so a point cloud is essentially a two-dimensional surface embedded in three-dimensional space, and the point count falls with the square rather than the cube of the side length. The practical rule of thumb is simply this: double the voxel side length and the point count falls to about a quarter.

The only departure from the straight line is the leftmost point, at 0.01 meters. The reason is not hard to see: once the voxel is smaller than the average spacing of the original points, most voxels contain only one point, downsampling does almost nothing, and the count is pinned by the original sampling density. This gives a useful test—if you shrink the voxel and find the point count no longer rising appreciably, you have reached the resolution floor of the sensor itself, and going smaller only wastes computation.

How to Choose the Voxel Side Length

The rule: d should be slightly smaller than the smallest feature size you care about. To see a 5-centimeter pipe, the voxel cannot be 10 centimeters.

Indoor mapping commonly uses 0.05 to 0.10 meters; large outdoor areas or vehicle-mounted work may use 0.20 to 0.50 meters.

The point cloud used for registration may be sparser than the one used for display—the registration experiment of Figure 17-3 in this chapter was run at 0.06 meters.

The check: shrink the voxel step by step and record the point count. When the

count no longer rises with it, the resolution limit of the sensor has been reached.

Recent systems such as KISS-ICP have one detail worth noting: they keep one original point in each voxel rather than the centroid, to avoid the discretization error that computing a centroid introduces (see Section 17.8).

Other Downsampling Strategies

Besides the voxel method, several other strategies are common, each with its own place:

- Random downsampling: discard a fixed proportion of the points at random. The fastest, but it can leave the distribution uneven.
- Uniform downsampling: take one point every fixed interval; simple, but it ignores density.
- Distance threshold: guarantee that output points are at least a given distance apart.
- Farthest point sampling: repeatedly choose the point farthest from those already chosen; the most evenly distributed but slower to compute, and common as preprocessing for deep learning.
- Edge-preserving downsampling: a recent line of improvement that adjusts the downsampling density according to point cloud features and deliberately retains edge information, so that key features are not lost.

The implementations in this book use voxel downsampling throughout—it strikes the best balance among effect, speed, and implementation difficulty, and it is the default choice of mainstream libraries such as PCL and Open3D.

17.3 ICP Point Cloud Registration

Point cloud registration is the central computation of 3D SLAM: given two partly overlapping point clouds, find the rotation and translation needed to align one with the other. This is precisely the three-dimensional version of the scan matching of Section 13.3, and ICP remains the classic method.

The Three-Step Loop of ICP

The procedure of Iterative Closest Point (ICP) is exactly the same as the 2D version introduced in Chapter 13, only carried out in three-dimensional space:

5. Find correspondences: for each point in the source cloud, find the nearest point in the target cloud and pair them.
6. Solve for the transformation: compute a rotation R and a translation t (the rigid-body transformation of Chapter 10) that minimize the sum of squared distances between all paired points.
7. Update and iterate: apply the transformation to the source cloud, return to step 1 and find correspondences afresh, repeating until the change in the transformation falls below a threshold (convergence).

The Key to Efficiency: the KD-Tree

Finding the nearest point in step 1 is the most time-consuming part of ICP. With brute force, every point must be compared against every point of the target cloud, which is entirely impractical for a cloud of several hundred thousand points.

The remedy is a KD-tree—a data structure that recursively partitions space and can reduce nearest-neighbor search from linear to logarithmic time. In practice the efficiency of building and searching the KD-tree often determines the speed of ICP as

a whole. It is also why modern registration libraries place such emphasis on parallelized KD-tree implementations.

Important Variants of ICP

- Point-to-point ICP: the original form, in which the error is the straight-line distance between two points. Simple to implement, but slow to converge when sliding along a plane.
- Point-to-plane ICP: the error becomes the perpendicular distance from the source point to the plane containing the target point. This change lets points slide freely along a plane without penalty and greatly accelerates convergence—it is the version most used in practice. The price is that a normal vector must first be estimated for every point.
- Generalized ICP (GICP): considers the local planar structure of both source and target, and is more robust still.
- Voxelized accelerated versions: downsample before registering, or use a voxel structure to accelerate the correspondence search, raising speed while holding accuracy.

Three Limitations of ICP

- It needs a good initial guess: ICP is a local optimization, and if the initial positions of the two clouds are too far apart it converges to a wrong local optimum. This is the old problem already stressed in Section 13.3—in practice odometry or an IMU supplies the initial value.
- It fears structural degeneracy: in a long corridor, a tunnel, a large flat surface, or any scene short of geometric constraints, the point cloud can slide freely in some direction with no change in error, and ICP cannot determine the

displacement along it. This is the three-dimensional version of the corridor predicament of Section 13.6.

- It is sensitive to outliers: dynamic objects (people walking by) and noisy points drag the registration result around. In practice it must be paired with outlier rejection or a robust kernel (Chapters 12 and 15).

17.4 NDT: the Normal Distributions Transform

ICP has one fundamental bottleneck in efficiency: every iteration must find nearest-point correspondences. The Normal Distributions Transform (NDT) proposes an entirely different line of attack—do not look for corresponding points at all, but turn the point cloud into a continuous probability distribution.

The Central Idea: Turn the Point Cloud into a Probability Field

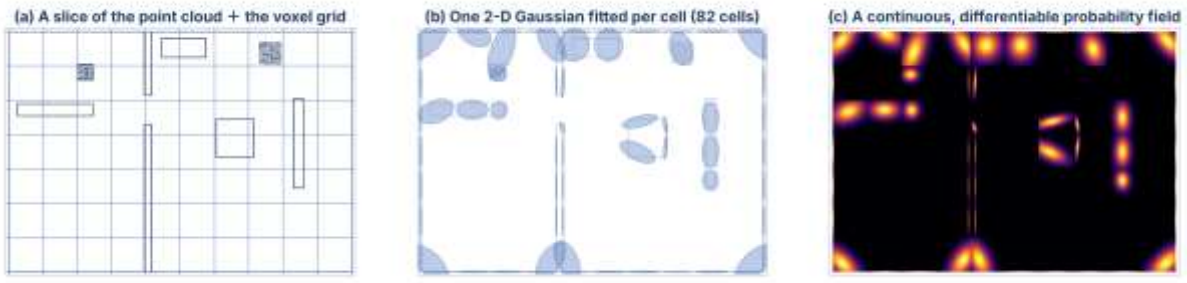
What NDT does is this:

8. Cut the space occupied by the target point cloud into voxels (cells).
9. For each voxel, compute the mean and the covariance of all points inside it—that is, use a three-dimensional Gaussian distribution to describe roughly what the points of that cell look like (the Gaussian distribution of Chapter 11).
10. The whole point cloud thereby becomes a continuous probability field pieced together out of many Gaussian distributions: at any position in space one can compute the probability density that a surface is there.
11. To register, find the transformation that maximizes the total probability of all points of the source cloud within this probability field.

This transformation brings one decisive advantage: the objective function becomes a smooth, differentiable, continuous function that can be solved directly by

optimization methods such as Newton's method (Chapter 12), and there is no need whatever to search repeatedly for nearest points during the iterations.

Figure 17-2 The Core of NDT: Turning Discrete Points into a Continuous Probability Field



"Turn the point cloud into a probability field" sounds abstract, but drawn out it is in fact very intuitive. For clarity this figure takes a horizontal slice of the indoor space of the previous figure at a height of 1.0 meters, reducing the problem to two dimensions for the demonstration; the three-dimensional procedure is exactly the same, only with the Gaussian distributions becoming ellipsoids instead of ellipses.

Panel (a) is the sliced point cloud with a voxel grid of side length 1.0 meters laid over it. At this stage the data is still discrete points, and to ask whether there is a surface at some position in space one can only search for nearby points—exactly what ICP must do at every iteration, and the most time-consuming thing it does. Panel (b) then computes the mean and covariance of the points in each cell and describes them with a two-dimensional Gaussian distribution. Note that the shape of each ellipse itself carries information: in a cell lying on a wall, the ellipse is drawn out into a long thin sliver whose major axis runs along the wall; in a cell at a corner, the ellipse is closer to circular. In other words, the covariance matrix automatically records which way the surface in that cell extends.

Panel (c) pieces these 82 Gaussian distributions together into a continuous probability field. Now any point in space can be substituted straight into a formula to obtain the probability density that a surface is there, with no search and no KD-tree. More important still, this function is smooth and differentiable, so it can be handed directly to the Newton's method of Chapter 12 to find an extremum. The two-step find-correspondences-then-solve loop of ICP is compressed in NDT into a single optimization problem.

The figure also honestly exposes one weakness of NDT: look closely at panel (c) and the probability field is seen to break at the voxel boundaries, so that where one wall crosses two cells a seam appears in the bright band. This is because each cell computes its own Gaussian and nothing is continuous from cell to cell. Real NDT implementations use overlapping voxels or interpolation to soften the problem, and this is precisely why the voxel resolution discussed in the next section is NDT's most sensitive parameter.

Reading the Ellipse: the Geometric Meaning of the Covariance

The direction of the ellipse's major axis = the direction of greatest variance in the cell's point distribution, which is the direction in which the surface extends.

The flatter the ellipse = the more the cell's points concentrate on a single plane, and the stronger the constraint in the direction of the normal.

Close to circular = the point distribution has no marked directionality (a corner, a cluttered area, or too few points).

The implementation in this book sets a lower bound of 0.05 meters on the covariance: a real surface carries measurement noise and should not be described as having zero thickness. Without this regularization the Gaussian

degenerates into a line and the optimization becomes extremely unstable.

A cell must hold at least five points before a Gaussian is built—with too few points the covariance estimate is unreliable, and it is better to leave the cell blank.

NDT Compared with ICP

Item of Comparison	ICP	NDT
Core computation	Finding nearest-point correspondences	Gaussian distributions within voxels
Does it search for correspondences	Yes (accelerated by a KD-tree)	No
Speed on large point clouds	Slower	Faster
Accuracy	Usually higher	Slightly below ICP
Sensitivity to the initial value	More sensitive	Less sensitive, with a wider basin of convergence
Key parameter	Correspondence distance threshold	Voxel resolution
Suitable scenes	Fine registration with fewer points	Large areas, large point clouds, a poor initial value

Table 17-2 ICP Compared with NDT

Comparative studies report that because NDT does not compute with the features of corresponding points during registration, it is faster than ICP on large point clouds

but less accurate. This conclusion appears quite consistently across a number of experiments.

Figure 17-3 ICP against NDT Head to Head, and Structural Degeneracy in a Long Corridor

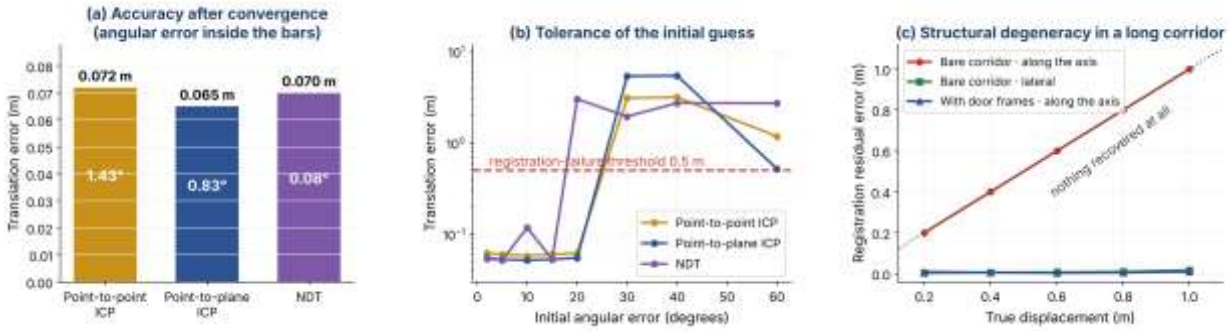


Table 17-2 compared the two methods in words; this figure turns that into measurement. The setup is a genuine SLAM situation: a LiDAR scans the same indoor space once from each of two positions (about 3.9 meters apart and rotated by 34 degrees), the two clouds overlap only partly, and each is voxel downsampled at 0.06 meters to about twenty-three thousand points. The task is to align the second back onto the first, and it is deliberately given an inaccurate initial guess.

Panel (a) is the accuracy after convergence. The translation error of all three methods falls between 6.5 and 7.2 centimeters, a narrow spread—this figure is in fact the floor imposed by the two viewpoints sampling the surface differently, not the limit of the algorithms. What really separates them is the angular error (the numbers inside the bars): 1.43 degrees for point-to-point ICP, 0.83 degrees for point-to-plane ICP, and 0.08 degrees for NDT. Point-to-plane is clearly better than point-to-point, which bears out the argument of Section 17.3 that points can slide freely along a plane without penalty; and NDT, because the covariance itself encodes the direction of the surface, does best of all on angle.

Panel (b) is tolerance of the initial guess, and it is the most practical panel in the figure. The horizontal axis is the initial angular error and the vertical axis the final translation error (on a logarithmic scale). All three methods converge stably to the centimeter level within 15 degrees; NDT begins to fail at 20 degrees, while both forms of ICP hold at 20 degrees and collapse at 30. The shape of the curves deserves notice: it is not gradual degradation but failure off a cliff—once outside the basin of attraction the result is not worse but wholly wrong, with the error jumping straight to the meter level. This explains why odometry or an IMU must supply the initial guess in practice (the first limitation of Section 17.3): they need not be very accurate, only accurate enough to hold the error within a dozen or so degrees.

Panel (c) then examines the second limitation of Section 17.3 on its own: structural degeneracy. The scene becomes a featureless corridor 90 meters long and 3.2 meters wide, far longer than the LiDAR's 25-meter range, so that the robot never sees either end. The red line is motion along the corridor's axis: the residual error equals the true displacement itself—displace by 1.0 meter and the error is 1.0 meter, meaning that registration has recovered no displacement whatever. The green line is lateral motion, with an error of only a centimeter or two, entirely normal. The blue line is the axial test repeated after doorways and columns are added along the same corridor: the error falls at once to the centimeter level.

These three lines could hardly make structural degeneracy clearer: the problem lies not in the algorithm but in the geometric information itself. After a translation along the axis of a featureless corridor the point cloud looks exactly the same, and the registration algorithm has nothing on which to judge how far it has moved—no optimizer, however good, can conjure information that does not exist. Only two

remedies exist: use another sensor (wheel odometry, an IMU, UWB) to supply the constraint in that direction, or make sure the environment holds enough geometric features. This also answers the tunnel difficulty of Case 17-2.

An Honest Note About This Comparative Experiment

Speed is not included in the comparison. The NDT here is a teaching implementation written in Python while the ICP is Open3D's C++ implementation, and with an order of magnitude between them in engineering optimization, comparing run times would not be fair. The conclusion in the literature that NDT is faster (Section 17.4) comes from implementations optimized to the same degree.

The tolerance result differs from the common claim that NDT is less sensitive to the initial value. In this experiment ICP uses a coarse-to-fine correspondence distance schedule (1.5 → 0.75 → 0.3 meters), which favors ICP considerably; NDT's basin of attraction depends on the voxel size, and this book tried a number of schedules from 2 meters to 8 meters, of which starting at 2 meters was best.

How the conclusion should be read: all three methods work given a reasonable initial value, and point-to-plane ICP and NDT beat point-to-point on angular accuracy; while the two limitations that the initial value must be good enough and that a degenerate scene has no solution apply to all three alike.

You can reproduce these three panels in Parts Two and Three of Exercise 17-1; different numbers are normal, and what matters is the trend.

NDT's Key Parameter: Voxel Resolution

The most important parameter of NDT is the side length of the voxel (cell). Too

small and there are too few points per cell for a reliable Gaussian estimate; too large and the description of space is too coarse, so accuracy falls. One study measuring point clouds in a tunnel environment found that a voxel resolution of 0.5 meters met the resolution requirement with the shortest registration time. This is a reminder that the best parameter is closely tied to the scale of the scene—indoors, in a tunnel, and outdoors all differ, and the value must be measured and tuned.

NDT and ICP in Cooperation

Since NDT is fast and robust while ICP is accurate, the natural thought is to combine them: use NDT for a coarse registration that quickly closes the gap, then ICP for a fine registration that raises the accuracy. This pairing of coarse NDT with fine ICP is quite common in practical systems, and it again embodies the idea this book stresses repeatedly: there is no universal algorithm, only an appropriate combination.

17.5 3D Maps and Indoor Space Scanning

With downsampling and registration in hand, a succession of LiDAR scans can be stitched into a complete three-dimensional map. This section explains how a 3D map may be represented and the practical procedure for scanning indoors.

Three Representations of a 3D Map

- Raw point cloud map: store all the (downsampled) points directly. Highest in fidelity and best-looking, but large, unable to tell empty from unknown, and awkward for planning paths directly. Suited to surveying and to display.
- 3D occupancy grid (octree): cut space into voxels and record an occupancy probability, subdividing recursively with an octree—subdividing only where something is present and representing open space with large cells, which saves

a great deal of memory. This representation can distinguish the three states of occupied, free, and unknown, and suits robot navigation and obstacle avoidance. OctoMap, mentioned in Chapter 3, is the representative example.

- Mesh model: convert the point cloud into a surface model of triangular facets. Small in file size, able to carry textures, and suited to architectural and game applications, but the reconstruction is more complex and may distort the result.

The Procedure for Scanning an Indoor Space

12. Plan the scanning route: make sure all the space you want is covered, and leave enough overlap at the critical places (corners, between rooms), since insufficient overlap causes registration to fail.
13. Collect the data: scan slowly, moving with a backpack, a handheld unit, or a robot platform (Section 15.6). Move slowly and avoid violent shaking, which causes motion distortion.
14. Preprocess: remove outliers, crop the extent as needed, and voxel downsample (Section 17.2).
15. Register and stitch: align neighboring scans with NDT or ICP (Sections 17.3 and 17.4), together with the pose estimates from SLAM.
16. Optimize globally: detect loop closures and run graph optimization (Chapters 12 and 15) to remove accumulated drift and secure overall consistency.
17. Post-process and output: convert to an octree or a mesh or add color as needed (Section 17.6), and output in a standard format (PLY, PCD, or LAS).

Quality Assessment

Echoing the assessment of 2D maps in Section 13.7, a 3D map must also be checked: whether the walls have grown thick (a sign of registration error, with one wall

drawn as two layers), whether straight structures are straight, whether the loop closure lines up, and how measured key dimensions compare with surveyed values. Thickened walls are the most typical quality indicator in 3D mapping—an ideal wall should be a thin single layer of points.

17.6 Point Cloud Colorization and Colored Visual Modeling

A purely geometric point cloud is accurate but looks like a gray-white skeleton, hard to interpret and unattractive. Only with color added does a point cloud truly become a digital twin.

Where the Color Comes From: the Projection Principle

The color comes from camera imagery. The principle is the pinhole projection model of Chapter 9 used in reverse:

18. The camera's intrinsic parameters K (Chapter 9) and the extrinsic parameters of the camera relative to the LiDAR (the coordinate transformation of Chapter 10) are known.
19. For each three-dimensional point, first use the extrinsics to take it from LiDAR coordinates into camera coordinates.
20. Then use the intrinsics to project it onto the image plane, obtaining the corresponding pixel coordinates.
21. Read the RGB value of that pixel and assign it to this three-dimensional point.

This procedure again confirms the central value of Chapter 10: accurate coordinate transformation and extrinsic calibration are the foundation of multi-sensor fusion. Let the extrinsics be off by a few degrees or a few centimeters and the color is pasted crooked—the color of the wall ends up on the window.

Practical Challenges

- Time synchronization: LiDAR and camera timestamps must be aligned. In motion, a discrepancy of a few tens of milliseconds misplaces the color (the synchronization problem of Chapters 5 and 10).
- Occlusion handling: a point the camera cannot see (blocked by something in front) should not be given the color of the thing in front. A visibility test is needed.
- Exposure and color difference: images taken at different moments differ in brightness, so patches of color appear in the stitched cloud. Color balancing is needed.
- Mismatched fields of view: a LiDAR is often 360 degrees while a camera's field of view is limited. Several cameras are needed, or one accepts that some points have no color.

The Value of a Colored Point Cloud

- Readability: a colored model can be understood at a glance even by non-technical people—which matters enormously when presenting project results to a client or to the university.
- Semantic cues: color provides clues to material and object, an important input to later AI semantic segmentation (Chapter 32).
- Preservation of cultural assets: in the digital archiving of monuments and artifacts, color and texture are indispensable information.
- Defect detection: wall cracks, corrosion, and water stains are not obvious geometrically but are plain to see in color.

17.7 3D Spatial Modeling of Factories, Campuses, and Exhibition Venues

This section explains the practical value of 3D mapping, with particular attention to directions that can be carried out on a university campus.

Factories: Facility Management and Digital Twins

A three-dimensional model of a factory can serve production line layout planning (will the new machine fit? will the transport path collide with anything?), the recording of piping and fire safety equipment, and spatial queries for equipment maintenance. Applications of this kind combine a preloaded floor plan outline to plan a coverage path that reaches every accessible room, corridor, and machine space, while keeping the robot safely clear of people and equipment during the building's operating hours.

The key advantage of LiDAR over a camera is especially plain here: unlike camera-based methods, which perform poorly in low light or in textureless areas (machine rooms and parking structures, say), LiDAR works reliably whatever the lighting, which makes it the clear choice for facility maintenance applications.

Campuses: Directions for Implementation on a University Site

- Architectural surveying: scan the interior of a department building to produce a three-dimensional model usable for teaching, space management, and accessibility planning.
- Indoor navigation maps: build the spatial foundation for the semantic navigation of Part IX, so that the robot knows where the elevator is and where the stairs are.

- A digital twin of the laboratory: scan the intelligent robotics classroom as the basis of a simulation environment—which lets students test algorithms in a virtual classroom and then move to the physical one for verification.
- Teaching material: the scan itself is excellent teaching material, letting later students see a three-dimensional model of what our own school looks like.

Exhibition Venues and Cultural Assets

A three-dimensional model of an exhibition venue can serve exhibit planning, virtual visits, and circulation analysis; and the digital preservation of monuments and historic buildings is one of the socially most valuable applications of 3D scanning—once a building is damaged by disaster or by time, an accurate three-dimensional record becomes the only basis for restoration. Taiwan has an abundance of historic buildings, which makes this a project direction of both technical and humanistic significance.

The Technical Foundation: Edge Computing

It is worth noting that modern 3D mapping leans more and more on edge computing platforms. SLAM algorithms run on edge processors such as the NVIDIA Jetson, stitching individual scan frames together and correcting positional drift through loop closure—the technique by which a robot recognizes an area it has visited before and uses that recognition to correct accumulated error in its trajectory estimate. Recent years have also brought GPU-accelerated voxel reconstruction libraries that can build signed distance fields and occupancy grids in real time, letting a robot reconstruct the geometry of a facility while it navigates, supporting mapping and obstacle avoidance at once.

This means 3D SLAM is no longer a technology that demands a workstation; a

handheld Jetson is equal to it—excellent news for teaching and student projects, and the direction pursued in depth in Part VII (edge AI SLAM) and Chapter 32.

17.8 Where Point Cloud Processing Is Going: Four Developments of 2024–2026

The original ICP paper was published in 1992 and NDT in 2003—by the standards of the techniques in this chapter, they are thirty- and twenty-year-old methods. Interestingly, far from being superseded, they have lately been the subject of a return to basics. This section gathers four developments worth noting.

1. A Return to Simplicity: KISS-ICP's "In Defense of Point-to-Point ICP"

KISS-ICP, published by a team at the University of Bonn in 2023, carries the subtitle "In Defense of Point-to-Point ICP" outright. Their claim is quite provocative: rather than piling complexity onto LiDAR odometry—feature extraction, deep learning modules, loop closure detection, pose graph optimization—get a few parts of the most basic point-to-point ICP right and the result can stand alongside the state of the art.

The parts they got right number four: predict motion with a constant-velocity model and compensate scan distortion with it (needing neither IMU nor wheel encoder), build the local map by voxel downsampling, optimize with a robust kernel, and, most crucially, use an adaptive correspondence distance threshold—a threshold not set by hand but computed automatically from a statistic of how far the odometry actually departs from the motion prediction (three times the standard deviation). The result is a system that needs almost no tuning, with one set of settings carrying across vehicle-mounted, aerial, and handheld platforms.

For students taking this course the lesson of this case matters more than its technical

detail: when your registration works poorly, the first response should not be to reach for a more complex algorithm but to check the basics of downsampling, initial guess, and correspondence distance threshold. The cliff-edge curve of Figure 17-3(b) in this chapter has already shown that the initial value and the threshold matter far more than the choice of algorithm.

2. Engineering Efficiency: from Comparing Algorithms to Comparing Implementations

The second development is that the fact that one algorithm can differ several-fold between implementations has formally been put on the table. The representative work is the small_gicp library published in 2024 by Kenji Koide of Japan's National Institute of Advanced Industrial Science and Technology: it rewrites and parallelizes downsampling, KD-tree construction, and registration throughout, and reports voxel downsampling about 1.3 times faster than PCL single-threaded and 3.2 times faster with six threads, KD-tree construction up to 6 times faster than nanoflann, and generalized ICP about 2.4 times faster than PCL's version on a single thread alone.

This echoes the discussion of tool choice in Case 17-3 and adds a dimension this chapter had not yet mentioned: when you write in a project report that "we used ICP for registration," that sentence has in fact said nothing definite about performance—which implementation, whether multithreading was enabled, and how the KD-tree was built may affect the speed you actually see more than the difference between point-to-point and point-to-plane.

3. A New Option for Map Representation: 3D Gaussian Splatting

The third development strikes at the list of three 3D map representations in Section

17.5. Over the last two years, 3D Gaussian Splatting has rapidly become a fourth option: it represents a scene with a great many three-dimensional Gaussians carrying position, shape, color, and transparency, so that it can be fitted directly from data like a point cloud and yet rendered into photorealistic images like a mesh.

This line has already reached the LiDAR field. Splat-LOAM in 2025 takes Gaussian splatting directly as the representation for LiDAR odometry and mapping, while work such as LiV-GS integrates LiDAR and camera into one Gaussian splatting SLAM for outdoor scenes. Students may notice an interesting coincidence: NDT describes a surface with a Gaussian inside a voxel (Figure 17-2 of this chapter), and Gaussian splatting describes the appearance of a scene with Gaussians in space—twenty years apart, both arrived at the same answer of describing the world with a collection of Gaussian distributions.

4. The Division of Labor between Learned Registration and Traditional Methods

The last development is that the place of deep learning in the registration problem has gradually become clear. The strength of learned methods lies in the step of finding correspondences—a learned three-dimensional descriptor can find the correspondence between two clouds outright even with very little overlap and a very poor initial guess, exactly where ICP and NDT are weakest (the cliff in Figure 17-3(b)). But once the correspondences are found, solving for the transformation remains the least squares problem of Chapter 12, at which traditional methods are both fast and accurate.

The practical division of labor at present is therefore quite clear: global registration (with no initial value, as in relocalization and loop closure) tends to use learned

descriptors or random sample consensus (RANSAC); local refinement (where a decent initial value already exists, as between consecutive frames) remains the domain of ICP and NDT. And the existence of KISS-ICP reminds us that in the particular task of LiDAR odometry over consecutive scans the initial value has always been good enough, which is why traditional methods have to this day had no reason to be replaced.

The Common Thread of the Four Developments

Method (KISS-ICP): complexity does not equal effect, and getting the basics right is more useful than changing algorithms.

Engineering (small_gicp): different implementations of one algorithm can differ several-fold, and performance is a question of implementation and not only of theory.

Representation (Gaussian splatting): map representation expands from points, voxels, and meshes to a collection of Gaussians, echoing NDT's idea across the years.

Learning (descriptors): what deep learning supplies is finding correspondences, not solving for the transformation; the two divide the labor rather than one replacing the other.

The common thread: not one of this chapter's four tools has been retired, and what has changed is how they are used and the quality with which they are engineered. Build the foundation solidly and every later change becomes readable.

Carrying Forward

This chapter has assembled the tools for processing three-dimensional point clouds:

downsampling brings the data volume back within reach, ICP and NDT let neighboring scans be aligned, and map representation and colorization let the result be used and displayed. The three sets of experiments in Figures 17-1 through 17-3 verify the law, the principle, and the limits of these three things respectively—especially the red line of Figure 17-3(c), which reminds us that some errors are not the algorithm's fault but the absence of geometric information itself.

But register neighboring scans frame by frame and accumulate, and the error still grows with distance—this is the three-dimensional version of the drift problem of Chapter 7. The LOAM family of the next chapter gives an elegant answer: rather than registering the whole cloud, first extract two classes of feature from the scan, edge points and planar points, and register only the features. This idea makes real-time three-dimensional LiDAR odometry possible, and it opens the true subject of Part V.

Formulas and Procedures

1. Estimating the Point Count of Voxel Downsampling

Apply a voxel of side length d to a point cloud and the output point count is about equal to the number of occupied voxels. Because of the nature of three-dimensional space:

$$\text{output point count} \propto 1 \;/\; d^2 \quad \text{(a surface-type point cloud)}$$

Note that this is the square and not the cube—because what the LiDAR scans is a surface and not a solid volume, so the points lie mainly on a two-dimensional surface. The practical experience: double the voxel side length and the point count falls to about a quarter. Only if the point cloud filled a three-dimensional volume (as in medical imaging) would the relation be cubic.

2. The Objective Function of ICP

Point-to-point ICP seeks a rotation R and a translation t that minimize the sum of squared distances between paired points:

$$(R^*, t^*) = \operatorname{argmin} \Sigma_i \, \| \, (R \cdot p_i + t) - q_i \, \|^2$$

where p_i is a point of the source cloud and q_i its nearest point in the target cloud. Point-to-plane ICP instead takes the error as the projection along the target point's normal n_i:

$$(R^*, t^*) = \operatorname{argmin} \Sigma_i \, ((\, (R \cdot p_i + t) - q_i \,) \cdot n_i \,)^2$$

The difference is that point-to-point penalizes departure in every direction while point-to-plane penalizes only departure perpendicular to the surface, letting points slide along the plane—which is exactly why it converges faster.

3. The Probabilistic Objective of NDT

NDT describes each voxel of the target cloud with a Gaussian distribution (μ, Σ). A source point p transformed by T falls in some voxel, where its score is:

$$score(p) \propto \exp(- (\, (T \cdot p - \mu \,)^{\top} \Sigma^{-1} \, (T \cdot p - \mu \,) \,) / 2 \,)$$

The registration objective is to maximize the total score over all points:

$$T^* = \operatorname{argmax} \Sigma_i \, score(T \cdot p_i)$$

The term $(x - \mu)^{\top} \Sigma^{-1} (x - \mu)$ in the expression is precisely the core term of the multivariate Gaussian distribution of Chapter 11 (the Mahalanobis distance). You can see that the probabilistic mathematics of Part III becomes directly a tool for 3D registration here.

4. The Complete 3D Mapping Procedure

Figure 17-4 The Complete Flow from LiDAR Scan to 3D Map

① The 3D LiDAR outputs a raw point cloud (several hundred thousand to over a million points per second)

↓

② Preprocessing: outlier rejection, range cropping, motion compensation

↓

③ Voxel downsampling: reduce the point count and even out the distribution (Section 17.2)

↓

④ Feature / normal estimation: prepare for point-to-plane ICP or later processing

↓

⑤ Registration: coarse NDT → fine ICP (Sections 17.3 and 17.4)

with odometry / IMU supplying the initial guess

↓

⑥ Accumulation and stitching: merge the new scan into the global map

↓

⑦ Loop closure detection + graph optimization: remove accumulated drift (Chapters 12 and 15)

↓

⑧ Post-processing: convert to an octree / mesh, colorize (Section 17.6), output

Cases

Case 17-1 Why Walls Grow Thick: the Quality Indicator of 3D Mapping

Students doing 3D mapping for the first time often meet a puzzling phenomenon: having scanned a room and looked down on the point cloud from above, a wall that ought to be a thin single layer has become a slab more than ten centimeters thick; sometimes two parallel wall surfaces appear with a gap between them.

The cause is direct: there is registration error. The robot scans the same wall at different times from different positions, and if the pose estimate is accurate these scans should overlap perfectly into one layer; if the pose is off, the same wall is drawn at slightly different places and thickens when stacked. Let the error grow beyond a point and an obvious double wall appears (this is the three-dimensional version of ghosting, corresponding to the 2D symptom of Section 13.7).

Wall thickness is therefore the most practical quality indicator in 3D mapping: measure the spread of a stretch of wall point cloud in the direction perpendicular to the wall, and the ideal value should approach the measurement accuracy of the LiDAR itself (a few centimeters). A thickness clearly in excess of that means something is wrong with the registration or with SLAM. The check is simple, intuitive, and needs no ground truth, and students are strongly urged to make it after every mapping run—it tells you at once whether the map is usable.

Case 17-2 From Tunnels to Mines: the Three-Dimensional Challenge of Degenerate Scenes

Section 13.6 discussed how a long corridor defeats 2D scan matching, and the problem only grows worse in three dimensions. Tunnels, underground galleries, and

long corridors share one feature: along the long axis, the geometry of the environment is almost unchanging. The point cloud can slide freely in that direction without increasing the registration error—this is called degeneracy.

Researchers have developed a number of remedies for scenes of this kind. One team working in tunnel environments adopted the combined procedure of first voxel downsampling to preserve the overall structure while reducing the point count, then solving for the coordinate transformation with NDT while optimizing the voxel resolution for the features of the environment, and finally introducing a KD-tree into ICP to accelerate the point-pair search and solving with the Gauss–Newton method; measurement showed the best results at a voxel resolution of 0.5 meters.

This case offers three lessons. First, degenerate scenes are a difficulty common to all 3D SLAM, not something you did wrong. Second, NDT and ICP each have their strengths, and using them in combination often beats using either alone. Third, parameters must be tuned to the scene; no setting holds everywhere. It also foreshadows how the LOAM family of Chapter 18 attacks this problem by means of feature extraction.

Case 17-3 Choosing Tools: PCL and Open3D

Do 3D point cloud processing and you will almost certainly use two open-source libraries: PCL (the Point Cloud Library) and Open3D. The former is an established, mainly C++ tool, extremely complete in function and standard equipment in the ROS ecosystem; the latter, developed by Intel, is known for its friendly Python interface and convenient visualization, and has become quite popular in research and teaching in recent years.

Open3D provides implementations of several advanced surface registration methods, including pairwise global registration, pairwise local refinement, and multiway registration using pose graph optimization—students should recognize the last at once as the graph optimization of Chapter 12, only applied to stitching point clouds.

On performance, each has its wins. Benchmarks comparing the registration speed of different libraries show that point-to-plane ICP is usually faster than point-to-point, and that NDT's speed is quite competitive as well; recent years have also brought lightweight registration libraries designed for parallelism, which parallelize the preprocessing steps of downsampling, KD-tree construction, and normal estimation throughout, with multithreaded versions several times faster than single-threaded ones.

The advice for readers of this book: use Open3D for teaching and rapid prototyping (quick to pick up in Python, with good visualization), use PCL when integrating into a ROS system, and consider the new generation of parallelized libraries for a project that chases performance. This follows the same line as the learning path of Chapter 8—simulate first, then cheap hardware, and pursue accuracy last.

Hands-On Activities

Exercise 17-1 Point Cloud Processing and Registration

Purpose: carry out the loading, downsampling, visualization, and registration of a point cloud with your own hands, and understand the behavioral differences between ICP and NDT and the influence of their parameters. This exercise uses Python with Open3D and can be completed without 3D LiDAR hardware.

Part One: Point Cloud Basics and Downsampling (everyone)

1. Install Open3D, load one of the built-in sample point clouds (or download an indoor scan from a public dataset), and rotate it in an interactive window to get a feel for what a three-dimensional point cloud looks like.
2. Print the basic information of the point cloud: point count, coordinate range (bounding box), and whether it carries color and normals.
3. Voxel downsampling experiment: downsample at side lengths of 0.01, 0.05, and 0.2 meters, record the output point count each time, and verify whether it follows the square relation of Equation 1.
4. Visual comparison: display the original cloud and the three downsampled results side by side, and observe at what voxel size details such as wall corners and table legs begin to disappear.
5. Estimate normals and display them as colors (color corresponding to normal direction), and observe how walls, floor, and ceiling are clearly distinguished—this is the preparation for point-to-plane ICP.

Part Two: ICP Registration (everyone)

6. Take a point cloud, copy it, and apply a known rotation and translation (20 degrees about the Z axis and a translation of 0.3 meters, say) to serve as the source cloud.
7. Register with point-to-point ICP, compare the estimated transformation matrix with the truth, compute the error, and record the iteration count and elapsed time.
8. Switch to point-to-plane ICP (normals must be estimated first), compare the differences in convergence speed and accuracy, and verify the argument of Section 17.3.

9. Initial value experiment: increase the initial offset step by step (5, 20, 60, and 120 degrees), find the critical point at which ICP begins to fail, and verify for yourself that ICP needs a good initial guess.

Part Three: NDT and Combined Registration (university level)

10. Perform the same registration with NDT, compare its speed, accuracy, and tolerance of the initial value with ICP, and fill in your own version of Table 17-2.
11. Voxel resolution experiment: vary NDT's voxel size (0.2, 0.5, and 1.0 meters), observe the effect on success rate and accuracy, and compare with the discussion in Case 17-2.
12. Implement the combined procedure of coarse NDT followed by fine ICP, and compare it with using either method alone.
13. Degenerate scene test: synthesize a corridor point cloud of your own (two parallel long walls plus a floor), test whether ICP registration along the corridor loses accuracy, and reproduce the phenomenon of structural degeneracy.

Discussion Questions

1. Increasing the voxel side length from 0.01 to 0.2 meters, by what factor does the point count fall? Does it follow the square relation or the cube relation? Why?
2. How much faster is point-to-plane ICP than point-to-point? Explain the difference from the principles of Section 17.3.
3. At what initial offset does ICP begin to fail? What does this mean for a real robot system (hint: the role of odometry and the IMU)?
4. In the corridor test, in which direction does the registration lose accuracy? Is this the same problem as the 2D corridor predicament of Section 13.6?

5. (University level) If you had to choose a registration method for a campus inspection robot dog, would you choose ICP, NDT, or a combination of the two? Give your reasons and your considerations about the site.

Extended Challenge (university level)

First: download a public 3D LiDAR dataset (a sequence from KITTI, say) and implement a simple LiDAR odometry—register consecutive pairs of frames and accumulate the poses, then compare the estimated trajectory with the truth and observe how drift accumulates with distance (exactly the problem Chapter 18's LOAM sets out to solve). Second: if your school has a 3D LiDAR, actually scan a laboratory, carry out the full procedure of Figure 17-4, output a colored point cloud model, and then assess its quality by the wall thickness method of Case 17-1. This result can serve directly as the basis of Project Two in Chapter 31.

Further Reading

1. P. Besl and N. McKay, "A Method for Registration of 3-D Shapes," IEEE TPAMI, 1992. (The original ICP paper, the theoretical source shared by Chapters 13 and 17.)
2. P. Biber and W. Straßer, "The Normal Distributions Transform: A New Approach to Laser Scan Matching," IROS, 2003. (The original NDT paper.)
3. Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A Modern Library for 3D Data Processing," arXiv, 2018. (The paper introducing Open3D, with examples of registration workflows.)
4. Point Cloud Library (PCL) official tutorials (pointclouds.org). (Complete examples of downsampling, normal estimation, and registration.)

5. Open3D official documentation and Python tutorials (open3d.org). (The main tool for this chapter's exercises, friendly for teaching.)
6. A. Hornung et al., "OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees," Autonomous Robots, 2013. (The representative work on 3D occupancy grid maps, echoing Section 17.5.)
7. X. Li et al., "A Tutorial Review on Point Cloud Registrations: Principle, Classification, Comparison, and Technology Challenges," Mathematical Problems in Engineering, 2021. (A complete review of point cloud registration, with a comparison of downsampling strategies.)

Summary

1. A 3D point cloud is a set made up of a large number of three-dimensional points, which may carry intensity, color, ring number, and timestamp; it has the three characteristics of being unordered, sparse and uneven, and enormous in volume, and unlike an occupancy grid map it cannot distinguish definitely empty from unknown.
2. Voxel downsampling cuts space into cubic cells and takes the centroid of each as its representative point, which greatly reduces the count and evens out the distribution; the centroid is more accurate than the cell center, and the approximate version may produce spurious points. The voxel side length should be slightly smaller than the smallest feature size of interest.
3. ICP registers point clouds by finding correspondences, solving for a transformation, and iterating, accelerating the nearest-neighbor search with a KD-tree; point-to-plane ICP converges faster because it penalizes only departure perpendicular to the surface, and it is the first choice in practice. Its

three limitations are the need for a good initial value, a fear of structural degeneracy, and sensitivity to outliers.

4. NDT turns the target point cloud into Gaussian distributions within voxels, forming a continuous differentiable probability field that needs no search for corresponding points; it is faster than ICP and less sensitive to the initial value but slightly less accurate, and its key parameter is the voxel resolution, which must be tuned to the scale of the scene.
5. A 3D map has three representations: the raw point cloud (high in fidelity), the octree occupancy grid (able to distinguish three states, suited to navigation), and the mesh model (small in file size, suited to display); the indoor scanning procedure is to plan the route, collect, preprocess, register, optimize globally, and post-process and output.
6. Point cloud colorization uses the camera projection model of Chapter 9 to assign image RGB values to three-dimensional points, the key being accurate intrinsics and extrinsics and time synchronization; a colored point cloud improves readability, provides semantic cues, and serves the preservation of cultural assets and the detection of defects.
7. 3D modeling is applied to factory digital twins, campus surveying and navigation maps, and exhibition venues and the preservation of monuments; LiDAR's indifference to illumination makes it superior to a camera at low-light and textureless sites, and advances in edge computing platforms have already taken 3D SLAM off the workstation.

This chapter has assembled the tools for processing three-dimensional point clouds: downsampling, registration, map representation, and colorization. But register

neighboring scans frame by frame and accumulate, and the error still grows with distance—exactly the three-dimensional version of the drift problem of Chapter 7. The LOAM of the next chapter gives a handsome answer: it registers not all the points but intelligently extracted edge features and planar features, and with a two-layer architecture of high-frequency odometry plus low-frequency mapping it strikes an excellent balance between real-time operation and accuracy. This is the technical backbone of 3D localization for autonomous inspection robots and self-driving cars.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 18

LOAM and LiDAR Odometry

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

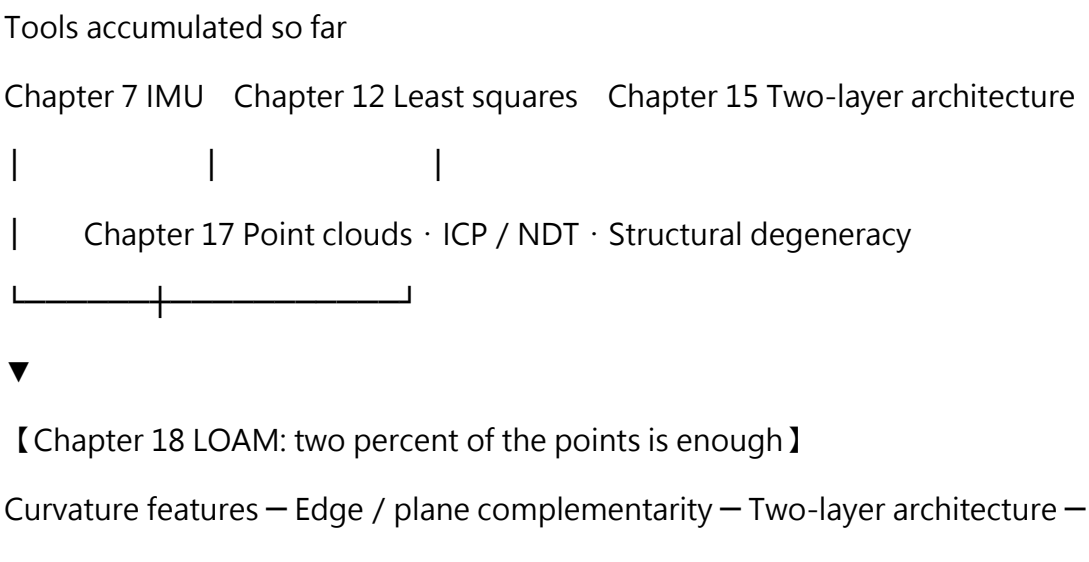
University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 17 assembled the tools for processing point clouds, but it also left an awkwardness: both ICP and NDT spend time finding correspondences, and a single 3D scan routinely holds tens of thousands of points, so registering frame by frame and accumulating is both slow and persistently drifting. The answer LOAM gives in this chapter sounds almost like a trick—do not use all the points, use only the two or three percent that have character. But there is a rigorous geometric reason behind the trick, and understanding that reason is the key to the whole of Part V.

The structure of this chapter echoes LOAM's two great contributions. Section 18.2 is about which points to pick, using curvature as the yardstick; Section 18.3 is about how to arrange the computation, using a two-layer high-frequency and low-frequency architecture—which students will find is the same design philosophy as Cartographer's local SLAM plus global SLAM in Chapter 15, incarnated in another dimension. From Section 18.4 onward come the descendants and applications of this architecture, leading all the way to the inspection projects of Part VII.

Figure 18-0 The Link Map of Chapter 18



compensation

▼

LeGO-LOAM ─ LIO-SAM ─ Fast-LIO (tightly coupled LiDAR-inertial)

▼

Part VI Visual SLAM Part VII Inspection and field applications

Learning Objectives

In the previous chapter we learned how to downsample and register point clouds, but accumulating frame-by-frame ICP over neighboring scans is both slow and persistently accumulating error. LOAM gives a handsome answer—it registers not all the points but intelligently extracted edge and planar features, and with a two-layer architecture of high-frequency odometry plus low-frequency mapping it balances real-time operation against accuracy. This is the technical backbone of 3D localization for autonomous inspection robots and self-driving cars. After studying this chapter, you should be able to:

1. Explain the central problem LOAM addresses and its place in the history of 3D LiDAR SLAM.
2. Explain how curvature (roughness) distinguishes edge features from planar features, and why registration with feature points alone suffices.
3. Explain the two-layer architecture of high-frequency odometry and low-frequency mapping, and why it can serve real-time operation and accuracy at once.
4. Compare the principal improvements of later systems such as LeGO-LOAM, LIO-SAM, and Fast-LIO.

5. Explain the motivation, the advantages, and the implementation essentials of tightly coupling LiDAR with an IMU.
6. Explain how 3D SLAM is applied to autonomous inspection robots.
7. Plan an inspection project set on a university campus (corridors, elevators, laboratories).

Core Concepts

中文名詞	English Term	Explanation
光達里程計與建圖	LOAM	The classic method of real-time LiDAR localization and mapping using edge and planar features.
光達里程計	LiDAR Odometry	The technique of inferring a robot's motion from consecutive LiDAR scans alone.
曲率 / 粗糙度	Curvature / Smoothness	The measure of how uneven the local neighborhood of a point is, used to classify features.
邊緣特徵	Edge Feature	A point of high curvature, lying at something sharp such as a corner, a column, or a tree trunk.
平面特徵	Planar Feature	A point of low curvature, lying on something flat such as a wall or the ground.
點到線距離	Point-to-Edge Distance	The error quantity used when registering edge features.
點到面距離	Point-to-Plane Distance	The error quantity used when registering planar features.
運動變形	Motion Distortion	The warping of one frame of point cloud caused by vehicle motion during the

中文名詞	English Term	Explanation
		LiDAR's revolution.
地面分離	Ground Segmentation	Treating ground points separately from non-ground points, the key to LeGO-LOAM.
兩步最佳化	Two-step Optimization	Solving some degrees of freedom from the ground plane first and the rest from edges afterward.
光達慣性里程計	LIO	An odometry method tightly coupling LiDAR with an IMU.
迭代擴展卡爾曼濾波	iEKF	The efficient tightly coupled estimation architecture adopted by Fast-LIO.

18.1 LOAM: LiDAR Odometry and Mapping

LOAM was proposed in 2014 by Ji Zhang and Sanjiv Singh at the CMU Robotics Institute, in a paper titled "LOAM: Lidar Odometry and Mapping in Real-time." It is the most influential single work in the field of 3D LiDAR SLAM—almost every mainstream system since has been built on its ideas.

The Problem It Addresses

The end of Chapter 17 left a difficulty: registering consecutive 3D scans frame by frame with ICP and accumulating meets two bottlenecks. The first is speed—with hundreds of thousands of points per frame, the nearest-neighbor search of ICP is a heavy burden and hard to keep real-time. When a scan contains a great many points, ICP may bear too high a computational cost. The second is accuracy—accumulate frame by frame and the error drifts continually.

LOAM's central insight is that not all the points are needed. What really supplies geometric constraint in an environment are the points with character—wall corners, columns, tree trunks (edges), and wall surfaces and floors (planes). Capture these feature points for registration and one can reach comparable or even better accuracy with less than one percent of the original point count.

LOAM's Two Great Contributions

- Feature extraction: classify points into edge features and planar features by curvature (roughness), greatly reducing the number of points that enter the computation (Section 18.2).

- Two-layer architecture: split the problem into a high-frequency but coarse odometry and a low-frequency but accurate mapping, two parallel tasks that serve real-time operation and accuracy together (Section 18.3).

LOAM splits the SLAM problem into two parallel tasks—odometry and mapping—and by extracting edge and planar features from the LiDAR scan and handling feature correspondence in a long-term optimization, achieves high accuracy at low computational cost in structured environments. This architecture of feature extraction plus KD-tree correspondence search has since been adopted as the core procedure by a great deal of LiDAR odometry research.

Its Place in History

Since Zhang and Singh proposed LOAM, LiDAR-based SLAM research has advanced rapidly; LOAM extracts edge and planar features by curvature and estimates a 6-degree-of-freedom pose by scan-to-map matching with a KD-tree, achieving computational efficiency and accuracy together by separating odometry and mapping into different execution frequencies.

For readers of this book, understanding LOAM is tantamount to holding the key to all of Part V: the LeGO-LOAM, LIO-SAM, and Fast-LIO of Section 18.4 are all its descendants, and research on university autonomous inspection robots builds on this same technical line.

18.2 Edge Features and Planar Features

This section covers LOAM's most recognizable design: how to pick the few genuinely useful feature points out of hundreds of thousands.

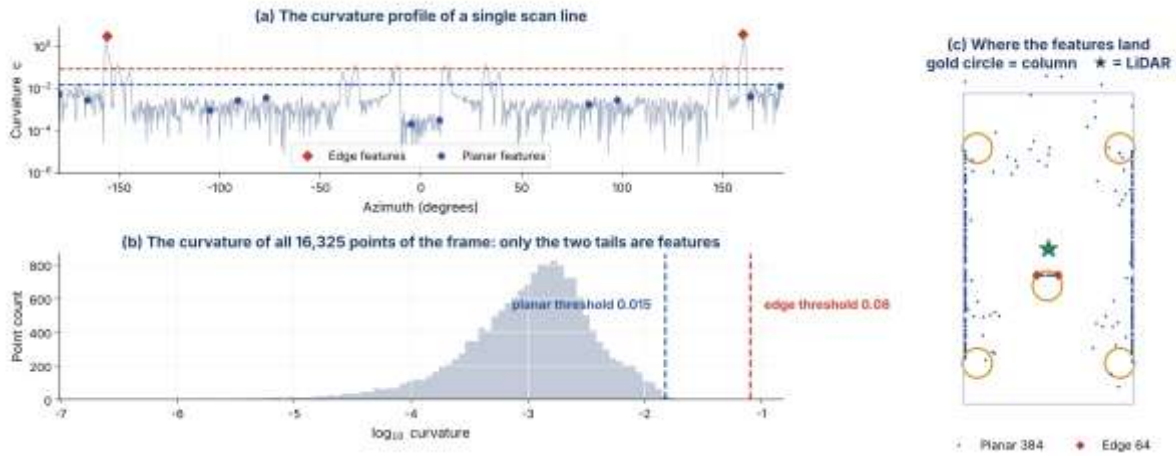
Curvature: the Yardstick for Identifying Features

LOAM classifies each point by a simple and ingenious measure—local roughness (commonly called curvature). The method is to take a point together with its neighbors on the same scan line and compute how far this point departs from the mean position of its neighbors.

- High roughness: the point is not collinear with the arrangement of its neighbors and lies at something sharp—this is an edge feature. Typical examples: wall corners, the edges of columns, tree trunks, and poles.
- Low roughness: the point lies nearly in a straight line with its neighbors, on something flat—this is a planar feature. Typical examples: wall surfaces, floors, ceilings, and large flat panels.

In each frame of the point cloud, only the few points of highest roughness need be taken as edge features and the few of lowest roughness as planar features. The illustrations in the LeGO-LOAM paper show this process clearly: after segmentation of the raw point cloud, edge and planar features are marked in blue and yellow respectively.

Figure 18-1 Curvature as a Yardstick: from One Scan Line to a Handful of Feature Points



This figure comes from a genuine simulation: a corridor 7 meters wide and 80 meters

long, with a colonnade, door frames, and a square column along it, scanned for one frame by a 32-beam LiDAR for 32,603 valid points. Panel (a) takes one of the scan lines and plots the curvature computed by Equation 1 against azimuth (the vertical axis on a logarithmic scale). For most of its length the curve murmurs along near 10^{-3}—that is the flat wall surface; and each time a column or a door frame is scanned a spike appears, leaping two orders of magnitude in an instant.

Panel (b) plots the curvature of all the points of the whole frame as a histogram, showing a single-peaked distribution with its peak near 10^{-3}. What LOAM wants is not this peak but the two tails of the distribution: the right tail (high curvature) is the edge features and the left tail (low curvature) the planar features. The great lump in the middle, which holds the overwhelming majority of the points, contributes almost nothing further to registration—this is the most direct visual evidence for what Case 18-1 calls the redundancy of most points.

Panel (c) verifies whether this screening really picks the right things. The red diamonds are edge features, and they can be seen falling accurately on the columns circled in gold and at the positions of the door frames and the square column; the small blue dots are planar features, spread evenly over the two long walls and the floor. The final numbers: 64 edge features and 384 planar features, together only 2.7 percent of the original point count. In other words, LOAM throws away 97 percent of the data, and what it throws away is precisely what could supply no new information.

This book also measured the price of that reduction. On the same scan, point-to-plane registration with all 32,513 points gave an angular error of 0.031 degrees, a translation error of 1.1 millimeters, and a time of 0.18 seconds; with only 768 planar features (2.4 percent), the angular error was 0.042 degrees, the translation error 1.3 millimeters, and

the time 0.038 seconds. Accuracy is barely lost while the time falls to a fifth. This is the fundamental reason LOAM can run in real time.

The Curvature Thresholds and Those Two Rejections

The thresholds in this figure are $c > 0.08$ for edges and $c < 0.015$ for planes, with at most 2 edge points and 4 planar points taken from each subregion of each scan line, so that features do not crowd into one place (the even distribution of Section 18.2).

The original LOAM paper adds two rejections, which this figure also implements: first, points on an occlusion boundary (where the range jumps between neighbors, the farther side being in fact occluded and bound to look different in the next frame); second, points whose scan line is nearly parallel to the surface (where the measurement is extremely unstable).

The rejection rate is about 8 percent. Without these two, false spikes caused by measurement noise on a distant wall are mistaken for edge features—exactly what happened in this book's first implementation, with edge points scattered over a bare wall rather than on the columns.

The thresholds must be adjusted to the LiDAR's beam count and to the site. The fewer the beams and the coarser the angular resolution, the noisier the curvature and the wider the thresholds must be.

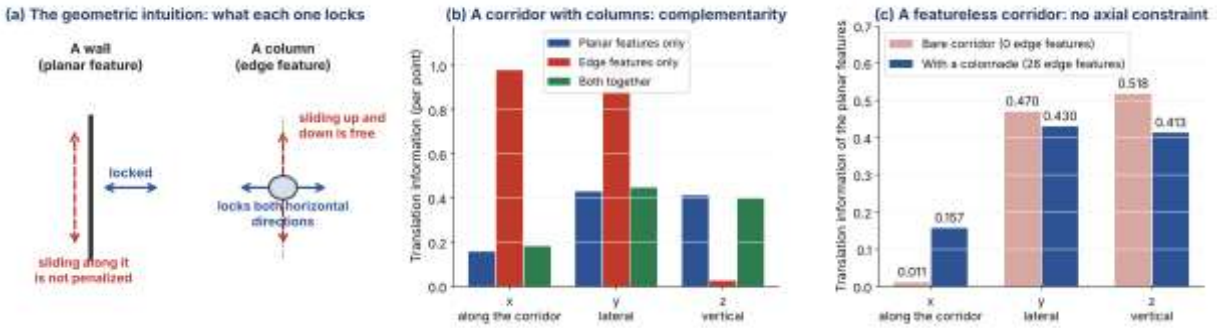
Why Both Kinds of Feature Are Needed

This is the most important idea in the section. The geometric constraints the two kinds of feature provide are complementary:

- Planar features constrain motion perpendicular to the plane. A wall can tell you how far you are from the wall but not how far you have walked along it—slide along the wall and the planar constraint does not change at all.
- Edge features constrain motion perpendicular to the edge line. A column pins your position in the two horizontal directions but has no say over motion up and down along the column.

Only by combining the two can all six degrees of freedom be pinned down. This also explains the structural degeneracy mentioned in Section 17.3: in a long corridor the walls all supply planar constraints in the same direction and edge features along the corridor are lacking, so the displacement along the corridor cannot be determined.

Figure 18-2 Why Neither Kind of Feature Can Be Spared: the Directions They Lock Are Exactly Complementary



This is the most important idea in the chapter, and it can be quantified exactly. The method is to write the sensitivity of the registration error to pose as an information matrix—the greater the information in some direction, the more a small departure in that direction raises the error, and so the more tightly that direction is locked. Panel (a) first gives the geometric intuition: a wall can lock only the one direction perpendicular to it, and sliding along the wall surface is not penalized at all; a column locks the two horizontal directions but has no say over sliding up and down along it.

Panel (b) is the measurement. With planar features alone, the information along the corridor is 0.157 while laterally it is 0.430 and vertically 0.413—along the corridor is clearly the weakest link. With edge features alone the situation is precisely reversed: the two horizontal directions are 0.980 and 0.994, all but saturated, while the vertical is only 0.026, which amounts to no constraint at all. The shapes of the two sets of numbers are complements of each other, which is the quantified version of Section 18.2's statement that the geometric constraints the two kinds of feature provide are complementary.

Panel (c) then pushes the conclusion to an extreme and answers the question left over from Chapter 17 along the way. Remove all the columns and door frames from the corridor, leaving a purely empty passage, and recompute: the information of the planar features along the corridor is only 0.011, against 0.470 laterally and 0.518 vertically—a difference of more than forty-fold. More telling still, in this empty corridor the number of edge features found by curvature screening is zero: the environment holds no sharp structure whatever that could supply an axial constraint.

This is the exact description, in LOAM's framework, of the structural degeneracy of Section 17.3: it is not that the algorithm is not good enough but that the geometric information itself does not exist. And put the colonnade back and 26 edge features appear at once, while the information of the planar features along the corridor rises from 0.011 to 0.157 (the door frames and the square column contribute surfaces perpendicular to the corridor). This also foreshadows the conclusion of Section 18.5—when the environment itself can supply no constraint, the only recourse is to take it from another sensor, and the IMU is that source.

How to Read the Information Number

It is the average constraint strength supplied by each feature point, dimensionless and between 0 and 1. A value of 1 means the direction is completely locked and 0 that it is completely free.

The information matrix of planar features is $\Sigma\, n\, n^T$ (n being the surface normal): whichever way the normal points is the direction locked.

The information matrix of edge features is $\Sigma\,(I - u\,u^T)$ (u being the direction of the edge line): all directions perpendicular to the line are locked. The columns in this figure are all vertical, so u is almost always the z axis and what is locked is the horizontal plane.

This analysis is also the basis of degeneracy detection in modern systems: compute the smallest eigenvalue of the information matrix in real time, and once it falls below a threshold the system knows it is walking into a degenerate scene and can raise its trust in the IMU (see Section 18.7).

Even Distribution: Avoiding Clustered Features

LOAM has one more practical detail: divide each scan line into several subregions and select a fixed number of feature points within each. This ensures that features are distributed evenly across the whole field of view rather than all clustered in one corner—if features cluster on one side the geometric condition of the registration is poor (much the same idea as the DOP of GNSS in Chapter 5).

The Definition of the Registration Error

Once features are extracted, how they are registered also depends on the feature type:

- Edge features: find the corresponding edge line in the previous frame (or in the map), determined by two neighboring edge points, and define the error as the perpendicular distance from the point to that line.
- Planar features: find the corresponding plane in the previous frame (or in the map), determined by three neighboring planar points, and define the error as the perpendicular distance from the point to that plane.

Minimize the sum of squared errors over all feature points and the best pose transformation follows—which is again the nonlinear least squares problem of Chapter 12. Students should feel by now that the mathematical tools of Part III appear again and again in every concrete system.

18.3 High-Frequency Odometry and Low-Frequency Map Optimization

LOAM's second core innovation is to split SLAM into two tasks that run in parallel at different frequencies. This architectural design is a classic and deserves careful study.

The Division of Labor between the Two Tasks

- LiDAR odometry (high frequency): compare the features of two neighboring scans and quickly estimate how far I moved in this tenth of a second. It uses only two frames, so it is light and fast but lower in accuracy and subject to drift.
- LiDAR mapping (low frequency): match the current scan against the accumulated global map for a finer registration. The map holds a great deal of historical information so the registration is more accurate, but it is heavy and runs at a low frequency.

In LOAM's original design the mapping module projects the point cloud onto the map at a frequency of 1 Hz; finally the pose transformations published by the two

algorithms are integrated to produce an output at about 10 Hz, representing the pose of the LiDAR relative to the map.

Figure 18-3 LOAM's Two-Layer Architecture

【LiDAR odometry】high frequency (about 10 Hz)

Match features between two neighboring frames → quickly estimate the relative motion

Characteristics: real-time, lightweight, lower in accuracy, subject to drift

↓ supplies the motion estimate and the initial guess

【LiDAR mapping】low frequency (about 1 Hz)

Match the current scan's features against the global map → fine registration

Characteristics: accurate, heavy to compute, low in frequency

↓

【Pose integration】

Integrate the two transformations → output an accurate pose at about 10 Hz

Why This Design Is Clever

The beauty of this architecture is that it keeps real-time operation and accuracy from conflicting:

1. High-frequency odometry meets the need of real-time control. A robot's motion control loop needs a high-frequency pose and cannot wait a full second for an update.

2. Low-frequency mapping takes charge of removing drift. It has ample time for a fine registration and pulls back the error accumulated by the high-frequency odometry.
3. The high-frequency odometry supplies the low-frequency mapping with a good initial guess. This is exactly what Section 17.3 stressed—ICP-type methods need a good initial value to converge, and odometry provides it.

Students should notice that this architecture of a fast but coarse front end plus a slow but accurate back end is cut from the same cloth as Cartographer's local SLAM and global SLAM subsystems in Chapter 15. This is no coincidence but a general pattern in robot system design: treat problems of different natures on different timescales.

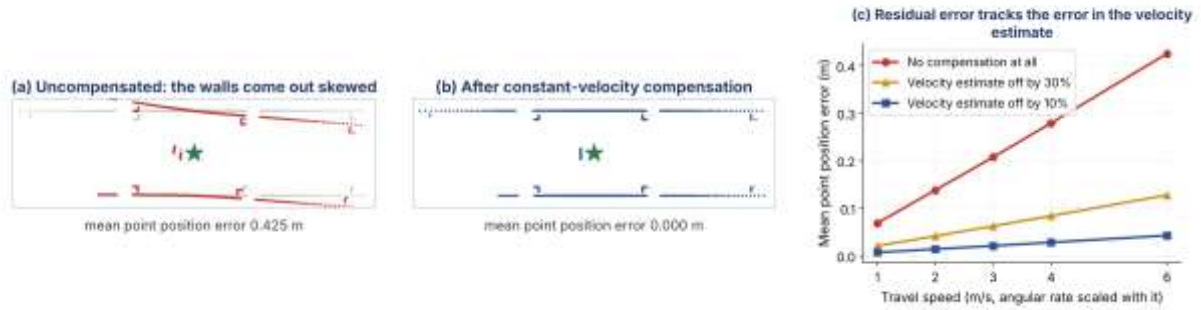
Motion Distortion Compensation

One further problem peculiar to 3D LiDAR must be handled. A mechanical LiDAR takes about 0.1 seconds (10 Hz) to complete one revolution, and during that time the vehicle itself is moving. The result is that the points of one frame of point cloud were in fact measured at different positions—the cloud is thereby warped, which is called motion distortion (mentioned in Section 8.4).

LOAM handles it by using the motion estimated by the odometry to project each point, according to the instant at which it was measured, back into the coordinate frame of the start of the sweep, removing the distortion. This is exactly why Section 17.1 mentioned that a point cloud needs timestamps. Later systems such as LIO-SAM also require explicitly that when the LiDAR spins at 10 Hz, each point's timestamp should vary between 0 and 0.1 seconds.

Figure 18-4 Motion Distortion: the Vehicle Moves While the LiDAR Completes a

Revolution



A mechanical LiDAR takes about 0.1 seconds to complete a revolution, and within that 0.1 seconds the vehicle itself is moving and turning. The result is that the points in one frame of point cloud were in fact measured at different poses. Take them naively as measurements made at one instant from one pose and the whole cloud is warped. Panel (a) simulates the phenomenon: the vehicle advances at 6 meters per second while turning at 1.6 radians per second; light gray is the true geometry and red the uncompensated point cloud—the two long walls are clearly skewed, with each point departing from its true position by 0.570 meters on average.

Panel (b) is after compensation. The method is exactly as described in Section 18.3: use a constant-velocity model to work out the pose at each point's measurement instant and project it back into the frame of the start of the sweep. The blue cloud coincides exactly with the gray truth and the mean error returns to zero. An honest note is in order here: it returns to zero because the velocity used for compensation is exactly the velocity used in the simulation—an ideal case, meant to verify that the compensation formula itself is written correctly.

The truly interesting panel is (c). The red line is the mean point position error with no compensation at all, and it is proportional to speed: 0.062 meters at 1 meter per second and 0.405 meters at 6. The amber and blue lines are the residual errors when the

velocity estimate is off by 30 percent and 10 percent respectively. The three lines display a clean proportionality: the residual error after compensation is about equal to the percentage error of the velocity estimate times the error with no compensation at all. Get the velocity 10 percent wrong and the residual error is 10 percent of the uncompensated value.

This proportionality ties Sections 18.3 and 18.5 together. It shows that the quality of motion compensation depends entirely on the quality of the velocity estimate, and the velocity estimate is exactly what an IMU does best—an update rate of several hundred hertz and extreme accuracy over short intervals. This is why virtually every modern LiDAR system carries an IMU: not only to supply the initial value for registration but to ensure that every frame of point cloud is already clean and undistorted before it enters registration at all.

Three Practical Points About Motion Distortion

The faster the speed and the sharper the turn, the worse the distortion; and turning matters more than driving straight, because rotation displaces distant points in proportion to their range. At 6 m/s the largest single-point error in this figure reaches 2.36 meters.

Compensation requires a timestamp for every point. This is exactly the use of the timestamp field mentioned in Section 17.1, and one of the reasons Case 18-3 says a missing required field stops the system from running.

A solid-state LiDAR has no rotating mechanism, but its scan still takes time and it suffers distortion just the same, only in a different pattern.

The constant-velocity model is the simplest assumption. It loses accuracy

under sharp acceleration or a sharp turn, and then the angular rate and acceleration measured by an IMU are more reliable than anything the model infers.

18.4 An Introduction to LeGO-LOAM, LIO-SAM, and Fast-LIO

LOAM opened the road, and the decade and more since has brought a great many improved systems. This section introduces three of the most representative descendants, each of which addresses a different weakness of LOAM.

1. LeGO-LOAM: Lightweight and Ground-Optimized

LeGO-LOAM stands for Lightweight and Ground-Optimized LiDAR Odometry and Mapping, designed for real-time 6-degree-of-freedom pose estimation on ground vehicles. It is lightweight, achieving real-time pose estimation on a low-power embedded system; and it is ground-optimized, because it makes good use of the presence of the ground plane in its segmentation and optimization steps.

Its two key improvements are:

- Point cloud segmentation and ground separation: segment the cloud first to filter out noise, then extract features to obtain discriminative planar and edge features. Segmentation effectively removes unstable points such as vegetation and noise—which matters especially outdoors.
- Two-step optimization: use a two-step Levenberg–Marquardt optimization, solving different components of the 6-degree-of-freedom transformation with planar and with edge features separately. Specifically, first use the planar features extracted from ground points to solve for the vertical displacement

and the roll and pitch angles, then use the edge features of the segmented cloud to estimate the horizontal displacement and the yaw angle, with the result of the first step as a constraint.

Measurements show that LeGO-LOAM achieves accuracy comparable to or better than LOAM at lower computational cost on ground vehicle datasets over varied terrain. It is optimized in particular for a 16-beam LiDAR mounted horizontally on a ground vehicle and assumes that a ground plane is always present in the scan—an assumption that limits its range of application but is also exactly what makes it efficient.

2. LIO-SAM: Tightly Coupled LiDAR-Inertial Odometry with a Factor Graph

LIO-SAM's innovation lies in tightly coupling the IMU with a factor graph and a Bayes tree, achieving motion-aware submap matching and integrating drift-aware loop closure detection.

- Tightly coupled IMU: add IMU preintegration (Chapter 7) to the graph as a factor, greatly improving robustness under fast motion.
- Factor graph architecture: use the GTSAM and iSAM2 incremental solvers introduced in Chapter 12 to fuse constraints from LiDAR, IMU, GPS, and loop closure efficiently.
- Loop closure detection: its loop closure function is adapted from LeGO-LOAM and is ICP-based; because ICP runs rather slowly, the authors recommend playing back at the original speed when testing.

An implementation limitation worth noting: like the original LOAM implementation, LIO-SAM can only be used with a nine-axis IMU (one able to supply roll, pitch, and yaw estimates), and it currently supports mechanical LiDARs only. Details of this kind are critical in project work, and students should always confirm them before

choosing hardware.

3. The Fast-LIO Family: Efficiency Taken to Its Limit

Fast-LIO takes another road: it performs no feature extraction, uses the raw point cloud directly, and estimates by tight coupling with an iterated extended Kalman filter (iEKF). Its key innovation is an efficient formula for computing the Kalman gain that makes the computational complexity depend on the dimension of the state rather than on the number of measurements, greatly raising its speed.

Fast-LIO2 goes further and introduces an incremental KD-tree (ikd-Tree) data structure that can update the map incrementally and efficiently, reaching very high processing rates and suiting platforms with limited computing resources and vehicles in fast motion (drones, say). This line proves that, given a good motion prior from an IMU, one can be both fast and accurate without feature extraction at all.

System	Year	Key Improvement	Suitable Scenes
LOAM	2014	Feature extraction + two-layer architecture	General purpose; the founding work
LeGO-LOAM	2018	Ground separation, two-step optimization, lightweight	Ground vehicles, embedded platforms
LIO-SAM	2020	Tight IMU coupling, factor graph, multi-source fusion	Sites needing high robustness and loop closure
Fast-LIO2	2021	Feature-free, iEKF, incremental KD-tree	Computation-limited platforms, fast motion

Table 18-1 The Main Systems of the LOAM Family Compared

18.5 Tightly Coupled LiDAR and IMU

Two of the three systems in Section 18.4 emphasize the IMU. This section explains why LiDAR SLAM depends more and more on the IMU, and what tight coupling means.

Three Weaknesses of LiDAR Used Alone

- Failure under fast motion: pure LiDAR methods degrade under fast dynamic motion; consecutive scans differ too much and registration struggles to converge (the old problem of Section 13.3, that ICP needs a good initial value).
- Geometrically degenerate environments: in geometrically degenerate environments such as long corridors or open spaces, insufficient geometric constraint makes pose estimation fail. This is the structural degeneracy of Section 17.3 appearing once again.
- Motion distortion: as described in Section 18.3, vehicle motion during a revolution warps the point cloud, and motion information is needed to compensate it.

The IMU Fills Exactly These Three Gaps

Recall Chapter 7: an IMU has a very high update rate (several hundred hertz), is extremely accurate over short intervals, and is entirely indifferent to the structure of the environment. It can therefore:

4. Supply a high-frequency motion estimate between two LiDAR frames as an excellent initial value for registration, solving the fast motion problem.
5. Supply an independent constraint in the geometrically degenerate direction—in a corridor the LiDAR cannot say how far it has traveled, but integrating the IMU's acceleration yields an estimate.
6. Supply an accurate attitude at the measurement instant of every point, for use in motion distortion compensation.

Conversely, the LiDAR gives the IMU what it most needs: a non-drifting pose observation with which to estimate and compensate the IMU's bias continually (Section 7.3). LiDAR-inertial odometry (LIO) has therefore drawn a great deal of attention; it fuses LiDAR measurements and IMU data tightly, combining LiDAR's robustness to changes in light with its ability to capture dense geometric information.

Loose Coupling vs. Tight Coupling

Section 7.6 discussed this pair of concepts in the context of visual-inertial SLAM, and it applies in full to the LiDAR field:

- Loose coupling: LiDAR SLAM and the IMU each compute a pose and the two are then fused with weights. Simple to implement, but the information has already been compressed and accuracy is limited.
- Tight coupling: put the IMU preintegration constraints and the LiDAR feature registration errors into one and the same optimization problem (or one and the same filter) and solve them together. Markedly better in accuracy, and the unanimous choice of modern systems.

LIO-SAM realizes tight coupling with a factor graph (the optimization road, Chapter 12) and Fast-LIO with an iEKF (the filtering road, Chapter 11)—different routes to the same destination, confirming once again that Chapters 11 and 12 are precisely the two trunk lines of SLAM.

Implementation Essentials

- Time synchronization: the timestamps of LiDAR and IMU must be aligned precisely, and an error of a few milliseconds affects accuracy appreciably (the engineering detail stressed repeatedly in Chapters 5 and 10).

- Extrinsic calibration: the relative mounting position and orientation of the IMU and the LiDAR must be measured accurately (the extrinsics of Chapter 10), or the fused result carries a systematic bias.
- IMU specification: as noted above, some systems require a nine-axis IMU. The quality of the IMU (the grade table of Chapter 7) also bears directly on how long it can hold out in a degenerate scene.

18.6 3D SLAM for Autonomous Inspection Robots

One of the most important practical applications of 3D LiDAR SLAM is the autonomous inspection robot. This section explains the requirements and design considerations of such systems.

Why Inspection Needs 3D Rather Than 2D

- Uneven terrain: an inspection robot (a robot dog above all) must go up and down stairs, cross thresholds, and traverse ramps. A 2D map cannot describe changes in height, nor judge whether a given step can be cleared.
- Three-dimensional obstacles: the suspended piping of a factory, the top edge of a rack in a machine room, and tree branches all lie off the 2D scan plane. The height blind spot of Chapter 8 causes collisions directly at an inspection site.
- Three-dimensional positions of inspection targets: to record that the second pipe on the east wall of the third floor is corroded requires three-dimensional coordinates rather than a planar position.
- Asset modeling: the by-product of an inspection is often a three-dimensional model of the facility (the digital twin of Section 17.7), which has value in itself.

A Typical System Architecture

Figure 18-5 The 3D SLAM System Architecture of an Autonomous Inspection Robot

【Sensing layer】3D LiDAR + IMU + (optionally) camera, GNSS, wheel encoders

↓

【Front end】feature extraction (edge / plane) or direct point cloud registration

↓

【Tightly coupled estimation】LIO: LiDAR constraints + IMU preintegration → high-frequency 6-DoF pose

↓

【Back end】factor graph optimization + loop closure detection (Chapter 12)

↓

【Map】3D point cloud map → converted to an octree for navigation (Section 17.5)

↓

【Application layer】path planning, obstacle avoidance, navigation to inspection points, image / thermal data collection

Platform Choice: Wheeled, Tracked, or Quadruped

- Wheeled: highest in efficiency and best in endurance, but confined to level ground and unable to climb stairs. Suited to factory floors and corridor inspection.

- Tracked: able to cross larger obstacles and ramps, and common at disaster sites and in outdoor inspection.
- Quadruped (robot dog): strongest in adaptation to terrain, able to climb stairs and step over obstacles, and a popular choice for inspection in recent years. The price is shorter endurance and higher cost, and the violent vibration of its gait is a severe test for SLAM—which is exactly why tight IMU coupling is required.

The Special Requirements of Inspection Tasks

- Repetition: inspection is a periodic task, walking the same route every day. The pattern of mapping once and localizing thereafter is therefore particularly apt (Case 14-3 of Chapter 14), and data from different times can be compared to find changes (a newly appeared crack, say).
- Accurate positioning at inspection points: the robot must stop at a specific position and photograph an instrument or piece of equipment from a specific angle. The positioning error must be held within a few centimeters, and this is the source of 3D SLAM's accuracy requirement.
- Long running times: an inspection task may last several hours, placing very high demands on drift control and system stability—loop closure detection is therefore indispensable.
- Data association: the images, thermal images, and audio collected during inspection all need to be tagged with an accurate three-dimensional position so that problems can be located in the digital twin.

18.7 After LOAM: Four Directions in LiDAR-Inertial Odometry, 2024–2026

LOAM was published in 2014, and for more than a decade its two great designs—feature extraction and the two-layer architecture—have been the standard skeleton of LiDAR SLAM. But over the last two years both of these pillars have been seriously challenged, and the challenges point in the same direction: computation has become cheap, so the compromises made for real-time operation back then may no longer pay. This section gathers four directions.

1. Feature Extraction Is Being Abandoned

The first direction was already foreshadowed in Section 18.4's introduction to Fast-LIO: perform no feature extraction and use the raw point cloud directly. This line has become all but mainstream in recent years. The reason is pragmatic—extracting features in 2014 was a way of reducing hundreds of thousands of points to a few thousand, without which the computation could not keep up; but on today's processors and GPUs, voxelized registration on tens of thousands of points directly is entirely feasible, and dropping the feature extraction step drops a set of thresholds that need tuning along with failure modes such as a column too slender to be caught.

This is the same current as the KISS-ICP of Section 17.8: rather than design ingenious screening rules, do the most basic registration solidly. Yet the analysis of Figure 18-2 in this chapter has not gone out of date—feature-free methods face geometric degeneracy just the same, only they no longer describe it in the vocabulary of edges and planes but look directly at the covariance of the voxel distribution. The concept is the same, differently worded.

2. Global Consistency Returns to the Front: GLIM

The second direction is a correction to the two-layer architecture itself. LOAM's low-

frequency mapping does correct drift, but once the front end has fixed the pose of a frame it never looks back. GLIM, published in 2024 in Robotics and Autonomous Systems by Kenji Koide's team at the National Institute of Advanced Industrial Science and Technology, argues that this fix-it-and-never-change design is precisely the root of collapse in degenerate scenes.

GLIM's approach has two keys: first, the odometry end adopts fixed-lag smoothing (a five-second window) so that the poses of the last few seconds continue to be corrected rather than settled at once; second, global optimization minimizes the registration error between submaps directly rather than first compressing them into Gaussian constraints on a pose graph. Add GPU-accelerated voxelized GICP and the whole system takes 15.1 milliseconds per frame of preprocessing and 29.3 milliseconds of odometry on ordinary equipment.

The effect is clearest in degenerate scenes. The paper tests a solid-state LiDAR on eight degenerate sequences, where GLIM's absolute trajectory error falls between 0.017 and 0.389 meters while FAST-LIO2's range is 0.297 to 15.551 meters—a fortyfold difference on the worst sequence. This contrast is worth remembering: under good conditions the various systems differ little, and what really separates them is always the worst case.

3. Degeneracy Is No Longer Merely Endured but Actively Detected

The third direction follows directly from the analysis of Figure 18-2 in this chapter. Since degeneracy can be quantified by the smallest eigenvalue of the information matrix, it can be computed in real time: once the constraint in some direction is found insufficient, the system automatically lowers its trust in the LiDAR and raises the weight on the IMU, or simply declines to update in that direction. Degeneracy-aware LiDAR-inertial odometry of this kind has been quite an active topic around 2025.

For a student project this offers a direction of modest difficulty and real value: there is no need to write a complete LIO of your own, only to add a monitor of the information matrix to an existing system, plot the smallest eigenvalue against time, and compare it with the site the robot actually walked, which yields a map of where in this building SLAM will fail. That is directly useful for the inspection project of Chapter 19.

4. From LiDAR-Inertial to LiDAR-Inertial-Visual

The fourth direction brings the camera into the same tightly coupled framework. Systems of the FAST-LIVO2 kind add a direct photometric image error on top of Fast-LIO's iterated Kalman filter architecture, forming a tight coupling of LiDAR, inertial, and visual sensing. The motivation agrees exactly with the analysis of sensor complementarity in Part II: LiDAR gives geometry, the camera gives texture and semantics, and the IMU gives high-frequency motion, and their failure modes barely overlap.

Its significance for degenerate scenes is especially worth noting. A long corridor leaves the LiDAR unable to say how far it has traveled, but the posters on the walls, the door plates, and the floor texture are clear features to a camera—which fills exactly the 0.011 gap of Figure 18-2(c). In other words, the visual SLAM of Part VI is not merely another parallel route but also the rescue plan for this chapter's systems in degenerate scenes.

The Common Thread of the Four Directions

Computation (feature-free): the screening done for real-time operation back then may not pay on today's hardware.

Architecture (GLIM): the front end should not fix a pose at once, and keeping room for correction is what carries a system through a degenerate stretch.

Diagnosis (degeneracy awareness): degeneracy turns from a disaster discovered after the fact into a metric measurable in real time.

Sensing (LiDAR-inertial-visual): a gap one sensor cannot fill is filled by another whose failure modes do not overlap with it.

The common thread: the three fundamentals of this chapter—curvature, complementary constraints, and motion compensation—have not been retired but have become the standard by which a new system is judged. Any new method must answer the same three questions: which points are useful, which direction has no constraint, and how far the vehicle moved during the sweep.

Carrying Forward

This chapter has completed the core technology of 3D LiDAR SLAM. From the point cloud processing of Chapter 17 to the LOAM family here, students now possess the full knowledge needed to localize and map in a three-dimensional world: Figure 18-1 shows which points are worth keeping, Figure 18-2 shows why neither kind of feature can be spared, and Figure 18-4 shows that motion during a sweep must be compensated. These three things are also the starting points from which every later system—LeGO-LOAM, LIO-SAM, Fast-LIO, GLIM—made its own improvements.

But the figure 0.011 in Figure 18-2(c) is a problem this chapter cannot solve on its own: when the geometry of the environment itself supplies no constraint, no LiDAR algorithm, however good, can conjure information. Section 18.5 gave the first answer—the IMU—and Part VI will give the second—the camera.

Before that, the next chapter must first answer another question: what happens when these techniques are put into a robot that has to run every day for a year? Chapter 19

takes 3D SLAM into the field of autonomous inspection and handles three things that algorithm textbooks do not teach but that decide whether a system succeeds: the ordering of inspection points, repeat positioning accuracy, and long-term map maintenance—and this chapter's degeneracy analysis in Figure 18-2 becomes, there, the concrete consequence of one inspection point being photographed crooked every other visit.

Formulas and Procedures

1. Local Roughness (Curvature)

For the i-th point p_i, take the set S of neighboring points on the same scan line; its roughness is defined as:

$$c = \left(1 \,/\, \left(|S| \cdot \|p_i\|\right)\right) \cdot \left\| \sum_{j \in S} \left(p_i - p_j\right) \right\|$$

The meaning of the expression is that the difference vectors between p_i and all its neighbors are summed. If p_i lies on something flat, the difference vectors to the left and right neighbors cancel and c is small; if it lies at a sharp corner, the difference vectors add in the same direction and c is large. The $\|p_i\|$ in the denominator normalizes the influence of distance (points far away are spaced more widely and should not be mistaken for edges on that account).

- c greater than a threshold → edge feature
- c less than a threshold → planar feature

2. Point-to-Line and Point-to-Plane Distance

The error of an edge feature is the perpendicular distance from the point to the corresponding edge line. Let the current edge point be p and the corresponding line

be determined by two points a and b in the map:

$$d_edge = \| (p - a) \times (p - b) \| / \| a - b \|$$

The numerator is the area of the parallelogram formed by the three points (computed as a cross product), and dividing by the length of the base gives the height—which is the distance from the point to the line.

The error of a planar feature is the perpendicular distance from the point to the corresponding plane. Let the plane be determined by three points a, b, and c in the map, with normal $n = (b - a) \times (c - a)$:

$$d_plane = | (p - a) \cdot n | / \| n \|$$

3. LOAM's Optimization Objective

Combine the errors of all feature points and seek the pose transformation T that minimizes the total:

$$T^* = \text{argmin} \left(\Sigma\, d_edge(T \cdot p_i)^2 + \Sigma\, d_plane(T \cdot p_j)^2 \right)$$

This is a nonlinear least squares problem, solved iteratively by the Levenberg–Marquardt method (Chapter 12). Students can see that from the abstract theory of Chapter 12, through the 2D scan matching of Chapter 13 and the ICP of Chapter 17, to the LOAM of this chapter, it is one body of mathematics applied to different data.

4. The Complete LOAM Processing Flow

Figure 18-6 LOAM's Processing Flow for Each Frame

① Receive one frame of 3D point cloud (with a timestamp and ring number for each point)

↓

② Motion distortion compensation: project each point back to the start of the sweep using the motion estimate

↓

③ Compute the local roughness c of every point

↓

④ Select features by region: in each subregion take the points of highest roughness (edges) and lowest (planes)

↓

⑤【Odometry, 10 Hz】match features against the previous frame → quickly estimate the relative motion

↓

⑥【Mapping, 1 Hz】match features against the global map → a fine pose, and update the map

↓

⑦ Integrate the two outputs → an accurate pose at about 10 Hz + a global 3D map

Cases

Case 18-1 Why Using One Percent of the Points Is More Accurate

When beginners hear that LOAM registers with only a few feature points, the instinctive reaction is usually: discard so much data and will accuracy not fall? The answer is unexpected—not only does it not fall, it often rises.

There are three reasons. First, most points are redundant: a flat wall carries tens of thousands of points, but the geometric constraint they supply is in fact only one—how far you are from this wall. Express that constraint with ten thousand points or with a hundred and the result is all but the same, while the computation differs a hundredfold.

Second, some points are harmful: leaves of vegetation, a fluttering plastic bag, a person walking by—these points are no longer in place in the next frame, and forcing a registration on them only injects error. LeGO-LOAM's point cloud segmentation step exists precisely to filter out unstable points of this kind.

Third, feature points are carefully chosen: points of very high or very low curvature happen to be the points most easily recognized correctly in the next frame—exactly the reason visual SLAM in Chapter 9 prefers corners ("not one point can be recovered on a white wall").

This case offers students a lesson beyond the technical: more data does not mean a better result, and what matters is the quality and representativeness of the data. The principle applies throughout machine learning, statistics, and engineering judgment.

Case 18-2 The Value of the Ground: LeGO-LOAM's Ingenuity

LeGO-LOAM has "ground-optimized" in its name, and this seemingly plain design is in fact very clever.

Consider: what is the ground to a robot driving on it? It is a planar feature that is always present, extremely stable, and enormous in area. And the constraint it supplies

is very definite—the ground tells the robot exactly how high above it the body is and whether the body is tilting forward, back, or to one side, that is, the three degrees of freedom of vertical displacement, roll, and pitch.

LeGO-LOAM therefore splits the six degrees of freedom into two groups solved separately: first use the planar features of ground points to solve for the vertical displacement and the roll and pitch angles, then use the other edge features to estimate the horizontal displacement and the yaw angle, taking the result of the first step as a constraint. The benefit is that each step has fewer unknowns to solve, so the problem converges better, computes faster, and is more stable.

This case demonstrates an important kind of engineering thinking: make good use of the prior knowledge of the site. That a ground vehicle is certainly on the ground is a free and reliable assumption, and not to use it is to waste it. The price, of course, is generality—the assumption does not hold at all on a drone. This again echoes the observation of Chapter 15: the design orientation of a technology is decided by the needs of its application.

Case 18-3 The Devil in the Details of Implementation: the Pitfalls of Hardware Specifications

Many students eagerly download LIO-SAM or LeGO-LOAM for a project only to be stuck at not being able to get it running. The reason is usually not the algorithm but a mismatch in hardware specification. Here are several real pitfalls:

- Too few IMU axes: like the original LOAM implementation, LIO-SAM can only be used with a nine-axis IMU (one able to supply roll, pitch, and yaw estimates). A student who bought a six-axis IMU is stopped outright.

- Unsupported LiDAR type: only mechanical LiDARs are currently supported. Solid-state LiDARs (Chapter 8) have a different data format and require extra adaptation.
- Missing required fields: the system needs a timestamp and a ring number for every point. The official documentation states explicitly that when the LiDAR spins at 10 Hz each point's timestamp should lie between 0 and 0.1 seconds, while the ring number indicates which channel of the sensor the point belongs to and is used to organize the cloud correctly. If the driver does not output these fields they must be added by hand.
- Optimization for a particular LiDAR: LeGO-LOAM is optimized in particular for a 16-beam LiDAR mounted horizontally on a ground vehicle and assumes a ground plane is always present in the scan. Used in other configurations, its performance is discounted.

The pedagogical point of this case is that an open-source system is not download-and-run. Reading the hardware requirements section of the official documentation is required homework before a project begins. The habit saves weeks of frustration—which is also why every chapter of this book sets out prerequisites and common failures in detail.

Hands-On Activities

Exercise 18-1 LiDAR Odometry and Feature Extraction

Purpose: implement curvature feature extraction by hand, observe the distribution of edge and planar features, and run a complete LiDAR odometry system to understand the roles of drift and loop closure. This exercise works mainly from public datasets and needs no 3D LiDAR hardware.

Part One: Implementing Feature Extraction (everyone)

1. Download a segment of public 3D LiDAR data (KITTI, or the campus dataset provided officially with LeGO-LOAM) and take a single frame of point cloud from it.
2. Implement the roughness computation of Equation 1 in Python: for each point, take several neighbors on each side with the same ring number and compute the value of c.
3. Classify by c: take the highest 2 percent as edge features and the lowest 20 percent as planar features, display them in different colors (Open3D will serve), and observe where in the environment each class falls.
4. Verify the intuition: do the edge features cluster on columns, tree trunks, and wall corners? Do the planar features cluster on the ground and the walls? If not, adjust the neighbor count and the thresholds and try again.
5. Count what proportion of the original points the feature points make up, to get a feel for LOAM's reduction.

Part Two: Running LiDAR Odometry (everyone, or university level)

6. Install LeGO-LOAM or A-LOAM in a ROS environment (taking care to check the hardware and data format requirements of Case 18-3).
7. Play back the dataset and observe in RViz in real time: the feature point cloud, the estimated trajectory, and the 3D map accumulating step by step.
8. Observe the drift: let the data play through one large loop, compare the positions of start and end, and quantify the accumulated drift.

9. Turn on loop closure detection and run again, observing how the whole trajectory and map are straightened when the loop closure fires (the striking moment of Chapters 12 and 15).

Part Three: System Comparison and a Degeneracy Experiment (university level)

10. Run LeGO-LOAM, LIO-SAM (if IMU data is available), and Fast-LIO on the same segment of data, and compare their trajectory accuracy, computational burden, and map quality.
11. Use the wall thickness method of Case 17-1 in Chapter 17 to compare the quality of the three maps quantitatively.
12. Degenerate scene experiment: find a segment of data dominated by a long corridor (or collect one yourself) and compare the performance of pure LiDAR against LiDAR plus IMU, verifying the argument of Section 18.5.
13. If IMU data is available, deliberately disable the IMU fusion and run again, quantifying the improvement the IMU brings.

Discussion Questions

1. What percentage of the original points are your feature points? If you doubled that proportion, would the registration accuracy improve? By how much would the computation time rise?
2. Examine the features you extracted: in areas thick with vegetation or clutter, is the distribution of features reasonable? Does this bear out why LeGO-LOAM segments the point cloud first?
3. How many meters is the accumulated drift over the large loop? What percentage of the total distance traveled is that? How much did loop closure correction improve it?

4. (University level) In the long corridor scene, in which direction is the error largest? Is this the same problem as the 2D corridor predicament of Section 13.6 and the structural degeneracy of Section 17.3?
5. (University level) Using Table 18-1, which system would you choose for a university inspection robot dog? What factors enter the decision?

Extended Challenge (university level)

First: implement the simplest possible LiDAR odometry—use planar features only, register consecutive pairs of frames with point-to-plane ICP (Chapter 17) and accumulate the poses, compare the trajectory with the dataset's ground truth, and plot the curve of error growing with distance. Then add edge features and observe how much the accuracy improves, verifying for yourself the argument of Section 18.2 that the two kinds of feature are complementary. Second: collect a segment of real data in a university corridor (following the method of Stage One in Section 19.8), run this chapter's systems on it and assess the quality, as a preliminary study for Project Four of Chapter 31.

Further Reading

1. J. Zhang and S. Singh, "LOAM: Lidar Odometry and Mapping in Real-time," Robotics: Science and Systems, 2014. (The original LOAM paper, the main basis of this chapter, from the CMU Robotics Institute.)
2. T. Shan and B. Englot, "LeGO-LOAM: Lightweight and Ground-Optimized Lidar Odometry and Mapping on Variable Terrain," IROS, 2018. (The original paper on ground optimization and two-step optimization.)

3. T. Shan et al., "LIO-SAM: Tightly-coupled Lidar Inertial Odometry via Smoothing and Mapping," IROS, 2020. (The representative work on factor graph tight coupling.)
4. W. Xu and F. Zhang, "FAST-LIO: A Fast, Robust LiDAR-inertial Odometry Package by Tightly-Coupled Iterated Kalman Filter," IEEE RA-L, 2021. (And FAST-LIO2, IEEE T-RO, 2022.)
5. The LeGO-LOAM and LIO-SAM open-source projects (GitHub). (With official datasets, hardware requirement notes, and parameter settings; required reading for implementation.)
6. The A-LOAM open-source project (GitHub, HKUST Aerial Robotics). (A clean reimplementation of LOAM with readable code, suited to study in teaching.)
7. The official KITTI Vision Benchmark Suite website. (The standard benchmark dataset for self-driving 3D LiDAR, with ground truth trajectories.)

Summary

1. LOAM was proposed by Zhang and Singh in 2014, and its central insight is that not all the points are needed; it splits SLAM into the two parallel tasks of odometry and mapping and, by extracting edge and planar features, achieves high accuracy at low computational cost in structured environments, becoming the common foundation of later LiDAR SLAM research.
2. Features are classified by local roughness (curvature): high curvature gives edge features (columns, wall corners) and low curvature planar features (walls, floors); the two supply complementary geometric constraints—planes constrain motion perpendicular to the surface and edges motion perpendicular to the line—and without either, the degrees of freedom cannot be fully locked.

3. The two-layer architecture is LOAM's second innovation: high-frequency odometry (about 10 Hz) supplies a real-time pose and a good initial value, while low-frequency mapping (about 1 Hz) registers finely against the global map to remove drift, and the two are integrated into a high-frequency, accurate pose output. Motion distortion compensation must also be performed.
4. LeGO-LOAM filters out noise by point cloud segmentation and makes good use of the ground plane in a two-step Levenberg–Marquardt optimization, running in real time on a low-power embedded system and achieving comparable or better accuracy at lower computational cost, but assuming a ground plane is always present in the scan.
5. LIO-SAM achieves tight IMU coupling and drift-aware loop closure integration with a factor graph and a Bayes tree; the Fast-LIO family takes the road of no feature extraction and tight coupling with an iterated extended Kalman filter, suiting computation-limited platforms and fast motion.
6. Pure LiDAR fails under fast motion and in geometrically degenerate environments (long corridors, open ground), and the IMU fills exactly these three gaps (supplying an initial value, an independent constraint in the degenerate direction, and motion distortion compensation), while the LiDAR returns non-drifting observations that correct the IMU's bias; tight coupling beats loose.
7. Autonomous inspection needs 3D rather than 2D because it must handle uneven terrain, three-dimensional obstacles, three-dimensional inspection targets, and asset modeling; the platform may be wheeled, tracked, or

quadruped, and the gait vibration of a quadruped makes tight IMU coupling a necessity.

This chapter has completed the core technology of 3D LiDAR SLAM. From the point cloud processing of Chapter 17 to the LOAM family here, students now possess the full knowledge needed to localize and map in a three-dimensional world. In the next chapter we take these techniques into a real site—the autonomous inspection robot of Chapter 19—handling the ordering of inspection points, repeat positioning accuracy, and long-term map maintenance, the three things that decide whether a system succeeds, and so closing Part V.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 19

Autonomous Inspection Robots and Field Practice

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapters 17 and 18 assembled the technology of three-dimensional LiDAR SLAM: how to process a point cloud, how to pick features, how to estimate a pose, and how to compensate motion during a sweep. This chapter asks another question—what happens when these techniques must run by themselves for three hours a day in the same building, every day for a year.

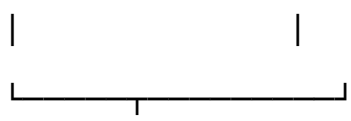
The answer is that technical metrics are no longer the only thing that matters. Whether an inspection robot succeeds often turns not on how accurate its SLAM is but on three more mundane things—whether the inspection points are well ordered, whether it can stop at the same position every day, and whether a map built three months ago still works today. These three things are the subjects of Sections 19.4, 19.5, and 19.6, and the questions the chapter's three experimental figures set out to answer.

The role of this chapter in the book is symmetrical with that of the cleaning robot in Chapter 16: Chapter 16 was the practical landing of 2D SLAM, and this chapter is the practical landing of 3D SLAM. Both demonstrate the same thing—put an algorithm into a product that must be used every day and the real difficulties only begin.

Figure 19-0 The Link Map of Chapter 19

The technical foundation of Part V

Chapter 17 Point cloud processing　　Chapter 18 LOAM and feature registration



▼

【Chapter 19 Autonomous inspection: making the technology run every day for a year】

Route planning ─ Repeat positioning ─ Map maintenance ─ From data to a digital twin

▼

Part VI Visual SLAM　　Part VII Drones and edge AI

Part VIII ROS implementation　　Part IX Semantic navigation

Learning Objectives

Inspection is the most mature commercial application of 3D SLAM and the site that best ties together the first eighteen chapters of this book. Starting from requirements, this chapter treats in turn the technology stack of an inspection robot, platform selection, route planning, repeat positioning, and long-term maintenance, closing with the state of research and industry in Taiwan and with an executable campus project. After reading this chapter, you should be able to:

1. Explain the problems the automation of inspection sets out to solve, and how it differs fundamentally from an ordinary navigation task.
2. Describe the technology stack of an inspection robot and point out which chapters of this book each layer corresponds to.
3. Compare the suitable sites and the costs of the four platforms: wheeled, tracked, quadruped, and aerial.

4. Explain the essential nature of the inspection route planning problem and estimate the effect of ordering on endurance.
5. Explain why repeat positioning accuracy is the key metric of inspection, and how it relates to the geometry of the site.
6. Explain why map maintenance is necessary in long-term operation, and design a map update strategy.
7. Give examples of Taiwan's research and industrial achievements in inspection robots and plan a campus inspection project on that basis.

Core Concepts

Term	Meaning	Related Chapters
Waypoint	A position the robot must reach and at which it must collect data in a particular attitude	19.4, 19.5
Repeat positioning accuracy	How widely the pose scatters over repeated returns to the same inspection point	19.5, Chapter 11
The difference between coverage and inspection	Coverage must visit every cell; inspection need only visit designated points	19.4, Chapter 16
Route optimization	Finding the shortest tour among the inspection points (the traveling salesman problem)	19.4
Map	Updating the prior map as the	19.6, Chapter 13

maintenance	environment changes, to keep localization possible for the long term	
Multi-session SLAM	Merging trajectories from several runs on different dates into one map	19.6, Chapter 15
Digital twin	The three-dimensional model and asset record built from inspection data	19.7, Chapter 17
Degenerate site	An area of insufficient geometric constraint where localization easily fails (long corridors, empty halls)	19.5, Chapters 17 and 18

19.1 Why Inspection Should Be Autonomous

Inspection is an old and tedious job: someone with a record sheet walks a fixed route once a day, reads instruments, listens to equipment, feels the temperature of pipes, and checks doors, windows, and fire safety equipment. The job looks simple, but it has three problems that human labor cannot overcome.

Three Structural Problems of Manual Inspection

- Consistency: people tire, people are distracted, and people skip what they checked yesterday. When the same inspection point is checked by different people at different times, the standard is hard to keep consistent.
- Frequency: labor is limited, and the inspection frequency is usually pressed to a minimum. An inspection twice a week means a piece of equipment may spend as long as three days in an abnormal state with no one aware of it.
- Environmental risk: substations, high-voltage equipment, underground culverts, buildings after a disaster, radiation areas—some places people should

not be going to often in the first place.

The autonomous inspection robot answers all three directly. Its records are entirely consistent, it can run once a day or even once an hour, and it can enter places people should not. More importantly, it raises inspection from finding faults to predicting them—because only when the data are dense and consistent enough can a trend be seen.

Four Differences between Inspection and Ordinary Navigation

Students readily take inspection to be navigation plus photography, but the engineering emphases of the two are in fact quite different:

Aspect	Ordinary Navigation	Inspection Task
Route	May differ each time; the aim is to arrive	The same every day; the aim is repetition
Localization requirement	Meter level suffices for safe travel	Centimeter level, or the same angle cannot be photographed
Map	Built once per run	Built once and used for a year; must be maintained
Criterion of success	Whether it arrived	Whether the data are comparable

The word repeat is the key. The value of inspection data comes from comparison across time—today's infrared thermal image must be compared with last week's to know whether the temperature is rising. If the position and angle of two shots differ by half a meter, the images cannot be compared directly and all trend analysis loses its foundation. This is the problem Section 19.5 addresses.

19.2 The Technology Stack of an Inspection Robot

An inspection robot is the sum of the first eighteen chapters of this book. This section takes its five layers apart from the bottom up and points out which chapters each corresponds to.

Layer	Content	Chapters of This Book
Sensing layer	3D LiDAR, IMU, wheel encoders, RGB and infrared thermal imaging, microphone	Chapters 7, 8, and 9
Localization and mapping layer	3D SLAM mapping, localization against a prior map, loop closure correction	Chapters 17 and 18
Navigation layer	Global path planning, local obstacle avoidance, control of arrival at inspection points	Chapter 3, Part VIII
Task layer	Inspection point management, route scheduling, battery and recharging decisions	19.4, 19.6
Data layer	Tagging data with position and time, anomaly detection, digital twin and reporting	19.7, Part IX

The Inspection Task Layer: the Layer This Chapter Adds

The first three layers were treated fully in Chapters 17 and 18 and in Part VIII; what this chapter genuinely adds are the task layer and the data layer. The questions the task layer must answer sound administrative but are in fact all optimization problems: in what order should the twelve inspection points be visited? With forty percent of the battery left, return to charge first or finish the round? If positioning at some point failed today, how many retries before giving up?

The quality of these decisions decides directly how much work the robot completes each day. Section 19.4 will show with an experiment that merely putting the inspection points in the right order saves forty-five percent of the distance traveled—and distance traveled is battery, and battery is how many rounds a day can be run.

19.3 Platform Selection: Wheeled, Tracked, Quadruped, and Aerial

There is no universal platform for inspection robots, and the choice is decided almost entirely by the site. This section compares the four mainstream options.

Platform	Advantages	Limitations	Typical Sites
Wheeled	Best endurance, lowest cost, most stable SLAM	Cannot climb stairs, poor over obstacles	Plants, corridors, machine rooms, substations
Tracked	Good over obstacles and ramps, highly stable	Large and heavy, wears the floor when turning	Outdoors, disaster sites, construction sites
Quadruped	Strongest adaptation to terrain, can climb stairs	Short endurance, high cost, gait vibration unkind to SLAM	Multiple floors, culverts, uneven ground
Aerial	Can reach heights and the underside of bridges, fast	Endurance measured in minutes, hard to localize indoors	Bridges, towers, the exterior of large structures

The SLAM Difficulty of a Quadruped Platform

Quadruped robots have become a popular choice for inspection in recent years, but they are a severe test for SLAM. The reason is gait vibration: every footfall produces

high-frequency pitch and roll of the body, and the motion distortion compensation discussed in Section 18.3 is especially prone to error under violent motion of this non-constant-velocity kind. This is also why a quadruped platform almost always requires a high-quality IMU and tightly coupled LiDAR-inertial odometry (Section 18.5).

Three Questions for Choosing a Platform

Does the site have changes in level? Stairs or a threshold rule out a wheeled platform.

How long is one inspection round? Beyond an hour, the endurance of quadruped and aerial platforms becomes the bottleneck.

At what height are the inspection targets? On a ceiling or under a bridge, no ground platform, however capable, can see them.

The last question is the most practical: will SLAM degenerate at this site? A long corridor or an empty hall (Section 17.3, Figure 18-2) is a difficulty for any platform and must be confirmed before purchase.

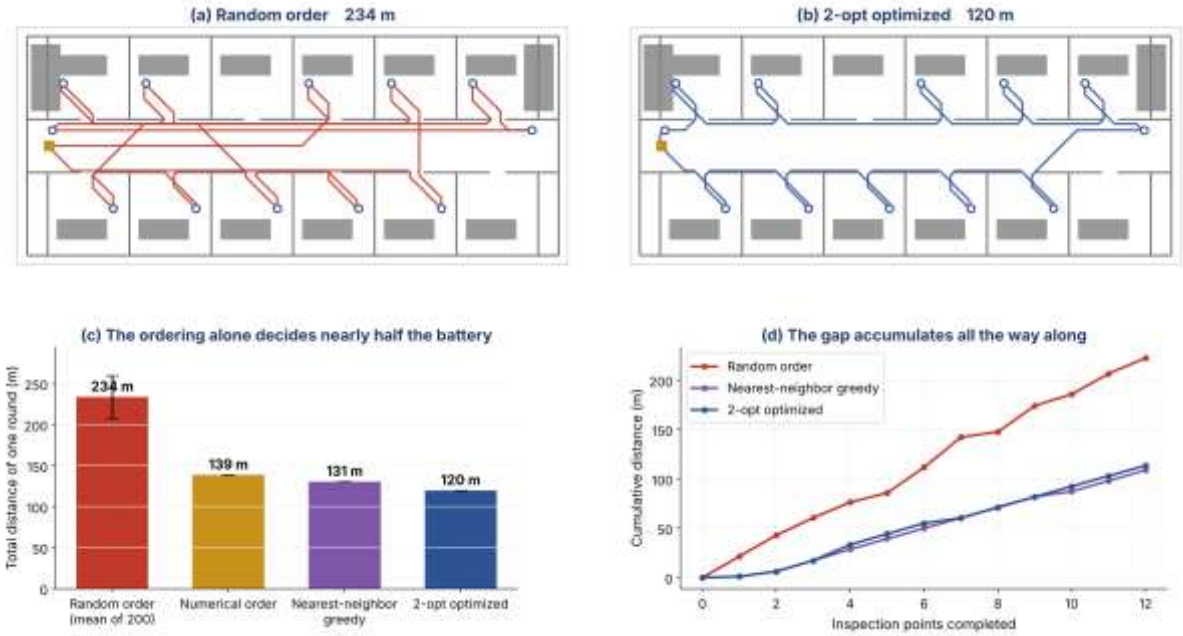
19.4 Inspection Point Planning and Route Optimization

Inspection differs fundamentally from the cleaning task of Chapter 16 in one respect: cleaning must visit every cell (coverage path planning) while inspection need only visit designated points. That difference turns the problem from a coverage problem into a traveling salesman problem—given a set of points that must be visited, find the shortest tour.

The problem is hard in theory (growing factorially with the number of points), but at the practical scale of inspection it is very easy to handle. A floor usually has only a dozen

or so inspection points, and a nearest-neighbor greedy initial solution refined repeatedly with 2-opt gives a near-optimal route within milliseconds.

Figure 19-1 Ordering the Inspection Points: One Route Decides Forty-Five Percent of the Battery



The experiment in this book uses a 34 × 14 meter floor of a department building: one corridor running the length of it, two rows of classrooms and laboratories above and below, an elevator shaft, and a stairwell, with twelve inspection points set along the way—the elevator door, a card reader, a distribution box, a fire hydrant, an air conditioning unit, and laboratory doorways. The robot sets out from the dock at the left end of the corridor, visits all twelve points, and returns to the dock.

Panel (a) is the result of a user tapping the inspection points into an app in no particular order: the path shuttles back and forth along the corridor for a total of 234 meters. Panel (b) is the same twelve points after 2-opt optimization: the robot advances along the corridor in one direction, entering and leaving the rooms on either side as it passes, for a total of 119.7 meters. The points visited are exactly the same and only the order

differs.

Panel (c) sets four orderings side by side. Random order (the mean of 200 trials) is 233.9 meters, numerical order 138.7 meters, nearest-neighbor greedy 130.5 meters, and 2-opt optimized 119.7 meters. From random to optimized saves 49 percent, while going from greedy to 2-opt saves only 8 percent more—which shows something very practical: most of the benefit of ordering comes from not wandering, not from an ingenious algorithm. A ten-line nearest-neighbor method captures ninety percent of the gain.

Panel (d) shows how the gap accumulates. The three curves nearly coincide over the first two or three points, but the random order pays a further price at every additional point, and by the twelfth it has fallen nearly a hundred meters behind. Translated into practical terms: at an inspection speed of 0.5 meters per second, the extra path costs more than three minutes, and if the robot runs eight rounds a day that is 26 minutes more of battery and motor life consumed every day.

Three Corrections in Implementation

The distances in this experiment are the actual shortest traversable paths through the corridor and rooms (computed by BFS on the occupancy grid), not straight-line distances. Ordering by straight-line distance is badly distorted on a floor with partitions.

Real inspection also has ordering constraints: some points must be photographed at a particular time (access control before working hours, say), which makes it a traveling salesman problem with time windows and requires the constraint to be added to the optimization.

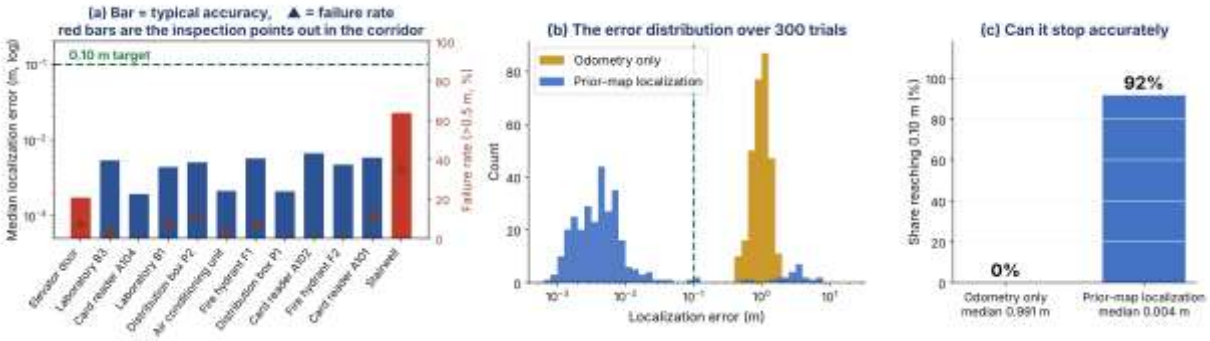
When crossing floors, the elevator is a bottleneck node and the wait is often longer than the walk, so finishing all the points on one floor at once should be set as a hard constraint.

When the battery is not enough for the whole round, the problem becomes maximizing inspection value within a limited distance, and each inspection point then needs a priority.

19.5 Repeat Positioning Accuracy: Can It Stop at the Same Place Every Day

This is the most critical metric of inspection and the one most easily underrated. To photograph the same instrument and compare thermal images of the same wall, the robot must stop at the same position at the same angle every day. The industry requirement is commonly a few centimeters—far above the meter-level requirement of ordinary navigation, which is only not to hit anything.

Figure 19-2 The Same Inspection Point, Repeated 25 Times: Does It Stop Accurately



The experiment is set up as follows. The robot leaves the dock and arrives at an inspection point, having accumulated about 1 meter of odometry error on the way (equivalent to a drift rate of 1.5 percent over a 60-meter route). At this moment there

are two choices—trust the odometry outright, or relocalize by matching a LiDAR scan against the prior map. Each of the twelve inspection points is repeated 25 times, for 300 trials in all.

The two distributions of panel (b) tell the whole story. With odometry alone the error concentrates near 1 meter and not a single trial reaches the 0.10-meter target; with prior-map localization the overwhelming majority of trials fall below 1 centimeter, with a median of 0.004 meters and 92 percent of trials meeting the target. Panel (c) condenses the contrast into one number: 0 percent against 92 percent.

But panel (a) is the real point of this section. It shows the positioning accuracy point by point, and students will notice at once that the blue in-room inspection points have median errors between 1 and 6 millimeters with failure rates of 0 to 8 percent, while the red corridor inspection points—the elevator door and the stairwell—are clearly worse, the stairwell failing 48 percent of the time, with nearly half of all attempts failing to localize.

The reason should by now be very familiar: a long corridor lacks geometric constraint along its axis (the structural degeneracy of Section 17.3, the information matrix analysis of Figure 18-2). The robot knows which wall it is hugging but cannot say how far along the corridor it has traveled. What was still only a theoretical limitation in Chapters 17 and 18 becomes, at an inspection site, a concrete product defect: that inspection point is photographed crooked every other time.

This brings out an important design principle: the position of an inspection point should be decided not only by what is to be checked but also by how accurately the robot can localize there. Moving an inspection point two meters from the middle of the

corridor toward a doorway, so that the robot can see the door frame and the walls on both sides at once, is often more effective than upgrading the sensor—and it costs nothing.

Four Means of Improving Repeat Positioning Accuracy

Move the inspection point: prefer positions rich in geometric features (corners, door frames, places dense with equipment).

Add artificial landmarks: put AprilTags or retroreflective markers (Chapter 4) in degenerate areas to supply an absolute position reference at very low cost.

Complement with multiple sensors: where LiDAR degenerates, as in a corridor, the visual features of a camera (Part VI) or a UWB tag (Chapter 4) can fill in the missing direction.

Accept failure and handle it: when localization confidence is insufficient, have the robot back up a meter and try again, or mark the record as position unreliable, rather than taking a photograph no one knows is crooked.

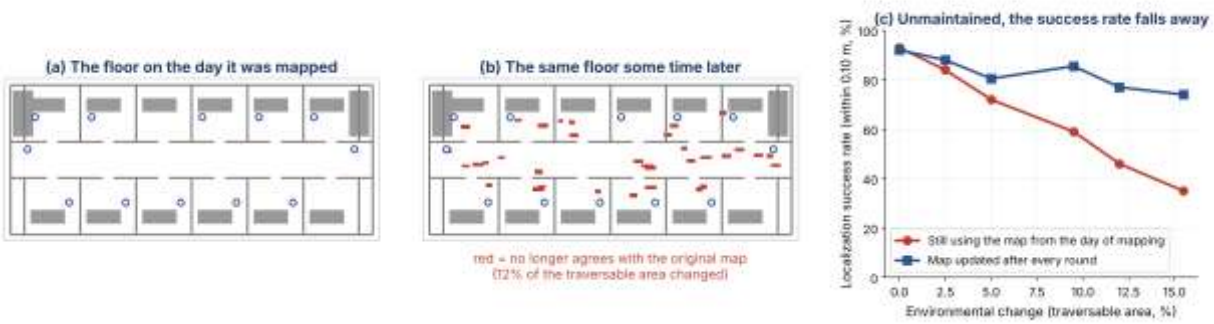
19.6 Long-Term Operation: Map Maintenance and a Changing Environment

The previous two sections dealt with today's round; this one asks whether it will still run a year from now. This is the greatest difference between inspection and a one-off mapping task: the map is not a deliverable but an asset requiring maintenance.

A real site changes. Desks and chairs are moved, cardboard boxes pile up in the corridor, a laboratory swaps its equipment, a door is open today and closed tomorrow. Each of these changes is small on its own, but together they suffice to defeat localization—

because these are exactly the things scan matching compares.

Figure 19-3 Maps Expire: the Effect of Environmental Change on Localization Success



Panel (a) is the floor on the day it was mapped and panel (b) the same floor some time later—red marks what no longer agrees with the original map: desks and chairs added in the rooms, objects set down temporarily in the corridor, altering about 12 percent of the traversable area in all. From a person's point of view the floor is still the same floor; from the point of view of scan matching, one observation in eight no longer matches.

Panel (c) is the quantified result, and it is the figure in this chapter most worth remembering. The red line is the localization success rate when the old map from the day of mapping continues to be used: 93 percent with no environmental change, falling to 72 percent at 5 percent change and to only 35 percent at 15 percent—in less than a year, half the system is useless. The blue line is the map updated after each inspection round: even at 15 percent environmental change the success rate holds at 74 percent.

The divergence of the two lines shows that map maintenance is not a finishing touch but the watershed of whether an inspection system can operate for the long term. It is worth noting that the blue line is not flat either—it declines gently from 92 percent to 74 percent, because the greater the environmental change, the larger the share of temporary objects (boxes, carts) in the map, and these things will not be there

tomorrow. This is exactly the lifelong mapping problem mentioned in Section 15.8: the map must both remember permanent structure and forget temporary obstacles.

Three Strategies for Map Maintenance

- Periodic rebuilding: rebuild the map at fixed intervals (quarterly, say). The simplest approach, but the system cannot operate during the rebuild and historical correspondences are lost.
- Incremental updating: merge new observations into the map after each inspection round, controlling how fast old information is forgotten by means of upper and lower bounds on the occupancy probability (Section 13.5). The blue line of this experiment is of this kind.
- Layered maps: divide the map into a permanent layer (walls, columns, door frames) and a transient layer (furniture, stored items). Localization uses the permanent layer only while obstacle avoidance uses both. This is the most common practice in commercial systems today and best satisfies remembering and forgetting at once.

19.7 The Functions of Inspection Robots and Their Industrial Applications

The preceding six sections dealt with how to get to a given point. But what an inspection robot does once it arrives, and how well it does it, is the real reason a user pays for it. This section lays out the functions of an inspection robot, then explains how requirements differ across industries by site, and finally introduces the roles that make up this industry chain.

1. The Core Functions of an Inspection Robot

The functions of an inspection robot fall into four classes: moving, collecting, interpreting, and managing. The list below gathers the capabilities that commercial models generally possess or are now introducing, and marks the technology each requires and the corresponding chapters of this book.

Class	Function	Technology Required	Chapters of This Book
Moving	Autonomous navigation and obstacle avoidance	3D SLAM, path planning, cost maps	Chapters 17 and 18, Part VIII
Moving	Docking at a point (centimeter-level repeat positioning)	Localization against a prior map, fine positioning control	19.5, Chapter 11
Moving	Automatic recharging and endurance management	Battery model, scheduling decisions	19.4, 19.6
Moving	Movement between floors (elevator integration)	Topological map, equipment communication interface	19.4, 19.8
Collecting	Visible-light photography at fixed points	Gimbal control, exposure compensation	Chapter 9
Collecting	Infrared thermal measurement	Thermal camera, temperature calibration	Chapter 9, 19.7
Collecting	Capturing abnormal sound and recording spectra	Microphone array, acoustic analysis	19.7

Collecting	Gas, temperature and humidity, and radiation detection	Environmental sensor integration	19.7
Interpreting	Reading instrument needles and digits	Image recognition, OCR	Part IX
Interpreting	Detecting defects in equipment appearance	Object detection, anomaly detection	Part IX
Interpreting	Analyzing hot spots and temperature rise trends	Comparison of data across time	19.5, 19.6
Managing	Tagging data with three-dimensional coordinates and time	Pose recording, time synchronization	Chapter 10, 19.6
Managing	Grading anomalies and dispatching work orders	Rule engine, back-office integration	19.7
Managing	Digital twin and asset records	Three-dimensional reconstruction, point cloud colorization	Sections 17.5 and 17.6

Students can see a clear dividing line in this table: the two classes of moving and collecting are already quite mature in commercial models, and their technical foundations all lie in Parts I through VIII of this book; interpreting is the frontier at present, with most models still only transmitting data back for a person to interpret, while automatic interpretation is gradually being introduced—which is precisely the

subject of Part IX.

2. How Functional Requirements Differ by Site

There is no universal specification for an inspection robot. The functions needed at a power facility and in an underground pipe have almost nothing in common, though both are called inspection. The table below gathers the differences across four common sites, and may serve as a starting point for choosing a project topic or writing a requirements document.

Site	Principal Inspection Targets	Key Functions	Environmental Difficulties
Power facilities	Transformers, switchgear, capacitors, terminals	Infrared thermography, abnormal sound detection, instrument reading	Outdoors in all weather, electromagnetic interference, high-voltage clearance
Bridges and structures	Cracks, corrosion, cable force	High-resolution imagery, three-dimensional positioning, surface orthophotos	No satellite signal, work at height, sub-millimeter resolution required
Plants and warehouses	Piping, machines, fire safety equipment, racking	Imagery at fixed points, temperature, gas detection	People and vehicles mixed, daily environmental change, the line cannot stop
Underground pipes and	Structural deformation,	Waterproofing and dust protection, gas	No light source, confined,

culverts	standing water, gas	detection, lighting	communications cut off, rescue impossible

Bridge inspection best illustrates how function decides platform. To identify cracks of 0.2 to 0.3 millimeters, the vehicle must be able to approach the surface of the structure closely and the camera must have sufficient resolution; and since satellite signals do not reach under a bridge or inside a structure, the vehicle must maintain its position entirely by its own SLAM. These two requirements pull against each other—the closer it flies the harder localization becomes—which is the core engineering difficulty of structural inspection drones and the meeting point of Parts V and VII.

3. Quantified Benefits: How Functions Turn into Value

A purchasing decision for an inspection robot usually looks not at technical metrics but at three quantifiable benefits. The figures below come from cases actually deployed in Taiwan and may serve as a reference for estimation.

- Inspection frequency: after one domestic power utility introduced an outdoor inspection robot, the inspection frequency of the same substation rose from twice a week by staff to once a day by robot. This means the maximum delay before an anomaly is found fell from three days to one, and it gave predictive maintenance a sufficiently dense data foundation for the first time.
- Computing resources: one domestic research institute reports that the navigation algorithm it developed for inspection robots occupies only 25 percent of the robot's computing resources, where traditional algorithms require 65 to 70 percent. For a robot that must run navigation and image recognition at the same time, the computing resources saved decide directly whether it can react in real time.

- Inspection coverage: taking bridges as an example, Taiwan has more than 23,000 road bridges and the competent authority's goal is 100 percent inspection. A quantity of this order makes manual inspection infeasible in principle, and shows that at some sites automation is not a question of efficiency but of feasibility.

4. The Roles That Make Up the Industry Chain

The inspection robot is one of the few robotics applications with a complete supply chain in Taiwan. The following explains the role of each link by industry, so that students may understand the shape of this industry and possible directions for employment.

Link	Industry	Role in the Inspection Robot
Sensing elements	LiDAR manufacturing	Supplies 2D / 3D LiDAR, deciding mapping quality and cost
Sensing elements	Optical and thermal imaging modules	Visible-light cameras, infrared thermal cameras, gimbals
Actuation components	Electrical machinery and motor manufacturing	Wheel sets and joint motors, deciding endurance and the ability to cross obstacles
Control components	Industrial and robot controller industry	Motion control, real-time systems
Computing platform	Edge computing and embedded systems	The onboard computer, deciding how complex an algorithm can be run
Whole-machine integration	Electronics contract manufacturing and systems integration	Mechanical design, whole-machine assembly, mass production and validation

Software and algorithms	Robot software and AI services	SLAM, navigation, image recognition, back-office platform
Research institutes	Industrial technology research institutes	Platform technology development, industry connection, field validation
End users	Utilities, manufacturers, transport authorities	State the requirements, provide the validation site, purchase and deploy

The link worth noting is the research institute. In Taiwan, non-profit research institutes often play the role of platform developer and industry connector: the institute designs the body and the AI core, suppliers in various industries provide components and whole-machine integration, and a utility or manufacturer finally provides the site for validation. This division of labor lets a single company of limited resources take part in highly complex robot development, and it is the model of cooperation students are most likely to encounter should they enter this field later.

From Function to Specification: Writing an Inspection Requirements Document

Define the inspection targets and the criteria of interpretation first: what is to be examined, and to what degree something counts as abnormal. This step decides the requirements for camera resolution, thermal accuracy, and docking accuracy.

Then define the site conditions: whether there are changes in level, whether it is outdoors, whether satellite signals reach it, and how often the environment changes. This step decides the platform type and the map maintenance

strategy.

Then define the frequency and the endurance: how many rounds a day, how long a round takes, and how long charging takes. This step decides the battery capacity and how important route optimization is.

Only then discuss technical specifications. Writing the requirements before choosing the specifications is the most effective way to avoid buying the most expensive sensor and still not solving the problem—an extension of the discussion of sensor selection in Chapter 8.

19.8 A Field Project: the Corridors, Elevators, and Laboratories of a University of Technology in Southern Taiwan

This section turns the preceding seven sections into an executable project plan. The site is a department building at a university of technology in southern Taiwan—a campus of this kind is an excellent site for teaching inspection: it has long corridors (degenerate scenes), elevators and stairs (crossing floors), laboratories (valuable equipment and access control), and genuine management needs.

Site Analysis: the Different Challenges of Three Environments

Site	Principal Challenge	Technical Remedy	Corresponding Chapters
Long corridor	Geometric degeneracy along the axis, high localization failure rate	Tight IMU coupling, artificial landmarks, inspection points moved to doorways	Sections 17.3, 18.5, 19.5

Elevators and stairwells	Crossing floors, a discontinuous map, metal reflections	Layered maps, the elevator as a topological node, a division of labor between quadruped and wheeled	19.4, 19.6
Laboratories	Daily environmental change, people moving about, equipment replaced	Layered maps, dynamic object rejection, incremental updating each round	19.6, Section 13.5
Lobbies and atriums	Open space, sparse returns at a distance, glass curtain walls	Reduce reliance on LiDAR, assist with vision, shorten the spacing between inspection points	Chapter 8, Part VI

A Five-Stage Project Plan

1. Stage 1—data collection: carry a 3D LiDAR and an IMU on a handcart or a robot platform and record datasets in the corridor, the stairwell, and the laboratory separately. This step needs no autonomous navigation—pushing it along by hand will do—and suits an introductory undergraduate assignment.
2. Stage 2—offline mapping and comparison: run LeGO-LOAM, LIO-SAM, and Fast-LIO (Section 18.4) on the recorded data, compare their trajectory accuracy, map quality, and computational burden at each site, and quantify map quality with the wall thickness method of the case in Chapter 17.
3. Stage 3—degenerate scene experiment: focusing on the long corridor, compare pure LiDAR against tightly coupled LiDAR plus IMU and quantify the improvement the IMU brings. This is the part with the most research value, and it corresponds directly to the two tall red bars of Figure 19-2(a) in this chapter.

4. Stage 4—setting inspection points and optimizing the route: set ten to fifteen inspection points on the map you built, implement the nearest-neighbor plus 2-opt planning of Section 19.4 of this chapter, and actually measure the distance traveled and the time taken before and after ordering, reproducing Figure 19-1.
5. Stage 5—long-term operation and map maintenance: run the inspection daily for several consecutive weeks, record the localization success rate at each inspection point, observe how it falls as the environment changes, then introduce incremental updating or a layered map and quantify the improvement. This stage can be extended into a master's topic.

Value for Teaching and Research

The strength of this project is that its levels are distinct: Stages 1 and 2 suit undergraduate coursework, Stages 3 and 4 suit project students, and Stage 5 can be extended into a master's thesis. And the results are concrete and presentable—a three-dimensional model of the campus and a set of daily inspection records are far more persuasive than any slide deck when explaining the work to outsiders.

More importantly, it uses everything in the first eighteen chapters of this book: Chapter 8 to choose the LiDAR, Chapter 7 to pair it with an IMU, Chapter 10 to calibrate the extrinsics, Chapter 17 to process the point cloud, Chapter 18 to run LOAM, and Chapter 12 for loop closure optimization, plus this chapter's route planning, repeat positioning, and map maintenance. A student who has completed these five stages already has the full capability to join an industrial inspection robot team.

Formulas and Procedures

1. Three Evaluation Metrics for an Inspection Task

The quality of an inspection system can be measured by three metrics, which pull against one another:

inspection completion rate = inspection points at which data were collected successfully / total inspection points

repeat positioning accuracy = the standard deviation of the pose over repeated visits to the same inspection point

time per round = total distance traveled / average speed + Σ the collection time at each point

Raising the completion rate usually means more retries and so lengthens the time per round; and shortening the time may sacrifice positioning accuracy. This engineering trade-off is the core of product design, and it is exactly isomorphic with the three metrics of the cleaning robot in Section 16.4.

2. The Route Optimization Problem of Inspection

Let the set of inspection points be $P = \{ p_1, \ldots, p_n \}$, the dock be p_0, and $d(i, j)$ be the length of the shortest traversable path between two points (not the straight-line distance); find a permutation π that minimizes the total distance traveled:

$$\min \; d(p_0, p_{\pi(1)}) + \sum_{k=1}^{n-1} d(p_{\pi(k)}, p_{\pi(k+1)}) + d(p_{\pi(n)}, p_0)$$

This is the traveling salesman problem. The practical solution has two steps: find an initial solution with the nearest-neighbor greedy method, then swap the order of any two segments of the route repeatedly with 2-opt until no further improvement is possible. The experiment of Figure 19-1 in this chapter shows that the greedy method alone already captures more than ninety percent of the benefit.

3. The Occupancy Probability Update of Map Maintenance

The core of incremental updating is to let old information be overwritten by new

observations. Use the log odds of Section 13.2, but set upper and lower bounds on it:

$$l_t = \mathrm{clamp}(l_{t-1} + l_{meas}, -l_{max}, +l_{max})$$

The bound l_{max} decides the plasticity of the map: the larger l_{max}, the more the map trusts history and the harder it is for a new observation to change it; the smaller l_{max}, the more readily the map forgets. A long-term inspection system usually takes a smaller l_{max} in exchange for adaptability to a changing environment.

4. A Day in the Life of an Inspection Robot

Stage	Action
① Startup	Leave the dock and perform global relocalization against the prior map (Chapter 11)
② Planning	Compute this round's route from the inspection point list and the battery level (Section 19.4)
③ Inspection	Travel to each inspection point → position finely to the centimeter → collect imagery, thermal images, and sound
④ Monitoring	Record the localization confidence in real time; retry or mark the data unreliable when confidence is insufficient
⑤ Recharging	Return to the dock once the battery threshold is reached, then resume the points not yet completed
⑥ Updating	Merge this round's scans into the map (incremental updating or a layered map, Section 19.6)
⑦ Reporting	Tag the data with three-dimensional coordinates and time, and output the anomaly list and trend charts (Part IX)

Cases

Case 19-1 From Twice a Week to Once a Day: What a Change in Inspection Frequency Changed

One domestic power utility began research on intelligent inspection robots in 2017 and, working with a non-profit research institute and domestic robot companies, took about three years to complete an outdoor inspection robot whose design and manufacture were entirely in domestic hands, deploying it at a secondary substation.

Its sensor configuration is almost a complete implementation of Part II of this book: a 3D LiDAR builds the surrounding three-dimensional terrain, a 3D camera detects obstacles to avoid collisions, a dual imaging module on a gimbal (including infrared thermography) serves as its eyes, a microphone array captures the sound of the equipment, and an antenna supports remote control. It walks a fixed route, photographs at set points, and returns to its own shed to charge wirelessly when the task is done—precisely the inspection point and repeat positioning requirements described in Sections 19.4 and 19.5.

But the real value of this case lies not in the hardware but in an institutional change: the inspection frequency of the same substation rose from twice a week by staff to once a day by robot. That seemingly ordinary number means the maximum delay before an anomaly is found fell from three days to one, and it gave predictive maintenance a sufficiently dense data foundation for the first time—so that when a transformer terminal begins to degrade, for instance, an outage for replacement can be scheduled before the failure occurs.

The case also presents the limitations of the time honestly: the robot only transmitted imagery back and could not yet identify equipment or interpret sounds by itself, with AI interpretation of instrument needles and detection of abnormal transformer noise

planned for later. This order of solving position first and understanding second is exactly the structure of this book—Parts I through VIII treat where, and only Part IX treats what.

Case 19-2 When There Are Twenty-Three Thousand Things to Inspect

Taiwan has more than 23,000 road bridges, and the competent authority's goal is to inspect them all. The number alone explains why automation is necessary—if manual inspection of each bridge takes several days, one round of national inspection is a job measured in years. This is not a question of efficiency but of feasibility.

The approach developed by domestic research teams splits the problem in two: first digitize the bridge with a drone, building a three-dimensional model and producing an orthophoto of the surface; then let AI find cracks and deterioration in the imagery and locate the defects back onto the three-dimensional model. The system can also detect occluded areas automatically and raise coverage with additional compensating imagery—the same line of thinking as the coverage path planning of Chapter 16, only moved from the ground into the air.

Two technical limitations are worth remembering. The first is positioning: satellite signals do not reach under a bridge or inside a structure, so the drone must maintain its position entirely by its own SLAM, which is exactly where Parts V and VII intersect. The second is resolution: to identify cracks of 0.2 to 0.3 millimeters the drone must fly close enough and the camera must be good enough, and flying close makes positioning harder still. This contradiction is the core engineering problem of structural inspection drones.

Another route is to abandon a general-purpose platform for a dedicated mechanism.

For cable inspection, a domestic engineering consultancy developed a cable-climbing vehicle: suitable for cables up to 300 millimeters in diameter, weighing 13.5 kilograms, climbing at angles above 60 degrees and at speeds above 40 centimeters per second, with sensors mounted at several points on the same cable to measure modes, frequencies, and cable force directly in place of the traditional vibration method. This reminds students that inspection need not use a robot that walks: a dedicated mechanism designed for a single task is often more reliable and cheaper.

Case 19-3 A Domestic Quadruped Platform: Industrial Division of Labor and Simulation First

In July 2026 a domestic competent authority announced the results of a domestic quadruped robot development platform, led by a non-profit research institute, which took about a year and a half to produce two domestic prototypes. The value of this case lies not in any single technology but in the industrial structure it reveals.

The division of labor is quite clear: an electronics contract manufacturer handled whole-machine integration, a vision systems company supplied the imaging payload, an electrical machinery company supplied the joint motors, a LiDAR company supplied the sensing elements, and a controller company handled motion control, while the research institute took charge of the body design, the AI core, the simulation environment, and the inertial measurement unit. In other words, from sensing to actuation and from controller to whole machine, the entire supply chain is domestic, and the platform uses an open architecture so that other teams can develop applications on it.

The applications already validated include underground culvert inspection,

autonomous navigation inspection, swarm cooperation, and firefighting rescue—the first three being the subject of this chapter, while firefighting rescue echoes the discussion of extreme environments in Part VII. The institute expects the first wave of demand in industrial inspection, heavy-load semiconductor transport, and logistics.

What students should note most is the training method: the team used a supercomputer with a general-purpose simulation platform to train more than four thousand virtual robot dogs in parallel simultaneously, then deployed to the physical machines. This simulate-first-then-deploy procedure is the full subject of Chapter 33, and it is already standard practice in a national-level project rather than an experiment in a research laboratory.

Hands-On Activities

Exercise 19-1 Inspection Route Planning and Repeat Positioning Assessment

Purpose: implement in code the two core problems of the inspection task layer—route optimization and repeat positioning assessment—and measure their effects in simulation or at a real site. This exercise is mainly in Python, and Parts One and Two can be completed without a 3D LiDAR.

Part One: Route Optimization (everyone)

1. Take a floor plan (the occupancy grid map produced in the Chapter 13 exercise will serve, or define one yourself on graph paper) and mark 10 to 15 inspection points and a dock position on it.
2. Compute the shortest traversable path length between every pair of points with BFS or A* and build a distance matrix. Note particularly that traversable distance

must be used, not straight-line distance.

3. Implement the nearest-neighbor greedy method to find an initial tour order and compute the total distance.
4. Implement 2-opt refinement: repeatedly try reversing any segment of the route, accept it if the total distance shortens, and stop when no further improvement is possible. Record the improvement.
5. Compare with a random order (the mean of 100 trials), reproduce the four bars of Figure 19-1(c), and compute how much distance ordering saves at your own site.

Part Two: Repeat Positioning Assessment (everyone, or university level)

1. In a simulation environment (or on a real robot), have the robot travel from the same starting point to the same inspection point 20 times, adding different odometry noise each time.
2. Record the final pose error under two conditions: odometry alone, and relocalization by matching a scan against the prior map.
3. Plot the error distributions of the two (a logarithmic horizontal axis is recommended), compute the median and the success rate, and reproduce Figures 19-2(b) and (c).
4. Repeat at a different inspection point, choosing in particular one in the middle of a long corridor and one at a corner of a room, compare their failure rates, and verify the argument of Section 19.5 about site geometry.
5. Adjust the inspection point positions according to the result: move the worst-performing point by one or two meters, measure again, and record the improvement.

Part Three: Long-Term Map Maintenance (university level and project students)

1. Add several new objects at random in the simulation environment to simulate environmental change, and compute the proportional change in traversable area.
2. Run localization with the original map and record the success rate; then run it with the updated map, compare the two, and reproduce Figure 19-3(c).
3. Implement a layered map: mark cells observed on several consecutive rounds as the permanent layer and those appearing on only one or two rounds as the transient layer. Localize using the permanent layer alone and compare its success rate with localization on the full map.
4. (Advanced) Map a real site daily for a week and observe which areas change most often, drawing a heat map of change as a basis for siting inspection points and for a map maintenance strategy.

Discussion Questions

1. At your site, how much distance does 2-opt save relative to a random order? If the robot's battery carries it 2 kilometers, how many more rounds a day does optimized ordering give it?
2. Which inspection point has the worst repeat positioning? What are the geometric features around it? Is this the same thing as the structural degeneracy of Section 17.3 and the information matrix analysis of Figure 18-2?
3. If an inspection point is moved from the middle of the corridor to two meters from a doorway, how much does localization improve? What does this free improvement say, set against the cost of upgrading the sensor?

4. At what proportion of environmental change does the localization success rate of the old map begin to fall appreciably? Using that proportion, how often does your site need its map updated?
5. (University level) Using the platform comparison of Section 19.3, which inspection robot would you choose for your department building? How would the need to cross floors change your choice?

Extended Challenge (university level)

First: implement inspection scheduling with time windows—some inspection points must be visited during a designated period (access control before working hours, air conditioning while it runs)—adding this constraint to the acceptance condition of 2-opt and comparing the route length with and without it. Second: design a localization confidence monitor that computes the smallest eigenvalue of the scan matching information matrix in real time (the method of Figure 18-2), flags an inspection record when it falls below a threshold, and plots the confidence of the whole floor as a heat map to find every degenerate area at the site.

Further Reading

- Reports on the outdoor inspection robot of a domestic power utility. (A complete case of a domestic outdoor inspection robot, with its sensor configuration and the change in inspection frequency.)
- Releases from a domestic non-profit research institute on inspection robots and the domestic quadruped robot development platform. (The industrial ecosystem and division of labor of inspection robots.)
- Reports on domestic bridge inspection drones and AI deterioration recognition

systems. (The automation route and technical limitations of structural inspection.)

- T. Shan and B. Englot, "LeGO-LOAM," IROS 2018; T. Shan et al., "LIO-SAM," IROS 2020. (The two 3D SLAM systems most used on inspection robots, extending Chapter 18.)
- S. Macenski et al., the official documentation of slam_toolbox and Nav2. (Map serialization, pure localization mode, and the navigation stack, the software foundation of an inspection system.)
- D. Applegate et al., The Traveling Salesman Problem: A Computational Study, Princeton Univ. Press. (The authoritative reference on route optimization, covering 2-opt and more advanced methods.)
- Literature on long-term localization and map maintenance (lifelong mapping, multi-session SLAM). (The theoretical foundation of Section 19.6 of this chapter.)

Summary

- Manual inspection has the three structural problems of consistency, frequency, and environmental risk; the autonomous inspection robot answers all three with entirely consistent records, a frequency that can be raised, and the ability to enter dangerous sites, and it raises inspection from finding faults to predicting them.
- The four differences between inspection and ordinary navigation are: the route seeks repetition rather than arrival, localization demands centimeters rather than meters, the map must be maintained rather than discarded after use, and the criterion of success is the comparability of the data.

- The technology stack has five layers—sensing, localization and mapping, navigation, task, and data; the first three were treated in Chapters 17 and 18 and in Part VIII, and what this chapter adds are the task layer (inspection points and routes) and the data layer (digital twins and reports).
- Platform selection is decided by the site: wheeled is best in endurance but cannot climb stairs, tracked is good over obstacles, quadruped adapts best to terrain but has short endurance and a gait vibration unkind to SLAM, and aerial can reach heights but has endurance measured in minutes.
- Inspection is a traveling salesman problem, not a coverage problem. The experiment shows 234 meters for a random order, 130 meters for nearest-neighbor greedy, and 119.7 meters for 2-opt optimization, so ordering saves 49 percent, nine tenths of which comes from the simple greedy method.
- Repeat positioning accuracy is the most critical metric of inspection. Odometry alone cannot reach the 0.10-meter target (0 percent), while localization against a prior map reaches 92 percent with a median of 0.004 meters; but inspection points in a corridor can fail 48 percent of the time because of structural degeneracy—the position of an inspection point should take account both of what is to be checked and of how accurately the robot can localize there.
- The map is an asset requiring maintenance. Continuing to use the old map, the localization success rate falls from 93 percent to 35 percent as the environment changes; updating the map every round holds it above 74 percent. The three maintenance strategies are periodic rebuilding, incremental updating, and layered maps.
- The functions of an inspection robot fall into the four classes of moving, collecting, interpreting, and managing; moving and collecting are already quite

mature, interpreting still mainly means transmitting data back for a person to interpret, and automatic interpretation is the frontier at present (Part IX).

- Functional requirements differ enormously by site: power facilities emphasize thermal imaging and abnormal sound, bridge structures need sub-millimeter resolution and positioning without satellites, plants and warehouses face daily change, and underground pipes require waterproofing, dust protection, and gas detection.
- Taiwan's inspection robots have a complete supply chain, made up of LiDAR, optics, motors, controllers, edge computing, whole-machine integration, software algorithms, research institutes, and end users together; deployed cases show that the inspection frequency can be raised from twice a week to once a day.

Carrying Forward

Part V is complete. From the point cloud processing of Chapter 17 and the LOAM and feature registration of Chapter 18 to this chapter's placing of those techniques into an inspection system that must run every day for a year, students should be able to see a clear thread: the algorithm sets the ceiling of a system, while the mundane decisions of the task layer decide how much of that ceiling is actually reached.

The two unusually tall red bars of Figure 19-2(a) in this chapter also point once more to the limit of LiDAR—in a geometrically degenerate long corridor, no 3D LiDAR, however good, can say how far it has traveled. In the next part we change sensors: from Chapter 20 onward we see with a camera. Visual SLAM has difficulties of its own (white walls, repetitive texture, scale ambiguity), but it serves exactly where LiDAR is helpless—the

posters on the walls, the door plates, and the floor texture are all clear features to a camera.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Part VI Visual SLAM and the Vision-Navigated Robot

Chapter 20

An Introduction to Visual SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Part VI changes the sensor but not the line of thought. Part V used a laser to measure the three-dimensional world, and in Chapter 19 it carried that technology into an inspection site that must run every day for a year; this part uses a camera to look instead. Students will quickly find, however, that not one of LOAM's four steps—extract features, estimate quickly at the front end, optimize finely at the back end, correct by loop closure—is missing from visual SLAM. The difference lies only in the nature of the features: a LiDAR's features are geometric and come into the world already carrying three-dimensional coordinates, while a camera's features are features of appearance, with only a two-dimensional pixel position, and their depth must be obtained some other way.

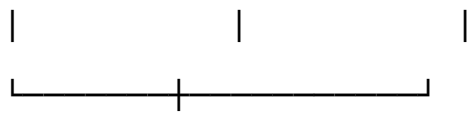
That difference gives this part its two themes. The first is how to choose features and how to match them, which is the content of Sections 20.3 and 20.4 of this chapter; the second is what to do without depth, which is the scale ambiguity of Section 20.5 and the question epipolar geometry sets out to answer in Chapter 21. One more thing is worth keeping in mind from the start: what Section 19.5 measured—a long corridor whose geometric degeneracy leaves a LiDAR unable to say how far it has traveled—is exactly where a camera is at its best, because the posters on the wall, the room numbers, and the texture of the floor are all distinct visual features. Before any of that, this chapter puts the whole framework in place so that students can see which piece each of the next four chapters supplies.

Figure 20-0 The Link Map of Chapter 20

The Foundations Already Laid

Chapter 9 The camera model Chapter 12 Optimization Chapters 18 and 19 LOAM and field

experience



▼

【Chapter 20 Visual SLAM: the framework and the fundamentals of features】

Feature-based / direct methods ─ ORB ─ matching and rejection ─ scale ambiguity

▼

Chapter 21 Epipolar geometry (where depth comes from)

Chapter 22 ORB-SLAM2 Chapter 23 ORB-SLAM3 (visual-inertial)

▼

Part VII Miniature robots and drones Part VIII ROS implementation Part IX AI and semantic navigation

Learning Objectives

In Part V we used a laser to measure the three-dimensional world; from this part on we use a camera to look at it instead. The sensor has changed, but students will find the central ideas entirely common to both—features, registration, optimization, loop closure—only with image pixels in place of a geometric point cloud. This chapter is the doorway to Part VI, and it establishes the overall framework of visual SLAM together with the fundamentals of feature technique. After studying this chapter, students should be able to:

1. Describe the basic pipeline of visual SLAM and indicate which chapters of this book correspond to each of its stages.
2. Compare the principles, the strengths and weaknesses, and the proper domains of the two great schools, the feature-based method and the direct method.

3. Explain the role and the characteristics of ORB, SIFT, SURF, FAST, and BRIEF, and understand why ORB became the mainstream choice for SLAM.
4. Distinguish feature tracking from feature matching and describe the common strategies for rejecting false matches.
5. Explain the scale ambiguity of monocular SLAM and the three ways of resolving it.
6. Compare stereo SLAM with RGB-D SLAM in their differences and their proper domains.
7. Give examples of applications of visual SLAM in AR / VR and in robotics.

Core Concepts

Chinese Term	English Term	Explanation
視覺 SLAM	Visual SLAM	Simultaneous localization and mapping with a camera as the principal sensor.
視覺里程計	Visual Odometry	Estimating camera motion from consecutive images; it accumulates drift.
特徵法	Feature-based / Indirect	Extracts and matches feature points, then solves for motion from the geometric relation.
直接法	Direct Method	Extracts no features and estimates motion directly from differences of pixel brightness.
關鍵點	Keypoint	A salient and repeatably detectable position in an image, such as a corner.
描述子	Descriptor	A vector describing the appearance around a key point, used for comparison.
尺度不變性	Scale Invariance	The ability to detect the same feature as

Chinese Term	English Term	Explanation
		an object's distance changes.
旋轉不變性	Rotation Invariance	The ability to match correctly after the image has been rotated.
漢明距離	Hamming Distance	The number of differing bits between two binary descriptors; extremely fast to compare.
特徵匹配	Feature Matching	Finding the same feature point in correspondence between two images.
特徵追蹤	Feature Tracking	Following the position of the same feature through consecutive images.
尺度不確定性	Scale Ambiguity	The inherent limitation that a monocular camera cannot determine the true size of the map.
詞袋模型	Bag of Words	Representing an image as a statistic over visual words, used for place recognition.

20.1 The Basic Pipeline of Visual SLAM

In Section 9.6 we drew a ladder of knowledge: image processing → feature extraction → multiple view geometry → visual odometry → visual SLAM. This section unfolds that ladder into a complete system pipeline.

Five Stages

1. Sensor data acquisition and preprocessing: read the image and undistort it (the camera calibration of Section 9.2). For a stereo or RGB-D camera, stereo matching or depth alignment is also required.
2. Visual odometry (the front end): estimate the camera motion between consecutive images and triangulate local three-dimensional points. This stage is fast but accumulates drift (the LiDAR odometry of Chapter 18 is its laser counterpart).
3. Back-end optimization: perform a nonlinear optimization over a batch of poses and map points (the bundle adjustment of Chapter 12) to remove the accumulated error.
4. Loop closure detection: recognize that this is a place I have been before, and generate a loop closure constraint to correct the global drift (Chapters 12 and 15). Visual SLAM commonly uses a bag-of-words model for this.
5. Mapping: output a map according to the needs of the application—a sparse map of feature points, a dense point cloud, or a semantic map (Chapters 1 and 3).

Figure 20-1 The Standard Pipeline of Visual SLAM

Image input (monocular / stereo / RGB-D)

↓

① Preprocessing: undistortion, stereo matching (Chapter 9)

↓

② Visual odometry (the front end): feature extraction and matching → estimating camera motion (Chapter 21)

↓

③ Back-end optimization: bundle adjustment refining poses and map points (Chapter 12)

↓

④ Loop closure detection: recognizing a revisited place by bag of words → global correction (Chapter 22)

↓

⑤ Map output: sparse point cloud / dense map / semantic map (Chapter 32)

A Comparison with LiDAR SLAM

Set this pipeline alongside the LOAM of Chapter 18 and the architectures prove strikingly alike: both run feature extraction → fast front-end estimation → fine back-end optimization → loop closure correction. The difference lies only in the nature of the features:

- A LiDAR's features are geometric (edges, planes) and carry three-dimensional coordinates and ranges directly.

- A camera's features are features of appearance (a corner and the texture around it), with only two-dimensional pixel coordinates; their depth must be obtained separately.

That difference gives rise to the central difficulty of visual SLAM: how to recover three-dimensional camera motion and scene structure from two-dimensional observations that carry no depth. This is precisely the question epipolar geometry sets out to answer in Chapter 21.

20.2 Feature-Based Methods and Direct Methods

Visual SLAM has two principal technical routes, and to understand how they diverge is to understand the lay of the land in this field.

The Feature-Based Method (also called the indirect method)

The feature-based method proceeds in three steps:

6. Detection: find key points such as corners in the image.
7. Description: compute a descriptor for each key point, recording what its surroundings look like.
8. Matching: find corresponding points between different images according to the similarity of their descriptors.

The feature-based method is widely adopted in visual odometry because it is robust to camera motion, to changes of illumination, and to moving objects. It is called indirect because it does not use pixel values directly but first abstracts the image into the intermediate representation of a set of feature points, and then solves for motion by geometric means.

- Advantages: robust to changes of illumination (descriptors generally have some invariance to brightness); able to handle fairly large changes of viewpoint; easy to use for loop closure detection (bag-of-words comparison of descriptors); mature and stable.
- Disadvantages: computing features and descriptors takes time; most of the pixel information is thrown away; where texture is scarce (a white wall) no features are found at all and the method fails (the curse of the white wall of Chapter 9).

The Direct Method

The direct method skips feature extraction entirely and uses pixel brightness directly. Its central assumption is brightness constancy: the same point in space should have the same brightness in different images. Estimating motion then becomes this: find a camera motion such that projecting the pixels of the previous frame into the current frame under that motion makes the difference in brightness as small as possible.

- Advantages: no features or descriptors need be computed, so it can be very fast; it can use every pixel in the image that has a gradient (not only corners), and so it still works in scenes where texture is scarce but a faint gradient remains; and it produces a semi-dense or dense map naturally.
- Disadvantages: the assumption of brightness constancy is fragile—changes of illumination, automatic exposure, and non-Lambertian surfaces all break it; convergence is difficult under large motion (a good initial value is needed); and loop closure detection is harder to do.

Representative systems include LSD-SLAM and DSO (Direct Sparse Odometry).

Item of Comparison	Feature-based (indirect)	Direct
Information used	Feature points and descriptors	Pixel brightness
Central assumption	Features can be detected and matched repeatably	Brightness constancy
Against changes of illumination	Fairly robust	Sensitive
Scenes of weak texture	Fails easily	Can still use a faint gradient
Large motion	Handles it fairly well	Diverges easily; needs a good initial value
Loop closure detection	Easy (bag of words)	Harder
Map density	Sparse	Semi-dense or dense
Representative systems	The ORB-SLAM family	LSD-SLAM, DSO

Table 20-1 A Comparison of Feature-Based and Direct Methods

Part VI of this book takes the feature-based method as its main line (the ORB-SLAM family of Chapters 21 through 23), because it is mature and stable, richly resourced, and best suited to teaching. But students should know that the direct method exists—and recent years have also brought hybrid methods that combine the strengths of both.

20.3 ORB, SIFT, SURF, FAST, and BRIEF

These five names often appear together, but their roles are in fact different: some are responsible only for detection, some only for description, and some do both.

Getting this straight is the first point of this section.

Detectors and Descriptors: a Division of Labor

- FAST: a detector only. It finds corners extremely quickly but produces no descriptor.
- BRIEF: a descriptor only. It produces a binary descriptor vector for a given key point but is not responsible for finding points.
- SIFT, SURF, ORB: both at once, detecting as well as describing.

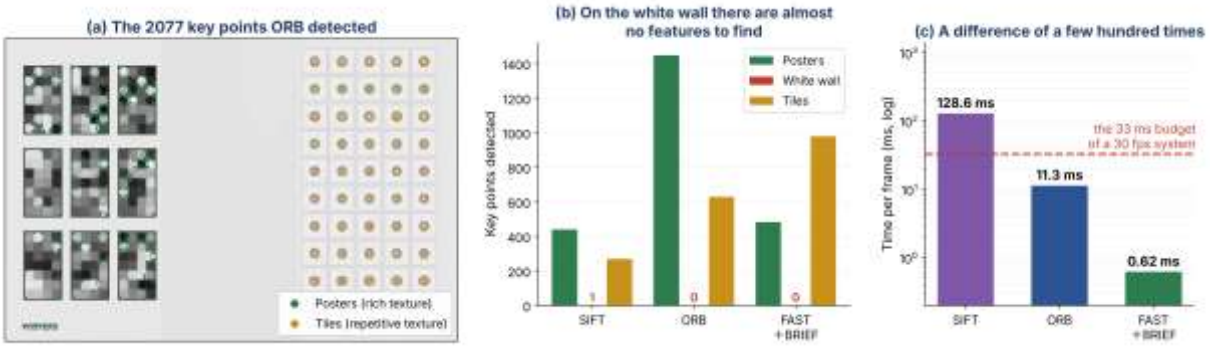
In practice these methods can be used independently and separately as detectors or as descriptors, and different combinations give different performance. This explains why combined forms such as FAST-BRIEF and SIFT-SURF appear in the literature.

An Introduction to the Five Principal Algorithms

- SIFT (Scale-Invariant Feature Transform): the classic of classics. It finds extrema across different scales by a difference of Gaussians and produces a 128-dimensional descriptor with excellent invariance to scale and rotation. SIFT essentially merges feature detection and the generation of the description vector into a single key derivation step. Its drawback is its computational cost.
- SURF (Speeded-Up Robust Features): an accelerated version of SIFT that approximates the Gaussian operations with an integral image and box filters. On the whole SURF demands less computation than SIFT.
- FAST (Features from Accelerated Segment Test): simple and fast, it compares a pixel with the pixels on a Bresenham circle around it, first checking whether the pixel is brighter or darker than N consecutive pixels on that circle; if it is, it may be a corner. The algorithm then applies non-maximum suppression to identify and retain the most discriminative key points.

- BRIEF (Binary Robust Independent Elementary Features): produces a binary string descriptor by comparing the brightness of randomly chosen pairs of points. Its advantages are that it is extremely fast to produce and that comparison requires only a Hamming distance (a bitwise operation), which is highly efficient. Its drawback is that the original version has neither rotation nor scale invariance.
- ORB (Oriented FAST and Rotated BRIEF): combines FAST and BRIEF while improving each—it adds orientation information to the FAST corner (oriented FAST) and lets the BRIEF descriptor rotate with that orientation (rotated BRIEF), thereby gaining rotation invariance.

Figure 20-2 One Wall, Three Feature Algorithms: Number, Distribution, and Cost



To give the comparison a control, this book synthesized a classroom wall deliberately divided into three regions: on the left are nine different posters (rich and distinctive texture), in the middle a stretch of nearly textureless white wall, and on the right a row of identical tiles (repetitive texture). These three regions correspond exactly to the three situations visual SLAM will meet.

Panel (a) draws the 2,077 key points ORB detected onto the image, and the result is plain at a glance: the poster region is thick with them, the tile region has a good many as well, and on that stretch of white wall in the middle there is not a single point. Panel

(b) sets the per-region counts of the three algorithms side by side; the numbers for the white wall region are 1 for SIFT, 0 for ORB, and 0 for FAST—the most direct evidence there is for the curse of the white wall of Chapter 9. Note that this is not a matter of one algorithm being inferior: a place with no variation in brightness simply has no information to extract, and all three methods fail there impartially.

Panel (c) is the reason for choosing ORB. On the same 960×640 image, SIFT takes 128.6 milliseconds per frame, ORB 11.3 milliseconds, and plain FAST detection only 0.62 milliseconds. SIFT is more than eleven times slower than ORB and more than two hundred times slower than FAST. The red dashed line marks the total budget of 33 milliseconds per frame for a 30 fps system—and into those 33 milliseconds must also be fitted matching, pose solving, and the map update. SIFT overruns the budget nearly fourfold on feature extraction alone, while ORB uses up only a third of it. What Case 20-1 calls the cost and the reward of speed is exactly this red line.

Panel (b) also hides a trap worth noting now: the number of features in the tile region is in fact considerable, with ORB finding 628 points there and FAST as many as 979. A large number does not mean a useful one—Figure 20-3 in the next section will reveal that these points are almost all a source of disaster.

The Experimental Conditions of This Figure

The image is a 960×640 grayscale synthetic image; ORB is capped at 2000 points, SIFT and FAST use OpenCV's default parameters, and the times are averages of five repetitions on the same image.

The absolute numbers will vary with image size, parameters, and hardware; what matters is the difference in order of magnitude: more than an order of

magnitude between SIFT and ORB, a conclusion that is quite robust.

The original ORB paper reports it to be about 14 times faster than SURF and SURF about 24 times faster than SIFT (Section 20.3), and the SIFT / ORB ratio in this experiment agrees with that magnitude.

Students may rerun this in Part One of Activity 20-1 with classroom photographs of their own, and should look in particular for a white wall and a tiled floor to reproduce the three situations of panel (b).

Why ORB Became the Mainstream in SLAM

The answer is speed. ORB's developers report it to be 14 times faster than SURF, and SURF in turn 24 times faster than SIFT; this newer algorithm also performs roughly as well, and in some situations considerably better. Other literature puts ORB's extraction speed at about 15 times that of SURF while retaining robustness to changes of rotation and scale, an efficiency that makes it particularly suited to real-time applications.

Among the various feature extraction algorithms of visual SLAM, the advantage of the ORB feature lies in the speed of extraction and matching, while little accuracy is lost against SIFT and SURF. ORB has therefore become the mainstream method of feature point detection in SLAM systems—it trades a portion of accuracy and robustness for a gain in computational speed, a bargain that is extremely favorable for a real-time system.

The Ingenuity of the ORB Descriptor

A detail of the design of the ORB descriptor is worth mentioning. In ORB the BRIEF descriptor is a 256-bit vector, each bit of which records the result of comparing the

mean intensities of two 5×5 pixel windows within the same 31×31 pixel patch; there are 676 such windows in a patch, and therefore more than two hundred thousand possible pairwise comparisons. To maximize the discriminative power of the descriptor, ORB selects from those two hundred thousand combinations the 256 comparisons that are farthest from the mean and least correlated with one another.

This design displays an important kind of engineering thinking: not choosing at random, but picking the most informative comparisons in a data-driven way. Should students one day design features or metrics of their own, this principle of maximizing discriminative power while minimizing redundancy is worth taking as a model.

Algorithm	Role	Form of descriptor	Speed	Invariance
SIFT	Detection + description	128-dimensional floating point	Slowest	Scale, rotation
SURF	Detection + description	64 / 128-dimensional floating point	Medium	Scale, rotation
FAST	Detection only	None	Extremely fast	None (original version)
BRIEF	Description only	Binary string	Extremely fast	None (original version)
ORB	Detection + description	256-bit binary	Fast	Rotation, and scale in part

Table 20-2 A Comparison of the Five Feature Algorithms

20.4 Feature Tracking and Feature Matching

Once features have been found, the next step is to establish correspondences

between different images. There are two lines of thought here, which students often confuse; this section sets them apart.

Feature Matching: Comparing Descriptors

Matching works like this: extract features and descriptors independently in each of two images, then, for every feature of the first, find the one in the second whose descriptor is most similar.

- Similarity measure: floating-point descriptors (SIFT, SURF) use the Euclidean distance; binary descriptors (ORB, BRIEF) use the Hamming distance—one need only count how many bits differ between two bit strings, which an XOR and a bit count accomplish extremely quickly. This is the greatest advantage of the binary descriptor.
- Where it applies: when the two images differ considerably (a changed viewpoint, a long interval of time, loop closure detection), matching is the only workable method.

Feature Tracking: Searching Nearby

Tracking works like this: after extracting features in the first image, do not extract them again in the second, but assume that the features have moved only a short distance and search directly near their original positions for the most similar image patch. The typical method is optical flow (Section 9.4).

- Advantages: no descriptor need be computed, so it is faster; and it works well on consecutive images.
- Limitations: it applies only where the image changes little (a high frame rate, slow motion); and after tracking for a while drift accumulates, so a feature may be lost or wander off onto something else.

False Matches: a Reality That Must Be Faced

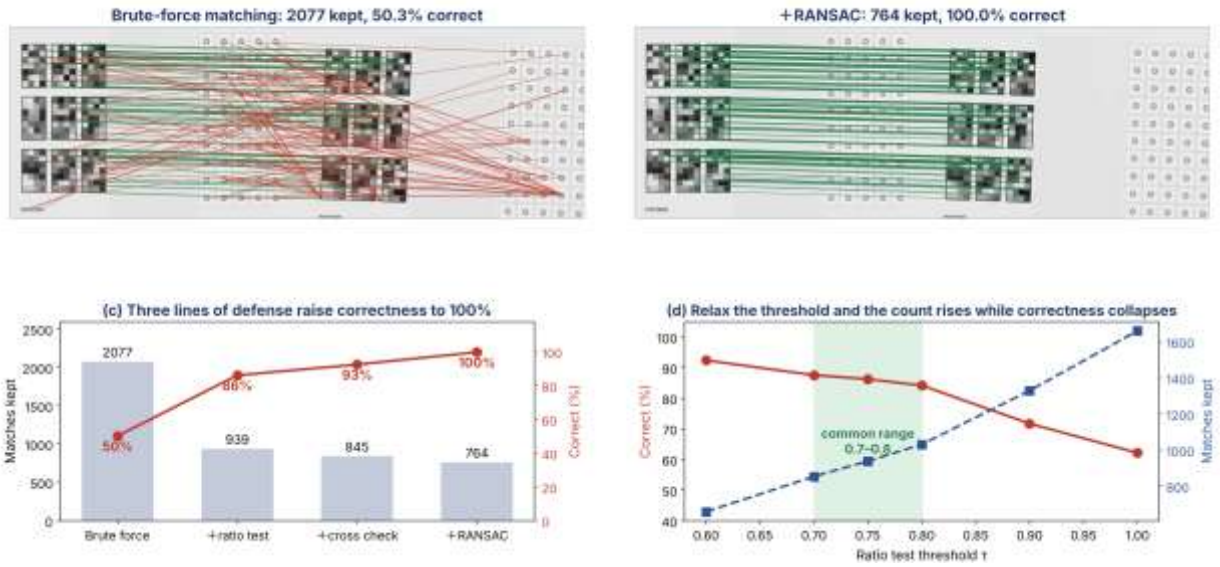
Whether one matches or tracks, wrong correspondences are inevitable. The causes include repetitive texture (tiles, railings, leaves, all looking the same everywhere), too great a change of viewpoint, changed illumination, and moving objects. And in a geometric solution a few severe false matches may be enough to pull the result off course—one of the most common causes of failure in visual SLAM.

Three Strategies for Rejecting False Matches

9. The ratio test: for each feature, find the two most similar candidates, the best and the second best. If their distances are close together, the match is hard to distinguish and therefore unreliable, and it is discarded. This is the classic approach proposed in the SIFT paper, simple and effective.
10. The cross check: the feature a of image A matches the feature b of image B, and b in turn matches back to a; only then is the match accepted. A one-way match is taken as unreliable.
11. Geometric verification (RANSAC): the most important move of all. Using the geometric constraint that all correct matches must conform to one and the same camera motion, it samples at random and solves repeatedly, counts those that conform, and rejects as outliers those that do not. In practice, estimating the essential matrix and the inlier set with RANSAC is the standard way of estimating the relative pose of a camera from feature correspondences that contain outliers. This method will be unfolded in detail in Chapter 21.

Figure 20-3 Three Lines of Defense Against False Matches: from 50 Percent

Correct to 100 Percent



The effect of rejecting false matches can usually be judged only by eye, from whether the lines look orderly; but if the second image has been produced from the first by a known homography, then every match has a right answer—project the feature point of the first image through that matrix, and if the result falls within four pixels of the matched point, the match is correct. This figure establishes its ground truth in exactly that way.

Panel (a) is brute-force matching with no rejection at all: 2,077 matches, the green lines correct and the red ones wrong, for a correctness of 50.3 percent—the same as tossing a coin. Note that the red lines are not evenly distributed: a great many of them run from the tile region on the right across to the poster region on the left, a plainly systematic confusion. Panel (b) is the result after all three lines of defense: 764 matches at 100 percent correct, the lines neatly displaying a single direction of translation.

Panel (c) takes the process apart. The ratio test cuts 2,077 matches down to 939 at a stroke, and correctness leaps from 50.3 percent to 86.3 percent—this is the decisive cut. The cross check trims further to 845 at 92.7 percent; and finally RANSAC clears away

the remaining outliers by geometric consistency, leaving 764 at 100 percent. The three lines of defense discard 63 percent of the matches in all, and what they buy is 764 correspondences that can be trusted completely. For back-end optimization this is an excellent bargain: a few correct constraints are far better than a great many constraints adulterated with wrong ones.

Panel (d) answers the second discussion question. As the ratio threshold τ is relaxed from 0.6 to 1.0, the number of matches retained rises from 659 to 1,661, but correctness collapses all the way from 92.4 percent to 62.2 percent. The curve turns sharply after 0.8—which explains why the values recommended in the literature almost all fall between 0.7 and 0.8 (the green band in the figure). The meaning of τ is in fact quite intuitive: it asks whether the best candidate is appreciably better than the second best, and $\tau = 1.0$ amounts to not asking at all, degenerating naturally into brute-force matching.

What Happened in the Tile Region: Case 20-2 Quantified

Region by region, brute-force matching is 69.8 percent correct in the poster region and only 1.0 percent in the tile region—of 598 matches, just 6 are right. Repetitive texture is not merely harder; it is all but bound to go wrong.

The ratio test behaves quite differently in the two regions as well: in the poster region it keeps 587 of 1,479 matches (about four tenths), while in the tile region it keeps only 10 of 598 (under 2 percent).

This is precisely the design intent of the ratio test: where everything looks alike, the distances of the best and second-best candidates are bound to be close, and the algorithm honestly says it cannot tell them apart and gives up, rather

than forcing a choice.

After RANSAC at the end, only 2 matches remain in the tile region. Few, but right—and far safer than keeping 598 of which 99 percent are wrong.

20.5 The Scale Ambiguity of Monocular SLAM

This is the most fundamental and the most inescapable limitation of monocular visual SLAM. Section 9.1 pointed out its geometric root in the pinhole projection formula; this section explains its consequences and its remedies.

The Nature of the Problem

Recall the projection formula of Section 9.1: $x = f \cdot X/Z$. Note that in this expression, if the three-dimensional coordinates of every point in the scene are scaled up by a factor of k while every translation of the camera is scaled up by k as well, the result projected onto the image is exactly the same.

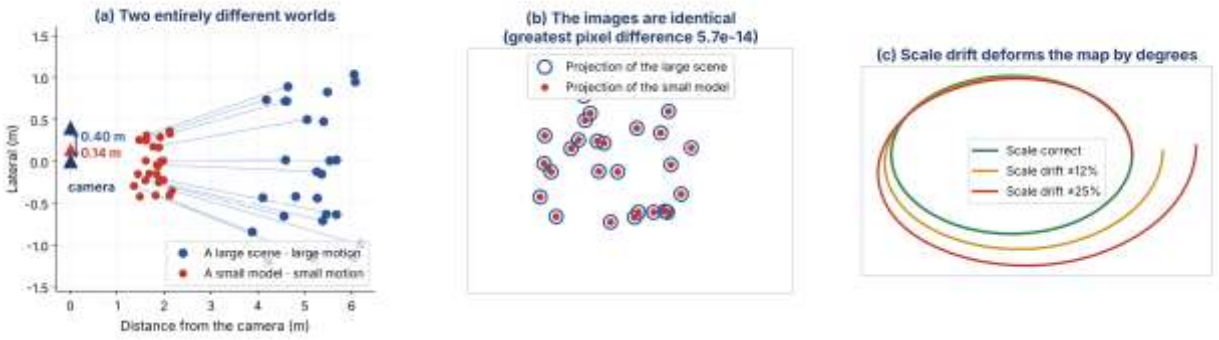
In other words, a sequence of images taken by a monocular camera may correspond to real motion in a real room, or equally to a minute motion in a model house—and the two cannot be told apart in the image at all. Monocular SLAM can therefore recover a trajectory and a map only up to an unknown scale factor.

Consequences

- The map has no true size: the map built can say that this wall is 3.7 units from that one, but not how many meters a unit is.
- It cannot be used for navigation directly: the robot does not know whether the obstacle ahead is 0.5 meters or 5 meters away, and so of course cannot avoid it safely.

- Scale drift: more troublesome still, this unknown scale factor also changes slowly over a long run (called scale drift), so that the map gradually deforms.

Figure 20-4 Scale Ambiguity: Two Worlds a Monocular Camera Is Inherently Unable to Tell Apart



Section 20.5 says that scaling the scene by k and the camera translation by k as well leaves the image exactly the same, and that statement can be verified by direct calculation. Panel (a) is a side view: in blue is a real room, with points distributed between 3.6 and 6.2 meters from the camera and the camera translating 0.40 meters; in red the whole scene and the translation have been scaled down to 0.35 times—a model house set in front of one's eyes, with the camera moving only 0.14 meters. Physically the two are entirely different worlds.

Panel (b) projects the two sets of three-dimensional points into images with the same camera intrinsics. The hollow blue circles are the projection of the large scene and the solid red dots that of the small model—they are not merely close together but exactly coincident. This book actually computed the greatest pixel difference between the two projections: 1.1×10^{-13}, which is to say the rounding error of floating-point arithmetic, and mathematically zero. This is not a matter of the algorithm not being clever enough; it is decided by the projection equation itself: a monocular image simply carries no information about scale.

Panel (c) then displays what this ambiguity actually looks like in practice—scale drift. The green line is the true trajectory at the correct scale, a closed ellipse. If the scale factor of each displacement drifts slowly as a random walk by ±12 percent (the amber line) or ±25 percent (the red line), the trajectory gradually deforms and its two ends no longer meet. Note that the direction of every step here is correct; the only thing wrong is how far each step went, yet the accumulation is enough to distort the whole map.

This is why monocular SLAM is almost never used alone. The three remedies of Section 20.5—add an IMU, change to a stereo or RGB-D camera, or introduce a reference object of known size—all amount to supplying a definition of the meter from outside. It is worth mentioning that this limitation has not been broken to this day: even the newest monocular SLAM systems built around a foundation model, to be introduced in Section 20.8, still optimize on Sim(3) and still perform a scale alignment before reporting their evaluation—because what they face is the same geometric fact.

The Difference Between Scale and Depth Estimation

A deep learning model (of the Depth Anything kind) can guess depth from a single image, but that is a prior learned from training data—seeing a door frame, it infers a height of about 2 meters, rather than measuring it.

Such a prior works well in ordinary scenes and errs systematically in scenes of unusual size: a model house, an enlarged prop, an aerial top-down view are all typical failures.

By contrast, an accelerometer measures acceleration in physical units and a stereo baseline is a fixed distance measured with a ruler—what these two supply is a measurement, not a guess.

Practical advice: where safety must be assured (obstacle avoidance, grasping, navigation), use a measurement; where a visual effect is wanted (AR overlays, dense reconstruction), a prior is acceptable.

Three Ways of Resolving It

12. Introduce an IMU (visual-inertial): an accelerometer measures acceleration in physical units, and integrating it supplies true scale. This is the most general remedy, and it is the central theme of Chapter 7 and Chapter 23. The price is that sufficient excitation of motion is required before scale becomes observable (Section 7.7).
13. Use a stereo or RGB-D camera: the baseline of a stereo pair is a known fixed distance, and an RGB-D camera outputs depth in meters directly. Solving the problem in hardware is the least troublesome approach (Section 20.6).
14. A reference object of known size: if the scene contains an object of known size (an AprilTag or a checkerboard, as in Chapter 4), or if the height of the camera above the ground is known, the scale can be recovered. This is quite practical in particular applications.

It should be added that recent years have also brought the practice of estimating depth from a single image by deep learning (models of the Depth Anything kind in Section 9.4) to supply a prior on scale, though its accuracy and reliability still fall short of the three physical methods above.

20.6 Stereo SLAM and RGB-D SLAM

Section 9.3 introduced three ways for a camera to obtain depth; this section focuses on how they differ within a SLAM system.

Stereo SLAM

A stereo camera photographs with two lenses of known baseline at the same moment and computes depth directly from the disparity between the left and right images (the formula Z = f·b/d of Section 9.6).

- Advantages: depth from a single frame; absolute scale; operation indoors and out alike (it does not depend on active projection); and a measuring range that can be extended by enlarging the baseline.
- Limitations: the relative position of the two cameras (the extrinsics) must be calibrated precisely; on a textureless surface no correspondence is found and so there is no depth; the depth error grows with the square of the distance (Section 9.6); and the computational load is fairly heavy (stereo matching must be performed).
- Where it applies: outdoor robots, drones, self-driving cars—anywhere that must work in sunlight and over a fairly large range of distances.

RGB-D SLAM

An RGB-D camera actively projects an infrared pattern to obtain dense depth (the structured light or active infrared stereo of Section 9.3).

- Advantages: it outputs the depth of every pixel directly, with no stereo matching to do oneself; it has depth even on a textureless white wall (because the projection is active); it has absolute scale; and a dense three-dimensional map is easily built.
- Limitations: it mostly relies on infrared and fails in strong sunlight; its effective range is usually between 0.2 and 10 meters; it is equally helpless before glass

and mirrors (the same weakness as the LiDAR of Chapter 8); and its power consumption is fairly high.

- Where it applies: indoor service robots, desktop three-dimensional reconstruction, indoor surveying—which is also why the implementation of Chapter 30 chooses RGB-D.

Item of Comparison	Monocular SLAM	Stereo SLAM	RGB-D SLAM
Absolute scale	None (must be supplied externally)	Yes	Yes
Source of depth	Triangulation from motion	Left-right disparity	Active projection
Textureless surfaces	Fails	No depth	Still has depth
Strong sunlight outdoors	Usable	Usable	Mostly fails
Effective range	Unlimited (but without scale)	Fairly far	About 0.2–10 m
Map density	Sparse	Can be semi-dense	Dense
Hardware cost	Lowest	Medium	Medium

Table 20-3 A Comparison of the Three Visual SLAM Configurations

One great merit of ORB-SLAM2 and ORB-SLAM3 (Chapters 22 and 23) is that they support all three of these modes at once—one and the same system, with only the input mode changed. This lets students compare the three within a single framework,

which makes excellent teaching material.

20.7 Applications of Visual SLAM in AR / VR and in Robotics

Section 1.4 of Chapter 1 mentioned AR / VR as one of the four great applications of SLAM; this section goes deeper into the role of visual SLAM within it and extends to robotic applications.

AR / VR: Visual SLAM in Your Pocket

For AR glasses or AR on a phone to pin a virtual object stably to a real tabletop, the device must know its own six-degree-of-freedom pose in space in real time. The industry commonly calls this capability spatial localization or inside-out tracking, and technically it is nothing other than visual-inertial SLAM.

- Why vision and not LiDAR: an AR device is extremely sensitive to weight, power consumption, and cost, and a camera plus an IMU is the only workable combination (the advantages of vision in Section 9.5).
- Special requirements: the latency must be extremely low (otherwise the virtual object drifts and causes discomfort), and tracking must be maintained even as the user turns their head quickly—a severe test for visual SLAM, and the reason an IMU is indispensable (Section 7.6).
- The evolution of the VR headset: early VR required external base stations for localization, while a modern headset uses cameras on the body of the device to perform visual SLAM, so that the user can walk about freely—an example of SLAM changing the form of a product directly.

Robotic Applications

- Indoor service robots: what visual SLAM supplies is not only geometry but also semantics (Section 9.7)—the robot can recognize that this is a door and that is an elevator, which a LiDAR cannot give (the theme of Chapter 32).
- Drones: payload and electrical power are extremely constrained, and a camera plus an IMU is the best choice (the theme of Chapter 26).
- Low-speed autonomous driving and campus vehicles: the application of visual SLAM in low-speed autonomous driving is an active direction of research, and the feature-based method is widely adopted for its robustness to camera motion, to changes of illumination, and to moving objects.
- Place recognition and relocalization: visual features are naturally suited to judging whether I have been here before. The bag-of-words technique is among the most widespread approaches, using local feature descriptors as visual words; and DBoW2 brings in the efficient FAST detector together with the BRIEF descriptor, in the form of a binary bag of words, cutting the time needed for feature extraction sharply against the traditional practice of using SURF and SIFT, so that the system is both more efficient and more robust. This is precisely the technical basis of loop closure detection in ORB-SLAM (Chapter 22).

The Trend: the Arrival of Deep Learning

Traditional hand-crafted feature methods have one problem: they cannot adapt effectively to every complicated and unforeseeable situation; many researchers have therefore adopted convolutional neural networks to extract deep features from images, applying them at every stage—visual odometry, pose estimation, loop closure detection, and the rest.

This is one of the most active directions in visual SLAM at present: replacing hand-

crafted features with learned ones, performing place recognition with a neural network, and even estimating pose end to end. In working systems, however, the traditional geometric methods remain the mainstream for their interpretability and their stability; learned methods mostly serve as an auxiliary or are used for particular difficult scenes. Chapter 32 returns to this theme and discusses how AI and SLAM are brought together.

At this point students have established the overall framework of visual SLAM. In the next chapter we go deeper into the mathematical core of the front end—epipolar geometry: what geometric relation is in fact hidden between two images? What are the essential matrix and the fundamental matrix? How is the PnP problem solved? How does triangulation recover three-dimensional points? These are the bridge that turns two-dimensional pixel correspondences into three-dimensional camera motion.

20.8 After Features: Three Turns in the Visual Front End, 2024–2026

The SIFT and ORB introduced in this chapter are hand-crafted features, and Case 20-3 has already heralded the change deep learning brings. That change has accelerated markedly in the past two years, and it runs deeper than merely changing to another kind of feature. This section sets out three turns.

One: Learned Features Can Finally Run Fast Enough

Learned local features (SuperPoint, for instance) long ago surpassed ORB in accuracy, but for years they were stuck on the same problem: too slow, and in need of a GPU. That is exactly the barrier set by the 33-millisecond red line of Figure 20-2(c). XFeat, of 2024, was designed against precisely this point—it is built to run in real time on the CPU of an inexpensive laptop, with no specialized hardware acceleration, and it is up to

five times faster than existing lightweight deep features while matching the accuracy of far larger models such as SuperPoint.

There has been a similar evolution on the matching side. Methods of the LightGlue kind let a deep matcher adaptively stop computing early according to the difficulty of the images, so that an easy image pair runs through fewer layers. Taken together the meaning is this: the three lines of defense of Figure 20-3 (the ratio test, the cross check, and RANSAC) are being replaced by a network that has learned to judge whether a correspondence can be trusted. It must be stressed, however, that the geometric consistency verification RANSAC stands for has not disappeared; it has merely retreated from being the principal line of defense to being the final confirmation.

Two: From Between Two Images to Putting Out Three Dimensions Directly

The second turn is more fundamental. The front end of traditional visual SLAM is a long chain: extract features → match → reject → solve the epipolar geometry → triangulate. The line of work of DUSt3R and MASt3R proposes instead that the network simply put out the three-dimensional coordinates of every pixel from two images, skipping that whole string of intermediate steps at once.

MASt3R-SLAM (CVPR 2025), built upon it, is a complete monocular dense SLAM system: it replaces the KD tree or brute-force search with dense point map matching, and it does not even need to know the camera intrinsics beforehand. On the TUM RGB-D dataset its mean absolute trajectory error is 0.030 meters when calibrated and 0.060 meters when uncalibrated; on an ordinary desktop machine with a high-end graphics card it reaches about 15 fps.

But there is a detail here that students must see clearly, and it answers directly to Figure 20-4: MASt3R-SLAM has not solved the scale ambiguity. That paper states plainly that scale is the principal source of inconsistency among its predictions, which is why it optimizes pose on Sim(3), and its monocular evaluation results are reported only after a scale alignment has been performed. In other words, even with the entire front end replaced by a foundation model, it remains true that a monocular camera cannot see absolute scale—because that is a property of the geometry and not a defect of the algorithm.

Three: Map Representation, from a Sparse Point Cloud to a Renderable Scene

The third turn belongs to the same current as the three-dimensional Gaussian splatting mentioned in Section 17.8, only it is still more natural in the visual domain. What traditional feature-based SLAM builds is a sparse map of feature points—adequate for localization, but no more than a cloud of scattered dots to look at, useless for rendering and unhelpful for any later semantic understanding. The dense visual SLAM of the past two years represents the scene directly in a renderable form, making localization and the reconstruction of a lifelike scene two sides of one thing.

This matters especially for the AR / VR applications discussed in Section 20.7: what AR needs has never been only the camera pose but also a knowledge of which real surfaces will occlude the virtual object. A sparse point cloud cannot give that; a dense renderable representation has it inherently. The price is equally plain, however—both memory and computation are far greater than for a sparse map, which is why robot navigation still works mainly with sparse or semi-dense maps.

The Thread Common to the Three Turns

Front-end components (XFeat / LightGlue): learned features have at last crossed the threshold of real-time operation and are beginning to replace ORB.

Front-end structure (MASt3R-SLAM): the whole chain of feature—match—geometry is crossed at a stride by an end-to-end network.

Map representation (dense and renderable): the map turns from sparse points for localizing into a scene that can be seen, can occlude, and can be understood.

What has not changed: the scale ambiguity is still there (Figure 20-4), geometric consistency verification is still needed, and there is still no information to extract from a white wall.

The common thread: what is being replaced is how it is computed, not what is computed. The four basic questions of this chapter—where are the features, which correspondences can be trusted, where does depth come from, and who supplies the scale—remain the standard by which any new system is judged.

Carrying Forward

This chapter has established the overall framework of visual SLAM together with the fundamentals of features. Figure 20-2 showed where features are to be found and why ORB is the choice; Figure 20-3 showed that matching must go wrong and how three lines of defense rescue correctness from 50 percent back to 100 percent; and Figure 20-4 showed the inherent limit of the monocular camera. These three things are also the background to every design decision of the ORB-SLAM family in Chapters 22 and 23.

But we have not yet answered the most central question: given a set of trustworthy two-dimensional correspondences, how exactly does one compute how far the camera has moved, and where those feature points lie in three-dimensional space? The next chapter enters epipolar geometry—the essential matrix, the fundamental matrix, the homography, and triangulation. This is the mathematical heart of the whole of visual SLAM, and the true face of that geometric constraint RANSAC verifies in Figure 20-3.

Formulas and Procedures

One: the Hamming Distance, for Comparing Binary Descriptors

The similarity of two binary descriptors d_i and d_j is measured by the Hamming distance—the number of bits at which they differ:

$$Hamming(d_i, d_j) = popcount(d_i \ XOR \ d_j)$$

Count the ones after an XOR (popcount). For a 256-bit ORB descriptor this takes only a few bitwise instructions, far faster than computing the Euclidean distance of a floating-point descriptor. This is one of the keys to ORB's real-time performance. The smaller the distance, the greater the similarity.

Two: the Ratio Test

For a given feature, let d_1 be its distance to the most similar candidate and d_2 its distance to the second most similar; the match is accepted on the condition that

$$d_1 \ / \ d_2 < \tau \quad (\tau \text{ is commonly taken between 0.7 and 0.8})$$

Intuitively: if the gap between the best and the second best is not pronounced enough (a ratio near 1), this feature has several similar candidates in the image and is hard to distinguish, so it should be discarded. This simple criterion rejects a great many

false matches.

Three: the Photometric Error of the Direct Method

The direct method does not compare descriptors but minimizes the photometric error—the difference in brightness after projecting the pixels of the previous frame into the current frame under the camera motion T:

$$T^* = \arg\min \Sigma_i \| I_2(\pi(T \cdot P_i)) - I_1(u_i) \|^2$$

where I_1 and I_2 are the brightnesses of the two frames, u_i a pixel in the first frame, P_i the three-dimensional point corresponding to it, and $\pi(\cdot)$ the projection function of Chapter 9. Compared with the feature-based method: the feature-based method minimizes a reprojection error (a geometric distance, in pixels), while the direct method minimizes a photometric error (a difference in brightness). Both are least squares problems (Chapter 12); only the definition of the error differs.

Four: the Front-End Processing Pipeline of Visual SLAM

Figure 20-5 The Per-Frame Processing Flow of a Feature-Based Front End

① Read the image → undistort (Section 9.2)

↓

② Detect key points: find corners with FAST and apply non-maximum suppression

↓

③ Compute orientation and descriptor: oriented FAST + rotated BRIEF → a 256-bit descriptor

↓

④ Match against the previous frame (or the map): compare by Hamming distance

↓

⑤ Reject false matches: the ratio test → the cross check → geometric verification by RANSAC

↓

⑥ Solve for pose: estimate the camera motion from the verified matches (Chapter 21)

↓

⑦ Triangulate new three-dimensional points → hand them to the back-end optimization (Chapters 12 and 22)

Cases

Case 20-1 The Cost and the Reward of Speed: Why ORB Won the SLAM Battlefield

Judged on the quality of the features alone, SIFT is superior to ORB on nearly every measure—its scale invariance is more complete, its matching more accurate, its tolerance of changes in viewpoint greater. Why then do the great majority of real-time SLAM systems choose ORB?

The answer is the harsh demand of running in real time. A SLAM system running at 30 frames per second has a budget of about 33 milliseconds for each frame, and within that time it must also complete feature extraction, matching, pose solving, the map update, and everything else. ORB's developers report it to be 14 times faster than SURF, and SURF in turn 24 times faster than SIFT—which works out to ORB being roughly two orders of magnitude faster than SIFT. That gap is not a matter of being somewhat quicker; it is the dividing line between running in real time and not.

More important still, the accuracy ORB sacrifices can be made good within a SLAM system. SLAM has back-end optimization and loop closure correction (Chapter 12), and even if a single match is slightly inferior, a great many redundant observations and a global optimization can press the error down. In other words: design at the level of the system compensates for a shortfall at the level of the component.

This case gives students an important engineering outlook: in choosing a technical component one must look not at that component's isolated figures of merit but at its contribution and its cost within the whole system. This is of a piece with sensor selection is an overall judgment in Chapter 8 and the design orientation is decided by the needs of the application in Chapter 15.

Case 20-2 The Trap of Repetitive Texture: Tiled Floors and Railings

Here is a failure experiment easily reproduced in a university classroom: have a robot run visual SLAM while moving over a floor paved with identical tiles, or along a row of identical railings. The result is often that the pose estimate suddenly jumps to something wrong and the map shows a severe misalignment.

The cause is systematic false matching produced by repetitive texture. The corner of every tile looks exactly the same and the descriptors are nearly identical. In comparing them the algorithm may well match the corner of this tile to the corner of the one next door—and it looks like a perfect match, with an extremely small descriptor distance.

This is precisely the point of the ratio test: when the distances of the best and second-best candidates are very close together (as they must be in a tiled scene), the ratio test judges the match indistinguishable and discards it, preferring fewer matches to wrong ones. Set the ratio threshold too loosely and scenes of this kind will go wrong.

Set beside the long corridor of Section 13.6, the structural degeneracy of Section 17.3, and the curse of the white wall of Chapter 9, students should see a pattern running through the whole book: repetitiveness of the environment and a poverty of features are the common enemies of SLAM, whether the sensor is a laser or a camera. Understand this and you will be able to foresee, at the planning stage of a project, which sites will give trouble.

Case 20-3 From Hand-Crafted Features to Deep Learning: a Change Now Under Way

SIFT, SURF, and ORB, introduced in this chapter, are all hand-crafted features—

human researchers designed rules for detection and description out of their own understanding of images. These methods dominated computer vision and SLAM for more than twenty years.

But they share one weakness: they cannot adapt effectively to every complicated and unforeseeable situation; many researchers have therefore turned to convolutional neural networks to extract deep features from images, applying them at the stages of visual odometry, pose estimation, and loop closure detection. Learned features can adapt to situations that hand-crafted rules describe only with difficulty—changes of season, the difference between day and night, the effects of weather.

Students should note, however, that this change is not yet complete, and that it will not necessarily supplant the old methods entirely. The traditional geometric methods have several advantages that learned methods can hardly match at present: they are interpretable (when they go wrong one knows why), they need no training data, their behavior is predictable in scenes they have not seen before, and their demands on computational resources are definite. Mainstream working systems still take geometric methods as their skeleton, with learned methods as a reinforcement at particular points.

The advice for readers of this book is to learn the traditional methods thoroughly first. Understanding why ORB is designed as it is, why RANSAC works, and what bundle adjustment is solving—these principles are a framework of thought that will not go out of date. With that foundation, when you come to look at the deep learning methods you will know what they are replacing and what they are keeping. Chapter 32 returns to this theme.

Hands-On Activities

Activity 20-1 Feature Extraction, Matching, and the Rejection of False Matches

Purpose: to implement the complete front-end pipeline of visual SLAM with your own hands, to compare the performance of different feature algorithms, and to witness the power of rejecting false matches. This activity uses Python with OpenCV and needs only an ordinary camera or images already to hand, and it is a direct warm-up for Chapters 21 through 23.

Part One: a Comparison of Feature Algorithms (everyone)

1. Prepare two photographs of the same scene from slightly different viewpoints (photograph the classroom with a phone, stepping a short distance forward or back between shots).
2. Detect key points with SIFT, ORB, and FAST in turn, draw the results on the images and compare them side by side: how many points does each detect? Where are they distributed?
3. A timing comparison: record the extraction time of the three methods on the same image and verify the differences in speed mentioned in this chapter (ORB far faster than SIFT).
4. Repeat the experiment on a photograph of a large expanse of white wall and observe that all three methods detect almost no features at all—verifying once again the curse of the white wall of Chapter 9.
5. A rotation test: rotate the second image by 45 degrees and compare the matching success rate of ORB against the original BRIEF, verifying the value of rotation invariance.

Part Two: Matching and the Rejection of False Matches (everyone)

6. Use ORB to perform brute-force matching between the two images (by Hamming distance), draw all the match lines, and observe by eye how many are plainly wrong.
7. Apply the ratio test (threshold 0.75), redraw the matching result, and count how many matches were rejected and how good the remainder is.
8. Add the cross check and observe the degree of improvement once again.
9. Use RANSAC to estimate the geometric relation between the two images (a homography will do to begin with), display inliers and outliers in different colors, and observe which matches RANSAC rejects.
10. A ratio threshold experiment: set the threshold to 0.6, 0.75, and 0.9, compare the number of matches retained against the correctness, and find the value best suited to your own scene.

Part Three: Repetitive Texture and Tracking (university level)

11. A repetitive texture experiment: photograph a tiled floor, a railing, or a latticed window, run the matching, and observe the false matches, reproducing Case 20-2. Compare how important the ratio test is in scenes of this kind.
12. Optical flow tracking: record a video of slow motion, track feature points with OpenCV's Lucas-Kanade optical flow, draw out the tracks, and observe which points are tracked stably and which are lost.
13. Tracking against matching: on the same video, run both extract and match afresh every frame and optical flow tracking, and compare the time taken and the stability of the two against the discussion of Section 20.4.

Discussion Questions

1. By what factor do the extraction times of ORB and SIFT differ? Does this agree with the figures quoted in this chapter? If your SLAM system must run at 30 fps, which of them is workable?
2. What percentage of the matches did the ratio test reject? Is the quality of the remainder appreciably better? What happens if the threshold is relaxed to 0.9?
3. In a scene of repetitive texture, what pattern do the false matches display (are they random, or systematically biased in some direction)? Why is a systematic false match more dangerous than a random one?
4. Under what conditions does optical flow tracking lose a point? What does this suggest about the relation between the speed of camera motion and the frame rate?
5. (University level) Following Table 20-3, if you had to choose a visual configuration for an indoor tour-guide robot at a university, would you choose monocular plus IMU, stereo, or RGB-D? Give your reasons.

Extended Challenge (university level)

First: implement the simplest possible visual odometry—read the frames of a video in sequence, match consecutive frames with ORB, reject false matches with RANSAC, estimate the relative motion and accumulate it, and draw out the trajectory. Observe how drift accumulates with time (precisely the problem Chapters 21 and 22 set out to solve). Second: implement a simple place recognition—extract ORB descriptors from all the frames of a video, build a simple bag-of-words representation, and test whether it can correctly find images of the same place taken at different moments. This is the embryo of the loop closure detection of Chapter 22.

Further Reading

1. E. Rublee et al., "ORB: An Efficient Alternative to SIFT or SURF," ICCV, 2011. (The original ORB paper, the principal source for Section 20.3 of this chapter.)
2. D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," IJCV, 2004. (The classic SIFT paper, including the proposal of the ratio test.)
3. E. Rosten and T. Drummond, "Machine Learning for High-Speed Corner Detection," ECCV, 2006. (The original paper on the FAST detector.)
4. M. Calonder et al., "BRIEF: Binary Robust Independent Elementary Features," ECCV, 2010. (The pioneering work on binary descriptors.)
5. D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," IEEE T-RO, 2012. (DBoW2, the basis of loop closure detection in ORB-SLAM.)
6. Gao Xiang and Zhang Tao, Fourteen Lectures on Visual SLAM, Lecture 7 (Visual odometry: the feature point method) and Lecture 8 (The direct method). (The clearest teaching material on feature-based and direct methods in the Chinese-speaking world, with complete source code.)
7. The official OpenCV tutorial on Feature Detection and Description (docs.opencv.org). (The principal tool for the implementation in this chapter, with complete examples.)

Summary

1. The standard pipeline of visual SLAM is preprocessing and undistortion → visual odometry (the front end) → back-end optimization → loop closure detection → mapping; its architecture agrees closely with the LiDAR SLAM of Chapter 18, the difference being that a visual feature has only two-dimensional pixel coordinates and its depth must be obtained separately.

2. The feature-based method first extracts key points and descriptors and then solves geometrically; it is robust to changes of illumination and to moving objects but fails where texture is weak. The direct method minimizes a photometric error over pixel brightness; it can use faint gradients and produce a dense map, but it rests on the fragile assumption of brightness constancy.
3. FAST only detects, BRIEF only describes, and SIFT / SURF / ORB do both; ORB combines oriented FAST with rotated BRIEF and achieves extremely high efficiency with a 256-bit binary descriptor and the Hamming distance—about 14 times faster than SURF, and SURF in turn about 24 times faster than SIFT—and has thereby become the mainstream choice for SLAM.
4. Matching compares descriptors and suits a large difference of viewpoint and loop closure detection; tracking searches near the original position, suits consecutive images, and is faster. False matches are inevitable, and the three strategies for rejecting them are the ratio test, the cross check, and geometric verification by RANSAC.
5. Monocular SLAM has an inherent scale ambiguity: scale the scene and the motion by k together and the image is exactly the same. The three remedies are to introduce an IMU, to use a stereo or RGB-D camera, or to use a reference object of known size.
6. A stereo camera measures depth from the disparity of a known baseline, has absolute scale, and works indoors and out, but it fears a lack of texture and its error grows with the square of the distance; an RGB-D camera obtains dense depth by active projection and still works on a white wall, but it fears strong light and glass and its range is limited.

7. Visual SLAM is, technically, what inside-out tracking for AR / VR really is, and it is also widely used in service robots, drones, and low-speed autonomous driving; the bag-of-words model (DBoW2's binary bag of words, for instance) is the key technique for place recognition and loop closure detection, while deep-learned features are steadily entering this field.

This chapter has established the overall framework of visual SLAM together with the fundamentals of features. But we have not yet answered the most central question: given feature correspondences between two images, how exactly does one compute how far the camera has moved? The next chapter, Epipolar Geometry and the Estimation of Camera Motion, supplies that piece of the puzzle—the essential matrix and the fundamental matrix describe the geometric constraint between two views, the PnP problem handles the correspondence of three-dimensional points to a two-dimensional image, triangulation recovers three-dimensional coordinates, and RANSAC makes all of it workable in a reality that contains false matches. It is the most mathematical and the most elegant chapter of visual SLAM.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 21

Epipolar Geometry and the Estimation of Camera Motion

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 20 taught the robot to find the same thing in two images. This chapter answers the next question: having found it, how does one compute how far the camera has moved?

This is the chapter of highest mathematical density in the book, but its reward is also the most direct—the three experimental figures of this chapter will prove one thing: epipolar geometry is not an approximation and not a rule of thumb but a set of exact relations whose error is zero in the absence of noise. Figure 21-2 will demonstrate that, even with 0.3 pixels of noise on every corresponding point, the rotation recovered by the eight-point algorithm is in error by only 0.16 degrees and the direction of translation by 0.70; and that among the four mathematical solutions the cheirality check picks out the one correct solution by 235 votes to 0.

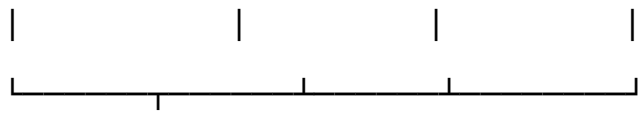
Equally important, though, is its fragility. Figure 21-3 will measure how noise is magnified into pose error, and why depth cannot be trusted at all when the baseline is short; and Figure 21-4 will show that a mere one match in ten being wrong brings this elegant mathematics down entirely—which is exactly the reason RANSAC exists, and the motive behind every robustness measure in ORB-SLAM2 in Chapter 22.

Figure 21-0 The Link Map of Chapter 21

The Foundations Already Laid

Chapter 9 The camera model Chapter 10 Coordinate transformation Chapter 12

Optimization Chapter 20 Feature matching



▼

【Chapter 21 Computing three-dimensional motion from two-dimensional correspondences】

The epipolar constraint ─ the essential matrix ─ PnP ─ triangulation ─ RANSAC

▼

Chapter 22 ORB-SLAM2 Chapter 23 ORB-SLAM3 (visual-inertial)

▼

Chapter 24 The vision-navigated robot Part VIII ROS implementation

Learning Objectives

In the last chapter we learned how to find corresponding feature points between two images, but we have not yet answered the most central question: given these correspondences, how exactly does one compute how far the camera has moved? This chapter supplies that piece of the puzzle. It is the most mathematical and the most elegant chapter of visual SLAM—once you understand epipolar geometry you will find that two seemingly ordinary photographs conceal a three-dimensional constraint of remarkable rigor. After studying this chapter, students should be able to:

1. Describe the overall line of thought in estimating camera motion from two images, and the conditions under which it applies.
2. Describe the central elements of epipolar geometry: the baseline, the epipole, the epipolar line, and the epipolar constraint.

3. Distinguish the essential matrix from the fundamental matrix and explain how camera motion is recovered from them.
4. Describe the definition of the PnP problem, the methods of solving it, and its role in SLAM.
5. Describe the principle of triangulation and the factors that affect its accuracy.
6. Describe how RANSAC operates and understand why it can stand up to a great many false matches.
7. Connect the knowledge of an image processing course to the geometric solutions of visual SLAM.

Core Concepts

Chinese Term	English Term	Explanation
對極幾何	Epipolar Geometry	The theory describing the intrinsic geometric relation between two views of the same scene.
基線	Baseline	The line joining the optical centers of two cameras.
對極點	Epipole	The intersection of the baseline with each image plane.
對極線	Epipolar Line	The line in one image corresponding to a point in the other.
對極約束	Epipolar Constraint	The geometric restriction that a corresponding point must fall on the epipolar line.
本質矩陣	Essential Matrix, E	The 3×3 matrix describing the relation between two views when the intrinsics are known.

Chinese Term	English Term	Explanation
基礎矩陣	Fundamental Matrix, F	The 3×3 matrix describing the relation between two views when the intrinsics are unknown.
單應矩陣	Homography, H	The matrix describing the relation between two views of a planar scene or under pure rotation.
PnP 問題	Perspective-n-Point	Solving for camera pose from known three-dimensional points and their image projections.
三角測量	Triangulation	Recovering three-dimensional coordinates from corresponding points in two views.
視差角	Parallax Angle	The angle a point subtends at the two optical centers, which affects the accuracy of triangulation.
隨機取樣一致	RANSAC	A robust estimation method that rejects outliers by random sampling and voting.
內點與外點	Inlier / Outlier	Data points that do and do not conform to the estimated model.

21.1 How Two Images Yield an Estimate of Camera Motion

Let us first form the overall picture of this chapter. Suppose a camera takes a photograph at position A, moves to position B, and takes another. Chapter 20 has already found for us the corresponding feature points in the two images. The question now is this: from these correspondences, can the rotation and translation of the camera from A to B be computed?

The Intuition: the Movement of Corresponding Points Gives the Motion Away

Intuitively the answer is yes. Imagine standing at the front of a classroom to take a photograph, then stepping two paces to the right and taking another: everything in the frame has moved to the left. If instead you step forward, things in the frame spread outward radially (near things moving a great deal, far things only a little). Different motions produce different patterns of displacement in the image—and that pattern is the clue we are to read.

Three Situations for Solving

In visual SLAM, three different methods of solution are used according to what information is in hand. This classification is the skeleton of the chapter and students should fix it firmly in mind:

Situation	What is known	Method of solution	Section of this chapter
2D–2D	Corresponding pixels in two images	Epipolar geometry (the essential / fundamental	20.2–20.3

Situation	What is known	Method of solution	Section of this chapter
		matrix)	
3D–2D	Known three-dimensional points + their image projections	The PnP problem	20.4
3D–3D	Two corresponding sets of three-dimensional points	ICP (Chapter 17)	Chapter 17

Table 21-1 Three Situations in Estimating Camera Motion

- 2D–2D: when the system has just started there are no three-dimensional map points at all, and initialization can rest only on the pixel correspondences of two images. This is the opening move of monocular SLAM.
- 3D–2D: once a map has been built, every subsequent frame can solve for pose using three-dimensional points already known in the map together with their two-dimensional projections seen in the current image. This is the method most used while SLAM is running, being both efficient and stable.
- 3D–3D: when both views carry depth information (stereo, RGB-D), two sets of three-dimensional points can be registered directly—which is precisely the ICP of Chapter 17, which students have already studied.

This chapter focuses on the first two. Understand this classification and students will be able to see, in the source code of ORB-SLAM (Chapter 22), when each kind of solver is called.

21.2 Epipolar Geometry

Epipolar geometry is the theoretical core of this chapter. It answers a question that seems simple but runs very deep: where in the second image can a given point of the first image appear?

The Search Space Falls from Two Dimensions to One

Intuitively the answer seems to be anywhere—after all, we do not know how far away the point is. But epipolar geometry tells us something remarkable: it can appear only on one particular straight line.

Why? Consider a pixel p_1 in the first image. The three-dimensional point P it corresponds to must lie on the ray that passes from the optical center of the first camera through p_1—only we do not know where on that ray (precisely the loss of depth of Chapter 9). Now project that entire ray into the second image and a straight line results. Wherever P actually lies on the ray, its projection in the second image must fall on this line.

That line is the epipolar line, and that constraint is the epipolar constraint. Its practical value is enormous: where the corresponding point had to be sought over the whole image (two dimensions), it need now be sought only along a line (one dimension). This both speeds up the search and greatly reduces the chance of a false match.

The Four Elements of Epipolar Geometry

- The baseline: the line joining the two optical centers O_1 and O_2. It is the backbone of the whole geometry.
- The epipolar plane: the plane determined by the two optical centers together with the point P in space. A different P gives a different epipolar plane, but all of

them contain the baseline—all epipolar planes form a pencil about the baseline.

- The epipole: the intersection of the baseline with each image plane, written e_1 and e_2. Geometrically it is the position at which the other camera's optical center projects into this image. Interestingly, every epipolar line in a given image passes through that image's epipole.
- The epipolar line: the intersection of the epipolar plane with the image plane. The epipolar line of p_1 in the second image is written l_2.

Figure 21-1 The Structure of Epipolar Geometry

A point P in space

├──── projects into image 1 → the pixel p_1

└──── projects into image 2 → the pixel p_2

O_1 ──────baseline──────→ O_2 (the two optical centers)

| |

└→ epipole e_1 └→ epipole e_2

The epipolar plane = the plane determined by O_1, O_2, and P

The epipolar line l_2 = the intersection of the epipolar plane with image 2

【The central constraint】p_2 must fall on the epipolar line l_2

→ the search space falls from two dimensions (the whole image) to one (a line)

A Verification from Everyday Life

Students can verify this phenomenon for themselves: close the left eye and look at a pen in front of you with the right, noting its position in the field of view; then close the right eye and look with the left, and the pen jumps to another position. Move the pen nearer and farther and repeat, and you will find that the track along which it jumps in the field of view always lies on one fixed straight line—that is the epipolar line. A pair of human eyes is a natural stereo system.

21.3 The Essential Matrix and the Fundamental Matrix

Described only in words, the epipolar constraint cannot be computed with. This section writes it in matrix form—and that is the essential matrix and the fundamental matrix.

The Matrix Form of the Epipolar Constraint

The mathematical form of the epipolar constraint is extremely concise and elegant. Let p_1 and p_2 be a pair of corresponding points in the two images (in homogeneous coordinates); then:

$$p_2^T \cdot F \cdot p_1 = 0 \qquad \textit{(the fundamental matrix form, in pixel coordinates)}$$

$$x_2^T \cdot E \cdot x_1 = 0 \qquad \textit{(the essential matrix form, in normalized coordinates)}$$

These two expressions are the most important formulas of the chapter. They say that the result of multiplying the corresponding points by the matrix must be zero—in other words, any correct pair of corresponding points must satisfy this constraint. Conversely, if we gather enough corresponding points, we can solve back for the matrix.

How the Two Differ

- The fundamental matrix F: acts on the raw pixel coordinates and does not require the camera intrinsics K (Chapter 9) to be known. It mixes together two kinds of information, the internal parameters of the cameras and the relative motion between them. It has 7 degrees of freedom.
- The essential matrix E: acts on normalized coordinates (pixel coordinates already transformed by the intrinsics K) and therefore contains only information about the relative motion, which makes it the purer of the two. It

has 5 degrees of freedom (three of rotation and three of translation, less one because the direction of the translation is known but not its magnitude).

The relation between them is

$$E = K_2^T \cdot F \cdot K_1$$

In practice the cameras of a SLAM system are calibrated (Section 9.2) and the intrinsics are known, so the essential matrix is generally used directly. The fundamental matrix is used where the intrinsics are unknown (handling photographs of uncertain origin from the internet, say).

Recovering Camera Motion from E

Once the essential matrix has been found, a singular value decomposition (SVD) can factor it into a rotation R and a translation t. But two important properties must be noted here:

- The translation is known in direction only, not in magnitude: in the definition of E, multiplying t by any nonzero constant leaves the constraint satisfied. This is exactly the mathematical embodiment of the monocular scale ambiguity of Section 20.5—epipolar geometry by itself cannot tell us how many meters the camera has moved, only in which direction.
- The decomposition has four solutions: mathematically, factoring E yields four possible combinations of (R, t). To pick the correct one from among them requires a physical criterion: after triangulating the corresponding points into three-dimensional coordinates, only the correct solution puts every point in front of both cameras (with positive depth). This is called the cheirality check.

A Special Case: the Homography

Epipolar geometry has one important exception. When every point in the scene lies on one and the same plane (the camera facing a wall, a tabletop, or the floor, for instance), or when the camera undergoes pure rotation with no translation, the epipolar constraint degenerates—solving for the essential matrix becomes unstable or even impossible.

In such a case one should use a homography H instead, which describes directly how the points of one image map into another:

$$p_2 = H \cdot p_1$$

One clever feature of ORB-SLAM's initialization is that it computes the fundamental matrix and the homography at the same time and judges automatically, from the fitting scores of the two, whether this scene is more planar or more three-dimensional, then chooses the appropriate model. This detail will be mentioned again in Chapter 22.

Computing the Whole Mathematical Chain Through Once

The three sections above have all been derivation. Derivations read elegantly, but a doubt inevitably forms: can this machinery really produce a number? This subsection takes a set of fully reproducible synthetic data and walks the whole chain through once: projection → the eight-point algorithm → decomposition → the cheirality check → triangulation.

The setup is this: the camera intrinsics are a focal length of 520 pixels, a principal point at the image center, and a resolution of 640×480; 240 three-dimensional points are scattered at random through the scene at depths between 4 and 12 meters; and the second camera is rotated about 9 degrees and translated 0.66 meters relative to the

first. These points are projected into the two images, Gaussian noise of 0.3 pixels is added, and then, pretending not to know the true motion, we recover it from these two-dimensional correspondences alone.

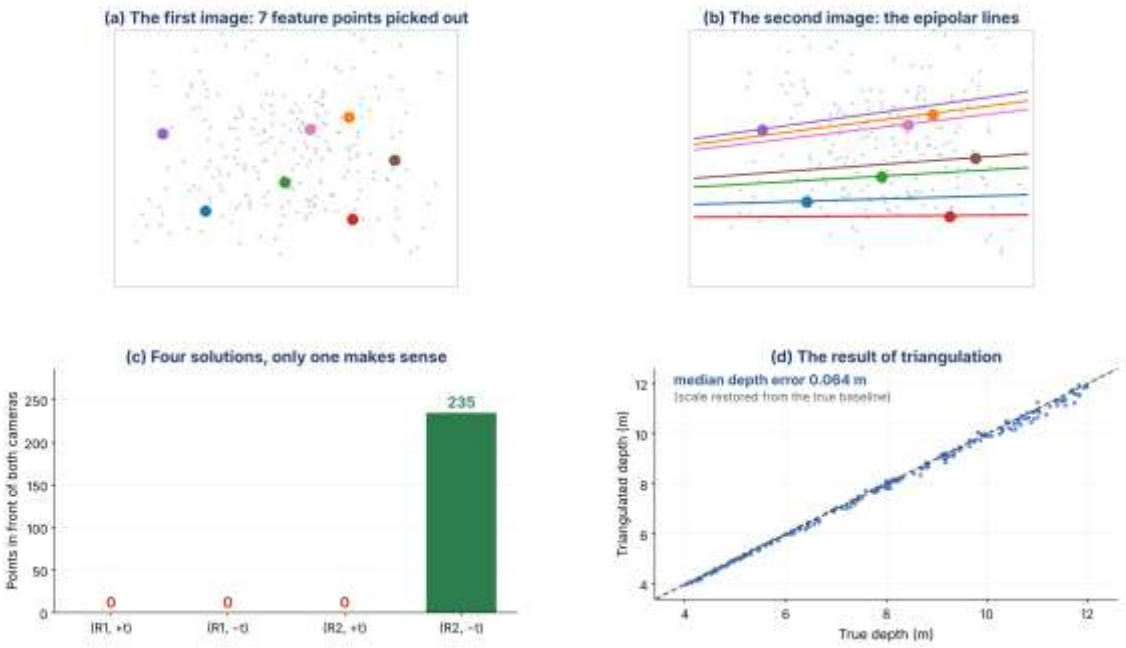


Figure 21-2 A Numerical Verification of the Epipolar Constraint: from Corresponding Points to Three-Dimensional Coordinates

Panel (a) is the first image, with 7 of the 240 points picked out and colored. Panel (b) is the second image, and each colored straight line is the epipolar line of the point of that color—computed from the fundamental matrix by $l_2 = F \cdot p_1$. Note that every colored dot falls exactly on the line of its own color: this is what the epipolar constraint looks like in an image. Where the corresponding point had to be sought over the whole 640×480 image, it need now be sought only along one line.

Panel (b) also reveals a detail: the seven epipolar lines are not parallel but converge slightly to one side. Extended, they meet at a single point—that is the epipole, the position at which the optical center of the first camera projects into the second image. In this example the camera mainly translates sideways with a small rotation, so the

epipole falls far outside the image and the lines look nearly parallel. Had the camera advanced straight ahead, the epipole would fall at the center of the image and all the epipolar lines would spread out from the center like rays of light—students can change the parameters and try it for themselves.

Panel (c) is the figure that best illustrates letting mathematics be mathematics and physics be physics. Take the singular value decomposition of the essential matrix and mathematics gives four combinations of (R, t), all of which satisfy the epipolar constraint, so that algebra alone cannot tell which is right. But take each of them, triangulate, and check whether those three-dimensional points lie in front of both cameras, and the result is this: the fourth combination has 235 points of positive depth and the other three have not one. 0 to 0 to 0 to 235—this is not a narrow win but an overwhelming one. The cheirality check works precisely because the three wrong solutions put the whole scene behind the camera, and that could not physically have been photographed at all.

Panel (d) is the last step: the selected (R, t) is used to triangulate the 235 points back into three dimensions and the result compared with the true depths. The points lie almost entirely on the diagonal, with a median depth error of 0.064 meters—about eight parts in a thousand in a scene running from 4 to 12 meters. It must be explained that the true depth on the horizontal axis can be matched at all only because we restored the scale with the known true baseline; in a genuine monocular system the whole figure would keep its shape but be scaled by an unknown factor, which is the scale ambiguity of Section 20.5.

With Zero Noise, the Error Really Is Zero

Set the noise of the experiment above to 0, and the rotation error recovered by the eight-point algorithm is 0.0000 degrees and the translation direction error 0.0000 degrees as well.

This is worth pausing over. It means that epipolar geometry is not an adequate approximation but a set of exact algebraic relations—the error comes entirely from measurement and not at all from the method. This is something quite different from the occupancy grid map of Chapter 13 or the particle filter of Chapter 14, which are approximations in their very nature.

So when students find in an implementation that the pose is estimated badly, they should not doubt the formulas but check three things: the accuracy of feature localization, the accuracy of the intrinsic calibration, and whether any false matches have crept in. The next two sections handle the first two and the third respectively.

21.4 The PnP Problem

Epipolar geometry handles the case of two images. But once SLAM is running we already have a three-dimensional map in hand—and then there is a better way to find pose.

The Definition of the Problem

The PnP (Perspective-n-Point) problem is defined thus: given the coordinates of n points in three-dimensional space together with the positions at which they project into the image in two dimensions, find the pose of the camera (the rotation R and the translation t).

Students should find this familiar—Section 4.6 used the same idea when it introduced AprilTag: the three-dimensional coordinates of the marker's four corners are known (the side of the square being known) and their pixel coordinates in the image can be detected, so the complete 6-DoF pose of the camera relative to the marker can be solved. That is a PnP problem with n = 4.

Why PnP Is More Useful Than Epipolar Geometry

- It has absolute scale: the coordinates of the three-dimensional points carry units (meters), so the translation solved for has a true magnitude as well—unlike epipolar geometry, which can give only a direction.
- No degeneracy to watch for: there is no problem of degeneracy in a planar scene.
- Few points required: a solution exists with as few as three points (P3P), far fewer than epipolar geometry requires.
- It is efficient: in SLAM each frame need only project and compare the points already known in the map, which makes it the mainstay method of the tracking stage.

Common Methods of Solution

- P3P: solves with three points and yields at most four solutions, requiring a fourth point to eliminate the rest. It is the basis of many methods.
- EPnP: an efficient linear solution able to handle any number of points, fast and of good accuracy, and the default choice of OpenCV and of many SLAM systems.

- The direct linear transform (DLT): converts the problem into a system of linear equations. Simple in concept but of poorer accuracy, it often serves to provide an initial value.
- Nonlinear optimization (BA): having obtained an initial value by one of the methods above, refine it by minimizing the reprojection error (Chapter 12). This is the final step, obligatory in practice.

The Reprojection Error: the Objective of PnP

An exact solution of PnP is in essence the minimization of the reprojection error—the central idea of bundle adjustment in Section 12.5:

1. Using the currently estimated pose, project each three-dimensional point into the image to obtain a predicted pixel position.
2. Compare it with the pixel position actually detected; the difference between the two is the reprojection error.
3. Adjust the pose so as to minimize the sum of the squared reprojection errors over all points.

Students can see that PnP is in fact a simplified bundle adjustment that optimizes only the camera pose and holds the three-dimensional points fixed, while a full BA optimizes poses and three-dimensional points together. The two are the same mathematics applied over different scopes—another appearance of the mathematical tools of Chapter 12.

21.5 Triangulation

The two preceding sections found camera motion from corresponding points. This section works the other way round: given the camera motion, find three-dimensional

coordinates from corresponding points. This is triangulation, and it is what allows monocular SLAM to build a map.

The Principle: the Intersection of Two Rays

The principle is extremely intuitive. Given the poses of the camera at two positions:

4. From the optical center of the first camera, draw a ray through the pixel p_1. The three-dimensional point P must lie on this ray.
5. From the optical center of the second camera, draw a ray through the pixel p_2. P must lie on this ray as well.
6. The intersection of the two rays is the three-dimensional position of P.

Perfect in theory, but there is a problem in practice: because the pixel coordinates carry measurement error, the two rays generally do not intersect exactly but pass each other by. The actual practice is therefore to find the point nearest to both rays, or to solve by minimizing the reprojection error.

The Key to Accuracy: the Parallax Angle

This is the most important idea of the section. The accuracy of triangulation depends on the angle between the two rays—called the parallax angle.

- A large parallax angle (the camera's displacement being large relative to the distance of the object): the two rays intersect at nearly a right angle, the position of the intersection is definite, and the depth estimate is accurate.
- A small parallax angle (the camera having hardly moved, or the object being very far away): the two rays are nearly parallel, and a minute error in the pixels lets the intersection slide a long way along the rays, so that the depth is extremely inaccurate.

This explains several important phenomena in practice:

7. Distant points cannot be measured accurately: for the same camera displacement the parallax angle of a distant object is far smaller, and so the depth of the sky or of distant hills can hardly be estimated at all. This is the same thing as the stereo depth error growing with the square of the distance in Section 9.6.
8. Pure rotation builds no map: if the camera only turns and does not move, the two optical centers coincide, the parallax angle is zero, and triangulation is impossible. This is a common cause of failed initialization in monocular SLAM—when students find in the implementation of Chapter 30 that initialization will not succeed, the reason is usually turning without moving.
9. A sufficient baseline is required: a SLAM system therefore waits until the camera has moved far enough (producing enough parallax) before triangulating new map points. This is one of the considerations behind the strategy for selecting key frames (Chapter 22).

The Tension, and How It Is Resolved

There is a subtle tension here: a large parallax angle requires the camera to move farther; but the farther it moves, the greater the difference in appearance between the two images, and the harder feature matching becomes (Chapter 20).

The resolution adopted in working systems is to track stably using neighboring frames with a short baseline, and to triangulate using key frames that are farther apart. This strategy of tracking on near frames and mapping on far frames is one reason ORB-SLAM distinguishes an ordinary frame from a key frame.

Measuring It: How Much Is Noise Worth, and How Much the Baseline

Section 21.5 says that depth is inaccurate when the parallax angle is small. That is true but not specific enough. How inaccurate? And how long must the baseline be? This subsection measures both questions with two sweep experiments.

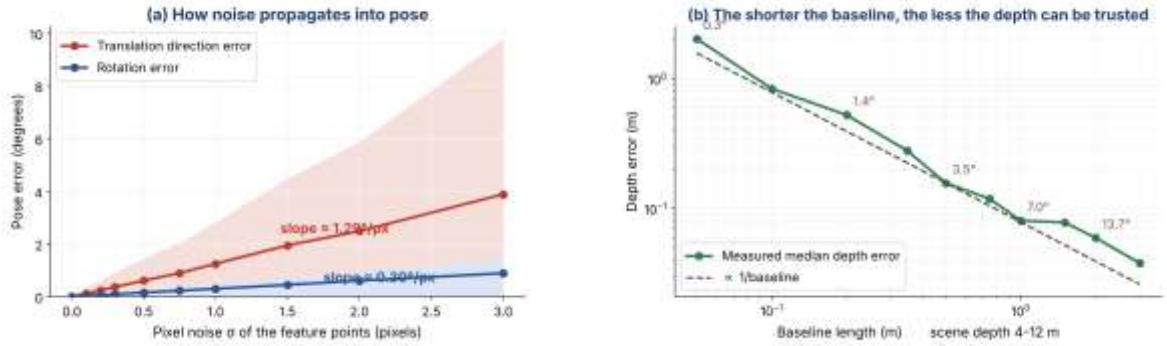


Figure 21-3 How Noise Propagates into Pose, and How the Baseline Decides Depth Accuracy

Panel (a) holds the scene and the motion fixed and varies only the pixel noise σ of the feature points, running 40 trials at each setting and taking the median, with the pale band showing the 90th percentile. Both lines are very nearly straight—pose error is proportional to pixel noise, with no threshold at which it suddenly collapses. That is good news: improve the accuracy of feature localization by a factor of two and the accuracy of the pose improves by a factor of two.

What is really worth remembering is the difference in the slopes of the two lines. The rotation error is about 0.30 degrees per pixel and the translation direction error about 1.29 degrees per pixel—the direction of translation is four times as sensitive to noise as the rotation. This asymmetry matters greatly in practice: the trajectory of visual odometry drifts off not mainly because the turns are estimated wrongly but because the direction of advance at every step is a little askew. It also explains why the visual-inertial SLAM of Chapter 23 brings in the IMU—an accelerometer supplies an

independent constraint on the direction of translation, exactly where vision is weakest.

Panel (b) holds the noise at 0.5 pixels, varies the length of the baseline, and measures the depth error of triangulation. Both axes are logarithmic, and the measured points lie almost on the dashed line of inverse proportionality to the baseline: at a baseline of 5 centimeters the depth error is 2.02 meters (of no value whatever for reference in a scene running from 4 to 12 meters), at 0.5 meters it falls to 0.16 meters, and at 3 meters to 0.038 meters. Marked beside each point is the corresponding parallax angle, running from 0.3 degrees all the way to 19 degrees.

Read backwards the figure is more useful still: if a student's project requires a depth error within 5 centimeters at a scene depth of about 10 meters, then the baseline must be at least 2.5 meters—which is to say that the camera must move a good two meters and more before it is fit to triangulate new map points. This is the concrete number behind the sentence a sufficient baseline is required in Section 21.5, and the arithmetic behind the key frame selection strategy of Chapter 22.

Why Practice Uses the Parallax Angle Rather Than the Baseline

The horizontal axis of Figure 21-3(b) is the length of the baseline, but what really decides the accuracy is the parallax angle—the ratio of the baseline to the depth of the scene. The same baseline of 0.5 meters gives a parallax of 5.7 degrees for an object 5 meters away and only 0.57 degrees for one 50 meters away.

So when a SLAM system judges whether it can triangulate yet, what it looks at is the parallax angle and not the distance moved. A common threshold is 1 to 2 degrees: below that, a new point is not created and the camera must travel

farther first.

This also explains why an open outdoor scene is especially unkind to monocular SLAM—distant buildings and the horizon never reach sufficient parallax, and all that can be used for mapping is the nearby road surface and the objects beside it.

21.6 Rejecting False Matches with RANSAC

The mathematics of the preceding sections is elegant, but it all shares one premise: that the correspondences fed in are correct. And Section 20.4 told us that false matches are bound to exist. This section introduces the key weapon against them.

Why a False Match Is So Deadly

Ordinary least squares (Chapter 12) minimizes the sum of the squared errors of all the data points. The trouble is that squaring magnifies large errors. One severe false match may produce an error of tens or even hundreds of pixels, which after squaring dominates the whole objective function and drags the estimate bodily off course.

By way of analogy: all ninety-nine students in a class are between 160 and 180 centimeters tall, but one entry in the data has been mistyped as 1800 centimeters. Estimate the height of the class by the mean and the result is severely distorted by that one absurd entry. Least squares facing a false match is in exactly this position.

The Central Idea of RANSAC

RANSAC (RANdom SAmple Consensus) adopts an entirely different strategy: rather than letting all the data take part, pick a few at random and see how many agree.

10. Random sampling: from all the correspondences, pick at random exactly the smallest number needed to solve the model (5 or 8 points for the essential matrix, 3 or 4 for PnP, say).
11. Solve the model: compute a candidate camera motion (or matrix) from those few points.
12. Count the votes: test all the correspondences against this candidate model and compute the error of each point. Those whose error is below a threshold are called inliers, and their number is recorded.
13. Iterate: return to step 1 and repeat tens to hundreds of times.
14. Choose the best: keep the model with the most inliers and solve once more, exactly and by least squares, using all of its inliers.

Why This Works

The key insight is this: if the few points picked at random happen to be all correct matches, then the model computed is correct, and the other correct matches will all agree with it (becoming inliers), so the inlier count is high. Conversely, if a false match is picked, the model computed is wrong, almost no other point agrees with it, and the inlier count is low.

The greatest number of inliers thus becomes an effective criterion for judging whether a model is correct. RANSAC does not try to correct false matches but simply excludes them—and this spirit of majority rule lets it tolerate a very high proportion of outliers (sometimes more than half).

Practical Points

- The number of iterations: how many are needed for a high probability of drawing at least one sample that is entirely of inliers? This can be estimated from a probability formula (see Formulas and Procedures). The lower the inlier ratio and the more sample points required, the more iterations are needed.
- Setting the threshold: the error threshold decides how close counts as an inlier. Too strict and correct matches are excluded too; too loose and false matches get in. It is usually set according to the level of pixel noise, at 1 to 3 pixels, say.
- Its relation to the robust kernel: RANSAC is the hard approach of rejecting first and optimizing after; the robust kernel mentioned in Chapters 12 and 15 is the soft approach of reducing the weight of outliers. The two are often used together—RANSAC first to reject the obvious false matches, then a robust kernel in the optimization to handle the outliers that remain.
- Its generality: RANSAC is not confined to vision. The point cloud registration of Chapter 17, the plane fitting of Chapter 13, and indeed any problem of model fitting in the presence of outliers can use it. It is among the most practical methods in an engineer's toolbox.

Measuring It: How Much Does RANSAC Actually Rescue

Section 21.6 calls a false match deadly. This section measures that word.

The method of the experiment is this: among correct matches at 0.5 pixels of noise, deliberately replace the second-image coordinates of a proportion of the correspondences with random positions (simulating false matches), then compare two approaches—throwing all the matches straight into the eight-point algorithm, and screening for inliers with RANSAC first and then estimating. Each proportion is run 25 times and the median taken.

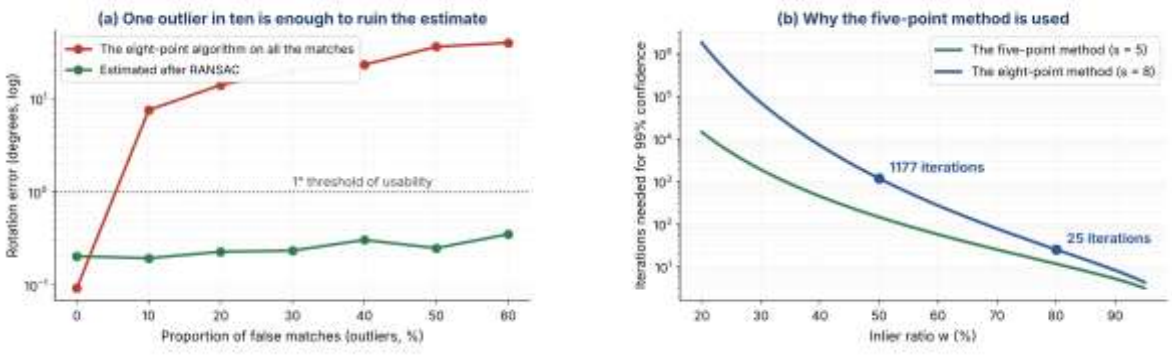


Figure 21-4 The Cost of False Matches, and Why the Number of Sample Points Is Decisive

The red line of panel (a) shows how serious the problem is. With no outliers, the rotation error of the direct eight-point algorithm is 0.09 degrees, which is quite accurate. But let in a mere 10 percent of false matches and the error explodes to 7.5 degrees—eightyfold. At 20 percent it is 14.0 degrees, and at 50 percent 36.1 degrees, which is to say wholly meaningless. Note that the vertical axis is logarithmic: this line does not degrade slowly but collapses outright from one outlier in ten.

The reason was given in Section 21.6: least squares sums the squares of the errors, and one false match may cause an error of hundreds of pixels, which after squaring dominates the whole objective function. In other words, this is not a slight loss of accuracy but the objective of the optimization itself being held hostage by a few points.

The green line is the result with RANSAC added. At 10 percent outliers the error is 0.19 degrees, scarcely worse than the direct estimate with no outliers at all; all the way to 40 percent outliers it stays within 0.31 degrees, far below the 1-degree threshold of usability; and even at 50 and 60 percent it is still 0.25 and 0.35 degrees. The theory nevertheless sets a limit on how far this can be pushed: when the inlier ratio falls to a half, the probability of drawing eight points that are all inliers is only $0.5^8 \approx 0.4$ percent, so the number of iterations must be raised accordingly—which is exactly what panel (b)

computes.

An honest addition: in this experiment the consensus set RANSAC found was a faithful one—at 50 and 60 percent outliers the proportion it judged to be inliers was 49 and 39 percent respectively, against true inlier ratios of 50 and 40 percent. That is the comfortable case, with the outliers scattered at random. Where false matches are systematic (a repeated texture, for instance, of the kind the tile region of Figure 20-3 produces) they may conform to one and the same wrong geometry, and RANSAC can then settle confidently on the wrong model. The remedy in practice is to increase the number of iterations, or to change to a variant more robust to outliers (MAGSAC or LO-RANSAC, say).

Panel (b) answers a question Section 21.6 raised but did not compute: how many iterations are enough? For an inlier ratio w and a minimum sample of s points, the number required for 99 percent confidence is $N = \log(1-0.99) \,/\, \log(1-w^s)$. At an inlier ratio of 50 percent the eight-point algorithm needs 1,177 iterations and the five-point algorithm only 145; at an inlier ratio of 80 percent the figures are 25 and 12. This is why the five-point algorithm for the essential matrix has in practice almost entirely displaced the eight-point algorithm—not because it is more accurate, but because it makes RANSAC an order of magnitude cheaper.

Why the Number of Sample Points Matters So Much

The key is the term w^s: s appears in the exponent. At an inlier ratio of 50 percent, the probability of drawing eight points that are all inliers is $0.5^8 = 0.39$ percent, while for five points it is $0.5^5 = 3.1$ percent—eight times greater.

This observation will appear again in Section 21.8: a new solution of recent

years that uses monocular depth estimation needs only three points, and at the same 50 percent inlier ratio it requires only 34 iterations—thirty-four times fewer than the eight-point method.

So when students design any algorithm of the RANSAC kind, the first question to ask is always: can I pin the model down with fewer points?

21.7 Connecting an Image Processing Course to Visual SLAM

The mathematical density of this chapter is fairly high, so this section steps back to connect the whole line of knowledge for students and to explain how one walks, step by step, from an image processing course already taken to visual SLAM.

How the Knowledge Follows On

Stage	The central question	Key techniques	Chapter of this book
Image processing	How to make an image clearer	Filtering, edge detection, morphology	Prerequisite knowledge
The camera model	How three dimensions become two	The pinhole model, intrinsics, distortion correction	Chapter 9
Feature engineering	How to find points that can be recognized repeatably	ORB, descriptors, matching	Chapter 20
Multiple view geometry	What geometric relation two images conceal	The epipolar constraint, E / F, PnP, triangulation	This chapter
Visual	How to estimate the	Solving frame by frame and	Chapters 20 and

Stage	The central question	Key techniques	Chapter of this book
odometry	camera trajectory continuously	accumulating	22
Visual SLAM	How to remove drift and build a consistent map	BA, loop closure, key frames	Chapters 22 and 23

Table 21-2 The Ladder of Knowledge from Image Processing to Visual SLAM

Three Key Turns of Thought

15. From processing an image to understanding geometry: image processing treats an image as a two-dimensional signal to be processed (filtering, enhancement); visual SLAM regards an image as a projection of the three-dimensional world and cares about the geometric structure behind it. This is the most fundamental change of perspective.
16. From a single image to the relation among several views: the epipolar geometry of this chapter is the core of this turn—a single image has lost depth, but the relation between two images suffices to recover motion and structure.
17. From exact assumptions to robust estimation: the algorithms of image processing mostly assume the input is clean; real SLAM must face false matches, noise, and moving objects, so RANSAC and robust optimization become indispensable tools. This also echoes the spirit of handling uncertainty by probability in Chapter 11.

Advice on Study

- Build geometric intuition first, then look at the formulas: the formulas of epipolar geometry are concise, but intuition matters more. Be sure to do the hands-on work of this chapter and draw the epipolar lines out to look at.
- Make good use of visualization: OpenCV can draw epipolar lines, inliers and outliers, and reprojection results. Once you see the picture, the abstract matrices acquire meaning.
- Compare with LiDAR SLAM: go back and compare the ICP of Chapter 17 (3D–3D) with this chapter's PnP (3D–2D) and epipolar geometry (2D–2D), and you will find them variants of one problem under different conditions of information.
- There is no need to derive every formula by hand: the details of the SVD factorization of the essential matrix, the derivation of EPnP, and the rest are already implemented in OpenCV. Understanding what it is solving, when it is used, and where it will fail matters more than memorizing a derivation. Those bent on research can go deeper into the classic textbook of Hartley and Zisserman.

At this point the mathematical foundation of visual SLAM is in place. In the next chapter we assemble these parts into a complete system—ORB-SLAM2. How do its three threads work together? How are key frames and map points managed? What is the covisibility graph? How are loop closures detected and corrected? A visual SLAM system that really runs is about to be laid out whole before the student's eyes.

21.8 After Epipolar Geometry: Three Turns in Two-View Estimation, 2024–2026

The mathematics of this chapter was already mature in the 1990s, and it remains to this

day the foundation of every visual SLAM system. But the way it is used has been changing in the past few years. This section sets out three turns worth knowing about.

One: Fewer Points, by Letting Monocular Depth Take Part in the Solution

The first turn makes use of a new fact: monocular depth estimation networks (mentioned in Chapter 9) are now quite usable, even though the depth they give carries an unknown scale and offset. A study of 2025 writes this imperfect depth straight into the solver: while estimating the relative pose it solves for the scale and the offset of the depth as unknowns at the same time.

The result is that the minimum number of correspondences required falls from five to three. By the iteration formula of Section 21.6, at an inlier ratio of 50 percent the three-point method needs only 34 iterations against 145 for the five-point method and 1,177 for the eight-point. What the paper reports in measurement is this: on a standard dataset the three-point solution has a median rotation error of 0.89 degrees and takes 22.4 milliseconds, while the five-point method paired with the best current dense matcher, though more accurate (0.56 degrees), needs 184.4 milliseconds. Accuracy, that is, is traded for an eightfold gain in speed—a decisive bargain on an embedded platform.

Two: Denser Matching, from a Few Hundred Features to the Whole Image

The second turn comes from the matching side. What Chapter 20 introduced was detecting feature points and then comparing descriptors, which usually yields a few hundred to a thousand or two correspondences per image. The dense matching methods of recent years instead find a correspondence for nearly every pixel in the image, producing tens of thousands to hundreds of thousands at a stroke, and are far

more robust than feature points under weak texture and large changes of viewpoint.

This creates a new problem for the mathematics of this chapter: RANSAC was designed for a few hundred correspondences, and faced with hundreds of thousands, merely testing every point once per iteration becomes unacceptably slow. Hence a line of research appeared in 2025 devoted to precisely this—first summarizing the dense matches into a small set of representative correspondences, then feeding those to the traditional two-view estimator. This is a good example: the new front end has not made the old geometry obsolete but has forced it to redesign its interface.

Three: Direct Regression, Skipping the Essential Matrix

The third turn is the most radical: do not solve for the essential matrix at all, but train a network that takes two images as input and outputs the relative pose between them directly.

A representative work of 2025 trained on about 8 million image pairs and reached a relative pose accuracy (AUC@20) of 75.56 on an indoor dataset, plainly above the 65.57 of comparable methods; on an indoor localization task its mean median errors were 0.04 meters and 1.02 degrees, against 0.36 meters and 18.38 degrees for the best comparable method before it. As for speed, it takes only 42 milliseconds per image pair (about 24 pairs a second), while a dense matching method needs 300 milliseconds and a reconstruction-based method 441.

Does this mean this chapter need no longer be studied? Quite the contrary, for three reasons. First, the training labels for networks of this kind are produced by the methods of this chapter. Second, their evaluation metrics (rotation error, translation direction error, AUC) rest entirely on this chapter's representation of pose. Third, and most

practical of all: when one of them estimates wrongly, you need this chapter's knowledge to judge where the fault lies—whether the parallax was insufficient, the scene degenerated into a plane, or there were simply too many outliers.

	Traditional epipolar geometry	Dense matching + geometry	Direct regression of pose
Input	A few hundred feature correspondences	Tens of thousands to hundreds of thousands of correspondences	Two images
Minimum number of points	5 to 8 (3 with depth)	The same	Not applicable
Interpretability	High; every step can be traced	Medium	Low
Handling degeneracy	Must be detected oneself (homography)	The same	Implicit in the training data
Typical time	A few milliseconds	A few hundred milliseconds	Tens of milliseconds
Present role	The foundation of every system	High-accuracy offline reconstruction	Fast initialization and relocalization

Table 21-3 A Comparison of Three Routes to Two-View Pose Estimation

The practical conclusion is that the three routes are dividing the labor rather than replacing one another. Traditional epipolar geometry, being fast, interpretable, and free of any need for training data, remains the default choice for a real-time SLAM front

end; dense matching is used for offline high-accuracy reconstruction; and direct regression suits occasions such as initialization and relocalization, where speed matters and a fair tolerance of error is acceptable. The ORB-SLAM family to be dissected in Chapters 22 and 23 takes the first route—and it is the one students should understand thoroughly first.

Formulas and Procedures

One: the Epipolar Constraint

Let p_1 and p_2 be a pair of corresponding points in two images (in homogeneous pixel coordinates) and K_1 and K_2 the camera intrinsics (Chapter 9); then the epipolar constraint is

$$p_2^{T} \cdot F \cdot p_1 = 0$$

Converting the pixel coordinates to normalized coordinates with the intrinsics, $x = K^{-1} \cdot p$, rewrites the constraint in essential matrix form:

$$x_2^{T} \cdot E \cdot x_1 = 0$$

The relation between the two matrices, and the composition of the essential matrix, are

$$E = K_2^{T} \cdot F \cdot K_1 \quad , E = [t]_x \cdot R$$

where $[t]_x$ is the skew-symmetric matrix of the translation vector (Chapter 10) and R the rotation matrix. This expression shows clearly that the essential matrix is a compact encoding of translation and rotation. Factoring R and t out of E gives the camera motion, but t has only a direction and no magnitude (the scale ambiguity).

Two: Computing the Epipolar Line

For a point p_1 in the first image, the epipolar line in the second image is

$$l_2 = F \cdot p_1 \qquad \textit{(the coefficients of the line equation } a \cdot u + b \cdot v + c = 0\textit{)}$$

Conversely, the epipolar line of p_2 in the first image is $l_1 = F^T \cdot p_2$. Drawing l_2 on the second image in an implementation lets one see with one's own eyes the constraint that the corresponding point must fall on this line—the most illuminating step of this chapter's hands-on work.

Three: the Reprojection Error (the Objective of PnP and BA)

For a three-dimensional point P, a camera pose (R, t), intrinsics K, and an observed pixel u_obs, the reprojection error is

$$e = u_obs - \pi(K, R, t, P)$$

Solving PnP means minimizing the sum of the squared reprojection errors over all points:

$$(R^*, t^*) = \operatorname{argmin} \Sigma_i \, \| u_obs_i - \pi(K, R, t, P_i) \|^2$$

Note: PnP optimizes only (R, t) and holds the three-dimensional points fixed, while a full bundle adjustment (Section 12.5) optimizes poses and three-dimensional points together. The two are different applications of one mathematical framework.

Four: the Number of RANSAC Iterations

Let the inlier ratio be w, let each sample require s points, and let the desired probability of drawing at least one all-inlier sample be p; then the number of iterations required is

$$N = \log(1 - p) \,/\, \log(1 - w^s)$$

For example: with an inlier ratio w = 0.5, s = 8 points for the essential matrix, and a desired success probability p = 0.99, N ≈ 1177. Change to a method needing only s = 5 points and N falls to about 145—which shows how precious a solver requiring few points is within RANSAC, and why the five-point algorithm is preferred to the eight-point.

Five: the Complete Solution Pipeline of the Visual Front End

Figure 21-5 The Complete Flow from Feature Correspondences to Camera Pose

① Feature matching (Chapter 20) → a set of correspondences that may contain false matches

↓

② Judge the situation:

• The system is initializing, with no map points → take 2D–2D (epipolar geometry)

• Map points already exist → take 3D–2D (PnP)

↓

③ RANSAC iteration: sample at random → solve the model → count inliers

↓

④ Take the model with the most inliers and reject the outliers

↓

⑤ Refine the pose by a nonlinear optimization over all the inliers (minimizing the reprojection error)

↓

⑥ Triangulation: for new correspondences of sufficient parallax angle, recover three-dimensional coordinates and add them to the map

↓

⑦ Output the camera pose + the new map points → hand them to the back end (Chapter 22)

Cases

Case 21-1 Why Monocular SLAM Dies When You Stand in One Place and Turn Around

This is the most common frustration students meet in implementing monocular visual SLAM. Phone or camera in hand, the system keeps displaying initializing and will not succeed however it is tried. The camera calibration, the number of features, and the lighting have all been checked and are all normal—so where is the trouble?

The answer is usually extremely simple: you have only been turning in place, without walking.

By the principle of Section 21.5: under pure rotation the positions of the optical center at the two exposures coincide exactly, the length of the baseline is zero, and the parallax angle is zero. The rays from the two optical centers through the corresponding pixels then overlap completely and there is no intersection to speak of—triangulation has no mathematical solution, and the system naturally cannot build an initial map.

More subtly still, epipolar geometry also degenerates under pure rotation: the translation vector t is zero, the essential matrix $E = [t]_x \cdot R$ becomes the zero matrix, and the solution naturally fails. What should be used then is the homography (Section 21.3)—which is exactly why ORB-SLAM computes both models at initialization and chooses between them automatically.

The remedy is simple: translate. Hold the camera and move it a few tens of centimeters from side to side or back and forth while keeping the features in view, and initialization usually succeeds within a few seconds. This case echoes IMU initialization requires excitation of motion in Section 7.7—the shared lesson being that both visual

and inertial systems need motion with content before certain quantities become observable.

Case 21-2 Visualizing the Epipolar Line: Seeing a Three-Dimensional Constraint at a Glance

The formulas of epipolar geometry look abstract, but the moment they are drawn out everything becomes clear. This is one of the most popular demonstrations in a university classroom.

The procedure: take two photographs of the same scene from slightly different viewpoints, find corresponding points with ORB, estimate the fundamental matrix F with RANSAC, and then, for every feature point in the first image, compute its epipolar line in the second image by Formula Two and draw it.

The result is a spectacular picture: many straight lines appear on the second image, and every corresponding feature point falls exactly on its own line. More remarkable still, if the camera moved forward, all these lines converge on one point in the frame—that is the epipole, the position at which the other camera's optical center projects, geometrically corresponding to the direction in which you advanced.

The teaching value of this experiment is very high: it lets students see with their own eyes that two ordinary photographs conceal a three-dimensional constraint of such rigor. The abstract matrix F suddenly becomes a visible line. Teachers are strongly encouraged to put this demonstration into the classroom and to have students photograph and implement it themselves—the hands-on activity of this chapter takes it as its core.

Case 21-3 The Universal Value of RANSAC: a Methodology Beyond Vision

RANSAC was proposed by Fischler and Bolles in 1981, and it was proposed for the pose estimation problem of computer vision in the first place (interestingly, what they treated in proposing it was precisely a PnP-type problem). But in the forty years since, its influence has reached far beyond vision.

It has been used for fitting planes and cylinders in point clouds (Chapter 17), for separating the ground in LiDAR data (LeGO-LOAM in Chapter 18), for calibrating the sensors of a robot, for sequence alignment in bioinformatics, for trend fitting in financial data—anywhere that outliers are mixed into the data and the mainstream model must be found.

RANSAC is so general because it embodies a profound methodology: when you cannot trust every datum, do not try to average over all the data but look for the explanation that the most data agree with. This is a change of thought from believing the data to believing the consensus.

The lesson for students is this: in studying an algorithm, besides remembering how it is done, understand the pattern of thought behind it. RANSAC's random sampling plus majority rule, the Kalman filter's weighting by reliability (Chapter 11), graph optimization's spreading of contradiction evenly (Chapter 12)—these frameworks of thought will serve you far longer than any particular formula.

Hands-On Activities

Activity 21-1 Epipolar Geometry and the Estimation of Camera Motion

Purpose: to compute the fundamental matrix with your own hands, draw the epipolar lines, carry out PnP and triangulation, and witness the power of RANSAC in rejecting false matches. This activity uses Python with OpenCV and needs no more than a phone camera, and it is among the experiments in this book that build geometric intuition best.

Part One: Visualizing the Epipolar Lines (everyone; the core of this chapter)

1. Take the material: photograph a classroom or a corridor twice, the second shot translated about 30 centimeters to the right of the first (note that this is a translation, not a turn in place). Make sure the frame has rich texture.
2. Extract and match features with ORB (the fruits of Chapter 20), applying the ratio test for a first rejection of false matches.
3. Estimate the fundamental matrix F with OpenCV's findFundamentalMat together with RANSAC, obtaining the inlier and outlier labels at the same time.
4. By Formula Two, compute the epipolar lines in the second image for the inliers of the first and draw them; draw the corresponding feature points as well.
5. Observe and verify: does every corresponding point fall on its own epipolar line? How large is the error (in pixels)? Do all the epipolar lines converge on one point (the epipole)?
6. A control experiment: take another set of photographs turning in place, repeat the procedure above, and observe whether the estimate becomes unstable—verifying the degeneracy of Case 21-1 at first hand.

Part Two: the Power of RANSAC (everyone)

7. Estimate F without RANSAC (by least squares instead), draw the epipolar lines, and observe whether they are plainly shifted.

8. Compare the inlier count: what percentage of all the matches does RANSAC judge to be inliers? What do the rejected outliers look like as matches in the image (draw them separately and look)?
9. Introduce false matches deliberately: add a number of random wrong correspondences to the match list, rerun RANSAC, and observe whether it still rejects them correctly. At what proportion does it fail?
10. Compute by Formula Four: at your inlier ratio, how many iterations are needed in theory for a 99 percent probability of success? Compare with OpenCV's default.

Part Three: PnP and Triangulation (university level)

11. A PnP implementation: with the camera calibrated in Chapter 9, photograph a checkerboard (or an ArUco marker); the three-dimensional coordinates of the grid points being known, solve for the camera pose with solvePnP and verify by projecting the coordinate axes back into the image.
12. A triangulation implementation: using the camera motion obtained in Part One, triangulate the matched inliers (OpenCV's triangulatePoints) and display the recovered three-dimensional points with matplotlib or Open3D.
13. A parallax angle experiment: triangulate with two sets of photographs, one translated 10 centimeters and the other 50, and compare the quality of the recovered point clouds (their scatter, the plausibility of the depths), verifying the effect of parallax angle on accuracy in Section 21.5.
14. A scale check: measure the true distance between two points in the scene and compare it with the triangulation result—you will find the proportions right but

the absolute values wrong, experiencing monocular scale ambiguity at first hand (Section 20.5).

Discussion Questions

1. In your visualization of the epipolar lines, by how many pixels do the corresponding points depart from their lines? Where does this error come from (error in feature localization? error in camera calibration?)
2. For the set of photographs taken while turning in place, why is the estimate of the fundamental matrix unstable? By Section 21.3, what model should be used instead?
3. What characteristics do the outliers RANSAC rejects display in the image (are they randomly distributed, or concentrated in certain regions such as repetitive texture)?
4. By how much do the triangulation results at 10 centimeters and at 50 centimeters of translation differ? What does this suggest about when a SLAM system ought to triangulate new map points?
5. (University level) The three-dimensional points recovered by triangulation have the right proportions but the wrong absolute values. To correct this, which three choices do you have by Section 20.5? What does each cost?

Extended Challenge (university level)

First: string the parts of this chapter into a complete two-frame visual odometry—feature matching → estimating the essential matrix with RANSAC → factoring out R and t → selecting the correct solution by the cheirality check → triangulating the initial map points. This is a simplified version of ORB-SLAM's initialization module, and on completing it you will have a very deep understanding of the system architecture of

Chapter 22. Second: implement ORB-SLAM's model selection strategy—compute the fundamental matrix and the homography at the same time, judge automatically from the fitting scores whether the scene is planar or three-dimensional, and initialize with the appropriate model. Test it on a white wall and on a three-dimensional scene in turn and verify that the choice is correct.

Further Reading

1. R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004. (The authoritative bible of multiple view geometry and the source of the complete derivations of every topic in this chapter.)
2. M. A. Fischler and R. C. Bolles, "Random Sample Consensus," Communications of the ACM, 1981. (The original RANSAC paper, applied to the pose estimation problem at its proposal.)
3. D. Nistér, "An Efficient Solution to the Five-Point Relative Pose Problem," IEEE TPAMI, 2004. (The five-point method for the essential matrix, which greatly reduces the iterations RANSAC requires.)
4. V. Lepetit, F. Moreno-Noguer, and P. Fua, "EPnP: An Accurate O(n) Solution to the PnP Problem," IJCV, 2009. (The original EPnP paper, OpenCV's default PnP solver.)
5. Gao Xiang and Zhang Tao, Fourteen Lectures on Visual SLAM, Lecture 7 (Visual odometry: the feature point method). (The best Chinese-language teaching material on epipolar geometry, PnP, and triangulation, with complete source code.)

6. The Camera Calibration and 3D Reconstruction module of the official OpenCV documentation (docs.opencv.org). (Instructions for findFundamentalMat, solvePnP, triangulatePoints, and the rest.)
7. The Multiple View Geometry unit of UZH's Vision Algorithms for Mobile Robotics course (introduced in Chapter 2). (Freely available course videos and notes, to be studied alongside this chapter.)

Summary

1. Camera motion estimation falls into three situations according to what is known: 2D–2D uses epipolar geometry (system initialization), 3D–2D uses PnP (the mainstay while SLAM is running), and 3D–3D uses ICP (Chapter 17). Understand this classification and you can see when a SLAM system calls which kind of solver.
2. The core of epipolar geometry is the epipolar constraint: a point in the first image must fall on one epipolar line in the second, which reduces the search space from two dimensions to one. Its four elements are the baseline, the epipolar plane, the epipole, and the epipolar line, and every epipolar line in a given image passes through its epipole.
3. The matrix forms of the epipolar constraint are $p_2^{\mathrm{T}} \cdot F \cdot p_1 = 0$ and $x_2^{\mathrm{T}} \cdot E \cdot x_1 = 0$; the fundamental matrix uses pixel coordinates and needs no intrinsics, while the essential matrix uses normalized coordinates and is the purer of the two. Factoring $E = [t]_x \cdot R$ gives the motion, but the translation is known only in direction (the scale ambiguity), and the four solutions must be screened by the cheirality check. In a planar scene or under pure rotation a homography should be used instead.

4. PnP solves for camera pose from known three-dimensional points and their projections in the image; its merits are absolute scale, no degeneracy, few points required, and high efficiency. Common solutions include P3P, EPnP, and DLT, refined by a nonlinear optimization that minimizes the reprojection error—it is in essence a simplified bundle adjustment with the three-dimensional points held fixed.
5. Triangulation recovers three-dimensional coordinates from the intersection of two rays, and its accuracy depends on the parallax angle: when the angle is small (the camera having hardly moved, or the object being far away) the depth is extremely inaccurate; under pure rotation the angle is zero and triangulation is impossible—the most common cause of failed initialization in monocular SLAM.
6. RANSAC stands against false matches by sampling at random, solving a model, counting inliers, iterating, and taking the model with the most inliers; it does not correct outliers but excludes them, and can tolerate a very high proportion of them. The number of iterations required is set by the inlier ratio and the number of sample points, which is why the five-point method is preferred to the eight-point.
7. Three turns of thought lead from image processing to visual SLAM: from processing an image to understanding geometry, from a single image to the relation among several views, and from exact assumptions to robust estimation. Understanding what an algorithm is solving, when it is used, and where it will fail matters more than memorizing a derivation.

Carrying Forward

But all of these are still parts—features, epipolar geometry, PnP, triangulation, RANSAC. How are they assembled into a complete system that runs stably over long periods, builds a consistent map, and recovers when tracking is lost? In the next chapter we dissect ORB-SLAM2: how it works with three threads in concert, how key frames and map points are managed, how the covisibility graph organizes spatial relations, and how loop closures are detected and corrected. It is worth taking the three figures of this chapter along to every one of its design decisions: why a key frame waits for sufficient parallax before creating new points (Figure 21-3(b)), why RANSAC must be run at every step (Figure 21-4(a)), and why initialization computes both the fundamental matrix and the homography before choosing one. A visual SLAM system that really runs is about to be laid out whole.

This chapter has assembled the geometric tools of visual SLAM. But all of these are still parts—features, epipolar geometry, PnP, triangulation, RANSAC. How are they assembled into a complete system that runs stably over long periods, builds a consistent map, and recovers when tracking is lost? In the next chapter we dissect ORB-SLAM2: how it works with three threads in concert, how key frames and map points are managed, how the covisibility graph organizes spatial relations, and how loop closures are detected and corrected. A visual SLAM system that really runs is about to be laid out whole.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 22

ORB-SLAM2 in Theory and in Practice

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 20 gave us features and Chapter 21 gave us geometry. This chapter assembles them into a system that really runs.

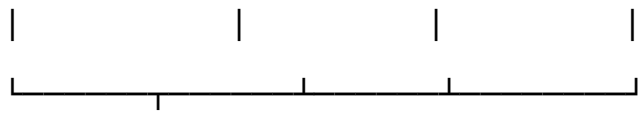
ORB-SLAM2 is worth a whole chapter's reading, not because any one of its algorithms is especially clever, but because it demonstrates how much engineering judgment it takes to put the right parts together: when a key frame should be inserted, how far a new map point should be trusted, what to do when the map grows too large, what happens if loop closure detection is wrong. The formulas of a paper hold no answer to these questions; only the design of a system can answer them.

The three figures of this chapter exist to quantify that engineering judgment. Figure 22-1 counts how many new map points are left after passing five gates and how many false matches were blocked; Figure 22-2 demonstrates something counterintuitive—the loop closed, and yet the map was not repaired; and Figure 22-3 draws the covisibility graph out to explain why it is not the same as the frames adjacent in time.

Figure 22-0 The Link Map of Chapter 22

The Parts Already Gathered

Chapter 20 ORB features Chapter 21 Epipolar geometry and PnP Chapter 12 BA Section 20.7 Bag of words



▼

【Chapter 22 ORB-SLAM2: assembling the parts into a system】

Three threads ─ key frames and map points ─ the covisibility graph ─ loop closure and global

BA

▼

Chapter 23 ORB-SLAM3 (visual-inertial + multiple maps)

▼

Chapter 24 The vision-navigated robot Part VIII ROS implementation

Learning Objectives

In Chapters 20 and 21 we gathered every part of visual SLAM: features, epipolar geometry, PnP, triangulation, RANSAC. But parts are not a system—how are they made to work in concert, to stay stable over long periods, to recover when tracking is lost, to correct themselves at a loop closure? This chapter dissects ORB-SLAM2, acknowledged as the most mature open-source visual SLAM system, and assembles the theory into an architecture that really runs. After studying this chapter, students should be able to:

1. Explain the role of the ORB feature within ORB-SLAM2 and the design merits of one set of features running through all three great tasks.
2. Explain how initialization differs among the monocular, stereo, and RGB-D modes.
3. Describe the workflow of the tracking thread and its three tracking strategies.
4. Explain how the local mapping thread adds map points, performs local BA, and culls redundancy.
5. Explain how the loop closing thread detects a loop, verifies the similarity transform, and corrects the global drift.
6. Describe the data structures and the uses of key frames, map points, the covisibility graph, and the spanning tree.

7. Plan and carry out an implementation and evaluation of ORB-SLAM2 on a public dataset.

Core Concepts

Chinese Term	English Term	Explanation
關鍵幀	KeyFrame	An image frame retained to represent a viewpoint, holding its pose and features.
地圖點	MapPoint	A three-dimensional feature point in the map, recording coordinates, descriptor, and observations.
共視圖	Covisibility Graph	The graph formed by linking key frames that observe map points in common.
生成樹	Spanning Tree	The minimum spanning tree linking all key frames; a lightweight structure.
本質圖	Essential Graph	Edges of high covisibility plus the spanning tree and the loop closure edges, used for pose graph optimization.
追蹤	Tracking	The front-end thread that estimates the camera pose for every frame.
局部建圖	Local Mapping	The thread that manages and optimizes the local map.
閉環	Loop Closing	The thread that detects a revisit and corrects the global drift.
僅運動 BA	Motion-only BA	A bundle adjustment that holds the map points fixed and optimizes only the camera pose.
局部 BA	Local BA	A bundle adjustment over the local key frames and the map points they see.
全域 BA	Full BA	A bundle adjustment over all key frames

Chinese Term	English Term	Explanation
		and map points.
地點辨識	Place Recognition	The module that judges by bag-of-words comparison whether a place has been visited before.
重定位	Relocalization	Finding one's position again within the map after tracking has failed.
相似變換	Similarity Transform	A transformation including scale, used in monocular loop closure correction.

22.1 ORB Feature Extraction

ORB-SLAM2 was developed by Mur-Artal and Tardós of the University of Zaragoza in Spain and is the most influential open-source system in the field of visual SLAM. Its name states its core outright: the whole system is built upon the ORB feature.

Why ORB: Three Reasons

Chapter 20 has already discussed ORB's advantage in speed, but within ORB-SLAM2 the choice of ORB rests on deeper considerations at the level of the system. These features are robust to rotation and to scale, and they also have good invariance to the camera's automatic gain, its automatic exposure, and changes of illumination; besides this they are fast to extract and to match, which supports real-time operation, and they show good precision and recall in bag-of-words place recognition.

- Speed: the hard requirement of a real-time system (Section 20.3).
- Invariance to exposure and changes of illumination: this point is often overlooked yet is absolutely crucial—a camera's automatic gain and automatic exposure continually alter the brightness of the image, and without this invariance the system loses tracking frequently. It is also precisely the soft spot of the direct method (Section 20.2).
- Suitability for a bag of words: a binary descriptor builds a vocabulary tree efficiently, making place recognition both fast and accurate.

A Key Design: One Set of Features, Three Uses

This is one of the most elegant decisions in the architecture of ORB-SLAM2. The system uses one and the same set of ORB features for all three tasks of tracking, mapping, and place recognition.

The benefits of this design are enormous:

1. Duplicate computation is saved: features need be extracted from an image only once and the three modules share them. Were tracking to use ORB and loop closing SIFT, two sets of features would have to be computed for every frame and the computational load would double.
2. The data structures are unified: a map point need store only one kind of descriptor, and a key frame only one bag-of-words representation.
3. The behavior is consistent: a feature that tracking can match, loop closing can match too, and no contradiction arises in which tracking sees something that loop closing cannot recognize.

Implementation Details: the Pyramid and Even Distribution

- The image pyramid: to gain scale invariance, ORB-SLAM2 scales the image into a pyramid of several levels and extracts FAST corners at every level. This lets the system match the same object at different distances.
- Even distribution: extracting FAST corners directly causes features to concentrate where texture is rich (crowding onto a single picture, for instance) while other regions are left empty. ORB-SLAM2 subdivides the image recursively with a quadtree to ensure that features are spread evenly over the whole frame—exactly the same intent as LOAM's selection of features by subregion in Section 18.2 (avoiding a degenerate geometric condition).

22.2 Initialization for Monocular, Stereo, and RGB-D

A great feature of ORB-SLAM2 is that one system supports three kinds of camera. But initialization differs greatly among the three, and this is one of the keys to understanding the system.

Monocular: the Most Difficult Initialization

A monocular camera has no depth information and must build its initial map from the parallax of two images (Chapter 21). And as Case 21-1 of Chapter 21 described, if the scene is planar or the camera rotates purely, epipolar geometry degenerates.

ORB-SLAM2's remedy is extremely clever: it computes two geometric models in parallel—a homography for a planar scene and a fundamental matrix for a non-planar one—and then chooses between them according to their relative scores.

- If the homography scores distinctly higher, the scene is judged planar and the motion is factored out of the homography.
- Otherwise the solution proceeds by the fundamental matrix (the essential matrix).
- If neither is good enough (insufficient parallax, for instance), this attempt at initialization is abandoned and the next frame is awaited for a retry.

This is the mechanical reason behind the initialization that will not succeed in students' implementations: the system is in fact protecting you, refusing to build a wrong map from an unreliable model. The remedy is as Chapter 21 described—translate, and create enough parallax.

Stereo and RGB-D: Able to Begin at Once

Initialization for stereo and RGB-D cameras is far simpler: there is depth information in the very first frame, so feature points with depth can be turned into map points directly, an initial map is built, and the system enters normal tracking immediately.

- Stereo: depth is computed from the disparity of the left and right images (Section 9.6).
- RGB-D: the depth image is read directly.
- Their common merits: absolute scale, no waiting for parallax to accumulate, and no failure under pure rotation.

Distinguishing Near Points from Far Points

ORB-SLAM2 has an important design in its stereo and RGB-D modes: feature points are divided by depth into near points and far points. The depth measurement of a near point (one whose depth is less than some multiple of the baseline) is reliable and can be triangulated into a map point directly; the depth of a far point is unreliable (the depth error growing with the square of the distance in Section 9.6), so it is triangulated only after observations from several viewpoints have accumulated enough parallax, just as in the monocular case.

This design displays engineering pragmatism: not all measurements are equally credible, and a system ought to handle them according to their credibility—of a piece with the spirit of weighting by reliability in the Kalman gain of Chapter 11.

22.3 The Tracking Thread

ORB-SLAM2 runs in three parallel threads. The system has three principal parallel threads: tracking, which localizes the camera for every frame by matching features against the local map and applying a motion-only BA to minimize the reprojection error; local mapping, which manages and optimizes the local map and performs local BA; and loop closing, which detects large loops and corrects the accumulated drift by pose graph optimization. This section looks at the first.

The Task of Tracking

The tracking thread is the front end, and its sole task is to compute, for every image that comes in, where the camera is now, as fast as possible. It must run in real time (in step with the camera's frame rate), so its computation must be light.

Three Tracking Strategies

ORB-SLAM2 adopts three strategies according to circumstance, attempted in order from the easiest to the hardest:

4. Tracking by the motion model: assume the camera moves at constant velocity, predict the current pose from the velocity of the previous frame, then match against the map points of the previous frame and refine. This is the fastest and the principal path under normal conditions.
5. Tracking by the reference key frame: if the motion model fails (the camera suddenly changing its motion, say), match features against the nearest reference key frame with the help of the bag of words and then solve for pose.
6. Relocalization: if both of the above fail (tracking is lost), the place recognition module is started to search the key frame database of the whole map for a similar view, and the pose is solved by PnP with RANSAC so that the system finds itself again.

Tracking the Local Map

Having obtained a preliminary pose, the tracking thread does one more crucial thing: it projects further map points from the local map into the current frame, looks for additional matches, and optimizes once more. This lets the pose estimate use more observations, and the accuracy improves appreciably.

The local map is obtained by way of the covisibility graph (Section 22.6): find the

key frames that share many observations with the current frame, together with the map points they see. These graph structures let the system take a local window of key frames, so that tracking and local mapping operate within a local range and can therefore work in a large environment.

The Selection of Key Frames

The last task of the tracking thread is to decide whether this frame ought to become a key frame. The criteria broadly include: enough frames have passed since the last key frame was inserted; the local mapping thread is idle; and the current frame tracks enough map points but its proportion of shared observations with the reference key frame has fallen (indicating an appreciable change of viewpoint).

ORB-SLAM2's strategy is to insert loosely and cull strictly: insert key frames as freely as possible to keep tracking robust, then let the local mapping thread cull the redundant ones (Section 22.4). This design makes the system hard to lose under fast motion while keeping the key frames from swelling without limit.

22.4 The Local Mapping Thread

The local mapping thread is the middle end; it need not run in real time, but it is responsible for maintaining the quality of the map. The local mapping thread inserts new points into the local map and optimizes the local map by performing local bundle adjustment.

Five Steps

7. Insert the key frame: the new key frame is inserted into the covisibility graph and into the spanning tree that links a key frame to the key frame with which it

shares the most points, and a bag-of-words representation of it is built (for the data association used in triangulating new points).

8. Cull poor map points: a newly created map point must pass a strict test before it is kept—it must be observed by enough key frames and its tracking success rate must be high enough. Those that do not qualify are deleted at once, lest they pollute the map.
9. Create new map points: new map points are produced by triangulating the ORB features of key frames connected in the covisibility graph; unmatched ORB features in one key frame are compared with unmatched ORB features in another, and a pairing is valid only if it satisfies the epipolar constraint; after a pair of ORB features has been triangulated, the depth in both frames must also be checked to be positive, and the parallax, the reprojection error, and the consistency of scale are all tested.
10. Local bundle adjustment: perform a local BA (Section 12.5) over the current key frame, the key frames connected to it in the covisibility graph, and all the map points those key frames see. This is the principal source of accuracy.
11. Cull redundant key frames: if nine tenths or more of the map points seen by a key frame are also seen by at least three other key frames, it is redundant and is deleted. This keeps the scale of the map from growing without limit over time.

Strict Quality Control

Students should notice that steps 2, 3, and 5 are all deleting things. This is precisely the secret of ORB-SLAM2's accuracy—it is extremely severe about the quality of the map, preferring fewer map points and key frames to unreliable data.

The multiple tests applied in triangulating a new point are especially rigorous: the

epipolar constraint, positive depth, the parallax angle (Section 21.5), the reprojection error, and the consistency of scale—five gates, not one of which may be omitted. This reflects an engineering principle: a wrong map point is more dangerous than no map point, because it will go on misleading every later estimate of pose.

A Consideration of the Baseline

One further detail of the implementation is worth mentioning: in triangulating new points the system takes neighboring key frames from the covisibility graph (more of them in monocular mode, fewer otherwise) and uses only those whose baseline is large enough. This is a direct application of the parallax angle decides the accuracy of triangulation of Section 21.5—a pair of key frames with too short a baseline triangulates to points whose depth cannot be trusted.

Measuring It: How Much Does Each of the Five Gates Block

Section 22.4 says that a new map point must pass five tests and that not one of the five gates may be omitted. That sounds reasonable, but how much does each of them actually block? This subsection counts it out with a reproducible simulation.

The setting is a laboratory 14×10 meters with 1,400 landmarks in all on the walls, the floor, and the furniture; the camera travels twice around an elliptical path (the second circuit of slightly smaller radius, since a real revisit does not follow the same line), and key frames are selected by the criteria of Section 22.3. Then, for every pair of covisible key frames the system would really attempt, the landmarks they see in common are taken as correct candidate matches, and 25 percent false matches are mixed in—half of them random mismatches and half deliberately made to fall on the epipolar line but at the wrong depth, because real false matches are often of exactly that kind (matching searches along the epipolar line to begin with).

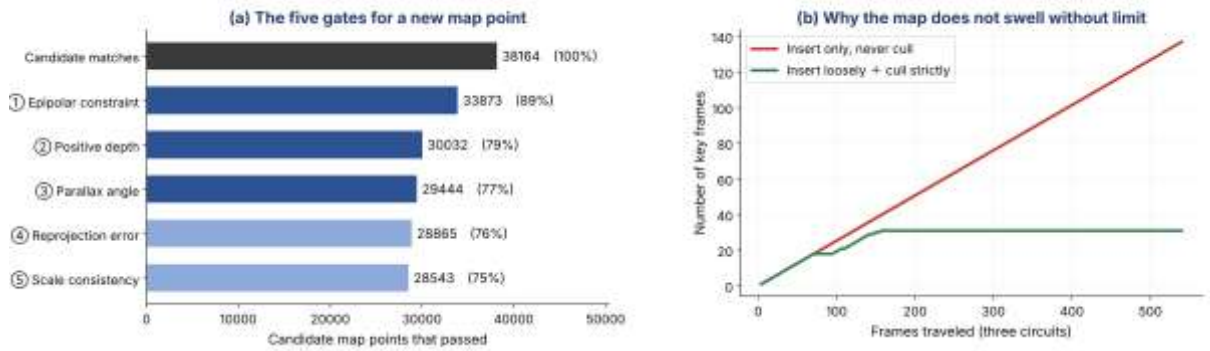


Figure 22-1 The Five Gates a New Map Point Must Pass, and Why Key Frames Do Not Grow Without Limit

Panel (a) is the funnel. Of 38,164 candidate matches, gate ① the epipolar constraint blocks 4,291 (89 percent remaining), gate ② positive depth blocks a further 3,841 (79 percent remaining), gate ③ the parallax angle blocks 588 (77 percent remaining), and gate ④ the reprojection error and gate ⑤ the consistency of scale block 579 and 322, leaving 28,543 at the end (75 percent).

What is most worth discussing in this figure is the gap between the first three gates and the last two. The epipolar constraint, positive depth, and the parallax angle together block more than nine tenths of all the candidates blocked, while the reprojection error and the consistency of scale between them block 2.4 percent of the candidates. Does this mean the last two could be dispensed with? It does not, for two reasons. First, the few hundred points they block are exactly those that passed the first three gates and are still wrong—and such points are the most dangerous of all, because they look entirely plausible. Second, this simulation does not model pyramid levels, whereas the scale consistency test of a real system compares the ratio of observation distances against the difference in pyramid level, and on real data the proportion it blocks is far higher.

More important than the number is the quality. Among the 28,543 surviving candidates,

97.1 percent are correct matches—a great improvement on the original 75 percent; of 9,541 false matches 8,706 were blocked (91 percent), while of 28,623 correct matches only 915 were killed by mistake (3.2 percent). This allocation, erring on the side of strictness, is the engineering realization of the sentence a wrong map point is more dangerous than no map point in Section 22.4.

Panel (b) answers another question: since key frames are inserted loosely, will they explode in number if the robot circles the same place three times? The red line is insertion with no culling—137 key frames accumulate over three circuits, and the growth is still linear. The green line adds the nine-tenths rule for culling redundancy (delete a key frame if nine tenths or more of the map points it sees are also seen by at least three other key frames), and the result settles at 31 by the end of the first circuit and stays at 31 for the remaining two.

This is precisely the key to ORB-SLAM2's ability to run for long periods. The scale of the map is decided by how large the environment is and not by how long one has been running—the same idea as the map maintenance of the inspection robot in Section 19.6 and the submap management of Cartographer in Chapter 15.

Insert Loosely, Cull Strictly: Why Not Simply Be Strict from the Start

A natural question: since seven tenths will be deleted in the end, why not insert fewer to begin with?

Because the amount of information available at insertion and at culling is entirely different. When the tracking thread decides whether this frame should be a key frame, it can see only the present; it does not know where the camera will go next or whether this viewpoint will later prove important. The local

mapping thread, when it culls, has already seen the later key frames and can judge whether a frame is truly redundant.

So this is not wasting first and tidying up after but deferring the decision until there is information enough. The same thinking appeared in the resampling of the particle filter in Chapter 14 and in the timing of submap freezing in Chapter 15, and it is a fairly general principle in system design.

22.5 The Loop Closing Thread

The loop closing thread is the back end, and it handles the most fundamental problem of SLAM: accumulated drift. The loop closing thread is responsible for detecting a revisited region and correcting the accumulated error by pose graph optimization in the g2o framework; after the pose graph optimization it launches a fourth thread to perform a full bundle adjustment, optimizing the whole map together with the trajectory estimate among the key frames.

Step One: Detecting Candidate Loop Closures

ORB-SLAM2 recognizes a revisited place by means of the DBoW2 library; it compares the features of the current image, in binary form, with the binary features stored in the hierarchical DBoW2 database, and if the similarity of the current frame to some representation in the database reaches a certain score, a loop closure has been detected.

The key here is efficiency: the database may hold thousands of key frames, and matching features against each in turn is out of the question. The bag-of-words model compresses each image into a statistical vector of visual words, so that comparison

requires only the similarity of two vectors and is extremely fast—precisely the value of DBoW2 introduced in Section 20.7.

Step Two: Geometric Verification

Similarity in the bag of words means only that it looks alike, which is not enough to confirm. The system next performs a strict geometric verification: it matches features between the candidate loop frame and the current frame and solves for the transformation between them. Only if the similarity is supported by enough inliers is the loop closure accepted.

This step is of the first importance. Chapter 12 and Section 15.4 both stressed it: one wrong loop closure drags the map bodily into a wrong position, with catastrophic consequences. Better, therefore, to miss a loop closure than to accept a wrong one.

Step Three: the Similarity Transform and the Correction of Scale

Here is a detail peculiar to monocular SLAM. Because monocular SLAM has scale drift (Section 20.5), the current scale at a loop closure may differ from the initial scale. What is solved for is therefore not an ordinary rigid transformation (rotation plus translation, Chapter 10) but a similarity transformation—with one further scale factor.

In correcting, the system spreads this difference of scale over the whole trajectory as well, correcting position and scale drift together. Stereo and RGB-D modes, having absolute scale, need only an ordinary rigid transformation.

Step Four: Pose Graph Optimization and Global BA

- Pose graph optimization: perform a pose graph optimization (Chapter 12) on the essential graph (Section 22.6) to pull the whole trajectory straight quickly.

Because only poses are optimized (without the map points) it is fast and can be completed in real time—exactly the strategy of Cartographer in Chapter 15.

- Global BA: after the pose graph optimization, the system launches a fourth thread to perform a full bundle adjustment, optimizing all key frame poses and all map points together for the best overall solution in structure and motion. This step is slower and is therefore run in the background, where it does not block the front-end tracking.

This two-stage design of pull straight quickly first and optimize finely afterward should feel familiar to students—it is the same pattern of system design embodied in the two-layer architecture of LOAM in Chapter 18 and the two subsystems of Cartographer in Chapter 15.

Measuring It: What Exactly Did the Loop Closure Repair

Section 22.5 says that loop closure corrects accumulated drift. That is true, but there is a point commonly misunderstood: joining the two ends of a loop does not mean the middle is now correct. This subsection measures the matter.

The experiment uses a simplified planar pose graph (the camera circling on a horizontal plane, so that a two-dimensional pose suffices, which lets the experiment be reproduced in full without losing the point). Noise is added to each step of the odometry, and after one circuit an appreciable drift has accumulated. Three settings of pose graph optimization are then compared: adding a single loop closure edge, adding the edges of the essential graph, and adding the edges of the full covisibility graph. Every residual is divided by the standard deviation of its measurement (the information weighting of Chapters 11 and 12).

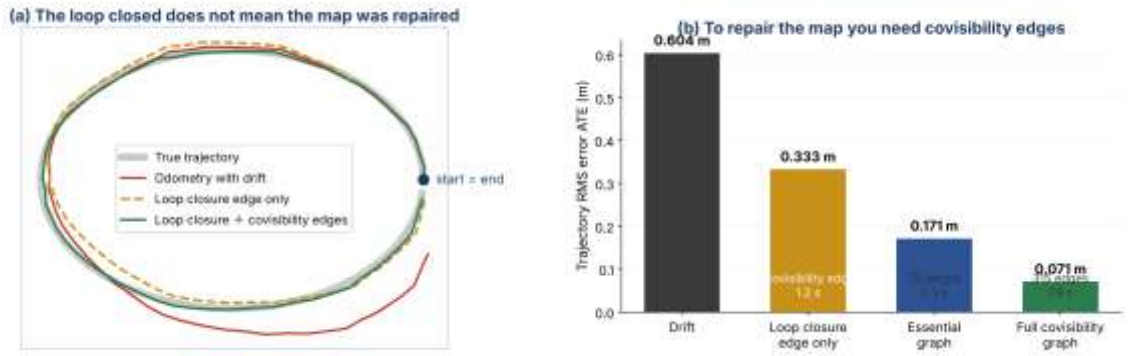


Figure 22-2 The Loop Closed Does Not Mean the Map Repaired

The thick gray line of panel (a) is the true trajectory and the red line the odometry estimate after drift, whose root mean square error over the whole trajectory (ATE) is 0.604 meters, and which on returning to the start is 1.48 meters away from where it should be.

The orange dashed line is the result of adding a single loop closure edge. It does indeed join the two ends—the endpoint error falls from 1.48 meters to 0.18, and the seam that had gaped open now closes. Judged by whether the trajectory closes, it succeeds completely.

But look at the whole trajectory: the orange dashed line still bulges plainly away from the gray one along the top and the lower left, at an ATE of 0.333 meters—only about half of what it was before optimization, which is a poor return for having removed almost all of the endpoint error. In other words, the loop closure has mostly moved the error away from the endpoint and spread it thinly over the middle; it has not removed it. The reason is intuitive: a single loop closure edge constrains only the relative relation between the first frame and the last, while the several dozen poses in between have no constraint among themselves beyond the odometry, and the optimizer has little reason to move them.

The green line is the result with covisibility edges added. The ATE falls to 0.071 meters, one eighth of the drift, and coincides almost exactly with the true trajectory. The key is that a covisibility edge binds together directly those poses that are not adjacent but see the same batch of landmarks, forming a net; the correction of the loop closure can therefore propagate along that net through the whole trajectory instead of stopping at the seam.

Panel (b) sets out the cost of the three settings as well. Loop closure edge only: 0 covisibility edges, 1.2 seconds, ATE 0.333 meters. The essential graph: 110 edges, 3.3 seconds, ATE 0.171 meters. The full covisibility graph: 315 edges, 7.6 seconds, ATE 0.071 meters. It must be said honestly that in this simulation the essential graph, while it halves the error again, still falls short of the full covisibility graph by a factor of more than two. What it buys is a third of the edges and under half the time.

That trade-off is exactly ORB-SLAM2's design: when a loop closure occurs, first perform pose graph optimization on the essential graph, because it is fast enough to finish in real time and will not block the front-end tracking; then launch a global BA in a background thread to refine the map with the full information. Pull the shape roughly straight by the cheap method, then make good the accuracy by the expensive one—the third appearance of the pattern already seen in the two-layer architecture of LOAM in Chapter 18 and the two subsystems of Cartographer in Chapter 15.

A Diagnosis Very Useful for a Project

If a student's SLAM system looks as though the loop closed but the map is still crooked, Figure 22-2 is describing your situation.

What to check is not loop closure detection but the covisibility relations: has

the system correctly created edges between non-adjacent key frames? Is the covisibility threshold set so high that the graph is too sparse? Does the pose graph optimization use the essential graph, or only odometry plus the loop closure?

Conversely, if there is a plain break at the seam of the loop closure, that is a problem of loop closure detection or geometric verification. The causes of the two symptoms are entirely different; tell them apart before setting to work.

22.6 The KeyFrame, the MapPoint, and the Covisibility Graph

The data structures of a system decide what it can do. This section describes the four core data structures of ORB-SLAM2.

The key frame (KeyFrame)

A key frame is a retained image frame representing a viewpoint of some significance. It stores: the camera pose (the transformation matrix of Chapter 10), all ORB feature points and their descriptors, the bag-of-words vector of that frame, and its connections to other key frames.

Why not keep every frame? Because the information in neighboring frames is highly redundant, and keeping all of them would make the scale of the BA explode. A key frame is the outcome of a trade-off: covering the most viewpoints with the fewest frames.

The map point (MapPoint)

A map point is a feature point in three-dimensional space. It stores: its three-dimensional coordinates, a representative descriptor (the most representative one

chosen from all its observations), the mean direction of observation, and a list of the key frames that have seen it.

That last item is of the first importance: it establishes a two-way link between map point and key frame, letting the system query quickly both which points this key frame sees and who has seen this point. The covisibility graph is derived from exactly this relation.

The covisibility graph

The system maintains a covisibility graph linking any two key frames that observe a map point in common, together with a minimum spanning tree linking all key frames.

- Nodes: every key frame.
- Edges: two key frames are joined by an edge if they observe enough map points in common; the weight of the edge is the number of map points observed in common.

The use of the covisibility graph is to define neighbors: a key frame that shares many observations with the current frame is one that is near in space and similar in viewpoint. Tracking and local mapping both operate only within this local range, which is how real-time performance is maintained in a large environment (the principle quoted in Section 22.3).

The Spanning Tree and the Essential Graph

- The spanning tree: a minimum spanning tree linking all key frames, each key frame joined to the key frame with which it shares the most observations. It is extremely light and guarantees that all key frames are connected.

- The essential graph: the structure on which pose graph optimization is performed at a loop closure. It contains all the edges of the spanning tree, the edges whose covisibility weight exceeds a threshold, and the loop closure edges. Having far fewer edges than the full covisibility graph, it lets the pose graph optimization finish quickly.

This layered design—the full covisibility graph for local operations, the pared-down essential graph for global optimization—is the key to ORB-SLAM2's balance of accuracy against efficiency. These graph structures also serve as the structure used for pose graph optimization at a loop closure.

Data structure	Contents	Principal use
KeyFrame	Pose, ORB features, bag-of-words vector, connections	The skeleton of the map; a variable of the BA
MapPoint	Three-dimensional coordinates, descriptor, list of observations	The reference for tracking; a variable of the BA
Covisibility graph	Key frame nodes + covisibility-weighted edges	Obtaining the local map; defining neighbors
Spanning tree	The edges of the minimum spanning tree	Guaranteeing connectivity; a lightweight structure
Essential graph	The spanning tree + strong covisibility edges + loop closure edges	Pose graph optimization at a loop closure

Table 22-1 The Core Data Structures of ORB-SLAM2

Drawing the Covisibility Graph Out

The covisibility graph is the most abstract idea in the chapter and the most easily mistaken for the key frames adjacent in time. This subsection draws it out. The

simulation is the same as that of Figure 22-1: the camera circles twice around a laboratory, the radius of the second circuit three tenths smaller than the first.



Figure 22-3 The Covisibility Graph, the Local Map, and the Essential Graph

Panel (a) is the full covisibility graph: 91 key frames and 561 edges (an edge is drawn where 15 or more map points are shared, and the thickness of a line represents the strength of the covisibility). Note the dense tangle of lines between the inner and the outer circuit—those are the cross-circuit covisibility edges, joining two frames dozens of key frames apart in time but looking at the same wall in space. The most important message of this figure is that the covisibility relation is decided by space and not by time.

Panel (b) is the local map. The red star is the current key frame and the orange its local map—the neighbors with which it shares most, plus the neighbors of those neighbors, 18 key frames in all. The tracking thread need only project into the current frame the map points seen by these 18 key frames, rather than trying the map points of all 91. Note that the orange points span both circuits at once: the robot can now use the map points built when it passed this way on the first circuit, which is the fundamental difference between SLAM and pure odometry.

More important still, the size of the local map does not grow with the map. In this figure it is 18 out of 91; if the robot were to circle ten more times and the key frames become

four hundred, the local map would still be only those twenty-odd near the current position. This is the concrete meaning of the covisibility graph letting the system work in a large environment in Section 22.6—the computation of tracking has nothing to do with the total size of the map, only with the local density.

Panel (c) is the essential graph: pared from 561 edges to 205, about a third. What is kept is the minimum spanning tree (ensuring every key frame is reachable) plus the edges of greatest covisibility strength. It preserves the overall connective structure of the graph and the principal cross-circuit connections while discarding a great many redundant short edges—this is the graph used for pose graph optimization at a loop closure, and the same kind of pruning as the 110 edges in Figure 22-2(b).

Three Panels for Three Threads

(a) The full covisibility graph: maintained by the local mapping thread, it is the connective structure of the whole map.

(b) The local map: used by the tracking thread, it decides which map points each frame is to be matched against.

(c) The essential graph: used by the loop closing thread, it is the object of pose graph optimization.

One and the same set of connections, from which three threads each take what they need—this is what Section 22.6 means by the data structures decide what a system can do. If the covisibility graph is built wrongly, all three threads go wrong at once, and with different symptoms.

22.7 Implementing ORB-SLAM2 and Testing It on Datasets

This section sets out the points of implementation, in preparation for the full implementation of Chapter 30 and the project of Chapter 31.

Compilation and Dependencies

ORB-SLAM2 is a C++ project, and several dependencies must be in place before compiling:

- OpenCV: image processing and feature extraction (Chapters 9 and 20).
- Eigen3: linear algebra.
- Pangolin: three-dimensional visualization, for displaying the map and the trajectory.
- g2o and DBoW2: already included in the project's Thirdparty folder, responsible respectively for graph optimization (Chapter 12) and place recognition (Section 20.7).

A note on implementation: ORB-SLAM2 was released several years ago, and on newer operating systems and versions of OpenCV a small amount of code may have to be modified before it will compile. Consulting a community-maintained fork, or using a containerized environment to avoid dependency trouble, is recommended—this is where students most often get stuck.

The Configuration File: Camera Parameters

A YAML configuration file must be prepared before running, containing:

- The camera intrinsics: fx, fy, cx, cy (the K matrix of Section 9.1).
- The distortion coefficients: k1, k2, p1, p2 (Section 9.2).
- The image size and the frame rate.

- The baseline times the focal length for stereo, and the depth scaling factor for RGB-D.
- ORB parameters: the number of features per frame (1000 to 2000 is common), the number of pyramid levels and the scale ratio, and the FAST threshold.

The importance of Chapter 9 must be stressed again here: if the camera calibration is inaccurate, there is no accuracy in the whole system to speak of. Be sure to perform a full calibration with your own camera, and do not simply apply the example parameters.

Public Datasets

Dataset	Camera type	Scene	Characteristics
TUM RGB-D	RGB-D	Indoor offices	Motion capture ground truth; suited to evaluating accuracy
KITTI	Stereo	Outdoor roads	Self-driving scenes; long trajectories over a large area
EuRoC MAV	Stereo + IMU	Indoor machine rooms and factory halls	A drone's point of view; includes fast-motion sequences

Table 22-2 Public Datasets Commonly Used for Visual SLAM

The recommended order of study is this: run an easy sequence of TUM RGB-D first (slow motion, indoors, with depth) to establish an experience of success, then take on the fast-motion sequences of EuRoC, and finally attempt the large outdoor scenes of KITTI.

Methods of Evaluation

- Absolute trajectory error (ATE): align the estimated trajectory with the ground truth and compute the root mean square of the position differences at corresponding times. This is the most commonly used measure of overall accuracy.
- Relative pose error (RPE): compute the error of the relative motion over a fixed interval of time, which reflects the degree of local drift.
- Tools: evo is at present the most convenient tool for evaluating trajectories, supporting a variety of dataset formats and able to align, compute metrics, and plot automatically.
- A special note for monocular: a monocular trajectory has no absolute scale (Section 20.5), and a scale alignment with the ground truth (a Sim3 alignment) must be performed before evaluation, or the error will be meaningless.

What to Watch For

While it runs, students are advised to watch several things in the Pangolin window:

12. The process of initialization: in monocular mode, observe how many frames and how much motion the system needs before it succeeds (against Section 22.2).
13. The distribution of key frames: observe how key frames are distributed along the trajectory—are they denser at turns and where the viewpoint changes greatly?
14. Covisibility relations: the lines between key frames in the visualization are the covisibility graph; observe its structure.

15. The moment of loop closure: on returning to the start, observe the instant at which the trajectory and the map are pulled straight (the striking picture of Chapters 12 and 15, reappearing here).
16. Loss of tracking and relocalization: cover the camera deliberately for a few seconds and then uncover it, and observe how the system loses itself and how it relocalizes—one of ORB-SLAM2's most impressive abilities.

At this point a complete visual SLAM system has been laid out whole. But ORB-SLAM2 still has two limitations: it is purely visual (with no IMU, so monocular has the scale problem and fast motion easily loses tracking), and once tracking is severely lost and relocalization fails, the system can go no further. The ORB-SLAM3 of the next chapter was born to solve exactly these two—a tightly coupled visual-inertial fusion, and a multiple-map system that can open a new map when tracking is lost and merge it later.

22.8 After ORB-SLAM2: Two Forks in Visual SLAM, 2024–2026

ORB-SLAM2 was published in 2017. Nearly a decade on, it is still the baseline most often used in comparing new methods—which in itself says something about its weight. But research in visual SLAM has indeed forked in two directions, and this section explains what each sets out to solve and how each relates to this chapter.

One: the First Fork, Replacing the Sparse Map with a Renderable Dense Map

ORB-SLAM2's map is sparse—tens of thousands of three-dimensional points, enough to localize with but not enough to see with. A principal line of recent years brings renderable representations of the 3D Gaussian splatting kind into SLAM, so that the

system builds, while localizing, a dense map that can be rendered back into an image.

What students should note is that systems of this kind have not discarded the content of this chapter. A representative work of late 2024 couples ORB features tightly with Gaussian splatting within a single optimization: tracking minimizes the photometric error and the reprojection error together, and the back end retains a lightweight bundle adjustment to suppress accumulated drift. The authors' reasoning is plain—a Gaussian splatting SLAM that relies on photometry alone is prone to artifacts and noise, chooses its viewpoints poorly, and lacks global optimization, and those three things are exactly what this chapter's key frame strategy, covisibility graph, and loop closure handle.

As to results, that work has a mean trajectory error of 0.91 centimeters on the TUM dataset against ORB-SLAM2's 1.00 centimeters—only a slight advantage in localization accuracy, the real difference lying in rendering quality (about 3.93 decibels better than the pure Gaussian splatting baseline). The authors also state the limitation honestly: training takes considerable time and the system cannot yet localize in real time. This is an important reminder—the price of a dense map is still high.

Two: the Second Fork, Replacing the Front End with a Learned Three-Dimensional Reconstruction Model

The second fork is more radical: hand even feature matching and epipolar geometry to a network. A class of methods appearing in 2025 takes a learned three-dimensional reconstruction model as the front end, predicting dense three-dimensional correspondences directly from two images and then attaching a SLAM back end.

Its most striking property is that it needs no camera calibration—the system uses a generic central camera model and can therefore handle video of unknown origin directly, which will come as something of a shock to students who spent a whole

section of Chapter 9 on calibrating intrinsics. As to speed, it runs at about 15 frames a second on a high-end graphics card and about 5 to 7 on a mid-range one; in accuracy it wins and loses against ORB-SLAM3 and other dense methods depending on the dataset.

Three: How Much of This Chapter Is Left

	ORB-SLAM2 (this chapter)	Gaussian splatting SLAM	SLAM with a learned front end
Form of the map	Sparse feature points	A renderable dense field	A dense point cloud
Front end	ORB matching + PnP	Photometric + features	A neural network predicts correspondences
Intrinsics required	Yes	Yes	Not necessarily
Key frame mechanism	Yes	Yes	Yes
Covisibility graph and loop closure	Yes	Mostly retained	Mostly retained
Real-time operation	Real time on a CPU	Mostly not yet real time	Needs a high-end GPU
Present position	The mature baseline	Rendering quality first	Calibration-free; generalization first

Table 22-3 A Comparison of the Three Routes

One thing can be seen from this table: what the two new routes replace is how the map is represented and how correspondences are found, while key frames, covisibility, loop closure detection, and pose graph optimization—that is, the whole of Sections 22.3

through 22.6 of this chapter—are kept almost untouched.

The reason is not hard to see. What these mechanisms solve is not how to compute a pose but how to manage information over a long run: which frames are worth remembering, which data are related to which, when one ought to admit that one has returned to an old place, and how the error should be redistributed. Whatever the front end is changed to, these questions remain. That is also the reason this chapter is worth reading carefully—what it teaches is system design, and system design outlives algorithms.

Formulas and Procedures

One: a Comparison of the Three Kinds of BA

ORB-SLAM2 uses three kinds of bundle adjustment (Section 12.5) at different stages, differing in which variables are optimized:

$$\textit{Motion-only BA: } \operatorname{argmin}_{R,t} \; \Sigma_i \, \rho\left(\| u_i - \pi(K, R, t, P_i) \|^2 \right)$$

The map points P_i are fixed and only the camera pose (R, t) is sought. Used on every frame in the tracking thread, it is the fastest.

$$\textit{Local BA: } \operatorname{argmin}_{R,t,P} \; \Sigma \text{ the reprojection errors of the local key frames and their map points}$$

Optimizes the poses of the local key frames together with the map points they see. Used in the local mapping thread, it is the principal source of accuracy.

$$\textit{Global BA: } \operatorname{argmin}_{\text{all } R,t,P} \; \Sigma \text{ the reprojection errors of all key frames and map points}$$

Optimizes the whole map; it is the heaviest and is run only in the fourth thread after a loop closure. In these expressions ρ is a robust kernel (Chapters 12 and 15), used to

reduce the influence of outlying observations.

Two: Pose Graph Optimization (Loop Closure Correction)

At a loop closure, pose graph optimization is performed on the essential graph, optimizing only key frame poses (without the map points):

$$argmin\ \Sigma_{(i,j)\in essential\ graph} \| log(\Delta T_{ij}^{-1} \cdot T_i^{-1} \cdot T_j) \|^2_{\Sigma}$$

where ΔT_{ij} is the relative pose measurement recorded on the edge and T_i and T_j are the key frame poses to be optimized. In monocular mode T is a similarity transform including scale (Sim3), while for stereo and RGB-D it is a rigid transform (SE3). This is the concrete realization of the pose graph optimization of Chapter 12.

Three: the Concerted Flow of the Three Threads

Figure 22-4 The Three-Thread Architecture of ORB-SLAM2

【Tracking thread】real time, run every frame

Extract ORB → solve for an initial pose by the motion model / reference frame / relocalization

→ track the local map → refine by motion-only BA → judge whether to insert a key frame

↓ (a new key frame)

【Local mapping thread】not real time, triggered by a key frame

Insert into the covisibility graph and spanning tree → cull poor map points → triangulate new map points

→ local BA → cull redundant key frames

↓ (a new key frame)

【Loop closing thread】not real time, run in the background

DBoW2 detects a candidate → geometric verification → solve the similarity transform (Sim3)

→ pose graph optimization on the essential graph (pulling the trajectory straight)

↓

【The fourth thread】global bundle adjustment, optimizing the whole map and trajectory

Four: the Five Tests for Triangulating a Map Point

Figure 22-5 The Tests a New Map Point Must Pass

① The epipolar constraint: the paired ORB features must satisfy the epipolar geometry (Chapter 21)

↓

② The positive depth test: after triangulation the depth must be positive in both frames

↓

③ The parallax angle test: the parallax must be large enough, or the depth is unreliable (Section 21.5)

↓

④ The reprojection error test: the error on projecting back into both images must be small enough

↓

⑤ The scale consistency test: the observed scale and distance must be plausible

↓

All five passed → it becomes a map point; any one failed → it is discarded

Cases

Case 22-1 Cover the Camera and Uncover It: the Magic Moment of Relocalization

This is the most dramatic demonstration of ORB-SLAM2's capability, and nearly every class is worth doing it once.

Let the system run normally until a stretch of map has been built, then cover the camera lens completely with your hand. The frame goes black, the features drop to zero instantly, the system immediately displays TRACK LOST, and the trajectory stops updating. Now take your hand away and let the camera see a region it has traversed before—in under a second the system jumps back to the correct position and the trajectory continues seamlessly.

What happened in that second? The system started the place recognition module: it turned the ORB features of the current image into a bag-of-words vector, searched the key frame database quickly for the most similar candidates, matched features against the candidates, solved for pose by PnP with RANSAC (Chapter 21), and resumed tracking once the verification passed.

A more interesting variation: after covering the camera, carry it to another corner of the room before uncovering it. The system can find itself just the same—the visual version of the kidnapped robot problem of Chapter 3. By contrast, pure visual odometry (with no map and no place recognition) could never come back after such an occlusion.

This demonstration lets students feel the difference between SLAM and VO at first hand: VO can only reckon forward, while SLAM possesses a map and a memory and can recover from failure. It is also the best gloss on the value of the loop closure detection module in the flow chart of Section 20.1.

Case 22-2 Insert Loosely, Cull Strictly: an Engineering Philosophy

ORB-SLAM2's treatment of key frames looks at first a little contradictory: the criteria for insertion are loose (key frames are inserted often), yet local mapping actively

culls the redundant ones. Why not simply be stricter from the start?

Because the two serve different ends. The aim at insertion is robust tracking: if key frames are too sparse, then under fast motion or a large turn the viewpoint of the current frame differs too much from that of the nearest key frame, matching fails, and tracking is lost. Loose insertion amounts to buying the system insurance.

The aim in culling is a compact map: key frames increasing without limit would swell the scale of the BA, exhaust memory, and gradually slow the system down. So once it is confirmed that the information of a key frame is adequately covered by others (nine tenths of its map points being observed by at least three other key frames), it is removed.

This strategy of stability first and refinement afterward is an engineering philosophy worth remembering: in an uncertain present, do more rather than less (keep your options), and pare down once there is enough information. Students will find it echoes the advantage of graph optimization in Chapter 12, that it can go back and correct history—it is precisely because the system can correct in hindsight that it dares to be loose in the moment.

Case 22-3 The Family That Grew Out of ORB-SLAM2

Once ORB-SLAM2 was open-sourced it became the basis of countless studies. Watching the branches of this technical family spread out shows the lay of the land in visual SLAM research.

- Adding other features: some work adds line segment features alongside point features. The system proposed by PL-SLAM is likewise built on three threads—visual odometry, local mapping, and loop closing; its map consists of key

frames, detected three-dimensional landmarks (both key points and line segments), a covisibility graph, and a spanning tree. Line features are especially valuable in man-made environments (corridors, buildings), where point features may be scarce but lines are abundant.

- Acceleration and lightening: some work proposes a descriptor-free variant. FastORB-SLAM is structured on ORB-SLAM2 and implements three threads, but establishes key point correspondences between adjacent frames by intensity matching rather than descriptor matching, handles the outlier problem with a two-stage, descriptor-independent key point matching method, and finally recovers the camera motion by minimizing the reprojection error.
- Extensions in application: some work applies ORB-SLAM2 to a wearable navigation device for the visually impaired—a sighted person wears the device and walks to build a virtual guide path, which the visually impaired then follow. That study explains that the tracking thread localizes the camera by matching features against the local map, the local mapping thread inserts new points and optimizes by local BA, and the loop closing thread detects large loops and optimizes the pose graph.
- Persistent maps: other work adds map saving to the visual localization of a self-driving car so that a map once built can be stored and reused, the system shutting down the local mapping and loop closing threads in localization mode and running tracking alone.

What these extensions have in common is that they all keep ORB-SLAM2's three-thread skeleton and only replace or add modules within it. This shows the value of a good architecture—it solves not only the problem of the moment but becomes a

platform for later innovation. For a student undertaking a visual SLAM project, improving upon ORB-SLAM2 is a practical path with predecessors to follow.

Hands-On Activities

Activity 22-1 Implementing and Evaluating the ORB-SLAM2 System

Purpose: to compile and run ORB-SLAM2 in full, to evaluate its accuracy on a public dataset, to observe the three threads working in concert, and to experiment with the limits of the system. This is the most important implementation of Part VI and the foundation of Chapters 30 and 31.

Part One: Setting Up the Environment and Running a Dataset (everyone)

1. Install the dependencies (OpenCV, Eigen3, Pangolin) and compile ORB-SLAM2. If compilation errors arise, record the problem and the remedy—an indispensable training in practical engineering.
2. Download an easy sequence of the TUM RGB-D dataset (freiburg1_xyz or freiburg2_desk are recommended to begin with), run it in RGB-D mode, and watch the map and the trajectory in the Pangolin window.
3. Run the same sequence in monocular mode and compare the two: the time taken to initialize, the completeness of the trajectory, the density of the map points.
4. Observe and record: how many key frames did the system create? How many map points? What pattern do the key frames follow along the trajectory?

Part Two: Observing the Behavior of the Three Threads (everyone)

5. Observing initialization (monocular): record the number of frames and the distance of camera movement from start-up to successful initialization, against the principles of Section 22.2.
6. Observing the covisibility graph: watch the lines between key frames in the visualization, judge which key frames share many observations, and compare with the camera trajectory.
7. Observing loop closure: choose a sequence containing a loop (KITTI sequence 00, say), watch the process by which the trajectory and the map are pulled straight at the instant the loop closure fires, and record it on video.
8. A relocalization experiment: interrupt the image input deliberately for a few seconds while running (or choose a sequence containing an occlusion) and observe the whole process of losing tracking and relocalizing, reproducing Case 22-1.

Part Three: Evaluating Accuracy (university level)

9. Install the evo tool, compare the trajectory ORB-SLAM2 outputs with the dataset's ground truth, and compute the absolute trajectory error (ATE) and the relative pose error (RPE).
10. Plot the trajectory comparison and the error against time, find the segment of greatest error, then go back and look at the images of that segment and infer the cause (fast motion? weak texture? moving objects?)
11. A note for monocular evaluation: remember to perform the scale alignment (Sim3) first, or the error is meaningless. Compare the error figures with and without the scale alignment to understand scale ambiguity at first hand.

12. A comparison of the three modes: if the dataset supports it, run the same scene in monocular, stereo, and RGB-D in turn and compare the ATE and the computational load of the three, filling in a comparison table of your own.

Part Four: Parameter Experiments and Testing the Limits (university level)

13. A feature count experiment: set the number of ORB features extracted per frame to 500, 1000, and 2000 in turn and compare the changes in accuracy, frame rate, and number of map points, finding the value suited to your scene.
14. A weak texture challenge: record a video of your own containing a large expanse of white wall and observe where the system loses tracking, verifying the curse of the white wall of Chapters 9 and 20.
15. A fast motion challenge: record a video with quick turns, observe whether tracking fails, and consider how an IMU might help (a trailer for Chapter 23).

Discussion Questions

1. How many frames does initialization in monocular mode require? What happens if the camera only rotates purely? Which statements of Section 22.2 and Section 21.5 does this bear out?
2. How many key frames did the system cull? What would happen over a long run if the culling of redundant key frames were switched off?
3. At the instant of loop closure correction, which segments of the trajectory move the most? Why those segments? (Hint: spreading contradiction evenly, Chapter 12.)
4. By how much do the ATEs of the three modes differ? Is RGB-D necessarily the most accurate? Under what conditions would stereo win?

5. (University level) Raising the feature count from 500 to 2000 improves accuracy by how much? And lowers the frame rate by how much? What does this trade-off mean for a real-time system?

Extended Challenge (university level)

First: read the source of ORB-SLAM2's Tracking.cc, find where the three tracking strategies (the motion model, the reference key frame, relocalization) are implemented, add output messages at the key places, and observe while running under what conditions the system switches strategy and how often. This is the best exercise there is for putting the theory of this chapter beside real code. Second: try running ORB-SLAM2 with your own camera (a phone or a webcam)—first complete the camera calibration of Chapter 9, write the YAML configuration file, then record video or feed it live, and build a map of a classroom or corridor in your own department building. This result can serve directly as material for the project of Chapter 31.

Further Reading

1. R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras," IEEE T-RO, 2017. (The original paper on which this chapter is chiefly based.)
2. R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," IEEE T-RO, 2015. (Its predecessor, the complete design of the monocular version.)
3. The ORB-SLAM2 open-source project (GitHub: raulmur/ORB_SLAM2). (Source code, compilation instructions, and examples of running on datasets.)

4. D. Gálvez-López and J. D. Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," IEEE T-RO, 2012. (DBoW2, the technical basis of loop closure and relocalization.)
5. H. Strasdat, J. M. M. Montiel, and A. J. Davison, "Scale Drift-Aware Large Scale Monocular SLAM," RSS, 2010. (The source of the covisibility graph and of Sim3 loop closure correction.)
6. Gao Xiang and Zhang Tao, Fourteen Lectures on Visual SLAM, Lecture 9 (The back end) and the practical chapters. (The best Chinese-language supplement for dissecting the ORB-SLAM system.)
7. The evo trajectory evaluation tool (GitHub: MichaelGrupp/evo). (Computation of ATE and RPE and plotting of trajectories; indispensable for evaluation.)

Summary

1. ORB-SLAM2 uses one and the same set of ORB features for all three tasks of tracking, mapping, and place recognition; ORB has good invariance to rotation, to scale, to the camera's automatic gain and exposure, and to changes of illumination, is fast to extract and match, and shows good precision and recall in bag-of-words place recognition.
2. Monocular initialization computes a homography (for a planar scene) and a fundamental matrix (for a non-planar one) in parallel and chooses between them by their relative scores; stereo and RGB-D have depth in the first frame and can map at once, and divide features by depth into reliable near points and far points that must accumulate parallax.
3. The tracking thread localizes the camera by minimizing the reprojection error with a motion-only BA, attempting in turn the motion model, the reference key

frame, and relocalization, and obtains a local map through the covisibility graph for further refinement; key frames are inserted by a loose policy.

4. The local mapping thread inserts the key frame into the covisibility graph and the spanning tree and builds its bag of words, culls poor map points, triangulates new points from connected key frames (which must pass five tests: the epipolar constraint, positive depth, parallax, the reprojection error, and consistency of scale), performs local BA, and culls redundant key frames.
5. The loop closing thread detects a revisit by comparing binary features with DBoW2, accepts the loop closure once geometric verification yields enough inliers, solves for a similarity transform (including scale correction for monocular), performs pose graph optimization on the essential graph, and launches a fourth thread for a global BA.
6. The core data structures are the key frame, the map point, the covisibility graph (linking key frames that observe map points in common), the minimum spanning tree, and the essential graph; the covisibility graph serves local operations and so supports large environments, while the pared-down essential graph serves the pose graph optimization of loop closure.
7. Implementation requires OpenCV, Eigen3, and Pangolin, together with accurate camera intrinsics and distortion coefficients supplied in YAML; the datasets commonly used are TUM RGB-D, KITTI, and EuRoC, evaluation uses ATE and RPE as its measures, and monocular requires a scale alignment first.

Carrying Forward

This chapter has assembled the parts of Chapters 20 and 21 into a system that really runs, and has measured several key numbers along the way: the five gates for a new

map point screen the candidates from 38,164 down to 28,543, raising correctness from 75 percent to 97.1 percent while killing only 3.2 percent of the correct matches by mistake; the nine-tenths rule for culling redundancy holds the key frames of three circuits at 31 rather than 137; and Figure 22-2 demonstrated something counterintuitive—with only one loop closure edge the trajectory did close (the endpoint error falling from 1.48 meters to 0.18) while the error over the whole trajectory improved only by half (ATE 0.604 against 0.333 meters); to remove the error in earnest one needs covisibility edges to carry the correction along the whole trajectory (ATE falling to 0.071 meters).

ORB-SLAM2 is the benchmark of maturity in visual SLAM, but it still has two definite limitations: its purely visual architecture leaves monocular with the scale problem and loses tracking easily under fast motion; and when tracking is severely lost and relocalization fails, the system can go no further. The ORB-SLAM3 of the next chapter was born for exactly this—a tightly coupled visual-inertial fusion founded on maximum a posteriori estimation (where the IMU preintegration of Chapter 7 makes its entrance), and a multiple-map system that opens a new map when tracking is lost and merges it seamlessly later. Read Chapter 23 with a question in mind: the covisibility graph of Figure 22-3 is a single graph, so how must this machinery change when the system holds several unconnected maps at once? The summit of Part VI is about to appear.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 23

ORB-SLAM3 and Visual-Inertial SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

The ORB-SLAM2 of Chapter 22 is already a complete system, but it has two hard faults: monocular has no absolute scale, and once tracking is severely lost and relocalization fails, the whole system ends there. The ORB-SLAM3 of this chapter was born against exactly these two—the first solved by the IMU, the second by multiple maps.

This is also the summit of Part VI and the meeting point of Part II with Part VI. Chapter 7 spent a whole chapter on the IMU: why it drifts, what bias is, why excitation of motion is needed before it can be initialized—every one of those conclusions is put to use in this chapter, and quantified.

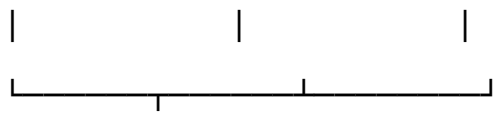
The three figures of this chapter answer three questions apiece: Figure 23-1 measures how long an IMU can hold out when vision fails, and the price of estimating the bias inaccurately; Figure 23-2 turns scale is unobservable under constant-velocity straight-line motion from a maxim into a condition number that can be computed; and Figure 23-4 compares the three fates after tracking is lost—stopping, opening a new map without merging, and Atlas's seamless merge.

Figure 23-0 The Link Map of Chapter 23

The Foundations Already Laid

Chapter 7 The IMU and preintegration Chapter 21 Epipolar geometry Chapter 22 The

ORB-SLAM2 architecture



▼

【Chapter 23 ORB-SLAM3: visual-inertial + multiple maps】

Tight coupling ─ IMU initialization ─ Atlas ─ relocalization and map merging

▼

Chapter 24 The vision-navigated robot (putting SLAM to work on a task)

▼

Part VII Drones and edge AI　　Part VIII ROS implementation

Learning Objectives

The ORB-SLAM2 of the last chapter is already the benchmark of maturity in visual SLAM, but it still has two limitations: its purely visual architecture leaves monocular with the scale problem and loses tracking easily under fast motion; and when tracking is severely lost and relocalization fails, the system can go no further. ORB-SLAM3 was born for exactly this. This chapter is the summit of Part VI and the culmination of the visual line of the whole book. After studying this chapter, students should be able to:

1. Explain the three core advances of ORB-SLAM3 over ORB-SLAM2.
2. Explain the meaning and the value of the four kinds of data association (short-term, mid-term, long-term, and multi-map).
3. Describe the architecture of monocular-inertial and stereo-inertial SLAM and the contributions the IMU makes.
4. Describe the three-stage procedure of IMU initialization founded on maximum a posteriori estimation.

5. Describe the composition of the Atlas multiple-map system and the two-stage strategy of map merging.
6. Describe the mechanism of relocalization and map merging after a loss of tracking.
7. Give examples of applications of visual-inertial SLAM in phones, drones, and wearable devices.

Core Concepts

Chinese Term	English Term	Explanation
視覺慣性 SLAM	Visual-Inertial SLAM	Simultaneous localization and mapping with camera and IMU tightly coupled.
多地圖系統	Atlas	An architecture maintaining several maps at once, one active and several inactive.
活躍地圖	Active Map	The map the tracking thread currently localizes in.
非活躍地圖	Inactive Map	A map in Atlas not in use for the time being but kept for a later merge.
短期資料關聯	Short-term	Matching map elements from the last few seconds.
中期資料關聯	Mid-term	Matching map elements near the camera that carry slight accumulated drift.
長期資料關聯	Long-term	Matching a previously visited region by place recognition, unaffected by accumulated drift.
多地圖資料關聯	Multi-map	Matching and using map elements from an earlier mapping session.
最大後驗估測	MAP Estimation	Seeking the most probable state estimate given the observations.

Chinese Term	English Term	Explanation
IMU 初始化	IMU Initialization	The start-up procedure that estimates scale, the direction of gravity, bias, and velocity.
地圖合併	Map Merging	The operation of fusing two maps seamlessly into one.
焊接視窗	Welding Window	The local range of key frames fused first during a map merge.
多階段作業	Multi-session	A mode of operation spanning several runs and reusing an earlier map.

23.1 The System Characteristics of ORB-SLAM3

ORB-SLAM3 was published in 2021 by Campos, Elvira, Rodríguez, Montiel, and Tardós, and it is the system already introduced in outline in Section 7.7. It is the culmination of the bloodline of feature-based SLAM, evolved by way of PTAM, ORB-SLAM, and ORB-SLAM2, and on its release it was the most complete and most accurate open-source SLAM library then available.

Three Core Advances

1. Tightly coupled visual-inertial SLAM: the first tightly coupled system founded on maximum a posteriori (MAP) estimation, with even the IMU initialization stage built upon MAP (Section 7.7). This solves the monocular scale problem and the question of robustness under fast motion.
2. A multiple-map system (Atlas): on losing tracking it no longer collapses but opens a new map and carries on, merging seamlessly when it later returns to the old region. This solves ORB-SLAM2's fatal weakness that losing tracking is the end.
3. Thoroughgoing use of four kinds of data association: this is the most fundamental architectural innovation. ORB-SLAM3 is the first system to use short-term, mid-term, long-term, and multi-map data association at every algorithmic stage.

The Sensor Configurations Supported

As Section 7.7 mentioned, ORB-SLAM3 supports an extremely wide range of configurations, an important source of its practical value:

- Camera types: monocular, stereo, RGB-D.

- Lens models: pinhole and fisheye (Chapter 9). Fisheye support is especially valuable for indoor wide-angle applications.
- With or without an IMU: every camera mode has a corresponding visual-inertial version.

Combining these, the system supports monocular, monocular-inertial, stereo, stereo-inertial, RGB-D, RGB-D-inertial, and other modes—one and the same codebase, with only a parameter changed. This makes it an excellent platform for research and teaching: students can compare the performance of different sensor configurations within exactly the same framework.

Accuracy

Section 7.7 has already quoted the key figures; here we add its performance in multi-session operation. In the EuRoC multi-session experiments, Atlas's map merging improves accuracy by more than a factor of two over CCM-SLAM and VINS-Mono; its advantage over VINS-Mono grows from 2.6 times in a single session to 3.2 times across sessions. On the whole, the paper's abstract claims an accuracy two to ten times better than previous methods; after its open-source release it became the standard basis of comparison for hundreds of later papers.

This phenomenon of winning already in a single session and winning by more across sessions shows the value of the multiple-map architecture plainly: a system that can reuse a map it built before is naturally more accurate. For robots actually deployed (running in the same site every day) this matters greatly.

23.2 Four Kinds of Data Association: the Spine of the Architecture

This section is the key to understanding the design philosophy of ORB-SLAM3. The authors divide data association—judging whether observations at different moments correspond to the same map element—into four levels, and argue that previous systems generally used only one or two of them.

Type	What is matched	Technical means	What it solves
Short-term	Map elements from the last few seconds	Projection matching	Basic frame-by-frame tracking
Mid-term	Elements near the camera carrying slight drift	Projection matching against the local map	Reduces the accumulation of drift and improves accuracy
Long-term	A previously visited region (however great the drift)	Place recognition (DBoW2)	Loop closure correction, relocalization
Multi-map	Map elements from an earlier mapping session	Place recognition + map merging	Reuse across sessions; recovery after a loss of tracking

Table 23-1 A Comparison of the Four Kinds of Data Association

Short-Term and Mid-Term: the Basis of Everyday Operation

Short-term and mid-term data association between a frame and the active map is found routinely by the tracking and mapping threads—projecting map points at the estimated camera pose and searching for a match within a window of only a few pixels in the image.

Mid-term association is an important source of ORB-SLAM3's advantage in

accuracy. Experiments show that in single-session monocular-inertial operation on the EuRoC dataset, ORB-SLAM3 is 2.6 times as accurate as VINS-Mono, which is credited to its ability to use mid-term data association.

Why does mid-term association matter so much? Because many visual odometry systems perform only short-term association (comparing only with the last few frames), so that once a map point leaves the field of view for a time and comes back, the system treats it as a new point—which wastes information and lets drift go on accumulating. ORB-SLAM3 can rematch these old points that are still nearby but somewhat drifted, which amounts to making small corrections continually.

Long-Term and Multi-Map: Recovery and Reuse

For the long-term data association required by relocalization and loop closure detection, ORB-SLAM uses the DBoW2 bag-of-words place recognition system, an approach adopted by most recent VO and SLAM systems as well.

Multi-map association is the fourth level, peculiar to ORB-SLAM3. It lets the system match map elements from different mapping sessions—whether a new map opened after a loss of tracking within one run, or a map built and saved yesterday. This opens the possibility of multi-session operation.

A Summary of the Idea

These four levels can be understood through an analogy with memory: short-term is what I just saw, mid-term what is familiar hereabouts, long-term where I have been before, and multi-map what I experienced on another occasion. A good SLAM system should make good use of every level of memory, as a person does—and that is the design philosophy of ORB-SLAM3.

23.3 Monocular / Stereo Visual-Inertial SLAM

Section 7.6 has already explained the four great contributions of the IMU in visual-inertial SLAM (absolute scale, the direction of gravity, filling the gaps where vision fails, and high-rate output); this section focuses on how ORB-SLAM3 realizes tight coupling.

Why Tight Coupling Is Needed

An IMU measures angular rate and linear acceleration but drifts appreciably, so that double integration of IMU data alone cannot be used for long-term pose estimation. That is the central conclusion of Section 7.3.

Conversely, pure vision fails under fast motion, weak texture, and violent changes of illumination. The two must therefore be fused—and as Section 7.6 described, tight coupling (putting the IMU preintegration constraint and the visual reprojection error into one and the same optimization problem) is appreciably more accurate than loose coupling.

How ORB-SLAM3 Realizes Tight Coupling

All three threads of the system are extended on account of the IMU:

- The tracking thread: besides minimizing the reprojection error, it adds the inertial error term of the IMU preintegration (Section 7.5) and estimates pose, velocity, and IMU bias together.
- The local mapping thread: the local mapping module adds key frames and map points to the current active map, removes redundant frames, and optimizes the map over the key frames neighboring the current one by a visual or a visual-inertial bundle adjustment.

- The loop closing and map merging thread: in inertial mode, loop closure correction must also handle the consistency of the direction of gravity and of scale.

The Extension of the State Vector

Against pure vision, visual-inertial SLAM has more to estimate for each key frame:

- Pure vision: pose (rotation R, translation t), 6 degrees of freedom in all.
- Visual-inertial: pose + velocity + accelerometer bias + gyroscope bias, 15 degrees of freedom in all.

More state means more computation, but what it buys is absolute scale (monocular no longer has a scale ambiguity), a known direction of gravity (the map is leveled automatically and roll and pitch become observable), and the ability to keep tracking for several seconds when vision briefly fails. In the great majority of applications this is an excellent bargain.

Stereo-Inertial: the Most Robust Configuration

If cost and bulk are no object, stereo-inertial is ORB-SLAM3's most robust configuration: the stereo pair supplies absolute scale and single-frame depth (with no waiting for parallax to accumulate) and the IMU supplies high-rate motion and robustness under fast motion. The EuRoC accuracy of 3.5 centimeters quoted in Section 7.7 is the result of the stereo-inertial mode. For a campus inspection robot or a drone project, this is the configuration of first choice where conditions permit.

Measuring It: How Long Can the IMU Actually Hold Out

Section 23.3 says that tight coupling can keep tracking for several seconds when vision briefly fails. How many is several? This subsection computes it, and answers a more

important question along the way: what does that ability depend on?

The experimental setting: let the body undergo a motion containing accelerations, decelerations, and rotations, with the IMU putting out specific force readings at 200 Hz (with bias and noise). From some moment on, vision is assumed to fail completely for τ seconds, during which the position is reckoned by double integration of the IMU alone, and at the end the reckoned position is compared with the true one. The key variable is how accurately the system has estimated the accelerometer bias, since the estimated bias must be subtracted before reckoning.

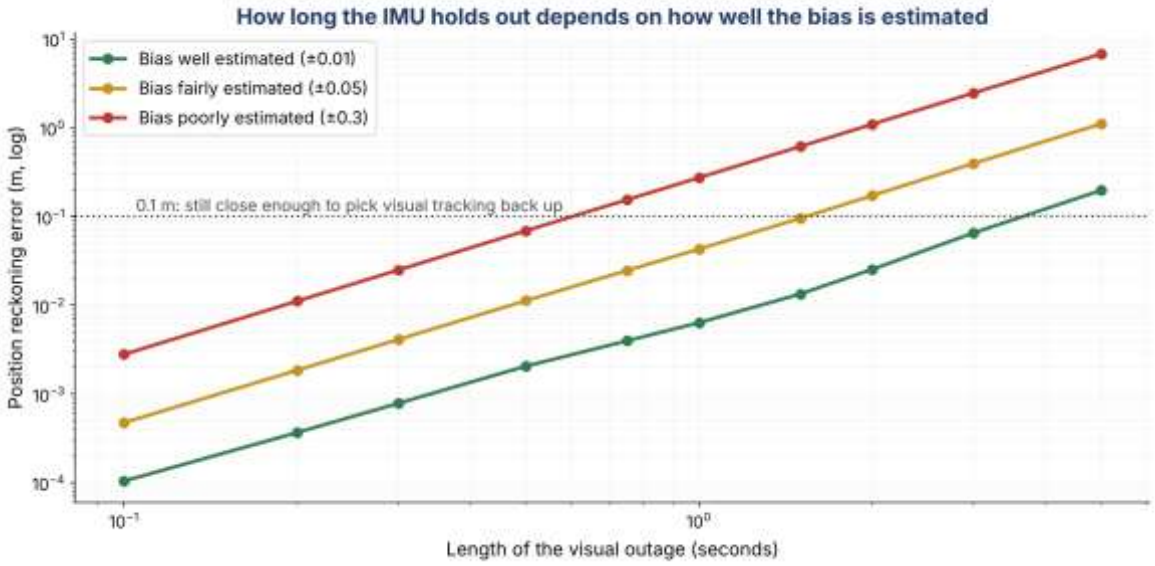


Figure 23-1 How Long the IMU Holds Out Depends on How Well the Bias Is Estimated

All three lines grow quadratically—the inevitable result of Section 7.3: an error in acceleration integrated twice becomes an error in position, so the error accumulates with the square of time. The real difference lies in the vertical distance between the three lines.

If the bias is estimated to the level of ±0.01 meters per second squared (the green line), the position error after 1 second of visual failure is 0.006 meters, after 2 seconds 0.025

meters, and even after 5 seconds only 0.20 meters. This accuracy is enough for the system to project the map points back into the frame and pick up again the moment vision returns—the technical basis of the user hardly noticing the interruption at all.

If the bias is estimated only to ±0.3 meters per second squared (the red line), the same 1-second error is 0.28 meters, the 2-second error 1.10 meters, and the 5-second error 6.82 meters. Against the green line, the difference at 2 seconds is fortyfold. An error of 1.10 meters means that reprojection will not line up with the original map points at all, and the system can only fall back on relocalization, or lose tracking outright.

These figures explain something: the IMU in visual-inertial SLAM is not simply a backup sensor. Whether it can save the situation depends on whether the bias has been estimated continually and correctly—and the estimate of the bias depends in turn on the positional constraint vision supplies. The two depend on each other, which is the fundamental reason Section 23.3 stresses tight and not loose coupling: attach the IMU as an independent module and the bias will never be estimated accurately.

Why Bias Matters Far More Than Noise

The random noise of an IMU is zero-mean, and integration cancels part of it, so the error grows roughly as $\sqrt{t}$. But bias is a constant offset, and after double integration the error grows as t^2—the two are not of the same order at all.

Take the numbers of this experiment: the accelerometer noise is 0.02 meters per second squared, while an uncalibrated bias may reach 0.1 meters per second squared or more. After five seconds the error contributed by the noise is under a few centimeters, while that contributed by the bias is on the order of meters.

So when students are tuning a visual-inertial system and find that the position drifts away slowly while standing still, what to check is the bias estimate and not the noise parameters.

23.4 IMU Initialization and Preintegration

Section 7.7 has introduced the difficulty of IMU initialization and the necessity of excitation of motion; this section goes deeper into its three-stage procedure—one of ORB-SLAM3's most important technical contributions.

What Initialization Must Solve For

At start-up a number of mutually coupled unknowns must be determined together:

- Scale: the true size of the map in monocular mode (known for stereo and RGB-D, which are therefore simpler).
- The direction of gravity: which way is down, so that gravity can be subtracted from the accelerometer readings (the idea of specific force in Section 7.1).
- IMU bias: the offsets of the accelerometer and the gyroscope (Section 7.3).
- Initial velocity: the velocity at each key frame.

The Three-Stage Procedure

ORB-SLAM3 splits initialization into three stages that converge by degrees:

4. Vision-only MAP estimation: first build a short stretch of map and trajectory from vision alone (of unknown scale in monocular mode). This provides the motion reference the later inertial estimate requires.

5. Inertial-only MAP estimation: hold the visual trajectory fixed and use the IMU measurements alone to estimate scale, the direction of gravity, bias, and velocity. With the visual trajectory already settled, this subproblem becomes relatively simple.
6. Joint visual-inertial BA: finally put all the variables (poses, velocities, biases, scale, gravity, map points) into one optimization problem for a joint refinement.

This strategy of solving separately first and refining jointly afterward greatly improves both the speed and the reliability of initialization.

The Performance of Initialization

IMU initialization reaches a 5 percent scale error within 2 seconds, and about 1 percent after refinement. Against the earlier systems mentioned in Section 7.7 (ORB-SLAM-VI requiring about 15 seconds) this is an enormous advance—2 seconds is all but imperceptible to a user, while 15 is plainly trying.

A Note on Implementation: Excitation of Motion

Let us stress again the key reminder of Section 7.7: IMU initialization requires sufficient excitation of motion. If the body moves only at constant velocity in a straight line, or not at all, the accelerometer reads almost nothing but gravity, and the system cannot separate scale from bias (they are mathematically unobservable).

In practice: after start-up, hold the device and move it for a few seconds with accelerations, decelerations, and rotations (small movements forward and back and side to side, with some turning), and initialization will complete smoothly. Together with pure rotation cannot triangulate of Chapter 21, this is one of the two great implementation traps of visual-inertial SLAM—students must remember it when they

come to implement Chapter 30.

Measuring It: Why You Really Must Shake It About

Section 23.4 says that if the body moves only at constant velocity in a straight line the system cannot separate scale from bias (they are mathematically unobservable). That statement matters, but unobservable sounds like an abstract word. This subsection turns it into a number that can be computed.

The method is to write the second stage of initialization (the visual trajectory held fixed, inertial estimation alone) as a system of linear equations. The unknowns are the velocity at each key frame, the gravity vector, the scale factor, and the accelerometer bias. The equations come from the preintegration relations between neighboring key frames—one set from position and one from velocity. Take the singular value decomposition of the coefficient matrix of this system, and the ratio of the largest to the smallest singular value (the condition number) tells us directly how well the problem can be solved.

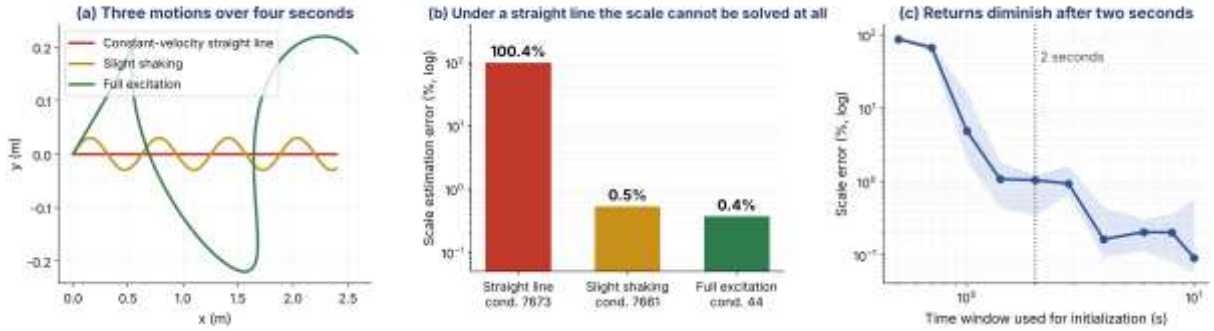


Figure 23-2 Excitation of Motion Decides Whether Initialization Can Be Solved at All

Panel (a) draws three motions over four seconds: constant-velocity straight line (red), slight shaking (amber, swaying about 3 centimeters sideways), and full excitation (green, with acceleration and deceleration fore and aft, lateral and vertical movement, and rotation in yaw and pitch). Note that all three trajectories are visibly moving—from

the trajectory alone it is hard to foresee that their initialization results will differ so greatly.

Panel (b) is the result, with the vertical axis logarithmic. The scale error of the constant-velocity straight line is 100.4 percent—not inaccurate but entirely insoluble; the scale actually solved for may even come out negative. The condition number reaches 7,673. Slight shaking brings the scale error down to 0.54 percent and full excitation to 0.37 percent, with the condition number falling to 44, a hundred and seventy times smaller than for the straight line.

Why does a constant-velocity straight line fail completely? The reason is quite plain. What an accelerometer measures is specific force, that is, the projection of true acceleration minus gravity into the body frame. Under constant-velocity straight-line motion the true acceleration is zero and the attitude does not change, so the accelerometer always reads the same constant vector. That constant can be explained as the projection of gravity, or equally as a bias, and the two explanations are mathematically identical—the system of equations simply has not enough information to separate them.

One further detail is worth noting here: slight shaking rescues the scale (0.54 percent), yet its condition number is still 7,661, as high as for the straight line, and its accelerometer bias error is in fact the largest of the three. In other words, a slight shake suffices to make scale observable but not to separate out the bias. This answers exactly to the conclusion of Figure 23-1—estimate the bias inaccurately and the IMU cannot carry you across a gap in vision.

Panel (c) answers another question: how long initialization takes. Holding the motion

at full excitation, the time window is swept from 0.5 seconds to 10. At 0.5 seconds the scale error is 87.9 percent—which is to say unsolved; at 1 second it falls at a stroke to 4.95 percent, at 1.4 seconds to 1.09 percent and at 2 seconds to 1.05 percent, after which the curve flattens markedly—only at 10 seconds does it reach 0.09 percent. A second and a half or so, that is, is the point of diminishing returns: waiting longer improves the accuracy little while consuming the user's patience linearly. This is why reaching a 5 percent scale error within 2 seconds, quoted in Section 23.4, is regarded as an important advance.

How This Experiment Differs from a Real System

To be fully reproducible, this experiment makes several simplifications: there is no gyroscope bias, the noise of the visual trajectory is independent and identically distributed, and the sensitivity of the preintegration to bias is taken only to first order. The absolute numbers here (0.7 percent at 2 seconds, say) are therefore optimistic against a real system.

A real system is at about 5 percent at 2 seconds and is refined to about 1 percent by the joint visual-inertial BA of the third stage. But the three qualitative conclusions agree entirely: a constant-velocity straight line is unobservable, a shake suffices to rescue the scale, and returns diminish after two seconds.

Should students redo this experiment on their own device, the most valuable product is not the absolute numbers but how much and how long your device must be shaken—which goes straight into your operating manual.

23.5 The Atlas Multiple-Map System

This is the second core innovation of ORB-SLAM3, and the design that best embodies engineering wisdom.

The Composition of Atlas

Atlas is a multi-submap system made up of a set of discrete maps. It maintains one active map, in which the tracking thread localizes the current frame and which the local mapping thread continually optimizes and updates with new key frames; the other maps in Atlas are inactive. The system also builds a database of key frame information founded on the bag-of-words model, used for relocalization, loop closure detection, and map merging.

- The active map: the one in use, on which tracking and local mapping both operate.
- An inactive map: one built earlier and not currently in use, but kept whole in memory awaiting a later merge.
- A shared bag-of-words database: the key frames of every map are registered in one and the same database—the key to being able to compare across maps.

What Happens When Tracking Is Lost

This is the very reason Atlas exists. Recall ORB-SLAM2's position: when tracking is severely lost and relocalization fails, the system simply stops there and all later data is wasted.

ORB-SLAM3's practice is this: if relocalization does not succeed shortly after tracking is lost, set the current map inactive, open a brand new map, and carry on mapping and tracking from nothing. The user hardly notices the interruption. And when the robot later walks back into a region covered by the old map, place recognition

detects a cross-map match and the system merges the two maps at once, restoring a single complete and consistent map.

How It Differs from Other Multiple-Map Schemes

It is worth noting that other systems have proposed multiple-map extensions of ORB-SLAM2, but they keep the submaps as separate entities; ORB-SLAM3 performs a seamless merge and builds a more accurate global map.

This difference is crucial. Merely storing maps separately without merging them loses no data, but you end up with several mutually unrelated maps, unable to plan a path across regions and unable to let their accuracies improve one another. Only a seamless merge realizes the true value of multiple maps.

The Value of Multi-Session Operation

Another important application of Atlas is reuse across runs: save the map built today, load it on starting up tomorrow, and the system can relocalize directly within the old map and go on extending it. This is extremely practical for service robots and inspection robots that run in the same site every day, and it explains the greater multi-session advantage in accuracy mentioned in Section 23.1.

23.6 Relocalization and Map Fusion After a Loss of Tracking

This section describes the technical details of how Atlas operates: how a cross-map association is detected, and how the merge is performed.

Place Recognition: a Design for High Recall

Whenever the mapping thread creates a new key frame, place recognition starts and attempts to detect a match with any existing key frame in Atlas. If the matching key

frame found belongs to the active map, a loop closure is performed; otherwise it is a multi-map data association, and the active map and the matching map are merged.

This is an elegant unification: loop closure and map merging are in fact two outcomes of one mechanism—the only difference being whether the matched key frame belongs to the same map or a different one.

The Trade-off Between Precision and Recall

Place recognition faces a classic dilemma:

- Recall must be high: every true match must be found if possible, or opportunities for loop closure and merging are missed.
- Precision must be a hundred percent: one wrong match drags the map bodily into a wrong position and causes a catastrophe (as Chapters 12, 15, and 22 have stressed repeatedly).

ORB-SLAM3's practice is this: for high recall, query the DBoW2 database for several similar key frames in Atlas for every new active key frame; and for a hundred percent precision, put every candidate through several steps of geometric verification.

Geometric verification is performed by a direct Sim(3) or SE(3) alignment (using RANSAC and Horn's method). Here again we see the shadow of RANSAC from Chapter 21, and the reason monocular requires a similarity transform (including scale) from Chapter 22.

The Two-Stage Strategy of Map Merging

Once a cross-map association is confirmed the merge is to begin. But there is an engineering difficulty here: the inactive map may be very large, and merging it all at once would block the system with computation.

ORB-SLAM3's remedy is to split the merge into two steps: first, perform the merge within a welding window defined by the matched key frames and their neighbors; and in the second stage, propagate the correction to the rest of the merged map by pose graph optimization.

Figure 23-3 The Two-Stage Strategy of Map Merging

A cross-map association is detected: key frame Ka of the active map Ma ↔ key frame Km of the inactive map Mm

↓ estimate the aligning transformation Tam

【Stage one: the welding window】

Perform the merge within the local window formed by the neighbors of Ka and Km

Search densely for mid-term data associations within this window to improve accuracy

→ locally usable at once; the tracking thread is not interrupted

↓

【Stage two: propagating the correction】

Propagate the correction to the rest of the merged map by pose graph optimization

→ global consistency is restored

This strategy of welding locally first and propagating globally afterward lets the merge proceed in real time without blocking tracking. Students should notice that it

agrees entirely with the architectural thinking of Cartographer's local in real time, global in the background in Chapter 15 and LOAM's high-rate odometry, low-rate mapping in Chapter 18—a universal pattern in the design of robotic systems.

A Detail: Strengthening Mid-Term Association

The authors' second innovation is this: once the relative pose between the new key frame and the matched map has been estimated, a local window is defined by the matched key frame and its neighbors in the covisibility graph, and mid-term data associations are searched for densely within this window to improve accuracy.

This echoes once more the spine of Section 23.2: ORB-SLAM3's advantage in accuracy comes in large measure from using mid-term data association wherever it can, at every point in the system.

Measuring It: Three Fates After Tracking Is Lost

Sections 23.5 and 23.6 described how Atlas operates. This subsection quantifies its value with a simulation: for one and the same trajectory and one and the same loss of tracking, what does each of three system designs end up with?

The setting is a robot circling indoors twice over 120 key frames, with vision failing completely between frames 66 and 78 (through a fast turn or facing a white wall, say). The three cases are: ORB-SLAM2's practice (stopping after relocalization fails), opening a new map without merging, and Atlas's seamless merge. Two measures compare them at the end: how much of the trajectory was covered, and the root mean square error of the final map.

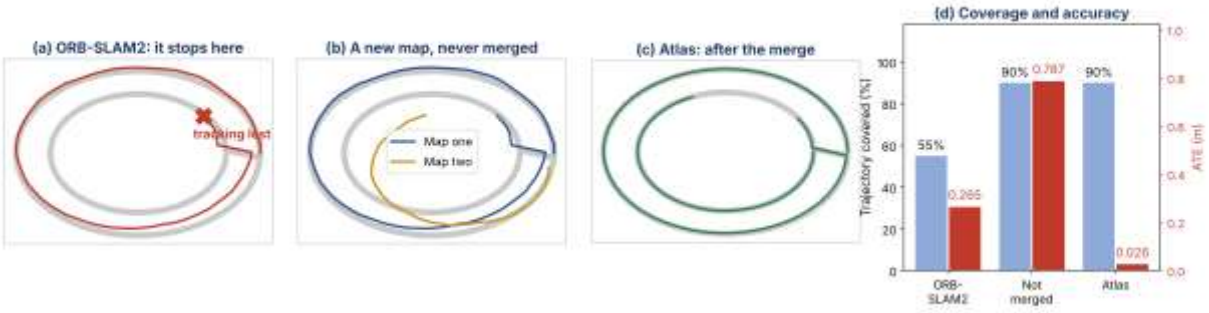


Figure 23-4 Three Fates After Tracking Is Lost

Panel (a) is ORB-SLAM2's position. The system loses tracking at frame 66, fails to relocalize, and stops there, covering only 55 percent of the trajectory while everything after is wasted. The ATE of the part already built is 0.265 meters—and note that this error is not small, because it never had the chance to be corrected by a loop closure on the second circuit.

Panel (b) is opening a new map without merging. Coverage rises to 90 percent and no data is indeed lost. But the two maps each have their own origin, and viewed in a common coordinate frame the overall ATE is 0.787 meters—three times worse than building only half. The blue and orange segments in the figure are plainly out of register, which is the visual form of unable to plan a path across regions in Section 23.5.

Panel (c) is Atlas. When the robot walks back into the region covered by the first map, place recognition detects a cross-map match and the system merges the two into one. Coverage is likewise 90 percent, but the ATE falls to 0.026 meters—one thirtieth of the unmerged case, and ten times better than ORB-SLAM2's half-built map. In the figure the merged green trajectory very nearly coincides with the true one, leaving a gap only over the stretch where tracking was lost.

Panel (d) sets the two measures side by side. The point this figure makes is that the value of Atlas is not only that no data is lost but that the data can correct one another.

The second map can be pulled back into the right position only by virtue of its cross-map covisibility with the first—exactly the conclusion of Figure 22-2 in Chapter 22: error can be redistributed only through the connectivity of the graph. Multiple maps that are never merged are merely data put into storage.

Why Merging Is So Much Harder Than Keeping

Saving the data after a loss of tracking as a new map is easy enough in engineering—just open a new container. What is really difficult is the merge: the two maps have different coordinate frames, their scales may differ (in monocular), and the merge must not block the tracking thread that is running.

That is the reason for the two-stage strategy of Section 23.6: fuse locally within the welding window first (a small range, completable in real time), then propagate the correction outward along the pose graph (a large range, done in the background). Again, pull straight quickly first and optimize finely afterward—the fourth appearance of this pattern in the book.

23.7 Visual-Inertial SLAM on the GoPro, the Phone, and the Drone

Visual-inertial SLAM is no longer merely a research topic; it is running in the everyday devices of you and me. This section describes several representative applications.

Phones and AR / VR

Section 20.7 mentioned AR's inside-out tracking, which technically is nothing other than visual-inertial SLAM. A phone's camera together with its built-in IMU (Chapter 7) makes a perfect visual-inertial combination: the camera supplies a reference to the

environment that does not drift, and the IMU supplies high-rate motion and robustness when the head turns quickly.

One particularly persuasive figure in the ORB-SLAM3 paper bears on applications of this kind. On the fisheye TUM-VI benchmark it performs better overall than VINS-Mono and Basalt; in the room sequences, which resemble AR / VR use, the mean absolute trajectory error is 0.009 meters in stereo-inertial mode and 0.011 meters in monocular-inertial.

Millimeter-level accuracy—precisely the level needed to pin a virtual object stably to a real tabletop. Students may consider: that AR ruler on your phone, able to measure the length of a table, rests on technology of this grade.

Wearable Cameras and Action Photography

An action camera (a GoPro, say) is worn on a helmet or on the chest and records while cycling, running, or climbing. Video of this kind is extremely challenging for SLAM: violent vibration, quick turns, motion blur, and strong changes of illumination (in and out of tree shade or a tunnel).

Pure visual methods are all but bound to lose tracking on such sequences, while visual-inertial SLAM, with the IMU holding out for several seconds when vision fails (the third contribution in Section 7.6), can maintain a continuous trajectory. Typical uses of such applications include the three-dimensional reconstruction of a sporting route, the analysis of trajectories in extreme sports, and the backpack spatial surveying mentioned in Part V.

Drones

The drone is one of the most important applications of visual-inertial SLAM and

the theme of Chapter 26. Its particular requirements include:

- Extremely constrained weight and power: a camera plus an IMU is the only workable combination (Section 9.5).
- Fast motion in six degrees of freedom: hard for pure vision to cope with, and the IMU is indispensable.
- GPS-denied environments: indoors, under a bridge, in a tunnel, visual-inertial is the only means of localization (Case 5-3 of Chapter 5).
- Stringent real-time requirements: a delay in localization bears directly on the stability and the safety of flight.

The EuRoC dataset mentioned in Case 7-3 of Chapter 7 was recorded with a micro air vehicle, and ORB-SLAM3's excellent performance on it (the 3.5-centimeter accuracy of Section 7.7) shows its practical value in drone applications.

Robots and the Campus Site

For readers of this book, several concrete project directions are worth considering:

- A vision-navigated robot: build a campus map with stereo-inertial ORB-SLAM3, then combine it with the visual navigation of Chapter 24 and the semantic understanding of Chapter 32.
- Multi-session mapping: use Atlas's multi-session capability to map a whole department building over several runs and verify the effect of map merging.
- A recovery-from-loss experiment: design an experimental procedure that loses tracking deliberately, and quantify Atlas's advantage over ORB-SLAM2 in the ability to recover—an excellent topic for a comparative study.

- A comparison with LiDAR SLAM: run ORB-SLAM3 (visual-inertial) and the LIO system of Chapter 18 (LiDAR-inertial) in the same site and compare their performance in different environments (a bright corridor, a dim machine room, a glass-curtained lobby). This comparison lets students feel deeply the complementarity of sensors stressed repeatedly in Chapters 8 and 9.

At this point the core systems of Part VI have been introduced in full. In the next chapter we widen our view from localization and mapping to navigation and application—how does a vision-navigated robot use these capabilities to carry out a task? Line following, visual servoing, AprilTag navigation, YOLO semantic navigation, and even VLM-assisted navigation will connect the technology of this book to real robotic tasks.

23.8 After Visual-Inertial: Two Directions in 2024–2026

ORB-SLAM3 was published in 2021 and is to this day the standard object of comparison in visual-inertial SLAM. Progress in the past few years has run mainly in two directions, and they are nearly opposite.

One: Downward, Squeezing onto a Milliwatt-Class Chip

The first direction shrinks visual-inertial odometry to the limit. A work of 2026 deploys a complete monocular visual-inertial pipeline (ORB features, the eight-point algorithm and EPnP with RANSAC, IMU preintegration, sliding-window pose graph optimization) onto a RISC-V ultra-low-power chip clocked at only 370 MHz.

The result is 20 frames a second, under 100 milliwatts of power, and less than 1 MB of memory (about 110 KB in the first level and about 850 KB in the second). Put these figures together and they are quite striking—a complete visual-inertial SLAM running

on a chip two orders of magnitude weaker than a phone.

But the price is stated honestly as well. That system does without loop closure detection, and on the standard EuRoC dataset its mean trajectory error is about 3.46 meters against ORB-SLAM3's 0.25 or so—an order of magnitude apart. The authors themselves point out that the reason is precisely the lack of global correction. This comparison belongs at the end of this chapter: it shows that loop closure and global optimization are not a flourish but the sole source of long-term consistency. The milliwatt-class platforms of Chapter 25 return to this theme.

Two: Upward, Letting Learned Modules Take Over the Front End

The second direction hands certain parts of the front end to neural networks: learned features and matching in place of ORB, a learned optical flow consistency check in place of purely geometric outlier rejection, or learned line features added in scenes of violently changing illumination.

These works have one thing in common worth the student's notice: what they replace is almost always the layer of how correspondences are found, while IMU preintegration, tightly coupled optimization, key frames, and map management—the whole of Sections 23.3 through 23.6 of this chapter—are mostly left untouched. This is exactly the phenomenon observed in Section 22.8 of Chapter 22.

	ORB-SLAM3 (this chapter)	Milliwatt-class VIO	VIO with a learned front end
Front end	ORB + RANSAC	ORB + RANSAC (pared down)	Learned features / optical flow
IMU fusion	Tightly coupled,	Tightly coupled	Tightly coupled

	with bias estimation	(sliding window)	
Loop closure and multiple maps	Yes	No	Mostly retained
Typical hardware	A desktop machine or a Jetson	RISC-V, 370 MHz	A GPU is required
Power	A few watts to tens of watts	Under 100 milliwatts	Tens of watts
Mean EuRoC error	About 0.25 meters	About 3.46 meters	Comparable to ORB-SLAM3
Where it applies	General robots and AR	Micro drones, wearables	Scenes of violently changing illumination and weak texture

Table 23-2 A Comparison of the Three Routes

From this table one may read a judgment quite useful in choosing a project: if your platform has a power budget of a few watts, ORB-SLAM3 remains the least troublesome and most reliable choice; if you are working on a gram-scale miniature platform, what you need is the technology of Chapter 25, and you must accept the consequences of having no loop closure; and if the lighting conditions of your site are poor, a learned front end is worth trying, though the system architecture remains the one in this chapter.

Formulas and Procedures

One: the State Vector of Visual-Inertial SLAM

In visual-inertial mode, the state to be estimated at each key frame i is

$$S_i = \{ R_i , p_i , v_i , b_g_i , b_a_i \}$$

where R_i is rotation and p_i position (6 degrees of freedom together), v_i velocity (3), and b_g and b_a the gyroscope and accelerometer biases (3 each), for 15 degrees of freedom in all—velocity and bias beyond the 6 of pure vision. Monocular-inertial mode must additionally estimate the scale s and the direction of gravity R_wg during initialization.

Two: the Joint Objective of Tight Coupling

ORB-SLAM3's visual-inertial bundle adjustment minimizes errors of both kinds, visual and inertial, at once:

$$min \Sigma \rho(|| e_visual ||^2_\Sigma) + \Sigma || e_inertial ||^2_\Sigma + || e_prior ||^2$$

- The visual error e_visual: the reprojection error (Chapters 21 and 22), requiring the projected position of a map point to agree with the actual observation.
- The inertial error e_inertial: the IMU preintegration residual (Section 7.5), requiring the changes of pose and velocity between neighboring key frames to agree with the IMU measurements.
- The prior error e_prior: retains the information accumulated by earlier optimizations.

ρ is a robust kernel. This expression is the concrete form of tight coupling in Section 7.6, and another application of the least squares framework of Chapter 12—by now students should be able to see that the mathematics of this whole book is highly unified.

Three: the Three Stages of IMU Initialization

Figure 23-5 The IMU Initialization Procedure

① Vision-only MAP estimation

Build a short stretch of map and trajectory with the camera alone (monocular scale unknown)

↓ supplies the motion reference

② Inertial-only MAP estimation (the visual trajectory held fixed)

Estimate: the scale s, the direction of gravity, the gyroscope bias, the accelerometer bias, and the velocity at each frame

↓ obtains initial values for the inertial parameters

③ Joint visual-inertial bundle adjustment

All variables (poses, velocities, biases, scale, gravity, map points) refined together

↓

Complete: a scale error of about 5 percent within 2 seconds, about 1 percent after refinement

【Precondition】there must be sufficient excitation of motion, or scale and bias are unobservable

Four: the Unified Flow of Loop Closure and Map Merging

Figure 23-6 The Two Outcomes Place Recognition May Trigger

The mapping thread creates a new key frame

↓

Place recognition: query the DBoW2 database for similar key frames in Atlas (high recall)

↓

Geometric verification: multi-step verification + Sim(3) / SE(3) alignment (RANSAC + Horn's method)

↓ (seeking 100 percent precision)

Which map does the matched frame belong to?

├─ The active map → 【loop closure】pose graph optimization + global BA (Chapter 22)

└─ Another map → 【map merging】fusion in the welding window + propagation by pose graph

Cases

Case 23-1 Losing Tracking Is No Longer the End: Atlas in Practice

This is the most dramatic experiment for contrasting ORB-SLAM2 with ORB-SLAM3, and demonstrating the two side by side in class is strongly recommended.

The design of the experiment is simple: run both systems on the same video, a video containing in the middle a severe visual failure—the camera completely covered for ten seconds, say, or swung quickly across a great expanse of white wall (the curse of the white wall of Chapter 9).

ORB-SLAM2's behavior: after losing tracking it attempts to relocalize, fails because it has strayed too far from the original region, and comes to a standstill. The display freezes and the rest of the video goes entirely unused—even if splendid scenes follow, all of it is wasted.

ORB-SLAM3's behavior: after losing tracking it tries briefly to relocalize, and on failing it opens a new map at once and goes on mapping and tracking normally, the user hardly noticing the interruption. And when the camera later circles back to the old region, place recognition detects the cross-map association and the two maps merge seamlessly—the screen showing the striking picture of two trajectories, once separate, suddenly joined.

This contrast lets students understand intuitively the value of robustness in a real system. On a clean dataset in the laboratory the difference in accuracy between the two may be only a number; but in challenging scenes in the real world, the difference is between usable and not. It also echoes the observation about Cartographer in Chapter 15: in a productized system, robustness often matters more than the last word in accuracy.

Case 23-2 The Value of Two Seconds: Why the Speed of Initialization Is Decisive

IMU initialization shortening from fifteen seconds to two sounds like being merely a little faster. But those thirteen seconds are decisive in practice.

Imagine a user opening an AR application on a phone. If the phone must be waved about for fifteen seconds before it can be used, the great majority of people will close the app by the fifth second—such a product could never be brought to market. A two-

second initialization, by contrast, finishes while the user is still adjusting their grip, all but imperceptibly.

The same holds for robots. A drone that must be shaken about on the ground for fifteen seconds before takeoff not only wastes time but is simply impracticable in some situations (an emergency rescue, say); a two-second initialization can be completed incidentally while taxiing for takeoff.

Looked at more deeply, fast initialization brings an indirect benefit as well: when the system loses tracking and must initialize afresh on a new map (Section 23.5), fast initialization means a very short interruption. Were initialization to take fifteen seconds, Atlas's seamless switch would not deserve the name. This shows how the various techniques in a system design support one another—the speed of initialization, the multiple-map architecture, and data association together constitute ORB-SLAM3's robustness.

The lesson for students is this: in assessing a technical improvement, do not look only at the change in a single number but ask what this change makes possible that was impossible before—that is the true measure of technical value.

Case 23-3 From PTAM to ORB-SLAM3: Twenty Years of One Bloodline

ORB-SLAM3 did not appear out of nowhere; it is the crystallization of twenty years of evolution in the technical bloodline of feature-based visual SLAM. It is the culmination of that bloodline, which began with PTAM and ORB-SLAM, and on its release it was the most complete and most accurate open-source SLAM library then available.

- PTAM (2007): first created the architecture of separating tracking and mapping into two threads, making real-time visual SLAM possible. That two-thread idea is the basis of every later system.
- ORB-SLAM (2015): unified the three tasks of tracking, mapping, and place recognition with the ORB feature (Section 22.1), added a loop closing thread, and introduced the covisibility graph and the essential graph (Section 22.6).
- ORB-SLAM2 (2017): extended to stereo and RGB-D and became the most widely used open-source visual SLAM (Chapter 22).
- ORB-SLAM3 (2021): added tightly coupled visual-inertial estimation and the Atlas multiple-map system, and used all four kinds of data association throughout.

Watching this bloodline, students can learn an important research methodology: no generation overturns the one before it; each keeps the architecture that works and makes its advance against a clearly identified weakness. PTAM's two threads, ORB-SLAM's covisibility graph and bag of words, ORB-SLAM2's support for several cameras—all of these are still present in ORB-SLAM3.

This also gives students bent on research a practical piece of advice: rather than pursuing the invention of something wholly new, understand one mature system deeply, identify its definite weakness, and propose a targeted improvement. ORB-SLAM3 could propose Atlas precisely because its authors knew intimately the pain of ORB-SLAM2's losing tracking is the end. Understand a limitation, and only then can you break through it.

Hands-On Activities

Activity 23-1 Implementing Visual-Inertial SLAM with ORB-SLAM3

Purpose: to run the several modes of ORB-SLAM3, to experience at first hand the difference the IMU makes and Atlas's ability to recover from a loss of tracking, and to complete a comparison with ORB-SLAM2. This is the concluding implementation of Part VI and an important foundation for Chapters 30 and 31.

Part One: Setting Up the Environment and Basic Operation (everyone)

1. Compile ORB-SLAM3 (the dependencies are similar to those of Chapter 22: OpenCV, Eigen3, Pangolin; mind the question of version compatibility).
2. Download a sequence of the EuRoC dataset (MH_01_easy is recommended to begin with), run it first in pure stereo mode, and record the trajectory and the map.
3. Run the same sequence in stereo-inertial mode and compare the two in trajectory, number of map points, and stability of the system.
4. Observe initialization: in monocular-inertial mode, record the time and the number of image frames from start-up to the completion of IMU initialization, against the two seconds of Section 23.4.

Part Two: the Value of the IMU (everyone, or university level)

5. A challenging sequence: choose a difficult sequence of fast motion (EuRoC's V1_03_difficult or MH_05_difficult, say) and run it in pure visual and in visual-inertial mode in turn, comparing whether and how often tracking is lost.
6. A scale check: run once in monocular and once in monocular-inertial mode; in evaluating with evo the former requires a scale alignment and the latter does not—verifying at first hand the value of the IMU's absolute scale (Section 20.5).

7. An excitation experiment: if using your own camera and IMU, try three ways of starting up—standing still, moving at constant velocity in a straight line, and moving with accelerations and rotations—and observe which succeeds in initializing, verifying the argument of Section 23.4.

Part Three: the Atlas Multiple-Map Experiment (the core; university level)

8. Cause a loss of tracking deliberately: while running a dataset, interrupt the image input by program for a few seconds (or choose a sequence with a severe occlusion) and observe whether the system opens a new map.
9. Observe the state of Atlas: from the terminal output or the visualization, confirm how many maps there are at present and which is active.
10. Map merging: let the camera return to the region covered by the old map, observe the instant the merge occurs, and record it on video—the picture most worth demonstrating in this chapter.
11. A comparative experiment: run ORB-SLAM2 and ORB-SLAM3 in turn on the same sequence containing a severe loss of tracking and compare the completeness of the final trajectory, reproducing Case 23-1.
12. A multi-session experiment: using several sequences of the same EuRoC scene (MH_01 through MH_05, say), run them in order and let the system merge them, observing how multi-session operation extends the map by degrees.

Discussion Questions

1. By how much do the accuracies (ATE) of pure stereo and stereo-inertial differ? On which kind of sequence is the gap greatest? Why?

2. On a difficult sequence, how many times did pure vision lose tracking? How much did adding the IMU improve matters? Against Section 7.6, which contribution of the IMU came into play here?
3. How long did IMU initialization take in your experiment? What happens if the body is held still during initialization? Which argument of Section 23.4 does this bear out?
4. What changes in the trajectory and the map when a merge occurs? How does this resemble and differ from the loop closure correction of Chapter 22? (Hint: the unified flow of Section 23.6.)
5. (University level) By the four kinds of data association of Section 23.2, which kind is the principal source of ORB-SLAM3's advantage in accuracy over VINS-Mono? Why does it matter so much?

Extended Challenge (university level)

First: a field test on campus—record data in the corridors and lobby of your department building with a stereo-inertial camera of your own (or a depth camera containing an IMU) and run ORB-SLAM3 to build a map. Design a route deliberately that passes through a dim machine room to cause a loss of tracking and then returns to a bright corridor, test Atlas's ability to merge, and assess the quality of the map by the method of Section 13.7. Second: a cross-modal comparison—record visual-inertial and LiDAR-inertial data in the same site at the same time, run ORB-SLAM3 and the LIO system of Chapter 18 respectively, and compare their performance in three environments (a bright corridor, a dim machine room, a glass-curtained lobby), verifying quantitatively the argument for the complementarity of sensors in Chapters 8 and 9. This topic has both teaching and research value and suits being the spine of a project.

Further Reading

1. C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE T-RO, 2021. (The principal source of this chapter.)
2. R. Elvira, J. D. Tardós, and J. M. M. Montiel, "ORBSLAM-Atlas: A Robust and Accurate Multi-Map System," IROS, 2019. (The precursor work on the Atlas multiple-map architecture.)
3. C. Campos, J. M. M. Montiel, and J. D. Tardós, "Inertial-Only Optimization for Visual-Inertial Initialization," ICRA, 2020. (The theoretical source of the three-stage IMU initialization procedure.)
4. C. Forster et al., "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," IEEE T-RO, 2017. (IMU preintegration, the theoretical basis of Section 7.5.)
5. The ORB-SLAM3 open-source project (GitHub: UZ-SLAMLab/ORB_SLAM3). (Source code, compilation instructions, and examples of running each mode.)
6. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE T-RO, 2018. (Another classic visual-inertial system, to be read alongside this chapter.)
7. The official websites of the EuRoC MAV and TUM-VI datasets. (The standard benchmarks used in this chapter's experiments and in the papers' evaluations.)

Summary

1. ORB-SLAM3 has three core advances: tightly coupled visual-inertial SLAM founded on maximum a posteriori estimation, the Atlas multiple-map system, and the thoroughgoing use of four kinds of data association at every

algorithmic stage; it supports monocular, stereo, and RGB-D cameras with pinhole and fisheye lenses in many configurations.

2. The four kinds of data association are short-term (the last few seconds), mid-term (elements nearby with slight drift), long-term (matching a previously visited region by place recognition), and multi-map (across mapping sessions); mid-term association is the key source of its advantage in accuracy, making it 2.6 times as accurate as VINS-Mono in single-session monocular-inertial operation on EuRoC.

3. In visual-inertial mode the state vector grows from the 6 degrees of freedom of pure vision to 15 (adding velocity and two biases); the IMU brings absolute scale, the direction of gravity, support when vision fails, and high-rate output, while vision repays it by correcting the IMU bias with observations that do not drift.

4. IMU initialization proceeds in three stages: vision-only MAP estimation → inertial-only MAP estimation with the visual trajectory held fixed → a joint visual-inertial BA. It reaches a 5 percent scale error within 2 seconds and about 1 percent after refinement, but there must be sufficient excitation of motion, or scale and bias are unobservable.

5. Atlas consists of one active map and several inactive maps sharing a bag-of-words key frame database; when tracking is lost it opens a new map and carries on, and performs a seamless merge when a cross-map association is later detected—unlike other schemes, which keep their submaps as separate entities.

6. Place recognition queries Atlas for similar frames for every new key frame to achieve high recall, then ensures a hundred percent precision with several steps

of geometric verification and a Sim(3) / SE(3) alignment; a match in the active map means a loop closure, otherwise a merge. The merge proceeds in two stages, fusion within a welding window and propagation by pose graph, so as not to block tracking.

7. Visual-inertial SLAM is widely applied in phone AR, wearable cameras, and drones; in the TUM-VI room sequences, which resemble AR / VR use, the mean ATE is 0.009 meters in stereo-inertial mode and 0.011 meters in monocular-inertial, which is millimeter-level accuracy.

Carrying Forward

This chapter has joined the IMU of Chapter 7 to the system architecture of Chapter 22, and measured three sets of numbers along the way: for the IMU to carry across a gap in vision the key is the bias—with the bias estimated to ±0.01 the reckoning error after two seconds is 0.025 meters, and with it estimated to ±0.3 it is 1.10 meters, forty times greater; under constant-velocity straight-line motion the condition number of initialization reaches 7,673 at a scale error of 100 percent, while full excitation brings the condition number down to 44 and the error to 0.37 percent; and after a loss of tracking, Atlas's merge presses the ATE from 0.787 meters unmerged down to 0.026 while covering 90 percent of the trajectory.

The core systems of Part VI have now been introduced in full. But SLAM is not an end in itself—once a robot has built a map and knows where it is, what is it to do with that? In the next chapter we widen our view from localization and mapping to navigation and tasks: how does visual navigation differ from SLAM? How do line following, visual servoing, and navigation by artificial markers work? How do object

detection and vision-language models let a robot understand an instruction such as please go to the registrar's office? And how does a robot dog or a campus inspection robot integrate all of this? Chapter 24 will connect the technology of this book to real robotic tasks.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 24

The Vision-Navigated Robot

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

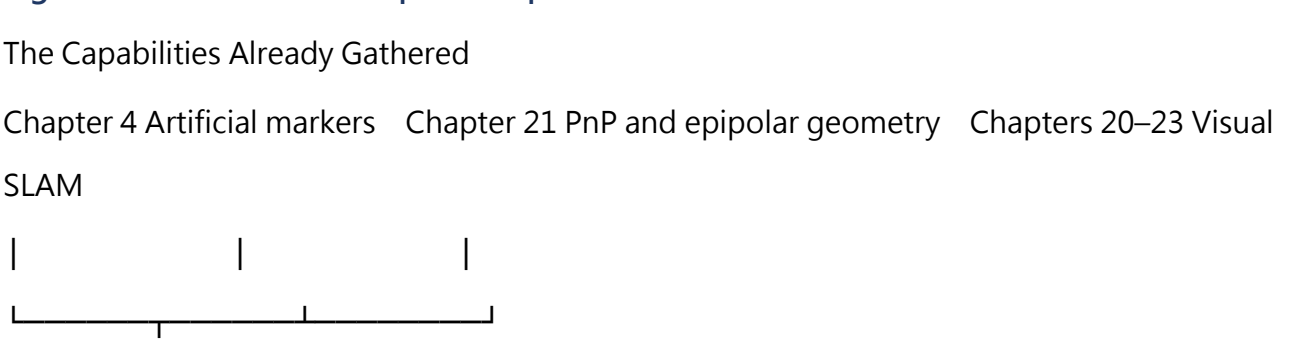
Where This Chapter Sits in the Book

Chapters 20 through 23 built visual SLAM all the way from features to a complete system. This chapter asks the next question: once a robot knows where it is, what is it to do with that?

The character of this chapter is quite unlike that of the four before it. Those four pursued accuracy and robustness; this one is about trade-offs—many navigation tasks do not need SLAM at all. Laying down a line, putting up a few markers, or simply doing visual servoing straight at the target is cheaper and more reliable than full SLAM in the right setting. The point of engineering judgment is not to use the strongest technology but to use just enough of it.

The three figures of this chapter all quantify these trade-offs: Figure 24-1 compares the convergence behavior of two kinds of visual servoing and explains why precise docking must choose the right one; Figure 24-2 measures how the pose accuracy of a marker varies with distance and viewing angle; and Figure 24-3 answers a very practical question—how far apart should markers be put up?

Figure 24-0 The Link Map of Chapter 24



navigation

▼

Part VII Miniature robots and drones Part VIII ROS implementation

▼

Chapter 32 Semantic SLAM Chapter 34 Language navigation and embodied intelligence

Learning Objectives

In the last four chapters we established a complete capability in visual SLAM: the robot can know where it is and what the environment looks like. But SLAM is not an end in itself—once a map has been built, what is it to be used for? This chapter, the last of Part VI, connects the technology to real robotic tasks, from the plainest line following all the way to VLM navigation that can understand human speech. After studying this chapter, students should be able to:

1. Explain how visual navigation differs from and complements visual SLAM.
2. Compare the three basic strategies of visual navigation: line following, landmark navigation, and visual servoing.
3. Describe four typical uses of AprilTag / ArUco in robot navigation.
4. Explain how YOLO object recognition supports semantic navigation, and understand its limitations from a robot's point of view.
5. Explain how a VLM vision-language model lets a robot understand a natural-language navigation instruction.
6. Describe the particular challenges of visual navigation on a robot dog and the measures taken against them.
7. Plan the system architecture of a campus inspection robot or a recycling-sorting robot.

Core Concepts

Chinese Term	English Term	Explanation
視覺導航	Visual Navigation	The technology of guiding a robot to a goal by visual information.
線追蹤	Line Following	The most basic form of visual navigation, traveling along a line on the floor.
視覺伺服	Visual Servoing	A method of controlling a robot's motion directly by image feedback.
影像式視覺伺服	IBVS	Takes the feature error in the image directly as the control signal.
位置式視覺伺服	PBVS	Solves for the three-dimensional pose error first and then controls.
地標導航	Landmark Navigation	Deciding the direction of travel from recognizable landmarks.
語意導航	Semantic Navigation	Navigating by the meaning of objects and scenes rather than geometric coordinates.
物件目標導航	Object-Goal Navigation	Accepting a task instruction of the form go find such-and-such an object.
視覺語言導航	VLN	Navigating an environment by natural-language instruction.
視覺語言模型	VLM	A multimodal AI model able to understand image and language at once.
視覺語言動作模型	VLA	A model that generates action commands directly from image and language.
邊界探索	Frontier Exploration	Advancing toward the boundary between the known and the unknown to explore an environment.
零樣本	Zero-Shot	The ability to perform a new task without task-specific training.

24.1 How Visual Navigation Differs from SLAM

This is the first idea that must be made clear in this chapter. Many students conflate visual SLAM with visual navigation, but the two answer different questions.

Two Different Questions

- Visual SLAM answers: where am I, and what does the environment look like? What it produces is a pose and a map—the theme of Chapters 20 through 23.
- Visual navigation answers: which way should I go? What it produces is a motion command—the theme of this chapter.

Set against the three questions of navigation in Chapter 3: SLAM addresses the first (where am I), while visual navigation covers the second and the third (where am I going, and how do I get there).

Visual Navigation Does Not Necessarily Need SLAM

This is an important fact that is often overlooked. Some methods of visual navigation build no map at all:

- A line-following robot looks only at the line on the floor; it does not know where it is and does not need to (Section 24.2).
- Visual servoing turns the offset of the target in the image directly into a motor command, passing through no map at all.
- A corridor robot following visual landmarks need only recognize that on seeing the fire extinguisher, turn right; it needs no global coordinates.

These methods belong to the reactive architecture introduced in Chapter 3—perception drives behavior directly, with no model and no planning. They are simple, fast, and robust, and very practical in a controlled environment.

SLAM Raises the Level of Navigation

But reactive navigation has a clear ceiling: it cannot plan a global path, cannot handle going to a place no line leads to, and cannot change its goal midway. With SLAM added, a robot gains:

1. Global path planning: computing the best route on a map (Chapters 3 and 29).
2. Navigation to any goal: able to go to any coordinate on the map, not only along a preset route.
3. Recovery from a break: able to relocalize after losing tracking and carry on with the task (Chapters 22 and 23).
4. Semantic tasks: combined with a semantic map, able to understand an instruction such as go to the registrar's office (Section 24.5).

Item of Comparison	Visual SLAM	Visual navigation
The central question	Where am I? What does the environment look like?	Which way should I go?
Output	Pose and map	Motion commands (speed, direction)
Is a map needed	It produces a map	Not necessarily
Type of architecture	Deliberative (a model is required)	May be reactive or deliberative
Which of the three questions of navigation	The first	The second and third
Chapters of	Chapters 19–22	This chapter and Chapter 29

Item of Comparison	Visual SLAM	Visual navigation
this book		

Table 24-1 Visual SLAM Compared with Visual Navigation

Working systems mostly use both: SLAM supplies global localization and planning while reactive visual navigation handles real-time obstacle avoidance and precise docking. This is exactly the hybrid architecture of Chapter 3.

24.2 Line Following, Landmark Navigation, and Visual Servoing

This section introduces three basic strategies of visual navigation that require no full SLAM, from the simple to the elaborate.

One: Line Following

This is the plainest robot, and the first that countless students ever build: lay a strip of black tape on the floor, have the robot detect the position of the line with a downward camera or an infrared sensor, and turn left if the line is to the left and right if it is to the right.

- The principle: convert the image to grayscale or binarize it (the basic skill of image processing), find the offset between the center of the line and the center of the frame, and turn that into a steering command.
- Advantages: extremely low cost, extremely simple code, extremely robust, and no need for a map or localization.
- Limitations: it can only follow a preset route, cannot go around an obstacle, and the environment must be modified (the line must be laid).

- In practice: guidance by magnetic strip or colored tape is still widely used for factory AGVs—in a controlled environment it is irreplaceably cheap and reliable (echoing the idea of the Amazon warehouse case in Chapter 4: the more controlled the environment, the simpler the choice of technology may be).

Two: Landmark Navigation

One step beyond line following: instead of laying a continuous line, place recognizable landmarks at key positions, and the robot turns right on seeing landmark A and stops on seeing landmark B.

A landmark may be artificial (the AprilTag, QR code, or color patch of Section 4.6) or natural (a door, a fire extinguisher, a fire hydrant). The former is reliable but requires modifying the environment, the latter requires no modification but is harder to recognize—which is where AI recognition technologies such as YOLO come into their own (Section 24.4).

Landmark navigation in essence establishes a topological map (Chapter 3): the nodes are the landmarks and the edges are the actions of traveling from one landmark to the next. It requires no precise coordinates yet can accomplish quite complicated route tasks.

Three: Visual Servoing

Visual servoing is the most precise of the three, and it puts the camera directly into the control loop: it continually compares the difference between the current image and a target image and generates motion commands to reduce that difference until the two agree.

It has two great schools:

- Image-based visual servoing (IBVS): takes the feature error in the image directly as the control signal (the target point should be at the center of the frame and is now 30 pixels to the left, say). Its advantages are that no three-dimensional reconstruction is required and that it is insensitive to errors in camera calibration; its drawback is that the robot's actual trajectory may be counterintuitive.
- Position-based visual servoing (PBVS): first solves from the image for the three-dimensional pose error of the camera relative to the target (using the PnP of Chapter 21) and plans its motion accordingly. Its advantage is that the trajectory is sensible in three-dimensional space; its drawback is that accurate camera calibration and a model of the target are required.

The typical application of visual servoing is precise docking: a robot arm grasping an object, a cleaning robot aligning itself to its dock, a drone landing precisely. It can reach millimeter-level relative positioning accuracy—far above the global localization accuracy of SLAM, because it cares only about position relative to the target in front of it.

Method	Requires modifying the environment	Accuracy	Flexibility	Typical applications
Line following	Yes (a line must be laid)	Medium	Low (a fixed route)	Factory AGVs, teaching robots
Landmark navigation	Depends on the landmarks	Medium	Medium (a topological route)	Corridor patrol, exhibition guiding
Visual servoing	No	Extremely high (millimeter	Low (only toward the target)	Precise docking, grasping, landing

Method	Requires modifying the environment	Accuracy	Flexibility	Typical applications
		level)		

Table 24-2 A Comparison of the Three Basic Strategies of Visual Navigation

Measuring It: Where the Two Kinds of Visual Servoing Differ

Section 24.2 says that in image-based visual servoing the robot's actual trajectory may be counterintuitive while in the position-based kind the trajectory is sensible in three-dimensional space. Those two statements sound like a difference of style; they are in fact a substantive difference that decides success or failure. This subsection computes it.

The experimental setting: the camera is to move from a displaced starting pose to a position facing the target squarely at a distance of 0.65 meters. The target is the four corners of a square 20 centimeters on a side. Both control laws take their standard form—the image-based one multiplies the feature error by the pseudoinverse of the image Jacobian, and the position-based one solves for the three-dimensional pose error first and commands a velocity directly. The key variable is how many degrees the camera is rotated about its optical axis at the start.

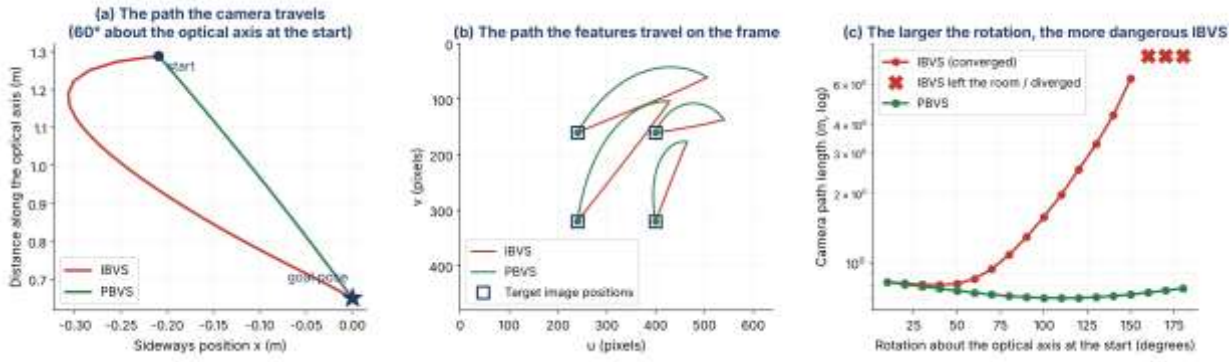


Figure 24-1 The Convergence Behavior of Image-Based and Position-Based Visual

Servoing

Panel (a) is the path the camera travels in space, starting rotated 60 degrees about the optical axis. The position-based control (green) runs almost straight from the starting pose to the goal, 0.74 meters in all; the image-based control (red) first swings further sideways, out to 0.31 meters off the axis of the goal pose when it started only 0.21 meters off it, and only then closes on the target, 0.85 meters in all. This swinging away before closing in is the same phenomenon as the famous camera retreat of image-based visual servoing: the control is shortening the error on the frame, not the error in space, so the camera is free to take a strange route through the room.

Panel (b) is the same process as it appears on the image plane, and here the verdict is entirely reversed. The four feature tracks of the image-based control are very nearly straight—which is exactly its design aim: it minimizes error in the image, so the features move along the shortest path on the frame. The feature tracks of the position-based control bend plainly, because it cares only about three-dimensional pose and not how the features travel across the frame.

This contrast shows the nature of the two: the image-based kind guarantees good behavior on the frame (the features are unlikely to leave the field of view) while the camera's actual motion may be strange; the position-based kind guarantees good motion of the camera while the features may leave the frame along the way. Which to choose depends on what your failure mode is—an arm working in a confined space that might collide should take the position-based kind; a target occupying only a small part of the frame that might be lost should take the image-based kind.

Panel (c) sweeps the starting rotation from 10 degrees to 180. The position-based path length stays between 0.70 and 0.82 meters throughout, very nearly the straight-line

distance from start to finish, and it converges every time. The image-based control is equally fine within 50 degrees (about 0.80 meters), beyond which the path swells rapidly—0.85 meters at 60 degrees, 1.09 at 80, 1.30 at 90, 2.55 at 120, 4.41 at 140 and 6.37 at 150—and beyond that it no longer converges at all: at 160, 170 and 180 degrees the camera retreats clean out of the room.

The Famous Counterexample at 180 Degrees

The best-known failure of image-based visual servoing is precisely the target being rotated exactly 180 degrees relative to the camera. The pseudoinverse of the image Jacobian then commands the camera to retreat straight back along the optical axis—because as far as the image is concerned, the farther back it goes the nearer the four corners come to the center of the frame, so the error appears to be decreasing. The result is that the camera retreats toward infinity while the error never comes down.

This experiment reproduces the phenomenon above 150 degrees: path lengths run to millions of meters, which is not the robot really traveling that far but the control law having lost control.

There are three remedies in practice: switched control (position-based at a distance, image-based close in), hybrid visual servoing (image-based for translation, position-based for rotation), or the simplest of all—align the attitude roughly by some other means first and let visual servoing finish the job.

24.3 Navigation with AprilTag / ArUco

Section 4.6 introduced the principle of visual fiducial markers—a complete 6-DoF

pose solved from four corners and the camera intrinsics (the PnP problem of Chapter 21). This section focuses on how they are actually used in navigation tasks.

Four Typical Uses

5. Absolute pose correction: put markers in corridors, at elevator doors, and at laboratory entrances. The moment the robot sees any marker it obtains an absolute pose that does not drift, eliminating SLAM's accumulated error at a stroke. This is the commonest and most practical use (the idea of fusion in Section 4.7).
6. Precise docking: put markers at the charging station, at pick-and-place points, and in front of elevator buttons, and complete the final alignment to the millimeter with visual servoing (Section 24.2). SLAM's localization accuracy is usually a few centimeters, not enough for an action requiring precise docking.
7. Identifying floors and zones: as Section 5.7 mentioned, LiDAR SLAM cannot tell apart different floors that look alike. Put markers of different ID at the elevator door of each floor and the problem is solved.
8. Evaluating SLAM accuracy: markers of known coordinates placed densely about a site can serve as an approximate ground truth trajectory for evaluating the accuracy of a SLAM system (the ATE evaluation of Section 22.7 requires a ground truth). This is extremely practical for a teaching laboratory with no motion capture system.

Points of Implementation

- Marker size and detection distance: the detection distance is roughly proportional to the side length of the marker. A corridor marker that is too small must be approached closely before it can be seen at all, which defeats the

purpose of correction. In practice markers of different sizes are made as required.

- Mounting height and angle: a marker should be put up at a sensible height within the robot camera's field of view and should face the passage as squarely as possible. At too oblique an angle the detection accuracy of the four corners falls and the error of the solved pose grows.
- Light and reflection: a marker should avoid direct reflection (a laminated marker may be entirely unrecognizable under a lamp). Printing on a matte material is recommended.
- Managing uniqueness: the ID of every marker and its world coordinates must be filed and managed. This marker map is itself a simple kind of semantic map (Chapter 3).

Levels of Fusion with SLAM

Section 4.7 mentioned three levels of fusion; here they are made concrete:

- Loose coupling: SLAM runs normally and the pose is simply pulled to the value the marker gives whenever a marker is seen. The simplest to implement and suitable for teaching.
- Tight coupling: the marker observation is added as a constraint in SLAM's back-end optimization (an edge, in the sense of Chapter 12). The most accurate, and a research-grade practice.
- Aiding initialization and relocalization: the most practical of all—SLAM runs on its own ordinarily and markers supply an absolute reference only at start-up or after a loss of tracking. This solves the most troublesome problem at a very low cost.

Measuring It: How Large, How Oblique, How Densely to Put Markers Up

The points of implementation of Section 24.3 gave three principles: a marker too small cannot be seen far, accuracy falls at too oblique an angle, and uniqueness must be managed. The first two can be quantified, and quantifying them yields something more useful than a principle—numbers that can be written straight into a planning document.

The method of the experiment: take a marker 16 centimeters on a side, observe it with a camera of 520 pixels focal length at a resolution of 640×480, add localization noise to the four corners, solve PnP, and compare with the true pose. The model of the corner noise is this: the smaller the marker projects on the frame (through distance or the foreshortening of an oblique view), the greater the noise—which is closer to the behavior of a real detector than a fixed noise would be.

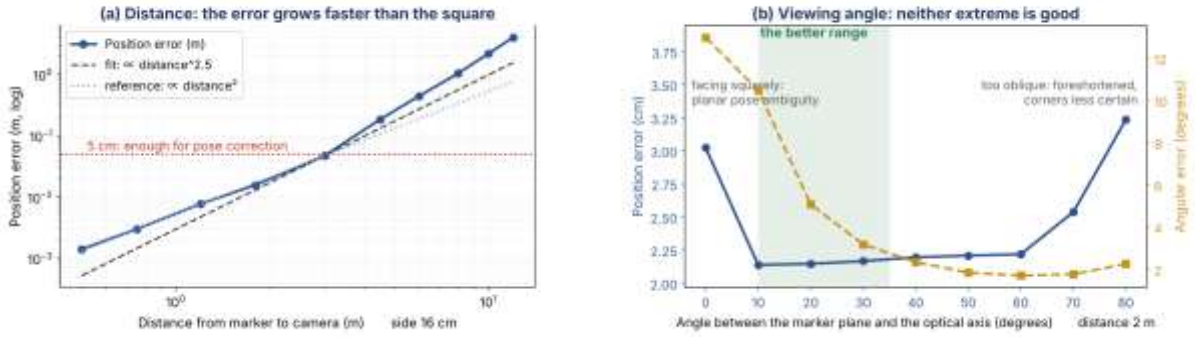


Figure 24-2 How the Pose Accuracy of a Marker Varies with Distance and Viewing Angle

Panel (a) is the effect of distance, with both axes logarithmic. At 0.5 meters the position error is only 1.4 millimeters, at 1.2 meters 7.7 millimeters, at 3 meters 4.8 centimeters, at 6 meters 0.44 meters, and at 12 meters 4.0 meters. The fitted exponent is 2.52—steeper than a simple inverse square. The extra half a power comes from corner detection itself: the smaller the marker on the frame, the harder the corner positions are to pin down, and the noise multiplies with the geometric magnification.

This curve yields a very practical design rule. Taking a position error of 5 centimeters as the threshold at which pose correction is worthwhile (SLAM's own localization error is usually a few centimeters too), the effective range of a 16-centimeter marker is about 3 meters. For a corridor where it must be usable at 6 meters, the side length must be doubled to about 32 centimeters. This is the concrete conversion behind the sentence the detection distance is roughly proportional to the side length of the marker in Section 24.3.

Panel (b) is the effect of viewing angle, and the result is more nuanced than the statement of Section 24.3. In principle the pose error grows at too oblique an angle is correct—at 80 degrees the position error is 3.2 centimeters, half as much again as the best value. But facing the marker squarely is no good either: at 0 degrees the angular error reaches 13.0 degrees, the worst point on the whole curve.

Facing squarely is poor because of the pose ambiguity of a planar target: when the marker faces the camera squarely, the four corners projected by tilting the marker a little to the left and a little to the right are nearly identical, and the direction of rotation is mathematically close to unobservable. Tilt it slightly and that symmetry is broken. Each end has its own trouble, and the optimum lies at about 10 to 35 degrees.

A Mounting Rule You Can Use Directly

Putting the two conclusions of Figure 24-2 together, the recommendation for mounting corridor markers is this: take the side length as the expected detection distance divided by 20 (16 centimeters for 3 meters, 32 centimeters for 6); and do not put it squarely on the wall straight ahead of the passage, but a little to one side, or let the robot pass obliquely, so that the viewing angle

falls between 10 and 35 degrees.

Neither of these costs anything, yet together they can make the same batch of markers several times more accurate. When planning a site, students would do well to run the experiment of Figure 24-2 once with their own camera—a different focal length and resolution will give different numbers.

Measuring It: How Far Apart Should Markers Be

What was measured above was the accuracy of a single marker; next comes the more practical question at the level of the system: how many markers does a patrol route need?

The experimental setting is a 90-meter U-shaped corridor patrol route of 360 key frames. Each step of the odometry carries noise, and an appreciable drift accumulates over the route. Markers are put up at a fixed interval, and the absolute pose measured from them carries noise (a standard deviation of 2 centimeters in position and 2 degrees in angle, inferred from the results of Figure 24-2). Three practices are compared: no markers at all, loose coupling (pull the pose over whenever a marker is seen), and tight coupling (take the marker observation as a constraint in the optimization and solve it together with the odometry).

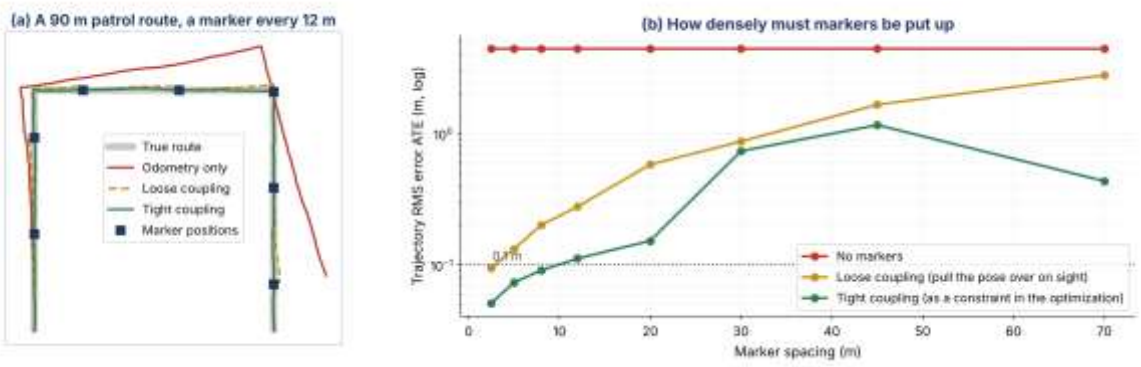


Figure 24-3 The Effect of Marker Spacing on Localization Accuracy

Panel (a) draws the actual trajectories with a marker every 12 meters. Pure odometry (red) has plainly departed from the route by the end of one circuit; loose coupling (the orange dashed line) is pulled back at every marker but drifts away again between them, forming a sawtooth trajectory; tight coupling (green) hugs the true route throughout.

Panel (b) sweeps the marker spacing. With no markers the ATE is fixed at 4.44 meters. Loose coupling gives 0.095 meters at a spacing of 2.5 meters, 0.132 at 5 meters, 0.278 at 12 meters, and 0.581 meters at 20. Tight coupling at the same spacings gives 0.051, 0.074, 0.112, and 0.153 meters—better at every point, and the greater the spacing the plainer the difference: at a spacing of 20 meters the error of tight coupling is only a quarter of that of loose coupling.

Why is the advantage of tight coupling greater when markers are sparse? Because loose coupling can correct only at the instant a marker is seen, and the correction cannot be carried back to the poses already traversed; tight coupling takes the marker as a constraint within the optimization, so the error is redistributed along the whole chain of poses. This is the same thing as the conclusion of Figure 22-2 in Chapter 22—a correction can spread only through the connectivity of the graph.

One detail must be pointed out honestly: at spacings of 45 and 70 meters only one

marker is left on the whole route, and the result is then governed by where that one marker happens to be rather than by the spacing itself. This is why the error at the 70-meter point in the figure is in fact lower than at 45. The figure can therefore be read directly only over the range in which the markers are numerous enough to form a chain.

Three Numbers for Planning a Site

Taking the setting of this experiment (a 90-meter route with odometry noise of a few centimeters):

To press the ATE within 10 centimeters, tight coupling requires a spacing of about 5 meters and loose coupling a spacing within 2.5—which is to say that adopting tight coupling halves the number of markers.

If 30 centimeters will do (enough for most inspection tasks), tight coupling may be relaxed to a spacing of about 20 meters, so that the whole route needs only four or five markers. This difference decides directly both the cost of deployment and the work of maintenance.

24.4 YOLO Object Recognition and Semantic Navigation

In the navigation of the three preceding sections the robot does not understand what it sees—a line is a line and a marker is a number. Only with object recognition added does a robot begin to understand the meaning of a scene.

From Geometry to Semantics

Chapters 1 and 3 mentioned the idea of a semantic map: upon the geometric map, labeling this is a door, that is a staircase, over there is a fire extinguisher. Real-time object detection models of the YOLO kind are exactly the tool for producing these

labels.

With semantics, the way a navigation task is expressed changes entirely:

- Geometric navigation: go to coordinates (12.5, 8.3)—the user must know the coordinates.
- Semantic navigation: go to the nearest water fountain—the user need only name the object.

The latter is the natural way for a human being to communicate, and the premise of a service robot that is genuinely usable.

Three Roles for YOLO in Navigation

9. Semantic labeling: a detected object is written into the map together with its three-dimensional position (depth supplied by RGB-D or stereo), forming a semantic map.
10. Handling moving objects: recognize movable things such as people and vehicles and remove the features of those regions within SLAM, keeping moving objects from disturbing the pose estimate (the weakness of visual SLAM mentioned in Section 9.5). This is an extremely practical technique.
11. Detecting task targets: an inspection robot recognizing instruments, cracks, and litter; a recycling robot recognizing kinds of bottle and can—this is the core function of the task itself.

The Challenge of the Robot's Point of View

Here is a problem easily overlooked yet absolutely crucial in practice: most vision models are trained on photographs from a human point of view, while a robot's point of view is often altogether different.

Research points out that autonomous navigation on a quadruped robot requires robust semantic perception, and yet, because such platforms operate at a height below 50 centimeters, vision models trained from a human-centric point of view often do not work well.

This problem holds for cleaning robots, robot dogs, and small wheeled robots alike: a chair seen looking up from 30 centimeters, or a door seen from floor level, differs greatly from the photographs in the training data, and the recognition rate falls accordingly.

- Remedy one: collect data from the robot's point of view and retrain or fine-tune the model—which is exactly the intent of stage four of the project in Chapter 31 (training YOLO to recognize inspection targets) and of the semantic labeling of Chapter 32.
- Remedy two: use a newer model with open-vocabulary ability, whose generalization is stronger.
- Remedy three: adjust the mounting height and pitch angle of the camera to approach the point of view of the training data as nearly as possible.

This case bears out once again an idea stressed repeatedly in this book: the performance of a technology depends greatly on the circumstances of its deployment, and one must not look only at the numbers in a paper.

24.5 Navigation Aided by a VLM Vision-Language Model

YOLO lets a robot recognize objects, but it can still answer only what is in the frame. A VLM (vision-language model) goes a step further—it can understand a scene, reason with common sense, and accept natural-language instructions. This is a paradigm shift

now under way in robot navigation.

What a VLM Brings

A VLM, pretrained on a vast corpus of images and text, displays a powerful ability to unify the visual and linguistic modalities, and is increasingly important for navigation tasks that require rich semantic perception. Early models aligned image and text features into a common embedding space, while CLIP introduced contrastive learning to achieve open-vocabulary visual recognition, greatly improving the generalization of multimodal representations; more recent VLMs go further and provide a unified interface for vision-language reasoning and decision support.

For navigation, the key value of a VLM lies in common-sense reasoning. If the instruction is go find a washbasin, for instance, the robot need not have labeled the position of a washbasin beforehand—a VLM knows that washbasins are usually in a bathroom or a kitchen and will explore regions of that kind first. This ability is called a semantic prior.

Three Modes of Application

12. Vision-language navigation (VLN): accepts a natural-language instruction such as walk to the end of the corridor, turn right, and stop at the second door. The system must understand the instruction, relate it to the current field of view, and carry it out step by step.
13. Object goal navigation: accepts a goal such as go find a trash can, and the system must explore an unknown environment and find it. A VLM's common-sense reasoning makes the exploration more efficient.

14. Socially aware navigation: understands the people and the situation in a scene and decides on fitting behavior (not forcing through a crowd, slowing down outside a classroom).

Zero-Shot Ability: No Task-Specific Training

The most pathbreaking property of VLM navigation is its zero-shot ability—no retraining is required for each new environment or new task.

Related research uses a VLM as a global planner, aggregating the submaps of several robots from different viewpoints into one unified global map that encodes the robots' states and the frontier regions; the VLM then allocates frontiers among the robots, bringing about coordinated and efficient exploration. Experiments show that methods of this kind surpass existing baselines in success rate and navigational efficiency without any task-specific training, and ablation experiments further confirm the importance of the VLM's semantic prior.

This stands in sharp contrast with traditional learned navigation, which generally requires collecting a great deal of data in the target environment for training and must start over in another site.

A Frontier Direction: the Vision-Language-Action Model (VLA)

A newer development is the VLA (Vision-Language-Action) model—which does not merely understand but generates action commands directly from image and language.

Recent research proposes a training framework achieving omni-modal goal conditioning: a high-capacity vision-language-action backbone trained simultaneously on three goal modalities—2D pose, first-person image, and natural

language—and on their combinations. This echoes a human capability: we interpret and combine different forms of goal specification flexibly, whether a verbal instruction, a spatial coordinate, or a visual reference.

Other research demonstrates one and the same agent core deployed on four heterogeneous real robots—a dual-arm wheeled platform, a quadruped, a humanoid, and a self-built quadrotor—with only the low-level controller changed. This generality across platforms is a concrete manifestation of the vision of a robotics foundation model.

Practical Limitations: Do Not Forget Engineering Reality

VLM navigation is exciting, but students must recognize its limitations:

- Computational burden: inference with a large VLM takes time and is hard to run in real time on a robot. Hence a variety of efficiency studies, such as attention-based assessment of token importance and query-guided spatiotemporal filtering, which retain the navigation-relevant information without retraining or modifying the pretrained model, and which have been verified for reliable, low-latency instruction-following navigation on a real quadruped robot.
- The risk of hallucination: a VLM may see things that are not there or reason wrongly. A robot that trusts it completely may run into a wall or take the wrong path. A working system must retain a geometric layer of safety checking (the obstacle avoidance of Chapter 29).
- Latency and safety: a VLM suits high-level decisions (where to go) and not real-time control (dodging an obstacle that appears suddenly). A layered

architecture is the inevitable choice—the modern version of the hybrid architecture of Chapter 3.

These limitations are exactly what Chapter 32 will discuss in depth. For now students need only establish one central understanding: a VLM lets a robot understand and reason, but walking steadily and not running into people still rests on the geometry and control of the preceding twenty-three chapters. Neither can be dispensed with.

24.6 Visual Navigation on a Robot Dog

The quadruped robot (the robot dog) has in recent years become a popular platform for research and application in visual navigation. This section explains its particular challenges and the measures taken against them.

Why Choose a Robot Dog

- Adaptability to terrain: it can go up and down stairs, cross thresholds and gravel, and walk over uneven ground—which a wheeled robot cannot do (Section 3.6).
- Suitability for campus and factory sites: on a campus with stairs, slopes, and outdoor passages, a robot dog's ability to get about is plainly better than a wheeled robot's.
- A mature platform: commercial quadruped robots are now quite widespread and provide open development interfaces, which suits them to research and teaching.

Four Particular Challenges

15. Gait vibration: in quadruped walking the body heaves continually up and down and sways from side to side, causing blurred images and violent changes of

pose. This is a severe test for visual SLAM—precisely why the tight IMU coupling of Chapter 23 is indispensable, and it echoes the argument of Chapter 18 that a robot dog needs tightly coupled LiDAR-inertial estimation.

16. The problem of the low viewpoint: as Section 24.4 described, a robot dog operates at a height below 50 centimeters, and vision models trained from a human-centric point of view often do not work well. The accuracy of semantic perception falls accordingly.

17. Understanding three-dimensional terrain: a wheeled robot need only know whether it can go forward, while a robot dog must also know how high this step is, whether it can be climbed, and whether the foothold is secure. Navigation rises from 2D to 2.5D or even full 3D (Section 3.6).

18. Endurance and computational limits: quadruped walking consumes far more energy than wheels, and the computing platform that can be carried is limited as well. This makes lightweight algorithms and edge computing especially important (the theme of Part VII).

Measures and System Design

- The sensor combination: a 3D LiDAR or depth camera (terrain) + an IMU (attitude and vibration compensation) + an RGB camera (semantics). Multi-sensor fusion is here not an option but a necessity.
- Tightly coupled estimation: with the LIO of Chapter 18 or the visual-inertial SLAM of Chapter 23 as the skeleton, the IMU is the key to stability.
- Layered control: the upper layer makes semantic decisions and plans paths (a VLM may be used), the middle layer assesses terrain and plans footholds, and the bottom layer controls the gait. The layers work on different time scales, a typical realization of the hybrid architecture of Chapter 3.

- Localizing the semantic model: collecting data from the robot dog's point of view to fine-tune the model is the most direct way to raise the recognition rate.

The Research Frontier

The robot dog has become the standard platform for verifying frontier navigation technologies. Recent research in vision-language navigation generally verifies the practicality of its methods on a real quadruped robot—because it has both the complexity of the real world and controllable experimental conditions. Work proposing a vision-language-action navigation model designed specifically for legged robots has already appeared as a representative piece, which shows how active this direction is.

24.7 The Campus Inspection Robot and the Recycling-Sorting Robot

This section integrates the technology of this chapter into two concrete project plans, as the conclusion of Part VI and in preparation for the project of Chapter 31.

Project One: the Campus Inspection Robot

This is the spine already mentioned in Sections 5.7 and 18.7; this section plans it fully from the standpoint of visual navigation.

Figure 24-4 The System Architecture of a Campus Inspection Robot

【Perception layer】an RGB-D or stereo camera + an IMU + (optionally) a LiDAR and GNSS

↓

【Localization and mapping】ORB-SLAM3 visual-inertial SLAM (Chapter 23) + AprilTag absolute pose correction (Section 24.3)

↓

【Semantic understanding】YOLO object detection → a semantic map (Section 24.4)

↓

【Task decision】a list of inspection points / a VLM natural-language task (Section 24.5)

↓

【Path planning】global planning (the map) + local obstacle avoidance (real-time perception)

↓

【Execution and recording】move to the inspection point → photograph / measure → label with three-dimensional coordinates and save

The Visual Essentials of an Inspection Task

- Reading instruments: detecting and reading the values of pressure gauges, electricity meters, and thermometers. This requires precise alignment (visual servoing, Section 24.2) together with digit recognition.

- Detecting anomalies: cracks, traces of leakage, rust, piled-up clutter—mostly things only vision can see (Chapter 8 mentioned that a LiDAR cannot supply semantics).
- Detecting change: comparing images of the same position at different times to find anomalies that were not there yesterday and are there today. This requires an accurate ability to relocalize repeatedly (the multi-session operation of Atlas in Chapter 23 applies exactly).
- Access control and safety: detecting whether a door is properly shut, whether a fire escape is blocked, whether a fire extinguisher is in place.

Project Two: the Recycling-Sorting Robot

This is a topic of both technical challenge and educational value, particularly suited to collaboration between high school and university students.

- The task: the robot patrols the campus, recognizes and records (or collects) recyclable items, and sorts them into PET bottles, aluminum cans, paper, and general waste.
- Visual challenges: the objects take many attitudes (lying, standing, crushed), the materials reflect (aluminum cans) or are transparent (PET bottles, a difficulty for depth cameras as well, echoing the limitations of Chapter 9), and the lighting varies (outdoors).
- Building the dataset: this is the core work of the project—photographing recyclables about the campus at various angles, in various lighting, against various backgrounds, building a labeled dataset, and training a YOLO model. This is exactly the method of stage four of the project in Chapter 31.

- System integration: SLAM supplies position (where it was found), YOLO supplies recognition, navigation lets the robot patrol autonomously, and a database records the results.

Planning by Level of Teaching

Stage	Content	Suited to
One	Line following and AprilTag navigation; understanding basic visual navigation	High school
Two	Training YOLO to recognize campus objects; building a dataset	Advanced high school / university
Three	SLAM mapping + semantic labeling; producing a semantic map	University
Four	Autonomous inspection: planning the route, carrying out the task, producing a report	A university project
Five	VLM natural-language tasks; detecting change across sessions	A project / graduate study

Table 24-3 Five Teaching Stages of a Visual Navigation Project

The merit of this plan by levels is that every stage has a result that can be demonstrated, so that students need not wait until the end to see something move; and the technologies of the stages build upon one another, so that the results of the earlier ones are never wasted.

With this, Part VI, Visual SLAM and the Vision-Navigated Robot, is complete. We set out from the fundamentals of features in Chapter 20, passed through the geometry of Chapter 21, the ORB-SLAM2 of Chapter 22, and the ORB-SLAM3 of Chapter 23, and in this chapter settled the technology into real tasks. In the next part we turn our view

to extremely constrained platforms—miniature robots and drones. When the computational resource is only a few milliwatts and the payload only tens of grams, how is SLAM to be realized? Part VII, Miniature Robots, Drones, and Edge AI SLAM, will bring thinking of an altogether different kind.

24.8 The State of Language Navigation: an Honest Assessment of 2025

Section 24.5 described the vision of navigation aided by a vision-language model. This section supplies something equally important: what, with today's technology, it can actually do.

One: a Benchmark for Embodied Navigation

A benchmark published at the end of 2025 gives a fairly clear answer. Its design is this: give the model a photograph of a real scene, a textual instruction, and a type of body (a person, a quadruped robot, a wheeled robot, a bicycle), and have the model draw on the image the trajectory it should take. The dataset has 1,000 situations and over 3,000 expert trajectories, and the scoring considers the shape of the trajectory, the accuracy of the endpoint, and a penalty for places this body ought not to go.

Method	Score
Human expert	75.40
Predicting only the path (told directly where the goal is)	56.55
A straight-line baseline to a known goal	51.89
Predicting the complete trajectory (the best model)	34.38

Predicting only the goal point	29.65

Table 24-4 Evaluation Results of the Embodied Navigation Benchmark (a higher score is better)

Two: Three Numbers Worth Remembering

First, the best model's 34.38 points fall 41.02 points short of the human expert's 75.40. This is not a gap that will be closed in another six months.

Second, and most worth pondering: a straight line to a known goal scores 51.89, far above the best model's 34.38 for a complete trajectory. A straight line that avoids no obstacle, takes no detour, and does not look at the environment at all beats the complete plan of the strongest contemporary model. This shows that the model loses its points chiefly not in the shape of the path but in not having worked out where the goal is.

The third point bears out that judgment: when the goal is given directly, the same model predicting only the path scores 56.55; conversely, predicting only the goal point drops it to 29.65. The model can walk but cannot recognize—mapping the goal in the language to the right place in the image is the real bottleneck at present.

Three: What This Means for the Layered Architecture of This Chapter

These three numbers support directly the position of Sections 24.1 through 24.3: since the model's weakest link is locating the goal, do not let it guess at a position that a marker could settle. A marker put up at the door of the registrar's office answers where is the registrar's office more reliably than any vision-language model.

Conversely, what the model is relatively good at should be handed to it: understanding spoken instructions, reading the content of a frame, generating a report, and

describing a route when the goal is known. The practical question is not whether a language model can do navigation but which parts of navigation it is fit to be given.

This also echoes the low-viewpoint problem of Case 24-2: the model's performance from a robot's point of view falls plainly short of its performance from a human's. Chapter 32 returns to the practice of collecting data and fine-tuning, and Chapter 34 unfolds this whole chain from language to action in full.

Formulas and Procedures

One: Proportional Control for Line Following

Let the width of the image be W, the detected center pixel of the line x_line, and the center of the frame x_center = W / 2; then the lateral error is

$$e = x_line - x_center$$

The most basic proportional controller turns the error directly into an angular rate command:

$$\omega = -K_p \cdot e$$

where K_p is the proportional gain. The minus sign means that when the line is to the right (e positive) the robot should turn right. In practice a derivative term is often added to damp oscillation (PD control), or the linear speed is adjusted by the size of the error—slowing when the error is large and speeding up on a straight run. This is the simplest visual feedback control and the best starting point for understanding visual servoing.

Two: the Control Law of Image-Based Visual Servoing

The aim of IBVS is to bring the current image features s to the target features s*.

Define the feature error

$$e = s - s^*$$

The control law is (driving the error down exponentially)

$$v = -\lambda \cdot L^{+} \cdot e$$

where v is the six-degree-of-freedom velocity command of the camera, λ the rate of convergence, L the image Jacobian (describing how far an image feature moves when the camera moves by one unit), and L^{+} its pseudoinverse. The spirit of this expression is of a piece with the optimization of Chapters 11 and 12: define an error, find the sensitivity of that error to the control variable, and move in the direction in which the error decreases.

Three: the Conversion of a Goal in Semantic Navigation

The core of semantic navigation is converting a semantic goal into a geometric one:

Figure 24-5 The Complete Flow from a Language Instruction to a Motion Command

The user's instruction: please go to the nearest water fountain

↓

① Language understanding (VLM / LLM): parse out the target object class = water fountain

↓

② Query the semantic map: has a water fountain been labeled in the map?

├─ Yes → obtain its three-dimensional coordinates and go straight to ④

└─ No → go to ③ and explore

↓

③ Semantic exploration: choose a promising direction by the VLM's common-sense reasoning

(a water fountain is usually near a corridor or a pantry) → frontier exploration + YOLO detection

↓

④ Geometric path planning: compute the path to the goal on the map (Chapters 3 and 29)

↓

⑤ Local obstacle avoidance and motion control: real-time perception, outputting velocity commands

↓

⑥ Precise docking on arrival (visual servoing, if required)

This flow displays clearly the overall architecture of this book: the upper layer understands semantics with AI, the middle layer supplies geometry with SLAM and planning, and the bottom layer executes the action with control. Not one of the three layers may be dispensed with—the central understanding stressed at the end of Section 24.5.

Four: Fusing an AprilTag Pose Correction

When the robot detects a marker of known world coordinates, it can reckon its own pose from that marker:

$$T_{world \leftarrow robot} = T_{world \leftarrow tag} \cdot T_{tag \leftarrow camera} \cdot T_{camera \leftarrow robot}$$

where T_world←tag is looked up in the marker map (known), T_tag←camera is solved by PnP (Chapter 21), and T_camera←robot is the extrinsic calibration (the camera's mounting position, Chapter 10). Multiplying the three transformations gives the robot's pose in world coordinates—a direct application of composing transformations is multiplying matrices in Chapter 10.

Cases

Case 24-1 From Laying a Line to Understanding Human Speech: Forty Years of Visual Navigation

Arrange the technologies of this chapter in time and a clear line of evolution appears, every step of which corresponds to a rise in the depth at which a robot understands the world.

- The 1980s—line following: the robot recognizes only differences of brightness, and the environment must be modified for it (a line laid down). It does not know where it is or where it is going; it only corrects an error continually.
- The 2000s—landmarks and visual servoing: the robot begins to recognize particular patterns and to localize itself precisely with respect to them. The need to modify the environment is reduced (only markers need be put up) but it has not gone away.
- The 2010s—visual SLAM: the robot can build a map and localize itself, and the environment need no longer be modified. But what it understands is geometry, not meaning.
- The 2020s—semantic and VLM navigation: the robot begins to understand that this is a door and that is a water fountain, and can accept natural-language instructions. At last it can communicate in a human way.

Watching this line, students will find an interesting regularity: what each generation reduces is the preparatory work a human being must do for the robot. From modifying the environment, to labeling landmarks, to merely giving an instruction—the robot adapts more and more to the human world instead of requiring the human to accommodate it.

But here is an important reminder: the new technologies have not made the old ones obsolete. Factory AGVs today still make heavy use of magnetic strip guidance, because in a controlled environment it is the most reliable and the cheapest; precise docking still requires visual servoing, because SLAM is not accurate enough. The engineer's task is not to use the newest technology always but to choose the fittest technology for each problem—the judgment this book has stressed repeatedly since

Chapter 8.

Case 24-2 The Predicament of the Low Viewpoint: When the Model Does Not Know the World as the Robot Sees It

This looks like a technical detail but is in fact of far-reaching effect, and it is the pit students most easily fall into when doing a project.

The situation is this: a student downloads an object detection model that performs excellently on a standard dataset, installs it eagerly on a robot dog or a cleaning robot, and finds the recognition rate dismal—a chair is plainly a chair, and the model cannot make it out; a door is plainly ahead, and the model labels it a wall.

The reason is as the research points out: a quadruped robot operates at a height below 50 centimeters, and vision models trained from a human-centric point of view often do not work well. The photographs of chairs in the training data were mostly taken by a standing person at a height of about 150 centimeters; seen looking up from 30 centimeters, a chair's shape, its perspective, and what occludes what are all entirely different.

This problem has several variants: a cleaning robot sees the lower half of furniture and the texture of the floor; a drone sees the top view of objects; an underwater robot faces images with a severe color cast. What they have in common is that the distribution of the deployment situation does not match that of the training situation.

There is no shortcut—collect data from the target point of view and retrain or fine-tune. That sounds troublesome, but it is exactly the core of stage four of the project in Chapter 31, and the most valuable part of such a project: a local dataset of campus objects from a robot's point of view is in itself a publishable and extensible research

result.

The deeper lesson of this case for students is that an AI model is not a universal black box. Understanding the difference between the distribution of its training data and the circumstances of deployment is a basic skill in using AI responsibly.

Case 24-3 Cooperative Exploration by Several Robots: a VLM as Global Planner

One study mentioned in Section 24.5 deserves unfolding on its own, because it displays the full potential of VLM navigation.

The framework that study proposes integrates a vision-language model as a global planner to achieve multi-robot visual semantic navigation with common-sense reasoning. It aggregates the submaps of several robots from different viewpoints into one unified global map encoding the robots' states and the frontier regions; the VLM then allocates frontiers among the robots, bringing about coordinated and efficient exploration.

What is splendid about this design is that the VLM is not used for low-level control but plays the part of a commander. It looks at the global map and uses common sense to judge which of several unexplored directions is most likely to hold the goal, and then assigns the task to one robot or another. Low-level movement and obstacle avoidance remain the business of traditional planning and control.

The experimental results show that this method surpasses existing baselines in both success rate and navigational efficiency, without any task-specific training; ablation experiments further confirm the importance of the VLM's semantic prior. The research team also verified the framework with a quadruped robot in a real setting.

The lesson for a campus site is that this is precisely the technical path of a project on campus inspection by several robot dogs (Chapters 27 and 31). Several robots explore separately, a VLM coordinates and allocates, each localizes and maps with SLAM, and at last the results are merged into a complete semantic map of the campus—a project topic rich in levels, divisible among a team, and in step with the international frontier.

Hands-On Activities

Activity 24-1 A Staged Implementation from Line Following to Semantic Navigation

Purpose: to build up a capability in visual navigation by degrees, following the five-stage plan of Section 24.7. This activity may be carried to whichever stage the level and resources of the class allow, and every stage has a result that can be demonstrated.

Stage One: Line Following (suitable for high school; no SLAM required)

1. Prepare a teaching cart with a camera (or work in a simulated environment) and lay out on the floor a route with straight stretches and turns.
2. Read the image with OpenCV, convert to grayscale, binarize, and find the center position of the line.
3. Implement the proportional controller of Formula One, turning the lateral error into a steering command so that the cart follows the line by itself.
4. A parameter experiment: adjust the proportional gain K_p and observe the phenomena of it being too small (sluggish response, running wide on a bend) and too large (oscillating from side to side), and find a suitable value.

5. A challenge: add a crossroads or a fork to the route and design rules by which the cart makes a choice.

Stage Two: AprilTag Navigation and Pose Correction (advanced high school / university level)

6. Put several AprilTags up around the classroom, and measure and record the world coordinates of each (building a marker map).
7. Detect the markers with the camera calibrated in Chapter 9 and solve by PnP for the pose of the camera relative to the marker (Chapter 21).
8. By Formula Four, multiply the three transformations to compute the robot's pose in world coordinates and display it in real time.
9. A comparison with SLAM: run visual SLAM at the same time and draw the two trajectories together—observe that the SLAM trajectory is smooth but drifts away by degrees while the marker trajectory jumps about but is always correct (reproducing Case 4-3 of Chapter 4).
10. A simple fusion: implement loose-coupled correction—use SLAM ordinarily and pull the pose back to the value the marker gives whenever one is seen, and observe whether the drift is suppressed.

Stage Three: YOLO Semantic Labeling (university level)

11. Collect data: with a camera at the robot's point of view (mind the mounting height), photograph target objects about the campus—doors, water fountains, fire extinguishers, trash cans, stairheads—at least 200 images of each class covering various angles and lighting.
12. Label them, train a YOLO model, and evaluate its accuracy on a validation set.

13. A viewpoint comparison experiment: photograph another set of test images from a human point of view (about 150 centimeters high) and compare the model's recognition rate at the two viewpoints, reproducing the phenomenon of Case 24-2 quantitatively.
14. Semantic labeling: combine the YOLO detections with RGB-D depth to compute the three-dimensional positions of the objects and label them onto the map built by SLAM, producing a semantic map.

Stage Four: a Semantic Navigation Task (a university-level project)

15. With the semantic map built, implement a simple semantic navigation: input the name of an object (water fountain, say), have the system look up that object's coordinates in the map, plan a path, and navigate to it.
16. Add the handling of moving objects: detect people with YOLO, remove the feature points on them within SLAM, and observe whether the stability of localization improves (the second role of Section 24.4).
17. Implement an inspection task: set several inspection points, have the robot visit them in order, photograph and record each, label them with three-dimensional coordinates, and produce an inspection report at the end.

Stage Five: VLM Assistance (a project / graduate level)

18. Feed the robot's current image to a VLM and ask questions in natural language: is the way ahead passable? Are there stairs in the frame? Where might a door be? Record the accuracy of the answers.
19. Compare the VLM with YOLO: on the same batch of images, compare the ability of the two to recognize open-vocabulary objects (classes not seen in training).

20. Measure the latency: record how long VLM inference takes and assess which layer of the system it belongs in (high-level decisions? real-time control?), against the discussion of limitations in Section 24.5.

Discussion Questions

1. In Stage One, what phenomena do too large and too small a proportional gain produce? Which ideas of control theory does this relate to?
2. In Stage Two, how does the difference between the SLAM trajectory and the AprilTag trajectory change with time? By how much does adding the correction improve matters?
3. In the viewpoint comparison of Stage Three, by how much do the recognition rates at the two viewpoints differ? What warning does this give about simply applying a pretrained model from the internet?
4. In Stage Four, did the stability of localization improve after removing the feature points on people? If there are many people in the scene (a corridor between classes, say), can the system still work?
5. (Project) What is the inference latency of the VLM? If the robot moves at 0.5 meters a second, how far does that latency mean the robot has traveled while thinking? What does this imply for safety design?

Extended Challenge (university level)

First: the recycling-sorting robot—following the plan of Project Two in Section 24.7, build a dataset of campus recyclables (paying particular attention to photographing transparent PET bottles and reflective aluminum cans), train a recognition model, and integrate SLAM so that the robot records what was found where. This topic covers AI, SLAM, and environmental education at once and demonstrates well. Second: a

simplified version of cooperative exploration by several robots—have two robots (or two simulated instances) explore the same site separately, merge their maps afterward (the idea of Atlas in Chapter 23 may be used), and compare the time cooperative and single-robot exploration take to complete coverage, echoing the research direction of Case 24-3.

Further Reading

1. F. Chaumette and S. Hutchinson, "Visual Servo Control, Part I: Basic Approaches" and "Part II: Advanced Approaches," IEEE Robotics & Automation Magazine, 2006–2007. (The authoritative tutorial on visual servoing, with complete accounts of IBVS and PBVS.)
2. E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," ICRA, 2011. (The theoretical source of the fiducial markers common to Chapters 4 and 24.)
3. A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision (CLIP)," ICML, 2021. (A milestone in open-vocabulary visual recognition and the basis of VLM navigation.)
4. B. Yu et al., "Co-NavGPT: Multi-Robot Cooperative Visual Semantic Navigation Using Vision Language Models," arXiv, 2023 (revised 2025). (The source of Case 24-3, on cooperative multi-robot exploration with a VLM.)
5. N. Yokoyama et al., "VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation," ICRA, 2024. (A representative work on zero-shot semantic navigation.)
6. The official Ultralytics YOLO documentation (docs.ultralytics.com). (Training and deployment of object detection models; the principal tool for stage four of the project in Chapter 31 and the semantic labeling of Chapter 32.)

7. The ArUco module and the image processing tutorials of the official OpenCV documentation (docs.opencv.org). (The implementation basis for line following and marker detection.)

Summary

1. Visual SLAM answers where am I and what does the environment look like, while visual navigation answers which way should I go; visual navigation does not necessarily need SLAM (line following and visual servoing belong to the reactive architecture), but only with SLAM added does it gain global planning, navigation to any goal, recovery from a break, and semantic tasks.
2. The three basic strategies: line following is extremely cheap but can only follow a preset route; landmark navigation builds a topological map and is more flexible; visual servoing controls directly by image feedback and can reach millimeter-level relative positioning accuracy, in the two schools of IBVS and PBVS.
3. AprilTag / ArUco has four uses in navigation: absolute pose correction, precise docking, identifying floors and zones, and serving as an approximate ground truth for evaluating SLAM accuracy; fusion with SLAM falls into the three levels of loose coupling, tight coupling, and aiding initialization.
4. YOLO plays three roles in navigation: semantic labeling, the removal of moving objects, and the detection of task targets; but because quadruped and small robots operate at a height below 50 centimeters, models trained from a human-centric point of view often do not work well, and data from the robot's point of view must be collected for fine-tuning.

5. A VLM, pretrained on a vast corpus of images and text, supplies semantic priors and common-sense reasoning, letting a robot accept natural-language instructions and perform new tasks zero-shot; but limited by computational burden, the risk of hallucination, and latency, it suits high-level decisions rather than real-time control.
6. Visual navigation on a robot dog faces four particular challenges: gait vibration, the semantic problem of the low viewpoint, the understanding of three-dimensional terrain, and limits of endurance and computation; the measures taken are tightly coupled multi-sensor estimation and a layered control architecture.
7. Campus inspection and recycling sorting are ideal projects for integrating the technology of this part, and may be planned in the five stages of line following, AprilTag, YOLO semantics, autonomous inspection, and VLM tasks, serving levels from high school to graduate study.

Carrying Forward

This chapter has carried the technology of the four preceding chapters into real tasks, and measured three sets of directly usable numbers along the way: in visual servoing, the position-based camera path stays between 0.70 and 0.82 meters throughout and converges every time, while the image-based path swells rapidly once the starting rotation exceeds 50 degrees and no longer converges at all beyond 150 degrees; for markers, a 16-centimeter marker has a position error of about 5 centimeters at 3 meters, the error growing with the 2.52nd power of distance, and the best viewing angle falls between 10 and 35 degrees (facing squarely meets the pose ambiguity of a plane); and for deployment, tight coupling at a 20-meter spacing has an ATE of 0.153 meters, only

a quarter of loose coupling's, which amounts to halving the number of markers needed.

Part VI, Visual SLAM and the Vision-Navigated Robot, is complete with this. We set out from features and geometry, built a complete visual SLAM system, and at last settled it into real robotic tasks. But all the discussion so far has presumed that the robot has computation and power enough. In the next part we go to the other extreme: when a robot weighs only 44 grams and its computation consumes only milliwatts, is SLAM still possible? Part VII, Miniature Robots, Drones, and Edge AI SLAM, will display the most astonishing engineering achievements of this field.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Part VII Micro Robots, Drones, and Edge-AI SLAM

Chapter 25

NanoSLAM and Low-Power SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

The visual SLAM of Part VI is mature enough, but it rests on a premise it never states aloud: that the robot carries a computer with power to spare, and that it can either be plugged in or carry a battery heavy enough to feed it. This chapter removes that premise and asks what is left.

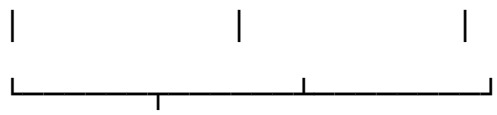
The subject of Part VII is the severely constrained platform—a 27-gram micro drone, a chip that draws 66 milliwatts, a battery that lasts seven minutes. Under that budget the occupancy grid of Chapter 13, the scan registration of Chapter 17, and the key frame management of Chapter 22 must every one of them be weighed again.

The three figures of this chapter are the scales on which they are weighed: Figure 25-1 explains why many cores running slowly use less power than one core running fast; Figure 25-3 asks whether measurements this sparse can still localize and map, and by how much GAP9's energy efficiency really leads; and Figure 25-4 computes something students may find surprising—that the weight of a compute module eats more flight time than its power draw does.

Figure 25-0 The Link Map of Chapter 25

What has been built up so far now goes on the scales

Chapter 8 Ranging sensors Chapter 13 Occupancy grid maps Chapter 17 Scan registration



▼

【Chapter 25 Compressing SLAM into a milliwatt budget】

Sparse ToF ─ the PULP parallel architecture ─ power and weight budgets ─ TinyML

▼

Chapter 26 Drone SLAM Chapter 27 Multi-robot collaboration

▼

Chapter 33 Edge deployment (Jetson, quantization, and distillation)

Learning Objectives

Every system in the first six parts has presumed that the robot has computing resources and electrical power enough—ORB-SLAM3 needs a whole processor, LiDAR SLAM a computing platform of tens of watts. But what if the robot weighs only 44 grams, fits in the palm of a hand, and has a power budget at the milliwatt level? This chapter displays the most astonishing engineering achievement of that field, and opens the extreme challenges of Part VII. After studying this chapter, students should be able to:

1. Explain the three great constraints on running SLAM on a micro robot, and the trouble with the traditional remedy.
2. Explain NanoSLAM's design goals and its five core contributions.
3. Explain how RISC-V and the parallel ultra-low-power computing paradigm underwrite milliwatt-level SLAM.

4. Explain the design trade-offs of a milliwatt-level SLAM system and how power is allocated among its modules.
5. Give examples of applications in micro drones, swarm robots, and AR glasses.
6. Explain the influence of edge AI and TinyML on the development of SLAM.
7. Plan a high-school-level simulation project on mapping with a micro drone.

Core Concepts

中文名詞	English Term	Explanation
微型無人機	Nano-UAV / Nano-drone	An ultra-small aircraft the size of a palm, weighing tens of grams.
機載運算	Onboard Computation	All the computation is done on the robot itself, depending on no external facility.
運算卸載	Computation Offload	Sending the computation to an external computer, subject to the bandwidth and the latency of communication.
平行超低功耗	PULP	A computing paradigm that trades multi-core parallelism for energy efficiency.
系統單晶片	SoC	Integrating processor, memory, and peripherals on a single chip.
開放指令集架構	RISC-V	An open and extensible instruction set architecture for processors.
飛時測距矩陣	ToF Matrix Sensor	A small depth sensor that measures distance in several directions at once.
掃描匹配	Scan Matching	Comparing successive measurements to estimate relative motion (Chapters 13 and 17).
位姿圖優化	Pose Graph	Correcting accumulated error by graph

中文名詞	English Term	Explanation
	Optimization	optimization (Chapter 12).
動態頻率調節	Dynamic Frequency Scaling	Adjusting clock and voltage in real time according to the load, to save power.
深度睡眠	Deep Sleep	A power-saving state that brings the power very low while idle.
邊緣人工智慧	Edge AI	Running AI inference on the device rather than in the cloud.
微型機器學習	TinyML	Running machine learning on severely constrained hardware such as a microcontroller.

25.1 Why SLAM Is So Hard on a Micro Robot

The first question of Part VII is this: why is it so difficult to fit SLAM into a micro robot? The answer is not simply that the chip is slower; it is the superposition of three constraints.

The Three Great Constraints

- Computing power: both the front end of SLAM (feature extraction, scan matching) and its back end (graph optimization, BA) are extremely expensive to compute. The ORB-SLAM family of Chapters 22 and 23 needs a complete multi-core processor; a chip at the microcontroller level simply cannot run it.
- Memory: Chapter 13 did the arithmetic—a 2D map of 20 × 20 meters at 5-centimeter resolution already has 160,000 cells, and the Gmapping of Chapter 14 must multiply that by the number of particles. A micro platform may have only a few hundred kilobytes to a few megabytes.
- Electrical power: this is the most severe constraint of all. A micro drone's battery holds very little energy, and flight itself consumes the overwhelming part of it. Let computation take several watts more and the flight time falls from minutes to tens of seconds—which leaves the system useless.

The Traditional Remedy: Offloading the Computation, and the Trouble with It

Because of the enormous computational load and memory requirement of running a SLAM algorithm, fully onboard mapping can at present be achieved only on robot platforms able to carry a processor of many watts; and precisely for that reason, pocket-sized robots constrained in hardware have had to offload the execution of SLAM to external facilities.

This practice carries obvious costs:

- Dependence on infrastructure: a wireless network and an external computer are required, and the robot is never truly autonomous. At a disaster site, in an underground tunnel, or out in the field this is simply not workable.
- Communication latency: the images or sensor data go out and the commands come back, and the delay may reach tens to hundreds of milliseconds. For a drone flying fast, that delay may mean flying into a wall.
- Bandwidth and power: continuously transmitting large volumes of data itself consumes power, and when several robots operate at once bandwidth becomes the bottleneck (the multi-robot question of Chapter 27).
- Reliability: let the network drop and the robot loses its ability to navigate.

The Central Question of This Chapter

The question this chapter must answer is therefore: can SLAM be made to run entirely onboard, at a power low enough for a micro drone to afford?

The answer is yes, and it has already been done. It required a simultaneous breakthrough at three levels: the algorithm had to be made light, the hardware had to be made efficient, and the sensor had to be made frugal. The sections that follow are the story of how those three were made to work together.

25.2 The Design Goals of NanoSLAM

NanoSLAM was proposed in 2023 by teams at ETH Zürich and the University of Bologna, and is the representative work referred to repeatedly in Chapters 7 and 25. Its goal is clear and bold: to let a micro drone run SLAM entirely by itself.

The Central Claim

NanoSLAM is a lightweight and optimized end-to-end SLAM method designed to run on centimeter-sized robots within a power budget of only 87.9 milliwatts. The research team demonstrated its mapping ability in real scenes and deployed NanoSLAM on a micro drone weighing 44 grams and carrying GAP9, a new commercial RISC-V low-power parallel processor.

To give students a feel for these numbers: 87.9 milliwatts is about the power of an LED indicator lamp, and 44 grams is lighter than an egg. To run complete SLAM under such conditions—this is the most astonishing thing in the chapter.

Five Core Contributions

The NanoSLAM paper lists five contributions explicitly, and they form the technical skeleton of this chapter:

1. A parallel optimized implementation: an optimized parallel implementation of the algorithms, with an assessment of their execution time and memory requirement.
2. Onboard ICP: a parallel implementation and evaluation of ICP, the benchmark algorithm of scan matching (Chapters 13 and 17), able to complete onboard in 55 milliseconds.
3. A sensing expansion board: a custom add-on companion board for the commercial Crazyflie 2.1 micro drone, extending the drone's sensing with four ToF matrix sensors so that it can perform scan matching and autonomous navigation.
4. A communication protocol: a protocol coordinating the integration and the exchange of data between the drone's original microcontroller and the GAP9 system on chip, prescribing how the graph structure is stored, how the poses of

the graph are exchanged, how edges are added, and how graph optimization is performed.

5. Validation in the field: an extensive field-experimental evaluation demonstrating the system's loop-closing mapping ability—a reduction in trajectory error of up to 67 percent with NanoSLAM, and a mapping accuracy of 4.5 centimeters.

The Key Result Figures

The design of this algorithm makes good use of the parallel capability of the RISC-V processing cores, and can build a map of an ordinary environment to an accuracy of 4.5 centimeters with an end-to-end execution time under 250 milliseconds.

Set these numbers beside the earlier chapters and their value is felt more keenly: the 3.5-centimeter accuracy of ORB-SLAM3 on EuRoC in Chapter 23 was obtained on a complete computer, while NanoSLAM's 4.5 centimeters was achieved on a milliwatt-level micro platform. The research team points out that by combining a new kind of low-power depth sensor with GAP9's parallel capability, the system attains an indoor mapping accuracy comparable to that of larger and computationally more powerful drones (usually called micro air vehicles or standard-sized drones).

25.3 RISC-V, GAP9, and Low-Power Parallel Computing

However light the algorithm, it needs suitable hardware to be realized. This section introduces the key hardware that makes milliwatt-level SLAM possible.

New Trends in Microcontroller Design

Recent trends in microcontroller design stress parallel processing, hardware accelerators, and energy efficiency. GreenWaves Technologies' GAP9 system on chip is

exactly the embodiment of these trends, suited to the specialized application of a micro drone; with RISC-V cores it supplies advanced parallel capability, power optimization, and sensor integration, giving a micro drone real-time edge computing, longer flight time, and stronger data processing.

PULP: the Parallel Ultra-Low-Power Paradigm

GAP9 is built on the parallel ultra-low-power (PULP) computing paradigm; it is small, and draws under 180 milliwatts.

The central idea of PULP is worth understanding, because it runs against intuition: rather than one core that runs very fast, use several cores that run slowly and work in parallel. The reason is that the power of a chip is roughly proportional to the square of the voltage and to the frequency—halve the frequency and let two cores work in parallel, and the total amount of computation is the same while the power may fall to less than half. This is the engineering wisdom of trading parallelism for efficiency.

A Comparison of Chip Specifications

Setting side by side the several chips common on micro drones shows plainly where GAP9 stands:

Chip (architecture)	CPU cores	Max frequency	Power	Where it sits
STM32 (ARM)	1	168 MHz	116 mW	Flight control and state estimation
ESP32 (Xtensa)	2	240 MHz	300 mW	Communication and light computation
GAP8 (RISC-V)	8 + 1	175 MHz	96 mW	The previous generation of AI acceleration

Chip (architecture)	CPU cores	Max frequency	Power	Where it sits
GAP9 (RISC-V)	9 + 1	370 MHz	66 mW	The subject of this chapter

Table 25-1 A Comparison of Systems on Chip Common on Micro Drones

This table says three things. First, GAP9 has far more cores than a traditional microcontroller, which embodies the parallel thinking of PULP. Second, even with more cores and a higher frequency its power is lower, so its advantage in efficiency is plain. Third, the STM32 used for flight control and the GAP9 used for computation stand in a division of labor—which is exactly why the fourth contribution of Section 25.2, the communication protocol, exists at all.

Why RISC-V

RISC-V is an open instruction set architecture that anyone may use and extend freely, with no licensing fee. This is extremely favorable to research and to startups:

- It can be customized: special instructions can be added for a particular computation (the matrix arithmetic of SLAM, say), raising efficiency greatly.
- There is no licensing cost: for academic research and small production runs the effect on the cost structure is appreciable.
- The ecosystem is growing fast: RISC-V has risen quickly in edge AI and the internet of things in recent years, and its tool chains are maturing.

For the reader of this book the value of knowing RISC-V is this: it represents the trend by which computing hardware is no longer monopolized by a few manufacturers, and it is a direction worth watching for Taiwan's semiconductor and robotics industries.

Advanced Features: Dynamic Scaling and Deep Sleep

GAP9 supports adjusting the clock frequency (up to 370 MHz) and the voltage at run time, together with automatic clock gating; its idle power can fall to 45 microwatts.

This means the system can run fast where it must and save where it may: at full speed while running SLAM, and at a reduced frequency or even in deep sleep while flight is steady and no mapping is needed. An idle power of 45 microwatts is all but negligible—and this fine-grained power management is exactly the key to a longer flight time.

Measuring It: Do Many Cores Running Slowly Really Use Less Power

Section 25.3 says that rather than one core that runs very fast one should use several cores that run slowly in parallel, on the ground that power is roughly proportional to the square of the voltage and to the frequency. The inference is worth working through with real arithmetic, because it has an equally important second half: more cores are not always better.

The model is simple. To reach a fixed total throughput, N cores each need run at only 1 / N of the frequency. The dynamic power is $N \times C \times V^2 \times f$, and in the near-threshold region the voltage falls roughly linearly with the frequency, but with a floor (a transistor cannot go below the threshold voltage). On top of that every core contributes leakage current, which has nothing to do with the frequency.

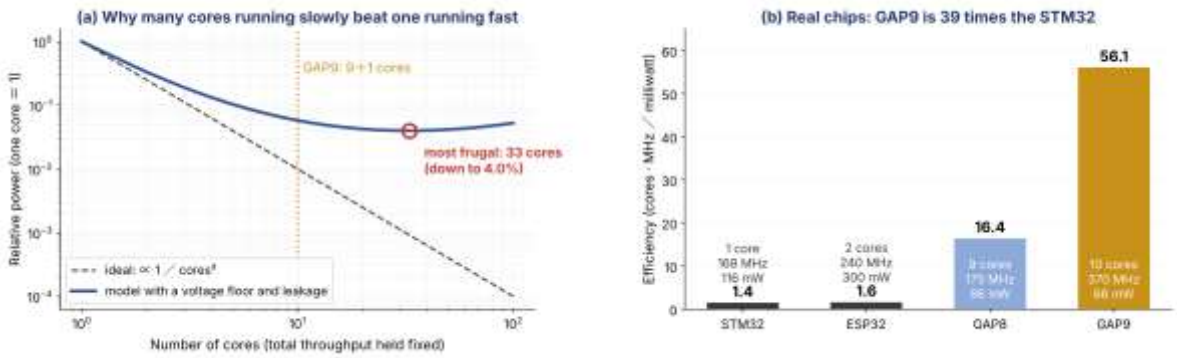


Figure 25-1 The Energy-Efficiency Principle of PULP Set Against Real Chips

Panel (a) is the result of the model, with both axes logarithmic. The gray dashed line is the ideal case—if the voltage could fall with the frequency without limit, the power would be proportional to the inverse square of the number of cores, which is to say that doubling the cores would quarter the power. The blue line is the real curve once the voltage floor and the leakage are included: it hugs the ideal line at first, the power falling to 6.8 percent from 1 core to 8, then departs from it gradually, bottoming out at about 32 cores (down to 4.0 percent), and rising again beyond that because of leakage.

That minimum explains something students may not have thought to ask: if many cores save power, why does GAP9 carry 9 + 1 cores and not ninety? Because beyond the optimum, the leakage each additional core brings exceeds the dynamic power it saves. The optimal number of cores on a real chip must further take account of the cost in area, of memory bandwidth, and of parallelization efficiency—a real workload can seldom be cut into ninety perfect pieces.

Panel (b) verifies the principle against the real chip figures of Table 25-1. Taking the number of cores times the frequency as a rough index of throughput and dividing by the power, the efficiencies are: 1.4 per milliwatt for STM32, 1.6 for ESP32, 16.4 for GAP8, and 56.1 for GAP9. GAP9 is 39 times STM32 and 35 times ESP32.

That multiple does not come entirely from parallelism. From GAP8 to GAP9 the efficiency leaps from 16.4 to 56.1 (3.4 times) while the number of cores rises only from 9 to 10—the difference comes mainly from improvements in process, in the memory hierarchy, and in dedicated accelerators. In other words, the parallel thinking of PULP supplies the first order of magnitude and the hardware optimizations that follow supply another. Only with the two superposed does SLAM come down from watts to milliwatts.

25.4 The System Design of Milliwatt-Level SLAM

This section explains how NanoSLAM compresses the technology of Chapters 13 and 17 into a milliwatt budget.

The Choice of Sensor: a ToF Matrix Rather Than a LiDAR or a Camera

The rotating 2D LiDAR introduced in Chapter 8, useful as it is, is too heavy and too power-hungry for a micro drone. A camera needs a great deal of computation before depth can be extracted (Chapter 9). NanoSLAM's choice is the ToF matrix sensor—a small time-of-flight ranging chip that measures the distance in several directions at once.

- Its merits: a few millimeters in size, extremely low power, a distance output directly (with no computation needed), and no moving parts.
- Its arrangement: as described in Section 25.2, four ToF matrix sensors face in four directions and together supply a distance measurement over approximately 360 degrees—which amounts to a minimal version of a 2D LiDAR.

- Its limitations: the resolution and the measuring range are far below those of a true LiDAR, so the algorithm must be able to work on sparse measurements.

The Algorithmic Pipeline

The overall flow of NanoSLAM is an architecture students already know well; it is only that every module has been made light to the last degree:

Figure 25-2 The System Pipeline of NanoSLAM

【Sensing】4 ToF matrix sensors → sparse distance measurements over approximately 360 degrees

↓

【Front end】Parallelized ICP scan matching (Chapters 13 and 17)

estimates the relative pose between successive instants → completed onboard in 55 ms

↓

【Building the graph】poses stored as nodes, the results of scan matching as edges (the pose graph of Chapter 12)

↓

【Loop closure detection】recognize a revisited position → add a loop closure edge

↓

【Back end】Parallelized pose graph optimization (Chapter 12)

corrects the accumulated drift → trajectory error reduced by up to 67%

↓

【Output】a map and a trajectory completed onboard end to end < 250 ms, at 87.9 milliwatts

Three Key Strategies for Keeping It Light

1. Parallelize rather than simplify: NanoSLAM did not turn ICP or graph optimization into a simple approximate version; it parallelized the standard

algorithms across many cores. This preserves the accuracy while making good use of what the PULP hardware is good at.

2. Sparse rather than dense: sparse measurements from a ToF matrix take the place of a dense point cloud, which greatly reduces both the volume of data and the computation—of a piece with LOAM's idea in Chapter 18 that not all the points are needed.
3. Divide the labor rather than do everything: flight control and state estimation are the business of the original STM32 and the SLAM computation that of GAP9, the two coordinated by a dedicated communication protocol. Each to its own office, and no single chip overloaded.

What the Allocation of Power Reveals

Putting the relevant figures together shows the trade-offs of the system design:

Task	Power / latency	Note
NanoSLAM mapping	about 87.9 mW, latency < 250 ms	The subject of this chapter
Monte Carlo localization	23 mW on average	Localization given a known map (Chapter 11)
YOLO object detection	below 100 mW, as little as 17 ms	Edge AI inference (Chapters 24 and 32)
Deep sleep	45 µW	Very low power while idle

Table 25-2 The Power Performance of Each Task on a Micro Platform

Holding the system within a power envelope below 100 milliwatts—above all for computationally intensive tasks—is vital to extending flight time and to supporting complex real-time task loads.

It is worth noting that localization (23 mW) uses far less power than mapping (87.9 mW). This bears out the observation of Case 14-3 in Chapter 14—that mapping for the first time and localizing afterwards are different tasks. For a micro robot it implies a practical strategy: build the map well first (it may be slower and more costly in power), and let the tasks that follow need only low-power localization.

Measuring It: With Measurements This Sparse, Can It Still Localize

Section 25.4 says that the resolution and measuring range of a ToF matrix are far below those of a true LiDAR, so that the algorithm must be able to work on sparse measurements. That sentence leaves two questions unanswered: how sparse may it be and still be usable? And if something must be given up, should one keep the number of rays or the range? This subsection measures the two separately.

The experimental setting is an indoor scene of 10 × 7 meters containing six pieces of furniture. The robot moves along an elliptical trajectory, and at each position estimates its own pose by likelihood-field scan registration (the method of Chapter 17) and compares it with the true pose. Two sensor configurations are used: four ToF matrices (45 degrees of field of view each, about 180 degrees in all) and a rotating LiDAR (360 degrees, 12 meters).

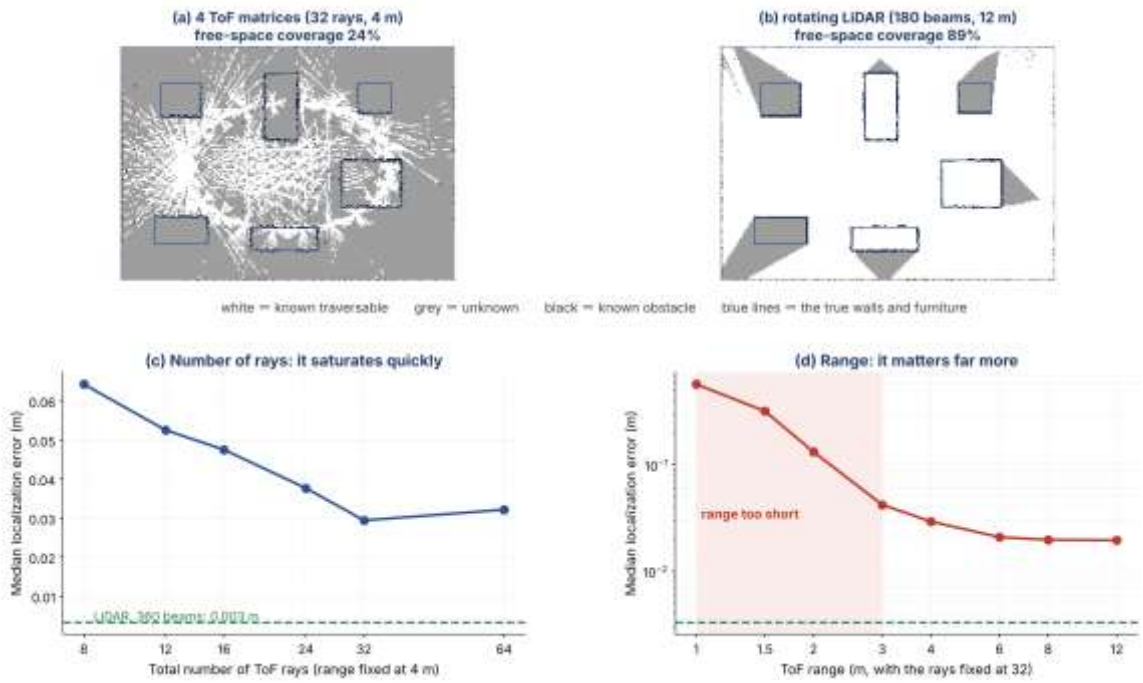


Figure 25-3 Mapping and Localization with a Sparse ToF and with a Rotating LiDAR

Panels (a) and (b) are the maps the two build. The LiDAR sweeps the room clean, with 89 percent coverage of the free space and the outlines of all six pieces of furniture clear. The ToF map is plainly incomplete: the coverage is only 24 percent, the white traversable region radiates outward in spokes, and large areas remain the gray of the unknown. There are two reasons—four sensors cover only about half of the bearings, and the range is only 4 meters, which cannot reach the far walls.

Panel (c) answers the first question, that of the number of rays. With only 8 the localization error is 0.064 meters; raise it to 12 and it falls to 0.053, to 16 and it falls to 0.048, and by 32 rays it has reached 0.030 meters, after which the curve is flat—0.032 at 64. That is to say, going from 32 rays to 64 (twice as many) buys nothing at all. This is a conclusion of real importance: for localization, a few dozen measurements already come close to saturation.

Panel (d) answers the second question, that of range, and the answer is entirely

different. At a range of 1 meter the localization error is 0.576 meters (not usable at all), at 2 meters 0.133, at 4 meters 0.030, and only beyond 6 meters does it flatten at 0.021 meters. From 1 meter to 4 the error improves nineteenfold. Set panels (c) and (d) side by side and the conclusion is plain: in choosing a ToF sensor for a micro platform, range matters far more than the number of zones.

Finally, set the ToF beside the LiDAR: a 4 × 8 ToF arrangement at a range of 4 meters gives a localization error of 0.030 meters against the LiDAR's 0.003—nine times worse, with an eleventh of the rays and a third of the range. This is exactly what makes milliwatt-level SLAM possible: localization needs far less density of measurement than intuition suggests, though it does need to see far enough.

Enough for Localization Does Not Mean Enough for a Map

There is a gap in this figure that must be made clear: the ToF's localization accuracy is nine times worse than the LiDAR's, but its map coverage is barely a quarter of the LiDAR's (24 percent against 89).

The reason is that the two need different quantities of measurement. Localization fits the current frame to a known structure, and a few correct distance measurements are enough to pin down six degrees of freedom; mapping, on the other hand, must actually sweep every piece of space once, and there is no shortcut.

This also explains why a real milliwatt-level system mostly builds only a coarse map that will serve—not because the algorithm is wanting but because the sensor's total quantity of measurement is simply that much. In planning a project on a micro platform, students should set the goal at being able to return to the start and avoid obstacles rather than at building a map as

handsome as the LiDAR's.

25.5 Applications: Micro Drones, Swarm Robots, and AR Glasses

What applications, impossible before, does milliwatt-level SLAM open up? This section describes three representative directions.

Micro Drones: Safety and Accessibility

A micro drone fits in the palm of the hand, suits indoor environments, and is very safe around people; yet such drones often cannot deliver the performance a complex task requires, for want of sufficient sensing and computing hardware.

Safety deserves particular emphasis. A standard drone of several kilograms that goes out of control and strikes a person may cause serious injury; a 44-gram micro drone is next to harmless. This lets them operate in classrooms, offices, hospitals, and other spaces where people are—which a large drone cannot do.

- Indoor exploration and survey: exploring and mapping spaces people cannot or should not enter (narrow ducts, buildings after a disaster, ceiling voids).
- Agricultural inspection: research already exists in which several autonomous micro drones detect pests visually and a single slower heavy vehicle then goes to the detected outbreak to apply treatment; to meet the extreme constraints of a micro drone (a low-resolution sensor and a computing power budget below 100 milliwatts), the team designed and optimized a tiny image-based convolutional neural network for pest detection.

- Education and research: a micro drone is relatively affordable and very safe, and makes an excellent teaching platform (Section 25.7).

Swarm Robots

Once both the cost and the size of a single robot have come down, using a great many of them becomes a workable strategy. The value of swarm robots lies in:

- Efficiency of coverage: several robots explore separately and the time to finish a large task falls greatly (the multi-robot collaborative exploration of Case 24-3 in Chapter 24).
- Fault tolerance: the failure of one does not affect the task as a whole—an advantage a single expensive robot cannot match.
- Distributed perception: observing from several viewpoints at once allows a more complete model of the environment to be built.

But swarm robots have one crucial premise: every robot must be able to localize itself. If all of them depend on external facilities or a central computer, the communication bandwidth collapses as the number of robots grows (as Section 25.1 noted). Onboard SLAM is therefore a necessary condition for swarm robots to scale—and that is exactly the strategic value of a technology like NanoSLAM. Chapter 27 goes deeply into the collaborative questions of multi-robot SLAM.

AR Glasses and Wearable Devices

Chapters 20 and 23 discussed AR's need for visual-inertial SLAM, but the constraints on AR glasses are more severe than those on a phone: they must be light enough to wear for long periods, the battery must be small, and they must not grow hot against the face.

These constraints are astonishingly like those on a micro drone—both amount to running SLAM within a very small power budget. The technical line of this chapter (a dedicated low-power chip, parallel ultra-low-power computing, a lightweight algorithm) therefore applies directly to wearable devices too.

This commonality across fields is an important idea in this chapter: low-power SLAM is not a specialized technology for micro drones alone, but a shared requirement of every constrained platform. Master this line and students hold the key to a great many applications—AR glasses, smart watches, miniature sensing nodes, and more.

Measuring It: Is the Price of Computation Really Its Power

Section 25.1 says the difficulty for a micro robot lies in the three constraints of computation, power, and weight. Which of the three is the most deadly? Intuition says power, but the arithmetic makes the answer weight—and by a wide margin.

The basis of the calculation is a micro drone of the 27-gram class: a battery of about 0.89 watt-hours, a hover power of about 7.6 watts, and an endurance in pure hover of about 7.0 minutes. By momentum theory the hover power is roughly proportional to the mass raised to the power 1.5, so hanging one more board on it does not merely consume that board's own power—it makes all four motors work harder.

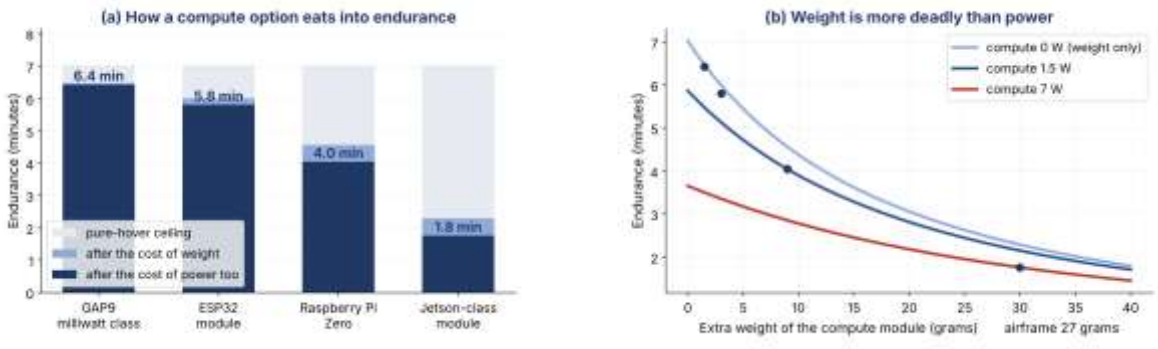


Figure 25-4 How the Weight and the Power of a Compute Option Eat into

Endurance

Panel (a) splits the endurance loss of four compute options into two parts. Take GAP9: at only 1.5 grams and 66 milliwatts, its weight brings the endurance down from 7.03 minutes to 6.48, and its power brings it down further to 6.43—a loss of only 8.5 percent in all.

A module of the Jetson class is the other extreme: 30 grams and 7 watts. Its weight presses the endurance from 7.03 minutes down to 2.29, and its power presses it further to 1.76. Note the ratio of these two parts—the weight costs 4.74 minutes and the power only 0.53, the former nine times the latter. For the four options this ratio is 10.6, 5.3, 4.7, and 9.0 times, without a single exception.

Panel (b) takes the weight out on its own. Even with a compute power of 0 watts (imagine a board that draws nothing at all), simply hanging 30 grams on the drone brings the endurance down from 7.0 minutes to 2.3. The vertical distance between the three curves—which is to say the effect of power—is far smaller than the fall of the curves themselves.

This conclusion bears directly on the choice of parts for a project. In choosing a compute module for a micro platform the first question to ask is not how many watts it draws but how much it weighs. A frugal but bulky board may cost more endurance than a thirsty but light one. This is also why Section 25.2 insists that milliwatts and a few grams must hold together—let either fail and micro SLAM does not stand.

When Does This Rule Reverse

The conclusion that weight comes first presumes that the hover power far exceeds the computing power. For a 27-gram drone, 7.6 watts against 0.066 is

a difference of more than a hundredfold, so weight dominates entirely.

Change to a ground robot and it is otherwise. A wheeled robot of 5 kilograms may use only 10 watts to move, and hanging another 100 grams on it scarcely affects the energy at all—there the computing power becomes the main consideration instead.

So this is not a universal rule but a rule for flying platforms. Before applying it, students should compute for their own platform the ratio of the power of motion to the power of computation, and the ratio of the added weight to the platform's own.

25.6 The Influence of Edge AI and TinyML on SLAM

This section widens the view from SLAM to the trend of edge intelligence as a whole, and explains its links with the later chapters of this book.

What Edge AI Means

Edge AI means running AI inference on the device rather than in the cloud. Its value is exactly of a piece with the reasons Section 25.1 gave for onboard SLAM: no dependence on a network, low latency, protection of privacy, and the ability to scale.

TinyML is the extreme version of edge AI—running machine learning on hardware such as a microcontroller with only a few hundred kilobytes of memory. It sounds impossible, but through model quantization, pruning, knowledge distillation, and the like, useful inference can already be run on very small hardware.

Three Ways It Joins with SLAM

1. Enhanced perception: running object detection on a micro platform. Benchmarks show that an inference task such as YOLO object detection can be completed below 100 milliwatts, with a shortest inference time of 17 milliseconds at about 1.59 millijoules. This lets a micro robot do semantic perception too (Chapter 24).
2. Learning in place of computation: using a small neural network to estimate certain quantities directly (optical flow, depth, even pose) in place of traditional geometric computation. If the model is small enough, inference may use less power than the traditional algorithm.
3. Online learning: a more advanced direction still is to let the robot adapt by learning for itself while in flight. Research already exists on on-device self-supervised learning aboard a micro drone within an extremely low power budget—which means the robot does not merely execute a model trained in advance but can keep adapting to new environments.

The Co-Design of Hardware and Algorithm

The most important methodological lesson of this chapter is this: under extreme constraints, hardware and algorithm cannot be designed apart.

NanoSLAM succeeded not because the algorithm was especially clever, nor because the chip was especially fast, but because the two were fitted to each other: the algorithm was tailored to the parallel architecture of the RISC-V multi-core, and the hardware supplied the efficiency and the integration the algorithm required. This way of thinking—hardware-software co-design—is a core competence in the field of edge computing.

For students of an intelligent robotics program this is an important reminder:

besides learning to use algorithms off the shelf, understanding the properties of the underlying hardware (parallelism, the memory hierarchy, the behavior of power) matters just as much. Where those two meet is exactly where Taiwan's advantage in semiconductors and its applications in robotics can be joined.

Links with the Chapters to Come

- Chapter 26: visual-inertial navigation for a micro drone, continuing this chapter's discussion of the constraints of the platform.
- Chapter 27: multiple robots and cloud-edge collaboration, answering to the swarm robots of Section 25.5.
- Chapters 32–34: semantic SLAM, training in simulation, and edge-AI deployment (Jetson, the NPU; Chapter 33), together with VLM language navigation (Chapter 34), are the extension of this chapter's discussion of edge computing.

25.7 A High School Project: Simulating Mapping with a Micro Drone

The technology of this chapter sounds out of reach, but it in fact makes an excellent high school project—because the central ideas can be experienced in full through simulation, with no expensive hardware.

Why It Suits High School

- The idea is intuitive: building a map with distance sensors in four directions is far easier to understand than the features and the geometry of visual SLAM.
- It can be simulated completely: a 2D simulation of a micro drone can be written in Python, with no aircraft needed.

- The connection is felt: micro drones, swarm robots, and AR glasses are all subjects students find interesting.
- The levels are distinct: from the simplest sensing and drawing, to adding drift, to correcting by loop closure, the difficulty can be dialed up.

The Design of the Project: Four Stages

1. Stage one—build the simulated environment: draw a 2D room (walls and obstacles) in Python, put a virtual micro drone in it, and implement distance sensing in four directions (from the drone's position, find the nearest wall in each of four directions).
2. Stage two—ideal mapping: fly the drone along a prescribed route and at every step use the four distance measurements to draw the detected wall points onto the map. This stage assumes the position is perfectly accurate, and students will watch a handsome map of the room emerge bit by bit.
3. Stage three—add drift: put a small random error into the drone's estimate of its own position (simulating odometry drift, Chapters 1 and 7) and build the map again. Students will see with their own eyes the map begin to distort and the wall lines double—a concrete reproduction of the map pathologies of Section 13.7.
4. Stage four—correct by loop closure: let the drone go once around and return to the start, add the constraint that the end should equal the start, and spread the error back over the whole trajectory by a simplified method (the paper method of Implementation 12-1 in Chapter 12 applies directly), then watch the map being pulled straight.

Directions for Extension

- Computing the power budget: given the battery capacity and the power of each module (see Table 25-2), compute the flight time, and experiment with how long it can fly if the mapping rate is halved—an excellent exercise in engineering trade-offs.
- Swarm simulation: add several virtual drones exploring separately and compare the time needed to cover the whole area (answering to Section 25.5 and Chapter 27).
- An experiment on the number of sensors: compare distance sensing in 4, 8, and 16 directions and its effect on the quality of the map—which corresponds to the trade-off in a real system between the number of sensors and the power and weight.

Teaching Suggestions

The teaching value of this project is that it lets students walk through the whole core logic of SLAM in the most simplified setting: sense → map → discover the drift → correct by loop closure. This is a miniature of Parts I to IV of this book.

Teachers are advised to let students first produce the perfect map of stage two and feel the achievement, and then to add drift deliberately to break it—that moment when the handsome map suddenly goes crooked will make students truly understand why SLAM is a hard problem. This rhythm of building, then breaking, then repairing is far more effective than expounding the theory directly.

At this point students have seen SLAM realized on a platform under extreme constraint. The next chapter focuses on the navigation challenges of the drone itself: how is one to fly in a GPS-denied environment? How are fast motion and vibration to be handled? How does visual-inertial odometry work on a drone? Chapter 26 carries

the discussion of Part VII to the heart of flight.

25.8 After the Milliwatt: Three Advances of 2025–2026

NanoSLAM proved that milliwatt-level SLAM is possible. The advances of the last few years have been mainly in three directions, and the price of each is worth knowing.

One: Moving a Complete Visual-Inertial Odometry onto RISC-V

A piece of work from 2026 deployed a complete monocular visual-inertial pipeline onto GAP9: ORB features, the eight-point algorithm and EPnP with RANSAC, IMU preintegration, and sliding-window pose graph optimization—which is to say the whole apparatus of Chapters 21 and 23, packed into a chip running at 370 MHz.

The result is 20 frames per second, a power below 100 milliwatts, and a memory footprint under 1 MB (about 110 KB in the first level and about 850 KB in the second). But the price is explicit: it does without loop closure detection, and on the standard EuRoC dataset its mean trajectory error is about 3.46 meters against about 0.25 meters for ORB-SLAM3 with loop closure—a difference of an order of magnitude. The authors themselves attribute the reason to the absence of global correction.

This comparison is worth reading together with Figure 22-2 in Chapter 22. That figure proved that loop closures and covisibility edges are the only route by which error is redistributed; and a micro platform gives them up not because they do not matter but because they require the whole map and the whole trajectory to be kept—which is exactly what will not fit in 1 MB of memory.

Two: Sensors Grow Sparser and Cheaper

The second direction is to simplify the sensor further still. The conclusion of Figure 25-3(c)—that a dozen or so measurements already come close to saturation—is precisely

the theoretical basis of this direction. Practices that have appeared in recent years include replacing several fixed sensors with a single multi-zone ToF on a rotating mount, and using a grayscale camera of very low resolution (160 × 160, say) for optical flow and place recognition.

The common logic of such designs is this: rather than pursue the quality of a single measurement, increase the number and the variety of measurements and then fuse them by the probabilistic methods of Chapter 11. It is the same belief as that of Chapters 8 and 9—that there is no all-powerful sensor, only a fitting combination—only pushed to its limit under a milliwatt budget.

Three: Computation Moves Toward Computing Only When It Must

The third direction lies neither in the algorithm nor in the hardware but in scheduling. Section 25.3 noted that GAP9's idle power can fall to 45 microwatts, which makes event-driven computation worth its while: reduce the frequency or even sleep while flight is steady, and run at full speed only when turning, when meeting an obstacle, or when the uncertainty of localization is detected to be rising.

This idea is of a piece with the map maintenance of Section 19.6 and the staggered execution of Section 33.8—all of them ask whether this step really needs doing now. For a platform with only seven minutes of endurance, saving half the computing time amounts to nearly half a minute more in the air.

	The NanoSLAM of This Chapter	A complete VIO on RISC-V	Event-driven scheduling
Sensor	4 ToF matrices	A monocular camera + an IMU	Not restricted

Loop closure	Yes (lightweight)	No	Depends on the configuration
Memory	A few hundred KB	Under 1 MB	Unchanged
Power	Milliwatt level	Below 100 milliwatts	The average can fall further
Accuracy	Centimeter level (a small site)	About 3.46 meters on EuRoC	Unchanged
The chief price	A sparse map	No global correction	Complexity of implementation

Table 25-3 A Comparison of the Three Directions

Formulas and Procedures

One: The Relation Between Power and Flight Time

The flight time of a micro drone is decided by the battery energy and the total power:

$$\textit{flight time} = \textit{battery energy} / (\textit{flight power} + \textit{computing power} + \textit{sensing power})$$

Taking the Crazyflie 2.1 as an example, a single flight with a 380 mAh battery reaches about 7 minutes. Supposing that the power of flight itself is far greater than that of computation, then:

$$\textit{if computing power is a fraction } r \textit{ of the total power, the flight time falls by about } r$$

This simple relation explains why milliwatts matter so much. If SLAM drew 5 watts, then for a micro drone of about 10 watts in total the flight time would be cut straight

in half; whereas 87.9 milliwatts is only about 1 percent and scarcely affects endurance at all. This is the engineering significance of NanoSLAM's pressing the power down to the milliwatt level.

Two: The Upper Bound on the Benefit of Parallel Speedup

When an algorithm is parallelized across N cores, the theoretical speedup is limited by the part that cannot be parallelized:

$$\text{speedup } S(N) = 1 \;/\; ((1 - p) + p \;/\; N)$$

where p is the fraction that can be parallelized. This is called Amdahl's law. With p = 0.9 and N = 8, for instance, the speedup is about 4.7 times rather than 8.

This formula bears directly on the design of NanoSLAM: the nearest-point search of ICP is highly parallel (every point searches independently), so p is high and the benefit of parallelism marked; whereas the sparse matrix factorization within graph optimization is harder to parallelize completely. Understanding which parts are worth parallelizing is the key to making good use of the PULP architecture.

Three: The Measure of Energy Efficiency

In comparing the computing power of different platforms, speed alone is not enough; what matters is how much computation can be done per joule:

$$\text{energy efficiency} = \text{amount of computation} \;/\; \text{energy consumed} \quad (\text{common units: GOPS / W or Ops / J})$$

This is precisely the goal the PULP paradigm pursues (Section 25.3). Taking this chapter's data as an example, YOLO object detection has a shortest inference time of 17 milliseconds at about 1.59 millijoules—and measuring by how much energy each inference consumes reflects the real usability of a constrained platform far better than

how many inferences it can run per second.

Four: A Decision Procedure for the Design of Low-Power SLAM

Figure 25-5 A Decision Procedure for SLAM Design on a Constrained Platform

① What is the power budget? (battery capacity ÷ target running time − the power needed to move)

↓

② The choice of sensor: what can be afforded within the budget?

• a few watts → 3D LiDAR / depth camera

• a few hundred milliwatts → 2D LiDAR / monocular camera

• a few tens of milliwatts → ToF matrix sensor (this chapter's solution)

↓

③ The choice of algorithm: can the volume of data and of computation be afforded?

• a dense method → needs much memory and computation; not feasible on a constrained platform

• a sparse / feature-based method → a great reduction (the shared idea of Chapters 18 and 20)

↓

④ Assess the parallelism: which parts can be parallelized? (estimate the benefit by Amdahl's law)

↓

⑤ Power management: dynamic frequency scaling, deep sleep, switching mode by task

↓

⑥ Design the division of labor: are flight control and computation to be separated onto different chips? How is the communication coordinated?

Cases

Case 25-1 The Power of One LED: Rethinking the Cost of Computation

What is most worth stopping to think about in this chapter is that figure of 87.9 milliwatts.

Let us make a few comparisons. A phone's LED flashlight consumes some hundreds of milliwatts; a laptop running ORB-SLAM3 may draw tens of watts in the processor—some hundreds of times NanoSLAM. And within such a budget NanoSLAM attains a mapping accuracy of 4.5 centimeters, comparable with that of larger and computationally more powerful drones.

This comparison overturns a common intuition: we tend to think that a better result requires more computation. But NanoSLAM proves that through the co-design of algorithm and hardware, comparable accuracy can be maintained while the power falls by two or three orders of magnitude.

This gives students two lessons. First, the bottleneck in performance often lies not in a want of computing power but in the use of the wrong method. LOAM's not all the points are needed in Chapter 18, ORB's trading some accuracy for speed in Chapter 20, and this chapter's trading parallelism for efficiency are all the same way of thinking: find what is essential to the problem and compute only what is necessary.

Second, innovating under constrained resources often calls forth more real

engineering wisdom than working with resources to spare. This has particular meaning for Taiwan's robotics and semiconductor industries: we may not be able to compete with the giants on raw computing power, but on the road of doing more with less there is ample room to work.

Case 25-2 Who Flies and Who Thinks: the Philosophy of a Two-Chip Division of Labor

One design in the NanoSLAM system that is easily overlooked and yet full of engineering wisdom is its two-chip architecture: the original STM32 handles flight control and state estimation, the added GAP9 handles the SLAM computation, and the two are coordinated by a specially designed communication protocol—covering how the graph structure is stored, how the poses of the graph are exchanged, how edges are added, and how graph optimization is performed.

Why not have one chip do everything? There are three reasons:

- The demands on timeliness differ: the flight control loop must run at a very high rate with very low latency, and any delay may bring the aircraft down; SLAM may be slower (250 milliseconds is acceptable). Put the two on the same chip and a peak in the SLAM computation may drag the flight control down with it.
- Isolation for reliability: even if SLAM crashes, flight control can keep the aircraft safe—a basic principle of safety design.
- Each to its strength: the STM32 suits real-time control and GAP9 suits parallel computation. Use the right chip for the right thing and the efficiency of the whole is best.

Students should notice that this division into a fast reflex layer and a slow

deliberative layer is exactly the same design pattern as the hybrid architecture of Chapter 3, the local and global subsystems of Cartographer in Chapter 15, and the high-rate odometry and low-rate mapping of LOAM in Chapter 18.

This shows one thing: a good principle of system design holds generally across levels. Whether at the level of the algorithm, of the software architecture, or of the hardware, layering by time scale and giving each layer its own office is an effective strategy. Mastering this way of thinking in patterns is worth more than remembering the details of any single system.

Case 25-3 A Micro Drone in the Field: from the Laboratory to the Farm

The value of a micro drone is displayed to the full in one concrete agricultural application.

A research team proposed the following idea: efficient crop production requires that an outbreak of pests be detected early and treated in time, and they considered using a fleet of several autonomous micro drones to detect pests visually, after which a single slower heavy vehicle goes to the detected outbreak to apply treatment.

This division of labor between light cavalry for reconnaissance and heavy troops for the work is extremely clever: micro drones are cheap, safe, and deployable in quantity, and so suit a fast search over a wide area; the heavy vehicle, slow as it is, can carry the chemical and do the actual work. Together they are far more efficient than either alone.

The technical challenge is equally worth noting: to meet the extreme constraints of a micro drone (a low-resolution sensor and a computing power budget below 100 milliwatts), the team designed, tuned, and optimized a tiny image-based convolutional

neural network for pest detection. This network needs only 0.58 GOps per inference, and on their dataset it reaches a mean average precision of 0.79 in detecting harmful insects—14 percent lower in mean precision than the best-performing network in the literature, but with 32 times less computation.

This trade of 14 percent of accuracy for 32 times less computation is a miniature of engineering on a constrained platform. The case also suggests a direction for a local project: Taiwan's agricultural settings (orchards, paddy fields, greenhouses) face the same need to detect pests and disease, and the safety and low cost of a micro drone make it better suited than a large drone to working among densely planted crops.

Implementation Activities

Implementation 25-1 Simulating Mapping with a Micro Drone

Purpose: to experience in full, by simulation alone, the core logic of low-power SLAM—from mapping with sparse sensing, through observing the drift, to correcting by loop closure—and to carry out an engineering trade-off analysis of power against the number of sensors. This activity needs only Python and suits every level from high school to the undergraduate.

Part One: Build the Simulated Environment and Map Ideally (everyone)

1. Define a 2D room in Python: represent the walls and obstacles by line segments and place in it a virtual micro drone (represented by a position and a heading).
2. Implement distance sensing in four directions: cast a ray forward, backward, left, and right from the drone's position and compute the distance to the intersection with the nearest wall. This simulates the four ToF matrix sensors of Section 25.4.

3. Move the drone along a prescribed route (once around the room, say), recording the four distances at every step and drawing the detected wall points onto the map by the coordinate transformation of Chapter 10.
4. Display the growing map with matplotlib and watch the outline of the room emerge. This stage assumes the position is perfectly accurate, and the map should be quite handsome.

Part Two: Drift and Loop Closure (everyone)

1. Add odometry drift: put a small random error into the drone's estimate of position and angle at every step (a standard deviation of 2 centimeters in position and 0.5 degrees in angle, say) and build the map again.
2. Observe the pathologies: what has gone wrong with the map? Have the wall lines doubled? After a complete circuit, do the end and the start fail to meet? Measure how large that discrepancy is. Compare with the three map pathologies of Section 13.7.
3. Implement a simplified loop closure correction: when the drone returns near the start, add the constraint that the end should equal the start and spread the accumulated error evenly over every step of the whole trajectory (the method of Implementation 12-1 in Chapter 12), then draw the map again.
4. Quantify the improvement: compare the discrepancy between end and start before and after the correction, together with the visual quality of the map, compute the percentage by which the error fell, and set it against the 67 percent of the NanoSLAM paper.

Part Three: An Engineering Trade-off Analysis (undergraduate; the distinctive part of this chapter)

1. An experiment on the number of sensors: change the four-direction sensing to 8, 16, and 32 directions and compare the quality of the map (the completeness of the walls, the resolution of detail). Find the turning point beyond which the improvement in quality slows.
2. Estimate the power: supposing each ToF sensor draws 20 milliwatts and the power of each scan match grows linearly with the number of sensed points, compute the total power for each number of sensors.
3. Compute the flight time: by Formula One, with the battery energy fixed and flight itself drawing 5 watts, compute the flight time for each configuration and draw the three-way trade-off among the number of sensors, the quality of the map, and the flight time.
4. A decision exercise: if the task requires at least 5 minutes of flight and a map error below 10 centimeters, which configuration would you choose? Explain why.

Part Four: Further Exploration (advanced)

1. An experiment on the mapping rate: change the rate at which SLAM runs from once per step to once every three steps and once every five, and observe how much the map quality falls and how much power is saved. This corresponds to the dynamic frequency scaling strategy of a real system (Section 25.3).
2. Swarm simulation: add a second and a third virtual drone exploring separately, merge their maps afterwards, and compare the number of steps needed to cover the whole area by collaborative exploration against a single drone. This answers to Section 25.5 and Chapter 27.
3. A localization mode: build the map with one drone first, then let a second only localize within the known map (without mapping) and compare the

computation of the two. Set this against the difference between localization at 23 mW and mapping at 87.9 mW in Table 25-2.

Discussion Questions

1. What pathologies appeared in the map once drift was added? How large was the discrepancy at the end of a circuit? Does this agree with the description in Section 13.7?
2. The loop closure correction spreads the error over the whole trajectory rather than correcting the last step alone. Why is spreading it better? (Hint: the case in Chapter 12.)
3. Going from 4 sensors to 32, by how much did the quality of the map improve? By how much did the power rise? What number do you think most suitable, and why?
4. If you were the system designer of a micro drone, how would you trade a better map against a longer flight time? Does the trade-off differ from task to task?
5. (Undergraduate) By Amdahl's law (Formula Two), if 95 percent of ICP can be parallelized, by how much can GAP9's 9 cores speed it up? How far is that from the ideal 9 times, and where does the difference come from?

Extended Challenge (university level)

First: implement a simplified pose graph optimization—take the pose at every step of the simulation as a node, the motion between adjacent steps as an edge, and the loop closure as an extra edge, and optimize the whole trajectory with scipy's least-squares solver (the programmed version of Implementation 12-1 in Chapter 12), then compare its accuracy with the simple method of spreading the error evenly. Second: if your school has a Crazyflie or a similar micro drone platform, try reading its sensor data and

running your own SLAM program on a computer, and experience the difference between real data and simulation—real sensor noise, time delays, and the disturbances of flight will give you a deeper feeling for the engineering challenges of this chapter.

Further Reading

1. V. Niculescu et al., "NanoSLAM: Enabling Fully Onboard SLAM for Tiny Robots," IEEE Internet of Things Journal, 2023 (arXiv:2309.12008). (The principal source for this chapter; the code is public on GitHub at ETH-PBL/NanoSLAM.)
2. H. Müller et al., "GAP9Shield: A 150GOPS AI-capable Ultra-low Power Module for Vision and Ranging Applications on Nano-drones," arXiv:2407.13706, 2024. (The sensing and computing module of a micro drone, with measured power data.)
3. D. Rossi et al., the PULP platform papers and the official site pulp-platform.org. (The technical source of the parallel ultra-low-power computing paradigm.)
4. The official Bitcraze Crazyflie documentation (bitcraze.io). (Hardware specifications, development environment, and community resources for this chapter's experimental platform.)
5. GreenWaves Technologies GAP9 product documentation. (Specifications and development tools for the RISC-V low-power processor.)
6. P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers, O'Reilly, 2019. (The classic introduction to TinyML.)
7. L. Crupi et al., "Training on the Fly: On-device Self-supervised Learning aboard Nano-drones within 20 mW," arXiv:2408.03168, 2024. (On-device learning aboard a micro drone, the advanced direction of Section 25.6.)

Summary

1. Running SLAM on a micro robot meets three constraints—computation, memory, and electrical power; the traditional practice is to offload the computation to external facilities, but this brings dependence on infrastructure, communication latency, bandwidth and power costs, and problems of reliability, so that the robot is never truly autonomous.
2. NanoSLAM is a lightweight and optimized end-to-end SLAM method designed to run on centimeter-sized robots within a power budget of only 87.9 milliwatts; it was deployed on a micro drone weighing 44 grams and carrying GAP9, a RISC-V low-power parallel processor, attaining a mapping accuracy of 4.5 centimeters and an end-to-end execution time under 250 milliseconds.
3. GAP9 is built on the parallel ultra-low-power (PULP) computing paradigm, trading multi-core parallelism for efficiency at a power below 180 milliwatts; it supports dynamic scaling of clock and voltage and automatic clock gating, with an idle power as low as 45 microwatts. The openness of RISC-V makes it customizable and free of licensing cost.
4. NanoSLAM replaces the LiDAR with four ToF matrix sensors, matches scans with a parallelized ICP (55 milliseconds onboard), and corrects drift by pose graph optimization (trajectory error reduced by up to 67 percent); its three strategies for keeping light are parallelize rather than simplify, sparse rather than dense, and divide the labor rather than do everything.
5. A micro drone fits in the palm of the hand, suits indoor spaces, and is safe around people; onboard SLAM is a necessary condition for swarm robots to scale; and the constraints on AR glasses are astonishingly like those on a micro

drone, which makes low-power SLAM a shared requirement of every constrained platform.

6. Edge AI and TinyML join with SLAM in three ways: enhanced perception (YOLO can complete an inference below 100 milliwatts in as little as 17 milliseconds), learning in place of computation, and online self-learning; the central methodology is the co-design of hardware and algorithm.
7. Simulating mapping with a micro drone suits a high school project admirably: the idea is intuitive, it can be simulated completely, and its levels are distinct; the recommended teaching rhythm is to build the perfect map first, then add drift deliberately to break it, and finally repair it by loop closure.

Carrying Forward

This chapter has compressed SLAM into a milliwatt budget and measured three sets of key numbers along the way: localization needs far less density of measurement than intuition suggests—32 ToF rays already come close to saturation (0.030 meters), and doubling them to 64 buys nothing; but range matters far more, and widening it from 1 meter to 4 improves the error nineteenfold. In hardware, at a fixed throughput many cores running slowly can bring the power down to 4.0 percent, but past about 33 cores leakage takes its revenge; on real chips GAP9's efficiency is 39 times that of the STM32. And at the level of the platform, the weight of a compute module affects endurance 4.7 to 10.6 times as much as its power does—in choosing a board the first question is how much it weighs, not how many watts it draws.

This chapter has displayed SLAM realized on a platform under extreme constraint, and has brought out an idea that runs through the whole book: the bottleneck in

performance lies more often in a method not yet optimized than in a want of computing power. The next chapter focuses on the challenges of drone navigation itself—how is one to fly in a GPS-denied environment? How are fast motion, vibration, and motion blur to be handled? How do optical flow, visual-inertial odometry, and visual SLAM divide the labor on a drone? Chapter 26 carries the discussion of Part VII to the heart of flight.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 26

Drone SLAM and Visual-Inertial Navigation

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 25 compressed SLAM into a milliwatt computing budget. This chapter changes the dimension: speed.

The typical speed of a ground robot is half a meter per second, and a drone may be ten times that or fifty. The difference is not merely running the same method faster—it blurs into a smear the clean corners on which Chapter 20 lives, it leaves no time at all to establish the mid-term data association on which Chapter 22 depends, and it has the propellers manufacture, in a way one would not expect, the very IMU bias discussed in Chapter 7.

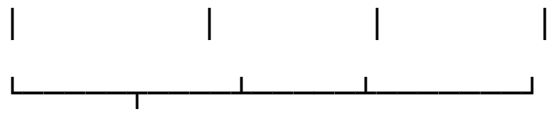
The three figures of this chapter measure exactly these things: Figure 26-1 compares the drift behavior of the three tiers—optical flow, VIO, and VI-SLAM; Figure 26-2 takes apart and measures the triple blow of Section 26.4 and finds which remedy is the most effective; and Figure 26-3 explains something counterintuitive—that raising the IMU sampling rate cannot save you from the aliasing vibration causes.

Figure 26-0 The Link Map of Chapter 26

The foundations built up so far

Chapter 7 The IMU and bias Chapter 20 Features Chapter 23 Visual-inertial SLAM

Chapter 25 Micro platforms



▼

【Chapter 26 The new problems flight brings: fast, shaking, and hard to see】

GPS-denied ─ optical flow / VIO / VI-SLAM ─ motion blur ─ vibration ─ graceful degradation

▼

Chapter 27 Multi-robot collaboration and the cloud-edge division of labor (closing Part VII)

▼

Part VIII ROS implementation　　Part IX AI and semantic navigation

Learning Objectives

In the last chapter we saw how SLAM is compressed into a milliwatt budget. This chapter focuses on the challenges of drone navigation itself: in the air the robot loses the support of the ground, possesses a full six degrees of freedom, moves far faster, and may be brought down by any error in localization. After studying this chapter, students should be able to:

1. Explain the four particular difficulties of localizing a drone as against a ground robot.
2. Explain how GPS-denied environments arise and what the typical settings are, and list the means of localization available.
3. Compare the levels of capability of optical flow, VIO, and visual SLAM, and where each is suitable.
4. Explain how fast motion, vibration, and motion blur affect visual localization, and propose remedies.

5. Plan the procedure and the safety measures for an indoor drone navigation experiment.
6. Explain the properties and the uses of the two great visual-inertial datasets, EuRoC and TUM-VI.
7. Explain the line of technical development from the micro drone to the vision-navigated drone.

Core Concepts

中文名詞	English Term	Explanation
微型飛行器	MAV	A small unmanned aircraft, the principal platform of visual-inertial navigation research.
GPS 拒止環境	GPS-denied	An environment where the satellite signal is blocked or jammed and cannot be used.
光流	Optical Flow	The vector field of the brightness pattern in an image moving over time.
光流定位	Optical Flow Positioning	Estimating horizontal velocity from the optical flow of a downward camera to hold a hover.
視覺慣性里程計	VIO	Coupling camera and IMU tightly to estimate motion; it accumulates drift.
視覺慣性 SLAM	VI-SLAM	VIO plus mapping and loop closure, which removes long-term drift.
動態模糊	Motion Blur	Blur in an image caused by the camera moving during the exposure.
特徵軌跡縮短	Shortened Feature Tracks	At high speed a feature leaves the field of view quickly and is tracked for a

中文名詞	English Term	Explanation
		shorter time.
滾動快門	Rolling Shutter	The sensor exposes row by row, which distorts the image at high speed.
全域快門	Global Shutter	The whole frame is exposed at once, which suits fast motion.
積極機動	Aggressive Maneuver	A flight maneuver of large angle and high acceleration.
光度校正	Photometric Calibration	Calibrating the camera's brightness response and vignetting; important for direct methods.
動作捕捉系統	Motion Capture	Laboratory equipment supplying millimeter-level true poses from external cameras.

26.1 The Particular Difficulties of Localizing a Drone

Section 3.6 compared the navigation requirements of five platforms; this section goes deeper into why the drone is the most severe of them.

Four Particular Difficulties

1. A full six degrees of freedom: a ground robot moves on a plane and has only three degrees of freedom (the x, y, θ of Section 10.3); a drone may translate and rotate as it pleases in three-dimensional space. With more states and more complex motion, the difficulty of estimation rises at once.
2. No odometry: this is the point most easily overlooked and yet absolutely crucial. A wheeled robot has wheel encoders supplying odometry (Chapters 3 and 13), which is a steady reference for motion even where it is imperfect; a drone in the air touches nothing and has no equivalent sensor at all. It can only integrate the IMU—and Section 7.3 told us that an IMU used alone drifts quickly.
3. High speed and violent dynamics: a drone flies far faster than an indoor ground robot, and its attitude changes violently. This leads directly to motion blur and to difficulty in tracking features (Section 26.4).
4. The consequences of an error are severe: a wheeled robot that localizes wrongly will at worst bump into a wall and stop; a drone that localizes wrongly may fall out of the sky, damaging the equipment and even injuring people. This makes the demand on the robustness of the system far higher than the demand on its accuracy.

The Double Constraint of Payload and Power

Chapter 25 set out the problems of a constrained platform in detail, and for a drone

they are sharper still: every additional gram of sensor weight consumes extra thrust and extra power, and flight itself has already consumed the overwhelming part of the charge. This constrains the choice of sensors on a drone severely—and it is the fundamental reason Section 9.5 called a camera together with an IMU the ideal combination for a drone.

The Severe Demand on Timeliness

If a ground robot's localization is half a second late, it merely stops half a beat behind; if a drone's localization is half a second late, it may already have moved several meters in flight, and the control loop loses its stability.

A drone system therefore tolerates latency very poorly. This explains why Chapter 25 said NanoSLAM must press its end-to-end execution time within 250 milliseconds, and why the attitude control loop of a high-grade flight controller runs at hundreds of hertz—which is exactly the value of the IMU's high-rate output in Chapter 7.

26.2 Navigation in GPS-Denied Environments

Chapter 5 discussed the principles and the limitations of GNSS in detail; this section focuses on the drone's position when the satellite signal is unavailable, and what can be done.

Typical GPS-Denied Settings

- Indoors: building materials block the satellite signal completely (Section 5.5). Inspection and stocktaking in factories, warehouses, shopping centers, and gymnasiums all belong here.
- Beneath bridges and structures: bridge inspection is an important drone application, but the deck itself is the obstruction.

- Tunnels and underground spaces: mines, sewers, underground car parks—no satellite signal at all.
- Urban canyons: the corridors between tall buildings mentioned in Section 5.5 not only block the signal but suffer severe multipath that produces a wrong position—which is more dangerous than no signal at all, because the system may not know that it is wrong.
- Under forest and vegetation: the canopy makes the signal weak and unsteady.
- Deliberate interference: the jamming and spoofing mentioned in Section 5.5 are a serious security question for a drone.

The Means of Localization Available

In a GPS-denied environment the means of localization a drone can rely on, ordered by reliability, are as follows:

Means	What it supplies	Limitations	Chapter in this book
IMU	High-rate attitude and motion	Drifts quickly when used alone	Chapter 7
Barometer	Relative height	Affected by air currents and temperature	This chapter
Optical flow (downward camera)	Horizontal velocity	Needs ground texture; supplies no absolute position	Section 26.3
Ranging sensor (downward)	Height above the ground	Unevenness of the ground causes misjudgment	Chapter 8
VIO (camera +	A full 6-DoF pose	Drifts; needs texture	Chapter 23

Means	What it supplies	Limitations	Chapter in this book
IMU)		and light	
VI-SLAM	Pose + map + loop closure	Heavier computation	Chapter 23
UWB / visual markers	An absolute correction of position	Infrastructure must be deployed beforehand	Chapter 4

Table 26-1 Means of Localizing a Drone in a GPS-Denied Environment

A Layered Strategy

A practical system combines these means in a layered design:

1. The bottom layer (hundreds of Hz): the IMU supplies attitude and keeps the flight stable. This layer may not be interrupted at all, or control is lost outright.
2. The middle layer (tens of Hz): optical flow and ranging supply velocity and height and sustain a hover in place. Even with no map the drone can stay where it is without drifting away.
3. The upper layer (10 to 30 Hz): VIO or VI-SLAM supplies a full pose and a map and sustains path planning and autonomous navigation.
4. The correcting layer (at irregular intervals): UWB, a visual marker, or a return to a region where GPS is available supplies an absolute correction of position (the fusion idea of Chapters 4 and 5).

The importance of this layering is fault tolerance: if the VIO of the upper layer fails, the drone can still hover on the optical flow of the middle layer and need not fall; and if the middle layer fails too, the IMU at the bottom can at least keep the attitude steady so that the operator may take over. The central principle of safety design is to let failure

degrade step by step rather than collapse in an instant.

26.3 Optical Flow, VIO, and Visual SLAM

This section sorts out the levels of capability of three visual localization technologies—they are often used interchangeably, but what they supply differs greatly.

Optical Flow: the Most Basic Layer

Section 9.4 introduced optical flow as the movement of the brightness pattern in an image over time. On a drone it has one extremely practical application: downward optical flow positioning.

The practice is to fit a downward-facing camera to the belly of the aircraft and compute continuously the speed at which the texture of the ground moves across the frame. Together with the height above the ground supplied by a ranging sensor, this converts into the drone's horizontal speed.

- What it supplies: horizontal velocity (not position).
- What it can do: hold a hover. If the optical flow shows the ground moving backward, the drone is drifting forward, and the flight controller corrects backward. This lets a drone hold its place steadily indoors with no GPS.
- Its limitations: it gives only velocity, which must be integrated to give position, so it still drifts; it needs the ground to have texture enough (over a plain white floor or water it fails); and an inaccurate measurement of height produces a wrong conversion of speed directly.

Optical flow positioning is standard equipment for the indoor hover of a consumer drone, costs very little computation, and is a good example of solving the most crucial

problem with the fewest resources.

VIO: a Full Six-Degree-of-Freedom Odometry

VIO couples camera and IMU tightly (Sections 7.6 and 23.3) and outputs a full 6-DoF pose and velocity. What it supplies is far more complete than optical flow, enough to sustain path following and autonomous flight.

But VIO is in essence still an odometry—it accumulates drift (the property of visual odometry noted in Section 20.1). Let the flight go on long enough and the estimated position departs from the true one.

Visual-Inertial SLAM: Adding a Map and Loop Closure

VI-SLAM adds mapping and loop closure detection on top of VIO (Chapter 23), and can correct the accumulated drift on revisiting the same place. The price is a rise in computation and in memory.

Technology	Output	Does it drift	Computation required	Typical use
Optical flow	Horizontal velocity	Yes, once integrated	Very low	Holding a hover
VIO	6-DoF pose + velocity	It accumulates	Moderate	Path following, autonomous flight
VI-SLAM	Pose + map + loop closure correction	Loop closure corrects it	Higher	Exploratory mapping, long tasks

Table 26-2 A Comparison of the Capabilities of the Three Visual Localization Technologies

How to Choose

- If the task is to hover and film: optical flow is enough, and saves both power and computation.
- If the task is to fly a prescribed route: VIO is needed to track position.
- If the task is to explore an unknown space and map it: VI-SLAM is needed.
- If the task is to inspect the same site repeatedly over long periods: VI-SLAM together with map reuse (the multi-session working of Atlas in Chapter 23) suits best.

This way of thinking about levels answers to a principle this book stresses repeatedly: do not use technology beyond what the task requires, because on a drone every ounce of computation and power is paid for in flight time.

Measuring It: How Much Does Each of the Three Tiers Drift

Table 26-2 distinguished the three technologies by the column headed does it drift, but did not say how much, or in what form. This subsection measures it with one complete flight simulation.

The setting is a drone flying six circuits of an elliptical route over 72 seconds at an average speed of about 2.5 meters per second. Each of the three estimators is modeled on its own physical properties: optical flow measures horizontal velocity in the body frame, which must be multiplied by the height measured by a ranging sensor to convert into true velocity (the height carries a 4 percent error) and then rotated back into the world frame by a heading integrated from the gyroscope (the heading drifts); VIO outputs relative poses directly and its error accumulates step by step; VI-SLAM sits on top of VIO and, on the completion of every circuit, forms a loop closure with the same position on the circuit before and performs pose graph optimization. Each setting is

run five times and the median taken.

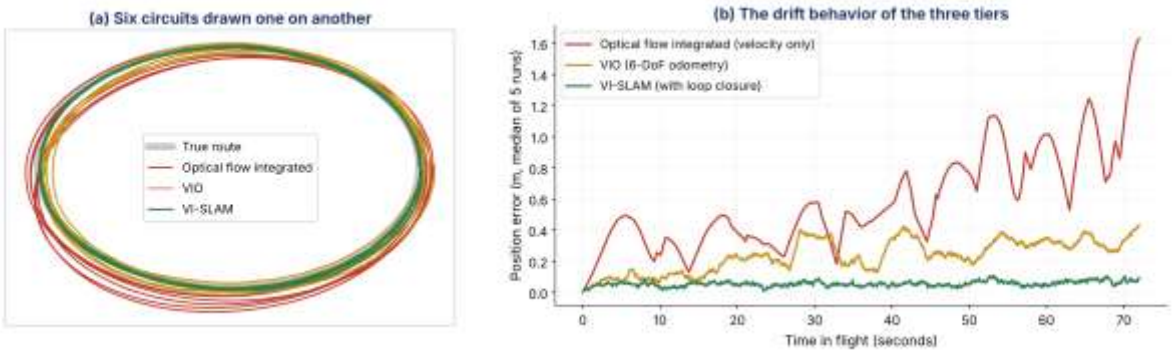


Figure 26-1 The Drift Behavior of Optical Flow, VIO, and VI-SLAM

Panel (a) is the six circuits drawn one on top of another. The green of VI-SLAM nearly coincides with the true route, and the six circuits drawn together show no divergence; the amber of VIO wanders outward circuit by circuit; and the red of optical flow wanders more plainly still, and in no fixed direction—which is what comes of a heading error turning the whole velocity vector.

Panel (b) is the position error against time. Over the second half (after 36 seconds) the mean errors are 0.810 meters for optical flow, 0.296 meters for VIO, and 0.060 meters for VI-SLAM; the peaks are 1.63, 0.43, and 0.11 meters. VI-SLAM is 5 to 13 times better than the other two.

But what deserves attention is the shape of the three lines rather than their height. The green line is flat from beginning to end—its error does not grow with the time in flight but is held by loop closure at a fixed level. The red and amber lines show a plain sawtooth: around each circuit the error first rises and then partly falls back. It falls back not because it has been corrected but because the route is closed, so that on coming round near the start the accumulated error happens to cancel in part—a coincidence, not a mechanism. Were the route one-way, these two lines would only climb.

This figure also shows the ground of those few recommendations under How to Choose in Section 26.3. Hovering requires no position but only velocity, so the drift of optical flow is no trouble at all; but let the task require flying to a given coordinate or coming back when done, and a scheme without loop closure will accumulate an error of meters within tens of seconds.

Why Optical Flow Is Worse Than VIO

On the face of it, an optical flow measurement is simpler and its error should be smaller. But it has two weaknesses VIO does not.

First, it gives only velocity and no attitude at all, so that turning the velocity into the world frame must borrow a heading integrated from the gyroscope—and an error in heading rotates the whole velocity vector directly, producing a position error proportional to the distance flown rather than to the square root of the time.

Second, its conversion of velocity depends on the measurement of height. This simulation gave the height only a 4 percent error, and that alone produced a 4 percent error of scale throughout. Should the drone fly over a table, the height the ranging sensor measures suddenly shrinks and the optical flow misjudges it as flying more slowly—which is the commonest cause of indoor drift in a consumer drone.

26.4 Fast Motion, Vibration, and Motion Blur

This section is the most central technical challenge of visual localization on a drone.

The Triple Blow

Research indicates that existing VIO systems degrade at high speed because of

motion blur, shortened feature tracks, and increased latency in the control loop. These three happen to form a vicious circle:

1. Motion blur: the camera moves during the exposure and the image blurs. At such speeds new problems emerge and the existing difficulties are aggravated—motion blur becomes more marked and damages the tracking of visual features. In a blurred image a corner becomes a smear and feature extraction fails outright (the features of Chapter 20 need clean corners).
2. Shortened feature tracks: in fast flight the same feature point sweeps across the field of view and is gone. The mid-term data association stressed in Section 23.2 is thus hard to establish, the system can rely only on short-term association, and accuracy falls.
3. Latency in the control loop: the visual processing itself takes time, and at high speed that delay corresponds to a larger displacement, so that the quality of control deteriorates and oscillation may set in.

What the Datasets Show of Reality

These challenges are plainly displayed on the standard benchmark datasets. Research has observed that because of the drone's fast motion the images are somewhat blurred, and markedly so in the difficult sequences MH_04, MH_05, V1_03, and V2_03.

More interesting still is one detail: the machine-hall sequences of EuRoC all begin with fast motion to assist IMU initialization (Section 23.4), but the motion blur this produces is a great challenge to visual tracking; the authors of DSO therefore cut off the beginning and use only the part where the drone is in the air, whereas for a VIO system IMU initialization is indispensable and the beginning cannot be discarded.

This detail shows two things at once. First, IMU initialization needs excitation of motion (as Section 23.4 stressed repeatedly), but the excitation itself causes blur, so that the two pull against each other. Second, different methods may be evaluated under different conditions on the same dataset, and students should be alert to such details when reading papers and comparing figures.

The Order of Magnitude of Speed

To give students a concrete idea of what fast means: the EuRoC micro air vehicle dataset seldom exceeds 5 meters per second; a racing drone dataset may reach 28 meters per second, but usually with RGB cameras on a structured indoor course or a short outdoor track. Research has supported up to 17.5 meters per second with a stereo RGB VIO, but it depends on stereo vision and presumes favorable lighting.

By contrast, an indoor ground robot usually moves below half a meter per second—the drone is ten times faster, or tens of times. This is why visual localization on a drone is so much harder.

Five Remedies

1. Raise the shutter speed: a shorter exposure reduces blur directly, at the price of a darker image, which calls for a better sensor or a brighter environment.
2. Use a global shutter: a rolling shutter exposes row by row and skews the image under fast rotation; a global shutter exposes the whole frame at once and is a necessary choice for high-speed applications.
3. Lean on the IMU: research has integrated an IMU-based motion prediction module to improve feature tracking across frames, particularly during fast motion. The IMU can predict where a feature ought to have moved to, which

narrows the search greatly and raises the rate of successful tracking. This is the concrete realization of Section 7.6's the IMU supplies vision with an initial value.

4. Enhance the image: preprocess a blurred or low-contrast image. Research has combined contrast-limited adaptive histogram equalization (CLAHE), gradient filtering, and descriptor verification to maintain stable feature tracks under motion blur.
5. Adaptive weighting: adjust the relative trust in vision and inertia dynamically according to image quality. Research has validated an adaptive covariance method on the EuRoC and TUM VI datasets and in field tests, markedly improving a drone's navigation accuracy in situations with motion blur.

The Problem of Vibration

Besides the speed of flight, the high-frequency vibration caused by the propellers and motors is a problem peculiar to drones:

- On the IMU: vibration contaminates the accelerometer readings, and if its frequency approaches the sampling rate it may even cause aliasing. In practice mechanical isolation (soft rubber mounts) and filtering are required.
- On the camera: the slight blur caused by high-frequency vibration lowers the quality of features, and is usually relieved by a damped mount and a faster shutter.
- On the mounting: if the relative position of sensor and airframe (the extrinsics, Chapter 10) works loose and changes through vibration, the accuracy of the whole system suffers.

Measuring It: How Much Does Each of the Triple Blows Take Away

Section 26.4 gathered the difficulties of fast flight under three heads: motion blur,

shortened feature tracks, and latency in the control loop. This section measures the first two separately, and checks along the way which of the five remedies is the most effective.

The method of the experiment: synthesize an image of the ground carrying texture at several scales together with geometric corners (simulating brick joints, floor markings, and a door frame), compute the corresponding pixel displacement from speed × exposure time × focal length ÷ depth, apply a linear motion blur, then detect with ORB and cross-match, counting how many features can still be matched correctly against the clean image. The camera parameters are those of the wide-angle configuration common on drones (a focal length of 380 pixels), with a scene depth of 4 meters.

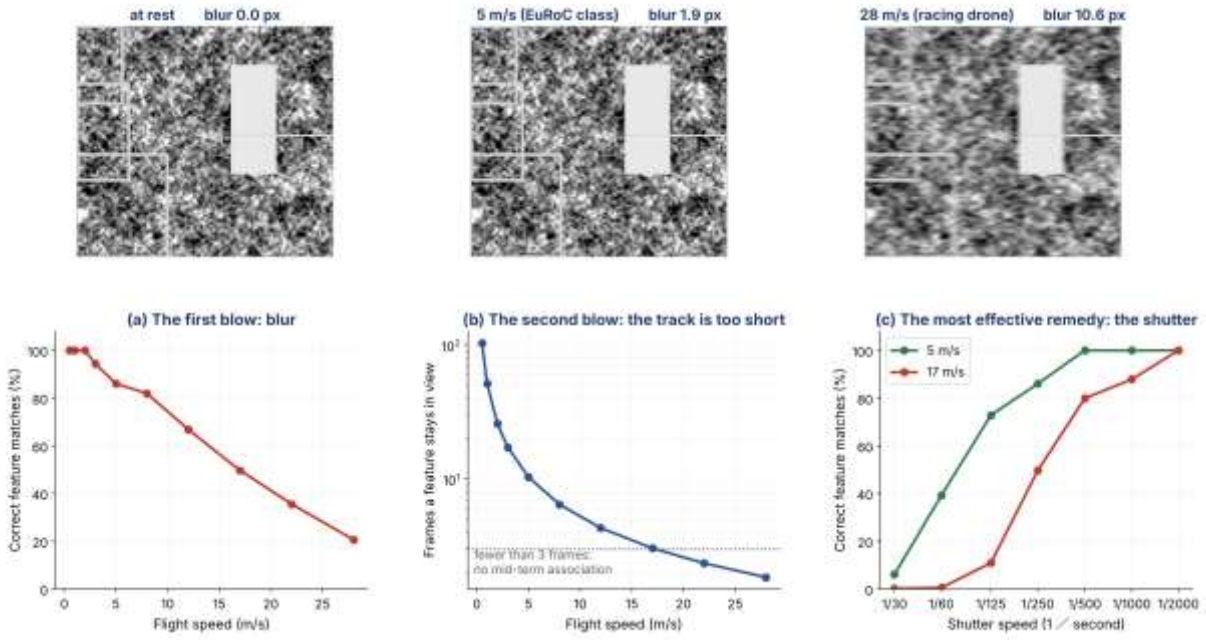


Figure 26-2 The Triple Blow of Speed, and the Most Effective Remedy

The top row is the same patch of texture at three speeds. At rest the corners are clean; at 5 meters per second (the typical speed of the EuRoC micro air vehicle dataset) the blur is 1.9 pixels and the edges can already be seen to soften with the naked eye; at 28 meters per second (a racing drone) the blur is 10.6 pixels, the fine texture has all but

vanished, and only large patches of light and dark remain.

Panel (a) quantifies the first blow. Within 2 meters per second the feature matching rate is still 100 percent; at 3 meters it is 94 percent, at 5 meters 86 percent, at 12 meters 67 percent, at 17 meters 50 percent, and at 28 meters only 21 percent remains. Note that the turning point falls between 2 and 5 meters per second—which is exactly the cruising speed of an ordinary indoor drone, and explains why the difficult sequences of EuRoC are precisely those flown faster.

Panel (b) quantifies the second blow, and this one is the more deadly. The vertical axis is how many frames a feature point can be seen in from entering the field of view to leaving it (the camera runs at 30 fps). At half a meter per second it is 104 frames (3.5 seconds); at 2 meters 26 frames; at 5 meters 10 frames; at 17 meters only 3; and at 28 meters—only 2.

Two frames is what? It means the feature is detected in this frame and is already off the screen in the next but one. With barely a second observation there is no reliable triangulation (Section 21.5 requires parallax), no mid-term data association (Section 23.2), and no sound way to create a map point. In other words, at 28 meters per second visual SLAM can hardly proceed even if the images are perfectly clean. Blur merely adds frost to the snow.

Panel (c) tests the first remedy mentioned in Section 26.4, raising the shutter speed. Take 17 meters per second: at a shutter of 1/30 second the matching rate is 0.2 percent, at 1/60 0.5 percent, at 1/125 10.9 percent, at 1/250 49.7 percent, at 1/500 79.9 percent, at 1/1000 87.9 percent, and at 1/2000 100 percent. Going from 1/125 to 1/500 takes the matching rate from one tenth to eight—the cheapest of the five remedies and the

one that acts most immediately.

The price is there too, of course: double the shutter speed and half the light is lost. High-speed visual localization on a drone is therefore in practice an optical problem—either the environment must be bright enough, or one must change to a fast lens and a sensitive imager. This is also why research on racing drone vision is mostly confined to a structured indoor course or an outdoor track in good light.

> **A Practical Suggestion for a Project**
>
> Setting panels (a) and (b) side by side gives a very clear upper limit on speed: within 2 meters per second both the blur and the track length are still healthy (100 percent matching, 26 frames and more); 5 meters per second is the usable boundary (86 percent, 10 frames); and beyond 10 meters per second one must deal with the shutter, the light, and the algorithm all at once.
>
> In planning a drone project, rather than pursue speed, students should first hold the speed within 2 or 3 meters per second and make the localization and the mapping steady; speed is the last parameter that should be relaxed.

Measuring It: What Does the Vibration of the Propellers Turn Into

Section 26.4 ended by noting that vibration contaminates the accelerometer readings, and that if its frequency approaches the sampling rate it may even cause aliasing. That sentence deserves unfolding, because the consequence of aliasing is far more serious than noise growing larger.

The experimental setting: superpose propeller vibration (a fundamental plus a second harmonic, of amplitude 2.5 meters per second squared) on a slowly varying true acceleration signal, sample at 400 Hz or at 1000 Hz, and compare the mean difference

between the sampled signal and the truth—a mean difference which, in the eyes of the estimator downstream, is simply an accelerometer bias.

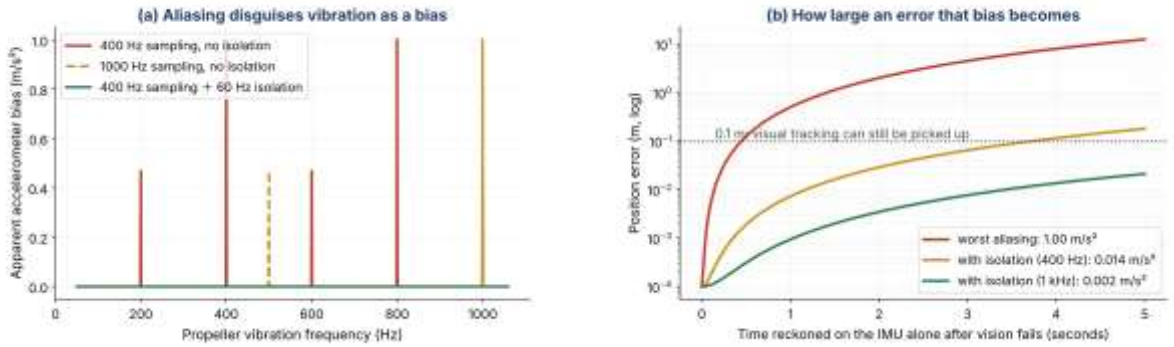


Figure 26-3 The Aliasing of Vibration, and Why It Disguises Itself as a Bias

Panel (a) sweeps the vibration frequency. The red line is 400 Hz sampling with no isolation: at most frequencies the apparent bias is near zero, but at 200, 400, 600, and 800 Hz it leaps suddenly to 1.00 meters per second squared. These are exactly the integer and half-integer multiples of the sampling rate—at these frequencies the vibration is folded down into a DC component, which is to say a constant offset that looks exactly like a bias.

The amber dashed line raises the sampling rate to 1000 Hz. The positions of the spikes change, but their height does not change at all—still 1.00 meters per second squared. This is the most important message of the figure: raising the sampling rate cannot cure aliasing; it can only move the problem to other frequencies. This is a fairly common misunderstanding.

The green line adds mechanical isolation with a cutoff at 60 Hz (a soft rubber mount plus an analog low-pass). The greatest apparent bias falls from 1.00 to 0.014 meters per second squared, some seventy times smaller. The key is that this filtering happens before the sampling—the high-frequency content is stopped before it ever reaches the

analog-to-digital converter, and so never has the chance to be folded down. Digital filtering after sampling is wholly useless against aliasing, because the signal is already spoiled.

Panel (b) shows how serious that 1.00 meters per second squared is. A bias integrated twice is a position error: a visual outage of 1 second produces an error of 0.50 meters, of 2 seconds 2.0 meters, and of 4 seconds 8.0 meters. Set beside the conclusion of Figure 23-1 in Chapter 23—that with the bias estimated to ±0.01 the error after 2 seconds is only 0.025 meters—this false bias from vibration is eighty times worse than the best case discussed there.

More troublesome still, this offset is not random but changes with the rotor speed. As the drone accelerates the rotor speed rises and the vibration frequency moves with it, so that the false bias may leap from zero to 1.00 and drop back to zero. The IMU bias estimation of Section 23.4 assumes that the bias varies slowly, and against a jump of this kind it is wholly powerless. Isolation is therefore not a detail of tuning but a precondition of the system working at all.

A Check That Can Be Done in the Laboratory

Fix the drone to a table (with the propellers removed or a guard fitted), raise it slowly from idle to full throttle, and record the raw accelerometer readings throughout.

Then compute the mean over each band of rotor speed. If the mean departs plainly from gravity at a few particular speeds, that is aliasing—and you have just found the bands of rotor speed that must be avoided.

A fuller practice is to take a fast Fourier transform of the raw signal, see where

the propellers' fundamental and harmonics fall, and choose the softness of the damped mount (which is to say its cutoff frequency) accordingly. This experiment needs only a drone and a recording program, and can save weeks of debugging afterwards.

26.5 An Indoor Drone Navigation Experiment

This section explains how to plan a drone indoor navigation experiment that is both safe and instructive, which is also the foundation of a project.

Safety First: What Must Be Said Before Anything Else

The greatest difference between a drone experiment and a ground robot experiment is the risk. The following principles are not to be compromised:

- A net or a cage: an indoor experiment should be conducted in a space with a safety net, or with a model whose propellers are guarded by a frame.
- Choose a micro platform: the micro drones introduced in Chapter 25 weigh only tens of grams, and even out of control and striking a person will scarcely cause injury. For a teaching experiment, starting from a micro platform is strongly recommended.
- An emergency stop: the operator must be able to cut the power with one key at any moment, and this function must not depend on the autonomous system that is being debugged.
- Test in stages: verify in simulation first, then test on a tether, and only then fly freely. Skipping a stage is the principal cause of accidents.

- Where people stand: during the experiment everyone not needed should be outside the net, and the operator should stand behind the drone and not in front of it.

Preparing the Site

- Texture: the floor and the walls need visual texture enough (the curse of the white wall of Chapters 9 and 20 holds on a drone just as well). Patterned mats may be laid on the floor if necessary.
- Light: even and ample, avoiding backlight and strong contrasts of light and dark. A window may cause overexposure, and drawing the curtains to keep the lighting steady is advised.
- Air currents: avoid air conditioning blowing directly or a draft through an open window, which disturbs the stability of a micro drone severely.
- A ground truth system (optional): a motion capture system supplies millimeter-level true poses for quantitative evaluation (Section 26.6). Failing that, the AprilTag of Chapter 4 can supply an approximate ground truth (the fourth use in Section 24.3).

The Procedure of the Experiment

1. Check the system: confirm that the IMU calibration, the camera calibration (Section 9.2), the sensor extrinsics (Chapter 10), and the communication link are all in order.
2. Test the hover: the most basic ability of all. Start optical flow positioning, let the drone hover in place for 30 seconds, and measure the drift in position. If the hover is not steady, there is no point attempting autonomous flight afterwards.

3. Initialize VIO: following Section 23.4, complete the IMU initialization with a motion carrying acceleration, deceleration, and rotation, and confirm that the scale estimate is correct (a known distance will verify it).
4. A simple route: set a simple square or straight route, let the drone fly it autonomously, and record the trajectory.
5. A loop closure test: let the drone go once around and return to the start, measure the discrepancy between end and start, and assess the accumulated drift (the evaluation method of Section 13.7).
6. A challenge test: raise the difficulty step by step—increase the speed, add sharp turns, cover the camera briefly—and observe under what conditions the system fails. This is the most instructive part of all.

Recording the Data

Every flight should be recorded in full: images, raw IMU data, flight controller state, estimated trajectory, and (if available) the true trajectory. These data can be analyzed offline again and again, and can serve as a dataset of your own making—for project research, a dataset of your own site is itself a valuable result.

26.6 Visual-Inertial Datasets: EuRoC and TUM-VI

Research on drone SLAM can be done without drone hardware—which is exactly the value of a public dataset. This section introduces the two great standard datasets.

The EuRoC MAV Dataset

Case 7-3 in Chapter 7 introduced EuRoC already; details are added here. It contains eleven sequences recorded on a micro air vehicle, supplying stereo camera images, IMU measurements, and true poses from a motion capture system.

Each sequence covers 30 to 60 seconds of flight, contains high-fidelity 6-DoF ground truth poses from external motion capture together with a high-resolution point cloud scan of the operating environment, so that both trajectory estimation and dense three-dimensional reconstruction may be evaluated against geometrically consistent reference data. The typical evaluation procedure passes 20 Hz monocular images and 200 Hz IMU data to a feature-based VIO (ORB-SLAM3 with loop closure turned off, say), producing a camera trajectory at true scale together with sparse three-dimensional landmarks.

- The classes of sequence: the machine hall (MH) sequences were recorded in an industrial plant, the room (V1, V2) sequences in a laboratory with motion capture.
- The grades of difficulty: every group of sequences is graded easy, medium, and difficult, the difficulty coming from speed, from light, and from how violent the motion is. This lets students test an algorithm step by step.
- Advice on use: start from MH_01_easy to build up an experience of success, then take on difficult sequences such as V1_03_difficult.

The TUM-VI Dataset

TUM-VI is another widely used benchmark. It supplies camera images of 1024 × 1024 resolution at 20 Hz with high dynamic range and photometric calibration; the IMU measures three-axis acceleration and angular rate at 200 Hz, and camera and IMU are time-synchronized at the hardware level.

By comparison the EuRoC MAV sequences are shorter and less varied, containing recordings of only one machine hall and one laboratory; EuRoC MAV moreover carries no photometric calibration, which matters considerably for evaluating direct methods

(Section 20.2).

- Its distinctive features: it covers a variety of scenes indoors and out, its sequences are longer, it has photometric calibration, and its time synchronization is done at the hardware level.
- Relevance to AR / VR: Section 23.7 quoted ORB-SLAM3's millimeter-level accuracy on the TUM-VI room sequences, which are exactly the room scenes of this dataset.

Property	EuRoC MAV	TUM-VI
Platform	Micro air vehicle	Handheld sensor rig
Images	Stereo, 20 Hz	Stereo 1024×1024, 20 Hz
IMU	200 Hz	200 Hz, hardware time synchronization
Ground truth	Motion capture / laser tracker	Motion capture (some sequences)
Photometric calibration	None	Yes
Variety of scenes	One machine hall + one laboratory	A variety of scenes indoors and out
Additional data	A point cloud scan of the environment	High dynamic range images

Table 26-3 A Comparison of the EuRoC and TUM-VI Datasets

Advice on Using Datasets

- Comparing algorithms: compare different systems on the same dataset (as in Implementation 23-1 of Chapter 23), and only then are the figures comparable.

- Mind the evaluation conditions: as Section 26.4 noted of whether the beginning is cut off, the conditions of evaluation may differ from paper to paper, and comparing numbers directly calls for caution.
- A dataset of your own: a public dataset cannot cover your particular site. Recording data at a university site by the method of Section 26.5 is a contribution of real value for a project.
- Newer datasets: datasets of greater speed and more sensing modalities have appeared in recent years, such as racing drone datasets (up to 28 meters per second) and datasets containing thermal imagery, which suit advanced research.

26.7 From NanoSLAM to the Vision-Navigated Drone

This section joins Chapters 24 and 25 together and traces the development of autonomous drone navigation and the directions ahead.

Three Levels of Drone Autonomy

Level	Capability	Key technology	Representative platform
Stable	Does not drift away; can hover	IMU + optical flow + ranging	Consumer drone
Autonomous	Can fly a route and avoid obstacles	VIO + path planning	Professional drone
Intelligent	Can explore, map, and understand a task	VI-SLAM + semantics + VLM	Research platforms, future products

Table 26-4 Three Levels of Autonomous Capability in a Drone

This table of levels can help students place the aim of their own project. Most teaching projects should confirm the foundations at the stable level and then take on the autonomous level; the intelligent level suits graduate work or a long-running project.

Three Main Lines of Technical Development

1. Computation settles downward: from computing on an external computer (offloading), to high-performance onboard computing (Jetson, say), to the milliwatt-level dedicated chip of Chapter 25. This line lets autonomous capability enter smaller and cheaper platforms.
2. Sensing grows lighter: from the heavy LiDAR, to a camera with an IMU, to the ToF matrix sensors of Chapter 25. This line lowers the threshold in weight and in power.
3. Intelligence rises: from geometric navigation, to semantic understanding (Chapter 24), to task-level autonomy driven by a VLM. This line raises the complexity of the task a robot can carry out.

It is worth noting that these three lines are converging: the micro platform of Chapter 25 can already run YOLO object detection (below 100 milliwatts, in as little as 17 milliseconds), which means a drone that is small, frugal, and capable of semantic understanding is becoming a reality.

Project Directions at a University Site

- Indoor survey: scan the interior of a departmental building with a micro drone (a high atrium, a stairwell) and produce a three-dimensional model. These are

spaces a ground robot can hardly reach, and they display the drone's distinctive value.

- Facility inspection: checking rooftop solar panels, exterior walls, and high pipework—a drone can safely take the place of a person working at height.
- Agricultural applications: answering to Case 25-3 in Chapter 25, Taiwan's orchards and greenhouses are ideal sites for a micro drone.
- A teaching dataset: record and publish a visual-inertial dataset of a university site for the students and researchers who come after. This is relatively easy to begin and yet yields value over a long time.
- Ground-air collaboration: let a robot dog (Chapters 18 and 24) and a drone explore together—the ground robot has long endurance and a large payload, the drone a wide view and the ability to go high. This is a concrete application of the multi-robot collaboration of Chapter 27.

Advanced Directions

A few research directions worth watching, for students minded to go deeper:

- Perception in fast flight: as Section 26.4 described, existing VIO degrades at high speed. Breaking the speed limit with unconventional sensors such as a thermal camera is an active direction of research.
- Joining learning to geometry: Section 20.7 noted that deep-learned features are entering SLAM. Research has integrated a deep feature extractor and a sparse feature matcher with traditional hand-crafted keypoint detection and optical flow tracking, accelerating the inference with TensorRT, and improved the performance of front-end tracking under fast camera motion and changing light.

- Learned inertial odometry: a learning method on the IMU alone can already approach the performance of VIO in certain situations—an important fallback where vision fails (darkness, smoke).
- Swarm collaboration: several micro drones mapping and exploring together is the subject of Chapter 27, and the last piece of this part.

At this point the central challenges and the technical lines of drone navigation have been presented in full. The next chapter widens the view from a single machine to a group: how do several robots map together? How are their separate maps fused? How do the cloud and the edge divide the labor? And what questions arise in communication, synchronization, and security? Chapter 27 completes the last piece of Part VII.

26.8 The State of Visual Localization on Drones: Where Is the Limit of Speed

This chapter has measured a great many places where things break. By way of closing, this section sets out the present boundary of the technology and three roads through it.

One: the Three Speed Bands of the Present State

Speed band	Typical setting	The state of visual localization	Chief limitation
Within 3 meters per second	Indoor inspection, filming	Mature; VI-SLAM runs steadily	Computation and endurance
3 to 10 meters per second	Outdoor inspection, survey	Workable, with the shutter and isolation to match	Motion blur

10 to 20 meters per second	Racing, fast search and rescue	At the research stage; needs stereo and good light	Feature tracks too short
Above 20 meters per second	The limit of racing	Vision can hardly carry it alone	Gone in a single frame

Table 26-5 Four Speed Bands of Visual Localization on a Drone

The dividing points of this table are exactly the two turnings Figure 26-2 measured: between 3 and 5 meters per second blur begins to bite, and at about 17 meters per second the feature track falls below three frames. The two limits are of different natures—the first can be relieved by optical means, the second is a hard geometric limit.

Two: Three Roads Through

The first is to change the sensor. An event camera outputs no frames but an event at each pixel whenever its brightness changes, with a time resolution down to microseconds, so that in principle motion blur does not exist. The price is that the form of its output is entirely different, and the whole apparatus of Chapters 20 to 23 must be rewritten.

The second is to let the IMU carry more. Section 26.4 noted IMU-based motion prediction improving feature tracking across frames, which is in essence using inertial information to predict where a feature ought to have moved to, narrowing the search from the whole image to a small window. At high speed this is of particular value—but it presumes that the IMU bias is estimated accurately, and Figure 26-3 has just shown how vibration destroys exactly that. The two hold each other in check, and this is the thorniest part of designing a drone system.

The third is to accept the limit and change the task. Since vision cannot hold up at high

speed, keep attitude at high speed on the IMU and the barometer alone, and slow down periodically to 2 or 3 meters per second for one visual correction of position. This is the same line of thought as the map maintenance of the inspection robot in Section 19.6 and the event-driven computation of Section 25.8—not to have the system meet the highest standard at every instant, but to have it meet the standard when it must.

For a student project the third road is the most practical. It needs no special hardware and no rewriting of the algorithm, only the writing of when to slow down into the plan of the task.

Formulas and Procedures

One: Converting Optical Flow to Speed

What a downward camera measures as optical flow is how many pixels the image moves per second; to convert it into a true horizontal speed one needs the height above the ground and the focal length of the camera:

$$v = (\text{flow_pixels} / f) \times h$$

where flow_pixels is the pixel displacement per second, f the focal length (in pixels, the intrinsics of Section 9.1), and h the height above the ground (supplied by the ranging sensor).

This formula reveals one crucial risk: the height h appears in the numerator. If the ranging sensor misjudges the height (if the height suddenly shrinks while flying over a tabletop, say), the converted speed is wrong, and the flight controller may make a wrong correction and drift or even collide. This is the commonest cause of failure in optical flow positioning, and students must mind the unevenness of the floor in their

experiments.

Two: Quantifying Motion Blur

The length of the blur on the image during the exposure is approximately:

$$blur_pixels \approx (v / h) \times f \times t_exposure$$

where v is the flight speed, h the distance to the scene, and t_exposure the exposure time.

Put in real numbers for the feel of it: at a flight speed of 5 m/s, a distance of 5 meters, a focal length of 400 pixels, and an exposure of 10 milliseconds, the blur is about 4 pixels—already enough to affect corner detection seriously (the features of Chapter 20 need clean local structure). Raise the speed to 20 m/s and the blur reaches 16 pixels, at which feature extraction fails almost entirely.

The formula also points directly to the remedy: shortening the exposure time t_exposure reduces the blur linearly—which is exactly the mathematical ground of the first remedy of Section 26.4.

Three: The State and the Error Terms of VIO

The state vector of a drone's VIO is the same as in Section 23.3, and for every key frame is:

$$S = \{ R, p, v, b_g, b_a \} \quad (15 \text{ degrees of freedom in all})$$

Tightly coupled optimization minimizes the visual and the inertial error together:

$$\min \Sigma \rho(|| e_visual ||^2) + \Sigma || e_inertial ||^2$$

For a drone one practical point is the dynamic adjustment of the two weights: when blur or a shortage of features is detected, the weight of the visual term should be

lowered and that of the inertial term raised—which is exactly how the fifth remedy of Section 26.4 (adaptive weighting) is realized, and answers to the central idea of the Kalman gain in Chapter 11, that of weighting by credibility.

Four: A Layered Architecture for GPS-Denied Drone Navigation

Figure 26-4 The Layered Architecture of Drone Navigation and Its Graceful Degradation

【Correcting layer】at irregular intervals: UWB / visual markers / return to a GPS region → an absolute correction of position

↓ degrade on failure

【Upper layer】10–30 Hz: VI-SLAM / VIO → a full 6-DoF pose + a map

sustains: path planning, autonomous navigation, exploratory mapping

↓ degrade on failure

【Middle layer】tens of Hz: optical flow + ranging → horizontal velocity + height above the ground

sustains: holding a hover (at least not drifting away)

↓ degrade on failure

【Bottom layer】hundreds of Hz: IMU → attitude

sustains: keeping the flight steady so the operator may take over

【Design principle】let failure degrade step by step rather than collapse in an instant

Cases

Case 26-1 The Price of Having No Wheels: Why a Drone Is Harder Than a Ground Robot

A student who has done SLAM on both a ground robot and a drone will feel keenly how much harder the latter is. The reason is not merely that it flies faster, but a structural difference often overlooked: a drone has no odometry.

Recall the line of this book: Section 13.4 explained that 2D SLAM works in a cycle of prediction by odometry and correction by LiDAR; Section 14.5 noted that of Gmapping's three preconditions odometry is necessary; and the LOAM of Section 18.3 likewise needs a motion estimate as the initial value for registration. Odometry drifts, but it supplies a steady, continuous, free reference for motion.

A drone has no such luxury at all. Its only internal source of motion is the IMU, and Section 7.3 has already proved that a consumer-grade IMU integrated alone drifts by meters within seconds. This means that a drone's visual system must bear alone the responsibility odometry used to share—and once vision fails, the system has no reliable fallback.

This explains three things: why the drone field sets such store by the tight coupling of VIO (Chapter 23), why optical flow positioning is standard equipment on a consumer drone (Section 26.3), and why the layered degradation architecture of Section 26.2 is so crucial.

The lesson for students is this: understanding what a platform lacks often explains the trade-offs of its system design better than understanding what it has. This way of reasoning backward from constraints to design is the core of engineering judgment.

Case 26-2 The Dilemma of Initialization: Too Little Motion Will Not Do, and Neither Will Too Much

This is a contradiction of great teaching value, and one students are certain to meet in practice.

On the one hand, Section 23.4 stressed repeatedly that IMU initialization needs excitation of motion enough, without which scale and bias are unobservable. At rest or moving in a straight line at constant velocity, the system simply cannot initialize.

On the other hand, the instance mentioned in Section 26.4 shows that the machine hall sequences of EuRoC all begin with fast motion to assist IMU initialization, but that the motion blur this produces is a great challenge to visual tracking. Fast motion meets the need of the inertial system while spoiling the quality of the visual system's input.

That this contradiction exists shows exactly what visual-inertial fusion is in essence: the needs of the two sensors do not agree and even pull against each other. The IMU likes motion with acceleration in it; the camera likes a steady, clean picture.

The practical solution is a compromise: make a motion of moderate amplitude, with acceleration, deceleration, and rotation, but not too fast. Concretely, hold the device or fly the drone back and forth and side to side in small movements while turning for a few seconds, keeping the speed below the point at which the picture blurs noticeably.

This case also reminds students that when a method from a paper will not run in your hands, the trouble need not be in the program but may be that your data do not meet the method's implicit assumptions. Understanding what a method requires of its input is a precondition of implementing it successfully.

Case 26-3 The Boundary of Speed: How Fast Can VIO Fly

The answer to the question of how fast a flight visual localization on a drone can support has been pushed forward steadily in recent years, and displays the development of the technology clearly.

Take the baseline first: the EuRoC micro air vehicle dataset seldom exceeds 5 meters per second—which is the evaluation condition of most VIO papers. Against that baseline, most of the accuracy figures students see were obtained at relatively gentle speeds.

Pushing upward: research has supported up to 17.5 meters per second with a stereo RGB VIO, but it depends on stereo vision and presumes favorable lighting. A racing drone dataset may reach 28 meters per second, but usually with RGB cameras on a structured indoor course or a short outdoor track.

Breaking the limit: more advanced research has begun to use unconventional sensors. One work achieved real-time operation at 30 meters per second in low-altitude flight with a monocular thermal-inertial odometry. The advantage of thermal imagery is that it is unaffected by light and is less sensitive to certain sources of blur.

But this road has its price too. Research points out that although methods based on deep learning show good gains in performance on thermal imagery, their inference time remains incompatible with the strict limits of latency and power on a drone's onboard hardware—certain methods need more than 30 milliseconds per frame even on a laptop-class GPU. A hand-designed pipeline therefore remains the most practical solution for embedded high-speed closed-loop flight.

This case distills a theme that appears again and again in this book: the splendid

figures of an advanced technology often rest on particular conditions, while a system that can really be deployed must satisfy accuracy, speed, power, and reliability all at once. Learning to read a paper for what was achieved and under what conditions at the same time is an important part of the ability to do research.

Implementation Activities

Implementation 26-1 A Drone Visual-Inertial Navigation Experiment

Purpose: to understand the challenges of visual localization on a drone and what can be done about them, through a public dataset and (optionally) real flight. Parts One and Two need no hardware and suit everyone; Part Three needs a micro drone platform.

Part One: Analyzing a Dataset (everyone; no hardware needed)

1. Download the EuRoC dataset, choose the two sequences MH_01_easy and V1_03_difficult, run each in the stereo-inertial mode of ORB-SLAM3 from Chapter 23, and compute the ATE with evo.
2. Compare the difference in accuracy between the two, and play the images to see in which passages of the difficult sequence motion blur appears plainly, setting this against the discussion of Section 26.4.
3. Quantify the blur: write a program to compute the sharpness of every frame (the variance of the Laplacian will serve), draw the curve of sharpness against time, and mark the blurriest passages.
4. An analysis of correlation: draw the sharpness curve together with the curve of trajectory error and see whether the two are related—is the moment of blurriest image also the moment of greatest error?

5. Observe the initialization: watch the fast motion at the start of the sequence, record the time the system needs to complete IMU initialization, observe how blurred the images are over that stretch, and verify the dilemma of Case 26-2 for yourself.

Part Two: Simulating Optical Flow and Blur (everyone)

1. Using OpenCV's Lucas-Kanade optical flow, compute the flow on a downward-looking video (a phone moved horizontally above a tabletop will serve) and convert it into a horizontal speed by Formula One.
2. A sensitivity experiment on height: set the height wrongly on purpose (enter 0.5 meters where it is really 1 meter), observe the error in the converted speed, and understand the risk of failure mentioned in Section 26.5.
3. Simulate blur: add motion blur artificially to a clean image (with OpenCV's motion blur kernel), the blur length computed by Formula Two, and test how the number of ORB features detected falls as the blur grows.
4. Draw the curve of blur length in pixels against the number of features detectable, and find the critical point at which the number of features falls away sharply. That critical value is the ground of the upper speed limit your system can tolerate.

Part Three: Real Flight (for classes with a micro drone platform)

Read the safety principles of Section 26.5 first without fail, and work inside a net with a micro platform.

1. Test the hover: start optical flow positioning, let the drone hover in place for 30 seconds, and measure the extent of the horizontal drift against markings on the floor.

2. An experiment on ground texture: hover in turn over a patterned carpet, a plain floor, and a reflective floor, compare the extent of the drift, and verify optical flow's dependence on texture.
3. An experiment on height: hover at several heights above the ground (0.5, 1.0, 1.5 meters), observe the difference in steadiness, and think about the effect of an error in the height in Formula One.
4. A simple route: set a square route to be flown autonomously, measure the discrepancy on returning to the start, and assess the accumulated drift.
5. A challenge test: raise the flight speed step by step, safety permitting, record the speed at which the system begins to be unsteady, and set this against the discussion of Case 26-3.

Discussion Questions

1. By what factor do the ATEs of the easy and the difficult sequences differ? How well are image sharpness and trajectory error correlated?
2. By Formula Two, if your camera's exposure is 5 milliseconds, its focal length 400 pixels, and the scene 3 meters away, at what flight speed does the blur reach 5 pixels? What does this imply for an upper speed limit?
3. In the optical flow conversion experiment, how much error in speed does a 50 percent error in height produce? If the drone corrects on that wrong speed, what will happen?
4. (For those who fly) Over which floor was the hover steadiest? What made the least steady one least steady? Which limitation of Section 26.3 does this bear out?

5. By the layered architecture of Figure 26-1, if your drone's VIO fails suddenly in flight, how should the system respond? Design a procedure for degrading gracefully.

Extended Challenge (university level)

First: implement adaptive weighting—add an assessment of image quality to a VIO system (the sharpness index of Part Two, say), lower the weight of the visual term automatically and raise that of the inertial term when the image is blurred, and test on the EuRoC difficult sequences whether the accuracy improves. This corresponds to the fifth remedy of Section 26.4 and is a complete small research topic in itself. Second: a dataset of a university site—by the method of Section 26.5, record visual-inertial data inside a departmental building with a micro drone or a handheld sensor rig (images, IMU, and an approximate ground truth from AprilTags), put it into a standard format, and publish it for the students who come after. This is a contribution that yields value over a long time, and a fine form in which to present the result of a project.

Further Reading

1. M. Burri et al., "The EuRoC Micro Aerial Vehicle Datasets," International Journal of Robotics Research, 2016. (The official description of the EuRoC dataset, the basis of Section 26.6.)
2. D. Schubert et al., "The TUM VI Benchmark for Evaluating Visual-Inertial Odometry," IROS, 2018. (The TUM-VI dataset, with an account of its photometric calibration and hardware time synchronization.)
3. C. Campos et al., "ORB-SLAM3," IEEE T-RO, 2021. (The system of Chapter 23, and the principal tool for this chapter's implementation.)

4. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE T-RO, 2018. (Another representative work in drone visual-inertial systems.)
5. D. Scaramuzza and Z. Zhang, "Visual-Inertial Odometry of Aerial Robots," Encyclopedia of Robotics, Springer, 2019. (The authoritative survey of visual-inertial navigation for aerial robots, from the UZH group, answering to Chapter 2.)
6. K. Sun et al., "Robust Stereo Visual Inertial Odometry for Fast Autonomous Flight," IEEE RA-L, 2018. (A representative work on VIO in fast flight, mentioned in Case 26-3.)
7. The documentation on integrating visual localization with the PX4 and ArduPilot open-source flight controllers. (Practical guidance on feeding VIO output into a flight controller; essential reading before real flight.)

Summary

1. Localizing a drone has four particular difficulties: full six-degree-of-freedom motion, no odometry (only a drifting IMU), high speed and violent dynamics, and severe consequences of an error; together with the constraints of payload and power and the severe demand on timeliness, these make it one of the hardest SLAM platforms of all.
2. GPS-denied settings include indoors, beneath bridges, tunnels, urban canyons, under vegetation, and deliberate interference; the means available, in order, are the IMU, the barometer, optical flow, ranging, VIO, VI-SLAM, and UWB or visual markers, and practice adopts a layered architecture so that failure degrades step by step rather than collapsing in an instant.

3. The levels of the three visual localization technologies: optical flow gives only horizontal velocity but costs very little computation and sustains a hover in place; VIO gives a full 6-DoF pose but drifts; VI-SLAM adds a map and loop closure and corrects long-term drift. One should choose by what the task requires and use no technology beyond it.
4. Fast flight deals a triple blow: motion blur damages feature tracking, shortened feature tracks make mid-term association hard to establish, and latency in the control loop worsens; the remedies include raising the shutter speed, using a global shutter, improving tracking with IMU prediction, enhancing the image, and adjusting the weights of vision and inertia adaptively.
5. The safety principles of an indoor flight experiment are not to be compromised: a net, a micro platform, an emergency stop, and testing in stages; the site needs attention to texture, light, and air currents; and the procedure should advance from a hover test by degrees to a challenge test.
6. EuRoC supplies eleven micro air vehicle sequences with stereo images and motion capture ground truth, at speeds seldom above 5 meters per second; TUM-VI supplies 1024 × 1024 images with photometric calibration and hardware time synchronization, with a greater variety of scenes. In comparing figures one must mind the evaluation conditions of each study.
7. Drone autonomy may be divided into three levels—stable, autonomous, and intelligent; the technology advances along three main lines, computation settling downward, sensing growing lighter, and intelligence rising, and the three are converging: a drone that is small, frugal, and capable of semantic understanding is becoming a reality.

Carrying Forward

This chapter has quantified the three new problems flight brings. On drift: over the second half of a 72-second flight the mean error is 0.810 meters for optical flow, 0.296 for VIO, and 0.060 for VI-SLAM, and the real difference lies in the shape—only VI-SLAM's error does not grow with time. On speed: the feature matching rate falls from 100 percent at 2 meters per second to 21 percent at 28, and more deadly still the number of frames a feature stays in view falls from 104 to 2; the most effective remedy is the shutter (at 17 m/s, going from 1/125 to 1/500 raises the matching rate from 11 percent to 80). On vibration: aliasing disguises the propellers' high-frequency vibration as a false bias of 1.00 meters per second squared (which produces an 8-meter position error in 4 seconds), and raising the sampling rate does nothing—only mechanical isolation before sampling presses it down to 0.014.

This chapter has completed the discussion of navigation for a single drone. But however advanced it is, the field of view, the endurance, and the computing power of one robot are limited in the end. The next chapter widens the view to the group: how do several robots map together? How are their separate maps fused into one consistent global map? How do the cloud and the edge divide the labor? And what questions arise in communication bandwidth, time synchronization, and data security? Chapter 27 completes the last piece of Part VII and prepares the ground for the implementation of Part VIII and the AI semantic navigation of Part IX.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 27

Multi-Robot SLAM and Cloud / Edge Collaboration

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

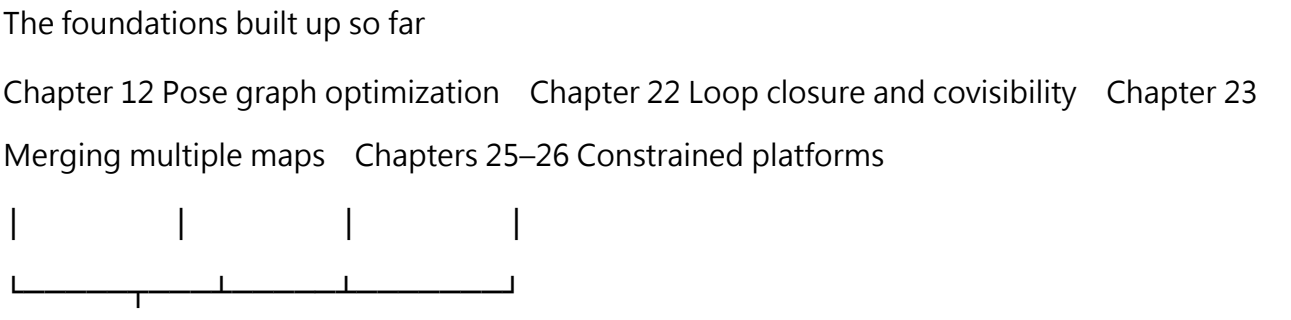
Where This Chapter Sits in the Book

Chapter 25 made the robot small and Chapter 26 set it flying. The shared conclusion of the two chapters is that the ability of a single platform is limited in the end. This chapter's answer is not to make it stronger but to bring more of them.

But bringing more of them is not multiplying the ability of one by N. A multi-robot system has problems of its own—how the maps are aligned, who talks to whom, what is to be sent first when the bandwidth will not stretch, and whether one robot mistaking its place will bring all the others down with it. None of these exists in a single-robot system.

The three figures of this chapter quantify three of these things: Figure 27-1 computes how much faster several robots exploring together really are, and what sharing a map is worth; Figure 27-2 measures how much damage one wrong cross-robot loop closure can do; and Figure 27-3 answers a very practical question—when the bandwidth is enough for only a few loop closures, which ones should be sent.

Figure 27-0 The Link Map of Chapter 27



security

▼

Part VIII ROS implementation (making all of this run)

▼

Part IX AI semantic navigation (the case in Chapter 34: collaborative exploration by several machines)

Learning Objectives

The twenty-five chapters so far have discussed one robot. But the field of view, the endurance, and the computing power of a single machine are limited in the end: one robot dog needs hours to patrol a whole departmental building, where ten might need only twenty minutes. This chapter widens the view to the group and completes the last piece of Part VII. After studying this chapter, students should be able to:

1. Explain the advantages of multi-robot SLAM over a single machine and the three great challenges it adds.
2. Explain the procedure of map fusion and what is crucial in detecting a cross-robot loop closure.
3. Compare the merits and defects of the centralized, distributed, and decentralized architectures of collaboration.
4. Explain the principles by which cloud SLAM and edge SLAM divide the labor, and where each is suitable.
5. Plan the system architecture and the allocation of tasks for a campus inspection by several robot dogs.
6. Explain how SLAM and an AI agent work together, and the role of a VLM in coordinating several machines.

7. Explain the questions a multi-robot system raises in formation, communication, data synchronization, and security.

Core Concepts

中文名詞	English Term	Explanation
協作式 SLAM	Collaborative SLAM	The technology by which several robots build one consistent map together.
集中式架構	Centralized	All the data are gathered on a central server to be processed.
分散式架構	Distributed	The computation is spread over several nodes, which exchange information.
去中心化	Decentralized	No central node; the robots communicate directly and each fuses for itself.
地圖融合	Map Fusion / Merging	Aligning several maps and merging them into one consistent global map.
跨機器人閉環	Inter-robot Loop Closure	A constraint established where different robots have observed the same place.
相對變換	Relative Transform	The rotation and translation between two robots' coordinate frames.
會合	Rendezvous	The event of two robots meeting and exchanging information.
感知混淆	Perceptual Aliasing	A wrong association caused by different places looking the same.
離群點剔除	Outlier Rejection	Excluding wrong loop closures so that they do not spoil the global map.
邊緣運算	Edge Computing	Computing on a local node close to the robot.
雲端運算	Cloud Computing	Computing and storing in a remote data

中文名詞	English Term	Explanation
		center.
異質團隊	Heterogeneous Team	A team made up of robots of different kinds (ground, aerial).

27.1 Single-Robot SLAM and Multi-Robot SLAM

This section explains why several robots are used, and what new problems the choice brings.

Four Advantages of Several Robots

1. Speed: several machines explore separately and the time to finish a large task falls greatly. This is the most direct motive—the collaborative multi-robot exploration of Case 24-3 in Chapter 24 is exactly this line of thought.
2. Fault tolerance: the failure of one does not affect the task as a whole. By contrast, when a single expensive robot fails the whole task stops.
3. Complementary viewpoints: observing from several positions at once allows a more complete model of the environment and reduces the blind spots occlusion causes.
4. Complementary capabilities: a heterogeneous team can combine the strengths of different platforms—a ground robot has long endurance and a large payload, a drone a wide view and the ability to go high (the ground-air collaboration mentioned in Section 26.7).

Three Great New Challenges

But several robots are not simply one multiplied by N; three problems arise that do not exist for a single machine:

- Unifying the coordinate frames: every robot takes its own starting point as its origin when it is switched on (Chapter 10). To put their maps together one must first know the relative transformation between their coordinate frames—and that transformation is unknown at the outset.

- The widening of data association: single-robot SLAM need only judge whether what I now see is something I have seen before (the long-term association of Chapter 23); several robots must further judge whether what I see is something someone else has seen. The search space grows and with it the risk of a wrong association.
- The limits of communication: the bandwidth between robots is limited, the link may drop, and the latency is not fixed. Every mechanism of collaboration must work within that reality—the theme that recurs throughout this chapter.

Its Relation to the Single-Machine Technology

The good news is that students have already learned the core technologies of multi-robot SLAM.

- Each robot localizing and mapping for itself: the various single-machine SLAMs of Chapters 13 to 26.
- Detecting cross-robot loop closures: the place recognition (the bag-of-words model) and geometric verification of Chapters 20 and 22.
- The optimization of map fusion: the pose graph optimization of Chapter 12.
- Handling wrong loop closures: the robust kernels and outlier rejection of Chapters 12 and 15.

In other words, multi-robot SLAM extends the single-machine technology to several coordinate frames and several sources of data. Understand this and students will not find the content of this chapter strange—it is the natural extension of all the chapters before.

Measuring It: How Much Faster Are a Few More Machines

Section 27.1 listed four advantages of multi-robot SLAM, the first being speed. It sounds the most self-evident of them and is also the most easily overestimated. This subsection computes it.

The experimental setting is a departmental floor of 34 × 18 meters (a central corridor with twelve rooms on either side). N robots set out from different positions along the corridor and each explores by the strategy of going to the nearest frontier, with a sensing radius of 3 meters, until 98.5 percent of the free space has been explored. Two settings are compared: sharing a single map (so that each can see where the others have been) and mapping separately, with no exchange of information at all.

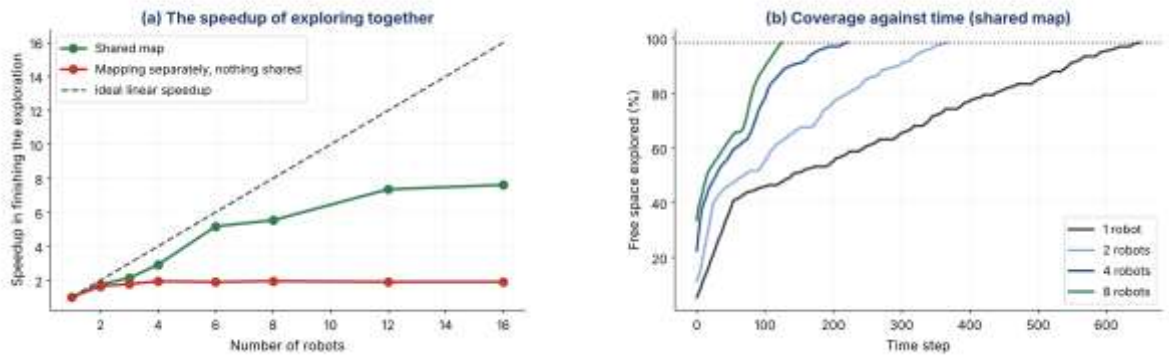


Figure 27-1 The Speedup of Collaborative Exploration by Several Robots

Panel (a) is the speedup. With a shared map two machines give 1.75 times, three 2.14, four 2.91, six 5.17 and eight 5.54—after which it flattens: twelve give 7.35 and sixteen 7.61. That is to say, the ceiling of parallelism on this floor is about twelve machines. More robots than that are not useless; it is that the time they need to travel to the last few unexplored corners has become a bottleneck that cannot be compressed further.

Note also that the speedup never reaches the dashed line of ideal linear scaling, and falls further below it the more machines there are: four give 2.91 rather than 4, and sixteen 7.61 rather than 16. Part of what is lost is travel—the robots must still walk to

the frontier—and part is overlap, where two of them approach the same unexplored corner from opposite sides.

What really deserves attention is the red line. With no map shared, two machines give only 1.64 times, four 1.94, and the curve then stops dead—eight give 1.95 and sixteen 1.91, however many are added. That flat line is itself the conclusion: robots that do not share a map explore again the regions the others have already covered, so that beyond two of them the extra machines add almost nothing. For eight machines, sharing the map shortens the time to completion by 82 percent.

This is why the map fusion of Section 27.2 is the central operation of the whole chapter. Without fusion, several robots are merely several single machines each going its own way; with fusion they truly become one system.

Once the Speedup Saturates, What Do the Extra Robots Do

Figure 27-1(a) shows that beyond six machines it grows no faster. But this does not mean the seventh is of no value.

The other three advantages mentioned in Section 27.1—fault tolerance, variety of viewpoint, and the division of labor—do not saturate with number. The thirteenth machine can take over when one fails, can observe the same object from another angle to make a semantic judgment more reliable (Chapter 32), or can be dedicated to serving as a communication relay.

In planning the size of a fleet one should therefore ask two questions separately: how fast must it be finished decides the number needed for the exploration, and how reliable must it be and what else must it do decide the number beyond that.

27.2 Map Fusion

Map fusion is the central operation of multi-robot SLAM: aligning two (or more) separately built maps and merging them into one consistent global map.

What It Shares with and What Distinguishes It from Single-Machine Loop Closure

Students should think at once of the Atlas map merging of Section 23.6. The mathematical essence of the two is the same—find the relative transformation between two maps, then propagate the correction by pose graph optimization. The difference is this:

- Several maps on one machine: the two maps come from different periods of the same robot, with exactly the same sensors and parameters.
- Map fusion across robots: the two maps come from different robots and may use different sensors, different viewpoints, and even different map representations. Fusion in a heterogeneous team is harder still.

The Three Steps of Fusion

1. Detect the encounter: judge whether two robots have observed the same place. This may be a physical meeting (the two robots see each other) or an overlap in place recognition—the scene robot A sees now, robot B saw a little earlier.
2. Estimate the relative transformation: once the overlap is confirmed, solve for the rotation and translation between the two robots' coordinate frames from the features or scans they observed in common (the geometric solution of Chapter 21, or the point cloud registration of Chapter 17).

3. Fuse and optimize: join the two pose graphs with this cross-robot edge and run pose graph optimization (Chapter 12) so that the whole system becomes globally consistent.

The Crucial Risk: a Wrong Cross-Robot Loop Closure

Chapters 12, 15, and 22 all stressed that one wrong loop closure drags the map bodily to the wrong place. In a multi-robot system that risk is higher still, because:

- The space of candidates is larger: what must be compared is not only one's own history but the observations of every other robot.
- Perceptual aliasing is worse: different robots may be at different but similar places (two classrooms of identical layout, say) and produce a wrong match that looks right.
- The consequences are more serious: one wrong cross-robot edge spoils the maps of several robots at once.

Robust outlier rejection is therefore an indispensable component of multi-robot SLAM. Systems in this line integrate robust outlier rejection to strengthen the practice and to adapt it to sporadic communication between robots. Another system adopts a distributed graduated non-convexity algorithm and incorporates semantic data into the map it produces.

Considerations of Communication Efficiency

Map fusion requires an exchange of data, and bandwidth is limited. How to achieve the best fusion with the least data is therefore the central question of the field. One system introduces a novel technique for prioritizing cross-robot loop closures by maximizing algebraic connectivity, so as to reduce communication and speed

convergence; it also adopts decentralized neighbor management and pose graph optimization suited to sporadic inter-robot communication, and supports descriptors for different sensor types—CosPlace and NetVLAD for images, Scan Context for LiDAR scans.

The central insight here is that not all loop closure candidates are of equal importance. Some contribute a great deal to global consistency and others scarcely matter. Transmitting the high-value candidates first obtains the greatest benefit within a limited bandwidth—a line of thought full of engineering wisdom.

Measuring It: How Dangerous Is One Wrong Cross-Robot Edge

Section 27.2 says that one wrong cross-robot edge spoils the maps of several robots at once. The sentence deserves to be said once in numbers, because its severity exceeds most people's intuition.

The experimental setting: three robots each walk a partly overlapping circular route of 50 key frames, with noisy odometry. Between them there are 42 correct cross-robot loop closures. A number of wrong edges from perceptual aliasing are then mixed in deliberately—the robot believes it is seeing the other's frame j when in fact it is another place that merely looks much like it. Ordinary least squares and a robust kernel (graduated non-convexity) are compared as ways of solving.

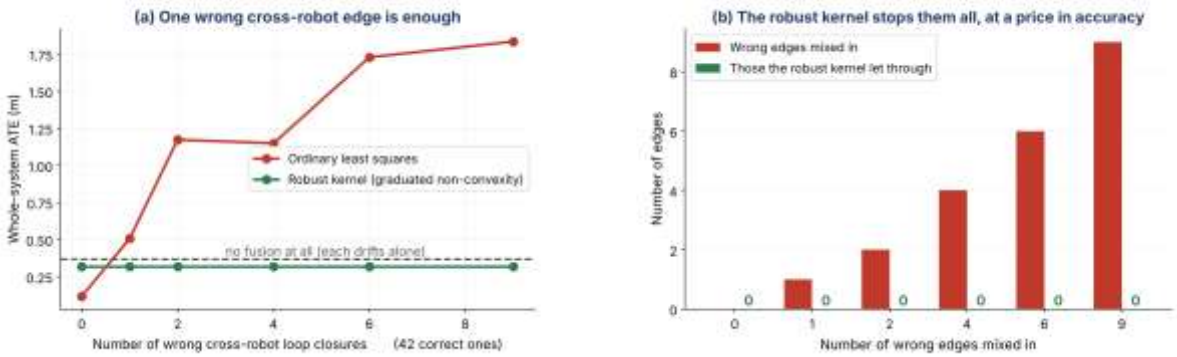


Figure 27-2 Wrong Cross-Robot Loop Closures and Robust Rejection

The red line of panel (a) is the point. With no wrong edges, the whole system's ATE after fusion is 0.118 meters, more than three times better than the 0.369 meters of no fusion at all—this is the positive value of map fusion. But let a single wrong edge in and the ATE leaps to 0.507 meters; two give 1.17 meters; and nine give 1.84.

Note what one wrong edge does: from 0.118 up to 0.507, more than fourfold, and worse than not fusing at all. In other words, in this situation fusing with one wrong edge mixed in is worse than not fusing in the first place. That is why Section 27.2 lists outlier rejection as an indispensable component—it is not there to raise accuracy but to avert disaster.

The green line is the result with the robust kernel added. However many wrong edges are mixed in (up to nine were tested), the ATE holds steady at 0.319 meters and does not worsen at all as the wrong edges multiply. Panel (b) shows its power of discrimination: every wrong edge was stopped, and not one slipped through.

But there is a price here that must be stated honestly. Even with no wrong edges at all, the robust kernel's ATE is 0.319 meters against ordinary least squares' 0.118—nearly three times worse. The reason is that a robust kernel lowers the weight of all large residuals, including correct edges that merely happen to be noisier. It has traded

accuracy for immunity to outliers.

The practice is to work in two stages: use the robust kernel first to identify the inlier set, then redo an ordinary least squares on the inliers alone. In theory this obtains the benefits of both, but the threshold of discrimination must be set with great care—loosen it and a wrong edge gets in, and one is enough to undo everything. This is the part of multi-robot SLAM that most needs tuning in practice and most easily goes wrong.

Measuring It: When the Bandwidth Will Not Stretch, Which Ones Should Be Sent

Section 27.2 ended by noting that not all loop closure candidates are of equal importance, and introduced the practice of ordering them by maximizing algebraic connectivity. The idea is rather abstract, and this subsection makes it concrete by experiment.

Algebraic connectivity is the second smallest eigenvalue of a graph's Laplacian matrix, and measures intuitively how tightly the graph hangs together—the larger it is, the harder the graph is to cut in two and the more easily error is redistributed across the whole of it (the reasoning of Figure 22-2 in Chapter 22). The experiment compares two strategies of selection: picking k candidates at random, and greedily picking each time the one that raises the algebraic connectivity most.

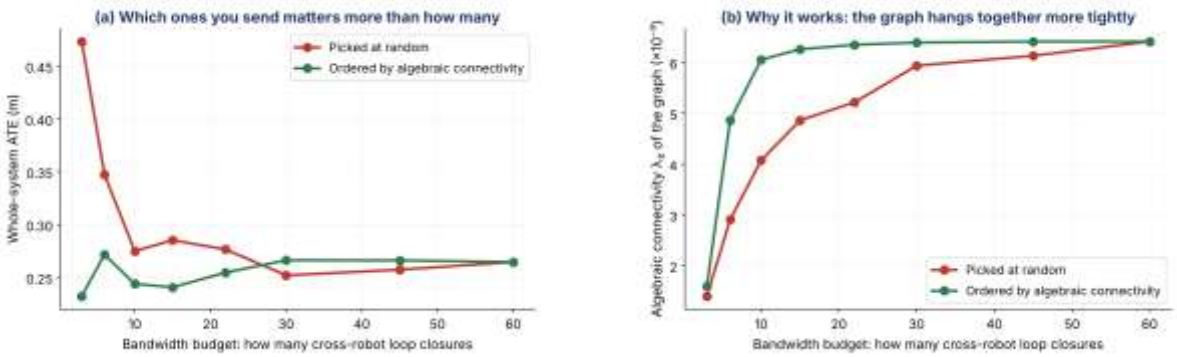


Figure 27-3 Under a Limited Bandwidth, Which Loop Closure Candidates Should Be Sent First

Panel (a) is the result. With a budget of only 3, random selection gives an ATE of 0.473 meters and ordering by connectivity 0.232 meters; with a budget of 10 the figures are 0.275 and 0.244 meters; with a budget of 22 they are 0.277 and 0.255. At a small budget the ordering halves the error, and at a middling one it still holds a clear lead.

A more useful way to read it is horizontally: ordering by connectivity needs only 10 to reach 0.244 meters, where random selection needs 30 to reach a comparable level. That is to say, simply choosing which ones to send can bring the bandwidth requirement down to a third. For a team of robots with only sporadic chances to communicate, this difference is decisive.

Panel (b) explains why it works. The vertical axis is the algebraic connectivity of the graph, and the green line is above the red at every budget—the ordering strategy has bought a more tightly connected graph with the same number of edges. The two lines meet at a budget of 60 (everything sent), because there the two are choosing the same set.

This experiment also demonstrates a general line of engineering thought: when resources are limited, rather than asking whether a few more can be had, ask first

whether those on hand are being spent where they count. The same thought appeared in the key frame culling of Chapter 22 and the event-driven computation of Chapter 25.

Why Algebraic Connectivity Is the Right Index

Intuitively one would want to pick the most certain loop closure, or the nearest. But such loop closures often connect two nodes that are already close together—the information is redundant and helps global consistency little.

Algebraic connectivity catches something else: whether this edge joins two pieces of the graph that were far apart. Stitching two loosely connected regions together is far more useful than putting another stitch where it is already tight.

This is the same reasoning as Chapter 22's covisibility edges carry the correction along the whole trajectory, only turned from a qualitative observation into a number that can be computed and ordered by.

27.3 Three Architectures for Collaborative SLAM

The system architecture of multi-robot SLAM is the most important design decision of this chapter. The spectrum of multi-robot SLAM architectures runs from the centralized all the way to the fully decentralized.

One: the Centralized Architecture

All robots send their data to a central server, which processes and optimizes them together and distributes the results back.

Centralized systems of the CCM-SLAM kind gather the data on a server for joint optimization; this simplifies map fusion but introduces a communication bottleneck

and a single point of failure.

- Merits: map fusion is simple (all the data are in one place), global optimization is easy, and the computational burden on the robots is light.
- Defects: a communication bottleneck (everything must be transmitted), a single point of failure (the server goes down and the whole system stops), and poor scalability.
- Representative systems: CCM-SLAM, COVINS (for visual-inertial sensing, sparsifying by removing redundant key frames), and LAMP 2.0 (a LiDAR system that predicts the priority of every cross-robot loop closure candidate with a graph neural network).

Two: the Distributed and Decentralized Architectures

In a distributed system the processing is spread over several nodes or agents; the robots collect data and carry out their own localization and mapping while sharing information with one another through continuous communication, so as to build a complete global map.

In the decentralized practice the robots exchange information with one another directly and complete all data fusion onboard, depending on no central entity to coordinate the system.

- Merits: no single point of failure, good scalability, and greater resilience to a break in communication.
- Defects: high complexity of synchronization, a need for distributed optimization algorithms, and a heavier computational burden on each robot.

- Representative systems: DOOR-SLAM and Kimera-Multi perform peer-to-peer pose graph optimization with outlier rejection, trading higher complexity of synchronization for greater scalability; Swarm-SLAM offers a decentralized ROS 2 framework supporting several sensing modalities; and LAMP and LAMP 2.0 handled underground multi-robot SLAM in the DARPA Subterranean Challenge (Case 1-2 in Chapter 1).

What Decides the Architecture: the Conditions of Communication

The defect of a centralized architecture is that it requires good communication with a common back end, and that if that node fails the whole system fails; moreover it cannot scale to a large team of robots.

A decentralized solution depending only on sporadic, intermittent communication between robots therefore suits a large setting better.

Architecture	Where the computing is	Merits	Defects	Where it suits
Centralized	A central server	Fusion is simple; the robots are light	A communication bottleneck, a single point of failure	A small team with good communication
Distributed	Several nodes working together	No single point of failure; scales well	Synchronization is complex	A medium or large team
Decentralized	Onboard each robot	Resilient to a break in	A heavy burden of onboard	A large-scale task with

Architecture	Where the computing is	Merits	Defects	Where it suits
		communication	computation	sporadic communication

Table 27-1 A Comparison of the Three Collaborative SLAM Architectures

Hybrids in Practice

A real system is often not purely one or the other. For instance: the robots run their own SLAM onboard (the decentralized part) but send key frames to a server for global optimization when communication is good (the centralized part). Such a hybrid design takes the merits of both, and is the engineering way of thinking this book stresses repeatedly—there is no need to choose one of two textbook categories.

27.4 Cloud SLAM and Edge SLAM

The last section discussed how robots divide the labor among themselves; this one discusses how a robot divides the labor with computing resources outside it.

The Division of Labor Among Three Tiers

Tier	Where	Latency	Computing power	Tasks it suits
Onboard	The robot itself	Very low	Constrained (Chapter 25)	Real-time control, attitude estimation, obstacle avoidance
Edge	A local server / base station	Low (a few to tens of milliseconds)	Moderate to high	The SLAM back end, map fusion, AI inference

Tier	Where	Latency	Computing power	Tasks it suits
Cloud	A remote data center	High (tens to hundreds of milliseconds)	Very high	Large-scale map storage, model training, cross-site analysis

Table 27-2 The Division of Labor Among the Onboard, Edge, and Cloud Tiers

The Basic Principles of the Division

Three criteria decide what should be done where:

1. The demand on timeliness: the control loop must be onboard (the drone case of Section 26.1 is the plainest of all); only a task that can tolerate tens of milliseconds of latency is fit to be sent to the edge.
2. The volume of data: transmitting large volumes of raw data is costly. Compressing or extracting features onboard first and transmitting only the condensed result is far more efficient (which is exactly the line of thought of Section 27.2 on communication efficiency).
3. Security and privacy: is image data bearing on personal privacy fit to be sent to the cloud? In settings such as a hospital or a school this must be considered.

Edge-Assisted Collaborative SLAM

Edge computing suits multi-robot SLAM particularly well, because it solves two problems at once: the shortage of computation on the robot and the latency of a centralized architecture.

One system optimizes the server side by offloading the heavy computation while maintaining the agents' autonomy with onboard visual odometry. This is a typical

division: the robot does the real-time front end itself (depending on no network) and leaves the time-consuming back-end optimization and map fusion to an edge server.

Advances in communication technology are pushing this direction too. Research has used 5G communication to handle real-time dense volumetric data from several robots, showing how high throughput and low latency make it possible to offload large sensor streams. Section 5.4 mentioned 5G in robot localization; here is its value in collaborative computing.

Communication Quality as a Planning Resource

An important change of idea in recent years deserves students' attention. The central idea of one study is to treat communication quality as a planning resource and, combined with multi-agent collaboration, to make the navigation cliff a manageable event of degradation rather than a hard limit of operation.

This line of thought is most suggestive: rather than accept passively that there is no signal here, bring the quality of communication into path planning actively—have one robot stop where the signal is good to serve as a relay, say, or arrange for robots to return periodically to a communicating region to exchange data. It is of a piece with the design philosophy of Section 26.2, letting failure degrade step by step.

27.5 Campus Inspection by Several Robot Dogs

This section integrates the technology above into one concrete system plan, which is also the foundation of Project Four in Chapter 31.

Why Several Are Needed

Estimating by the scale of an ordinary university campus: one robot dog inspecting

at half a meter per second may need several hours to walk every corridor and floor of a departmental building, while its battery often lasts less than an hour. Only with several robot dogs working separately can the task be finished on a single charge.

The System Architecture

Figure 27-4 The System Architecture of Campus Inspection by Several Robot Dogs

【Onboard layer】on every robot dog

• LiDAR-inertial SLAM (Chapter 18) or visual-inertial SLAM (Chapter 23)

• Real-time obstacle avoidance and gait control

• Semantic perception: YOLO detection of inspection targets (Chapter 24)

↓ sent: key frame descriptors, the pose graph, inspection findings

【Edge layer】a local server on campus

• Cross-dog loop closure detection and map fusion (Section 27.2)

• Global pose graph optimization (Chapter 12)

• Task allocation and route coordination

↓ sent: the condensed map, the inspection report

【Cloud layer】(optional)

• Long-term map storage and comparison across time (change detection)

• AI model training and updating

• The management interface and reports

Strategies for Allocating the Task

- By area: the simplest—each robot dog takes a floor or a region. The merits are that they do not interfere with one another and that it is easy to implement; the defect is that it cannot be adjusted dynamically, so that a dog whose region finishes early stands idle.
- By frontier, allocated dynamically: the practice mentioned in Case 24-3 of Chapter 24—allocate the boundary between the explored and the unexplored (the frontier) dynamically to the most suitable robot. It is more efficient, but needs a mechanism of real-time coordination.
- By capability: the practice of a heterogeneous team—the robot dogs take the stairs and the interiors, the drones the atrium and the roof, and the wheeled robots the flat long corridors (where their endurance is best).

Practical Questions

1. Aligning the initial coordinates: let all the robot dogs set out from the same charging station, or place an AprilTag at the starting point (Section 24.3), and a common coordinate frame is established directly, which simplifies map fusion greatly. This is a fine example of avoiding a hard problem by a simple method.
2. Communication coverage: a campus Wi-Fi does not necessarily reach everywhere (basements, stairwells). The system must tolerate a dropped link and send the missing data when it recovers—which is exactly the value of the decentralized architecture of Section 27.3.
3. Safety and giving way: several robot dogs may meet in a corridor and must coordinate their passage. Interaction with pedestrians must also be considered (the socially aware navigation of Chapter 24).

4. Battery management: a robot dog's endurance is short, and the system must plan a charging rotation—two inspecting and one charging, say, taking turns to maintain continuous coverage.

27.6 SLAM Working Together with an AI Agent

Section 24.5 introduced VLM-assisted navigation; this section focuses on the role of AI in coordinating several robots.

The VLM as a Global Planner

Case 24-3 in Chapter 24 set out this direction in detail: the VLM does no low-level control but plays the commander—looking at the global map, reasoning from common sense about which directions are most likely to find the target, and allocating the tasks to the several robots.

The elegance of this division is that the VLM's strength is semantic understanding and common-sense reasoning (slow but clever) while the robot's is real-time perception and control (fast but without understanding of meaning). Letting each do what it is good at is the most reasonable architecture.

Three Roles for an AI Agent in a Multi-Machine System

1. The planner of tasks: decomposing a high-level goal (inspect every classroom on the third floor) into concrete subtasks and allocating them to the robots. This requires an understanding of the semantic structure of the site (the semantic map, Chapters 3 and 24).
2. The reader of anomalies: an inspection robot returns a great many images, and examining them one by one by hand is impractical. An AI agent can filter out

the anomalies that need human attention (a crack, a leak, a blocked passage) and lighten the burden of labor greatly.

3. The arbiter of coordination: when the needs of several robots conflict (all want the same region, all need charging), the agent decides by the global goal.

Sharing the Semantic Map

One advanced direction in multi-robot collaboration is to share semantics and not only geometry. One system incorporates semantic data into the map it produces, and can build a globally consistent metric-semantic three-dimensional mesh model in real time while depending on local communication alone, with resilience to wrong loop closures.

The value of sharing semantics is this: once robot A has recognized this is an elevator, robot B need not recognize it again but may use the result directly. This not only saves computation but keeps the whole team's understanding of the environment consistent.

A Practical Caution

The limitations Section 24.5 stressed apply in a multi-robot system too, and are even more severe:

- Latency: VLM inference takes time, and if every decision must wait on it the system reacts too slowly. AI should therefore handle decisions at the scale of seconds and above, while real-time avoidance remains the business of traditional methods.

- Hallucination: a wrong judgment by the AI may send the whole team exploring in the wrong direction. The system should retain verification at the geometric level and an interface for human supervision.
- Communication: an AI decision is usually made at the edge or in the cloud, which means it depends on communication. When the link drops the robots must be able to fall back on a default autonomous behavior (the degradation thinking of Section 27.4).

27.7 Formation, Communication, Data Synchronization, and Security

This section gathers the four engineering questions of a multi-robot system; unglamorous beside the algorithms, they often decide whether the system can really work at all.

One: Formation and Spatial Coordination

- Avoiding collisions: several robots moving in the same space must avoid striking one another. The practices include sharing positions in real time, dividing regions of travel, and setting rules of passage at narrow places (corridors, doorways).
- Maintaining the communication chain: if the team must stay connected, the formation must take account of communication range. Sometimes one robot is stationed deliberately at a relay position (treating communication as a planning resource, as Section 27.4 noted).
- Efficiency of coverage: the formation also affects the efficiency of exploration. Spreading out speeds coverage, but spreading too far loses the ability to support one another and to communicate.

Two: Communication

- Limited bandwidth: raw images and point clouds are enormous in volume and cannot all be transmitted. What is transmitted in practice is a condensed representation—key frame descriptors, the pose graph, sparse features (the considerations of efficiency in Section 27.2).
- Intermittent connection: in a real setting the link will drop. The design must assume that it may drop at any moment and be able to send the missing data and resynchronize when it recovers.
- The communication topology: can every robot communicate with every other, or only with its neighbors? This bears directly on the choice of architecture (Section 27.3).

Three: Data Synchronization

- Time synchronization: this is the difficulty most easily underrated in a multi-robot system. Every robot's clock differs a little, and without calibration the fusion will be misaligned in time. Chapters 5 and 10 stressed the importance of time synchronization repeatedly, and in a multi-machine system its effect is greater still. The usual practices are the network time protocol or timing from GNSS (Section 5.1 noted that GNSS supplies precise timing as well).
- Coordinate synchronization: the problem of unifying coordinate frames mentioned in Section 27.1. Besides estimating it by map fusion, a common starting point or a known marker in the environment may be used (the practice of Section 27.5).
- Consistency of version: when the map is updated (after a loop closure correction, say), every robot needs the newest version. If one robot acts on an old map, conflict may follow.

Four: Security

- Communication security: if the communication between robots is eavesdropped on or tampered with, an attacker may inject wrong map data and mislead the whole team. Encryption and authentication are necessary.
- Data privacy: the images an inspection robot takes may contain faces and private spaces. In settings such as a campus or a hospital the collection, storage, and access rights of the data must be regulated explicitly. This is not a technical question only but one of ethics and of law.
- Physical safety: with several robots working at once the risk to the people around them rises. The system needs an emergency stop for all of them, and every robot should have an independent safety floor (so that it does not run out of control even having lost communication).
- Fault isolation: when one robot fails or behaves abnormally, the system should be able to isolate it so that its wrong data do not contaminate the global map—which is exactly the extension of Section 27.2's outlier rejection to the level of the system.

The Close of Part VII

Part VII set out from the milliwatt-level micro platform of Chapter 25, passed through the drone navigation of Chapter 26, and arrives at the group collaboration of this chapter, presenting one clear line of thought: when the ability of a single robot is constrained, the answer is not to make it bigger and stronger but to make it smaller and more frugal and then to have more of them work together.

This line of thought agrees with nature's own strategy—an ant or a bee can do little by itself, yet the colony accomplishes astonishing things. The technology of this part is

exactly the foundation on which a team of robots acquires such collective intelligence.

In the next part we return to the most practical level: how all of this is actually made to run. Part VIII, ROS / Python / C++ Implementation, will take students through setting up a development environment, implementing 2D LiDAR mapping and navigation, running visual SLAM, and completing a full project. The theory has come this far; it is time to make the robot really move.

27.8 Three Practical Boundaries of Multi-Robot SLAM

This chapter has measured three things, and each points to a practical boundary. This section gathers them together.

One: the Boundary on Number, Decided by the Site and Not by the Budget

Figure 27-1 showed the speedup flattening at about twelve machines. That number is not universal—it is decided together by the size of the site, the complexity of its topology, and the sensing radius. In planning a fleet, students should first run the simulation of Figure 27-1 on their own site, find the saturation point that belongs to them, and only then decide how many to buy.

A rough way to estimate it is to divide the total path length of the site by the distance one machine can walk within its endurance, which gives how many are needed at least; and the saturation point usually falls between one and two times that number. Machines added beyond the saturation point have their value in fault tolerance and in the division of labor, not in speed.

Two: the Boundary on Trust—Better to Fuse Too Little Than to Fuse Wrongly

The numbers Figure 27-2 gives are extreme: one wrong edge makes the system twice

as bad as no fusion at all. This asymmetry decides a design principle for multi-robot systems—the threshold for fusion should be set stricter than for a single-machine loop closure.

Concrete practices include requiring more inliers before a cross-robot match is accepted, requiring the same match to be detected over several consecutive frames, and running a check of geometric consistency before acceptance (that the transformations among three robots should compose end to end back to the origin, say). All these checks make the system miss some correct loop closures, but by the figures of Figure 27-2 that price is far smaller than accepting one wrong edge.

Three: the Boundary on Bandwidth—Order First, Then Compress

Faced with a limit on bandwidth, the intuitive practice is to compress the data—lower the image resolution, reduce the number of feature points, sparsify the key frames. Figure 27-3 points to another dimension often overlooked: selection. Sending the same volume of data, choosing the right things to send can improve the result two or three times over.

The practical order is to select first (decide which to send) and compress after (decide how finely each is sent). Reverse the order and you will spend the bandwidth evenly across all the candidates, with the result that none is sent clearly enough and none can be used.

	What this chapter measured	What it means for design
Number of machines	Twelve flatten at a 7.4-fold speedup	The saturation point is decided by the site;

		simulate before buying
Sharing the map	With eight, it shortens the time by 82 percent	Map fusion is the premise of a multi-machine system, not a bonus
A wrong loop closure	One takes the ATE from 0.118 to 0.507 meters	The threshold for fusion must be stricter than for a single-machine loop closure
The robust kernel	It stopped every one, but with no wrong edges the accuracy fell from 0.118 to 0.319	Insurance has its premium; two-stage handling obtains both
Ordering the candidates	10 ordered ≒ 30 at random	Order first, then compress

Table 27-3 What the Three Experiments of This Chapter Imply for Design

Formulas and Procedures

One: the Coordinate Transformation Across Robots

Let the world frames of robots A and B be W_A and W_B. The heart of map fusion is to find the relative transformation between them:

$$T_{W_A \leftarrow W_B}$$

Once this transformation is had, any pose of B can be converted into A's frame:

$$p_{W_A} = T_{W_A \leftarrow W_B} \cdot p_{W_B}$$

This is exactly Chapter 10's composing transformations is multiplying matrices, applied to several robots. If the team has N robots, N − 1 relative transformations

suffice in principle to bring every map into one frame (forming a spanning tree, which answers to the idea of Section 22.6).

Two: the Objective Function of a Multi-Robot Pose Graph

Once the pose graphs of all the robots are merged, the objective of the overall optimization is:

$$\min \Sigma_{\text{edges within a robot}} \|e\|^2_\Sigma + \Sigma_{\text{edges across robots}} \rho(\|e\|^2_\Sigma)$$

The first term holds each robot's own odometry and loop closure constraints, the second the loop closure constraints across robots.

Note particularly the robust kernel ρ outside the second term—the risk of error in a cross-robot edge is far higher than in an edge within one robot (Section 27.2), and a robust kernel must therefore protect it. This detail embodies the idea of treating things differently according to their credibility, of a piece with the Kalman gain of Chapter 11 and the information matrix of Chapter 12.

Three: Estimating the Volume of Communication

The communication a multi-robot system requires may be roughly estimated as:

$$\text{total communication} \approx N \times (N-1) \times \text{the data exchanged between each pair} \times \text{the rate}$$

Note the growth with the square of N—which is exactly the mathematical reason the centralized and fully connected architectures are hard to scale. With 10 robots the number of pairs is 45; raise it to 20 and the number of pairs leaps to 190.

This formula explains the necessity of Section 27.2's prioritizing of loop closure candidates and Section 27.3's decentralized architecture: as N grows, the actual

communication must somehow be held far below this upper bound—communicating only when needed, and sending only what is most valuable.

Four: The Complete Procedure of Multi-Robot SLAM

Figure 27-5 The Processing Flow of Collaborative Multi-Robot SLAM

【Onboard each robot】

each runs single-machine SLAM independently (Chapters 13–25) → its own pose graph and map

↓ extract a condensed representation (key frame descriptors, sparse features)

【Communication】

exchange candidate information by priority (bandwidth is limited; send only what is valuable)

↓

【Cross-robot loop closure detection】

place recognition matching → geometric verification → outlier rejection

↓ obtain the relative transformation across robots

【Map fusion】

establish the cross-robot edges → merge the pose graphs

↓

【Global optimization】

pose graph optimization (with a robust kernel) → a globally consistent map

↓

【Distribution】send the corrected results back to every robot

Cases

Case 27-1 The Subterranean Challenge: the Ultimate Examination Hall of Multi-Robot SLAM

Case 1-2 in Chapter 1 introduced the DARPA Subterranean Challenge; here that competition is looked at again from the multi-robot point of view.

The competition required a team of robots to explore, map, and find target objects autonomously in man-made tunnels, an underground city, and natural caves. That setting gathers very nearly the worst version of every challenge in this chapter:

- Communication is terrible: radio attenuates severely underground, and a robot often loses contact entirely once it has gone deep—which is exactly the extreme case of Section 27.3's sporadic, intermittent communication.
- The environment is degenerate: the geometry of a tunnel repeats itself highly (the structural degeneracy of Section 17.3), the risk of perceptual aliasing is very high, and the probability of a wrong loop closure rises greatly.
- The team is heterogeneous: the competing teams generally combined wheeled robots, quadruped robot dogs, and drones, each suited to different terrain (the complementary capabilities of Section 27.1).

This competition gave rise directly to several important systems. LAMP and LAMP 2.0 were developed precisely to handle underground multi-robot SLAM in the DARPA Subterranean Challenge; LAMP 2.0 introduced a centralized prioritization mechanism based on a graph neural network, predicting the outcome of every cross-robot loop closure candidate after pose graph optimization.

This practice of using AI to predict which loop closure candidates are worth handling is most suggestive: when resources (communication, computation) are limited, rather than treat every candidate alike, predict the value by a learned method

and then allocate the resources. It is the same thought as the prioritizing of Section 27.2, by a different technical means.

The lesson for students is that an extreme setting is often the hotbed of a technical breakthrough. The systems entered in the Subterranean Challenge have since become important foundations of multi-robot SLAM—a hard problem forced out good answers.

Case 27-2 How Badly Does the System Break When Communication Is Unreliable

Papers on multi-robot SLAM mostly evaluate under good communication. But in a real setting communication is often broken and restored by turns—how much does this affect the system?

Research has quantified this question specifically, and the result is quite striking: against a baseline of reliable communication, the error of Swarm-SLAM rises 5.08 times and that of Kimera-Multi 6.17 times.

In other words, the same system may see its error worsen five or six times when communication is unreliable. The figure reminds us that handsome laboratory data need not be reproducible in the field.

It also explains why recent research sets more and more store by the conditions of communication. Section 27.4's treating communication quality as a planning resource is exactly such a response: rather than accept the consequences of poor communication passively, plan the route actively so as to stay connected, or design an architecture that degrades gracefully when the link drops.

For students doing a project this is an important reminder: in evaluating your multi-robot system, be sure to include a test under poor communication. Testing only under ideal conditions will overestimate the real ability of the system severely. This is of a piece with the spirit of Section 14.6's common failure cases and Section 26.5's challenge test—seeking out the boundary of failure actively is responsible engineering practice.

Case 27-3 From Simulation to the Field: Development Tools for a Heterogeneous Team

The development of a multi-robot system has one practical difficulty: the cost of hardware. The price of ten robot dogs is a heavy burden for most laboratories, and the labor and the space needed to run an experiment with ten robots are considerable too.

Simulation is therefore a necessary tool in multi-robot research. Open-source simulation environments designed for heterogeneous multi-robot work have appeared in recent years. There is, for instance, an open-source simulator and data collection pipeline built on an Unreal Engine 5 simulator, which resolves key architectural limitations of earlier frameworks so as to support unmanned aerial and ground vehicles operating together in large-scale, photorealistic, dynamic environments; it introduces a new waypoint-following ground vehicle controller matching the existing drone control interface, and supplies a shared navigation stack for heterogeneous platforms covering mapping, traversability analysis, planning, and control.

Tools of this kind are of particular value in teaching: students can build teams of ten or twenty robots in simulation and test task allocation, map fusion, and

communication strategies without a corresponding quantity of physical hardware. Once the algorithm has been shown to work in simulation, two or three real robots suffice to verify the crucial parts.

This is exactly the path of learning this book has recommended since Chapter 8: simulate first, then cheap hardware, and pursue scale afterwards. A multi-robot system magnifies the value of that principle to the utmost—without simulation, multi-robot research can hardly move a step.

Implementation Activities

Implementation 27-1 Simulating Collaborative Mapping by Several Robots

Purpose: to experience in full, through simulation, the central questions of multi-robot SLAM—map fusion, the limits of communication, task allocation, and the risk of a wrong loop closure. This activity is mainly a simulation and needs no fleet of real robots.

Part One: Fusing the Maps of Two Robots (everyone)

1. Reuse the simulated environment of Implementation 25-1 in Chapter 25 (or the 2D mapping simulation of Chapter 13), create two virtual robots setting out from different corners of the room, and have each explore by its own route and build its own map.
2. Observe the problem: display the two maps laid one on the other and you will find that they do not agree at all—because each takes its own starting point as its origin (the problem of unifying coordinate frames in Section 27.1).

3. Fuse by hand: supposing you know the relative position of the two starting points, transform coordinates by Formula One, bring B's map into A's frame, and display them together. Observe whether the fused map is complete.
4. Estimate the transformation automatically: let the two robots' routes overlap in part, register the two maps (or the points in the overlapping region) with the ICP of Chapter 17, and estimate the relative transformation automatically. Compare the error against the truth.
5. Add drift: give both robots' pose estimates an accumulated error, fuse again, observe how the map is thrown out of register, and correct it by pose graph optimization (Chapter 12).

Part Two: An Experiment on the Limits of Communication (everyone, or undergraduate)

1. Set a communication range: rule that the two robots may exchange data only when they are closer than some distance. Simulate their exploring separately and meeting now and then.
2. Compare three conditions of communication: connected at all times, communicating only on meeting, and dropping at random (no communication with probability 30 percent at every step, say), and compare the quality of the final map.
3. Quantify the effect: compute the map error under the three conditions, set it against the five- to sixfold increase in error mentioned in Case 27-2, and see whether your simulation shows a similar trend.
4. Count the data: record the total communication under the three conditions and think about the trade-off between the volume of communication and the quality of the map.

Part Three: Task Allocation and Wrong Loop Closures (undergraduate)

1. Compare the efficiency of exploration: let 1, 2, and 4 robots each explore the same room until it is wholly covered, and record the number of steps each needs. Draw the curve of the number of robots against the time to completion and see whether the benefit grows linearly (hint: usually it does not, because some regions are explored twice).
2. Strategies of task allocation: compare the efficiency of random exploration against division by area, setting this against the discussion of Section 27.5.
3. An experiment on a wrong loop closure: add a wrong cross-robot loop closure edge by hand (connecting some pose of A wrongly to an unrelated pose of B, say), run the pose graph optimization, and observe how the map is spoiled.
4. Make it robust: add a robust kernel to the optimization (scipy's soft_l1 and similar loss functions will serve), run it again, and observe whether the damage of the wrong loop closure is suppressed. This verifies directly the work of ρ in Formula Two.

Discussion Questions

1. What do the two maps look like laid one on the other with no coordinate transformation? What new problem of multi-robot SLAM against a single machine does this show?
2. How large is the error of the relative transformation ICP estimates automatically? What happens if the two robots' routes hardly overlap at all?
3. By what factor does the map error rise when the condition of communication changes from connected at all times to communicating only on meeting? How does this compare with the data of Case 27-2?

4. Going from 1 robot to 4, by what fraction is the time to completion shortened? Why is it not a quarter?
5. (Undergraduate) After the wrong loop closure was added, which parts of the map were spoiled worst? By how much did the robust kernel suppress the damage? What does this imply for the design of a real system?

Extended Challenge (university level)

First: implement a simple prioritizing of loop closures—when communication is limited and only some candidates can be sent, design a scoring mechanism (by how far apart the two stretches of trajectory a loop closure connects are, say, or whether the robots involved already have other links), send the high-scoring candidates first, and compare the result with random selection. This corresponds to the central idea of Section 27.2. Second: simulate a heterogeneous team—add one robot that is fast but has a small sensing range and one that is slow but has a large one, design a task allocation strategy that plays to each one's strength, and compare the efficiency with a homogeneous team. This answers to the complementary capabilities of Section 27.1 and the ground-air collaboration of Section 26.7.

Further Reading

1. Y. Tian et al., "Kimera-Multi: Robust, Distributed, Dense Metric-Semantic SLAM for Multi-Robot Systems," IEEE T-RO, 2022. (The representative work in distributed multi-robot semantic SLAM; the code is public.)
2. P.-Y. Lajoie and G. Beltrame, "Swarm-SLAM: Sparse Decentralized Collaborative SLAM Framework for Multi-Robot Systems," IEEE RA-L, 2024. (A decentralized ROS 2 framework, with the loop closure prioritization technique.)

3. Y. Chang et al., "LAMP 2.0: A Robust Multi-Robot SLAM System for Operation in Challenging Large-Scale Underground Environments," IEEE RA-L, 2022. (The representative system of the DARPA Subterranean Challenge.)
4. P. Schmuck and M. Chli, "CCM-SLAM: Robust and Efficient Centralized Collaborative Monocular SLAM," Journal of Field Robotics, 2019. (The classic of centralized collaborative SLAM.)
5. P.-Y. Lajoie et al., "DOOR-SLAM: Distributed, Online, and Outlier Resilient SLAM for Robotic Teams," IEEE RA-L, 2020. (The distributed architecture and outlier rejection.)
6. The awesome-Collaborative-SLAM open-source list (GitHub). (A complete collection of papers and code on collaborative SLAM, and a good place to begin research.)
7. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022. (ROS 2's support for several robots, the foundation of the implementation in Part VIII.)

Summary

1. The four advantages of multi-robot SLAM are speed, fault tolerance, complementary viewpoints, and complementary capabilities; its three great new challenges are unifying the coordinate frames, the widening of data association, and the limits of communication. Its core technologies are the single-machine technologies (SLAM, place recognition, pose graph optimization, outlier rejection) extended over several coordinate frames and several sources of data.

2. Map fusion has three steps: detect the encounter, estimate the relative transformation, and fuse and optimize; the greatest risk is a wrong cross-robot loop closure—the space of candidates is larger, perceptual aliasing is worse, and the consequences are more serious, so robust outlier rejection is an indispensable component, while prioritizing the loop closure candidates obtains the greatest benefit within a limited bandwidth.
3. The three architectures each have their trade-offs: the centralized is simple to fuse but has a communication bottleneck and a single point of failure and cannot scale to a large team; the distributed and decentralized perform peer-to-peer pose graph optimization, scale well, and are resilient to a break in communication, at the price of complexity in synchronization and a burden of onboard computation; and a decentralized solution depending only on sporadic, intermittent communication suits a large setting better.
4. The onboard, edge, and cloud tiers divide the labor by timeliness, by volume of data, and by privacy; the edge-assisted practice maintains autonomy with onboard visual odometry and offloads the heavy computation to a server; and recent work treats communication quality as a planning resource, making the navigation cliff a manageable event of degradation rather than a hard limit.
5. Campus inspection by several robot dogs adopts the three-tier onboard, edge, and cloud architecture; the task may be allocated by area, dynamically by frontier, or by capability; the practical questions include aligning the initial coordinates (which a common charging station or an AprilTag simplifies), communication coverage, giving way to one another, and a battery rotation.
6. An AI agent plays in a multi-machine system the planner of tasks, the reader of anomalies, and the arbiter of coordination; sharing semantics and not only

geometry keeps the team's understanding of the environment consistent, but three limitations—latency, hallucination, and dependence on communication—must be kept in mind.

7. The four engineering questions: a formation must balance collision avoidance, the communication chain, and the efficiency of coverage; communication must face limited bandwidth and intermittent connection; data synchronization covers consistency of time, of coordinates, and of version; and security covers encrypted communication, data privacy, physical safety, and fault isolation.

Carrying Forward

This chapter has quantified three key parameters of bringing more of them. The speedup of collaborative exploration flattens at about twelve machines at 7.4 times, and the difference made by sharing a map is very large—with eight machines, sharing shortens the time to completion by 82 percent, and without it the speedup stops dead at about 1.9 however many are added. The positive value of map fusion is to bring the ATE from 0.369 meters down to 0.118, but let a single wrong cross-robot edge in and it worsens to 0.507 meters, which is worse than no fusion at all, while a robust kernel holds it steady at 0.319. As for bandwidth, ordering by algebraic connectivity needs only ten cross-robot loop closures to achieve what thirty picked at random achieve.

Part VII is complete. From the milliwatt-level micro platform of Chapter 25, through the drone navigation of Chapter 26, to the group collaboration of this chapter, one clear line of thought has emerged: when the ability of a single robot is constrained, the answer is not to make it bigger and stronger but to make it smaller and more frugal and then to have more of them work together—which agrees with the strategy of the

ant and the bee.

In the next part we return to the most practical level: how all of this is actually made to run. Part VIII, ROS / Python / C++ Implementation, will take students through setting up a development environment, implementing 2D LiDAR mapping and navigation, running visual SLAM, and completing a full project with its assessment. The theory has come this far; it is time to make the robot really move.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Part VIII Implementation in ROS / Python / C++

Chapter 28

The SLAM Programming Environment

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

The twenty-seven chapters so far have been about how it ought to be done. From this chapter on, Part VIII deals with how to make it actually run.

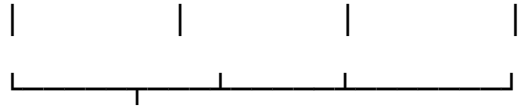
The character of this part is wholly different from what came before. It discusses no new algorithm but the things that will hold a project up for three days: versions that do not match, a topic with no data on it, a broken transform tree, two hours of recorded data and a full disk. None of these appears in a paper, and yet they take up the better half of the time spent implementing.

The three figures of this chapter are all measurements actually run rather than numbers copied from a specification sheet: Figure 28-1 takes the inner loop of scan registration and implements it in pure Python, in NumPy, and in C++, and times each; Figure 28-2 measures with real inter-process communication the difference between a topic and a service; and Figure 28-3 generates realistic sensor data and measures how much space an hour of recording takes.

Figure 28-0 The Link Map of Chapter 28

The theory of the twenty-seven chapters before

Part IV 2D SLAM Part V 3D SLAM Part VI Visual SLAM Part VII Constrained platforms



▼

【Chapter 28 Getting the tool chain ready】

Ubuntu and ROS 2 ─ Python / C++ ─ the communication mechanisms ─ rosbag ─

visualization

▼

Chapter 29 A 2D LiDAR rover Chapter 30 Visual SLAM in practice

▼

Chapter 31 Integrating a project and assessing it

Learning Objectives

The first seven parts have built up the theory and the systems knowledge in full. From this part on we return to the most practical level: how all of it is actually made to run. This chapter is the gateway to Part VIII, and its business is to get the development environment ready and the tool chain working—the foundation of everything implemented afterwards. After studying this chapter, students should be able to:

1. Explain how the versions of Ubuntu and ROS 2 correspond, and complete the installation of the environment.
2. Explain the division of labor between Python and C++ in ROS 2, and understand the role of CMake and the build system.
3. Explain the differences among topics, services, and actions, and where each is suitable.
4. Record and replay sensor data with rosbag2, and understand what the MCAP format is for.
5. Use RViz2, Gazebo, and Foxglove for visualization, simulation, and debugging.
6. Explain a systematic way of reading an open-source project on GitHub.
7. Plan a path of learning from block-based implementation at high school to work on source code at university.

Core Concepts

中文名詞	English Term	Explanation
長期支援版	LTS	A stable release with a long period of maintenance, suited to teaching and to products.
工作空間	Workspace	The directory structure holding the source of ROS packages and the products of the build.
套件	Package	The basic unit in which code is organized in ROS.
建置工具	colcon	The tool ROS 2 uses to compile every package in a workspace.
節點	Node	An independent program carrying out a particular function (Chapter 3).
話題	Topic	The channel on which nodes exchange data in the publish-subscribe pattern.
服務	Service	A synchronous communication mechanism of one question and one answer.
動作	Action	A communication mechanism for a long task that can report progress and be cancelled.
啟動檔	Launch File	A script that starts several nodes at once and sets their parameters.
資料錄製	rosbag2	The tool for recording and replaying the data on ROS topics.
訊息擷取格式	MCAP	A self-describing recording format allowing fast random access.
視覺化工具	RViz2	The three-dimensional visualization tool built into ROS 2.

中文名詞	English Term	Explanation
模擬器	Gazebo	An environment supplying physical simulation and sensor simulation.

28.1 Ubuntu and ROS / ROS 2

Section 3.5 introduced the role of ROS and its central ideas; this section focuses on the practicalities of setting the environment up.

Why Ubuntu

ROS 2 supports several operating systems, but Ubuntu is what is called a Tier 1 platform—the one the project develops and tests on principally. In practice the documentation, the dependencies, and the community support of nearly every open-source SLAM project presume Ubuntu. The implementations in this book therefore all take Ubuntu as their basis.

Matching the Versions: the First Pitfall of All

The versions of ROS 2 and of Ubuntu are bound together—each ROS 2 distribution supports only particular versions of Ubuntu. Installing the wrong one leads to a train of dependency errors that are hard to track down, and it is the commonest source of frustration for a beginner.

ROS 2 releases a new version each May, and the even years are long-term support (LTS) releases. As this book is written, the principal correspondences are these:

ROS 2 Distribution	Released	Ubuntu version	Type	Notes
Humble Hawksbill	2022	22.04	LTS	Supported to May 2027; the most mature and stable
Jazzy Jalisco	2024	24.04	LTS	rosbag2 changed to MCAP as its default format
Kilted Kaiju	2025	24.04	Ordinary	rosbag2 performance

ROS 2 Distribution	Released	Ubuntu version	Type	Notes
			release	improved
Lyrical Luth	2026	26.04	LTS	The newest long-term support release

Table 28-1 ROS 2 Distributions and the Corresponding Ubuntu Versions

Advice on Choosing a Version

- For teaching and for beginners: a mature LTS version is advised. Humble is at present the choice richest in teaching resources—the overwhelming majority of tutorials, open-source projects, and questions and answers online take it as their basis, so an answer is easiest to find when trouble comes.
- For a new project: if it is to be maintained for a long time, choose the newest LTS version for the longer period of support.
- For continuing an existing project: align directly with the version that project specifies, and do not try to upgrade while you are at it—that usually wastes a great deal of time on dependency problems.
- An important reminder: a whole class should settle on one version. Recordings made under different versions need not be interchangeable, and an inconsistent version across a class causes no end of trouble in teaching.

Three Ways to Install

1. A native installation: install Ubuntu and ROS 2 directly on the physical computer. The performance is best and the hardware support most complete, and it is the necessary choice when a real robot is connected. The drawback is that the machine must be reinstalled or dual-booted.

2. A virtual machine: run Ubuntu in a virtual machine on Windows or macOS. It installs easily and does not disturb the original system, and it suits purely software simulation exercises. The drawbacks are poorer 3D acceleration (Gazebo may stutter) and the awkwardness of connecting USB devices.
3. A container (Docker): package the whole environment in a container. The merits are a consistent environment, quick rebuilding, and no contamination of the host system—which is of particular value in teaching, since the teacher can supply one container image and spare the class the confusion of each installing separately. The drawback is that extra configuration is needed for the graphical interface and for USB devices.

A Practical Recommendation for Teaching

On years of teaching experience, the strategy advised is this: use a container or a virtual machine at the high school and introductory undergraduate stage, so that every student works in the same environment on simulations; and change to a native installation on entering the stage of work with real robots. This keeps the interference of environment problems to a minimum and lets class time be spent on SLAM itself rather than on debugging an installation.

28.2 Python, C++, and CMake

ROS 2 supports both Python and C++, and in SLAM development the two have a clear division of labor.

The Division of Labor Between the Two Languages

Aspect	Python (rclpy)	C++ (rclcpp)
Speed of development	Fast; suits rapid prototyping	Slower; must be compiled
Execution performance	Slower	Fast; suits high-rate computation
Tasks it suits	Control logic, data analysis, visualization	The core of SLAM, image processing, real-time control
Threshold of learning	Low	Higher (memory management, templates)
Its use in this book	Implementation exercises, data processing	Reading the source of open-source SLAM

Table 28-2 The Division of Labor Between Python and C++ in ROS 2

One useful principle is to get the logic working in Python first and rewrite the critical parts in C++ only if the performance will not do. Students will find that the mainstream SLAM systems (ORB-SLAM3, Cartographer, the LOAM family) are nearly all written in C++—because they must handle images and point clouds at a high rate, for which Python is not fast enough.

CMake and the Build System

A C++ project must be compiled, and the tool that manages the compilation is CMake. Its business is to find where the dependent libraries are, decide the order of compilation, and generate the actual compiler commands.

For a student of SLAM, CMake appears most often when an open-source project will not compile. Section 22.7 noted that ORB-SLAM2 may fail to build on a new system, and the trouble usually lies in a library version specified in CMakeLists.txt not matching the version on the system.

- An essential skill: being able to read the two commands find_package (locate a dependency) and target_link_libraries (link a library) in CMakeLists.txt will solve most compilation problems.
- Common errors: a package not found (it must be installed or its path given), a version mismatch (the requirement must be changed or the matching version installed), and a C++ standard mismatch (the compiler options must be adjusted).

The Workspace and colcon

ROS 2 code is organized into a workspace, inside which several packages are placed. The build tool colcon analyzes the dependencies among the packages automatically and compiles them all in the right order.

The typical workflow is: create the workspace directory → put the packages' source into the src subdirectory → run the colcon build → load the environment settings (source) → run the node. Students will use this workflow again and again throughout Part VIII.

Common Traps in Setting Up the Environment

- Forgetting to source: after a build the environment settings must be loaded before the system can find the newly compiled packages. This is the commonest reason for a node that cannot be found although the build plainly succeeded.
- Several workspaces overlaid: with several workspaces at once, the order in which they are loaded decides which version of a package is used. When behavior turns strange, check the environment variables first.

- Not rebuilding after changing the code: a Python node may take effect directly under some settings, while C++ must always be recompiled. This difference often confuses a beginner.

Measuring It: How Much Do Python and C++ Really Differ

Table 28-2 advises that Python do the prototyping and the integration and C++ the parts critical to performance. The division is right, but where does critical to performance begin? This subsection measures it with a real piece of code.

What is tested is the inner loop of scan registration—transform N points by a pose, look them up in a distance field, and accumulate the likelihood (the likelihood-field scoring of Chapter 17, used in the experiments of Chapters 19 and 25). The same logic is written three ways: a pure Python loop, a vectorized NumPy version, and C++ (g++ with -O2), and all are timed on the same machine.

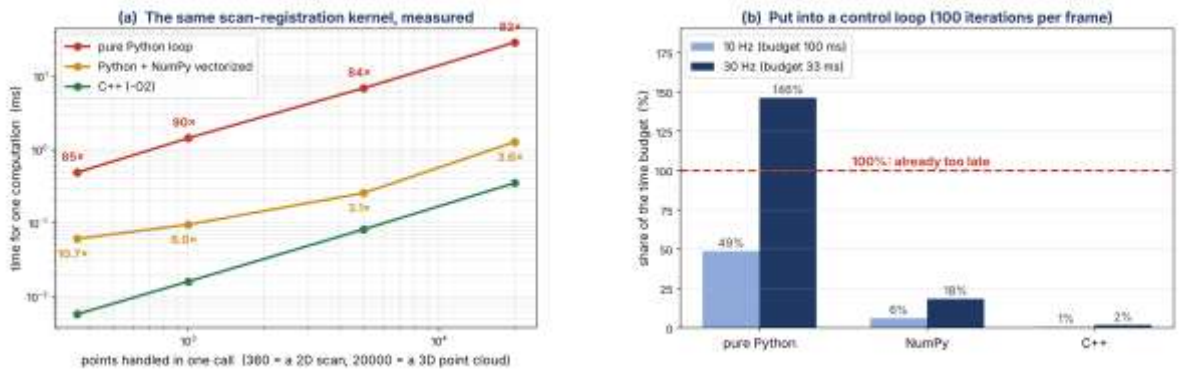


Figure 28-1 The Measured Performance of One Computation in Three Languages

Panel (a) is the time for a single computation, with both axes logarithmic. Take 360 points (a typical 2D LiDAR scan): C++ takes 0.0057 milliseconds, NumPy 0.0609 milliseconds, and pure Python 0.4870 milliseconds. Pure Python is 85 times slower than C++.

But the really useful information is that the relative positions of the red and the amber

lines change. The multiple between pure Python and C++ holds nearly fixed between 82 and 90 times and does not vary with the number of points; the gap between NumPy and C++, however, narrows from 10.7 times at 360 points to about 3 times at 5,000 and at 20,000 points (the scale of a 3D point cloud).

The reason is that NumPy's cost has two parts: a fixed overhead on every call (type checking, memory allocation) and only then the actual arithmetic. With few points the fixed overhead is the greater part, so it looks slow; with many points that overhead is spread thin and the performance approaches C++. This gives a very practical criterion: NumPy suits handling a large batch at once, and not handling a little at a time with a great many calls.

Panel (b) puts these numbers back into a real control loop. Suppose scan registration takes 100 iterations per frame (a very common setting). At 10 Hz (a budget of 100 milliseconds per frame) pure Python takes 49 percent of it, NumPy 6 percent, and C++ 0.6 percent. Forty-nine percent sounds usable, but it means only half the time is left for navigation, communication, and everything else.

At 30 Hz (33 milliseconds per frame) pure Python needs 146 percent of the budget—which is to say it simply cannot keep up, and the system begins to drop frames. NumPy needs 18 percent and C++ 2 percent, both with room to spare. The practical boundary this figure gives is clear: a pure Python loop suits offline analysis and the proving of a prototype; once a real-time loop is entered, vectorization is the minimum.

When C++ Is Really Needed

Figure 28-1(b) shows NumPy taking only 10 percent of the budget at 30 Hz—which for the great majority of teaching and project purposes is enough. So

the thought that it must be C++ is mostly needless worry.

C++ is really needed in three situations: where the iterations far exceed 100 (a large-scale bundle adjustment, say); where the computation cannot be vectorized (a branching tree search, an incremental update of a data structure); and where it runs on a milliwatt platform of the kind in Chapter 25, which has no room even for the Python interpreter.

The practical practice is to write a working version wholly in Python first, measure which part really eats the time, and rewrite only that part in C++. This is the same principle as the debugging order of Chapter 31—measure first, optimize after.

28.3 ROS Topics, Services, and Actions

Section 3.5 introduced these three mechanisms briefly; this section explains their differences and when to choose each—the central judgment in designing a ROS system.

The Topic: a Continuous Stream of Data

A topic follows the publish-subscribe pattern: the publisher keeps sending data and the subscriber keeps receiving it. Neither need know that the other exists; they need only agree on the name of the topic and the format of the message.

- Its properties: asynchronous, one to many, continuously flowing, with no guarantee that the receiver got it.
- It suits: sensor data (LiDAR scans, images, IMU), broadcasts of state (pose, map), and control commands (velocity).

- An example in SLAM: the LiDAR driver publishes /scan and the SLAM node subscribes to it and publishes the map (the implementation flow of Section 14.5).

The Service: one Question, one Answer

A service is the synchronous request-response pattern: the client sends a request and waits for the server to reply.

- Its properties: synchronous, one to one, with an explicit reply, and the caller waits.
- It suits: a query or a setting that can be completed quickly—get the current map, reset the odometry, switch mode.
- A caution: because the caller waits, it does not suit a lengthy operation—that would block the whole node.

The Action: a Long Task That Can Be Watched

The action is designed for a task that takes time and whose progress one wants to know. It combines three elements: the goal (what is to be done), the feedback (how far it has got), and the result (how it ended).

- Its properties: asynchronous, able to report progress, and able to be cancelled part way.
- It suits: navigating to a goal point, executing a trajectory, completing a scanning task.
- An example in SLAM: the Nav2 navigation of Chapter 29 is offered through an action interface—you send the goal go to this coordinate, the system keeps reporting the distance remaining, and you may cancel at any time.

Mechanism	Pattern of communication	Does it wait	Can it be cancelled	Typical use
Topic	Publish-subscribe, one to many	Does not wait	Not applicable	Sensor data, state, commands
Service	Request-response, one to one	Waits for the reply	No	Quick queries and settings
Action	Goal, feedback, result	Does not wait; can be watched	Yes	Navigation, long tasks

Table 28-3 A Comparison of the Three Communication Mechanisms

The Procedure for Choosing

When wondering which mechanism a function should use, ask three questions in order:

1. Is this a continuously flowing stream of data? Yes → a topic.
2. Is it a one-off request that can be answered quickly? Yes → a service.
3. Is it a lengthy task whose progress must be watched? Yes → an action.

The judgment looks simple, and yet it decides the quality of a ROS system's design. Using the wrong mechanism (performing a lengthy navigation through a service, say) makes the system block and hard to debug—a detail worth watching for in practice.

Measuring It: Where the Costs of a Topic and a Service Differ

Table 28-3 compared the properties of the three mechanisms, and the most crucial column of it is whether the caller is blocked. That column looks a mere technical detail, but its effect on system design is greater than all the other columns together. This

subsection measures it.

The method of the experiment: use real inter-process communication (two separate operating-system processes joined by a pipe) to send a message of 4 KB. The topic mode sends and moves on without waiting for a reply; the service mode sends a request and waits for the answer, with the server spending different lengths of time on the work. What is measured is how long the caller is actually held up, and how many exchanges can be completed per second.

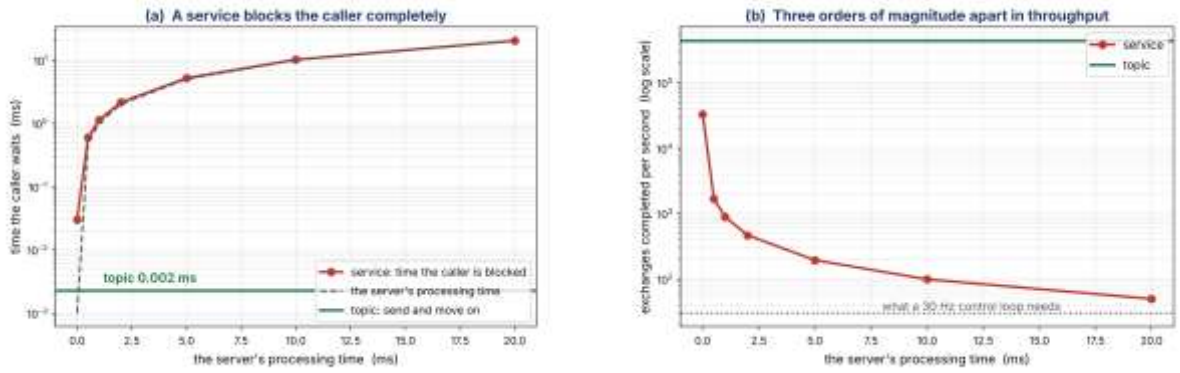


Figure 28-2 The Measured Latency and Throughput of a Topic and a Service

The green line of panel (a) is the topic mode: a latency of 0.002 milliseconds, and it does not change whatever the server is busy with—because the sender does not wait at all. The red line is the service mode, and it lies almost exactly along the grey dashed line of the server's processing time: let the server spend 5 milliseconds and the caller is held 5.17; let it spend 20 and the caller is held 20.24.

The line looks self-evident and yet is the trap most easily overlooked. Many students will call a service from inside a control loop to ask for the current map or to request a planned path, and then wonder why the robot has become sluggish—because the whole loop is waiting on that service to return.

Panel (b) converts this into throughput, with the vertical axis logarithmic. The topic

mode can send 440,673 messages per second; the service mode manages only 33,245 even with the server doing nothing at all (thirteen times fewer, purely from the cost of the round trip); and with the server spending 20 milliseconds, only 49 per second remain. The grey dotted line marks what a 30 Hz control loop requires—let the server's processing time exceed about 30 milliseconds and the service can no longer keep up.

This is the reason the action mechanism exists. An action separates sending the request from waiting for the result: the caller sends the goal and returns at once to its own loop, while the server reports progress periodically and sends the result when it is done. It is in essence the non-blocking character of a topic wrapped to give the there-and-back semantics of a service.

Three Criteria for Choosing

By the measurements of Figure 28-2, three questions may be asked in choosing a communication mechanism:

One: does the downstream need a reply? If not, use a topic—it is more than ten times faster and will not hold the upstream up. Sensor data, poses, and velocity commands all belong here.

Two: a reply is needed, but will it come back quickly? If the server's processing time is far shorter than the control period (looking up a parameter, switching a mode), a service is the simplest choice.

Three: a reply is needed and it will take a long time? Then an action is required. Navigating to a goal, carrying out a scan, docking to charge—these tasks of seconds to minutes would lock the whole system if done through a service.

28.4 Recording and Replaying Data with rosbag

rosbag2 is one of the most important tools in SLAM development: it makes record once, analyze any number of times possible.

Why rosbag Matters So Much

- Reproducibility: replaying the same data again and again allows a fair comparison of different algorithms or parameters—which is exactly what the Cartographer versus Gmapping comparison of Implementation 15-1 in Chapter 15 requires.
- Offline development: once the data are recorded, an algorithm can be developed without the robot. This is extremely practical in a teaching environment with limited hardware.
- Debugging: record the data when something goes wrong and replay it again and again to find the cause, rather than rerunning the experiment on site time after time.
- Building a dataset: the dataset of your own site recommended in Section 26.5 is recorded with rosbag.

MCAP: the Recording Format of the New Generation

The storage format of rosbag2 has evolved importantly in recent years. MCAP (Message Capture) is a recording container format from Foxglove, and it has been rosbag2's default storage format since ROS 2 Jazzy.

Its properties include being indexed for fast random access, being self-describing by carrying the message definitions within it, and supporting several formats—ROS, protobuf, JSON, flatbuffers, and others.

For students the self-describing part matters particularly: the recording itself contains the definitions of the message formats, so that it can be read correctly even on a computer where the corresponding message packages are not installed—which improves the feasibility of exchanging data and of keeping it for the long term a great deal.

Practical Points in Recording

1. What to record: SLAM development needs at least the raw sensor data (/scan or images), the IMU, the odometry, and the TF transforms. TF is often forgotten, but without it nothing lines up on replay.
2. What not to record: the outputs (the map, the estimated trajectory) usually need not be recorded, since they will be produced again on replay. Recording them takes space and may cause confusion.
3. Time stamps: confirm that the time stamps of all the data are correct—the engineering detail stressed repeatedly in Chapters 5, 10, and 27, and crucial above all on replay.
4. File size: images and point clouds are enormous in volume, and one minute may be several gigabytes. Estimate before recording and confirm that the storage will suffice.

Common Problems in Replaying

Three problems are common enough in practice to deserve attention: a version mismatch—data recorded under one version may not replay smoothly under another; missing TF transforms—many datasets lack the transform information, so that visualization in RViz fails; and a mismatch of topic names—if your algorithm expects /scan while the data carry /lidar_scan, the topics must be remapped.

The third point is particularly practical: on replay one may rename, so that recorded data can feed an algorithm expecting a different topic name with no change to the code. This is a common technique when integrating someone else's dataset.

Measuring It: How Much Space Does an Hour Take

Section 28.4 explained the value of rosbag but not its cost. This section computes the cost, because a full disk is one of the commonest accidents in the laboratory.

The method of the experiment: generate realistic synthetic sensor data (not random noise—random data do not compress and would underestimate the compression ratio badly), containing 2D LiDAR scans of smooth walls with discontinuities, a 16-beam 3D point cloud, RGB images with texture and geometric structure, a smooth depth map, and an IMU. Measure the raw bit rate of each kind of data, and the bit rate after general compression (zlib) and after an image-specific encoding (JPEG).

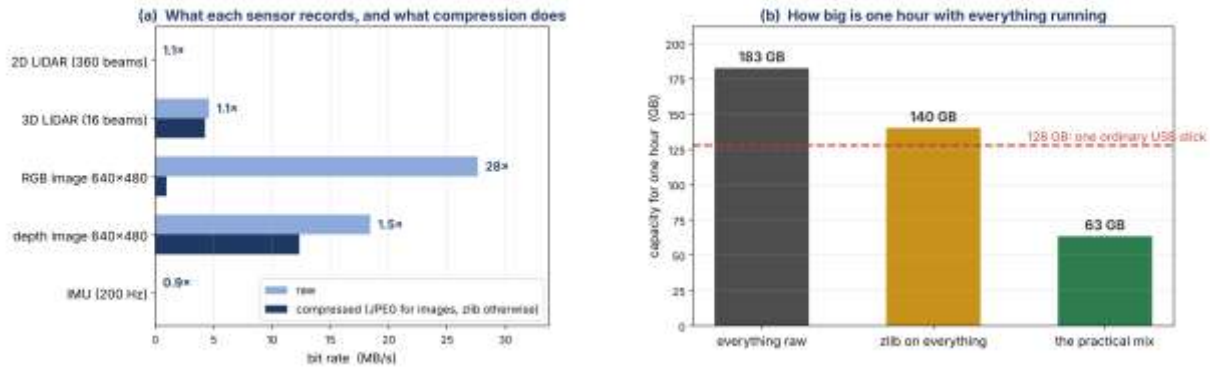


Figure 28-3 The Recorded Volume of Each Sensor and the Effect of Compression

Panel (a): the first finding is how far apart the orders of magnitude are. A 2D LiDAR is only 0.01 MB per second and the IMU likewise 0.01 MB—these two could be recorded for days together with no trouble. An RGB image is 27.65 MB per second, a depth image 18.43, and a 3D point cloud 4.61. One RGB-D camera alone takes more than nine tenths of the whole.

The second finding matters more: general compression is next to useless on sensor data. The compression ratio is only 1.1 times for the 3D point cloud, 1.5 for the depth image, and 1.2 for the RGB image. The reason is that these data are by nature high-entropy floating point or near-random low bits, and zlib can find no repeated pattern to exploit. The IMU even grows larger after compression (0.9 times), because the messages are so small that the format's own header takes the advantage.

What really works is an encoding specific to the domain. The same RGB images compressed with JPEG (quality 85) fall from 27.65 MB/s to 0.98—twenty-eight times. The contrast shows one thing: the effect of compression depends on whether the encoder understands the structure of the data, and not on how high the compression level is turned.

Panel (b) converts this into actual capacity. With every sensor running and nothing compressed, an hour is 183 GB; with zlib on everything it is 140 GB, a saving of only 23 percent; and the practical combination of JPEG for the images and zlib for the rest is 63 GB, 35 percent of the raw. The red dashed line is 128 GB—the capacity of a common USB stick. Which is to say that without a trade-off a stick will not hold even an hour.

Three Decisions Before Recording

By the figures of Figure 28-3, three decisions should be made in planning a recording.

One: which topics really need recording? Most offline SLAM analysis needs only the LiDAR, the IMU, and the odometry—under 5 MB a second together, and only 17 GB for an hour. Images need not be recorded throughout unless visual SLAM or semantic labeling is to be done.

Two: in what format are the images? A compressed image topic (JPEG) and a raw one differ twentyfold. If the purpose is to run a feature-based method afterwards, JPEG at quality 85 or above usually suffices; if a photometric method is intended (the direct method of Section 20.2), raw images must be recorded.

Three: should the rate be reduced? Images at 30 fps are a waste for most offline analysis and 10 fps is often enough, which cuts two thirds at a stroke.

28.5 RViz, Gazebo, and Foxglove

This section introduces the three tools that run through Part VIII: visualization, simulation, and debugging analysis.

RViz2: Seeing the Robot's World

RViz2 is the three-dimensional visualization tool built into ROS 2, and the debugging weapon this book has mentioned repeatedly since Chapter 10. It can display at once:

- Sensor data: LiDAR scan points, point clouds, images.
- Coordinate frames: the axes of the TF tree (the when the map is crooked, look at TF first check of Chapter 10).
- SLAM output: the occupancy grid map, the estimated trajectory, the particle cloud (Case 11-2 in Chapter 11), key frames and the covisibility graph (Chapter 22).
- Navigation information: the planned path, the cost map, the goal point (Chapter 29).

RViz2 has kept improving in recent years—adding a regular-expression TF filter so that transforms may be shown or hidden by name, showing a topic's rate automatically in the status bar, and allowing subscription to point_cloud_transport topics, among other things.

Gazebo: Experimenting Without a Robot

Gazebo supplies physical simulation and sensor simulation, so that students may verify a SLAM algorithm in full in a virtual environment. Since Chapter 8 this book has advised simulate first, then cheap hardware, and pursue accuracy afterwards, and Gazebo is the first station on that road.

- What is simulated: the kinematics and dynamics of the robot, the measurements of LiDAR and camera (with noise), and the geometry and collisions of the environment.
- Its value in teaching: every student can have a robot of their own, with no limit from the number of machines; scenes may be designed at will (a long corridor, a glass partition) to reproduce the various difficult situations discussed in Chapters 13 and 14; and dangerous experiments may be run safely.
- Matching the versions: from Jazzy onward ROS 2 recommends a particular Gazebo version for each ROS release and supplies Gazebo vendor packages so that users may install and begin quickly. Follow the official pairing when installing, to avoid problems of compatibility.

Foxglove: More Powerful Debugging and Analysis

Foxglove Studio is a multi-protocol visualization tool developed by a company founded in 2021 by engineers formerly of a self-driving car company. It runs on the desktop, in the browser, and in the cloud, supports ROS 1, ROS 2, MCAP, MAVLink,

MQTT, and raw WebSocket, and offers panels for 3D, plotting, images, state transitions, maps, and more.

The reason it has been taken up is that RViz2, capable as it is, has its limits: it depends on ROS and can hardly inspect a non-ROS system; its ability to analyze recordings is weak; it is hard to collaborate on or share (only by screenshot and screen recording); and it has no support in the browser or on a tablet.

- Its value in teaching: the browser version lets students inspect data with nothing to install, and a link may be shared with a teacher or a classmate, which makes discussion and marking convenient.
- Its value in research: powerful plotting and time-series analysis, which suits the trajectory error analysis of Section 22.7.
- A recommendation: use RViz2 for live three-dimensional visualization (watching what the robot is doing as it runs) and Foxglove for analyzing the data afterwards (finding where it went wrong). The two complement each other rather than replace each other.

28.6 A Method for Reading an Open-Source Project on GitHub

One great good fortune of the SLAM field is that nearly every important system is open source. But faced with a project of tens of thousands of lines, students often do not know where to begin. This section offers a systematic method of reading.

Five Steps

1. Read the paper first, then the code: this is the most important advice of all. ORB-SLAM3, Cartographer, and LOAM all have papers of their own; understand

the architecture of the algorithm first (as this book did in Chapters 15, 18, 22, and 23) and reading the code will take half the effort. Jumping straight into the code usually means getting lost in the details.

2. Read the README and the documentation: confirm the dependencies, the supported platform versions, and the commands that run the examples. Mind the hardware requirements particularly—Case 18-3 in Chapter 18 noted that many students are held up because the hardware specification does not match, and that this is usually stated plainly in the documentation.

3. Get it running first, then read the code: run the system through on the dataset the project supplies and build up an experience of success. Read the code once it runs and you will have a concrete reference—that trajectory just now on the screen was produced by this piece of code.

4. Follow it down from the entry point: find the main function or the ROS node's entry point and read downstream along the flow of data. Do not try to understand the whole at once; trace instead which functions one frame of data passes through, in order.

5. Add print statements to verify your understanding: put output statements at the key places, run it, and observe when and how often they fire. The extended challenge of Implementation 22-1 in Chapter 22 is exactly this method—tracing under what conditions ORB-SLAM2 switches among its three tracking strategies.

What to Look for While Reading

In reading a SLAM project it is advised to look for the modules corresponding to the structure of this book:

- Where the front end is: the code for feature extraction or scan matching (Chapters 13 and 20).
- Where the back end is: the code that calls the optimization library (g2o, Ceres, GTSAM of Chapter 12).
- Where the loop closing is: the code for place recognition and verification (Chapters 22 and 23).
- The data structures: how key frames, map points, and the pose graph are defined (Section 22.6).

Map the modules of the code onto the chapters of this book and you will find the correspondence between theory and implementation very clear—which is also why this book deliberately takes architecture rather than mathematical detail as its main thread.

Making Good Use of Community Resources

- Issues: a project's issue tracker is a treasury. The problem you have met has very likely been asked already and answered. Searching for the error message will often find the answer directly.
- Forks and branches: the original project may no longer be maintained, but the community often has a maintained fork (the advice of Section 22.7). Look for forks with more stars and recent updates.
- Contributing back: if you solve a problem not yet recorded, write the solution up as an issue or a pull request and give it back to the community. This is the most direct way of taking part in the open-source ecosystem, and good material for a résumé besides.

28.7 From Block-Based Implementation at High School to Source Code at University

This chapter closes by laying out a complete path of learning from high school to university and on to graduate work. It is also the master plan of the whole book's teaching design.

Stage	Aim	Chief tools	Chapters of this book
One: observing and experiencing	Understanding what SLAM does	Simulators, observing a robot vacuum	Chapters 1–3, 16
Two: block-based operation	Mapping and navigating with a ready-made system	Gazebo, launch files, tuning parameters	Chapters 14, 15, 29
Three: data and analysis	Recording data and assessing quality	rosbag2, Python, evo	Sections 13.7, 22.7
Four: reading source code	Understanding how the system is realized	GitHub, debugging tools	Chapters 22, 23
Five: modifying and innovating	Improving or integrating systems	C++, CMake, algorithm design	Chapters 30, 31

Table 28-4 The Five Stages of Learning SLAM

Teaching Advice for Each Stage

- Stage one (suitable for high school): no code at all; the point is to build intuition. Operate a simulator, observe the robot vacuum at home (Section 16.7), do the paper activities (the implementations of Chapters 12, 16, and 25). The aim is that students know what problem SLAM solves and why it is hard.
- Stage two (advanced high school / undergraduate): use a ready-made system and learn to start it, set its parameters, and observe the results. The point is to understand what the parameters mean and what they do, not to write programs. The implementations of Chapters 14 and 15 are designed at exactly this level.
- Stage three (undergraduate): begin to write programs—but programs for analysis and tooling, not the core of SLAM. Record data, compute errors, draw figures. What this stage cultivates is the engineering habit of letting the data speak.
- Stage four (later undergraduate / project): read the source of an open-source system and understand how the theory becomes a program. The experience of the three stages before now becomes the scaffolding of that understanding.
- Stage five (project / graduate): modify an existing system or integrate several of them to solve a particular problem. The project of Chapter 31 and the AI integration of Chapter 32 both belong at this level.

Three Principles of Teaching

1. Succeed first, go deep after: let students have something to show at every stage. Asking them to read source code too early easily brings frustration and abandonment.
2. Demonstrate failure deliberately: that every chapter of this book discusses common failures is by design. Seeing under what conditions a system breaks

teaches more than seeing it run smoothly—the challenge tests of Sections 14.6 and 26.5 are in this spirit.

3. Keep environment problems apart from the subject matter: as Section 28.1 advised, use a container or a uniform environment to keep installation problems to a minimum. Class time should be spent on SLAM and not on clearing dependency errors.

Carrying Forward

This chapter has measured the three costs of the tool chain. On languages: for the same scan registration kernel, pure Python is 147 times slower than C++ and overruns the budget of a 30 Hz loop by 63 percent, while NumPy on a large batch is only 1.5 times slower than C++ and takes 10 percent of the budget at 30 Hz. On communication: a topic has a latency of 0.022 milliseconds and a throughput of 89,862 messages per second, while a service holds the caller for the whole of the server's processing time (20 milliseconds at the server means 20.3 at the caller, and the throughput falls to 49 per second). On recording: an hour with every sensor and no compression is 183 GB, general compression saves only 28 percent, and only an image-specific encoding brings a difference of twenty-two times.

The environment and the tools are now ready. In the next chapter we make the 2D SLAM theory of Part IV actually run: from assembling the hardware, setting up TF, and publishing odometry, to mapping with Gmapping and Cartographer, localizing with AMCL, and the complete navigation stack. Chapter 29 will be this book's first complete system implementation from end to end.

28.8 Three Practical Principles for the Tool Chain

This chapter has measured three things, and each corresponds to a principle that can be applied directly.

One: Measure First, Optimize After

Figure 28-1 shows NumPy taking under a fifth of the time budget in a 30 Hz control loop. This means that most of the agonizing over should I rewrite this in C++ is beside the point. The right order is: write a working version in Python first, time it to find which part is really slow, and rewrite only that part.

The principle sounds like a platitude, but it matters particularly in a SLAM project, because the bottleneck is often in a counterintuitive place. Many suppose it to be in the matrix arithmetic, where in fact it often lies in copying or serializing data, or in waiting on a service as in Section 28.3.

Two: Blocking Is More Dangerous Than Slowness

The central message of Figure 28-2 is this: a slow module merely slows itself, while a blocking call drags the whole loop down. The former can be solved with faster hardware and the latter cannot—it is a question of architecture.

This is the same principle as the staggered execution of Section 33.8 and the asynchronous design of Section 34.5, and it is why this book asks of every system first what the update rate of this layer is. Between modules of different rates the interface must not block.

Three: Data Will Overflow

Figure 28-3 shows an hour at 183 GB where no trade-off is made. The number bears directly on planning an experiment: a day of experiments recording eight hours is 1.5 TB. And these data must further be backed up, transferred, and kept for the long term.

The practical practice is to work in tiers: record only the light topics ordinarily (LiDAR, IMU, odometry, pose) and turn the images on only when visual analysis is needed, at a reduced rate and compressed. This answers to the map maintenance of the inspection robot in Section 19.6 and the bandwidth ordering of Chapter 27—in a system with limited resources, choosing what to keep is itself a piece of design work.

What was measured	The figure	The practical principle
The cost of pure Python	147 times slower than C++; overruns the budget by 63% at 30 Hz	A real-time loop must at least be vectorized
The cost of NumPy	Only 1.5 times slower than C++ on a large batch	Rewriting in C++ is unnecessary in most cases
A service blocks	The server spends 20 ms and the caller is held 20.3 ms	Always use an action for a long task
The throughput of a topic and a service	89,862 against 9,101 per second	Use a topic where no reply is needed
The volume recorded	183 GB for an hour with everything on	Choose the topics first, then compress, and reduce the rate if need be
The effect of compression	zlib only 1.1–1.5 times, JPEG 22 times	Use an encoder that understands the structure of the data

Table 28-5 What the Three Experiments of This Chapter Imply in Practice

Formulas and Procedures

One: The Complete Procedure for Setting Up the Environment

Figure 28-4 The Procedure for Building an Environment from Nothing to Running SLAM

① Confirm the version correspondence: choose the ROS 2 distribution → find the matching Ubuntu version (Table 28-1)

↓

② Install Ubuntu: native / virtual machine / container (chosen by purpose, Section 28.1)

↓

③ Install ROS 2: add the software source and install the desktop version (with RViz2) by the official documentation

↓

④ Install the simulator: the Gazebo version officially recommended for that ROS release

↓

⑤ Create the workspace: make the directory → put in the package sources → build with colcon

↓

⑥ Load the environment settings (source) → verify: start the example nodes and confirm that they communicate

↓

⑦ Install the SLAM packages: slam_toolbox / Cartographer / Nav2 and so on

↓

⑧ The smoke test: start the simulator and SLAM and watch the map grow in

RViz2

The eighth step, the smoke test, is the crucial checkpoint: if the map can be seen growing bit by bit, the whole tool chain is working and one may go on to the implementation of Chapter 29 with confidence. If it sticks, work through the common traps of Sections 27.1 to 28.2 one by one.

Two: The Judgment for Choosing a Communication Mechanism

Figure 28-5 Deciding Among Topic, Service, and Action

What kind of communication does this function need?

↓

① Is it a continuously flowing stream of data? (sensors, state, control commands)

├─ Yes → 【Topic】publish-subscribe, asynchronous, one to many

└─ No → go to ②

↓

② Is it a one-off request that can be answered quickly? (a query, a setting)

├─ Yes → 【Service】request-response, the caller waits

└─ No → go to ③

↓

③ Is it a lengthy task whose progress must be watched? (navigation, a scanning task)

└─ Yes → 【Action】can report progress, can be cancelled

Three: The Complete Procedure for Recording and Replaying with rosbag

Figure 28-6 The Workflow from Recording Data to Analyzing Them

【The recording stage】

① Decide the topics to record: sensor data + IMU + odometry + TF (do not forget TF)

② Estimate the file size and the storage space

③ Record → produce a recording in MCAP format

↓

【The replay stage】

④ Check that the topic names agree with what the algorithm expects, and remap if need be

⑤ Replay the data → start the SLAM node at the same time

⑥ Watch live in RViz2, or analyze afterwards in Foxglove

↓

【The analysis stage】

⑦ Output the estimated trajectory → compare with ground truth → compute ATE / RPE (Section 22.7)

⑧ Adjust the parameters → return to ⑤ and replay the same data → a fair comparison

Note the eighth step: because the same data are replayed, a comparison of different parameters is a fair one. This is exactly rosbag's greatest value to research and to teaching—it holds fixed the variable of the randomness of the sensor data.

Cases

Case 28-1 The Cost of Half a Class: the Chain Reaction of a Version Mismatch

This scene happens in very nearly every SLAM course, and students would do well to take warning from it.

The situation is this: a student downloads some open-source SLAM project, follows the README to compile it, and gets a long train of error messages. They search online, try one remedy after another, install several packages that look relevant—and two hours later the problem is not only unsolved but the system has been broken as well.

The root cause is usually extremely simple: the project was written for one version of ROS 2 and the student installed another. The dependent libraries are of different versions, and the chain reaction produces dozens of seemingly unrelated errors.

There are three lessons in this case:

- First, confirm the version correspondence. Before compiling, spend five minutes confirming that the ROS version the project supports matches your environment—those five minutes will save two hours.
- Second, read the error messages from the first one. Compilation errors often have a chain effect, and the dozens that follow may all come from the first. Reading backward from the end is a common mistake.
- Third, containers are your friend. Package the environment in a container; if you break it, rebuild it, with no contamination of the host system, and the whole class is guaranteed the same environment (the advice of Section 28.1).

The deeper lesson is this: in engineering practice, environment problems take up far more time than expected. This is no want of ability on the student's part but the normal state of the field. The difference between an experienced engineer and a

beginner is not that the one meets no problems, but that the one can judge quickly what class of problem it is and find the matching remedy.

Case 28-2 From a rosbag to a Paper: the Many Values of One Recording

The value of rosbag is displayed most clearly in a real research workflow.

Suppose a student is to do a project comparing how different SLAM systems perform in the corridors of a departmental building. Without rosbag they must run a physical experiment once for each system—but the route, the speed, the light, and the flow of people differ from run to run, so the comparison carries no conviction; and each time the robot must be borrowed and the site prepared, which takes time and labor.

With rosbag the workflow is wholly different: record one high-quality dataset (images, IMU, LiDAR, TF) and test every system on that same data afterwards. This brings four benefits:

- A fair comparison: every system faces exactly the same input, and the differences come purely from the algorithms.
- Reproducibility: someone given your data can reproduce your result—the basic requirement of scientific research.
- Efficiency: parameters can be tuned again and again with no experiment to rerun. Dozens of parameter sets can be tested in an afternoon.
- Long-term value: the students of next year and the year after can still use these data. The record and publish a dataset of your local site recommended in Section 26.7 is exactly this line of thought.

The concrete advice is this: let every year's project students contribute one dataset

of their own site, and the accumulation becomes a valuable local dataset. For the research that follows, for teaching demonstrations, and even for exchange with other schools, it is a real asset.

Case 28-3 The Evolution of the Tools: from rosbag to MCAP, from RViz to Foxglove

The tool chain introduced in this chapter has evolved markedly in recent years, and the trend deserves students' attention.

In recording formats, MCAP has become the new standard in fact, displacing the original rosbag format, and has been rosbag2's default storage format since ROS 2 Jazzy. Its key advantage is being self-describing—the recording carries the message definitions within it and needs no extra packages to be read.

In visualization, Foxglove makes good RViz2's limits: RViz2 depends on ROS, can hardly inspect a non-ROS system, is weak at analyzing recordings, is hard to collaborate on or share, and has no support in the browser or on a tablet. Foxglove runs on the desktop, in the browser, and in the cloud, and handles ROS 1, ROS 2, MCAP, MAVLink, MQTT, and other protocols.

There is a pattern behind this evolution worth noting: what drives the change is often not the ROS core team but the practical needs of industry. Foxglove was founded by engineers formerly of a self-driving car company, and they turned the pain points of self-driving development—analyzing great volumes of data, collaborating across teams, integrating non-ROS systems—into tools.

The lesson for students is that tools will keep evolving and there is no need to cling to any particular one. What matters is understanding what problem this tool solves—

know the essence of the problem and you can take up a new tool quickly, and judge when it is time to change tools. This is the same claim this book makes throughout: understanding the principle matters more than remembering the operations.

Implementation Activities

Implementation 28-1 Setting Up a SLAM Development Environment and Completing a Smoke Test

Purpose: to build a working ROS 2 development environment in full, grow familiar with the three communication mechanisms and the rosbag tool, and complete a first mapping run in simulation. This activity is the necessary preparation for Chapters 29 and 30.

Part One: Building the Environment (everyone)

1. Choose a ROS 2 version and the corresponding Ubuntu version by Table 28-1, and record your choice and the reason for it.
2. Choose a way of installing by Section 28.1 (native, virtual machine, or container) and complete the installation of Ubuntu and the ROS 2 desktop version.
3. Verify the installation: start ROS 2's built-in example nodes (talker and listener, say) and confirm that the two communicate.
4. Create a workspace: build the directory structure, build an empty workspace with colcon, and load the environment settings.
5. Install the simulator and the TurtleBot3 packages, start the simulated environment, and confirm that the virtual robot can be seen on screen.

Part Two: Exploring the Communication Mechanisms (everyone)

1. List every topic active in the system and find the SLAM-related ones—/scan, /odom, /tf.
2. Inspect the contents of a message on /scan, set it against the LaserScan format of Section 8.3, and find the fields angle_min, angle_increment, and ranges.
3. Observe the publishing rates of the topics, record the rate of the LiDAR, the odometry, and TF, and consider why they differ.
4. List the services and actions available and find the action interface related to navigation (which Chapter 29 will use).
5. Write the simplest possible node in Python: subscribe to /scan, compute and print the distance and the direction of the nearest obstacle. This is the ROS version of the extended challenge of Implementation 8-1 in Chapter 8.

Part Three: Recording and Replaying with rosbag (everyone)

1. Drive the simulated robot about the environment by remote control while recording /scan, /odom, /tf, and /imu (if present) with rosbag2, and observe the format and the size of the file produced.
2. Stop the simulator, replay the recorded data on its own, display the LiDAR scan and TF in RViz2, and confirm that it is reproduced correctly.
3. An exercise in remapping topics: rename /scan to /laser_scan on replay and observe whether the node that subscribed to /scan still works, so as to understand the remapping technique mentioned in Section 28.4.
4. Leave TF out of the recording deliberately and replay again, and observe what happens in RViz2—verifying for yourself the do not forget TF warning of Section 28.4.

Part Four: the Smoke Test—a First Map (everyone)

1. Start the simulator and a SLAM package (slam_toolbox or Cartographer are advised), and display the map, the LiDAR, and TF in RViz2.
2. Drive the robot slowly once around the environment and watch the occupancy grid map grow bit by bit (the theory of Chapter 13 made concrete here).
3. Save the map and assess its quality by the method of Section 13.7: are the walls straight? Does the loop close?
4. Write the whole procedure into one launch file, so that next time the simulator, the SLAM, and RViz2 start with a single command.

Discussion Questions

1. Which ROS 2 version did you choose? By the advice of Section 28.1, what are the merits and defects of that choice for finding teaching resources and for long-term maintenance?
2. What are the publishing rates of the LiDAR, the odometry, and TF? Why do they differ? (Hint: the sensor properties of Chapters 7 and 13.)
3. What happened on replaying data recorded without TF? Which idea of Chapter 10 does this bear out?
4. By the decision procedure of Figure 28-2, which communication mechanism should be used to have the robot carry out one complete scanning task of a room, with its progress visible and the task cancellable? Why?
5. How did your first map turn out? If it was poor, which of the failure modes of Section 14.6 do you think the likeliest cause?

Extended Challenge (university level)

First: containerize your development environment—write a container configuration packaging the complete ROS 2 environment and the SLAM packages needed, so that

classmates may obtain an identical environment with one command. This is extremely practical for teaching assistants and project teams, and an essential skill in modern software engineering. Second: an exercise in reading an open-source project—choose one of the open-source SLAM projects introduced in this book (slam_toolbox is advised to begin with, its code being of moderate size), read it by the five steps of Section 28.6, and draw a flow chart of how the data pass from input to output. Compare your chart with the architecture figures of the corresponding chapters of this book and see how theory and implementation correspond.

Further Reading

1. The official ROS 2 documentation (docs.ros.org). (Installation guides, tutorials, and API documentation; the first-hand reference for the implementations.)
2. REP-2000: ROS 2 Releases and Target Platforms (ros.org/reps). (The official specification of the version and platform correspondence, the basis of Section 28.1.)
3. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022. (The authoritative introduction to the design philosophy of ROS 2.)
4. Installing Gazebo with ROS, in the official Gazebo documentation (gazebosim.org). (The official advice on pairing ROS and Gazebo versions.)
5. The official MCAP documentation (mcap.dev) and the Foxglove documentation (docs.foxglove.dev). (The recording format and the visualization tool of the new generation.)
6. The slam_toolbox open-source project (GitHub). (The mainstream 2D SLAM of ROS 2, of moderate size and a good place to begin reading source code.)

7. The ROBOTIS TurtleBot3 e-Manual (emanual.robotis.com). (A complete implementation guide for the teaching platform, covering both simulation and the real robot.)

Summary

1. The versions of ROS 2 and Ubuntu are bound together, and installing the wrong one brings a chain of dependency errors; ROS 2 releases each May and the even years are LTS, and for teaching a mature LTS version richest in resources is advised, with the whole class on one version. There are three ways to install—native, virtual machine, and container—and a container is advised at the teaching stage to keep the environment consistent.
2. Python suits rapid prototyping and data analysis and C++ the core of SLAM and high-rate computation; CMake manages the compilation of C++, and being able to read find_package and target_link_libraries will solve most compilation problems; colcon builds a workspace by its dependencies, and the environment settings must be loaded after a build.
3. Each of the three communication mechanisms has its place: a topic for a continuously flowing stream (sensors, state, commands), a service for a quick one-off request, and an action for a lengthy task whose progress must be watched (navigation, say); the wrong mechanism makes the system block and hard to debug.
4. rosbag2 makes record once, analyze any number of times possible, and underwrites reproducibility, offline development, debugging, and the building of datasets; MCAP has been the default format since ROS 2 Jazzy and is indexed and self-describing; do not forget TF when recording, and mind the three common problems of version, TF, and topic names on replay.

5. RViz2 is for live three-dimensional visualization, Gazebo supplies simulation with no hardware (the versions must be paired as officially recommended), and Foxglove makes good RViz2's limits in analyzing recordings, in collaboration and sharing, and with non-ROS systems; the three complement each other rather than replace each other.
6. The five steps for reading an open-source project: read the paper before the code, read the README to confirm the dependencies and the hardware requirements, get it running before reading the code, follow the data flow down from the entry point, and add print statements to verify your understanding; and make good use of the issue tracker and of forks maintained by the community.
7. The path of learning has five stages: observing and experiencing, block-based operation, data and analysis, reading source code, and modifying and innovating; the principles of teaching are to succeed first and go deep after, to demonstrate failure deliberately, and to keep environment problems apart from the subject matter.

The environment and the tools are now ready, and students have the means to make all the theory before this actually run. In the next chapter we implement the 2D SLAM of Part IV from end to end: assembling the hardware and setting up TF, publishing odometry, mapping with Gmapping and Cartographer, localizing with AMCL, and finally connecting the complete navigation stack so that the robot really walks to a goal by itself. This will be the book's first complete end-to-end system implementation.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 29

Mapping with a 2D LiDAR Rover

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

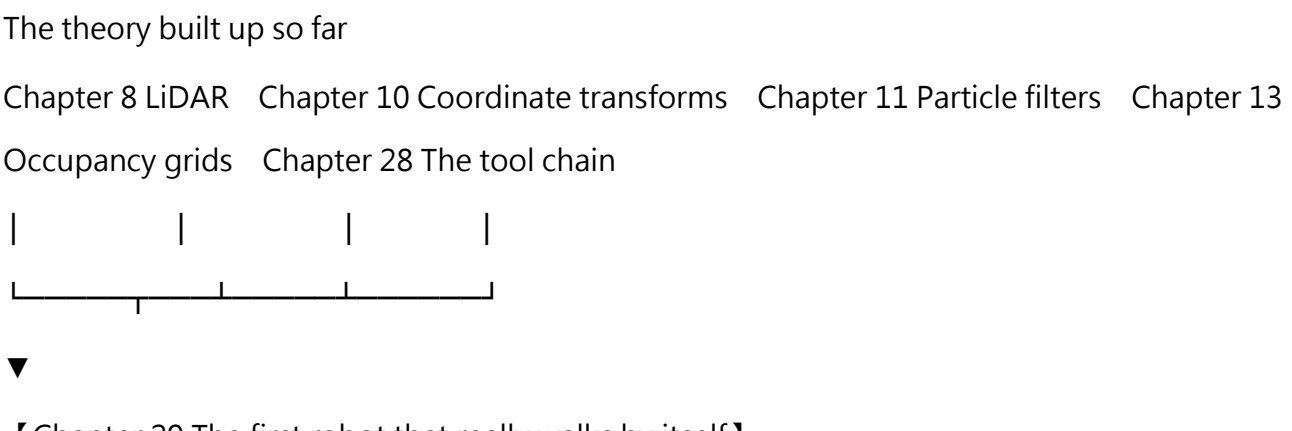
Where This Chapter Sits in the Book

This is the book's first end-to-end system implementation—from the turning of a wheel to the robot walking to a goal by itself, with every link in between to be connected by hand.

What most distinguishes this chapter from those before is that it deals with the places where things are wired together. The algorithms themselves were covered in Chapters 13, 14, and 15; what really holds a project up is a coordinate transform set wrongly, odometry parameters never calibrated, and not knowing where to begin looking when the system will not move. What these problems have in common is that they raise no error—they only make the result look somehow odd.

The three figures of this chapter exist to turn somehow odd into something measurable: Figure 29-2 demonstrates the standard method of calibrating odometry and explains why one must drive once clockwise and once counterclockwise; Figure 29-3 sets side by side the maps built under four common coordinate transform errors, as a table of symptoms; and Figure 29-4 runs an actual global localization and sees how many steps the particles need to converge.

Figure 29-0 The Link Map of Chapter 29



debugging

▼

Chapter 30 Visual SLAM in practice (changing to a camera)

▼

Chapter 31 Integrating a project and assessing it

Learning Objectives

The theory and the tools of the twenty-seven chapters before are now all in hand. This chapter is the book's first complete system implementation from end to end: beginning with the hardware and the coordinate settings, through mapping and localization, and ending with the robot walking to a goal by itself. After studying this chapter, students should be able to:

1. Explain what a 2D SLAM rover is made of and the role of each component.
2. Write a URDF robot description and set up the TF coordinate tree correctly.
3. Compute and publish odometry from the wheel encoders.
4. Run 2D SLAM to build a map and save the map file.
5. Explain the key settings of AMCL localization and its common problems.
6. Explain the architecture of the Nav2 navigation stack and the complete data flow of one navigation task.
7. Troubleshoot a navigation system in a systematic order.

Core Concepts

中文名詞	English Term	Explanation
機器人描述檔	URDF	An XML description of a robot's links, joints, and sensor mountings.
連桿與關節	Link / Joint	The basic elements of URDF: the rigid bodies and the joints connecting them.
狀態發布器	robot_state_publisher	The node that publishes the TF transforms among the links from the URDF.
差速驅動	Differential Drive	A drive in which direction is controlled by the difference between the two wheel speeds.
里程計	Odometry	The pose reckoned by integrating wheel speeds, supplying the odom→base_link transform.
代價地圖	Costmap	A navigation map annotating an occupancy grid with the cost of passage.
膨脹層	Inflation Layer	The cost map layer expanding obstacles outward by a margin of safety.
全域規劃器	Global Planner	Plans a complete path from start to goal on the static map.
控制器	Controller	The local planning component turning a path into velocity commands.
行為樹	Behavior Tree	The structure Nav2 uses to organize the flow of a navigation task.
復原行為	Recovery Behavior	Actions attempted to escape when navigation fails—turning in place, backing up.
生命週期節點	Lifecycle Node	A kind of ROS 2 node with explicit transitions among start-up states.
速度指令	cmd_vel	The linear and angular velocity command sent to the chassis.

29.1 The Hardware: Chassis, Motors, Encoders, and a 2D LiDAR

This section explains what components a 2D rover able to run SLAM needs, and what to consider in choosing them.

The Five Components

- The chassis and motors: the commonest arrangement is differential drive—two driven wheels with one or two casters. It is simple in structure and intuitive to control (the 2D pose of Section 10.3 corresponds to it exactly), and it is the standard choice for a teaching platform.
- The encoders: fitted to the motor shaft or the wheel axle, they count how far the wheel has turned. They are the source of the odometry (Section 29.3), and the indispensable half of Section 13.4's prediction by odometry, correction by LiDAR.
- The 2D LiDAR: the rotating laser scanner introduced in Chapter 8, which supplies the /scan topic. An entry-level model for teaching measures to about 8 to 12 meters, which is enough for indoor mapping.
- The microcontroller: it drives the motors and reads the encoders and communicates with the computer above it. What it handles is real-time control at the millisecond scale, which does not belong on a single-board computer.
- The single-board computer: it runs ROS 2 and the SLAM algorithm. This division—the microcontroller for real-time control and the single-board computer for computation—is exactly the two-chip architecture discussed in Case 25-2 of Chapter 25, embodied in a teaching platform.

Three Considerations in Choosing

1. The LiDAR's mounting height and occlusion: the LiDAR must have a clear 360-degree view. If the body or another component blocks part of the angle, those directions are blind forever. Check on mounting that the scan is really unobstructed.
2. The resolution of the encoders: too low a resolution and the odometry will be inaccurate. The encoders of an ordinary teaching platform suffice, but with a chassis of your own making, confirm that the pulses per revolution are enough to resolve a small movement.
3. Computing power: 2D SLAM does not demand much computation (as against the 3D and visual SLAM of Parts V and VI), and a common single-board computer will serve. If Nav2 navigation is to run at the same time, mind whether the memory suffices.

Three Choices of Platform

- A ready-made teaching platform: completely integrated, fully documented, and supported by a community, it lets students get quickly to SLAM itself rather than to debugging hardware. It suits a course with limited time.
- A platform of your own assembly: cheaper, and deeper as a lesson—students handle every link by hand and understand the system far better than with a ready-made platform. It suits a project and a course with time to spare.
- Simulation alone: no hardware at all, and yet the whole of the procedures of Sections 28.2 to 29.7 can be experienced. Chapter 28's advice of simulate first, then hardware applies to this chapter particularly—get the whole pipeline running in simulation and then move it to the real rover, and a great deal of debugging time is saved.

29.2 Setting Up TF and URDF

Chapter 10 stressed that when the map is crooked, look at TF first; this section explains how to build the coordinate tree correctly. It is the part of the whole implementation most apt to go wrong and most in need of care.

URDF: the Skeleton of the Robot

URDF (the Unified Robot Description Format) describes the physical structure of a robot in XML: what rigid bodies (links) there are, how they are joined (joints), and the size and relative position of each.

For SLAM, URDF's most important function is to describe where the sensors are mounted—the extrinsics mentioned repeatedly in Chapters 5 and 10. If the LiDAR position written in the URDF differs from the actual mounting by a few centimeters or a few degrees, the map built will be blurred or out of register.

The Typical Structure of the Coordinate Tree

Section 10.7 introduced the standard hierarchy of frames in ROS; on a real rover it takes this concrete form:

Figure 29-1 The Complete TF Tree of a 2D SLAM Rover

map (the map frame) globally fixed, does not drift, jumps

↓ published by SLAM or AMCL

odom (the odometry frame) continuous and smooth, drifts over time

↓ published by the odometry node (Section 29.3)

base_footprint (the robot's projection on the ground; optional)

↓

base_link (the center of the robot body)

├──→ laser (the LiDAR) ↖

├──→ imu_link (the IMU) published from the URDF by

├──→ left_wheel robot_state_publisher on the fixed mountings

└──→ right_wheel ↙

Who Publishes Which Segment

This is where a beginner is most apt to be confused, and it must be made clear:

- map → odom: published by the SLAM system (while mapping) or by AMCL (while navigating). The official documentation states explicitly that this transform is one of the principal requirements of the Nav2 system.
- odom → base_link: published by the odometry system. The role of the odometry system is precisely to supply this transform, and its source may be LiDAR, radar, wheel encoders, visual-inertial odometry, or an IMU; the aim is to supply a smooth, continuous local frame based on the robot's motion.

- base_link → each sensor: supplied by robot_state_publisher from the URDF, which publishes the transform from base_link to each sensor.

Common TF Errors

1. A broken coordinate tree: some segment has no node publishing it, so that the chain from map to laser is broken. RViz shows a cannot find transform error.
2. Wrong extrinsics: the LiDAR's mounting position or orientation is entered wrongly. The symptom is that the scan points are out of register with the real environment—put the robot against a wall, and if the scan points in RViz do not line up with the wall, this is the problem.
3. Duplicate publishing: two nodes publish the same segment at once, which causes conflict and jitter. This happens easily when several packages are integrated.
4. Time stamp problems: the transform's time stamp is not synchronized with the sensor data, which causes plain misalignment during fast motion (Section 10.6 noted the importance of time stamps).

How to Verify

- A visual check: display the TF axes and the LiDAR scan together in RViz and confirm that the scan points sit sensibly relative to the robot.
- Produce a picture of the TF tree: generate a graph of the coordinate tree with the tool and confirm that the structure is complete, with nothing broken or duplicated.
- A static check: place the robot at a known position (hard against a corner, say) and check whether the scan points fall at the right distances.

29.3 Publishing the Odometry

Odometry is the other cornerstone of 2D SLAM. This section explains how it becomes a ROS odometry message from the readings of the encoders.

From Encoder to Pose

The odometry computation of a differential-drive rover is a direct application of the dead reckoning of Chapter 3 and the coordinate transforms of Section 10.3:

1. Read the encoders: obtain how far the left and the right wheel each turned within this interval.
2. Convert to wheel displacement: multiply the revolutions by the circumference of the wheel to get how far each wheel travelled.
3. Compute the body motion: the mean of the two displacements gives the distance advanced, and their difference divided by the wheel base gives the turn (the formula is Formula One of this chapter).
4. Update the pose: by the 2D pose composition of Section 10.3, add the relative motion onto the current pose, remembering to handle the wrapping of the angle.
5. Publish: publish the odometry message (carrying pose and velocity), and publish the odom → base_link transform at the same time.

Calibration: the Key to Accuracy

Section 14.6 noted that wheel slip and inaccurate odometry are the first cause of a failed map, and what within that is most often overlooked is the calibration of the parameters. Two parameters are crucial:

- The wheel diameter: if the value set differs from the real one, the distance travelled in a straight line is systematically off, and the whole map is scaled up or down in proportion. To calibrate, drive the rover straight a fixed distance (3 meters, say) and compare the odometry reading with a tape measure.
- The wheel base: the distance between the centers of the two driven wheels. Set it wrongly and the angle turned is off, which affects the map more severely than the wheel diameter does—because an error in angle is magnified by the movement that follows (Section 7.3 discussed the magnifying effect of an attitude error). To calibrate, turn the rover ten times in place and compare the angle the odometry accumulates with 3,600 degrees.

These two calibrations should be done before the first map is built. Many students spend hours tuning SLAM parameters without noticing that the trouble lies in uncalibrated odometry—the commonest wrong road in implementation.

What Is Required of the Odometry

The aim of odometry is to supply a smooth, continuous local frame, while the global localization system updates the transform relative to the global frame so as to compensate the odometry's drift.

Note the requirement of smooth and continuous: odometry need not be absolutely accurate (SLAM will correct the drift), but it must be smooth and not jump. If the odometry jumps because of a bug in the program or a dropped packet, SLAM's motion prediction fails and the map is bound to fail with it.

Going further: Fusing the IMU

The angle estimated from wheel speed alone goes wrong easily through slip. In practice a filter often fuses the wheel speeds with the IMU's angular rate (Chapters 7

and 11) for a more robust odometry. In the ROS 2 ecosystem the EKF node of the robot_localization package can supply the odom → base_link transform, which helps particularly where the ground is uneven or slippery.

Measuring It: What Happens When the Odometry Parameters Are Wrong

Section 29.3 says the odometry must be calibrated, but not how far wrong it goes uncalibrated, nor how to tell which parameter is at fault. This subsection answers both at once with the standard square path test.

A differential-drive odometry has only two crucial parameters: the wheel radius and the wheel base. Each causes an error of its own character: set the wheel base too small and the robot believes it is turning more than it is, so that every turn falls short; let the left and right wheels differ in diameter and the robot veers to one side while driving straight, though its odometry believes it went straight. The trouble is this—from one experiment alone the two errors look exactly alike.

The method of the experiment is the classic square path test: command the robot to drive a square 2 meters on a side by its own odometry and return to the start, then measure where it actually stopped. What is crucial is to drive it once clockwise and once counterclockwise.

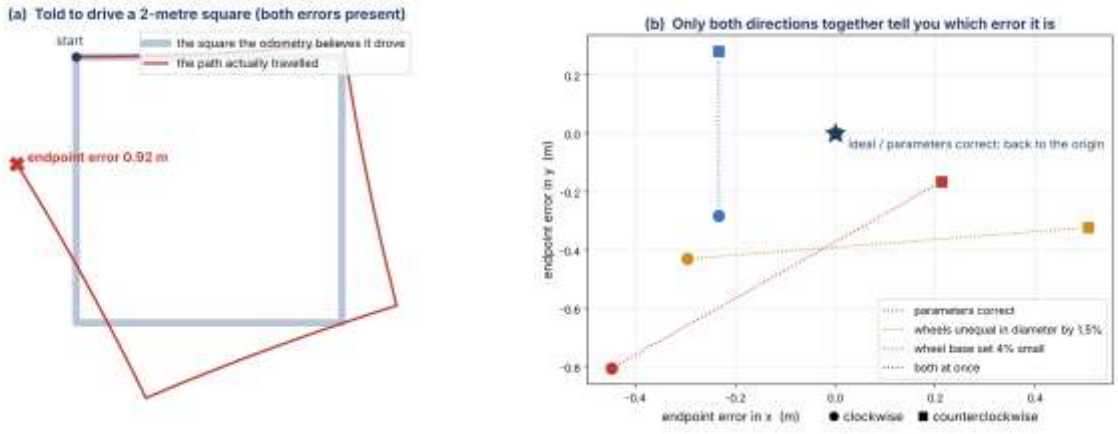


Figure 29-2 The Square Path Test: the Fingerprint of an Odometry Error

Panel (a) draws the result of a rover carrying both errors at once (the wheel base set 4 percent small, the right wheel actually 1.5 percent larger than the left) driving the square clockwise. The thick grey line is the path the odometry believes it travelled—a perfect square; the red line is the path it actually travelled. Every corner falls short of the turn and every straight stretch curves slightly, and over four sides the end stands 0.92 meters from the start. In a site 2 meters on a side, that error is nearly half the length of a side.

Panel (b) is the point of this figure. Four cases are each run clockwise (circles) and counterclockwise (squares), and the endpoint error of each is drawn on one plot. With the parameters correct, both points fall at the origin (an error of 0.001 meters).

The two points for a wheel base 4 percent wrong (blue) fall at the same x with y equal and opposite: (−0.233, −0.282) and (−0.233, +0.282). That arrangement—same x, mirrored y—is the fingerprint of a pure wheel base error, because the error in the turn angle is symmetric between the two directions.

Wheels unequal in diameter by 1.5 percent (amber) is wholly different: the two points fall at (−0.296, −0.430) and (+0.506, −0.322), far apart and with no plain symmetry. The

direction in which it veers while driving straight does not change with the direction of travel, but the order of the turns changes how that veer accumulates.

With both errors present at once (red), the endpoint error is 0.92 meters clockwise and only 0.27 counterclockwise. This is exactly why driving one direction alone will deceive you: a student who tried only counterclockwise might see 0.27 meters and think it tolerable, with no idea that the other direction is out by 0.92.

The Order of Calibration

By the fingerprints of Figure 29-2(b), the calibration may be done in two steps.

The first step deals with the wheels being unequal in diameter: adjust the ratio of the left and right radii until the clockwise and counterclockwise endpoints fall roughly symmetric about the origin. What remains is then mainly the wheel base error.

The second step adjusts the wheel base: if the turns fall short (the endpoint lies to the inside of the path), raise the wheel base value; otherwise lower it. Rerun both directions after each adjustment, until both points are near the origin.

One practical reminder: the result of a calibration changes with the floor material (carpet and tile differ in how much the wheels slip) and drifts as the tires wear. This test is therefore worth repeating at the start of every semester or on changing site—it takes ten minutes and saves all the debugging of why is the map crooked afterwards.

29.4 Building a Map with 2D SLAM

With the preparation done, a map can at last be built. This section explains the procedure and the choice of tool.

The Choice of Tool

Chapters 14 and 15 introduced Gmapping and Cartographer, and in a modern ROS 2 environment there is a third common choice. Nav2 officially lists SLAM Toolbox as its default SLAM algorithm, used to localize and to produce a static map.

Tool	Theoretical basis	Features	Where it suits
slam_toolbox	Graph optimization (Chapter 12)	The ROS 2 default; few parameters; supports continuing a map	The first choice for teaching
Cartographer	Graph optimization + submaps	Strong loop closure; supports 3D; more parameters	A large site needing loop closure
Gmapping	Particle filter (Chapter 11)	Intuitive in concept; the mainstream of the ROS 1 era	Understanding the particle filter

Table 29-1 A Comparison of Three 2D SLAM Tools

Teaching advice: build an experience of success with slam_toolbox first (it has few parameters and is easy to take up), then feel the power of loop closure with Cartographer (Implementation 15-1 in Chapter 15), and finally set Gmapping beside them to understand the difference between a particle filter and graph optimization.

The Mapping Procedure

1. Start the hardware: bring up the robot driver, the LiDAR, and the odometry node, and confirm that both /scan and /odom carry data.
2. Check TF: confirm that the coordinate tree is complete (the verification methods of Section 29.2). This step may not be skipped.

3. Start SLAM: start the chosen SLAM node. Confirm that it supplies the map → odom transform and publishes the /map topic.
4. Start RViz: display the map, the scan, TF, and the robot model, so that it may be watched live.
5. Drive slowly and map: Section 14.6 stressed that driving a little slower is the single variable that most raises a beginner's rate of success. Be slower still in the turns.
6. Assess the quality: by Section 13.7, check whether the walls are straight, whether the loop closes, and whether anything is doubled.
7. Save the map: save it with nav2_map_server, which produces two files, a .yaml and a .pgm, to represent the map.

The Two Files of a Map

- The .pgm: a grayscale image file, which is the occupancy grid map visualized—black for obstacle, white for free space, grey for unknown (the three states of Section 13.1). It can be opened and inspected in any image viewer.
- The .yaml: the description file, recording the image file's name, the resolution (meters per pixel), the coordinates of the origin, and the probability thresholds for judging occupied and free.

These two files are the map product mentioned in Section 14.5. In navigation afterwards they are loaded by map_server for AMCL to localize against and for path planning.

Measuring It: What a Map Looks Like When the Transform Is Wrong

Section 29.2 set up the coordinate transform tree and Section 29.7 lists it first on the debugging checklist. But what does a wrong transform actually look like? This

subsection sets four common errors deliberately and puts the resulting maps side by side, as a table of symptoms.

The experimental setting is an indoor scene of 10 × 7 meters; the robot goes once around an elliptical trajectory and builds an occupancy grid map with a 240-beam 2D LiDAR. The real LiDAR is mounted at the center of the body, while the extrinsics the system believes are set wrongly on purpose.

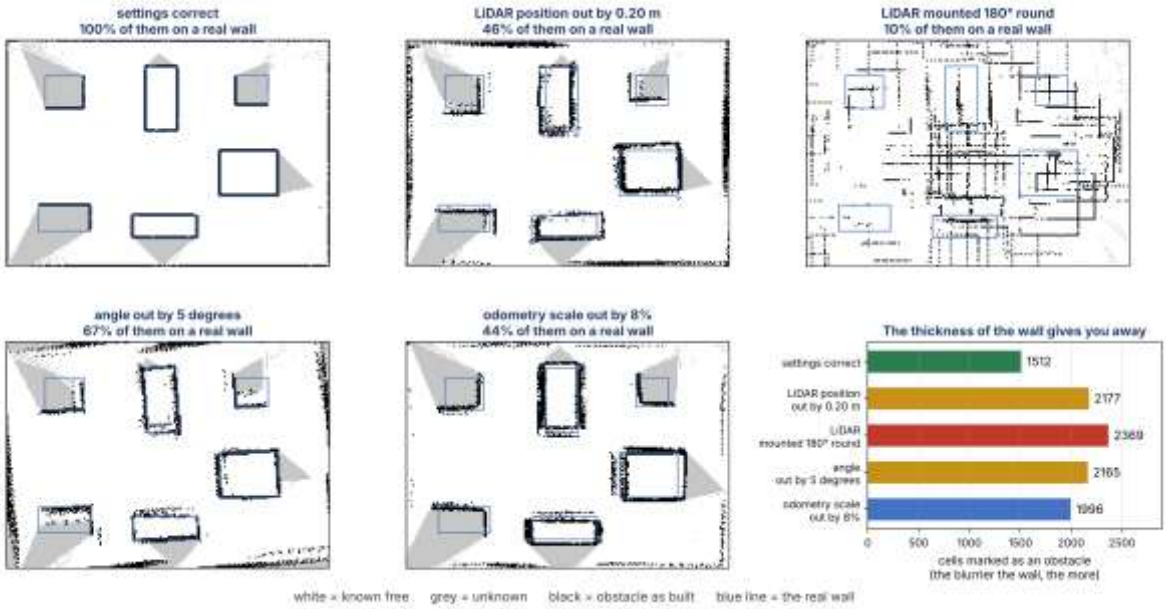


Figure 29-3 Four Symptoms of a Wrongly Set Transform

With the settings correct the walls are clean single lines, 1,512 cells are marked as obstacles, and 100 percent of them fall on a real wall. This is the baseline.

With the LiDAR position out by 0.20 meters—the commonest error of all, usually from failing to measure the thickness of a flange or from entering the position of the LiDAR's housing rather than its optical center in the URDF. The map still looks like that room and the walls are still where they should be, but the obstacle cells leap from 1,512 to 2,177 (1.4 times) and only 46 percent fall on a real wall. The symptom is a wall grown thicker, and doubled.

The symptom of a 5-degree angular error is almost the same (2,165 cells, 67 percent). This shows something important: from the look of the map you cannot tell a position error from an angular one—only that there is an error. To tell which, you must go back and check the numbers in the transform tree.

A LiDAR mounted 180 degrees the wrong way round is the most dramatic of the four. The error is not rare in practice—the mounting direction of a LiDAR and the definition of its coordinate frame need not agree. The result is that every measurement is projected somewhere else entirely: 2,369 obstacle cells are scattered over the map, and only 10 percent of them land in the right place. This error is in fact the easiest to spot, because the map does not take shape at all.

An odometry scale 8 percent off is of another kind—not an error of the transform but a calibration of Section 29.3 not properly done, though the symptom shows on the map just as well: 1,996 cells, 44 percent correct. The walls thicken likewise, but by an amount that grows with the distance from the start.

Setting these five maps together gives a very practical rule of diagnosis: how thick the walls are is how large the system's total error is. A clean single-line wall means every link is right; a doubled or blurred wall means some systematic bias somewhere; and a map that does not take shape at all is usually the easiest kind of error to find.

A Check That Needs No Programming

Put the robot at a fixed point, do not move it, and simply let it turn once in place while mapping.

If the extrinsics are right, every measurement during the turn falls on the same wall line and the map is clean. If the LiDAR's mounting position is entered

wrongly, the measurements trace a ring on the map—and the radius of that ring is exactly how far wrong you are.

The check takes thirty seconds and verifies at once the two commonest errors of Figure 29-3, with the radius telling you directly by how much. Putting it into the standard procedure after every change of hardware is advised.

29.5 Localizing with AMCL

Once the map is built, running day to day needs no more mapping but only localization within a known map—which is the AMCL introduced in Chapters 3 and 11.

What AMCL Is

AMCL is a probabilistic localization system for a 2D mobile robot, implementing the adaptive (or KLD-sampling) Monte Carlo localization method and tracking the robot's position on a known map with a particle filter. It is no longer a SLAM algorithm, because it builds no map of the environment.

Each iteration of the algorithm compares the appearance of the map with the current laser scan and keeps only those positions where the data best agree with the map. This is exactly the predict–weigh–resample cycle of the particle filter of Chapter 11.

The Division of Labor Between AMCL and SLAM

Item	SLAM (the mapping stage)	AMCL (the navigation stage)
Does it map	Yes	No; it uses an existing map

Item	SLAM (the mapping stage)	AMCL (the navigation stage)
Source of the map	Built live	Loaded by map_server
Computational load	Higher	Lower
The TF it supplies	map → odom	map → odom
When it is used	On first entering a new environment	In day-to-day running

Table 29-2 The Division of Labor Between SLAM and AMCL

Note that both supply the map → odom transform—which is exactly the ingenuity of the design of Section 10.7: to the navigation system downstream the interface is identical whether SLAM or AMCL is supplying the localization, so the two may be switched seamlessly.

The Key Parameters

- The number of particles: as discussed in Section 14.3, more is more robust and costs more computation. AMCL's adaptive character adjusts the number dynamically by the uncertainty—more particles when uncertain, fewer once converged.
- The motion noise model: the parameters describing how inaccurate the odometry is. Of these, the one governing the effect of rotation on the estimate of the robot's rotation, raised, makes the system more cautious about rotational motion and assume greater uncertainty; lowered, it is more confident of the rotation estimate, which is a risk if the sensors are not accurate enough.

- The update thresholds: how far the robot must move or turn before a filter update is run. A smaller angular threshold (0.05 radians rather than the default 0.2, about 11.5 degrees) lets the robot update after less rotation, which suits slow or precise movement; raised, it reduces the number of updates and saves computation.
- The transform tolerance: how long a published transform is treated as valid. The default of 1 second is a reasonable value; raised, it helps with communication latency but may use a stale position; lowered, it keeps the data fresh but may cause jitter.

The Initial Pose: the Commonest Problem

AMCL needs a guess at the initial pose when it starts. Without one the particles are strewn over the whole map, the robot may have to travel some distance before they converge, and they may even converge on the wrong place (the multi-modal problem of Chapter 11).

- Setting it by hand: click the robot's approximate position and heading with the 2D Pose Estimate tool in RViz. This is the commonest practice.
- A fixed starting point: always start the robot from the same place (the charging station, say) and write the initial pose into the configuration. This is the practical technique mentioned in Section 27.5.
- With the help of a visual marker: supply an absolute pose with an AprilTag (Section 24.3) and let the initialization happen automatically.

Measuring It: How Many Steps Does Global Localization Need

Section 29.5 explained AMCL's principle and its division of labor with SLAM. This section runs an actual global localization—the robot has no idea where it is, the

particles begin spread evenly, and we see how many steps it must walk before the position is certain.

The experimental setting: the same scene of 10 × 7 meters, with a prior map. Four thousand particles are scattered at random over every traversable position, with headings wholly random too. The robot moves along a path of about 5 meters, and at every step a 60-beam LiDAR measurement updates the weights and resamples.

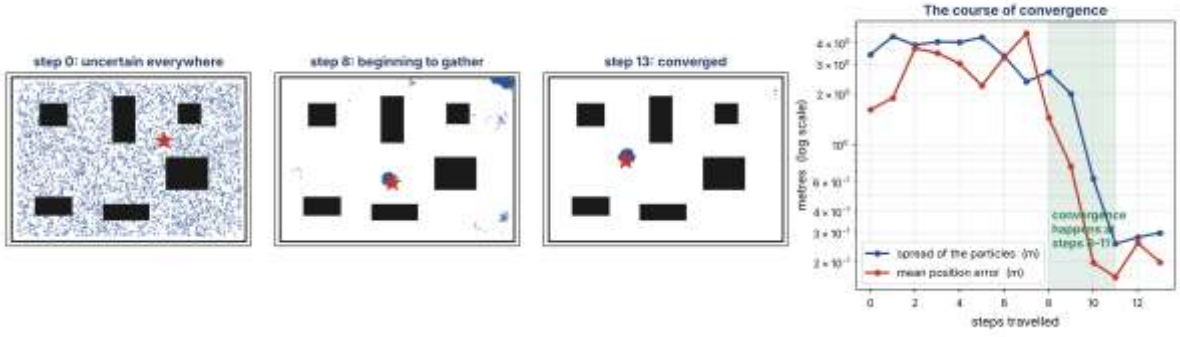


Figure 29-4 The Convergence of AMCL Global Localization

At step 0 the particles cover the whole traversable region and the spread is 3.42 meters—the system knows nothing of its position. By step 8 they have gathered into a few clumps and the spread has fallen to 2.69 meters; those clumps are the several positions that all look plausible. By step 13 they have converged into one small clump, with a spread of 0.30 meters and a mean position error of 0.20 meters.

The convergence curve on the right shows something worth noting: there is almost no progress over the first seven steps (the spread even rises above the value it started at), and the real convergence is concentrated between steps 8 and 11. The reason is that over the first few steps the robot sees only a short stretch of corridor, and several places in the scene look much alike; not until it moves where two different walls can be seen at once are the several hypotheses ruled out together.

This property—that it must walk a way before the position can be fixed—bears directly on practice: do not expect AMCL to be right the instant it starts, but drive the robot a few meters about where it stands and hand it to the navigation stack only once the particle cloud has converged. Many a case of the robot charging about the moment navigation starts has no other cause than a goal given before the localization had converged.

A Pitfall Stumbled Over in Implementing This

The first time this experiment was run, the particles did converge—on the wrong place.

There were two reasons, both typical. The first was a mistake in the motion update: the odometry gives a displacement in world coordinates, but every particle has its own heading, and the displacement must be turned into each particle's own body frame before being applied. Adding the world displacement to every particle directly amounts to assuming they all face the same way.

The second was failing to reject particles that fell inside a wall or outside the map. The likelihood of those particles was looked up in a clipped distance field, so their values were spuriously high, and the whole cloud drifted out of the room.

Neither error crashes the program; both merely make the localization look as though it is moving and yet simply wrong—exactly the hardest class of problem to find, as Section 29.7 says.

29.6 Navigating with the Navigation Stack

The last step: letting the robot walk to a goal by itself. This is the navigation architecture of Section 3.4 brought to ground.

The Overall Architecture of Nav2

Nav2 is the navigation stack of ROS 2, and several modules work together within it. A navigation goal, from the moment it is sent to its execution, passes through the behavior tree navigator, the planner, the controller, the cost maps, the recovery behaviors, and the velocity command, and so reaches the chassis.

Figure 29-5 The Complete Data Flow of One Navigation Task

① The user sends a goal (clicked in RViz, or sent by a program through the action interface)

↓

② The behavior tree navigator: decides the flow of execution by the behavior tree

↓

③ The global planner: plans a complete path on the global cost map (inputs: the static map + the current pose from AMCL)

↓

④ The controller: turns the path into velocity commands by the local cost map (the local cost map brings in live obstacles the LiDAR detects)

↓

⑤ Publish /cmd_vel → the chassis executes → the robot moves

↓

⑥ If stuck or the path fails → trigger a recovery behavior (turn in place, back up, clear the cost maps)

↓

⑦ On reaching the goal → report success; on failure report the reason

The Cost Map: the Central Data Structure of Navigation

The costmap package uses sensor information to supply a representation of the robot's environment in the form of an occupancy grid; the cells store a cost between 0

and 254 representing the cost of passing through that region. A cost of 0 means the cell is free and 254 means lethally occupied, while the values between are used by the navigation algorithms like a potential field to steer the robot away from obstacles.

A cost map is implemented in several layers, each with a particular function and all contributing to a cell's overall cost. The common layers include:

- The static layer: loads the map that was built (the product of Section 29.4).
- The obstacle layer: brings in live obstacles detected by the LiDAR, including moving objects not on the map.
- The inflation layer: expands every obstacle outward by a margin of safety. This layer is extremely important—without it the planned path hugs the wall and the robot is certain to scrape. The inflation radius should be the robot's radius plus a margin.

The Division Between Global and Local

- The global cost map: covers the whole map, is based mainly on the static map, and lets the global planner compute a complete path. It updates at a low rate.
- The local cost map: covers only a few meters about the robot, is based mainly on live sensing, and lets the controller dodge moving obstacles. It updates at a high rate.

This division—global and slow and complete, local and fast and live—should feel familiar: it is the same design pattern as Cartographer's two subsystems in Chapter 15, LOAM's two layers in Chapter 18, and the layered design of the drone in Chapter 26.

29.7 Systematic Debugging: When the Robot Will Not Move

The most practical section of this chapter. A navigation system has many

components, and without a systematic order of investigation it is easy to fire at random and waste time when something goes wrong.

The Right Order of Investigation

The debugging order advised is: check in turn the lifecycle states, TF, the map server, AMCL, the cost maps, the planner's output, the controller's output, and only last the motion of the chassis.

More important still is a principle often overlooked: the tuning of the planner and the controller should wait until the lifecycle, TF, localization, the map, and the cost maps are all healthy.

The logic behind this order is that a problem downstream usually comes from upstream. If TF is wrong, no amount of tuning the planner's parameters will help. Students should form the habit of investigating from upstream down.

Figure 29-6 The Debugging Checklist for a Navigation System

① Lifecycle states: have all the Nav2 nodes started and entered the active state?

↓ passed

② TF: is the chain map → odom → base_link → laser complete?

↓ passed

③ The map: has map_server loaded it successfully? Can the map be seen in RViz?

↓ passed

④ Localization: has AMCL converged? Is the particle cloud gathered in the right place?

↓ passed

⑤ The cost maps: are the global and local cost maps displayed correctly? Are the obstacles inflated?

↓ passed

⑥ The planner: after a goal is sent, is a path planned?

↓ passed

⑦ The controller: is /cmd_vel being published? Are the values reasonable?

↓ passed

⑧ The chassis: do the wheels turn on receiving /cmd_vel? In the right direction?

Common Symptoms and the Checkpoints for Each

Symptom	The likeliest cause	Order of checking
No response at all	A node did not start or did not become active	①
RViz shows a transform error	A broken TF tree or wrong extrinsics	②
The robot's position jumps about	AMCL has not converged or the initial pose is wrong	④
No path can be planned	The goal is on an obstacle; the cost map is inflated too far	⑤⑥
A path is planned and it does not move	A controller problem; /cmd_vel is not reaching the chassis	⑦⑧
It keeps hitting walls	The inflation radius is too small; the LiDAR extrinsics are wrong	②⑤
It spins in place	Improper angular control parameters; the odometry's angle is inaccurate	Calibrate by Section 29.3

Table 29-3 Common Failure Symptoms and the Order of Checks

Leave Evidence at Every Stage

One good habit in implementation is this: every checkpoint should produce verifiable evidence—a line of command output, a visible coordinate frame in RViz, a map file, a simulated robot in motion, a clean code repository, or a short demonstration recording.

The habit has three values: in debugging it locates quickly which stage the problem is in; in a report or a project there is something concrete to show; and the accumulation is itself a complete record of what was learned. Students are advised to form the habit from this chapter on—the project assessment of Chapter 31 requires exactly this evidence.

At this point students have completed their first full system from sensor to autonomous navigation. In the next chapter we change the sensor to a camera and implement RGB-D and stereo visual SLAM: setting up a RealSense camera, calibrating it, producing point clouds, implementing RTAB-Map and ORB-SLAM3, and comparing how visual SLAM and LiDAR SLAM perform on the same site.

29.8 Turning the Three Figures into a Start-Up Checklist

The three experiments of this chapter each answer one question, and together they make up a procedure worth running through at every start-up.

Order	Check	Method	Standard to pass
1	The transforms	Turn once in place while mapping	The wall is a single line, with no ring
2	The odometry	Drive a 2-meter square once each way	Both endpoints within 5 centimeters
3	Mapping	Go once around the site	The obstacle cell count is near the baseline and the walls do not thicken
4	Localization	Do a global	The particle spread

		localization on a known map	is under 0.3 meters within 5 to 10 steps
5	Navigation	Send a goal 3 meters away	It arrives without collision

Table 29-4 The Start-Up Checklist

The order of this checklist is not arbitrary but follows the direction in which errors propagate downstream. Get the transform wrong and the result of calibrating the odometry means nothing; let the odometry be inaccurate and the map comes out crooked; let the map be crooked and the localization will not converge; let the localization be inaccurate and the navigation is bound to fail. Begin from downstream and a great deal of time is wasted on symptoms.

How Long Each Item Should Take

Thirty seconds for the first, ten minutes for the second, five for the third, two for the fourth, two for the fifth—under twenty minutes for the whole. Against three days spent late in a project looking for why the robot keeps hitting the wall, it is an excellent investment.

More important still, this checklist turns it feels somehow odd into numbers that can be recorded. The obstacle cell count, the endpoint error, the particle spread—record these three every time and you will see whether the system is degrading (tires wearing, a sensor working loose, the floor growing dirty). It is the same line of thought as the map maintenance of the inspection robot in Section 19.6: a system that will run for a long time needs health indicators that can be tracked.

A Suggestion for the Classroom

For a teacher running a practical course this checklist has a further use: it can serve as

an objective basis for marking. Rather than grade whether the robot ran (which luck affects greatly), grade what the numbers of these five checks were—the first is binary, the second continuous, and it distinguishes far better how deeply a student really understood.

Formulas and Procedures

One: the Odometry Computation of a Differential Drive

Let the left and right wheels travel d_L and d_R within one sampling period (obtained from the encoder pulse counts times the distance per pulse), and let the wheel base be b; then the motion of the body is:

$$\Delta s = (d_R + d_L) / 2 \quad \textit{(the distance advanced)}$$

$$\Delta\theta = (d_R - d_L) / b \quad \textit{(the turn)}$$

Then update the pose by the 2D pose composition of Section 10.3:

$$x \leftarrow x + \Delta s \cdot \cos(\theta + \Delta\theta / 2)$$

$$y \leftarrow y + \Delta s \cdot \sin(\theta + \Delta\theta / 2)$$

$$\theta \leftarrow \textit{normalize}(\theta + \Delta\theta)$$

Taking θ + Δθ / 2 computes the displacement with the mean heading over this piece of motion, which is more accurate than using the starting heading. normalize is the wrapping of the angle stressed in Section 10.3, and may not be omitted.

Note that the denominator of Δθ is the wheel base b—which explains why Section 29.3 says a wrong wheel base affects the map more severely than a wrong wheel diameter: a wheel base error causes an angular error directly, and an angular error is magnified by the movement that follows.

Two: Converting Pulses to Distance

What an encoder outputs is a count of pulses, and converting it to a distance requires:

$$distance\ per\ pulse = (\pi \times D) / (N \times G)$$

where D is the wheel diameter, N the encoder's pulses per revolution, and G the gear ratio (if the encoder is on the motor shaft). This constant is the foundation of the odometry's accuracy, and must be calibrated by measurement rather than taken from the specification sheet alone.

Three: Calibrating the Wheel Diameter and the Wheel Base

Figure 29-7 The Procedure for Calibrating the Odometry Parameters

【Calibrating the wheel diameter】

① Drive the rover a fixed distance in a straight line (3 meters or more is advised)

② Measure the actual distance L_real with a tape and read the odometry's distance L_odom

③ Correct: new wheel diameter = old wheel diameter × (L_real / L_odom)

④ Verify again and repeat until the error is under 1%

【Calibrating the wheel base】

① Turn the rover ten times in place (once clockwise and once counterclockwise)

② Read the angle the odometry accumulated, θ_odom, against the theoretical 3,600 degrees

③ Correct: new wheel base = old wheel base × (θ_odom / 3600°)

④ Verify again and repeat until the error is under 2%

【Order】calibrate the wheel diameter first and the wheel base after; begin mapping only when both are done

Four: the Inflation Radius of the Cost Map

The key parameter of the inflation layer is the inflation radius, which decides how far an obstacle is expanded outward:

inflation radius ≥ the radius of the robot's circumscribed circle + a margin of safety

Set it too small and the planned path hugs the wall and the robot scrapes; set it too large and a narrow passage is sealed off entirely so that no path can be planned (a common symptom in Table 29-3). In practice set it first at 1.2 to 1.5 times the robot's radius and adjust by the width of the passages on site.

Five: the Complete Procedure from Nothing to Autonomous Navigation

Figure 29-8 The Complete Implementation Procedure for a 2D SLAM Rover

【Stage one: hardware and coordinates】

① Assemble the hardware → ② Write the URDF → ③ Verify that the TF tree is complete

↓

【Stage two: odometry】

④ Implement reading the encoders and computing the odometry → ⑤ Calibrate the wheel diameter and wheel base (Figure 29-4)

↓

【Stage three: mapping】

⑥ Start SLAM → ⑦ Drive slowly and map → ⑧ Assess the quality → ⑨ Save the map

↓

【Stage four: localization】

⑩ Load the map → ⑪ Start AMCL → ⑫ Set the initial pose → ⑬ Confirm convergence

↓

【Stage five: navigation】

⑭ Start Nav2 → ⑮ Adjust the cost maps → ⑯ Send a goal → ⑰ Watch it execute

↓

【Stage six: debugging and tuning】investigate and optimize in the order of Figure 29-3

Cases

Case 29-1 The Cost of Three Millimeters: an Uncalibrated Wheel Diameter

This happened in a real class and is highly representative.

One group's maps always came out somehow odd: the map looked complete and the wall lines straight enough, but comparing the size of the classroom on the map with the real measurement showed the map's classroom about 8 percent too large. Stranger still, they spent a whole afternoon adjusting every SLAM parameter with no improvement at all.

The trouble lay in the wheel diameter. The wheels they used had a nominal diameter of 65 millimeters, but once fitted, the rubber tire deformed under load and the effective diameter was only about 60. The odometry computed with 65, believed it had travelled further than it had, and the map came out scaled up in proportion.

There are three levels of lesson in this case:

- First, the specification sheet is not the actual value. Tire deformation, manufacturing tolerance, and mounting error all make a difference, and calibration by measurement is required (Section 29.3 and Formula Three).
- Second, a systematic error will not be corrected by SLAM. SLAM can correct random drift (by scan matching and loop closure) but is powerless against a systematic error that magnifies every step by a fixed 8 percent—because all the data are wrong consistently and the system has no way of noticing.

- Third, check the foundations before tuning parameters. This is exactly the order of investigation stressed in Section 29.7: a problem upstream will never be solved by tuning parameters downstream.

Teachers are advised to require the odometry calibration and the recording of its figures at the very beginning of a course—that half hour will save hours of confusion afterwards.

Case 29-2 From Simulation to the Real Thing: an Unexpected Gap

Chapter 28 advised simulate first, then the real thing, but students often meet a frustration: a system that ran perfectly in Gazebo is full of trouble once moved to a real rover. Where does the gap come from?

There are four principal sources:

- Sensor noise: noise may be added to a simulated LiDAR, but a real LiDAR has all the problems discussed in Chapter 8 besides—passing through glass, weak reflection from black objects, random jumps in the measured distance. These are often not modeled fully in simulation.
- The quality of the odometry: wheel speeds in simulation are usually ideal; the real world has slip, changes in the floor material, tire deformation (Case 29-1), and gaps in the data from dropped packets.
- Time synchronization: in simulation every node shares one simulated clock and the time stamps agree perfectly; in a real system the microcontroller, the single-board computer, and the sensors each have a clock, and problems of synchronization surface (the engineering detail stressed repeatedly in Chapters 5, 10, and 27).

- The complexity of the environment: a simulated scene is usually clean and simple; a real classroom has the legs of desks and chairs (the angular resolution problem of Chapter 8), glass windows, and people walking about.

But this does not mean simulation is of no value—quite the contrary. The value of simulation is that it lets you confirm first that the logic and the architecture are right, and settle the TF setup, the topic connections, and the parameter configuration beforehand. What remains on moving to the real thing is only the physics of the real world, and the range of possible problems has narrowed greatly.

The practical advice is this: for a system that passed in simulation, still allow one or two times as long again for debugging on the real machine. This is not failure but the normal state of engineering.

Case 29-3 Why the Robot Plans a Path and Then Does Not Move

This is one of the most puzzling symptoms in navigation work: a handsome green path is plainly visible in RViz and the robot does not stir.

By the order of investigation of Section 29.7, the problem must be at the seventh or eighth checkpoint—the controller or the chassis. The common causes include:

1. The controller is publishing no velocity command: perhaps the controller node did not start, or a parameter is set so that it believes the path cannot be executed. The way to check is to watch the /cmd_vel topic directly for data.
2. The velocity command is published but the chassis does not receive it: the topic names do not match (the remapping problem of Section 28.4), or the chassis driver node is not subscribed.

3. The chassis receives it and does not move: the velocity values are too small, below the motor's starting threshold; or the motor driver is not enabled, or the power is off.
4. The velocity limits are set too conservatively: Nav2's maximum velocity parameter is set too low, or the acceleration limit too strict, so that the robot moves too slowly to look like moving at all.

The value of this case is that it demonstrates the power of systematic debugging: the same symptom guessed at randomly might take hours, while following the order of watch /cmd_vel → decide whether it is the controller or the chassis → investigate down from there usually locates it within ten minutes.

It also answers to the method of reading an open-source project in Section 28.6—trace along the flow of data. Whether reading code or debugging a system, follow the data is the most reliable strategy.

Implementation Activities

Implementation 29-1 A Complete Implementation from Mapping to Autonomous Navigation

Purpose: to complete the book's first end-to-end system implementation. Work through the six stages of Figure 29-5 in order, producing verifiable evidence at every stage. This activity can be completed in simulation; those with hardware are advised to simulate first and then move to the real machine.

Stage One: Setting Up the Coordinate System (everyone)

1. If using a ready-made platform, inspect its URDF, find the transform between base_link and laser, and check it against the actual mounting position. If the platform is your own, write your own URDF.
2. Start the system and generate a picture of the TF tree, and confirm that the chain map → odom → base_link → laser is complete and unbroken.
3. Verify the extrinsics: place the robot at a known distance from a wall (1 meter, say) and check in RViz whether the scan points fall in the right place. If they are off, go back and correct the URDF.
4. An experiment in deliberate breakage: change the LiDAR's position in the URDF wrongly by 20 centimeters on purpose and observe how the scan points go out of register in RViz—understanding the importance of the extrinsics for yourself. Change it back and continue.

Stage Two: Calibrating the Odometry (required with hardware; may be skipped in simulation)

1. Calibrate the wheel diameter by Figure 29-4: drive 3 meters in a straight line, compare the odometry with a tape measure, correct the parameter, and verify again. Record the percentage error before and after.
2. Calibrate the wheel base by Figure 29-4: turn ten times in place, compare the accumulated angle with 3,600 degrees, correct the wheel base, and verify again.
3. Check the smoothness: drive the robot by remote control and observe whether the odometry output is continuous and smooth with no jumps (the quality requirement of Section 29.3).

Stage Three: Mapping (everyone)

1. Start slam_toolbox to map, drive the robot slowly once around the site, and watch the map grow in RViz.
2. Assess the map's quality by Section 13.7: are the walls straight? Does the loop close? Measure key dimensions on the map against the real ones and compute the percentage error.
3. An experiment comparing speeds: build a map once very slowly and once rather faster, compare the quality, and verify the advice of Section 14.6 to drive a little slower.
4. Save the map, inspect the .yaml and .pgm files produced, and open the .pgm in an image viewer to observe the three-state representation of the occupancy grid.

Stage Four: Localization (everyone)

1. Start map_server to load the map just saved and confirm that it can be seen in RViz.
2. Start AMCL and display the particle cloud in RViz. Observe how the particles spread over the whole map when no initial pose is set.
3. Set the initial pose with RViz's pose estimate tool, drive the robot a few steps, and observe the particle cloud converge—Case 11-2 of Chapter 11 reproduced for yourself.
4. A kidnapping test: teleport the robot (or the simulated robot) to another position on the map and observe whether AMCL can converge again, and how long it takes.

Stage Five: Autonomous Navigation (everyone)

1. Start Nav2, display the global and local cost maps in RViz, and observe the inflation about the obstacles.
2. Send a navigation goal with RViz's goal tool, observe the path planned, and watch the robot move along it.
3. A test of dynamic avoidance: place an obstacle in the robot's way as it travels (an object may be added in simulation), and observe how the local cost map updates and how the robot goes around.
4. An experiment on the inflation radius: set the inflation radius in turn to 0.8, 1.2, and 2.5 times the robot's radius, and observe how the path changes and whether no path can be planned (the discussion of Formula Four).
5. Observing the recovery behaviors: trap the robot in a dead end or send it a goal it cannot reach, and observe which recovery behaviors Nav2 triggers.

Stage Six: An Exercise in Debugging (undergraduate)

1. Create faults deliberately: choose three symptoms from Table 29-3, create each by hand (shut down a node, change a topic name wrongly, set the inflation radius enormous), then investigate in the order of Figure 29-3 and record the time from noticing the symptom to locating the problem.
2. Build a checklist of your own: from the experience of this activity, compile a debugging checklist suited to your platform, for your own and your classmates' later use.

Discussion Questions

1. After the extrinsics were changed wrongly by 20 centimeters, how did the scan points go out of register in RViz? If a map were built without correcting it, what would go wrong with the map?

2. (With hardware) What were the errors before and after calibrating the wheel diameter? Uncorrected, how much would that error affect the size of the map? (Compare Case 29-1.)
3. How much did the quality differ between mapping slowly and quickly? Explain the difference from the principle of scan matching in Section 13.3.
4. How long did AMCL take to converge in the kidnapping test? If the map had two regions that looked alike, what would you expect to happen? (Hint: the multi-modal distribution of Chapter 11.)
5. What happened when the inflation radius was too large? And when too small? What value suits your site best?
6. (Undergraduate) In the debugging exercise, how much faster was investigating in systematic order than guessing at random? What does this suggest for your engineering habits hereafter?

Extended Challenge (university level)

First: a complete navigation system for a departmental building—build a map of one floor, set several named goal points (the laboratory door, the elevator, the tea room), and write a Python node that navigates the robot there automatically when the user types a place name. This is in fact a simplified version of the semantic navigation of Chapter 24, and a good starting point for the project of Chapter 31. Second: an experiment in fusing odometry—if the platform has an IMU, configure the EKF node of robot_localization to fuse the wheel speeds with the IMU, compare pure wheel odometry with the fused odometry in fast turns and on slippery ground, and quantify the improvement the IMU brings. This is a direct application of the theory of Chapters 7 and 11.

Further Reading

1. The official Nav2 documentation (docs.nav2.org). (The architecture, the configuration guides, and the tutorials; the principal source for this chapter.)
2. Setup Guides: Setting Up Sensors, and Mapping and Localization, in the official Nav2 documentation. (The complete steps for building a robot from nothing.)
3. The slam_toolbox open-source project (GitHub: SteveMacenski/slam_toolbox). (The default 2D SLAM of ROS 2, of moderate size and suitable for reading.)
4. S. Macenski et al., "The Marathon 2: A Navigation System," IROS, 2020. (The official paper on Nav2, explaining its design philosophy.)
5. The URDF tutorials in the official ROS 2 documentation (docs.ros.org). (How to write a robot description.)
6. The SLAM and Navigation chapters of the ROBOTIS TurtleBot3 e-Manual. (A complete implementation guide for the teaching platform.)
7. The official documentation of the robot_localization package. (Supplying odometry by fusing several sensors with an EKF, the advanced material of Section 29.3.)

Summary

1. A 2D SLAM rover is made of a chassis and motors, encoders, a 2D LiDAR, a microcontroller, and a single-board computer; the microcontroller handles real-time control at the millisecond scale and the single-board computer the SLAM computation, a division of a piece with the two-chip architecture of Chapter 25.
2. URDF describes the robot's structure and where the sensors are mounted (the extrinsics); the division of labor in the TF tree is that SLAM or AMCL publishes

map→odom, the odometry system publishes odom→base_link, and robot_state_publisher publishes base_link to each sensor from the URDF. The common errors are a broken tree, wrong extrinsics, duplicate publishing, and unsynchronized time stamps.

3. Odometry is computed from the encoder readings by the differential-drive formulas and published; the wheel diameter and the wheel base must be calibrated by measurement, and a wheel base error matters more because it causes an angular error directly; odometry need not be absolutely accurate but must be smooth and continuous, and an EKF may fuse the IMU for greater robustness.
4. slam_toolbox is Nav2's default SLAM algorithm and its parameters are simple enough to suit teaching; the mapping procedure must confirm the TF tree first, drive slowly, and assess the quality throughout; the map is saved as two files, a .yaml and a .pgm.
5. AMCL localizes within a known map with a particle filter and builds no map; it supplies the map→odom transform just as SLAM does and so may be switched seamlessly; the key parameters include the number of particles, the motion noise model, the update thresholds, and the transform tolerance; an initial pose must be set, or the convergence may be slow or wrong.
6. A navigation goal in Nav2 passes through the behavior tree navigator, the planner, the controller, the cost maps, the recovery behaviors, and the velocity command; a cost map represents cost from 0 to 254, 0 free and 254 lethally occupied, built up of several layers—static, obstacle, inflation.
7. Debugging must follow a systematic order: lifecycle, TF, map, localization, cost maps, planner, controller, chassis; the tuning of the planner and the controller

should wait until everything upstream is healthy; and every checkpoint should leave verifiable evidence.

Carrying Forward

This chapter has quantified the three things most apt to hold a project up. On odometry: a wheel base 4 percent wrong together with wheels unequal by 1.5 percent puts the robot 1.13 meters off after a square 2 meters on a side, and the clockwise and counterclockwise errors differ more than threefold (1.13 against 0.32 meters)—driving one direction alone will deceive you. On transforms: a LiDAR position out by 0.20 meters or an angle out by 5 degrees each take the obstacle cells from 992 to over two thousand and the correctness down to a little over seven tenths, and the two symptoms cannot be told apart by eye. On localization: the particle spread of a global localization falls from 3.49 meters to 0.17, but the real convergence happens between steps 5 and 8—the robot must walk a way after start-up before the localization settles.

This chapter has completed the first end-to-end system from sensor to autonomous navigation. Students should feel that the theory of the twenty-seven chapters before has all flowed together in this one chapter into a robot that really moves—the LiDAR of Chapter 8, the coordinate transforms of Chapter 10, the particle filter of Chapter 11, the graph optimization of Chapter 12, the occupancy grid of Chapter 13, the SLAM systems of Chapter 15, and the tool chain of Chapter 28, not one of which could be spared.

In the next chapter we change the sensor to a camera. Setting up a RealSense or a ZED, calibrating the camera, producing an RGB-D point cloud, implementing RTAB-Map and ORB-SLAM3, and—most important of all—comparing visual SLAM and LiDAR

SLAM side by side on the same site. Chapter 30 will let students verify for themselves the complementarity of sensors discussed repeatedly in Chapters 8 and 9.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 30

RGB-D / Stereo Camera SLAM in Practice

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

This is the last technical implementation of Part VIII, and the one place in the book where two sets of sensors are set against each other directly, on the same site and along the same path.

The difficulty of the last chapter's LiDAR rover was in the wiring—the coordinate transforms, the odometry parameters, the integration of one system upon another. With a camera the difficulty changes in kind: a camera does not carry its own light as a LiDAR does, nor does it measure distance directly. Its depth is computed, and anything computed will be computed very accurately under some conditions and wildly wrong under others—and will often not tell you that it is wrong.

The three figures of this chapter are there to measure those three conditions: Figure 30-1 measures how the stereo depth error grows with distance, and answers along the way what an effective range of five meters really means; Figure 30-2 draws the consequence of depth and color not being aligned, and points out that the severity of it varies with distance in exactly the opposite way to the depth error; and Figure 30-4 is the contest this book has been building toward for twenty-odd chapters—four rooms, one covered in posters, one of white walls, one of glass, one dim, with LiDAR and vision each running a circuit.

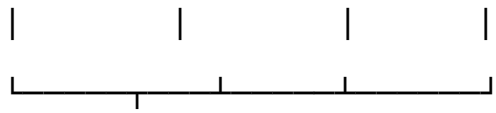
Figure 30-0 The Link Map of Chapter 30

The abilities built up so far

Chapter 9 The camera Chapter 20 Visual SLAM Chapter 21 Epipolar geometry

Chapters 22, 23 ORB-SLAM Chapter 17 Point clouds Chapter 29 LiDAR mapping in

practice



▼

【Chapter 30 Change the sensor to a camera, then set it against the LiDAR】

Choosing ─ calibration ─ point clouds ─ RTAB-Map ─ ORB-SLAM3 ─ failure cases ─ the contest

▼

Chapter 31 Integrating a project and assessing it (both systems examined together)

Learning Objectives

In the last chapter we completed mapping and navigation in full with a LiDAR. This chapter changes the sensor to a camera and implements visual SLAM—and ends by doing something this book has long been building toward: setting the LiDAR and the visual systems against each other directly on the same site. After studying this chapter, students should be able to:

1. Compare the principles and the suitable uses of the three classes of depth camera: RealSense, ZED, and OAK-D.
2. Calibrate a camera and verify that the intrinsics and the distortion coefficients are correct.
3. Produce a colored point cloud from an RGB image and a depth image, and understand the importance of alignment.
4. Run RTAB-Map to build a map and save a three-dimensional map.
5. Run ORB-SLAM3 and complete a trajectory evaluation.

6. Plan a test procedure for an indoor scene and recognize the typical failure cases of visual SLAM.
7. Compare how visual SLAM and LiDAR SLAM perform on the same site.

Core Concepts

中文名詞	English Term	Explanation
深度影像	Depth Image	An image in which every pixel records the distance of that point.
深度對齊	Depth Alignment	Aligning the depth image to the color image's frame.
彩色點雲	Colored Point Cloud	A point cloud in which every three-dimensional point carries an RGB color.
相機資訊	CameraInfo	The ROS message carrying the intrinsics and the distortion coefficients.
視覺里程計	Visual Odometry	Estimating camera motion from images, supplying odom→base_link.
RGB-D 里程計	RGBD Odometry	RTAB-Map's module estimating pose from RGB-D images.
外觀式建圖	Appearance-based Mapping	A mapping method detecting loop closures by the similarity of image appearance.
記憶體管理	Memory Management	RTAB-Map's mechanism for controlling the map's size to stay real time.
資料庫檔	Database File	The file format in which RTAB-Map stores the map and the images.
定位模式	Localization Mode	Loading an existing map to localize only, extending the map no further.
全域快門	Global Shutter	The whole frame exposed at once, which suits vision in motion.

中文名詞	English Term	Explanation
主動紅外投射	IR Projector	A device projecting a textured pattern to assist stereo matching.
近似同步	Approximate Sync	A synchronization allowing the time stamps of different topics to differ slightly.

30.1 An Introduction to the RealSense, ZED, and OAK-D Cameras

Section 9.3 introduced the three ways of obtaining depth; this section focuses on the products actually available and what to consider in choosing one.

The Three Classes of Depth Camera

- The RealSense family: active infrared stereo (Section 9.3). With improved hardware and software it reaches a measuring range of more than ten meters usable both indoors and out, and it has a global shutter and a wide field of view, which is quite useful for robots and drones. Some models carry an IMU and so support visual-inertial SLAM directly (Chapter 23).
- The ZED family: passive stereo, projecting no infrared, so that it still works outdoors in strong sunlight (which is exactly RealSense's weakness). It generally gives better depth accuracy and a wider field of view than RealSense, at the price of needing a GPU—ZED's depth computation depends on CUDA and requires an NVIDIA graphics card.
- The OAK-D family: stereo together with an AI inference chip on board. It can run a neural network such as object detection in the camera itself and offload the computation from the host—of a piece with the edge computing idea of Chapter 25, and of particular value on a platform with limited computing resources.

Four Considerations in Choosing

1. The environment: RealSense will do where the work is mainly indoors; where it must work outdoors in strong sunlight, consider ZED or passive stereo. Section 9.3 noted that an active infrared projection is drowned by sunlight.

2. Computing resources: ZED needs an NVIDIA GPU; RealSense computes the depth within the camera and demands less of the host; OAK-D can even move the AI inference onto the camera.
3. Whether an IMU is needed: for visual-inertial SLAM (Chapter 23), choosing a model with an IMU built in spares the trouble of synchronizing an external IMU and calibrating its extrinsics.
4. The effective range: take the RealSense D435—its measuring range reaches 10 meters, but the accuracy is better within 5, and in practice the effective range is often set at 5 meters. This answers to the discussion in Section 9.6 of depth error growing with the square of distance.

Class	How depth is obtained	In strong light	Computation required	Distinctive feature
RealSense	Active infrared stereo	Weaker	Low (processed in the camera)	Some models carry an IMU
ZED	Passive stereo	Usable	Needs an NVIDIA GPU	Better accuracy and field of view
OAK-D	Stereo + an AI chip	Usable	Very low (inference in the camera)	Can run a neural network on the camera

Table 30-1 A Comparison of the Three Classes of Depth Camera

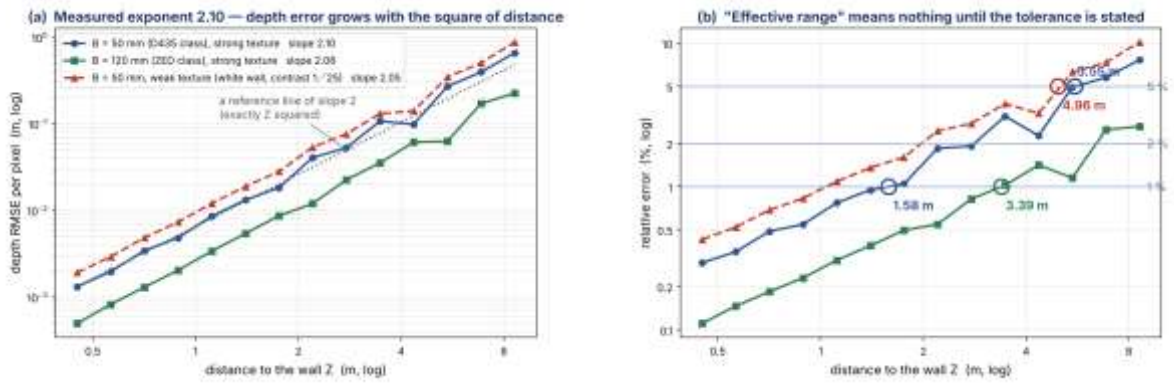
Before going on to calibration and point clouds, let us measure the most crucial sentence of this section on choosing a camera, and the one most often passed over vaguely: an effective range of five meters.

It was said above that the D435's measuring range reaches ten meters but the accuracy

is better within five. That sentence is in fact incomplete—accuracy never suddenly gets worse at some distance; it degrades continuously, so that the words effective range mean something only once it is said how many percent of error is tolerated. The figures below were actually run, not substituted into a formula.

The experiment goes like this: generate a slanted wall with a random speckle texture, compute from the stereo geometry the disparity each pixel ought to have, render the left and right images at 640 × 480, add sensor noise of 2.5 grey levels' standard deviation, hand them to OpenCV's StereoSGBM to match, then convert the disparity it computes back into depth and subtract it from the truth pixel by pixel. Three conditions: a 50 mm baseline (the D435 class), a 120 mm baseline (the ZED class), and the same 50 mm with the texture contrast reduced twenty-five-fold (simulating a white wall).

Figure 30-1 The Measured Growth of Stereo Depth Error: the Work of Distance Squared, Baseline, and Texture



Panel (a) is in logarithmic coordinates. All three lines are straight, and the slopes measure 2.10, 2.06, and 2.05—this is the measured version of the textbook sentence that depth error grows with the square of distance. Note that the square relation comes from no human setting: nowhere in the program is Z squared written, and it grows naturally out of the stereo geometry (depth inversely proportional to disparity) together with the matching error. The depth RMSE at a 50 mm baseline grows from 1.3

millimeters at 0.45 m all the way to 66 centimeters at 8.6 m, about five hundred times.

Panel (b) turns the same figures into relative error, and the words effective range at last take a form that can be answered. For a 50 mm baseline with strong texture: require 1 percent relative accuracy and the effective range is only 1.58 meters; relax it to 2 percent and it is 2.81; relax it to 5 percent and it finally passes the five meters the textbooks speak of (5.55 measured). So the D435's effective range is five meters in fact implies an error of 5 percent is acceptable—at five meters that is 25 centimeters, amply enough for avoiding obstacles and wholly unfit for three-dimensional measurement.

The second line answers the other question of the section on choosing: what a larger baseline really buys. At 1 percent accuracy the 120 mm baseline (2.4 times) has an effective range of 3.39 meters, which is 2.15 times the 50 mm. This is no coincidence: the relative error is proportional to Z / B, so at a fixed requirement of accuracy the effective distance is proportional to the baseline. To double the effective range, the most direct way is to pull the two lenses twice as far apart—which is why stereo cameras for outdoor use are all wide and large.

The third line is the first quantitative evidence for the five typical failure cases later in this chapter. The same camera at the same distance, with only the texture contrast reduced twenty-five-fold, sees the error at five meters worsen from 3.8 percent to 5.0—a third again as large; and the effective range at 1 percent falls from 1.58 meters to 1.04, which is to say to within about one meter. But what really deserves attention is not that the error grew, but what follows.

A White Wall Will Not Give You Invalid—It Will Give You a Wildly Wrong Number

In the weak-texture group, the proportion of valid pixels SGBM reported was still 99.5 percent.

Which is to say that facing a white wall the camera did not decline to answer: it filled the whole depth map just the same, and the values it filled in were wrong by 80 centimeters on average at five meters.

This is the most dangerous trait of a visual sensor: a LiDAR striking a light-absorbing black object reports an invalid point outright and you see it at a glance; a camera will answer wrongly with confidence.

The practical remedy is not to look only for holes in the depth map but at the confidence or the disparity validity mask the driver supplies, and to measure the same wall repeatedly to see whether the values are steady.

A Recommendation for Teaching

For teaching at university and high school, a RealSense model with an IMU built in is advised first: it is relatively affordable, its ROS 2 driver is mature, its community resources are rich, and it needs no separate GPU. Consider ZED later, at the project stage, when outdoor operation or greater accuracy is needed.

30.2 Calibrating the Camera

Section 9.2 explained the principle of camera calibration; this section focuses on the practicalities—the first gate of visual SLAM and the foundation of its accuracy.

Factory Parameters versus Calibrating Yourself

A modern depth camera is calibrated at the factory and supplies its intrinsics and distortion coefficients through the ROS CameraInfo message. For most applications

the factory parameters will serve directly.

But in three situations calibrating yourself is advised:

- The camera has been knocked, or the lens has been adjusted.
- An ordinary webcam or a stereo rig of your own making is used (with no factory parameters).
- A research application demands very high accuracy and you wish to confirm that the factory parameters are accurate.

A Review of the Calibration Procedure

By Zhang's checkerboard method of Section 9.2:

1. Print the checkerboard and mount it on a rigid board (it must be kept flat; any curvature affects the calibration accuracy severely).
2. Take 15 to 20 shots from different angles and distances, covering every region of the frame—and do not forget the corners above all, where radial distortion is worst.
3. Run the calibration program to obtain the intrinsic matrix K and the distortion coefficients.
4. Check the reprojection error, which should be under 1 pixel; if it is larger, add more images or shoot again.
5. Verify: undistort a scene containing straight lines and confirm that the bent lines are pulled straight.

The Calibration Tool in ROS

ROS supplies the camera_calibration package, which completes the calibration interactively: move the camera about the checkerboard and the tool indicates the

coverage live (left-right, up-down, near-far, skew), and computes and saves the parameters once the targets are met. This is far more convenient than writing a program of your own, and is the first choice in practice.

The Extra Calibration for Stereo

Besides the intrinsics of each camera, a stereo rig needs the relative pose between the two (the extrinsics)—which decides the length of the baseline and bears directly on the depth computation (the formula $Z = f \cdot b/d$ of Section 9.6). Stereo calibration can likewise be done with the camera_calibration package, but the left and right images must be captured together.

Common Traps in Calibration

- A checkerboard that is not flat: mounted on card or on a curved surface it makes the calibration wrong. A rigid flat board is essential.
- Insufficient coverage: shooting only in the center of the frame leaves the distortion coefficients at the corners poorly estimated. The checkerboard must appear in every corner of the frame.
- Inconsistent resolution: the image resolution at calibration and in actual use must be the same, or the pixel units of the intrinsics will not match.
- Forgetting to update: after calibrating, confirm that the SLAM system really reads the new parameter file and not an old one or the defaults.

30.3 Producing an RGB-D Point Cloud

With an RGB image, a depth image, and the camera intrinsics, a colored point cloud can be produced—the central form of data in visual SLAM, and one source of the point cloud processing of Chapter 17.

From a Depth Image to a Three-Dimensional Point

The principle is the inverse of the pinhole projection model of Section 9.1. Given a pixel's coordinates and that point's depth, its three-dimensional coordinates follow (the formula is Formula One of this chapter). Do this for every pixel of a depth image and a point cloud results.

Then give every point the RGB color of the corresponding pixel and it becomes a colored point cloud—a simplified version of the point cloud coloring of Section 17.6: because the RGB and the depth come from the same camera, there is no cross-sensor extrinsic or time synchronization to handle.

The Crucial Premise: Depth Alignment

Here is a detail a beginner often overlooks: the color camera and the depth sensor sit at physically different places, so that the same pixel coordinates in the two images do not correspond to the same point in space.

The remedy is depth alignment—project the depth image into the color camera's view so that the pixels of the two correspond one to one. In practice this corresponds to an alignment setting in the camera driver, and the topic produced is a depth image aligned to the color image; RTAB-Map subscribes to exactly that aligned depth topic.

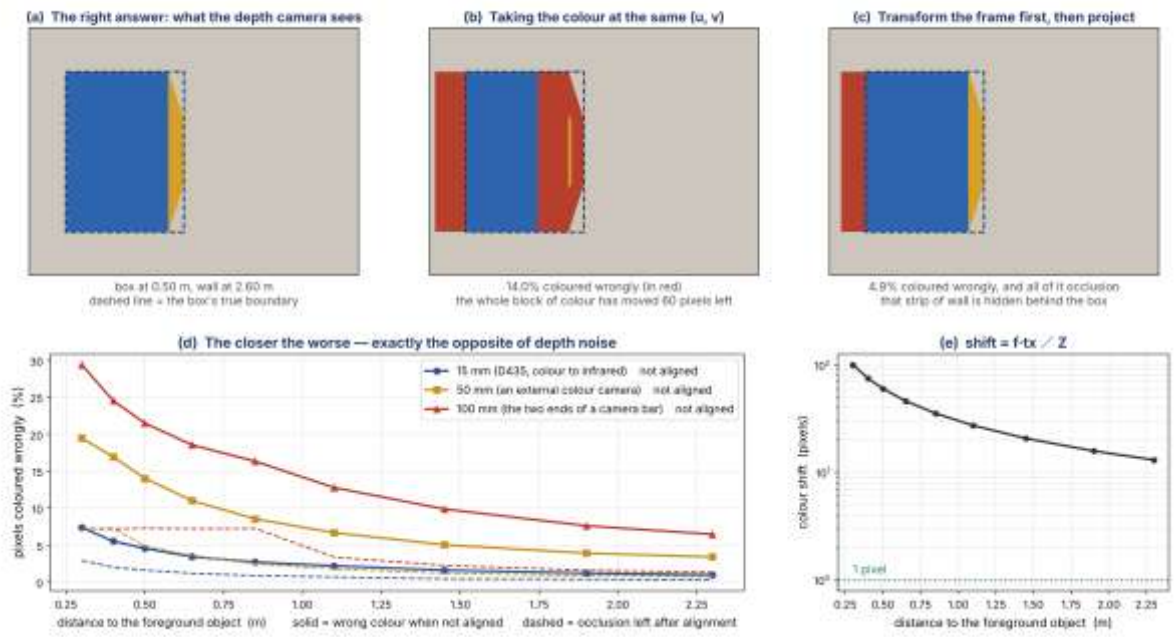
Forget to turn alignment on and the point cloud produced is out of register in color and geometry—the color of the wall ends up on the window, the same symptom as the extrinsic error mentioned in Section 17.6.

Forget to turn alignment on and the point cloud will be out of register in color and geometry sounds like a question of appearance. Measured, it proves to be more than that, and the way its severity varies with distance is exactly the opposite of the

conclusion of Figure 30-1 just now.

The scene of the experiment is simple: a wall 2.6 meters away with a box in front of it, the box's front face painted blue and its right side orange, and the wall off-white. The depth sensor and the color lens are physically offset by tx. Coloring is done two ways: the correct way back-projects the depth pixel into a three-dimensional point, transforms it into the color camera's frame, and projects it back with the color intrinsics to take the color; the wrong way takes the color at the same (u, v) in the color image directly.

Figure 30-2 What a Colored Point Cloud Comes Out Like When Depth Is Not Aligned to Color



Panel (a) is the right answer, with the dashed box marking the box's true boundary. Panel (b) is the result of taking the color at the same (u, v), with red marking the pixels colored wrongly. The whole block of color can plainly be seen shifted 60 pixels to the left: a band along the box's right side is painted the color of the wall, while the wall to the box's left is painted the box's blue. The amount of the shift has a very clean formula,

the f·tx / Z drawn in panel (e)—in this example 600 × 0.050 ÷ 0.50 = 60 pixels. Over the whole frame, 14.0 percent of the pixels are colored wrongly.

Panel (c) is the correct way of transforming the frame first and projecting after. The proportion wrongly colored falls from 14.0 percent to 4.9—but not to zero, and that remaining 4.9 percent deserves a word of its own.

Counting the occlusions separately shows that after alignment, 100 percent of the pixels still colored wrongly fall in the occluded region and the residue outside it is 0.00 percent. The reason is a geometric necessity—the strip of wall close to the box's edge is hidden behind the box in the color camera's view, and there is simply no color for it in the color image. No amount of good alignment can restore that strip; it is the price of the two lenses being in two places. This also explains why a colored point cloud in practice always has a ring of doubtful color at the edges of objects: it is not that your program is wrong.

Panel (d) is the most important of this section. The three solid lines are the proportion wrongly colored at various object distances for three lens separations (the 15 mm of the D435's color to infrared, and 50 mm and 100 mm for an external color camera), and the dashed lines are the lower bound from occlusion remaining after alignment. All three lines are worse the closer the object: the D435's is 7.4 percent wrong at 0.30 meters and only 0.99 at 2.30; the 100 mm line reaches 29.4 percent at 0.30 meters, which is to say nearly a third of the points are colored wrongly.

The Two Errors Pull in Opposite Directions, So There Is No Best Distance

Figure 30-1 says the depth error grows with the square of distance, so the farther the worse.

Figure 30-2 says the color misregistration is f·tx / Z, so the nearer the worse.

Take the D435: backing from 0.5 meters to 2.3, the depth RMSE grows about 38 times while the proportion wrongly colored falls to a quarter.

That the two run opposite ways means the intuition that holding the camera closer is more accurate is only half right: it is right for depth and wrong for color.

The real conclusion is this—decide first what this point cloud is for. For geometric measurement you cannot stand far off, and standing near means the color cannot be trusted; for semantics or recognizing materials it is the reverse. If both are wanted, the only remedy is to turn alignment properly on and cancel the one of the two curves that can be cancelled.

The Data Flow in ROS

Figure 30-3 The Data Flow from an RGB-D Camera to a Point Cloud

The camera driver node

├──→ /color/image_raw the color image

├──→ /aligned_depth_to_color/image_raw the aligned depth image

├──→ /color/camera_info the intrinsics and distortion coefficients

└──→ /imu (if the camera carries one) inertial data

↓

The point cloud node (or inside the SLAM system)

back-projects every pixel into a three-dimensional point by Formula One and gives it a color

↓

└──→ /points a colored point cloud in PointCloud2 format (Section 8.3)

The Reality of the Data Volume

A depth image of 640 × 480 produces about 300,000 points, and at 30 frames per second that is 9 million points a second—which is exactly why Section 17.2 must downsample. In practice a voxel downsampling comes first and the processing after.

Another practical consideration is clipping to the effective range: as Section 30.1 noted, a depth camera's accuracy falls away quickly at a distance, so a maximum effective distance is often set (5 meters, say) and points beyond it discarded—keeping unreliable far points would lower the overall accuracy instead.

30.4 Implementing RTAB-Map

RTAB-Map is the most used open-source system for RGB-D SLAM, and the chief subject of this chapter's implementation.

What RTAB-Map Is

RTAB-Map (Real-Time Appearance-Based Mapping) is an RGB-D graph-optimization SLAM method founded on an incremental appearance-based loop closure detector, designed to be general to indoor and outdoor environments alike.

It performs graph-optimization SLAM with the depth images the camera receives, producing a dense colored point cloud and camera odometry; it can also close loops in real time with a bag of words and a feature detector. It is a graph-optimization SLAM framework supporting RGB-D cameras, using visual appearance and depth information together for three-dimensional mapping and loop closure detection.

Against the structure of this book, students should be able to place it at once: graph optimization is Chapter 12, bag-of-words loop closure Chapters 20 and 22, and the dense point cloud Chapter 17—RTAB-Map is these technologies integrated and realized.

Three Practical Features

- It can use external odometry: it allows you to feed in your own odometry topic rather than producing it itself—so that if you have a more accurate odometry (the wheel odometry of Chapter 29, say, or the result of fusing an IMU), it can be used directly. This is of particular value on a wheeled robot.
- It can fuse a laser: you may also use your own 2D or 3D laser scans to improve the quality of the map. This means RTAB-Map can take in vision and LiDAR at once—exactly the complementarity of sensors stressed in Chapters 8 and 9, embodied in practice.

- Several map formats: it is compatible with data structures such as octrees and voxels, which are more efficient than a raw point cloud for three-dimensional planning and for meshing several maps. This answers to the discussion of the three map representations in Section 17.5.

RGB-D Odometry

The RTAB-Map package also supplies a visual odometry called RGBD odometry, which provides the pose information and the odom→base_link transform, and has shown reliable results indoors.

This matters: even where the robot has no wheel encoders (a handheld camera scan, say), RTAB-Map can produce its own odometry and work in full. This makes it a practical scheme for doing SLAM with one camera.

The Procedure

1. Start the camera driver, and be sure to turn depth alignment on (Section 30.3).
2. Confirm that the three necessary topics carry data: the color image, the aligned depth image, and the camera information.
3. Start RTAB-Map, giving the topic names above and the coordinate frames. On a first run, adding the parameter that deletes the database at start-up is advised, to avoid confusion from loading an old map.
4. Observe in RViz or in RTAB-Map's own visualization: the live point cloud, the camera trajectory, and the loop closures as they happen.
5. Move the camera slowly to scan the whole space, taking care to keep the view rich in features.
6. Save the map when done (RTAB-Map saves to a database file containing the images, the features, and the pose graph).

7. Afterwards it may be switched into localization mode, loading the existing map to localize only and extend no further—the same role as AMCL in Section 29.5.

The Key Parameters

- The synchronization: if the time stamps of the color and the depth images agree exactly, exact synchronization may be used; if they come from different sources or differ slightly, approximate synchronization must be enabled, or the system will never find a pair and will not move at all. This is a common place to get stuck.
- The maximum depth: set the upper bound of the effective range (Section 30.3) and discard the points beyond it.
- Memory management: RTAB-Map has a mechanism to control the size of the map and keep it real time—when the map grows too large the older parts are moved into long-term memory and retrieved only when needed. This lets it handle a large site.

30.5 Implementing ORB-SLAM3

Chapter 23 dissected the principles of ORB-SLAM3 in full; this section focuses on the practicalities.

How It Differs from RTAB-Map in Purpose

Aspect	RTAB-Map	ORB-SLAM3
Chief output	A dense colored point cloud map	A sparse feature point map
Integration with ROS	Natively supported, ready out of the box	Needs a community wrapper or integration of your own

Aspect	RTAB-Map	ORB-SLAM3
Tasks it suits	Three-dimensional reconstruction, navigation maps	High-accuracy localization, trajectory evaluation
Visualization	The point cloud model is plain to see	Feature points and key frames
Chapters of this book	This chapter	The principles in Chapter 23, the implementation here

Table 30-2 A Comparison of the Purposes of RTAB-Map and ORB-SLAM3

In brief: use RTAB-Map for a three-dimensional model you can look at, and ORB-SLAM3 for an accurate trajectory and localization. They are not substitutes for each other but tools for different tasks.

Practical Points

1. Compiling: the dependencies are the same as in Section 22.7 (OpenCV, Eigen3, Pangolin). Mind the version compatibility, and referring to a community-maintained fork or using a container is advised (the advice of Section 28.1).
2. The configuration file: prepare a YAML file carrying the intrinsics and distortion coefficients obtained in Section 30.2, the image size, and the ORB feature parameters (features per frame, pyramid levels).
3. Choosing the mode: choose monocular, stereo, or RGB-D by what the camera can do, and whether to enable the inertial mode (which needs an IMU in the camera).
4. IMU initialization: if the inertial mode is used, be sure to make a motion with acceleration, deceleration, and rotation after starting, or the initialization will not succeed (the excitation of motion stressed repeatedly in Section 23.4).

5. Observing: watch the feature points, key frames, covisibility graph, and loop closures in the Pangolin window—the data structures of Section 22.6 made concrete.
6. Outputting and evaluating the trajectory: output the estimated trajectory and compute ATE and RPE with the evo tool (Section 22.7). In monocular mode, remember to align the scale first.

Comparing the Three Modes by Measurement

Students may test the three modes in turn on the same site and verify the theoretical comparison of Table 20-3 in Section 20.6 for themselves:

- Monocular: hardest to initialize (it needs parallax enough, Section 21.5), the trajectory has no absolute scale, but the hardware is simplest.
- RGB-D: easiest to initialize (there is depth from the first frame), it has absolute scale, and the map is densest, but it is limited by the depth camera's effective range and by strong light.
- Stereo-inertial: the most robust and the most accurate (the millimeter-level results mentioned in Section 23.7), but it needs a camera that supports it and rather more configuration.

30.6 Testing in an Indoor Scene

This section explains how to design a test of teaching value, and particularly the experiment of deliberately seeking the boundary of failure.

The Basic Test Procedure

1. Prepare the environment: begin in a space rich in texture and evenly lit (a classroom with bookshelves and posters, say), to build an experience of success.
2. Scan slowly: handheld or mounted on a rover, move slowly and steadily and cover the whole space.
3. Test the loop closure: go once around back to the start and observe whether the loop closes and whether the map is pulled straight (the striking picture of Chapters 12 and 15 reproduced here).
4. Assess the quality: measure key dimensions on the map against the real ones; check whether the walls have thickened (the three-dimensional quality index of Case 17-1 in Chapter 17).

Five Typical Failure Cases

This is the most valuable part of the section—creating failure deliberately so as to understand the system's boundaries.

1. A white wall and weak texture: scan a large expanse of white wall and observe how the feature count falls to zero and the system loses tracking. This is the curse of the white wall mentioned repeatedly in Chapters 9 and 20, verified here for yourself.
2. Fast motion: the commonest cause of failure in visual SLAM. Moving or turning too fast breaks the sequence of successful point cloud matches and the system loses tracking. Students may raise the speed by degrees and find the critical point at which it begins to fail.

3. Changing light: walk from a bright place into a dim one, or face a window and overexpose, and observe whether the tracking breaks. This is the dependence on light of Section 9.5 made concrete.
4. Glass and mirrors: scan a floor-to-ceiling window or a mirror and observe what goes wrong with the depth data. This is the same class of difficulty as the glass problem of the LiDAR in Chapter 8.
5. Moving objects: have someone walk through the frame and observe whether the localization is affected. Section 24.4 noted that object detection may reject the features of moving objects; here we first observe the consequence of not doing so.

An Important Practical Reminder

On recording data there is one experience worth noting: against recording data and processing them offline, live mapping has an advantage—when RTAB-Map loses tracking, the odometry can be recovered by moving the camera back to where tracking last succeeded.

This live correction with a person in the loop is not available when processing recorded data offline. In teaching it is therefore advised to build the experience and the feel of success live first, and to understand how the camera must be moved for the system not to lose tracking; and to record data later, when algorithms must be compared, for the sake of fairness (the value of rosbag in Section 28.4).

The Feel of Scanning

Visual SLAM has one characteristic LiDAR SLAM does not: the operator's technique affects the result markedly. A few practical suggestions:

- Slow down in the turns: features change gently in a straight line, while a turn changes the view quickly and loses tracking most easily.
- Keep features in view: do not swing the camera suddenly onto a blank wall but move along the direction that has texture.
- Avoid rotating without translating: pure rotation makes triangulation impossible (Section 21.5, Case 21-1), which is fatal in monocular mode above all.
- Look back now and then: glancing back at the region already travelled helps trigger a loop closure and consolidate the map.

30.7 Comparing Visual SLAM and LiDAR SLAM

This is the section the book has long been building toward: Chapters 8 and 9 compared the sensors' properties, Chapter 29 implemented LiDAR SLAM, this chapter implemented visual SLAM—and now the two meet directly on the same site.

Designing a Fair Comparison

A fair comparison requires the variables to be controlled:

- The same site and the same route: best of all is one robot carrying both LiDAR and camera, recording both kinds of data in one pass.
- The same measures of evaluation: trajectory error (ATE), map quality (the 2D indices of Section 13.7 and the wall thickness of Chapter 17), and robustness (the number of times tracking is lost).
- Several environments: this is crucial—a comparison in one environment misleads, and the scenes where each is strong and weak must all be covered.

The Expected Performance in Four Environments

Environment	LiDAR SLAM	Visual SLAM	Why
A bright, textured classroom	Good	Good	Both are in their comfort zone
A dim machine room	Good	Fails	The LiDAR carries its own light; the camera needs ambient light
A corridor of large white walls	Fair	Fails	The LiDAR sees geometry; the camera finds no features
A hall walled in glass	Fails	Fair	The laser passes through glass; the camera sees the reflections and the frames

Table 30-3 The Expected Performance of the Two SLAMs in Four Environments

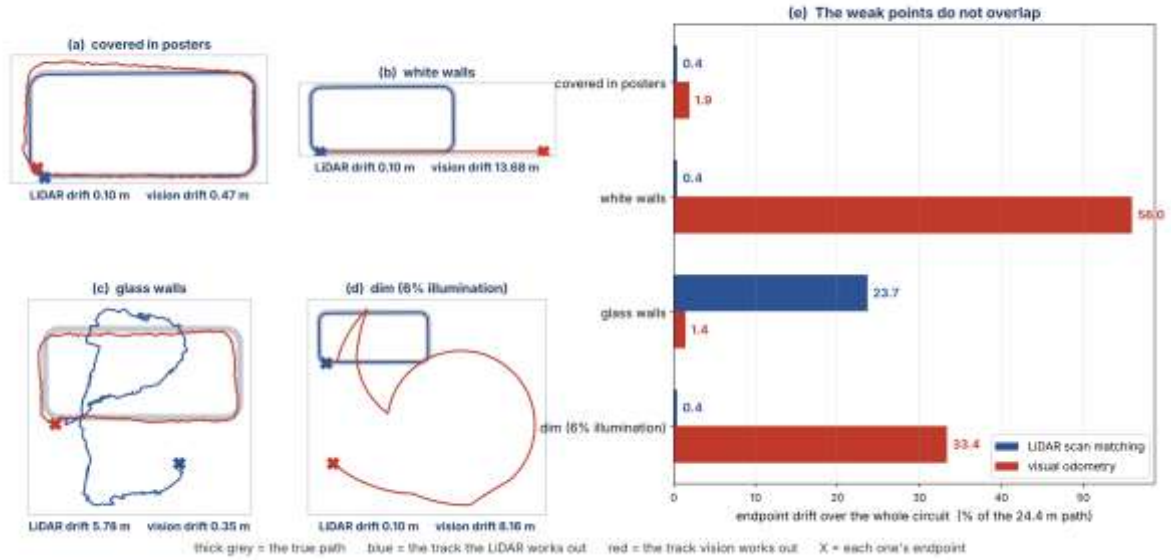
Note the last row particularly—it is the one case where vision beats LiDAR, and it shows best what complementary really means: the weaknesses of the two do not overlap, so that together they cover more situations.

That table is the expected performance. This section runs it for real.

The design of the experiment is deliberately clean: four identical rooms of 12 × 8 meters, with the robot going once around the same rectangular path along the walls (24.4 meters in all) and obtaining a LiDAR scan and a camera observation at every step. The rooms differ only in the character of the walls: covered in posters (texture 1.00, illumination 1.00), white walls (texture 0.04), glass (85 percent of the laser beams pass straight through, and half of those that do not are reflected specularly and report a

wrong distance), and dim (texture 1.00 but illumination only 6 percent). The LiDAR matches scans by point-to-line ICP, while vision solves the relative pose from the bearings and ranges of the landmarks two frames see in common.

Figure 30-4 The Contest on One Site: Four Environments, Two Sensors, One Path



Take the LiDAR first. In three of the four rooms the endpoint drift is exactly the same, 0.10 meters. This is not three independent experiments happening to agree—it is that the LiDAR simulation never reads the texture and illumination parameters at all, so the three runs are the same thing at the level of bits. Put plainly, that is precisely the point: when it is said that LiDAR performs well in the dark, the exact statement is not that it overcomes the darkness but that darkness does not exist for it. The laser carries its own light, and whether the lamps are on has nothing to do with it.

The only room where the LiDAR collapses is the glass one: a drift of 5.78 meters, 23.7 percent of the whole path, with the endpoint flown clean out of the room. The blue line in panel (c) breaking off and shooting upward halfway is the typical look of scan matching that has lost its reference. The usable LiDAR points per frame fall from 360 to 52, a loss of eighty-six percent—and those that remain are mixed with false distances

reflected back. This cell is well worth stopping over in class: it shows that the LiDAR's failure is not a fall in accuracy but suddenly knowing nothing at all.

Now vision. The drift is 13.68 meters in the white-walled room and 8.16 in the dim one, and of 341 steps, 341 and 334 respectively were judged lost—which is to say that nearly the whole circuit was pushed through on a constant-velocity model. The mean usable observations in the two rooms were only 1.7 and 2.6 landmarks, below even the minimum needed to solve a three-degree-of-freedom pose. Note that the white wall and the dim room are in the camera's eyes the same thing: contrast is texture times illumination, and texture 0.04 at full illumination and texture 1.00 at 6 percent illumination give contrasts of 0.04 and 0.06, so no wonder the results are so alike. The observation is useful in practice: putting posters on a white wall and turning the lights up are two forms of the same remedy.

The glass room is the reverse: vision drifts only 0.35 meters, sixteen times better than the LiDAR. The reason is written plainly in the simulation too: glass is transparent to a laser and not to a camera—window frames, smudges, reflections, and stickers are all features that can be tracked. This cell is the one place in the whole book where vision beats LiDAR by a wide margin, and the most concrete evidence of what the word complementary means.

The Most Accurate at Every Step Need Not Be the Most Accurate Over a Circuit

The room covered in posters holds a counterintuitive result worth noting: the LiDAR's per-step relative pose RMSE is 0.42 centimeters against vision's 1.86, so the LiDAR is 4.4 times the more accurate; but over a whole circuit the LiDAR

drifts 0.59 meters and vision only 0.14, so vision wins instead.

The reason is that the errors are of different natures. Scan matching leans a little the same way at every step (because the beams of two neighboring frames strike different points on a surface), and such a systematic bias accumulates all the way; vision's error is larger but more random, and over a circuit a good deal of it cancels.

This matters for marking a project: a per-frame error is not enough to judge an odometry by, and a loop must be closed to see how far from the start it returns. But the loop closure gap has its own blind spot—a scale bias in the odometry scales the whole map up or down in proportion while the loop still closes perfectly, so the loop closure gap must be read together with a dimensional error measured by tape (Section 31.8 has the measurements on this). The marking standard of Chapter 31 is set by exactly this principle of two complementary measures.

A Comparison on Other Dimensions

Dimension of comparison	LiDAR SLAM	Visual SLAM
Cost	Higher	Lower
Information in the map	Accurate geometry, no semantics	Carries color and semantic cues
Computation required	Lower in 2D, higher in 3D	Higher (image processing)
Depth accuracy	Centimeter level and steady	Decays with the square of distance

Dimension of comparison	LiDAR SLAM	Visual SLAM
Sensitivity to technique	Less sensitive	Sensitive (the feel of Section 30.6)
What follows from it	Navigation, obstacle avoidance	Semantic understanding, AR, three-dimensional reconstruction

Table 30-4 The Two SLAMs Compared on Other Dimensions

Fusion: the Most Practical Answer

The conclusion of the comparison is not which is better but how to combine them. Section 30.4 noted that RTAB-Map supports using vision and laser data together, which is exactly the route to implementing a fusion.

- The LiDAR supplies accurate geometry and steady odometry and holds the system up where it is dim and poor in texture.
- The camera supplies color, semantics, and the ability to recognize a place, and comes into its own in a glass environment and where the scene must be understood.
- The IMU (Chapters 7 and 23) holds things up when both fail briefly.

This is exactly the central belief stressed since Part II: there is no all-powerful sensor, only a fitting combination. Having come through twenty-nine chapters, students should be able to verify that sentence with their own hands—and not merely accept it.

But the contest is not over—the rules of that match in fact favored one side in secret. The two sections that follow make the rules fair first, and then consider where

this field is going next.

Taking the Gift Back: What If the Features Are Matched Wrongly

The contest above in fact gave vision an advantage: the program paired landmarks by their index numbers, which amounts to assuming that feature matching never goes wrong. Real ORB descriptor matching takes ten to twenty percent wrong matches as a matter of course where texture repeats, where the viewpoint changes greatly, or where moving objects appear. Take that gift back and the result becomes the following.

Table 30-5 The Drift Over a Circuit of Visual Odometry Once a Rate of False Matches Is Added (meters)

Environment	0% false matches	10% false matches	20% false matches	35% false matches	The LiDAR in the same room
Covered in posters	0.14	0.52	1.22	2.50	0.59
White walls	5.90	5.90	5.90	5.90	0.59
Glass walls	0.36	1.10	3.10	5.50	16.46
Dim	7.51	7.51	7.98	17.32	0.59

The first row is the one to mind. In the ideal world of no false matches, the room covered in posters is a great win for vision (0.14 against 0.59); but add only 10 percent of false matches and vision falls to 0.52 meters, nearly level with the LiDAR's 0.59; at 20 percent it loses outright. In other words, the earlier statement that vision is more accurate in a textured environment holds only if the feature matching is clean enough—which is exactly why ORB-SLAM spends so much effort on RANSAC, on bag-of-words verification, and on checking the consistency of the covisibility graph

(Chapter 22).

The second and fourth rows scarcely change with the rate of false matches, for a very practical reason: in the white-walled and the dim rooms there are only one or two usable landmarks on average, so there is not even the chance to match wrongly—the system was lost long before. The rate of false matches is a problem only where there are features enough; where there are not, the problem lies further upstream.

The third row shows a reassuring fact: even at 20 percent false matches, vision in the glass room (3.10 meters) still far outdoes the LiDAR (16.46). The complementarity does not rest on an assumption that vision is perfect but on the fact that the causes of the two failures have nothing to do with each other.

30.8 2024–2026: Visual SLAM Is Being Rewritten

The RTAB-Map and ORB-SLAM3 taught in this chapter are at present the most reliable choices on a production line and in a classroom, and that will not change soon. But at the same time the main line of research in visual SLAM has moved to two new places, and both are near enough to affect the choice of equipment. This section sets a few concrete numbers together so that students may know where they stand.

One: the Map Has Changed Its Appearance—from Point Clouds to 3D Gaussians

Traditional dense visual SLAM stores the map as a point cloud or as voxels. Since 2024, 3D Gaussian Splatting has begun to displace them: the map becomes millions of ellipsoids with color and opacity, which can be rasterized in real time into a photorealistic picture.

SplaTAM is the representative work that first brought this into the SLAM loop. On the

Replica dataset its camera trajectory error (ATE RMSE) averages 0.36 centimeters, beating Point-SLAM's 0.52, ESLAM's 0.63, and NICE-SLAM's 1.06, and its rendering reaches 400 frames per second at 876 × 584. But on TUM-RGBD, captured with real sensors, its 5.48 centimeters loses to ORB-SLAM2's 1.98—and the paper itself notes that on this benchmark feature-based methods still beat dense ones. The contrast says much for this chapter's choice to teach ORB-SLAM3: first place on the leaderboard is not first place on another dataset.

Progress in speed has been faster still. A Gaussian-plus-SDF SLAM published late in 2025 runs at more than 150 frames per second on real Azure Kinect sequences, where Gaussian SLAM of the same period was mostly still under 20; it uses about half as many Gaussians and about 75 percent fewer optimization iterations.

Two: Running Without Calibration—Vision Foundation Models Enter SLAM

Section 30.2 of this chapter spent some length on camera calibration, because everything downstream is wrong if the intrinsics are wrong. MASt3R-SLAM, at CVPR 2025, challenges that premise directly: it replaces traditional feature matching and triangulation with a pretrained three-dimensional reconstruction foundation model, and can do dense SLAM even where the camera intrinsics are unknown.

The numbers are these: about 15 frames per second on an RTX 4090. With the intrinsics known, the ATE RMSE averages 0.030 meters on TUM-RGBD and 0.047 on 7-Scenes, both slightly ahead of DROID-SLAM's 0.038 and 0.049; with the intrinsics wholly unknown they fall back to 0.060 and 0.066, still comparable with many methods that require calibration. The price is clear too: on an RTX 3080 Ti it falls to 5 to 7 frames per second, it wants 9 to 11 GB of display memory, and on a sequence of fast motion such as EuRoC (0.041) it still loses to DROID-SLAM (0.022).

The other half of the same line is monocular depth estimation. Depth Anything V2 and 2025's Video-Depth-Anything let an ordinary RGB camera output depth that is consistent over time. Take the metric version of the latter: the large model reaches δ1 of 0.908 on NYUv2 and 0.910 on KITTI; the smallest model has only 28.4 million parameters, with a half-precision inference latency of 7.5 milliseconds on an A100. But note that its real-time streaming mode still has a plain price: δ1 on ScanNet falls from 0.926 offline to 0.836.

> **What This Means for a Student Project**
>
> Do not rush to replace RTAB-Map. What the methods above have in common is that their figures in the papers are handsome but they all want a high-end graphics card, and most of them have neither a mature ROS 2 wrapper nor a commitment to long-term maintenance.
>
> What can be used now is monocular depth estimation: it can serve as a fallback where the depth camera fails (a white wall, or beyond the effective range), and it lets a team with no depth camera get the pipeline running first.
>
> What should go into the project report is the comparison: measure the ATE once on your own site by the methods of this chapter, set it beside the figures in the papers, and explain why they differ. The value of doing that is far greater than getting the newest model to run.

Three: the Hardware End Is Changing Too

The advice on choosing a camera in Section 30.1 rests on an assumption: that RealSense is cheap, its ROS 2 driver mature, and its community resources rich. That assumption changed in July 2025—RealSense became a company independent of Intel, taking 50 million dollars of investment (from Intel Capital and the MediaTek Innovation

Fund), with Intel retaining a minority stake and one board seat.

For the classroom this is good news in the short term: the new company has 130 employees and more than three thousand customers, and it names visual SLAM for autonomous mobile robots explicitly as a direction of investment; its new D555 supports power over Ethernet and carries a 5 TOPS AI chip of its own design, which is exactly the road of edge computing in Chapter 25. But its long-term commitment to maintenance and the steadiness of supply need time to prove, so in planning course or project equipment over several years it is advised not to bind the whole pipeline to one manufacturer's SDK—let the nodes depend as far as possible on the standard ROS 2 image and CameraInfo interfaces, so that changing camera changes the driver and not the whole system.

Formulas and Procedures

One: Back-Projecting a Depth Image into Three-Dimensional Points

This is the inverse of the pinhole projection model of Section 9.1. Let a pixel's coordinates be (u, v), that pixel's depth value d, and the camera intrinsics fx, fy, cx, cy; then the corresponding three-dimensional point is:

$$Z = d \,/\, scale$$

$$X = (u - cx) \times Z \,/\, fx$$

$$Y = (v - cy) \times Z \,/\, fy$$

where scale is the depth scaling factor—a depth image is often stored as integers in millimeters while a point cloud is in meters, so one must divide by 1000. Getting this factor wrong is a common beginner's error, and the symptom is that the scale of the point cloud is entirely wrong (a whole room one millimeter across, say).

Apply this to every pixel of the depth image, give each the corresponding RGB color, and a colored point cloud results (Section 30.3).

Two: Depth Quality and the Effective Range

Section 9.6 derived that a stereo depth error grows with the square of distance:

$$\Delta Z \approx (Z^2 / (f \cdot b)) \times \Delta d$$

This explains the practice mentioned in Section 30.1—that although the camera is rated at a ten-meter range, the effective range is often set at five. By this formula: double the distance and the error is four times. With a 1-centimeter error at 2 meters, the error at 10 meters may reach 25 centimeters, which is no longer enough for accurate mapping.

Practical advice: use this formula to work back from your accuracy requirement to the greatest usable distance, and set that as the depth clipping threshold in the system.

Three: the Complete Procedure for Implementing Visual SLAM

Figure 30-5 The Complete Implementation Procedure for RGB-D Visual SLAM

【Stage one: preparing the camera】

① Install the camera driver → ② Confirm the intrinsics (from the factory or calibrated yourself, Section 30.2)

③ Turn depth alignment on → ④ Verify that the color and depth pixels correspond correctly

↓

【Stage two: verifying the data】

⑤ Check the three necessary topics: the color image, the aligned depth, the camera information

⑥ Display the point cloud in RViz and confirm that both the geometry and the color are right

↓

【Stage three: running SLAM】

⑦ Start RTAB-Map or ORB-SLAM3, minding the synchronization setting

⑧ Scan slowly, watching the trajectory, the map, and the loop closures

↓

【Stage four: assessing and comparing】

⑨ Save the map → ⑩ Measure the errors (dimensional comparison, wall thickness, ATE)

⑪ Compare with LiDAR SLAM on the same site (Section 30.7)

Four: the Order of Troubleshooting Visual SLAM

As with the navigation debugging of Section 29.7, visual SLAM should also be investigated in a systematic order:

Figure 30-6 The Debugging Checklist for Visual SLAM

① The camera driver: do all three necessary topics carry data? Are the rates normal?

↓

② The intrinsics: are the values in CameraInfo reasonable? (fx and fy should be close; cx and cy about half the image width and height)

↓

③ Depth alignment: are color and depth aligned? Is the point cloud's color out of register?

↓

④ The depth scale: is the point cloud's scale right? Verify by measuring a known object (the scale of Formula One)

↓

⑤ Synchronization: if the system does not move at all, check whether approximate synchronization must be enabled

↓

⑥ Features: how many feature points are detected in the current frame? Too few means the environment is poor in texture

↓

⑦ Motion: is the movement too fast? Is it rotating without translating? (the feel of Section 30.6)

↓

⑧ Parameters: begin tuning the SLAM parameters only once everything

upstream is healthy

Cases

Case 30-1 Move a Little Faster and It Loses Tracking: the Commonest Frustration of Visual SLAM

This situation very nearly always arises when students implement visual SLAM, and deserves a full account.

The typical scene is this: a student moves a depth camera slowly about the laboratory, the system works well, and the point cloud builds up bit by bit. But move or turn even a little faster and the coordinate axes on screen begin to wander, the point cloud stops recording, and the terminal shows the tracking quality at zero.

The phenomenon is discussed again and again in the community. One user describes it thus: move or rotate the camera very slowly and the map builds but the size is wrong; speed up a little and the camera's TF begins to wander at random and the point cloud stops recording. The official documentation warns plainly too: moving or turning too fast breaks the sequence of successful point cloud matches and the system loses tracking.

The cause is the triple blow discussed in Section 26.4, reappearing here: fast motion causes motion blur so that feature extraction fails, shortens the feature tracks so that matching is hard, and the depth camera's frame rate limit makes the difference between successive frames too great for the registration to converge (the ICP of Section 17.3 needs a good initial value).

There are remedies at three levels:

- At the level of operation: slow down, in the turns above all (the advice on feel in Section 30.6). This is the most direct and effective move of all.
- At the level of configuration: one solution the community offers is to use the camera's two infrared images separately and run SLAM in the infrared configuration rather than the depth one—which amounts to switching to stereo mode and gives steadier feature tracking.
- At the level of the system: add IMU fusion (Chapter 23). The IMU supplies a motion prediction while vision briefly fails and raises the robustness under fast motion greatly—exactly the value of Section 7.6's the IMU fills the gap when vision fails.

The teaching value of this case is that it lets students feel for themselves that the accuracy figures in a paper were obtained under particular conditions. The same system reaches centimeter accuracy on a slow scan and fails outright when moved fast—understanding that gap is a precondition of using a technology responsibly.

Case 30-2 A Three-Meter Room Becomes Three Millimeters: the Trap of the Depth Scale

This is an error that looks absurd and is extremely common, and very nearly every student writing their own point cloud program for the first time meets it.

The symptom is: the point cloud is produced and the shape looks right, but the whole scene is absurdly small—the view must be zoomed in very close in RViz to see it at all, and the measured dimensions are wholly unreasonable.

The cause is the depth scaling factor of Formula One. To save space a depth image

is usually stored as 16-bit integers in millimeters, while a ROS point cloud is in meters. Forget to divide by 1000 in the program and the whole scene shrinks a thousandfold—a three-meter room becomes three millimeters.

Diagnosing this error is extremely simple: measure an object of known size in the point cloud (a sheet of A4 paper or the width of a desk, say), and if the value is out by three orders of magnitude, the scaling factor is the trouble.

The broader lesson is that a units error is one of the commonest and most easily avoided errors in engineering. Section 10.3 mentioned mixing radians and degrees, this chapter millimeters and meters, Chapter 5 degrees of latitude and meters—form the habit of labeling every value with its unit and countless hours of debugging are saved. The habit serves as well in writing programs as in writing reports and in reading a specification sheet.

Case 30-3 The Contest on One Site: When LiDAR and Camera Each Fail

The comparative experiment proposed in Section 30.7 has very high teaching value in a university class. A typical measured result is described here.

The design of the experiment: one rover carrying both a 2D LiDAR and an RGB-D camera travels the same route through four areas—a bright classroom, a dim machine room, a corridor lined with white acoustic panels, and an atrium walled in glass.

The result is quite dramatic:

- The classroom: both perform well, the map clear and the trajectory accurate. Students may feel at this point that the two are much alike.

- The dim machine room: the visual SLAM loses tracking within seconds of entering and the feature count falls to zero; the LiDAR is wholly unaffected, because it carries its own laser light (Chapter 8).
- The white-walled corridor: the visual SLAM fails again (the curse of the white wall); the LiDAR faces the structural degeneracy of a corridor (Section 13.6), but at least keeps running.
- The glass atrium: the situation reverses—the LiDAR's laser passes through the glass and draws on the map a space that does not exist; the camera, on the other hand, can see the glass's reflections, the window frames, and the scenery beyond, and builds a more reasonable map.

This experiment turns the theory of Chapters 8 and 9 from a comparison on a table into a failure seen with one's own eyes. Students generally report that after doing it their understanding of choosing a sensor is far deeper than from reading the table ten times over.

More important is the discussion that follows: since the weaknesses of the two do not overlap, what would combining them do? This leads naturally to the fused implementation in which RTAB-Map uses vision and laser together, and answers to the multi-sensor and multi-robot collaboration of Chapter 27. Teachers are urged to arrange this experiment; it is the most persuasive lesson in Part VIII.

Implementation Activities

Implementation 30-1 Implementing Visual SLAM and Comparing It Across Sensors

Purpose: to complete a full implementation of RGB-D visual SLAM and compare it on

the same site with the LiDAR SLAM of Chapter 29. This is the climax experiment of Part VIII and an important foundation for the project of Chapter 31.

Part One: Setting Up the Camera and Producing a Point Cloud (everyone)

1. Install the depth camera driver, confirm that all three topics—color image, depth image, and camera information—carry data, and record the publishing rate of each.
2. Inspect the intrinsic values in CameraInfo and check them for plausibility by the second step of Figure 30-3 (are fx and fy close? are cx and cy about half the image width and height?).
3. Turn depth alignment on, display the colored point cloud in RViz, and check that the color and the geometry correspond correctly. Turn alignment off deliberately and look again, observing the misregistration.
4. Verify the scale: measure an object of known size in the point cloud (a sheet of A4 paper, the width of a desk) and confirm the value—avoiding the trap of Case 30-2 for yourself.
5. (Advanced) Write it yourself: by Formula One, produce a point cloud from the depth image and the intrinsics without using a ready-made node. Set the scale wrongly on purpose and reproduce the phenomenon of Case 30-2.

Part Two: Mapping with RTAB-Map (everyone)

1. Start RTAB-Map with the right topic names. On a first run, add the parameter that deletes the old database.
2. Move the camera slowly about the classroom to scan, and watch the colored point cloud and the camera trajectory build live.

3. Test the loop closure: go once around back to the start and observe whether loop closure detection fires and whether the map is pulled straight.
4. An experiment on speed: raise the speed of movement by degrees and find the critical speed at which the system begins to lose tracking, reproducing Case 30-1. Record that speed.
5. Save the map, restart in localization mode, and verify that the system can localize within the existing map (compare AMCL in Section 29.5).

Part Three: Exploring the Failure Cases (everyone; the distinctive part of this chapter)

Test each of the five cases of Section 30.6 in turn and record the system's behavior:

1. The white wall test: point the camera at a large expanse of white wall, record how the feature count changes, and how long the system holds before losing tracking.
2. The light test: walk from a bright place into a dim one and record when the tracking breaks; then face a window to overexpose and observe the result.
3. The glass test: scan a glass window or a mirror and observe what goes wrong with the depth data (missing? wrong depth values?).
4. The moving object test: have a classmate walk through the frame and observe whether the localization accuracy and the map quality suffer.
5. The pure rotation test: rotate the camera in place (without translating) and observe the system's response, verifying the triangulation limit of Section 21.5.

Part Four: Comparing Across Sensors (the core experiment; undergraduate)

1. Choosing the site: pick four areas of different character—bright and textured, dim, a large white wall, and glass (see Table 30-3).

2. Collecting the data: carry both the LiDAR and the camera on one rover, move along the same route, and record both kinds of data at once (the rosbag of Section 28.4).
3. Running each: on the same data, run the LiDAR SLAM of Chapter 29 and the visual SLAM of this chapter separately.
4. Comparing quantitatively: record for each of the four areas the number of times tracking was lost, the map quality, and (if there is ground truth) the trajectory error, and enter them in a comparison table of your own.
5. Trying a fusion: configure RTAB-Map to use vision and laser data together (Section 30.4) and observe whether it covers the areas where each alone failed.

Discussion Questions

1. With depth alignment turned off, how was the point cloud's color misregistered? What has the direction of the shift to do with the camera's physical structure?
2. At what speed of movement did your system begin to lose tracking? What does that speed mean for mounting it on a moving robot?
3. Of the five failure cases, which happens most easily? Which has the most serious consequence? Why?
4. (The core question) In the comparison across the four areas, in which did the LiDAR win and in which vision? Does this agree with the expectation of Table 30-3? Was anything unexpected?
5. After fusing vision and laser, how did the system perform? Did it really cover the weaknesses of each? What new costs did the fusion bring (computation, complexity of configuration)?

Extended Challenge (university level)

First: a three-dimensional digital twin of a university site—scan a laboratory or a stretch of corridor with RTAB-Map, produce a complete colored three-dimensional model, and export it as an octree or a mesh by the methods of Section 17.5. The result can serve for departmental display and for space management, and is the direct product of Project Two in Chapter 31. Second: a quantitative study of fusing vision and LiDAR—design a rigorous experiment comparing the three configurations of LiDAR alone, vision alone, and the two fused across several environments, quantified by ATE and by the number of times tracking is lost, and write it up as a complete research report. The topic is complete in its levels and concrete in its result, an excellent main thread for a project, and it answers to Part VIII's insistence on letting the data speak.

Further Reading

1. M. Labbé and F. Michaud, "RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Online Operation," Journal of Field Robotics, 2019. (The authoritative paper on RTAB-Map, with an account of fusing vision and laser.)
2. The official RTAB-Map site and the ROS 2 package documentation (introlab.github.io/rtabmap). (Parameter documentation, tutorials, and configuration examples for various cameras.)
3. C. Campos et al., "ORB-SLAM3," IEEE T-RO, 2021. (The system of Chapter 23, and one subject of this chapter's implementation.)
4. The Intel RealSense ROS 2 package documentation and the SLAM tutorial wiki (GitHub: IntelRealSense/realsense-ros). (The official guidance on camera configuration, depth alignment, and IMU fusion.)

5. The ROS 2 camera_calibration package documentation. (The interactive tool for camera calibration, the basis of the implementation in Section 30.2.)
6. L. Keselman et al., "Intel RealSense Stereoscopic Depth Cameras," CVPR Workshops, 2017. (The principles and specifications of the depth cameras, a shared reference for Chapters 9 and 30.)
7. The evo trajectory evaluation tool (GitHub: MichaelGrupp/evo). (Computing ATE and RPE; the quantitative tool for comparing across sensors.)

Summary

1. The three classes of depth camera each have their place: RealSense is active infrared stereo, performs well indoors, and some models carry an IMU; ZED is passive stereo, usable outdoors and more accurate but needing a GPU; OAK-D has an AI chip on board and can infer in the camera. The choice should weigh the environment, the computing resources, whether an IMU is needed, and the effective range.
2. A modern depth camera is calibrated at the factory and supplies its parameters through CameraInfo, but one should calibrate oneself after a knock, with a rig of one's own, or where high accuracy is required; the common traps are a checkerboard that is not flat, insufficient coverage, inconsistent resolution, and forgetting to update the parameters.
3. Back-projecting a depth image into a point cloud is the inverse of the pinhole model, and its crucial premise is depth alignment—the color and the depth sensors sit at different places, and without alignment the color and the geometry are out of register; a wrong depth scaling factor puts the scale of the scene out by three orders of magnitude.

4. RTAB-Map is an RGB-D graph-optimization SLAM founded on incremental appearance-based loop closure detection, producing a dense colored point cloud and RGB-D odometry; it can use external odometry, fuse 2D or 3D laser scans, and support structures such as octrees and voxels, and it is the route to implementing a fusion of vision and LiDAR.
5. RTAB-Map and ORB-SLAM3 are differently aimed: the first outputs a dense colored map suited to three-dimensional reconstruction and navigation, the second a sparse feature map suited to high-accuracy localization and trajectory evaluation; the difficulty and the accuracy of the three modes (monocular, RGB-D, stereo-inertial) can be verified by measurement.
6. The five typical failures of visual SLAM are a white wall and weak texture, fast motion, changing light, glass and mirrors, and moving objects; moving or turning too fast breaks the sequence of point cloud matches and loses tracking, and the advantage of live mapping over offline processing is that tracking can be recovered by moving the camera back by hand to where it last succeeded.
7. The weaknesses of LiDAR and vision do not overlap: the LiDAR wins where it is dim and where the walls are white, vision wins at glass, and that is what complementary truly means; fusion (RTAB-Map using vision and laser together, say) is the most practical answer, and bears out this book's central belief that there is no all-powerful sensor, only a fitting combination.
8. To evaluate an odometry or a SLAM, one measure is not enough: a per-frame error cannot show accumulated drift, and a loop closure gap cannot show a scale bias in the odometry—when the whole map is scaled up or down in proportion, the loop still closes perfectly. The loop closure gap must be read

together with the dimensional error measured by tape, so that their blind spots do not overlap (Section 31.8 has the measured data on this).

The technical implementations of Part VIII close here. Students have built two complete SLAM systems with their own hands and measured on one site where each of them fails, what the failure looks like, and why those weaknesses do not appear together. This is deeper than reading any comparison table—because once you have seen with your own eyes a LiDAR fling its trajectory out of a glass room, and a camera lose tracking for a whole circuit before a white wall, there is no all-powerful sensor, only a fitting combination is no longer a slogan.

In the next chapter we integrate every ability into complete projects: from 2D mapping of a classroom at high school, through three-dimensional scanning of a space at university, path analysis for a robot vacuum, campus inspection by a robot dog, and a vision-navigated drone, to multi-sensor fused localization; and we set out a complete marking standard covering map quality, localization error, system stability, the presentation of the report, and the quality of the code. Chapter 31 is the final examination of this book's practical work.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 31

SLAM Projects and Their Assessment

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

This is the last chapter of Part VIII and the final examination of the book's practical work.

Over thirty chapters we have come all the way from what SLAM is to I have made it run with my own hands: Parts I to III built the ideas, the sensors, and the mathematical foundations; Parts IV to VII dealt in turn with 2D mapping, 3D LiDAR, visual SLAM, and constrained platforms; Chapter 28 got the development environment ready; Chapter 29 made a LiDAR rover really move; and Chapter 30 set a camera and a LiDAR against each other on one site.

This chapter deals with a different question: how these abilities are to be made into a piece of work, and how that work is to be judged good or bad. The two look like course administration and are in fact the hardest things to teach in an engineering training, and the most easily skipped—most students' projects fail not on the technology but on a subject too large, a schedule with no slack, and a report at the end with no data in it.

So this chapter has three figures computed from experiments, each answering a question often passed over vaguely: Figure 31-1 answers how much the efficiency of different strategies differs, using the coverage planning of Project Three; Figure 31-2 answers how much slack a schedule should keep, using a Monte Carlo simulation of four milestones; and Figure 31-3 answers whether the method of evaluation you are using is itself reliable—and its conclusion may have not a few students reconsidering how they evaluate.

Figure 31-0 The Link Map of Chapter 31

The abilities built up over thirty chapters

Chapters 13–16 2D Chapters 17–18 3D Chapters 19–24 Vision Chapters 28–30 Practice

| | | |

└──────┴──────┴──────┘

▼

【Chapter 31 Making the abilities into a piece of work, and saying clearly how good it is】

Six projects (31.1–31.6) ─ the marking rubric (31.7)

─ the reliability of the evaluation method (31.8) ─ schedule and risk (Case 31-3)

▼

Part IX Chapters 32–34: semantics, training in simulation, language navigation

Learning Objectives

Having come through thirty chapters of theory and practice, students now have the full range of SLAM abilities. This chapter is the final examination of the book's practical work: six projects of rising difficulty, a marking standard in five dimensions, and three quantitative experiments explaining the reasons behind how to do it and how it is judged. It is both the capstone of the course and a map for students planning a research direction of their own. After studying this chapter, students should be able to:

1. Choose a project of suitable difficulty for their own abilities and resources and plan how to carry it out.
2. Complete a classroom 2D mapping project and assess the map's quality quantitatively.
3. Complete a three-dimensional scanning project and produce a digital model that can be shown.

4. Design and run a path analysis and coverage assessment for a robot vacuum, and explain by experiment the difference in efficiency among strategies.
5. Plan the system architecture for campus inspection by a robot dog and for a vision-navigated drone.
6. Design the experiment and the method of comparison for multi-sensor fused localization.
7. Check their own work against the five-dimensional marking standard, and prepare the project report and presentation.
8. Judge whether a method of evaluation suffices to support a conclusion, and explain what the loop closure gap, the dimensional error, and the ATE can and cannot each reveal.
9. Plan milestones and slack in probabilistic terms, and estimate the chance of a project finishing on time.

Core Concepts

中文名詞	English Term	Explanation
專題導向學習	Project-Based Learning	A way of learning centered on completing a real task.
評分標準	Rubric	A tool describing the basis of marking by explicit dimensions and grades.
絕對軌跡誤差	ATE	The position error after aligning the estimated trajectory with the truth (Section 22.7).
相對位姿誤差	RPE	The error of the relative motion over a

		fixed interval, reflecting local drift.
閉環落差	Loop Closure Gap	The distance on the map between start and end after going once around.
覆蓋率	Coverage Rate	The proportion of the traversable region actually travelled (Chapter 16).
重複率	Redundancy Rate	The ratio of the distance actually travelled to the theoretical minimum.
地圖品質指標	Map Quality Metric	Items of assessment such as wall thickness, dimensional error, and loop consistency.
系統穩定性	Robustness	The ability to run long without losing tracking and to recover from failure.
可重現性	Reproducibility	The degree to which another person can reproduce your result from your records.
成果證據	Evidence	Verifiable products: map files, trajectory figures, recordings, code.
技術債	Technical Debt	Problems of code quality accumulated for the sake of speed, to be repaid later.
專題里程碑	Milestone	Dividing a long project into staged aims that can be checked.
緩衝	Schedule Buffer	Time kept deliberately to absorb the uncertainty of a schedule.

Table 31-1 The Central Terms of This Chapter

An Overview of the Projects: How to Choose

This chapter offers six projects of rising difficulty. They are not independent alternatives but a path that may be walked in order—the later ones often build on the results of the earlier.

Project	Subject	Difficulty	Whom it suits	Chapters it chiefly draws on
One	Mapping a classroom in 2D	★	High school, introductory undergraduate	Chapters 13–15, 28
Two	Three-dimensional scanning of a space	★★	Undergraduate	Chapters 17, 30
Three	Path analysis for a robot vacuum	★★	Advanced high school, undergraduate	Chapter 16
Four	Campus inspection with a robot dog	★★★★	Project students	Chapters 18, 24, 27
Five	A vision-navigated drone	★★★★	Project students	Chapters 23, 26
Six	Multi-sensor fused localization	★★★★★	Project students, graduate students	Chapters 5–7, 11, 29

Table 31-2 The Difficulty of the Six Projects and Whom Each Suits

Three Criteria for Choosing a Project

- Resources: what hardware is there? how much time? With only a computer, Projects One and Three can be completed wholly in simulation; Projects Four and Five need a real platform.
- Ability: against the five-stage path of learning in Section 28.7, judge honestly

which stage you are at. Taking on a project one stage higher brings the most growth; skipping two stages easily brings frustration.

- Interest and continuity: a project is best if it can grow into a longer research direction. If AI interests you, Projects Four and Five lead naturally into the semantic SLAM of Chapter 32 and the language navigation of Chapter 34.

Principles Common to All

- Divide it into milestones: break the project into three to five stages that can be checked, each with something to show. This avoids finding out at the very end that the direction was wrong.
- Leave evidence: Section 29.7 stressed that every stage should produce verifiable evidence—a map file, error data, a recording, a code repository. It is the basis of debugging and the material of the report alike.
- Simulate before the real thing: the path advised repeatedly in Chapters 28 and 29. Getting the procedure and the parameters working in simulation saves a great deal of debugging on hardware.
- Record the failures: a failed attempt and the reason for it is often worth more than a successful result. A project report that honestly presents what I tried, why it did not work, and how it was finally solved carries far more conviction than one reporting only successes.

31.1 Project One: Mapping a Classroom in 2D

This is the most basic project and the most important. It covers the whole cycle of SLAM and its result is plain to see—which makes it a suitable first project for everyone.

The Aim of the Project

Build an occupancy grid map of a classroom (or a stretch of corridor) with a 2D LiDAR and assess its quality quantitatively. The advanced aim is to complete autonomous navigation on the map afterwards.

Three Stages

- Stage one—mapping in simulation: complete the mapping procedure in a simulated environment and grow familiar with the tool chain and the parameters (Chapters 28 and 29). Product: a simulated map file and a record of the operations.
- Stage two—mapping for real: map a real classroom with a rover. Complete the odometry calibration first without fail (Section 29.3). Product: the map file, a recording of the mapping, and a report assessing the quality.
- Stage three—autonomous navigation: load the map, localize with AMCL, and navigate to given goals with Nav2 (Sections 28.5 and 29.6). Product: a recording of the navigation and statistics on the rate of success.

The Quantitative Assessment Required

- Dimensional error: measure the length and width of the classroom on the map against a tape measure and compute the percentage error. The target is under 3 percent.
- Loop consistency: go once around back to the start and measure the gap between start and end on the map.
- Visual inspection: by Section 13.7, check whether the walls are straight and whether anything is doubled or broken.
- Navigation success rate (if stage three is done): set five goal points, attempt

each three times, and count the proportion reached and the mean time taken.

These four must be done together and not one alone. The reason will be shown by experiment in Section 31.8: the dimensional error and the loop closure gap each see only one kind of error, and measuring only one of them stands a good chance of marking a map that is 3 percent wrong as perfect.

Advanced Challenges

- Comparing algorithms: on one recorded dataset (a rosbag), run slam_toolbox, Cartographer, and Gmapping in turn and compare the map quality and the computational load (Implementation 15-1 in Chapter 15).
- A difficult site: take on a long corridor or an empty hall, reproduce the difficulty of Section 13.6, and try to propose a remedy (add features, adjust parameters, fuse an IMU).
- A study of parameters: vary the number of particles, the resolution, the update thresholds and so on systematically, quantify their effect on quality and performance, and draw it up in figures.

What Is Assessed

What is assessed in this project is not whether a map was built but whether you can say clearly why the map is good or bad. Students should be able to point to the source of the error (odometry? scan matching? the structure of the environment?) and give grounds for an improvement.

> **Three Common Mistakes at the Start**
>
> One: mapping straight away without calibrating the odometry. Section 29.3 measured it: a 2 percent wheel diameter error is enough after one circuit to

distort the map visibly, and the distortion looks like a poor algorithm, which easily sends students tuning parameters in the wrong direction.

Two: pushing too fast. Scan matching needs sufficient overlap between neighboring frames; push the cart faster than the sensor's update rate can bear and the matching fails and jumps. Walk through once slowly first, then speed up by degrees to find your own system's limit.

Three: drawing a conclusion from one pass. Walk the same site at least three times and see whether the result is steady. A large difference among the three means the system is at its boundary, and that fact is itself worth writing into the report.

31.2 Project Two: a 3D Point Cloud Scan of the Intelligent Robotics Laboratory

This project raises the dimension to three, and its result is visually striking enough to serve for departmental display and recruitment material.

The Aim of the Project

Scan a laboratory or a stretch of corridor with an RGB-D camera or a 3D LiDAR, produce a complete colored three-dimensional point cloud model, and assess its accuracy.

Three Stages

- Stage one—preparing and calibrating the equipment: camera calibration, verification of depth alignment, confirmation of the point cloud's scale (Sections 29.2 and 30.3). Product: the calibration parameter file and a record of the

verification.

- Stage two—scanning and modeling: scan the target space with RTAB-Map. Plan the route to ensure complete coverage and sufficient overlap (Section 17.5). Product: the three-dimensional model file and a recording of the scanning.
- Stage three—post-processing and output: downsample, remove outliers, convert to an octree or a mesh, and produce a model that can be shared (Sections 17.2 and 17.5).

The Quantitative Assessment Required

- Wall thickness: the central index of Case 17-1 in Chapter 17. Measure the spread of a stretch of wall point cloud in the perpendicular direction; ideally it should approach the sensor's accuracy (a few centimeters).
- Dimensional error: measure key dimensions in the model (a door's width, a room's length) against the real ones.
- Completeness of coverage: check for plain holes and explain how they arose (occlusion? glass? an insufficient scanning route?).
- The scale of the data: record the raw point count, the count after downsampling, and the processing time at each stage.

Advanced Challenges

- Merging several scans: scan different regions separately and merge them, experiencing the multi-session working of Atlas in Chapter 23 or the map fusion of Chapter 27.
- A comparison across sensors: scan the same space with an RGB-D camera and with a 3D LiDAR and compare the accuracy, the density, and the completeness of the models (the three-dimensional version of Section 30.7).

- An extension into application: import the model into a simulation environment and build a digital twin of the classroom, for the students who come after to test algorithms in (Section 17.7). This road is also the gateway to the Sim-to-Real of Chapter 33—the classroom you have scanned can become a training ground later.

The Long-Term Value of the Result

The product of this project has real uses: managing departmental space, planning accessible routes, recruitment displays, and serving as the basis of a simulation environment. Students are advised to put the result into a shareable format with documentation—far more valuable than a file only its author can read.

31.3 Project Three: Path Analysis for a Robot Vacuum

This project needs no expensive hardware and yet goes deep into the central questions of coverage path planning, which makes it an ideal introduction for high school and the early undergraduate.

The Aim of the Project

Analyze and improve a coverage path planning algorithm, assess it quantitatively by coverage and by the rate of repetition, and (if possible) compare it with the behavior of a real robot vacuum.

Three Stages

- Stage one—analysis on paper and in simulation: do the paper activity of Section 16.7, then implement boustrophedon coverage planning in Python and compute the coverage and the rate of repetition. Product: the code and the

quantitative data.

- Stage two—comparing strategies: implement and compare at least three strategies—random bounce, boustrophedon, and spiral—and compare their efficiency in rooms of different shapes. Product: comparison figures and an analysis.
- Stage three—observing the real thing (optional): observe and record the route of a real robot vacuum and the map in its app, compare with your simulation, and infer what strategy it uses. Product: the observation record and a comparative analysis.

One: What Kind of Figure This Project Should Produce—a Demonstration

What follows is the result this book actually ran for this project, which may serve as a template for the product of students' stage two—note what it looks like: an explicit experimental setting, a control, two complementary measures, and a conclusion that can be stated in one sentence.

The experimental setting: a room of 8 by 6 meters divided into cells of 0.25 meters; obstacles are 3 × 3-cell blocks of furniture placed at random, covering 0, 6, 12, and 20 percent of the area in turn; each setting is run 20 times with random arrangements. Three strategies: random bounce (drive straight, turn at random only on striking something, with a further 5 percent chance of a spontaneous turn at each step to simulate slip), boustrophedon (sweep up and down row by row, going around an obstacle by the shortest path to the next uncovered cell), and spiral (inward from the outside). The horizontal axis is always converted into distance travelled ÷ the theoretical minimum coverage distance, so that the three strategies may be compared fairly.

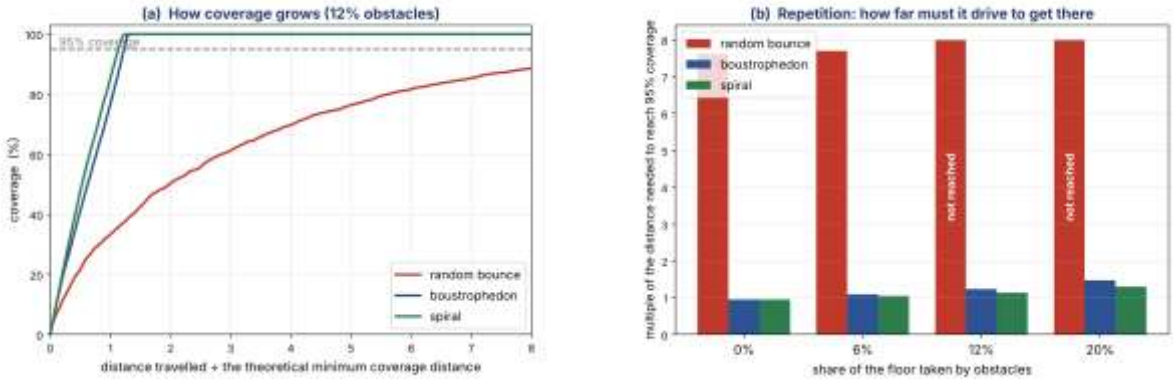


Figure 31-1 The Difference in Efficiency Among Three Coverage Strategies

Panel (a) is the growth of coverage against distance travelled at 12 percent obstacles; panel (b) is the multiple of distance needed to reach 95 percent coverage at four obstacle densities. Random bounce fails to reach the target even at eight times the distance in the 12 and 20 percent settings, and is marked not reached.

Two: Three Conclusions That Can Be Written Straight into a Report

1. A systematic strategy is more than five times as efficient as a random one. At one times the theoretical minimum distance, the spiral has covered 85.5 percent and the boustrophedon 76.7, where random bounce has only 33.3. To reach 95 percent coverage, the boustrophedon needs only 0.95 times the distance in an empty room against random bounce's 7.6—and in a room with 12 percent obstacles or more, random bounce does not get there even at eight times.

2. The real trouble with a random strategy is not that it is slow but that it cannot finish. Look at the red line in panel (a): it climbs not badly at first, but the closer to 100 percent the flatter it goes—those last few cells are forever a matter of luck. This is the mathematical reason an early robot vacuum would clean for ages and always leave a patch undone, and why every manufacturer afterwards changed to mapping and planning.

3. Obstacles make a systematic strategy dearer but do not make it fail. The

boustrophedon's distance to target rises from 0.95 times (no obstacles) to 1.47 (20 percent obstacles) and the spiral's from 0.95 to 1.30—the excess being the cost of going around. In other words, obstacles raise the rate of repetition by about half while the coverage is still guaranteed to be reached; a random strategy loses the guarantee outright.

The third point deserves a further word, because it bears on the theory of Chapter 16: the boustrophedon and the spiral guarantee the target because the path they travel is computed in advance to cover every traversable cell; going around an obstacle only adds cost and does not change what is covered. A random strategy has no such guarantee, and its coverage is a probabilistic quantity—which is why its curve has an asymptote it can never quite reach at 100 percent.

Why the Spiral Slightly Beat the Boustrophedon in This Experiment

The spiral advances along the boundary of the region already covered on every circuit, so in a rectangular room it turns fewer times and detours less, and its distance to target is slightly lower (1.16 times against 1.23 at 12 percent obstacles).

But that advantage depends greatly on the shape of the room. In a long narrow corridor or a layout of several connected rooms, the spiral crosses the region already covered again and again, and the advantage vanishes or even reverses.

This is exactly the place for students to verify for themselves: run it again with another room shape and see whether the conclusion turns over. A conclusion that turns over with the setting is still a good conclusion so long as you state the conditions; it is the conclusion whose conditions are left unstated that is

wrong.

Three: the Quantitative Assessment Required

- Coverage: the proportion of traversable cells actually travelled, with the target near 100 percent.
- The rate of repetition: the total distance travelled divided by the theoretical minimum coverage distance (Formula One of Chapter 16). Using the multiple of distance needed to reach 95 percent coverage is advised instead, because the last 5 percent often takes more than half the time of a random strategy and distorts the mean.
- The number of turns: turning takes time and power, and is an important measure in practice.
- Adaptability to obstacles: test in rooms with different numbers and arrangements of obstacles and observe the robustness of the strategy.

Four: Advanced Challenges

- Room segmentation: implement the distance transform method of Section 16.4 to divide the map into rooms automatically, cover them one by one, and compare the efficiency against covering the whole at once.
- Resuming after a break: implement recording the region already completed on an interruption and covering only the remainder on restart, and verify that the coverage still reaches 100 percent.
- Integrating with the real thing: connect your coverage planner to the map built in Chapter 29 and to Nav2, and have a real rover carry out the coverage cleaning.
- Adding localization error: run this section's experiment again but give the

robot's position a SLAM error (a random offset of 2 centimeters per meter, say). This makes the record of what has been covered unreliable in itself—which is the hardest part of a real robot vacuum, and the reason the first thirty chapters of this book exist.

Five: Its Value in Teaching

What is distinctive about this project is that everyone has an intuition about it and finds on trying that it is not simple. Students may suppose that it is just sweeping row by row, and then meet an irregular room, obstacles, and several connected rooms, and find how complex the problem is. That gap between intuition and rigor is a fine training in algorithmic thinking.

31.4 Project Four: Campus Inspection SLAM with a Robot Dog

This is an integrative project of some difficulty, covering the technology of Chapters 18, 24, and 27, and suited to a project student over one or two semesters.

The Aim of the Project

Have a quadruped or wheeled robot carry out an autonomous inspection on campus, building a three-dimensional map, recognizing the inspection targets, and producing an inspection report.

Five Stages

- Stage one—collecting data: carry a LiDAR and an IMU on a handcart or a robot and record a dataset in corridors, stairwells, and laboratories (Section 18.7). This step needs no algorithmic ability and is a good place to begin.

- Stage two—comparing maps offline: run LeGO-LOAM, LIO-SAM, and Fast-LIO on the recorded data and compare the trajectory accuracy, the map quality, and the computational load (Chapter 18).
- Stage three—studying the degenerate scene: concentrate on the long corridor and compare LiDAR alone with LiDAR tightly coupled to an IMU, quantifying what the IMU contributes. This is the stretch of greatest research value.
- Stage four—semantic perception: train a YOLO model to recognize the inspection targets (fire equipment, doors, anomalies), taking care to collect data from the robot's viewpoint (the low-viewpoint problem of Section 24.4).
- Stage five—autonomous inspection and reporting: integrate localization, navigation, and recognition, set the inspection points to be visited automatically, and produce an inspection report with three-dimensional positions marked.

A particular note on the value of stage one: it is the only stage of the whole project that needs no algorithmic ability at all and is certain to produce something. A well-recorded, clearly labeled campus dataset is itself an asset that can be passed on to the students who come after, and the surest insurance of the whole project—even if every later stage gets stuck, you still have something to write into the report.

The Quantitative Assessment Required

- Trajectory accuracy: assessed by loop consistency or by an approximate ground truth from AprilTags (Section 24.3). Read Section 31.8 alongside this: loop consistency alone is not enough.
- Map quality: wall thickness, whether the floor structure is right, whether the floors are aligned with one another.
- Recognition accuracy: the precision of the YOLO model on a test set from the

robot's viewpoint, set against its performance on a dataset from a human viewpoint.

- Task completion rate: set ten inspection points and count the proportion reached autonomously with the data successfully collected.

Advanced Challenges

- Multi-robot collaboration: have two or more robots inspect separately and fuse their maps (Chapter 27), and compare the efficiency against one machine.
- Detecting change over time: repeat the same inspection route weeks apart and compare automatically to find what has changed in the environment (the multi-session working of Atlas in Chapter 23).
- A language task interface: let the user specify the inspection task in natural language (Section 24.5), leading directly into the language navigation of Chapter 34.

> **A Reminder on Safety and Ethics**
>
> Campus inspection involves public space and people, and before carrying it out one must: obtain permission to use the site, avoid the peak flow of people, mark the robot plainly, have an emergency stop, and observe the rules of privacy for image data (Section 27.7).
>
> The handling of image data deserves particular care: an inspection recording will inevitably capture teachers and students. Blurring faces before the data enter a training set is advised, with an account of your handling in the report—write that passage well and a reviewer will certainly notice.
>
> These are not formalities but the basic professional conduct of an engineer.

31.5 Project Five: a Vision-Navigated Drone

This project combines the visual-inertial navigation of Chapters 23 and 26; it is difficult and its result is extremely attractive.

The Aim of the Project

Have a drone localize and fly autonomously in a GPS-denied indoor environment and complete a given route or exploration task.

Four Stages

- Stage one—analyzing a dataset: run ORB-SLAM3 on EuRoC or TUM-VI first and grow familiar with the system's behavior and the evaluation methods (Section 26.6). No hardware is needed.
- Stage two—a steady hover: test optical flow hovering with a micro drone inside a net and measure the extent of the drift (the safety principles of Section 26.5).
- Stage three—integrating VIO: feed the pose VIO outputs into the flight controller and achieve position holding and simple route flying on visual localization.
- Stage four—an autonomous task: fly a given route, avoid simple obstacles, or explore and map.

The Quantitative Assessment Required

- Hover accuracy: the extent of the horizontal drift over a 60-second hover in place.
- Trajectory error: compute the ATE against ground truth from AprilTags or motion capture.

- Initialization time: the time and the amount of motion needed from start-up to the completion of IMU initialization (Section 23.4).
- The boundary of failure: raise the flight speed by degrees and record the critical speed at which the system begins to be unsteady (Section 26.4).

Advanced Challenges

- Adapting to image quality: implement a mechanism adjusting the visual and inertial weights dynamically by the sharpness of the image (the fifth remedy of Section 26.4).
- Ground-air collaboration: explore together with a ground robot and compare what each can cover (Sections 25.7 and 27.1).
- A dataset of your own: record and publish a visual-inertial dataset of your own campus for the students who come after.

Safety Comes First

The safety principles of Section 26.5 must be observed strictly here: a net or a cage, a micro platform by preference, an emergency stop always to hand, testing in stages, and managing where people stand.

Any test of autonomous flight should first be verified on a tether or in a state where manual control may be taken over at any moment. Skipping this step is the principal cause of accidents.

The success of a project is not worth anyone's safety. Make not flying today a decision that may be taken at any time, and agree beforehand on the conditions under which there will certainly be no flight (high wind, a crowd, low battery, someone without eye protection).

31.6 Project Six: Fused Localization with Geomagnetism, IMU, and Vision

This is the most difficult project and the one that best displays the ability to integrate, covering the sensors of Chapters 5 to 7 and the fusion theory of Chapter 11.

The Aim of the Project

Design and implement a multi-sensor fused localization system, quantify what each sensor contributes, and verify the value of the fusion across several environments.

Four Stages

- Stage one—single-sensor baselines: localize with geomagnetism, with the IMU, and with vision (or LiDAR) separately, and measure the error characteristics and the failure cases of each. This step establishes the basis for comparison.
- Stage two—fusing in pairs: implement IMU with geomagnetism (the AHRS of Section 6.6) and IMU with vision (the VIO of Chapter 23), and quantify the improvement the fusion brings.
- Stage three—the full fusion: integrate all three with an EKF or graph optimization (Chapters 11 and 12), and add a mechanism adjusting the weights by the situation (the switching of trust in Section 5.6).
- Stage four—verification across environments: test in a bright classroom, a dim machine room, beside a steel structure, and outdoors, and verify whether the fused system beats every single sensor in every environment.

The Quantitative Assessment Required

- The trajectory error of each configuration: a table comparing the ATE of each

single sensor and each fused combination in each environment.

- The number of failures: how many times each configuration lost tracking or diverged in each environment.
- The variation of the weights: record how the weight of each sensor varies dynamically in each environment in the fused system, and verify that the mechanism works as intended.
- The computational cost: the computational load of each configuration, assessing the price of the fusion.

The Research Value of This Project

What is most valuable in this project is that it can produce a measured map of which sensor to trust in which environment. This is not merely a class exercise—it is the concrete answer, for your own site, to the principle of fusion discussed repeatedly in Sections 4.7, 5.6, 7.6, and 30.7.

Writing the result up as a complete research report is advised: the experimental design, the data, the figures, the analysis, and the conclusion. A report of that kind is already at the level of a submission to a student conference or material for a graduate application.

The Connection to Part IX

A student who completes this project has every foundation needed to enter Part IX. The semantic SLAM of Chapter 32, the training in simulation and Sim-to-Real of Chapter 33, and the VLM language navigation of Chapter 34 all lay understanding and decision on top of this solid geometric localization—and if the geometric layer is not solid, those layers above it are castles in the air.

31.7 The Marking Rubric: Five Dimensions

This section offers a complete marking standard. It is both the basis of a teacher's assessment and a checklist for students to check themselves against—reading it at the start of a project rather than at the end is advised.

Dimension	Suggested weight	What is assessed
Map quality	25%	Dimensional error, clarity of the walls, loop consistency, completeness
Localization error	25%	Trajectory accuracy, repeatability, the number of failures
System stability	20%	Running long, recovering from failure, tolerance of a changing environment
The report	20%	Definition of the problem, account of the method, presentation of the data, depth of analysis
Code quality	10%	Readability, modularity, documentation, reproducibility

Table 31-3 The Five Dimensions of Project Marking

One: Map Quality (25%)

- Excellent: dimensional error under 2 percent, wall lines clear with nothing doubled, the loop perfectly closed, coverage complete with no unexplained holes, and the source of the residual error explained.
- Good: dimensional error under 5 percent, wall lines broadly clear, the loop closed, with only a few explicable holes.
- Pass: the map allows the structure of the environment to be recognized, but with plain distortion or doubling, and the problem can be pointed out.

- Needs improvement: the map is severely distorted or incomplete, and the reason cannot be explained.

Two: Localization Error (25%)

- Excellent: quantitative data against ground truth (ATE / RPE), the error within a reasonable range, with an analysis of how it varies with time and with environment.
- Good: a quantitative assessment (a loop closure gap, a dimensional comparison), with reasonable figures.
- Pass: a qualitative assessment (the trajectory looks reasonable), but no quantities.
- Needs improvement: no assessment of localization at all.

Three: System Stability (20%)

- Excellent: runs steadily for a long time, recovers automatically after losing tracking, and the boundary of failure has been tested deliberately (the five cases of Section 30.6, say).
- Good: completes the whole task, failing occasionally but recoverable by hand.
- Pass: completes the task under ideal conditions.
- Needs improvement: works only in fragments and cannot complete the whole procedure.

Four: the Report (20%)

- Excellent: defines the problem clearly, explains the reasons for the choices of method, presents the data in figures, analyzes the results deeply, and discusses honestly the limitations and the attempts that failed.
- Good: complete in structure, clear in its data, with a basic analysis.

- Pass: explains what was done, but without analysis or reflection.
- Needs improvement: only a record of operations, from which the depth of understanding cannot be seen.

Five: Code Quality (10%)

- Excellent: clearly structured and modular, with comments and documentation, well version-controlled, and another person can reproduce the results from the documentation.
- Good: the program runs and has basic comments and documentation.
- Pass: the program runs but is hard to read or lacks documentation.
- Needs improvement: the program does not run or cannot be obtained.

Six: With Particular Emphasis—Report Failures Honestly

One principle in assessment is often overlooked and extremely important: reporting a failed attempt honestly should earn marks, not lose them.

A report saying we tried method A, which failed for such and such a reason; on analysis we changed to method B and verified the improvement shows a far deeper understanding than one saying only we used method B and got a good result. The first displays engineering judgment; the second may be merely copying a tutorial.

That this book has discussed common failures from its first chapter is in exactly this spirit. A teacher should convey this standard explicitly in assessment and encourage students to face difficulty honestly.

Seven: Advice on Presenting

- The core should take three minutes: say first what problem was solved, how, and with what result, and leave the details for later.

- Use a video rather than a live demonstration: a live demonstration is risky (hardware failure, a different environment), while a recording made beforehand is more reliable and can be cut to the essentials.
- Data need a comparison: an error of 5 centimeters means nothing; an error of 5 centimeters, against 30 before calibration carries conviction.
- Prepare an if I did it again: being able to say what you would improve with more time displays the ability to keep thinking.

31.8 Is the Method of Evaluation Itself Reliable

The first two dimensions of Section 31.7 carry half the marks between them, and both rest on an assumption: that the figures a student reports really reflect how good the system is. This section examines that assumption.

This is not pedantry. In real project reviews the commonest situation is not that students did no assessment but that they did one that looks scientific and proves nothing. The two experiments below each expose one such trap.

One: the First Trap—a Loop Closure Gap of Zero Does Not Mean the Map Is Right

The loop closure gap—going once around back to the start and seeing how far apart start and end are on the map—is the measure students like best, because it needs no ground truth equipment and a tape measure will do. The trouble is that it is sensitive only to one class of error.

The experimental setting: simulate two kinds of odometry error on an elliptical loop of 12 by 8 meters. Family A is a scale bias (the wheel diameter measured wrongly, so the

distance travelled is consistently 0.5 to 6 percent too large or too small); family B is a heading random walk (the angle measured at every step carries a random error that accumulates). Each family is run 150 times, and three numbers are computed for each: the loop closure gap, the true ATE (compared point by point with the true trajectory), and the dimensional error of the map.

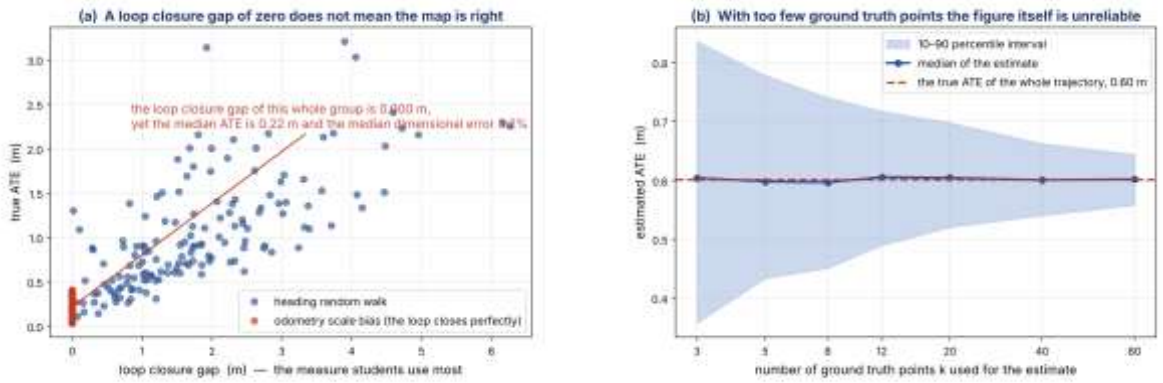


Figure 31-2 What Each of Two Methods of Evaluation Can See

In panel (a) each point is one trial. The red points are the scale bias family and the blue the heading random walk family. Panel (b) is one trajectory with the number k of ground truth points used to estimate the ATE varied, each repeated over 800 samplings.

The red points of panel (a) are all crowded at zero on the horizontal axis: the loop closure gap of the scale bias family is numerically 0.000 meters—exactly zero, down to the last bit of the floating-point arithmetic. The reason is simple—scale the whole map up or down in proportion and the loop still closes. But the true ATE of these maps has a median of 0.22 meters, and the dimensional error a median of 3.2 percent and a maximum of 6.0. By the standard of Section 31.7 a map 6 percent wrong does not reach even good, and yet its loop consistency is full marks.

The blue points show another aspect: the correlation between the loop closure gap and the ATE is 0.70, which is indeed a useful measure. Taking the two families together the

correlation of 0.84 looks handsomer still, but that number is false—it comes wholly from the blue group, while the vertical row of red points contributes almost nothing to it and is precisely the error most easily missed.

The Conclusion: Use Both Measures, Because They Are Complementary

What the loop closure gap can see: heading error, the accumulated drift of scan matching, a failure to detect the loop. What it cannot see: a scale bias.

What the dimensional error (compared by tape) can see: a scale bias. What it cannot see: local distortion and doubling—the map may be right in length and width and crooked in the middle.

This is why Section 31.1 requires all four of dimensional error, loop consistency, visual inspection, and navigation success rate to be done together. Each has a blind spot, and the blind spots do not overlap.

If ground truth equipment can be borrowed (motion capture, a total station, or the AprilTag scheme of Section 24.3), compute the ATE / RPE directly—but read the next subsection first.

Two: the Second Trap—with Too Few Ground Truth Points, the ATE Itself Wobbles

Suppose you have borrowed the equipment and can lay a few ground truth points about the site. The question is: how many are enough?

Panel (b) of Figure 31-2 uses a representative trajectory from family B whose true ATE is 0.60 meters (computed over all 1,200 samples). Then the case of measuring only k ground truth points is simulated: k points are drawn at random from the trajectory and the ATE computed, repeated 800 times for each k, and the 10th to 90th percentile

interval of the estimate drawn.

The result: at k = 3 the 10–90 interval of the estimate runs from 0.36 to 0.84 meters—the same system may be reported as 0.4 or as 0.8, a difference of more than twofold. At k = 8 the interval narrows to 0.45–0.74, at k = 20 to 0.52–0.70, and only at k = 80 to 0.56–0.64.

What does this mean? If you measure ATE = 0.25 meters with three ground truth points and the next group measures 0.55 with three, you cannot say your system is better—both numbers lie in the same distribution and the difference may be entirely the luck of the sampling. For my system beats the control to stand up, the ground truth points must be numerous enough that the intervals do not overlap.

Ground truth points k	The 10–90 interval of the ATE estimate (m)	Departure from the true 0.60 m
3	0.36 – 0.84	−40% to +40%
5	0.43 – 0.78	−28% to +30%
8	0.45 – 0.74	−25% to +23%
12	0.49 – 0.72	−18% to +20%
20	0.52 – 0.70	−13% to +17%
40	0.54 – 0.66	−10% to +10%
80	0.56 – 0.64	−7% to +7%

Table 31-4 The Number of Ground Truth Points and the Uncertainty of the ATE Estimate

Practical advice: for a comparison between groups or between methods, at least 20 ground truth points; to report only a rough order of magnitude, 8 to 12 are acceptable,

but the number must be stated in the report together with the note that the figure carries about ±25 percent of uncertainty. Writing the uncertainty down will not lower your mark—by the standard of Section 31.7 it is a mark of the excellent grade.

Three: What Students Should Take Away from These Two Experiments

1. A method of evaluation has blind spots, and the blind spots are a property of the method and not a matter of luck. Before choosing a measure, ask what class of error it is insensitive to.
2. Using two or more measures whose blind spots do not overlap is more effective than refining one measure finely.
3. Every estimate carries an uncertainty, including the number with which you prove your system good. With too small a sample, the difference between two figures may mean nothing at all.
4. Writing in the report what my evaluation method cannot see shows that you really understand, better than three more decimal places would.

Advice to Teachers on Planning the Course

This section offers project arrangements for three lengths of course, for teachers' reference.

Kind of course	Hours	Projects advised	How to adjust
High school elective	18 weeks × 1–2 hours	Project One (in simulation) or Three	Mainly simulation and paper activities, with no source code
Undergraduate	18 weeks × 3	Project One +	Walk the whole of mapping,

	hours	Two, or Three	localization, navigation, and assessment
Project / research	One or two semesters	Project Four, Five, or Six	Stress research method, quantitative comparison, and writing the report

Table 31-5 Project Arrangements Suggested for Three Kinds of Course

Principles of Scheduling

- The first third on theory and tools: let students get the environment ready and the basics in hand, with small simulation exercises interspersed to keep up the motivation.
- The middle third on basic implementation: complete the guided implementations of Chapters 29 and 30 and build an experience of success.
- The last third on the project: by now students can explore on their own and the teacher turns adviser.
- Keep slack: a practical course has a great many accidents (hardware failure, environment problems, dependency errors). At least two weeks of slack is advised—and the next section will show that at least two weeks is in fact still optimistic.

Advice on Grouping

- Three or four to a group is ideal: fewer and the workload is too heavy, more and someone easily rides along.
- Divide the work explicitly but rotate: four roles of mapping, navigation, assessment, and reporting, say, rotated part way, so that everyone meets the whole procedure.

- Make assessment a role of its own: it is the work most often sacrificed, because it does not bring the immediate satisfaction that mapping does. Appoint someone to it and require data from them at every milestone.

Cases

Case 31-1 A Map Worth Full Marks, 4 Percent Wrong

One group's Project One result: a classroom map with straight wall lines, nothing doubled, and start and end exactly coincident after a circuit—flawless on visual inspection, and the group wrote in their report that the loop closure error is zero and the map quality excellent.

The reviewer measured the classroom with a tape: 9.2 meters long in fact, 8.83 on the map, an error of 4.0 percent. The cause was that the wheel diameter parameter used the nominal value from the specification sheet while the tire, underinflated, had a smaller real radius—which is the real-world version of the red family in Figure 31-2.

The lesson of this case is not that the students were lazy; quite the contrary: they did an assessment, and the one they did simply has no feeling for this kind of error. The remedy was simple too—run the odometry calibration of Section 29.3 once, and after remapping the dimensional error fell to 0.8 percent. The whole thing took less than an afternoon, but had nobody measured with a tape, that map would have carried its 4 percent error on into the navigation stage, and then had them doubting their navigation parameters while Nav2 missed the goal again and again.

Case 31-2 An Improvement with No Control

One group implemented an improved coverage planner in Project Three and wrote in

their report that the coverage reached 97 percent, a good result. The reviewer asked: what was it with the original method? They could not say.

This is the commonest way of losing marks. The number 97 percent carries no information in itself—Figure 31-1 shows that in a room with 12 percent obstacles the plainest boustrophedon passes 95 percent by 1.23 times the distance, while random bounce does not get there at eight times. Whether 97 percent is remarkable or a matter of course depends on what you compare it with and under what conditions.

The right way to write it is: under the same room arrangements and the same budget of distance travelled, the baseline boustrophedon covered 92.4 percent and this method 97.1, and over 20 random arrangements this method was the better in 17. That sentence has a control, a controlled variable, and a number of repetitions; lacking any one of the three, the conclusion is only an impression.

Case 31-3　Three Typical Modes of Project Failure

Mode one: the aim is too large

Setting out confidently at the start of term to build a robot dog that can inspect the whole campus autonomously, and handing in at the end a report containing only simulations. The trouble is not the ambition but the failure to define a minimum viable result—when the first stage sticks, there is nothing to fall back on.

Mode two: environment problems eat the time

Dependency conflicts, an incompatible driver, hardware arriving late, a site that cannot be booked. Each of these alone is a small thing, and together they can eat half a semester. The simulation in the next section will tell you this is not bad luck but the normal state.

Mode three: no quantitative assessment

A great deal was done and it really ran, but the whole report contains only photographs and a record of operations. By the standard of Section 31.7 such a project bleeds badly across the 50 percent carried by map quality and localization error together—and those fifty percent are the easiest to make good: a tape measure and an afternoon will take a pass to a good.

The remedy common to all three modes is to think clearly at the start of the project about what must be handed in at the end. Keep the marking standard of Section 31.7 and the checklist of Figure 31-4 before you and work backward to what each stage must do—the basic skill of project management, and one of the most practical abilities a student takes into industry or graduate school.

Case 31-4 How Much Slack a Schedule Needs: a Simulation

Keep two weeks of slack is common advice, but where do two weeks come from? This section gives an answer with grounds, by Monte Carlo simulation.

The model: a project is divided into four milestones planned at 3, 4, 4, and 3 weeks, 14 weeks in all. The actual duration of each milestone follows a log-normal distribution whose median equals the planned value—which is to say the plan does not systematically underestimate, it merely has uncertainty. Let σ stand for the size of that uncertainty: σ = 0.25 is a software project that goes smoothly, σ = 0.45 the ordinary case, and σ = 0.65 a project involving hardware, a site, and outside dependencies. Each setting is run 40,000 times.

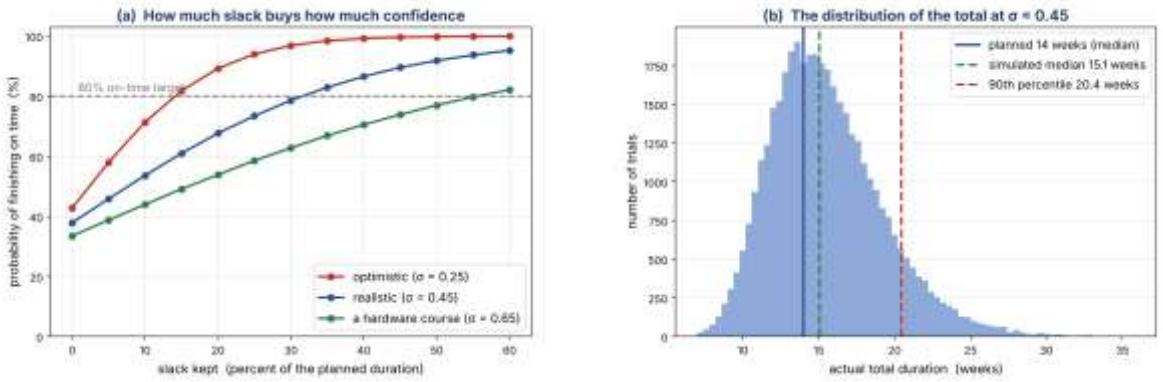


Figure 31-3 The Proportion of Slack and the Rate of Finishing on Time

In panel (a) the horizontal axis is the slack kept as a percentage of the planned duration and the vertical axis the probability of finishing within it. Panel (b) is the distribution of the total duration at σ = 0.45.

One: Three Counterintuitive Numbers

1. Keep no slack and the chance of finishing on time is only three or four in ten. At σ = 0.45 it is 37.9 percent and at σ = 0.65 only 33.5. Note that the median of every milestone here equals the planned value—nobody underestimated, and together they still overrun with high probability.

2. The reason is that the distribution is asymmetric. Look at the histogram in panel (b): the left is blocked at 0 and the right trails a long tail. Finishing a stage early saves a few days at most, while getting stuck can cost three weeks. So the median total duration of 15.1 weeks is already above the planned 14, and the 90th percentile is 20.4 weeks—1.46 times the plan.

3. To reach 80 percent on time, σ = 0.45 needs about 32 percent slack (14 weeks becoming 18.5) and σ = 0.65 about 56. Converted back: for a 14-week project, two weeks of slack buys only a 60 percent chance of being on time; for eight tenths of confidence a software project needs four and a half weeks and a hardware project nearly eight.

Two: Four Implications for Planning a Project

- Put the slack at the end and do not spread it among the stages. Slack spread about is eaten by Parkinson's law (work expands to fill the time available); only slack gathered at the end lets you see how much is left.
- Define a minimum viable result, and reach it at the first milestone. The simulation tells us that overrunning is normal, so it must be guaranteed that there is something in hand when the overrun comes.
- Put the stages of greatest uncertainty first. Hardware arriving, booking a site, installing dependencies—these things of large σ should be stumbled over early, so that the time remaining can respond.
- Re-estimate at the end of every milestone. If the first stage took 1.5 times as long, that is evidence that σ is large for this project of yours, and the expectation for the three stages after must be revised upward too, rather than hoping they will make it back.

The Limits of This Model (and students should write them so in their own reports)

The model assumes the durations of the four stages independent. In reality they are often correlated—unstable hardware slows several stages at once, which makes the long tail heavier, so the figures above are an optimistic estimate.

The model also assumes the median of each stage equals the planned value. Where the plan is itself optimistic (as most people's are), the reality will be worse than Figure 31-3.

The log-normal is a convenient choice and not fitted from data. If you have

records of the actual durations of earlier years' projects, substituting the real distribution and running it again would make a valuable report on improving the course.

Implementation Activities

Implementation 31-1 Planning and Proposing a Project

Purpose: to complete a full planning proposal before committing to a project. The exercise is itself important training—most projects are decided in the planning stage.

Part One: Choosing a Subject and Assessing Its Feasibility

- Choose two or three candidate projects by Table 31-2 and assess each: can the hardware needed be obtained? do you have the ability needed (against the five stages of Section 28.7)? is there time enough?
- For the project that interests you most, list the minimum viable result and the ideal complete result. The first is the floor that must be reached, the second the aim if there is capacity to spare.
- Assess the risks: list the three links most likely to fail and prepare a fallback for each. For instance, if the real rover will not work, complete stage two in simulation.

Part Two: Planning the Milestones

- Divide the project into three to five milestones, each defining explicitly what is to be completed, what evidence it produces, and how long it is expected to take.
- Mark the checkpoints on the schedule. For the proportion of slack follow Case

31-4: keep 30 percent for a project of software or simulation alone, 50 percent or more for one involving hardware, a site, or outside dependencies, and gather the slack at the end.

- For every milestone, list what I will do if this step sticks for more than a week.

Part Three: Designing the Method of Evaluation (crucial)

- By the five dimensions of Section 31.7, design concrete methods of evaluation for your project. What is to be measured for map quality, say, and what will serve as ground truth for localization error.
- Check the measures you chose by Section 31.8: what class of error are they insensitive to? Have you a second measure whose blind spot does not overlap?
- Estimate the number of ground truth points: how many do you intend to measure? By Table 31-4, how much uncertainty does your ATE estimate carry at that number? Is that uncertainty small enough to support the conclusion you want to draw?
- Design a control: what will your result be compared with? (A different method? different parameters? the presence or absence of some component?) Without a control the data mean nothing (Case 31-2).
- Plan the data collection: what data are to be collected at every milestone? In what format recorded? This avoids finding at the end that the data are insufficient (mode three of Case 31-3).

Part Four: Presenting the Proposal

- Put the above into a two-page proposal and prepare a five-minute oral report.
- Present to your classmates and your teacher, with the aim of convincing them that the subject is worthwhile, that you can complete it, and that you know what

difficulties will come.

- Take questions and revise the proposal. Other people's doubts often point out your blind spots—far more efficient than working head-down for three weeks and finding the problem yourself.

Discussion Questions

1. What is your minimum viable result? If only half the time were left, what would you give up?
2. Where does the ground truth in your method of evaluation come from? Without ground truth, how will you argue for the quality of your result?
3. What class of error can your measure not see? What makes it good? (Compare Figure 31-2.)
4. What is your control? Why does that control bring out the value of your result?
5. By the three failure modes of Case 31-3, which is your project likeliest to fall into? What is your precaution?
6. By Case 31-4, about which band does your project's σ fall in? Is the slack you kept enough?

Extended Challenge

Write your proposal up in the formal shape of a research plan: the background and motivation, a literature review (the further reading of this book's chapters may be cited directly), the method, the expected results, the schedule, and the references. The document is not only the starting point of the project but the basis of a graduate application, a National Science and Technology Council undergraduate research proposal, or a competition entry later on. The act of writing it will itself force your ideas clear—the greatest value of this activity.

Implementation 31-2 Stress-Testing Your Method of Evaluation

Purpose: to examine, by the methods of Section 31.8, the way you yourself intend to evaluate. This activity needs only a computer and can be done in one class, but students who have done it usually go back and change their original design of evaluation.

1. Take a trajectory of your own (or generate one going round a circuit in simulation to serve as ground truth).
2. Add two errors by hand: (a) multiply every step length by 1.03, simulating a wheel diameter measured wrongly; (b) add a zero-mean random error to the heading at every step and accumulate it. Generate 50 trajectories of each.
3. For each, compute three numbers: the loop closure gap, the dimensional error, and the ATE against the ground truth.
4. Draw the two scatter plots of loop closure gap against ATE and dimensional error against ATE, and compute the correlation of each.
5. Answer: which error is the measure you originally intended to use blind to? What measure must be added to make it good?
6. Advanced: draw k points at random from your trajectory to estimate the ATE (k of 3, 5, 10, 20, 50), repeat 500 times for each k, and draw the distribution of the estimates. How many ground truth points do you actually intend to lay? At that number, could you confidently distinguish a system with an error of 0.3 meters from one with 0.5?

The product of this activity—two scatter plots and a paragraph—can go straight into the evaluation methods section of your project report. By the standard of Section 31.7, a report that can say I know what my measure cannot see already stands at excellent on the dimension of the report.

Further Reading

- J. Sturm et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," IROS, 2012. (The authoritative source for the ATE and RPE methods, and the paper describing the TUM RGB-D dataset.)
- The evo trajectory evaluation tool (GitHub: MichaelGrupp/evo). (An essential tool for project assessment, supporting many dataset formats and figure outputs.)
- Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," IROS, 2018. (A complete tutorial on trajectory evaluation, including details such as scale alignment; the concerns of Section 31.8 agree with this paper.)
- C. Cadena et al., "Past, Present, and Future of SLAM," IEEE T-RO, 2016. (The authoritative review cited as early as Chapter 1, and a good starting point for a project's literature review.)
- Section 13.7 of this book (assessing map quality), Case 17-1 in Chapter 17 (the wall thickness index), and Section 22.7 (trajectory evaluation). (Internal references for the methods of project assessment.)
- S. Macenski et al., "The Marathon 2: A Navigation System," IROS, 2020. (The Nav2 paper, whose methodology for assessing system performance is worth consulting.)
- E. Galceran and M. Carreras, "A Survey on Coverage Path Planning for Robotics," Robotics and Autonomous Systems, 2013. (A review of coverage path planning, the theoretical basis of Project Three.)

- Collections of outstanding undergraduate research proposals and project competition entries from various universities. (To see how others organize a complete project report.)

Two Figures Worth Pinning Above Your Desk

Figure 31-4 The Complete Flow of a Project from Planning to Presentation

【The planning stage】

① Choose the project by Table 31-2 → ② Assess the resources and the ability → ③ Divide it into three to five milestones

④ Decide the proportion of slack by Case 31-4 (30% for software, 50% or more for hardware, gathered at the end)

↓

【The execution stage】

⑤ Verify in simulation (get the procedure and the parameters working first)

⑥ Implement for real (allow twice the time for debugging)

⑦ Produce verifiable evidence at every milestone (Section 29.7), and re-estimate the time remaining

↓

【The assessment stage】

⑧ Check yourself against the five dimensions of Section 31.7

⑨ Assess quantitatively: ATE / RPE / dimensional error / coverage (as the project requires)

⑩ Check by Section 31.8 whether the measures have blind spots and whether the ground

truth points suffice

⑪ Record honestly the attempts that failed and why

↓

【The presentation stage】

⑫ Write the report: the problem, the method, the data, the analysis, the limitations

⑬ Prepare a three-minute core account + a demonstration video

⑭ Prepare a reflection on what I would do differently

Figure 31-5 The Checklist Before Handing In

□ Map quality: is there quantitative data on the dimensional error? Can the source of the error be explained?

□ Localization accuracy: is there data on the ATE or the loop closure gap? Were both measured?

□ The measure's blind spot: what class of error can my measure not see? Has a second measure been added?

□ Ground truth points: are there enough to support the conclusion I want to draw? Is the uncertainty written down?

□ System stability: has the boundary of failure been tested? Is it known under what conditions the system breaks?

□ The control: is there a comparison with another method or another setting of the parameters? Are the controlled variables stated?

□ A record of failures: are the failed attempts and how they were solved recorded honestly?

□ Complete evidence: are the map files, the trajectory figures, the recordings, and the code all there?

□ Reproducibility: can another person reproduce the result from your documentation?

□ Reflection: can you say what you would do differently?

Summary

- The six projects rise in difficulty: 2D mapping of a classroom (★), three-dimensional scanning (★★), path analysis for a robot vacuum (★★), campus inspection with a robot dog (★★★★), a vision-navigated drone (★★★★), and multi-sensor fused localization (★★★★★); in choosing, weigh the resources, the ability, and the continuity.
- Project One covers the whole cycle of SLAM and its result is plain to see; what is assessed is not whether a map was built but whether it can be said clearly why the map is good or bad. The product of Project Two has long-term value for managing departmental space and for a simulation environment, and may become the scene for the training in simulation of Chapter 33 directly.
- The experimental result of Project Three: at one times the theoretical minimum distance, the spiral covers 83.2 percent, the boustrophedon 75.6, and random bounce only 35.3; to reach 95 percent coverage the boustrophedon needs 0.95 times the distance in an empty room and random bounce 6.9, while in a room with 12 percent obstacles or more random bounce does not get there even at eight times. The advantage of a systematic strategy is not merely speed but its guarantee of coverage.
- Marking takes five dimensions: map quality (25%), localization error (25%), system stability (20%), the report (20%), and code quality (10%); reading the standard at the start of the project and planning backward from it is advised.
- The method of evaluation has blind spots of its own: a map error from a scale bias is wholly invisible to the loop closure gap—in the experiment this family's loop closure gap was 0.000 meters while the median ATE was 0.25, the median dimensional error 3.2 percent, and the maximum 6.0. The loop closure gap and the dimensional error must be used together, because their blind spots do not

overlap.

- The figures of an evaluation carry uncertainty themselves: for one trajectory whose true ATE is 0.60 meters, the 10–90 interval estimated from 3 ground truth points is 0.20–0.88 meters and only narrows to 0.46–0.73 at 20 points. For a comparison between groups, use at least 20 ground truth points and state the number and the uncertainty in the report.
- The uncertainty of a schedule is asymmetric: a project of four milestones planned at 14 weeks, under ordinary uncertainty and with no slack, finishes on time only 37.9 percent of the time, with a median total duration of 15.1 weeks and a 90th percentile of 20.4. For eight tenths of confidence a software project needs about 30 percent of slack and a hardware project about 55, and the slack should be gathered at the end rather than spread about.
- The three typical modes of project failure are an aim too large, environment problems eating the time, and no quantitative assessment; the remedy common to all is to think clearly at the start about what must be handed in at the end, and to plan each stage backward from the marking standard.
- Reporting a failed attempt honestly should earn marks rather than lose them—displaying the whole course of finding the problem, analyzing the cause, and verifying the improvement says far more about the depth of understanding than presenting a handsome result alone; and writing what my evaluation method cannot see is likewise a mark of the excellent grade.
- Advice on presenting: the core should take three minutes, use a recorded video rather than a live demonstration, the data need a control, and prepare a reflection on what I would do differently.

The Close of Part VIII

Part VIII, Implementation in ROS / Python / C++, is now complete. From the environment of Chapter 28, through the LiDAR mapping and navigation of Chapter 29 and the visual SLAM and cross-sensor comparison of Chapter 30, to this chapter's integration and assessment of a project, students have walked the whole path from theory to practice.

And remember the two scatter plots of Section 31.8. The thirty chapters before taught how to let a robot know where it is; this chapter adds how to know that it really knows—and the latter is the more easily skipped, and the better at telling the quality of a piece of work.

In the last part we turn our eyes to the future. Traditional SLAM solves where am I and what does the map look like; AI and the VLM are to solve what am I looking at, what does this place mean, and what should be done next. Part IX has three chapters: Chapter 32, Semantic SLAM: Giving the Map Meaning, raises a geometric map into a semantic one; Chapter 33, Training in Simulation and Sim-to-Real, lets a robot learn a skill in a virtual environment and deploy it on a real body; and Chapter 34, VLM, Language Navigation, and Embodied Intelligence, lets a robot understand an instruction such as please go to the registrar's office, and brings the book to its close.

The habits of evaluation you have built in this chapter will matter more in Part IX, not less. A failure at the layer of semantics and language usually raises no error on the spot—it merely takes the robot quietly to the wrong room.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Part IX AI, VLMs and Semantic Navigation SLAM

Chapter 32

Semantic SLAM: Giving the Map a Meaning

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Learning Objectives

Thirty-one chapters on, students can now have a robot answer where am I and what does the map look like. This chapter adds the third question: what is this place? We begin with the limits of classical SLAM, explain the division of labor among AI vision models within SLAM and the four ways of representing a semantic map, then take up two practical difficulties—dynamic objects and the semantic cost map—and close with a survey of the international research landscape. By the end of this chapter students should be able to:

1. Explain the limits of a geometric map, and why the three levels of robot intelligence are laid one upon another rather than replacing one another.
2. Distinguish the roles of five classes of AI vision model in SLAM: detection, segmentation, depth, description, and scene understanding.
3. Explain the four levels of a semantic map, and why open vocabulary and scene graphs have become the focus of recent research.
4. Explain why a dynamic object pollutes a map, and design a multi-pass consistency rule for removing it.
5. Explain how semantics enter the cost map and change a navigation decision, and estimate what that costs and what it buys.
6. Explain how an error in a semantic label propagates onto the map through the extrinsics and the localization.
7. Give examples of the principal lines of research in semantic SLAM internationally, and of the systems that represent them.

Thirty-one chapters on, students can now have a robot answer where am I and what

does the map look like. But that is not enough—a robot of real practical use must also answer what have I seen, what does this place stand for, and what should I do next. This chapter is the first of Part IX, and the doorway into the decade to come. By the end of it students should be able to:

1. Explain the limits of classical SLAM, and why AI has become a necessary supplement to it.
2. Explain the division of labor among YOLO, SAM, Depth Anything, DINO and CLIP in robot perception.
3. Explain the line of development from Visual SLAM to Semantic SLAM and on to embodied AI navigation.
4. Plan the collection and labeling of training data for a robot, and train a model that recognizes scenes about a campus.
5. Explain how a VLM understands a navigation instruction given in natural language, and how RAG supplies spatial knowledge of a campus.
6. Explain the key techniques of deploying AI at the edge: quantization, distillation, and the management of memory.
7. Analyze the risks of AI-assisted SLAM, and look ahead to the union of SLAM with foundation models.

Core Concepts

Term	Meaning	Related Sections
Semantic map	Category, instance and relational information attached to a geometric map	32.3

Instance segmentation	Cutting an image into separate objects rather than categories alone	32.2
Open vocabulary	Categories need not be defined in a fixed list beforehand	32.3, 32.6
Three-dimensional scene graph	Describing space by an object–region–floor hierarchy and its relations	32.3
Removal of dynamic objects	Keeping what moves out of the map	32.4
Semantic cost map	Turning what a thing is into a cost for navigation	32.5
Propagated labeling error	The semantics right but the extrinsics or the localization out, so the label still goes on crooked	32.5, Chapter 10
Geometry–semantics–task	The three levels of robot intelligence, each holding up the one above	32.1

中文名詞	English Term	Explanation
語意 SLAM	Semantic SLAM	SLAM that annotates a geometric map with objects and the meaning of a scene.
具身智慧	Embodied AI	Artificial intelligence that learns and acts by interacting with a physical environment.
開放詞彙	Open-Vocabulary	Not bound by predefined categories, able to recognize objects unseen at training time.
物件目標導航	ObjectNav	A navigation task that takes an instruction

中文名詞	English Term	Explanation
		to go and find a certain object.
場景圖	Scene Graph	A structure describing the objects of a scene and their spatial relations as nodes and relations.
視覺語言模型	VLM	A multimodal model that understands images and language at once.
視覺語言動作模型	VLA	A model that produces action commands directly from images and language.
檢索增強生成	RAG	Retrieving relevant material before generating an answer, so as to improve its correctness.
模型蒸餾	Distillation	A method of training that transfers the ability of a large model to a small one.
量化	Quantization	Representing a model's parameters in fewer bits to lower memory and computation.
邊緣部署	Edge Deployment	Running AI inference on the computing platform aboard the robot itself.
幻覺	Hallucination	The phenomenon of AI producing content that looks reasonable and is in fact wrong.
世界模型	World Model	A model that learns the dynamics of an environment and can predict its future state.

32.1 Why Classical SLAM Still Needs AI

What the first thirty chapters of this book built is an extremely solid geometric system. It can tell you precisely where the robot is and what shape the environment has—but it has one fundamental blind spot.

Classical SLAM Knows Shape and Not Meaning

A robot running Cartographer, facing a door, knows this: 1.2 meters ahead there is a gap 0.9 meters wide, with a continuous plane on either side. It does not know that the thing is a door—does not know that a door can be passed through, that a door may close, that behind a door there is usually another room.

The difference is of no consequence in a simple task (avoiding an obstacle does not require knowing what the obstacle is), and yet it is fatal in a real service task:

- The user says go to the registrar's office, and the map holds only coordinates, no names.
- There is a puddle on the floor: geometrically it is flat ground, semantically the robot ought to go around it.
- Ahead is a glass door, which the LiDAR cannot see (Chapter 8) but which vision can recognize by its frame and handle.
- A person is standing in the corridor: geometrically an obstacle, semantically someone the robot should slow down and give way to, rather than skirt as though he were a wall.

The Gap Between the Three Levels

Level	The question it answers	Technique	Chapters of this book
Geometric	Where am I? What shape is the environment?	SLAM	Chapters 1–30
Semantic	What have I seen? What sort of place is this?	Object detection, segmentation, VLMs	This chapter
Task	What should I do next?	LLM agents, VLAs	This chapter

Table 32-1 The Three Levels of Robot Intelligence

These three levels are laid one upon another and do not replace one another—the most important idea in this chapter. Without a solid geometric level beneath it, the semantic level leaves the robot knowing that the thing is a door and unable to reach the doorway; and without a semantic level above it, the geometric level leaves the robot able only to execute coordinate commands, and unable to interact naturally with people.

What AI Brings

1. An enlargement of perception: recognizing objects, segmenting a scene, estimating depth—turning pixels into information that means something.
2. An interface of language: letting the user give a task in natural language rather than by entering coordinates.
3. Inference from common sense: a washbasin is usually in a toilet or a tea room—knowledge of this kind cannot be derived from a geometric map, and yet it raises the efficiency of exploration a great deal (Section 24.5).
4. The ability to generalize: to judge sensibly even when faced with an environment or an object not seen during training.

32.2 The Role of AI Vision Models in SLAM

Sections 23.4 and 24.5 introduced YOLO and VLMs; this section sets out systematically the division of labor among the classes of vision model—a map for planning an AI perception system.

The Division of Labor Among Five Classes of Model

Recent surveys point out the strengths and weaknesses of each family with some clarity. Image-text contrastive models (CLIP, SigLIP) are the default front end of a vision-language model, good at semantic retrieval and weaker at dense prediction; self-supervised vision transformers (DINOv2, DINOv3) retain fine spatial structure because of their training objective and do excellently at dense prediction—segmentation, depth, correspondence—though their features are not aligned with language; segmentation foundation models (SAM, SAM 2) are the experts at proposing and tracking objects; and the dense prediction specialists (Depth Anything, MiDaS) handle depth.

Class of model	Representatives	What it is good at	Its role in SLAM
Object detection	YOLO	Boxing an object's category in real time	Semantic labeling, removal of dynamic objects
Image-text contrastive	CLIP, SigLIP	Open-vocabulary semantic retrieval	Place recognition, queries in language
Self-supervised	DINOv2 / v3	Dense prediction,	Feature extraction, segmentation

Class of model	Representatives	What it is good at	Its role in SLAM
vision		spatial structure	
Segmentation foundation models	SAM, SAM 2	Proposing and tracking objects	Precise object boundaries, tracking what moves
Depth estimation	Depth Anything	Estimating depth from a single image	Supplying depth for a monocular camera, dense reconstruction

Table 32-2 The Roles of Five Classes of AI Vision Model in SLAM

One Crucial Engineering Constraint

Here is a fact of great importance to deploying a robot: a practical video system on a wearable device, a robot, or a smart camera cannot possibly carry one backbone network for every capability; it can neither compromise on capability nor bear the cost in memory and latency.

Put otherwise: although every task has its own best specialized model, a robot's computing resources will not allow five large models to run at once. In practice a trade-off must be made—choose one or two critical models, or use a unified backbone that can serve several tasks. This is precisely the thinking of Chapter 25, hardware and algorithm designed together, carried into the age of AI.

Advice to the Reader of This Book

- To begin: start from YOLO. It is mature, light, and rich in resources, and produces a visible result at once (the implementation of Section 32.4).

- Going further: add a depth estimation model to supply what a monocular camera lacks (Section 9.4), or use a segmentation model to obtain precise object boundaries.
- For research: explore open-vocabulary models, so that the robot can recognize objects not defined at training time—the key to whether semantic navigation can be made practical.

32.3 From Visual SLAM to Semantic SLAM to Embodied AI Navigation

This section sets out that line of development, so that students may see clearly where what they have learned stands and in what direction it runs.

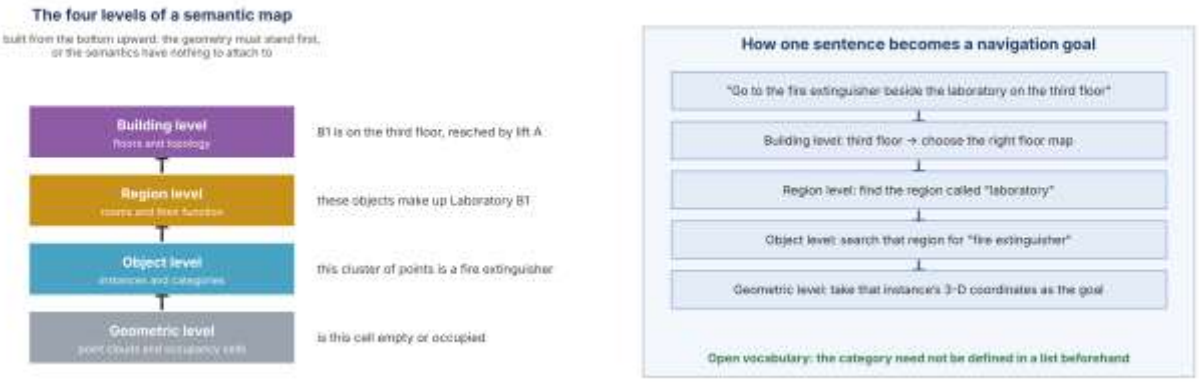
Three Stages

5. Visual SLAM (Chapters 20 to 23): the output is the camera's trajectory and a sparse or dense geometric map. Every point in the map has coordinates and no meaning.
6. Semantic SLAM: object recognition and segmentation are carried out while the map is built, and semantic labels are attached to the elements of the map. A point in the map is no longer merely there is something here but this is part of a chair.
7. Embodied AI navigation: the robot has not only a semantic map but the ability to understand a task, reason from common sense, and plan an action. It can accept an instruction such as go and turn off the light in the meeting room and carry it out by itself.

Two Ways of Doing Semantic SLAM

- As post-processing: build the geometric map first with classical SLAM, then project the results of object detection onto it. Simple to implement and loosely coupled in its modules, it is the first choice for teaching and for a student project (the route taken by the implementation of Section 32.4).
- Tightly coupled: bring the semantic information into SLAM's own estimation—replacing point-level association with object-level association, say, or using semantic priors such as a wall is a plane and a floor is horizontal to constrain the optimization. More accurate, and more complicated to implement.

Figure 32-1 The Four Levels of a Semantic Map, and How One Sentence Becomes a Navigation Goal



A semantic map is often misunderstood as sticking labels on a map, but semantics that are really useful have a hierarchy. The left half of the figure divides it into four levels. At the bottom is the geometric level, which answers only whether this cell is empty or occupied; above it the object level, which takes a cluster of points to be one instance—this is a fire extinguisher; above that the region level, which assigns a group of objects to one functional space—Laboratory B1; and at the top the building level, which records floors and the topology among them—B1 is on the third floor and is reached by lift A.

The hierarchy is built from the bottom upward and the order cannot be reversed:

without correct geometry the object instances fall in the wrong places, and without objects there is nothing from which to induce a region. This too is why this book puts semantics in Part IX—it needs all eight preceding parts in place before it can stand at all.

The right half of the figure then demonstrates what the hierarchy is for: parsing a sentence from the top downward. The user says go to the fire extinguisher beside the laboratory on the third floor; the system first chooses the right floor map at the building level, then finds the laboratory at the region level, then searches the object level within that region for a fire extinguisher, and finally takes that instance's three-dimensional coordinates as the navigation goal. Each level need answer only one simple question, rather than one model having to handle the whole sentence at once.

Here is exactly where the value of a three-dimensional scene graph lies. If the map were no more than a heap of cells marked with categories, the spatial relation beside the laboratory could not be expressed at all; and an open vocabulary frees the last step of the search from a fixed list of categories—even with no category fire extinguisher at training time, it can still be found by matching text features.

What the Four Levels Mean in Practice

The geometric level decides safety: whether a thing will be run into depends on this level alone. A semantic error is not fatal; a geometric one is.

The object level decides interaction: whether it can go and fetch that thing depends on whether the instance is correctly segmented and located.

The region level decides efficiency: only with the notion of a room can one say clean the kitchen alone or do the third floor first (Chapters 16 and 19).

The building level decides reachability: navigating between floors needs topology, and a lift is a typical topological node rather than a geometric obstacle.

Advice for a project: reaching the object level alone is already of value; the region level can be segmented automatically by the distance transform method of Section 16.4; and the building level can usually be labeled by hand.

The Frontier: Open Vocabulary and Scene Graphs

Recent research is pressing forward in two directions:

- Open-vocabulary online semantic mapping: freeing a SLAM system from a predefined set of categories, so that it can recognize and label arbitrary objects while it maps—which matters greatly given the complexity and variety of real environments.
- Scene graph representations: recording not merely that there is a chair here but the structure of relations—the chair is beside the table, the table is in the meeting room. Recent work has shown zero-shot embodied navigation supported by a multimodal three-dimensional scene graph.

The value of a scene graph lies in its ability to support inference: when the user says go and fetch the cup on the table, the robot must understand the spatial relation on the table—which a purely geometric map cannot express.

The Convergence of the Academy and Industry

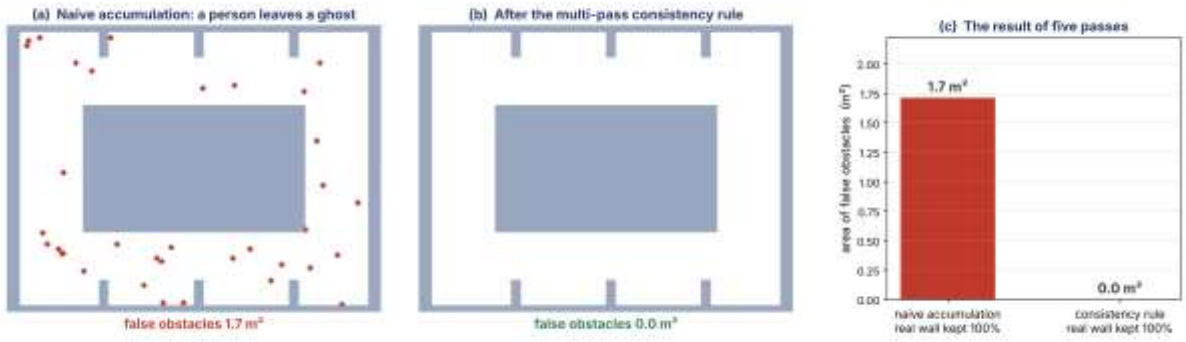
It is worth noting that along this line of development the academy and industry are converging quickly. Research in the field has already brought SLAM together with semantics, three-dimensional Gaussians, the bird's-eye view, navigation, large

language models, and multi-stage operation into a single range of concerns for one research community. For students this means that the geometric foundation this book teaches is precisely the ticket into that lively field.

32.4 A Dynamic Environment: What Moves Should Not Go into the Map

The most direct use of semantics, and the earliest the industry adopted, is in fact not recognizing what a thing is but recognizing what ought not to be recorded. Section 13.5 discussed how upper and lower bounds on the occupancy probability keep a map malleable, but that is a passive forgetting; with semantics the robot can judge actively: this is a person, so do not write it into the map.

Figure 32-2 The Ghosts People Leave Behind, and How to Clear Them Away



In this experiment the robot goes five times around the same floor, with seven people at random positions in the environment on every pass. Panel (a) is the result of naive accumulation: a cell is recorded as an obstacle if it was ever observed occupied on any pass. Grey is the real wall and red the false obstacles added to the map—1.7 square meters in all. They are scattered up and down the corridor, and every patch is a place where somebody happened to be standing on some pass.

Panel (b) is the same with a multi-pass consistency rule added: a cell must be observed

occupied on at least three of the five passes before it is recorded as an obstacle. The result is that the false obstacles fall from 1.7 square meters to zero, while the real wall is still kept at 100 percent. The reason is plain—the wall is there on every pass, a person is not.

The rule works because it exploits one semantic fact: permanent structure and temporary objects behave entirely differently in time. It is worth noting that no deep learning model is used here at all, only the statistical method of voting across passes—which is the concrete implementation of the layered map strategy of Section 19.6.

When, then, is AI really needed? The answer is when you cannot wait for five passes. Voting across passes takes time, and the robot must avoid the person walking toward it on the very first pass. What is needed then is instance segmentation in real time: recognize in a single frame that this is a person, and exclude the corresponding points from the mapping outright. The two methods are used together in practice—AI handles the present moment, voting handles the long term.

The Three Lines of Defense Against Dynamic Objects

The immediate layer (milliseconds): remove known dynamic categories—people, trolleys—within a single frame by instance segmentation or object detection. Quick to react, and recognizes only the categories it was trained on.

The short-term layer (seconds to minutes): detect a moving object by the inconsistency between consecutive frames, needing no knowledge of categories, though insensitive to slowly moving objects.

The long-term layer (between passes): the multi-pass consistency voting of this figure. It needs no model at all and handles changes of every kind, at the cost

of needing time to accumulate.

The three lines of defense solve problems at different scales of time, and a practical system usually uses all three rather than picking the best one.

32.5 How Semantics Change a Navigation Decision

This section explains how semantics really change the behavior of the planner, and what the traps are.

From Category to Cost: Three Ways of Injecting It

- Injection at the region level: mark a whole region with a category (a forbidden zone, a busy thoroughfare), then map the category to a cost. The simplest, and the one most often used for forbidden zones set by hand.
- Injection at the object level: when a particular object is detected (glass, a mirror, the head of a staircase), add a ring of cost around it. Suited to objects on which a sensor will fail.
- Learned traversability: predict directly from the image how easy this place is to drive over, with no explicit category in between. Popular in recent years outdoors and on unstructured terrain, at the cost of making a failure hard to explain.

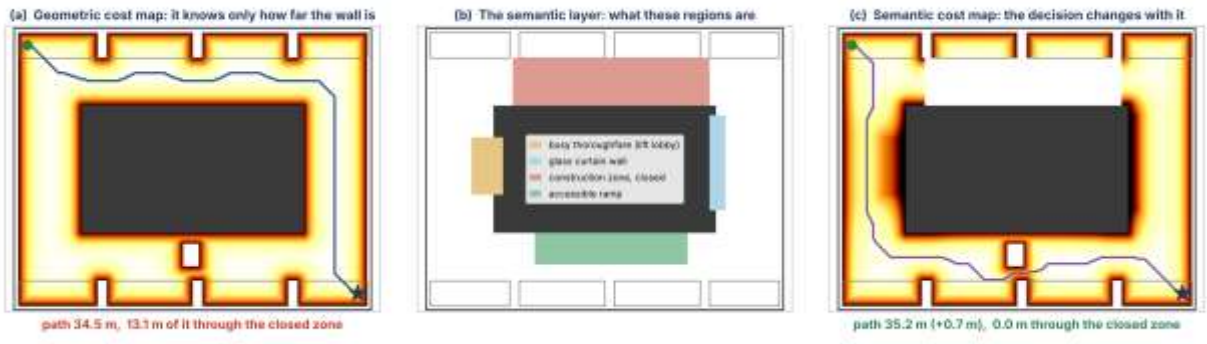
A Trap Easily Overlooked: the Label Goes on Crooked

The accuracy of semantic segmentation is often treated as the only measure, but for SLAM recognizing it correctly and putting it in the right place are two different things. A model that recognizes a fire extinguisher perfectly will, if the extrinsics between camera and LiDAR are two degrees out, put the label tens of centimeters from where it belongs on the map.

Formula Two has already written out that chain of propagation: a detection on the image must be back-projected through the camera intrinsics, then carried through the extrinsics from camera to chassis, from chassis to odometry, and from odometry to map, before it lands at a final map coordinate. An error at any link of that chain accumulates. Put otherwise, the quality of a semantic map depends at once on the AI's rate of recognition and on the quality of the coordinate transforms of Chapter 10—and in practice the latter is more often the bottleneck.

This leads to a very practical order of debugging: when a semantic label goes on crooked, do not rush to change to a larger model. First verify the extrinsics and the localization with an object whose position is known (a target stuck on the wall), and only once the geometric chain is confirmed sound go back and examine the recognition model. The contents of Parts I to VIII of this book are still the foundation here.

Figure 32-3 The Same Map, and How the Decision Changes Once Semantics Are Added



The experiment of this book uses a floor whose corridors run in a loop: at the center a core of lifts and machine rooms, corridors around it, and classrooms and laboratories outside them. The robot is to carry something from the room at the top left to the one at the bottom right—and since either side of the core will serve, which side to take

becomes a real decision.

Panel (a) is the cost map with geometric information alone. The redder the color the closer to an obstacle and the higher the cost, so the planner tends to keep to the middle of the corridor. It chose the clockwise route along the top, 34.5 meters in all—and by a purely geometric standard that is the right answer. The trouble is that 13.1 meters of that route pass through a construction zone that is closed, and the geometric map has no idea the place is closed: to it that is merely a stretch of corridor with nothing in the way.

Panel (b) is the same floor with a semantic layer added. Four regions are marked out: the busy thoroughfare by the lift lobby, the glass curtain wall, the construction zone, and the accessible ramp. None of this can be seen geometrically at all—the glass curtain wall is a thing the LiDAR cannot even see, and it appears on the map only because another sensing modality and a semantic model put it there.

Panel (c) is the result of turning the semantics into cost. The closed zone is set impassable, the thoroughfare and the glass wall are given an extra cost that decays with distance, and the ramp is given a lower cost because it is easy going. The planner therefore takes the counterclockwise route along the bottom, 35.2 meters in all—only 0.7 meters more, and yet the 13 meters of passage through the closed zone fall to nothing. Those 0.7 meters are the price of semantics, and what they buy is a route that is compliant, safe, and better matched to what a person would expect.

How a Semantic Cost Should Reasonably Be Set

Hard constraints against soft costs: a forbidden zone or a construction zone should be set impassable (a hard constraint); a busy thoroughfare or a glass

curtain wall suits an extra cost that can be weighed against others (a soft cost).

The decay distance matters more than the number: this figure sets a decay distance of 1.0 meters for the glass curtain wall and 1.3 for the busy thoroughfare. That distance decides how far away the robot begins to keep clear, and affects its behavior more than the numerical cost does.

A negative cost is useful too: the accessible ramp is given a low cost, which amounts to encouraging the robot to use it. Semantics are not only for avoiding bad places but for preferring good ones.

Beware of over-weighting: set every semantic region to a high cost and the planner becomes conservative, or finds no path at all. In practice it is advised to add one or two semantic layers at a time, and to confirm the behavior is reasonable before adding more.

32.6 A Map of the International Research: Three Lines in Semantic SLAM

Semantic SLAM has developed internationally along roughly three lines, each with its representative research groups and systems. This section offers a map, so that students know where to look for material.

One: Metric-Semantic SLAM—Fitting Semantics into an Existing SLAM Framework

The claim of this line is: do not start afresh, but add a layer of semantics to a mature SLAM system. Representative work attaches instance segmentation to the back end of a visual-inertial SLAM, maintaining object-level instances while the geometric map is built and organizing them in a three-dimensional scene graph. Its merit is that it inherits

the stability of the existing system, and it fits the architecture of Parts I to VIII of this book best.

Two: Open-Vocabulary Maps—Escaping the Fixed List of Categories

Classical semantic segmentation can recognize only the few dozen categories defined at training time, and the things in a real site are far more than that. The open-vocabulary line stores the features of a vision-language model directly in every element of the map and matches text against them at query time—so that a query never defined in advance, find the red fire extinguisher, will work. The price is a sharp rise in memory use, and a result that cannot be guaranteed.

Three: Neural Fields and Renderable Representations—Localization and Reconstruction in One

Three-dimensional Gaussian splatting, mentioned in Sections 17.8 and 20.8, has its counterpart in the semantic field: store semantic features in every Gaussian element as well, so that the map can both render a lifelike image and answer a semantic query. This line has the advantage in the quality of its reconstruction and the highest cost in computation and memory, and is at present used more for offline reconstruction than for navigation in real time.

The trade-off among the three lines may be understood thus: the first is the steadiest and the best suited to deployment; the second the most flexible and the best suited to an open site; the third the handsomest and the best suited to demonstration and offline analysis. For a student project the first is advised—it is wholly compatible with this book's stack of techniques, and when something goes wrong you know which layer to look at.

How to Follow the Progress of This Field

Conferences: ICRA, IROS, RSS, CVPR and CoRL are the chief venues at which semantic SLAM is published.

Keywords: semantic SLAM, open-vocabulary mapping, 3D scene graph, instance-aware mapping, dynamic SLAM.

Datasets for assessment: indoors, chiefly ScanNet and Replica; outdoors and for autonomous driving, SemanticKITTI and nuScenes.

Where to start implementing: attach an off-the-shelf segmentation model to an existing ORB-SLAM or RTAB-Map, project the result onto the map, and walk the chain of propagation of Formula Two with your own hands—this gives a better feel for it than reading ten papers.

Formulas and Procedures

One: the Measures by Which Object Detection Is Assessed

In training a detection model such as YOLO the measure most used is the mean average precision:

$$mAP = (1/C)\ \Sigma_c AP_c$$

where C is the number of categories and AP_c the average precision of category c (the area under the precision-recall curve). Common variants are mAP@0.5, computed at an intersection-over-union threshold of 0.5, and mAP@0.5:0.95, averaged over several thresholds (the stricter of the two).

For a robot application two practical measures deserve attention as well:

- The miss rate: the proportion not detected. For objects bearing on safety (a person, the head of a staircase) a miss is far more dangerous than a false alarm.
- The performance of each category: a high overall mAP does not mean that every category is good. If the precision of the category staircase is particularly low, and that is precisely what safety turns on, then the overall figure is misleading.

Two: Projecting a Detection onto a Three-Dimensional Map

The central computation of semantic labeling is turning a detection box on the image into a three-dimensional coordinate. Taking the center pixel (u, v) of the box and its depth d, back-project by Formula One of Chapter 30:

$$P_{camera} = \left((u - c_x) \cdot Z / f_x ,\ (v - c_y) \cdot Z / f_y , Z \right) \quad , Z = d / scale$$

and then transform it into the map frame by the chain of coordinate transforms of Chapter 10:

$$P_{map} = T_{map \leftarrow odom} \cdot T_{odom \leftarrow base} \cdot T_{base \leftarrow camera} \cdot P_{camera}$$

This formula joins the AI of this chapter to the coordinate transforms of Chapter 10—semantic labeling looks like the work of AI, and yet whether it lands correctly on the map depends on the quality of the extrinsics and the localization. Which bears out once more the point of Section 32.1: the semantic level must be built upon a solid geometric level.

Four: the Complete Data Flow of AI-Assisted SLAM

Figure 32-4 The End-to-End Flow from Sensing to a Semantic Decision

① Sensors: camera, LiDAR, IMU

↓

② Geometric SLAM: localization and mapping (Chapters 13–22) → pose + geometric map

↓

③ AI perception: YOLO detection / segmentation / depth estimation (Section 32.2)

↓ project the detection into map coordinates by Formula Two

④ Semantic map: geometry + object labels + spatial relations

↓

⑤ RAG knowledge base: campus topology, the attributes of places, the rules of passage (Section 32.5)

↓

⑥ The VLM / LLM agent: understand the instruction, reason, choose the goal (Section 32.6)

↓ output a structured geometric goal

⑦ Path planning and control (Chapter 29) → velocity commands

↓

⑧ Execute → the environment changes → return to ①

【The guard of safety】step ⑦ has the right to overrule the decision of ⑥

Cases

Cases

Case 32-1 The Semantics Right, the Position Wrong

A failure extremely common in student projects: the student attaches an object detection model better than ninety percent accurate, the robot does indeed recognize the fire extinguisher in the corridor, and yet a look at the map shows the label floating a meter outside the wall, or even landing in the room next door.

The instinctive reaction is usually that the model is not good enough, so a larger model is fetched and more data labeled afresh. But the real cause is almost never in the model. What detection gives is a box on an image; to turn it into a point on the map it must pass through depth estimation, the camera intrinsics, the extrinsics from camera to chassis, and the localization of the moment—four links. The one that most often goes wrong is the extrinsics: the camera was not calibrated again after it was mounted, or the design value was used rather than the measured one, and two or three degrees are enough to cause an offset of tens of centimeters.

The way to diagnose it is simple: take an object whose position is known (a target stuck on the wall) as the subject of the test, have the robot detect it ten times from different angles, and see whether the labeled points converge on one position. If the labeled points spread out with the viewing angle, the trouble is in the extrinsics or the localization; if they are tight but displaced as a body, the trouble is in the translation of the extrinsics; only when the labeled points themselves jump about is it necessary to go back and examine the model.

The lesson of this case is the same as that of the repeatability of localization in Section

19.5: for the output of AI to land in the right place depends on the coordinate transforms of Chapter 10 and the localization of Chapter 11. However strong the semantic layer, it stands on the shoulders of the geometric one.

Case 32-2 Why a Robot Vacuum Does Not Record Your Feet as a Wall

An application of semantics that happens every day and is seldom noticed. While a robot vacuum works, people walk about the house; if it recorded every foot it detected as an obstacle, the map would soon be strewn with walls that do not exist, and the coverage planning of Chapter 16 would fail along with it.

What it does is exactly the three lines of defense of Figure 32-2. The immediate layer: judge by infrared or by camera that what is ahead is a moving object, and go around it for the moment without writing it into the map. The short-term layer: a position that is occupied on some scans and not on others over several consecutive scans is judged an unstable observation. The long-term layer: an obstacle that has not appeared again over several cleanings has its occupancy probability lowered step by step until it vanishes from the map (the mechanism of upper and lower bounds of Section 13.5).

It is worth noting that a low-end model often has only the long-term layer, which is why you will see it record a backpack left on the floor as an obstacle and go on driving around a thing that is no longer there for days. That difference is the concrete value of semantic ability in a product—it shows itself not necessarily as being cleverer but as doing fewer stupid things.

Case 32-3 What an Open Vocabulary Costs

An open-vocabulary map sounds nearly all-powerful: no need to define categories beforehand, and the user describes in words whatever he wishes to find. But its costs

are quite concrete, and worth students knowing in advance.

The first is memory. A classical semantic map stores one category number for each element (a few bytes); an open-vocabulary map must store a feature vector from a vision-language model—commonly several hundred floating-point dimensions, which is to say a thousand bytes and more per element. For the same map the memory used may differ by two or three orders of magnitude, which is precisely the difficulty of edge deployment that Chapter 33 takes up.

The second is reliability. A model with fixed categories has at least clear measures of assessment and clear boundaries of failure; an open-vocabulary query may give wholly different results for two similar descriptions, and there is no good way of knowing in advance which queries will fail. For a navigation task that needs a guarantee of safety, this is a serious problem.

The practical course at present is a mixture: categories critical to safety (people, staircases, forbidden zones) still use a model with fixed categories, with explicit assessment and guarantees; the open vocabulary is used for interacting with the user and for queries—occasions where being wrong does nothing worse. Students may apply this principle of the division of labor to a project directly.

Implementation Activities

Implementation Activities

Implementation 32-1 A Semantic Cost Map and the Removal of Dynamic Objects

Purpose: to fit semantic information into the map and the planner with one's own

hands, to verify how it changes a decision, and to implement the multi-pass consistency removal of dynamic objects. The implementation is chiefly in Python; parts one and two need no robot hardware.

Part One: the Semantic Cost Map (for everyone)

1. Take an occupancy grid map (the product of the implementation of Chapter 13 will serve), produce a geometric cost map from it by a distance transform, plan a path from a start to a goal with A*, and record its length.
2. Mark two or three semantic regions on the map as rectangles (a forbidden zone, a busy thoroughfare, a preferred route), and assign each class a cost and a decay distance.
3. Add the semantic cost on top of the geometric cost map, plan again, and compare the lengths of the two paths and the distance each runs through a semantic region, reproducing the three numbers of Figure 32-3.
4. An experiment in sensitivity: raise the cost of the busy thoroughfare from zero by degrees, find the threshold at which the path jumps to the other route, and explain what that threshold means.

Part Two: Removing Dynamic Objects (for everyone)

1. Place several people at random in a simulated environment, run one pass of scanning and mapping, and repeat for five passes.
2. Build the map by naive accumulation (recorded as an obstacle if observed on any pass), compare it with the true map, compute the area of the false obstacles, and reproduce Figure 32-2(a).
3. Change to voting for multi-pass consistency (recorded as an obstacle only if observed on at least k passes), sweep k from 1 to 5, draw the two curves of area

of false obstacles and proportion of the real wall kept, and find the best k.

4. For discussion: if the number of passes rose from 5 to 20, how would the best k change? What does that suggest for the strategy by which an inspection robot (Chapter 19) maintains its map?

Part Three: Projecting the Semantic Label (university level)

1. With an RGB-D camera or with simulated data, run an off-the-shelf object detection model on one image and obtain the detection box and the depth.
2. Project the detection into the map frame by Formula Two, and draw the labeled point on the map.
3. Alter the rotation of the extrinsics deliberately by 1, 2 and 5 degrees, record how far the labeled point moves, and so quantify the effect of an extrinsic error, reproducing the phenomenon of Case 32-1.
4. (Going further) Detect and project the same object from five different viewpoints, and compute the spread of the labeled points as a combined measure of the quality of the whole chain of propagation.

Discussion Questions

1. How much farther does the semantic path travel than the geometric one? What does that cost buy? If the extra distance were 50 percent rather than 2, would you still choose the semantic path?
2. What is the best value of k for multi-pass consistency voting? What happens when k is too small, and when it is too large?
3. With the rotation of the extrinsics 2 degrees out, how far does a labeled point ten meters away move? How does that figure compare with the centimeter-level localization accuracy required in Section 19.5?

4. Which categories suit a model with a fixed list, and which an open vocabulary? Classify them by the consequence of being wrong.
5. (University level) If you were to design a semantic map for a campus tour robot, which four semantic layers would you put in it? How would you set the cost and the decay distance of each?

Extended Challenge (university level)

The first: implement a semantic layer editor—let the user draw regions on the map, choose a class, adjust the cost, and see at once how the planned path changes. A tool of this kind is standard equipment in inspection and service robot products. The second: raise the removal of dynamic objects from voting to a layered map: maintain two maps, a permanent layer and a transient one, localize against the permanent layer alone and avoid obstacles with both, and compare its rate of successful localization with that of plain voting (the method of assessment of Section 19.6 may be reused).

Further Reading

1. N. Yokoyama et al., "VLFM: Vision-Language Frontier Maps for Zero-Shot Semantic Navigation," ICRA, 2024. (The representative work on zero-shot semantic navigation, referred to in Chapters 24 and 32 alike.)
2. T. B. Martins, M. R. Oswald, and J. Civera, "Open-Vocabulary Online Semantic Mapping for SLAM," IEEE RA-L, 2025. (Open-vocabulary semantic mapping, the frontier direction of Section 32.3.)
3. D. Shah et al., "LM-Nav: Robotic Navigation with Large Pre-trained Models of Language, Vision, and Action," CoRL, 2022. (The pioneering work on navigation driven by a language model.)

4. Embodied AI with Foundation Models for Mobile Service Robots: A Systematic Review, arXiv, 2025. (A complete survey of embodied intelligence and foundation models, suitable as a starting point for research.)
5. Ultralytics YOLO official documentation (docs.ultralytics.com). (The training, assessment and deployment of object detection models, the chief tool of this chapter's implementation.)
6. The awesome-and-novel-works-in-slam open-source list (GitHub). (A continuously updated collection of new directions: semantic SLAM, three-dimensional Gaussians, navigation, large language models and more.)
7. NVIDIA Jetson developer documentation and the TensorRT optimization guide. (A practical reference for deploying AI at the edge, the technical foundation of Section 33.8.)

Summary

Summary

- Classical SLAM knows shape and not meaning; robot intelligence may be divided into three levels—geometric, semantic and task—which are laid one upon another and do not replace one another: without a solid geometric level beneath it, the semantic level only puts its labels on crooked.
- AI vision models divide five ways within SLAM: detection, segmentation, depth, description and scene understanding; the crucial constraint on the choice is the budget of computation and memory on an edge platform.
- A semantic map divides into four levels—geometric, object, region and building—built from the bottom upward; an open vocabulary frees the categories from having to be defined in advance, and a scene graph makes

spatial relations something that can be queried and reasoned over.

- A dynamic object written straight into the map leaves a ghost behind. The experiment of this chapter shows naive accumulation producing 1.7 square meters of false obstacles, while voting for multi-pass consistency brings that to zero and keeps 100 percent of the real wall.
- Dynamic objects are handled by three lines of defense—instance segmentation in real time, inconsistency between frames in the short term, and voting across passes in the long term—which work at different scales of time and are used together in practice rather than one instead of another.
- Semantics enter navigation through the cost map. The experiment of this chapter shows the semantic path travelling only 0.7 meters farther (34.5 → 35.2 meters) while the 13 meters of passage through a forbidden zone fall to nothing; hard constraints and soft costs should be designed separately, and the decay distance affects behavior more than the numerical cost does.
- The quality of a semantic label depends at once on the rate of recognition and on the chain of coordinate transforms; two or three degrees of error in the extrinsics are enough to cause an offset of tens of centimeters, and in debugging the geometric chain should be verified before the model is examined.
- International research runs along three lines: metric-semantic SLAM is the steadiest, the open vocabulary the most flexible, and neural fields and renderable representations give the best reconstruction at the highest cost.

Carrying Forward

This chapter has carried the map from shape to meaning. Figure 32-1 set out the four levels of a semantic map, Figure 32-2 showed what ought not to go into a map, and

Figure 32-3 demonstrated how semantics change a decision. But all of this presupposes one thing: that a good enough AI model is already in your hands.

Where does the model come from? Real data are expensive and labeling takes time, and the cost of a robot's trial and error in the real world is higher still. In the next chapter we enter simulation: Chapter 33 introduces how to train a robot on a large scale in a simulated environment and then deploy the learned policy on real edge hardware—a route of simulate first, then deploy, which is by now the standard procedure from national projects to volume production in industry.

(The chapter ends here; the next enters training in simulation and deployment at the edge.)

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 33

Training in Simulation and Sim-to-Real

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 32 gave the map a meaning—the robot knows that the thing is a door. But what after it knows? To have the robot actually walk over, push the door open, and stand firm on uneven ground, there must be a control policy behind it, and such a policy can hardly be learned from nothing on a real robot. The reason is simple: reinforcement learning needs tens of millions of attempts, and a real robot can neither afford that many falls nor spare that much time.

What this chapter takes up is exactly that contradiction: practice in simulation first, then carry what has been learned over to the real robot. The route is called sim-to-real, and since 2020 it has become all but the standard practice for legged robots and dexterous manipulation. This chapter follows NVIDIA's Isaac Sim and Isaac Lab ecosystem as its main line, because it is at present the most complete set of tools and the easiest to obtain; but the ideas this chapter is about—why parallel simulation works, where the gap between simulation and reality comes from, and how to close it—do not change with the tool.

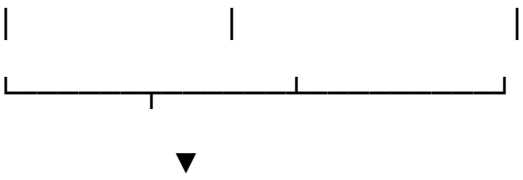
The relation of this chapter to the earlier parts is direct: the coordinates and probability of Chapters 10 and 11 are the language of a simulator, the sensor models of Chapters 8 and 9 decide what the simulator must imitate, and the edge computing of Chapter 25 is the hardware foundation of Section 33.8, on how to deploy what has been trained. This, one might say, is the chapter that carries the techniques of the whole book from running to learning.

Figure 33-0 The Link Map of Chapter 33

The foundations the earlier parts supply

Chapters 8–9 Sensors Chapters 10–11 Coordinates and Probability Chapter 25 Edge

Computing



【Chapter 33 Train in Simulation, Then Carry It to a Real Robot】

Parallel simulation ─ Sensor simulation ─ Synthetic data ─ Domain randomization ─ Edge deployment

▼

Chapter 34 VLMs, Language Navigation and Embodied Intelligence (the close of the book)

Learning Objectives

This chapter starts from why simulate first and takes up in turn the principle of parallel simulation, the simulation of sensors, the generation of synthetic data, the sources of and remedies for the sim-to-real gap, and finally deployment on an edge device. It has three figures computed from experiments, each answering one question: what parallelism really buys, where simulation and reality differ, and what domain randomization costs. By the end of this chapter students should be able to:

1. Explain why a reinforcement-learned robot policy must nearly always be trained in simulation first, and estimate the number of interactions needed.
2. Explain the division of labor between Isaac Sim and Isaac Lab, and the difference between GPU-native simulation and a classical simulator.
3. Explain how the number of parallel environments affects throughput, gradient noise and the stability of training all at once, and know the limit of each.
4. Explain how the sensors a robot commonly uses are imitated in simulation, and which parts are imitated well and which still fall short.

5. Explain how synthetic data are generated and a real scene reconstructed, and plan the sources of data for one's own project.
6. Point out the chief sources of the sim-to-real gap, and distinguish when system identification and when domain randomization is the remedy to use.
7. Explain what domain randomization costs: the trade-off between robustness and nominal performance.
8. Plan a complete route from training in the cloud to deployment on a Jetson, and estimate the memory needed after quantization.

Core Concepts

Term	Meaning	Related Sections
Sim-to-Real	Training a policy in simulation and then deploying it on a real robot	33.1, 33.6
Parallel simulation	Advancing thousands of environments at once on one GPU	33.3
GPU-native	The state living in GPU memory as tensors, with no round trip to the CPU	33.3
OpenUSD	An open scene format for describing scenes and robots	33.2
Domain randomization	Sampling environment parameters at random	33.6

	during training, buying robustness in deployment	
System identification	Measuring a real robot and working back to the simulation's parameters	33.6, Chapter 11
Synthetic data	Training data produced and labeled automatically by a simulator	33.5, Chapter 32
Domain gap	The loss of performance caused by a training distribution that does not match the deployment one	33.5, 33.6
Quantization	Representing a model's parameters in fewer bits to save memory	33.8
Distillation	Transferring the ability of a large model to a small one that can run at the edge	33.8

Table 33-1 The Core Terms of This Chapter

33.1 Why a Robot Must Learn in Simulation First

Let us state the scale of the problem plainly first.

One: the Problem of the Numbers

To train a walking policy for a legged robot by reinforcement learning, the typical requirement in samples is of the order of a billion (10^9) steps of interaction with the environment. Suppose the control frequency is 50 Hz: a billion steps come to 20 million

seconds, which is about 231 days of continuous running. And that is under the ideal assumption that the robot will not break, the battery needs no charging, and it picks itself up after every fall.

The real world has three further costs that simulation does not: the hardware breaks (one fall may damage a gearbox or a shell), a person must watch over it throughout (it cannot run unattended), and an experiment cannot be repeated (today's floor is not as damp as yesterday's, so two algorithms cannot be compared cleanly). These three together make reinforcement learning on a real machine, in the great majority of cases, not a question of cost but something fundamentally impracticable.

Two: What Simulation Offers

- Time can be sped up: simulation is not bound by real time, and one second can advance thousands of environments by tens of steps.
- Failure costs nothing: a fall merely resets the state, and breaks no part.
- The state is wholly known: the simulator knows the robot's true pose and velocity, which makes asymmetric training possible—the critic network may see the true state while the policy network sees only sensor data.
- Experiments can be repeated: fix the random seed and the same experiment can be run a hundred times, which is the premise of comparing algorithms (the methods of assessment of Chapter 31).
- Environments can be generated at will: terrain, lighting and the arrangement of objects can all be produced programmatically, a variety the real world cannot offer.

Three: What Simulation Does Not Offer

The limits of simulation must be stated just as honestly. A simulator solves the

equations you wrote into it, not the physics of the real world. Whatever was not written into those equations—the motor's rise in temperature, the backlash of the gears, the tug of the cabling, the change in the coefficient of friction once the foot pads have worn, those compensations the engineers added in the firmware to make it pleasant to use—the simulator knows nothing of. These unmodeled dynamics are the chief source of the sim-to-real gap, which Section 33.6 takes up one by one.

> **Why SLAM Took Up Training in Simulation Later than Control Did**
>
> Most of the SLAM algorithms of the first thirty-two chapters of this book need no training—they are derived, not learned. This is the fundamental difference between SLAM and a control policy: SLAM has an explicit probabilistic model and objective of optimization (Chapters 11 and 12), while nobody can write a closed-form solution for how to walk steadily on gravel.
>
> But that line is blurring. Deep-learned visual odometry, learned loop closure descriptors, and the semantic segmentation of Chapter 32 all need a great deal of data; and the generation of synthetic data in Section 33.5 is exactly the route by which SLAM too begins to benefit from simulation.

33.2 What Isaac Sim and Isaac Lab Are

The two names are often used interchangeably, though the division of labor between them is in fact quite clear.

One: the Division of Labor Between the Two Layers

	Isaac Sim	Isaac Lab
What it is	The simulator and the	A framework for robot

	scene editing environment	learning
What it does	Physics solution, sensor rendering, importing scenes and robots	Defining tasks, observations and rewards, the training procedure
Who uses it	Those making scenes, attaching ROS 2, or generating data	Those training policies
What it rests on	PhysX physics + RTX rendering + OpenUSD scenes	Built upon Isaac Sim
An analogy	The laboratory and its instruments	The lab manual and the marking sheet

Table 33-2 The Division of Labor Between Isaac Sim and Isaac Lab

Put more plainly: Isaac Sim is the world and Isaac Lab is the course. You may use Isaac Sim alone to make a scene, put a robot in it, and attach ROS 2 to test your navigation stack, without touching reinforcement learning at all; or use Isaac Lab to draw directly on the dozens of training environments built into it, without having to build a scene from nothing.

Two: the Three Pillars

- PhysX: the physics solver, which handles rigid bodies, joints, contact and friction. What matters most for a robot is the quality of the contact solution—a foot on the ground and a gripper on an object both depend on it.
- RTX: ray-traced rendering, which produces the camera images. It is at the same time the foundation of simulating LiDAR and depth cameras (Section 33.4).
- OpenUSD: an open format for describing a scene, proposed by Pixar and by

now the standard for exchanging scenes between tools. Its value lies in one scene shared by several tools, and it also lets a robot model (links, joints, actuators, sensors) be described in a standard structure.

Three: the Important Changes of 2025

At SIGGRAPH on 11 August 2025, Isaac Sim 5.0 and Isaac Lab 2.2 became generally available, and the extensions of Isaac Sim were open-sourced on GitHub at the same time (isaac-sim/IsaacSim and isaac-sim/IsaacLab). For teaching and research this is a considerable matter: a simulator that could formerly be used only as a black box can now be read, altered, and submitted to.

One point must be made clear, lest students miss a step: the Omniverse Kit beneath it is not wholly open-sourced, and at present takes no outside submissions. Which is to say that you may modify Isaac Sim's extension layer and not the core at the very bottom. For a course project this is not usually a restriction, but it is worth knowing in advance when planning research for the long term.

The same release also brought several features that bear directly on robots: a robot structure description and an import wizard in OpenUSD, which makes a URDF conversion harder to get wrong; a model of joint friction, which lets the real resistance of a gearbox be written into the simulation; and support for ROS 2 Jazzy together with a standardized simulation interface, so that one set of ROS 2 nodes can be switched between different simulators (Part VIII).

What Isaac Lab 2.2 Brings

A cloud training environment that starts at one click (a Brev launchable), with no CUDA or drivers to install oneself.

The two-armed training environments of GR00T-Mimic, demonstrating learning manipulation from human demonstration data.

The integration of Omniverse Fabric, which lowers the memory and synchronization overhead of large scenes.

A tensorized suction gripper model, so that vacuum picking tasks may be simulated in parallel.

An open-source framework for evaluating policies, giving a trained policy a standard way of being compared (echoing Chapter 31).

33.3 GPU-Native Parallel Simulation

This is the technical heart of the whole ecosystem, and the question the first experimental figure of this chapter answers.

One: What GPU-Native Means

The procedure of a classical simulator (Gazebo, say) is: the CPU computes the physics → the state is copied to the learning program → the learning program computes an action → it is copied back to the simulator. Every step moves data between CPU and GPU, and once the environments are many the time spent moving data exceeds the time spent computing physics.

The GPU-native way is to keep the state of every environment as tensors directly in GPU memory, and to carry out the physics solution, the computation of observations, the computation of rewards and the inference of the neural network all on the GPU, so that the whole training loop never moves data back to the CPU. The difference cannot be seen when the environments are few, and at thousands of environments it is a

difference of orders of magnitude.

Two: What Parallelism Really Buys

Anyone can say that opening more environments makes it faster, but faster in what, faster by how much, and at what point it stops getting faster are three different questions. The experiment below, which can be reproduced in full, separates the three.

The experimental setting: a joint-tracking task with one degree of freedom. The joint has four parameters—rotational inertia, viscous damping, Coulomb friction and actuation delay; the controller is a set of PD gains with friction feedforward, which is to say three parameters to be learned. The training method is the plainest random hill climbing, each iteration evaluating the candidate parameters on N randomly sampled environments. The task is far simpler than real legged locomotion, and yet all three effects of parallelism show themselves fully here.

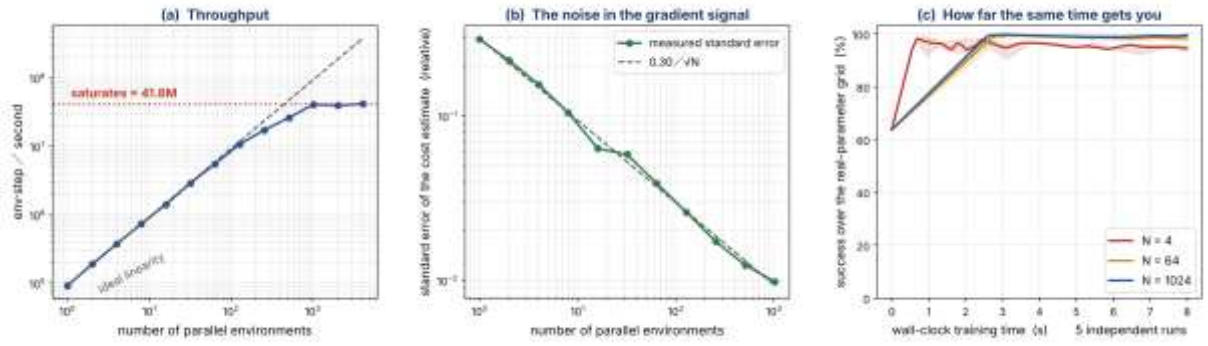


Figure 33-1 The Three Effects of the Number of Parallel Environments: Throughput, Gradient Noise, and the Stability of Training

Take (a) first. The horizontal axis is the number of environments N advanced together and the vertical axis the environment steps advanced per second, both on a logarithmic scale. The dashed line is ideal linearity—if parallelism were wholly free, doubling the environments should double the throughput. The measurement follows that line almost exactly while N is below about 32: from 93 thousand steps per second at N = 1

all the way to 2.91 million at N = 32, thirty-two times the environments bought with thirty-one times the throughput. The stretch is free because a single environment cannot fill the GPU at all, and the added environments merely put idle computing units to work.

Beyond N = 32 the curve begins to leave the dashed line, and by N = 4096 the throughput halts at about 41.8 million steps per second. Against N = 1 that is 452 times, and against the 379 million of ideal linearity only a ninth. The reason for the saturation is that the GPU's computing units are full already, and further environments can only queue. It is worth noting that from N = 1 to N = 4096 the wall-clock time of each batch rises only from 0.004 seconds to 0.039: the environments increase by a factor of 4,096 and the waiting by a factor of 9.8. Which is precisely the most immediate value of parallel simulation—samples had very nearly for nothing.

Panel (b) answers the second question: what those samples are for. Every update in reinforcement learning must estimate how good this set of parameters is, and that estimate, computed from the average over a finite number of environments, necessarily carries noise. The vertical axis is the standard error of that estimate, and the measured points fall almost perfectly on the dashed line of $1/\sqrt{N}$—multiply each point's standard error by $\sqrt{N}$ and the eleven results all lie between 0.25 and 0.33. From N = 1 to N = 1024 the standard error falls from 0.293 to 0.0098, close to the factor of $\sqrt{1024} = 32$. This is Formula One.

Panel (c) puts the first two together: given the same budget of wall-clock time (8 seconds), how far different degrees of parallelism get. To keep the luck of a single run out of it, each setting is trained five times independently; the heavy line in the figure is the median and the pale band the interquartile range. The median of N = 4 reaches

only 94.6 percent and goes no further, with an interquartile range of 93.3 to 95.8. The median of N = 64 is 98.3 percent and of N = 1024 is 99.6, the latter with an interquartile range of only 98.8 to 99.6.

What panel (c) is meant to convey is not that high parallelism is faster but that high parallelism is more reliable. N = 4 does not walk slowly; it walks erratically—the gradient noise is too large, many a step goes in the wrong direction, and the last few percent are never closed. The phenomenon is worse in real reinforcement learning, and it is why a framework such as Isaac Lab treats being able to open thousands of environments at once as a central measure.

An Honest Caveat

The three figures above were computed on a toy task with one degree of freedom, not on a legged robot. The absolute numbers of a real task will be wholly different: the point of saturation depends on your GPU and the complexity of the environment, and the number of environments needed depends on the dimension of the action and the sparsity of the reward.

But the three qualitative conclusions are general: throughput grows linearly and then saturates, gradient noise falls as $1/\sqrt{N}$, and with too little parallelism the variation in the outcome of training is too large to compare. Drawing the same three figures for your own task is the most effective way of planning computing resources.

Three: the Numbers at Real Scale

For comparison, look at figures at industrial scale. The Isaac Lab technical report (November 2025), on a configuration of 8 GPUs with 16,384 environments each, reports

the teacher policy of a dexterous-hand grasping task reaching more than 900 thousand steps per second, and a Franka arm cabinet-opening task more than 1.6 million, with near-linear scaling across several machines and cards.

Another figure more practical for students: on the cabinet-opening task the throughput of one consumer-grade GeForce 5090 is about equal to two professional cards, and on the dexterous-hand task it is even about 25 percent faster, because the bottleneck of that task is the CPU and not the GPU. Which means that a desktop machine with a single high-end graphics card is enough to support quite serious research in robot learning—something unimaginable five years ago.

Four: Formula One—Parallelism and the Quality of an Estimate

Let the estimate of the return from a single environment be a random variable with standard deviation σ; taking the average over N independent environments as the estimate, the standard error of the estimator is:

$$SE(N) = \sigma / \sqrt{N}$$

To lower the noise of the estimate to a tenth requires a hundred times the environments. This penalty of the square root explains why parallelism is counted in thousands: raising the environments from 64 to 4,096 lowers the noise only to an eighth. It explains at the same time why the saturation of panel (a) is acceptable—even with the throughput down to a sixth of the ideal, the added environments are still lowering the noise.

33.4 Simulating Sensors: from RTX LiDAR to Touch

For the reader of this book this section is probably the closest to home: what does every sensor used in the past thirty-two chapters look like in simulation?

One: the Four Families of Sensor

Family	How it is implemented in simulation	Corresponding chapters	Quality of the imitation
Proprioception	Read directly from the physical state	Chapters 7, 10	High
Camera	RTX ray-traced rendering	Chapters 9, 20–23	Fairly high
Ranging	Warp ray casting	Chapters 8, 17–18	Medium
Contact and touch	The contact solver + a vision-based tactile model	Chapter 19	Fairly low

Table 33-3 The Four Families of Sensor in Simulation

Two: One by One

Proprioception is the easiest and is imitated the best. The IMU, the joint encoders, contact detection and the frame transformer can all be read straight off the physical state, with bias and noise added by the error model of Chapter 7. But note: what the simulator gives is a clean truth plus the noise you wrote, while a real IMU's error has temperature drift and vibration coupling in it, and is not independent identically distributed Gaussian noise.

The camera is where RTX rendering is at home. Besides RGB, depth, semantic segmentation, instance segmentation, surface normals and motion vectors all come free—data that would need extra labeling in the real world are a by-product of the rendering pipeline in simulation. This is precisely the technical foundation of the synthetic data of Section 33.5. The lens model supports both pinhole and fisheye, the

latter mattering for the wide-angle visual navigation mentioned in Chapter 9.

Ranging sensors (LiDAR, and the stereo matching of a depth camera) use Warp ray casting: rays are cast from the sensor origin in the specified scan pattern and their intersections with the scene computed. This handles occlusion and geometry correctly and can imitate the different beam arrangements of various LiDARs; the terrain height scan that legged robots commonly use is implemented by the same mechanism. But two things it does not easily get right: the specular reflection and transmission of Chapter 8 (glass, water, a highly reflective sign), and the motion distortion of Chapter 18—the latter needing the simulator to keep updating the sensor pose during the scan, rather than casting every ray at one instant.

Contact and touch are at present the class with the largest gap. The contact force itself is solved by PhysX, but real contact involves the microscopic deformation of the material, the roughness of the surface and viscous effects, which a rigid-body solver can only approximate by a coefficient of friction. The simulation of vision-based tactile sensors (which infer contact force from a camera's view of the deformation of an elastic membrane) is usable by now, but if a student's project is to do fine insertion or assembly, the sim-to-real gap of that class of task must be expected to be the largest.

A Detail Often Overlooked: the Timing of the Sensors

Real sensors do not trigger together. A LiDAR takes 100 milliseconds to turn once, a camera's exposure a few milliseconds, the IMU runs at 200 Hz and the wheel encoders at 50—all discussed in Chapters 7 and 18.

A simulator by default often treats every sensor as sampling at the same instant. If your algorithm is sensitive to time synchronization (a tightly coupled

LiDAR-inertial odometry, say), then fine in simulation and drifting on the real machine is quite likely not a fault of the algorithm but of a simulation that did not build the time differences in.

The remedy is to set the update rate and the phase of every sensor explicitly in simulation, and to add random jitter in time deliberately.

33.5 Generating Synthetic Data and Reconstructing a Real Scene

Chapter 32 discussed the necessity of collecting one's own data, and the cost of labeling. This section offers another road: let the simulator produce the data and the labels together.

One: the Temptation of Synthetic Data, and Its Trap

The temptation is plain: in simulation it is known which object every pixel belongs to, so that segmentation labels, bounding boxes, depth maps and six-degree-of-freedom poses are all free and without error. Ten thousand training images under different lighting, from different viewpoints, with objects differently arranged, are a script and a few hours of computation away.

The trap is just as plain: a simulated image is not the same as a real one. The reflectance of materials, the noise and compression artifacts of a sensor, the dirt and color cast of a lens, the clutter in a real scene that nobody would trouble to model—these differences together are called the domain gap. A model trained on synthetic data alone will often do poorly on real images.

Two: Three Roads to Narrowing the Domain Gap

1. Reconstructing the real scene: do not model it yourself, but scan the real scene in. The present practice is to reconstruct an interactive three-dimensional scene from real video (the NuRec library and the open-source 3DGUT method, for instance), so that the background and the materials are real to begin with and only the dynamic objects are synthetic.
2. Visual domain translation: use a generative model to translate a simulated image into one that looks as though it were really photographed. Isaac Sim's data writer can already be connected directly to models of this kind (Cosmos Transfer, say).
3. Visual domain randomization: the reverse—do not imitate reality, but randomize the simulated materials, lighting and background to a degree so extreme and exaggerated that the model has no choice but to rely on the shape of the object rather than its appearance. This road is the simplest and the most used.

Three: Mixing Synthetic and Real

What works best in practice is usually not one or the other but a mixture: pre-train on a great deal of synthetic data and fine-tune on a little real data. This echoes a point made in Chapter 32—fine-tuning from pre-trained weights needs far less real data than training from nothing. A practical recipe for a campus project is: several thousand synthetic images for pre-training, 200 to 500 real images per category for fine-tuning, and a test set made of real data only.

Figure 33-2 The Iterative Flow of Data from Collection to Deployment

① Define the categories and plan the collection

↓

② Obtain the data (really photographed / generated in simulation / a mixture of both)

↓

③ Label (set a standard for the real data and spot-check it; synthetic data are labeled automatically)

↓

④ Split the dataset (split by place rather than at random; the test set is real data only)

↓

⑤ Train (pre-train on synthetic data → fine-tune on real data)

↓

⑥ Assess (compute the accuracy on the real test set; and test on the real site)

↓

⑦ Analyze the errors (which categories go wrong most often? under what conditions?)

↓

⑧ Add data aimed at the weak points → return to ⑤ and iterate

↓

⑨ Optimize for deployment (quantize, accelerate) → the edge device (Section 33.8)

Note the seventh and eighth steps particularly: analyzing the errors and adding data aimed at them is often far more effective than blindly adding training epochs. Synthetic data are especially valuable at this step—when you find the model goes wrong most often on a staircase against the light, you can generate a thousand backlit staircases in simulation on purpose, which in the real world would mean waiting on the weather.

Two Tools Worth Knowing

MobilityGen: generates trajectories for a mobile robot and the corresponding

sensor data automatically, suited to producing datasets for training or assessing SLAM and navigation.

Describing a scene automatically with a vision-language model: once the simulator has rendered an image, a VLM produces a textual description as the label—which joins the semantic labeling of Chapter 32 to the synthetic data of this chapter, and is one of the chief sources of the language navigation data of Chapter 34.

33.6 Domain Randomization and the Sim-to-Real Gap

This is the most important section of the chapter. The five sections before it were about how to train in simulation; this one is about why it fails when carried to a real machine once trained, and what to do about it.

One: Where the Gap Comes From

Source of the gap	What it consists of	Where it appears in this book
Actuator dynamics	The virtual inertia created by firmware compensation, the bandwidth of the current loop, torque saturation	This section
Error in link inertia	The CAD model underestimates the real inertia (cabling, shells, sensors)	This section

Friction and damping	The Coulomb friction of a gearbox and damping that changes with the state	This section
Time delay	The total delay caused by command transmission, filtering and the control period	Chapter 25
Joint offset	The zero offset caused by errors of assembly and calibration	Chapter 10
Sensor characteristics	The noise model, the sampling time, the field of view and occlusion	Chapters 7–9
Differences in the environment	Ground friction, terrain, load, temperature and ageing	Chapter 19

Table 33-4 Seven Sources of the Sim-to-Real Gap

Two: the First Remedy—System Identification

The first road is to make the simulation right: measure the real robot's response and work back to the parameters the simulation ought to use. The standard practice is to hang the robot in the air (so that the feet touch nothing and contact forces do not interfere), send each joint a frequency sweep (a sine wave whose frequency rises continuously from low to high), record the encoders' response, and work back to the parameters by optimization. Recent research notes that about 20 seconds of sweep data in the air per robot is enough, with about four parameters per joint to be identified (armature inertia, viscous damping, Coulomb friction, joint offset) plus one global

command delay.

The procedure is demonstrated below with an experiment that can be reproduced in full. The real robot is a joint model with Coulomb friction and a 7.5-millisecond command delay; the URDF model has only the inertia of the links—no damping, no friction, no delay—which is precisely the typical state of a URDF exported straight from CAD.

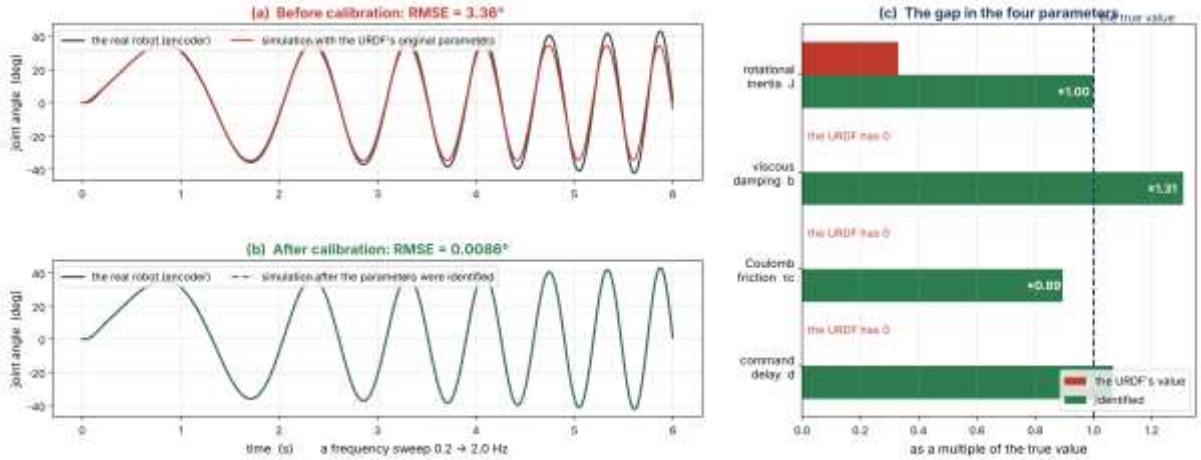


Figure 33-3 System Identification: from the Guess in the URDF to the Fitted Simulation

Panel (a) is before calibration. The grey line is the angle recorded by the encoder of the real robot, and the red the same motion simulated with the URDF parameters. The two curves are alike in shape, but the red plainly leads—the consequence of the delay not being modeled; its amplitude is slightly larger too, because nothing in the simulation is consuming energy by friction and damping. The root-mean-square error over the whole stretch is 3.36 degrees. For a walking policy, a systematic bias of three degrees and more at every joint is enough to throw the gait entirely out of shape.

Panel (b) is after calibration. The green dashed line lies exactly over the grey, and the root-mean-square error falls to 0.0086 degrees, a four-hundredth of what it was. It must be stressed that the model was not made more complicated: the parameters of

the same model were set right, and the form of the equations did not change from beginning to end.

Panel (c) shows how far the parameters were out. The red bars are the URDF's original values as a multiple of the true ones and the green bars the identified results. In rotational inertia the URDF holds only 0.33 of the true value—the missing two thirds coming from the virtual inertia created by the firmware's compensation and from cabling and shells that the CAD did not count. Viscous damping, Coulomb friction and command delay are simply zero in the URDF. After identification the inertia is restored exactly, to 1.00 of the true value; the delay, searched on a grid at intervals of 2 milliseconds, lands at 8.0 milliseconds against a truth of 7.5, and damping and friction come out at 1.31 and 0.89 times the truth—they have absorbed between them the half grid point by which the delay is out. Which is the compensation among parameters that the next subsection warns of, seen here in miniature.

A Pitfall in Implementation: the Delay Cannot Be Fitted by a Gradient Method

In a discrete-time simulation the command delay is an integer number of time steps, so the residual is a step function of the delay—and within each step the gradient is zero. Throw the delay into a gradient method such as least_squares and it will sit at its initial value without moving.

The right course is to search the delay on an integer grid in the outer loop and fit the continuous parameters by a gradient method in the inner one. The first time the experiment of this section was done wrongly, the delay fitted came out at 0.0001 seconds (the truth being 0.0075), and the inertia was overestimated by a factor of 2.3 to make up for the delay that had not been

identified—the trajectories looked right and the parameters were all wrong. This is the most characteristic trap of system identification: a small error does not mean the parameters are right, and different parameters may compensate for one another.

Three: the Second Remedy—Domain Randomization

The second road is the reverse: do not pursue a correct simulation, but make the policy immune to the uncertainty of the parameters itself. The method is to draw the dynamic parameters of every environment at random from a distribution during training, forcing the policy to learn a behavior that works for any reasonable parameters.

The question is: how wide should the randomization be? The experiment below compares three settings. On the same joint-tracking task, three controllers are trained: (a) on the nominal parameters alone; (b) randomized within ±10 percent of the nominal values; and (c) randomized over a wide range—inertia from 0.5 to 2.0 times, friction from 0.3 to 2.0 times, delay from 0 to 28 milliseconds. The three are then assessed on the same grid of real parameters.

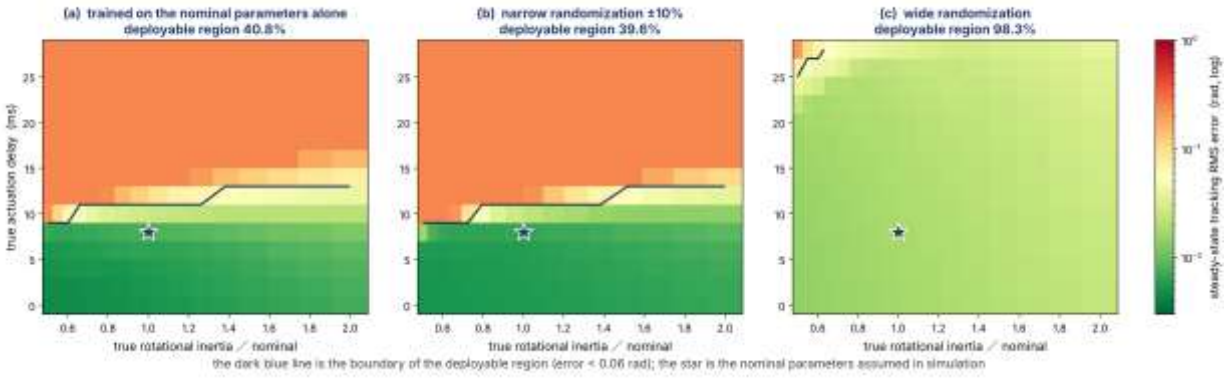


Figure 33-4 How the Range of Domain Randomization Affects the Rate of Successful Deployment

Take the plainest conclusion first: the deployable region (the area enclosed by the dark

blue line) leaps from 40.8 percent in (a) and 39.6 in (b) to 98.3 in (c). Narrow domain randomization is of almost no help—a range of ±10 percent is too narrow, and the policy learned is much the same as one trained at the nominal point alone. The point is worth students remembering: domain randomization is not a matter of adding a little randomness, and the range must be wide enough to cover the uncertainty you really have about the real parameters.

The second conclusion is hidden in the colors. Within the deployable region (a) and (b) are deep green, meaning a very small tracking error; (c) is pale green all over, meaning it works everywhere and is nowhere good enough. Quantitatively: at the nominal parameters the controller trained on the nominal parameters alone has an error of 0.0069 radians and the widely randomized one 0.0179—two and a half times as much. This is what domain randomization costs: robustness is not free, and it is bought with nominal performance.

The third conclusion is the shape of the boundary of failure. The boundary in (a) and (b) is nearly a horizontal line, falling at a delay of between 9 and 13 milliseconds, and rising only slightly with the rotational inertia. Which is to say that in this task being out by a factor of two in inertia will not make the policy fail, while a few milliseconds more of delay will. The message is of practical use in planning a project: rather than spend effort weighing every link, measure the delay of the control loop first.

Four: How to Choose Between the Two Roads

	System identification	Domain randomization
The method	Measure the real machine and make the simulation	Leave the simulation alone and train the policy to take

	right	punishment
What it needs	A real machine that can be measured, and equipment to excite it	An estimate of the range of uncertainty in the parameters
Its merits	High nominal performance, and explicable	No real machine needed in order to train
Its faults	The parameters may compensate for one another; it fails as parts wear	Nominal performance falls; the range is hard to set
It suits	Having a real machine already, and wringing the best performance out of it	The real machine not yet in hand, or a fleet whose machines differ greatly

Table 33-5 System Identification Compared with Domain Randomization

In practice the two are usually used together: calibrate the center of the simulation to the right place by system identification first, then randomize moderately about that correct center to cover wear, temperature and the differences between individual machines. This neither sacrifices nominal performance as pure domain randomization does, nor fails once a part has worn as pure system identification does.

33.7 How to Begin, and How to Join the Community

This section is very practical: what a student who has read the six sections above and wants to begin should do next.

One: Three Routes In

Route	What it needs	Whom it suits
A one-click start in the cloud	A browser and an account	Students with no graphics card of their own
Deployment in a container	A Linux host with CUDA + a container environment	A laboratory with a workstation
Installing from source	The same, and handling the package dependencies oneself	Those who are to modify the simulator

Table 33-6 Three Routes In

For most students, beginning from a one-click start in the cloud is advised. It settles the driver, the CUDA version and the package dependencies—the things most apt to hold one up—so that you can see a robot moving in simulation on the first day instead of spending three days installing an environment. Once the procedure is familiar and you are sure of committing to it for the long term, move to a local installation.

One detail is worth knowing before a local installation: the simulator's display is streamed out over WebRTC, using ports 49100 and 47998 by default. If you cannot reach the display on a school network, nine times in ten the firewall has blocked those two ports and the software is not broken.

Two: A Suggested Order of Learning

1. Get the built-in example running first: change nothing, and confirm that your environment can train a robot that walks. The point of this step is only to establish a baseline.
2. Change the robot and not the task: replace the robot in the example with another built-in model, and observe how the same rewards perform on a

different morphology. The framework has about eleven legged morphologies built in, enough to compare for a long while.

3. Change the rewards: this is the step from which most is learned. Raise the weight on velocity tracking, raise the penalty on energy, and observe how the gait changes. You will soon feel that designing rewards is a craft.

4. Import your own robot: convert your own URDF into a form the simulation can use with the import wizard, at which point the real problems of joint limits, inertia and friction settings begin.

5. Attach ROS 2: let the simulator serve as the test bench of your navigation stack (Part VIII). This step needs no reinforcement learning, and is of the greatest value to students working on SLAM.

6. Do system identification and sim-to-real: once a real machine is at hand, go once through the procedure of Section 33.6.

Three: How to Join the Community

Robot simulation is a field that changes quickly, and the documentation often lags behind the code. Knowing where to ask is therefore nearly as important as knowing how to do it.

- The developer forum: the official robotics developer forum is the most effective place to ask, and the engineers reply directly. Attach the version number, the model of the graphics card and the complete error message, and the chance of a useful reply is much higher.
- GitHub: the code of Isaac Sim and Isaac Lab is public on GitHub. When behavior and documentation disagree, reading the source is often quicker than searching. Issues and changes may be submitted too—the most substantial way of taking

part.

- Official courses and newsletters: the free introductory robotics course suits students with no background at all, and the robotics newsletter is the least effortful way of keeping up with changes between versions.
- Papers: the figures in technical reports and papers (such as the throughput cited in Section 33.3) are the most reliable basis on which to plan a hardware purchase, and far more accurate than marketing material.

Three Reminders for the Project Student

Pin the version. This ecosystem updates quickly and the API changes between versions. Write the version number into the documentation when the project begins, and have the whole team use the same one.

Do not pursue perfection in simulation. The value of simulation is in iterating quickly, not in realism at the level of the pixel. Three weeks spent making the materials beautiful usually does nothing whatever for the sim-to-real success of the policy.

Think out what you are going to measure first. Training a robot that walks is only the starting point; the value of a project lies in what you measured and what you compared (Chapter 31).

33.8 Deployment at the Edge: Jetson, Quantization and Distillation

Training happens in the cloud or on a workstation, and execution on the robot. This section takes up the last leg of that train and deploy chain: how to fit the model into a computing module the size of a palm and running on a battery.

One: the Scale of the Problem

The computational demands of a vision-language model usually confine it to the cloud, and running in the cloud brings every problem discussed in Section 25.1: dependence on the network, latency, bandwidth and reliability. For a robot working in an underground car park or a factory, stopping dead when the network drops is not an acceptable design.

Two: Choosing the Hardware

Module	AI performance	Memory and bandwidth	Power	Typical use
AGX Thor T5000	2,070 TFLOPS (FP4 with sparsity)	128 GB, 273 GB/s	40–130 W	Running a large model aboard
AGX Orin 64GB	275 TOPS (INT8)	64 GB, 204.8 GB/s	15–60 W	SLAM + several models in parallel
Orin NX 16GB	100 / 157 TOPS	16 GB, 102.4 GB/s	10–40 W	The mainstream of inspection robots
Orin Nano 8GB	40 / 67 TOPS	8 GB, 68 GB/s	7–25 W	Teaching and student projects

Table 33-7 Edge Computing Modules Compared

Two traps are to be avoided in reading this table. First, Thor's figure is in TFLOPS at FP4 with sparsity and the Orin series' in TOPS at INT8, so the two cannot simply be divided one by the other—a figure seven times larger does not mean performance seven times

better. Second, and more important: for a robot running SLAM and an AI model at once, what decides whether it can run at all is often not the column of computing power but the column of memory. Case 33-2 explains this.

Three: the Three Key Techniques

Quantization: representing the model's parameters in fewer bits. Vision-language and vision-language-action models at the edge at present take 4-bit weights with 16-bit activations as their default configuration; a format of 4-bit weights with 4-bit activations unlocks the newest generation of hardware platforms and is the production-grade upgrade where it is supported; and 8-bit weights with 8-bit activations remain the safer fallback for mature vision and segmentation models.

Distillation: transferring the ability of a large model to a small one. Research has proposed a distillation strategy that transfers the complex spatial-semantic reasoning of a large frontier model to a light local model of 4 billion parameters, so that it may run on an embedded GPU device; the experiments show that fine-tuning the small model on about 500 reasoning traces from the frontier model is enough to give it navigation ability, with a success rate of 34.5 percent against the 39.7 of the large cloud model—quite close.

Memory management: a link often overlooked and extremely important. Research running SLAM and a VLM together on a Jetson Orin Nano found that a map management strategy founded on geometry suffered out-of-memory failures and stalls in execution; after changing to a key-frame loading and unloading mechanism partitioned by semantics, it not only did away with both problems and held the localization steady but raised the throughput and lowered the latency by 21.7 percent.

Four: Formula Two—Estimating the Memory After Quantization

The memory a model needs after quantization is approximately:

$$\text{memory} \approx \text{number of parameters} \times \text{bits per parameter} \ / \ 8 \quad \text{(bytes)}$$

Take a model of 4 billion parameters: 16-bit precision needs about 8 GB, 8-bit about 4 GB, and 4-bit about 2 GB. This explains why 4-bit quantization can squeeze a large model onto an embedded platform—the difference is fourfold. Set against Table 33-7: a 4-bit quantized model of 4 billion parameters takes about 2 GB, and on an 8 GB Orin Nano it must share the remaining 6 GB with the operating system, SLAM's map and the image buffers, which is already quite tight.

But note: the memory actually used includes activations, intermediate results and the overhead of the framework as well, and is usually 1.2 to 2 times the estimate above. Leave margin when planning.

Five: Three Remedies

1. Limit the size of the map: load only the region currently needed, partitioned by semantics. This echoes the multiple maps of Atlas in Chapter 23 and the submaps of Cartographer in Chapter 15—cutting a large problem into manageable pieces.
2. Choose a small quantized model: there is no need to carry one large model for every task. One quantized general model is often more practical than three unquantized specialized ones.
3. Stagger the execution: AI inference need not run on every frame. Once or twice a second is usually enough to support decisions at the semantic level, while SLAM's front end keeps its original rate.

A Checklist Before Deployment

Have you measured the peak memory on the target hardware? (Not the average—the peak.)

Have you run SLAM and the AI model together? Both being fine apart does not mean they are fine together.

Have you measured the end-to-end latency? From the image coming in to the decision going out, not merely the model's inference.

Have you tested with the network down? If there is any dependence on the cloud, this must be tested.

Have you tested running for a long time? A memory leak and the growth of the map take hours to show themselves (Section 19.6).

Cases

Case 33-1 With an Uncalibrated Simulation, the Policy Trained Falls Over at Once

This is the real experiment that states the system identification of Section 33.6 most clearly.

A research team did the same thing to several legged robots: hung the robot in the air, sent its joints a frequency sweep lasting 20 to 60 seconds, and, using the encoder data alone, fitted the dynamic parameters by evolutionary optimization over 4,096 parallel environments. The parameters to be identified were the armature inertia, viscous damping, Coulomb friction and zero offset of every joint, plus one global command delay—about forty-nine in all.

The crucial controlled comparison was this: a policy trained with the uncalibrated URDF parameters, placed on a real quadruped, failed even to walk forward and had to be caught by the hoist; while a policy trained with the identified parameters walked normally. The same algorithm, the same rewards, the same training procedure—the difference lay only in whether the simulation's parameters were right.

The team quantified the sources of the gap at the same time, and every one answers to Table 33-4: the firmware's compensating behavior created a virtual inertia of about 8.1×10^{-3} kg·m², the CAD underestimated the inertia of the thigh link, the global command delay was about 7.5 milliseconds, and the zero offset of a joint reached about 5 degrees at most. Not one of these figures can be read off a CAD drawing.

Calibration brought more than being able to walk. The cost of transport measured by the same procedure on different models fell from 1.86 to 1.27, an improvement of about three tenths in energy; as for endurance, one quadruped covered 4.12 kilometers on a single charge and another 6.10; and a small quadruped of 4.2 kilograms climbed twenty stairs of 18 centimeters each in 46 seconds.

There are two lessons for students. First, about 20 seconds of data suffice for the identification—an experiment a one-semester project can well afford. Second, and more important: after identification the team used no domain randomization of the dynamics at all. This agrees with the conclusion of Figure 33-4 in Section 33.6—once you can calibrate the simulation accurately enough, there is no need to buy robustness with nominal performance. Domain randomization is the insurance for when you cannot calibrate, not the default.

Case 33-2 Memory Is the Real Bottleneck

This is a problem many students run into only late in a project, and knowing of it in advance saves a great deal of frustration.

The situation is this: the student has SLAM running successfully on a Jetson and the object detection model running successfully too, but the moment the two run together the system becomes extremely slow, or crashes outright. And yet a check of the processor and graphics processor shows neither fully loaded—which is puzzling.

The real bottleneck is memory. Research has quantified the phenomenon clearly: running SLAM and a vision-language model together on a Jetson Orin Nano, a map management strategy founded on geometry met out-of-memory failures and stalls in execution; while a key-frame loading and unloading mechanism partitioned by semantics not only did away with both and held the localization steady but raised the throughput and lowered the latency by 21.7 percent.

The root of the problem is that SLAM's map keeps growing as exploration goes on (the memory estimate of Section 13.5, the number of particles times the size of the map of Chapter 14), while the AI model takes a great deal of memory in itself. With the two in contention the system is bound to fail.

The ingenuity of the solution lies in partitioning by semantics: the robot need not remember the detail of a whole building at every moment, but only load the map of the region it is in. The idea descends directly from the multiple maps of Atlas in Chapter 23 and the submaps of Cartographer in Chapter 15—cutting a large problem into manageable pieces.

A concrete piece of advice for a project: estimate the memory needed by Formula Two at the planning stage, and consider a mechanism for limiting the size of the map first.

Dealing with it only once the system crashes usually means a large restructuring.

Case 33-3 Putting a Large Model into a Robot: the Power of Distillation

This case answers the doubt students are most likely to have: if a large model will not run, is there nothing for it but to give up?

The background is plain enough: vision-language models have become a powerful tool in robotics, able to understand an environment in the context of language, and have given rise to open-vocabulary tasks such as object-goal navigation; but their computational load usually confines them to the cloud.

What the research team did was to distill: first build a high-quality pipeline founded on a scene graph with a large frontier model, reaching a success rate of 39.7 percent on the standard open-vocabulary object navigation benchmark; then transfer that reasoning ability to a local model of only 4 billion parameters, so that it might run on an embedded GPU device.

The crucial result is this: fine-tuning the small model on about 500 reasoning traces from the frontier model alone was enough to give it navigation ability, with a success rate of 34.5 percent, narrowing the gap to the large cloud model greatly.

Five hundred pieces of data—the figure is worth savoring. Set against the hundreds of thousands of labels needed to train from nothing, the cost is tiny. And what it buys is a change in kind, being able to run on the robot: no network needed, no cloud latency, and no data leaving the machine.

The methodological lesson of this case agrees exactly with the NanoSLAM case of Chapter 25: the bottleneck in performance is often not a want of computing power but

a method not yet optimized. When you feel that this technique is too heavy and will not run, the first question should not be whether to give up but whether there is a cleverer way.

Implementation: from Training in Simulation to Deployment at the Edge

The implementation of this chapter falls into three stages of increasing difficulty. Stage one needs no special hardware; stage three needs an embedded computing module.

Stage One The Three Figures of Parallelism (no robot needed)

Reproduce the three measurements of Figure 33-1 of this chapter. The task may be of your own choosing; the simplest is the single-joint tracking used here, though any vectorized environment you are familiar with will serve.

1. Measure the throughput: fix the policy, run one batch for each of N = 1, 2, 4, …, 4096, record the wall-clock time and convert it to environment steps per second. Draw it on logarithmic axes and mark the point of saturation.
2. Measure the gradient noise: fix the policy, sample forty times over for each N, and compute the standard deviation of the cost estimate. Verify whether it falls as 1/√N (multiply by √N and see whether it is nearly constant).
3. Measure the stability of training: fix the budget of wall-clock time, run five independent trainings for each N, and draw the median and the interquartile range. What matters is not whose median is higher but whose interquartile range is narrower.

The product of this stage is a parallelism curve for your own hardware. It will tell you at what number of environments it stops being worth it on your machine—the basis on

which every training experiment afterwards is planned.

Stage Two System Identification and Domain Randomization (one joint with an encoder needed)

The least that is required is a servo motor or gearbox module with an encoder, able to take a position command and report the actual angle. One joint of a robot arm will do.

1. Record the sweep: let the joint carry no load, send a swept position command from 0.2 to 2 Hz, record the command and the encoder reading at as high a rate as possible, for 20 to 60 seconds.
2. Build the model: write down a discrete-time model with rotational inertia, viscous damping, Coulomb friction and a command delay. Start with the CAD value of the inertia and zero for the rest, simulate once and compare with the measurement—that is your Figure 33-3(a).
3. Do the identification: search the delay on an integer grid in the outer loop and fit the continuous parameters by least squares in the inner one. Remember to verify whether the parameters compensate for one another: verify again on a different excitation signal, and if the parameters run away when the signal changes, the identification is not yet steady enough.
4. Measure the cost: train one set of controllers with the identified parameters and another with wide-range domain randomization, and compare their tracking error on the real joint. You should see the same phenomenon as in Figure 33-4: the randomized set takes more punishment and is less accurate at the nominal point.

Stage Three Deployment at the Edge (an embedded computing module needed)

1. Establish a baseline: run SLAM alone on the module and record the peak memory, the latency and the update rate. Then run your AI model alone and record the same three.
2. Run them together: run both at once and measure the three again. Compare the combined figures with the sum of the separate ones; the difference is the loss caused by contention.
3. Quantize: quantize the model to 8 bits and to 4, and measure the memory and the accuracy of each. Set the estimate of Formula Two against the measurement and see how much of it is the framework's overhead.
4. Apply the remedies: try the three remedies of Section 33.8 one by one, changing one thing at a time and measuring again, and build a table of remedy against effect.

The product of this stage is a deployment report made of figures you measured yourself. It carries more conviction than any specification sheet, and it is the most practical close to this book's technical line.

Further Reading

- The Isaac Lab technical report (2025). (The architecture and throughput figures of GPU-native parallel simulation, the basis of Section 33.3.)
- The GitHub projects and official documentation of Isaac Sim and Isaac Lab. (The practical foundation of Sections 33.2 and 33.7.)
- Research on systematic sim-to-real transfer for legged robots (2025). (The source of Section 33.6 and Case 33-1.)
- Survey papers on learned legged locomotion control. (The background to

Section 33.1, with comparisons among the various methods.)

- The OpenUSD specification and the robot structure description documentation. (A reference for the scene and robot description formats of Section 33.2.)
- NVIDIA Jetson developer documentation and the inference optimization guide. (The technical foundation of the edge deployment of Section 33.8.)
- The edge computing of Chapter 25, the methods of assessment of Chapter 31, and the data collection and labeling of Chapter 32. (The direct joints between this chapter and the rest of the book.)

The Points of This Chapter

- A reinforcement-learned robot policy needs interactions with the environment of the order of a billion, which on a real machine comes to about 231 days of continuous running, so that training must be done in simulation first. Simulation offers accelerated time, failure at no cost, a wholly known state, repeatable experiments and environments that can be generated; but it solves the equations you wrote into it, not real physics.
- Isaac Sim is the simulator and the scene environment (PhysX physics, RTX rendering, OpenUSD scenes) and Isaac Lab the robot learning framework built upon it (tasks, observations, rewards, the training procedure). Both became generally available in August 2025 with their extension layers open-sourced, though the Omniverse Kit beneath is not wholly open-sourced and takes no outside submissions.
- The key to GPU-native simulation is that the state lives in GPU memory as tensors, so that the whole training loop never goes back and forth to the CPU. The measurement shows throughput nearly linear while $N \leq 32$ (from 93

thousand steps per second to 2.91 million), saturating thereafter at about 41.8 million; and yet with the environments increased 4,096-fold the waiting per batch rises only 9.8-fold.

- The real value of parallelism is in lowering gradient noise. The measured standard error of the cost estimate falls precisely as 1/√N (multiplied by √N, all eleven points lie between 0.25 and 0.33). Within the same budget of 8 seconds, the median success of N = 4 is only 94.6 percent with an interquartile range spanning 93.3 to 95.8, while N = 64 reaches 98.3 and N = 1024 reaches 99.6 with an interquartile range of only 98.8 to 99.6—with too little parallelism the training stops short and its outcome varies more widely.
- Sensor simulation falls into four families: proprioception (high), camera (fairly high), ranging (medium) and contact and touch (fairly low). The camera's segmentation, depth and normals are by-products of rendering, which is the technical foundation of synthetic data; while specular reflection in LiDAR, motion distortion, and the time differences between sensors are the sources of the gap most often overlooked.
- The temptation of synthetic data is that the labels are free and without error, and the trap is the domain gap. The three roads to narrowing it are reconstructing the real scene, visual domain translation and visual domain randomization; what works best in practice is pre-training on synthetic data with fine-tuning on a little real data, with a test set of real data only.
- The sim-to-real gap has seven sources, of which actuator dynamics, errors in link inertia and time delay are most often underestimated. The system identification experiment shows the URDF's rotational inertia to be only 0.33 of the true value, with damping, friction and delay simply zero; after calibration the

root-mean-square error between simulation and reality falls from 3.36 degrees to 0.0086, about a four-hundredth.

- System identification has one characteristic trap: in discrete time the delay is a step function, so that a gradient method stops dead at its initial value while the inertia is overestimated to compensate—the trajectories agree and the parameters are all wrong. The right course is to search the delay on a grid in the outer loop.
- The range of domain randomization must be wide enough to cover the real uncertainty: a narrow ±10 percent is of almost no effect (a deployable region of 39.6 percent against 40.8 with no randomization), while a wide range reaches 98.3. The cost is that the tracking error at the nominal point rises from 0.0069 radians to 0.0179, about two and a half times—robustness is bought with nominal performance.
- The measured boundary of failure is decided almost entirely by the delay (about 9.5 milliseconds) and is nearly independent of the rotational inertia. Which shows that in planning, the delay of the control loop should be measured before the mass of every link is computed precisely.
- The two remedies are usually used together: calibrate the center of the simulation correctly by system identification first, then randomize moderately about it to cover wear, temperature and the differences between individual machines. Case 33-1 shows that when the calibration is good enough, domain randomization of the dynamics may be dispensed with entirely.
- The three great techniques of edge deployment are quantization (4-bit weights are at present the default for vision-language models at the edge), distillation (fine-tuning a 4-billion-parameter model on about 500 reasoning traces reaches

34.5 percent, close to the 39.7 of the large cloud model), and memory management (key-frame loading and unloading partitioned by semantics does away with out-of-memory failures and lowers the latency by 21.7 percent).

- In choosing an edge module, what decides whether it can run is often the column of memory rather than the column of computing power; and different generations are labeled in different units (TFLOPS at FP4 with sparsity cannot be compared directly with TOPS at INT8). The memory actually used is about 1.2 to 2 times the estimate of Formula Two.

Carrying Forward

This chapter has carried the techniques of the book from running to learning: Chapter 32 gave the map a meaning, and this chapter has given the robot a way of acquiring a skill in simulation and of sending the trained model all the way to the computing module on its body.

But one piece of the puzzle is still missing. What this chapter trains is a policy for how to walk, and what Chapter 32 built is a map of what is here—between them is missing why go there. When the user says go to the meeting room and turn off the projector, the robot must break one sentence into a succession of executable subgoals, must reason in an environment it has never seen about where a meeting room usually is, and must know where to explore when it cannot find the goal.

That is the subject of Chapter 34: vision-language models, language navigation, and embodied intelligence. It is also the last chapter of the book—we shall gather up the threads from Chapter 1 to Chapter 34 and use one complete task to explain where each of these thirty-four chapters plays its part.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 34

VLMs, Language Navigation and Embodied Intelligence

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

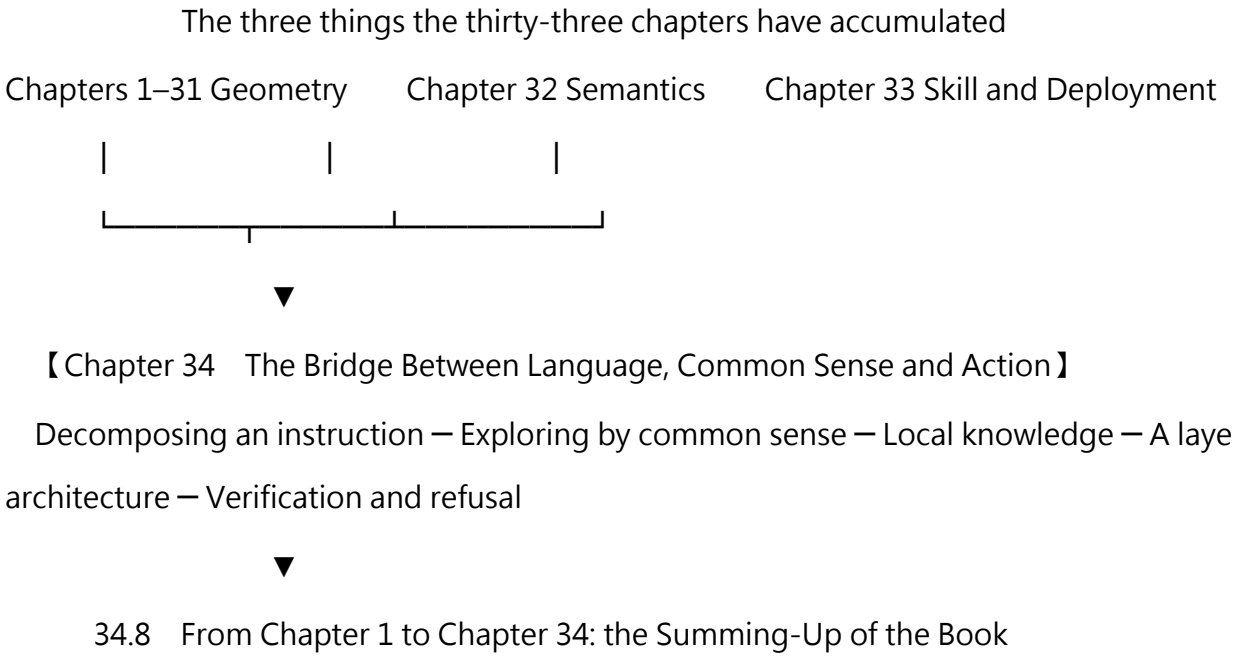
This is the last chapter of the book, and the one that gathers up the thirty-three before it.

Up to now we have three things: Chapters 1 to 31 gave the robot an accurate geometric map, so that it knows where I am and what shape the environment has; Chapter 32 gave the map a meaning, so that it knows that is a door and that is a person; and Chapter 33 let it learn a skill in simulation and sent the model into the computing module on its body.

One thing is still missing: a purpose. When the user says go to the meeting room and turn off the projector, the robot must turn one sentence into a succession of executable subgoals, must reason on a floor it has never visited about which side the meeting room is likely to be on, and must know where to go on looking when it cannot find the goal. This is the subject of this chapter—the bridge between language, common sense and action.

This chapter carries the summing-up of the whole book as well. Section 34.8 will use one complete task to put every part from Chapter 1 to Chapter 34 back where it belongs, and explain why not one of the nine parts could be spared.

Figure 34-0 The Link Map of Chapter 34



Learning Objectives

This chapter begins from how one sentence becomes a coordinate and takes up in turn the common-sense prior, the local knowledge base, the layered architecture and the mechanism of verification, then examines honestly the real level of today's vision-language models in navigation, and closes with the summing-up of the book. It has three figures computed from experiments, each answering one question: what common sense is worth, what happens when common sense is wrong, and why there must be a layer of verification. By the end of this chapter students should be able to:

1. Explain the stages by which a natural language instruction becomes a navigation goal, and point out the mode of failure possible at each.
2. Explain how a common-sense prior shortens the cost of searching in object-goal navigation, and quantify what it is worth.
3. Explain what a prior pointing the wrong way costs, and judge from that when

the semantic level should not be trusted.

4. Design a local spatial knowledge base, and explain three advantages of retrieval-augmented generation over fine-tuning a model.
5. Explain a system architecture layered by time scale, and point out the update rate and the responsibility of each layer.
6. Estimate the overall success of a multi-step task by a model of multiplied probabilities, and explain why verification and retrying improve it so greatly.
7. Explain how far today's vision-language models stand from human performance in embodied navigation, and read the results of the relevant benchmarks.
8. Map the contents of the book's nine parts onto the stages of one complete task.

Core Concepts

Term	Meaning	Related Sections
Embodied intelligence	The intelligence that arises as an agent interacts with its environment through a body	34.1, Chapter 33
Language navigation	Navigating an unknown environment by an instruction in natural language	34.2, 34.6
Object-goal navigation	Given the name of an object, finding where it is	34.3

Common-sense prior	The statistical knowledge that an object of some kind is usually in a space of some kind	34.3
Semantic exploration	Deciding the next direction of exploration by a semantic prior	34.3, Chapter 16
Retrieval-augmented generation	Retrieving the relevant information from a knowledge base before inferring	34.4
Scene graph	A structured representation describing a scene by nodes and relations	34.1, Chapter 32
Layered architecture	Dividing a system into layers by time scale, each with its own office	34.5, Chapter 3
Mechanism of refusal	A lower layer may overrule an upper one's decision to keep things safe	34.5, 34.7
Vision-language-action model	A model that outputs actions directly from images and language	34.6, Chapter 33

Table 34-1 The Core Terms of This Chapter

34.1 From Knowing What a Thing Is to Knowing What to Do

At the close of Chapter 32 the robot had a map marked with semantics. This section explains what is still missing between that and being able to execute one sentence.

One: the Three Levels, Once More

Chapter 32 described robot intelligence in three levels: the geometric level answers where am I and what shape the environment has, the semantic level answers what have I seen, and the task level answers what should I do next. The first two this book has dealt with; this chapter deals with the third.

The task level differs from the first two in kind: the input of the first two is a sensor and the output a description of the world; the input of the task level is a person's intention and the output an action. A description may be made accurate, but an intention is often vague—turn off the light in the meeting room says neither which meeting room nor where the switch is. The better half of the task level's work is supplying what was not said.

Two: How Many Assumptions Hide in One Sentence

Take go to the meeting room and turn off the projector: the information the robot must supply for itself includes at least these.

- Which meeting room? (If there are several, it may have to be inferred from where the user is or from the calendar.)
- Where is the meeting room? (Local spatial knowledge is needed, Section 34.4.)
- What does a projector look like? (The recognition of the semantic level is needed, Chapter 32.)
- Where in the meeting room is the projector? (Common sense is needed: usually on the ceiling or on the lectern.)

- How is it to be turned off? (The remote? A button on the body? Or an infrared signal?)
- How is it confirmed that it really went off? (Verification is needed, Section 34.5.)
- What if it cannot be turned off? (Degradation and reporting are needed, Section 34.7.)

Of these seven, only the third was dealt with in Chapter 32. The other six need local knowledge, common-sense reasoning, knowledge of actions and a mechanism of verification—which is exactly what this chapter is to supply.

Three: Why a Scene Graph Is More Useful than a Semantic Map

Chapter 32 mentioned the scene graph: recording not merely that there is a chair here but the structure of relations—the chair is beside the table, the table is in the meeting room. At the task level that difference becomes crucial.

The reason is that spatial relations in language are relative. The cup on the table, the fire extinguisher beside the door, the third seat in the row by the window—none of these can be expressed on a flat semantic map, which records only that there is an object of some class at some coordinate. To answer the cup on the table, the system must know which cup's relation of support points to which table.

A Distinction: the Three Forms a Navigation Goal Takes

A point goal: a coordinate given directly (go to (12.4, 8.1)). This is the form the first thirty-one chapters deal with, and the form every upper layer must finally convert into.

An object goal: a category of object given (go and find a water cooler). The

position of the goal is unknown and must be searched for—the subject of Section 34.3.

An instruction goal: a passage of natural language given (go to the end of the corridor, turn right, and go into the second classroom). It must be understood and executed step by step—what Section 34.6 assesses, and the hardest of the three at present.

34.2 How a Language Instruction Becomes a Navigation Goal

This section takes that chain of conversion apart, because every link in it is a possible point of failure.

One: Five Stages

Stage	Input	Output	Typical failure
① Parsing the intention	Natural language	A structured description of the task	An unclear reference, an ambiguous verb
② Locating the place	The description of the task	A region on the semantic map	The name of the place is not in the knowledge base
③ Locating the goal	The region + the object category	A three-dimensional coordinate	The object is not where it was, or is recognized wrongly
④ Planning the path	A three-dimensional coordinate	A feasible path	The goal is unreachable, the door is shut

⑤ Executing and confirming	The path	Completion or a report	It arrived and did not confirm; it was interrupted on the way

Table 34-2 The Five Stages from Language to Action

It is worth noting that everything after stage ④ has already been dealt with entirely by the first thirty-one chapters of this book. What this chapter and Chapter 32 add are stages ① to ③. Which is to say that language navigation does not replace SLAM but attaches a stretch of conversion in front of it.

Two: The Output Should Be Structured, Not Free Text

An important principle in implementation: what an upper layer sends a lower one should be structured data (goal coordinates, a list of objects, an action code) and not free text.

The reason is practical. Free text cannot be parsed reliably, nor verified. If the language model answers walk toward that room ahead on the left that looks like a tea room, how is the downstream system to turn it into a coordinate? Whereas if it answers goal region: tea room, confidence 0.72; candidate coordinate (23.3, 13.9), the downstream can query the map and verify whether that coordinate is reachable, whether it lies in a forbidden zone, and whether it conflicts with the cost map.

This too is why having a language model output velocity commands directly is poor engineering design. It skips every intermediate representation that could be verified, and turns a system that can be checked into a black box that cannot.

Three: Uncertainty Must Be Passed Down

Chapter 11 spent a whole chapter on probability and uncertainty, and the idea applies just as well at the task level, where it is often overlooked. A language model will usually say the meeting room is on the east side of the third floor in the same confident tone whether it looked the fact up in a knowledge base or guessed it.

The practical course is to have every goal the upper layer outputs carry a confidence and a mark of its source (looked up in the knowledge base / inferred from common sense / a pure guess). The lower layer decides from that mark how careful to be—a goal from the knowledge base may be driven to directly, while a goal that is a pure guess should have a cheap exploratory trip sent out first.

34.3 The Common-Sense Prior and Semantic Exploration

A washbasin is usually in a toilet or a tea room—knowledge of this kind cannot be derived from a geometric map, and yet it changes a robot's efficiency of search a great deal. This section quantifies what it is worth with an experiment that can be reproduced in full.

One: the Experimental Setting

The scene is one floor of a departmental building: a central corridor with twelve rooms on either side, of six kinds (classroom, laboratory, office, tea room, meeting room, toilet, storeroom). The robot sets out from the entrance to find a specified object (a water cooler, a fire extinguisher, a projector, a first-aid box, a ladder, or a copier).

The crucial part of the setting is that both strategies are given the same map. This is not a comparison of having a map against having none—that would not be fair—but a comparison of the order of search under one and the same prior map:

- Geometric order: go each time to the nearest room not yet searched. This is the baseline, using no semantic information at all.
- Semantic order: sort by the probability that an object of this kind appears in a room of this kind, from high to low, taking the nearer room where the probabilities are equal.

The objects are placed at random by a set of real conditional probability distributions (a water cooler, say, is in the tea room with probability 55 percent, in the corridor 30 percent, and in a meeting room 15). On entering a room the robot sweeps back and forth (the length of the sweep estimated by the coverage planning of Chapter 16), and the object counts as found when it comes within a sensing range of 3.5 meters with the line of sight not blocked by a wall. Six hundred random trials are run in all.

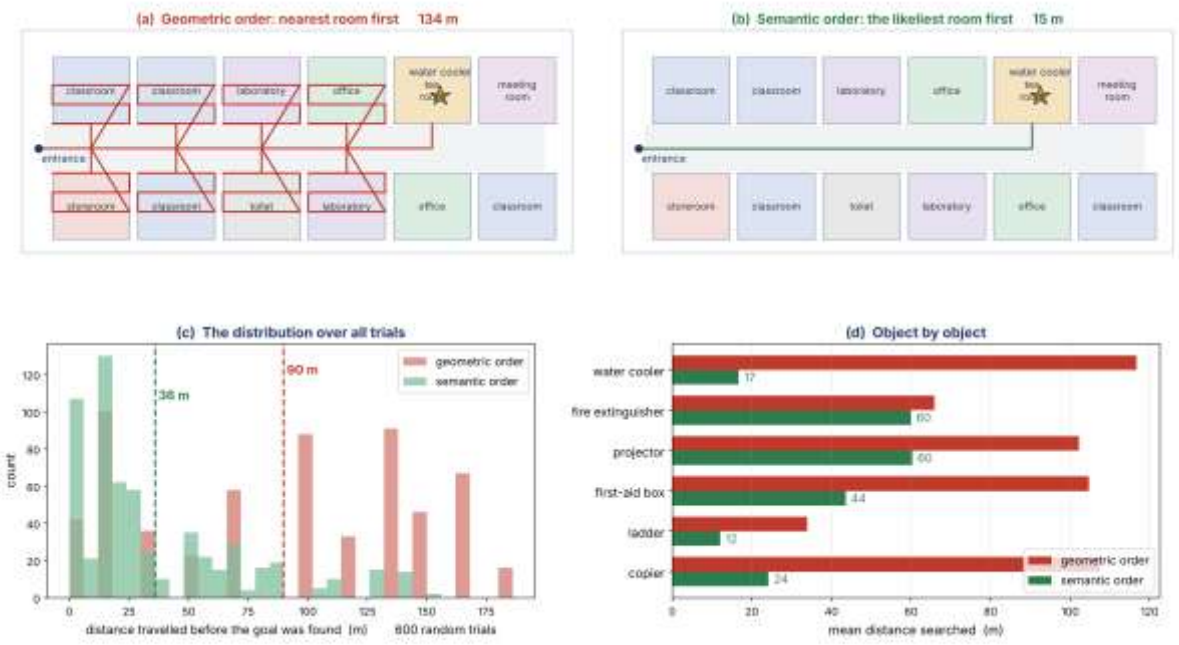


Figure 34-1 The Same Map, Two Orders of Search

Panels (a) and (b) draw the same trial: the water cooler is in the tea room. The geometric order begins from the classroom nearest the entrance and presses eastward room by room, travelling 134 meters before reaching the tea room. The semantic order goes

straight to the tea room and travels only 15. What is drawn is the path of movement between rooms; the sweeping within a room is not drawn, though its distance is counted.

Panel (c) is the overall distribution over 600 trials. The mean of the geometric order is 90 meters and of the semantic order 36, a mean shortening of 60.1 percent; by the median it is 100.9 meters against 18.3, a shortening of 81.9. The two distributions differ in shape as well: the geometric order shows distinct peaks, each corresponding to which room in the order of search the object fell in, while the semantic order is heavily concentrated within 30 meters.

Panel (d) takes the objects one by one, and the benefit differs greatly. The ladder falls from 34 meters to 12 and the copier from 108 to 24, because their distributions are highly concentrated (the ladder is in the storeroom 70 percent of the time, the copier in an office 65). The fire extinguisher falls only from 66 meters to 60, because it is in the corridor with probability 60 percent—and the corridor is where the robot passes in any case, so a semantic prior is of little help there.

That difference is itself a useful conclusion: the value of a common-sense prior depends on how concentrated the object's distribution is. For objects that are everywhere (fire extinguishers, waste bins, hydrants) the return on effort spent building semantic reasoning is low; for objects that are only in particular places (a ladder, a copier, a particular instrument) the return is high.

A Number Often Skipped Over

In these 600 trials the semantic order travelled farther than the geometric one

in 15.3 percent of cases.

The reason is that the room of highest probability is not necessarily the room the object is really in this time. When the water cooler falls in the meeting room on its 15 percent, the robot that rushed off to the tea room has made a wasted trip.

Which is a reminder that a semantic prior improves the expectation and not every single run. In assessing any semantic navigation system, looking only at the mean underestimates its variation—a principle of assessment stressed over and over in Chapter 31.

Two: What Happens When the Prior Is Wrong

The experiment above assumes that the robot's common sense is right. But a vision-language model's common sense is learned from data on the internet and need not match your building. The copier is in an office holds in most places, but what if this building puts all its copiers in an open area of the corridor?

The experiment below sweeps the accuracy α of the prior: with probability α the true distribution is used, and otherwise the ordering of room types is shuffled at random. One extreme case is added as well—a prior that points the wrong way, so that the higher the true probability of a room the later it comes in the order.

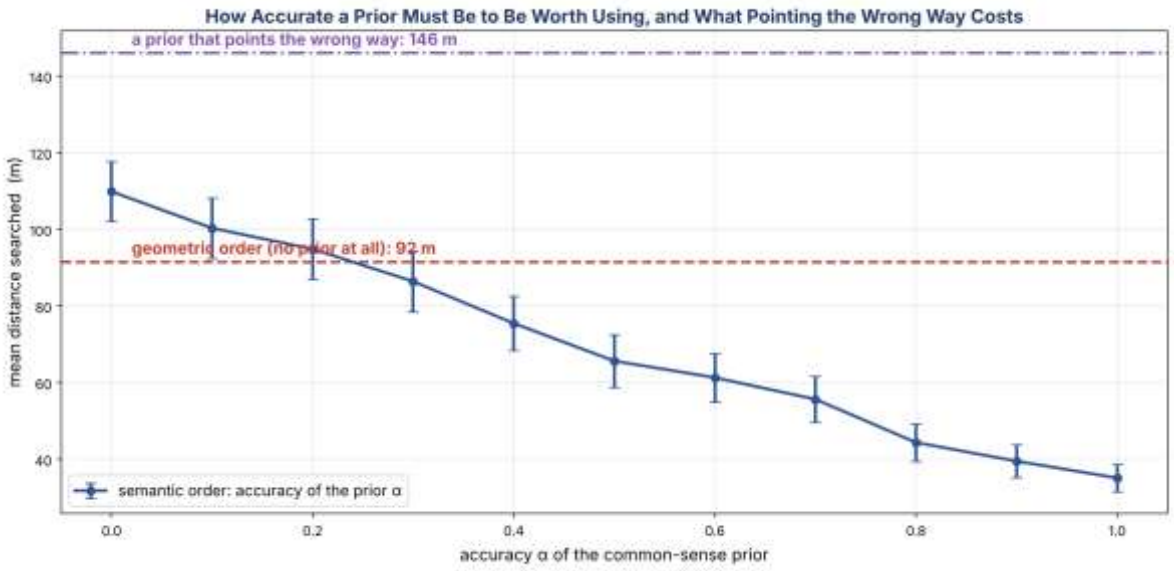


Figure 34-2 How Accurate a Prior Must Be to Be Worth Using, and What Pointing the Wrong Way Costs

The blue line is the semantic order as α varies, with 95 percent confidence intervals for error bars. It falls from 110 meters at α = 0 all the way to 35 at α = 1, roughly linearly. That linear relation is itself good news: a common-sense prior need not be perfect to be useful, and every tenth of accuracy gained saves about seven meters.

The red dashed line is the geometric order with no prior at all (92 meters). Note where the blue line crosses it: at about α = 0.25. Below that the prior is worse than useless—a wholly random order of search (110 meters at α = 0) is in fact somewhat worse than one that goes from near to far, since going to the nearer room first at least saves travel at the start, while a random order does not even do that.

The real risk is in the purple dotted line: when the prior points the wrong way the mean distance of search is 146 meters, 60 percent worse than using no prior at all. This is the most dangerous mode of failure of the semantic level—it does not merely fail to help but actively leads the robot the wrong way, and does so in the same confident tone.

The lesson for system design is clear: the semantic level's suggestions must be able to

be overruled. A reasonable course is to set a budget—if the first two suggestions of the semantic order have been searched without finding the object, fall back on the geometric order and trust the prior no further. This is exactly the concrete form of the mechanism of refusal of Section 34.5.

34.4 RAG: Letting a Robot Read the Spatial Knowledge of Its Own Site

The common-sense prior of the last section deals with knowledge in general. This section deals with the other half: the knowledge peculiar to this one building.

One: the Problem and the Solution

Here is a practical problem: however clever a vision-language model may be, it does not know which building and which floor this school's registrar's office is on—that is the knowledge of a particular institution, and it is not in the model's training data.

The solution is retrieval-augmented generation (RAG): build a local spatial knowledge base, and when the robot receives an instruction, retrieve the relevant information from the knowledge base first and then hand it to the model to reason over. The model is responsible for understanding and reasoning in language, and the knowledge base for the facts.

Figure 34-3 An Example Structure for a Campus Spatial Knowledge Base

【The topology of the floors】

First floor: the gate → the hall → the registrar's office → the student affairs office → the

stairs → the lift

Second floor: the lift → the corridor → the computer classroom → the meeting room

【The attributes of a place】

The registrar's office: first floor, east of the hall, open 08:00–17:00, the door an automatic glass one

The intelligent robotics classroom: third floor, entrance → work benches → the robot dog test area → the LiDAR mapping area → the charging area

【The rules of passage】

The robot may not enter: the inside of a laboratory, the stairs (except a stair-climbing platform)

Regions in which it must slow: the hall, the doorway of a classroom, a corner

【What common queries correspond to】

"office" → the registrar's office, the student affairs office "charging" → the charging stations on each floor

Two: the Three Advantages of RAG

1. It can be maintained: when the layout of the campus changes, only the knowledge base need be updated and the model need not be retrained. A classroom changes its use and one line of text is changed.
2. It can be verified: the robot's answer can be traced to the entry in the knowledge base it came from, which makes debugging and accountability easier. This bears directly on the question of explainability in Section 34.7.
3. It is cheap: the cost of building a knowledge base is far lower than fine-tuning a large model, and it suits a teaching project better—one structured text file is

enough to begin.

Three: Three Practical Examples of a Query

- How do I get from the entrance to the robot dog test area?—RAG retrieves the internal topology of the intelligent robotics classroom, the model generates a description of the route from it, and that is turned into a sequence of navigation goals.
- If there is a student sitting on the floor ahead, how should the robot go around?—real-time perception (a person detected) is combined with the rules of passage in the knowledge base (slow down, give way) to produce a strategy for going around.
- How should a robot vacuum plan to cover a whole classroom?—the map (Chapter 13) is combined with coverage planning (Chapter 16), a typical example of the semantic and geometric levels working together.

Four: A Practical Recommendation on Architecture

For a project implementation the division of labor advised is this: a structured text file or a simple database will serve for the RAG knowledge base (there is no need to start with a vector database); the language model is responsible for understanding language and judging what it sees; and every decision bearing on safety (obstacle avoidance, speed limits) is still guarded by the classical geometric system. This is the principle the next section stresses repeatedly.

> **The Division of Labor Between the Knowledge Base and the Common-Sense Prior**
>
> A point easily confused: the common-sense prior of Section 34.3 and the

knowledge base of this section are both knowledge outside the model, and the two are wholly different in character.

The common-sense prior is statistical and general across sites (a ladder is usually in the storeroom), and answers where to look first. The knowledge base is definite and particular to a site (the storeroom is on the west side of the first floor), and answers where that place is.

A good system wants both: look up with the knowledge base all that can be looked up, and leave only what cannot to the common-sense prior to guess—remembering to mark it a guess (Section 34.2).

34.5 The Layered Architecture and the Mechanism of Verification

This section explains how to assemble the components of the sections above into a system that does not run out of control when an upper layer goes wrong.

One: Layering by Time Scale

Figure 34-4 The Layered Architecture of a Language Navigation System

【The task layer】a second and above the language model agent

- Understand the natural language instruction, query the knowledge base (Sections 34.2, 34.4)
- Decompose the task, choose the direction of exploration, judge an anomaly (Section

34.3)

↓ output: a semantic goal (where to go, what to find) + a confidence + a source

【The semantic layer】0.1–1 second the AI vision models

- Object detection, segmentation, depth estimation (Chapter 32)
- Building and querying the semantic map

↓ output: geometric goal coordinates, labels of dynamic objects

【The geometric layer】10–30 Hz SLAM + navigation (Chapters 1–31)

- Localization and mapping, path planning, the cost map

↓ output: velocity commands

【The control layer】100 Hz and above the chassis / flight controller (Chapter 33)

- Motion control, immediate obstacle avoidance, the safe stop

【The principle of safety】an upper layer may go wrong; a lower layer must hold the line of safety

This layering, the hybrid architecture of Chapter 3, the two subsystems of Cartographer in Chapter 15, the two-layer design of LOAM in Chapter 18, and the degradation architecture of the drone in Chapter 26 are one and the same pattern of system design carried into the age of AI—layer by time scale, and let each layer do its own office. By this point students should feel deeply that a good principle of architecture holds good across eras.

Two: Four Practical Points in Integration

1. Design it asynchronously: AI inference takes time and must never block the navigation loop. It should be called asynchronously, and the robot kept in a safe behavior while it waits (continuing with the previous goal, or stopping).
2. An explicit interface: what the AI layer outputs should be structured data and

not free text (Section 34.2). This lets the downstream system parse and verify it reliably.

3. A mechanism of refusal: the geometric layer should be able to overrule the AI's decision. If the AI says go forward while the cost map shows an obstacle ahead, the geometric layer must win. The purple dotted line of Figure 34-2 shows what the want of such a mechanism costs.

4. A path of degradation: when the AI service is unavailable (disconnected, timed out, the model crashed), the system should fall back on purely geometric navigation rather than halt—exactly the principle of degrading failure by degrees of Section 26.2.

Three: Why a Multi-Step Task Must Have Verification

The mechanism of refusal above sounds like an insurance for safety. But it has a more fundamental use as well: without it a multi-step task cannot have a usable rate of success at all. A simple model below explains why.

Suppose a task has n steps, each succeeding with probability p, and the steps are independent. Then the overall rate of success is p to the nth power—this is Formula One. The destructiveness of that product is far greater than intuition suggests.

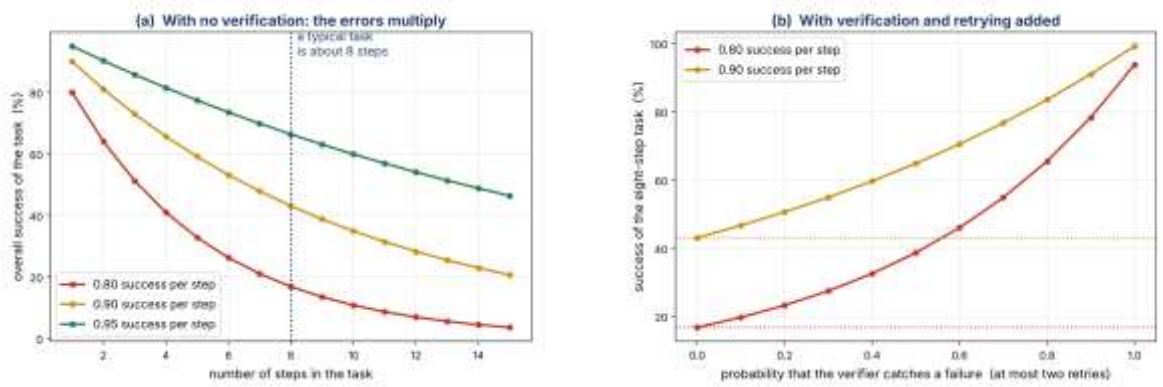


Figure 34-5 The Success of a Multi-Step Task, and the Effect of Verification and

Retrying

Panel (a) draws the consequence of the product. For a task of eight steps (the five stages of Section 34.2 plus opening the door, lining up, and reporting—eight is a conservative estimate), even with a 90 percent rate of success at every step the overall rate is only 43.0 percent; at 80 percent per step only 16.8 remains. For an eight-step task to reach 66.3 percent, every step must manage 95.

This explains a common puzzlement: why does every subsystem of a robot look all right while the whole thing keeps failing? It is not that any one link is particularly bad; the product itself is merciless. It also explains why dividing the task more finely is not necessarily a good thing—the finer the division, the more the steps, and the heavier the penalty of the product.

Panel (b) is the remedy. A verification is added after every step: if the verifier judges the step to have failed, it is retried (at most twice). The horizontal axis is the probability that the verifier catches a failure, and the dotted line is the level with no verification. Taking a system at 80 percent per step: the overall rate of success rises from 16.8 percent to 38.7 at a detection rate of 0.5 and to 93.8 at a detection rate of 1.0. A system at 90 percent per step rises from 43.0 to 99.2.

Note the size of that improvement: a perfect verifier turns a system that fails eight times in ten into one that succeeds nine times in ten, with the ability of every underlying step entirely unchanged. This is one of the most important engineering conclusions of the chapter—for a multi-step task, investing in knowing that one has got it wrong pays far better than investing in doing it better.

And verification is usually much easier than execution. Driving to the door of the

meeting room needs a complete navigation stack, while confirming whether one is at the door of the meeting room may need only one reading of the room number, or one comparison of the current pose with the goal pose. That asymmetry—verification being simpler than execution—is the fundamental reason a layered architecture can stand.

Four: Formula One—the Success of a Multi-Step Task

Let the task have n steps, the ith succeeding with probability p_i, the steps independent, with no verification and no retrying. Then:

$$P_{task} = p_1 \times p_2 \times \dots \times p_n$$

When every step succeeds with the same probability this simplifies to $P = p^n$. Adding verification (with detection rate r) and at most k retries, the effective rate of success of a single step rises to:

$$p_{eff} = p + (1 - p) \cdot \Sigma_{j=1..k} (r (1 - p))^{j - 1} \cdot r \cdot p$$

where r·p is the probability of being caught and retried successfully. When r → 1 and k is large enough, p_eff → 1 and the overall rate of success approaches 1; when r → 0, p_eff → p and we return to the case of panel (a). The curves of Figure 34-5(b) are the numerical results of this formula at n = 8 and k = 2.

34.6 Where Things Stand: How Good Vision-Language Models Really Are at Navigation

The five sections above described an ideal architecture. This section examines honestly how well the topmost layer of that architecture does at the level of the technology of 2025 and 2026.

One: A Benchmark for Embodied Navigation

A benchmark for embodied navigation published at the end of 2025 gives a fairly clear answer. Its design is this: the model is given a photograph of a real scene, an instruction in text, and a type of embodiment (a person, a quadruped robot, a wheeled robot, a bicycle), and is to draw on the image the trajectory it ought to travel. The dataset holds 1,000 situations and more than 3,000 expert trajectories, and the scoring combines the dynamic time warping distance of the trajectory, the endpoint error, and a penalty computed from semantic segmentation for the places this embodiment ought not to go.

Eight contemporary vision-language models were assessed. The results are as follows:

Method	Score
A human expert	75.40
Predicting the path alone (told directly where the goal is)	56.55
The straight-line baseline to a known goal	51.89
Complete trajectory prediction (the best model)	34.38
Predicting the goal point alone	29.65

Table 34-3 The Results of the Embodied Navigation Benchmark (a higher score is better)

Two: Three Numbers Worth Remembering

First, the gap between the best model's 34.38 points and the human expert's 75.40 is 41.02 points. This is not a gap to be described as caught up with in another six months. The vision-language models of 2026 are still far from human at embodied navigation.

Second, and the most worth savoring: a straight line to a known goal scores 51.89 points, far above the 34.38 of the best model's complete trajectory. A straight line that looks at nothing in the environment, avoids no obstacle and goes around nothing beats the complete planning of the strongest model of the day. Which shows that the model loses its marks not chiefly on the shape of the path but on having no clear idea where the goal is.

Third, which bears that judgment out: when the goal is given directly and the same model predicts the path alone, it scores 56.55; the other way round, predicting the goal point alone, it drops to 29.65. Which is to say that the model can walk and cannot recognize—mapping the goal named in the language onto the right place in the image is the real bottleneck at present.

Three: What This Means for System Design

These three numbers support the design principles of Sections 34.4 and 34.5 directly. Since the model's weakest link is locating the goal, the system should so far as possible not let it do that at all—a place that can be looked up in the knowledge base is never left for the model to guess (Section 34.4); where it must guess, the guess is marked as such and a budget for searching is set (Sections 34.2, 34.3); and the cost of a wrong guess is absorbed by the mechanisms of verification and refusal (Section 34.5).

Conversely, what the model is relatively good at should be left to it: understanding language, reading an image, generating a report, describing a route when the goal is known. This is a practical division of labor—the question is not whether a language model can do navigation but which parts of navigation suit being left to it.

Another Fact of the Present: Real Deployment Is Still Rare

A survey of 2026 points out that progress in language navigation research is mostly still confined to real-to-sim settings at the level of software, with relatively little research on sim-to-real and real-world deployment, and that practical schemes for edge deployment are quite scarce as well.

This echoes the conclusion of Chapter 33: a result in simulation and performance on a real machine are two different things. For students this is in fact an opportunity—a language navigation system that really runs on a real campus, however simple in function, has substantial research value at present.

34.7 Risks, Limits and the Road Ahead

Before closing, three things must be discussed: the risks, the limits, and the direction.

One: Four Great Risks

1. Hallucination: AI may see things that are not there or reason wrongly, and it usually does not express uncertainty. A robot that trusts AI entirely may hit a wall or take the wrong road. The purple dotted line of Figure 34-2 is the quantified form of this very risk. The remedy is to keep the geometric layer's right of verification and refusal (Section 34.5).
2. The systematic character of a misjudgment: an AI's errors are often not random but systematically biased—always mistaking a certain class of object under a certain light, say. This is more dangerous than random error, because it keeps happening and retrying cannot save it (the model of verification in Section 34.5 assumes retries to be independent, and a systematic error breaks that assumption). The remedy is thorough testing on the site and analysis of the

errors (Chapter 32, Section 33.5).

3. Latency: inference with a large model takes time, and putting it inside a real-time control loop is a disaster. The remedy is a strict layered architecture—AI makes only decisions on the scale of a second and above (Section 34.5).
4. Security: which includes information security (the model or the knowledge base being tampered with), privacy (the images a robot takes may contain personal information), and accountability (whether the grounds of a decision can be traced when something goes wrong). These are not only technical questions but questions of ethics and regulation.

Two: Four Present Limits

- The demand for data: training or fine-tuning a model needs labeled data, and labeling is expensive. This is the chief obstacle for a project or a small team—and the reason distillation, few-shot fine-tuning and synthetic data (Sections 33.5 and 33.8) are regarded so highly.
- The gap in generalization: a model does well in the laboratory and falls off at another site. The discussion of the domain gap in Section 33.5 applies to the semantic level just as well.
- Explainability: why did the AI judge so? When it goes wrong, diagnosis is often difficult. This stands in sharp contrast to classical geometric methods, where every step can be traced, and is one of the reasons Section 34.4 argues for a knowledge base rather than fine-tuning.
- Computation and energy: as Section 33.8 described, even after quantization and distillation AI remains one of the most resource-hungry components on a robot.

Three: Four Directions for the Future

1. A deep integration of SLAM with foundation models: no longer SLAM plus an AI module, but one architecture in which a foundation model handles perception, mapping and planning together.
2. World models: letting a robot understand not only what is now but predict what will happen next. This matters greatly for planning in a dynamic environment, and connects directly with the training in simulation of Chapter 33—a good enough world model is itself a simulator one can train inside.
3. Robot foundation models: a general agent across platforms—one and the same model deployable on wheeled, quadruped, humanoid and flying platforms alike. This would lower the threshold of robot development greatly.
4. Lifelong learning: a robot that goes on learning and adapting after deployment, rather than staying at the ability it left the factory with. The on-device self-supervised learning mentioned in Section 25.6 is an early attempt in this direction, and the map maintenance of Section 19.6 its most practical rudiment.

34.8 From Chapter 1 to Chapter 34: the Complete Journey of One Robot

As the last section of the book, let us use one complete task to string the thirty-four chapters together.

One: One Task, Nine Parts of Knowledge

The situation: a service robot at a university of technology in southern Taiwan receives an instruction—please go to the intelligent robotics classroom on the third floor, check the state of the equipment in the charging area, and report anything amiss.

1. Understand the instruction: the model parses out the goal place and the content of the task (Section 34.2). The output is a structured description of the task, not free text.
2. Query the knowledge base: RAG retrieves that the intelligent robotics classroom is on the third floor, and that its internal topology is entrance → work benches → robot dog test area → LiDAR mapping area → charging area (Section 34.4). This step saves the robot from guessing.
3. Plan the route: plan a path on the existing map from the current position to the third floor, the lift included (Chapters 3 and 29).
4. Move and localize: LiDAR SLAM or visual-inertial SLAM keeps estimating the pose (Chapters 13 to 23), localizing within the existing map (Chapters 11 and 29), with the IMU holding things up through turns and vibration (Chapter 7).
5. Pass the lift lobby: seeing an artificial marker, obtain an absolute pose correction and confirm the floor (Chapters 4 and 24).
6. Avoid a pedestrian: on detecting a person, remove his features so that they do not disturb the localization, and at the same time trigger slowing and giving way (Chapters 24 and 32).
7. Reach the charging area: find the goal by the semantic map and complete the precise alignment by visual servoing (Chapter 24). If it is not to be found on the semantic map, decide where to look by the common-sense prior, and fall back on a geometric search after two attempts (Section 34.3).
8. Inspect and judge: photograph the equipment, have the AI judge whether anything is amiss, and mark the finding with a three-dimensional coordinate (Chapters 31 and 32). Every skill of movement and manipulation comes from the

training in simulation of Chapter 33.

9. Verify every step: confirm the result after each stage is finished, and retry on failure (Section 34.5). Without this step the overall success of the eight steps above would be too low to use.
10. Report: generate a report in natural language, with the position and the photographic evidence attached (Section 34.2).

Every part of this book contributes to that one errand: the ideas of Part I, the sensors of Part II, the mathematics of Part III, the SLAM systems of Parts IV to VI, the edge computing of Part VII, the implementation tools of Part VIII, and the AI semantics, training in simulation and language navigation of Part IX.

And the crucial thing is this: not one of the nine parts could be spared. Without the geometric level the robot hits a wall; without the semantic level it does not understand the instruction; without training in simulation it never learns to move; without the toolchain nothing runs at all. This is the message the book has meant to convey from its first page to its last—SLAM is not a single technique but a whole body of knowledge whose parts hold one another up.

Two: One Thread Running Through the Book

The book began at Chapter 1 with a robot that has no map and does not know where it is, and has passed through sensors, mathematics, 2D and 3D systems, visual methods, miniature and swarm robots, and a complete implementation, to arrive at last at AI semantic navigation, training in simulation and the understanding of language.

If one thread is to be said to run through these thirty-four chapters, it is this: only by understanding a limit can a limit be broken.

We have discussed over and over the weakness of every sensor (Part II), the mode of failure of every algorithm (Parts IV to VI), the constraint on resources of every platform (Part VII), the order of debugging of every system (Part VIII), and the source of the gap in every model (Part IX). This is not pessimism; it is that the nature of engineering is to find a solution within limits.

The arrival of AI has not changed that nature. A language model lets a robot understand and think things through, while walking steadily and not running into people still rests on the geometry and control of the thirty-one chapters before. The numbers of Section 34.6 say it plainly: at embodied navigation the strongest model of the day is still far from human, and a straight line to a known goal even beats its complete planning. Neither can be spared—and so it will be for the foreseeable future.

Three: A Few Last Words to the Student

The two settings of this book—a university's intelligent robotics program and a high school's AI and robotics club course—stand for two stages of learning, and share one conviction: SLAM is not some far-off frontier technology. Its core version will run on a processor of the grade of a mobile phone, its principles can be worked out with pen and paper, and its fruits are turning quietly in the robot vacuum in your own living room.

Every experimental figure in this book was computed on an ordinary computer. The loop closure of Chapter 15, the registration of Chapter 17, the motion compensation of Chapter 18, the inspection route of Chapter 19, the parallel simulation and parameter identification of Chapter 33, and the semantic search of this chapter—not one of them needed expensive equipment. What was needed was only to state the problem clearly, do the experiment cleanly, and look at the numbers honestly.

May students take the foundation this book has laid and go on to build robots of their own—whether a robot dog inspecting a campus, a drone scanning a historic site, or a little car that can only follow a line. Behind every robot that finds its own way is the same body of understanding that took humanity forty years to build.

Now it is your turn.

Cases

Case 34-1 One Sentence, Eight Steps: Why Verification Matters More than Ability

This case puts the numbers of Section 34.5 back into a concrete situation.

Suppose a student has built a campus delivery robot, whose task is to take this document to the registrar's office on the third floor. Taken apart, that is eight steps: understand the instruction, look up the place, plan the route, take the lift, walk to the door, recognize the room number, enter, hand over and confirm.

Suppose every step manages a success rate of 80 percent—which does not sound bad, every subsystem succeeding eight times in ten. But the overall rate is 0.8 to the eighth power, which is 16.8 percent. Eight errands in ten will go wrong somewhere.

A team's instinct is usually to improve the weakest link. Suppose a whole semester is spent raising every step from 80 percent to 90—a considerable engineering effort. And the result? The overall rate rises from 16.8 percent to 43.0. A large improvement, and still nearly six failures in ten.

The other road is to change nothing in any step but add a check after each. Was the lift button pressed? Compare the floor sensor. Are we at the door? Compare the current

pose with the goal pose. Is the room number right? Take a photograph and read the text. If the check fails, do that step again, at most twice.

Computed by the model of Section 34.5: if the check catches half the failures, the overall rate rises from 16.8 percent to 38.7; if the check is nearly perfect, to 93.8. And the ability of every underlying step has not changed at all.

What this case is meant to convey is more than a trick with numbers. It is a judgment about the order of engineering priorities: when a multi-step system performs poorly as a whole, asking whether it knows it has got something wrong is usually more effective than asking how to make it do better. This is the same line of thought as the order of debugging in Chapter 31 and the degradation design of Chapter 26.

Case 34-2 When the Robot's Common Sense Points the Wrong Way

This case takes up a mode of failure students are quite likely to meet in a project and are seldom warned of beforehand.

The situation: a student adds semantic search to an inspection robot. To find a copier, the robot first asks the language model where a copier usually is, receives the answer an office, and so searches every office first. In the building they tested in, the strategy did excellently—the distance searched fell by seven tenths.

Then they moved the system to another building. To save power and simplify management, that building had put all its copiers together in the open area beside the lift on each floor, and there was not one in any office. The result was that the robot went through every office first, every time, and only then found the goal in the corridor it thought least likely.

Figure 34-2 quantifies the situation. When the prior points exactly the wrong way the mean distance of search is 138 meters, against the 90 of a geometric order using no prior at all—the semantic level has set the system back by 54 percent. Worse, the system will not know it is wrong: every time it reasonably goes to the offices first, and every time it reasonably fails.

The middle stretch is worth noting. The blue line of Figure 34-2 stands at 84 meters at $\alpha = 0$ (a wholly random prior), slightly better than the geometric order's 90. Which is to say that an ignorant prior is not to be feared; what is to be feared is a confident and wrong one. And that is precisely the kind a language model is most apt to produce—it seldom says I do not know.

The remedies come at three levels. The least is to set a budget for searching: search the first two places the semantics suggest, and fall back on the geometric order if they fail. Further is to write local observation back into the knowledge base: once a copier has been found in the corridor, that fact should be recorded rather than guessed at afresh from general common sense next time (Section 34.4). The most thorough is to let the prior itself be learned—which is exactly the problem the lifelong learning direction of Chapter 33 is to address.

Case 34-3 A Straight Line Beats the Strongest Model

This is a case fit for the end of the book, because it states at once the value of AI and the boundary of it.

A benchmark for embodied navigation at the end of 2025 asked models to draw on a photograph of a real scene the trajectory a robot ought to travel. Eight of the strongest contemporary vision-language models were assessed, and the scoring weighed the

shape of the trajectory, the accuracy of the endpoint, and a penalty for the places this embodiment ought not to go.

The most surprising item in the results is this: a straight line to a known goal position scored 51.89 points, while the best model's complete trajectory prediction scored only 34.38. A straight line that avoids no obstacle, goes around nothing, and looks at the environment not at all beat the complete planning of the strongest model of the day.

The first reaction may be to laugh at the model. But thought over, the result says not that the model is stupid but that the great bulk of the score comes from the position of the goal. A further experiment confirms it: the same model, told the goal position directly and predicting only the path, leaps to 56.55; the other way round, predicting the goal point alone, it drops to 29.65.

Put otherwise, the model can walk and cannot recognize. It understands a spatial description such as go forward along the corridor and avoid that pile of boxes, and cannot reliably judge which door in the picture is the meeting room the user spoke of.

This diagnosis is of great value to system design, because it says where the engineering effort should go. Since locating the goal is the bottleneck: what can be looked up in the knowledge base is not guessed at (Section 34.4); where an artificial marker will serve, vision alone is not relied on (Chapters 4 and 24); and where it must be guessed, the confidence is marked and a budget set (Sections 34.2 and 34.3).

It says at the same time where the real value of AI lies. The 56.55 the model scored with the goal known and only the path to predict is above the straight line's 51.89—which is to say that with the goal clear, the model really can plan better than a straight line. This is the conclusion of Section 34.6: do not ask whether a language model can do

navigation, but which parts of navigation suit being left to it.

One last word is worth adding. The human expert scores 75.40 on this benchmark against the best model's 34.38—a gap of 41.02 points. The technology is still moving quickly as this book reaches its last page; but for the foreseeable future, that body of geometric method humanity took forty years to build remains the fundamental guarantee of whether a robot can reach its destination safely.

Implementation: a Campus Semantic Navigation System

This is the final implementation of the book, in four stages, and how far it is carried may be chosen by the level of the course. It draws on Chapters 32, 33 and 34 together.

Stage One A Dataset of Campus Objects (suitable for high school)

1. Define the categories: discuss with the class and settle on five to ten objects common about a campus (suggested: classroom doors, whiteboards, lecterns, chairs, stairs, lifts, corridors, waste bins, hydrants, water coolers, barriers).
2. Plan the viewpoint: decide the height and angle of the shots, and be sure to imitate the robot's real viewpoint (30 to 50 centimeters high) rather than photographing while standing—the most important detail of this implementation.
3. Collect in groups: each group takes two or three categories and photographs at least 200 images of each, covering different angles, distances, lighting and occlusion, and deliberately including difficult samples.
4. Label: set a standard for labeling first (does a half-hidden door count? where does the box go?), then label, and spot-check at least 10 percent of one another's

work.

5. Split the dataset: split by the place of photography rather than at random, to avoid an inflated assessment.
6. Train the model, assess it on the test set, and analyze the errors of the categories that do worst.

Stage Two Semantic Search and the Common-Sense Prior (the core of this chapter)

This stage needs no real robot and can be done on a map alone; it is a reproduction of the three figures of this chapter.

1. Build a semantic map of the floor: digitize the plan of your own departmental building and mark the type of every room and the position of its door.
2. Build the distribution of the objects: survey how often five to eight objects actually appear in each kind of room. This step is a piece of valuable fieldwork in itself.
3. Implement the two orders of search: geometric and semantic, run each several hundred times over randomly placed objects, and compare the distributions of the distance searched. This is your Figure 34-1.
4. Sweep the quality of the prior: shuffle or reverse the prior deliberately, measure how the distance searched changes, and find the crossing point at which the prior becomes worth using. This is your Figure 34-2.
5. Design the way out: implement the strategy of falling back on the geometric order when the first two semantic suggestions fail, and measure how much it recovers when the prior is wrong.

Stage Three A Local Knowledge Base and a Language Interface

1. Build the knowledge base: by the structure of Figure 34-3, write out the topology of the floors of your own building, the attributes of its places, the rules of passage and what common queries correspond to. A plain text file will serve.
2. Implement the retrieval: given one instruction, retrieve the relevant entries from the knowledge base. Keyword matching will do to begin with; there is no need to start with vector retrieval.
3. Attach the language model: have the model output a structured goal from what was retrieved (the name of the region, the coordinate, the confidence, the mark of the source).
4. Test the failing case: ask deliberately for a place not in the knowledge base, and confirm that the system marks the answer a guess rather than pretending to know.

Stage Four The Verification Layer and End-to-End Integration

1. Measure the success of every step: divide your task into n steps, test each separately 30 times and more, and obtain the p_i of each.
2. Predict the overall success: compute the theoretical value by Formula One, then run 30 complete tasks and compare the prediction with the measurement. The difference is itself good material for discussion.
3. Add verification: design a check for every step, and measure the check's detection rate r (create failures deliberately and see how many the check catches).
4. Measure the improvement: measure the overall success again with verification and retrying added, and compare with the prediction of Formula One. This is

your Figure 34-5.

5. Write the report: the heart of the report is not that we built it but what we measured, where it disagreed with the prediction, and why. This is what the book has asked for consistently from Chapter 1 to Chapter 34.

Further Reading

- The benchmark for assessing vision-language models on embodied navigation (2025). (The source of Section 34.6 and Case 34-3, with the complete method of scoring.)
- A survey of vision-language navigation enhanced by large models (2026). (The basis of Section 34.6 on the present state of real deployment.)
- Research on open-vocabulary zero-shot object navigation. (The theoretical foundation of the semantic exploration of Section 34.3.)
- The semantic maps and scene graphs of Chapter 32, and the training in simulation and edge deployment of Chapter 33. (The direct precursors of this chapter; the three chapters make up Part IX.)
- The hybrid architecture of Chapter 3, the degradation design of Chapter 26, and the methods of assessment and debugging of Chapter 31. (The source of the thinking behind the layered architecture and the mechanism of verification of Section 34.5.)
- The probability and uncertainty of Chapter 11. (The theoretical foundation of uncertainty must be passed down in Section 34.2.)

The Points of This Chapter

- The task level differs from the perceptual levels in kind: the input of perception is a sensor and its output a description of the world; the input of the task level is a person's intention and its output an action. An intention is often vague, and the better half of the task level's work is supplying the assumptions that were not spoken.
- Language to action falls into five stages: parsing the intention, locating the place, locating the goal, planning the path, executing and confirming. Everything after the fourth stage the first thirty-one chapters have dealt with entirely; language navigation attaches a stretch of conversion in front of SLAM and does not replace it.
- What an upper layer sends a lower one must be structured data (coordinates, lists, confidences, marks of source) and not free text—free text can neither be parsed nor verified. Having a language model output velocity commands directly turns a system that can be checked into a black box that cannot.
- The common-sense prior is worth a good deal: under the same map, the mean distance searched in the semantic order is 36 meters against the geometric order's 90, a shortening of 60.1 percent (81.9 by the median). But in 15.3 percent of the trials the semantic order went farther—it improves the expectation and not every single run.
- The benefit depends on how concentrated the object's distribution is: the ladder (70 percent in the storeroom) is shortened by 65 percent and the copier (65 percent in an office) by 77, while the fire extinguisher (60 percent in the corridor) is shortened by only 9. For objects that are everywhere, the return on building semantic reasoning is low.
- The accuracy α of the prior and the distance searched are roughly linear (110

meters at α = 0, 35 at α = 1), crossing the geometric order at about α = 0.25. A wholly random prior (110 meters) is already a little worse than using none (92); what is really dangerous is a prior that points the wrong way—146 meters, 60 percent worse than using none at all. And a confident, wrong prior of just this kind is what a language model is most apt to produce.

- RAG supplies with a local knowledge base the institutional knowledge a model lacks, and its three advantages are that it can be maintained, that it can be verified, and that it is cheap. The common-sense prior (statistical, across sites, answering where to look first) and the knowledge base (definite, particular to a site, answering where that place is) do different offices, and a good system wants both.
- The layered architecture divides by time scale: the task layer (seconds), the semantic layer (0.1–1 second), the geometric layer (10–30 Hz) and the control layer (100 Hz and above). The four practical points are asynchronous design, an explicit interface, a mechanism of refusal, and a path of degradation. The principle of safety is that an upper layer may go wrong and a lower layer must hold the line.
- The success of a multi-step task is a product: an eight-step task is only 43.0 percent at 90 percent per step and a mere 16.8 at 80. This explains why every subsystem looks all right and the whole keeps failing.
- Verification and retrying do far more than raising the ability of a single step: an eight-step task at 80 percent per step, with verification at a detection rate of 0.5 (and at most two retries), rises from 16.8 percent to 38.7, and to 93.8 as the detection rate approaches 1—with the underlying ability entirely unchanged. Verification is usually much easier than execution, and that asymmetry is the

fundamental reason a layered architecture can stand.

- The embodied navigation benchmark of late 2025 shows the best vision-language model scoring 34.38 against the human expert's 75.40, a gap of 41.02. More notable still, a straight line to a known goal scores 51.89, beating the model's complete trajectory; while the same model, with the goal known and only the path to predict, reaches 56.55, and predicting the goal point alone drops to 29.65. The model can walk and cannot recognize—locating the goal is the real bottleneck at present.
- The four great risks are hallucination, the systematic character of a misjudgment, latency, and security. Of these the systematic misjudgment is the most dangerous, because it breaks the assumption that retries are independent, and retrying cannot save it.
- The thread running through the book is that only by understanding a limit can a limit be broken. AI lets a robot understand and think things through, while walking steadily and not running into people still rests on the geometry and control of the first thirty-one chapters. For the foreseeable future, neither can be spared.

The Close of the Book

Part IX is complete here, and with it the book.

From Chapter 32, which gave the map a meaning, through Chapter 33, which taught the robot to move, to this chapter, which let it understand human speech, these three chapters have carried the geometric foundation of the thirty-one before to the frontier of application. But remember those numbers of Section 34.6: the most advanced model is still far from human at embodied navigation, and a straight line to a known goal even

beats its complete planning.

This is not to pour cold water but to see the position clearly. The geometric and probabilistic methods you have learned from Chapter 1 to Chapter 31 are not an old technique about to be replaced by AI; they are the ground any language model must stand upon.

Thirty-four chapters, nine parts, and here we are. What to build next is for you to decide.